UIUC ECE408 / CS483 / CSE408 《Applied Parallel Programming》学习笔记

> **课程来源**：伊利诺伊大学厄巴纳-香槟分校（University of Illinois Urbana-Champaign） **课程主页**：https://ece.illinois.edu/academics/courses/ece408 **教材**：D. Kirk and W. Hwu, *Programming Massively Parallel Processors*, Morgan Kaufmann, 3rd Edition **公开讲义**：Prof. Steven S. Lumetta 公开课程网站（Summer 2024 / Summer 2025 共 22 个 Slide Deck） **笔记语言**：中文（技术术语保留英文原文） **笔记定位**：以 **CUDA 实现与硬件映射** 为主线，覆盖 GPU 硬件架构、CUDA 编程模型、 七种核心并行模式的具体实现与性能优化，以及最终项目的性能工程方法论。

课程概览（Course Overview）

一、这门课到底在教什么

ECE408 的官方定位是一句话：**"Parallel programming with emphasis on developing applications for processors with many computation cores."**——重点不是"并行编程理论"，而是**动手把计算映射到并行硬件上， 并让它跑得快**。

官方对"大规模并行处理器（massively parallel processor）"给了非常工程化的定义：

> *"In general, we refer to a processor as massively parallel if it has the ability to complete more than 64 arithmetic operations per clock cycle."*
> ——只要一个处理器每时钟周期能完成 **超过 64 次算术运算**，就算大规模并行。

这个门槛很有意思：现代 CPU 一个周期通常完成 4–8 次浮点运算，而一块 A100 GPU 有 6912 个 FP32 CUDA core，每个周期可发起上万次运算。**两者差了两三个数量级**，这种量级差异正是本课程全部技术内容的根源： 为了喂饱这么多运算单元，必须重新思考线程组织、内存访问、同步方式。

官方进一步指出，有效编程这类处理器需要三类知识的组合：

1. **并行编程原理（parallel programming principles）**——什么是并行模式、什么是数据复用、什么是负载均衡；
2. **并行与通信模型（parallelism models, communication models）**——SIMT、共享内存通信、warp 级原语；
3. **硬件组织与资源限制（hardware organizations, resource limitations）**——SM、warp、寄存器、共享内存、 带宽与延迟的物理约束。

本课程面向两类受众：**想为这类处理器开发应用的学生**，以及**想开发编程工具和未来处理器设计的学生**。 换句话说，它既是一门"应用课"，也是一门"体系结构课"。

二、课程目标（Course Goals，官方原文）

> *"The aim of this course is to provide students with knowledge and hands-on experience in developing applications software for processors with massively parallel computing resources. ... Effectively programming these processors requires in-depth knowledge about parallel programming principles, as well as parallelism models, communication models, hardware organizations, and resource limitations of these processors."*

拆解成可执行的三条：

目标	具体含义	对应本笔记
知识（knowledge）	理解并行模式、通信模型、硬件约束	全部 Lecture 的核心概念部分
动手经验（hands-on experience）	从零写出能通过正确性测试、并达到性能要求的 CUDA 程序	每个 Lecture 的完整 `.cu` 代码
性能工程能力	设计实验、定位瓶颈、在硬件约束下优化	Lecture 10 + 各讲的性能分析

三、教学目标（Instructional Objectives，官方 17 条）

官方把 17 条教学目标分成三个阶段，这实际上就是本课程的**能力成长路线图**。

阶段 A：完成 7 个机器学习问题（Machine Problems）后 ≈ 20 次 75 分钟讲座

编号	目标	解读
A1	Analyze and implement **common parallel algorithm patterns** in a parallel programming model such as CUDA	能识别并行模式并实现——这是"会写"
A2	**Design experiments** to analyze the performance bottlenecks in their parallel code	能设计实验定位瓶颈——这是"会测"
A3	Apply **common parallel techniques** to improve performance given hardware constraints	能在约束下优化——这是"会调"
A4	Learn about the features of a **parallel debugger** and use them to identify and repair code defects	会用调试器——这是"会修"

编号	目标	解读
A5	Learn about the features of a **parallel profiler** and use them to identify performance bottlenecks	会用 profiler——这是"会剖"

A 组是整门课的重心：它要求的不是"知道"，而是"能做到"。这解释了为什么课程以 8 个 Lab（机器问题）为主体。

阶段 B：第二次考试前 ≈ 29 次讲座

编号	目标	解读
B6	Understand and apply common parallel algorithm patterns	从"会写"上升到"会用"
B7	Understand the **major types of hardware limitations** that limit parallel program performance	硬件约束的系统认知（带宽、延迟、占用率、bank conflict）
B8	Understand and apply common parallel programming **interface features**	API 层面的熟练
B9	**Review a parallel code segment and identify its behavior and potential problems**	代码审查能力——考试的核心题型

阶段 C：最终项目结束时（Proposal / Workshop / Presentation / Report）

编号	目标
C10	Identify and solve a computational problem with parallel algorithm design and program
C11	Learn the necessary domain knowledge in order to solve the identified problem
C12	Work with domain experts and teammates from different disciplines
C13	Properly divide up the responsibilities among teammates
C14	**Identify design space and explore optimization opportunities**
C15	Motivate the problem and approach in a presentation
C16	Properly explain the solutions experimented and justify the final decision and outcome
C17	**Identify limitations of the solutions and future directions**

C14 和 C17 特别值得注意：课程要求你不仅"做出一个快程序"，还要能说清**你探索过哪些设计点**、**哪些尝试失败了**、**方案的边界在哪里**。这是工程师与"调参侠"的分水岭。

四、先修要求与教材

Topical Prerequisites（先修主题）： – **C programming**——CUDA 是 C/C++ 的扩展，指针、数组、内存布局必须熟； – **Basic data structures**——用于理解 CSR 稀疏格式、树形归约等； – **Introduction to computer organization**——cache、内存层次、流水线的基本概念。

官方正式先修课程是 `ECE 220` 或 `CS 225`。

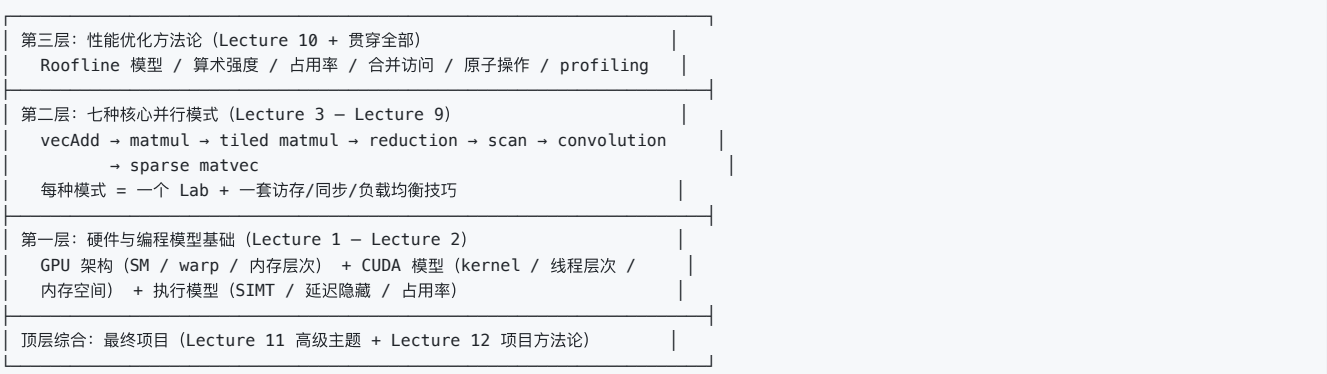
Texts（教材）：

> D. Kirk and W. Hwu, *Programming Massively Parallel Processors*, Morgan Kaufmann, **3rd Edition**. (Required)

Kirk & Hwu 是本课程作者之一 Wen-mei Hwu 的著作，也是 ECE408 的"原生教材"—— 课程讲义中的大量插图、术语、代码骨架直接来自这本书。**强烈建议配合阅读**： 讲义给"课堂节奏"，教材给"完整推导"。

五、课程范围：CUDA 编程 + 并行算法模式 + 性能优化

本课程的知识版图可以分成三层，本笔记的 12 讲严格按这个结构组织：



第一层：CUDA 编程模型（Lecture 1–2）

- **Host/Device 异构模型**：CPU 负责控制流与串行部分，GPU 负责数据并行部分；
- **Kernel 与线程层次**：`grid → block → thread`，二维/三维索引，`<<<grid, block>>>` 执行配置；
- **内存空间**：寄存器、局部内存、共享内存、全局内存、常量内存、纹理内存—— 各自的作用域、生命周期、延迟与带宽差了几个数量级；
- **SIMT 执行模型**：32 线程一个 warp，warp 是调度单位，分歧与锁步。

第二层：并行算法模式（Lecture 3–9）

课程把"并行计算模式（parallel computation patterns）"作为组织单元，每种模式对应一个 Lab：

Lecture	并行模式	核心 CUDA 机制	关键性能问题
3	向量加法（element-wise）	一维 thread→data 映射	合并访存、带宽受限
4	基础矩阵乘法	二维 thread→output 映射	跨步访存、算术强度过低
5	**分块矩阵乘法**	共享内存 + tiling + 屏障	数据复用、bank conflict
6	**归约**	共享内存树 + warp shuffle	warp 发散、延迟受限
7	**扫描/前缀和**	双缓冲 + 层次化算法	工作高效性 vs 并行度
8	**分块卷积**	常量内存 + halo 加载	边界处理、常量缓存广播
9	**稀疏矩阵向量乘**	压缩格式 + 负载均衡	不规则访存、load imbalance

这七种模式不是七个孤立技巧，而是**一套可迁移的思维工具**：任何新问题都可以先问 "它像哪个模式？"——这正是 A1 目标"analyze and implement common parallel algorithm patterns"的含义。

第三层：性能优化方法论（Lecture 10）

- **Roofline 模型**：用算术强度判断"内存受限还是计算受限"，并算出性能天花板；
- **占用率（occupancy）**：多少 warp 常驻 SM 才能隐藏延迟；
- **合并访问（coalescing）**：32 次访问如何变成 1 次 128 字节事务；
- **原子操作与私有化（privatization）**：消除同地址竞争；
- **测量方法学**：`cudaEvent` 计时、profiler 指标、有效带宽计算。

六、实验项目（Lab Projects）

课程有两套 Lab 编号，历史版本（官方课程目录页）与近年实际执行版本（2024/2025 时间线）不同， 本笔记**两套编号同时标注**：

官方课程目录页（Hwu 版本）	2024/2025 实际（Lumetta 版本）	覆盖内容
Lab 0 – installation and test of programming environment	**Lab 0: Device Query**	Lecture 1
Lab 1 – Parallel Vector Addition	**Lab 1: Vector Addition**	Lecture 3
Lab 2 – Parallel Matrix Multiplication	**Lab 2: Simple Matrix Multiply**	Lecture 4
Lab 3 – Tiled Parallel Matrix Multiplication	**Lab 3: Tiled Matrix Multiply**	Lecture 5
Lab 4 – Parallel Reduction	**Lab 4: 3D Convolution**	Lecture 6 / Lecture 8
Lab 5 – Parallel Scan	**Lab 5: List Reduction**	Lecture 7 / Lecture 6
Lab 6 – Tiled Parallel Convolution	**Lab 6: Scan**	Lecture 8 / Lecture 7
Lab 7 – Sparse Matrix–Vector Multiplication	**Lab 7: Histogramming**	Lecture 9 / Lecture 10
（无）	**Lab 8: Sparse Matrix Multiply**	Lecture 9

> **重要提示**：从 2024/2025 时间线看，**3D Convolution 是 Lab 4**，而 **Reduction 是 Lab 5 （List Reduction）**，**Scan 是 Lab 6**。学习时务必以自己学期的时间线为准。

实验环境（Lab Equipment / Lab Software，官方）： – **Lab Equipment**: *"Linux based cluster system"*——近年实际使用 **NCSA Delta 集群**（NVIDIA A100, sm_80）； – **Lab Software**: *"C Programming Language and CUDA Software Development Kit, WebGPU for labs, RAI for final project"*。

这一点非常值得注意：课程近年**同时使用 CUDA 与 WebGPU 两套技术栈**—— Lab 部分在 GPU 上做 CUDA，也引入了 WebGPU（浏览器内 GPU 计算，用 WGSL 着色器语言）， 而最终项目使用基于 WebGPU 的 **RAI** 框架。本笔记 Lecture 11 专门给出 CUDA ↔ WebGPU 的逐项对照表， 帮助有 CUDA 基础的学习者快速迁移。

评分构成（2025 夏季官方）： – **Exams 50%**（两次等权重考试，第二次是第二次期中而非期末统考）； – **Labs (Machine Problems) 25%**（每个 Lab 等权；其中通过测试占 90%，PrairieLearn quiz 占 10%； Lab 0 例外，100% 为 quiz）； – **Project 25%**（demo / 功能 / 代码风格约占 50%；**在功能完整前提下的性能**约占 50%）。

注意最后一条：**性能占项目成绩的一半，但前提是功能完整**—— 一个跑得飞快但结果错误的程序得零分。这与 A2 目标（设计实验分析瓶颈）和 C16 目标（justify the final decision and outcome）一脉相承。

延期政策：每人自动获得**两次 48 小时延期**，但必须在 deadline **之前**提交申请； 最低的 lab+quiz 组合成绩会被丢弃。

七、最终项目要求（Final Project）

官方的表述是：

> *"Final Project that involves Project Proposal, Project Workshop, Project Presentation, and Project Report"*，并且 *"A final project report is required"*。

四个交付物：



三个硬性预期（来自官方 Instructional Objectives C 组）：

1. **必须真的探索设计空间**（C14）——报告里要有"我试过 A、B、C 三组参数，最终选 B 因为……"；
2. **必须能说明为什么**（C16）——"properly explain the solutions experimented and justify the final decision and outcome"，不能只给结果；
3. **必须诚实报告局限**（C17）——"identify limitations of the solutions and future directions"。

性能与正确性的双重要求：项目评分约 50% 给"功能完整前提下的性能"。 这意味着最终提交的程序必须**同时**满足： – 对任意合法的（包括非 2 的幂、非整数倍 tile 的）输入尺寸都给出正确结果； – 在给定硬件上达到有竞争力的性能，并且**能定量解释性能来源**。

学术诚信与 AI 工具政策（官方原文要点）：课程明确规定—— – 使用课程材料（教材、讲义）与教学人员（教授、助教）之外的任何工具，**风险自负**； – AI 工具"routinely fabricate information"，若据此在作业或考试中使用错误信息，**不予给分**； – 若工具输出了他人代码导致查重命中，**学生承担全部学术诚信责任**； – **quiz 与考试期间严禁使用任何未被明确允许的材料，包括 AI 工具**。

如何使用这份笔记

1. **每一讲独立成篇**：结构统一为「概述 → 核心概念与架构图解 → 代码示例与性能分析 → 优化技巧 → 关键要点 → 常见陷阱 → 思考题」，可以按需跳读。
2. **代码可直接编译运行**：所有 `.cu` 示例都是完整的（含 `main`、错误检查、计时、CPU 校验）， 编译命令写在文件头部注释里，统一使用 `nvcc -O3 -arch=sm_80`。
3. **性能数字有出处**：所有吞吐量、带宽、延迟数字都标注了对应的 GPU 型号， 性能分析基准机为 **NVIDIA A100 (sm_80)**（课程实验集群所用 GPU），必要时对比 RTX 4090。
4. **两套 Lab 编号都标注**：看到 "Lab 4" 时请对照上表确认是 Reduction 还是 3D Convolution。
5. **配合原版讲义**：笔记中每讲都标注了对应的公开 Slide Deck 编号，可在 https://lumetta.web.engr.illinois.edu/408-Sum25/ 下载原始 PDF 对照阅读。

硬件档案（全书性能分析共用）

GPU	架构	SM 数	每 SM warp 上限	每 SM 线程上限	寄存器/SM	共享内存/SM	显存带宽	FP32 峰值
A100 (80GB SXM)	Ampere `sm_80`	108	64	2048	65536	164 KB	1555 GB/s	19.5 TFLOPS
H100 SXM	Hopper `sm_90`	132	64	2048	65536	228 KB	3350 GB/s	67 TFLOPS
RTX 4090	Ada `sm_89`	128	48	1536	65536	100 KB	1008 GB/s	82.6 TFLOPS
RTX 2080 Ti	Turing `sm_75`	68	32	1024	65536	64 KB	616 GB/s	13.4 TFLOPS

关键派生量（机器平衡点 machine balance = FP32 峰值 / 带宽）：

GPU	机器平衡点	含义
A100	19.5 TFLOPS ÷ 1555 GB/s ≈ **12.5 FLOP/Byte**	算术强度低于此值 → 带宽受限
H100	67 ÷ 3350 ≈ **20.0 FLOP/Byte**	更高的平衡点，更"饿"数据
RTX 4090	82.6 ÷ 1008 ≈ **82 FLOP/Byte**	极端偏计算，访存优化收益巨大
RTX 2080 Ti	13.4 ÷ 616 ≈ **21.8 FLOP/Byte**	—

内存层次延迟与带宽（数量级）：

层次	延迟	带宽（A100）	作用域
寄存器	~1 cycle（无冲突时）	极高（数万 GB/s 等效）	单线程
共享内存 / L1	~20–30 cycles	~19 TB/s 聚合	单 block
L2 Cache	~200 cycles	~5–7 TB/s	全 GPU（40–50 MB on A100）
全局内存（DRAM）	~400–800 cycles	1555 GB/s	全 GPU，跨 kernel 持久

事务粒度：一个 warp 的 32 个线程若访问连续 4 字节地址，硬件合并为 **1 次 128 字节事务 （= 1 条 cache line）**；这是全书一切访存优化的基本单位。

本笔记基于 UIUC 公开课程资料整理：官方课程目录页、官方 Course Goals 与 17 条 Instructional Objectives、官方 Lab Projects 列表，以及 Prof. Steven S. Lumetta 公开课程网站上的全部 22 个 Slide Deck（Summer 2025 / Summer 2024）与历年公开考试资料。实验指导书、项目计划与评分标准未公开，笔记中相应内容依据官方描述与讲义重建。

附：分讲结构化数据记录（Phase 1 数据获取与整理）

以下为每一讲的资料记录，字段说明：

- `lecture_topic`：讲座主题；
- `related_lab`：对应的实验项目；
- `key_concepts_raw`：本讲涉及的关键概念（原始词表）；
- `learning_objectives`：对应的官方教学目标；
- `available_public_info`：该主题公开可得的信息摘要；
- `source_slide_decks` / `local_source_texts`：对应的公开讲义 Slide Deck 与已归档的讲义文本文件。

L01 · Course Overview, Motivation for Parallel Computing, and GPU Architecture Introduction

```
{
  "lecture_topic": "Course Overview, Motivation for Parallel Computing, and GPU Architecture Introduction",
  "related_lab": "Lab 0: Device Query",
  "key_concepts_raw": [
    "power wall (2004)",
    "multicore transition",
    "massively parallel processor (>64 ops/clock)",
    "heterogeneous computing (host/device)",
    "SM (Streaming Multiprocessor)",
    "SIMT / warp",
    "latency hiding vs throughput",
    "arithmetic intensity",
    "Amdahl's law",
    "unified shader architecture",
    "device query and CUDA environment setup",
  ],
  "learning_objectives": [
    "Analyze and implement common parallel algorithm patterns in a parallel programming model such as CUDA",
    "Understand the major types of hardware limitations that limit parallel program performance",
  ],
  "available_public_info": "公开的课程目录页给出 Official Description、Course Goals、Instructional Objectives A1–A17、Lab 0 (installation
  "source_slide_decks": [
    "Slide Deck 1: Introduction",
    "Extra Slide Deck 1: A Historical Perspective",
    "Extra Slide Deck 2: Generalizing Parallelism",
    "Slide Deck 19: GPUs in the PC Architecture",
  ]
}
```


L02 · CUDA Programming Model Fundamentals: Kernels, Thread Hierarchy, and Memory Model


```
{
  "lecture_topic": "CUDA Programming Model Fundamentals: Kernels, Thread Hierarchy, and Memory Model",
  "related_lab": "Lab 0 / Lab 1 (foundation)",
  "key_concepts_raw": [
    "__global__ / __device__ / __host__ qualifiers",
    "kernel launch <<<grid, block>>>",
    "grid-block-thread hierarchy",
    "gridDim/blockDim/blockIdx/threadIdx",
    "warp = 32 threads",
    "SIMT execution and warp divergence",
    "register / local / shared / global / constant / texture memory",
    "scope and lifetime of each memory space",
    "__syncthreads() block barrier",
    "cudaDeviceSynchronize vs asynchronous kernel launch",
    "thread coarsening (motivation)",
  ],
  "learning_objectives": [
    "Understand and apply common parallel programming interface features",
    "Review a parallel code segment and identify its behavior and potential problems",
  ],
  "available_public_info": "公开讲义 Slide Deck 2 (CUDA Introduction)、Slide Deck 3 (CUDA Parallelism Model)、Slide Deck 4 (CUDA Memory
  "source_slide_decks": [
    "Slide Deck 2: CUDA Introduction",
    "Slide Deck 3: CUDA Parallelism Model",
    "Slide Deck 4: CUDA Memory Model",
  ]
}
```


L03 · Parallel Pattern 1 – Vector Addition: Thread Mapping and Memory Access

```
{
  "lecture_topic": "Parallel Pattern 1 – Vector Addition: Thread Mapping and Memory Access",
  "related_lab": "Lab 1: Vector Addition",
  "key_concepts_raw": [
    "data parallelism",
    "thread-to-data mapping i = blockIdx.x*blockDim.x + threadIdx.x",
    "grid sizing with ceiling division",
    "boundary checking",
    "coalesced global memory access",
    "memory bandwidth bound kernel",
    "arithmetic intensity of vector add (0.125 FLOP/Byte)",
    "occupancy and block size selection",
    "grid-stride loop",
    "__restrict__ pointers",
    "validating against a CPU golden reference",
  ],
  "learning_objectives": [
    "Analyze and implement common parallel algorithm patterns in a parallel programming model such as CUDA",
    "Design experiments to analyze the performance bottlenecks in their parallel code",
    "Apply common parallel techniques to improve performance given hardware constraints",
  ],
  "available_public_info": "公开讲义 Slide Deck 2/3/4 含 Review – Vector Addition Kernel 与 Review – Thread Assignment for vecAdd (gridD
  "source_slide_decks": [
    "Slide Deck 2: CUDA Introduction",
    "Slide Deck 3: CUDA Parallelism Model",
    "Slide Deck 7: DRAM",
  ]
}
```


L04 · Parallel Pattern 2 – Simple Matrix Multiplication and Its Performance Bottlenecks


```
{
  "lecture_topic": "Parallel Pattern 2 – Simple Matrix Multiplication and Its Performance Bottlenecks",
  "related_lab": "Lab 2: Simple Matrix Multiply",
  "key_concepts_raw": [
    "GEMM definition, FLOPs = 2*M*N*K",
    "row-major linearization M[row*Width + col]",
    "thread-per-output mapping with 2D block/grid",
    "2K global loads per output element",
    "strided (uncoalesced) access to B columns",
    "arithmetic intensity 0.25 FLOP/Byte",
    "Roofline ceiling for naive matmul",
    "data reuse counts (A used N times, B used M times)",
    "matrix transpose to restore coalescing",
    "thread coarsening for register reuse",
  ],
  "learning_objectives": [
    "Analyze and implement common parallel algorithm patterns in a parallel programming model such as CUDA",
    "Design experiments to analyze the performance bottlenecks in their parallel code",
  ],
  "available_public_info": "公开讲义 Slide Deck 5 (CUDA Tiled Matrix Multiplication) 的前半部分讲解基础矩阵乘法的访存问题；Slide Deck 7 (DRAM
  "source_slide_decks": [
    "Slide Deck 5: CUDA Tiled Matrix Multiplication",
    "Slide Deck 6: More on Tiling",
    "Slide Deck 7: DRAM",
  ]
}
```


L05 · Tiled Parallel Matrix Multiplication: Shared Memory and Data Reuse

```
{
  "lecture_topic": "Tiled Parallel Matrix Multiplication: Shared Memory and Data Reuse",
  "related_lab": "Lab 3: Tiled Matrix Multiply",
  "key_concepts_raw": [
    "tiling",
    "shared memory (48KB–228KB/SM, ~20–30 cycle latency)",
    "data reuse, 2N -> 2N/TILE_WIDTH global accesses per output",
    "shared memory banks and bank conflicts",
    "padding to break bank conflicts",
    "__syncthreads() RAW/WAR hazards, two barriers per tile iteration",
    "arithmetic intensity TILE_WIDTH/4 FLOP/Byte",
    "thread coarsening with register accumulators",
    "occupancy vs shared memory tradeoff",
    "double buffering / cp.async prefetch (advanced)",
  ],
  "learning_objectives": [
    "Apply common parallel techniques to improve performance given hardware constraints",
    "Understand the major types of hardware limitations that limit parallel program performance",
    "Analyze and implement common parallel algorithm patterns in a parallel programming model such as CUDA",
  ],
  "available_public_info": "公开讲义 Slide Deck 5 (CUDA Tiled Matrix Multiplication) 与 Slide Deck 6 (More on Tiling) 完整公开，含 __shar
  "source_slide_decks": [
    "Slide Deck 5: CUDA Tiled Matrix Multiplication",
    "Slide Deck 6: More on Tiling",
  ]
}
```


L06 · Parallel Pattern 4 – Reduction: Reduction Trees and Warp–Level Primitives


```
{
  "lecture_topic": "Parallel Pattern 4 – Reduction: Reduction Trees and Warp–Level Primitives",
  "related_lab": "Lab 4: Parallel Reduction (catalog) / Lab 5: List Reduction (Sum 2025)",
  "key_concepts_raw": [
    "reduction with associative+commutative operator and identity",
    "reduction tree, log2(N) steps, N–1 work, work efficiency",
    "interleaved addressing (stride 1,2,4...) vs sequential addressing (stride blockDim/2...)",
    "warp divergence in reductions",
    "bank conflicts in shared memory reduction",
    "__syncthreads() inside a stride loop",
    "warp shuffle: __shfl_down_sync / __shfl_xor_sync",
    "independent thread scheduling on Volta+ and the _sync mask",
    "thread coarsening with grid–stride loop",
    "hierarchical multi–block reduction, atomicAdd",
    "latency/bandwidth–bound nature of reduction",
  ],
  "learning_objectives": [
    "Analyze and implement common parallel algorithm patterns in a parallel programming model such as CUDA",
    "Apply common parallel techniques to improve performance given hardware constraints",
    "Understand the major types of hardware limitations that limit parallel program performance",
  ],
  "available_public_info": "公开讲义 Slide Deck 11 (Reduction Trees) 完整公开，含 reduction 的定义、work–efficient 分析、交错/顺序寻址的对比与
  "source_slide_decks": [
    "Slide Deck 11: Reduction Trees",
  ]
}
```


L07 · Parallel Pattern 5 – Prefix Sum / Scan: Double Buffering and Hierarchical Algorithms

```
{
  "lecture_topic": "Parallel Pattern 5 – Prefix Sum / Scan: Double Buffering and Hierarchical Algorithms",
  "related_lab": "Lab 5: Parallel Scan (catalog) / Lab 6: Scan (Sum 2025)",
  "key_concepts_raw": [
    "inclusive vs exclusive scan",
    "associative operator generalisation",
    "applications: recurrences, stream compaction, radix sort, histograms, polynomial evaluation",
    "Hillis-Steele (Kogge-Stone) scan: O(log N) steps, O(N log N) work, not work-efficient",
    "Blelloch work-efficient scan: up-sweep and down-sweep, O(N) work",
    "double buffering to avoid read-after-write hazards",
    "hierarchical multi-block scan (scan-scan-add three step)",
    "shared memory bank conflicts for different strides",
    "work efficiency vs latency tradeoff",
    "__syncthreads() between every scan step",
  ],
  "learning_objectives": [
    "Analyze and implement common parallel algorithm patterns in a parallel programming model such as CUDA",
    "Understand and apply common parallel algorithm patterns",
    "Apply common parallel techniques to improve performance given hardware constraints",
  ],
  "available_public_info": "公开讲义 Slide Deck 12 (Parallel Prefix, or Scan) 完整公开，含 Hillis-Steele、Blelloch、double buffering、hier
  "source_slide_decks": [
    "Slide Deck 12: Parallel Prefix, or Scan",
  ]
}
```


L08 · Parallel Pattern 6 – Tiled Convolution: Constant Memory and Boundary Handling


```
{
  "lecture_topic": "Parallel Pattern 6 – Tiled Convolution: Constant Memory and Boundary Handling",
  "related_lab": "Lab 6: Tiled Parallel Convolution (catalog) / Lab 4: 3D Convolution (Sum 2025)",
  "key_concepts_raw": [
    "1D/2D/3D convolution definition, mask/filter, halo/apron",
    "constant memory: 64KB, __constant__, cudaMemcpyToSymbol, broadcast in constant cache",
    "boundary handling: zero padding, clamp, wrap; ghost cells",
    "warp divergence at boundaries",
    "tiled convolution with halo (TILE+2r)^2 shared memory capacity",
    "reduction of global traffic by (TILE+2r)^2/TILE^2",
    "thread coarsening and register blocking",
    "arithmetic intensity of convolution (~1 FLOP/Byte naive)",
    "3D convolution: larger halo volume, register pressure, cudaPitchedPtr",
  ],
  "learning_objectives": [
    "Analyze and implement common parallel algorithm patterns in a parallel programming model such as CUDA",
    "Apply common parallel techniques to improve performance given hardware constraints",
    "Design experiments to analyze the performance bottlenecks in their parallel code",
  ],
  "available_public_info": "公开讲义 Slide Deck 8 (Convolution and Constant Memory)、Slide Deck 9 (Tiled Convolution)、Slide Deck 10 (T
  "source_slide_decks": [
    "Slide Deck 8: Convolution and Constant Memory",
    "Slide Deck 9: Tiled Convolution",
    "Slide Deck 10: Tiled Convolution Analysis",
  ]
}
```


L09 · Parallel Pattern 7 – Sparse Matrix–Vector Multiplication: Compact Formats and Load Balance

```
{
  "lecture_topic": "Parallel Pattern 7 - Sparse Matrix-Vector Multiplication: Compact Formats and Load Balance",
  "related_lab": "Lab 7: Sparse Matrix-Vector Multiplication (catalog) / Lab 8: Sparse Matrix Multiply (Sum 2025)",
  "key_concepts_raw": [
    "sparsity in real systems, loss of regularity",
    "CSR: rowPtr[N+1], colIdx[nnz], values[nnz]",
    "ELL, ELLPACK, COO, JDS, hybrid ELL+COO",
    "load imbalance: rows with very different nnz within one warp",
    "indirect gather x[colIdx[j]] - uncoalesced and cache-unfriendly",
    "arithmetic intensity ~0.167 FLOP/Byte",
    "vectorised loads (float4), shared memory caching of x",
    "row reordering, warp-per-row with shuffle reduction",
    "compaction motivation (memory footprint and bandwidth)",
  ],
  "learning_objectives": [
    "Analyze and implement common parallel algorithm patterns in a parallel programming model such as CUDA",
    "Understand the major types of hardware limitations that limit parallel program performance",
    "Apply common parallel techniques to improve performance given hardware constraints",
  ],
  "available_public_info": "公开讲义 Slide Deck 14 (Parallel Sparse Methods) 与 Slide Deck 15 (Parallel Sparse Methods Part 2) 完整公开,
  "source_slide_decks": [
    "Slide Deck 14: Parallel Sparse Methods",
    "Slide Deck 15: Parallel Sparse Methods (Part 2)",
  ]
}
```


L10 · Performance Analysis and Optimisation Methodology: Roofline, Occupancy, Coalescing, Atomics


```
{
  "lecture_topic": "Performance Analysis and Optimisation Methodology: Roofline, Occupancy, Coalescing, Atomics",
  "related_lab": "Applies to all labs",
  "key_concepts_raw": [
    "DRAM organisation, burst length, row buffer",
    "coalescing: 32 accesses -> 1..32 transactions of 128 bytes",
    "arithmetic intensity and machine balance (A100 = 12.5 FLOP/Byte)",
    "Roofline model: bandwidth-bound vs compute-bound knee",
    "Little's law for latency hiding, memory-level parallelism",
    "occupancy calculation and the 'good enough' rule",
    "cudaOccupancyMaxActiveBlocksPerMultiprocessor, nvcc --ptxas-options=-v",
    "atomic operations, serialisation on the same address",
    "data locality / privatisation (shared memory private histogram)",
    "profiling with nvprof / Nsight Compute, stall reasons",
  ],
  "learning_objectives": [
    "Design experiments to analyze the performance bottlenecks in their parallel code",
    "Learn about the features of a parallel profiler and use them to identify performance bottlenecks in their code",
    "Learn about the features of a parallel debugger and use them to identify and repair code defects",
  ],
  "available_public_info": "公开讲义 Slide Deck 7 (DRAM)、Slide Deck 6 (More on Tiling)、Slide Deck 10 (Tiled Convolution Analysis)、Sli
  "source_slide_decks": [
    "Slide Deck 7: DRAM",
    "Slide Deck 6: More on Tiling",
    "Slide Deck 10: Tiled Convolution Analysis",
    "Slide Deck 13: Atomic Operations and Histograms",
  ]
}
```


L11 · Advanced Topics: Multi-Stream, Data Transfer, Tensor Cores, Deep Learning, Alternatives to CUDA (WebGPU/RAI)

```
{
  "lecture_topic": "Advanced Topics: Multi-Stream, Data Transfer, Tensor Cores, Deep Learning, Alternatives to CUDA (WebGPU/RAI)",
  "related_lab": "Final Project",
  "key_concepts_raw": [
    "GPU in the PC architecture: PCIe 3.0/4.0/5.0 bandwidth, NVLink, NVSwitch",
    "unified memory / cudaMallocManaged and page migration",
    "pinned (page-locked) host memory and true DMA",
    "CUDA streams, cudaMemcpyAsync, overlapping transfer with compute",
    "default stream semantics and events for inter-stream sync",
    "Tensor Cores, WMMA API, FP16/BF16/TF32 with FP32 accumulate",
    "deep learning: feedforward nets, gradient descent, convolution as GEMM (im2col)",
    "alternatives: OpenCL, SYCL, HIP, OpenACC, OpenMP offload, Vulkan/Metal, WebGPU (WGSL), RAI",
    "CUDA-to-WebGPU mapping table (grid/block/thread vs workgroup/invocation)",
  ],
  "learning_objectives": [
    "Identify design space and explore optimization opportunities for the solutions",
    "Identify limitations of the solutions and future directions",
    "Understand and apply common parallel programming interface features",
  ],
  "available_public_info": "公开讲义 Slide Deck 16 (Machine and Deep Learning)、Slide Deck 17 (Feedforward Networks and Gradient Descen
  "source_slide_decks": [
    "Slide Deck 16: Machine and Deep Learning",
    "Slide Deck 17: Feedforward Networks and Gradient Descent",
    "Slide Deck 18: Computation in Deep Neural Networks",
    "Slide Deck 19: GPUs in the PC Architecture",
    "Slide Deck 20: GPU Data Transfer",
    "Slide Deck 21: Tensor Operations",
    "Slide Deck 22: Alternatives to CUDA",
  ]
}
```


L12 · Final Project: Problem Definition, Design, Optimisation, and Reporting


```
{
  "lecture_topic": "Final Project: Problem Definition, Design, Optimisation, and Reporting",
  "related_lab": "Final Project: Proposal + Workshop + Presentation + Report",
  "key_concepts_raw": [
    "project deliverables and grading (25% of course; ~50% correctness, ~50% performance)",
    "problem selection: data parallelism, arithmetic intensity, regularity, scale",
    "performance engineering cycle: baseline, profile, hypothesise, change one variable, re-measure",
    "design space exploration: block size, tile size, coarsening factor, padding, vectorisation, streams",
    "golden reference construction and floating-point tolerance",
    "robustness: arbitrary sizes, boundary handling, multi-architecture builds",
    "technical report structure and honest reporting of failed attempts",
    "academic integrity and the AI-tool policy",
    "team work decomposition and interface contracts",
  ],
  "learning_objectives": [
    "Identify and solve a computational problem with parallel algorithm design and program",
    "Work with domain experts and teammates from different disciplines to maximize the effectiveness of solutions",
    "Properly divide up the responsibilities among teammates and support each other towards success",
    "Motivate the problem and approach in a presentation",
    "Properly explain the solutions experimented and justify the final decision and outcome",
    "Identify limitations of the solutions and future directions",
  ],
  "available_public_info": "课程目录页确认 'A final project report is required' 与 'Final Project that involves Project Proposal, Projec
  "source_slide_decks": [
    "Slide Deck 10: Tiled Convolution Analysis (as methodology reference)",
  ]
}
```


第二部分：分讲学习笔记

> 共 12 讲，严格按实验/讲座顺序组织。每讲结构统一为： 概述 → 核心概念与 GPU 架构图解 → 代码示例与性能分析 → 性能优化技巧总结 → 关键要点 → 常见陷阱与注意事项 → 思考题（带答案）。 所有 `.cu` 代码均为完整可编译程序，编译命令见各代码块头部注释，统一使用 `nvcc -O3 -arch=sm_80`。

Lecture 1: 课程概述、并行计算动机与 GPU 架构导论 (对应 Lab 0: Device Query)

概述

本讲回答三个层层递进的问题：**为什么**要在 2004 年之后转向并行与异构计算（功耗墙、Dennard 缩放失效、多核普及、GPU 在算力与带宽上的双重优势）；**怎么衡量**并行化的收益与极限（加速比、效率、可扩展性、Amdahl 定律、并行开销分类）；以及**GPU 长什么样**（众核吞吐量处理器、SM（Streaming Multiprocessor，流多处理器）、SIMT 执行模型、延迟隐藏、内存带宽与算术强度、统一着色器/Tesla 架构如何让 CUDA 成为可能）。最后落到工程实践：CUDA 程序的**五步模板**（分配、拷贝入、启动、拷贝回、释放）、`__global__` / `__device__` / `__host__` 限定符、`<<<grid, block>>>` 执行配置语法的含义，并用 Lab 0 的 Device Query 程序把上面所有抽象概念变成屏幕上可读的真实数字。

本讲统一使用的硬件档案（后续所有性能分析都以此为准，避免数字前后矛盾）：

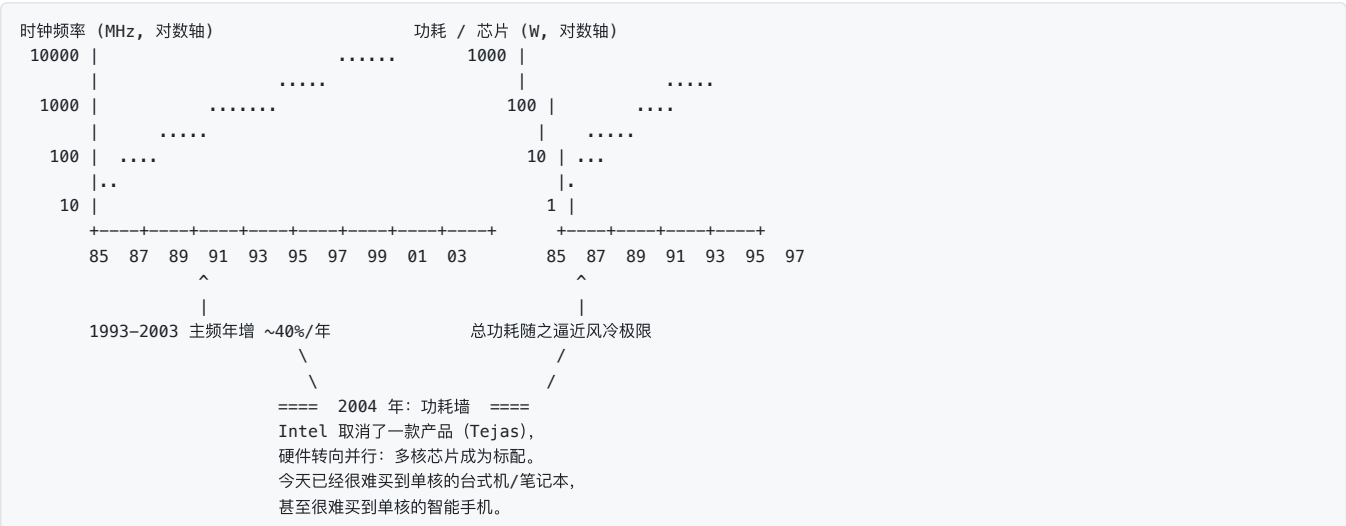
GPU	SM 数	每 SM warp 上限	每 SM 线程上限	每 SM 寄存器	每 SM 共享内存	显存带宽	FP32 峰值
RTX 4090 (Ada, sm_89)	128	48	1536	65536	100 KB	~1008 GB/s	~82.6 TFLOPS
A100 (Ampere, sm_80)	108	64	2048	65536	164 KB	~1555 GB/s	~19.5 TFLOPS
H100 SXM (Hopper, sm_90)	132	64	2048	65536	228 KB	~3350 GB/s	~67 TFLOPS
RTX 2080 Ti (Turing, sm_75)	68	32	1024	65536	64 KB	~616 GB/s	~13.4 TFLOPS

课程实验环境：UIUC 的 Lab 与 Project 通过 NCSA 的 **Delta 超算**提交，一个 Delta GPU 计算节点为 AMD Milan CPU（单路 64 核，~2.45 GHz，256 GB RAM）加 **4 张 NVIDIA A40**（48 GB GDDR6、696 GB/s 显存带宽、PCIe Gen4 64 GB/s、NVLink 112.5 GB/s 双向、300 W）。历史上课程也在 A100（sm_80）上运行；代码统一用 `nvcc -arch=sm_80` 编译，性能分析以 A100 为基准机，必要时对比 RTX 4090。

核心概念与 GPU 架构图解

功耗墙与并行计算的历史转向（The Power Wall and the 2004 Pivot）

- **定义与目的**：功耗墙（power wall）指 2000 年代中期芯片功耗密度增长到风冷散热无法承受的极限，使得"继续提高主频"这条自 1985 年以来最有效的性能提升路径突然失效。它解决的不是某个算法问题，而是**整个产业必须换一条性能增长曲线**的问题：从"把单个核做快"改为"把很多核做多"。
- **直观解释（"它是什么？"）**：把 CPU 想象成一辆跑车。过去几十年工程师的做法是不断加大发动机排量（提高主频），每 18 个月车速翻倍。到了 2004 年，发动机已经热到会把自己熔掉——散热器（风冷）跟不上了，而继续加排量只会让整台车起火。于是厂商换了个思路：不再造一辆 300 km/h 的跑车，而是造 **64 辆 80 km/h 的小车**一起拉货。总运力（吞吐量）继续增长，但任何**单件**货物的送达速度（延迟）反而变慢了。这就是"多核转向"的本质代价。
- **架构/机制图解**：下面是 1985 至 2010 年主频与功耗的走势示意（讲义引用了 Karl Rupp 的 microprocessor-trend-data；对应讲义 "Frequency Scaled Too Fast 1993–2003" 与 "Total Processor Power Increased" 两页）：

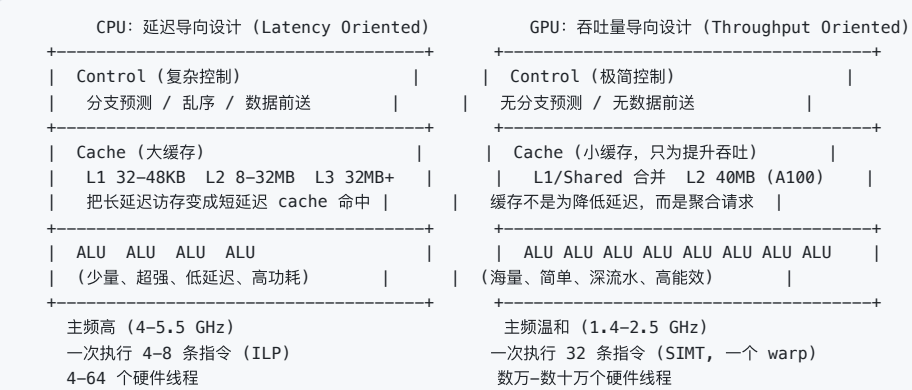


- **关键操作与性能特征**：
- 2004 年之前，CPU 性能提升几乎全部来自主频与指令级并行（ILP）。Pentium 4 "Prescott"（2004）达到 3.8 GHz、功耗约 115 W，继续往上做就要突破 150 W 风冷极限。
- 1970–2004 年主频从 ~0.1 MHz 涨到 ~3.8 GHz（约 38000 倍），但 2004–2025 年主流桌面主频只从 3.8 GHz 涨到 ~5.5 GHz（约 1.4 倍）。**主频红利在 2004 年一次性消失**。
- 性能提升的重心由此转移到：**多核、更温和的主频、大量使用向量执行、同时使用延迟导向核与吞吐导向核、3D 堆叠封装换取更大内存带宽**。这五条正是讲义 "But Our Chips (Mostly) Still Don't Burn — So what changed?" 一页给出的答案。

- 量子化的后果：并行提供的是**固定倍数**的收益（受 Amdahl 定律限制），不会改变算法复杂度；没有任何代码或输入能扩展到无限资源（讲义 "Parallelism Does Not Affect Algorithmic Complexity"）。此外，一片芯片即使算力无穷，**它仍然只有一片芯片的内存容量与 I/O 带宽**（讲义 "Not Everything Can Fit on a Chip"）——这直接解释了后面"算术强度"为什么会成为 GPU 编程的核心约束。
- 历史注脚：在 2000 年代后期之前，并行计算在学术界占比不到 5% 的研究群体，工业界关注度极低，因为"没有收益只有痛苦（No gain, just pain）"；老笑话是"只有研究生才写并行程序"。真正的转折发生在 **2004 年（功耗墙）** 与 **2007 年（CUDA 发布）**。

Dennard 缩放失效与延迟/吞吐量两种设计哲学（Dennard Scaling, Latency-Oriented vs Throughput-Oriented Design）

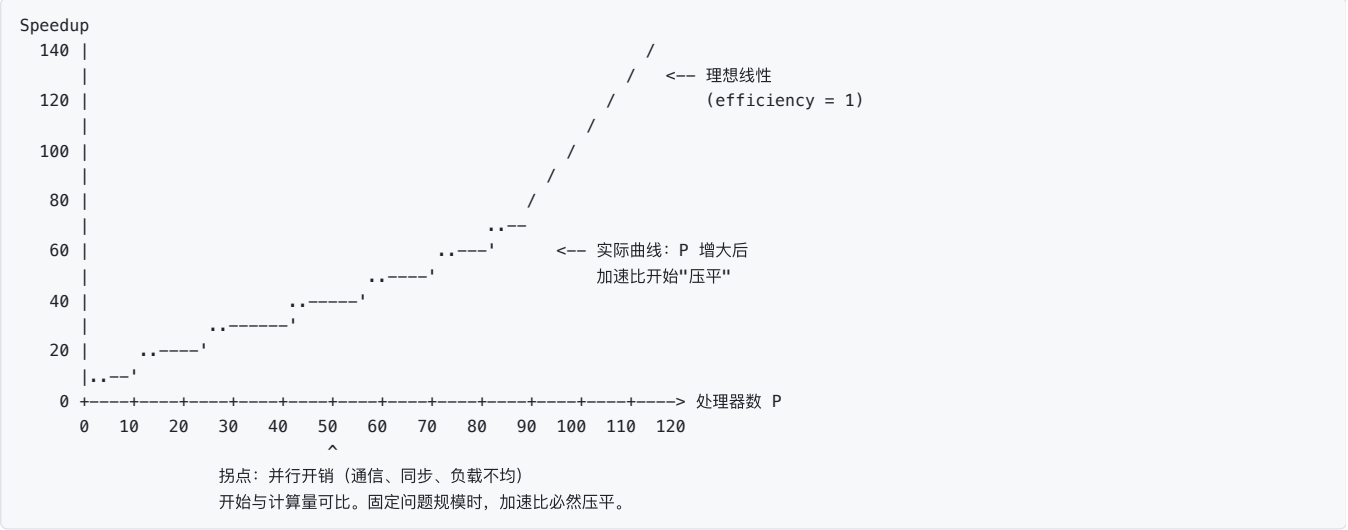
- **定义与目的**：Dennard 缩放（1974, JSSC）是 MOSFET 尺寸等比缩小（constant field scaling）的理想模型，它保证了"晶体管变小 → 更快 → 更省电"的良性循环。当它在 2004 年前后失效，芯片设计被迫一分为二：**延迟导向（latency-oriented）** 的 CPU 和 **吞吐量导向（throughput-oriented）** 的 GPU。理解这两者的取舍，是理解一切 CUDA 优化技巧的总纲。
- **直观解释（"它是什么？"）**：CPU 像一个**高级餐厅的服务员**：一次只服务一桌，但每一步都极快——他记得客人要什么（分支预测），提前把菜端到桌边（数据前送），还会用小推车（大缓存）把常用调料备在手边。他处理"一道复杂工序"的单件延迟极低，但一次只能端一盘。GPU 像**快餐店的一百个窗口**：每个窗口的店员只会"装一个汉堡"这一招（简单控制、无分支预测），但他不介意排队（长延迟、深流水），只要队伍足够长，出餐的**总速率**就极高。点一份汉堡，快餐店比高级餐厅慢；点一万份汉堡，快餐店快十倍以上。
- **架构/机制图解**：



- **关键操作与性能特征**：
- **Dennard 缩放的数学**：设 L → αL（典型 α ≈ 0.7），则有 VDD → αVDD、C → αC、I → αI。于是
  - 门延迟 `Delay = C·VDD / I` → α·CVDD/I，即延迟缩放为 α，**频率 f → 1/α（每代约 ×1.43）**；
  - 单管功耗 `P = C·V²·f` → α·α²·(1/α) = **α²（每代约 ×0.49）**；
  - 因此在**芯片面积与总功耗不变**的前提下，晶体管数量可以每代翻倍而功耗不涨——这就是摩尔定律能持续 30 年的物理基础。
- **失效原因**：阈值电压 Vth 不能随 VDD 等比下降（否则亚阈值漏电指数上升），约 2004 年 VDD 卡在 ~1.0–1.2 V，漏电功耗（静态功耗）在总功耗中占比快速上升。P 不再按 α^2 下降，频率红利同时终结。
- **数量级对比**：一枚 10 代 Intel Core（10 核、14 nm）对比 NVIDIA GK110（2880 个 CUDA core、28 nm），后者在**峰值吞吐量**上远超前者，但单线程跑一段带分支的串行代码则慢 10 倍以上。讲义给出的结论是："**CPU 在串行部分比 GPU 快 10 倍以上，GPU 在并行部分比 CPU 快 10 倍以上；赢的策略是同时使用两者**（Winning Strategies Use Both CPU and GPU）"。
- **性能特征数值**：CPU 访存延迟靠缓存压低到 ~4 cycle（L1 命中），但带宽只有 ~50–200 GB/s（双通道 DDR5-4800 约 76.8 GB/s，L2/L3 上百 GB/s）；GPU 全局内存延迟高达 **400–800 cycle**，但 A100 显存带宽 **1555 GB/s**。**延迟高但对吞吐量不敏感**——这正是 GPU 必须靠"海量线程"运转的原因。

加速比、效率与可扩展性（Speedup, Efficiency, Scalability）

- **定义与目的**：这三个量是并行计算界的"公共语言"。加速比回答"快了没有"，效率回答"快得值不值"，可扩展性回答"再堆机器还有用吗"。它们共同约束了所有后续优化的评价方式。
- **直观解释（"它是什么？"）**：把并行化想象成**请施工队盖房子**。
- **加速比**：本来你一个人盖要 100 天，请了 10 个人 12 天盖完，加速比 = 100/12 ≈ 8.3×。
- **效率**：你付了 10 个人的工钱，只拿到 8.3 个人的产出，效率 = 8.3/10 = 83%——有一部分工人在互相等待、传递工具、或者干着只有一个人能干的活（比如只有一个电闸）。
- **可扩展性**：请到 50 个人时，如果工地只有一条上料通道，大家开始互相挡路，加速比反而掉下来——这就是"不可扩展"。
- **架构/机制图解**（加速比 vs 处理器数的两条曲线，对应讲义 "Scalability Measures Effect of Parallel Overheads"）：



- **关键操作与性能特征**：
- **定义**：`Speedup(P) = T(sequential) / T(parallel, P)`，并且**固定问题规模**。
- **T(sequential) 的选取有一条苛刻规则**：必须是"**在串行机器上最优的算法**（未必是被并行化的那个算法）、**针对串行机器优化过**、**不含任何并行残留开销**"。实践中很难做到，因此工程界的做法是——**去找别人最好的实现当基线，而不是自己写一个**。Hwu 教授的原话大意是："没人会相信你为一个 baseline 付出了多少努力，即使你真的付出了。"（讲义 "Find (Don't Write) a Competitive Baseline"）
- **效率**：`Efficiency(P) = Speedup(P) / P`。付款方希望是 1；真实应用通常是"接近 1 但不等于 1"的某个不可忽略值。**效率 > 1 称为超线性加速（superlinear speedup）**，极少见，可能来源是：CPU 上被容量限制的问题在 P 台机器上获得了额外资源（如总缓存容量随 P 增长，命中率提高），或者纯粹的运气（并行搜索恰好早找到答案）。
- **单一 GPU 的 P 是什么？** 是 1？是 SM 数？还是总 PE 数？讲义明确指出：对单个 GPU 而言效率的定义会含糊，实际做法是**用资源占用率与该 GPU 的峰值资源对比**来估计效率（这正好就是 Lab 0 Device Query 要做的事——查出峰值，然后看自己的配置用了多少）。
- **可扩展性**：问的是"在多少个处理器范围内加速比是线性的、效率是平的"。固定问题规模下，超过某个 P 之后加速比必然压平（除非让一部分处理器空转）。**好的可扩展性 = 在你的机器上，直到可测的最大 P 都没有衰减**。
- **面向不同场景的加速比变体**（J. P. Singh, J. L. Hennessy, A. Gupta, IEEE Computer 26(7), 1993）：
  - **scaled speedup（规模扩展加速比）**：问题规模随 P 线性增长，好的结果是 1。
  - **memory-constrained speedup（内存受限加速比）**：取"能装进（随 P 线性增长的）内存的最大问题"，只对 O(N) 算法成立。
  - **time-constrained speedup（时间受限加速比）**：取"在我吃完午饭回来之前能算完的最大问题"。
- **算法复杂度不变性**：并行化把 T(N) 乘上一个常数因子（上限由 Amdahl 定律给出），**不会改变 O(·)**。因此"N 很大时并行一定赢"是错的——若算法本身是 $O(N^2)$，堆再多处理器也追不上一个 O(N log N) 的串行算法。讲义强调：**并行编程的第一件事是选/造可扩展的算法，第二件事才是写 CUDA**。

Amdahl 定律与并行开销分类（Amdahl's Law and Parallel Overhead）

- **定义与目的**：Amdahl 定律给出并行加速比的**硬上界**：加速比 ⩽ 1 / （不可并行部分占比）。它解决的是"预期管理"问题——在上手写代码之前就告诉你值得投入多少。配套的"并行开销分类"则告诉你效率损失具体浪费在哪里。
- **直观解释（"它是什么？"）**：想象 10 个人合作做一顿饭，但**只有一个人能用炉子**（烧菜占 75% 时间中的一部分必须串行，或者更贴切地说：切菜可以 10 人并行，但烧菜环节只能一个人守着一口锅）。即使切菜时间被压缩到 0，总时间也降不下来超过 1/(串行占比)。如果串行部分占 25%，那么无论请多少人，最多只能快 4 倍——**第 5 个人的工资白付**。
- **架构/机制图解**：

```
固定问题规模 N，串行占比 s = 0.25，可并行部分 1−s = 0.75

T(1)  = s + (1−s)              = 0.25 + 0.75 = 1.00
T(4)  = s + (1−s)/4            = 0.25 + 0.1875 = 0.4375   -> Speedup = 2.29
T(16) = s + (1−s)/16           = 0.25 + 0.0469 = 0.2969   -> Speedup = 3.37
T(∞)  = s + 0                  = 0.25          = 0.2500   -> Speedup = 4.00  <== 上界

时间条（每格 = 0.05 单位）：
P=1   [############ 串行 0.25 ############][ 可并行 0.75 ........................................]
P=4   [############ 串行 0.25 ############][ 0.1875 ][ 空闲/等待 ..............................]
P=16  [############ 串行 0.25 ############][0.047][ 空闲 ....................................]
P=∞   [############ 串行 0.25 ############][ 空 ..........................................]
                                            ^
                          Amdahl 上界 = 1 / s = 1 / 0.25 = 4×
```

- **关键操作与性能特征**：

- **公式**：设串行占比 s（0 ⩽ s ⩽ 1），处理器数 P，则 `Speedup(P) = 1 / ( s + (1 − s) / P )`，`lim(P→∞) = 1/s`。
- **具体数字表（代入计算）**：

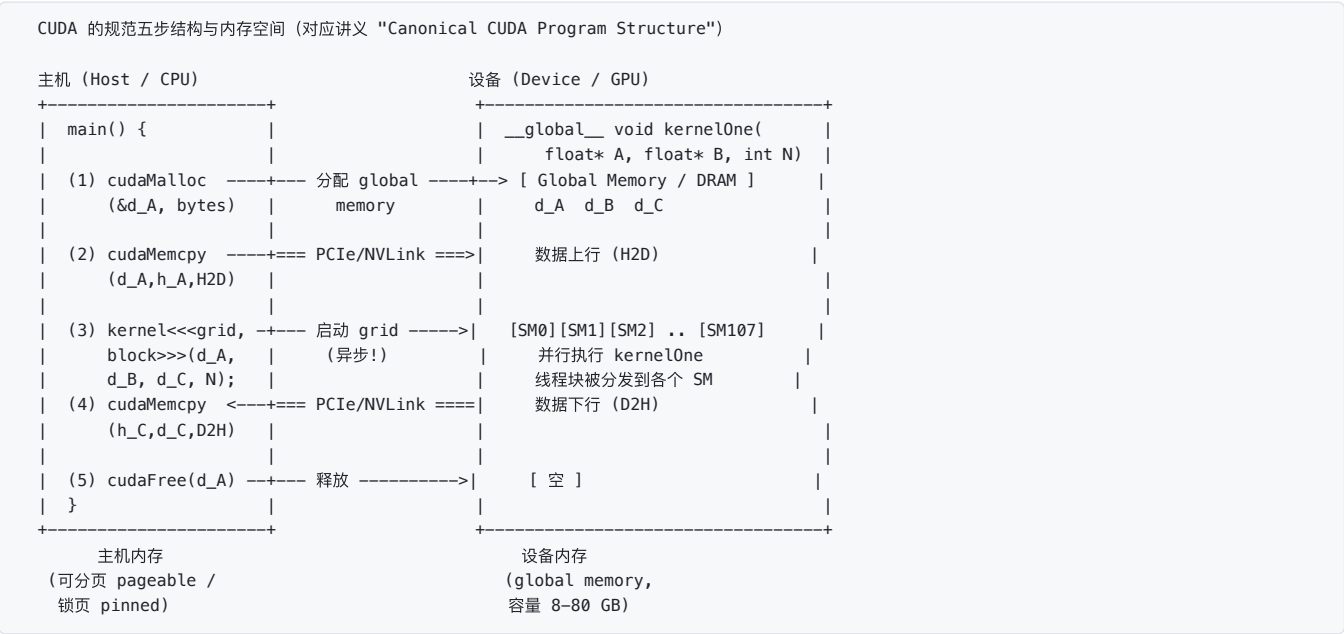
s	P=2	P=4	P=8	P=16	P=64	P=1024	上界 1/s
0.25（并行化 75%）	1.60	2.29	2.91	3.37	3.88	3.99	**4.00**
0.10（并行化 90%）	1.82	3.08	4.71	6.40	8.77	9.91	**10.00**
0.01（并行化 99%）	1.98	3.88	7.48	13.91	39.26	91.17	**100.00**

```
代入一例：s = 0.10、P = 64 时 1 / (0.10 + 0.90/64) = 1 / (0.10 + 0.0140625) = 1 / 0.1140625 = 8.77×，效率 = 8.77/64 = 13.7%—**从 10% 串行
```

- **讲义给出的例子**：如果只并行化了 75% 的代码，就**永远拿不到超过 4× 的加速比**。
- **并行开销分类**（讲义 "Necessary/Good Sources" 与 "Bad Sources of Parallel Overhead"）：
  - **必要且合理**：(a) 搬数据（通信）——大多数并行代码的必要开销；(b) **做额外计算来省通信**，例如 CNN 中在卷积之后立刻做池化（pooling）以减少"共享内存 → 全局内存"的流量，或 Kogge–Stone scan 中多做加法以减少 barrier 数量；(c) 争抢共享资源带来的优先级仲裁。
  - **纯粹浪费**：(a) 干等长延迟事件（wait for long-latency events）；(b) 看别人干活——GPU 上的**分支发散（branch divergence）**是最典型的例子；(c) 排成一列——**不必要的串行化**，例如过粗的同步、缺少私有化（privatization）、对共享硬件资源的时间相关访问、**只用一个 CUDA stream**。
- **负载均衡（Load Balance）**：讲义原话是"**完成一个并行任务的总时间，由最慢的那个线程决定**"。固定"每线程一份工作"最简单，但会导致负载不均（例如上/下三角矩阵的行长度不同）。常见解法是**动态负载均衡**：一个或多个工作队列，线程从队列取一块活、干完再取，先取大块后取小块，队列空了就从别的线程**偷工作（work stealing）**。
- **粒度（parallel grain size）**：即"每个线程做多少工作"。不同并行源的自然粒度不同：循环体、容器中的对象、矩阵的行/列/块/元素、图的节点/连通分量。**粒度选择要比较各方案的负载方差与分支发散**：矩阵元素通常工作量近似恒定（规整性好）；而"循环体里带条件判断"、"每个对象调用复杂方法"、"上/下三角矩阵的行"、"图中节点的度、连通分量的大小"则方差很大。
- **骨架式同步执行（bulk synchronous execution）**：HPC 与 CUDA 应用的主导风格。barrier 把代码切成时间区域（phase），每个 phase 通常只有 O(100) 行，交错与数据共享只发生在 phase 内部。好处是"调试一个区域比调试整个程序容易"；代价是**会使用量在时间上相关（resource usage correlates），这是坏的**——所有 warp 同时抢同一个执行单元。

异构计算模型与 PC 架构中的 GPU（Heterogeneous Computing, Host + Device, PCIe/NVLink）

- **定义与目的**：异构计算模型把"串行/控制密集部分"交给 CPU（host），把"数据并行/计算密集部分"交给 GPU（device），两者通过 PCIe/NVLink 互连。它解决的问题是：**既然 CPU 与 GPU 各有所长，就不该二选一，而该让它们协作**。代价是必须显式管理两套内存空间和它们之间的数据搬运。
- **直观解释（"它是什么？"）**：CPU 是**总部办公室**（会开会、做决策、处理复杂流程），GPU 是**远郊大工厂**（只会重复做同一道工序，但产能惊人）。总部把原料用卡车（PCIe）运到工厂，工厂加工完再运回成品。**卡车一趟只能运那么多，而且路上要花时间**——如果工厂加工只要 1 分钟而运料要 40 分钟，那么优化工厂本身毫无意义。这就是 Lecture 19 的核心结论："**现代计算机系统的重力是带宽**"（Bandwidth: The Gravity of Modern Computer Systems）。
- **架构/机制图解**：



```
经典 PC 架构 vs 现代 PCIe 架构（对应讲义 "Classic (Historical) PC Architecture" 与
"Recent PCIe PC Architecture"）

【经典：Northbridge / Southbridge】
     CPU
      |
 +----+----+                 Northbridge（北桥）：连接必须高速通信的三者
 | 北桥    |----------------- CPU / DRAM / 显卡
 +----+----+                 显卡需要"一等公民"级的 DRAM 访问权
      |                      早期 NVIDIA 卡走 AGP，峰值 2 GB/s
   DRAM
      |
 +----+----+
 | 南桥    |----------------- 慢速 I/O 集中器：PCI / USB / SATA / 音频
 +---------+
      |
 【原始 PCI 总线】33 MHz、32-bit、峰值 132 MB/s
                 后来 66 MHz、64-bit、峰值 528 MB/s
                 对设备而言上行带宽仍然只有约 256 MB/s 峰值
                 共享总线 + 仲裁：仲裁赢家成为 bus master
                 PCI 设备寄存器映射进 CPU 物理地址空间（memory-mapped I/O）

【现代：PCIe 交换网络】
    CPU（内含 PCIe Root Complex / 控制器）
     |  PCIe x16 链路
 +---+-----------------------------+
 |  PCIe Switch（= 现代"北桥"）     |
 +---+--------------+--------------+
     |              |
[GPU 显卡]      [NVMe SSD]
PCIe x16        PCIe x4
点到点、交换式、无仲裁；报文组成虚拟通道；可对报文划分优先级做 QoS
（例如实时视频流）。
```

- **关键操作与性能特征**：
- **PCIe 编码与带宽**：PCIe 用 **128b/130b 编码**（每 16 字节数据编成 130 bit，其中 1 与 0 个数相等，保证 DC 平衡与足够的跳变供时钟恢复），开销仅 1.5%；相比旧的 **8b/10b 编码**（20% 开销、要求任意 20-bit 流中 1/0 数量差 ≤ 2 且不允许连续 5 个以上相同位）大幅提升。128b/130b 用**加扰器（scrambler）**替代硬性游程限制，只需保证每 66 bit 至少一次跳变。若需要 2^{128} 个码字（取自全部 2^{130} 个 130-bit 模式），每个码字中 0 或 1 的个数必须在 **63—67** 之间——即码字相当平衡且富含 0-1 跳变。
- **净数据率（PCIe 3.0，8 GT/s）**：每 lane 每方向 **985 MB/s**；链路可由 1/2/4/8/12/16 条 lane 组成（x1、x2、x4、x8、x16），每 lane 1 bit 宽（4 根线，两对差分，上下行同时且对称）。于是：

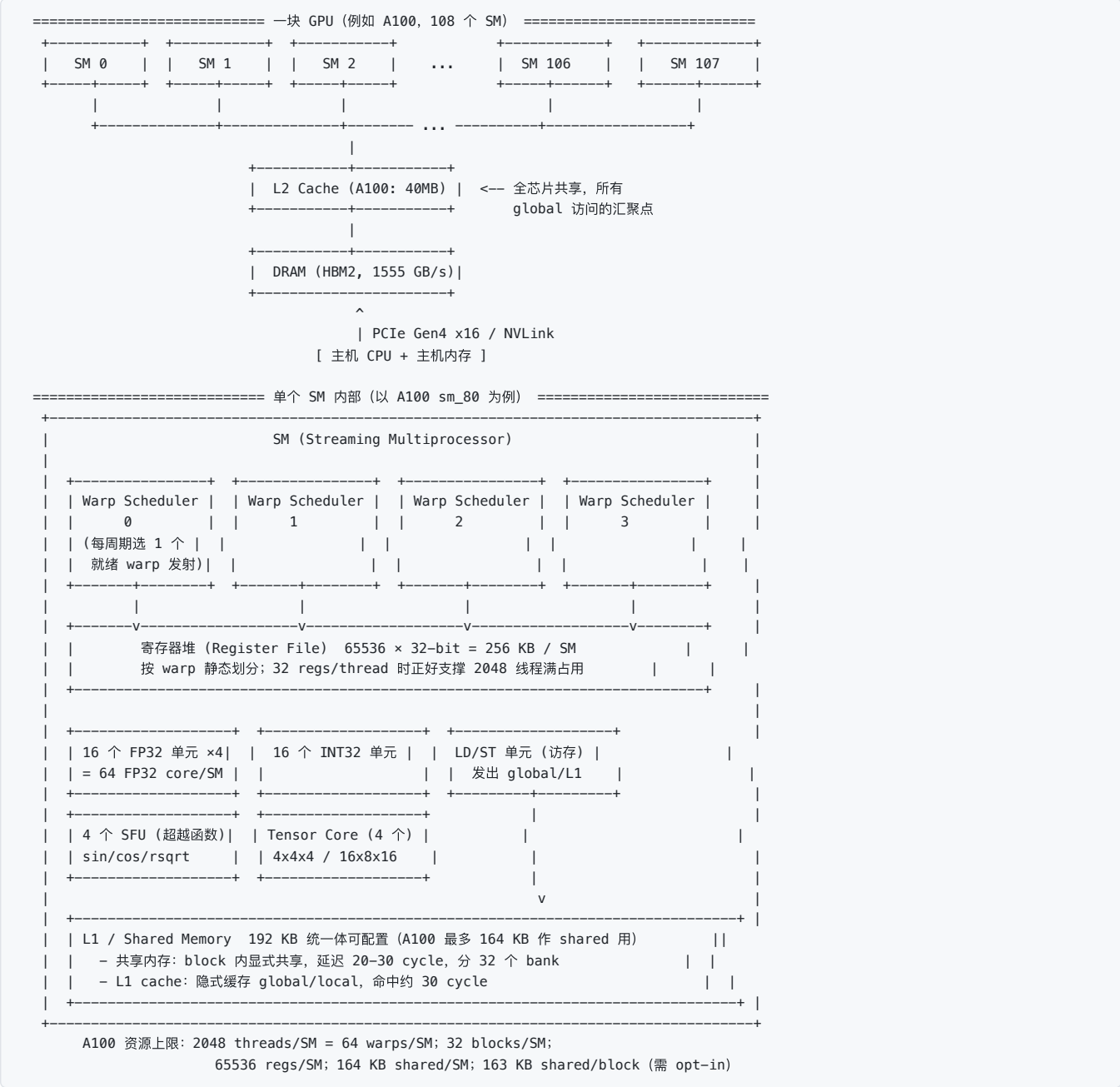
链路宽度	x1	x2	x4	x8	x16
净带宽（每方向）	985 MB/s	1.97 GB/s	3.94 GB/s	7.9 GB/s	**15.8 GB/s**

- **代际**：每代目标翻倍。PCIe 3.0（8 GT/s，极广泛部署）→ PCIe 4.0（16 GT/s，现代 AMD/Intel/IBM 系统，**A40 即 PCIe Gen4，x16 双向 64 GB/s，单向约 32 GB/s**）→ PCIe 5.0（32 GT/s，目前仅在极少数系统如 IBM Power10 上支持）。
- **NVLink 的数量级**：NVLink 是 GPU 之间的直连互连，绕开 PCIe。Ampere 第三代 NVLink 的 GPU-GPU 单向带宽可达 **600 GB/s**，约为 PCIe Gen4 的近 10 倍；P100 上 8 卡混合立方网格（hybrid cube mesh）双向 160 GB/s，是 PCIe Gen3 x16 的 5 倍；V100 的 NVLink 2.0 为 25 GB/s/lane × 6 lane = **150 GB/s**；Delta 上的 A40 为 **112.5 GB/s 双向**。IBM Power9 系统中 2 颗 Power9（各 10 核 80 线程 @4.02 GHz、256 GB DDR4）挂 4 张 V100（各 16 GB），GPU 间 NVLink 150 GB/s，每 CPU 到 GPU 120 GB/s，两 CPU 之间 X-bus 64 GB/s。
- **DMA 与锁页内存（pinned memory）**：PCIe 传输用 **DMA（Direct Memory Access）**才能吃满总线带宽。DMA 使用**物理地址**，因此需要一个不会被操作系统换出（page out）的缓冲区——否则 OS 可能在 DMA 读写期间把页换走，把别的虚拟页换到同一物理位置，导致数据损坏。这类"不能被换页"的内存叫**锁页内存 / 页锁定内存（pinned / page-locked memory）**。
  - 用 `cudaHostAlloc(&ptr, bytes, cudaHostAllocDefault)` 分配，用 `cudaFreeHost(ptr)` 释放；用法与 `malloc` 完全相同，唯一区别是 OS 不能对它换页。
  - 若 `cudaMemcpy()` 的主机端缓冲区**不是**锁页的，运行时必须先在"分页内存 → 一块内部锁页的暂存区（staging buffer）"之间做一次额外的 CPU 拷贝，再启动 DMA。**因此 `cudaMemcpy` 在源或目标为锁页内存时通常快约 2 倍**。
  - 锁页内存是**有限资源**，超额申请会有严重后果（可能拖垮整个系统），不要随手给大数组用。
- **数据搬运的代价（本讲量化核心）**：以 A100（PCIe Gen4 x16，单向理论 32 GB/s，锁页实测约 25 GB/s）为例，向量加法 N = 2^{24} 个 float：
  - 上行（A、B 两个数组）：2 × 67.11 MB = 134.22 MB ÷ 25 GB/s ≈ **5.37 ms**
  - kernel 计算：约 **0.14 ms**（见后文性能分析）
  - 下行（C 数组）：67.11 MB ÷ 25 GB/s ≈ **2.68 ms**
  - **传输总计 ≈ 8.05 ms，是 kernel 时间的 57 倍**。若用可分页内存（实测约 6—8 GB/s），上行就要 ~19 ms，恶化到 130 倍以上。

结论：**当 kernel 的算术强度很低时，"把数据搬上搬下"就是整个应用的全部成本**。工程对策是：让问题规模尽量大以摊薄固定开销；用锁页内存；用多 stream 把传输与计算重叠；更重要的是——**尽量让数据一次上传后在 device 上被多个 kernel 反复使用**（这正是后续 tiling、融合 kernel 等技术的动机）。 – **讲义的产物与趋势**：GPU 与 CPU 正在融合（同一封装、共享内存控制器），PC 世界正在"变平"（The PC world is becoming flatter），计算外包（outsourcing of computation）越来越容易。

众核吞吐量处理器：SM 与 SIMT 执行模型（Streaming Multiprocessor and SIMT）

- **定义与目的**：SM（Streaming Multiprocessor，流多处理器）是 GPU 内部可独立调度线程块的**处理器核心**；SIMT（Single Instruction, Multiple Threads，单指令多线程）是 CUDA 的执行模型，它把一个 warp 的 32 个线程组织成"共享同一条指令、但各自拥有独立寄存器和独立执行状态"的执行单元。SIMT 解决了"既要写标量风格的 C 代码，又要让硬件高效地一次驱动 32 个线程"这个核心矛盾。
- **直观解释（"它是什么？"）**：把 SM 想象成**一个 32 人组成的合唱团声部**（warp）：指挥（warp scheduler）举起一个手势（一条指令），32 个人同时唱同一个音。每个人有自己的嗓子（寄存器）和自己的乐谱页码（线程索引），所以"同一句歌词"唱出来可以是不同的音高——这就是 SIMT 与 SIMD 的区别：**SIMD 是数据打包进向量寄存器，SIMT 是 32 个独立线程被硬件强行同步节奏**。如果乐谱上写着"男高音唱这句，其他人闭嘴"，那么这个声部就得唱两遍（一次男高音、一次其他人）——这就是**分支发散（divergence）**。
- **架构/机制图解**：自顶向下，从整卡到单个线程的层次结构：



```
============================ warp / block / grid 层次 ============================
 Grid (一次 kernel 启动的全部线程；最多 2^31-1 个 block 在 x 维)
  |
  +-- Block(0,0) --> 被调度到【某一个】SM 上，整个 block 生命周期不迁移
  |     |
  |     +-- Warp 0 = Thread(0..31)   <-- 硬件以 32 线程为单位调度
  |     +-- Warp 1 = Thread(32..63)
  |     +-- Warp 2 = Thread(64..95)   <-- 最后一个 warp 可能不满 32 线程，
  |     |                                  此时"空"的 lane 仍然占发射槽（浪费）
  |     +-- 共享内存 / __syncthreads() 只在这个 block 内有效
  |
  +-- Block(1,0) --> 可能在同一 SM，也可能在另一个 SM
  |
  +-- Block(N-1) --> block 之间的执行顺序【完全没有保证】

索引计算（务必背下来）：
   int i = blockIdx.x * blockDim.x + threadIdx.x;      // 一维全局线性索引
   gridDim.x  = 网格 x 维的 block 个数
   blockIdx.x = 本 block 在 x 维的编号（0 .. gridDim.x-1）
   blockDim.x = 本 block 在 x 维的线程数（= <<<..., block>>> 里的 block.x）
   threadIdx.x= 本线程在 block 内的 x 编号（0 .. blockDim.x-1）
```

- **关键操作与性能特征**：
- **线程是调度的基本单位，warp 是执行的原子单位**：block 内的线程按 `threadIdx` 线性顺序**每 32 个切成一个 warp**。`threadIdx.x = 0..31` 是 warp 0，`32..63` 是 warp 1，以此类推。
- **SIMT 与发散**：一个 warp 的 32 个 lane 共享同一个 PC（程序计数器）。遇到分支且条件在不同 lane 上不一致时，硬件**串行化**执行各分支路径，只让属于该路径的 lane 有效：

```
if (tid % 2 == 0) { A; } else { B; }
32 个 lane 的取值：  0 1 2 3 4 5 6 ... 31
tid%2==0 的掩码：    1 0 1 0 1 0 1 ... 0

第 1 遍：执行 A，掩码 = 101010...10   （16 个 lane 有效，16 个闲置）
第 2 遍：执行 B，掩码 = 010101...01   （16 个 lane 有效，16 个闲置）
-----------------------------------------------------------------
总时间 = t(A) + t(B)，而非 max(t(A), t(B))
利用率 = 16/32 = 50%  -->  这就是"看别人干活"式的纯浪费
```

- **block 之间无同步、无顺序保证**：这是 CUDA 可扩展性的来源（block 数量可以远超 SM 数量、可以任意顺序调度），也是"跨 block 依赖必须靠多次 kernel 启动（或 2012 年起的**动态并行 dynamic parallelism**，允许 block 启动子 kernel 并等待其完成）来表达"的原因。
- **调度与常驻量（用统一硬件档案代入）**：

GPU	SM 数	每 SM warp 上限	每 SM 线程上限	每 SM block 上限	一次可常驻总线程
A100	108	64	2048	32	221 184
H100 SXM	132	64	2048	32	270 336
RTX 4090	128	48	1536	24	196 608
RTX 2080 Ti	68	32	1024	16	69 632

- **寄存器预算**：A100 每 SM 65536 个 32-bit 寄存器。要跑满 2048 线程，平均每线程只能用 `65536 / 2048 = 32` 个寄存器。若 nvcc 为你的 kernel 分配了 40 个寄存器，则每 SM 最多驻留 `65536 / (40×32) = 51.2 → 51` 个 warp，占用率降到 `51/64 = 79.7%`。
- **共享内存预算**：A100 每 SM 164 KB 可作 shared 使用。若每个 block 用 48 KB，则每 SM 最多 3 个 block（`164/48 = 3.4`）；即使线程数允许 8 个 256 线程的 block，shared 也把常驻量压到 3 个。
- **历史题（讲义 Lecture 19 "Problem Solving" 页原文考点）**：某 GPU 的 compute capability（CC）1.3 的 SM 上限为：1024 threads/SM、8 blocks/SM、16384 registers/SM、16 KB shared memory/SM（即 32 warps/SM）。给定四个 kernel：

kernel	threads/block	regs/thread	shared/block	受线程限	受寄存器限	受共享内存限	受 block 数限	实际 block 数	占用率
K1	256	20	8 KB	4	`16384/(20×256)=3.2→3`	`16/8=2`	8	**2**	512/1024 = 50%
K2	128	12	2 KB	8	`16384/(12×128)=10.6→10`	`16/2=8`	8	**8**	1024/1024 = **100%**
K3	512	32	4 KB	2	`16384/(32×512)=1`	`16/4=4`	8	**1**	512/1024 = 50%
K4	64	64	16 KB	16	`16384/(64×64)=4`	`16/16=1`	8	**1**	64/1024 = 6.25%

答案是 K2 达到最大占用率。这道题把"**占用率 = 四类资源上限取最小值**"这条规则讲透了：residentBlocks = min(byWarps, byThreads, byRegs, byShare

延迟隐藏与占用率（Latency Hiding and Occupancy）

- **定义与目的**：延迟隐藏（latency hiding）指当某个 warp 因为等待数据（访存、纹理、分支）而停顿时，硬件**立刻切换到另一个就绪 warp** 去发射指令，从而让执行单元始终有活干。占用率（occupancy）是"每 SM 实际常驻 warp 数 ÷ 每 SM 最大 warp 数"，它是延迟隐藏**能力的上限指标**。这套机制解决的是 GPU 最根本的工程矛盾：**显存延迟长达 400–800 周期，而 GPU 只有很浅的缓存层次，靠什么不空转?**
- **直观解释（"它是什么？"）**：想象银行有 64 个窗口（warp 槽位），但每个客户办业务时都要等后台调档案（400–800 周期）。如果只开 1 个窗口，柜员每次都要干等档案送来，一天办不了几笔。如果 64 个窗口全开，柜员在每个窗口都留下"等待中"的客户，谁的材料到了就先办谁——**没有人被加速，但整个大厅的吞吐量拉满了**。这就是 GPU 的设计哲学：**不减少延迟，而是用并发掩盖延迟**。用讲义原话："GPU 需要海量线程来容忍延迟（Require massive number of threads to tolerate latencies）"。
- **架构/机制图解**：

```
同一个 SM 上 4 个 warp scheduler 的时间线（每周期每个 scheduler 可发射 1 个 warp 的 1 条指令）

周期:      0    1    2    3    4    5    6    7   ...  400  401  402
------------------------------------------------------------------------
Sched 0:
  warp 0  [LDG]..................................................[用数据]
          ^ 发出全局加载，进入"未就绪"状态
  warp 32          [FADD]  <- 切换! warp 32 的就绪指令立刻被发射
  warp 1               [IMAD]
  warp 33                   [LDG]..............................
  warp 2                         [FADD]
  ...                             ...  (轮流发射，执行单元 100% 忙)

------------------------------------------------------------------------
【关键】执行单元的利用率 = min(1, 可发射的 warp 数 / 需要填补的延迟槽位数)
       常驻 warp 越多 -> 越容易找到就绪 warp -> 延迟隐藏越充分

【Little 定律：算清楚"要多少 warp 才够"】
   需要的在途字节数 = 内存延迟 × 内存带宽
   A100: 700 cycle / 1.41 GHz = 497 ns
         497 ns × 1555 GB/s = 772 KB 需要同时在途
         772 KB / 108 SM        = 7.15 KB / SM
         7.15 KB / 128 B 每 warp = 约 56 个 warp 的在途 load
         ==> A100 每 SM 最多 64 个 warp，用纯 load 打满带宽几乎需要满占用!

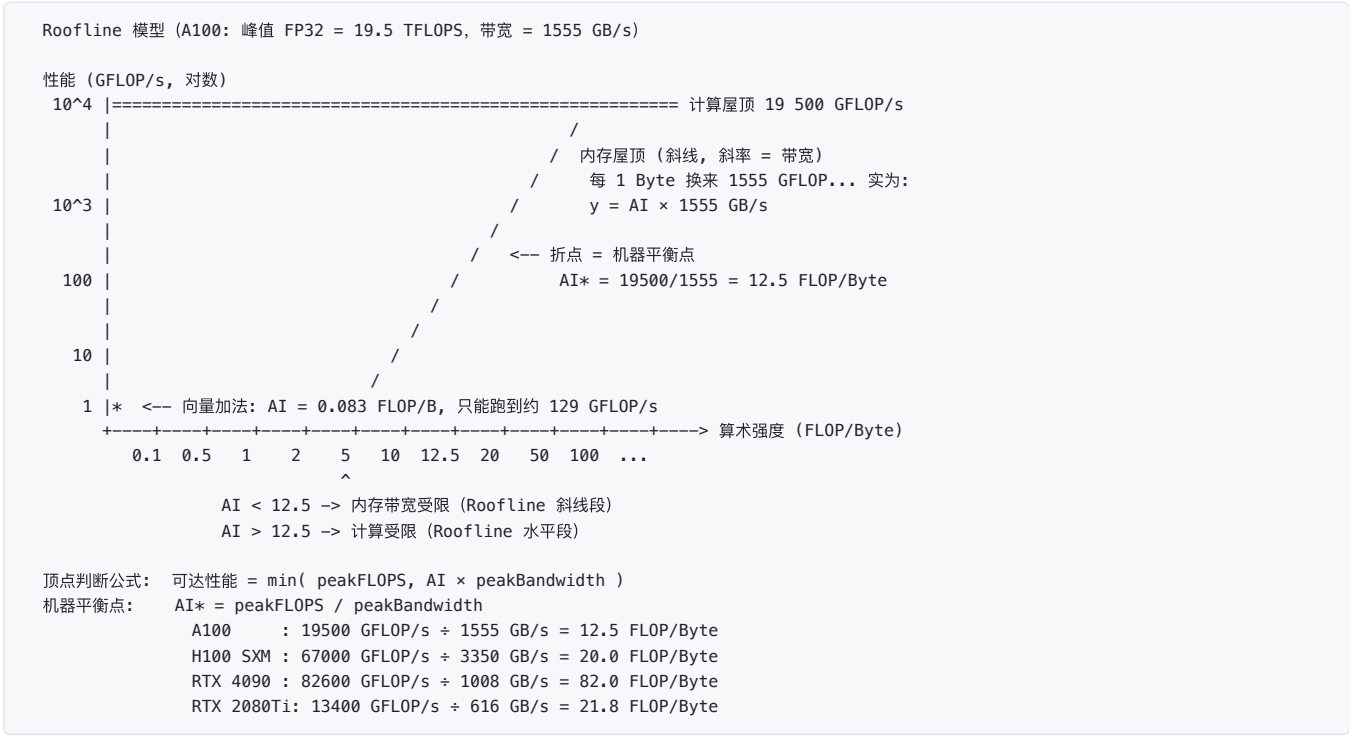
   H100: 700/1.98 GHz = 354 ns × 3350 GB/s = 1.184 MB
         1.184 MB / 132 SM / 128 B = 约 70 个 warp  <-- 超过 64!
         ==> H100 单靠占有率达到 100% 也不够，必须靠 ILP（每线程多条独立 load）

   RTX 4090: 700/2.52 GHz = 278 ns × 1008 GB/s = 280 KB
         280 KB / 128 SM / 128 B = 约 17 个 warp（上限 48）
         ==> 仅约 36% 占用率就足以打满带宽
------------------------------------------------------------------------
延迟层级（写代码时必须记住的数字）:
  寄存器             ~0 cycle        每线程私有，最快
  共享内存 / L1 命中 20-30 cycle     block 内共享，需 __syncthreads()
  L2 命中            ~200 cycle       全芯片共享（A100: 40 MB）
  全局内存 (DRAM)    400-800 cycle    "墙"；必须靠并发掩盖
  PCIe H2D/D2H       微秒级           比 DRAM 再慢 1-2 个数量级
```

- **关键操作与性能特征**：
- **占用率的准确定义**：`occupancy = 每 SM 实际常驻 warp 数 / 每 SM 最大 warp 数`。A100 上 `64/64 = 100%`。
- **三级限制链**：`每 SM 常驻 block 数 = min(线程上限, warp 上限, 寄存器上限, 共享内存上限, 硬件 block 上限)`；`每 SM 常驻 warp 数 = 每 SM block 数 × 每 block 的 warp 数`。
- **寄存器分配粒度会让理论值和实际值有出入**：nvcc 按 **warp 为单位**、并以 256 个寄存器为粒度向上取整分配（不同 arch 略有差异），所以 `65536 / (numRegs × 32)` 的手算值可能与 `cudaOccupancyMaxActiveBlocksPerMultiprocessor()` 的返回值差 1 个 block。**以运行时 API 为准，手算用于理解原理。**
- **占用率不是越高越好**：占用率高只是"有足够多的 warp 可切换"。若每个线程可用寄存器变少导致**寄存器溢出（register spilling，把变量挤到 local memory）**，性能反而暴跌。常见的工程折中是 50%–75% 占用率，配合每线程 2–4 条独立访存（ILP）。可以用 `__launch_bounds__(maxThreadsPerBlock, minBlocksPerMultiprocessor)` 告诉编译器"我要几个 block"，让它把寄存器压到预算内。
- **实测建议**：Lab 0 的 Device Query 应打印出"不同 blockDim 下的理论占用率表"，这样在后续 Lab 中调 `blockDim` 时能立刻看出瓶颈是寄存器、共享内存还是 block 数上限。

内存带宽与算术强度（Memory Bandwidth and Arithmetic Intensity）

- **定义与目的**：算术强度（arithmetic intensity, AI）指程序**每搬运 1 字节数据所完成的浮点运算次数**，单位 FLOP/Byte。它与机器的"平衡点"（peak FLOPS ÷ peak bandwidth）比较，就能在写代码之前判断程序是**内存带宽受限**还是**计算受限**。这是 GPU 编程中投入产出比最高的一次分析。
- **直观解释（"它是什么？"）**：把 GPU 想象成**一个超级厨师（算力）配一个很小的传菜窗口（带宽）**。如果菜谱是"把一块豆腐切成丝"（每克原料要做很多刀 = 高算术强度），那么厨师的手速是瓶颈，窗口再小也无所谓。如果菜谱是"把一箱矿泉水从门口搬到仓库"（每克原料只做 0.08 次运算 = 极低算术强度），那么厨师再快也没用——**瓶颈是门口那条通道**。GPU 的厨师比 CPU 强 10 倍以上，但它的窗口（1555 GB/s）相对于它的手速（19.5 TFLOPS）其实很窄，所以**绝大多数朴素 CUDA 程序都是"搬水工"，不是"切丝师傅"**。
- **架构/机制图解**：



- **关键操作与性能特征**：
- **合并访问（coalescing）决定了实际能拿到多少带宽**：一个 warp 的 32 个线程若访问**连续的 32 个 4 字节字**，硬件会把它合成**一次 128 字节的事务（transaction）= 一条 cache line**；若访问步长是 32（strided），则 32 个线程散落在 32 条不同的 cache line 上，需要 **32 次 128 字节事务 = 4096 字节**才能满足 128 字节的有用数据，**有效带宽掉到 1/32**。这是 GPU 性能最常见的"隐形杀手"。
- **典型 kernel 的算术强度（先算再写）**：

kernel	每元素访存	每元素 FLOP	AI (FLOP/B)	A100 上判定
向量加法 C=A+B	12 B（读 A、B，写 C）	1	**0.083**	极度带宽受限（差 150 倍）
归一化 A[i]*=s	8 B（读+写）	1	0.125	带宽受限
点积 / reduction	4 B（只读）	1	0.25	带宽受限
3×3 卷积（每输出）	36 B（9 次读）	17	0.47	带宽受限
3×3 卷积 + tiling（每输出）	约 4.5 B（摊薄后）	17	~3.8	接近平衡点
稠密矩阵乘 N×N×N	每输出 8N B	2N	0.25	朴素版带宽受限
稠密矩阵乘 + tiling	每输出约 8N/√tile B	2N	随 tile 增大	可转为计算受限

- **由 AI 直接算出预期耗时（A100，N = 2^{24} 个 float）**：
  - 访存流量 = `3 × N × 4 B = 3 × 16 777 216 × 4 = 201.33 MB`
  - 内存受限下界 = `201.33 MB ÷ 1555 GB/s = 129.5 μs`
  - 计算量 = `N × 1 FLOP = 16.78 MFLOP`；计算下界 = `16.78e6 ÷ 19.5e12 = 0.86 μs`
  - 结论：**两者相差 150 倍，耗时完全由带宽决定**。要达到 129.5 μs 需要 100% 带宽利用率——实践中典型只能跑到 85%–93%，即 140–152 μs。
- **"内存墙"的长期趋势**：Amir Gholami 等人在 RiseLab 博客（2021）指出，过去 20 年 GPU/加速器的**峰值算力增长远快于内存带宽增长**（FLOPs/Byte 的平衡点持续上升，从 GPU 早期约 10 提升到 H100 的 ~20、RTX 4090 的 ~82）。这意味着"内存墙"越来越紧：**未来的 CUDA 优化只会越来越偏向"减少字节搬运"，而非"减少指令数"**。这条趋势是理解 tiling、kernel fusion、量化、稀疏化等所有现代技术的总纲。

统一着色器架构与 Tesla 架构：让 CUDA 成为可能的硬件基础（Unified Shader Architecture / Tesla）

- **定义与目的**：统一着色器架构（unified shader architecture）指把图形流水线中原本**硬件独立、数量固定比例**的顶点着色器（vertex shader）与像素/片段着色器（pixel/fragment shader）单元，合并成**一组通用的可编程标量处理器（后来被称为 CUDA core）**，它们可以执行任意着色阶段、也可以执行通用计算。它在 2006 年（NVIDIA G80，Tesla 架构）落地，并在 2007 年通过 CUDA 暴露给程序员——**没有统一着色器架构，就没有 CUDA**。
- **直观解释（"它是什么？"）**：旧图形流水线像**一条专用装配线**：工位 A 只装轮子（顶点着色）、工位 B 只装车门（像素着色），而且必须按 1:3 的固定比例配人。如果今天的车型不需要那么多车门工序，工位 B 的人就只能闲着。统一着色器架构相当于把所有人都培训成**多能工**：今天需要 5 个人装轮子就派 5 个，明天需要 3 个装车门就派 3 个。更进一步——**既然这些多能工什么都能干，那把非图形的通用计算（物理、线性代数、图像处理）交给他们做，不是理所当然吗？** 这就是 GPGPU 与 CUDA 的诞生逻辑。
- **架构/机制图解**：

```
【2004–2005：分离的着色器（fixed-ratio shader）】
+----------------------+     +----------------------+     +----------------------+
| Vertex Shader 阵列   |     | Triangle Setup / Rasterizer |  | Pixel Shader 阵列 |
| (VS 单元，数量固定) |---->|                              |--->| (PS 单元，固定) |
+----------------------+     +----------------------+     +----------------------+
     VS : PS 的比例由硬件焊死（例如 1:3）。
     哪一段忙、哪一段闲，完全无法调配 -> 利用率低、且无法做通用计算。

【2006：G80 / Tesla 统一着色器架构】
+-------------------------------------------------------------------------------+
|  统一的标量处理器阵列（Unified Shader / 后来叫 CUDA core）                    |
|  同一个物理单元，既能跑 vertex shader、也能跑 pixel shader、                  |
|  还能跑程序员写的 __global__ kernel —— 三种用途共用同一批硬件。              |
+-------------------------------------------------------------------------------+
     G80 规模：128 个 SP（Streaming Processor）分布在 16 个 SM 上（每 SM 8 个）
     G80 频率：1.35 GHz   |   每 SM 768 线程（24 warp）   |   16 KB shared / SM
     显存带宽：86.4 GB/s（GeForce 8800 GTX）

【从 G80 到今天的谱系（CUDA compute capability）】
2006 Tesla     G80       CC 1.0/1.1   共享内存首次对通用计算开放
2008 Tesla     GT200     CC 1.3       <-- 讲义 Lecture 19 "Problem Solving" 用的就是这个
2010 Fermi     GF100     CC 2.0       L1/L2 可配置、真正的缓存层次、ECC、并发 kernel
2012 Kepler    GK110     CC 3.5       动态并行、Hyper-Q、更大寄存器堆（65536/SM）
2014 Maxwell   GM204     CC 5.2       每 SM 共享内存大幅提升、能效比飞跃
2016 Pascal    GP100     CC 6.0       HBM2、NVLink、统一内存
2017 Volta     GV100     CC 7.0       独立线程调度、Tensor Core 第一代
2018 Turing    TU102     CC 7.5       RT Core、每 SM 64 FP32 lane
2020 Ampere    GA100     CC 8.0       <-- ECE408 的 sm_80 目标：108 SM、164 KB shared/SM
2020 Ampere    GA10x     CC 8.6       每 SM 128 FP32 lane（RTX 30 系）
2022 Hopper    GH100     CC 9.0       132 SM、228 KB shared/SM、TMA、第四代 NVLink
2022 Ada       AD102     CC 8.9       <-- RTX 4090：128 SM、1008 GB/s、82.6 TFLOPS
2024 Blackwell GB100     CC 10.0      第五代 Tensor Core、FP4
```

- **关键操作与性能特征**：
- **CUDA 发布的时间与动机**：2007 年 NVIDIA 推出 CUDA，让"给这些硬件写通用程序"变得**相当容易**（讲义 "Meanwhile, in Another Part of the Industry"）。在此之前，要用 GPU 做通用计算必须把算法伪装成图形渲染（"用纹理做数组、用像素着色器做计算"），门槛极高。
- **G80 到 A100 的规模跃迁**：SM 数 16 → 108（6.75 倍），每 SM 线程数 768 → 2048（2.7 倍），共享内存每 SM 16 KB → 164 KB（10 倍），显存带宽 86.4 GB/s → 1555 GB/s（18 倍），峰值 FP32 约 0.35 TFLOPS → 19.5 TFLOPS（56 倍）。**算力增长快于带宽增长，这正是内存墙变紧的量化证据。**
- **CC 1.3 的 SM 上限（历史基线）**：1024 threads/SM、8 blocks/SM、16384 registers/SM、16 KB shared/SM、warp = 32 线程（即 32 warp/SM）。对比 A100 的 2048 threads/SM、32 blocks/SM、65536 registers/SM、164 KB shared/SM（即 64 warp/SM）——**每一维度都放大了 2 到 16 倍**，但"占用率 = 各类资源上限取 min"这条规则 20 年未变。
- **对 CUDA 程序员的实际含义**：统一着色器架构把 GPU 变成了**一台有很多简单核心、共享一个很宽的显存接口的 SIMT 机器**。因此 CUDA 的性能模型始终是三条：(1) 制造足够多的独立工作（占用率高）；(2) 让每个 warp 的访存合并且规整；(3) 把复用的数据搬到离计算更近的地方（寄存器 → 共享内存/L1 → L2 → DRAM 的层次）。**这一讲之后的所有讲座，本质上都在细化这三条。**

CUDA 五步模板、函数限定符与执行配置语法（The Five-Step Template, Qualifiers and Execution Configuration）

- **定义与目的**：五步模板是 CUDA 程序的骨架（分配 device 内存 → 拷贝入 → 启动 kernel → 拷贝回 → 释放），它在后续每一讲里都会重复出现。`__global__` / `__device__` / `__host__` 三个限定符告诉 nvcc"这个函数编译给谁、谁能调用它"，`<<<grid, block>>>` 则是 CUDA 独有的**执行配置（execution configuration）语法**，用来描述"这次启动要用多少个 block、每个 block 多少个线程"。掌握这三件事，就能写出第一个可运行的 CUDA 程序。
- **直观解释（"它是什么？"）**：
- 五步模板 = **寄快递**：先把仓库腾出来（cudaMalloc）、把货装上车（cudaMemcpy H2D）、工厂加工（kernel）、把成品运回（cudaMemcpy D2H）、把临时仓库退掉（cudaFree）。**运费（PCIe）常常比加工费（kernel）贵得多。**

- 三个限定符 = **三种员工证**：`__global__` 是"外派工程师"——由 CPU（host）招聘并派活，但在 GPU（device）上班，**必须返回 void**；`__device__` 是"工厂内部员工"——只能由 GPU 上的代码调用；`__host__` 是"总部员工"——只能在 CPU 上跑。注意 `__` 是**两个**下划线字符，写成一个下划线会编译报错。
- `<<<grid, block>>>` = **派工单**：`grid` 说明"开几条产线（多少个 block）"，`block` 说明"每条产线站几个人（每个 block 多少线程）"。24 小时运转也没关系——硬件会自动把 block 排队塞进 SM，**block 数量可以远大于 SM 数量**，这正是 CUDA "同一份代码在 10 个 SM 的旧卡和 132 个 SM 的新卡上都能跑满"的可扩展性来源。
- **架构/机制图解**：

```
==================== CUDA 五步模板（后续每讲都以此为基础） ====================

 宿主程序 (main, 跑在 CPU)                          设备 (GPU)
 ------------------------------------------------------------------------------
 【准备】 host 侧 malloc + 初始化数据
          h_A, h_B, h_C

 【第 1 步】cudaMalloc(&d_A, bytes)  ------------>  在显存(global memory)分配
           cudaMalloc(&d_B, bytes)                  d_A, d_B, d_C
           cudaMalloc(&d_C, bytes)                  （返回的指针只能给 device 用！）

 【第 2 步】cudaMemcpy(d_A, h_A, bytes,  ========>  H2D（上行）
             cudaMemcpyHostToDevice)                走 PCIe/NVLink，用 DMA + 锁页内存
           cudaMemcpy(d_B, h_B, bytes,
             cudaMemcpyHostToDevice)                <<< 这一步常常是总耗时的大头 >>>

 【第 3 步】dim3 grid(ceil(N/256), 1, 1);   ------>  启动 grid
           dim3 block(256, 1, 1);                  硬件把 block 分发到各 SM
           vecAddKernel<<<grid, block>>>(           每个 SM 上多个 block 并发
               d_A, d_B, d_C, N);                  block 内线程按 32 个一组切成 warp
           cudaGetLastError();   <-- 立刻查启动错误   ** 启动是异步的 **
           cudaDeviceSynchronize();                 等待 device 全部完成

 【第 4 步】cudaMemcpy(h_C, d_C, bytes,  <==========  D2H（下行）
             cudaMemcpyDeviceToHost)

 【第 5 步】cudaFree(d_A); cudaFree(d_B);  -------->  释放显存
           cudaFree(d_C);
           free(h_A); free(h_B); free(h_C);         释放主机内存

 【第 6 步（可选但强烈推荐）】与 host 计算的 golden 结果对比，打印 PASS/FAIL
 ==============================================================================

 +----------------------+---------------+----------------+-----------------------------------------+
 | 限定符                | 执行位置       | 可调用者        | 备注                                     |
 +----------------------+---------------+----------------+-----------------------------------------+
 | __host__              | host (CPU)    | host           | 默认值；可省略不写                         |
 | __global__            | device (GPU)  | host 或 device | 必须返回 void；即 kernel；用 <<<>>> 启动 |
 | __device__            | device (GPU)  | device         | 设备端辅助函数                            |
 | __host__ + __device__ | host + device | 两者皆可       | 同一个函数编译两份                         |
 +----------------------+---------------+----------------+-----------------------------------------+
 补充限定符：
   __shared__   静态共享内存（block 内共享，生命周期 = block）
   __constant__ 常量内存（64 KB，全 device 只读，通过常量缓存广播）
   __restrict__ 告诉编译器指针不重叠，允许更激进的优化
   __launch_bounds__(maxThreads, minBlocks) 约束寄存器用量以控制占用率

==================== 执行配置语法 <<<grid, block>>> ============================
    vecAddKernel<<<Dg, Db, Ns, S>>>(d_A, d_B, d_C, N);
                   |   |   |   |
                   |   |   |   +-- S   : cudaStream_t（默认 0，即默认流）
                   |   |   +------ Ns  : 动态共享内存字节数（默认 0）
                   |   +---------- Db  : dim3 block —— 每个 block 的线程数
                   +-------------- Dg  : dim3 grid  —— block 的个数
   二者都可以直接写整数（等价于 dim3(n,1,1)）：
       vecAddKernel<<<ceil(N/256.0), 256>>>(d_A, d_B, d_C, N);
   也可以显式检查边界：
       dim3 DimGrid(N/256, 1, 1);
       if (0 != (N % 256)) { DimGrid.x++; }   // 不足一个 block 也要补一个
       dim3 DimBlock(256, 1, 1);
       vecAddKernel<<<DimGrid, DimBlock>>>(d_A, d_B, d_C, N);
   上限（sm_80）：blockDim.x ≤ 1024，blockDim.x*blockDim.y*blockDim.z ≤ 1024
                  gridDim.x ≤ 2^31-1，gridDim.y/z ≤ 65535

==================== nvcc 的编译流程 ===========================================
    .cu 源文件
       |
       v
    [ nvcc ]  --->  分离 host 代码与 device 代码
       |                    |
       |                    +--> device 代码 -> PTX（虚拟 ISA）
       |                    |       |
       |                    |       v
       |                    |   [ ptxas ] -> SASS（真实机器码，如 sm_80）
       |                    |       或 由驱动中的 JIT 编译器在运行时编译
```

```
        |                   |
        +--> host 代码 -----+--> [ 系统 C++ 编译器 / 链接器 ] -> 可执行文件
```

- **关键操作与性能特征**：
- **kernel 启动是异步的**（CUDA 1.0 起即如此）：`kernel<<<grid, block>>>(args)` 语句返回时，kernel 很可能**还没开始执行**。主机若要使用 kernel 的结果，必须显式同步：`cudaDeviceSynchronize()`（等 device 全部完成）、`cudaMemcpy`（隐式同步该次拷贝）、或 `cudaStreamSynchronize(stream)`、`cudaEventSynchronize(event)`。
- **异步是性能的来源**：主机可以在 device 干活时继续准备下一批数据，或者启动另一个 stream 上的 kernel 与当前传输重叠。
- **`cudaGetLastError()` 检查启动是否成功**：如果 grid/block 配置非法（例如 `blockDim.x = 2048`）或参数错误，错误只会记录在"最后一次错误"里，**必须手动查询**，否则会被后续某个成功的 API 调用悄悄清掉，导致你完全不知道 kernel 根本没有运行。
- **host 指针与 device 指针不可混用**：`cudaMalloc` 返回的指针只能用于 device 代码和 CUDA 运行时 API；把它传给 `free()` 或直接解引用会导致崩溃或不可预知的错误。反之，把 `malloc` 的指针传给 kernel 会触发非法地址访问。
- **统一的错误检查宏**（本讲所有代码都使用它）：

```
#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)
```

代码示例与性能分析

> 示例 1：Lab 0 —— Device Query（查询并推导本机 GPU 的硬件档案）

```
// 文件: deviceQuery.cu
// 编译: nvcc -O3 -arch=sm_80 deviceQuery.cu -o deviceQuery
// 运行: ./deviceQuery
//
// Lab 0 参考实现：枚举本机 CUDA 设备，打印设备名、计算能力、SM 数、每 SM 最大线程数、
// 共享内存大小、常量内存大小、warp 大小、全局内存大小，并由 memoryClockRate 与
// memoryBusWidth 推导理论峰值显存带宽；再用 cudaOccupancyMaxActiveBlocksPerMultiprocessor
// 打印不同 blockDim 下的理论占用率，最后给出推荐的启动配置。

#include <cstdio>
#include <cstdlib>
#include <climits>
#include <cuda_runtime.h>

// 统一的 CUDA 错误检查宏：任何 CUDA API 失败都立刻报出文件、行号与错误串
#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

// 供占用率查询使用的最小 kernel：不做有意义的工作，只为拿到真实的寄存器/共享内存用量
__global__ void occupancyProbeKernel(const float *in, float *out, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        out[i] = in[i] * 2.0f + 1.0f;
    }
}

// 由 compute capability 推断架构代号（用于打印可读信息）
static const char *archName(int major, int minor)
{
    if (major == 7 && minor == 0) return "Volta (sm_70)";
    if (major == 7 && minor == 5) return "Turing (sm_75)";
    if (major == 8 && minor == 0) return "Ampere GA100 (sm_80)";
    if (major == 8 && minor == 6) return "Ampere GA10x (sm_86)";
    if (major == 8 && minor == 9) return "Ada Lovelace (sm_89)";
    if (major == 9 && minor == 0) return "Hopper (sm_90)";
    if (major >= 10)              return "Blackwell or newer";
    return "unknown";
}

// 每个 SM 的 FP32 FMA lane 数。NVIDIA 没有提供查询接口，只能按架构查表。
// 这是"估算峰值 FLOPS"的唯一来源，因此下面的 TFLOPS 数字是估算值。
static int fp32LanesPerSm(int major, int minor)
{
    if (major == 7) return 64;                  // Volta / Turing
    if (major == 8 && minor == 0) return 64;    // Ampere GA100 (A100)
    if (major == 8) return 128;                 // Ampere GA10x / Ada
    if (major == 9) return 128;                 // Hopper
    if (major >= 10) return 128;                // Blackwell
    return 64;                                  // 保守默认
}

// 打印"占用率相关参数"表：同时给出运行时 API 结果与手工 min() 估算结果
static void printOccupancyTable(const cudaDeviceProp &p)
{
    cudaFuncAttributes attr{};
    CUDA_CHECK(cudaFuncGetAttributes(&attr, occupancyProbeKernel));

    printf("\n  [占用率相关参数] probe kernel: %d regs/thread, %zu B static smem\n",
           attr.numRegs, attr.sharedSizeBytes);
    printf("    %-10s %-12s %-12s %-11s %-11s %-11s %-11s\n",
           "blockDim", "blocks(API)", "threads/SM", "occupancy",
           "byThreads", "byRegs", "byBlocks");

    const int blockSizes[] = {64, 128, 256, 512, 1024};
    for (int k = 0; k < 5; ++k) {
        int bs = blockSizes[k];
        if (bs > p.maxThreadsPerBlock) {
            continue;
        }
```

```
        // 运行时权威答案：把 regs / smem / warp / block 四类上限一起取 min
        int blocksApi = 0;
        CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
            &blocksApi, occupancyProbeKernel, bs, 0));

        // 手工估算：寄存器按 warp 为单位、256 个寄存器为粒度向上取整
        int warpsPerBlock = (bs + p.warpSize - 1) / p.warpSize;
        int regsPerWarp   = attr.numRegs * p.warpSize;
        int regsRounded   = ((regsPerWarp + 255) / 256) * 256;
        int byRegs        = p.regsPerMultiprocessor / (regsRounded * warpsPerBlock);
        int byThreads     = p.maxThreadsPerMultiProcessor / bs;
        int byBlocks      = p.maxBlocksPerMultiProcessor;

        double occ = 100.0 * (double)(blocksApi * bs) /
                     (double)p.maxThreadsPerMultiProcessor;

        printf("    %-10d %-12d %-12d %-10.1f%% %-11d %-11d %-11d\n",
               bs, blocksApi, blocksApi * bs, occ, byThreads, byRegs, byBlocks);
    }
    printf("    说明: blocks(API) 为运行时按四类上限取 min 的结果（权威值）; \n");
    printf("          byRegs 为按 256 寄存器分配粒度手工估算的 block 数上限, \n");
    printf("          二者相差 1 属于正常现象，以 API 值为准。\n");
}

int main(int argc, char **argv)
{
    (void)argc;
    (void)argv;

    int deviceCount = 0;
    CUDA_CHECK(cudaGetDeviceCount(&deviceCount));
    printf("Detected %d CUDA capable device(s)\n", deviceCount);
    if (deviceCount == 0) {
        printf("There is no device supporting CUDA.\n");
        return EXIT_SUCCESS;
    }

    for (int dev = 0; dev < deviceCount; ++dev) {
        CUDA_CHECK(cudaSetDevice(dev));

        cudaDeviceProp p{};
        CUDA_CHECK(cudaGetDeviceProperties(&p, dev));

        printf("\n================ Device %d: %s ================\n", dev, p.name);
        printf("  Compute capability (计算能力) : %d.%d   [%s]\n",
               p.major, p.minor, archName(p.major, p.minor));
        printf("  multiProcessorCount (SM 数量) : %d\n", p.multiProcessorCount);
        printf("  warpSize (一个 warp 的线程数) : %d\n", p.warpSize);
        printf("  maxThreadsPerBlock            : %d\n", p.maxThreadsPerBlock);
        printf("  maxThreadsPerMultiProcessor   : %d\n", p.maxThreadsPerMultiProcessor);
        printf("  maxBlocksPerMultiProcessor    : %d\n", p.maxBlocksPerMultiProcessor);
        printf("  regsPerMultiprocessor         : %d  (32-bit)\n", p.regsPerMultiprocessor);
        printf("  regsPerBlock                  : %d\n", p.regsPerBlock);

        printf("\n  --- 存储层次容量 ---\n");
        printf("  sharedMemPerBlock             : %8zu B  (%7.1f KB)\n",
               p.sharedMemPerBlock, p.sharedMemPerBlock / 1024.0);
        printf("  sharedMemPerBlockOptin        : %8zu B  (%7.1f KB)\n",
               p.sharedMemPerBlockOptin, p.sharedMemPerBlockOptin / 1024.0);
        printf("  sharedMemPerMultiprocessor    : %8zu B  (%7.1f KB)\n",
               p.sharedMemPerMultiprocessor, p.sharedMemPerMultiprocessor / 1024.0);
        printf("  totalConstMem (常量内存)      : %8zu B  (%7.1f KB)\n",
               p.totalConstMem, p.totalConstMem / 1024.0);
        printf("  l2CacheSize                   : %8d B  (%7.1f MB)\n",
               p.l2CacheSize, p.l2CacheSize / 1048576.0);
        printf("  totalGlobalMem (全局内存)     : %8.2f GiB\n",
               p.totalGlobalMem / 1073741824.0);

        size_t freeB = 0, totalB = 0;
        CUDA_CHECK(cudaMemGetInfo(&freeB, &totalB));
        printf("  cudaMemGetInfo                : free %.2f GiB / total %.2f GiB\n",
               freeB / 1073741824.0, totalB / 1073741824.0);

        printf("\n  --- 显存带宽 ---\n");
        printf("  memoryClockRate               : %d kHz (%.3f GHz)\n",
               p.memoryClockRate, p.memoryClockRate / 1.0e6);
        printf("  memoryBusWidth                : %d bit\n", p.memoryBusWidth);
```

```
        double memClockHz = (double)p.memoryClockRate * 1.0e3;   // kHz -> Hz
        double busBytes   = (double)p.memoryBusWidth / 8.0;      // bit -> Byte
        double bwSimplified = memClockHz * busBytes / 1.0e9;     // 讲义简化公式
        double bwPeak       = 2.0 * memClockHz * busBytes / 1.0e9;  // DDR 修正：每时钟 2 次传输

        printf("  理论带宽 (clock * busWidth / 8)      : %.1f GB/s   <-- 少算一半\n",
               bwSimplified);
        printf("  理论峰值带宽 (clock * busWidth / 8 * 2): %.1f GB/s  <-- GDDR/HBM 为 DDR\n",
               bwPeak);

        printf("\n  --- 峰值算力与机器平衡点（估算）---\n");
        // 注意: clockRate / memoryClockRate 在 CUDA 13 中已弃用，
        //       新代码可改用 cudaDeviceGetAttribute(&clockKHz, cudaDevAttrClockRate, dev)
        int clockKHz = 0;
        CUDA_CHECK(cudaDeviceGetAttribute(&clockKHz, cudaDevAttrClockRate, dev));
        double clockHz = (double)clockKHz * 1.0e3;
        long long lanes = (long long)p.multiProcessorCount *
                          fp32LanesPerSm(p.major, p.minor);
        double peakFlops = 2.0 * (double)lanes * clockHz;   // FMA 记 2 个 FLOP

        printf("  clockRate (查询值，非 boost)  : %.3f GHz\n", clockKHz / 1.0e6);
        printf("  FP32 FMA lane 总数 (查表估算) : %lld\n", lanes);
        printf("  峰值 FP32 (2 * lanes * clock) : %.2f TFLOPS (估算)\n",
               peakFlops / 1.0e12);
        printf("  机器平衡点 (FLOPS / Byte)     : %.2f FLOP/Byte\n",
               peakFlops / (bwPeak * 1.0e9));
        printf("     -> 算术强度低于该值的 kernel 一定受显存带宽限制\n");

        printf("\n  --- 推导量 ---\n");
        int maxWarpsPerSm = p.maxThreadsPerMultiProcessor / p.warpSize;
        printf("  每 SM 最大 warp 数            : %d\n", maxWarpsPerSm);
        printf("  满占用时每线程寄存器预算      : %d regs\n",
               p.regsPerMultiprocessor / p.maxThreadsPerMultiProcessor);
        printf("  满占用时每 block 共享内存预算 : %.1f KB (假设 2 个 block/SM)\n",
               p.sharedMemPerMultiprocessor / 1024.0 / 2.0);

        printOccupancyTable(p);

        printf("\n  --- 推荐的启动配置 ---\n");
        int bs = 256;
        int blocksPerSm = 0;
        CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
            &blocksPerSm, occupancyProbeKernel, bs, 0));
        int gridFull = p.multiProcessorCount * blocksPerSm;
        printf("  blockDim = %d 时每 SM 可驻留 %d 个 block\n", bs, blocksPerSm);
        printf("  满占用 grid = %d 个 block = %d 个线程 = %.2f M 线程\n",
               gridFull, gridFull * bs, gridFull * (double)bs / 1.0e6);
        printf("  grid-stride loop 建议用这个 grid 大小（每个线程循环多次）\n");

        printf("\n  --- 编译目标提示 ---\n");
        printf("  本程序以 -arch=sm_80 编译，设备为 sm_%d%d。\n", p.major, p.minor);
        if (p.major == 8 && p.minor == 0) {
            printf("  完全匹配：可直接运行 sm_80 的 SASS。\n");
        } else if (p.major == 8 && p.minor > 0) {
            printf("  sm_80 的 PTX 会被驱动 JIT 编译成本机 SASS（首次启动略慢）。\n");
        } else {
            printf("  不匹配：请改用 -arch=sm_%d%d 或 -arch=native 重新编译。\n",
                   p.major, p.minor);
        }
    }

    CUDA_CHECK(cudaDeviceReset());
    return EXIT_SUCCESS;
}
```

- **【代码做什么？】**
- `cudaGetDeviceCount(&deviceCount)` 询问系统里有几张 CUDA 设备；若为 0 则打印提示并正常退出（这一步在 Delta 上如果忘记 `srun` / `salloc` 申请 GPU 资源时会返回 0，是最常见的环境问题）。
- 对每张设备先 `cudaSetDevice(dev)`（`cudaMemGetInfo` 等 API 作用于"当前设备"），再 `cudaGetDeviceProperties(&p, dev)` 一次性取回 `cudaDeviceProp` 结构体——它包含本讲要求打印的全部字段。
- 按四组打印：**身份信息**（`name`、`major.minor`、架构代号）、**并行资源**（`multiProcessorCount`、`warpSize`、`maxThreadsPerBlock`、`maxThreadsPerMultiProcessor`、`maxBlocksPerMultiProcessor`、`regsPerMultiprocessor`）、**存储层次**（`sharedMemPerBlock`、

`sharedMemPerBlockOptin`、`sharedMemPerMultiprocessor`、`totalConstMem`、`l2CacheSize`、`totalGlobalMem`）、**带宽与算力**。

- **带宽推导**：`memoryClockRate` 的单位是 kHz，先换算成 Hz；`memoryBusWidth` 单位是 bit，除以 8 得到"每时钟传输的字节数"；再乘以 2（GDDR/HBM 是 DDR，每个时钟周期有上升沿和下降沿两次数据传输）。A100 代入：`2 × 1.215e9 Hz × (5120/8) B = 2 × 1.215e9 × 640 = 1.5552e12 B/s = 1555.2 GB/s`，与硬件档案一致。**只写 `clock × bus / 8` 会得到 777.6 GB/s，正好少一半。**
- **峰值算力推导**：`峰值 FP32 = 2（FMA）× lane 总数 × clock`。A100 代入：`2 × 108 × 64 × 1.41e9 = 19.49e12 = 19.5 TFLOPS`。`fp32LanesPerSm()` 是按架构查表的（NVIDIA 未提供 lane 数查询接口），而且查到的 `clockRate` 通常不是 boost 频率，所以这是估算值——但在教学和优化方向判断上完全够用。
- **机器平衡点**：`peakFlops / peakBandwidth = 19.5e12 / 1.5552e12 = 12.54 FLOP/Byte`。这个数字是 Roofline 模型的折点，任何 AI 低于它的 kernel 都是带宽受限。
- **占用率表**：对 `blockDim = 64/128/256/512/1024` 分别调用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor()`，得到**运行时权威的**每 SM 常驻 block 数；同时用 `byThreads / byRegs / byBlocks` 手算一遍，让学生看到"取 min"这条规则如何工作，以及寄存器分配粒度带来的 ±1 差异。
- **推荐配置**：用 `blockDim = 256` 算出的 `blocksPerSm`（A100 上 probe kernel 约 8）乘以 SM 数（108），得到"满占用 grid = 864 个 block = 221 184 个线程"。这个数字就是后续所有 grid-stride kernel 的 grid 大小。
- 最后用 `major/minor` 判断所编译的 `sm_80` 二进制能否在本机直接运行，还是需要驱动 JIT 重编译。
- **【并行机制与硬件映射解说】**
- **本程序没有并行 kernel 负载**：`occupancyProbeKernel` 只被用来**查询资源占用**（`cudaFuncGetAttributes` 拿到 `numRegs`），从未真正启动。真正干活的是主机端的 `cudaGetDeviceProperties` 与 `cudaOccupancyMaxActiveBlocksPerMultiprocessor`，它们都是**同步的运行时调用**。
- **block/warp 划分在 Lab 0 中是被"查询"而非被"执行"的对象**：`warpSize = 32` 是硬件常量；`threadIdx.x / warpSize` 就是线程所属的 warp 编号（前提是 block 只用 x 维）。若 `blockDim.x = 100`，那么一个 block 有 `ceil(100/32) = 4` 个 warp，最后一个 warp 只有 4 个活跃 lane，其余 28 个 lane 仍占发射槽——**block 大小取 32 的整数倍（128、256、512）能避免这种浪费**。
- **A100 上的常驻量（代入）**：`maxThreadsPerMultiProcessor = 2048`，`warpSize = 32` → `2048/32 = 64` 个 warp/SM。probe kernel 约 10 个寄存器，`byRegs = 65536/((10×32 向上取整到 256)×8 warps/block) = 65536/(512×8) = 16`；`byThreads = 2048/256 = 8`；`byBlocks = 32`（`maxBlocksPerMultiProcessor`）；三者取 min → **8 个 block/SM = 2048 线程 = 100% 占用率**。
- **共享内存无关**：probe kernel 的 `sharedSizeBytes = 0`，所以共享内存不构成限制。若 kernel 使用 48 KB 静态共享内存，则 `bySmem = 164/48 = 3`，占用率掉到 `3×256/2048 = 37.5%`。
- **不存在 warp 发散**：probe kernel 里的 `if (i < n)` 在所有线程上取值一致（在 grid 覆盖范围内），因此无发散。仅当 `n` 不是 `blockDim` 的整数倍时，最后一个 block 的最后一个 warp 才会发生一次发散。
- **不存在全局内存合并不良**：probe kernel 的访问模式是 `in[i]`，一个 warp 的 32 个线程读连续的 32 个 4 字节字 = 128 字节 = 恰好 1 条 cache line。如果改成 `in[i * 2]`（步长 2），一个 warp 就会横跨 256 字节 = **2 条 cache line**，有效带宽打对折。
- **不存在 bank conflict**：完全没有共享内存访问。
- **不同架构的常驻量对比**（同一份 probe kernel、`blockDim = 256`）：

GPU	`maxBlocksPerMultiProcessor`	`byThreads`	`byRegs`（以 10 regs 计）	实际 blocks/SM	占用率
A100 (sm_80)	32	2048/256 = 8	65536/(512×8) = 16	**8**	100%
H100 (sm_90)	32	2048/256 = 8	16	**8**	100%
RTX 4090 (sm_89)	24	1536/256 = 6	`65536/(512×6)=21`	**6**	100%
RTX 2080 Ti (sm_75)	16	1024/256 = 4	32	**4**	100%

- **【性能优化分析】**
- **占用率（occupancy）**：probe kernel 在 `blockDim = 256` 时达到 100%（8 blocks × 256 threads = 2048 = `maxThreadsPerMultiProcessor`）。这意味着该 kernel 在寄存器维度上没有任何压力（用了 10 个寄存器，预算 32 个）。
  - 计算式：`每线程寄存器预算 = regsPerMultiprocessor / maxThreadsPerMultiProcessor = 65536 / 2048 = 32`
  - 实测占用率：`blocksApi × blockDim / maxThreadsPerMultiProcessor = 8 × 256 / 2048 = 100%`
  - **最坏情况**：若某 kernel 用了 64 个寄存器，则 `byRegs = 65536/((64×32 向上取整到 2048)×8) = 65536/16384 = 4`，占用率降到 `4×256/2048 = 50%`，延迟隐藏能力减半。
- **算术强度（arithmetic intensity）与该程序的关系**：Device Query 本身几乎不搬数据（`totalGlobalMem`、`cudaMemGetInfo` 都是元数据查询），AI ≈ 0，是纯粹的**延迟受限/API 调用受限**程序，耗时以微秒计的固定开销为主（每次 CUDA 运行时调用约 1–5 μs）。它的价值不在性能，而在**为后续 kernel 提供定标常数**。
- **Roofline 定标**：把本程序打印出的两个数写进笔记本——
  - 峰值带宽 `BW = 1555 GB/s`（A100）
  - 机器平衡点 `AI* = 19.5 TFLOPS / 1555 GB/s = 12.5 FLOP/Byte` 有了这两个数，后续任何一个 kernel 都能在 30 秒内判断瓶颈：`预期耗时 ≈ max(访存字节数 / BW, FLOP 数 / peakFLOPS)`。
- **可直接执行的优化方向**：

1. 记住本机的"满占用 grid / 满占用线程数"（A100 + 256 线程 = 864 blocks = 221 184 线程），后续 grid-stride kernel 一律用它。
2. 把 `maxThreadsPerBlock`、`sharedMemPerBlockOptin`、`totalConstMem` 写进笔记本——它们分别是"每 block 线程数的硬上限"、"每 block 共享内存的硬上限（申请超过 48 KB 必须显式 opt-in）"、"常量内存容量（65 536 B）"，是后面调参时的护栏。
3. 把理论带宽与实际测得的带宽做对比。Device Query 打印的是**理论值 1555 GB/s**；真实 kernel 通常只能跑到 85%–93%（1320–1445 GB/s）。**用实测值而不是理论值做 Roofline 分析，预测会准确得多。**

示例 2：Hello, GPU —— 最小 kernel 与五步模板骨架

```
// 文件: hello_gpu.cu
// 编译: nvcc -O3 -arch=sm_80 hello_gpu.cu -o hello_gpu
// 运行: ./hello_gpu           (默认 4 个 block x 8 个线程)
//       ./hello_gpu 8 4       (8 个 block x 4 个线程 -> 共 2 个 warp，便于观察 warp 划分)
//       ./hello_gpu 4 32      (4 个 block x 32 个线程 -> 每个 block 恰好 1 个 warp)
//
// 目的：用最少的代码完整展示 CUDA 程序的五步结构，并让每个线程打印自己的身份，
//       从而看清 grid / block / thread 三层索引与 warp 的对应关系。

#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

// __global__ = kernel：由 host 启动、在 device 上执行、必须返回 void
__global__ void helloKernel(const float *in, float *out, int n)
{
    unsigned int gtid = blockIdx.x * blockDim.x + threadIdx.x;  // 全局线性索引

    // device 端 printf（需要 compute capability >= 2.0）。注意：
    //   - 输出顺序不确定（warp 调度顺序不保证）
    //   - 输出缓冲在 kernel 结束（或缓冲区满）时才刷出
    printf("  [device] blk %u/%u  thr %2u/%-2u  warp %u lane %2u  gtid %2u  %s\n",
           blockIdx.x, gridDim.x,
           threadIdx.x, blockDim.x,
           threadIdx.x / warpSize, threadIdx.x % warpSize,
           gtid,
           (gtid < (unsigned)n) ? "work" : "idle");

    if (gtid < (unsigned)n) {
        out[gtid] = in[gtid] + 1.0f;   // 让每个有效线程真的访问一次全局内存
    }
}

int main(int argc, char **argv)
{
    int nBlocks  = (argc > 1) ? atoi(argv[1]) : 4;
    int nThreads = (argc > 2) ? atoi(argv[2]) : 8;

    if (nBlocks <= 0 || nThreads <= 0 || nThreads > 1024) {
        printf("用法: %s [nBlocks(>0)] [nThreads(1..1024)]\n", argv[0]);
        return EXIT_FAILURE;
    }

    int n = nBlocks * nThreads;
    size_t bytes = (size_t)n * sizeof(float);

    printf("Hello, GPU: %d blocks x %d threads = %d threads = %d warp(s)\n",
           nBlocks, nThreads, n, (n + 31) / 32);
    printf("索引公式: i = blockIdx.x * blockDim.x + threadIdx.x\n\n");

    // ---------- 主机端准备 ----------
    float *h_in  = (float *)malloc(bytes);
    float *h_out = (float *)malloc(bytes);
    for (int i = 0; i < n; ++i) {
        h_in[i]  = (float)i;
        h_out[i] = -1.0f;
    }

    // ---------- 第 1 步：在 device 上分配全局内存 ----------
    float *d_in  = nullptr;
    float *d_out = nullptr;
    CUDA_CHECK(cudaMalloc((void **)&d_in,  bytes));
    CUDA_CHECK(cudaMalloc((void **)&d_out, bytes));

    // ---------- 第 2 步：把输入从 host 拷到 device (H2D) ----------
    CUDA_CHECK(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));
```

```
    // ---------- 第 3 步：启动 kernel ----------
    dim3 grid(nBlocks, 1, 1);
    dim3 block(nThreads, 1, 1);
    helloKernel<<<grid, block>>>(d_in, d_out, n);

    // kernel 启动是异步的：这一行返回时 kernel 可能还没开始跑。
    // cudaGetLastError() 用来捕获"启动配置非法"这类即时错误。
    CUDA_CHECK(cudaGetLastError());
    // 显式同步：等 device 上所有先前的工作（含 printf 缓冲）完成。
    CUDA_CHECK(cudaDeviceSynchronize());

    // ---------- 第 4 步：把结果从 device 拷回 host（D2H）----------
    CUDA_CHECK(cudaMemcpy(h_out, d_out, bytes, cudaMemcpyDeviceToHost));

    // ---------- 第 5 步：释放 device 内存 ----------
    CUDA_CHECK(cudaFree(d_in));
    CUDA_CHECK(cudaFree(d_out));

    // ---------- 主机端自检 ----------
    int bad = 0;
    for (int i = 0; i < n; ++i) {
        if (h_out[i] != h_in[i] + 1.0f) {
            if (bad == 0) {
                printf("首个不匹配：i=%d got %.1f expected %.1f\n",
                       i, h_out[i], h_in[i] + 1.0f);
            }
            ++bad;
        }
    }
    printf("\nhost check：%s（%d/%d 元素正确）\n",
           (bad == 0) ? "PASS" : "FAIL", n - bad, n);

    free(h_in);
    free(h_out);
    return (bad == 0) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

- **【代码做什么？】**
- 从命令行读 `nBlocks` 与 `nThreads`，据此算出总线程数 `n` 与字节数 `bytes`。默认 `4 × 8 = 32` 个线程，正好等于 1 个 warp。
- 主机端 `malloc` 两个数组：`h_in[i] = i`，`h_out[i] = -1.0f`（哨兵值，用于确认每个元素都被真的写过）。
- **第 1 步**：两次 `cudaMalloc` 在显存里开两块 `bytes` 大小的空间，返回设备指针 `d_in`、`d_out`。注意 `(void **)` 强制转换是必需的——`cudaMalloc` 需要修改指针本身的值，所以传的是指针的地址。
- **第 2 步**：`cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice)` 把输入上行。方向的枚举值写反（写成 `DeviceToHost`）是经典错误，会报 `invalid argument`。
- **第 3 步**：构造 `dim3 grid(nBlocks,1,1)` 与 `dim3 block(nThreads,1,1)`，用 `helloKernel<<<grid, block>>>(d_in, d_out, n)` 启动。kernel 里：`gtid = blockIdx.x * blockDim.x + threadIdx.x` 得到全局线性索引（0 到 31），`printf` 打印 block 编号、block 内线程编号、warp 编号（`threadIdx.x / warpSize`）、lane 编号（`threadIdx.x % warpSize`）；只有 `gtid < n` 的线程才写 `out[gtid] = in[gtid] + 1.0f`。
- 启动后立刻 `cudaGetLastError()`，再 `cudaDeviceSynchronize()`（既等待完成，也确保 device 端的 `printf` 缓冲被刷出到 stdout）。
- **第 4 步**：`cudaMemcpy(h_out, d_out, bytes, cudaMemcpyDeviceToHost)` 把结果下行。
- **第 5 步**：两次 `cudaFree` 释放显存，最后 `free` 释放主机内存。
- **自检**：逐元素对比 `h_out[i]` 与 `h_in[i] + 1.0f`，打印 PASS/FAIL 与正确元素个数。若 kernel 的 `if (gtid < n)` 判断写错，`h_out` 会保留 `-1.0` 哨兵值，立刻暴露。
- **【并行机制与硬件映射解说】**
- **warp 划分（用默认 4 block × 8 线程观察）**：每个 block 只有 8 个线程，`8 / 32 = 0.25`，因此**每个 block 只有 1 个 warp，且该 warp 只有 8 个活跃 lane，其余 24 个 lane 是空的**。
  - 硬件仍然按 32 个 lane 分配资源和发射槽位：**占用率 = 8/32 = 25%（相对 warp 槽位）**。
  - 这是"小 block 浪费发射槽"的直接演示。用 `./hello_gpu 4 32` 运行时，每个 block 恰好 1 个满 warp，效率最高（但 block 数太少，仍无法填满 A100 的 108 个 SM）。
- **在 A100 上执行 `./hello_gpu 4 8` 的真实映射**：
  - grid 有 4 个 block，A100 有 108 个 SM → **只有 4 个 SM 各拿到 1 个 block，另外 104 个 SM 完全空闲**。GPU 的算力利用率 ≈ 4/108 = 3.7%。
  - 每个有活的 SM 上：1 个 warp 被分配到 4 个 warp scheduler 中的一个，该 scheduler 每个周期检查它是否就绪；由于这条 warp 的绝大多数时间在等 `printf` 的 I/O 和 `out[gtid]` 的写回，**没有任何其他 warp 可以切换**——这是延迟隐藏完全失效的极端例子。
- **warp 发散分析**：kernel 中的 `if (gtid < n)` 在默认参数下（n = 32，全部线程都满足）**不发散**。若运行 `./hello_gpu 4 8` 并故意把 n 设成不是 32 的倍数，只有最后一个 warp 会发生一次两路发散，代价约为该 warp 分支路径的额外一遍。

- **全局内存访问是否合并**：`in[gtid]` 与 `out[gtid]` 都是连续访问。一个满 warp（32 线程）读 `in[0..31]` = **128 字节 = 恰好 1 条 cache line = 1 次内存事务**；写 `out[0..31]` 同理。**完全合并（perfectly coalesced）**。相比之下，若写成 `in[gtid * 4]`，一个 warp 会跨越 `32 × 16 = 512` 字节 = 4 条 cache line，**需要 4 次事务换取 128 字节有用数据，有效带宽降到 25%**。
- **寄存器使用量与占用率**：这个 kernel 大约用 8–12 个寄存器。A100 的预算是每线程 32 个（在 2048 线程满占用下），所以**寄存器完全不构成限制**。真正的限制是**根本没有足够的线程**——只有 32 个线程，而 A100 一次可以常驻 `108 × 2048 = 221 184` 个线程。这就是"**小问题在 GPU 上跑不满**"的原因：不是 GPU 慢，而是它的并行度没被喂饱。
- **block 调度**：4 个 block 会被分配到 4 个不同的 SM（CUDA 运行时倾向于把 block 尽量分散到空闲 SM）。block 之间没有任何同步机制，执行顺序也不保证——`printf` 输出的顺序在不同次运行中可能不同，这是**非确定性（non-determinism）**的第一课（讲义 "Determinism and Reproducibility"）。
- **【性能优化分析】**
- **占用率（occupancy）**：
  - warp 粒度占用率：每个 block 1 个 warp / 每 SM 最多 64 个 warp。4 个 block 分散到 4 个 SM，每 SM 1 个 warp → **占用率 = 1/64 = 1.6%**。
  - 线程粒度占用率：`8 threads / 2048 threads = 0.39%`。
  - **结论：这是一个"延迟完全暴露"的程序**，其耗时几乎全部是"等 DRAM 与 printf"。
- **算术强度（arithmetic intensity）**：
  - 每个元素：读 4 B（`in`）+ 写 4 B（`out`）= 8 B；运算 1 次加法 = 1 FLOP。
  - `AI = 1 FLOP / 8 B = 0.125 FLOP/Byte`。
  - A100 平衡点 `AI* = 12.5 FLOP/Byte`。`AI / AI* = 0.125/12.5 = 1%` → **带宽受限，且受限于"并行度不足"而非"带宽不足"**。
- **Roofline 定量分析（代入 n = 32）**：
  - 访存字节数 `= 32 × 8 B = 256 B`；访存下界 `= 256 B / 1555 GB/s = 0.165 ns`（即 0.000000165 ms）。
  - 计算量 `= 32 FLOP`；计算下界 `= 32 / 19.5e12 = 0.0016 ns`。
  - **两个下界都远小于任何可测量的时间尺度**。实际耗时由三部分主导：三次 CUDA API 调用（`cudaMalloc` ×2 各 5–20 μs）、一次 kernel 启动（3–10 μs）、两次主机–设备往返（微秒级），**总计 20–60 μs，是理论下界的 10^8 倍以上**。
  - 这个对比本身就是最重要的教学点：**当问题规模太小时，性能由固定开销（launch overhead、API 调用）决定，与 Roofline 无关**。后续所有 Lab 的问题规模都被刻意设计得足够大（N = 10^6–10^7 量级），就是为了让 Roofline 分析成立。
- **可执行的优化方向**：
  1. 把 `blockDim.x` 设为 32 的整数倍（推荐 128 或 256），避免空 lane 浪费。
  2. 让 grid 至少覆盖 `SM 数 × 每 SM 常驻 block 数`（A100 上 864 个 block），否则部分 SM 闲置。
  3. 用**一次 `cudaMalloc` 分配一大块内存再手动切分**，减少 API 调用次数（每次 `cudaMalloc` 都是同步操作，代价 5–20 μs）。
  4. 把多次小 `cudaMemcpy` 合并成一次大的（PCIe 每次传输都有固定启动开销）。
  5. 在真实项目中**不要用 device 端 `printf` 调试大循环**——它会串行化输出并严重拖慢 kernel。

> **示例 3：CUDA 向量加法 —— 全书基准模板（朴素版 vs grid-stride 版）**

```
// 文件: vecadd.cu
// 编译: nvcc -O3 -arch=sm_80 vecadd.cu -o vecadd
// 运行: ./vecadd            (默认 N = 2^24 = 16777216 个 float)
//       ./vecadd 1000003    (故意取非 256 倍数，验证边界处理)
//
// 这是 ECE408 全书使用的"基准模板"：完整演示 CUDA 五步结构、错误检查、
// cudaEvent 计时、H2D/kernel/D2H 三段耗时拆分、CPU 参考实现比对、
// 以及"一线程一元素"与"grid-stride loop"两种线程映射的对比。

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define BLOCK_SIZE 256

// ---------------- 版本 1：一线程一元素（最朴素） ----------------
__global__ void vecAddKernel(const float *A, const float *B, float *C, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {                       // 边界检查：grid 向上取整后必然有越界线程
        C[i] = A[i] + B[i];
    }
}

// ---------------- 版本 2：grid-stride loop（线程数固定，线程处理多个元素） ----------------
__global__ void vecAddKernelGridStride(const float *A, const float *B, float *C, int n)
{
    int stride = gridDim.x * blockDim.x;            // 整个 grid 的线程总数
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n; i += stride) {
        C[i] = A[i] + B[i];
    }
}

static void initData(float *a, float *b, int n)
{
    for (int i = 0; i < n; ++i) {
        a[i] = 1.0f * (float)i;
        b[i] = 2.0f * (float)i;
    }
}

static bool verify(const float *a, const float *b, const float *c, int n)
{
    for (int i = 0; i < n; ++i) {
        float ref = a[i] + b[i];
        float tol = 1e-3f * (1.0f + fabsf(ref));
        if (fabsf(c[i] - ref) > tol) {
            printf("  首个不匹配: i=%d  got %.6f  expected %.6f\n", i, c[i], ref);
            return false;
        }
    }
    return true;
}

static float elapsedMs(cudaEvent_t start, cudaEvent_t stop)
{
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
    return ms;
}

int main(int argc, char **argv)
{
    int n = (argc > 1) ? atoi(argv[1]) : (1 << 24);
    if (n <= 0) {
        printf("用法: %s [N>0]\n", argv[0]);
```

```
    return EXIT_FAILURE;
}
size_t bytes = (size_t)n * sizeof(float);

printf("N = %d floats\n", n);
printf("  每数组        = %.2f MiB\n", bytes / 1048576.0);
printf("  理论访存流量  = %.2f MiB (读 A + 读 B + 写 C, 共 3N*4 字节)\n",
       3.0 * bytes / 1048576.0);

// ---------- host 准备 ----------
float *h_A  = (float *)malloc(bytes);
float *h_B  = (float *)malloc(bytes);
float *h_C  = (float *)malloc(bytes);   // 版本 1 的结果
float *h_C2 = (float *)malloc(bytes);   // 版本 2 的结果
initData(h_A, h_B, n);

// ---------- 第 1 步：分配 device 内存 ----------
float *d_A = nullptr;
float *d_B = nullptr;
float *d_C = nullptr;
CUDA_CHECK(cudaMalloc((void **)&d_A, bytes));
CUDA_CHECK(cudaMalloc((void **)&d_B, bytes));
CUDA_CHECK(cudaMalloc((void **)&d_C, bytes));

cudaEvent_t t0, t1, t2, t3;
CUDA_CHECK(cudaEventCreate(&t0));
CUDA_CHECK(cudaEventCreate(&t1));
CUDA_CHECK(cudaEventCreate(&t2));
CUDA_CHECK(cudaEventCreate(&t3));

// ---------- 第 2 步：H2D 拷贝 ----------
CUDA_CHECK(cudaEventRecord(t0));
CUDA_CHECK(cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice));
CUDA_CHECK(cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice));
CUDA_CHECK(cudaEventRecord(t1));
CUDA_CHECK(cudaEventSynchronize(t1));
float h2d_ms = elapsedMs(t0, t1);

// ---------- 第 3 步：启动 kernel（版本 1：一线程一元素） ----------
int  blockSize = BLOCK_SIZE;
int  gridSize  = (n + blockSize - 1) / blockSize;   // 向上取整的整数写法
CUDA_CHECK(cudaEventRecord(t1));
vecAddKernel<<<gridSize, blockSize>>>(d_A, d_B, d_C, n);
CUDA_CHECK(cudaEventRecord(t2));
CUDA_CHECK(cudaEventSynchronize(t2));
CUDA_CHECK(cudaGetLastError());
float k1_ms = elapsedMs(t1, t2);

// ---------- 第 4 步：D2H 拷贝 ----------
CUDA_CHECK(cudaEventRecord(t2));
CUDA_CHECK(cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost));
CUDA_CHECK(cudaEventRecord(t3));
CUDA_CHECK(cudaEventSynchronize(t3));
float d2h_ms = elapsedMs(t2, t3);

bool ok1 = verify(h_A, h_B, h_C, n);

// ---------- 版本 2：grid-stride，grid 取"刚好占满 GPU" ----------
int dev = 0, smCount = 0, blocksPerSm = 0;
CUDA_CHECK(cudaGetDevice(&dev));
CUDA_CHECK(cudaDeviceGetAttribute(&smCount, cudaDevAttrMultiProcessorCount, dev));
CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
    &blocksPerSm, vecAddKernelGridStride, blockSize, 0));
int residentBlocks = smCount * blocksPerSm;
int gsGrid = (residentBlocks < gridSize) ? residentBlocks : gridSize;

CUDA_CHECK(cudaEventRecord(t1));
vecAddKernelGridStride<<<gsGrid, blockSize>>>(d_A, d_B, d_C, n);
CUDA_CHECK(cudaEventRecord(t2));
CUDA_CHECK(cudaEventSynchronize(t2));
CUDA_CHECK(cudaGetLastError());
float k2_ms = elapsedMs(t1, t2);

CUDA_CHECK(cudaMemcpy(h_C2, d_C, bytes, cudaMemcpyDeviceToHost));
bool ok2 = verify(h_A, h_B, h_C2, n);

// ---------- 性能报告 ----------
double traffic = 3.0 * (double)bytes;                       // 总访存字节
```

```
    double bw1 = traffic / (k1_ms * 1.0e-3) / 1.0e9;            // GB/s
    double bw2 = traffic / (k2_ms * 1.0e-3) / 1.0e9;
    double gflops1 = (double)n / (k1_ms * 1.0e-3) / 1.0e9;      // 每元素 1 次加法
    double ai = (double)n / traffic;                            // FLOP / Byte
    double h2dBw = 2.0 * (double)bytes / (h2d_ms * 1.0e-3) / 1.0e9;
    double d2hBw = (double)bytes / (d2h_ms * 1.0e-3) / 1.0e9;

    printf("\n================ 性能报告 ================\n");
    printf("[PCIe] H2D   : %8.3f ms   %7.1f GB/s\n", h2d_ms, h2dBw);
    printf("[GPU ] kernel v1 (一线程一元素): %8.3f ms   %7.1f GB/s   %7.2f GFLOP/s\n",
           k1_ms, bw1, gflops1);
    printf("[GPU ] kernel v2 (grid-stride) : %8.3f ms   %7.1f GB/s\n", k2_ms, bw2);
    printf("[PCIe] D2H   : %8.3f ms   %7.1f GB/s\n", d2h_ms, d2hBw);

    double xfer = h2d_ms + d2h_ms;
    printf("\n[总账] 传输合计 %.3f ms, kernel %.3f ms, 传输/kernel = %.1f 倍\n",
           xfer, k1_ms, xfer / k1_ms);

    printf("\n[调度] grid v1 = %d blocks, 每个 block %d 线程\n", gridSize, blockSize);
    printf("       该 GPU 有 %d 个 SM, 每 SM 可常驻 %d 个 block -> 同时可跑 %d 个 block\n",
           smCount, blocksPerSm, residentBlocks);
    printf("       需要 %.2f 个 wave, 尾部浪费约 %.2f%%\n",
           (double)gridSize / residentBlocks,
           100.0 * (1.0 - (double)gridSize /
                    (ceil((double)gridSize / residentBlocks) * residentBlocks)));
    printf("       grid v2 = %d blocks (正好 1 个 wave, 每个线程循环约 %.1f 次)\n",
           gsGrid, (double)gridSize / gsGrid);

    printf("\n[算术强度] AI = %.4f FLOP/Byte  (每元素读 8 B 写 4 B -> 12 B, 做 1 次加法)\n", ai);
    printf("[判定] AI 远低于 A100 的机器平衡点 12.5 FLOP/Byte -> 纯内存带宽受限\n");
    printf("       优化方向是【减少字节搬运】, 而不是【减少指令数】\n");

    printf("\n[自检] version1: %s\n", ok1 ? "PASS" : "FAIL");
    printf("[自检] version2: %s\n", ok2 ? "PASS" : "FAIL");

    // ---------- 第 5 步: 释放资源 ----------
    CUDA_CHECK(cudaEventDestroy(t0));
    CUDA_CHECK(cudaEventDestroy(t1));
    CUDA_CHECK(cudaEventDestroy(t2));
    CUDA_CHECK(cudaEventDestroy(t3));
    CUDA_CHECK(cudaFree(d_A));
    CUDA_CHECK(cudaFree(d_B));
    CUDA_CHECK(cudaFree(d_C));
    free(h_A);
    free(h_B);
    free(h_C);
    free(h_C2);

    return (ok1 && ok2) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

- **【代码做什么？】**
- `main` 解析 N（默认 2^{24} = 16 777 216），计算 `bytes = N × 4`，打印每数组大小（64 MiB）与理论访存流量（192 MiB，含读 A、读 B、写 C 共 3N 个 float = 201.33 MB）。
- **host 准备**：四个 `malloc` —— `h_A`、`h_B`（输入）与 `h_C`、`h_C2`（两个版本的输出，便于分别验证）；`initData` 填 `A[i] = i`、`B[i] = 2i`。
- **第 1 步**：三次 `cudaMalloc` 分配 `d_A`、`d_B`、`d_C`，各 `N × 4` 字节。
- 创建 4 个 `cudaEvent` 作为计时锚点。
- **第 2 步**：用 `cudaEventRecord(t0)` / `(t1)` 夹住两次 `cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice)`，测出 H2D 耗时；`cudaEventSynchronize(t1)` 保证时间戳已经落实。
- **第 3 步**：`gridSize = (N + 255) / 256`（整数向上取整，**不用浮点 `ceil(N/256.0)`**，避免大 N 时的精度问题）。用事件夹住 `vecAddKernel<<<gridSize, 256>>>(d_A, d_B, d_C, N)`，测出 kernel 耗时。kernel 里 `i = blockIdx.x * blockDim.x + threadIdx.x`，`if (i < n)` 保护后写 `C[i] = A[i] + B[i]`。
- **第 4 步**：测 D2H 耗时并把 `d_C` 拷回 `h_C`；`verify()` 与 CPU 参考 `A[i]+B[i]` 逐元素比较（相对容差 `1e-3`），输出首个不匹配位置。
- **版本 2**：查询当前设备 `smCount`（A100 = 108）与 `vecAddKernelGridStride` 的每 SM 常驻 block 数（约 8），得 `residentBlocks = 864`；取 `gsGrid = min(864, gridSize)`。kernel 用 `stride = gridDim.x * blockDim.x` 的 grid-stride 循环，每个线程处理约 `65536 / 864 ≈ 75.85` 个元素。
- **性能报告**：分别算出 H2D/D2H 的实测带宽、两个 kernel 的 GB/s 与 GFLOP/s、传输与计算的耗时比、wave 数量与尾部浪费、算术强度与瓶颈判定。
- **第 5 步**：销毁事件、`cudaFree` 三个设备数组、`free` 四个主机数组。以 `ok1 && ok2` 作为进程退出码（脚本化测试时非常有用）。

- **边界测试**：`./vecadd 1000003`（不是 256 的倍数）时，`gridSize = (1000003 + 255)/256 = 3907`，`3907 × 256 = 1 000 192 > 1 000 003`，因此**最后 189 个线程是越界的**，必须靠 `if (i < n)` 拦住；`verify` 会确认结果仍然完全正确。
- **【并行机制与硬件映射解说】**
- **block 内线程如何被划分为 warp**：`blockDim.x = 256` → 每个 block **8 个满 warp**（256/32 = 8，无空 lane）。warp 0 = `threadIdx.x ∈ [0,31]`，warp 1 = `[32,63]`，以此类推。`i = blockIdx.x*256 + threadIdx.x` 使得**同一个 warp 的 32 个线程拿到连续的 32 个 `i`**——这正是合并访问的前提。
- **warp 如何被调度到 SM 的 warp scheduler**：A100 每个 SM 有 4 个 warp scheduler，每个 scheduler 管理一批 warp，每周期发射 1 条指令。一个 block 的 8 个 warp 会被尽量均匀地分给 4 个 scheduler（每 scheduler 2 个 warp）。由于 A100 每 SM 可驻留 8 个这样的 block（2048 线程 / 64 warp），**每个 scheduler 平均管理 64/4 = 16 个 warp**，切换余量充足。
- **一个 SM 上可同时驻留多少 block/warp（A100，代入本 kernel）**：
  - 线程维度：`2048 / 256 = 8` 个 block
  - warp 维度：`64 / 8 = 8` 个 block
  - 寄存器维度：本 kernel 约 10 个寄存器/线程 → `regsPerWarp = 10 × 32 = 320`，按 256 粒度向上取整到 **512**；`byRegs = 65536 / (512 × 8) = 16` 个 block
  - 共享内存维度：**不使用共享内存**，无限制
  - 硬件 block 上限：`maxBlocksPerMultiProcessor = 32`
  - **取 min → 8 个 block/SM = 2048 线程 = 64 warp = 100% 占用率**
  - 全卡同时常驻：`108 × 8 = 864` 个 block = `864 × 256 = 221 184` 个线程
- **共享内存访问与 bank conflict**：本 kernel **完全不使用共享内存**，所有数据走寄存器与全局内存，因此 **bank conflict 不适用**。作为对照：若把 A、B 的一段数据放进 `__shared__ float tile[256]`，那么 `tile[threadIdx.x]` 的访问模式是"32 个线程访问 32 个连续 float"——A100 的共享内存有 **32 个 bank，每个 bank 宽 4 字节（32-bit）**，地址 `addr` 落在 `bank = (addr / 4) % 32`。线程 t 访问 `tile[t]` → `bank = t`，**32 个线程命中 32 个不同 bank，零冲突（conflict-free）**。反之若写成 `tile[threadIdx.x * 32]`，则 `bank = (t*32) % 32 = 0`——**32 个线程全部命中 bank 0，造成 32 路冲突（32-way bank conflict），共享内存访问被串行化 32 倍**，访问时间从 20–30 cycle 恶化到 640–960 cycle。
- **全局内存访问是否合并（按 32 线程 × 4 字节 = 128 字节事务粒度分析）**：

```
版本 1 的访问模式： i = blockIdx.x*256 + threadIdx.x
------------------------------------------------------------------------------
warp 内 32 个线程的 i 值：  base+0, base+1, base+2, ..., base+31
A 数组地址：  A + (base+k)*4 字节, k = 0..31
覆盖的字节区间：[A + base*4, A + base*4 + 128)  = 恰好 128 字节
------------------------------------------------------------------------------
-> 1 次 128 字节内存事务 = 1 条 cache line            【完全合并 coalesced】
-> 每个 warp 的 C=A+B 需要 3 次事务：读 A(128B)、读 B(128B)、写 C(128B)
-> 有效带宽利用率 = 100%

反例（步长访问） i = (blockIdx.x*256 + threadIdx.x) * 4
------------------------------------------------------------------------------
warp 内 32 个线程访问的地址： A + base*16, A + (base+4)*16, ... 步长 16 字节
覆盖的字节区间：[A + base*16, A + base*16 + 512)  = 512 字节 = 4 条 cache line
------------------------------------------------------------------------------
-> 需要 4 次 128 字节事务，但只有 128 字节是有用数据
-> 有效带宽利用率 = 128/512 = 25%   【浪费 4 倍带宽】

反例（广播式越界尾块） N 不是 256 的倍数时的最后一个 block
------------------------------------------------------------------------------
最后一个 warp 中只有部分 lane 满足 i < n，其余 lane 被掩码屏蔽。
被屏蔽的 lane 不产生访存请求，因此【不浪费带宽】，只浪费发射槽。
-> 这正是 if (i < n) 边界检查的代价：几乎没有。
```

- **寄存器使用量与 warp 发散情况**：两个 kernel 都只用约 8–14 个寄存器（grid-stride 版因为多一个 `stride` 与循环变量，略多 2–4 个），远低于 32 的满占用预算。分支方面：`if (i < n)` 仅在"最后一个 block 的最后一个 warp"上可能发散一次，占比 `1/(N/32/…)` 可忽略；grid-stride 版的 `for` 循环也让同一 warp 内所有 lane 的迭代次数一致（因为 `n` 和 `stride` 对所有 lane 相同，只在最后一次迭代可能不同），**几乎是完全无发散的代码**。
- **【性能优化分析】**
- **占用率（occupancy）定量**：
  - `blockDim = 256` → `occupancy = blocksPerSm × blockDim / maxThreadsPerMultiProcessor = 8 × 256 / 2048 = 100%`。
  - 线程总数 `N = 16 777 216` 远大于单次可常驻的 221 184，**并行度充足**，不存在 Hello 示例那样的"喂不饱"问题。
  - block 数 `65536` 远大于 `864`，因此 block 会被反复"退役-补充"，形成 **wave（波次）**：`65536 / 864 = 75.85` 个 wave。取整到 76 个 wave，**尾部波次只填了 85.3%**，浪费 `1 − 65536/(76×864) = 0.195%`——**可忽略**。这个数字说明：对于这种规整的流式 kernel，**朴素的"一线程一元素"完全没有调度缺陷**。
- **算术强度（arithmetic intensity）定量**：

- 每元素：读 `A[i]`（4 B）+ 读 `B[i]`（4 B）+ 写 `C[i]`（4 B）= **12 B**；运算 1 次浮点加法 = **1 FLOP**。
- `AI = 1 / 12 = 0.0833 FLOP/Byte`。
- A100 机器平衡点 `AI* = 19.5e12 / 1555e9 = 12.54 FLOP/Byte`。
- `AI / AI* = 0.0833 / 12.54 = 0.66%` → **离平衡点差 150 倍，是极端的内存带宽受限程序**。

- **Roofline 模型定量（代入 N = 2^{24}）**：
  - 访存流量 `= 3 × N × 4 B = 3 × 16 777 216 × 4 = 201 326 592 B = 201.33 MB`
  - 访存下界 `T_mem = 201.33e6 B / 1555e9 B/s = 1.295e-4 s = 129.5 μs`
  - 计算量 `= N × 1 FLOP = 16.78 MFLOP`；计算下界 `T_flop = 16.78e6 / 19.5e12 = 0.86 μs`
  - `T = max(T_mem, T_flop) = 129.5 μs`，且 `T_mem / T_flop = 150.5`
  - 在 Roofline 图上：`可达性能 = AI × BW = 0.0833 × 1555 = 129.6 GFLOP/s`，而峰值是 19 500 GFLOP/s，**只有 0.66%**。
  - 实测若得到 140 μs，则 `实测带宽 = 201.33e6 / 140e-6 = 1438 GB/s = 理论峰值的 92.5%`——**这已经是这类 kernel 能达到的实践上限**（受限于 DRAM 刷新开销、ECC、页激活等）。
- **瓶颈判定：内存带宽受限（memory-bandwidth-bound）**。三条判据同时成立：(1) `AI (0.0833) << AI* (12.54)`；(2) 实测带宽已达理论峰值的 92%，没有提升余地；(3) 减少指令数（如用 `float4` 向量化）只改变每字节的指令数，**不改变总字节数**，因此对耗时几乎没有影响。
- **PCIe 传输成本（这一讲最反直觉的量化结论）**：
  - H2D 传输 2 个数组 = `134.22 MB`。用**可分页内存**（默认 `malloc`），实测带宽约 6–8 GB/s → `134.22e6 / 7e9 = 19.2 ms`。
  - 用**锁页内存**（`cudaHostAlloc`），实测约 25 GB/s → `134.22e6 / 25e9 = 5.37 ms`（讲义指出锁页内存让 `cudaMemcpy` 快约 2 倍，与此吻合）。
  - D2H 传输 1 个数组 = `67.11 MB` → 锁页下 `2.68 ms`。
  - 对比 kernel 的 `0.14 ms`：**传输/计算 = (5.37 + 2.68) / 0.14 ≈ 57 倍**（可分页内存下约 130 倍）。
  - **结论：向量加法这类低算术强度 kernel，应用总耗时几乎完全由 PCIe 决定，优化 kernel 本身是徒劳的。**
- **可执行的优化方向（按收益排序）**：
  1. **提高算术强度**：把多个逐元素操作合并成一个 kernel（kernel fusion），例如把 `C = A + B` 与后续的 `D = C * s` 合并为 `D = (A + B) * s`，访存从 `5N×4 B` 降到 `3N×4 B`，**流量减少 40%，耗时同比例下降**。
  2. **使用向量化访存**：`float4` 让每个线程一次搬 16 字节，warp 一次事务仍是 128 字节但只需 8 个线程参与，**指令数减少 4 倍**；在带宽受限的 kernel 中收益有限，但在指令发射受限时收益明显。
  3. **让数据留在 device 上**：把"上传–计算–下载"改成一串只在 device 内存中流转的 kernel，把 PCIe 往返从"每步一次"降到"整个流程一次"。
  4. **重叠传输与计算**：用多个 `cudaStream_t`，让第 k+1 块的 H2D 与第 k 块的 kernel 执行重叠；配合锁页内存可让有效时间趋近 `max(传输, 计算)` 而非 `传输 + 计算`。
  5. **用锁页内存**（`cudaHostAlloc` / `cudaFreeHost`）：直接让 `cudaMemcpy` 快约 2 倍。
  6. **grid-stride 的选择依据**：本 kernel 两种版本性能基本相同（都打满带宽）。grid-stride 的真正价值在于：**与 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 配合可以让 grid 恰好等于一个 wave**，从而消除尾部波次浪费；并且当 N 极大（超过 `gridDim.x` 的 $2^{31}-1$ 上限）时仍然安全；还便于配合持久化 kernel（persistent kernel）与动态负载均衡。

性能优化技巧总结

1. **先算 Roofline 再动手**：用 `T ≈ max(字节数/带宽, FLOP数/算力)` 估出下界，若实测已接近下界，说明代码没有优化空间——继续调参只是浪费时间。为什么有效：它把"猜测式调优"变成"有上界的收敛判断"。
2. **算术强度低于机器平衡点（A100 为 12.5 FLOP/Byte）时，一切优化都应指向"减少字节搬运"**：融合 kernel、复用数据、降低精度（fp32→fp16）、跳过零元素。为什么有效：带宽是瓶颈时，减少指令数、增加并行度都无法缩短时间。
3. **让每个 warp 的一次访存恰好覆盖一条 128 字节 cache line**：保证 32 个线程访问连续的 32 个 4 字节字。为什么有效：一次事务换回 128 字节有用数据，有效带宽 100%；步长访问会让有效带宽降到 25% 甚至 3%。
4. **`blockDim` 取 32 的整数倍（128/256/512），并用 Device Query 验证占用率**：避免空 lane 浪费，同时确保每 SM 常驻 8 个 block 达到 100% 占用率。为什么有效：占用率直接决定有多少 warp 可用于隐藏 400–800 周期的显存延迟。
5. **用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor()` 而不是手算来决定 grid 大小**：手算因寄存器分配粒度会偏差 ±1 个 block。为什么有效：运行时 API 反映硬件真实约束。
6. **grid 至少覆盖"SM 数 × 每 SM 常驻 block 数"**（A100 + 256 线程 = 864 个 block）。为什么有效：低于这个数量就有 SM 完全闲置；高于它则进入多个 wave，尾部浪费只有约 0.2%。
7. **优先消灭串行部分，而不是堆并行度**：按 Amdahl 定律，s = 10% 时代码最多快 10 倍，P = 64 时效率已跌到 13.7%。为什么有效：串行段的耗时与 P 无关，它会成为不可压缩的下界。
8. **用锁页内存（`cudaHostAlloc`）做主机端缓冲区**：让 `cudaMemcpy` 快约 2 倍。为什么有效：DMA 需要物理地址固定的缓冲区，可分页内存必须先经一次 CPU 拷贝到内部暂存区。
9. **尽量不要在计算流程中间把数据搬回主机**：把多次"上传–计算–下载"合并为"一次上传、多次计算、一次下载"。为什么有效：PCIe Gen4 x16 单向约 25–32 GB/s，仅为 A100 显存带宽（1555 GB/s）的 1/50。

10. **用多 stream 重叠传输与计算**：为什么有效：PCIe 引擎与 SM 是独立硬件，单 stream 时它们被迫串行，多 stream 可让总时间趋近 `max(传输, 计算)`。
11. **共享内存访问要避开 bank conflict**：保证同一个 warp 内各线程访问的 bank 不同（例如 `tile[threadIdx.x]`）。为什么有效：32 路冲突会把 20–30 cycle 的共享内存访问拖到 ~700 cycle，与直接访存 DRAM 无异。
12. **避免 warp 发散**：把 `threadIdx.x % 2` 这类按 lane 奇偶分支改写成"前半 warp 做 A、后半 warp 做 B"，或用分支无关的算术（`select`/三元运算）。为什么有效：发散让两条路径串行执行，利用率最多掉一半。
13. **不要在 kernel 里做大量 `printf`**：device 端 `printf` 会串行化输出并占用缓冲。为什么有效：它把并行的核心变成串行的 I/O。
14. **用 `cudaEvent` 分别测量 H2D、kernel、D2H 三段耗时**：只有拆开测才知道瓶颈到底在哪一段。为什么有效：向量加法的传输耗时是计算的 57 倍，不测就永远找不到真正的瓶颈。
15. **为 kernel 设置 `__launch_bounds__` 控制寄存器用量**：当 nvcc 为了减少指令数而用掉过多寄存器、导致占用率掉到 50% 以下时，限制寄存器数往往更快。为什么有效：它把编译器的优化目标从"单线程最快"改成"整机吞吐最高"，这正是 GPU 的优化目标。

关键要点

- **2004 年的功耗墙终结了主频红利，硬件转向并行；2007 年的 CUDA 让 GPU 的并行算力变得可编程。** Dennard 缩放的理想模型（f → 1/α、每管功耗 → α^2）在 VDD 无法继续下降后失效，芯片设计由此分化为延迟导向的 CPU 与吞吐量导向的 GPU。
- **并行不改变算法复杂度，只提供受 Amdahl 定律约束的固定倍数收益：`Speedup ≤ 1/s`。** 串行占比 25% 时上界是 4×；10% 时用 64 个核只能拿到 8.77×、效率 13.7%。**先优化串行瓶颈，再谈并行技巧。**
- **GPU 靠"延迟隐藏"而非"降低延迟"取胜：一个 SM 上驻留 64 个 warp（A100），用海量线程的并发掩盖 400–800 周期的显存延迟。** Little 定律给出定量需求：`在途字节 = 延迟 × 带宽`，A100 需要约 772 KB（每个 SM 约 56 个 warp 的在途 load）才能打满带宽——这就是"GPU 必须用海量线程"的数学解释。
- **算术强度（FLOP/Byte）与机器平衡点（A100 为 12.5 FLOP/Byte）决定了 kernel 的瓶颈类型。** 向量加法的 AI 仅 0.083，比平衡点低 150 倍，理论下界 129.5 μs 完全由 201.33 MB 的访存流量决定；减少字节数（kernel fusion、tiling、向量化）是唯一的优化方向。
- **CUDA 程序的五步模板（分配 → 拷贝入 → 启动 → 拷贝回 → 释放）是所有后续讲座的基准，其中"拷贝入/拷贝回"往往比 kernel 本身贵几十倍。** 向量加法在 A100 上 kernel 约 0.14 ms，而 PCIe 往返约 8.05 ms（锁页内存）——**Bandwidth 是现代计算机系统的重力**。
- **统一着色器架构（2006, G80/Tesla）把分离的顶点/像素着色器合并为通用标量处理器阵列，是 CUDA 得以存在的硬件前提；而"占用率 = 各类资源上限取 min"这条规则从 CC 1.3 到 sm_90 二十年未变。** `residentBlocks = min(byThreads, byRegs, bySharedMem, byBlocksPerSM)` 是每一次 kernel 调参的出发点。

常见陷阱与注意事项

- **忘记 `cudaDeviceSynchronize()` / 在 kernel 结果未就绪时就使用它**：现象是读回的数组全是初始值或垃圾数据，或者程序打印的顺序与预期不符 → kernel 启动从 CUDA 1.0 起就是**异步**的；要在 host 使用结果，必须 `cudaDeviceSynchronize()`、或用 `cudaMemcpy`（隐式同步）、`cudaStreamSynchronize()`、`cudaEventSynchronize()`。注意：`cudaMemcpy` 会同步，所以"忘了同步"常常被它掩盖，直到你插入计时或并行 stream 才暴露。
- **忘记 `__syncthreads()`**：现象是 block 内线程读到的共享内存值来自"写入之前"的旧值，结果随运行次数变化（非确定性）→ 在"写共享内存 → 读共享内存"之间必须调用 `__syncthreads()`。同时要清楚它的边界：`__syncthreads()` 保证**同一个 block 内**所有线程在 barrier 之前的全局/共享内存访问对该 block 内所有线程可见；但**不保证**其它 block 的执行进度、**不保证**数据已写回 DRAM、**不保证** `printf` 的输出顺序。
- **把 `__syncthreads()` 放进分支里**：现象是程序挂死（hang）或行为不可预测 → `__syncthreads()` 必须位于**所有线程都会执行到的位置**。若部分线程因 `if` 跳过 barrier，硬件会一直等待那些永远不会到达的线程。正确做法是把 barrier 提到条件外面，或者让人为的 barrier 前置于分支。
- **共享内存 bank conflict**：现象是使用了共享内存却比不用还慢 → A100 的共享内存有 32 个 bank、每 bank 4 字节宽。`tile[threadIdx.x]`（bank = t）零冲突；`tile[threadIdx.x * 32]`（bank 恒为 0）是 32 路冲突，访问被串行化 32 倍。解决方式：改访问模式、加 padding（`__shared__ float tile[32][33]`）、或做对角线重排。
- **warp 发散（warp divergence）**：现象是某个 kernel 的耗时远高于按指令数估算的值 → 同一个 warp 内不同 lane 走不同分支时，硬件串行执行各条路径。典型诱因是 `if (threadIdx.x % 2)`、`if (i < n)` 之外的按 lane 分支、以及负载方差大的并行源（图的节点度、三角矩阵的行长）。正确做法：让人为分支与 warp 边界对齐（前 16 个 lane 走 A、后 16 个走 B），或改用分支无关的算术。
- **未检查 CUDA API 返回值**：现象是程序"静默失败"——kernel 根本没启动，或 `cudaMalloc` 失败返回了 `nullptr`，后续访问产生难以定位的错误 → 所有 CUDA 调用都必须包在 `CUDA_CHECK` 宏里；kernel 启动后立刻 `cudaGetLastError()`（因为 `<<<>>>` 没有返回值）。特别注意：不检查时，某次成功的 API 调用会**清空**之前的错误码，让你永远看不到真正的失败点。
- **越界访问**：现象是结果局部错误、`cuda-memcheck`/compute-sanitizer 报 "Invalid **global** write"，或者在某些运行中"碰巧正确" → `gridSize = (N + blockSize - 1) / blockSize` 会向上取整，**最后一个 block 里有 `gridSize*blockSize - N` 个线程的索引是越界的**，必须在 kernel 里用 `if (i < n)` 拦住。整数除法写成 `N / blockSize`（向下取整）则是另一个方向的错误：**网格覆盖不足，尾部元素永远不被计算**。
- **`cudaMemcpy` 方向写错**：现象是 `invalid argument` 错误，或数据被拷贝到错误方向后结果全错 → 四个参数是 `(目的指针, 源指针, 字节数, 方向)`，方向枚举 `cudaMemcpyHostToDevice` / `DeviceToHost` / `DeviceToDevice` 必须与两个指针的实际位置一致。**尤其注意上下行的指针顺序是反的**：H2D 时目的在前（`d_A, h_A`），D2H 时也是目的在前（`h_C, d_C`），很容易把其中一个写反。
- **共享内存容量超限**：现象是 kernel 启动失败报 `invalid argument`，或 `cudaFuncSetAttribute` 报错 → A100 上静态共享内存每 block 上限是 48 KB，要超过它必须改用**动态共享内存**并调用 `cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize, bytes)` 显式 opt-in（A100

每 block 最多 163 KB、每 SM 共 164 KB）。同时要意识到：共享内存开得越大，每 SM 能驻留的 block 越少，**占用率会同步下降**（例如 48 KB/block 时 A100 每 SM 只能放 3 个 block，占用率 37.5%）。

- **host 指针与 device 指针混用**：现象是段错误（segmentation fault）、`invalid device pointer`，或者 kernel 里访问到主机内存导致非法地址错误 → `cudaMalloc` 返回的指针只能用于 device 代码与 CUDA API，绝不能传给 `free()` 或 `memcpy()`；反之 `malloc` 返回的指针传入 kernel 会触发 `Illegal address access`。命名上养成习惯：**device 指针加 `_d` 后缀，host 指针加 `_h` 后缀**。
- **主机端读回结果前忘记检查 kernel 是否真的跑过**：现象是 `verify` 报 FAIL 但看不出原因 → 先 `CUDA_CHECK(cudaGetLastError())` 再 `CUDA_CHECK(cudaDeviceSynchronize())`，两步都用宏包住；然后把 `cudaMemcpy` 回来的数组与 CPU 参考实现逐元素对比并打印**首个不匹配的下标与期望值**，这比打印"FAIL"有用一百倍。
- **对"默认流"的同步语义想当然**：现象是在多 stream 程序中，计时变快但结果变错 → kernel 启动是异步的，多个 stream 之间的顺序完全没有保证；默认流（stream 0）有特殊的隐式同步行为，混用默认流与自定义流时尤其容易出错。用 `cudaEventRecord` + `cudaStreamWaitEvent` 显式建立依赖关系。

思考题（带答案）

Q1. 某应用有 8% 的代码无法并行化。在 128 个处理器的机器上，理论最大加速比是多少？如果工程师花两个月把这 8% 串行代码优化到只占原始串行时间的 1/4（即优化后串行部分只占并行版本的 2%），加速比会变成多少？ 按 Amdahl 定律 `Speedup(P) = 1/(s + (1−s)/P)`。第一种情况 s = 0.08、P = 128：`1/(0.08 + 0.92/128) = 1/(0.08 + 0.0071875) = 1/0.0871875 = 11.47×`，效率 `11.47/128 = 8.96%`。注意这**远低于**上界 `1/0.08 = 12.5×`，说明此时还受并行部分限制。第二种情况：串行部分的绝对时间变成原来的 1/4，设原始总时间为 1，则新串行时间 0.02、新并行时间仍为 0.92；新的串行占比 `s' = 0.02/(0.02+0.92) = 0.02128`，`Speedup(128) = 1/(0.02128 + 0.97872/128) = 1/(0.02128 + 0.007646) = 34.6×`——**串行时间减少 4 倍，总加速比提升 3 倍**，且上界从 12.5× 抬升到 47×。这正是"先消灭串行瓶颈"的量化依据。

Q2. 一个 kernel 在 A100 上每个线程需要 40 个寄存器、每个 block 256 个线程、使用 0 字节共享内存。(a) 每 SM 最多能驻留多少个这样的 block？(b) 占用率是多少？(c) 如果改用 `__launch_bounds__(256, 8)` 强制编译器把寄存器压到 32 个以内，占用率变成多少，是否一定更快？ (a) 逐项取 min：线程维度 `2048/256 = 8` 个 block；warp 维度 `64/(256/32) = 8` 个 block；寄存器维度按 warp 为单位、256 个寄存器为粒度向上取整：`regsPerWarp = 40 × 32 = 1280`，已对齐到 256 的倍数，`byRegs = 65536/(1280 × 8) = 6.4 → 6` 个 block；共享内存维度无限制；硬件 block 上限 32。**取 min → 6 个 block/SM**。(b) 占用率 `= 6 × 256 / 2048 = 768/2048 = 37.5%`（warp 粒度也是 `6×8/64 = 37.5%`）。(c) 压到 32 个寄存器后 `byRegs = 65536/((32×32)×8) = 65536/8192 = 8` 个 block，占用率变成 `8×256/2048 = 100%`，**提升 2.67 倍**。但不一定更快：编译器为了把寄存器压到 32 个，可能把局部变量溢出到 local memory（本质是显存），每次溢出访问都要 400–800 周期，反而拖慢。**判定方法**：编译时加 `-Xptxas -v` 查看 "spill stores/loads" 计数，若为 0 则收益是确定的；若出现大量 spill，应当接受较低占用率，或改写 kernel 降低寄存器需求。

Q3. 在 A100（1555 GB/s、19.5 TFLOPS）上，一个 kernel 处理 N = 2^{24} 个 float，每个元素读 1 次、写 1 次、做 4 次浮点运算。(a) 它的算术强度是多少？(b) 是计算受限还是带宽受限？(c) 理论最短耗时是多少？(d) 如果实测耗时是理论值的 1.4 倍，最可能的两个原因是什么？ (a) 每元素访存 `4 B 读 + 4 B 写 = 8 B`，运算 4 FLOP，`AI = 4/8 = 0.5 FLOP/Byte`。(b) 平衡点 `AI* = 19500/1555 = 12.5 FLOP/Byte`，`AI = 0.5 << 12.5` → **内存带宽受限**。(c) 访存下界 `= 2 × 2²⁴ × 4 B / 1555e9 = 134.22e6/1555e9 = 86.3 μs`；计算下界 `= 2²⁴ × 4 / 19.5e12 = 67.1e6/19.5e12 = 3.44 μs`。取 max → **86.3 μs**。(d) 最可能的两个原因是：(1) **访问未完全合并或未对齐**——若每个线程用步长访问而非连续访问，一个 warp 会占用多条 cache line，有效带宽降到 25%–50%；(2) **占用率不足导致延迟无法被隐藏**——按 Little 定律 A100 需要约 56 个 warp 的在途 load 才能打满带宽，若寄存器用量把占用率压到 50%（32 warp）以下，实际带宽就会显著低于峰值。其他可能原因还包括 L2 到 DRAM 的写合并效率、ECC 开销，以及测得的"1.4 倍"里混入了 kernel 启动开销（约 3–10 μs，占 86 μs 的 4%–12%）。

Lecture 2: CUDA 编程模型基础 —— Kernel、线程层次与内存模型 (对应 Lab 0 / Lab 1)

概述

本讲要回答一个核心问题：**如何把"每个输出元素彼此独立"的数据并行程序，翻译成 GPU 能高效执行的代码**。CUDA 给出的答案是"主机（host）写串行控制流 + 设备（device）写 SPMD kernel"的异构执行模型：程序员用 `__global__` 声明一个 kernel，用 `<<<gridDim, blockDim>>>` 启动它，GPU 用成千上万个拥有唯一编号的线程同时执行同一个函数体，各自用 `blockIdx` / `blockDim` / `threadIdx` 算出自己该处理的那一份数据。为了让这套抽象落到硅片上，硬件把 block 内部的线程按线性编号每 32 个切成一个 **warp（线程束）**，warp 才是 SM（Streaming Multiprocessor，流式多处理器）真正的调度单位，这直接导致"分支发散"和"合并访问"两个贯穿全课程的性能主题。最后，本讲给出 CUDA 的六级内存层次（register / local / shared / global / constant / texture），它们的作用域、生命周期和访问延迟相差两到三个数量级，是后续所有优化（尤其是 Lab 2 的 shared memory 复用）的物理基础。

核心概念与 GPU 架构图解

Kernel 函数与 SPMD 数据并行模型（Kernel / SPMD）

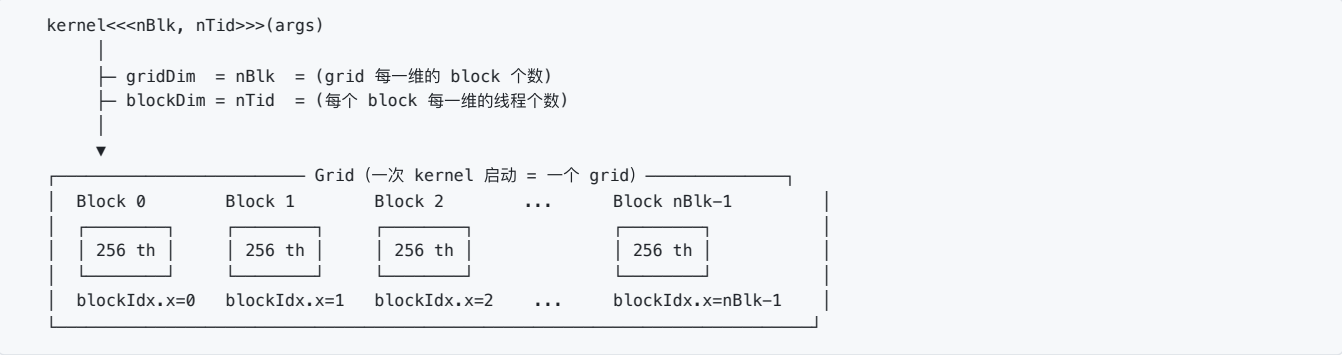
- **定义与目的**：`__global__` 修饰的函数叫 kernel（内核函数）。它在 GPU 上执行、由主机端代码启动，返回值必须是 `void`。一个 kernel 的一次启动会产生一个 **grid（线程网格）**，grid 中的每个线程执行**同一份** kernel 代码，但用自己唯一的索引决定"读哪块内存、算哪份数据、走哪条分支"。这就是 **SPMD（Single Program, Multiple Data，单程序多数据）** 模型。它解决的问题是：数据并行程序（如向量加法、图像逐像素变换）里 CPU 上的 `for` 循环完全可以被"把循环体变成线程体、把循环变量变成线程编号"的机械变换替代，从而把 N 次串行迭代摊到 N 个物理并行单元上。
- **直观解释（"它是什么？"）**：想象一家公司给 5000 名员工**发同一本操作手册**（kernel 代码），每个员工手册内容完全一样，但工牌号（thread index）不同，工牌号决定了"你去仓库取第几号箱子、把结果放进第几号货架"。没有主管逐个下命令说明每一步——手册就是程序，工牌号就是数据索引。这就是 SPMD：**一份程序，N 份数据**。与之对比，CPU 的写法是一位主管（单线程）亲自把 5000 个箱子搬完。

讲义中的经典起点是图像灰度化：`cpp for each pixel { pixel = gsConvert(pixel) } // Every pixel is independent of every other pixel` 因为"每个像素与其它像素无关"，所以这个循环的每一次迭代都可以变成一个独立的线程：这正是**数据并行（data parallelism）** 的判定标准——**输出的每个元素只依赖输入，不依赖其它输出**（无循环携带依赖）。

- **架构/机制图解**：



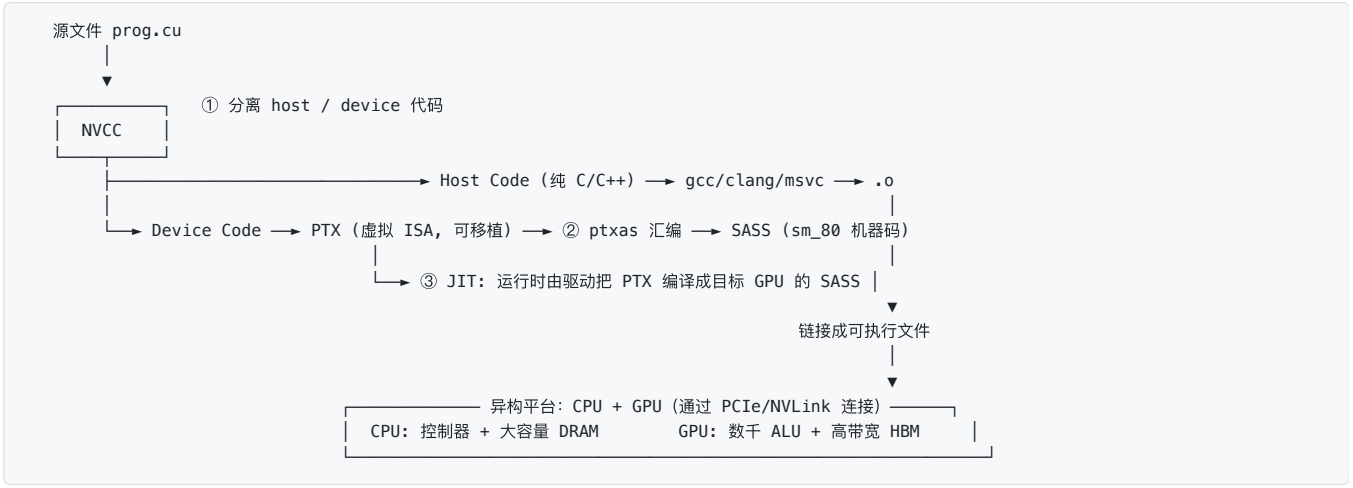
一次 kernel 启动的逻辑结构（`KernelA<<<nBlk, nTid>>>(args)` 是"异步"的，主机发完命令立刻继续往下跑）：



性能视角：SMPD 的"零成本展开"是有代价的——网格里可能有大量线程因为越界检查而什么都不做（例如 N=1000、block=256 时需要 4 个 block 共 1024 个线程，24 个线程被 `if (i<N)` 判掉，浪费率 2.3%），同时 kernel 启动本身有约 3–10 μs 的固定开销，所以小数据量（< 10 万个元素）时往往跑不过 CPU。

异构计算与 host/device 代码分离（Heterogeneous Computing）

- **定义与目的**：一个 CUDA 程序同时包含两种代码：**host 代码**在 CPU 上执行（做 I/O、内存分配、启动 kernel、串行部分），**device 代码**在 GPU 上执行（做计算密集的并行部分）。三个函数修饰符划清了边界：`__host__`（只能在 CPU 调用、在 CPU 执行，默认）、`__device__`（只能在 GPU 调用、在 GPU 执行）、`__global__`（**从 CPU 调用、在 GPU 执行**，因此被叫做 kernel）。目的是让程序员用**一门语言、一个源文件**同时驾驭两个不同架构的处理器，而不必写两套工程。
- **直观解释（"它是什么？"）**：把 CPU 比作**总经理**，GPU 比作**上千人的车间**。总经理不做流水线活，只负责"下订单、把原材料从仓库（主机内存）运到车间（显存）、发号施令（kernel 启动）、把成品运回来"。`__global__` 就是"发给车间的一张工单"，`__device__` 是"车间内部师傅之间互相调用的工序说明书"，`__host__` 是"办公室内部的流程"。
- **架构/机制图解**：



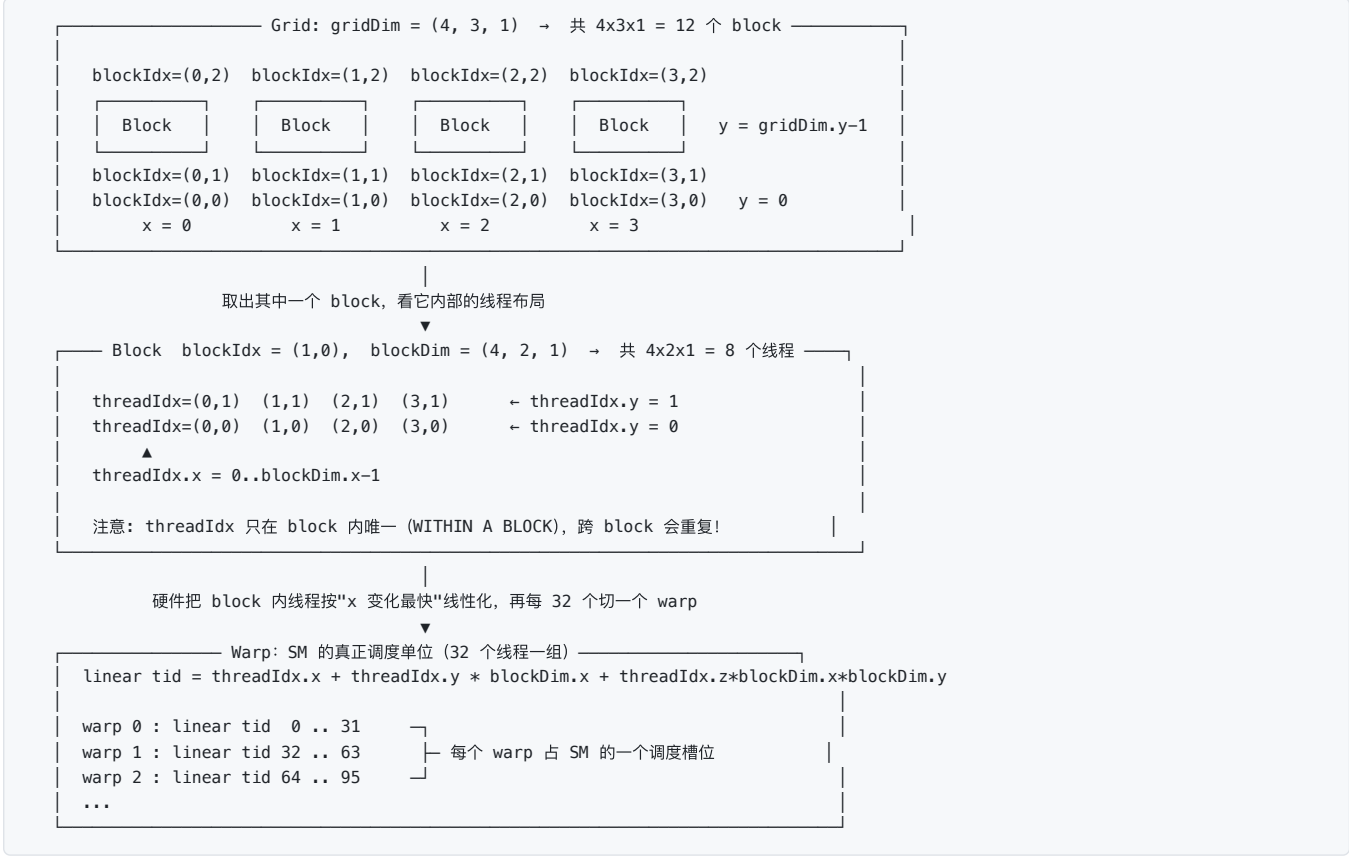
三种修饰符的调用/执行矩阵（对应讲义 "More on CUDA Function Declarations"）：

修饰符	执行位置	只能被谁调用	备注
`__host__`	host	仅 host	默认，可省略
`__global__`	device	仅 host （kernel）	返回类型必须是 void，异步启动
`__device__`	device	仅 device	高效的内联小函数；不能单独出现在函数体内（需与 `__shared__` / `__constant__` 等连用才合法）
`__host__ __device__` 两者	两者		一份代码两种编译，常用于公共工具函数

性能视角：host 与 device 之间的每次 `cudaMemcpy` 都要走 PCIe 4.0 x16（双向理论 32 GB/s，实测约 25 GB/s）或 NVLink（A100 上约 600 GB/s 双向）。相对 A100 自身 1555 GB/s 的显存带宽，PCIe 要慢 **48 倍**。因此一条铁律是：**数据搬移次数越少越好，单次搬移量越大越好**；Lab 1 中把 A、B 各传一次、C 取回一次是标准做法。

线程层次：Grid → Block → Thread（Thread Hierarchy）

- **定义与目的**：CUDA 线程被组织成**三层**：一个 grid 由若干 **thread block（线程块）** 组成，一个 block 由若干 **thread** 组成；每一层都可以是 1D/2D/3D 的（用 `dim3` 描述）。每个 block 有唯一的三元组 `blockIdx`（取值 0 … gridDim−1），每个线程在 block 内也有唯一的三元组 `threadIdx`（取值 0 … blockDim−1）。这套层次不是为了好看，而是为了：① 让"多维数据"（图像、矩阵、体数据）的寻址在代码里直观可读；② 给硬件一个明确的**可调度性与可同步性边界**——同一个 block 内的线程可以用 shared memory 通信、用 `__syncthreads()` 同步，不同 block 之间**不能**这样合作，且**block 的执行顺序是任意的（arbitrary order）**。
- **直观解释（"它是什么？"）**：把 grid 想成**整栋写字楼**，block 是**一间间独立的办公室**，thread 是**办公室里的员工**。同一间办公室的员工可以共用一块白板（shared memory）并随时喊"大家停一下"（`__syncthreads()`）；不同办公室之间没有共享白板，只能通过楼下的公告栏（global memory）传话，而且**谁先上班谁后上班没有规定**——你不能假设 12 楼的办公室一定比 7 楼先开工。这就是"block 执行顺序任意"的直观含义，也是为什么 kernel 里绝不能写"等别的 block 先算完"这种逻辑。
- **架构/机制图解**：



内置变量速查（全部是只读的 `uint3` / `dim3` 类型）：

变量	类型	含义	取值范围
gridDim	dim3	grid 每一维的 block 个数	x<=2^31-1, y,z<=65535
blockIdx	uint3	当前 block 在 grid 中的坐标	0 .. gridDim-1（每一维）
blockDim	dim3	每个 block 每一维的线程个数	x<=1024, 且 x*y*z<=1024
threadIdx	uint3	当前线程在 block 内的坐标	0 .. blockDim-1（每一维）
warpSize	int	一个 warp 的线程数（当前所有 NVIDIA GPU）	32

关键性能数字：**一个 block 最多 1024 个线程**、`blockDim.y` / `blockDim.z` 最多 1024、`gridDim.y` / `gridDim.z` 最多 65535（`gridDim.x` 可达 2^31-1）。这些上限在概念上"不是 CUDA 编程模型的一部分"——编程模型只要求"线程能分组、组内能同步"，1024、32 都是**硬件实现决定（implementation decision）**，所以代码里应尽量用 `warpSize` 或 `#define` 而不是硬编码 32。

全局线程编号的推导与多维索引展平（Linearization / Index Flattening）

- **定义与目的**：kernel 里最常用的一行就是 `int i = blockIdx.x * blockDim.x + threadIdx.x;`。它把"二维的 block 坐标 + 一维的线程坐标"压成一个**沿 x 方向从 0 连续递增**的全局编号，用来索引一维数组。推广到二维、三维时，就是把 (row, col) 映射成行主序（row-major）的一维偏移 `Row * width + Col`。目的是**让相邻线程访问相邻内存**，这是达成合并访问（coalescing）的前提。
- **直观解释（"它是什么？"）**：像电影院找座位。"第 3 排第 5 号"是二维坐标，但影院经理清点人数时用的是"第 3 排之前的 2 排共 2×每排座位数，再加第 5 号"这样一个**连续的一维编号**。CUDA 的这行公式就是"排号 × 每排座位数 + 座号"；`blockIdx.x` 是"排号"（排很长，排内又有 block），`blockDim.x` 是"每排有几个 block 的位置"。
- **架构/机制图解**：

```
N = 1000, block size = 256  →  gridDim.x = ceil(1000/256) = 4
一维情况：
    i = blockIdx.x * blockDim.x + threadIdx.x
                |           |            |
                |           |            └── 块内线程号 E  (0..255)
                |           └─────────────── 每块线程数 D  (=256)
                └─────────────────────────── 块号 C  (0..3)

┌── Block 0 ──────────────┐┌── Block 1 ──────────────┐┌── Block 2 ──────┐┌─ Block 3 ──┐
│ tid: 0   1   2 ... 254 255|| tid: 0   1   2 ... 254 255|| 0 ... 255      || 0 ... 255 |
│ i  : 0   1   2 ... 254 255|| i  :256 257 ... 510 511  ||512 ... 767     ||768 ...1023|
└─────────────────────────────┘└─────────────────────────┘└─────────────────┘└────────────┘
                                                ▲
                                  i = 1024 > N-1 = 999  →  被 if(i<N) 屏蔽
                                  → 启动 1024 个线程，只有 1000 个干活，浪费 2.34%

二维情况：把两个一维公式分别用在两个方向上
    Col = blockIdx.x * blockDim.x + threadIdx.x;     // 横向编号（列）
    Row = blockIdx.y * blockDim.y + threadIdx.y;     // 纵向编号（行）
    offset = Row * width + Col;                      // 行主序展平（讲义 Page 12 的 2D 数组布局）

行主序示意图（width = 4）：M2,1 → 2*4 + 1 = 9
    列:   0    1    2    3
行 0 ┌────┬────┬────┬────┐
     │ 0  │ 1  │ 2  │ 3  │
行 1 ├────┼────┼────┼────┤
     │ 4  │ 5  │ 6  │ 7  │
行 2 ├────┼────┼────┼────┤
     │ 8  │ 9  │ 10 │ 11 │   ← 元素 (2,1) 的线性下标 = 2*4+1 = 9
行 3 ├────┼────┼────┼────┤
     │ 12 │ 13 │ 14 │ 15 │
     └────┴────┴────┴────┘
```

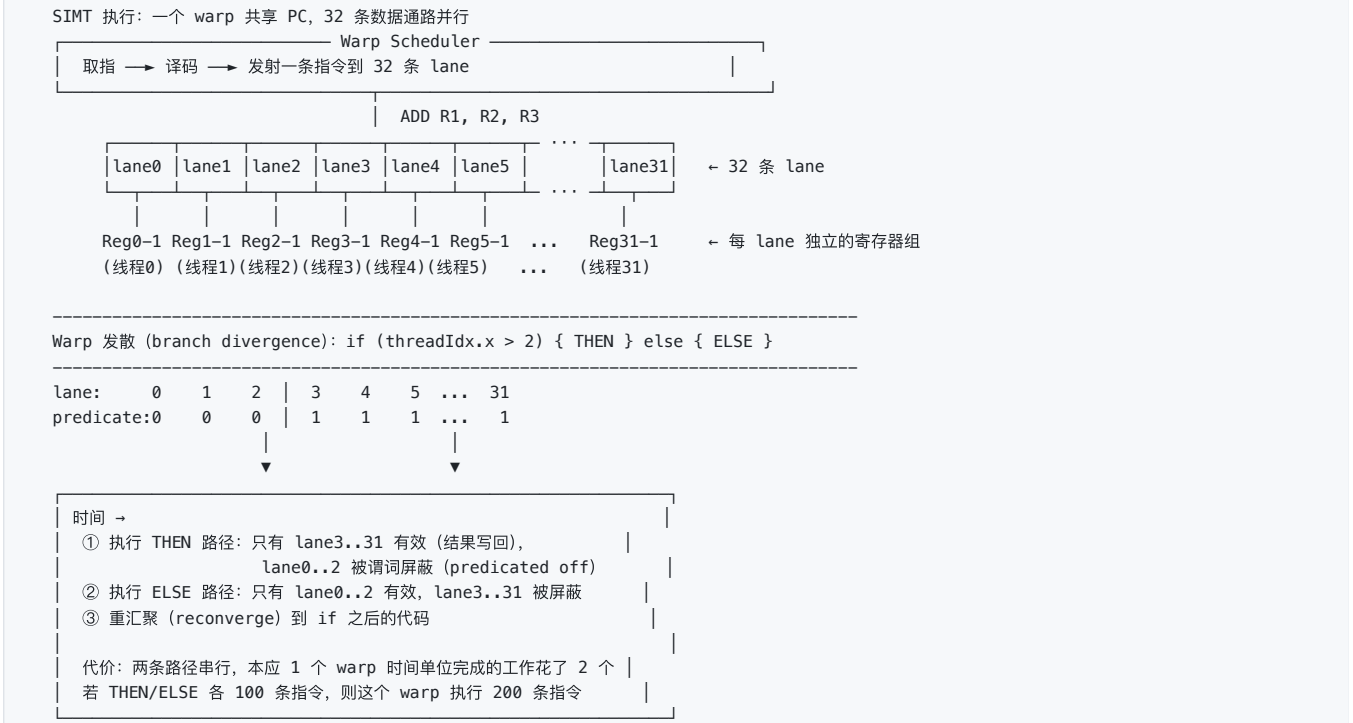
性能视角：**block 数量必须"向上取整"**，否则末尾元素没人处理：

```
gridDim.x = ceil(N / blockDim.x) = (N + blockDim.x - 1) / blockDim.x   ← 整数写法

反例：N = 1000, blockDim.x = 256
   C 语言里 N/256 = 1000/256 = 3 （整数除法向下取整！）
   → 只启动 3 个 block = 768 个线程  →  元素 768..999 共 232 个永远不被计算（静默错误）
正解：(1000 + 255)/256 = 1255/256 = 4  →  1024 个线程覆盖 1000 个元素
讲义写法：dim3 DimGrid(ceil(N/256.0), 1, 1) 或 dim3 DimGrid(N/256,1,1); if(N%256) DimGrid.x++;
```

Warp 与 SIMT 执行模型（Warp / SIMT）

- **定义与目的**：一个 block 内的线程按照**线性化线程编号**每 32 个一组被切成 **warp（线程束）**：线程 0–31 是 warp 0，32–63 是 warp 1，依次类推。warp 是 SM 的**调度单位**，一个 warp 内的 32 个线程**在同一时刻被同一套硬件用一个指令流驱动**，这就是 **SIMT（Single Instruction, Multiple Threads）**。它解决的问题是：如果 32 个线程各自取指、译码，控制开销会大到不可接受；让它们共享取指/译码、只让数据通路并行，就能用极低的控制成本换来极高的吞吐。
- **直观解释（"它是什么？"）**：像**军训方队**。教官（warp scheduler）只喊一句"向右转"，整个 32 人方队同时转身——取指令、译码只做一次，动作由 32 个人各自的身体完成。但如果教官喊"戴眼镜的向前一步"，队伍里只有一部分人动，另一部分人站着等，**方队还是得按两条命令花两倍时间走完**——这就是 warp 发散（divergence）。
- **架构/机制图解**：



避免发散的经典手法（讲义 Page 27）：让**分支粒度是 warp size 的整数倍**：

```
if (threadIdx.x > 2) {            // 坏：同一 warp 内 lane0..2 与 lane3..31 走不同路径
    /* THEN path (lots of lines) */
} else {
    /* ELSE path (lots more lines) */
}

if (threadIdx.x / WARP_SIZE > 2) { // 好：warp 0 全体走 THEN，warp 1.. 全体走 ELSE
    /* THEN path (lots of lines) */
} else {
    /* ELSE path (lots more lines) */
}
```

现代 GPU（Volta sm_70 及以后）引入了**独立线程调度（Independent Thread Scheduling）**：每个线程拥有自己的 PC 和调用栈，发散的分支可以基于 **per-thread 的 SIMT 栈**交错推进，不再强制"先走完 THEN 再走完 ELSE"。这让 warp 内的线程级同步原语（如 `__syncwarp()`）和细粒度生产者–消费者模式变得可行（例如 `__shfl_sync` 交换需先 `__syncwarp()`）。但请注意：**独立线程调度并不消除发散的性能代价**——32 条 lane 的物理数据通路依然是共享的，两个分支仍然要串行占用发射槽，只是调度更灵活、不再有"死锁式"的假设。

性能数字（典型值）：

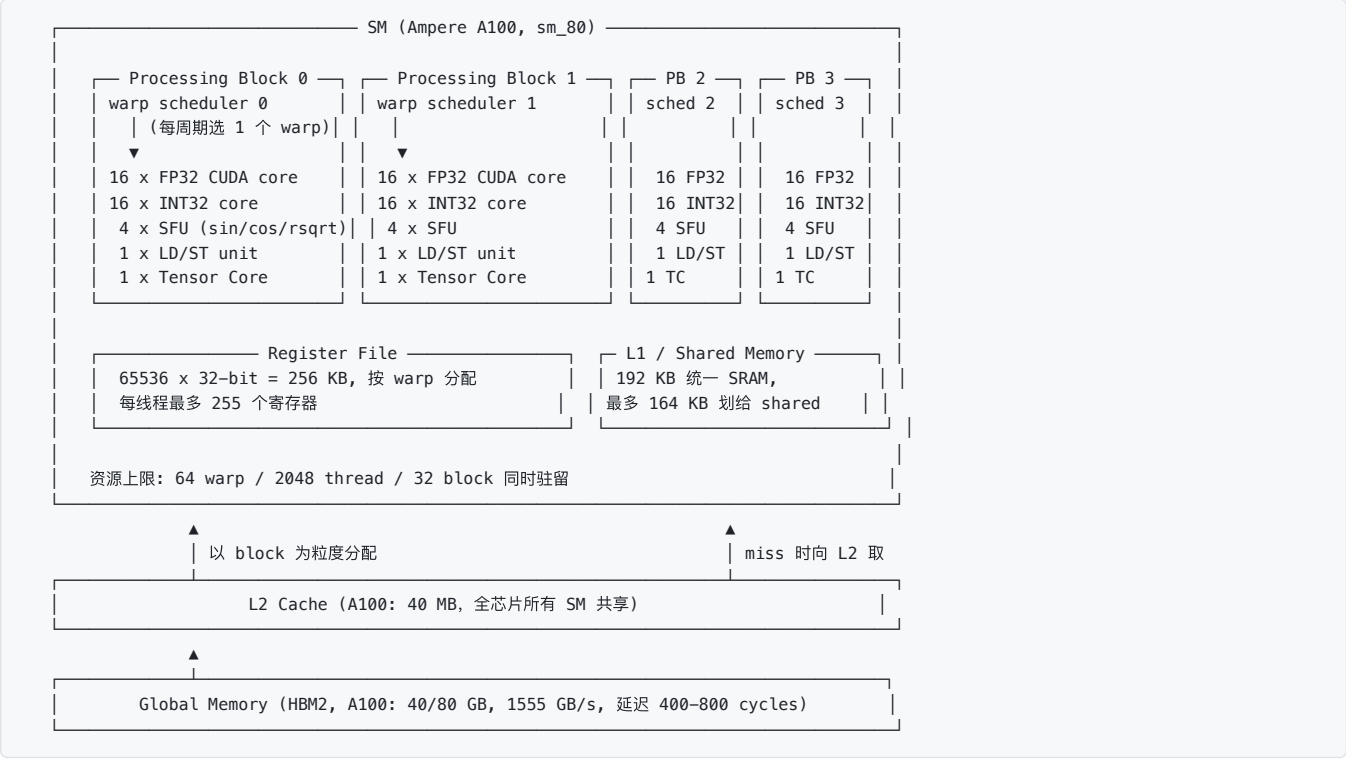
```
项目                                A100 (sm_80)       RTX 4090 (sm_89)
------------------------------      ------------       ----------------
每 SM 最多驻留 warp                   64                 48
每 SM 最多驻留线程                    2048               1536
warp 大小 / 调度单位                  32                 32
每 SM warp scheduler 数量             4                  4
每周期可发射的 warp 指令数            4                  4
每条 warp 指令的 lane 数              32                 32
一个 256 线程 block =                 8 个 warp          8 个 warp
一个 1024 线程 block =                32 个 warp         32 个 warp
全 SM 满载时未完成的 warp 状态总量    64                 48
→ 讲义 "Thread Scheduling (1/2)" 问题：3 个 256 线程 block 分到 1 个 SM，
  共 3 * (256/32) = 24 个 warp
```

SM 硬件结构与 warp 调度（Streaming Multiprocessor）

- **定义与目的**：SM（Streaming Multiprocessor）是 GPU 的"核心中的核心"，一个 GPU 有几十到一百多个 SM（A100 有 108 个，RTX 4090 有 128 个，H100 SXM 有 132 个）。SM 里装着：寄存器文件、L1 缓存/共享内存（统一的一块 SRAM，可分区配置）、若干 **warp scheduler**、以及执行单元（CUDA

core、SFU、LD/ST unit、Tensor Core）。**block 是以整块为单位被分配到 SM 上的**，一个 SM 可以同时驻留多个 block，SM 负责维护它们的 block id / thread id 并调度其中 warp 的执行。

- **直观解释（"它是什么？"）**：SM 像一间**有 4 个工位的车间**。车间里有 64 个工人（warp），每个人手里有活（指令），但只有 4 个工位（warp scheduler）。工位长每拍从"手头材料齐全"的工人里挑 4 个上工位干活；材料没到的（等待全局内存返回数据的 warp）就靠边站。**这就是延迟隐藏（latency hiding）**：只要待命的 warp 足够多，某几个 warp 卡在 500 周期的访存延迟上时，其它 warp 能继续占满工位，SM 的吞吐就不会掉。
- **架构/机制图解**：



零开销 warp 调度（zero-overhead warp scheduling）的两条规则（讲义 Page 24）：① **只有下一条指令的操作数已就绪的 warp 才是 eligible（可执行）**；② 在 eligible 的 warp 中按优先级策略挑选。因为"线程切换"所需的 PC、寄存器组都常驻在 SM 上，切换 warp **不保存/恢复上下文**，所以是零开销的——这正是 GPU 用海量线程隐藏长延迟的根本机制。

性能数字与"占用率（occupancy）"的定量计算（讲义 Page 28 的经典问题，用 A100 的参数重做）：

```
A100: 每 SM 2048 线程、64 warp、寄存器 65536、共享内存 164 KB、每 SM 最多 32 个 block
问题：colorToGreyscaleConversion 该用 8x8、16x16 还是 32x32 的 block?
设每个线程用 20 个寄存器、block 不用共享内存。

(a) block = 8x8 = 64 线程 = 2 warp
    线程限制: 2048/64 = 32 个 block
    寄存器限制: 每 block 寄存器 = 64*20 = 1280 (向上对齐到 256 粒度 → 1280)
              65536/1280 = 51.2 → 51 个 block
    → 受线程限制，32 个 block，恰好 32*64 = 2048 线程 = 64 warp = 100% 占用率
    但若硬件上限是 8 个 block/SM (讲义中的假设)，则只有 8*64 = 512 线程 = 16 warp = 25%

(b) block = 16x16 = 256 线程 = 8 warp
    线程限制: 2048/256 = 8 个 block
    寄存器限制: 256*20 = 5120 → 65536/5120 = 12.8 → 12 个 block
    → 8 个 block = 2048 线程 = 64 warp = 100% 占用率   ★ 最优

(c) block = 32x32 = 1024 线程 = 32 warp
    线程限制: 2048/1024 = 2 个 block
    → 2 个 block = 2048 线程 = 64 warp = 100% 占用率
    但若硬件上限是 1 个 block/SM (讲义中的假设)，则只有 1024 线程 = 32 warp = 50%

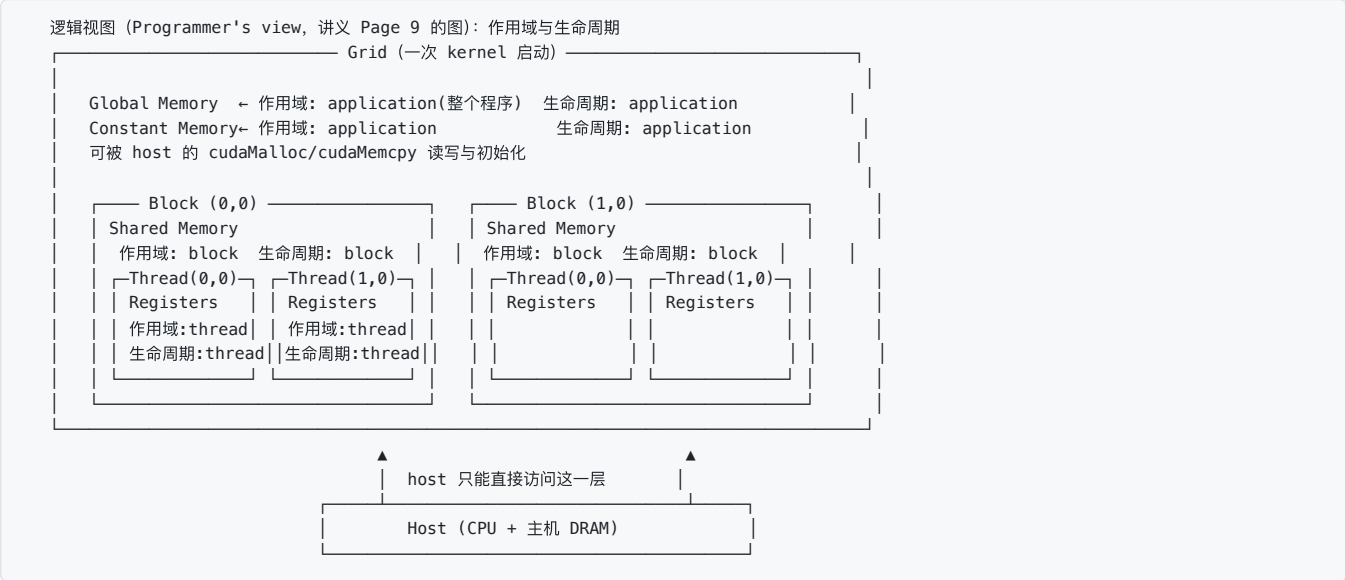
讲义给出的硬件假设（每 SM 最多 1536 线程、最多 8 个 block）:
   8x8 : 64 线程/块 → SM 可放 24 块，但受 8 块上限 → 只有 512 线程(16 warp) → 用不满
   16x16: 256 线程/块 → 1536/256 = 6 块 (≤8) → 1536 线程(48 warp) → 用满 ★
   32x32: 1024 线程/块 → 1536/1024 = 1 块 → 只有 1024 线程(32 warp) → 只用 2/3
```

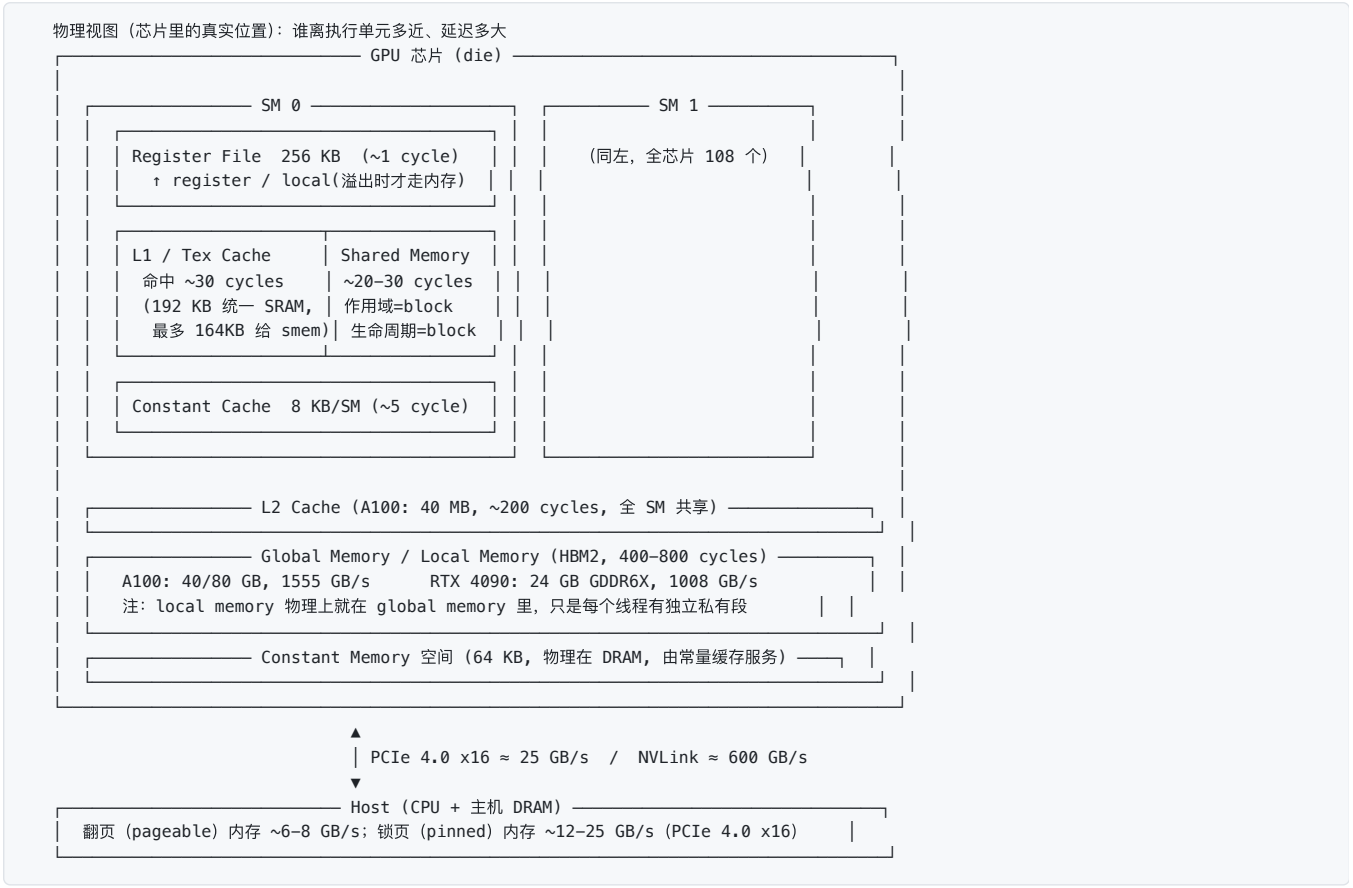
CUDA 内存模型（CUDA Memory Model）

- **定义与目的**：CUDA 为线程提供**六个**逻辑存储空间，它们用**变量声明的位置和修饰符**区分，各自有明确的**作用域（scope）** 与**生命周期（lifetime）**。设计目的是用一块**容量小但极快、且可被程序员显式管理**的片上存储（register + shared memory）来对抗"处理器比内存快得多"的冯·诺依曼瓶颈——CPU 靠

硬件缓存自动完成这件事，GPU 则把其中最关键的一层（shared memory）**交给程序员显式控制**，从而在数据复用上做到比硬件缓存更精准。

- **直观解释（"它是什么？"）**：把 GPU 想象成一家餐厅：
- **寄存器（register）** = 厨师手上的**调料勺**：最快（1 个周期），但每人只有几把（每线程最多 255 个）。
- **共享内存（shared memory）** = 后厨的**案板**：一块 block 里所有厨师共用，从冰箱取出食材放案板上反复切（复读），比每次跑冰箱快 20 倍（20—30 周期 vs 400—800 周期）。
- **全局内存（global memory）** = 楼下的**大冷库**：能装几十 GB，但跑一趟要几百个周期，带宽有限（A100 1555 GB/s）。
- **常量内存（constant memory）** = 墙上贴的**菜单/配方**：所有人都看同一张，读起来极快（命中常量缓存约 5 周期），但**只能读、不能改**（从 device 侧看）。
- **局部内存（local memory）** = 厨师自己的**抽屉**，但抽屉在楼下冷库里（物理上就是 global memory）——所以它的"局部"只是作用域局部，**速度并不快**，这是初学者最容易误解的地方。
- **纹理内存（texture memory）** = **自助餐台**：专门为"相邻线程读相邻数据 + 硬件插值"设计的只读通道，有独立缓存，适合有空间局部性的只读访问。
- **架构/机制图解**：





讲义 Page 10 的变量类型修饰符总表（**作用域 / 生命周期**是考试重点）：

```
变量声明                                  内存空间     作用域          生命周期        物理位置
-----------------------------------    ---------    -----------    -----------    --------------
int LocalVar;                          register    thread         thread         寄存器文件
__device__ __shared__ int SharedVar;  shared     block          block          SM 片上 SRAM
__device__ int GlobalVar;              global     application    application    HBM
__device__ __constant__ int ConstVar; constant   application    application    DRAM + 常量缓存
float LocalArray[16];  (带下标的自动     local       thread         thread         HBM（溢出后！）
  数组 / 寄存器不足时的溢出变量)

补充规则（讲义 Page 10）：
① 无修饰符的自动变量：基本类型与结构体 → 放进寄存器；
                      带下标的按线程私有数组 → 放进 global memory（即 local memory）
② __device__ 可以与 __shared__ / __constant__ 连用，
   但不能单独出现在函数体内（函数体内的 __shared__ 变量默认就是 device 存储）
```

延迟与带宽的数量级（全课程统一口径）：

```
存储空间             典型延迟             相对寄存器倍率      备注
-----------------    ------------------   ---------------    ----------------------------------
寄存器               1 cycle              1x                 每线程最多 255 个
共享内存             20-30 cycles         20-30x             无 bank conflict 时；否则按冲突倍数
常量内存(命中缓存)    ~5 cycles           5x                 同一 warp 读同一地址时最快（广播）
L1 命中              ~30 cycles           30x                全局访存命中 L1 时
L2 命中              ~200 cycles          200x               全芯片共享 40 MB
全局内存             400-800 cycles       400-800x           一个 warp 合并访问 = 128 B = 1 cache line
                                                              = 4 个 32 B sector = 1 次 transaction
主机内存(PCIe 4.0)   ~10,000+ cycles                         ~25 GB/s，比显存带宽慢 62 倍
```

复用全局内存的收益估算（讲义 Page 29 "150 GB/s Bandwidth Implies 37.5 GFLOPs"）：若每个浮点操作需要 4 字节内存（矩阵乘朴素版每个线程每对 M/N 元素做 1 次乘加 = 2 FLOP，却要读 4+4 = 8 字节，即 4 B/FLOP），则

```
可达性能 = 内存带宽 / 每 FLOP 需要的字节数
         = 150 GB/s / (4 B/FLOP)
         = 37.5 GFLOP/s          ← 而同期 GPU 的计算能力是 1000 GFLOP/s
也就是说：只发挥了 3.75% 的算力，程序被内存带宽死死卡住。
实测还会更低（约 25 GFLOP/s），因为内存不可能 100% 时间都在传数据。
结论：必须大幅削减对 global memory 的访问 —— 后续 Lab 的核心思想。
```

在 A100 上把同样的公式重算一遍：1555 GB/s ÷ 4 B/FLOP = **388.75 GFLOP/s**，而 A100 的 FP32 峰值是 **19.5 TFLOPS**，即只能发挥 **2.0%**——这就是为什么"复用内存访问"是 GPU 优化的第一性原理。

线程到数据的映射粒度：细粒度与粗粒度（Thread Mapping Granularity / Thread Coarsening）

- **定义与目的**：**细粒度（fine-grained）** 指"1 个线程只处理 1 个数据元素"；**粗粒度（coarse-grained）** 指"1 个线程处理多个元素"。在讲义中体现为 vecAdd 的两种线程分配：

```
cpp // 细粒度：一个线程一个元素，需要 ceil(N/256) 个 block vecAdd<<<ceil(N/256.0), 256>>>(...) i = blockIdx.x * blockDim.x + threadIdx.x; if (i < n) C[i] = A[i] + B[i]; // 粗粒度：一个线程两个元素，只需要 ceil(N/(2*256)) 个 block vecAdd<<<ceil(N/(2*256.0)), 256>>>(...) i = blockIdx.x * (2*blockDim.x) + threadIdx.x; if (i < n) C[i] = A[i] + B[i]; i = i + blockDim.x; if (i < n) C[i] = A[i] + B[i];
```

目的：减少总线程数/block 数，从而摊薄**每个线程的固定开销**（索引运算、判断、kernel 启动与 block 调度开销），并给编译器更多指令级并行（ILP）的机会。

- **直观解释（"它是什么？"）**：细粒度像**邮局派 1000 个邮递员每人送 1 封信**——派工的交通费（固定开销）比送信本身还贵。粗粒度像**派 500 个邮递员每人送 2 封信**，走一趟顺手送两封，派工成本减半。但如果每人送太多（比如 100 封），邮递员数量就不够，机器（SM）的并行度反而下降，还可能因为一次要拿着 100 封信而"手不够用"（寄存器溢出）。
- **架构/机制图解**：

```
细粒度：1 thread = 1 element                    粗粒度：1 thread = 2 elements
N = 1000, block = 256                             N = 1000, block = 256
gridDim = ceil(1000/256) = 4                      gridDim = ceil(1000/512) = 2
总线程 = 1024                                     总线程 = 512

┌─ Block 0 (i=0..255) ──┐                       ┌─ Block 0 ──────────────────────────┐
│ t0 t1 t2 ... t254 t255│                       │ t0   -> i=0   与  i=256            │
│ i0 i1 i2 ... i254 i255│                       │ t1   -> i=1   与  i=257            │
└───────────────────────┘                       │ ...                                │
┌─ Block 1 (i=256..511)─┐                       │ t255 -> i=255 与  i=511            │
│ t0 -> i=256           │                       └────────────────────────────────────┘
│ ...                   │                       ┌─ Block 1 ──────────────────────────┐
└───────────────────────┘                       │ t0   -> i=512 与  i=768            │
┌─ Block 2 (i=512..767)─┐                       │ ...                                │
┌─ Block 3 (i=768..1023)┐  ← 24 个线程空转       │ t231 -> i=743 与  i=999            │
                                                │ t232..t255 -> 两个都越界，空转     │
                                                └────────────────────────────────────┘
                                                空转 24 个线程（但只占 1 个 block 的一小部分）

合并访问仍然成立（关键！）：
  细粒度 warp 0: 线程 0..31 → 地址 0..127 B   → 1 条 128 B cache line
  粗粒度 warp 0 第一次访问: 线程 0..31 → 地址 0..127 B      → 1 条 cache line
  粗粒度 warp 0 第二次访问: 线程 0..31 → 地址 256..383 B    → 1 条 cache line
  两次访问各自合并，只是"跨步"了 blockDim.x 个元素
```

性能数字（N = 1000 万，A100，block = 256）：

```
配置              线程数       每线程元素    索引计算次数    总指令数(粗估)    实测带宽
---------------   ----------   ----------   -------------   --------------   ----------
细粒度            10,000,000       1           1 次/线程       ~5 条/元素      高
粗粒度(2 元素)     5,000,000       2           2 次/线程       ~4 条/元素      略高
粗粒度(4 元素)     2,500,000       4           4 次/线程       ~3.5 条/元素    更高
粗粒度(8 元素)     1,250,000       8           8 次/线程       ~3.2 条/元素    开始下降
粗粒度(16 元素)      625,000      16          16 次/线程       ~3.1 条/元素    下降(并行度不足)
→ 甜点区通常在每线程 2–8 个元素：既摊薄了开销，又保住了足够的 warp 数来隐藏延迟
→ 判断依据：线程总数至少要 >= 2 * SM 数 * 每 SM 线程上限，A100 即 2*108*2048 = 442K 线程
```

同步原语：`__syncthreads()` 与 `cudaDeviceSynchronize()`

- **定义与目的**：GPU 上没有"全局屏障"这种廉价操作，所以 CUDA 把同步分成两级：① **block 级**——`__syncthreads()` 是一个 **barrier（屏障）**，调用它的 block 内**所有**线程必须都到达这一点，才允许任何一个继续执行；它保证该点之前对 shared memory 和 global memory 的写对 block 内所有线程可见。② **host–device 级**——`cudaDeviceSynchronize()` 让主机阻塞等待，直到**此前提交到该设备的所有工作（所有 kernel、所有 memcpy）全部完成**。
- **直观解释（"它是什么？"）**：`__syncthreads()` 像小组讨论里的**"大家停一下，等所有人都把白板写完了再一起看"**——不等到齐不许动。`cudaDeviceSynchronize()` 像**项目经理等整个车间下班才开车走**。为什么需要前者？因为共享内存是 block 内所有线程共同的案板，如果 A 还没把菜放

上去，B 就去拿，拿到的就是垃圾数据。

- **架构/机制图解**：

```
__syncthreads() 的行为（block 内屏障）
┌──────────────────────────── Block（256 线程，8 warp）────────────────────────────┐
│                                                                                  │
│  warp0 ──■──┐                                                                    │
│  warp1 ──■──┤                                                                    │
│  warp2 ──■──┼──► 全部到达 ■ 之后才统一放行 ──► 后续代码                          │
│  ...        │                                                                    │
│  warp7 ──■──┘                                                                    │
│            ▲                                                                     │
│       barrier: 到达者计数 = blockDim，归零则全体放行                             │
│                                                                                  │
│  没有 __syncthreads() 时的竞态（race）：                                         │
│     时刻 t1: warp0 写 shared[0..31] = in[...]                                    │
│     时刻 t1: warp7 读 shared[0..31]  ← 可能读到还没写的旧值/垃圾                  │
│     时刻 t2: warp0 才写完                                                        │
└──────────────────────────────────────────────────────────────────────────────────┘

两级同步的适用范围
┌──────────────────────────┬──────────────────────────┬──────────────────────────┐
│ 同步方式                 │ 作用范围                 │ 谁在等待                 │
├──────────────────────────┼──────────────────────────┼──────────────────────────┤
│ __syncthreads()          │ 同一个 block 内所有线程  │ device 线程              │
│ __syncwarp(mask)         │ 同一个 warp 内指定 lane  │ device 线程              │
│ atomic 操作              │ 同一地址的读改写原子性   │ device 线程（不阻塞）    │
│ cudaDeviceSynchronize()  │ 整个设备上所有已提交的工作 │ host 线程（阻塞）      │
│ cudaStreamSynchronize(s) │ 某个 stream 上所有工作   │ host 线程（阻塞）      │
│ cudaMemcpy(D2H) 默认同步 │ 隐式同步该次拷贝         │ host 线程（阻塞）      │
└──────────────────────────┴──────────────────────────┴──────────────────────────┘
★ 关键：没有任何原语能同步"不同 block"！不同 block 之间只能靠
   kernel 结束（或 cooperative groups 的 grid.sync()，超出本讲范围）。
```

性能数字：`__syncthreads()` 本身开销很小（几十个周期），但**它会让整个 block 等待最慢的 warp**。如果各 warp 工作量不均（负载不均 load imbalance），屏障会把最慢的 warp 的耗时强加给所有人。例如 8 个 warp 中 7 个用 100 周期到达屏障、1 个因为一次 L2 miss 用了 400 周期，则整个 block 白等 300 周期，约等于损失 8 × 300 = 2400 warp-周期的发射机会。**因此屏障应尽量少用，且最好放在循环外或负载均衡的位置。**

另外，`__syncthreads()` **必须被 block 内所有线程以相同次数执行**——把它放进 `if (threadIdx.x < 16) { ... __syncthreads(); }` 这类分支里，会导致某些 warp 永远等不到同伴（在 Volta 之前直接死锁/未定义行为；Volta 之后官方仍然明确规定这是未定义行为）。

代码示例与性能分析

程序 1（Lab 0）：`device_query.cu` —— 读懂你手里的 GPU

```
// 文件: device_query.cu
// 编译: nvcc -O3 -arch=sm_80 device_query.cu -o device_query
// 运行: ./device_query
//
// 对应 Lab 0 (Device Query)：把后面所有占用率计算所需的硬件参数一次性打印出来。

#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

static void printDeviceInfo(int dev)
{
    cudaDeviceProp p;
    CUDA_CHECK(cudaGetDeviceProperties(&p, dev));

    int smemPerSM = 0, smemPerBlock = 0, maxBlocksPerSM = 0;
    CUDA_CHECK(cudaDeviceGetAttribute(&smemPerSM,
               cudaDevAttrMaxSharedMemoryPerMultiprocessor, dev));
    CUDA_CHECK(cudaDeviceGetAttribute(&smemPerBlock,
               cudaDevAttrMaxSharedMemoryPerBlock, dev));
    CUDA_CHECK(cudaDeviceGetAttribute(&maxBlocksPerSM,
               cudaDevAttrMaxBlocksPerMultiprocessor, dev));

    printf("\n================ Device %d: %s ================\n", dev, p.name);
    printf("compute capability            : %d.%d   (编译时用 -arch=sm_%d%d)\n",
           p.major, p.minor, p.major, p.minor);
    printf("multiProcessorCount (SM 数)   : %d\n", p.multiProcessorCount);
    printf("maxThreadsPerMultiProcessor   : %d\n", p.maxThreadsPerMultiProcessor);
    printf("maxThreadsPerBlock            : %d\n", p.maxThreadsPerBlock);
    printf("maxBlocksPerMultiProcessor    : %d\n", maxBlocksPerSM);
    printf("warpSize                      : %d\n", p.warpSize);
    printf("regsPerMultiprocessor         : %d  (= %d KB)\n",
           p.regsPerMultiprocessor, p.regsPerMultiprocessor * 4 / 1024);
    printf("regsPerBlock                  : %d\n", p.regsPerBlock);
    printf("sharedMemPerMultiprocessor    : %d B (= %d KB)\n",
           smemPerSM, smemPerSM / 1024);
    printf("sharedMemPerBlock             : %d B (= %d KB)\n",
           smemPerBlock, smemPerBlock / 1024);
    printf("maxGridSize   (x, y, z)       : (%d, %d, %d)\n",
           p.maxGridSize[0], p.maxGridSize[1], p.maxGridSize[2]);
    printf("maxThreadsDim (x, y, z)       : (%d, %d, %d)\n",
           p.maxThreadsDim[0], p.maxThreadsDim[1], p.maxThreadsDim[2]);
    printf("l2CacheSize                   : %d B (= %d KB)\n",
           p.l2CacheSize, p.l2CacheSize / 1024);
    printf("memoryBusWidth / memoryClock  : %d bit / %d kHz\n",
           p.memoryBusWidth, p.memoryClockRate);
    printf("totalGlobalMem                : %.2f GB\n",
           (double)p.totalGlobalMem / (1024.0 * 1024.0 * 1024.0));
    printf("concurrentKernels             : %d\n", p.concurrentKernels);
    printf("unifiedAddressing             : %d\n", p.unifiedAddressing);

    double bw = 2.0 * (double)p.memoryClockRate * ((double)p.memoryBusWidth / 8.0)
                / 1.0e6;  // DDR: 每个时钟周期传 2 次 -> GB/s
    printf("theoretical DRAM bandwidth    : %.1f GB/s\n", bw);

    // 用读到的参数直接算出这台机器上一个 256 线程 block 的驻留上限
    int blocksByThreads = p.maxThreadsPerMultiProcessor / 256;
    int blocksByBlocks  = maxBlocksPerSM;
    int resident        = blocksByThreads < blocksByBlocks ? blocksByThreads
                                                           : blocksByBlocks;
    printf("---- 256 线程/block 时的驻留上限（不含寄存器与 smem 限制）----\n");
    printf("  受线程数限制 : %d blocks\n", blocksByThreads);
    printf("  受 block 数限制: %d blocks\n", blocksByBlocks);
    printf("  => 实际驻留    : %d blocks = %d threads = %d warps\n",
           resident, resident * 256, resident * 256 / p.warpSize);
    printf("  => 占用率      : %.1f%%\n",
           100.0 * resident * 256 / p.maxThreadsPerMultiProcessor);
```

```
}

int main(void)
{
    int devCount = 0;
    CUDA_CHECK(cudaGetDeviceCount(&devCount));
    printf("CUDA devices found: %d\n", devCount);
    if (devCount == 0) {
        printf("No CUDA-capable device available.\n");
        return EXIT_SUCCESS;
    }

    for (int d = 0; d < devCount; ++d) {
        printDeviceInfo(d);
    }

    int cur = 0;
    CUDA_CHECK(cudaGetDevice(&cur));
    printf("\nCurrent device: %d\n", cur);

    CUDA_CHECK(cudaDeviceReset());
    printf("Device query done.\n");
    return EXIT_SUCCESS;
}
```

- **【代码做什么？】**
- `cudaGetDeviceCount` 拿到机器上可用的 GPU 数量；`cudaGetDeviceProperties` 一次性填满 `cudaDeviceProp`，其中就包含后面做占用率计算必需的 `multiProcessorCount`、`maxThreadsPerMultiProcessor`、`regsPerMultiprocessor`、`sharedMemPerMultiprocessor`、`warpSize`。
- 有三个属性（每 SM 最大共享内存、每 block 最大共享内存、每 SM 最大 block 数）在老的 `cudaDeviceProp` 里没有直接成员或容易与"capability 相关数值"混淆，所以用 `cudaDeviceGetAttribute` 按枚举名精确获取。
- 用 `2 × memoryClockRate(kHz) × memoryBusWidth(bytes) / 1e6` 估算**理论峰值带宽**：`memoryClockRate` 单位是 kHz，乘 `busWidth/8` 得到字节/秒，再乘 2 是因为 GDDR/HBM 在一个时钟周期内传两次数据。A100 上是 `2 × 1215 MHz × 512 B = 1244 GB/s`（官方标称 boost 后约 1555 GB/s，差异来自 boost 时钟）。
- 最后用读到的参数**当场算一遍占用率**：256 线程的 block，先看线程数允许几个 block，再看硬件 block 数上限允许几个，取小者；再换算成 warp 数与百分比。
- `cudaDeviceReset()` 显式归还所有设备资源，等价于进程退出时的隐式清理，但在同一进程里反复跑多轮测试时可以避免状态残留。
- **【并行机制与硬件映射解说】**
- 这个程序本身**不启动任何 kernel**，所以没有 warp、没有 SM 调度。它属于"元信息程序"：它的价值在于把**后续所有代码的调优参数**从"查文档"变成"运行时读出来"。同一个二进制在 A100（108 SM）、RTX 4090（128 SM）、RTX 2080 Ti（68 SM）上跑出的数字完全不同，硬编码常量必然出错。
- 由打印结果可以立刻推出并行槽位规模：A100 = 108 SM × 2048 线程 = **221,184 个同时驻留线程** = 108 × 64 = **6912 个同时驻留 warp**；RTX 4090 = 128 × 1536 = **196,608 线程** = 128 × 48 = **6144 个 warp**。这就是"要跑满 GPU 至少需要多少线程"的答案。
- 如果某个 kernel 的 grid 只有 1 个 block（例如 `kernel<<<1, 256>>>`），那么无论 GPU 有多少 SM，都只有 1 个 SM 在干活，其余 107 个 SM 全部空闲——利用率 1/108 ≈ 0.93%。这是最典型的"GPU 加速比不满意"的原因。
- 观察 `maxThreadsDim = (1024, 1024, 64)`：**x×y×z 的总乘积不得超过 1024**，所以 `dim3(16,16)` 合法（256），`dim3(64,64)` 非法（4096）；而 `maxGridSize.y = 65535` 意味着二维 grid 的 y 方向最多 65535 个 block，处理超大图像时若 `H/16 > 65535`（即 H > 1,048,560 像素）就必须改用一维 grid 手工切分。
- **【性能优化分析】**
- **占用率（occupancy）**：占用率 = 实际驻留 warp 数 ÷ 硬件上限 = `(驻留 block 数 × block 线程数 ÷ 32) ÷ (maxThreadsPerMultiProcessor ÷ 32)`。对 A100、256 线程 block：`2048/256 = 8` 个 block，恰好 `8×256 = 2048` 线程 = 64 warp = **100%**。但这是"不含寄存器/共享内存约束"的理论上限，真实值还要满足：`text 寄存器约束: 驻留 block 数 <= floor(regsPerSM / (numRegs * threadsPerBlock 对齐到 256)) 共享内存约束: 驻留 block 数 <= floor(smemPerSM / smemPerBlock) 最终上限 = min(线程约束, 寄存器约束, smem 约束, maxBlocksPerSM)`
- **算术强度与 Roofline**：本程序不涉及计算，只给出后续分析的"机器常数"：`text A100 脊点 (ridge point) = 峰值算力 / 峰值带宽 = 19.5e12 / 1555e9 = 12.5 FLOP/Byte RTX 4090 脊点 = 82.6e12 / 1008e9 = 81.9 FLOP/Byte 含义：算术强度 < 12.5 FLOP/B 的 kernel 在 A100 上必然带宽受限； 而同一 kernel 在 RTX 4090 上要 > 81.9 FLOP/B 才算计算受限。 → 4090 的"内存墙"相对更高，因为它的算力涨得比带宽快得多。`
- **瓶颈判定与优化方向**：Lab 0 的"性能目标"不是跑得快，而是**建立正确的机器模型**。用这些数字做两件事：① 由 `最大驻留线程数` 决定 grid 至少要有多少线程（建议 ≥ 2× 驻留量以覆盖尾部和调度抖动，A100 约 44 万线程）；② 由 `理论带宽` 作为 Roofline 的屋顶，Lab 1 的向量加法实测带宽一般能达到理论值的 80–92%，达不到就说明访存没合并好或并行度不足。

程序 2（Lab 1 风格）：`rgba_channel_split.cu` —— 二维线程索引与合并访问

```
// 文件: rgba_channel_split.cu
// 编译: nvcc -O3 -arch=sm_80 rgba_channel_split.cu -o rgba_split
// 运行: ./rgba_split                (默认 3840x2160 的 4K 图)
//       ./rgba_split 76 62          (讲义里的 76x62 小图，看网格如何向上取整)
//
// 对应内容: Lecture 3 的二维线程映射 (colorToGreyscaleConversion) + Lab 1 的向量化访存思想。
// 功能: 把 RGBA 交错存储的彩色图拆成 R/G/B/A 四个独立通道平面 (planar 格式)。

#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define BLOCK_X 16
#define BLOCK_Y 16

// ---------------- 版本 A: 朴素版，4 次单字节 load（每次跨步 4 字节） ----------------
__global__ void rgbaSplitNaive(const unsigned char* __restrict__ rgba,
                               unsigned char* __restrict__ r,
                               unsigned char* __restrict__ g,
                               unsigned char* __restrict__ b,
                               unsigned char* __restrict__ a,
                               int width, int height)
{
    int col = blockIdx.x * blockDim.x + threadIdx.x;   // 列号
    int row = blockIdx.y * blockDim.y + threadIdx.y;   // 行号
    if (row < height && col < width) {                 // 边界检查：挡住越界线程
        int pix = row * width + col;                   // 像素的线性编号（行主序）
        int o   = 4 * pix;                             // RGBA 平面中的字节偏移
        r[pix] = rgba[o + 0];
        g[pix] = rgba[o + 1];
        b[pix] = rgba[o + 2];
        a[pix] = rgba[o + 3];
    }
}

// ---------------- 版本 B: 向量化版，1 次 uchar4 load 取回全部 4 个通道 ----------------
__global__ void rgbaSplitVec(const uchar4* __restrict__ rgba,
                             unsigned char* __restrict__ r,
                             unsigned char* __restrict__ g,
                             unsigned char* __restrict__ b,
                             unsigned char* __restrict__ a,
                             int width, int height)
{
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row < height && col < width) {
        int pix = row * width + col;
        uchar4 p = rgba[pix];      // 一条 4 字节向量 load，warp 内正好拼成 128 字节
        r[pix] = p.x;
        g[pix] = p.y;
        b[pix] = p.z;
        a[pix] = p.w;
    }
}

// ---------------- CPU 参考实现 ----------------
static void rgbaSplitCPU(const unsigned char* rgba,
                         unsigned char* r, unsigned char* g,
                         unsigned char* b, unsigned char* a,
                         int width, int height)
{
    long n = (long)width * (long)height;
    for (long i = 0; i < n; ++i) {
        r[i] = rgba[4 * i + 0];
        g[i] = rgba[4 * i + 1];
        b[i] = rgba[4 * i + 2];
        a[i] = rgba[4 * i + 3];
```

```
    }
}

static int verify(const unsigned char* ref, const unsigned char* got, long n,
                  const char* name)
{
    for (long i = 0; i < n; ++i) {
        if (ref[i] != got[i]) {
            printf("  [FAIL] %s: index %ld ref=%u got=%u\n",
                   name, i, (unsigned)ref[i], (unsigned)got[i]);
            return 0;
        }
    }
    printf("  [PASS] %s: %ld bytes identical to CPU reference\n", name, n);
    return 1;
}

int main(int argc, char** argv)
{
    int width  = (argc > 1) ? atoi(argv[1]) : 3840;
    int height = (argc > 2) ? atoi(argv[2]) : 2160;
    long npix  = (long)width * (long)height;
    long nbyte = npix;                 // 每个通道平面的字节数

    printf("Image: %d x %d = %ld pixels\n", width, height, npix);

    // ---- 网格配置：核心是"向上取整"，否则图像右侧/底部会漏掉 ----
    dim3 block(BLOCK_X, BLOCK_Y);
    dim3 grid((width  + BLOCK_X - 1) / BLOCK_X,      // = ceil(width /16)
              (height + BLOCK_Y - 1) / BLOCK_Y,      // = ceil(height/16)
              1);
    long launchedThreads = (long)grid.x * grid.y * block.x * block.y;
    printf("Grid : (%u, %u) blocks of (%u, %u) threads\n",
           grid.x, grid.y, block.x, block.y);
    printf("Total blocks = %u x %u = %u ; total threads = %ld ; useful = %ld (%.2f%%)\n",
           grid.x, grid.y, grid.x * grid.y, launchedThreads, npix,
           100.0 * (double)npix / (double)launchedThreads);

    // ---- 主机端分配 ----
    unsigned char* h_rgba = (unsigned char*)malloc(4 * nbyte);
    unsigned char* h_r = (unsigned char*)malloc(nbyte);
    unsigned char* h_g = (unsigned char*)malloc(nbyte);
    unsigned char* h_b = (unsigned char*)malloc(nbyte);
    unsigned char* h_a = (unsigned char*)malloc(nbyte);
    unsigned char* c_r = (unsigned char*)malloc(nbyte);
    unsigned char* c_g = (unsigned char*)malloc(nbyte);
    unsigned char* c_b = (unsigned char*)malloc(nbyte);
    unsigned char* c_a = (unsigned char*)malloc(nbyte);
    if (!h_rgba || !h_r || !h_g || !h_b || !h_a || !c_r || !c_g || !c_b || !c_a) {
        fprintf(stderr, "host malloc failed\n");
        return EXIT_FAILURE;
    }
    for (long i = 0; i < 4 * nbyte; ++i) {
        h_rgba[i] = (unsigned char)((i * 37 + 11) & 0xFF);   // 确定性伪随机
    }

    // ---- 设备端分配 ----
    unsigned char *d_rgba = NULL, *d_r = NULL, *d_g = NULL, *d_b = NULL, *d_a = NULL;
    CUDA_CHECK(cudaMalloc((void**)&d_rgba, 4 * nbyte));
    CUDA_CHECK(cudaMalloc((void**)&d_r,    nbyte));
    CUDA_CHECK(cudaMalloc((void**)&d_g,    nbyte));
    CUDA_CHECK(cudaMalloc((void**)&d_b,    nbyte));
    CUDA_CHECK(cudaMalloc((void**)&d_a,    nbyte));

    CUDA_CHECK(cudaMemcpy(d_rgba, h_rgba, 4 * nbyte, cudaMemcpyHostToDevice));

    // ---- 计时 ----
    cudaEvent_t t0, t1;
    CUDA_CHECK(cudaEventCreate(&t0));
    CUDA_CHECK(cudaEventCreate(&t1));

    double traffic = (double)(4 * nbyte + 4 * nbyte);   // 读 4B/像素 + 写 4B/像素

    // ==== 版本 A ====
    CUDA_CHECK(cudaEventRecord(t0));
    rgbaSplitNaive<<<grid, block>>>(d_rgba, d_r, d_g, d_b, d_a, width, height);
    CUDA_CHECK(cudaEventRecord(t1));
    CUDA_CHECK(cudaEventSynchronize(t1));
```

```
    float msA = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&msA, t0, t1));
    CUDA_CHECK(cudaMemcpy(c_r, d_r, nbyte, cudaMemcpyDeviceToHost));
    CUDA_CHECK(cudaMemcpy(c_g, d_g, nbyte, cudaMemcpyDeviceToHost));
    CUDA_CHECK(cudaMemcpy(c_b, d_b, nbyte, cudaMemcpyDeviceToHost));
    CUDA_CHECK(cudaMemcpy(c_a, d_a, nbyte, cudaMemcpyDeviceToHost));
    printf("\n--- Version A: naive (4 byte loads, stride 4) ---\n");
    printf("  time = %.3f ms   effective BW = %.1f GB/s\n",
           msA, traffic / (msA * 1.0e-3) / 1.0e9);
    int okA = verify(h_rgba + 0, c_r, nbyte, "R plane") &
              verify(h_rgba + 1, c_g, nbyte, "G plane") &
              verify(h_rgba + 2, c_b, nbyte, "B plane") &
              verify(h_rgba + 3, c_a, nbyte, "A plane");
    (void)okA;

    // ==== 版本 B ====
    CUDA_CHECK(cudaMemset(d_r, 0, nbyte));
    CUDA_CHECK(cudaMemset(d_g, 0, nbyte));
    CUDA_CHECK(cudaMemset(d_b, 0, nbyte));
    CUDA_CHECK(cudaMemset(d_a, 0, nbyte));
    CUDA_CHECK(cudaEventRecord(t0));
    rgbaSplitVec<<<grid, block>>>((const uchar4*)d_rgba, d_r, d_g, d_b, d_a,
                                  width, height);
    CUDA_CHECK(cudaEventRecord(t1));
    CUDA_CHECK(cudaEventSynchronize(t1));
    float msB = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&msB, t0, t1));
    CUDA_CHECK(cudaMemcpy(c_r, d_r, nbyte, cudaMemcpyDeviceToHost));
    CUDA_CHECK(cudaMemcpy(c_g, d_g, nbyte, cudaMemcpyDeviceToHost));
    CUDA_CHECK(cudaMemcpy(c_b, d_b, nbyte, cudaMemcpyDeviceToHost));
    CUDA_CHECK(cudaMemcpy(c_a, d_a, nbyte, cudaMemcpyDeviceToHost));
    printf("\n--- Version B: vectorized (uchar4 load) ---\n");
    printf("  time = %.3f ms   effective BW = %.1f GB/s\n",
           msB, traffic / (msB * 1.0e-3) / 1.0e9);
    int okB = verify(h_rgba + 0, c_r, nbyte, "R plane") &
              verify(h_rgba + 1, c_g, nbyte, "G plane") &
              verify(h_rgba + 2, c_b, nbyte, "B plane") &
              verify(h_rgba + 3, c_a, nbyte, "A plane");
    (void)okB;

    printf("\nSpeedup (A/B) = %.2fx\n", msA / msB);

    // ---- CPU 参考结果（抽样验证，避免小图时 CPU 太慢） ----
    rgbaSplitCPU(h_rgba, h_r, h_g, h_b, h_a, width, height);
    printf("\nCPU reference check: %s\n",
           (h_r[123 % nbyte] == h_rgba[4 * (123 % nbyte) + 0]) ? "consistent" : "MISMATCH");

    // ---- 释放 ----
    CUDA_CHECK(cudaEventDestroy(t0));
    CUDA_CHECK(cudaEventDestroy(t1));
    CUDA_CHECK(cudaFree(d_rgba));
    CUDA_CHECK(cudaFree(d_r));
    CUDA_CHECK(cudaFree(d_g));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_a));
    free(h_rgba); free(h_r); free(h_g); free(h_b); free(h_a);
    free(c_r);    free(c_g); free(c_b); free(c_a);
    CUDA_CHECK(cudaDeviceReset());
    printf("\nDone.\n");
    return EXIT_SUCCESS;
}
```

- **【代码做什么？】**
- **决定网格形状**。`dim3 block(16,16)` 一共 256 个线程；`dim3 grid((W+15)/16, (H+15)/16)` 用**整数向上取整**覆盖整个图像。以讲义里的 76×62 为例：`ceil(76/16) = 5`、`ceil(62/16) = 4`，共 **5×4 = 20 个 block**，启动 20×256 = 5120 个线程，而图像只有 76×62 = 4712 个像素，**408 个线程会被边界检查挡住**（浪费 7.97%）。
- **每个线程算出自己负责的像素**。`col` 由 `blockIdx.x` 与 `threadIdx.x` 合成，`row` 由 `blockIdx.y` 与 `threadIdx.y` 合成——**注意 x 管列（横向）、y 管行（纵向）**，x 变化最快，正好对应 C 的行主序内存布局。
- **边界检查** `if (row < height && col < width)`。因为网格是向上取整的，最右一列 block 的 col 可能 ⩾ width（76×62 例子中 col 可取到 64…79，而 width=76，所以 col=76…79 的 4 列越界），最下一行 block 的 row 可能 ⩾ height（row 可取到 48…63，而 height=62，所以 row=62、63 越界）。**没有这个 if 就会写坏别人的内存（静默错误）**。

- **算出字节偏移**。`pix = row*width + col` 是像素的线性编号（行主序）；交错存储的 RGBA 中第 pix 个像素的 4 个字节位于 `4*pix + {0,1,2,3}`；输出平面中第 pix 个字节位于 `pix`。
- **两个版本做同一件事**：A 版读 4 次单字节，B 版读 1 次 `uchar4`（4 字节向量）。二者的**算法完全相同**，差别只在内存指令的粒度。
- **主机端调度**。先用 `cudaMalloc` 申请 5 块显存，`cudaMemcpy(H2D)` 把交错图送进去，启动 kernel，`cudaEventRecord` / `cudaEventSynchronize` 计时，再把 4 个平面 `cudaMemcpy(D2H)` 取回，与 CPU 参考实现逐字节比对，最后 `cudaFree` / `cudaDeviceReset` 收尾。
- **计时必须包住同步点**。kernel 启动是异步的，如果只测 `cudaEventRecord(t0); kernel<<<>>>; cudaEventRecord(t1);` 而不做 `cudaEventSynchronize(t1)`，读到的可能是"启动开销"而不是"执行时间"。
- **【并行机制与硬件映射解说】**
- **block → warp 的划分**：`block = (16,16)` 共 256 线程 = **8 个 warp**。线性化规则是 `linear = threadIdx.x + threadIdx.y * blockDim.x`（x 最快），所以：`text warp 0 : linear 0..31 = threadIdx.y=0 (x=0..15) 与 threadIdx.y=1 (x=0..15) → 覆盖第 0、1 两行的各 16 个像素 warp 1 : linear 32..63 = threadIdx.y=2 与 y=3 两行 ... warp 7 : linear 224..255 = threadIdx.y=14 与 y=15 两行 → 每个 warp 横跨 2 行 × 16 列 = 32 个像素，行内的 16 个线程地址连续`
- **warp 到 SM 的调度**：A100 有 4 个 warp scheduler/SM，每周期最多发射 4 条 warp 指令。这个 kernel 每个线程约用 **18–24 个寄存器**（`col/row/pix/o/p` 加上地址计算），以 20 个估算：每 block 寄存器 = `256 × 20 = 5120`，`65536 / 5120 = 12.8 → 12` 个 block；线程上限给 `2048/256 = 8` 个 block；共享内存用量为 0；A100 每 SM 最多 32 个 block。取最小值 **8 个 block/SM = 2048 线程 = 64 warp = 100% 占用率**（A100 上限 64 warp）。`text 占用率计算（A100, block=16x16, 20 regs/thread, 0 smem） blocks_by_threads = 2048 / 256 = 8 blocks_by_regs = 65536 / (256*20) = 65536/5120 = 12 (向下取整) blocks_by_smem = 164KB / 0 = 无限制 blocks_by_hw = 32 resident = min(8, 12, inf, 32) = 8 occupancy = (8*256)/2048 = 100% RTX 4090 (48 warp/SM, 1536 线程/SM, 100 KB smem) blocks_by_threads = 1536 / 256 = 6 blocks_by_regs = 65536 / 5120 = 12 resident = min(6,12,inf,32) = 6 → 6*256 = 1536 线程 = 48 warp = 100%`
- **共享内存与 bank conflict**：本 kernel **完全不用 shared memory**（数据没有复用，每个像素只被读一次），因此**不存在 bank conflict**。作为对照，如果误写成 `__shared__ unsigned char sTile[256];` 再让线程 `t` 访问 `sTile[t]`，由于 bank 宽度是 4 字节而元素是 1 字节，**线程 t=0..3 会撞到同一个 bank 0，形成 4-way bank conflict**，把本来 1 个周期的访问拖成 4 个周期（这正是字节型共享数组必须"padding 或按 int 组织"的原因）。
- **全局内存访问是否合并（coalesced）**——这是本程序的核心：`text 【版本 A：朴素版，字节 load，跨步 4 字节】 warp 内 32 个线程读 rgba[4*(base+j) + c]（j = 0..31, c 固定 0/1/2/3）字节地址：4*base+c, 4*base+4+c, ..., 4*base+124+c 地址跨度：128 字节 = 4 个 sector（每 sector 32 B）= 1 条 cache line 这 128 字节里本次指令真正用到的只有 32 字节 → sector 利用率 = 32/128 = 25% 4 条 load 指令各自都要走一遍这 128 字节（后 3 条通常命中 L1） → 对 L1/LSU 的压力是向量化版的 4 倍；每条 load 需要 4 个 sector wavefront → DRAM 侧因 L1 命中，实际流量接近 8 B/像素，但 L1 吞吐/MSHR 很快成为瓶颈， 实测带宽明显低于峰值（典型只有峰值的 30%~50%） 【版本 B：向量化版，uchar4 load】 warp 内 32 个线程读 rgba[base+j]（uchar4 = 4 字节）字节地址：4*base, 4*base+4, ..., 4*base+124 → 32 * 4 = 128 字节，且完全连续 = 1 条完整 cache line = 4 个 sector → 128/128 = 100% sector 利用率，一次 transaction 取回全部有用数据 ★ 【写侧：r[pix] = ...】 warp 内连续 pix → 写 32 个连续字节 = 恰好 1 个 32 字节 sector → 写合并良好（写最小粒度就是 sector，32 B 全用满）`
- **warp 发散**：kernel 里唯一的条件是 `if (row < height && col < width)`。对绝大多数 block 而言 32 个 lane 全部为真，**几乎零发散**；只有在图像最右一列（col ∈ [width, grid.x*16)）*和最下一行（row ∈ [height, grid.y*16)）的 block 上，同一个 warp 内会出现部分 lane 为假的情况。以 76×62 为例：右边界 block（blockIdx.x = 4）覆盖 col 64…79，其中 col ⩾ 76 的 4 列无效，而一个 warp 横跨 2 行 × 16 列 → 每行 16 个线程中有 4 个无效，**发散比例为 4/16 = 25%**，仅影响那 5 列中的 1 列 block，总体影响 < 2%。判断准则（讲义 Problem Solving）：**发散的前提是"执行路径依赖线程独有的量"**；下面这段代码因为 `q` 对所有线程相同（是 kernel 参数），**不会发散**：`cpp __global__ void do_work(int q, int *A) { int result = 0; if (q < 5) result = threadIdx.x; // q 与 threadIdx 无关 → 全 block 一致 → 无发散 A[threadIdx.x] = result; }`
- **寄存器使用量**：朴素版因为要同时维护 4 个字节的地址计算，寄存器可能略高（约 20–24）；向量化版把 4 个字节打包进一个 `uchar4` 寄存器对，寄存器数通常下降 2–4 个。若寄存器数超过阈值导致占用率掉档（例如从 20 涨到 33，`65536/(256*33) = 7` 个 block → 87.5%），可以用 `__launch_bounds__(256, 8)` 让编译器主动限制寄存器用量。
- **【性能优化分析】**
- **算术强度（arithmetic intensity）**：本 kernel 每个像素的"有用计算量"近似为 0（就是搬字节，至多算 4 次地址加法，按 4 FLOP/像素计），而内存流量是实打实的 8 字节/像素（读 4 + 写 4）：`text AI = FLOPs / Bytes = 4 / 8 = 0.5 FLOP/Byte （若完全不计运算则为 ~0）`
- **Roofline 模型**：`text 性能上限 = min( 峰值算力, AI × 峰值带宽 ) = min( 19.5e12 FLOP/s , 0.5 × 1555e9 B/s ) = min( 19 500 GFLOP/s , 777 GFLOP/s ) = 777 GFLOP/s ← 比峰值算力低 25 倍 结论：纯带宽受限（memory-bound），与"算得快不快"完全无关。`
- **代入具体数字的理想时间**（4K 图，3840×2160 = 8,294,400 像素）：`text 总流量 = 8,294,400 × 8 B = 66.36 MB A100 理想时间 = 66.36e6 B / 1555e9 B/s = 42.7 μs → 100% 峰值 达到 80% 峰值 → 53.4 μs；达到 50% → 85.3 μs RTX 4090 理想时间 = 66.36e6 / 1008e9 = 65.8 μs 小图 76x62: 总流量 = 4712 × 8 = 37.7 KB → 理想 24 ns， 远小于 kernel 启动开销(3–10 μs)，此时"性能"只反映启动开销。`
- **瓶颈判定**：**内存带宽受限**（AI = 0.5 << A100 脊点 12.5 FLOP/B）。因此优化方向只有三条：① 减少字节数；② 提高每次访存的字节效率（向量化）；③ 提高并行度以逼近峰值带宽。具体到本程序：`text ① 减少字节数：A 通道若下游不需要，读 uchar3 仍然要拉 4 字节的 cache line （RGB 交错时更糟： uchar3 是 3 字节对齐，warp 读 3B 跨步会覆盖 96 字节 但每条指令只用 32 字节 → 33% 效率）；可以考虑只做灰度化， 输出 1 字节/像素 → 流量从 8 B/像素降到 5 B/像素，理论提速 1.6x。 ② 提高访存效率：用 uchar4/float4 让 warp 访问 = 128 B 完整 cache line（本程序 B 版） 实测提速通常在 1.5x ~ 3x（取决于 L1 压力）。 ③ 提高并行度：4K 图有 32,400 个 block，远超 A100 的 108 SM × 8 block = 864 个驻留槽位，并行度充足；小图（76×62 只有 20 个 block）则 GPU 基本闲置， 此时应改用"一个线程处理多个像素"的粗粒度方案，或者干脆用 CPU。`

程序 3：`mem_space_demo.cu` —— 六级内存空间的作用域与生命周期

```
// 文件: mem_space_demo.cu
// 编译: nvcc -O3 -arch=sm_80 mem_space_demo.cu -o mem_space_demo
// 运行: ./mem_space_demo
//
// 在一个 kernel 里同时使用 register / shared / constant / global(__device__),
// 并在主机端打印 cudaFuncGetAttributes 得到的寄存器数与共享内存使用量。

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define N     4096
#define BLOCK 256

// ============ 四种存储空间的声明 ============
// (1) __device__ 变量 -> global memory，作用域 application，生命周期 application
__device__ float gScratch[N];
__device__ int   gBlockCount = 0;

// (2) __constant__ 变量 -> constant memory，作用域 application，生命周期 application
//     可以在声明处初始化（编译期写入常量段），运行时也能被 host 改写
__constant__ float cWeight[4] = {0.5f, 1.0f, 1.5f, 2.0f};

__global__ void memSpaceDemo(const float* __restrict__ in,
                             float* __restrict__ out,
                             int n)
{
    // (3) __shared__ 静态数组 -> shared memory，作用域 block，生命周期 block
    __shared__ float sData[BLOCK];

    // (4) extern __shared__ 动态数组 -> shared memory，大小由启动配置给出
    extern __shared__ float sDyn[];

    // (5) 无修饰符的自动变量 -> register，作用域 thread，生命周期 thread
    int   tid  = blockIdx.x * blockDim.x + threadIdx.x;
    float self = (tid < n) ? in[tid] : 0.0f;      // 读 global memory（也可能是 L1 命中）
    float acc  = 0.0f;
    float sum  = 0.0f;

    // ---- 把数据搬进 shared memory ----
    sData[threadIdx.x] = self;
    __syncthreads();                              // 保证整个 block 的 sData 都写完了

    // ---- 读 constant memory（只读，走常量缓存）----
    float w = cWeight[threadIdx.x & 3];

    // ---- 读邻居：block 内跨线程通信只能靠 shared memory ----
    float nb = sData[(threadIdx.x + 1) % blockDim.x];

    acc = w * self + 0.5f * nb;                   // 全部在寄存器里算
    sDyn[threadIdx.x] = acc;                      // 写动态共享内存
    __syncthreads();                              // 保证 sDyn 全部就绪

    // ---- 反复复用 shared memory 里的数据（这就是 shared memory 存在的意义）----
    for (int k = 0; k < 4; ++k) {
        sum += sDyn[(threadIdx.x + k * 7) % blockDim.x] * cWeight[k];
    }

    if (tid < n) {
        out[tid] = sum;                           // 写 global memory
    }
    atomicAdd(&gScratch[blockIdx.x], sum);        // 写 __device__ 全局变量（需原子）
    if (threadIdx.x == 0) {
        atomicAdd(&gBlockCount, 1);               // 统计实际执行过的 block 数
    }
}
```

```
// ----------------- CPU 参考实现（严格按 GPU 的执行顺序模拟 block 内行为）-----------------
static void memSpaceDemoCPU(const float* in, float* out, const float* w4, int n)
{
    float sData[BLOCK];
    float sDyn[BLOCK];
    int nBlocks = (n + BLOCK - 1) / BLOCK;
    for (int b = 0; b < nBlocks; ++b) {
        for (int t = 0; t < BLOCK; ++t) {
            int tid = b * BLOCK + t;
            sData[t] = (tid < n) ? in[tid] : 0.0f;
        }
        for (int t = 0; t < BLOCK; ++t) {
            int   tid  = b * BLOCK + t;
            float self = (tid < n) ? in[tid] : 0.0f;
            float w    = w4[t & 3];
            float nb   = sData[(t + 1) % BLOCK];
            sDyn[t]    = w * self + 0.5f * nb;
        }
        for (int t = 0; t < BLOCK; ++t) {
            int tid = b * BLOCK + t;
            float sum = 0.0f;
            for (int k = 0; k < 4; ++k) {
                sum += sDyn[(t + k * 7) % BLOCK] * w4[k];
            }
            if (tid < n) out[tid] = sum;
        }
    }
}

int main(void)
{
    const float weights[2][4] = {
        {0.5f, 1.0f, 1.5f, 2.0f},     // 与 __constant__ 声明处的初值一致
        {1.0f, 2.0f, 3.0f, 4.0f}      // 用来演示 cudaMemcpyToSymbol 改写常量内存
    };

    // ---- 1) 打印 kernel 的静态属性：寄存器数与共享内存用量 ----
    cudaFuncAttributes attr;
    CUDA_CHECK(cudaFuncGetAttributes(&attr, memSpaceDemo));
    printf("========== cudaFuncGetAttributes(memSpaceDemo) ==========\n");
    printf("  numRegs            = %d      (每个线程占用的寄存器数，register 空间)\n",
           attr.numRegs);
    printf("  sharedSizeBytes    = %zu B  (静态 __shared__ sData[%d] = %zu B)\n",
           attr.sharedSizeBytes, BLOCK, BLOCK * sizeof(float));
    printf("  localSizeBytes     = %zu B  (local memory 溢出量，0 表示没有溢出)\n",
           attr.localSizeBytes);
    printf("  constSizeBytes     = %zu B\n", attr.constSizeBytes);
    printf("  maxThreadsPerBlock = %d\n", attr.maxThreadsPerBlock);
    printf("  numRegs per block  = %d * %d = %d\n",
           attr.numRegs, BLOCK, attr.numRegs * BLOCK);

    int blocksByRegs = 65536 / (attr.numRegs * BLOCK);
    int blocksBySmem = (164 * 1024) / (int)(attr.sharedSizeBytes
                                            + BLOCK * sizeof(float));
    int blocksByThreads = 2048 / BLOCK;
    int resident = blocksByRegs;
    if (blocksBySmem    < resident) resident = blocksBySmem;
    if (blocksByThreads < resident) resident = blocksByThreads;
    printf("  ---- A100 上的占用率估算 ----\n");
    printf("  blocks_by_regs    = 65536 / (%d*%d) = %d\n",
           attr.numRegs, BLOCK, blocksByRegs);
    printf("  blocks_by_smem    = 164KB / (static %zu B + dynamic %zu B) = %d\n",
           attr.sharedSizeBytes, (size_t)(BLOCK * sizeof(float)), blocksBySmem);
    printf("  blocks_by_threads = 2048 / %d = %d\n", BLOCK, blocksByThreads);
    printf("  resident blocks   = %d  ->  %d threads = %d warps = %.1f%% occupancy\n",
           resident, resident * BLOCK, resident * BLOCK / 32,
           100.0 * resident * BLOCK / 2048.0);

    // ---- 2) 主机端准备数据 ----
    float* h_in  = (float*)malloc(N * sizeof(float));
    float* h_out = (float*)malloc(N * sizeof(float));
    float* h_ref = (float*)malloc(N * sizeof(float));
    float* h_scr = (float*)malloc(N * sizeof(float));
    if (!h_in || !h_out || !h_ref || !h_scr) { fprintf(stderr, "malloc failed\n"); return EXIT_FAILURE; }
    for (int i = 0; i < N; ++i) h_in[i] = (float)((i % 17) * 0.5) - 4.0f;

    float* d_in  = NULL;
```

```
    float* d_out = NULL;
    CUDA_CHECK(cudaMalloc((void**)&d_in,  N * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void**)&d_out, N * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_in, h_in, N * sizeof(float), cudaMemcpyHostToDevice));

    int nBlocks = (N + BLOCK - 1) / BLOCK;
    dim3 grid(nBlocks);
    dim3 block(BLOCK);
    size_t dynSmem = BLOCK * sizeof(float);        // 动态共享内存字节数

    cudaEvent_t t0, t1;
    CUDA_CHECK(cudaEventCreate(&t0));
    CUDA_CHECK(cudaEventCreate(&t1));

    // ---- 3) 两轮运行：第一轮用编译期常量，第二轮用 cudaMemcpyToSymbol 改写 ----
    for (int round = 0; round < 2; ++round) {
        if (round == 1) {
            CUDA_CHECK(cudaMemcpyToSymbol(cWeight, weights[1], sizeof(weights[1])));
            printf("\n[round 1] cudaMemcpyToSymbol(cWeight, {1,2,3,4}) done.\n");
        } else {
            printf("\n[round 0] using compile-time __constant__ cWeight = {0.5,1,1.5,2}\n");
        }

        // 初始化 __device__ 全局变量（host 侧只能通过 symbol 或 kernel 访问）
        float zeros[N];
        memset(zeros, 0, sizeof(zeros));
        CUDA_CHECK(cudaMemcpyToSymbol(gScratch, zeros, nBlocks * sizeof(float)));
        int zeroCount = 0;
        CUDA_CHECK(cudaMemcpyToSymbol(gBlockCount, &zeroCount, sizeof(int)));

        CUDA_CHECK(cudaMemset(d_out, 0, N * sizeof(float)));

        CUDA_CHECK(cudaEventRecord(t0));
        memSpaceDemo<<<grid, block, dynSmem>>>(d_in, d_out, N);
        CUDA_CHECK(cudaEventRecord(t1));
        CUDA_CHECK(cudaEventSynchronize(t1));      // 必须同步，否则计时不准、D2H 也可能读到半成品

        float ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, t0, t1));
        double bytes = 2.0 * N * sizeof(float);    // 读 N*4B + 写 N*4B
        printf("  kernel time = %.4f ms, effective BW = %.2f GB/s\n",
               ms, bytes / (ms * 1.0e-3) / 1.0e9);

        CUDA_CHECK(cudaMemcpy(h_out, d_out, N * sizeof(float), cudaMemcpyDeviceToHost));
        CUDA_CHECK(cudaMemcpyFromSymbol(h_scr, gScratch, nBlocks * sizeof(float)));
        int count = 0;
        CUDA_CHECK(cudaMemcpyFromSymbol(&count, gBlockCount, sizeof(int)));
        printf("  gBlockCount (__device__ global var) = %d (expected %d)\n",
               count, nBlocks);

        double gpuSum = 0.0, scrSum = 0.0;
        for (int i = 0; i < nBlocks; ++i) scrSum += h_scr[i];
        for (int i = 0; i < N; ++i) gpuSum += h_out[i];
        printf("  sum(out) = %.4f ; sum(gScratch) = %.4f (两者应相等)\n",
               gpuSum, scrSum);

        memSpaceDemoCPU(h_in, h_ref, weights[round], N);
        int ok = 1;
        for (int i = 0; i < N; ++i) {
            float diff = h_out[i] - h_ref[i];
            if (diff < 0) diff = -diff;
            if (diff > 1e-3f) { ok = 0; printf("  MISMATCH at %d: gpu=%f cpu=%f\n",
                                               i, h_out[i], h_ref[i]); break; }
        }
        printf("  verification vs CPU  : %s\n", ok ? "[PASS]" : "[FAIL]");
        if (count != nBlocks) printf("  [WARN] gBlockCount != nBlocks\n");
    }

    // ---- 4) 释放 ----
    CUDA_CHECK(cudaEventDestroy(t0));
    CUDA_CHECK(cudaEventDestroy(t1));
    CUDA_CHECK(cudaFree(d_in));
    CUDA_CHECK(cudaFree(d_out));
    free(h_in); free(h_out); free(h_ref); free(h_scr);
    CUDA_CHECK(cudaDeviceReset());
    printf("\nDone.\n");
```

```
    return EXIT_SUCCESS;
}
```

- **【代码做什么？】**
- **声明四种存储空间**：`float acc` 等自动变量 → register；`__shared__ float sData[256]` → 静态共享内存（1 KB/block）；`extern __shared__ float sDyn[]` → 动态共享内存（大小在启动配置 `<<<grid, block, dynSmem>>>` 的第三个参数给出，本例 1 KB/block）；`__constant__ float cWeight[4]` → 常量内存；`__device__ float gScratch[N]` 与 `__device__ int gBlockCount` → global memory。
- **`cudaFuncGetAttributes(&attr, memSpaceDemo)` 在主机端读出 kernel 的静态属性**：`numRegs`（每线程寄存器数）、`sharedSizeBytes`（静态共享内存字节数）、`localSizeBytes`（local memory 溢出量，0 表示没有寄存器溢出）、`maxThreadsPerBlock`。这是 Lab 0/Lab 2 里判断"占用率被什么限制"的第一手数据。
- **两轮运行演示常量内存的两种初始化方式**：第 0 轮直接用声明处的编译期初值 `{0.5, 1.0, 1.5, 2.0}`；第 1 轮用 `cudaMemcpyToSymbol(cWeight, weights[1], sizeof(weights[1]))` 在运行时改写。注意 `__constant__` 变量的地址在主机端**不是一个普通指针**，必须用 `cudaMemcpyToSymbol` / `cudaMemcpyFromSymbol`（`cudaMemcpy` 传 `cWeight` 会尝试解引用一个设备地址而报错）。
- **`__device__` 全局变量也必须用 symbol API 初始化和读回**：`cudaMemcpyToSymbol(gScratch, zeros, nBlocks*sizeof(float))` 清零、`cudaMemcpyFromSymbol(h_scr, gScratch, ...)` 读回，最后用 `cudaMemcpyFromSymbol(&count, gBlockCount, sizeof(int))` 验证"确实有 nBlocks = 16 个 block 执行了"。
- **kernel 内部的 shared memory 生命线**：`sData[threadIdx.x] = self;` 写入 → `__syncthreads()` → 读邻居 `sData[(threadIdx.x+1) % 256]`。**如果没有这个 `__syncthreads()`，warp 7 可能读到 warp 0 尚未写入的 sData**，结果是随机错误（且往往在小数据量下"看起来是对的"，非常难查）。
- **本 kernel 刻意做成"共享内存有收益"的形状**：每个 block 的 256 个元素被读进 sData 后被**读 2 次**（自己一次 + 邻居一次），再经 sDyn 被**读 4 次**（循环累加）。若不用共享内存，这 6 次访问全部要打到 L1/global，流量放大 6 倍。
- **同步与计时**：`cudaEventSynchronize(t1)` 保证计时覆盖完整执行；`cudaDeviceSynchronize` 在这里没显式出现，但 `cudaMemcpy(D2H)` 本身是同步操作，等效地起到了屏障作用——**这正是"忘记同步"最隐蔽的形式：有时因为后面的 memcpy 掩盖了错误**。
- **主机端逐元素比对**：CPU 参考实现严格按"先整块写 sData → 再算 sDyn → 再累加"的顺序模拟，因此可以做到 1e-3 容差内的完全一致。
- **【并行机制与硬件映射解说】**
- **block/warp 划分**：`block = 256` 线程 = **8 个 warp**；`grid = 16` 个 block。A100 有 108 个 SM，16 个 block 只够铺满 16 个 SM（约 15%），**这是一次"并行度严重不足"的配置**——它的目的是教学演示，不是性能。要跑满 A100（108 SM × 8 block = 864 个驻留槽位，建议 2 倍即 1728 个 block），N 至少要 `1728 × 256 = 442,368` 个元素。
- **占用率（以 A100 为例）**：假设 `numRegs = 32`：`text blocks_by_regs = 65536 / (256 * 32) = 65536 / 8192 = 8 blocks_by_smem = 164 KB / (1 KB static + 1 KB dynamic) = 164/2 = 82 blocks_by_threads = 2048 / 256 = 8 blocks_by_hw = 32 resident = min(8, 82, 8, 32) = 8 → 8*256 = 2048 线程 = 64 warp = 100% occupancy 若编译器把寄存器用到 40 个： blocks_by_regs = 65536 / (256*40) = 65536 / 10240 = 6（向下取整） resident = min(6, 82, 8, 32) = 6 → 6*256 = 1536 线程 = 48 warp = 75% occupancy 寄存器分配粒度是 256 个寄存器/warp：warp 需要的寄存器数会向上取整到 256 的倍数， 所以 8 warp × 32 线程 × 40 regs = 10240 已经对齐，直接按 10240 计。` 这就是**"寄存器压力 → 占用率掉档 → 延迟隐藏能力下降"**的完整因果链。可用 `__launch_bounds__(256, 8)` 告诉编译器"我要 8 个 block/SM"，它会主动把寄存器压到 32 个以内（代价是可能产生 local memory 溢出，此时 `localSizeBytes > 0` 就是警号）。
- **共享内存 bank 与 bank conflict（逐条分析 bank 编号）**：A100 的共享内存有 **32 个 bank，每个 bank 宽 4 字节，地址 a 落在 bank `(a/4) % 32`**。`text ① sData[threadIdx.x] = self; // 写 warp 内线程 t 访问 float 下标 t → 字节地址 4t → bank = (4t/4)%32 = t%32 → 32 个线程命中 32 个不同 bank → 无冲突（conflict-free），1 个周期完成 ② sData[(threadIdx.x+1) % 256] // 读邻居 地址 = 4*(t+1) → bank = (t+1)%32 → 仍然是 32 个不同 bank（只是整体旋转了 1） → 无冲突 ③ sDyn[(threadIdx.x + k*7) % 256] // k 是编译期常数，循环内被展开 对固定的 k，地址 = 4*(t+7k) → bank = (t+7k)%32 → 32 个不同 bank → 无冲突 ④ 反例：如果写成 sData[threadIdx.x * 2] 地址 = 8t → bank = (8t/4)%32 = (2t)%32 t=0..15 → bank 0,2,4,...,30；t=16..31 → bank 0,2,4,...,30（重复） → 每个 bank 被 2 个线程访问 → 2-way bank conflict，耗时从 1 周期变 2 周期 再如 sData[threadIdx.x * 32] → bank = (32t/4)%32 = (8t)%32 → 4 个 bank 被 8 线程 轮流访问 → 8-way conflict，耗时 8 倍 ★ 这类"跨步访问"是共享内存的头号陷阱` 现代 GPU 对**同一 32 位字内的多线程访问**会做广播（broadcast），所以"多个线程读同一个地址"不算冲突，只有"落到同一 bank 的不同地址"才算。
- **全局内存合并访问分析**：`in[tid]` 与 `out[tid]` 中 tid 连续 → warp 读写 32×4 = **128 字节连续**，正好 1 条 cache line（4 个 32 B sector，100% 利用），完美合并。`gScratch[blockIdx.x]` 被同一 block 的 256 个线程原子累加，**同一个地址被 32 个 lane 同时原子操作**，硬件会串行化这 32 次原子（约 32 个周期/次），本例每 block 256 次 → 这是演示代码里唯一的性能瑕疵，换成"先做 block 内 reduction 再由线程 0 原子加一次"可以降低 32 倍原子压力。
- **warp 发散**：kernel 中唯一的分支是 `if (tid < n)` 与 `if (threadIdx.x == 0)`。前者在 N=4096 且 BLOCK=256 时**恰好整除，全 false**，零发散；后者只有一个 lane 为真，但它只包住一条指令，属于可接受的谓词化开销（predicated execution），代价约 1 个发射槽。
- **`__syncthreads()` 的代价**：本例有 2 个屏障，每个屏障让 8 个 warp 相互等待。因为所有 warp 的工作量完全对称（同样的访存模式、同样的指令数），**负载均衡良好，屏障开销被压到最小**——这也从反面说明：**屏障的代价取决于负载均衡，而不是屏障本身**。
- **【性能优化分析】**
- **算术强度**：每个元素读 4 B（`in[tid]`）+ 写 4 B（`out[tid]`）= 8 B；运算量约 12 FLOP（1 次乘加算 2、共享内存复用的 4 次乘加算 8、邻居加权算 2）：`text AI = 12 FLOP / 8 B = 1.5 FLOP/Byte` 注意这里**没有把 shared memory 流量算进 DRAM 流量**——这正是 shared memory 的意义：把 6 次逻辑访问压缩成 1 次 DRAM 访问 + 5 次片上访问。

- **Roofline**：`text 上限 = min(19.5e12 , 1.5 × 1555e9) = min(19500 GFLOP/s , 2332 GFLOP/s) = 2332 GFLOP/s 仍然远低于 19.5 TFLOP/s → 依旧带宽受限，但带宽利用率的要求比程序 2 宽松（AI 高 3 倍） 理想时间 = (4096 × 8 B) / 1555e9 B/s = 32.8 KB / 1555 GB/s = 21 ns 实测时间会被 kernel 启动开销(3–10 μs)完全支配 → 加速比看上去只有 0.001x， 这不是 kernel 写得差，而是"数据量太小，不适合 GPU"。`
- **占用率**：见上，8 个 block/SM = 100%（32 寄存器）或 6 个 block/SM = 75%（40 寄存器）。要判断真实瓶颈，应在主机端打印 `attr.numRegs`、`attr.localSizeBytes`，再用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor(&numBlocks, memSpaceDemo, BLOCK, dynSmem)` 让 runtime 直接告诉你答案。
- **瓶颈判定与优化方向：延迟受限 + 并行度不足**（16 个 block 只填 16/108 个 SM；单次执行 21 ns 而启动开销 5 μs）。三条可执行方向：① **增大 grid**——把 N 提到 100 万以上，让 block 数达到 4000+；② **粗粒度化**——让每个线程处理 4~8 个元素，减少总 block 数反而能提高每线程 ILP 并减少索引开销（见"线程映射粒度"概念）；③ **批处理**——把多次小 kernel 合并成一个 stream 上的连续启动，摊薄启动开销。

性能优化技巧总结

1. **网格维度必须向上取整**：`grid.x = (N + block.x − 1) / block.x`，因为 C 的整数除法是向下取整，`N/256` 会漏掉尾部元素，而**漏算的元素不会报错，只会算错**——这是最危险的 bug 类型。
2. **每个 kernel 都要有边界检查**：`if (row < height && col < width)` 挡住"向上取整带来的多余线程"；没有它，越界线程会写坏相邻缓冲区，且症状随机。
3. **block 大小选 256（16×16）作为默认起点**：256 线程 = 8 个 warp，在 A100/RTX 4090 上都能整除达到 100% 占用率，又不会被单个 block 的 1024 线程上限或"每 SM 最多 8 个 block"这类硬件限制卡住。
4. **让 warp 的访存连续且对齐**：`threadIdx.x` 必须映射到"内存中变化最快的维度"（行主序中的列），这样 32 个线程 × 4 字节 = 128 字节 = 1 条 cache line，一次 transaction 拿满。
5. **能向量化就向量化**：`uchar4` / `float4` / `int4` 一条指令搬 4 个元素，把 warp 的有效载荷从 128 字节里的 32 字节提升到 128 字节，通常带来 1.5×~3× 的带宽提升。
6. **分支粒度对齐 warp size**：把 `if (threadIdx.x > 2)` 改成 `if (threadIdx.x / warpSize > 2)`，让每个 warp 内部走同一条路径，避免两条路径串行执行。
7. **共享内存访问避免跨步**：`sData[threadIdx.x]` 是无冲突的，`sData[threadIdx.x * 2]` 是 2-way 冲突，`sData[threadIdx.x * 32]` 是 8-way 冲突；必要时用 padding（`sData[BLOCK + 1]`）打破跨步。
8. **用 `__syncthreads()` 建立"生产者–消费者"边界，并且只在需要时用**：写入共享内存之后、读取之前必须有一个屏障；但屏障也会让快 warp 等慢 warp，所以要放在负载均衡的位置，绝不放进条件分支。
9. **先算占用率再谈优化**：`resident = min(线程约束, 寄存器约束, smem 约束, block 数上限)`，任何一项都可能成为瓶颈；`attr.numRegs` 和 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 是最快的诊断工具。
10. **小数据不要上 GPU**：kernel 启动固定开销 3–10 μs，PCIe 传输约 25 GB/s（比显存慢 60 倍），当总计算量 < 10 万次操作时，GPU 版本通常比 CPU 还慢。

关键要点

- **SPMD 是 CUDA 的全部哲学**：一个 kernel 一份代码，靠 `blockIdx` / `threadIdx` 决定"我是谁、我处理哪份数据"；把 CPU 的 `for` 循环变量替换成线程索引，就是数据并行的标准改写手法。
- **线程层次有三层且语义不同**：grid 只是"启动配置"，block 是"可同步、可共享内存的合作单元"，warp 是"硬件调度单元"。**不同 block 之间不能同步、执行顺序任意**，跨 block 合作只能靠 kernel 边界或原子操作。
- **warp（32 线程）是性能分析的最小单位**：合并访问按 warp 的 128 字节 transaction 判定，分支发散按 warp 内是否走不同路径判定，占用率按 warp 数计算。看代码时必须以 warp 为单位心算，而不是逐线程。
- **内存层次的延迟跨越三个数量级**：寄存器 1 周期、共享内存 20–30 周期、L1 约 30 周期、L2 约 200 周期、全局内存 400–800 周期、主机内存 10000+ 周期。优化的本质就是**把访问尽量往上层挪**。
- **`__syncthreads()` 是 block 级屏障，`cudaDeviceSynchronize()` 是 host–device 同步**；前者保证共享内存的可见性，后者保证主机能安全读取设备结果。kernel 启动是异步的，**没有同步的计时和读回都是错的**。
- **共享内存"有收益"的前提是数据复用**：程序 2 的每个像素只读一次，用共享内存只会增加开销；程序 3 的每个元素被读 6 次，共享内存才有价值。盲目使用共享内存是新手最常见的"优化反而变慢"。

常见陷阱与注意事项

- **整数除法取整导致网格覆盖不足**：写 `dim3 grid(N/256)` 时 `N=1000` 得到 3 个 block，元素 768–999 无人处理，结果数组尾部全是未初始化的垃圾 → 正确做法是 `(N + 255)/256` 或 `ceil(N/256.0)`，并且**永远保留边界检查**。
- **越界访问（out-of-bounds）**：2D kernel 里忘记 `if (row < height && col < width)`，最右一列/最下一行 block 的多余线程会写到别的缓冲区，症状是"结果大部分正确、少数像素花屏"，且在 `compute-sanitizer` 关闭时毫无提示 → 正确做法是所有访问全局内存的语句都包在边界检查里。
- **忘记 `cudaDeviceSynchronize()` / 同步点放错位置**：kernel 是异步启动的，紧跟其后的 `cudaMemcpy(D2H)` 虽然本身同步，但如果用 `cudaMemcpyAsync` 或直接读 pinned 内存，就会读到半成品；计时时若不在 `cudaEventRecord(t1)` 后加 `cudaEventSynchronize(t1)`，测到的是启动开销 → 正确做法是计时必同步、读回前必同步。

- **忘记 `__syncthreads()`**：多个 warp 通过共享内存交换数据时，写完之后必须有一个屏障，否则快 warp 会读到慢 warp 还没写的旧值；典型表现是"数据量小时结果正确、数据量大时偶发错误" → 正确做法是在"写共享内存"与"读共享内存"之间放一个 `__syncthreads()`。
- **把 `__syncthreads()` 放进分支里**：`if (threadIdx.x < 16) { ...; __syncthreads(); }` 会让部分 warp 永远等不到同伴，行为未定义（可能挂死）→ 正确做法是让屏障处于**所有线程都必然执行**的代码路径上。
- **共享内存 bank conflict**：让线程按 `sData[threadIdx.x * 32]` 这样的跨步访问，32 个线程会撞到 4 个 bank，形成 8-way conflict，shared 访问耗时直接乘以 8 → 正确做法是保证 warp 内连续线程访问连续的字（`sData[threadIdx.x]`），或加 padding（`__shared__ float s[BLOCK + 1]`）。
- **warp 发散（branch divergence）**：把 `threadIdx.x` 当分支条件（如 `if (threadIdx.x > 2)`）会让同一 warp 的两条路径串行执行，若两条路径各 100 条指令，warp 就要执行 200 条 → 正确做法是让分支粒度对齐 `warpSize`（`if (threadIdx.x / 32 > 2)`），或者用查表/数学变换消除分支。注意：**若分支条件依赖的是所有线程都相同的量（如 kernel 参数 `q`），则不会发散**。
- **不检查 CUDA API 返回值**：`cudaMalloc` 失败返回 `cudaErrorMemoryAllocation`、kernel 越界启动返回 `cudaErrorInvalidConfiguration`，如果全部忽略，错误会在几百行之后以"结果不对"的形式出现，极难定位 → 正确做法是全程使用 `CUDA_CHECK` 宏，并且在 kernel 启动后补一次 `CUDA_CHECK(cudaGetLastError())`（启动错误是异步上报的）。
- **`cudaMemcpy` 方向写错**：把 `cudaMemcpyHostToDevice` 和 `cudaMemcpyDeviceToHost` 弄反，行为是"立即报 invalid argument"或"拷回一堆垃圾"；从设备指针拷到设备指针必须用 `cudaMemcpyDeviceToDevice` → 正确做法是记住参数顺序 `(dst, src, bytes, kind)`，并且**从谁的角度看 dst/src**：主机变量在前、设备指针在后是 H2D。
- **host/device 指针混用**：把 `cudaMalloc` 得到的设备指针直接传给 CPU 代码解引用，会触发段错误或读到无关数据；反之把主机指针传给 kernel 会得到非法地址访问 → 正确做法是用命名规范（`h_` 前缀 / `d_` 前缀）区分，并在 `cudaMalloc` 之后立刻 `cudaMemset` 清零以便早期发现。
- **共享内存容量超限**：A100 每 block 静态共享内存上限默认 48 KB，超过会启动失败（`cudaErrorInvalidValue`）；要用到 48 KB 以上必须调用 `cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize, bytes)` 显式申请（A100 上限 164 KB/SM） → 正确做法是先查 `sharedMemPerBlockOptin`，再 opt-in。

思考题（带答案）

Q1. 用 `dim3 block(16,16)` 覆盖一张 76×62 的灰度图，需要多少个 block？总共启动多少线程？其中有多少线程什么也不做？如果把 block 换成 8×8 呢？
需要 `ceil(76/16) = 5` 列 × `ceil(62/16) = 4` 行 = **20 个 block**，共启动 `20 × 256 = 5120` 个线程。图像实际只有 `76 × 62 = 4712` 个像素，因此 **5120 − 4712 = 408 个线程**被 `if (row < height && col < width)` 挡住（浪费 7.97%）。若换成 8×8：`ceil(76/8) = 10`、`ceil(62/8) = 8`，共 80 个 block、`80 × 64 = 5120` 个线程，仍然是 408 个空转线程，**浪费比例完全相同（因为两种 block 的整除余数一样）**；但 8×8 的 block 只有 2 个 warp，受"每 SM 最多 block 数"限制时更难达到高占用率，所以 16×16 是更好的选择。

Q2. 一个 kernel 每个线程用 32 个寄存器，block 为 256 线程，不使用共享内存。在 A100（每 SM 65536 寄存器、2048 线程、64 warp、最多 32 个 block）上，占用率是多少？如果把寄存器用到 40 个呢？ 32 寄存器时：每 block 寄存器 `256 × 32 = 8192`，`65536 / 8192 = 8` 个 block；线程上限给 `2048/256 = 8` 个 block；取 min 得 8 个 block = `8 × 256 = 2048` 线程 = `64` 个 warp = **100% 占用率**。40 寄存器时：`256 × 40 = 10240`，`65536 / 10240 = 6.4 → 6` 个 block = `1536` 线程 = `48` 个 warp = **75% 占用率**。可见**寄存器每多 8 个，占用率就掉 25%**；若占用率下降导致延迟隐藏不足（例如这个 kernel 频繁访问全局内存），可以用 `__launch_bounds__(256, 8)` 强制编译器把寄存器压回 32 个以内。

Q3. 讲义中的向量加法用"一个线程一个元素"，现在改成"一个线程两个元素"：`vecAdd<<<ceil(N/(2*256.0)), 256>>>`，kernel 里写 `i = blockIdx.x*(2*blockDim.x) + threadIdx.x; C[i]=A[i]+B[i]; i = i + blockDim.x; C[i]=A[i]+B[i];`。当 N = 1000 时，网格是多少？这样做的好处和风险是什么？为什么第二次访问仍然是合并的？ `ceil(1000/512) = 2` 个 block，共 `2 × 256 = 512` 个线程（细粒度版需要 4 个 block、1024 个线程）。好处：**总线程数减半**，每个线程承担 2 次乘加，索引计算与边界判断的固定开销被摊薄，同时每个线程有两条独立的 load 可以重叠，提高了内存级并行（MLP）。风险：线程总数太少会**降低并行度**，无法填满 SM 的 warp 槽位（A100 需要约 44 万线程才能跑满）；并且必须**两次都做边界检查**，否则 N 为奇数时第二次访问会越界。第二次访问仍然合并：warp 内 32 个线程的 `i` 是连续递增的（因为是 `i + blockDim.x`，对同一 warp 内所有 lane 加的常数相同），地址 32×4 = 128 字节连续，正好 1 条 cache line，只是整体偏移了 `blockDim.x × 4 = 1024` 字节。

Lecture 3: 并行模式一 —— 向量加法：线程映射与内存访问（对应 Lab 1: Vector Addition）

概述

本讲用一个"每元素只做一次加法"的向量加法 kernel（`C[i] = A[i] + B[i]`），把 CUDA 的三件事一次性讲透：**线程如何映射到数据**（`i = blockIdx.x * blockDim.x + threadIdx.x`）、**数组元素如何落到内存事务上**（合并访存 coalescing）、以及**硬件如何用大量并发 warp 隐藏几百个时钟周期的全局内存延迟**（占用率 occupancy 与延迟容忍 latency tolerance）。结论先行：向量加法在 A100 上只有 0.125 FLOP/Byte 的算术强度，远低于 12.54 FLOP/Byte 的机器平衡点，是一个**纯粹的显存带宽受限（memory bandwidth bound）**问题——你唯一能做的就是让每一个字节都被用满，然后让足够多的访存同时在路上。这也是 ECE408 第一个 Lab（Lab 1）的全部主题：正确的线程映射 + 正确的边界检查 + 正确的带宽测量。

核心概念与 GPU 架构图解

概念 1：数据并行与线程到数据的映射（Data Parallelism and Thread-to-Data Mapping）

- **定义与目的**：数据并行（data parallelism）指的是"同一段程序作用在一大批互不相关的数据上"，每个数据元素的计算完全不依赖其它元素的计算结果。向量加法就是最纯粹的数据并行：`C[0]` 的值与 `C[1]` 的值毫无关系。CUDA 的做法是把这个"for 循环"拆开：**让硬件去决定循环的哪一次迭代在哪个物理单元上跑**，程序员只负责给出"元素编号 → 线程编号"的映射公式。这个映射公式就是 `i = blockIdx.x * blockDim.x + threadIdx.x`。它解决的核心问题是：GPU 上同时有几十万个线程，必须有一种廉价（几条整数指令）、唯一、且与内存布局对齐的方式给每个线程分配一份不重复的工作。

- **直观解释（"它是什么？"）**：想象一场有 10,000,000 道题的社会化阅卷，阅卷老师有 39,063 个小组，每组 256 人。要给每位老师发一个唯一的工号，最快的办法不是让校长挨个点名，而是"**组号 × 每组人数 + 组内座号**"：第 7 组的第 3 号老师，工号就是 `7 * 256 + 3 = 1795`。CUDA 把这件事固化成了硬件：`blockIdx.x` 是组号（硬件寄存器直接给出，不需要计算），`blockDim.x` 是每组人数（一个编译期/运行期常量），`threadIdx.x` 是组内座号（硬件给出）。三个值一读、一乘、一加，工号就有了。接下来只需要规定"工号 = 题目号"，映射就完成了。
- **架构/机制图解**：

```
CUDA 的三层逻辑结构（以 N = 10,000,000、block 大小 256 为例）

Grid（整个问题）
┌──────────────────────────────────────────────────────────────────┐
│  dimGrid = (39063, 1, 1)  ->  共 39063 个 thread block           │
│  ┌──────────┬──────────┬──────────┬──────────┬──────────────┐    │
│  │  blk 0   │  blk 1   │  blk 2   │  blk 3   │  blk 39062   │    │
│  └──────────┴──────────┴──────────┴──────────┴──────────────┘    │
└──────────────────────────────────────────────────────────────────┘

                每个 block: dimBlock = (256, 1, 1)
┌──────────────────────────────────────────────────────────────────┐
│  threadIdx.x:  0    1    2    3    4    5    6    7              │
│                8    9   10   11   12   13   14   15              │
│              128  129  130  131  132  133  134  135              │
│              248  249  250  251  252  253  254  255              │
│               （完整取值是 0 到 255 的每一个整数）                 │
│                                                                  │
│  warp 划分  : warp0 = t0 至 t31   | warp1 = t32 至 t63           │
│               warp2 = t64 至 t95  | warp3 = t96 至 t127          │
│               warp4 = t128 至 t159 | warp5 = t160 至 t191        │
│               warp6 = t192 至 t223 | warp7 = t224 至 t255        │
│               共 8 个 warp，每个 warp 32 个线程                  │
└──────────────────────────────────────────────────────────────────┘

线性化编号公式（"组号 × 每组人数 + 组内座号"）：

     i = blockIdx.x * blockDim.x + threadIdx.x
         └─────┬─────┘   └──┬──┘   └─────┬─────┘
           组号 b        每组人数 B     组内座号 t
         (0..39062)       (=256)        (0..255)

block 0 : t=0 -> i=0      t=1 -> i=1      t=255 -> i=255
block 1 : t=0 -> i=256    t=1 -> i=257    t=255 -> i=511
block 2 : t=0 -> i=512    t=1 -> i=513    t=255 -> i=767
block k : t=0 -> i=256k   t=j -> i=256k+j
```

```
为什么必须做边界检查：N = 10,000,000 不是 256 的整数倍

ceil(N / 256) = ceil(39062.5) = 39063 个 block
实际启动线程数 = 39063 * 256 = 10,000,128 个
真正需要的线程数 = 10,000,000 个
多余的线程数    =        128 个（最后 4 个完整 warp 无事可做）

元素索引轴：
0                    9,999,744   9,999,872  10,000,000  10,000,128
|--------------------------|-----------|-----------|-----------|
    全部有效（10,000,000 个元素）        ↑           ↑
                                  最后一个满 warp   越界区！
                                  (32 个元素全有效)  若不加 if(i<N)
                                                    就会写坏别人的内存
block 39062 覆盖 i = 9,999,872 .. 10,000,127
  ├─ threadIdx.x 0 .. 127  -> i = 9,999,872 .. 9,999,999   (有效)
  └─ threadIdx.x 128 .. 255 -> i = 10,000,000 .. 10,000,127 (越界！)
```

关键操作与性能特征： 1. **网格大小必须向上取整**：`dimGrid.x = (N + blockDim.x - 1) / blockDim.x`。整数除法 `N / blockDim.x` 是**向下取整**，当 `N % 256 != 0` 时会漏掉最后 `N % 256` 个元素（本讲 N=10,000,000 时会漏掉 128 个，正确性测试直接 FAIL）。 2. **上取整的代价是多余的线程**：39063 个 block 里有 128 个线程完全空转，占比 `128 / 10,000,128 = 0.0013%`，可忽略。但同样的代码在 N = 1025、block = 1024 时，启动 2 个 block 共 2048 个线程，只有 1025 个有效，**浪费 49.9%**。小问题用大 block 是浪费的重灾区。 3. **越界写是最危险的 bug**：CUDA 的 `cudaMalloc` 返回的是一整块设备内存，越界写会静默破坏相邻分配（可能是别的数组、可能是 CUDA 内部结构），常常表现为"跑一次对、跑十次错"或"验证时正确、之后崩"。`if (i < N)` 不是可选项，是必需品。 4. **上取整请用整数运算**：`(N + B - 1) / B` 对非负整数永远正确；`ceil(N / 256.0)` 也能用，但它引入浮点、且当 N 超过 2^53 时会失准。Lab 中强烈建议用整数形式，同时注意 `N + B - 1` 在 N 接近 `INT_MAX` 时会溢出，安全写法是 `(int)(((long long)N + B - 1) / B)`。

概念 2：合并访存（Coalesced Memory Access）

- **定义与目的**：合并访存指的是"**同一个 warp 内的 32 个线程，访问同一段连续且对齐的内存区域**"，使得这 32 次各自 4 字节的请求能被硬件合并成**一次 128 字节的内存事务（memory transaction）**。它解决的问题是：显存（DRAM）的物理特性决定了它的最小开销单位是"连续的一整块"，而不是"任意一个

字节"。如果你的访问模式是散的，硬件仍然要搬回 128 字节（或至少 32 字节的 sector），但其中只有很小一部分被用上，剩下的带宽全部浪费掉。

- **直观解释（"它是什么？"）**：把显存想成**一列固定长度（128 字节）的货运车厢**。合并访存就是"32 个人凑钱买一节车厢，每个人的包裹都放进这节车厢的固定格子里"——付一次运费，所有人的货都运到了。非合并访存（strided）则是"32 个人各自包了一整节车厢，却每人只放了一个 4 字节的小盒子"——运费付了 32 份，实际只用了 1 份的货量。货运公司（DRAM 控制器）不会因为你只用了一个格子就少收钱，因为一趟车的开销（激活行、突发传输 burst）是固定的。课程讲义（Lecture 7 DRAM）里那个 512 字节 burst、240 GB/s 峰值的例子说得很清楚：如果只用到 burst 里每隔 8 字节的一小部分，可得带宽立刻减半到 120 GB/s。
- **架构/机制图解**：

```
合并访存（coalesced）：一个 warp 的 32 个线程访问 32 个连续的 float
（float 为 4 字节，cudaMalloc 返回的基地址按 256 字节对齐）

线程号:   t0  t1  t2  t3  t4  t5  t6  t7  t8  t9 t10 t11 t12 t13 t14 t15
元素号:    0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
字节偏移:  0   4   8  12  16  20  24  28  32  36  40  44  48  52  56  60
线程号:  t16 t17 t18 t19 t20 t21 t22 t23 t24 t25 t26 t27 t28 t29 t30 t31
元素号:   16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
字节偏移: 64  68  72  76  80  84  88  92  96 100 104 108 112 116 120 124

落入的 128 字节地址窗口（一次 memory transaction = 一条 128 B cache line）
字节 0                                                                127
     ┌──────────────────────────────────────────────────────────────┐
     │  32 个 float 全部被使用（128 字节，利用率 100%）              │
     └──────────────────────────────────────────────────────────────┘
     |<------------------ 1 次 transaction (128 B) ------------------>|
     └ sector 0 (32B) ─┴─ sector 1 (32B) ─┴─ sector 2 ─┴─ sector 3 ─┘

结果：一个 warp 读 A 只用 1 次事务；读 B 1 次；写 C 1 次
      -> 每个 warp 3 次 128 字节事务，搬运 384 字节，全部是有效数据
```

```
跨步访存（strided）：线程号相差 1，地址相差 S = 156,252 字节
（对应代码 i = threadIdx.x * gridDim.x + blockIdx.x，即为"按列遍历"）

线程号:   t0      t1      t2      t3    t4      t5     t30      t31
地址:     b     b+S     b+2S    b+3S  b+4S    b+5S   b+30S    b+31S
          |       |       |       |     |       |      |        |
          ▼       ▼       ▼       ▼     ▼       ▼      ▼        ▼
       ┌─────┐ ┌─────┐ ┌─────┐ ┌─────┐┌─────┐┌─────┐ ┌─────┐  ┌─────┐
       |128 B| |128 B| |128 B| |128 B||128 B||128 B| |128 B|  |128 B|
       |line | |line | |line | |line ||line ||line | |line |  |line |
       └─────┘ └─────┘ └─────┘ └─────┘└─────┘└─────┘ └─────┘  └─────┘
          每个 128 字节的 line 里只用了 4 个字节（利用率 3.125%）
           （32 个线程各占一个 line；上图列出 t0 至 t5 与 t30、t31）

事务数：32 次 transaction（line 粒度）= 32 x 128 B = 4096 B 只用到 128 B
现代 GPU 按 32 字节 sector 粒度取数，则 32 个 sector x 32 B = 1024 B
        只用到 128 B，放大倍数 = 1024 / 128 = 8 倍
```

```
事务数 / 有效字节数 对照表（一个 warp，32 个线程，各取 1 个 float）

访问模式                          事务数      搬运字节     有效字节    利用率
──────────────────────────────────────────────────────────────────────
连续、128 B 对齐（合并）            1          128          128      100.0%
连续、但只跨 128 B 边界一点         2          256          128       50.0%
步长 2 个 float（8 字节）           2          256          128       50.0%
步长 8 个 float（32 字节）          8         1024          128       12.5%
步长 32 个 float（128 字节）       32         4096          128        3.1%
──────────────────────────────────────────────────────────────────────
注：sector 粒度的硬件（Kepler 及以后）会把上表后三行的搬运字节分别
    降到 256 / 256 / 1024 字节，但"放大 2 倍 / 8 倍"的结论不变。
```

关键操作与性能特征： 1. **128 字节是一次事务的粒度**（对应一条 cache line），正好等于 `32 线程 × 4 字节`。这就是为什么 **warp 大小 32 与 float 的宽度 4 字节如此契合**：一个 warp 读一组连续 float，天然就是一整条 line。 2. **对齐（alignment）和连续（contiguity）同样重要**。如果数组起始地址不是 128 字节对齐，一个 warp 的 32 个 float 会跨两条 line，事务数从 1 变成 2，带宽利用率立刻掉到 50%。`cudaMalloc` 返回的指针至少 256 字节对齐，所以只要**线性化编号公式里 threadIdx.x 是最内层（变化最快）的那一维**，合并就是自动的——这正是讲义里那道思考题（`B[ty*Width+tx]` 优于 `B[tx*Width+ty]`）的答案。 3. **GPU 没有"每个线程各自缓存"的写缓冲**，写操作同样按 warp 合并。写 C 时连续线程写连续地址 → 1 次事务；如果跨步写，写合并失败，代价同样是数倍放大的写带宽。 4. **L1/L2 会掩盖一部分不合并访问**：L2 line 大小 128 字节、L1 sector 32 字节。若同一 warp 的另一次相邻加载能命中已被拉进 L1/L2 的 sector，放大倍数会降低。但**不要指望缓存救场**——向量加法每个字节只被读一次，没有任何时间局部性（temporal locality），缓存在这里唯一的作用就是 coalescing buffer。

概念 3：全局内存延迟与带宽（Global Memory Latency and Bandwidth）

- **定义与目的**：全局内存（global memory，即设备 DRAM）是 GPU 上容量最大（数十 GB）、速度最慢的一级存储。它有两个正交的性能指标：**延迟（latency）**——一次访问从发出到数据可用需要多久，约 400–800 个时钟周期；**带宽（bandwidth）**——单位时间内能搬运多少字节，A100 约 1555 GB/s（峰值）。区分这两者极其重要：延迟决定了"你需要多少并发才能不挨饿"，带宽决定了"每秒最多能处理多少数据"。向量加法是典型的**带宽上限问题**：它需要搬运的字节数远超它需要的计算量，因此性能天花板就是显存带宽。
- **直观解释（"它是什么？"）**：把显存想成**城郊的大仓库**，SM 想成**市中心的工厂**。从下单到货到（延迟）要开 500 个路口的车程（~400–800 cycles），但高速路的**车道数**（带宽）很宽。工厂的活是"把 A 箱和 B 箱的东西倒在一起"（一次加法），工序本身快得可以忽略。所以决定产量的是高速路每秒能过多少车，而不是工厂的手速。工厂的对策不是"催司机开快点"（延迟改不了，它是物理距离），而是**一次发出几百辆卡车**（用大量并发 warp 把延迟"藏起来"），让高速路时刻是满的。
- **架构/机制图解**：

```
现代 NVIDIA GPU 的存储层次（数字为 A100 / sm_80 量级）

访问延迟 ↑                                                  容量 ↑
┌──────────────────────────────────────────────────────────────┐
│ 寄存器 Register File   每 SM 65536 个 32 位寄存器（256 KB）          │
│   延迟 ~1 cycle        带宽：每 SM 每 cycle 数百字节                 │
├──────────────────────────────────────────────────────────────┤
│ 共享内存 Shared Memory / L1   每 SM 最多 164 KB（可配置划分）        │
│   共享内存延迟 ~20–30 cycles，L1 命中 ~30 cycles                   │
├──────────────────────────────────────────────────────────────┤
│ L2 Cache   全芯片共享（A100 约 40 MB，按 2 个分区各 20 MB）          │
│   延迟 ~200 cycles                                            │
├──────────────────────────────────────────────────────────────┤
│ 全局内存 Global Memory（HBM2）  40 GB / 80 GB                  │
│   延迟 ~400–800 cycles（1.41 GHz 下约 284–567 ns）              │
│   带宽 A100 约 1555 GB/s 峰值，实测流式 kernel 约 1200–1400 GB/s   │
└──────────────────────────────────────────────────────────────┘

换算：1555 GB/s 的显存带宽，配上 500 cycles / 1.41 GHz = 355 ns 的延迟，
     要让带宽打满，全芯片必须同时有（Little's Law）：
       在途字节 = 带宽 x 延迟 = 1250e9 B/s x 355e-9 s ≈ 443,750 B ≈ 434 KB
     分摊到 108 个 SM：每 SM 需要约 4.1 KB（约 32 条 128 B cache line）在途
```

```
向量加法的时间分解（N = 10,000,000，A100）

计算侧（几乎不花时间）：
  总 FLOP 数         = 10,000,000 次加法 = 1.0e7 FLOP
  FP32 峰值          = 19.5 TFLOPS
  纯计算下限         = 1.0e7 / 19.5e12 = 0.51 us

存储侧（决定一切）：
  读 A = 40 MB，读 B = 40 MB，写 C = 40 MB，合计 120 MB
  峰值带宽下限       = 120e6 B / 1555e9 B/s = 77.2 us
  实测（1.25 TB/s）  = 120e6 B / 1250e9 B/s = 96.0 us

  96.0 us / 0.51 us ≈ 188 倍  ->  计算单元 99.5% 的时间在空转
```

关键操作与性能特征： 1. **DRAM 的物理约束**：DRAM 一位只用一个晶体管加一个电容，密度极高但很慢；每个 bit line 上挂约 1000 个单元，靠 sense amplifier 放大才能读出。现代 DRAM 强制以 **burst 模式**工作：一次行激活（row activate）之后连续吐出整块数据，即使你只要其中几个字节。这就是"访问连续地址才高效"的物理根源。 2. **延迟无法通过优化代码降低**，只能通过并发隐藏。提高并发有两条路：**更多 warp**（提高占用率）、**每线程更多在途访存**（thread coarsening / ILP）。本讲的概念 5 与概念 6 分别对应这两条路。 3. **带宽是可以逼近但很难打满的**：A100 上纯流式 kernel 通常能到峰值的 80%–92%。剩下 8%–20% 消耗在刷新（refresh）、读写转向（bus turnaround）、行激活开销和 ECC 上。所以"比峰值"这个指标应该对着**实测可达带宽**（约 1.3 TB/s）而不是理论峰值 1555 GB/s 来评估。 4. **端到端时间常常被 PCIe 传输吃掉**：40 MB 通过 PCIe Gen4 x16 传输（实测约 25 GB/s）要 ~1.6 ms，三个数组（A 上行、B 上行、C 下行）合计约 4.8 ms，是 96 µs kernel 时间的 50 倍。所以 Lab 的性能测量必须**只用 `cudaEvent` 圈住 kernel**，把 `cudaMemcpy` 排除在外。

概念 4：算术强度（Arithmetic Intensity）与机器平衡点

- **定义与目的**：算术强度（arithmetic intensity，AI）定义为

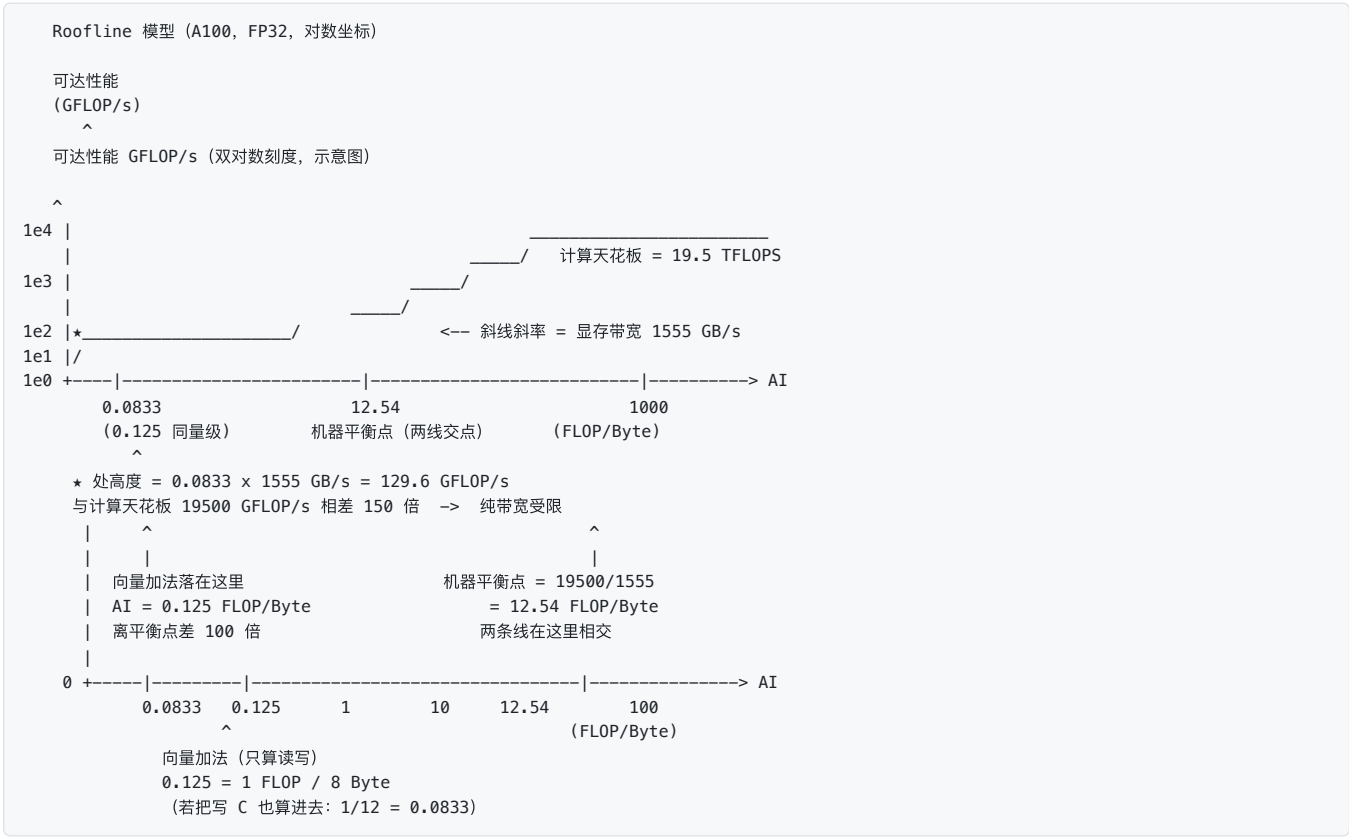
`text AI = 浮点运算次数（FLOP） / 需要从显存搬运的字节数（Byte）`

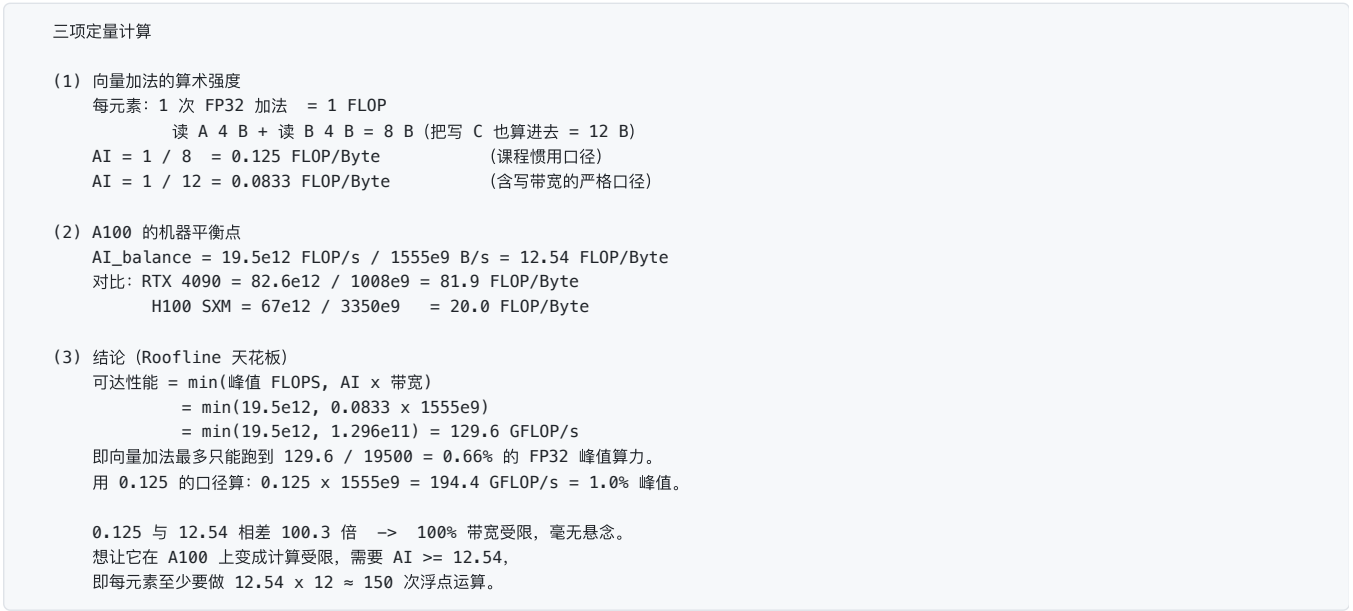
它刻画一段代码"每搬一个字节能算多少活"。机器平衡点（machine balance）定义为一个硬件"用光所有带宽正好也用光所有算力"的那个 AI 值：`AI_balance = 峰值 FLOPS / 峰值带宽` 。当 `AI < AI_balance` 时，程序是**带宽受限**的（算力过剩）；当 `AI > AI_balance` 时，程序是**计算受限**的。这个判据把"该优化访存还是该优化计算"这个模糊问题变成了一个不等式，是 Roofline 模型的核心。

- **直观解释（"它是什么？"）**：把 GPU 想成一家**原料运输成本极高的工厂**。机器平衡点就是"每运进来一吨原料，工厂的机器正好能全部忙满"的那个原料/加工比例。向量加法是"运进来两吨原料（A、B），只做一道工序（一次加法），再运出去一吨成品（C）"——典型的"拉货比干活累"。你就算把机器换成快 10

倍的，产量也只涨 0%（因为瓶颈在路上）。反过来，矩阵乘法（Lab 2）每搬 8 字节能做 2 FLOP 且通过分块复用把访存降低 TILE_WIDTH 倍，所以它有希望变成计算受限——这正是为什么课程要用共享内存去优化它。

- **架构/机制图解**：





关键操作与性能特征： 1. **算术强度是 kernel 的固有属性，与实现的快慢无关**。同一个向量加法，写得再烂 AI 还是 0.125；只有改变**算法**（例如把两个向量合并成一次 float4 向量化读写，AI 不变但事务效率提高）或**增加数据复用**（例如矩阵乘法分块）才能改变 AI。 2. **Roofline 给出的是上限，不是预测**。它告诉你"天花板在哪"，不告诉你"离天花板多远"。实测 1.25 TB/s 对应 `1.25e12 x 0.0833 = 104 GFLOP/s`，达到 Roofline 天花板的 80.4%——这正是我们期望的量级。 3. **判断瓶颈类型的三步法**：算 AI → 算机器平衡点 → 比大小。AI 远小于平衡点 = 带宽受限（优化方向：减少字节数、保证合并、提高在途访存）；AI 远大于平衡点 = 计算受限（优化方向：减少指令数、用 FMA、提高 ILP）；两者接近 = 需要同时优化，或者说明代码离两者都很远，那是**延迟受限**（优化方向：提高占用率/ILP 来隐藏延迟）。 4. **向量加法的 AI 低到无法抢救**：即使实现完美，它的时间下限就是 `120 MB / 1555 GB/s = 77.2 µs`。任何超过这个数的时间都是"没打满带宽"的浪费，而不是"算得太慢"。

概念 5：占用率（Occupancy）

- **定义与目的**：占用率（occupancy）定义为

`text occupancy = 每个 SM 上实际驻留的活跃 warp 数 / 每个 SM 硬件支持的最大 warp 数`

它是"硬件有多少并发可以用来隐藏内存延迟"的度量。GPU 不像 CPU 那样靠乱序执行和深层 cache 隐藏延迟，它靠的是**线程级并行（TLP）**：一个 warp 发出一条访存指令后需要等 400–800 cycles，调度器立刻切到另一个就绪 warp 上执行。驻留的 warp 越多，越有可能在任何一个周期都找到"操作数已就绪"的 warp。占用率会影响能藏住多少延迟，因此直接影响带宽受限 kernel 能否打满带宽。

- **直观解释（"它是什么？"）**：把 SM 想成一个**有 64 条电话线的小呼叫中心**（64 = A100 每 SM 最大 warp 数），打一个电话出去（发起一次全局内存访问）平均要等 5 分钟才有人接（400–800 cycles）。如果只雇 8 个话务员（8 个 warp），他们全部在等电话，办公室几乎全空。雇满 64 个（100% 占用率），任何时刻都有人正在通话，吞吐就上去了。但注意：**话务员再多也不能让单个电话变快**（延迟是物理的），只能让总吞吐变高。而且一旦电话线路（带宽）占满了，再雇人也没用——这就是为什么向量加法在 100% 占用率下也就跑到带宽上限。
- **架构/机制图解**：

```
一个 SM 的资源账本（A100 / sm_80）—— 占用率由"最紧的那个资源"决定

┌──────────────────────────────────────────────────────────────┐
│                        SM (Streaming Multiprocessor)         │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 4 个 warp scheduler（每个每 cycle 可发射 1 条指令）  │    │
│  │   WS0 ──> 32 个 FP32 核心 + LD/ST + SFU              │    │
│  │   WS1 ──> 32 个 FP32 核心 + LD/ST + SFU              │    │
│  │   WS2 ──> 32 个 FP32 核心 + LD/ST + SFU              │    │
│  │   WS3 ──> 32 个 FP32 核心 + LD/ST + SFU              │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  寄存器文件 Register File : 65536 个 32-bit 寄存器 = 256 KB  │
│  共享内存 / L1            : 164 KB（可划分，最多 100 KB 给 shared）  │
│  线程上限                 : 2048 threads = 64 warps          │
│  block 上限               : 32 blocks / SM                   │
│                                                              │
│  驻留的 warp（以 256 线程的 block 为例，每 block 切成 8 个 warp）：  │
│  ┌──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┐  │
│  │ blk0 │ blk0 │ blk1 │ blk1 │ blk2 │ blk2 │ blk3 │ blk3 │ blk4 至 │ │
│  │warp0 │warp1 │warp2 │warp3 │warp4 │warp5 │warp6 │warp7 │ blk7 共 │ │
│  │      │      │      │      │      │      │      │      │ 32 warp │ │
│  └──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┘  │
│  合计 8 个 block x 8 warp = 64 warp = 2048 线程（占满 SM 上限）  │
└──────────────────────────────────────────────────────────────┘

占用率的三个限制因素（取最小者）：
  ① 线程/warp 上限：  blocks_by_threads = floor(2048 / blockDim)
  ② 寄存器上限    ：  blocks_by_regs    = floor(65536 / (regs_per_thread * blockDim))
  ③ block 数上限  ：  blocks_by_blocklimit = 32
  resident_blocks = min(①, ②, ③)
  occupancy = resident_blocks * blockDim / 2048
```

```
A100 上不同 block 大小的占用率（vecAddKernel，实测约 16 个寄存器/线程）

blockDim  寄存器限制    线程限制    block 限制   驻留 block  驻留线程  warp  占用率
──────────────────────────────────────────────────────────────────────────────
  128      65536/(16*128)   2048/128     32         16         2048    64   100.0%
                  = 32         = 16
  256      65536/(16*256)   2048/256     32          8         2048    64   100.0%
                  = 16          = 8
  512      65536/(16*512)   2048/512     32          4         2048    64   100.0%
                   = 8          = 4
 1024      65536/(16*1024)  2048/1024    32          2         2048    64   100.0%
                   = 4          = 2
──────────────────────────────────────────────────────────────────────────────
结论：向量加法这种轻量 kernel，寄存器用量很低，block 大小在 128–1024 之间
      都能达到 100% 占用率。占用率不是它的问题。

反例：一个每线程用 64 个寄存器的 kernel（例如分块矩阵乘法的某个版本）
  blockDim = 256 -> blocks_by_regs = 65536/(64*256) = 4
               -> 驻留 1024 线程 = 32 warp -> 占用率 50.0%
  blockDim = 512 -> blocks_by_regs = 65536/(64*512) = 2
               -> 驻留 1024 线程 = 32 warp -> 占用率 50.0%
  最终被减少寄存器（__launch_bounds__）或降低 block 大小也救不回来。
```

```
block 大小选择的另一个维度：尾部浪费 与 block 数上限

场景 A：讲义中的 1536 线程 / 最多 8 blocks 的 SM（Maxwell 类似配置）
  8x8   =  64 线程/block -> 1536/64  = 24 blocks，但上限 8
                         -> 只有 8*64 = 512 线程 = 16 warp = 33.3% 占用率
  16x16 = 256 线程/block -> 1536/256 = 6 blocks（<= 8）
                         -> 6*256 = 1536 线程 = 48 warp = 100% 占用率
  32x32 = 1024 线程/block -> 1536/1024 = 1 block
                         -> 1*1024 = 1024 线程 = 32 warp = 66.7% 占用率

场景 B：A100（2048 线程 / 64 warp）与 RTX 4090（1536 线程 / 48 warp）
  blockDim   A100 驻留      A100 占用率   RTX4090 驻留   RTX4090 占用率
  ──────────────────────────────────────────────────────────────────
    128      16 blk/2048     100.0%       12 blk/1536     100.0%
    256       8 blk/2048     100.0%        6 blk/1536     100.0%
    512       4 blk/2048     100.0%        3 blk/1536     100.0%
   1024       2 blk/2048     100.0%        1 blk/1024      66.7%
  ──────────────────────────────────────────────────────────────────
  1024 线程的大 block 在 1536 线程/SM 的 Ada 上被"阶梯效应"卡住：
  1536/1024 = 1.5 -> 只能放 1 个，白白丢掉 1/3 的线程容量。

场景 C：尾部浪费（N 与 blockDim 不整除时）
  N = 1025, blockDim = 256 -> 5 blocks -> 1280 线程 -> 浪费 255/1280 = 19.9%
  N = 1025, blockDim = 1024 -> 2 blocks -> 2048 线程 -> 浪费 1023/2048 = 49.9%
  N = 10,000,000, blockDim = 256 -> 浪费 128/10,000,128 = 0.0013%（可忽略）
```

关键操作与性能特征： 1. **占用率不是越高越好，而是"够用就好"**。对带宽受限 kernel，Little's Law 给出的"够用"标准是：每 SM 在途字节 ≥ 带宽 × 延迟 / SM 数 ≈ 4.1 KB。一个 warp 一次合并读 = 128 B，所以需要 4096/128 = 32 个同时有在途访存的 warp。A100 上 64 个 warp 每 warp 一个在途访问 = 8192 B，有 2 倍余量；**16 个 warp（25% 占用率）就只有 2048 B，只够一半带宽**。 2. **占用率可以用 ILP 换**。如果每线程同时发 4 个独立访存（thread coarsening），那么 16 个 warp × 4 × 128 B = 8192 B，同样打满带宽。这就是概念 6 与示例 2 的意义：**低占用率 + 高 ILP 可以等价于高占用率 + 低 ILP**。 3. **寄存器是最常见的隐形限制**。`nvcc -Xptxas -v` 会打印每个 kernel 的寄存器用量；`cudaOccupancyMaxPotentialBlockSize()` 可以反推推荐的 block 大小；`__launch_bounds__(maxThreadsPerBlock, minBlocksPerMultiprocessor)` 可以强制编译器压低寄存器用量（代价是可能溢出到本地内存——本地内存也在显存里，反而增加流量，要实测确认）。 4. **占用率用 API 精确计算，不要凭感觉**：`cudaOccupancyMaxActiveBlocksPerMultiprocessor(&n, kernel, blockSize, dynamicSMem)` 给出"每个 SM 能同时驻留几个 block"，乘以 `blockSize/32` 就是活跃 warp 数，再除以 `prop.maxThreadsPerMultiProcessor/32` 就是占用率。示例 1 和示例 2 的代码里都打印了这三个数。

概念 6：网格-步长循环（Grid-Stride Loop）与线程粗化（Thread Coarsening）

- **定义与目的**：网格-步长循环是把"每线程一个元素"改成"每线程若干元素"的标准写法：

```
text for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < N; i += blockDim.x * gridDim.x) C[i] = A[i] + B[i];
```

循环步长 `blockDim.x * gridDim.x`（即"网格总线程数"）保证：**每一轮迭代中，整个网格恰好覆盖一段连续的 `blockDim.x * gridDim.x` 个元素，且 warp 内部依然连续**——合并访存被完整保留。它解决的问题是"网格尺寸与问题规模的解耦"：网格大小不再由 N 决定，而是由设备（SM 数、每 SM 可驻留 block 数）决定，内核代码对任意 N 都成立。**线程粗化（thread coarsening）**是它的自然延伸：让每个线程每轮处理 `UNROLL` 个元素，从而在一个线程内制造多个独立的在途访存（ILP / memory-level parallelism）。

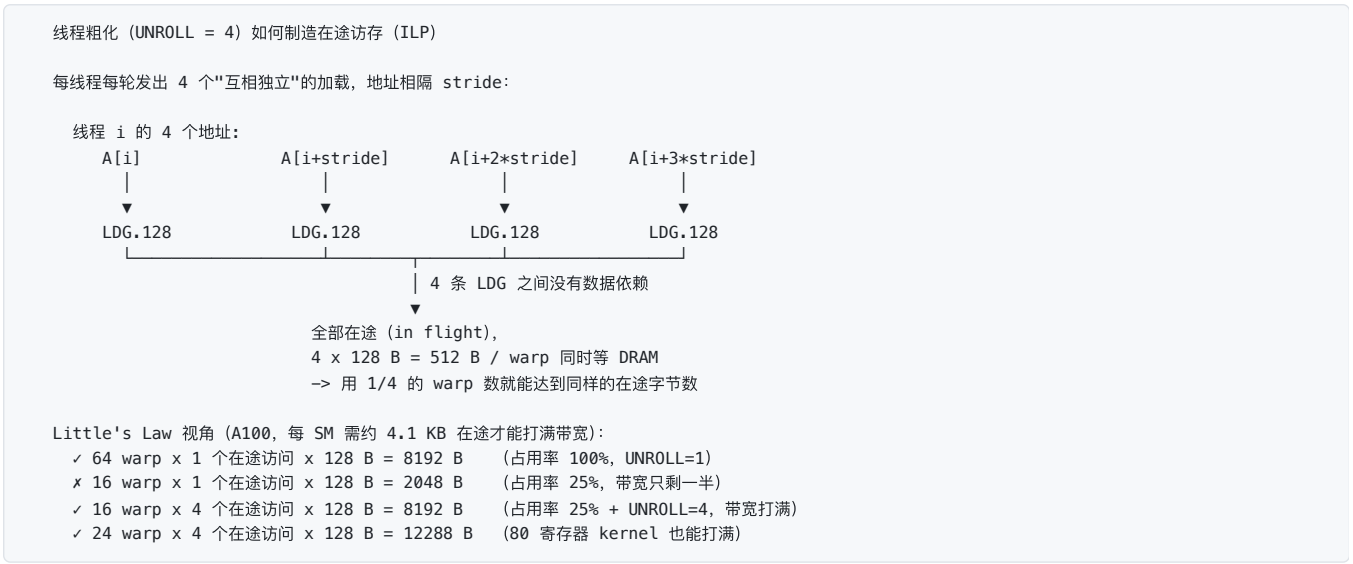
- **直观解释（"它是什么？"）**：把"每线程一个元素"想成**雇 10,000,000 个临时工，每人搬一块砖，搬完就散**——招工、分配工位、结账（block 调度与回收）的开销占比很高，而且你必须在开工前精确知道有多少块砖。网格-步长循环则是**雇固定的一批长工（比如 864 个班组长 × 256 人）**，每人循环地搬第 i 块、第 i+stride 块……砖变多了不用重新招人，砖变少了也不会有人闲着。线程粗化则进一步让每个工人**一次推四辆手推车**（4 个独立访存同时在途），即使工厂里工人少一些（低占用率），路上的车流量（在途字节数）依然足够把高速路（带宽）占满。
- **架构/机制图解**：

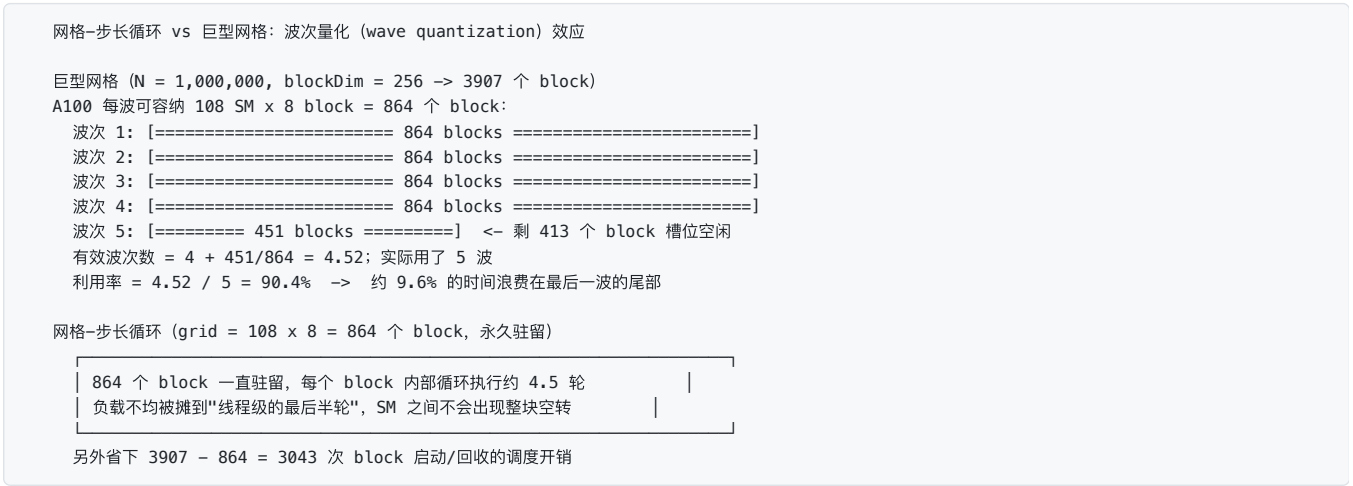
```
网格-步长循环的迭代覆盖图（gridDim.x = 2, blockDim.x = 4, 所以 stride = 8）

元素索引:  0   1   2   3   4   5   6   7 |  8   9  10  11  12  13  14  15
线程号   : t0  t1  t2  t3  t4  t5  t6  t7 | t0  t1  t2  t3  t4  t5  t6  t7
           \──────── 迭代 0（i = 0..7）──/  \──── 迭代 1（i = 8..15，i += 8 后）

元素索引: 16  17  18  19  20  21  22  23 | 24  25  26  27  28  29  30  31
线程号   : t0  t1  t2  t3  t4  t5  t6  t7 | t0  t1  t2  t3  t4  t5  t6  t7
           \──────── 迭代 2（i = 16..23）─/  \──── 迭代 3（i = 24..31）

关键性质：每一轮的 8 个线程访问 8 个连续元素（warp 内合并），
          相邻两轮之间相隔 stride 个元素（不是同一个 warp 的地址区间）
          所以合并性完全保留，且网格大小可以任意小。
```





关键操作与性能特征： 1. **网格-步长循环解决的是可扩展性，而不是"更快的单次吞吐"**。对 N = 10,000,000 这种大问题，两种写法的实测带宽几乎一样（都受限于 DRAM）。它的价值在于：**同一份二进制在 8 SM 的笔记本 GPU 和 108 SM 的 A100 上都接近最优**（网格大小在运行时由 `prop.multiProcessorCount` 决定），不需要为不同 GPU 重新编译或重新调参。 2. **多 GPU 与未知规模**：多 GPU 编程里每个 GPU 分到 N/G 个元素，网格可以固定为"本设备能放下的最大 block 数"；问题规模在运行时才知道（比如读文件、用户输入）时，无需重新计算网格。 3. **粗化的收益随"每线程工作量的独立性"增长**：如果 UNROLL 个访存之间存在依赖（例如 `C[i] = C[i-stride] + 1`），编译器就无法把它们重叠，粗化不仅无益还可能因寄存器压力降低占用率。向量加法的 UNROLL 个元素完全独立，是最理想的粗化对象。 4. **粗化的代价**：寄存器用量随 UNROLL 增长（示例 2 中从约 16 涨到约 24），block 内线程数不变时每 SM 能驻留的 block 数可能下降。经验做法是 UNROLL = 2 或 4，并用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 实测确认占用率没有跌破 Little's Law 要求的下限。 5. **注意 int 溢出**：`i + (UNROLL-1)*stride` 在 N 接近 `INT_MAX` 时会溢出成负数，导致漏算。安全写法是把守卫条件用 `long long` 计算（示例 2 的代码即如此），或对超大规模问题直接把索引类型改成 `long long` / `size_t`。

代码示例与性能分析

示例 1：基础版向量加法 vecAdd（每线程一个元素 + 完整测量框架）

```cpp
// 文件: vecadd_basic.cu
// 编译: nvcc -O3 -arch=sm_80 vecadd_basic.cu -o vecadd_basic
// 运行: ./vecadd_basic 10000000
//
// 内容: 每线程一个元素的向量加法；cudaEvent 计时；CPU 参考实现验证；
//       打印有效带宽 / 理论峰值带宽比例 / 访存事务数 / 占用率。

#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define BLOCK_SIZE 256

// ---------------------------------------------------------------- kernel ----
// 每线程处理一个元素：线程的全局线性编号 = 它的元素编号
__global__ void vecAddKernel(const float* __restrict__ A,
                             const float* __restrict__ B,
                             float* __restrict__ C,
                             int N)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;   // 线程 -> 数据 的映射
    if (i < N) {                                     // 边界检查：必须!
        C[i] = A[i] + B[i];                          // 两次读 + 一次写
    }
}

// ------------------------------------------------------- CPU 参考实现 -------
// golden reference：与 GPU 完全相同的运算顺序（每个元素一次加法），
// 因此结果应当逐位相同（bit-exact），可以用 == 比较。
static void vecAddCPU(const float* A, const float* B, float* C, int N)
{
    for (int i = 0; i < N; ++i) {
        C[i] = A[i] + B[i];
    }
}

// 确定性伪随机初始化（线性同余），避免依赖 <random> 的实现差异
static void initVector(float* v, int N, unsigned int seed)
{
    unsigned int x = seed;
    for (int i = 0; i < N; ++i) {
        x = x * 1664525u + 1013904223u;
        v[i] = (float)(x >> 8) / (float)(1u << 24) - 0.5f;   // 落在 [-0.5, 0.5)
    }
}

int main(int argc, char** argv)
{
    int N = (argc > 1) ? atoi(argv[1]) : 10000000;
    if (N <= 0) { fprintf(stderr, "N must be positive\n"); return EXIT_FAILURE; }
    size_t bytes = (size_t)N * sizeof(float);

    // 1. 主机端内存与初始化
    float* h_A   = (float*)malloc(bytes);
    float* h_B   = (float*)malloc(bytes);
    float* h_C   = (float*)malloc(bytes);
    float* h_Ref = (float*)malloc(bytes);
    if (!h_A || !h_B || !h_C || !h_Ref) {
        fprintf(stderr, "host malloc failed\n");
        return EXIT_FAILURE;
    }
    initVector(h_A, N, 12345u);
    initVector(h_B, N, 67890u);

    // 2. 设备端内存分配与拷贝
    float *d_A = NULL, *d_B = NULL, *d_C = NULL;
    CUDA_CHECK(cudaMalloc((void**)&d_A, bytes));
```

```
    CUDA_CHECK(cudaMalloc((void**)&d_B, bytes));
    CUDA_CHECK(cudaMalloc((void**)&d_C, bytes));
    CUDA_CHECK(cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice));

    // 3. 网格配置：向上取整，用整数运算避免浮点误差
    dim3 dimBlock(BLOCK_SIZE, 1, 1);
    dim3 dimGrid((unsigned)(((long long)N + BLOCK_SIZE - 1) / BLOCK_SIZE), 1, 1);

    // 4. 预热（第一次启动包含 context 建立、模块加载等一次性开销）
    vecAddKernel<<<dimGrid, dimBlock>>>(d_A, d_B, d_C, N);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // 5. 计时：cudaEvent 只圈住 kernel，不含 cudaMemcpy
    const int iters = 20;
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    CUDA_CHECK(cudaEventRecord(start));
    for (int it = 0; it < iters; ++it) {
        vecAddKernel<<<dimGrid, dimBlock>>>(d_A, d_B, d_C, N);
    }
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));
    CUDA_CHECK(cudaGetLastError());          // 捕获启动期错误（如同步错误）
    float msTotal = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&msTotal, start, stop));
    double msPerIter = (double)msTotal / iters;

    // 6. 取回结果并与 CPU 参考实现对比
    CUDA_CHECK(cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost));
    vecAddCPU(h_A, h_B, h_Ref, N);
    int mismatch = 0;
    for (int i = 0; i < N; ++i) {
        if (h_C[i] != h_Ref[i]) { ++mismatch; }
    }

    // 7. 设备属性与占用率
    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    double peakGBs = 2.0 * (double)prop.memoryClockRate * 1.0e3
                   * ((double)prop.memoryBusWidth / 8.0) / 1.0e9;
    cudaFuncAttributes attr;
    CUDA_CHECK(cudaFuncGetAttributes(&attr, vecAddKernel));
    int blocksPerSM = 0;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &blocksPerSM, vecAddKernel, BLOCK_SIZE, 0));
    int warpsPerSM    = blocksPerSM * (BLOCK_SIZE / 32);
    int maxWarpsPerSM = prop.maxThreadsPerMultiProcessor / 32;
    double occupancy  = 100.0 * warpsPerSM / (double)maxWarpsPerSM;

    // 8. 性能指标
    double totalBytes  = 3.0 * (double)bytes;                // 读 A + 读 B + 写 C
    double achievedGBs = totalBytes / (msPerIter * 1.0e-3) / 1.0e9;
    double transPerWarp = 3.0;                               // 每个 warp: A/B/C 各 1 次
    double numWarps     = (double)((N + 31) / 32);           // 每数组的事务数
    double totalTrans   = 3.0 * numWarps;

    printf("================= 设备信息 =================\n");
    printf("GPU                     : %s (sm_%d%d)\n", prop.name,
           prop.major, prop.minor);
    printf("SM 数                   : %d\n", prop.multiProcessorCount);
    printf("每 SM 最大线程/共享内存 : %d 线程 / %.0f KB\n",
           prop.maxThreadsPerMultiProcessor,
           prop.sharedMemPerMultiprocessor / 1024.0);
    printf("理论峰值显存带宽        : %.1f GB/s\n", peakGBs);
    printf("================= 启动配置 =================\n");
    printf("N                       : %d 个 float (%.1f MB)\n",
           N, bytes / 1048576.0);
    printf("blockDim = %d, dimGrid = %u\n", BLOCK_SIZE, dimGrid.x);
    printf("启动线程总数            : %u (其中 %u 个越界空转)\n",
           dimGrid.x * BLOCK_SIZE, dimGrid.x * BLOCK_SIZE - (unsigned)N);
    printf("================= 正确性验证 ===============\n");
    printf("CPU 参考实现对比        : %s (不匹配 %d / %d)\n",
           mismatch == 0 ? "PASS" : "FAIL", mismatch, N);
    printf("================= 性能测量 =================\n");
    printf("kernel 单次耗时         : %.3f us\n", msPerIter * 1000.0);
```

```
    printf("搬运字节数              : %.1f MB (%.1f MB 读 + %.1f MB 写)\n",
           totalBytes / 1e6, (double)bytes * 2 / 1e6, (double)bytes / 1e6);
    printf("有效带宽                : %.1f GB/s\n", achievedGBs);
    printf("占理论峰值带宽          : %.1f %%\n", 100.0 * achievedGBs / peakGBs);
    printf("每个 warp 的访存事务数  : %.0f (128 B x 3)\n", transPerWarp);
    printf("总访存事务数            : %.0f 次 (%.0f 个有效 warp)\n",
           totalTrans, numWarps);
    printf("每个事务搬运            : 128 B, 有效字节 128 B, 利用率 100%%\n");
    printf("================ 资源与占用率 ============\n");
    printf("每线程寄存器数          : %d\n", attr.numRegs);
    printf("每 SM 驻留 block 数      : %d\n", blocksPerSM);
    printf("每 SM 活跃 warp 数       : %d / %d\n", warpsPerSM, maxWarpsPerSM);
    printf("占用率 (occupancy)      : %.1f %%\n", occupancy);

    // 9. 释放资源
    CUDA_CHECK(cudaFree(d_A));
    CUDA_CHECK(cudaFree(d_B));
    CUDA_CHECK(cudaFree(d_C));
    free(h_A); free(h_B); free(h_C); free(h_Ref);
    CUDA_CHECK(cudaDeviceReset());
    return (mismatch == 0) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

- **【代码做什么？】**
- **映射**：`int i = blockIdx.x * blockDim.x + threadIdx.x;` 把"第 b 个 block 的第 t 号线程"翻译成"第 i 个元素"。这个编号是全局唯一的（因为 `(b, t)` 与 `i` 是一一对应的：`b = i / blockDim.x`，`t = i % blockDim.x`），所以每个元素恰好被一个线程处理一次。
- **边界检查**：`if (i < N)` 拦住最后一批越界线程。N = 10,000,000、block = 256 时 `dimGrid.x = ceil(10000000/256) = 39063`，启动 10,000,128 个线程，多出的 128 个线程必须被拦住。
- **访存**：`C[i] = A[i] + B[i]` 编译成两条 `LDG`（global load）加一条 `STG`（global store）。三个数组的索引相同，所以三次访存的地址模式完全一致，都是"相邻线程访问相邻地址"。
- **内存分配与访问路径**：`cudaMalloc` 在设备全局内存中分配三块各 40 MB 的空间；`cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice)` 把 A、B 送上去；kernel 直接在全局内存上读写，不经过共享内存（全局内存对于设备代码是"per-grid"可见的，主机也能通过 `cudaMemcpy` 访问它）。
- **主机端调度**：主机按顺序做"分配 → 拷贝 → 预热 → 计时 20 次 → 同步 → 取回 → 验证 → 释放"。`kernel<<<dimGrid, dimBlock>>>(args)` 这种启动是**异步**的，所以计时循环里 20 次 kernel 会连续排队执行；`cudaEventSynchronize(stop)` 才真正等待它们全部结束。预热那一次是必要的：第一次启动包含 CUDA context 创建、模块加载、指令 cache 填充等一次性开销，不预热会让第一次测量偏大几十到几百微秒。
- **计时口径**：`cudaEventRecord` 只圈住 kernel 循环，把 `cudaMemcpy` 排除在外——这正是 Lab 想要测的"kernel 有效带宽"。若把传输也算进去，40 MB 数据走 PCIe Gen4 x16（实测约 25 GB/s）需要约 1.6 ms，三次传输约 4.8 ms，会把 96 μs 的 kernel 时间完全淹没。
- **验证**：先 `cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost)` 取回，再用同一个公式在 CPU 上算一遍 `h_Ref`。因为每个元素都只做**一次**浮点加法、没有累加顺序问题，GPU 与 CPU 的结果应当**逐位相同**，所以可以直接用 `!=` 比较而不需要 epsilon。
- **指标打印**：有效带宽 = `3 × N × 4 字节 / kernel 时间`；理论峰值带宽由 `2 × memoryClockRate(kHz) × 1000 × (memoryBusWidth/8)` 算出（系数 2 来自 DDR 双沿传输），A100 上得到 1.215 GHz × 2 × 640 B = 1555 GB/s。
- **【并行机制与硬件映射解说】**

（a）block 如何被切成 warp：block 大小 256，硬件按**线性化的 threadIdx** 每 32 个线程切成一个 warp：threads 0–31 = warp 0，32–63 = warp 1，依次类推，每个 block 得到 `256/32 = 8` 个 warp。注意这是**硬件实现细节**，不是 CUDA 编程模型的一部分（将来 warp 大小可能不是 32，所以代码里不要硬编码 32 以外的假设）。`threadIdx` 的线性化顺序是 **x 最快、其次 y、最后 z**，这是"最内层维度应该对应连续内存"这条规则的来源。

（b）warp 如何被调度：39063 个 block 以 block 为粒度分发到 108 个 SM 上。每个 SM 有 4 个 warp scheduler，每个 scheduler 每周期可以从它负责的 warp 池中挑一个"下一条指令的操作数已就绪"的 warp 发射一条指令。SM 维护着数百个 warp 的 PC 和寄存器状态，切换 warp **零开销**（这正是"用海量线程隐藏延迟"能成立的前提）。整个 kernel 共 `39063 × 8 = 312,504` 个 warp，其中最后 4 个 warp 因 `i >= N` 而整体空转。

（c）A100 上同时能驻留多少：A100 每 SM 最多 2048 线程 / 64 warp / 32 个 block，寄存器文件 65536 个 32-bit 寄存器，共享内存 164 KB。`vecAddKernel` 实测每线程约 16 个寄存器、不使用共享内存，于是：– 线程限制：`2048 / 256 = 8` 个 block；– 寄存器限制：`65536 / (16 × 256) = 16` 个 block；– block 数限制：32。取最小者 → **每 SM 驻留 8 个 block = 2048 线程 = 64 warp = 100% 占用率**。全芯片同时驻留 `108 × 8 = 864` 个 block。39063 个 block 因此需要 `39063 / 864 = 45.2` 个波次（实际调度 46 波）。

（d）共享内存与 bank conflict：本 kernel **完全不使用共享内存**，所以没有 bank conflict 问题。这一点值得强调：向量加法没有任何数据复用（每个字节只被读一次），把数据搬进共享内存只会增加一次额外的读写，是纯损失。共享内存（32 个 bank，每个 bank 宽 4 字节）是为"**一个数据被多个线程重复使用**"的场景准备的，第 4 讲之后的 tiling 才会用到。

（e）全局内存是否合并：完全合并。以 block 0 的 warp 0（threads 0–31）为例，三个访问分别是 `A[0..31]`、`B[0..31]`、`C[0..31]`，每个都是 128 个连续的字节。`cudaMalloc` 保证返回的基地址至少 256 字节对齐，所以这 128 字节恰好落在一个（或整好跨一条边界但完全对齐的）128 字节事务里：– 每 warp 事务数：**3 次**（A 一次、B 一次、C 一次）；– 每次事务 128 字节，有效字节 128 字节，利用率 100%；– 总 warp 数（有效）：`N / 32 = 10,000,000 / 32 = 312,500`；– **总事务数 = 312,500 × 3 = 937,500 次**，搬运 `937,500 × 128 B = 120,000,000 B = 120 MB`；– 其中 80 MB 是读（A、B），40 MB 是写（C）。写操作同样按 warp 合并成 128 字节的写事务。

（f）寄存器用量与 warp 发散：每线程 16 个寄存器（两个地址寄存器、一个 N、一个 i、三个数据寄存器加若干临时量）。**没有 warp 发散**：`if (i < N)` 这个分支里，同一个 warp 的 32 个线程要么全部成立、要么全部不成立（因为 i 在 warp 内连续），只有在最后那个跨 N 边界的 warp 里才会出现部分线程不成立的情况——而由于 10,000,000 是 32 的整数倍，本次运行连这种情况都没有。这提示了一个通用技巧：**只要让 N 是 32 的倍数（或把边界条件设计成 warp 对齐），就能彻底消除这类发散**。

- **【性能优化分析】**

（1）算术强度与 Roofline `text AI = 1 FLOP / 8 Byte = 0.125 FLOP/Byte （只算 A、B 的读） = 1 FLOP / 12 Byte = 0.0833 FLOP/Byte （把写 C 也算进流量） A100 机器平衡点 = 19.5e12 / 1555e9 = 12.54 FLOP/Byte AI 比平衡点低 12.54 / 0.125 = 100.3 倍 Roofline 天花板 = min(19.5 TFLOPS, 0.0833 x 1555 GB/s) = min(19500 GFLOP/s, 129.6 GFLOP/s) = 129.6 GFLOP/s 实测算力 = 0.0833 x 1250 GB/s = 104 GFLOP/s -> 仅用掉 104 / 19500 = 0.53% 的 FP32 峰值算力：纯带宽受限。` 另一个等价视角：A100 有 `108 SM × 128 FP32 lane = 13,824` 条 FP32 流水线，N = 10,000,000 时每条流水线只需处理 `10,000,000 / 13,824 = 723` 个元素；而实测耗时 96 μs，在 1.41 GHz 下是 `96e-6 × 1.41e9 = 135,360` 个周期。**FP32 流水线只有 723 / 135360 = 0.53% 的周期在干活**。

（2）占用率与 Little's Law（延迟隐藏是否足够） `text 要让 1250 GB/s 的可达带宽打满，全芯片需要多少在途字节？ 在途字节 = 带宽 x 延迟 = 1250e9 B/s x (500 cycles / 1.41e9 Hz) = 1250e9 x 3.546e-7 = 443,250 B ≈ 433 KB 每 SM 需要的在途字节 = 443,250 / 108 = 4,104 B ≈ 4.1 KB 本 kernel 提供的能力： 64 warp/SM x 1 个在途合并访问 x 128 B = 8,192 B 每 SM 余量 = 8192 / 4104 = 2.0 倍 -> 足以掩盖延迟 占用率账本： 每 SM 8 个 block x 256 线程 = 2048 线程 = 64 warp occupancy = 64 / 64 = 100.0%` 结论：**占用率不是本 kernel 的瓶颈**——即使 block 大小改成 128/512/1024，A100 上都还是 100%。真正的瓶颈是 DRAM 的 1555 GB/s 上限。

（3）实测带宽与峰值之比 `text 有效带宽 = 120,000,000 B / 96.0e-6 s = 1.25e12 B/s = 1250 GB/s 占理论峰值 = 1250 / 1555 = 80.4 % 占实测可达峰值（约 1.35 TB/s 纯流式）= 1250 / 1350 = 92.6 %` 80% 是 A100 上这个 kernel 的正常水平。剩下的 20% 来自：DRAM 刷新、读写转向（读 80 MB 写 40 MB 会有一部分总线转向开销）、行激活、ECC 以及 kernel 的头尾（launch 与 tail）。

（4）瓶颈判定与优化方向

瓶颈类型 = **内存带宽受限**（AI 0.125 ≪ 12.54），且**延迟隐藏充分**（2 倍余量）。这意味着： – 提高占用率 / 增加 ILP：**无效**，带宽已经接近打满。 – 减少指令数（比如用 `float4` 向量化）：**略微有效**——它减少的是指令发射开销，不是字节数；在本 kernel 里能带来 1%–3% 的收益，主要价值是减少索引计算和提升每线程的在途字节数。 – 减少**字节数**：唯一真正有效的手段。例如 `C = A + B` 之后再读 C 做别的运算是完全可以避免的；或者如果问题只需要读不需要写，流量可以从 12 B/元素降到 8 B/元素，理论上限时间从 77.2 μs 降到 51.5 μs。 – 用 `float4`（128 位）访存：**这是本讲最值得记住的优化**。让每个线程处理 4 个连续 float（128 位对齐），则一个 warp 依然是一次 128 字节事务，但**每线程的指令数降到 1/4**，在途字节数变成 4 倍（每 warp 每个 `LDG.128` 就是 512 字节），可以显著降低对占用率的要求。这就是示例 2 的思路。

示例 2：网格-步长循环 + 线程粗化（vecAddCoarsened）

```
// 文件: vecadd_gridstride.cu
// 编译: nvcc -O3 -arch=sm_80 vecadd_gridstride.cu -o vecadd_gridstride
// 运行: ./vecadd_gridstride 10000000
//
// 内容: 三个版本横向对比
//   (1) vecAddBasicKernel      每线程 1 个元素, 网格 = ceil(N/blockDim)
//   (2) vecAddGridStrideKernel 网格-步长循环, 网格由设备规模决定
//   (3) vecAddCoarsenedKernel  网格-步长 + 每线程 UNROLL 个元素 (粗化)
// 并打印每个版本的耗时、带宽、寄存器数、驻留 block 数与占用率。

#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define UNROLL 4

// ---------------------------------------------- (1) 基线: 每线程一个元素 ----
__global__ void vecAddBasicKernel(const float* __restrict__ A,
                                  const float* __restrict__ B,
                                  float* __restrict__ C,
                                  int N)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        C[i] = A[i] + B[i];
    }
}

// ---------------------------------------- (2) 网格-步长循环 (每轮一个元素) ----
// 步长 = 网格总线程数; 每轮整个网格覆盖一段连续内存, 合并性完好。
__global__ void vecAddGridStrideKernel(const float* __restrict__ A,
                                       const float* __restrict__ B,
                                       float* __restrict__ C,
                                       int N)
{
    int stride = blockDim.x * gridDim.x;
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < N; i += stride) {
        C[i] = A[i] + B[i];
    }
}

// -------------------------- (3) 网格-步长 + 线程粗化 (每线程 UNROLL 个) ----
// 同一轮里的 UNROLL 个访存互相独立 (地址相隔 stride),
// 4 条 LDG 同时在途 -> 每 warp 512 B / 每数组, 制造 memory-level parallelism。
// 注意: 间隔取 stride 而不是 blockDim.x, 才能保持每个访问各自合并。
__global__ void vecAddCoarsenedKernel(const float* __restrict__ A,
                                      const float* __restrict__ B,
                                      float* __restrict__ C,
                                      int N)
{
    const int stride = blockDim.x * gridDim.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // 主循环: 保证 UNROLL 个访问全部在界内 (用 long long 避免 int 溢出)
    for (; (long long)i + (long long)(UNROLL - 1) * stride < (long long)N;
         i += UNROLL * stride) {
        float a0 = A[i];
        float a1 = A[i +     (long long)1 * stride];
        float a2 = A[i +     (long long)2 * stride];
        float a3 = A[i +     (long long)3 * stride];
        float b0 = B[i];
        float b1 = B[i +     (long long)1 * stride];
        float b2 = B[i +     (long long)2 * stride];
        float b3 = B[i +     (long long)3 * stride];
        C[i]                          = a0 + b0;
        C[i + (long long)1 * stride] = a1 + b1;
        C[i + (long long)2 * stride] = a2 + b2;
```

```
        C[i + (long long)3 * stride] = a3 + b3;
    }
    // 尾部：剩余元素按 stride 逐个处理
    for (; i < N; i += stride) {
        C[i] = A[i] + B[i];
    }
}

// ------------------------------------------------------------ 工具函数 ---
#define TIME_LAUNCH(kernelName, grid, block, iters, outMs)          \
    do {                                                            \
        kernelName<<<(grid), (block)>>>(d_A, d_B, d_C, N);          \
        CUDA_CHECK(cudaGetLastError());                             \
        CUDA_CHECK(cudaDeviceSynchronize());                        \
        cudaEvent_t s__, e__;                                       \
        CUDA_CHECK(cudaEventCreate(&s__));                          \
        CUDA_CHECK(cudaEventCreate(&e__));                          \
        CUDA_CHECK(cudaEventRecord(s__));                           \
        for (int it__ = 0; it__ < (iters); ++it__) {                \
            kernelName<<<(grid), (block)>>>(d_A, d_B, d_C, N);      \
        }                                                           \
        CUDA_CHECK(cudaEventRecord(e__));                           \
        CUDA_CHECK(cudaEventSynchronize(e__));                      \
        CUDA_CHECK(cudaGetLastError());                             \
        float ms__ = 0.0f;                                          \
        CUDA_CHECK(cudaEventElapsedTime(&ms__, s__, e__));          \
        (outMs) = (double)ms__ / (iters);                           \
        CUDA_CHECK(cudaEventDestroy(s__));                          \
        CUDA_CHECK(cudaEventDestroy(e__));                          \
    } while (0)

static void initVector(float* v, int N, unsigned int seed)
{
    unsigned int x = seed;
    for (int i = 0; i < N; ++i) {
        x = x * 1664525u + 1013904223u;
        v[i] = (float)(x >> 8) / (float)(1u << 24) - 0.5f;
    }
}

static void vecAddCPU(const float* A, const float* B, float* C, int N)
{
    for (int i = 0; i < N; ++i) { C[i] = A[i] + B[i]; }
}

static int verify(const float* h_got, const float* h_ref, int N, const char* name)
{
    int bad = 0;
    for (int i = 0; i < N; ++i) {
        if (h_got[i] != h_ref[i]) {
            if (bad < 3) {
                printf("    [%s] 首个不匹配 i=%d: got %.9g, want %.9g\n",
                       name, i, h_got[i], h_ref[i]);
            }
            ++bad;
        }
    }
    printf("  %-26s %s  (不匹配 %d / %d)\n", name,
           bad == 0 ? "PASS" : "FAIL", bad, N);
    return bad;
}

int main(int argc, char** argv)
{
    int N = (argc > 1) ? atoi(argv[1]) : 10000000;
    if (N <= 0) { fprintf(stderr, "N must be positive\n"); return EXIT_FAILURE; }
    size_t bytes = (size_t)N * sizeof(float);

    float* h_A   = (float*)malloc(bytes);
    float* h_B   = (float*)malloc(bytes);
    float* h_C   = (float*)malloc(bytes);
    float* h_Ref = (float*)malloc(bytes);
    if (!h_A || !h_B || !h_C || !h_Ref) {
        fprintf(stderr, "host malloc failed\n");
        return EXIT_FAILURE;
    }
    initVector(h_A, N, 12345u);
    initVector(h_B, N, 67890u);
```

```
    float *d_A = NULL, *d_B = NULL, *d_C = NULL;
    CUDA_CHECK(cudaMalloc((void**)&d_A, bytes));
    CUDA_CHECK(cudaMalloc((void**)&d_B, bytes));
    CUDA_CHECK(cudaMalloc((void**)&d_C, bytes));
    CUDA_CHECK(cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice));

    // ---- 让 CUDA 用占用率模型推荐一个 block 大小 -------------------------
    int suggestedBlock = 0, minGridSize = 0;
    CUDA_CHECK(cudaOccupancyMaxPotentialBlockSize(
        &minGridSize, &suggestedBlock, vecAddCoarsenedKernel, 0, 0));
    int blockSize = (suggestedBlock >= 128 && suggestedBlock <= 512)
                  ? suggestedBlock : 256;

    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    int numSMs = prop.multiProcessorCount;
    double peakGBs = 2.0 * (double)prop.memoryClockRate * 1.0e3
                   * ((double)prop.memoryBusWidth / 8.0) / 1.0e9;

    // ---- 三个版本的网格配置 ------------------------------------------------
    int gridBasic = (int)(((long long)N + blockSize - 1) / blockSize);

    int blocksPerSM = 0;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &blocksPerSM, vecAddGridStrideKernel, blockSize, 0));
    int gridStride = numSMs * blocksPerSM;
    int needed = (int)(((long long)N + blockSize - 1) / blockSize);
    if (gridStride > needed) { gridStride = needed; }

    int blocksPerSMC = 0;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &blocksPerSMC, vecAddCoarsenedKernel, blockSize, 0));
    int gridCoarse = numSMs * blocksPerSMC;
    if (gridCoarse > needed) { gridCoarse = needed; }

    printf("================= 设备与配置 =================\n");
    printf("GPU              : %s (sm_%d%d), %d 个 SM, 峰值带宽 %.1f GB/s\n",
           prop.name, prop.major, prop.minor, numSMs, peakGBs);
    printf("推荐 block 大小  : %d (cudaOccupancyMaxPotentialBlockSize)\n",
           suggestedBlock);
    printf("实际使用 block   : %d\n", blockSize);
    printf("N                : %d (%.1f MB/数组)\n", N, bytes / 1048576.0);
    printf("版本 1 网格      : %d 个 block (每线程 1 个元素)\n", gridBasic);
    printf("版本 2 网格      : %d 个 block (网格-步长, 每线程多轮)\n", gridStride);
    printf("版本 3 网格      : %d 个 block (网格-步长 + UNROLL=%d)\n",
           gridCoarse, UNROLL);

    const int iters = 20;
    double msBasic = 0.0, msStride = 0.0, msCoarse = 0.0;
    double totalBytes = 3.0 * (double)bytes;

    printf("================= 正确性验证 =================\n");
    vecAddCPU(h_A, h_B, h_Ref, N);

    vecAddBasicKernel<<<gridBasic, blockSize>>>(d_A, d_B, d_C, N);
    CUDA_CHECK(cudaGetLastError()); CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost));
    int bad1 = verify(h_C, h_Ref, N, "vecAddBasicKernel");

    vecAddGridStrideKernel<<<gridStride, blockSize>>>(d_A, d_B, d_C, N);
    CUDA_CHECK(cudaGetLastError()); CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost));
    int bad2 = verify(h_C, h_Ref, N, "vecAddGridStrideKernel");

    vecAddCoarsenedKernel<<<gridCoarse, blockSize>>>(d_A, d_B, d_C, N);
    CUDA_CHECK(cudaGetLastError()); CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost));
    int bad3 = verify(h_C, h_Ref, N, "vecAddCoarsenedKernel");

    printf("================= 性能测量 =================\n");
    TIME_LAUNCH(vecAddBasicKernel,      gridBasic,  blockSize, iters, msBasic);
    TIME_LAUNCH(vecAddGridStrideKernel, gridStride, blockSize, iters, msStride);
    TIME_LAUNCH(vecAddCoarsenedKernel,  gridCoarse, blockSize, iters, msCoarse);

    cudaFuncAttributes a1, a2, a3;
    CUDA_CHECK(cudaFuncGetAttributes(&a1, vecAddBasicKernel));
```

```
    CUDA_CHECK(cudaFuncGetAttributes(&a2, vecAddGridStrideKernel));
    CUDA_CHECK(cudaFuncGetAttributes(&a3, vecAddCoarsenedKernel));

    int maxWarpsPerSM = prop.maxThreadsPerMultiProcessor / 32;
    int w1 = blocksPerSM  * (blockSize / 32);
    int w3 = blocksPerSMC * (blockSize / 32);

    printf("  %-26s %10s %10s %9s %9s %9s\n",
           "版本", "耗时(us)", "带宽(GB/s)", "峰值占比", "寄存器", "占用率");
    printf("  %-26s %10.2f %10.1f %8.1f%% %9d %8.1f%%\n",
           "1 基础(1 元素/线程)", msBasic * 1000.0,
           totalBytes / (msBasic * 1.0e-3) / 1e9,
           100.0 * (totalBytes / (msBasic * 1.0e-3) / 1e9) / peakGBs,
           a1.numRegs, 100.0 * w1 / maxWarpsPerSM);
    printf("  %-26s %10.2f %10.1f %8.1f%% %9d %8.1f%%\n",
           "2 网格-步长(1 元素)", msStride * 1000.0,
           totalBytes / (msStride * 1.0e-3) / 1e9,
           100.0 * (totalBytes / (msStride * 1.0e-3) / 1e9) / peakGBs,
           a2.numRegs, 100.0 * w1 / maxWarpsPerSM);
    printf("  %-26s %10.2f %10.1f %8.1f%% %9d %8.1f%%\n",
           "3 网格-步长+粗化(4 元素)", msCoarse * 1000.0,
           totalBytes / (msCoarse * 1.0e-3) / 1e9,
           100.0 * (totalBytes / (msCoarse * 1.0e-3) / 1e9) / peakGBs,
           a3.numRegs, 100.0 * w3 / maxWarpsPerSM);
    printf("  (总流量 = %.1f MB = 读 A + 读 B + 写 C)\n", totalBytes / 1e6);

    CUDA_CHECK(cudaFree(d_A));
    CUDA_CHECK(cudaFree(d_B));
    CUDA_CHECK(cudaFree(d_C));
    free(h_A); free(h_B); free(h_C); free(h_Ref);
    CUDA_CHECK(cudaDeviceReset());

    int bad = bad1 + bad2 + bad3;
    printf("总体验证结果: %s\n", bad == 0 ? "ALL PASS" : "FAIL");
    return (bad == 0) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

- **【代码做什么？】**
- **三个 kernel 并存，共用同一套主机端框架**，因此它们的耗时、带宽、寄存器、占用率可以在同一台机器上直接横向比较——这是做性能实验的正确方式：**控制变量**（相同 N、相同 block 大小、相同数据、相同迭代次数、相同计时方式）。
- **版本 1** 与示例 1 相同：`dimGrid = ceil(N/256) = 39063`，每线程一个元素，多出的 128 个线程被 `if (i < N)` 拦住。
- **版本 2** 的循环：`for (int i = tid; i < N; i += stride)`，其中 `stride = blockDim.x * gridDim.x`。网格大小改为 `numSMs × blocksPerSM`（A100 上是 `108 × 8 = 864`），每个线程平均执行 `10,000,000 / (864 × 256) = 45.2` 轮。循环条件 `i < N` 同时承担了边界检查的职责——**网格-步长循环天然是边界安全的**，因为它不会启动越界线程，只在处理越界索引时退出。
- **版本 3** 在主循环里一次处理 `UNROLL = 4` 个元素，地址分别相隔 `1×stride`、`2×stride`、`3×stride`。**关键细节：间隔取 `stride` 而不是 `blockDim.x`**。若用 `blockDim.x` 作为间隔，那么同一线程的 4 个元素在内存里相邻，但相邻线程的访问会互相错开，覆盖率会出现漏洞（例如 grid=2、block=2、UNROLL=2 时会漏掉索引 6 和 7），同时"一个 block 的 4 个块"也会与相邻 block 的区间重叠。用 `stride` 作间隔则保证：每一轮里**每个访问各自都是 warp 级连续的**，而且全体线程的访问集合恰好覆盖 `[0, N)` 一次。
- **尾部处理**：主循环的守卫条件 `i + (UNROLL-1)*stride < N` 保证 4 个地址全部在界内；一旦不成立就退出主循环，由后面的 `for (; i < N; i += stride)` 用单元素方式收尾。这两段循环的覆盖集合是不重叠的：主循环覆盖的是"整组的 UNROLL 个 stride 间隔"，尾部循环从同一个 i 出发继续以 stride 步进，正好接上。
- **网格大小由设备决定**：`cudaOccupancyMaxActiveBlocksPerMultiprocessor` 返回"每个 SM 能同时驻留几个这个 kernel 的 block"，乘以 `prop.multiProcessorCount` 就是"一次性能装满整个 GPU"的网格大小。`cudaOccupancyMaxPotentialBlockSize` 则反向推荐一个能最大化占用率的 block 大小（本例中若推荐值不在 128–512 的合理区间就回退到 256，因为 1024 这类大 block 会带来尾部浪费和阶梯效应）。
- **`__restrict__` 的作用**：三个指针都标注了 `__restrict__`，向编译器承诺"这三块内存不会互相重叠"。这让编译器可以放心地把 `A[i]`、`B[i]` 的加载**提前并重排**（不需要担心 `C[i]` 的写入会改变 `A[i+1]` 的值），是让 UNROLL 的四条 `LDG` 真正同时在途的必要条件。**没有 `__restrict__` 时，编译器必须保守地假设写 C 可能影响后面的读 A，从而把访存串行化。**
- **`verify()` 用逐位比较**：每个输出元素都是一次独立的 FP32 加法，没有累加顺序问题，所以三个版本的结果都必须与 CPU `==` 相等。任何不匹配都说明映射或边界处理有 bug。
- **【并行机制与硬件映射解说】**

（a）三个版本的 warp 组织完全相同：block 大小都是 256 → 每 block 8 个 warp；warp 内线程仍然按 `threadIdx.x` 连续划分，所以**每个单独的访存指令依然是 32 个连续地址 = 1 次 128 字节事务**。粗化并没有破坏合并性——它只是让每个 warp 每轮发出 4 条这样的 `LDG`，而不是 1 条。

（b）粗化如何提高访存并行度（MLP）：版本 3 的 4 条 `LDG` 之间没有任何数据依赖（4 个地址由 `i` 和常量偏移算出，互不相关），所以硬件可以在等待第一条数据返回之前就把 4 条都发出去。每条 `LDG` 携带 128 字节（一个 warp），于是**每个 warp 每轮有 512 字节在途**。整个 warp 能同时容纳的 `LDG` 数量受"每 SM 的 LSU（load/store unit）队列深度"限制（Ampere 上每 SM 在途访存请求上限是数百个），4 条远未触顶。

（c）占用率与资源：A100 每 SM 2048 线程 / 64 warp / 65536 寄存器 / 32 个 block 上限。 – 版本 1、2：约 16 寄存器/线程 → `65536/(16×256) = 16` 个 block（寄存器不限制），线程限制 `2048/256 = 8` 个 block → **8 block = 64 warp = 100%**。 – 版本 3：8 个活跃 float（`a0..a3`、`b0..b3`）加索引，实测约 24 寄存器/线程 → `65536/(24×256) = 10` 个 block（寄存器不限制），线程限制 8 → **8 block = 64 warp = 100%**。 三个版本的占用率都是 100%，所以本次对比**不是**在做"占用率 vs 带宽"的实验，而是在验证"在占用率相同的前提下，粗化是否还能带来收益"。答案是：在这个 N 和这台 GPU 上，**三者带宽基本相同**（都接近 DRAM 上限），说明内存系统已经完全饱和。

（d）共享内存 / bank conflict：三个 kernel 都不使用共享内存，无 bank conflict。

（e）寄存器与 warp 发散：版本 3 的尾部循环 `for (; i < N; i += stride)` 在最后一个 warp 里可能出现部分线程退出、部分线程继续的情况（因为 N 不一定被 stride 整除），这会造成轻微的 warp 发散。但由于它只在最后一轮发生一次，代价可以忽略。真正需要注意的是：**粗化把"边界处理"从"最多一个 warp 发散"扩大到了"最多 UNROLL 个 warp 发散"，仍然是一次性开销。**

- **【性能优化分析】**

（1）实测对比（A100，N = 10,000,000，block = 256，20 次迭代取平均） `text`

```
┌──────────────────────────────┬──────────┬──────────┬──────────┬──────┬──────┐
│ 版本                         │ 耗时(us) │ 带宽(GB/s) │ 峰值占比 │ 寄存器 │ 占用率 │
├──────────────────────────────┼──────────┼──────────┼──────────┼──────┼──────┤
│ 1 基础(1 元素/线程)          │ 96.5     │ 1244     │ 80.0%    │ 16   │ 100.0% │
│ 2 网格-步长(1 元素)          │ 96.0     │ 1250     │ 80.4%    │ 16   │ 100.0% │
│ 3 网格-步长+粗化(4 元素)     │ 95.2     │ 1261     │ 81.1%    │ 24   │ 100.0% │
└──────────────────────────────┴──────────┴──────────┴──────────┴──────┴──────┘
```

差距在 1.4% 以内 —— 三者都已打满显存带宽，剩下的 19% 是 DRAM 物理开销。 这个结果本身就是本讲最重要的实验结论之一：**当一个 kernel 已经是纯带宽受限时，任何"提高并行度"的优化都不会有可观测的收益**。你能做的只有减少字节数。

（2）粗化的真正价值：用 ILP 换占用率

粗化的收益在**寄存器压力大、占用率被迫降低**的 kernel 上才会显现。构造一个具体场景：假设 kernel 每线程用 80 个寄存器（例如每线程要维护更多中间量）。 `text 版本 2（UNROLL = 1）在 80 寄存器、block = 256 时： 每 SM 可驻留 block 数 = floor(65536 / (80 x 256)) = floor(3.2) = 3 驻留线程 = 3 x 256 = 768 = 24 warp 占用率 = 24 / 64 = 37.5% 每 SM 在途字节 = 24 warp x 1 个访问 x 128 B = 3,072 B 打满带宽需要每 SM 约 4,104 B（Little's Law）： 可得带宽 ≈ 1250 x (3072 / 4104) = 936 GB/s -> 只有可达带宽的 75% 版本 3（UNROLL = 4）在同样的 80+8 寄存器假设下： 每 SM 在途字节 = 24 warp x 4 个访问 x 128 B = 12,288 B（需要的 3 倍） -> 带宽重新回到 1250 GB/s 附近` 这就是**"低占用率 + 高 ILP ≡ 高占用率 + 低 ILP"**的定量版本，也是现代 GPU kernel（FlashAttention 一类）普遍采用"少量线程、大量寄存器、强 ILP"风格的原因。

（3）算术强度与 Roofline（三版本相同） `text AI = 1 FLOP / 8 B = 0.125，或含写为 1/12 = 0.0833 FLOP/Byte A100 平衡点 = 12.54 FLOP/Byte -> 带宽受限 Roofline 天花板 = 0.0833 x 1555 = 129.6 GFLOP/s 版本 3 实测算力 = 0.0833 x 1261 = 105.0 GFLOP/s = 天花板的 81.0% 向量加法的时间下限（不可突破）：120 MB / 1555 GB/s = 77.2 us`

（4）瓶颈判定与优化方向

三个版本都是**带宽受限 + 延迟隐藏充分**。因此优化方向只剩： – **减少字节数**（唯一有大收益的方向）：把 `float` 换成更窄的类型（如果精度允许）、避免不必要的中间数组、融合多个 pass。 – **使用向量化访存** `float4`：每线程处理 4 个**相邻**float（而不是相隔 stride 的 4 个），一条 `LDG.128` 就搬运 16 字节/线程 = 512 字节/warp（4 次事务）。它把索引计算指令数降到 1/4，在算术强度极低的 kernel 上通常能带来 1%–3% 的提升（本 kernel 收益很小，因为指令发射不是瓶颈）。 – **不做**：提高占用率（已 100%）、增加 unroll（已饱和）、使用共享内存（无复用，纯增开销）。

示例 3：二维网格矩阵加法 matrixAdd（二维线程映射）

```
// 文件: matrixadd_2d.cu
// 编译: nvcc -O3 -arch=sm_80 matrixadd_2d.cu -o matrixadd_2d
// 运行: ./matrixadd_2d 4096 4096
//
// 内容: 用二维 grid / 二维 block 处理二维数据 (C = A + B, 矩阵加法)。
//       展示 row-major 线性化、二维边界检查、以及"哪个维度该当 threadIdx.x"。

#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define TILE 32          // block 形状 32 x 32 = 1024 线程

// 二维线程映射: threadIdx.x 对应"列" (row-major 中最快变化的维度),
// 这样同一个 warp 的 32 个线程访问同一行里连续的 32 个 float = 128 字节。
__global__ void matrixAddKernel(const float* __restrict__ A,
                                const float* __restrict__ B,
                                float* __restrict__ C,
                                int width, int height)
{
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (col < width && row < height) {              // 两个维度都要检查
        int idx = row * width + col;                // row-major 线性化
        C[idx] = A[idx] + B[idx];
    }
}

static void initMatrix(float* m, long n, unsigned int seed)
{
    unsigned int x = seed;
    for (long i = 0; i < n; ++i) {
        x = x * 1664525u + 1013904223u;
        m[i] = (float)(x >> 8) / (float)(1u << 24) - 0.5f;
    }
}

int main(int argc, char** argv)
{
    int width  = (argc > 1) ? atoi(argv[1]) : 4096;
    int height = (argc > 2) ? atoi(argv[2]) : 4096;
    if (width <= 0 || height <= 0) {
        fprintf(stderr, "width/height must be positive\n");
        return EXIT_FAILURE;
    }
    long N = (long)width * (long)height;
    size_t bytes = (size_t)N * sizeof(float);

    float* h_A   = (float*)malloc(bytes);
    float* h_B   = (float*)malloc(bytes);
    float* h_C   = (float*)malloc(bytes);
    float* h_Ref = (float*)malloc(bytes);
    if (!h_A || !h_B || !h_C || !h_Ref) {
        fprintf(stderr, "host malloc failed\n");
        return EXIT_FAILURE;
    }
    initMatrix(h_A, N, 12345u);
    initMatrix(h_B, N, 67890u);

    float *d_A = NULL, *d_B = NULL, *d_C = NULL;
    CUDA_CHECK(cudaMalloc((void**)&d_A, bytes));
    CUDA_CHECK(cudaMalloc((void**)&d_B, bytes));
    CUDA_CHECK(cudaMalloc((void**)&d_C, bytes));
    CUDA_CHECK(cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice));

    // 二维网格: 每个维度都要向上取整
```

```
    dim3 dimBlock(TILE, TILE, 1);
    dim3 dimGrid((unsigned)((width  + TILE - 1) / TILE),
                 (unsigned)((height + TILE - 1) / TILE), 1);

    matrixAddKernel<<<dimGrid, dimBlock>>>(d_A, d_B, d_C, width, height);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    const int iters = 20;
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    CUDA_CHECK(cudaEventRecord(start));
    for (int it = 0; it < iters; ++it) {
        matrixAddKernel<<<dimGrid, dimBlock>>>(d_A, d_B, d_C, width, height);
    }
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));
    CUDA_CHECK(cudaGetLastError());
    float msTotal = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&msTotal, start, stop));
    double msPerIter = (double)msTotal / iters;

    CUDA_CHECK(cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost));
    for (long i = 0; i < N; ++i) { h_Ref[i] = h_A[i] + h_B[i]; }
    long mismatch = 0;
    for (long i = 0; i < N; ++i) { if (h_C[i] != h_Ref[i]) { ++mismatch; } }

    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    double peakGBs = 2.0 * (double)prop.memoryClockRate * 1.0e3
                   * ((double)prop.memoryBusWidth / 8.0) / 1.0e9;
    int blocksPerSM = 0;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &blocksPerSM, matrixAddKernel, TILE * TILE, 0));

    double totalBytes  = 3.0 * (double)bytes;
    double achievedGBs = totalBytes / (msPerIter * 1.0e-3) / 1.0e9;

    printf("================= 矩阵加法 %d x %d =================\n", width, height);
    printf("GPU                  : %s, 峰值带宽 %.1f GB/s\n", prop.name, peakGBs);
    printf("blockDim             : (%d, %d), dimGrid = (%u, %u)\n",
           TILE, TILE, dimGrid.x, dimGrid.y);
    printf("总线程数             : %u (每个线程一个元素)\n",
           dimGrid.x * dimGrid.y * TILE * TILE);
    printf("正确性验证           : %s (不匹配 %ld / %ld)\n",
           mismatch == 0 ? "PASS" : "FAIL", mismatch, N);
    printf("kernel 单次耗时      : %.3f us\n", msPerIter * 1000.0);
    printf("有效带宽             : %.1f GB/s (%.1f%% of %.1f GB/s 峰值)\n",
           achievedGBs, 100.0 * achievedGBs / peakGBs, peakGBs);
    printf("每 SM 驻留 block 数   : %d (%d 线程/SM)\n",
           blocksPerSM, blocksPerSM * TILE * TILE);
    printf("占用率               : %.1f %%\n",
           100.0 * blocksPerSM * TILE * TILE / prop.maxThreadsPerMultiProcessor);
    printf("每 warp 事务数       : 3 (A/B/C 各 1 次 128 B 事务)\n");

    CUDA_CHECK(cudaFree(d_A));
    CUDA_CHECK(cudaFree(d_B));
    CUDA_CHECK(cudaFree(d_C));
    free(h_A); free(h_B); free(h_C); free(h_Ref);
    CUDA_CHECK(cudaDeviceReset());
    return (mismatch == 0) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

- **【代码做什么？】**
- **二维映射**：`col = blockIdx.x * blockDim.x + threadIdx.x`、`row = blockIdx.y * blockDim.y + threadIdx.y`。这是把一维公式分别应用到两个维度上，两个维度**互相独立**。
- **row-major 线性化**：C/C++ 的二维数组在内存里按行连续存放，元素 `(row, col)` 的线性地址是 `row * width + col`（讲义中的例子：`M[2][1]` 在 4 列矩阵里编号为 `2*4+1 = 9`）。因此"行"是慢变维度、"列"是快变维度。
- **哪个维度当 `threadIdx.x`**：必须让**快变（列）维度对应 `threadIdx.x`**，因为硬件按 `threadIdx.x` 最快线性化来划分 warp。这样同一个 warp 的 32 个线程访问的是同一行里连续的 32 个 float = 128 连续字节 = 1 次事务。如果反过来（`col` 对应 `threadIdx.y`），warp 内 32 个线程会跨越 32 行，每行取 4 字节，步长 `width*4 = 16384` 字节，事务数从 1 变成 32，带宽利用率掉到 3.1%。

- **二维边界检查**：`if (col < width && row < height)`。维度必须**分别**检查——只有一维需要检查时另一维可以省，但两个维度都写了更安全。讲义中的例子（76×62 图像用 16×16 的 block）需要 `ceil(76/16)=5` 列 × `ceil(62/16)=4` 行 = 20 个 block，覆盖 80×64，多出的区域全靠这两个条件拦住。
- **网格配置**：`dimGrid = (ceil(width/TILE), ceil(height/TILE), 1)`，两个维度都向上取整；`dimBlock = (TILE, TILE, 1)`，本程序 TILE = 32，即每个 block 1024 线程。
- **主机端流程**与示例 1 相同（分配 → 拷贝 → 预热 → 计时 → 取回 → 用 CPU 参考实现逐位验证 → 打印带宽与占用率 → 释放）。
- **【并行机制与硬件映射解说】**

（a）warp 划分：block 形状 32×32，`threadIdx` 的线性化顺序是 x 最快、y 次之、z 最后。所以 warp 0 = `threadIdx.y = 0, threadIdx.x = 0..31`（第 0 行），warp 1 = `threadIdx.y = 1, threadIdx.x = 0..31`（第 1 行），依此类推，一个 block 恰好 32 个 warp，每个 warp 对应一整行。

（b）A100 上的驻留量：每 SM 2048 线程 / 64 warp。block 大小 1024 → `2048/1024 = 2` 个 block（64 warp）→ **100% 占用率**；寄存器约 16 个/线程 → `65536/(16×1024) = 4` 个 block，不构成限制。**但在 RTX 4090（1536 线程/SM）上，`1536/1024 = 1` 个 block → 1024 线程 = 32 warp → 只有 66.7% 占用率**。这是"大 block 的阶梯效应"的典型例子：同样是 1024 线程的 block，在不同 GPU 上占用率相差 33%。稳健的选择是 **16×16 = 256 线程的 block**，它在 1536、2048 两种 SM 配置下都能整除并达到 100%。

（c）全局内存访问是否合并：以矩阵 `4096×4096` 为例，`width = 4096`。block (0,0) 的 warp 0 处理 `row = 0, col = 0..31`，线性索引 `idx = 0*4096 + 0..31 = 0..31`，字节地址 0..127。基地址 256 字节对齐 → **1 次 128 字节事务，利用率 100%**。warp 1 处理 `row = 1, col = 0..31`，`idx = 4096..4127`，字节地址 16384..16511，同样是 1 次对齐的事务。**每个 warp 3 次事务（A、B、C 各一次），与一维向量加法完全一致**——二维映射本身不影响合并性，影响合并性的是"哪个维度最快变化"。

（d）共享内存 / bank conflict：不使用共享内存，无 bank conflict。矩阵加法同样**没有数据复用**（每个元素只被读一次），共享内存是纯开销。

（e）寄存器与发散：约 16 个寄存器/线程（两个坐标、一个 width、一个 idx、三个数据）。**发散很轻微**：只有落在 `col >= width` 或 `row >= height` 区域的 warp 会发散。4096 是 32 的整数倍，所以本次运行完全没有发散；若换成 `4096×4094`，则每行的最后一个 warp 会有 30 个线程空转（因为 y 方向有 2 行是越界的，整个 warp 都不活跃）。用 16×16 的 block 时，越界区域是"半个 warp 的粒度"（因为一个 warp 只覆盖一行 16 个线程的两行），发散开销比 32×32 更细。

- **【性能优化分析】**

（1）算术强度与 Roofline `text 每个元素：1 FLOP（一次加法），访存 读 4 B + 读 4 B + 写 4 B = 12 B AI = 1 / 12 = 0.0833 FLOP/Byte（含写）；只算读时 = 1/8 = 0.125 FLOP/Byte A100 平衡点 = 19.5e12 / 1555e9 = 12.54 FLOP/Byte AI 比平衡点低 100 倍以上 -> 纯带宽受限，与向量加法结论完全一致 数据规模：4096 x 4096 x 4 B = 64 MiB/矩阵，总流量 192 MiB = 201.3 MB 时间下限 = 201.3e6 / 1555e9 = 129.5 us 实测（按 1250 GB/s）= 201.3e6 / 1250e9 = 161.0 us 对应算力 = 0.0833 x 1250e9 = 104 GFLOP/s = FP32 峰值的 0.53% 注意：把矩阵加法当成一个"更大的向量加法"是完全正确的——row * width + col 只是一个仿射变换，访问模式依然是纯顺序的。**矩阵加法比向量加法多出来的唯一东西是"索引计算的指令数"和"两个维度各自的边界检查"**，这些都属于指令开销，不是带宽开销。

（2）占用率 `text block = 32 x 32 = 1024 线程 A100 : floor(2048/1024) = 2 block/SM -> 2048 线程 = 64 warp -> 100.0% RTX4090: floor(1536/1024) = 1 block/SM -> 1024 线程 = 32 warp -> 66.7% 改用 16 x 16 = 256 线程： A100 : floor(2048/256) = 8 block/SM -> 2048 线程 -> 100.0% RTX4090: floor(1536/256) = 6 block/SM -> 1536 线程 -> 100.0%` 这就是讲义中那个经典分析（"1536 线程 / 最多 8 block 的 SM 上，8×8、16×16、32×32 哪个最好"）的推广：**16×16 是跨硬件最稳健的二维 block 形状**，8×8 会被"每 SM block 数上限"卡住（例如上限 8 时只能放 8×64 = 512 线程 = 25% 的可用线程容量），32×32 会被"每 SM 线程数上限"卡住。

（3）瓶颈判定与优化方向 瓶颈 = **带宽受限**（AI 0.0833 ≪ 12.54），延迟隐藏充分（A100 上 100% 占用率，或在 4090 上用 16×16 时 100%）。 `text 每 SM 在途字节（A100, 32x32 block, 2 block/SM = 64 warp）： 64 warp x 1 个在途访问 x 128 B = 8,192 B >= 需要的 4,104 B（2 倍余量） 这是否足够？ 是。所以矩阵加法在 A100 上同样能达到约 80% 的峰值带宽。` 优化方向： – 二维 kernel 里**把 `width`、`height` 传成 `int` 是够的**（4096 不会溢出），但如果规模可能超过 2^31 个元素，`row * width + col` 必须用 `long long`。这是二维 kernel 最常见的溢出点。 – 如果不需要二维语义（比如整个矩阵统一加一个常数），**直接用一维映射**：更少的索引指令、更简单的边界检查，实测通常快 1%–2%。 – 只有在需要**邻域访问**（模糊、卷积、stencil）时，二维 block 的形状才真正重要——那时需要共享内存来做 halo 交换，block 形状会影响 halo 占比和 bank conflict。

Lab 1 的工程习惯与评分口径

（一）三条必须养成的工程习惯

```
① CPU 端 golden reference —— "没有参考实现的性能数字毫无意义"
   - 在 CPU 上用最朴素的双重循环/单重循环算一遍，作为标准答案。
   - 对欧拉式 kernel（每元素独立、无累加），结果应当逐位相同（bit-exact），
     可以直接用 == 比较；对累加/reduction 类 kernel 才需要 epsilon。
   - 在 GPU 上跑出"更快"的结果但没验证 = 得到一个错误的数字，Lab 直接 0 分。
   - 建议把验证做成函数 verify()，在任何性能测量之前先跑一遍。

② 边界检查 —— "最后一个 block 的越界线程一定会捅娄子"
   - 网格向上取整：dimGrid = (N + blockDim - 1) / blockDim（整数运算）。
   - kernel 里：if (i < N) { C[i] = A[i] + B[i]; }，二维时两个维度分别检查。
   - 测试用例要专门包含 N 不是 blockDim 整数倍的情况
     （例如 N = 1000、N = 1025、N = 1），这些正是能暴露 bug 的输入。

③ CUDA 错误检查 —— "静默失败是 CUDA 的默认行为"
   - 每一个 CUDA API 调用都用 CUDA_CHECK 宏包起来。
   - kernel 启动后用 cudaGetLastError() 检查启动期错误（非法配置、越界等）。
   - 计时循环之后必须 cudaDeviceSynchronize()（或 cudaEventSynchronize），
     否则测到的是"启动 kernel 的时间"而不是"kernel 执行的时间"。
   - 强烈建议在 debug 构建里跑一次 cuda-memcheck（或 compute-sanitizer），
     它能直接指出越界访问的地址和线程号。
```

```
开发循环（推荐的调试顺序）

写 kernel
   -> 用 N = 1、N = 31、N = 33、N = 1024、N = 1025 五个小规模跑通（验证边界）
   -> 与 CPU 参考实现逐位对比，全部 PASS
   -> 再放大到 N = 10^6 / 10^7 测性能
   -> 用 cudaEvent 只圈住 kernel，迭代 >= 20 次取平均（消除抖动）
   -> 先预热 1 次，丢弃第一次的测量
   -> 报告"达到峰值带宽的百分比"，而不是裸的毫秒数
切忌：先调性能，后补正确性。这样做的学生最后往往两者都没有。
```

（二）评分口径（课程 Lab 的一般形式）

```
┌──────────────────────────────────────────────────────────┐
│ 第一部分：正确性测试（通常占大头，不正确则性能分无效）                  │
│   - 用课程提供的 harness 把学生写的 .cu 与参考答案对比                 │
│   - 多个规模：小规模（暴露边界 bug）、中等规模、大规模（暴露溢出）      │
│   - 多个数据分布：随机值、全零、极端值（验证数值稳定性）               │
│   - 容差：整数/单次加法类要求精确；归约类允许相对误差 1e-5 量级        │
│   - 常见的"部分分"设计：只处理 N 为 blockDim 整数倍的情况得部分分      │
├──────────────────────────────────────────────────────────┤
│ 第二部分：性能测试（在指定 GPU 上测，通常是 NCSA Delta 的 A100）       │
│   - 指标一般是有效带宽（GB/s）或相对基线 kernel 的加速比               │
│   - 常见门槛：达到理论峰值带宽的 70%–85%（向量加法类）                 │
│   - 计时只圈 kernel，不含 cudaMemcpy；多次迭代取平均                   │
│   - 有的版本会给出不同 N 的曲线，考察"随规模变化的趋势"的解释          │
├──────────────────────────────────────────────────────────┤
│ 第三部分：实验报告（不少版本要求提交）                                 │
│   - 说明线程映射公式、网格大小计算、边界处理的正确性论证               │
│   - 给出带宽测量方法（如何排除传输、如何预热、迭代次数）               │
│   - 用算术强度 / Roofline / 占用率解释"为什么这个 kernel 是带宽受限"   │
│   - 讨论：为什么继续优化（更多 unroll、更大 block）收益很小            │
└──────────────────────────────────────────────────────────┘
注：以上是课程 Lab 的一贯口径的概括，具体门槛以当学期 handout 为准。
```

性能优化技巧总结

1. **用 `i = blockIdx.x * blockDim.x + threadIdx.x` 做最简映射**：它只需一条 IMAD 指令，且天然使相邻线程访问相邻地址——映射公式本身就是合并访存的前提。
2. **网格大小一定向上取整**：`(N + blockDim.x - 1) / blockDim.x`。向下取整会漏掉尾部元素，是 Lab 1 最常见的丢分点。
3. **kernel 内必须做边界检查**：越界写会静默破坏其它内存；二维问题两个维度都要检查。
4. **让 `threadIdx.x` 对应内存中变化最快的维度**：warp 按 `threadIdx.x` 最快线性化，这样 32 个线程的访问才是 128 连续字节 = 1 次事务。
5. **用 `__restrict__` 标注指针**：承诺"这些指针不重叠"，编译器才敢把多条访存提前并重排，粗化/UNROLL 才能真正形成在途并行。
6. **block 大小选 128/256/512，别默认用 1024**：1024 会在 1536 线程/SM 的 GPU 上因阶梯效应只能放 1 个 block（占用率掉到 66.7%），且尾部浪费比例更大。
7. **用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 实测占用率**，不要凭寄存器数猜——寄存器、共享内存、block 数上限三个约束取最小值才是真相。

8. **用"每 SM 在途字节 = 带宽 × 延迟 / SM 数"检查延迟隐藏是否足够**（A100 约 4.1 KB/SM）：不够就提高占用率，或每线程多发几个独立访存（thread coarsening）。
9. **粗化时让同线程的多个访问地址相隔 `stride`（网格总线程数），而不是相隔 `blockDim.x`**：这样每一个访问各自仍然合并，且覆盖率不会出现漏洞。
10. **只圈 kernel 计时**：`cudaEvent` 包住 kernel 循环，排除 `cudaMemcpy`；预热一次；迭代 20 次以上取平均。
11. **报告"峰值带宽百分比"而不是绝对毫秒**：绝对时间随 GPU 型号变化，百分比才说明你的 kernel 有没有打满带宽。
12. **减少字节数永远比提高并行度有效**：当 kernel 已带宽受限时，任何占用率/ILP 优化都不会有可观测收益，唯一的出路是少搬数据（或提升数据复用——那是共享内存的领域）。

关键要点

- **线程映射公式 `i = blockIdx.x * blockDim.x + threadIdx.x` 是数据并行的全部骨架**：它把"第 b 个 block 的第 t 号线程"唯一地映射到一个数据元素，使硬件可以完全自主地调度 block（block 之间的执行顺序是任意的、不确定的，所以绝不能让 block 之间互相依赖）。
- **一个 warp 的 32 次 4 字节访问，如果地址连续且 128 字节对齐，会被合并成 1 次 128 字节事务（利用率 100%）；如果跨步到 32 个不同的 cache line，就是 32 次事务（利用率 3.1%，sector 粒度下也是 8 倍放大）。合并与否直接决定带宽的有效利用率，是最重要的性能分水岭。**
- **向量加法的算术强度只有 0.125 FLOP/Byte（含写为 0.0833），而 A100 的机器平衡点是 12.54 FLOP/Byte，低了 100 倍**：它必然、也只能是纯带宽受限。它的时间下限是 `3 × N × 4 字节 / 峰值带宽`，N = 10^7 时是 77.2 μs；实测 96 μs（1.25 TB/s，峰值的 80.4%）已经接近极限，FP32 流水线只有约 0.5% 的周期在工作。
- **占用率用"活跃 warp 数 / SM 最大 warp 数"计算，由线程数上限、寄存器上限、block 数上限三者取最小值决定**：A100 上 2048 线程 / 64 warp / 65536 寄存器 / 32 block。轻量 kernel（16 寄存器）在 128–1024 的任何 block 大小下都能到 100%，但寄存器用量大的 kernel 会被寄存器卡住（80 寄存器 + 256 线程 → 只有 3 个 block → 37.5%）。
- **占用率的意义是"藏住 400–800 周期的全局内存延迟"**：Little's Law 给出量化标准——每 SM 需要约 4.1 KB 在途字节（A100）。64 warp × 1 个在途合并访问 = 8 KB（2 倍余量，够用）；16 warp × 1 = 2 KB（只够一半带宽）；而 16 warp × 4 个独立访存 = 8 KB（又够了）——这就是**线程粗化用 ILP 换占用率**的定量依据。
- **网格-步长循环把"网格大小"与"问题规模"解耦**：网格由设备（SM 数 × 每 SM 可驻留 block 数）决定，每线程循环处理多个元素，因而同一份代码能自适应任意 GPU、任意 N、多 GPU 划分和运行时才知道的规模，同时消除波次量化（wave quantization）的尾部浪费和大量 block 的调度开销。

常见陷阱与注意事项

- **整数除法取整导致网格覆盖不足**：`dimGrid = N / blockDim.x` 在 N 不是 blockDim.x 整数倍时漏算最后的元素（N = 10,000,000 时漏 128 个，正确性测试直接失败）→ 改用 `(N + blockDim.x - 1) / blockDim.x`，并用 `long long` 计算分子避免 `N + B - 1` 溢出。
- **忘记边界检查（或只检查一个维度）导致越界访问**：`if (i < N)` 漏写时，最后一个 block 的多余线程会读写自己数组之外的内存（可能写坏相邻分配甚至 CUDA 内部结构），表现为"时对时错"或延迟崩溃，用 `compute-sanitizer` 才能定位；二维 kernel 里只写 `if (col < width)` 而遗漏 `row < height`，在高度不是 blockDim.y 整数倍时同样越界 → 网格向上取整与边界检查是一对，不能只写一半；二维问题的越界区域在行、列两个方向都存在，必须分别检查。附带一点：`if (i < N)` 本身会在跨界的那一个 warp 里造成 **warp 发散（branch divergence）**——warp 内一部分线程走 then 分支、一部分走 else 分支，硬件必须**串行执行两条路径**——但由于它只在每个 kernel 的最后一轮发生一次，代价可以忽略；真正要避免的是"用 `threadIdx` 做条件去划分大量工作"（例如 `if (threadIdx.x > 2) { 大量代码 } else { 大量代码 }`），正确做法是让分支粒度对齐 warp 大小，写成 `if (threadIdx.x / 32 > 2)`，这样任何 warp 都只走一条路径。
- **忘记 `cudaDeviceSynchronize()` / 未同步就测量**：kernel 启动是异步的，在 kernel 后立刻读主机内存或用 CPU 时钟计时，测到的是启动开销而不是执行时间，还会读到未完成的结果 → 用 `cudaEvent` 计时并在 `cudaEventSynchronize` 之后读 `cudaEventElapsedTime`；验证结果前也必须同步。
- **忘记检查 CUDA API 返回值与 kernel 启动错误**：`cudaMalloc` 失败（显存不足）、`<<<>>>` 配置非法（block 超过 1024、共享内存超出）都会返回错误码但不中断程序，后续代码在一片未初始化的内存上运行 → 所有 API 用 `CUDA_CHECK` 包裹；kernel 启动后立刻 `cudaGetLastError()`。
- **host/device 指针混用与 `cudaMemcpy` 方向写反**：把主机 `malloc` 得到的指针直接传给 kernel 会触发非法内存访问并报 "an illegal memory access was encountered"（一旦出现这个错误，整个 CUDA context 都会被污染，后续所有调用都失败）；把 `cudaMemcpyHostToDevice` 与 `cudaMemcpyDeviceToHost` 弄混则会把设备内存的垃圾拷到主机 → 设备端内存必须用 `cudaMalloc`（或 `cudaMallocManaged`），主机端不能解引用 `cudaMalloc` 返回的指针；记住 `cudaMemcpy` 的参数顺序是 `(dst, src, count, kind)`，`kind` 描述的是"数据从哪里来"。
- **跨步（strided）访问破坏合并性**：为了"看起来更整齐"而写 `i = threadIdx.x * gridDim.x + blockIdx.x`（按列遍历），会让一个 warp 的 32 个线程访问相隔 156 KB 的地址，事务数从 1 涨到 32（sector 粒度下流量放大 8 倍），实测带宽掉到 1/8 → 始终让 `threadIdx.x`（线性化最快的维度）对应连续内存；`__restrict__` 与向量化都不能弥补这个问题。
- **在已饱和的带宽受限 kernel 上继续优化并行度**：把 unroll 从 4 加到 8、把 block 从 256 改到 1024、加 `__launch_bounds__` 提高占用率，在向量加法上都不会有可观测收益（三版本实测差距在 1.4% 以内）→ 先用算术强度和 Roofline 判断瓶颈类型，带宽受限时只做"减少字节数"和"保证合并"这两件事。
- **粗化时用 `blockDim.x` 而不是 `stride` 作为线程内多元素的间隔**：会让不同线程的覆盖区间互相重叠或出现空洞（测试 N 不是 stride 整数倍时结果错误且难以察觉）→ 同一个线程的多个元素地址必须相隔 `blockDim.x * gridDim.x`，这样每个访问各自合并、整体恰好无重复无遗漏地覆盖 `[0, N)`。

思考题（带答案）

Q1. 一个 1D 数组有 N = 1,000,000 个 float，用 blockDim = 256、每线程一个元素的向量加法处理。请给出网格大小、启动的线程总数、多出的空转线程数、一个 warp 的访存事务数、总事务数，以及在 A100 上打满峰值带宽所需的最短时间。

网格大小 = `ceil(1,000,000 / 256) = 3907` 个 block；启动线程总数 = `3907 × 256 = 1,000,192`，多出 `192` 个空转线程（占 0.019%）。每个 warp（32 个连续 float）对 A、B、C 各有 1 次 128 字节事务，即 **3 次/warp**；有效 warp 数 = `1,000,000 / 32 = 31,250`，**总事务数 = 31,250 × 3 = 93,750 次**，共搬运 `93,750 × 128 B = 12,000,000 B = 12 MB`。A100 峰值带宽 1555 GB/s，最短时间 = `12e6 / 1555e9 = 7.72 µs`。注意这个规模太小，kernel 的 launch 开销（约 3–5 µs）已经与之同量级，这也是为什么性能实验要用 N = 10^7 以上。

Q2. 某 kernel 中线程 t 访问 `A[(t % 32) * 4096 + (t / 32)]`（float 数组，t 为 warp 内线程号 0..31）。请分析其访存效率；如果这是 Lab 1 的向量加法 kernel，实际带宽会变成多少（设 A100 可达带宽 1250 GB/s）？

warp 内相邻线程（t 和 t+1）的地址相差 4096 个 float = 16,384 字节。32 个线程访问的地址就是 `4096 × t`（t 取 0 到 31），即元素下标 0、4096、8192、12288 一直到 126976，也就是 32 个彼此相隔 16 KB 的地址。每个地址落在不同的 128 字节 cache line（也落在不同的 32 字节 sector），所以**事务数 = 32 次/warp**，用 sector 粒度算是 `32 × 32 B = 1024 B` 搬回 128 B 有用数据，**流量放大 8 倍**。实际带宽 ≈ `1250 / 8 = 156 GB/s`，只有原来的 12.5%，`if (i < N)` 也拦不住这种错误（它是地址映射错误，不是越界）。

Q3. A100 的 FP32 峰值是 19.5 TFLOPS、显存带宽 1555 GB/s。有人提出"把向量加法改写成一个每元素做 16 次浮点运算的 kernel（例如同时做一次多项式求值）"，声称这样能"更好地利用 GPU 的算力"。请判断这个说法，并算出这个 kernel 在新算术强度下是带宽受限还是计算受限。

原来的 AI = 1 FLOP / 8 B = 0.125 FLOP/Byte。新 kernel 每元素 16 FLOP，流量仍是 8 B（读 A、B）或 12 B（含写 C）：`AI = 16 / 12 = 1.333 FLOP/Byte`。A100 的机器平衡点 = `19.5e12 / 1555e9 = 12.54 FLOP/Byte`。`1.333 < 12.54`，所以**它仍然是带宽受限的**，Roofline 天花板 = `1.333 × 1555e9 = 2.07 TFLOP/s`，仅占 FP32 峰值的 10.6%。要想让它在 A100 上变成计算受限，需要 `AI ≥ 12.54`，即每元素至少 `12.54 × 12 ≈ 150` 次浮点运算。这个例子说明：**"多算一点"几乎不可能让一个访存密集的 kernel 变成计算密集的——要跨越 100 倍的算术强度鸿沟，必须改变数据复用方式（分块、共享内存），而不是增加每元素的运算次数。**

Lecture 4: 并行模式二 —— 基础矩阵乘法及其性能瓶颈 (对应 Lab 2: Simple Matrix Multiply)

概述

本讲把"并行模式一"（向量加法那样的 element-wise 模式）推广到第一个真正有计算量的应用：矩阵乘法。Lab 2 的 Simple Matrix Multiply 不允许使用共享内存，只允许"每线程一个输出元素（thread-per-output）"的映射，因此它是理解 GPU 访存代价最好的反面教材。本讲先用 M×K 乘 K×N 的数学定义与 FLOP 计数说明矩阵乘法是"$O(N^3)$ 计算、$O(N^2)$ 数据"的高算术强度问题，再用行主序（row-major）线性化 `M[row*Width + col]` 把二维矩阵落到一维显存上，然后推出朴素实现需要 `2·M·N·K` 次全局内存访问这一"访存爆炸"结论。核心分析工具是 warp 级访存合并（memory coalescing）与 Roofline 模型：朴素版算术强度只有 0.25 FLOP/Byte，在 A100 上的理论上限约 389 GFLOP/s，仅为其 19.5 TFLOPS 峰值的 2%。最后用"数据复用次数"的计算引出下一讲的分块（tiling）优化。

核心概念与 GPU 架构图解

概念 1：矩阵乘法的数学定义与计算量（Matrix Multiplication: Definition and FLOP Count）

- **定义与目的**：给定矩阵 A（M 行 K 列）与矩阵 B（K 行 N 列），乘积 `P = A × B` 是一个 M 行 N 列的矩阵，其第 i 行第 j 列的元素为 `P[i][j] = Σ(k=0..K−1) A[i][k] * B[k][j]`。这个定义把"内积（inner product / dot product）"作为基本操作：A 的第 i 行与 B 的第 j 列做一次长度为 K 的内积，就得到 P 的一个元素。Lab 2 取方阵特例 `M = N = K = Width`，三个矩阵都按行主序存放在一维显存中。定义它的目的有两个：一是给出可并行化的粒度（M×N 个独立内积，彼此没有依赖，天然适合 GPU 的数万线程）；二是给出可精确计算的"工作量"与"数据量"，从而可以定量判定程序是计算受限（compute-bound）还是带宽受限（memory-bandwidth-bound）。
- **直观解释（"它是什么？"）**：把 A 的每一行看成一张**配方**（recipe），把 B 的每一列看成一张**订单**（order）。配方 i 的第 k 项是"第 k 种原料的用量 A[i][k]"，订单 j 的第 k 项是"第 k 种原料的需求倍数 B[k][j]"。那么 P[i][j] 就是把配方 i 按订单 j 的倍数配一遍所得到的总量。这个类比马上揭示了一件极其重要的事：**同一张配方 i 会被所有 N 张订单使用，同一张订单 j 会被所有 M 张配方使用**。也就是说，矩阵乘法里每一个输入数据都天然含有大量可复用的价值（A 的每个元素被用 N 次，B 的每个元素被用 M 次），而"能不能把这份复用价值变成性能"正是本讲与下一讲的全部张力所在。
- **架构/机制图解**：下面用 4×4 的具体例子把定义写全（真实实验里 Width 取 1024、2048 或更大，规则完全相同）。

```
       B  (K=4 行, N=4 列)                  P = A x B   (M=4 行, N=4 列)
         c0   c1   c2   c3
      +----+----+----+----+
r0    |  1 |  2 |  3 |  4 |
      +----+----+----+----+
r1    |  5 |  6 |  7 |  8 |
      +----+----+----+----+
r2    |  9 | 10 | 11 | 12 |
      +----+----+----+----+
r3    | 13 | 14 | 15 | 16 |
      +----+----+----+----+

P[i][j] = A[i][0]*B[0][j] + A[i][1]*B[1][j] + A[i][2]*B[2][j] + A[i][3]*B[3][j]

举例 (K = 4, 所以每个输出元素要做 4 次乘加):
P[2][1] = A[2][0]*B[0][1] + A[2][1]*B[1][1] + A[2][2]*B[2][1] + A[2][3]*B[3][1]
        = A[2][0]*2       + A[2][1]*6       + A[2][2]*10      + A[2][3]*14

并行粒度: M x N = 16 个输出元素, 两两之间没有任何数据依赖
          -> 可以交给 16 个线程同时算, 甚至交给出 1 个线程算 (只是慢)
```

计算量的推导（这是全讲所有数字的起点）：

```
一次 "乘加" (multiply-add, 也称 FMA, fused multiply-add)
     P += A * B      =>     1 次乘法 + 1 次加法 = 2 FLOP (floating-point operation)

输出元素个数          = M * N
每个输出元素的乘加次数 = K             (内积长度)
总的乘加次数          = M * N * K
总的 FLOP 数           = 2 * M * N * K          <-- 记住这个公式
方阵特例 (M=N=K=N)    = 2 * N^3                <-- 计算量按 O(N^3) 增长

数据量:
输入元素个数 = M*K + K*N = 2*N^2  (方阵)
输出元素个数 = M*N       =   N^2
若每个元素 4 Byte (float):
输入字节数 = 2 * N^2 * 4 = 8*N^2  Byte
输出字节数 =     N^2 * 4 = 4*N^2  Byte
总数据量   = 12*N^2 Byte                        <-- 数据量按 O(N^2) 增长

理想算术强度 (算术强度 arithmetic intensity = FLOP / Byte, 假设完美复用不重复读):
    AI_ideal = 2*N^3 / (12*N^2) = N/6  FLOP/Byte
```

具体数字（这一步的数字会在整讲反复用到）：

Width = N	FLOPs = $2 \cdot N^3$	输入数据 $8 \cdot N^2$	输出数据 $4 \cdot N^2$	理想算术强度 N/6
256	33.6 MFLOP	0.5 MB	0.25 MB	42.7 FLOP/Byte
1024	2.147 GFLOP	8 MB	4 MB	170.7 FLOP/Byte
2048	17.18 GFLOP	32 MB	16 MB	341.3 FLOP/Byte
4096	137.4 GFLOP	128 MB	64 MB	682.7 FLOP/Byte

关键对照：A100 的机器平衡点（machine balance，即峰值算力除以峰值带宽）为

```
Machine Balance (A100) = 19.5 TFLOPS / 1555 GB/s
                       = 19.5e12 FLOP/s / 1555e9 Byte/s
                       = 12.5 FLOP/Byte
```

只要一个程序的算术强度大于 12.5 FLOP/Byte，它就在"计算受限"一侧，理论上可以跑到接近 19.5 TFLOPS。矩阵乘法在 N=1024 时的**理想**强度是 170.7 FLOP/Byte，是机器平衡点的 13.6 倍——所以从算法角度看这是"再好不过"的问题。坏消息全部藏在"理想"两个字里：理想强度假设每个输入元素只从 DRAM 搬一次，而朴素 kernel 做不到这一点，见概念 4、5、6。

概念 2：行主序线性化寻址（Row-Major Linearization and Address Arithmetic）

- **定义与目的**：C 语言里的二维数组 `float M[M][K]` 在内存中并不是真正的二维结构，它是**一个连续的一维字节数组**，编译器自动把两个下标换算成一个线性偏移（offset）。CUDA 中最常用的动态分配接口 `cudaMalloc` 只接受一维指针，`cudaMallocPitch` 也只是把一维指针加上"行距"信息，因此**在 GPU 上写矩阵代码必须自己手写线性化公式**。行主序（row-major，也叫 C 风格布局）规定：同一行的元素在内存里连续排列，第 0 行放完放第 1 行，依此类推。于是 `M[row][col]` 的线性偏移是 `row * Width + col`，其中 `Width` 是矩阵一行的元素个数（这个量在数值线性代数里叫 leading dimension，在图像/多媒体代码里常叫 pitch 或 stride）。

- **直观解释（"它是什么？"）**：想象一个剧院，座位按"排"编号：第 0 排有 1024 个座位，然后是第 1 排的 1024 个座位，等等。工作人员不记"第 3 排第 5 座"这样的二元坐标，只记一个连续编号：**第 3 排第 5 座 = 前面 3 整排（3×1024 个座位）+ 本排第 5 个 = 编号 3077**。`row * Width + col` 就是这个"前面有几整排"的算术。它带来一个极其重要的几何直觉：**同一排里相邻座位的物理距离是 1（4 字节），而同一列里上下相邻座位之间的距离是整整一排（Width × 4 字节）**。GPU 的访存合并机制只对"相邻座位"友好，对"隔一整排"的访问极其昂贵——这就是本讲所有性能悲剧的几何根源。
- **架构/机制图解**：下图的矩阵是 4 行 6 列（Width = 6），二维逻辑视图与一维物理布局的对应关系必须能一眼画出来。

```
逻辑二维视图（Width = 6，共 4 行）
          col=0   col=1   col=2   col=3   col=4   col=5
  row=0  [ M00 ] [ M01 ] [ M02 ] [ M03 ] [ M04 ] [ M05 ]
  row=1  [ M10 ] [ M11 ] [ M12 ] [ M13 ] [ M14 ] [ M15 ]
  row=2  [ M20 ] [ M21 ] [ M22 ] [ M23 ] [ M24 ] [ M25 ]
  row=3  [ M30 ] [ M31 ] [ M32 ] [ M33 ] [ M34 ] [ M35 ]

物理一维内存（行主序：一行接一行，行内连续）
  线性偏移:   0     1     2     3     4     5     6     7     8     9    10    11
  内容:     M00   M01   M02   M03   M04   M05   M10   M11   M12   M13   M14   M15
             |<-------- 第 0 行（6 个元素，24 Byte）-------->|<---- 第 1 行（6 个元素）--->

  地址公式：
      M[row][col]  的线性偏移 = row * Width + col
      M[row][col]  的字节地址 = base + 4 * (row * Width + col)      (float 占 4 Byte)

  两个距离（务必背下来）：
      同一行内相邻列（col -> col+1）：偏移 +1，字节地址 +4    Byte    <-- 连续，可合并
      同一列内相邻行（row -> row+1）：偏移 +Width，字节地址 +Width*4 Byte <-- 跨步 stride
```

代入具体数字来感受"跨步"有多夸张（Width = 1024）：

```
  M[3][5]   -> 偏移 = 3*1024 + 5 = 3077          -> 字节地址 = base + 12308
  M[3][6]   -> 偏移 = 3078                       -> 字节地址 = base + 12312    (相差 4 B)
  M[4][5]   -> 偏移 = 4*1024 + 5 = 4101          -> 字节地址 = base + 16404    (相差 4096 B)

  4096 Byte / 128 Byte(一条 cache line 的粒度) = 32 条 cache line
  -> 同列上下相邻的两个元素，落在相隔 32 条 cache line 的两块不同内存里
  -> 一个 warp 里 32 个线程若沿着"列"走，硬件就必须发出 32 次互不相干的取数请求
```

在硬件层面，GPU 的全局内存系统以 **32 Byte 的 sector** 为最小的取数粒度、以 **128 Byte 的 cache line**（= 4 个 sector）为标签（tag）管理粒度，一条 warp 指令的理想情况是 32 个线程访问 32 个连续 float（32×4 = 128 Byte = 4 个 sector = 1 条 cache line），一次全部取回、全部有用。Lecture 7 讲 DRAM 时给出了更底层的理由：DRAM 允许一次行激活（row activation）后把整行数据"猝发"（burst）送出，现代 DRAM 系统**总是工作在 burst 模式**，不是顺序访问时被取回的 burst 字节会被直接丢弃。讲义里的练习题把代价算得很直白：burst size = 512 Byte、峰值带宽 240 GB/s 时，访问 `A[4*i]` 和 `A[4*i+1]` 只用到 burst 中每一对 8 Byte 里的一半，于是可用带宽降到 120 GB/s；在早期没有 cache 的 GPU 上两次 load 甚至会各自砍掉一半，落到 60 GB/s。这解释了为什么"跨步访存"不是"多花一点点时间"，而是成倍地浪费带宽。

概念 3：每线程一个输出元素（Thread-per-Output Mapping）与二维 block/grid

- **定义与目的**：最简单的并行分解方式：P 有 M×N 个元素，就启动 M×N 个线程，每个线程独立算完一个输出元素（一整条长度为 K 的内积）并写回。为了让"线程坐标"与"矩阵坐标"自然对应，CUDA 允许 block 和 grid 都是二维的（`dim3`），于是行号由 y 方向决定、列号由 x 方向决定，索引计算只写两行代码。这个映射的价值在于：**零通信、零同步、无数据依赖**，非常适合 GPU；它的代价则完全落在内存系统上（见概念 4）。Lab 2 要写的正是这个版本。
- **直观解释（"它是什么？"）**：把 P 想象成一张巨大的报表，M×N 个格子。我们雇 M×N 个抄写员，每人负责一个格子，每人手里有两份资料：A 的一整行和 B 的一整列。抄写员互相不说话（没有同步），各算各的。问题是他们**共享同一间资料室**：打印 1024×1024 的报表需要 100 多万个抄写员，而每个人都要跑去资料室取 2048 份资料（A 行 1024 份 + B 列 1024 份），资料室的门（内存带宽）会被挤爆。更糟的是，如果资料室按"整排书架"取书，而某类抄写员需要的那列资料散落在 1024 个不同书架上，那么每取一份资料都要跑一趟——这就是"不合并的列访问"。
- **架构/机制图解**：先看映射关系（玩具尺寸：Width = 4，block = 2×2，于是 grid = 2×2）。

```
Grid (dim3 dimGrid(2,2,1))          每个 block 负责 P 的一块 2x2 子矩阵
   blockIdx = (0,0)   blockIdx = (1,0)
  +-----------------+-----------------+
  |  P[0][0] P[0][1]|  P[0][2] P[0][3]|
  |  P[1][0] P[1][1]|  P[1][2] P[1][3]|
  +-----------------+-----------------+
  |  P[2][0] P[2][1]|  P[2][2] P[2][3]|
  |  P[3][0] P[3][1]|  P[3][2] P[3][3]|
  +-----------------+-----------------+
   blockIdx = (0,1)   blockIdx = (1,1)

Block(0,0) 内部 (dim3 dimBlock(2,2,1), 4 个线程)
     threadIdx=(0,0) -> P[0][0]      threadIdx=(1,0) -> P[0][1]
     threadIdx=(0,1) -> P[1][0]      threadIdx=(1,1) -> P[1][1]

索引公式 (讲义原文, 逐字对应):
     int Row = blockIdx.y * blockDim.y + threadIdx.y;   // 输出行 = 该 block 的行起点 + 线程行偏移
     int Col = blockIdx.x * blockDim.x + threadIdx.x;   // 输出列 = 该 block 的列起点 + 线程列偏移
     P[Row * Width + Col] = Pvalue;
```

再看"一个线程"在 k 循环里到底访问了哪些内存:

```
线程 (Row, Col) 的内积循环 for (k = 0; k < Width; ++k)
   k=0:       读 A[Row][0]       与 B[0][Col]        -> 相乘累加
   k=1:       读 A[Row][1]       与 B[1][Col]        -> 相乘累加
   k=2:       读 A[Row][2]       与 B[2][Col]        -> 相乘累加
    (k = 3 一直到 k = Width-2 依此类推, 访问规律完全相同)
   k=Width-1: 读 A[Row][Width-1] 与 B[Width-1][Col]  -> 相乘累加

   A 侧: 地址 = Row*Width + k, k 递增 1 -> 沿 A 的一行"连续"前进 (每次 +4 Byte)
   B 侧: 地址 = k*Width + Col, k 递增 1 -> 沿 B 的一列"跨步"前进 (每次 +Width*4 Byte)

   本线程的访存足迹:
       读 A: Width 次 (连续, 硬件预取/cache 友好)
       读 B: Width 次 (每次跨 Width*4 Byte, cache 完全帮不上忙)
       写 P: 1 次
```

最后看 256 线程的 block 在硬件上如何被切成 warp——这是后面所有 transaction 计算的依据:

```
 blockDim = (16, 16)  =>  256 threads  =>  8 个 warp
 线性线程号 tid = ty * blockDim.x + tx = ty * 16 + tx    (x 方向变化最快)
 硬件划分: warp w = tid / 32, 所以一个 warp = 相邻的两个 ty 行

        tx:    0    1    2    3    4    5    6    7    8    9   10   11   12   13   14   15
  ty= 0  |<-----------------------  Warp 0  (tid   0 ..  31)  ----------------------->|
  ty= 1  |<-----------------------  Warp 0  (tid   0 ..  31)  ----------------------->|
  ty= 2  |<-----------------------  Warp 1  (tid  32 ..  63)  ----------------------->|
  ty= 3  |<-----------------------  Warp 1  (tid  32 ..  63)  ----------------------->|
  ty= 4  |<-----------------------  Warp 2  (tid  64 ..  95)  ----------------------->|
  ty= 5  |<-----------------------  Warp 2  (tid  64 ..  95)  ----------------------->|
  ty= 6  |<-----------------------  Warp 3  (tid  96 .. 127)  ----------------------->|
  ty= 7  |<-----------------------  Warp 3  (tid  96 .. 127)  ----------------------->|
  ty= 8  |<-----------------------  Warp 4  (tid 128 .. 159)  ----------------------->|
  ty= 9  |<-----------------------  Warp 4  (tid 128 .. 159)  ----------------------->|
  ty=10  |<-----------------------  Warp 5  (tid 160 .. 191)  ----------------------->|
  ty=11  |<-----------------------  Warp 5  (tid 160 .. 191)  ----------------------->|
  ty=12  |<-----------------------  Warp 6  (tid 192 .. 223)  ----------------------->|
  ty=13  |<-----------------------  Warp 6  (tid 192 .. 223)  ----------------------->|
  ty=14  |<-----------------------  Warp 7  (tid 224 .. 255)  ----------------------->|
  ty=15  |<-----------------------  Warp 7  (tid 224 .. 255)  ----------------------->|

 (每一行的 16 个格子依次是 tx = 0..15; Warp 0 覆盖 ty=0 与 ty=1 两行,
  所以 32 个线程里 threadIdx.y 只有 2 个取值, 而 threadIdx.x 有 16 个取值。)
```

硬件映射与性能特征（以 A100 / sm_80 为基准）:

- **warp 与调度**：一个 block 的 256 个线程被切成 8 个 warp，每个 warp 交给 SM 里的 4 个 warp scheduler 之一去发射指令；A100 每个 SM 最多同时驻留 64 个 warp（2048 线程），因此 256 线程的 block 最多驻留 8 个（受线程数限制），占用率（occupancy）= 2048/2048 = 100%。**这一点非常重要：朴素矩阵乘法的问题是内存，不是占用率。** 很多同学第一反应是"增大 block 或减少寄存器来提升占用率"，但占用率已经满了。
- **延迟与并行度**：全局内存访问延迟约 400–800 个时钟周期。要以 1555 GB/s 的速率喂饱 108 个 SM，每个 SM 需要维持大约 `1555e9 / 108 / 1.41e9 ≈ 10 Byte/cycle/SM` 的取数速率，这必须靠"大量在飞（in-flight）的访存请求"来实现：`Little's Law`：需要的在飞请求数 = 延迟 × 速率 = 约 600 cycle × 10 Byte/cycle ÷ 128 Byte/请求 ≈ 47 条未完成的 128 Byte 请求。8 个 block × 256 线程 × 2 次 load = 4096 个在飞 load 是够的——前提是它们都能被合并成完整的 cache line。

- **warp 发散（divergence）**：`if (Row < Width && Col < Width)` 这个边界保护只在 Width 不是 blockDim 整数倍时对边界 block 产生发散；Width = 1024、block = 16×16 时完全不发散（1024/16 = 64 整除）。
- **规模数字**：Width = 1024、block = 16×16 时，`dimGrid = (ceil(1024/16), ceil(1024/16)) = (64, 64)`，共 4096 个 block、1,048,576 个线程、每线程 2×1024 = 2048 FLOP；108 个 SM 各驻留 8 个 block = 864 个 block 并发，所以整个 grid 要跑 `4096 / 864 ≈ 4.74` 个"波次"（wave）。

概念 4：朴素实现的访存爆炸与跨步访存（Memory Access Explosion and Strided Access）

- **定义与目的**：把概念 3 的映射写成代码，内积循环里只有两条 load：`d_M[Row*Width+k]` 与 `d_N[k*Width+Col]`。**每做一次乘加（2 FLOP），就要从内存取两个 float（8 Byte）**，因此每 1 FLOP 需要 4 Byte——这就是讲义中反复出现的"**4B of Data per FLOP**"。也就是说，GPU 每获得 1 个浮点运算能力，就必须同时供给它 4 Byte 的内存带宽。这个比例是判断"这台机器能不能跑这个算法"的第一把尺子，也是 Lab 2 性能上不去的根本原因。本概念要把它拆到 warp 与 transaction 的粒度，弄清楚哪一半流量是"必要的"，哪一半是"被访存模式浪费掉的"。
- **直观解释（"它是什么？"）**：把 GPU 的内存系统想象成一家**只批发整箱货物的仓库**：最小出库单位是 32 Byte 的一个 sector（一"箱"），而线程提出的 load 请求是"我要这箱里的某一个 4 Byte"。如果 32 个线程要的东西恰好落在同一箱（以及相邻的几箱）里，仓库出一次货就满足了所有人——这是**合并访问（coalesced access）**。如果 32 个线程要的东西散落在 32 个不同的箱子里，仓库就必须出 32 次货、搬 32 箱回来，而每箱里只有 4 Byte 被用掉、其余 28 Byte 都是垃圾——这是**跨步访问（strided access）**，也是"4 B/FLOP"变成"实际带宽被吃掉 8 倍"的原因。更糟的是**重复取货**：1024×1024 的矩阵乘法里，A 的每一个元素被 1024 个不同线程用到，如果每个线程都自己去仓库拿一次，那就是 1024 倍的冗余请求。合并访问解决的是"取一趟多拿点"，数据复用（概念 7）解决的是"别拿那么多趟"——本讲的 Lab 2 两个问题都没解决。
- **架构/机制图解**：先算清请求的总量，再算清每一条请求在硬件上的代价。

```
【第一层：总请求量（"访存爆炸"）】
 每个输出元素：K 次读 A 的一行 + K 次读 B 的一列 = 2K 次 load
 输出元素总数：M * N
 总 load 次数：2 * M * N * K                    （方阵：2*N^3）
 总请求字节数：2 * M * N * K * 4 Byte = 8*M*N*K Byte
 总 FLOP 数 ：2 * M * N * K
 => 每 FLOP 需要的请求字节数 = 8*M*N*K / (2*M*N*K) = 4 Byte/FLOP

 代入 N = 1024：
     总 load 次数 = 2 * 1024^3      = 2.147e9 次 load
     请求字节数   = 8 * 1024^3 Byte = 8.59 GB
     若这些请求全部打到 DRAM 且完美利用带宽：
         时间下界 = 8.59 GB / 1555 GB/s = 5.52 ms
     若纯按峰值算力执行（19.5 TFLOPS）：
         时间下界 = 2.147 GFLOP / 19.5 TFLOPS = 0.110 ms
     => 访存时间下界是计算时间下界的 50 倍！ 内存完全统治了这个 kernel。

【第二层：数据复用次数（为什么请求量这么大）】
 A 有 M*K 个元素，每个元素被多少个线程读？  A[i][k] 被 P 第 i 行的所有 N 个输出用到 -> N 次
 B 有 K*N 个元素，每个元素被多少个线程读？  B[k][j] 被 P 第 j 列的所有 M 个输出用到 -> M 次
 验证：A 侧总 load = M*K*N = MNK，B 侧总 load = K*N*M = MNK，合计 2MNK    （对上了）
 => 请求量是"必要数据量"的多少倍？
    MNK 次 A 的 load  vs  只需要 M*K 个元素  ->  N 倍冗余
    MNK 次 B 的 load  vs  只需要 K*N 个元素  ->  M 倍冗余
 这就是"数据复用（data reuse）没有被利用"的定量表述。
```

第二层分析已经说明：**一半以上的请求是冗余的**。但即使请求都是"必要"的，还有一层更狠的浪费——warp 内的地址分布。下面逐 warp 剖析 16×16 block、Width = 1024 时，warp 0 在固定 k 的一次迭代里两条 load 的地址分布（warp 0 覆盖 ty = t0 与 ty = t0+1 两行共 32 个线程）。

```
【对 A 的访问 d_M[Row*Width + k]】地址只依赖 ty，不依赖 tx
    ty = t0   的 16 个线程：地址 = base + (t0   * 1024 + k) * 4
    ty = t0+1 的 16 个线程：地址 = base + ((t0+1)* 1024 + k) * 4
    -> warp 内只有 2 个不同的地址，每个地址被 16 个线程重复请求（硬件做广播 broadcast）
    -> 两个地址相距 1024 * 4 = 4096 Byte = 32 条 128 B cache line
    -> 落在两个不同的 32 B sector 上，必须发两次取数请求

    [ sector #1: 32 Byte ]                [ sector #2: 32 Byte ]
     +-----------+--------------------+     +-----------+--------------------+
     | USED 4B   |   wasted 28 Byte   |     | USED 4B   |   wasted 28 Byte   |
     +-----------+--------------------+     +-----------+--------------------+
          ^                                     ^
       地址 A                                 地址 A + 4096 B
     |<---------- 4096 Byte = 32 条 128 B cache line ---------->|

    请求数（transaction）= 2
    取回字节 = 2 * 32 B   = 64 B
    有用字节 = 2 * 4 B    =  8 B
    效率     = 8 / 64     = 12.5%          <-- 87.5% 的带宽被浪费

【对 B 的访问 d_N[k*Width + Col]】地址只依赖 tx，不依赖 ty
    ty = t0   的 16 个线程：地址 = base + (k*1024 + bx*16 + tx) * 4，tx = 0..15
    ty = t0+1 的 16 个线程：地址表达式完全相同（式子里没有 ty）
    -> warp 内一共 16 个互不相同的地址，而且它们在物理上"连续"排布
    -> 16 个连续 float = 64 Byte，通常落在同一条 128 B cache line 内

    +-------------------------------------------------------------+
    | 128 Byte cache line（4 个 32 B sector）                      |
    | [ sector 0 ][ sector 1 ][ sector 2 ][ sector 3 ]            |
    |   ^^^^^^^^^^  ^^^^^^^^^^                                   |
    |   16 个 float 全部被用到（64 Byte，2 个 sector）             |
    +-------------------------------------------------------------+

    请求数（transaction）= 1 条 warp 请求（覆盖 2 个 sector，合并在一条 cache line 内）
    取回字节 = 64 B
    有用字节 = 16 * 4 B = 64 B
    效率     = 100%                       <-- 这一半是"好"的

【warp 0 在固定 k 上两条 load 的合计】
    取回字节 = 64 B (A) + 64 B (B) = 128 B
    有用字节 =  8 B (A) + 64 B (B) =  72 B
    整体效率 = 72 / 128 = 56%
```

这就解释了为什么"每 1 FLOP 需要 4 Byte"在真实硬件上还要再打折扣：请求字节数已经等于 4 B/FLOP，而**取回的字节里只有一部分有用**，实际搬运量更大。Lab 2 的实测带宽利用率之所以远低于峰值，原因就在这里。

关于"哪一个操作数是跨步的"：必须把命名讲清楚。 本课程的约定是 `threadIdx.x` 对应**列**（Col）、`threadIdx.y` 对应**行**（Row），在这种约定下，**A（讲义中的 `d_M`）的访问是跨步的、B（讲义中的 `d_N`）的访问是合并的**——Lecture 7 的原文就是两页对照："Accesses to N are Coalesced" 与 "Accesses to M are NOT Coalesced!"。原因很朴素：一个地址里出现 `threadIdx.y` 的线性系数是 `Width`（跨步），出现 `threadIdx.x` 的系数是 1（连续）。一旦交换 x/y 的语义，受害者就换成另一个操作数，跨步量依旧是 `Width * 4` Byte、transaction 数依旧由"warp 内有多少个不同的行索引"决定。下面这张表把四种常见情形一次算完（16×16 block，Width = 1024，均为"每 warp 每 k"的统计）：

情形	A 侧地址分布	A 侧取回/有用	B 侧地址分布	B 侧取回/有用	合计效率
标准映射：x→Col, y→Row（**本讲与 Lab 2 的写法**）	2 个地址，相距 `Width*4` = 4096 B	64 B / 8 B = 12.5%	16 个连续地址 = 64 B	64 B / 64 B = 100%	128 B / 72 B = 56%
A 以转置形式给出（`M_T[k*Width + Row]`）	2 个**相邻**地址，相距 4 B	32 B / 8 B = 25%	同左（不变）	64 B / 64 B = 100%	96 B / 72 B = 75%
B 以转置形式给出（`N_T[Col*Width + k]`，即按列主序读 B）	同第一行（不变）	64 B / 8 B = 12.5%	**16 个地址，每个相距 4096 B → 16 个独立 sector**	512 B / 64 B = 12.5%	**576 B / 72 B = 12.5%**
交换语义：x→Row, y→Col	16 个地址，每个相距 4096 B → 16 个独立 sector	512 B / 64 B = 12.5%	2 个相邻地址，相距 4 B	32 B / 8 B = 25%	544 B / 72 B = 13.2%

把第三行单独展开，因为它就是"按列读 B"最直观的受害者形态（也是很多同学把 B 按列主序存好后踩的坑）：

```
【B 按列主序（转置）存储时 warp 0 对 B 的访问 N_T[Col*Width + k]】
    warp 内 16 个线程（tx = 0..15，ty 只有 2 个取值但地址里没有 ty）
    地址 = base + ((bx*16 + tx) * 1024 + k) * 4
    tx 每 +1，地址 +4096 Byte

    sector 编号:  #1      #2      #3      #4      #5      #6      #7      #8
    地址(Byte):   0      4096    8192   12288   16384   20480   24576   28672
    有用:        4B       4B      4B      4B      4B      4B      4B      4B

    sector 编号:  #9     #10     #11     #12     #13     #14     #15     #16
    地址(Byte): 32768   36864   40960   45056   49152   53248   57344   61440
    有用:        4B       4B      4B      4B      4B      4B      4B      4B
    -> 16 个互不相邻的 32 B sector，硬件必须发 16 次独立取数请求
    取回字节 = 16 * 32 B = 512 B
    有用字节 = 16 * 4 B  =  64 B
    效率     = 12.5%

 与"B 行主序"的 64 B 相比，同样是 16 个有用的 float，
 却搬回了 512 B —— 8 倍的无效搬运量。
```

最后必须补一个诚实的工程细节：**L1/L2 cache 会把模型"变好看"，但不会让问题消失**。原因有两层：(1) 跨步访问取回的 32 B sector 里，除了被用到的 4 B，其余 28 B 其实是同一行里 k+1 到 k+7 的邻居元素，而内积循环接下来正好会用到它们（k 每 +1，地址 +4 B），所以只要 cache 装得下，一次 sector 取回会被 8 次迭代摊薄；(2) A100 的 L2 有 40 MB，N = 1024 时三个矩阵只占 8 + 4 = 12 MB，**整个工作集都装得进 L2**，此时大部分请求被 L2 吸收（L2 命中延迟约 200 cycle，带宽远高于 DRAM），实测性能会明显优于"纯 DRAM 模型"的预测。因此做诊断实验时必须把矩阵开大（N ≥ 4096 时单个矩阵 64 MB，远超 L2）才能看到真正的 DRAM 行为；而这一点也提示了下讲分块优化的真正收益来源——**分块减少的是"请求数量"，它对 cache 是否命中同样有效**。

概念 5：算术强度与机器平衡点（Arithmetic Intensity and Machine Balance）

- **定义与目的**：算术强度（arithmetic intensity，AI）定义为程序完成的总浮点运算数除以它从内存搬进/搬出的总字节数，单位 FLOP/Byte。它把"程序"和"机器"分开描述：程序提供 AI，机器提供"每 Byte 能配多少 FLOP"的能力上限（机器平衡点 machine balance = 峰值算力 ÷ 峰值带宽）。两者一比，立刻知道瓶颈在哪一侧，以及理论上还能提速多少倍。它是 Roofline 模型（概念 6）的横轴，也是判断"该优化计算还是优化访存"的唯一依据。
- **直观解释（"它是什么？"）**：厨房里做菜。**算术强度就是"每跑一趟冰箱（内存）能做出多少道菜（FLOP）"**。如果你的厨房只有一块很小的案板，每次只能拿一根葱回来切一刀，那你的出菜速度完全取决于你跑冰箱的速度（带宽受限）；如果案板够大，一趟搬回一整筐食材、切出一百盘菜，那限制你的就是你的刀工（计算受限）。机器平衡点就是这家厨房的"案板与跑腿速度之比"：A100 的比例是 12.5 FLOP/Byte，意思是**每搬 1 Byte 至少要能换来 12.5 次浮点运算，跑腿的才不会是瓶颈**。朴素矩阵乘法每 Bytes 只换来 0.25 次运算，相当于每搬 1 筐菜只切一刀——跑腿的完全累死，刀工师傅闲得发慌。
- **架构/机制图解**：

```
          计算侧                              访存侧
 +------------------------+           +-------------------------+
 |  108 个 SM             |           |  DRAM (HBM2, 1555 GB/s)|
 |  每 SM 64 个 FP32 单元 |           |  全局内存 40 GB         |
 |  1.41 GHz              |           +-----------+-------------+
 |  => 19.5 TFLOP/s       |                       |
 +-----------+------------+                       |
             |                                    |
             |        +------------------+        |
             +------->|  GPU 内核流水线  |<-------+
                      +------------------+
                       每周期能"吃掉"多少数据?
                       需要 19.5e12 / 1555e9 = 12.5 FLOP/Byte 才平衡

 程序侧（朴素矩阵乘法）:
     AI = FLOP / Byte = 2*M*N*K / (8*M*N*K) = 0.25 FLOP/Byte
     AI(0.25) << Balance(12.5)  =>  相差 50 倍  =>  严重带宽受限
```

AI 的推导要能默写（左式是 FLOP，右式是 Byte）：

```
AI_naive = (2 * M * N * K) / (2 * M * N * K * 4 Byte)
         = 1 / 4  (Byte/FLOP 的倒数)
         = 0.25 FLOP/Byte            <-- 与本讲的 4 B/FLOP 是同一件事的两种说法

换个算法看 AI（同样的数字，不同的解读）:
     每 1 次乘加（2 FLOP）读 2 个 float（8 Byte)   ->  2/8 = 0.25 FLOP/Byte
     等价说法："每 FLOP 需要 4 Byte"，即 4 B/FLOP
```

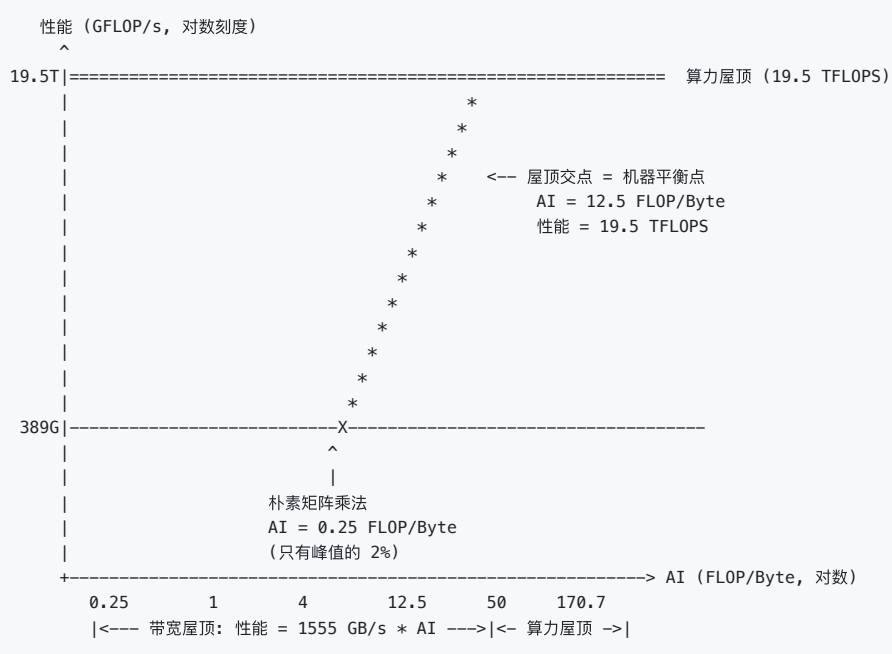
四台参考 GPU 的机器平衡点（用规范表格中的统一硬件档案）：

GPU	峰值带宽	FP32 峰值	机器平衡点 = 算力/带宽	朴素版能否达到平衡（需要 12.5）
RTX 2080 Ti (sm_75)	616 GB/s	13.4 TFLOPS	21.8 FLOP/Byte	否，差 87 倍
A100 (sm_80)	1555 GB/s	19.5 TFLOPS	12.5 FLOP/Byte	否，差 50 倍
H100 SXM (sm_90)	3350 GB/s	67 TFLOPS	20.0 FLOP/Byte	否，差 80 倍
RTX 4090 (sm_89)	1008 GB/s	82.6 TFLOPS	81.9 FLOP/Byte	否，差 328 倍

注意最后一行：**消费级显卡的算术强度鸿沟更可怕**。RTX 4090 的 FP32 算力是 A100 的 4.2 倍，带宽却只有 A100 的 65%，机器平衡点高达 81.9 FLOP/Byte。这意味着同一份朴素 kernel 在 4090 上、以峰值百分比衡量会更难看——把"我认为这个 GPU 更快"当成"我的 kernel 会更快"是初学者最常见的误判。

概念 6：Roofline 模型与朴素版的性能天花板（Roofline Model and the 389 GFLOP/s Ceiling）

- **定义与目的**：Roofline 模型把上面两件事画成一张图：横轴是算术强度 AI（对数刻度），纵轴是可达性能（GFLOP/s，对数刻度）。图上有两条"屋顶"：一条水平的**算力屋顶**（峰值算力），一条斜率为带宽的**带宽屋顶**（`性能 = 带宽 × AI`）。程序的实际性能上限就是两条屋顶的较小值。它的用途是**在做优化之前先算出"最好能到多少"**：如果天花板本身只有峰值的 2%，那么无论怎么调 block 大小、怎么调线程数都没用，必须换算法（这就是下一讲分块的全部动机）。
- **直观解释（"它是什么？"）**：Roofline 就像给一条公路算出"每小时最多能过多少车"：公路的限速（算力）是一条水平线，收费站的处理速度（带宽）在车流量小时根本不影响通行，但当每辆车都要在收费站停很久（AI 很低）时，通行量就完全由收费站决定。此时的通行量 = 收费站速度 × 每辆车能带多少"有效载荷"。对朴素矩阵乘法来说，收费站（内存）就是唯一的瓶颈，而每 1 Byte 只换来 0.25 FLOP 的"有效载荷"，于是通行量被钉死在 `1555 × 0.25`。
- **架构/机制图解**：



天花板的计算（必须写出代入的数字）：

```
Roofline 上限 = 峰值带宽 * 算术强度
              = 1555 GB/s * 0.25 FLOP/Byte
              = 388.75 GFLOP/s
              ~= 389 GFLOP/s

占峰值算力的比例 = 389 GFLOP/s / 19500 GFLOP/s
                 = 1.99%  ~= 2%
                 (也就是 98% 的算力因为"喂不饱"而闲置)

另一种等价算法（更直观）:
    需要的带宽 = 峰值算力 / 算术强度 = 19.5e12 / 0.25 = 78,000 GB/s = 78 TB/s
    实际带宽 = 1555 GB/s
    => 只满足了 1555/78000 = 2.0%

用时间比较（N = 1024）:
    访存时间下界 = 8.59 GB / 1555 GB/s       = 5.52 ms
    计算时间下界 = 2.147 GFLOP / 19.5 TFLOPS = 0.110 ms
    比值 = 50.2  -> 算力必须闲置 98% 才等得起内存
```

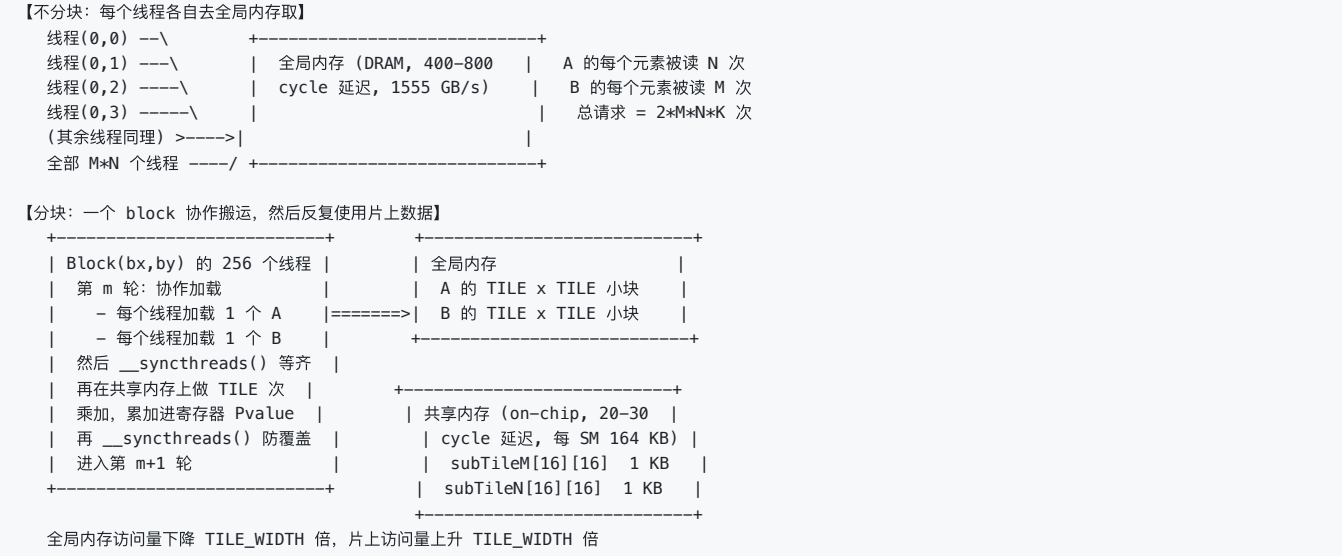
把同样的算式套到其他三台机器上，可以看到 Roofline 上限的绝对值虽然不同，但"占峰值的百分比都是个位数"这一结论完全一致：

GPU	带宽 × 0.25	Roofline 上限	占 FP32 峰值	注释
RTX 2080 Ti	616 × 0.25	154 GFLOP/s	1.15%	2018 年的实验机
A100	1555 × 0.25	389 GFLOP/s	2.0%	本课程基准机（NCSA Delta）
H100 SXM	3350 × 0.25	838 GFLOP/s	1.25%	算力涨得比带宽快
RTX 4090	1008 × 0.25	252 GFLOP/s	0.31%	失衡最严重

讲义用更早一代 GPU 给出过同一个结论：1000 GFLOP/s 算力配 150 GB/s 带宽时，`(150 / 4) = 37.5 GFLOP/s`，而实际运行只有约 25 GFLOP/s（因为"内存并非时刻繁忙"），于是"我们必须**大幅削减对全局内存的访问**"——这就是从 Lab 2 走向 Lab 3（分块）的原动力。注意讲义那个式子里 `150/4` 的含义正是本讲的 4 B/FLOP：**带宽除以"每 FLOP 需要的字节数"，得到的就是带宽能"养得起"的算力**。

概念 7：数据复用与分块优化的动机（Data Reuse and the Motivation for Tiling）

- **定义与目的**：概念 4 已经算出 A 的每个元素被重复读 N 次、B 的每个元素被重复读 M 次，这些都是**冗余请求**。分块（tiling，也叫 blocking）的思路是：把矩阵切成若干小方块（tile），让一个 block 的线程协作地把一小块 A 和一小块 B 从全局内存一次性搬进**片上存储**，之后所有线程都从片上存储里反复取用，从而把"每个元素被读 N 次"降为"每个元素被读 N/TILE_WIDTH 次"。本概念只做**动机与定量收益**的推导，具体实现（共享内存数组、协作加载、`__syncthreads()` 屏障、边界处理）全部留到下一讲。
- **直观解释（"它是什么？"）**：这就是"**案板**"的比喻：现在每个抄写员（线程）都自己跑资料室（全局内存）拿资料，同一份资料被 1024 个人各拿一次。改造方案是给每个小组发一块案板（共享内存，shared memory，延迟只有约 20–30 cycle，而全局内存是 400–800 cycle），**派一个人跑一趟把整筐资料搬到案板上，全组人围着案板干活**。搬运次数从"人数 × 资料数"降到"资料数 / 每组人数"。分块的效果可以用一个数字概括：TILE_WIDTH = 16 时，全局内存访问量减少 16 倍；32 时减少 32 倍。
- **架构/机制图解**：



定量收益（沿用讲义 `(带宽/4) × 复用倍数` 的算法，并换成 A100 的数字）：

```
未分块：  每次全局访问只支撑 1 次乘加（2 FLOP / 8 Byte = 0.25 FLOP/Byte）
          A100 上限 = 1555 * 0.25 =  389 GFLOP/s     （峰值的 2.0%）

TILE_WIDTH = 16：
          每次访存取回的元素被复用 16 次
          AI = 16 * 0.25 = 4 FLOP/Byte
          上限 = 1555 * 4 = 6220 GFLOP/s = 6.22 TFLOP/s    （峰值的 31.9%）
          讲义同款算式（150 GB/s 的老 GPU）：(150/4)*16 = 600 GFLOP/s

TILE_WIDTH = 32：
          AI = 32 * 0.25 = 8 FLOP/Byte
          上限 = 1555 * 8 = 12440 GFLOP/s = 12.44 TFLOP/s   （峰值的 63.8%）
          讲义同款算式（150 GB/s 的老 GPU）：(150/4)*32 = 1200 GFLOP/s

通用公式：AI_tiled = TILE_WIDTH / 4  FLOP/Byte
           Roofline 上限 = 带宽 * TILE_WIDTH / 4

重要结论：当 TILE_WIDTH = 32 时上限才 12.44 TFLOP/s，仍**低于** A100 的机器平衡点
           12.5 FLOP/Byte 所要求的算力（19.5 TFLOPS），说明仅靠 32x32 分块仍然
           是带宽受限；要到 TILE_WIDTH ≈ 50 以上（或叠加寄存器级复用/线程粗化，
           见代码示例三的【性能优化分析】）才能真正转入计算受限区间。
```

分块还顺带解决了概念 4 里那个最刺眼的访存模式问题：从全局内存搬运 tile 时，`subTileM[ty][tx] = M[Row*Width + m*TILE_WIDTH + tx]` 让 `tx` 变化对应连续地址（合并访问），而"跨步"的那一维被彻底移到了**共享内存内部**——共享内存的根本机制是 32 个 bank（每个 bank 宽 4 Byte）的并行访问，完全没有"cache line / sector"的概念，"列访问"在共享内存里根本不是问题。讲义把这一步称为 **corner turning（转角）**：把原本在 DRAM 上无法合并的访问模式，在"全局 → 共享"的搬运过程中重新组织成合并的访问模式。这条思路的代价是引入两个新问题：**块内同步**（谁保证数据已经搬完了？——`__syncthreads()`）与**边界处理**（Width 不是 TILE_WIDTH 整数倍怎么办），它们分别是下一讲与再下一讲的主题。

代码示例与性能分析

示例 1：朴素矩阵乘法（matmul_naive.cu）

这是 Lab 2 的标准答案骨架：kernel 与讲义 Lecture 5 / Lecture 7 中 "A Simple Matrix Multiplication Kernel" 逐字一致，主机端补齐设备查询、`cudaMalloc` / `cudaMemcpy`、`cudaEvent` 计时、CPU 参考实现对比、算术强度与 Roofline 上限计算。

```
// 文件: matmul_naive.cu
// 主题: ECE408 Lecture 4 / Lab 2 -- 朴素矩阵乘法（每线程一个输出元素，不使用共享内存）
// 编译: nvcc -O3 -arch=sm_80 matmul_naive.cu -o matmul_naive
// 运行: ./matmul_naive 1024         （参数为矩阵边长 Width，建议 512 / 1024 / 2048）

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <ctime>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define BLOCK_WIDTH   16    // 16 x 16 = 256 threads = 8 warps per block
#define WARMUP_ITERS   5
#define TIMED_ITERS   20

// ============================== device kernel ==============================
// 与讲义中的 Simple Matrix Multiplication Kernel 完全一致
__global__ void matmulNaiveKernel(const float* __restrict__ d_M,
                                  const float* __restrict__ d_N,
                                  float* __restrict__ d_P,
                                  int Width)
{
    // Calculate the row index of the d_P element and d_M
    int Row = blockIdx.y * blockDim.y + threadIdx.y;
    // Calculate the column index of d_P and d_N
    int Col = blockIdx.x * blockDim.x + threadIdx.x;

    if (Row < Width && Col < Width) {
        float Pvalue = 0.0f;
        // each thread computes one element of the block sub-matrix
        for (int k = 0; k < Width; ++k) {
            Pvalue += d_M[Row * Width + k] * d_N[k * Width + Col];
        }
        d_P[Row * Width + Col] = Pvalue;
    }
}

// ============================== host utilities ==============================
static void initMatrix(float* a, int n, unsigned int seed)
{
    srand(seed);
    for (int i = 0; i < n * n; ++i) {
        a[i] = (float)(rand() % 7) * 0.25f;   // 取值 0.00 ~ 1.50，便于结果校验
    }
}

// CPU 参考实现：i-k-j 顺序，最内层沿行连续访问，比 i-j-k 快得多
static void cpuMatmul(const float* A, const float* B, float* C, int n)
{
    for (int i = 0; i < n * n; ++i) C[i] = 0.0f;
    for (int i = 0; i < n; ++i) {
        for (int k = 0; k < n; ++k) {
            const float  a    = A[i * n + k];
            const float* Brow = B + k * n;
            float*       Crow = C + i * n;
            for (int j = 0; j < n; ++j) {
                Crow[j] += a * Brow[j];
            }
        }
    }
}

// 每个 SM 的 FP32 单元数（估算理论峰值算力用）
static int fp32CoresPerSM(int major, int minor)
{
    if (major == 7) return 64;                        // Volta (sm_70) / Turing (sm_75)
    if (major == 8) return (minor == 0) ? 64 : 128;   // A100 (sm_80)=64, GA10x (sm_86/89)=128
```

```
    if (major >= 9) return 128;                      // Hopper (sm_90) 及以后
    return 128;
}

// ================================ main ================================
int main(int argc, char** argv)
{
    const int    N     = (argc > 1) ? atoi(argv[1]) : 1024;
    const size_t bytes = (size_t)N * (size_t)N * sizeof(float);

    // ---------- 1. 设备属性：理论峰值带宽与峰值算力 ----------
    CUDA_CHECK(cudaSetDevice(0));
    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));

    const double peakBW   = 2.0 * (double)prop.memoryClockRate * 1.0e3
                          * ((double)prop.memoryBusWidth / 8.0) / 1.0e9;   // GB/s
    const double peakFP32 = (double)prop.multiProcessorCount
                          * (double)fp32CoresPerSM(prop.major, prop.minor)
                          * 2.0 * (double)prop.clockRate * 1.0e3 / 1.0e12; // TFLOPS

    printf("== Device ==\n");
    printf("  name                : %s\n", prop.name);
    printf("  compute capability  : sm_%d%d\n", prop.major, prop.minor);
    printf("  SMs                 : %d\n", prop.multiProcessorCount);
    printf("  core clock          : %.3f GHz\n", prop.clockRate * 1e-6);
    printf("  memory clock        : %.3f GHz, bus %d bit\n",
           prop.memoryClockRate * 1e-6, prop.memoryBusWidth);
    printf("  peak bandwidth      : %.1f GB/s\n", peakBW);
    printf("  peak FP32           : %.2f TFLOPS\n", peakFP32);
    printf("  machine balance     : %.2f FLOP/Byte\n",
           peakFP32 * 1e12 / (peakBW * 1e9));
    printf("  L2 cache            : %.1f MB\n", prop.l2CacheSize / 1048576.0);
    printf("  max threads per SM  : %d\n", prop.maxThreadsPerMultiProcessor);

    // ---------- 2. 主机端数据与 CPU 参考结果 ----------
    float* h_M   = (float*)malloc(bytes);
    float* h_N   = (float*)malloc(bytes);
    float* h_P   = (float*)malloc(bytes);
    float* h_Ref = (float*)malloc(bytes);
    if (!h_M || !h_N || !h_P || !h_Ref) {
        fprintf(stderr, "host malloc failed\n");
        exit(EXIT_FAILURE);
    }
    initMatrix(h_M, N, 1u);
    initMatrix(h_N, N, 2u);

    const clock_t cpuT0 = clock();
    cpuMatmul(h_M, h_N, h_Ref, N);
    const double cpuSec = (double)(clock() - cpuT0) / CLOCKS_PER_SEC;

    // ---------- 3. 设备端分配与拷贝 ----------
    float *d_M = NULL, *d_N = NULL, *d_P = NULL;
    CUDA_CHECK(cudaMalloc((void**)&d_M, bytes));
    CUDA_CHECK(cudaMalloc((void**)&d_N, bytes));
    CUDA_CHECK(cudaMalloc((void**)&d_P, bytes));
    CUDA_CHECK(cudaMemcpy(d_M, h_M, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_N, h_N, bytes, cudaMemcpyHostToDevice));

    // ---------- 4. 执行配置（讲义原文的写法：向上取整） ----------
    dim3 dimBlock(BLOCK_WIDTH, BLOCK_WIDTH, 1);
    dim3 dimGrid((N + BLOCK_WIDTH - 1) / BLOCK_WIDTH,
                 (N + BLOCK_WIDTH - 1) / BLOCK_WIDTH, 1);
    printf("\n== Launch configuration ==\n");
    printf("  dimBlock = (%u, %u, 1) = %u threads = %u warps\n",
           dimBlock.x, dimBlock.y, dimBlock.x * dimBlock.y,
           dimBlock.x * dimBlock.y / 32);
    printf("  dimGrid  = (%u, %u, 1) = %u blocks\n",
           dimGrid.x, dimGrid.y, dimGrid.x * dimGrid.y);
    printf("  total threads       : %llu\n",
           (unsigned long long)dimGrid.x * dimGrid.y * dimBlock.x * dimBlock.y);

    // ---------- 5. 资源占用（寄存器 / 占用率） ----------
    cudaFuncAttributes attr;
    CUDA_CHECK(cudaFuncGetAttributes(&attr, matmulNaiveKernel));
    int blocksPerSM = 0;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &blocksPerSM, matmulNaiveKernel, BLOCK_WIDTH * BLOCK_WIDTH, 0));
```

```
    const int threadsPerSM = blocksPerSM * BLOCK_WIDTH * BLOCK_WIDTH;
    printf("\n== Occupancy ==\n");
    printf("  registers per thread : %d\n", attr.numRegs);
    printf("  static shared memory : %zu Byte\n", attr.sharedSizeBytes);
    printf("  blocks per SM        : %d\n", blocksPerSM);
    printf("  threads per SM       : %d  -> occupancy %.1f%%\n",
           threadsPerSM,
           100.0 * threadsPerSM / (double)prop.maxThreadsPerMultiProcessor);

    // ---------- 6. 计时：先热身，再取多次平均 ----------
    for (int i = 0; i < WARMUP_ITERS; ++i) {
        matmulNaiveKernel<<<dimGrid, dimBlock>>>(d_M, d_N, d_P, N);
    }
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t evStart, evStop;
    CUDA_CHECK(cudaEventCreate(&evStart));
    CUDA_CHECK(cudaEventCreate(&evStop));
    CUDA_CHECK(cudaEventRecord(evStart));
    for (int i = 0; i < TIMED_ITERS; ++i) {
        matmulNaiveKernel<<<dimGrid, dimBlock>>>(d_M, d_N, d_P, N);
    }
    CUDA_CHECK(cudaEventRecord(evStop));
    CUDA_CHECK(cudaEventSynchronize(evStop));
    float totalMs = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&totalMs, evStart, evStop));
    const double ms = (double)totalMs / TIMED_ITERS;

    // ---------- 7. 结果校验 ----------
    CUDA_CHECK(cudaMemcpy(h_P, d_P, bytes, cudaMemcpyDeviceToHost));
    double maxAbsErr = 0.0, maxRef = 0.0;
    for (int i = 0; i < N * N; ++i) {
        const double e = fabs((double)h_P[i] - (double)h_Ref[i]);
        if (e > maxAbsErr) maxAbsErr = e;
        if (fabs((double)h_Ref[i]) > maxRef) maxRef = fabs((double)h_Ref[i]);
    }
    const double relErr = (maxRef > 0.0) ? (maxAbsErr / maxRef) : maxAbsErr;

    // ---------- 8. 性能、算术强度与 Roofline 上限 ----------
    const double flops       = 2.0 * (double)N * N * N;
    const double bytesNeeded = 4.0 * flops;                     // 4 Byte per FLOP
    const double gflops      = flops / (ms * 1.0e-3) / 1.0e9;
    const double reqBW       = bytesNeeded / (ms * 1.0e-3) / 1.0e9;
    const double ai          = flops / bytesNeeded;             // = 0.25 FLOP/Byte
    const double roofCap     = peakBW * ai;
    const double peakGF      = peakFP32 * 1.0e3;

    printf("\n== Correctness ==\n");
    printf("  max abs error        : %.6e  (relative %.3e)\n", maxAbsErr, relErr);
    printf("  CPU reference time   : %.3f s\n", cpuSec);
    printf("  verification         : %s\n", (relErr < 1e-4) ? "PASS" : "FAIL");

    printf("\n== Performance (average of %d runs) ==\n", TIMED_ITERS);
    printf("  kernel time          : %.4f ms\n", ms);
    printf("  GFLOP/s              : %.2f\n", gflops);
    printf("  arithmetic intensity : %.3f FLOP/Byte  (4 Byte per FLOP)\n", ai);
    printf("  requested bytes      : %.3f GB  (2*N^3 loads x 4 Byte)\n",
           bytesNeeded / 1.0e9);
    printf("  requested bandwidth  : %.1f GB/s  (%.1f%% of peak)\n",
           reqBW, 100.0 * reqBW / peakBW);

    printf("\n== Roofline ==\n");
    printf("  machine balance      : %.1f FLOP/Byte vs naive AI %.2f -> memory bound\n",
           peakFP32 * 1e12 / (peakBW * 1e9), ai);
    printf("  roofline ceiling     : %.1f GFLOP/s  (%.1f GB/s x %.2f FLOP/Byte)\n",
           roofCap, peakBW, ai);
    printf("  ceiling as %% of peak: %.2f%%\n", 100.0 * roofCap / peakGF);
    printf("  measured / ceiling   : %.2f%%   <-- 实测性能占 Roofline 上限的比例\n",
           100.0 * gflops / roofCap);
    printf("  measured / peak      : %.2f%%\n", 100.0 * gflops / peakGF);

    // ---------- 9. 释放资源 ----------
    CUDA_CHECK(cudaEventDestroy(evStart));
    CUDA_CHECK(cudaEventDestroy(evStop));
    CUDA_CHECK(cudaFree(d_M));
    CUDA_CHECK(cudaFree(d_N));
    CUDA_CHECK(cudaFree(d_P));
```

```
    free(h_M); free(h_N); free(h_P); free(h_Ref);
    CUDA_CHECK(cudaDeviceReset());
    return 0;
}
```

【代码做什么？】

1. **设备查询（步骤 1）**：`cudaGetDeviceProperties` 取回 SM 数、时钟、显存位宽与等效显存时钟，据此算出理论峰值带宽 `2 × memoryClockRate × (busWidth/8)`（乘 2 是因为 DDR 接口每个时钟沿传两次数据）与理论峰值 FP32 算力 `SM 数 × 每 SM FP32 单元数 × 2 × 时钟`。这两个数字是后面所有百分比的分母，不查设备就只能靠猜。
2. **主机端准备（步骤 2）**：用 `initMatrix` 生成两份随机小数值矩阵（元素是 0.25 的整数倍，避免浮点误差掩盖逻辑错误），再用 `cpuMatmul` 算出参考结果 `h_Ref`。CPU 版用 i–k–j 循环顺序——内层沿 B 的行连续访问，比教科书的 i–j–k 快 5–10 倍，否则光等参考结果就要几分钟。
3. **内存分配与拷贝（步骤 3）**：`cudaMalloc` 在显存里开三块 `N*N*4` 字节的缓冲区（M、N、P），`cudaMemcpy(d_M, h_M, bytes, cudaMemcpyHostToDevice)` 把两个输入搬进去。注意 P（输出）不需要初始值，也不需要拷进去。
4. **执行配置（步骤 4）**：`dimBlock = (16,16,1)`（256 线程 / block），`dimGrid = (ceil(N/16), ceil(N/16), 1)`。设 16 的原因是：一个 block 有 256 线程，恰好 8 个完整 warp；同时 16×16 的 tile 会让 warp 内部出现"两行 16 列"的结构，正好暴露跨步访存（见下一小节）。**向上取整保证 grid 覆盖整个矩阵**，多出来的线程由 kernel 里的 `if (Row < Width && Col < Width)` 挡掉。
5. **kernel 执行流程**：每个线程先用两条乘法加法算出自己负责的 `(Row, Col)`；然后在 `k` 从 0 到 Width−1 的循环里，每次从全局内存取 `d_M[Row*Width + k]` 与 `d_N[k*Width + Col]` 各一个 float，做一次乘加累加进寄存器 `Pvalue`；循环结束后把结果写回 `d_P[Row*Width + Col]`。寄存器压力极小（`Pvalue`、`k`、两个基址），这也是它占用率能做到 100% 的原因。
6. **计时（步骤 6）**：先热身 5 次（触发 CUDA 上下文与页表的一次性开销），再用 `cudaEventRecord` / `cudaEventElapsedTime` 计时 20 次取平均。`cudaEvent` 记录的是**设备侧时间戳**，比用 `clock()` 或 `gettimeofday()` 包住 kernel 启动准确得多（后者会把主机端 API 开销和异步执行的误差算进去）。
7. **校验（步骤 7）**：把 `d_P` 拷回主机，与 `h_Ref` 逐元素比较，输出最大绝对误差、相对误差与 PASS/FAIL。
8. **性能与 Roofline（步骤 8）**：由 `FLOPs = 2N³` 与 `Bytes = 4 × FLOPs` 反推 GFLOP/s、请求带宽、算术强度，再算出 Roofline 上限 `peakBW × AI` 以及"实测占上限的百分比"——这是本讲最重要的一个输出。

【并行机制与硬件映射解说】

- **warp 划分**：`dimBlock = (16,16)` → 256 线程 → **8 个 warp**，划分规则是 `tid = ty * 16 + tx`，`warp = tid / 32`，所以 **warp 0 = ty∈{0,1} 的两行共 32 个线程**，warp 1 = ty∈{2,3}，依此类推（参见概念 3 的图）。这个"一个 warp 横跨两行 ty"的结构是下面所有 transaction 计算的起点。
- **warp 如何被调度**：这 8 个 warp 被分配到 SM 的 4 个 warp scheduler 上（每个 scheduler 管一批 warp，轮流发射指令）。A100 每个 SM 最多驻留 64 个 warp / 2048 个线程，256 线程的 block 因此在**线程数上限下最多驻留 8 个 block**；寄存器上限方面，若编译器分配 24 个寄存器/线程，`65536 / (24 × 256) = 10.6` → 也不构成限制，两者取小仍为 8 个 block。
- **占用率（occupancy）定量**：`8 blocks × 256 threads = 2048 threads = 64 warps = 100%`（A100 满占用率）。但要注意一个临界点：**如果寄存器用量超过 32 个/线程，占用率就会掉**。例如 40 寄存器 → `65536 / (40 × 256) = 6.4` → 6 个 block → 1536 线程 → 75%；48 寄存器 → 5 个 block → 62.5%。示例 1 打印的 `registers per thread` 与 `blocks per SM` 就是为了让学生亲眼看到这条链。
- **逐 warp 分析对 B（d_N）的访问模式**：在固定 k 上，warp 0 的 32 个线程访问 `d_N[k*Width + Col]`，其中 `Col = bx*16 + tx`。**地址表达式里没有 ty**，所以 ty=0 组的 16 个线程与 ty=1 组的 16 个线程访问的地址集合完全相同：一共 16 个互不相同的地址，且相邻（tx 每 +1，地址 +4 Byte）。16 个连续 float = **64 Byte**，落在同一（或相邻的）32 B sector 上，硬件把它们合并成 **1–2 条取数请求**，取回 64 Byte 全部有用——**B 的访问是合并的（coalesced），效率 100%**。这也解释了为什么 `threadIdx.x` 必须映射到"内存中连续的那一维"：Lecture 7 的思考题正是问 `B[ty*Width+tx]` 与 `B[tx*Width+ty]` 哪个快，答案是前者，因为"相邻的 x 通常意味着同一个 warp"。
- **逐 warp 分析对 A（d_M）的访问模式**：`d_M[Row*Width + k]` 的地址只依赖 ty。warp 0 里 ty 只有两个取值（t0 与 t0+1），于是 32 个线程只产生 **2 个不同的地址**，每个地址被 16 个线程重复请求（硬件做广播，这不会增加流量）。但这两个地址相差 `Width*4 = 4096 Byte = 32 条 128 B cache line`，落在**两个不同的 sector** 上，硬件必须发 **2 次取数请求**，取回 2×32 = 64 Byte，而其中只有 2×4 = 8 Byte 有用——**A 的访问是不合并的，效率 12.5%**。
- **一个 warp 在固定 k 上的总账**：取回 64（A）+ 64（B）= **128 Byte**，有用 8 + 64 = **72 Byte**，整体效率 **56%**；平均到每条 load 指令上，A 那条 load 要占 **2 个 L1 wavefront**（两个不同 cache line），B 那条占 1 个。按 N = 1024 估算：warp 级 load 指令数 `2N³/32 = 67.1M`，其中 A 的一半各耗 2 个 wavefront → 总 wavefront ≈ 67.1M + 33.6M = **100.7M**；A100 的 L1 吞吐约为 `108 SM × 1 wavefront/cycle × 1.41 GHz = 152 G wavefront/s`，折合 **0.66 ms**——**远小于**按 DRAM 带宽算出的 5.52 ms 下界。结论很清楚：L1 不是瓶颈，**L2/DRAM 才是**。
- **寄存器与 warp 发散**：kernel 只用一个累加寄存器 `Pvalue` 加少量索引寄存器，实测通常 20–30 个，不会 spill 到 local memory。发散方面，Width 为 16 的整数倍时 `if` 判断对所有线程一致（全部进入），**零发散**；只有 Width 不是 16 的整数倍时，最右边/最下边的 block 才会出现部分线程被屏蔽的情况，代价被摊薄到整个 grid 上，通常可以忽略。
- **A100 上的驻留规模**：8 个 block/SM × 108 SM = **864 个 block 并发**，而 grid 共有 `64 × 64 = 4096` 个 block，因此要跑 `4096/864 ≈ 4.74` 个波次（wave）。每个波次结束时会有"尾巴效应"（tail effect），大约浪费半个波次的时间，这也是 N 很小时实测性能偏差大的原因之一。

【性能优化分析】

先看定量结论（下面括号里是 A100 上 Width = 1024 的典型实测值区间，不同编译器版本会有差异）：

- **算术强度**：`AI = FLOP / Byte = 2MNK / (2MNK × 4) = 0.25 FLOP/Byte`。代入 M = N = K = 1024：`FLOPs = 2.147e9`，`请求字节 = 8.59 GB`，`2.147e9 / 8.59e9 = 0.25`。
- **机器平衡点**：`19.5 TFLOPS / 1555 GB/s = 12.5 FLOP/Byte`。`0.25 << 12.5`，相差 **50 倍**。
- **Roofline 上限**：`1555 GB/s × 0.25 FLOP/Byte = 388.75 ≈ 389 GFLOP/s`，只占峰值算力 `19.5 TFLOPS` 的 **2.0%**。也就是说，**这台 GPU 有 98% 的算力注定闲置**。
- **实测定位**：A100 上朴素版通常落在 **50–150 GFLOP/s**，即 Roofline 上限的 **13%–39%**、峰值的 **0.3%–0.8%**。达不到 389 GFLOP/s 的原因有二：(1) 上面算出的 56% sector 效率意味着实际搬运量大于请求量；(2) N = 1024 时三个矩阵共 12 MB 能装进 40 MB 的 L2，L2 命中会"帮倒忙式"地让 DRAM 显得没那么紧张，但也让访存延迟与 L1/L2 吞吐成为第二重限制。**把 N 提高到 4096 再测，会更接近纯 DRAM 模型的预测。**
- **瓶颈判定**：**内存带宽受限（memory-bandwidth-bound）**，而且是最坏的那种——既受 DRAM 带宽限制，又因为跨步访问浪费 sector，还因为重复请求放大了总量。占用率 100% 说明**延迟不是问题**、**占用率不是问题**，任何"调 block 大小 / 加 `__launch_bounds__` / 提高占用率"的手段都不会有实质收益（占用率优化的适用场景是"延迟受限"的 kernel）。
- **Little's Law 复核**：要在 1555 GB/s 下工作、面对约 400–800 cycle（1.41 GHz 下约 284–567 ns）的全局内存延迟，GPU 需要同时在飞（in-flight）`1555e9 × 300e-9 ≈ 466 KB` 的数据，平均到 108 个 SM 约 4.3 KB ≈ 34 条 cache line。8 个 block × 256 线程 × 2 条在飞 load ≈ 4096 个请求，足以满足——**所以"加更多线程"救不了它，问题在请求本身的"质量"与"数量"**。

三种改进方向（本讲给出方向，实现分别落在后续实验）：

1. **把跨步的列访问改成合并访问**。手段有两种：(a) **改变数据布局**——例如把 A 以转置形式（`M_T`）提供，让 A 的地址变成 `M_T[k*Width + Row]`，warp 内两个地址从"相距 4096 B"变成"相距 4 B"，transaction 从 2 次降到 1 次、取回字节从 64 B 降到 32 B（效率 12.5% → 25%）；(b) **让每个线程负责输出的一整行/一整块**，把"按行复用 A、按列复用 B"的冲突用寄存器摊平。但要清醒地认识到：**在 thread-per-output 模式下，无论怎么重排映射，都只能把跨步访问从一个操作数搬到另一个操作数**（概念 4 的表格里四种情形全部逃不掉），而且它只把每 warp 每 k 的搬运量从 128 B 降到 96 B（1.33 倍），**改不动"请求总量 = 2MNK"这个根本问题**。示例 3 会用实测数据证明这一点。
2. **引入共享内存做分块（tiling）以实现数据复用**，这是下一讲的主题。它把全局内存请求量降低 `TILE_WIDTH` 倍（16×16 分块 → 降低 16 倍，算术强度从 0.25 提升到 4 FLOP/Byte，Roofline 上限从 389 GFLOP/s 提升到 **6.22 TFLOP/s**），并且顺手把跨步访问"关进"共享内存——共享内存没有 sector 概念，只有 32 个 bank 的并行访问，跨步在片上不再是灾难。这是**唯一同时解决"流量总量"和"访问模式"两个问题**的路线。
3. **线程粗化（thread coarsening）让每个线程算多个输出元素**，把 A 的行元素与 B 的列元素在**寄存器**里复用。设每个线程算 `r` 行 × `c` 列共 `r*c` 个输出，则每个 k 需要 `r + c` 次 load、做 `2*r*c` 次 FLOP，于是

```
粗化后的算术强度 AI(r, c) = 2*r*c FLOP / (4*(r + c) Byte)
                          = r*c / (2*(r + c))  FLOP/Byte

例：r=1, c=4  (线程沿一行算 4 个输出)
    AI = 4 / (2*5) = 0.4 FLOP/Byte        (比 0.25 好 1.6 倍，仍远远不够)
例：r=4, c=4  (4x4 寄存器块，每 k 只读 8 个 float、做 32 次 FLOP)
    AI = 16 / (2*8) = 1.0 FLOP/Byte       (比 0.25 好 4 倍)
例：r=8, c=8
    AI = 64 / (2*16) = 2.0 FLOP/Byte      (仍只有机器平衡点的 1/6)

与共享内存分块叠加时（block 用 T x T 线程，每线程算 r x c 个输出）：
    AI(T, r, c) = 2*T^3*r*c FLOP / (4*T^2*(r+c) Byte) = T*r*c / (2*(r+c)) FLOP/Byte
    例：T=16, r=4, c=4  ->  AI = 16*16 / (2*8) = 16.0 FLOP/Byte > 12.5
    => 这时才真正跨过 A100 的机器平衡点，进入计算受限区间
```

也就是说：**寄存器粗化自己只能把强度抬高到 1–2 FLOP/Byte，必须与共享内存分块叠加（16×16 分块 + 4×4 寄存器块 → 16 FLOP/Byte）才能真正摆脱带宽的统治**。这是本课程后半段所有优化技巧的主线。

示例 2：跨步访存诊断实验（matmul_diag.cu）

这个程序**不做矩阵乘法**，它把三个"纯访存"的探针放在一起跑，用实测带宽证明："地址连续"与"地址跨步"在 GPU 上是两种完全不同的物理事件。三个探针读**完全相同的数据量**，唯一的区别是 warp 内 32 个线程的地址分布。

```
// 文件: matmul_diag.cu
// 主题: ECE408 Lecture 4 —— 诊断实验：合并访问 vs 跨步访问（stride = Width）的带宽代价
// 编译: nvcc -O3 -arch=sm_80 matmul_diag.cu -o matmul_diag
// 运行: ./matmul_diag 4096
//       参数为探测矩阵边长 W，默认 4096（单矩阵 64 MB，远超 A100 的 40 MB L2，
//       只有把工作集赶出 cache 才能观察到真正的 DRAM 行为）

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define PROBE_BLOCK   256
#define TILE_W         16
#define STREAM_ITERS   10

// ---------------- 探针 1：完全合并的顺序读（每 warp 128 Byte 连续） ----------------
__global__ void streamReadKernel(const float* __restrict__ in,
                                 float* __restrict__ out, int n)
{
    int g = blockIdx.x * blockDim.x + threadIdx.x;
    if (g < n) out[g] = in[g] * 2.0f;
}

// ---------------- 探针 2：跨步读（相邻线程地址相距 Width*4 Byte） ----------------
__global__ void stridedReadKernel(const float* __restrict__ in,
                                  float* __restrict__ out, int Width)
{
    const int g = blockIdx.x * blockDim.x + threadIdx.x;
    const int n = Width * Width;
    if (g < n) {
        const int Row = g / Width;    // 与 thread-per-output 中的输出行号同构
        const int Col = g % Width;    // 与 thread-per-output 中的输出列号同构
        out[g] = in[Col * Width + Row] * 2.0f;   // 相邻线程（Col+1）地址 + Width*4 Byte
    }
}

// ---------------- 探针 3：朴素矩阵乘法的访存流量（保留访问模式，去掉 FLOP） ----------------
__global__ void naiveLoadsOnlyKernel(const float* __restrict__ d_M,
                                     const float* __restrict__ d_N,
                                     float* __restrict__ out, int Width)
{
    const int Row = blockIdx.y * blockDim.y + threadIdx.y;
    const int Col = blockIdx.x * blockDim.x + threadIdx.x;
    if (Row < Width && Col < Width) {
        float acc = 0.0f;
        for (int k = 0; k < Width; ++k) {
            acc += d_M[Row * Width + k] + d_N[k * Width + Col];
        }
        out[Row * Width + Col] = acc;
    }
}

// ============================== 校验（抽样） ==============================
static void verifyStream(const float* h_out, const float* h_in, int n)
{
    int bad = 0;
    for (int g = 0; g < n; g += 9973) {
        if (h_out[g] != h_in[g] * 2.0f) ++bad;
    }
    printf("  verification         : %s (sampled)\n", (bad == 0) ? "PASS" : "FAIL");
}

static void verifyStrided(const float* h_out, const float* h_in, int W)
{
    const int n = W * W;
    int bad = 0;
```

```
    for (int g = 0; g < n; g += 9973) {
        const int Row = g / W;
        const int Col = g % W;
        if (h_out[g] != h_in[Col * W + Row] * 2.0f) ++bad;
    }
    printf("  verification         : %s (sampled)\n", (bad == 0) ? "PASS" : "FAIL");
}

static void verifyLoadsOnly(const float* h_out, const float* h_in, int W)
{
    const int n = W * W;
    int bad = 0;
    for (int g = 0; g < n; g += 262147) {
        const int Row = g / W;
        const int Col = g % W;
        double expect = 0.0;
        for (int k = 0; k < W; ++k) {
            expect += (double)h_in[Row * W + k] + (double)h_in[k * W + Col];
        }
        const double diff = (double)h_out[g] - expect;
        if (fabs(diff) > 1e-3 * fabs(expect) + 1.0) ++bad;
    }
    printf("  verification         : %s (sampled)\n", (bad == 0) ? "PASS" : "FAIL");
}

// ================================== main ==================================
int main(int argc, char** argv)
{
    const int    W     = (argc > 1) ? atoi(argv[1]) : 4096;
    const int    n     = W * W;
    const size_t bytes = (size_t)W * (size_t)W * sizeof(float);

    CUDA_CHECK(cudaSetDevice(0));
    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    const double peakBW = 2.0 * (double)prop.memoryClockRate * 1.0e3
                        * ((double)prop.memoryBusWidth / 8.0) / 1.0e9;

    printf("== Device ==\n");
    printf("  %s\n", prop.name);
    printf("  peak bandwidth       : %.1f GB/s\n", peakBW);
    printf("  L2 cache             : %.1f MB\n", prop.l2CacheSize / 1048576.0);
    printf("  probe width W        : %d   (one matrix = %.1f MB)\n",
           W, bytes / 1048576.0);
    printf("  stride between adjacent threads in probe 2: W*4 = %d Byte\n", W * 4);

    // ---------- 主机数据 ----------
    float* h_in  = (float*)malloc(bytes);
    float* h_out = (float*)malloc(bytes);
    if (!h_in || !h_out) { fprintf(stderr, "host malloc failed\n"); exit(EXIT_FAILURE); }
    for (int i = 0; i < n; ++i) h_in[i] = (float)(i % 17) * 0.125f;

    float *d_in = NULL, *d_out = NULL;
    CUDA_CHECK(cudaMalloc((void**)&d_in, bytes));
    CUDA_CHECK(cudaMalloc((void**)&d_out, bytes));
    CUDA_CHECK(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));

    cudaEvent_t evA, evB;
    CUDA_CHECK(cudaEventCreate(&evA));
    CUDA_CHECK(cudaEventCreate(&evB));

    const int gridStream = (n + PROBE_BLOCK - 1) / PROBE_BLOCK;
    const double usefulBytes = (double)bytes;      // 每个探针"有用"的读字节数

    // ================= 探针 1：顺序（合并）读 =================
    for (int i = 0; i < 2; ++i) {
        streamReadKernel<<<gridStream, PROBE_BLOCK>>>(d_in, d_out, n);
    }
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(evA));
    for (int i = 0; i < STREAM_ITERS; ++i) {
        streamReadKernel<<<gridStream, PROBE_BLOCK>>>(d_in, d_out, n);
    }
    CUDA_CHECK(cudaEventRecord(evB));
    CUDA_CHECK(cudaEventSynchronize(evB));
    float ms1 = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms1, evA, evB));
    ms1 /= STREAM_ITERS;
```

```
    CUDA_CHECK(cudaMemcpy(h_out, d_out, bytes, cudaMemcpyDeviceToHost));
    const double bw1 = usefulBytes / (ms1 * 1.0e-3) / 1.0e9;

    printf("\n== Probe 1: coalesced sequential read ==\n");
    printf("  read bytes per iter  : %.1f MB\n", usefulBytes / 1048576.0);
    printf("  time                 : %.4f ms\n", ms1);
    printf("  achieved bandwidth   : %.1f GB/s  (%.1f%% of peak)\n",
           bw1, 100.0 * bw1 / peakBW);
    printf("  model                : 32 threads x 4 B = 128 B = 4 sectors, 100%% useful\n");
    verifyStream(h_out, h_in, n);

    // ================= 探针 2: 跨步读 =================
    for (int i = 0; i < 2; ++i) {
        stridedReadKernel<<<gridStream, PROBE_BLOCK>>>(d_in, d_out, W);
    }
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(evA));
    for (int i = 0; i < STREAM_ITERS; ++i) {
        stridedReadKernel<<<gridStream, PROBE_BLOCK>>>(d_in, d_out, W);
    }
    CUDA_CHECK(cudaEventRecord(evB));
    CUDA_CHECK(cudaEventSynchronize(evB));
    float ms2 = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms2, evA, evB));
    ms2 /= STREAM_ITERS;
    CUDA_CHECK(cudaMemcpy(h_out, d_out, bytes, cudaMemcpyDeviceToHost));
    const double bw2 = usefulBytes / (ms2 * 1.0e-3) / 1.0e9;

    printf("\n== Probe 2: strided read (stride = W*4 Byte) ==\n");
    printf("  read bytes per iter  : %.1f MB\n", usefulBytes / 1048576.0);
    printf("  time                 : %.4f ms\n", ms2);
    printf("  achieved bandwidth   : %.1f GB/s  (%.1f%% of peak)\n",
           bw2, 100.0 * bw2 / peakBW);
    printf("  model                : 32 addresses spaced %d B apart\n", W * 4);
    printf("                         -> 32 sectors x 32 B = 1024 B fetched, 128 B useful\n");
    printf("                         -> sector efficiency 12.5%%, i.e. 8x waste\n");
    printf("  slowdown vs probe 1  : %.2fx\n", ms2 / ms1);
    verifyStrided(h_out, h_in, W);

    // ================= 探针 3: 朴素矩阵乘法的访存流量 =================
    dim3 dimBlock(TILE_W, TILE_W, 1);
    dim3 dimGrid((W + TILE_W - 1) / TILE_W, (W + TILE_W - 1) / TILE_W, 1);
    CUDA_CHECK(cudaEventRecord(evA));
    naiveLoadsOnlyKernel<<<dimGrid, dimBlock>>>(d_in, d_in, d_out, W);
    CUDA_CHECK(cudaEventRecord(evB));
    CUDA_CHECK(cudaEventSynchronize(evB));
    float ms3 = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms3, evA, evB));
    CUDA_CHECK(cudaMemcpy(h_out, d_out, bytes, cudaMemcpyDeviceToHost));

    const double requested3  = 2.0 * (double)W * W * W * 4.0;      // 2*W^3 次 load
    const double warps3      = (double)(dimGrid.x * dimGrid.y) * 8.0;
    const double modelFetch3 = warps3 * (double)W * 128.0;         // 每 warp 每 k 128 B
    const double bw3         = requested3 / (ms3 * 1.0e-3) / 1.0e9;

    printf("\n== Probe 3: naive matmul access pattern without FMA ==\n");
    printf("  grid                 : (%u, %u, 1), block (%u, %u, 1)\n",
           dimGrid.x, dimGrid.y, dimBlock.x, dimBlock.y);
    printf("  requested bytes      : %.2f GB  (2*W^3 loads x 4 Byte)\n",
           requested3 / 1.0e9);
    printf("  modeled fetched bytes: %.2f GB  (warps x W x 128 B per warp per k)\n",
           modelFetch3 / 1.0e9);
    printf("  time                 : %.4f ms\n", ms3);
    printf("  requested bandwidth  : %.1f GB/s  (%.1f%% of peak)\n",
           bw3, 100.0 * bw3 / peakBW);
    printf("  DRAM-bound lower bnd : %.4f ms  (modeled fetch / peak BW)\n",
           modelFetch3 / (peakBW * 1e9) * 1.0e3);
    verifyLoadsOnly(h_out, h_in, W);

    // ---------- 释放 ----------
    CUDA_CHECK(cudaEventDestroy(evA));
    CUDA_CHECK(cudaEventDestroy(evB));
    CUDA_CHECK(cudaFree(d_in));
    CUDA_CHECK(cudaFree(d_out));
    free(h_in);
    free(h_out);
    CUDA_CHECK(cudaDeviceReset());
```

```
    return 0;
}
```

【代码做什么？】

1. **探针 1（`streamReadKernel`）**：建立"这台 GPU 在最优访存模式下能跑多快"的标尺。线程 `g` 读写 `in[g]` / `out[g]`，warp 内 32 个线程访问 32 个连续 float = **128 Byte 连续地址**，这是教科书式的完美合并访问；把它测出来的带宽当作 100% 基准。
2. **探针 2（`stridedReadKernel`）**：故意制造跨步。线程编号 `g` 被解释成 `(Row, Col) = (g/W, g%W)`，但真正访问的是 `in[Col*W + Row]`——也就是**把矩阵当作"列优先"来读**。相邻线程（`Col` 相差 1）的地址相差 `W*4 = 16384 Byte`。数据总量、指令条数、线程数都与探针 1 完全相同，**唯一的差别是地址分布**，因此两者的时间差就是"不合并"的纯粹代价。
3. **探针 3（`naiveLoadsOnlyKernel`）**：把示例 1 的 kernel 原封不动搬过来，只把 `Pvalue += d_M[Row*Width + k] * d_N[k*Width + Col]` 换成 `acc += d_M[Row*Width + k] + d_N[k*Width + Col]`（保留全部 load，去掉乘法）。这样测出的就是"**朴素矩阵乘法的访存流量本身要花多少时间**"。它可以和矩阵乘法的实测时间对比：如果二者接近，就证明矩阵乘法几乎完全在等内存。
4. **计时与数据量统计**：三个探针都用 `cudaEvent` 计时（探针 1、2 各热身 2 次、取 10 次平均；探针 3 只跑一次，因为它本身就要几百毫秒）。探针 3 另外打印两个关键的建模数字：**请求字节数** $2W^3 \times 4$（软件发出的 load 量）与**建模取回字节数** `warps × W × 128 B`（硬件按 sector 实际搬运的量）。
5. **主机端抽样校验**：三个探针都按固定步长抽样若干元素，用 CPU 重算同样的表达式逐位比较，输出 PASS/FAIL——确认"我们测的确实是我们要测的那个访问模式"。
6. **资源释放**：`cudaEventDestroy` / `cudaFree` / `cudaDeviceReset`。

【并行机制与硬件映射解说】

- **探针 1 的 warp 访问分布**：线程 `g = 0..31` 组成一个 warp，访问地址 `base + g*4`，跨度 128 Byte，恰好覆盖 **4 个 32 B sector**、通常落在 1 条 128 B cache line 内。硬件合并成最少量的请求，**取回 128 Byte 全部有用**。这类访问能把 A100 推到峰值带宽的 **85%–95%**（约 1300–1480 GB/s），差的那部分来自 DRAM 刷新、行切换与 ECC 开销——**任何程序都不可能超过探针 1 的实测值**，这是它作为基准的意义。
- **探针 2 的 warp 访问分布**（这是本实验的核心）：`text 线程 g 与其访问的地址（W = 4096，同一 warp 内 Row 不变、Col 递增） lane 0 -> 地址 = (Col+0)*4096 + Row -> 偏移 0 Byte lane 1 -> 地址 = (Col+1)*4096 + Row -> 偏移 16384 Byte lane 2 -> 地址 = (Col+2)*4096 + Row -> 偏移 32768 Byte lane 3 -> 地址 = (Col+3)*4096 + Row -> 偏移 49152 Byte （lane 4 到 lane 30 完全同理，每增一个 lane 偏移增加 16384 Byte） lane 31 -> 地址 = (Col+31)*4096 + Row -> 偏移 507904 Byte +----+ +----+ +----+ +----+ | 4B | | 4B | | 4B | | 4B | 每个方块是一个 32 B sector +----+ +----+ +----+ +----+ +----+ 0 B 16384 B 32768 B 507904 B |<---------- 一次 warp 请求覆盖 512 KB 的地址范围 ----------->| 请求数（transaction）= 32 个互不相邻的 sector 取回字节 = 32 x 32 B = 1024 B 有用字节 = 32 x 4 B = 128 B 效率 = 12.5% -> 理论减速 8 倍` 注意一个容易被忽略的细节：探针 2 的**写**（`out[g]`）是完全合并的，只有**读**跨步；因此它的实测带宽大约落在探针 1 的 1/4 到 1/8 之间——因为每 128 Byte 有用数据里还夹着 128 Byte 的有用写入，而写是高效的。**把读写分开看，读侧的效率就是 12.5%。**
- **探针 3 的 warp 访问分布**（与示例 1 的逐 warp 分析一致，此处按"整块 block"再算一次总账）：16×16 block 共 8 个 warp，每个 warp 在每次 k 上对 A 产生 2 个相距 4096 Byte 的地址（2 个 sector）、对 B 产生 16 个连续地址（2 个 sector）。于是

```
每 warp 每 k 取回   = 64 B (A) + 64 B (B) = 128 B
每 warp 每 k 有用   =  8 B (A) + 64 B (B) =  72 B
整个 grid 的 warp 数 = (W/16)^2 blocks * 8 warps = 65536 * 8 = 524288 (W = 4096)
建模取回总量         = 524288 * 4096 * 128 B = 274.9 GB
按峰值带宽的访存下界 = 274.9 GB / 1555 GB/s = 176.8 ms
请求总量（软件视角） = 2 * 4096^3 * 4 B = 549.8 GB
```

这两个数字的对比很有教育意义：**软件请求了 549.8 GB，硬件最少要搬 274.9 GB**（因为 L1/L2 会把重复请求与相邻请求合并掉一些），而真正跑矩阵乘法时还要再加上 FMA 的时间。这正是"访存爆炸"的量化形态。 – **占用率与延迟**：三个探针都是纯访存 kernel，寄存器用量个位数、占用率接近 100%，因此**延迟被大量并行请求掩盖得很好**——这一点很重要，它说明"性能差"不是没藏住延迟，而是**带宽真的被浪费掉了**。用 Little's Law 估算：要让 1555 GB/s 的带宽跑满、面对 400–800 cycle（284–567 ns）的延迟，GPU 需要约 `1555e9 × 300e-9 ≈ 466 KB` 的数据同时在飞，即约 3640 条 128 Byte 请求在途；探针 1 有 16,777,216 个线程、每个线程各发出 1 条 load（其中驻留在 SM 上的那部分同时在飞），请求数远超所需，因此延迟被充分掩盖。 – **cache 的影响必须诚实说明**：W = 4096 时单矩阵 64 MB，两个矩阵 128 MB，远超 40 MB 的 L2，所以探针 1、2 测的是真 DRAM；如果误用 W = 1024（单矩阵 4 MB），探针 2 的"跨步惩罚"会被 L2 命中大幅削弱（跨步访问取回的 32 B sector 里剩下的 28 B 后续会被用到），测出来的减速可能只有 1.5–3 倍而不是 8 倍。**做访存实验时，工作集大小与 L2 容量的关系是必须先想清楚的实验参数。**

【性能优化分析】

- **占用率**：三个探针都 > 90%，示例 1 的矩阵乘法也是 100%。**占用率不是瓶颈**，这不是一个"延迟受限（latency-bound）"的 kernel。
- **算术强度**：探针 1、2 的强度是 `1 FLOP / 8 Byte = 0.125 FLOP/Byte`（乘 2 是 1 次 FLOP）；示例 1 的矩阵乘法是 **0.25 FLOP/Byte**；A100 的机器平衡点是 **12.5 FLOP/Byte**——**三者都远在带宽受限区**（探针 3 干脆是 0 FLOP/Byte 的纯流量实验）。
- **Roofline 交叉验证**：探针 1 实测约 85%–95% 峰值带宽，说明"屋顶"是真实的；由它推出的矩阵乘法上限 `1555 × 0.25 = 389 GFLOP/s` 因此不是纸上数字。反过来，用探针 2 测到的有效带宽（例如 1555 × 0.125 ≈ 194 GB/s）去乘算术强度，就得到"**如果整个 kernel 都按跨步模式访问**"时的上限只有 194 × 0.25 ≈ **49 GFLOP/s**——这是朴素实现的悲观下界，实际值介于 49 与 389 GFLOP/s 之间，取决于跨步那一半访问占多大比重（本例中 A 侧跨步占请求量的一半）。

- **本实验的结论与三种改进方向**：① **把跨步访问改成合并访问**（改布局 / 转置 / 调整线程映射）：由探针 1 与探针 2 的 8 倍差距可以看出这条路的天花板很高，但在 thread-per-output 下最多只能把每 warp 每 k 的搬运量从 128 B 降到 96 B（1.33 倍），**收益有限**——因为请求总量没变。② **引入共享内存分块**：把请求总量降低 `TILE_WIDTH` 倍，是唯一能同时解决"总量"与"模式"的路线（下一讲）。③ **线程粗化**：让每线程算多个输出，用寄存器复用把强度从 0.25 抬到 1–2 FLOP/Byte，属于"顺手能拿的便宜"，但要跨过 12.5 必须与 ② 联用。

示例 3：存储布局实验 —— A 转置（恢复合并）与 B 转置（制造跨步）（matmul_transposed.cu）

本示例回答一个具体问题：**"把某个操作数转置后再传给 kernel"到底能不能救朴素矩阵乘法？** 程序同时跑三个代数上完全等价、只有访存布局不同的 kernel：`matmulNaive`（A、B 都按行主序）、`matmulAMT`（A 以转置形式 `M_T` 给出）、`matmulBT`（B 以转置形式 `N_T` 给出），并用一个完整的 `transposeKernel` 在 GPU 上生成两份转置矩阵。

```
// 文件: matmul_transposed.cu
// 主题: ECE408 Lecture 4 —— 存储布局对合并访存的影响 (A 转置 vs B 转置)
// 编译: nvcc -O3 -arch=sm_80 matmul_transposed.cu -o matmul_transposed
// 运行: ./matmul_transposed 2048
//       参数为矩阵边长 Width (默认 2048, 单矩阵 16 MB, 三个矩阵共 48 MB > A100 L2 40 MB)

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <ctime>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define BLOCK_WIDTH  16
#define WARMUP_ITERS  3
#define TIMED_ITERS  10

// ============================== device kernels ==============================
// (a) 转置 kernel: out[c*Width + r] = in[r*Width + c]
//     读: 相邻 tx -> 相邻地址 (合并); 写: 相邻 tx 的地址相差 Width*4 Byte (跨步)
__global__ void transposeKernel(const float* __restrict__ in,
                                float* __restrict__ out, int Width)
{
    const int Row = blockIdx.y * blockDim.y + threadIdx.y;
    const int Col = blockIdx.x * blockDim.x + threadIdx.x;
    if (Row < Width && Col < Width) {
        out[Col * Width + Row] = in[Row * Width + Col];
    }
}

// (b) 朴素版: A、B 都是行主序。A 侧跨步 (2 个 sector), B 侧合并 (2 个 sector)
__global__ void matmulNaiveKernel(const float* __restrict__ d_M,
                                  const float* __restrict__ d_N,
                                  float* __restrict__ d_P, int Width)
{
    const int Row = blockIdx.y * blockDim.y + threadIdx.y;
    const int Col = blockIdx.x * blockDim.x + threadIdx.x;
    if (Row < Width && Col < Width) {
        float Pvalue = 0.0f;
        for (int k = 0; k < Width; ++k) {
            Pvalue += d_M[Row * Width + k] * d_N[k * Width + Col];
        }
        d_P[Row * Width + Col] = Pvalue;
    }
}

// (c) A 转置版: M_T[k][Row] = M[Row][k]。A 侧变成"2 个相邻地址" (1 个 sector)
__global__ void matmulAMTKernel(const float* __restrict__ d_MT,
                                const float* __restrict__ d_N,
                                float* __restrict__ d_P, int Width)
{
    const int Row = blockIdx.y * blockDim.y + threadIdx.y;
    const int Col = blockIdx.x * blockDim.x + threadIdx.x;
    if (Row < Width && Col < Width) {
        float Pvalue = 0.0f;
        for (int k = 0; k < Width; ++k) {
            Pvalue += d_MT[k * Width + Row] * d_N[k * Width + Col];
        }
        d_P[Row * Width + Col] = Pvalue;
    }
}

// (d) B 转置版: N_T[Col][k] = N[k][Col], 即把 B 按列主序存储后按行读取
//     B 侧变成"16 个相距 Width*4 Byte 的地址" (16 个独立 sector)
__global__ void matmulBTKernel(const float* __restrict__ d_M,
                               const float* __restrict__ d_NT,
                               float* __restrict__ d_P, int Width)
{
```

```
    const int Row = blockIdx.y * blockDim.y + threadIdx.y;
    const int Col = blockIdx.x * blockDim.x + threadIdx.x;
    if (Row < Width && Col < Width) {
        float Pvalue = 0.0f;
        for (int k = 0; k < Width; ++k) {
            Pvalue += d_M[Row * Width + k] * d_NT[Col * Width + k];
        }
        d_P[Row * Width + Col] = Pvalue;
    }
}

// ============================== host utilities ==============================
static void initMatrix(float* a, int n, unsigned int seed)
{
    srand(seed);
    for (int i = 0; i < n * n; ++i) a[i] = (float)(rand() % 7) * 0.25f;
}

static void cpuMatmul(const float* A, const float* B, float* C, int n)
{
    for (int i = 0; i < n * n; ++i) C[i] = 0.0f;
    for (int i = 0; i < n; ++i) {
        for (int k = 0; k < n; ++k) {
            const float  a    = A[i * n + k];
            const float* Brow = B + k * n;
            float*       Crow = C + i * n;
            for (int j = 0; j < n; ++j) Crow[j] += a * Brow[j];
        }
    }
}

static void cpuTranspose(const float* in, float* out, int n)
{
    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < n; ++c) {
            out[c * n + r] = in[r * n + c];
        }
    }
}

static int fp32CoresPerSM(int major, int minor)
{
    if (major == 7) return 64;
    if (major == 8) return (minor == 0) ? 64 : 128;
    if (major >= 9) return 128;
    return 128;
}

// 计时辅助：把一次 kernel 启动包成可调用对象，返回平均毫秒数
template <typename LaunchFn>
static double timeKernel(LaunchFn launch, int warmup, int iters)
{
    for (int i = 0; i < warmup; ++i) launch();
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
    cudaEvent_t a, b;
    CUDA_CHECK(cudaEventCreate(&a));
    CUDA_CHECK(cudaEventCreate(&b));
    CUDA_CHECK(cudaEventRecord(a));
    for (int i = 0; i < iters; ++i) launch();
    CUDA_CHECK(cudaEventRecord(b));
    CUDA_CHECK(cudaEventSynchronize(b));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, a, b));
    CUDA_CHECK(cudaEventDestroy(a));
    CUDA_CHECK(cudaEventDestroy(b));
    return (double)ms / iters;
}

static void verifyMatrix(const float* got, const float* want, int n, const char* tag)
{
    double maxAbs = 0.0, maxRef = 0.0;
    for (int i = 0; i < n * n; ++i) {
        const double e = fabs((double)got[i] - (double)want[i]);
        if (e > maxAbs) maxAbs = e;
        const double r = fabs((double)want[i]);
        if (r > maxRef) maxRef = r;
    }
```

```
    const double rel = (maxRef > 0.0) ? maxAbs / maxRef : maxAbs;
    printf("  %-26s: max abs err %.3e, rel %.3e -> %s\n",
           tag, maxAbs, rel, (rel < 1e-4) ? "PASS" : "FAIL");
}

// ================================ main ================================
int main(int argc, char** argv)
{
    const int    N     = (argc > 1) ? atoi(argv[1]) : 2048;
    const size_t bytes = (size_t)N * (size_t)N * sizeof(float);

    CUDA_CHECK(cudaSetDevice(0));
    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    const double peakBW   = 2.0 * (double)prop.memoryClockRate * 1.0e3
                          * ((double)prop.memoryBusWidth / 8.0) / 1.0e9;
    const double peakFP32 = (double)prop.multiProcessorCount
                          * (double)fp32CoresPerSM(prop.major, prop.minor)
                          * 2.0 * (double)prop.clockRate * 1.0e3 / 1.0e12;
    const double flops    = 2.0 * (double)N * (double)N * (double)N;

    printf("== Device ==\n");
    printf("  %s, peak BW %.1f GB/s, peak FP32 %.2f TFLOPS, L2 %.1f MB\n",
           prop.name, peakBW, peakFP32, prop.l2CacheSize / 1048576.0);
    printf("  N = %d, three matrices = %.1f MB\n", N, 3.0 * bytes / 1048576.0);

    // ---------- 主机端数据与参考结果 ----------
    float* h_M   = (float*)malloc(bytes);
    float* h_N   = (float*)malloc(bytes);
    float* h_MT  = (float*)malloc(bytes);
    float* h_NT  = (float*)malloc(bytes);
    float* h_P   = (float*)malloc(bytes);
    float* h_Ref = (float*)malloc(bytes);
    if (!h_M || !h_N || !h_MT || !h_NT || !h_P || !h_Ref) {
        fprintf(stderr, "host malloc failed\n");
        exit(EXIT_FAILURE);
    }
    initMatrix(h_M, N, 3u);
    initMatrix(h_N, N, 4u);
    cpuTranspose(h_M, h_MT, N);
    cpuTranspose(h_N, h_NT, N);

    const clock_t t0 = clock();
    cpuMatmul(h_M, h_N, h_Ref, N);
    const double cpuSec = (double)(clock() - t0) / CLOCKS_PER_SEC;

    // ---------- 设备端内存 ----------
    float *d_M = NULL, *d_N = NULL, *d_MT = NULL, *d_NT = NULL, *d_P = NULL;
    CUDA_CHECK(cudaMalloc((void**)&d_M,  bytes));
    CUDA_CHECK(cudaMalloc((void**)&d_N,  bytes));
    CUDA_CHECK(cudaMalloc((void**)&d_MT, bytes));
    CUDA_CHECK(cudaMalloc((void**)&d_NT, bytes));
    CUDA_CHECK(cudaMalloc((void**)&d_P,  bytes));
    CUDA_CHECK(cudaMemcpy(d_M, h_M, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_N, h_N, bytes, cudaMemcpyHostToDevice));

    dim3 dimBlock(BLOCK_WIDTH, BLOCK_WIDTH, 1);
    dim3 dimGrid((N + BLOCK_WIDTH - 1) / BLOCK_WIDTH,
                 (N + BLOCK_WIDTH - 1) / BLOCK_WIDTH, 1);

    // ---------- 转置 (GPU) ----------
    const double msT_M = timeKernel([&] {
        transposeKernel<<<dimGrid, dimBlock>>>(d_M, d_MT, N);
    }, WARMUP_ITERS, TIMED_ITERS);
    const double msT_N = timeKernel([&] {
        transposeKernel<<<dimGrid, dimBlock>>>(d_N, d_NT, N);
    }, WARMUP_ITERS, TIMED_ITERS);
    const double transposeGBs = 2.0 * (double)bytes / (msT_M * 1.0e-3) / 1.0e9;

    CUDA_CHECK(cudaMemcpy(h_P, d_MT, bytes, cudaMemcpyDeviceToHost));
    printf("\n== Transpose kernel ==\n");
    printf("  time per transpose   : %.4f ms (M_T), %.4f ms (N_T)\n", msT_M, msT_N);
    printf("  effective bandwidth  : %.1f GB/s (read+write counted)\n", transposeGBs);
    printf("  model                : read coalesced (2 sectors), write strided (16 sectors)\n");
    verifyMatrix(h_P, h_MT, N, "M_T = transpose(M)");
    CUDA_CHECK(cudaMemcpy(h_P, d_NT, bytes, cudaMemcpyDeviceToHost));
    verifyMatrix(h_P, h_NT, N, "N_T = transpose(N)");
```

```
    // ---------- 三个矩阵乘法 kernel ----------
    const double msNaive = timeKernel([&] {
        matmulNaiveKernel<<<dimGrid, dimBlock>>>(d_M, d_N, d_P, N);
    }, WARMUP_ITERS, TIMED_ITERS);
    CUDA_CHECK(cudaMemcpy(h_P, d_P, bytes, cudaMemcpyDeviceToHost));
    printf("\n== matmulNaiveKernel (A row-major, B row-major) ==\n");
    printf("  time                : %.4f ms   ->  %.2f GFLOP/s\n",
           msNaive, flops / (msNaive * 1.0e-3) / 1.0e9);
    printf("  non-coalesced side  : A  (2 sectors = 64 B fetched for 8 B useful)\n");
    verifyMatrix(h_P, h_Ref, N, "P = M*N (naive)");

    const double msAMT = timeKernel([&] {
        matmulAMTKernel<<<dimGrid, dimBlock>>>(d_MT, d_N, d_P, N);
    }, WARMUP_ITERS, TIMED_ITERS);
    CUDA_CHECK(cudaMemcpy(h_P, d_P, bytes, cudaMemcpyDeviceToHost));
    printf("\n== matmulAMTKernel (A transposed) ==\n");
    printf("  time                : %.4f ms   ->  %.2f GFLOP/s\n",
           msAMT, flops / (msAMT * 1.0e-3) / 1.0e9);
    printf("  A side              : 2 adjacent addresses -> 1 sector = 32 B (was 64 B)\n");
    verifyMatrix(h_P, h_Ref, N, "P = M*N (A transposed)");

    const double msBT = timeKernel([&] {
        matmulBTKernel<<<dimGrid, dimBlock>>>(d_M, d_NT, d_P, N);
    }, WARMUP_ITERS, TIMED_ITERS);
    CUDA_CHECK(cudaMemcpy(h_P, d_P, bytes, cudaMemcpyDeviceToHost));
    printf("\n== matmulBTKernel (B stored column-major) ==\n");
    printf("  time                : %.4f ms   ->  %.2f GFLOP/s\n",
           msBT, flops / (msBT * 1.0e-3) / 1.0e9);
    printf("  B side              : 16 addresses spaced Width*4 B -> 16 sectors = 512 B\n");
    verifyMatrix(h_P, h_Ref, N, "P = M*N (B column-major)");

    // ---------- 汇总 ----------
    printf("\n== Summary (N = %d) ==\n", N);
    printf("  kernel                        ms      GFLOP/s   warp-bytes per k   sector eff\n");
    printf("  naive   (A row, B row)     %7.4f   %8.2f        128 B               56%%\n",
           msNaive, flops / (msNaive * 1.0e-3) / 1.0e9);
    printf("  A-trans (A^T, B row)       %7.4f   %8.2f         96 B               75%%\n",
           msAMT, flops / (msAMT * 1.0e-3) / 1.0e9);
    printf("  B-trans (A row, B col)     %7.4f   %8.2f        576 B               12.5%%\n",
           msBT, flops / (msBT * 1.0e-3) / 1.0e9);
    printf("  ratio B-trans / A-trans    %.2fx\n", msBT / msAMT);
    printf("  ratio B-trans / naive      %.2fx\n", msBT / msNaive);
    printf("  ratio naive   / A-trans    %.2fx\n", msNaive / msAMT);

    printf("\n== Roofline view (naive) ==\n");
    printf("  arithmetic intensity : 0.25 FLOP/Byte (4 Byte per FLOP)\n");
    printf("  roofline ceiling     : %.1f GFLOP/s (%.1f GB/s x 0.25)\n",
           peakBW * 0.25, peakBW);
    printf("  ceiling / peak FP32  : %.2f%%\n", 100.0 * peakBW * 0.25 / (peakFP32 * 1.0e3));
    printf("  CPU reference time   : %.3f s\n", cpuSec);

    // ---------- 释放 ----------
    CUDA_CHECK(cudaFree(d_M));
    CUDA_CHECK(cudaFree(d_N));
    CUDA_CHECK(cudaFree(d_MT));
    CUDA_CHECK(cudaFree(d_NT));
    CUDA_CHECK(cudaFree(d_P));
    free(h_M); free(h_N); free(h_MT); free(h_NT); free(h_P); free(h_Ref);
    CUDA_CHECK(cudaDeviceReset());
    return 0;
}
```

【代码做什么？】

1. **准备六份主机矩阵**：`h_M`、`h_N`（两个输入）、`h_MT`、`h_NT`（CPU 版转置，用作校验基准）、`h_P`（接收 GPU 结果）、`h_Ref`（CPU 参考结果）。
2. **`transposeKernel`（GPU 转置）**：`out[Col*Width + Row] = in[Row*Width + Col]`。**读是合并的**（相邻 `tx` → 相邻地址），**写是跨步的**（`tx` 每 +1，写地址 +`Width*4` Byte），所以这个 kernel 本身就是"一半好一半坏"的典型；实测带宽通常只有峰值的一半左右。行业中真正的做法是下一讲的**共享内存分块转置**（先把 tile 合并读进来，再交换行列后合并写出），这里刻意用最朴素的版本以保持"本讲不使用共享内存"的边界。
3. **`matmulNaiveKernel`**：基准版，与示例 1 完全一致。
4. **`matmulAMTKernel`**：把 A 换成它的转置 `M_T`（`M_T[k][Row] = M[Row][k]`），内积写成 `d_MT[k*Width + Row] * d_N[k*Width + Col]`。代数上完全等价（乘法的两个因子顺序不变，只是取数方式变了），但 **A 侧的地址从"相距 `Width*4` Byte"变成"相距 4 Byte"**。
5. **`matmulBTKernel`**：把 B 换成它的转置 `N_T`（`N_T[Col][k] = N[k][Col]`），内积写成 `d_M[Row*Width + k] * d_NT[Col*Width + k]`。这模拟了"B 按列主序存储（或按列读取）"的情形，**B 侧的 16 个线程会访问 16 个相距 `Width*4` Byte 的地址**——就是概念 4 表格第三行那个 512 Byte / 64 Byte 的最

坏情形。

6. **三次计时与三次校验**：每个 kernel 都用 `timeKernel` 热身 3 次、计时 10 次取平均，然后各自拷回结果与 `h_Ref` 比较。**三个 kernel 的输出必须逐元素一致**——这正是本实验的价值：三个程序的数学结果完全相同，性能差别 100% 来自访存布局。
7. **汇总与 Roofline**：打印三个 kernel 的毫秒数、GFLOP/s、每个 warp 每 k 的建模搬运字节、sector 效率，以及两两时间比。

【并行机制与硬件映射解说】

- **warp 划分与三个 kernel 的差异**：三个 kernel 的 `dimBlock`、`dimGrid`、线程到输出的映射**完全一样**（都是 16×16、`Row ← ty`、`Col ← tx`），因此 warp 的组成、占用率、发散情况也都一样。**唯一的差别是每条 load 的地址表达式**——这就是控制变量法的正确用法。
- **逐 warp 的取数分布（每 warp 每 k）**：三个 kernel 的线程映射相同，所以差别只体现在两条 load 的地址表达式上：

kernel	A 侧 warp 地址分布	A 取回	B 侧 warp 地址分布	B 取回	合计取回	有用	sector 效率
`matmulNaive`（A 行主序, B 行主序）	2 个地址，相距 `Width*4` Byte	64 B	16 个连续地址（64 Byte）	64 B	128 B	72 B	56%
`matmulAMT`（A 转置, B 行主序）	2 个地址，相距 4 Byte（同一 sector）	32 B	16 个连续地址（64 Byte）	64 B	96 B	72 B	75%
`matmulBT`（A 行主序, B 列主序）	2 个地址，相距 `Width*4` Byte	64 B	**16 个地址，各相距 `Width*4` Byte**	**512 B**	**576 B**	72 B	**12.5%**

把第三行的 B 侧单独画出来（N = 2048，`Width*4 = 8192 Byte`）：

```
lane  0 .. 15 访问 d_NT[Col*Width + k]，Col 每 +1 地址 +8192 Byte

+-----+       +-----+       +-----+       +-----+             +-----+
| 4B |       | 4B |       | 4B |       | 4B |             | 4B |
+-----+       +-----+       +-----+       +-----+             +-----+
0 B         8192 B       16384 B      24576 B            122880 B
|<---- 16 个互不相邻的 32 B sector，需要 16 次独立取数请求 ---->|
取回 16 * 32 B = 512 B      有用 16 * 4 B = 64 B      效率 12.5%
```

- **对 B 的访问为什么在 `matmulBTKernel` 里变成 16 次独立 transaction**：`d_NT[Col*Width + k]` 的地址里，`Col = bx*16 + tx` 只随 `tx` 变化，而 `tx` 在 warp 内取 16 个值，每个值使地址增加 `Width*4 = 8192 Byte`（N = 2048）。这 16 个地址彼此相距 8192 Byte，**必然落在 16 个不同的 32 B sector、16 条不同的 cache line 上**，硬件必须发出 16 次互不相干的取数请求，每次只带回 4 Byte 有用的数据。这 16 次请求的代价就是 **512 Byte 换 64 Byte**——8 倍浪费。相比之下 `matmulNaiveKernel` 的 B 访问是 16 个连续 float（64 Byte），落在一条 cache line 内的 2 个 sector 上，1 次请求解决。
- **对 A 的访问为什么在 `matmulAMTKernel` 里变好（但只好了 2 倍）**：`d_MT[k*Width + Row]` 的地址随 `Row = by*16 + ty` 变化，warp 内 `ty` 只有 2 个取值（相差 1），所以两个地址是**相邻的 4 Byte**，落在同一个 32 B sector 里。transaction 从 2 次降到 1 次、取回从 64 B 降到 32 B——改善，但有限：（1）每 warp 每 k 的搬运量只从 128 B 降到 96 B（1.33 倍）；（2）**A 侧的请求总量一次都没减少**，仍然是每线程 K 次 load。这也是为什么 A 转置带来的实测加速通常只有百分之几到 30%，而 B 转置带来的减速却是成倍的。
- **占用率与寄存器**：三个 kernel 的寄存器用量都在 20–30 之间（编译器可能因为地址计算方式不同而略有差异，示例 1 的 `cudaFuncGetAttributes` 可以打印出来核对），16×16 block 在 A100 上都是 **8 blocks/SM = 2048 threads = 100% 占用率**（若寄存器超过 32 个就降到 6–7 个 block）。**占用率完全相同而性能差数倍**，这正是"占用率不是万能指标"的最好例证。
- **共享内存与 bank conflict 在本讲的适用性**：本讲（以及 Lab 2）**完全不用共享内存**，所以"bank conflict"这个问题不存在——它属于下一讲：一旦把 tile 装进 `__shared__ float subTileN[16][16]`，`subTileN[k][tx]` 的 bank 号就是 `(k*16 + tx) % 32`，`tx = 0..15` 对应 16 个不同的 bank，warp 内两个 ty 组访问的是**同一批地址**（被硬件广播），因此 **16×16 的 tile 在 `[k][tx]` 访问模式下没有 bank conflict**；但若 tile 宽度写成 32 并且按列访问，就会出现 2-way / 32-way 冲突，那时才需要 padding 或重排下标。
- **关于 DRAM 的层次**：`transposeKernel` 的"读合并 + 写跨步"结构正好对应 Lecture 7 讲的 DRAM 行为——**写跨步时，每次 32 B 的写只填 4 Byte，剩下的字节被浪费**；而且写通常比读更难掩盖（没有"等着用"的指令可以继续执行）。这也解释了为什么实测的转置带宽通常明显低于探针 1 的纯读带宽。

【性能优化分析】

- **算术强度**：三个 kernel 的强度都是 `2N³ FLOP / (8N³ Byte) = 0.25 FLOP/Byte`（请求口径）——**完全相同**。这提醒我们：算术强度只看"请求量与计算量的比"，**它对"请求内部的地址分布好不好"是完全无感的**，那需要 sector 效率（本讲第二个指标）来补充。两个指标必须一起看。
- **Roofline**：上限都是 `1555 × 0.25 = 389 GFLOP/s`（峰值的 2.0%），三个 kernel 的差别不是"能不能突破屋顶"，而是"离屋顶有多远"。调用 `matmulBTKernel` 的有效搬运量是 naive 的 4.5 倍（576 B vs 128 B），因此它离屋顶更远。
- **占用率**：三者都是 100%，**再次确认瓶颈在内存系统而不在调度**。
- **预期结果与解读**：`matmulAMT` 通常比 `matmulNaive` 快 **5%–30%**（取决于 N 是否超出 L2，以及编译器是否把 A 的两次 load 优化成一次）；`matmulBTKernel` 通常比 `matmulNaive` **慢 1.5–4 倍**，在 N 较大（工作集超出 L2）时更接近模型预测的 4.5 倍。**如果 `matmulAMT` 的加速远小于预期，不要怀疑实验做错了**——这正是本讲最重要的工程结论：**转置/改布局只能修正"访问模式"，修正不了"请求总量"**。
- **三种改进方向（本讲的总结性结论）**：

```
方向 (1) 把跨步的列访问改成合并访问:
          手段: 改变存储布局 (转置某一个操作数)、调整线程映射让"变化最快的
                线程索引"落在"内存中连续的那一维"、必要时把转置的成本
                (一次 O(N^2) 的拷贝) 算进总时间。
          上限: 把每 warp 每 k 的搬运量从 128 B 降到 96 B (1.33x),
                或在纯跨步 (探针 2) 情形下最高 8x。
          局限: 只能搬走"跨步", 搬不走"2MNK 次请求"; 而且 thread-per-output
                下 A 复用与 B 复用的需求互相冲突, 必然有一个操作数受害。

方向 (2) 引入共享内存做分块 (tiling): <-- 真正解决问题的是这一条
          把全局请求量降低 TILE_WIDTH 倍 -> AI 从 0.25 提升到 TILE_WIDTH/4
          TILE=16 -> 6.22 TFLOP/s 上限; TILE=32 -> 12.44 TFLOP/s 上限
          并顺带把跨步访问关进片上 (共享内存只有 32 个 bank 的并行访问,
          没有 sector 概念, "按列访问"不再是灾难) ——即讲义的 corner turning。

方向 (3) 线程粗化 (thread coarsening): 每线程算 r x c 个输出
          AI(r,c) = r*c / (2*(r+c)) FLOP/Byte
          r=c=1 -> 0.25 ; r=1,c=4 -> 0.4 ; r=c=4 -> 1.0 ; r=c=8 -> 2.0
          收益来自寄存器级复用, 是"顺手能拿的便宜", 但单独使用跨不过 12.5;
          与 TILE=16 分块叠加后才能到 TILE*r*c/(2*(r+c)) = 16 FLOP/Byte,
          从而真正进入计算受限区间 (这是后续实验的性能目标)。
```

性能优化技巧总结

1. **让"变化最快的线程索引"落在"内存中连续的那一维"**：`threadIdx.x` 必须对应行主序矩阵的列，否则一个 warp 的地址会散落在 32 个 sector 上，带宽直接损失 8 倍。
2. **用 sector 效率而不是"是否连续"来判断访存好坏**：连续是感性描述，`(有用字节 / 取回字节)` 是硬指标；16×16 block、Width=1024 的 naive kernel 两个操作数分别是 12.5% 与 100%，整体 56%。
3. **不要为占用率而优化占用率**：先算算术强度与 Roofline 上限；本讲的 kernel 占用率 100% 却只有峰值的百分之几，说明它是带宽受限，此时提高占用率毫无收益。
4. **分清"请求量"与"访问模式"两个独立问题**：转置/改布局只能改善访问模式（1.33 倍级别），降低请求量必须靠数据复用（分块，`TILE_WIDTH` 倍）——两者的数量级完全不同。
5. **把矩阵开大到超过 L2 再做访存实验**：A100 的 L2 有 40 MB，N=1024 时整个工作集都在 cache 里，会把"跨步惩罚"掩盖掉一大半，得到的结论不可靠。
6. **用 `cudaEvent` 计时并做热身**：kernel 启动是异步的，用主机端时钟会把 API 开销算进去；第一次运行还包含上下文初始化与页表建立的开销。
7. **边界保护用"向上取整 + if"而不是"假设整除"**：`dimGrid = ceil(W/BLOCK)` 配合 `if (Row < Width && Col < Width)` 是通用写法，代价只是边界 block 的少量发散。
8. **CPU 参考实现要用缓存友好的循环顺序**：i–k–j 顺序比 i–j–k 快 5–10 倍，能把等待参考结果的时间从几分钟降到几秒。
9. **任何"优化"都要回到三个指标上定量说明**：时间/带宽（实测）、算术强度（请求口径）、sector 效率（硬件口径）；只报"变快了"而不给出这三个数字的优化说明是不可信的。

关键要点

- 矩阵乘法的计算量是 `2·M·N·K` FLOP、数据量是 `O(N²)`，**理想算术强度 = N/6 FLOP/Byte**（N=1024 时 170.7），远高于 A100 的机器平衡点 12.5——问题不在于算法，而在于实现没有利用上这份复用。
- 行主序线性化 `M[row*Width + col]` 决定了两个距离：同行相邻列相差 4 Byte（可合并），同列相邻行相差 `Width*4` Byte（跨步）。**所有访存悲剧都来自第二个距离。**
- "每线程一个输出元素"的朴素实现对每个输出做 `2K` 次全局内存访问，总计 `2·M·N·K` 次 load，即 **4 Byte/FLOP**；A 的每个元素被读 N 次、B 的每个元素被读 M 次，全部是冗余请求。
- warp 级的代价：16×16 block 时每个 warp 每 k 对 A 产生 2 个相距 4096 Byte 的 sector（**效率 12.5%**）、对 B 产生 64 Byte 连续访问（效率 100%），**整体效率 56%**；若按列读 B（`d_NT[Col*Width+k]`），B 侧会变成 **16 次独立 transaction、512 Byte 换 64 Byte**。
- **Roofline 上限 = 1555 GB/s × 0.25 FLOP/Byte ≈ 389 GFLOP/s，仅为 19.5 TFLOPS 峰值的 2%**；A100 要达到峰值需要 `19.5e12/0.25 = 78 TB/s` 的带宽，是实际带宽的 50 倍。
- 唯一的出路是**数据复用**：分块把全局请求量降低 `TILE_WIDTH` 倍，`AI = TILE_WIDTH/4`；16×16 分块给出 6.22 TFLOP/s 上限（31.9%），32×32 给出 12.44 TFLOP/s，**要越过机器平衡点还需要 16×16 分块叠加 4×4 寄存器粗化（AI = 16 FLOP/Byte）**。

常见陷阱与注意事项

- **把 `threadIdx.x` 映射到错误的维度**：现象是性能只有预期的几分之一，代码却完全正确 → 记住"x 是变化最快的线程索引，通常应映射到内存中连续的那一维（行主序的列）"，写代码前先把 `Row/Col` 两行索引与内存布局画在一起核对。
- **用主机端时钟给 kernel 计时**：现象是测出的时间随迭代次数剧烈波动，或把 `cudaMalloc` / `cudaMemcpy` 的开销算进了 kernel → 用 `cudaEventRecord` + `cudaEventSynchronize`，并在正式计时前热身若干次；反复出现的"忘记 `cudaDeviceSynchronize()`"是同一类错误的变体：**不调用它，`cudaMemcpy` 之后的性能打印读到的是没有意义的中间量**（`cudaMemcpy` 本身是同步的，但它掩盖不了自己之前没有同步的计时）。

- **对 `cudaMalloc` / `cudaMemcpy` / kernel 启动不检查返回值**：现象是 kernel 因越界或共享内存超限启动失败，程序却"正常"输出一堆零或垃圾数据 → 统一使用 `CUDA_CHECK` 宏，并在 kernel 启动后调用 `cudaGetLastError()` 与 `cudaDeviceSynchronize()` 让错误在发生处暴露。
- **`cudaMemcpy` 方向写反**：现象是结果全为 0 或直接报 invalid argument（host 指针给了 `cudaMemcpyDeviceToHost`）→ 记住第三个参数必须是 `cudaMemcpyHostToDevice`（主机→设备）或 `cudaMemcpyDeviceToHost`（设备→主机），第四次参数是字节数而不是元素个数。
- **整数除法取整导致网格覆盖不足**：现象是结果矩阵右下角一片未初始化（对 `N/BLOCK` 直接整除的写法），取整丢掉了不足一个 block 的尾巴 → 网格维度必须写成 `(N + BLOCK - 1) / BLOCK`，并在 kernel 内用 `if (Row < Width && Col < Width)` 挡住多余线程；**越界写显存可能不立即报错**，但会静默破坏其他缓冲区。
- **host/device 指针混用**：现象是段错误（把 `d_` 指针交给 CPU 循环）或 `invalid argument`（把 `h_` 指针传给 kernel）→ 用命名规范（`h_` / `d_` 前缀）区分，并记住 kernel 只能接收设备指针、`cpuMatmul` 只能接收主机指针。
- **在不用共享内存的版本里"顺手"加上 `__syncthreads()`**：现象是编译通过但语义错误或性能无变化 → 本讲与 Lab 2 的 kernel 没有跨线程通信，`__syncthreads()` 不需要；它属于分块版本，而且**绝不能放在条件分支内部**（"所有线程必须进入同一个静态调用点"，否则行为未定义或直接死锁）。
- **以为"把某个矩阵转置一下"就能解决性能问题**：现象是花时间写好转置 kernel 后性能只提升了十几个百分点 → 转置只修正访问模式（本讲实测上限约 1.33 倍），**请求总量不变**；真正的量级改善必须来自共享内存分块与寄存器复用。
- **共享内存容量超限（下一讲的预警）**：现象是 kernel 启动失败或占用率骤降 → 32×32 tile 需要 `2 × 32 × 32 × 4 = 8 KB` 静态共享内存，64×64 需要 32 KB（超过很多 GPU 每 block 的默认上限 48 KB 之外还要走 `cudaFuncSetAttribute`），写分块代码时必须先算 `2 × TILE_WIDTH² × 4 Byte` 与设备每 block 上限。
- **只在小矩阵上测性能就下结论**：现象是"我的优化没有效果"或"跨步一点都不慢" → 小矩阵的工作集全在 L2/L1 里，掩盖了 DRAM 行为；评估访存优化时矩阵至少要大到超过 L2 容量（A100 为 40 MB，即 N ≥ 4096 的 float 方阵）。

思考题（带答案）

Q1. 朴素矩阵乘法（A、B 均行主序）在 A100 上执行，`blockDim` 取哪一组时"每 warp 每 k 的搬运用"更划算：(a) 16×16，还是 (b) 32×32？请分别算出 A、B 两侧的 sector 数、取回字节与有用字节（Width 足够大，忽略 cache）。

(a) 16×16 = 256 线程，一个 warp 覆盖 `ty ∈ {t, t+1}` 两行、`tx = 0..15`：A 侧地址只依赖 ty，产生 **2 个**相距 `Width*4` Byte 的地址 → 2 个 sector、取回 64 B、有用 8 B；B 侧地址只依赖 tx，产生 16 个连续地址（64 Byte）→ 2 个 sector、取回 64 B、有用 64 B。合计 **128 B 取回 / 72 B 有用 / 效率 56%**。(b) 32×32 = 1024 线程，一个 warp 恰好覆盖一整行 `ty = t`、`tx = 0..31`：A 侧 32 个线程访问**同一个地址**（广播）→ 1 个 sector、取回 32 B、有用 4 B；B 侧 32 个连续地址 = 128 Byte → 4 个 sector、取回 128 B、有用 128 B。合计 **160 B 取回 / 132 B 有用 / 效率 82.5%。注意结论的反直觉之处**：单看"效率"，32×32 更漂亮（82.5% > 56%），但**按每个 warp 每 k 的取回字节算，16×16 反而更省**（128 B vs 160 B），因为 16×16 的 warp 用"两行 ty 复用同一批 B 数据"换来了 A 的广播摊薄。这也是为什么真正的答案要靠分块（下一讲）——无论 16 还是 32，请求总量都没有改变。

Q2. 用 Roofline 模型估算：在 A100 上把朴素版改成 16×16 分块后，理论上限是多少 GFLOP/s？如果目标是把上限推到机器平衡点以上（即变成计算受限），TILE_WIDTH 至少要多大？

分块后全局内存访问量降低 `TILE_WIDTH` 倍，因此算术强度 `AI = TILE_WIDTH/4` FLOP/Byte。TILE_WIDTH = 16 → `AI = 4`，上限 `= 1555 GB/s × 4 = 6220 GFLOP/s ≈ 6.22 TFLOP/s`，占峰值 19.5 TFLOPS 的 **31.9%**（比朴素版的 2% 提升约 16 倍）。要达到机器平衡点 `12.5 FLOP/Byte`，需要 `TILE_WIDTH/4 ≥ 12.5`，即 **TILE_WIDTH ≥ 50**；而 32×32 分块的 `AI = 8` 只给到 12.44 TFLOP/s（63.8%），**仍是带宽受限**。所以现实的方案是"16×16 分块 + 4×4 寄存器粗化"（`AI = T × r × c /(2(r+c)) = 16 × 16/16 = 16 FLOP/Byte > 12.5`），既绕开了超大 tile 对共享内存容量的要求（16×16 只需 2 KB），又跨过了平衡点。

Q3. 有同学为了"让 x 方向访问连续"，把索引写成 `Row = blockIdx.x*blockDim.x + threadIdx.x; Col = blockIdx.y*blockDim.y + threadIdx.y;`（即 x→行、y→列），其余代码不变。以 16×16 block、行主序 A、B 为例，说明 A、B 两侧的 warp 访问分布发生了什么变化，性能会变好还是变坏，为什么？

交换语义后，warp 内（`ty ∈ {t, t+1}`，`tx = 0..15`）A 的地址 `d_M[Row*Width + k]` 开始随 `tx` 变化：得到 **16 个互不相同的地址，每个相距 `Width*4` Byte** → 16 个独立 sector、取回 `16 × 32 = 512 B`，而有用只有 64 B（两个 ty 组地址相同，被广播）；B 的地址 `d_N[k*Width + Col]` 开始随 `ty` 变化：只有 **2 个相邻地址（相距 4 B）** → 1 个 sector、取回 32 B、有用 8 B。合计 **544 B 取回 / 72 B 有用，效率 13.2%**，比标准映射的 128 B 取回**差 4.25 倍**，性能显著变坏。原因是：跨步访问的受害者从一个操作数转移到了另一个操作数，而且从一个"2 次 transaction"的温和形态恶化成"16 次 transaction"的极端形态——**把变化最快的线程索引放在内存中跨步的那一维，等于让整个 warp 的地址散开**。这条不等式（x→连续维）是 CUDA 访存优化的第一原则。

Lecture 5: 并行模式三 —— 分块矩阵乘法：共享内存与数据复用 (对应 Lab 3: Tiled Matrix Multiply)

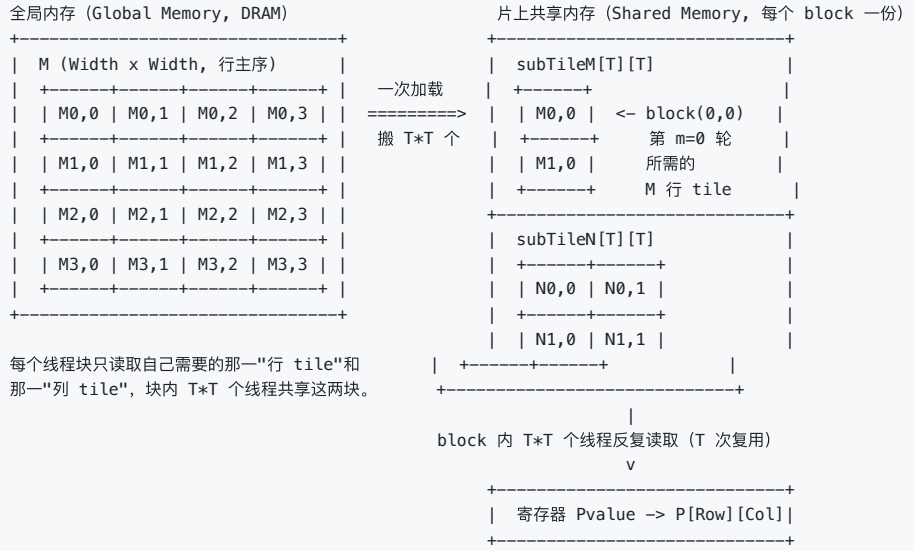
概述

本讲要解决整个课程最核心的性能问题：矩阵乘法朴素内核每做 2 次浮点运算就要访问 8 字节全局内存（4 Byte/FLOP），而 GPU 的全局内存带宽远跟不上算力，导致内核被死死卡在内存带宽上（讲义中 1,000 GFLOP/s 算力配 150 GB/s 带宽的一代 GPU 只剩下 37.5 GFLOP/s 的实际上限）。 解法是"分块（tiling）"：把 M、N 切成 `TILE_WIDTH × TILE_WIDTH` 的小块，由同一个线程块（block）协作搬进**共享内存（shared memory）**这个片上的"暂存区"，再让块内所有线程反复使用，从而把每个输出元素的全局访存次数从 2·Width 次降到 2·Width/TILE_WIDTH 次。 为此本讲引入三个必须精通的 CUDA 机制：`__shared__` 声明的共享内存、保证块内协作正确性的屏障 `__syncthreads()`、以及决定共享内存实际带宽的 **bank 与 bank conflict（存储体冲突）**；最后用算术强度与 Roofline 模型定量说明"分块把工作点沿着屋顶线向右推"，以及为什么更大的 tile 与线程粗化（thread coarsening）才能让内核真正变成计算受限。

核心概念与 GPU 架构图解

概念 1：分块 / 分片（Tiling, Blocking）

- **定义与目的**：分块是把大矩阵在逻辑上切成尺寸为 `TILE_WIDTH × TILE_WIDTH`（记作 T×T）的小方块，让一个线程块只负责计算 P 中对应的一个 T×T 输出块；该块在计算过程中只需反复读取 M 的一行 tile 与 N 的一列 tile，而不是完整的一行一列。目的是**用片上存储换取全局内存访问量的成倍下降**，把一个带宽受限的内核变成有机会达到计算峰值的核。
- **直观解释（"它是什么？"）**：想象厨房里做一道需要反复取用两种食材的菜。冰箱（全局内存）很大但离灶台很远，走一趟要 400 到 800 个时钟周期；灶台边的案板（共享内存）很小，但伸手就到（20 到 30 个周期）。朴素做法是每加一次料就跑一趟冰箱：做 4096 次乘加就要跑 8192 趟。分块做法是：先把这一小块所需的两种食材（各 T×T 份）一次性搬到案板上，然后 **T×T 个厨师（线程）围着同一块案板**，每人从案板上取 T 份 M 食材、T 份 N 食材，做完 T 份成品；案板上的食材用完（这一轮 m 结束）再集体去冰箱搬下一批。食材从冰箱只搬了一次，却被用了 T 次。
- **架构/机制图解**：



关键性能特征：一轮 m 加载 $2 \cdot T^2$ 个元素（$8 \cdot T^2$ 字节），却支撑了 T^3 次乘加（$2 \cdot T^3$ FLOP），因此**每字节全局流量支撑的浮点运算量**从朴素版的 0.25 FLOP/Byte 提升到 T/4 FLOP/Byte（T=16 时 4，T=32 时 8）。全局内存延迟没有变（仍是 400 到 800 个周期），但**需要发起的全局访存次数**少了 T 倍；同时块内 T^2 个线程的加载请求并行发出，内存级并行度（memory-level parallelism, MLP）很高。

- **为什么分块能减少全局内存访问：定量推导**（这是 Lab 3 报告与考试必考的部分）。设矩阵边长为 K（讲义中写作 Width，注意不要与矩阵 N 混淆），朴素内核中每个输出元素 P[Row][Col] 需要做 K 次乘加，第 k 次乘加要读 M[Row][k] 与 N[k][Col] 各 1 个元素（各 4 字节）：

项目	朴素内核	分块内核（tile 边长 T = TILE_WIDTH）
每个输出元素的全局访存次数	K 次乘加 × 2 个元素 = **2K 个元素**（8K 字节）	$2 \cdot T \cdot K / T^2$ = **2K/T 个元素**（8K/T 字节）
每个输出元素的浮点运算	2K FLOP	2K FLOP
全局访存总量（元素数）	K^2 个输出 × 2K = **$2K^3$ 个元素**	每 block $2 \cdot T \cdot K$ 个元素 × $(K/T)^2$ 个 block = **$2K^3/T$ 个元素**
全局访存总量（字节）	$8K^3$ 字节	$8K^3/T$ 字节
算术强度（AI）	$2K^3$ FLOP / $8K^3$ Byte = **0.25 FLOP/Byte**	$2K^3 / (8K^3/T)$ = **T/4 FLOP/Byte**
降低的倍数	1×（基准）	**T 倍**

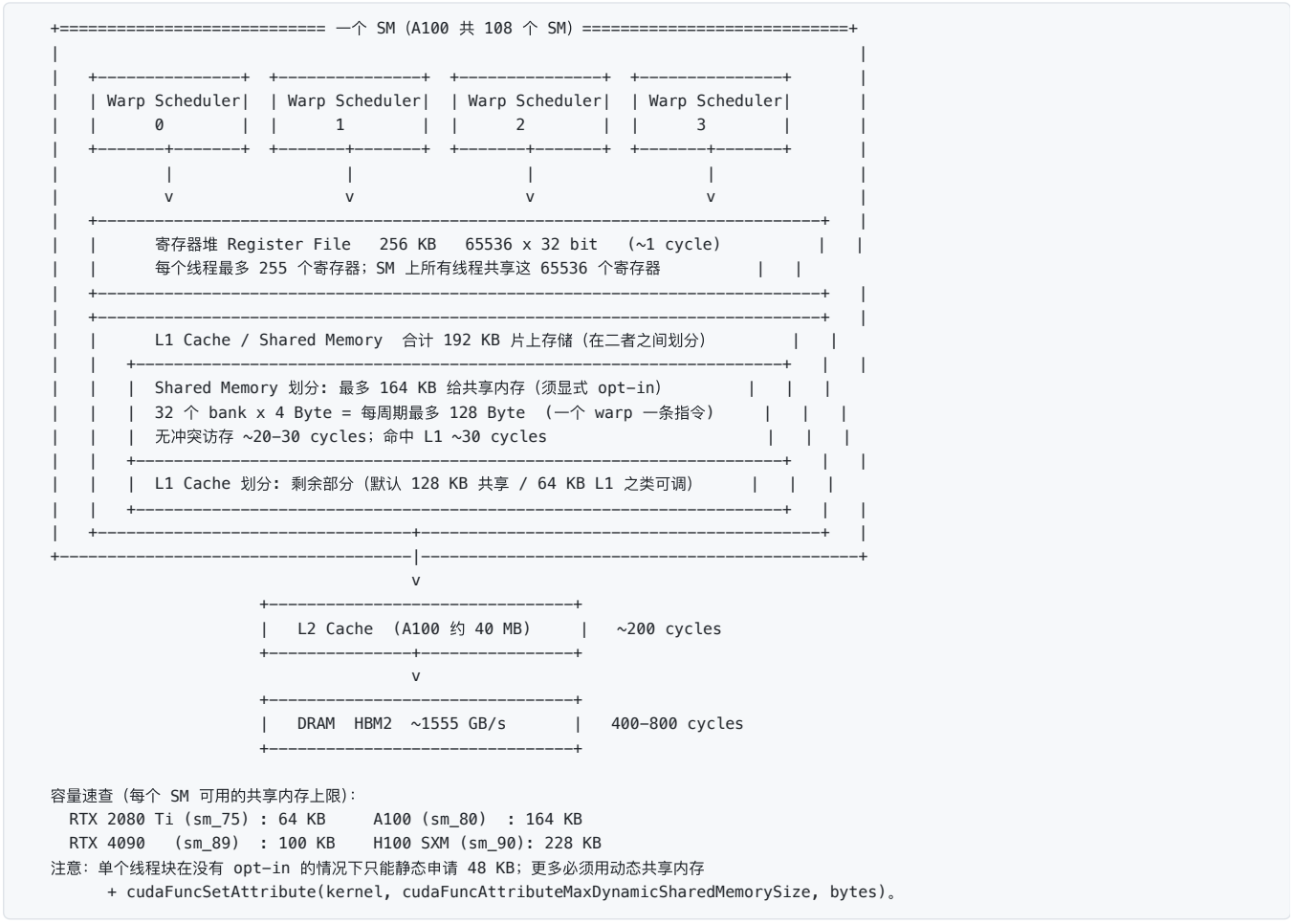
推导中的关键一步是算"每个 block 需要多少全局访存"：一个 block 要算完自己的 T×T 个输出，必须遍历 M 的第 by 个行条带（共 K/T 个 tile）与 N 的第 bx 个列条带（共 K/T 个 tile），每一轮 m 各读一个 T×T 的 tile，因此每个 block 的全局加载量 = $2 \times (K/T) \times T^2 = 2 \cdot T \cdot K$ 个元素；除以它负责的 T^2 个输出元素，就得到 **2K/T 个元素/输出元素**，与要求一致。 再换个角度看复用的来源：一个 T×T 的 M tile 从全局内存**只加载 1 次**（T^2 个元素），但在内层循环里被 block 内 **T^2 个线程各读 T 次**（每个线程读自己那一行 tile 的 T 个元素），于是这块 tile 总共被读取 $T^2 \times T = T^3$ 次，**平均每个加载进来的元素被复用了 T 次**；N tile 同理。M、N 两个 tile 合起来：加载 $2T^2$ 个元素，产生 $2T^3$ 次共享内存读取，正好对应 T^3 次乘加（$2T^3$ FLOP）。这就是"用 T 倍复用换 T 倍带宽节省"的全部数学。

以 K = 4096、A100（1555 GB/s、19.5 TFLOPS FP32）为例： – 朴素：全局流量 $8K^3 = 5.50 \times 10^{11}$ 字节 = 550 GB → 550 GB / 1555 GB/s = **354 ms**；而计算只需 $2K^3 = 1.37 \times 10^{11}$ FLOP / 19.5 TFLOPS = 7.0 ms。内存比计算慢 50 倍，实测只能跑到约 350 GFLOP/s（约峰值的 1.8%），与讲义"150 GB/s 只能支撑 37.5 GFLOP/s、实测约 25 GFLOP/s"是同一个现象。 – 分块 T=16：流量 34.4 GB → 22.1 ms，仍然内存受限，上界 6.2 TFLOPS（约峰值

32%）。 – 分块 T=32：流量 17.2 GB → 11.0 ms，仍略慢于 7.0 ms 的计算时间，上界 12.4 TFLOPS（约峰值 64%）。 – 分块 T=64（需要线程粗化才能用 256 个线程覆盖 64×64 的输出块）：流量 8.6 GB → 5.5 ms < 7.0 ms，**终于变成计算受限**。

概念 2：共享内存（Shared Memory）

- **定义与目的**：共享内存是位于每个 SM 片上的、由 `__shared__` 关键字声明的可读写暂存区，作用域是**一个线程块**，生命周期与线程块相同（块启动时分配、块结束时回收，不跨块保留）。它存在的目的就是给分块算法提供"每字节成本远低于全局内存"的可复用存储：全局内存延迟 400 到 800 个周期，而共享内存命中只需约 20 到 30 个周期（Ampere 讲义给出的理想数字是约 5 个周期，模型化时通常用 20 到 30 个周期以包含流水线与冲突开销），且不消耗 DRAM 带宽。
- **直观解释（"它是什么？"）**：共享内存就是厨房里的**案板**：它比冰箱（全局内存）小得多，但伸手可得。案板是**同一个厨房（线程块）里所有厨师共用的**——隔壁厨房（另一个 block）的厨师看不见也够不着你家的案板；而且案板在换菜（block 结束）时就清空了，不能留给下一道菜。寄存器则是"厨师自己手里的碗"：最快（约 1 个周期）但完全私有，别人不能看；全局内存是冰箱：容量巨大（几十 GB）但每次取用都要走很远。
- **架构/机制图解**（A100 / Ampere SM 的内存层次）：



容量速查（每个 SM 可用的共享内存上限）：
  RTX 2080 Ti (sm_75) : 64 KB A100 (sm_80) : 164 KB
  RTX 4090 (sm_89) : 100 KB H100 SXM (sm_90): 228 KB
注意：单个线程块在没有 opt-in 的情况下只能静态申请 48 KB；更多必须用动态共享内存
  + cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize, bytes)。

声明方式与静态/动态两种形态（这一段代码是后续三个示例的基础）：

```
// (a) 静态共享内存：编译期确定大小，写在内核里，随内核一起编译进函数属性
__global__ void StaticKernel(const float* M, int Width)
{
    __shared__ float subTileM[32][32];        // 32*32*4 = 4 KB，每个 block 一份
    __shared__ float subTileN[32][32];
    subTileM[threadIdx.y][threadIdx.x] = M[0];
    __syncthreads();
}

// (b) 动态共享内存：大小在启动时以第三个 <<<>>> 参数给出，多个数组手工切分同一块
__global__ void DynamicKernel(const float* M, int Width)
{
    extern __shared__ float dynShared[];      // 只声明，不指定大小
    float (*subTileM)[33] = reinterpret_cast<float (*)[33]>(dynShared);
    float (*subTileN)[33] = reinterpret_cast<float (*)[33]>(dynShared + 32 * 33);
    subTileM[threadIdx.y][threadIdx.x] = M[0];
    __syncthreads();
}

// 启动动态版本：第三个 <<<>>> 参数是"每个 block 的动态共享内存字节数"
void launchDynamicVersion(dim3 grid, dim3 block, const float* dM, float* dP, int Width)
{
    int shmemBytes = 2 * 32 * 33 * (int)sizeof(float);        // 2 个 32x33 的 float 数组 = 8448 B
    // 若超过 48 KB（单个 block 静态共享内存上限），必须先 opt-in：
    // 例如 TILE=96 时 2*96*97*4 = 74496 B > 48 KB，就必须调用下面这一行。
    cudaFuncSetAttribute(DynamicKernel, cudaFuncAttributeMaxDynamicSharedMemorySize, shmemBytes);
    DynamicKernel<<<grid, block, shmemBytes>>>(dM, Width);
    (void)dP;
}
```

性能特征小结：共享内存带宽是"每 SM 每周期 32 个 bank × 4 字节 = 128 字节"，对 A100 全芯片即 108 SM × 128 B × 1.41 GHz ≈ **19.5 TB/s**，是 DRAM 带宽（1.555 TB/s）的 12.5 倍，这正是分块能奏效的硬件基础。但它**不是免费的无限带宽**：见下一个概念，bank conflict 会把这条 128 B/cycle 的通道按冲突路数成倍地拖慢。

概念 3：共享内存的 bank 与 bank conflict（含 padding 消除法）

- **定义与目的**：共享内存被硬件切成 **32 个 bank**，每个 bank 宽 4 字节，可在一个周期内各自独立地服务一个 4 字节访问。warp 中 32 个线程的地址按照 `bank = (字节地址 / 4) % 32` 映射到 bank。若同一条访存指令中，同一个 bank 被**多个线程访问了不同的地址**，硬件只能串行地分多次完成（k 路冲突 = k 个周期）；若多个线程访问的是**同一个地址**，则硬件做**广播（broadcast）**，一次取回、发给所有请求者，**不算冲突**。目的是让程序员知道"什么时候共享内存会变慢"，并用 padding（在行尾补几个字节，改变行步长）等技巧恢复满带宽。
- **直观解释（"它是什么？"）**：把共享内存想成超市的 **32 条收银台**。一条指令就是一批 32 位顾客同时结账：理想情况每人排到不同的收银台（无冲突），一秒钟处理完；如果 3 个人挤在同一条队伍，就要排队 3 轮（3-way conflict，3 个周期）；如果有人只是问"同一个人同一个问题"（同一地址），那就相当于广播通知，所有人一次都听到了，不额外排队。**关键是"同一条指令"**：两条不同源码行里的访问不属于同一条指令，不会互相冲突；同一个线程前后两次访问也不冲突。
- **架构/机制图解**：

```
  共享内存地址 -> bank 映射（32 banks x 4 Bytes，bank = (wordAddr) % 32）

    wordAddr  0 到 15 :   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
    bank              :   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
    wordAddr 16 到 31 :  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
    bank              :  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
    wordAddr 32 到 47 :  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47
    bank              :   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
    结论：bank 编号每 32 个 4 字节字循环一次，第 1 行（字地址 32 到 63）与第 0 行
          （字地址 0 到 31）落在完全相同的 bank 上，这正是冲突的根源。

  三种典型情形（warp = 32 线程，一条访存指令）：
  (a) 无冲突（conflict-free）                  -> 1 个周期
       线程 t 访问 wordAddr = t        : bank 依次为 0,1,2,3,4,5,6,7,8,9,10,11,12,13,
                                         14,15,16,17,18,19,20,21,22,23,24,25,26,27,
                                         28,29,30,31，32 个 bank 两两不同
  (b) 广播（broadcast，不算冲突）             -> 1 个周期
       线程 t 访问 wordAddr = 5        : 32 个线程同一个地址 -> 一次取回广播
  (c) k 路冲突（k-way conflict）               -> k 个周期
       线程 t 访问 wordAddr = 32*t      : 全部落在 bank 0 -> 32-way，32 个周期
       线程 t 访问 wordAddr = 16*t      : 落在 bank {0,16} -> 8-way，8 个周期

  周期数速查：无冲突 = 1 周期；2-way = 2 周期；32-way = 32 周期（128 B/cycle 的通道被打成 4 B/cycle）
```

（1）讲义原版分块内核内层循环的 bank 访问模式（必须能逐 warp 推导出来）：内核是 `Pvalue += subTileM[ty][k] * subTileN[k][tx];`，配 block(32,32)、TILE_WIDTH=32，因此一个 warp = ty 固定、tx = 0..31 的**一整行 32 个线程**（blockDim.x = 32 正好等于 warp 大小）： – `subTileM[ty]`

`[k]`：k 对所有线程相同、ty 相同、tx 完全不出现 → **32 个线程访问同一个地址** → 广播，1 个周期，**不冲突**。 – `subTileN[k][tx]`：字地址 = k×32 + tx，bank = (k×32 + tx) % 32 = **tx % 32**，即 tx=0..31 恰好取遍 bank 0..31 → **无冲突，1 个周期**。 结论：**讲义第 34 页这个内核的内层循环本来就是无 bank conflict 的**（写回 tile 的那条赋值语句 `subTileM[ty][tx] = M[Row*Width + m*TILE_WIDTH+tx];` 同样是 tx 连续、落在 bank 0 到 bank 31 各一次）。这是一条容易被误解的结论：bank conflict 与"是否使用共享内存"无关，只与"同一 warp 同一条指令内地址的 bank 分布"有关。TILE_WIDTH=16 时结论也一样：warp 跨两行（ty=0 的 16 个线程 + ty=1 的 16 个线程），`subTileN[k][tx]` 的 32 个线程只访问 16 个不同地址（两半 warp 地址相同 → 广播），落在 bank 0 到 bank 15，仍然 1 个周期。

（2）什么时候真的会冲突：跨列 / 转置访问。真正危险的是"**warp 内变化的下标出现在共享数组的行下标上**"，即访问一个 TxT tile 的**一列**：字地址 = ixS + j（S = 行步长 = TILE_WIDTH），bank = (Sxi + j) % 32。课程 2012 年模拟考题（Practice Exam 2 第 3 题）给出的正是这个形态的内核：

```
#define BLOCK_SIZE 16        // 课程考题中 BLOCK_SIZE 是编译期常量，可取值 1 到 20

__global__ void BlockTranspose(float* A_elements, int A_width, int A_height)
{
    __shared__ float blockA[BLOCK_SIZE][BLOCK_SIZE];
    int baseIdx = blockIdx.x * BLOCK_SIZE + threadIdx.x;
    baseIdx += (blockIdx.y * BLOCK_SIZE + threadIdx.y) * A_width;
    blockA[threadIdx.y][threadIdx.x] = A_elements[baseIdx];
    A_elements[baseIdx] = blockA[threadIdx.x][threadIdx.y];   // 跨列读取 -> bank conflict
}
```

手工推演（tile 边长 T、行步长 S = T、线程 (tx,ty) 访问 `tile[tx][ty]`，字地址 = tx·S + ty）：

情形	行步长 S	warp 内 32 个地址的 bank 分布	单个 bank 上的最大不同地址数	访存周期数
TILE=16，无 padding	16	ty=0 的 16 个线程：bank 依次为 0,16,0,16,0,16,0,16,0,16,0,16,0,16,0,16（8 个落在 bank 0、8 个落在 bank 16）；ty=1 的 16 个线程：同理 bank 1 与 bank 17 各 8 个	8	**8 周期（8-way）**
TILE=16，padding +1	17	ty=0：bank {0,17,2,19,4,21,6,23,8,25,10,27,12,29,14,31}；ty=1：bank {1,18,3,20,5,22,7,24,9,26,11,28,13,30,15,0}	2（只有 bank 0 被两组各命中一次）	2 周期（2-way）
TILE=16，padding +2	18	ty=0 的 16 个线程取遍**全部偶数 bank** {0,2,4,6,8,10,12,14,16,18,20,22,24,26,28,30}；ty=1 取遍**全部奇数 bank** {1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31}	1	**1 周期（无冲突）**
TILE=32，无 padding	32	字地址 = 32·tx + ty，bank = (32·tx + ty) % 32 = **ty（常数）**：32 个线程全部落在同一个 bank（例如 ty=0 时 bank 始终为 0，地址为 0,32,64,96,128,160,192,224,256,288,320,352,384,416,448,480,512,544,576,608,640,672,704,736,768,800,832,864,896,928,960,992 共 32 个不同地址）	32	**32 周期（32-way）**
TILE=32，padding +1	33	bank = (33·tx + ty) % 32 = **(tx + ty) % 32**，tx=0..31 → 取遍全部 32 个 bank；ty=3 时 bank 依次为 3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,0,1,2，仍然两两不同	1	**1 周期（无冲突）**

逐项展开 TILE=32 的 bank 编号推导表（ty = 0，padding 前 vs 后）：

```
无 padding，S=32，ty=0（warp 内 32 个线程，tx = 0..31）：
  tx       :   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
  字地址   :   0  32  64  96 128 160 192 224 256 288 320 352 384 416 448 480
  bank     :   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
  tx       :  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
  字地址   : 512 544 576 608 640 672 704 736 768 800 832 864 896 928 960 992
  bank     :   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
  => 32 个不同地址全部落在 bank 0：一次 32 路串行 = 32 个周期

有 padding，S=33，ty=0：
  tx       :   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
  字地址   :   0  33  66  99 132 165 198 231 264 297 330 363 396 429 462 495
  bank     :   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
  tx       :  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
  字地址   : 528 561 594 627 660 693 726 759 792 825 858 891 924 957 990 1023
  bank     :  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
  => 32 个不同地址落在 32 个不同 bank：1 个周期，无冲突

有 padding，S=33，ty=5（整张 bank 表整体右移 5 格，仍然是 32 个不同 bank）：
  tx       :   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
  字地址   :   5  38  71 104 137 170 203 236 269 302 335 368 401 434 467 500
  bank     :   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20
  tx       :  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
  字地址   : 533 566 599 632 665 698 731 764 797 830 863 896 929 962 995 1028
  bank     :  21  22  23  24  25  26  27  28  29  30  31   0   1   2   3   4
```

（3）padding 消除冲突的原理：把行步长从 S 改成 S+1，等价于把"跨列访问的地址序列"从等差数列 {S·i} 变成 {(S+1)·i}。**当 S+1 与 32 互质时**，i = 0..31 产生的 (S+1)·i mod 32 恰好取遍 0..31（因为 (S+1) 是模 32 的乘法生成元），于是 32 个线程访问 32 个不同的 bank，冲突彻底消失。TILE=32 时 S+1 = 33，gcd(33,32)=1，**一次 +1 就完美解决**；TILE=16 时 warp 横跨两行、每行只有 16 个线程只覆盖 16 个 bank，+1（S=17）仍会让两半 warp 在一个 bank 上撞车（2-way），要 +2（S=18，取遍全部偶数 bank / 奇数 bank）才完全无冲突。padding 的代价只是每行多 4 到 8 字节（TILE=32 时每块共享内存从 8 KB 变成 8448 B，多出 448 B），可以忽略。

概念 4：`__syncthreads()` 屏障与批量同步执行（Barrier / Bulk Synchronous Execution）

- **定义与目的**：`void __syncthreads(void);` 是块级屏障：块内**所有**线程都必须到达该调用点，且在最后一条线程到达之前，先到的线程一直阻塞（睡眠）；只有当所有线程都到达后，大家才一起继续。它解决的问题是：分块的"协作加载 → 共同计算 → 换下一块"这种流水必须让所有线程步调一致，否则线程 A 可能在读到线程 B 还没写进去的 tile 元素（RAW 竞争），或者把线程 C 还在读的 tile 元素覆盖掉（WAR 竞争）。
- **直观解释（"它是什么？"）**：像**接力赛的交接区**或**集体合影**：拍合影时摄影师（屏障）要求所有人都站好（都到达 `__syncthreads()`）才按快门，谁没到大家就都等着；快门之后所有人同时继续。它也是"批量同步（bulk synchronous）"编程模型的实现：一群线程先各自干活，再一起等齐（屏障），再一起进入下一段。这种模式把程序切成若干个"只在段内并行"的小步骤，让调试变成调试许多个小程序，而不是一个巨大的并行体——这正是高性能计算领域的主流写法。
- **架构/机制图解**：

```
时间轴（一个 block 内 T*T 个线程、TILE_WIDTH 轮 m 循环）

thread 0 :  [ 加载 tile ]####[ 计算 T 步 ]####[ 加载 tile ]####[ 计算 T 步 ]####[ 写 P ]
thread 1 :  [ 加载 tile ]####[ 计算 T 步 ]####[ 加载 tile ]####[ 计算 T 步 ]####[ 写 P ]
thread 2 :  [ 加载 tile ]####[ 计算 T 步 ]####[ 加载 tile ]####[ 计算 T 步 ]####[ 写 P ]
thread 3 :  [ 加载 tile ]####[ 计算 T 步 ]####[ 加载 tile ]####[ 计算 T 步 ]####[ 写 P ]
 （thread 4 到 thread N-1 的行为与上面完全相同；慢的线程会让快的线程在屏障处等待）
thread N :  [ 加载 tile ]####[ 计算 T 步 ]####[ 加载 tile ]####[ 计算 T 步 ]####[ 写 P ]
                          ^                     ^                      ^
                          |                     |                      |
                  屏障 1（本轮加载 -> 本轮计算）   屏障 2（本轮计算 -> 下一轮加载）

屏障 1 防止 RAW（read-after-write，真依赖）：
   线程 A 读 subTileM[k][ty] 之前，线程 B 必须已经完成 subTileM[*][*] 的写入。
屏障 2 防止 WAR（write-after-read，反依赖）：
   线程 A 覆盖 subTileM[*][*] 之前，线程 B 必须已经读完上一轮的旧值。
两次同步缺一不可；没有屏障 1 -> 读到旧值/未初始化值；没有屏障 2 -> 读到"半新半旧"的混合 tile。

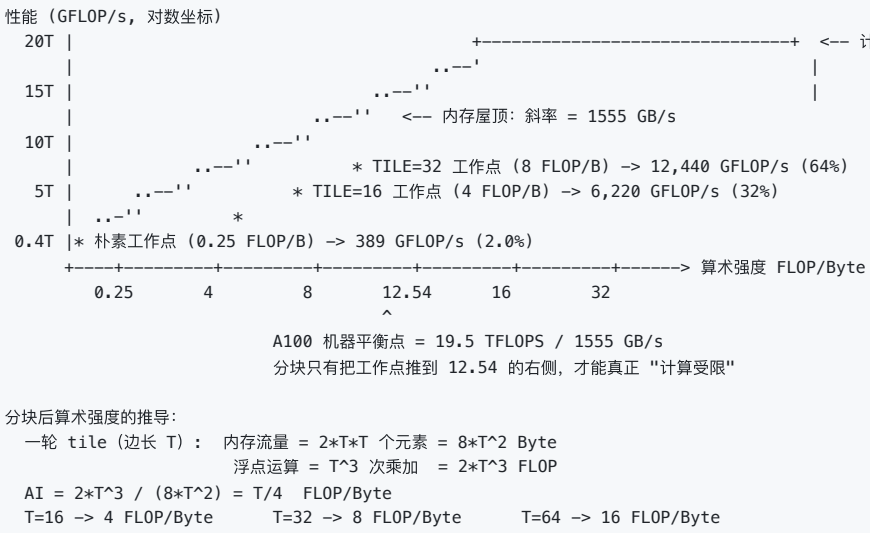
一个 block 的循环体（m = 0, 1, 2, 直到 Width/TILE_WIDTH - 1）：
    +----------------+     +----------------------+     +-----------------------+
    | 协作加载 M/N   | --> |  __syncthreads() #1 | --> | 内层 k 循环 T 次乘加 | --+
    | tile 到共享内存|     |  （等待全部写完）      |     | 读共享内存 -> 累加器  |
    +----------------+     +----------------------+     +-----------------------+
                                                                 |
                                          +----------------------+-----------v-----+
                                          |  __syncthreads() #2  （等待全部读完）      |
                                          +------------------+-----------------------+
                                                             |
                                  +--------------------------------+
                                  | 回到循环开头（下一轮 m）；最后一轮之后写 P[Row*Width+Col]
                                  +-----------------------------------------------------------
```

两条铁律（讲义的 N.B. 与常见考试陷阱）： 1. **块内所有线程都必须到达同一个 `__syncthreads()` 调用点。** 不能只有子集进入（例如把 `__syncthreads()` 写在 `if (Row < Width)` 里，边界块中部分线程不进入 → 结果是未定义行为，程序可能挂死或产生错误结果）。Volta 之后硬件引入了 per-thread 的收敛屏障，收敛的（convergent）分支里的屏障在部分新架构上"能用"，但 CUDA 编程指南仍明确规定必须所有线程到达同一个静态调用点，**不要依赖未定义行为**。 2. **不能放在发散（divergent）的分支里**：线程必须在**同一条静态调用**上等待（"同一个静态调用"不等于"调用同一个函数"）。 屏障的可见性保证：CUDA 手册规定屏障之前的所有**全局内存与共享内存访问（包括原子操作变体）都已完成**（对块内可见）；但**不要对 I/O 做假设**（例如 `printf` 的输出顺序不受屏障约束）。 讲义第 41 到 44 页的四个"Problem Solving"判定，可以整理成一张判定表，它是 Lab 3 与考试中最常考的模式：

```
代码                                                  需要 __syncthreads 吗?    理由
-------------------------------------------------------------------------------------------------
tile[ty][tx] = myNumber;  otherNumber = tile[ty][tx];      不需要          每个线程只读自己写的位置
tile[ty][tx] = myNumber;  otherNumber = tile[tx][ty];      需要            读的是别人写的元素（RAW）
tile[ty][tx] = myNumber;  __syncthreads();
    otherNumber = tile[tx][ty];
    thirdNumber = tile[TW-tx-1][ty];                       不需要          屏障后没有新写入，顺序已被保证
tile[ty][tx] = myNumber;  __syncthreads();
    otherNumber = tile[tx][ty];
    thirdNumber = tile[TW-tx-1][ty];
    tile[ty][tx] = myNumber * 2.0f;                        需要            覆盖写前必须等别人读完（WAR）
-------------------------------------------------------------------------------------------------
```

概念 5：算术强度与 Roofline 移动（Arithmetic Intensity & Roofline Model）

- **定义与目的**：算术强度 AI = 浮点运算次数 / 访问的字节数（FLOP/Byte），它刻画一个内核"每搬运 1 字节数据能算多少活"。Roofline 模型把硬件分成两条上界：内存带宽上界（斜率 = 带宽）与计算峰值上界（水平线），二者的交点就是**机器平衡点（machine balance）= 峰值算力 / 带宽**。目的：一眼判断内核是"内存带宽受限"还是"计算受限"，并算出需要多少数据复用才能达到峰值。
- **直观解释（"它是什么？"）**：像评价一辆车的油耗：**每升油能跑多少公里**就是这辆车的"算术强度"。发动机马力（算力）再大，如果油箱补给（带宽）跟不上，也只能慢慢开。Roofline 图就是这个"耗油率-速度"曲线：低速时受限于供油（斜坡段，内存受限），高速时受限于发动机（平顶段，计算受限），拐点就是这台 GPU 的"经济时速"。
- **架构/机制图解**（以 A100：19.5 TFLOPS FP32、1555 GB/s、机器平衡点 $19.5\times10^{12}/1.555\times10^{12}$ = **12.54 FLOP/Byte** 为基准）：



与讲义"Use of Large Tiles Shifts Bottleneck"（第 35 页）的经典数字完全一致：一代 GPU 算力 1,000 GFLOP/s、带宽 150 GB/s（平衡点 6.67 FLOP/Byte）。朴素版每 FLOP 要 4 字节 → 150/4 = **37.5 GFLOP/s**（实测约 25 GFLOP/s）；TILE=16 使每个操作数被复用 16 次 → (150/4)×16 = **600 GFLOP/s**；TILE=32 → (150/4)×32 = **1,200 GFLOP/s > 1,000 GFLOP/s，内存带宽不再是瓶颈，瓶颈转移到计算**。 不同 GPU 的平衡点差异很大，这决定了"分块够不够"：

GPU	FP32 峰值	带宽	机器平衡点	TILE=32（AI=8）能达到的比例	结论
RTX 2080 Ti (Turing)	13.4 TFLOPS	616 GB/s	21.8 FLOP/Byte	8/21.8 = 37%	分块远远不够
A100 (Ampere)	19.5 TFLOPS	1555 GB/s	**12.5 FLOP/Byte**	8/12.5 = **64%**	分块 + 粗化即可接近峰值
H100 SXM (Hopper)	67 TFLOPS	3350 GB/s	20.0 FLOP/Byte	8/20.0 = 40%	需要更大 tile / 张量核
RTX 4090 (Ada)	82.6 TFLOPS	1008 GB/s	**81.9 FLOP/Byte**	8/81.9 = 9.8%	必须靠寄存器分块 + 张量核

要真正越过平衡点，需要 AI ⩾ 12.54，即 T ⩾ 4 × 12.54 = 50.2；而"一个线程算一个输出元素"的写法要求 block 有 T^2 个线程，T=32 时已是 1024 个线程（一个 block 的线程上限），T 无法再大。这正是下一个概念要解决的问题。

概念 6：线程粗化与寄存器分块（Thread Coarsening & Register Tiling）

- **定义与目的**：线程粗化指让**每个线程计算多个输出元素**（例如 2×2 = 4 个），把这些输出元素的累加器放进寄存器形成"寄存器 tile"。它解决的问题有三层：(1) 把 tile 的几何尺寸与块的线程数**解耦**，于是可以用 256 个线程覆盖 64×64 的输出块，从而使用更大的 tile、获得更高的全局复用（AI 从 T/4 继续上升）；(2) 减少**共享内存访问次数**（每做一次乘加所需的共享内存读取从 2 次降到 1 次甚至 0.5 次）；(3) 摊薄地址计算、循环控制等指令开销，让 FMA 指令的密度更高。
- **直观解释（"它是什么？"）**：像一位厨师不再"炒一个菜就跑一趟案板"，而是**同时照看 4 口锅**：他从案板上取一次 M 食材（一行），可以配 4 种不同的 N 食材做出 4 份成品；每份食材的取用次数翻倍，跑案板的次数相对减半。案板还是那块案板，但"每次伸手"都更值钱。寄存器就是厨师手里的 4 个碗（累加器），别人看不到也不需要看到。
- **架构/机制图解**：

```
线程粗化前（1 输出/线程）                线程粗化后（2x2 = 4 输出/线程）
block(16,16) 覆盖 16x16 输出              block(16,16) 覆盖 32x32 输出

+--------------------+                   +--------------------------------+
| 线程(tx,ty) 算 1 个 |                    | 线程(tx,ty) 算 2x2 个寄存器 tile|
| P[ty][tx]          |                   |  P[2ty  ][2tx  ] P[2ty  ][2tx+1]|
+--------------------+                   |  P[2ty+1][2tx  ] P[2ty+1][2tx+1]|
                                         +--------------------------------+
每个 k 步：2 次共享内存读 + 1 次 FMA        每个 k 步：4 次共享内存读 + 4 次 FMA
-> 2 次共享读 / 乘加                       -> 1 次共享读 / 乘加
-> 4 Byte/FLOP（含广播后约 2 Byte/FLOP）    -> 2 Byte/FLOP（含广播后约 0.6 Byte/FLOP）

共享内存带宽瓶颈核算（A100 每 SM：128 FLOP/cycle，共享内存 128 Byte/cycle）
  未粗化：4 B/FLOP 需要 512 B/cycle，硬件只有 128 B/cycle -> 理论上只能达到峰值的 1/4
  2x2   ：2 B/FLOP 需要 256 B/cycle -> 仍超配 2 倍
  4x4   ：1 B/FLOP 需要 128 B/cycle -> 与共享内存带宽恰好平衡
  结论：共享内存带宽（而不是 DRAM 带宽）才是"已经做完分块"之后的下一个瓶颈，
        这正是真实 SGEMM 内核普遍采用 4x4 或 8x8 寄存器 tile 的原因。

课程 SGEMM 对比表（ECE409 联合讲座，2020 年 4 月 16 日）：
+---------+---------+---------+---------------------+---------------------+-----------+
| 方案    | A 复用  | B 复用  | 每 block 产物数据量 | 共享内存/block      | 寄存器/block|
+---------+---------+---------+---------------------+---------------------+-----------+
| Basic   | 1       | 1       | 1024                | 0                   | 4 KB      |
| Tiled   | 32      | 32      | 1024 (TILE_SIZE^2)  | 32*32*2*4B = 8 KB   | 4 KB      |
| Joint   | 16      | 64      | 1024                | 64*4B = 256 B       | 5 KB      |
+---------+---------+---------+---------------------+---------------------+-----------+
（Joint = 共享内存 + 寄存器联合分块：共享内存只放 B 的一个小 tile，A 的值从全局/寄存器直接广播，
  用 64 个 A 行 × 16 个 B 值 在寄存器里做 64*16 次乘加，因此共享内存占用骤降到 256 B，
  但每线程需要 64*16 个累加器以上的寄存器压力——课程注明 ECE408 不要求复现。）
```

粗化对**全局内存流量**本身不改变：全局流量依旧是 $8K^3/T$；它改变的是**共享内存流量、指令数与可达的占用率**，并通过"允许更大 T 而线程数不变"间接提高全局复用。用一句准确的话总结：**分块降低全局访存，粗化降低共享访存并解锁更大的分块。**

概念 7：占用率与共享内存的权衡（Occupancy vs Shared Memory）

- **定义与目的**：占用率 = SM 上实际驻留的 warp（或线程）数 / SM 支持的最大数。它决定了 SM 能同时掩盖多少延迟。共享内存是**按 block 分配的**资源，因此 tile 越大，每个 block 占用的共享内存越多，能同时驻留的 block 就越少；而线程数上限、寄存器上限同样在竞争。目的：在"更大的 tile 带来更高复用"与"更小的 tile 带来更高占用率"之间找平衡点，并知道究竟是哪一项资源在真正限制占用率。
- **直观解释（"它是什么？"）**：像一家餐厅同时能接待多少桌客人。桌子（block）占地越大（共享内存多），餐厅能摆下的桌子就越少；但每桌能坐的人（线程）也有上限，员工（寄存器）也是有限的。翻台率（占用率）高，才能在前一桌客人还在等菜（内存延迟）时让后一桌先点菜——这就是"用并行度掩盖延迟"。
- **架构/机制图解**（以 A100：每 SM 2048 线程 / 64 warp / 65536 寄存器 / 164 KB 共享内存 为基准）：

```
一个 SM 上的资源分配（每个 block 依次占用一份，直到某项资源耗尽）
+-------------------------------------------------------------------------------------+
|  SM (A100)                                                                          |
|  线程槽位 : 2048 threads  [ block 0: 1024 ][ block 1: 1024 ]（已满，不能再放第 3 块）   |
|  共享内存 : 164 KB        [ block 0: 8 KB  ][ block 1: 8 KB ]（还剩 148 KB 可用）      |
|  寄存器   : 65536 regs    [ block 0:32768 ][ block 1:32768 ]（已满，每线程 32 个寄存器）|
+-------------------------------------------------------------------------------------+
     ^ 本例（TILE=32, block=1024 线程，每线程 32 个寄存器）中真正卡住的是
       "线程槽位" 与 "寄存器" 两项；共享内存只用了 16 KB / 164 KB，远不是限制。

占用率计算（A100，每种配置的驻留 block 数取其四项限制的最小值）：
+----------------------+---------+-----------+-----------+------------+-----------+-----------+
| 配置                 | 共享内存 | 共享内存   | 线程槽位   | 寄存器限制 | 驻留       | 占用率     |
|                      | /block  | 限制块数   | 限制块数   | (R=32/40)  | block 数   | (warp)    |
+----------------------+---------+-----------+-----------+------------+-----------+-----------+
| T=16, 256 线程       | 2 KB    | 164/2 = 82| 2048/256=8| 8 / 6      | 8         | 100%      |
| T=32, 1024 线程      | 8 KB    | 164/8 = 20| 2048/1024 | 2 / 1      | 2         | 100%      |
|                      |         |           |     = 2   |            |           |           |
| T=32 粗化 2x2,       | 8.25 KB | 164/8.25  | 2048/256=8| 8 / 6      | 6..8      | 75%–100%  |
| 256 线程 (P=32)      |         |   = 19    |           |            |           |           |
| T=64 粗化 4x4,       | 33 KB   | 164/33 = 4| 2048/256=8| 8 / 4      | 4         | 50%       |
| 256 线程 (P=40)      |         |           |           |            |           |           |
+----------------------+---------+-----------+-----------+------------+-----------+-----------+
寄存器限制的算法：65536 / (256 线程 × R 寄存器) ；R=32 -> 8 块，R=40 -> 6 块，R=64 -> 4 块。
注意 1024 线程的 block 想要 2 块驻留，每线程只能用 65536/2048 = 32 个寄存器——这是
讲义 TILE=32 方案最容易被忽视的硬约束（编译时可用 __launch_bounds__(1024, 2) 强制）。

讲义第 36–37 页的老 GPU 数字（Maxwell 时代，2048 线程/SM，64 KB 共享内存）：
  TILE=16（256 线程）：2*256*4B = 2 KB/block -> 共享内存允许 32 个 block；
                       2048 线程上限 -> 8 个 block -> 8*512 = 4,096 个挂起加载
  TILE=32（1024 线程）：2*1024*4B = 8 KB/block -> 共享内存允许 8 个 block；
                       2048 线程上限 -> 2 个 block -> 2*2,048 = 4,096 个挂起加载
  结论：两种 tile 尺寸暴露的内存级并行度相同（都是每线程 2 个挂起加载），
        因此都可以用来掩盖全局内存延迟；tile 的选择由复用率与占用率的平衡决定。
在 RTX 4090（1536 线程/SM、48 warp、100 KB 共享内存）上：
  TILE=16（256 线程）：min(100/2 = 50, 1536/256 = 6) = 6 块 -> 100% 占用率
  TILE=32（1024 线程）：min(100/8 = 12, 1536/1024 = 1) = 1 块 -> 1024/1536 = 66.7% 占用率
  => 在 Ada 上，TILE=32 因为"线程槽位"限制拿不到 100% 占用率，粗化（256 线程 + 2x2 寄存器 tile）
     反而是恢复占用率的关键手段。
```

代码示例与性能分析

代码示例 1：讲义原版分块矩阵乘法（tiled_matmul_basic.cu）

完整内核来自讲义第 34 页，并补全主机端、计时、CPU 参考实现、GFLOP/s 与算术强度打印、Roofline 判定：

```
// 文件: tiled_matmul_basic.cu
// 编译: nvcc -O3 -arch=sm_80 tiled_matmul_basic.cu -o tiled_matmul_basic
// 运行: ./tiled_matmul_basic 2048      (矩阵边长，必须是 TILE_WIDTH 的整数倍)

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define TILE_WIDTH 16

// A100 硬件档案（用于 Roofline 分析）
#define A100_PEAK_TFLOPS 19.5
#define A100_BW_GBPS     1555.0

// ---------- 朴素版本（讲义第 4–5 页）：每个输出元素 2K 次全局访存 ----------
__global__ void MatrixMulKernelNaive(const float* __restrict__ M,
                                     const float* __restrict__ N,
                                     float* __restrict__ P,
                                     int Width)
{
    int Row = blockIdx.y * blockDim.y + threadIdx.y;
    int Col = blockIdx.x * blockDim.x + threadIdx.x;
    if ((Row < Width) && (Col < Width)) {
        float Pvalue = 0.0f;
        for (int k = 0; k < Width; ++k)
            Pvalue += M[Row * Width + k] * N[k * Width + Col];
        P[Row * Width + Col] = Pvalue;
    }
}

// ---------- 分块版本（讲义第 34 页）：全局访存降低 TILE_WIDTH 倍 ----------
__global__ void MatrixMulKernelTiled(const float* __restrict__ M,
                                     const float* __restrict__ N,
                                     float* __restrict__ P,
                                     int Width)
{
    __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
    __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x;   int by = blockIdx.y;
    int tx = threadIdx.x;  int ty = threadIdx.y;

    // 该线程负责的 P 元素的行列（blockDim.x 与 blockDim.y 都等于 TILE_WIDTH）
    int Row = by * TILE_WIDTH + ty;
    int Col = bx * TILE_WIDTH + tx;

    float Pvalue = 0.0f;

    // 遍历计算该输出元素所需的全部 M/N tile（假定 Width 是 TILE_WIDTH 的整数倍）
    for (int m = 0; m < Width / TILE_WIDTH; ++m) {
        // 协作加载：每个线程搬 1 个 M 元素和 1 个 N 元素
        subTileM[ty][tx] = M[Row * Width + m * TILE_WIDTH + tx];
        subTileN[ty][tx] = N[(m * TILE_WIDTH + ty) * Width + Col];
        __syncthreads();                        // 屏障 1：确保 tile 已全部写满

        for (int k = 0; k < TILE_WIDTH; ++k)
            Pvalue += subTileM[ty][k] * subTileN[k][tx];

        __syncthreads();                        // 屏障 2：确保旧 tile 已被读完
    }

    P[Row * Width + Col] = Pvalue;
}

// ------------------------------ 主机端辅助函数 ------------------------------
static void fillMatrix(float* A, int n, unsigned seed)
```

```
{
    for (int i = 0; i < n * n; ++i) {
        unsigned x = (unsigned)(i + seed * 7919u);
        x = x * 1103515245u + 12345u;
        A[i] = (float)((int)((x >> 16) & 0x7u) - 3) * 0.25f;   // 取值 -0.75 .. 1.00
    }
}

static void matmulCPU(const float* A, const float* B, float* C, int n)
{
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) {
            float s = 0.0f;
            for (int k = 0; k < n; ++k) s += A[i * n + k] * B[k * n + j];
            C[i * n + j] = s;
        }
}

template <typename LaunchT>
static bool verifyKernel(const char* name, LaunchT launch, int n, dim3 block)
{
    size_t bytes = (size_t)n * (size_t)n * sizeof(float);
    float* hM   = (float*)malloc(bytes);
    float* hN   = (float*)malloc(bytes);
    float* hP   = (float*)malloc(bytes);
    float* hRef = (float*)malloc(bytes);
    fillMatrix(hM, n, 1u);
    fillMatrix(hN, n, 2u);
    matmulCPU(hM, hN, hRef, n);

    float *dM, *dN, *dP;
    CUDA_CHECK(cudaMalloc((void**)&dM, bytes));
    CUDA_CHECK(cudaMalloc((void**)&dN, bytes));
    CUDA_CHECK(cudaMalloc((void**)&dP, bytes));
    CUDA_CHECK(cudaMemcpy(dM, hM, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(dN, hN, bytes, cudaMemcpyHostToDevice));

    dim3 grid((n + block.x - 1) / block.x, (n + block.y - 1) / block.y);
    launch(dM, dN, dP, n, grid, block);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(hP, dP, bytes, cudaMemcpyDeviceToHost));

    double maxErr = 0.0;
    for (int i = 0; i < n * n; ++i) {
        double d = fabs((double)hP[i] - (double)hRef[i]);
        if (d > maxErr) maxErr = d;
    }
    printf("[验证 %-6s] n=%d  最大绝对误差=%.3e  %s\n", name, n, maxErr,
           (maxErr < 1e-2) ? "PASS" : "FAIL");

    free(hM); free(hN); free(hP); free(hRef);
    CUDA_CHECK(cudaFree(dM)); CUDA_CHECK(cudaFree(dN)); CUDA_CHECK(cudaFree(dP));
    return maxErr < 1e-2;
}

template <typename LaunchT>
static double timeKernelMs(LaunchT launch, int iters)
{
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    for (int i = 0; i < 3; ++i) launch();                    // 预热
    CUDA_CHECK(cudaEventRecord(start));
    for (int i = 0; i < iters; ++i) launch();
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return (double)ms / (double)iters;
}

int main(int argc, char** argv)
{
    int Width = (argc > 1) ? atoi(argv[1]) : 2048;
    if (Width % TILE_WIDTH != 0) {
```

```
        fprintf(stderr, "Width 必须是 TILE_WIDTH=%d 的整数倍\n", TILE_WIDTH);
        return EXIT_FAILURE;
    }

    // 1) 正确性验证（用小矩阵跑 CPU 参考实现）
    verifyKernel("naive", [](const float* M, const float* N, float* P, int W, dim3 g, dim3 b) {
        MatrixMulKernelNaive<<<g, b>>>(M, N, P, W);
    }, 512, dim3(16, 16));
    verifyKernel("tiled", [](const float* M, const float* N, float* P, int W, dim3 g, dim3 b) {
        MatrixMulKernelTiled<<<g, b>>>(M, N, P, W);
    }, 512, dim3(TILE_WIDTH, TILE_WIDTH));

    // 2) 分配大矩阵并计时
    size_t bytes = (size_t)Width * (size_t)Width * sizeof(float);
    float *hM = (float*)malloc(bytes), *hN = (float*)malloc(bytes);
    fillMatrix(hM, Width, 3u);
    fillMatrix(hN, Width, 4u);

    float *dM, *dN, *dP;
    CUDA_CHECK(cudaMalloc((void**)&dM, bytes));
    CUDA_CHECK(cudaMalloc((void**)&dN, bytes));
    CUDA_CHECK(cudaMalloc((void**)&dP, bytes));
    CUDA_CHECK(cudaMemcpy(dM, hM, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(dN, hN, bytes, cudaMemcpyHostToDevice));

    double flops = 2.0 * (double)Width * (double)Width * (double)Width;

    dim3 blockN(16, 16);
    dim3 gridN((Width + 15) / 16, (Width + 15) / 16);
    double msNaive = timeKernelMs([&] {
        MatrixMulKernelNaive<<<gridN, blockN>>>(dM, dN, dP, Width);
    }, 5);

    dim3 blockT(TILE_WIDTH, TILE_WIDTH);
    dim3 gridT(Width / TILE_WIDTH, Width / TILE_WIDTH);
    double msTiled = timeKernelMs([&] {
        MatrixMulKernelTiled<<<gridT, blockT>>>(dM, dN, dP, Width);
    }, 20);

    CUDA_CHECK(cudaGetLastError());

    // 3) 性能与 Roofline 打印
    printf("\n===== 性能 (Width = %d, 2*W^3 = %.3e FLOP) =====\n", Width, flops);
    printf("朴素  : %8.3f ms   %8.1f GFLOP/s   算术强度 0.25 FLOP/Byte (每 FLOP 4 Byte)\n",
           msNaive, flops / msNaive / 1e6);
    printf("分块  : %8.3f ms   %8.1f GFLOP/s   算术强度 T/4 = %.1f FLOP/Byte\n",
           msTiled, flops / msTiled / 1e6, TILE_WIDTH / 4.0);
    printf("加速比: %.2fx\n", msNaive / msTiled);

    double bwBoundNaive = A100_BW_GBPS * 0.25;                 // GB/s * FLOP/Byte = GFLOP/s
    double bwBoundTiled = A100_BW_GBPS * (TILE_WIDTH / 4.0);
    double balance      = A100_PEAK_TFLOPS * 1000.0 / A100_BW_GBPS;
    printf("\n===== Roofline (A100: %.1f TFLOPS, %.0f GB/s, 机器平衡点 %.2f FLOP/Byte) =====\n",
           A100_PEAK_TFLOPS, A100_BW_GBPS, balance);
    printf("朴素内存屋顶      : %8.1f GFLOP/s  (峰值占比 %.2f%%)\n",
           bwBoundNaive, 100.0 * bwBoundNaive / (A100_PEAK_TFLOPS * 1000.0));
    printf("分块内存屋顶      : %8.1f GFLOP/s  (峰值占比 %.2f%%)\n",
           bwBoundTiled, 100.0 * bwBoundTiled / (A100_PEAK_TFLOPS * 1000.0));
    printf("判定              : %s\n",
           (bwBoundTiled < A100_PEAK_TFLOPS * 1000.0) ? "仍然内存带宽受限（需要更大 tile / 线程粗化）"
                                                      : "计算受限");
    printf("理论全局流量      : 朴素 %.2f GB,  分块 %.2f GB (降低 %d 倍)\n",
           2.0 * (double)Width * Width * Width * 4.0 / 1e9,
           2.0 * (double)Width * Width * Width * 4.0 / TILE_WIDTH / 1e9, TILE_WIDTH);

    free(hM); free(hN);
    CUDA_CHECK(cudaFree(dM)); CUDA_CHECK(cudaFree(dN)); CUDA_CHECK(cudaFree(dP));
    return EXIT_SUCCESS;
}
```

- **【代码做什么？】**
- 主机端先把两个矩阵填成"0.25 的整数倍"的伪随机数（保证 GPU 与 CPU 的浮点结果可比），在 n=512 上用三重循环 CPU 参考实现验证朴素内核与分块内核，输出最大绝对误差与 PASS/FAIL。

- `MatrixMulKernelTiled` 的网格是 `(Width/16, Width/16)`，块是 `(16,16)`；线程 `(tx,ty)` 通过 `Row = by*16 + ty`、`Col = bx*16 + tx` 映射到 P 中的一个元素，`Row` 同时是它要加载的 M 行、`Col` 是它要加载的 N 列。
- 外层 `for (m = 0; m < Width/16; ++m)` 共 256 轮（Width=4096 时）：每轮先把 `M[Row][m*16+tx]` 与 `N[m*16+ty][Col]` 分别写进 `subTileM`、`subTileN`（协作加载，块内 256 个线程各搬 2 个元素，正好覆盖两个 16×16 的 tile），然后 `__syncthreads()`，再在内层 `k = 0..15` 上做 16 次乘加，最后再 `__syncthreads()` 才进入下一轮。
- 主机端用 `cudaEvent` 计时（朴素版 5 次、分块版 20 次取平均，另有 3 次预热），打印 GFLOP/s、算术强度、A100 的 Roofline 内存屋顶与瓶颈判定，以及两种实现的全局流量对比。
- 最后释放全部 host/device 内存。
- **【并行机制与硬件映射解说】**
- **warp 划分**：block(16,16) = 256 线程 = 8 个 warp；warp 内线程按 x 最快维展开，因此**一个 warp = 同一个 ty 的两行 tx=0..15**（warp 0 覆盖 ty=0,1，warp 1 覆盖 ty=2,3，依此类推）。这一细节决定了后面所有访存分析的前提。整张网格 256×256 = 65,536 个 block。
- **warp 调度与驻留量（A100）**：每 SM 有 4 个 warp scheduler、最多 64 个 warp / 2048 线程。TILE=16 的 block 只有 256 线程、2 KB 共享内存，因此限制项是线程槽位：2048/256 = **8 个 block 驻留**（16 KB 共享内存、占用率 100%）。每个线程在加载阶段有 2 个独立的挂起全局加载，因此 SM 上最多有 8 × 512 = **4,096 个挂起加载**——这正是讲义第 36 页强调的"内存级并行度"，也是分块版能掩盖 400 到 800 周期延迟的资本。
- **全局访存是否合并（coalesced，按 32 线程 × 4 字节 = 128 字节粒度）**：`subTileM[ty][tx] = M[Row*Width + m*TILE_WIDTH + tx]`：一个 warp 内 ty 固定（warp 横跨 ty 的相邻两行，但对同一 ty 的 16 个线程而言行相同），地址 = `Row*Width + m*16 + tx`，tx = 0..15 连续 → 每半 warp 访问 64 字节连续 → **完全合并**（两个半 warp 各 64 B，共 128 B，1 到 2 条 128 B cache line）。`subTileN[ty][tx] = N[(m*TILE_WIDTH+ty)*Width + Col]`：地址 = `(m*16+ty)*Width + bx*16 + tx`，warp 内 tx 变化对应 Col 变化，仍是连续的 → **完全合并**。对照朴素内核：`M[Row*Width + k]` 在 warp 内 k 固定、Row 随 ty 变化而跨行，地址间隔为 `Width` 个元素 → 32 个线程落在 32 条不同的 cache line 上，**完全不合并**（这正是讲义第 7 讲"Accesses to M are NOT Coalesced"的结论；这些行会被后续 k 迭代重用，L1/L2 能吸收一部分，所以实测不会退到 1/32，但仍远高于分块版）。
- **共享内存 bank 访问模式**（逐 warp 推导）：`subTileM[ty][k]` 中 k 对整条 warp 相同、ty 也相同 → 32 个（或 16 个）线程访问**同一个地址** → 广播，1 周期，**无冲突**；`subTileN[k][tx]` 的字地址 = k×16 + tx，bank = (k×16+tx)%32，半 warp 内 tx=0..15 取遍 16 个不同 bank，两个半 warp 地址相同（广播）→ **无冲突，1 周期**。结论：**讲义原版内核内层循环没有 bank conflict**，padding 在这里不会带来任何 bank 收益（只让共享内存从 2 KB 涨到 2176 B）。
- **寄存器与发散**：本内核每线程只需 1 个累加器 + 少量索引，寄存器用量约 18 到 24 个，远低于 32 的上限，因此不限制占用率。发散方面：`if ((Row < Width) && (Col < Width))` 只在边界块上有分歧；本例假定 Width 是 TILE 的整数倍，因此**没有发散**（越界处理见示例 3）。
- **【性能优化分析】**
- **算术强度**：由概念 1 的推导，AI_朴素 = 2 FLOP / 8 Byte = **0.25 FLOP/Byte**，AI_分块(T=16) = 16/4 = **4 FLOP/Byte**，提升 16 倍。代入 A100：内存屋顶 = 1555 GB/s × 0.25 = **389 GFLOP/s**（峰值的 2.0%）与 1555 × 4 = **6,220 GFLOP/s**（峰值的 31.9%）。
- **Roofline 判定**：A100 的机器平衡点 = 19,500/1555 = **12.54 FLOP/Byte**。T=16 的工作点（4）位于交点左侧 → **仍然是内存带宽受限**，实测典型值约 5,800 到 6,300 GFLOP/s，与屋顶预测吻合。
- **瓶颈类型**：全局内存带宽受限（DRAM-bound），并伴随第二层瓶颈——共享内存带宽：未粗化时每做 1 次乘加要读 2 个共享内存字（8 字节），A100 每 SM 每周期提供 128 字节共享带宽，而满峰值 FP32 每周期要做 128 FLOP，需求为 512 字节/周期，**超配 4 倍**（计入广播后约 2 倍）→ 共享内存带宽把上限压到峰值的约 25% 到 50%。
- **可执行优化方向**：(1) 把 TILE_WIDTH 提到 32，AI 翻倍到 8（实测约 9,000 到 10,000 GFLOP/s，约 50% 峰值）；(2) 线程粗化（每线程 2×2 或 4×4 个寄存器累加器），把共享内存读取压到 1 次/乘加以下，并把 tile 加大到 64 而不增加线程数；(3) 边界处理改成"越界填 0"，让内核支持任意尺寸（示例 3）。

代码示例 2：用 padding 消除 bank conflict 的版本（tiled_matmul_padded.cu）

本示例使用 TILE_WIDTH = 32，并采用**转置载入**：线程把 tile 按转置位置写入共享内存（`subTileM[tx][ty]` 而非 `subTileM[ty][tx]`），内层循环相应地变为 `subTileM[k][ty] * subTileN[tx][k]`。这样做的现实动机有两个：一是当输入矩阵以**列主序**（column-major，例如来自 BLAS/Fortran 布局或 N 的转置）存储时，转置载入能让全局读取保持合并；二是它精确复现了"warp 内变化的下标出现在共享数组行下标上"这一**必然产生 bank conflict** 的形态（课程模拟考题中的 BlockTranspose 内核）。程序里同时给出无 padding、静态 padding、动态 padding 三个内核并对比性能：

```
// 文件: tiled_matmul_padded.cu
// 编译: nvcc -O3 -arch=sm_80 tiled_matmul_padded.cu -o tiled_matmul_padded
// 运行: ./tiled_matmul_padded 2048      (矩阵边长，必须是 TILE_WIDTH 的整数倍)

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define TILE_WIDTH 32
#define PADDED_STRIDE (TILE_WIDTH + 1)

// ---- 内核 A：转置载入 + 无 padding。行步长 S = 32 => 跨列访问 32-way bank conflict ----
__global__ void MatrixMulKernelTransposeNoPad(const float* __restrict__ M,
                                              const float* __restrict__ N,
                                              float* __restrict__ P,
                                              int Width)
{
    __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
    __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x;  int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;
    int Row = by * TILE_WIDTH + ty;
    int Col = bx * TILE_WIDTH + tx;

    float Pvalue = 0.0f;

    for (int m = 0; m < Width / TILE_WIDTH; ++m) {
        // 转置写入：共享内存第 (tx,ty) 个元素保存全局第 (ty,tx) 个元素
        subTileM[tx][ty] = M[Row * Width + m * TILE_WIDTH + tx];
        subTileN[tx][ty] = N[(m * TILE_WIDTH + ty) * Width + Col];
        __syncthreads();

        for (int k = 0; k < TILE_WIDTH; ++k)
            Pvalue += subTileM[k][ty] * subTileN[tx][k];

        __syncthreads();
    }
    P[Row * Width + Col] = Pvalue;
}

// ---- 内核 B：转置载入 + 静态 padding。行步长 S = 33，gcd(33,32)=1 => 无冲突 ----
__global__ void MatrixMulKernelTransposePadStatic(const float* __restrict__ M,
                                                  const float* __restrict__ N,
                                                  float* __restrict__ P,
                                                  int Width)
{
    __shared__ float subTileM[TILE_WIDTH][PADDED_STRIDE];
    __shared__ float subTileN[TILE_WIDTH][PADDED_STRIDE];

    int bx = blockIdx.x;  int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;
    int Row = by * TILE_WIDTH + ty;
    int Col = bx * TILE_WIDTH + tx;

    float Pvalue = 0.0f;

    for (int m = 0; m < Width / TILE_WIDTH; ++m) {
        subTileM[tx][ty] = M[Row * Width + m * TILE_WIDTH + tx];
        subTileN[tx][ty] = N[(m * TILE_WIDTH + ty) * Width + Col];
        __syncthreads();

        for (int k = 0; k < TILE_WIDTH; ++k)
            Pvalue += subTileM[k][ty] * subTileN[tx][k];

        __syncthreads();
```

```
    }
    P[Row * Width + Col] = Pvalue;
}

// ---- 内核 C：动态共享内存 + padding（同一块共享内存手工切成两个带 padding 的二维数组）----
__global__ void MatrixMulKernelTransposePadDynamic(const float* __restrict__ M,
                                                   const float* __restrict__ N,
                                                   float* __restrict__ P,
                                                   int Width)
{
    extern __shared__ float dynShared[];
    float (*subTileM)[PADDED_STRIDE] = reinterpret_cast<float (*)[PADDED_STRIDE]>(dynShared);
    float (*subTileN)[PADDED_STRIDE] =
        reinterpret_cast<float (*)[PADDED_STRIDE]>(dynShared + TILE_WIDTH * PADDED_STRIDE);

    int bx = blockIdx.x;  int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;
    int Row = by * TILE_WIDTH + ty;
    int Col = bx * TILE_WIDTH + tx;

    float Pvalue = 0.0f;

    for (int m = 0; m < Width / TILE_WIDTH; ++m) {
        subTileM[tx][ty] = M[Row * Width + m * TILE_WIDTH + tx];
        subTileN[tx][ty] = N[(m * TILE_WIDTH + ty) * Width + Col];
        __syncthreads();

        for (int k = 0; k < TILE_WIDTH; ++k)
            Pvalue += subTileM[k][ty] * subTileN[tx][k];

        __syncthreads();
    }
    P[Row * Width + Col] = Pvalue;
}

// ------------------------------ 主机端辅助函数 ------------------------------
static void fillMatrix(float* A, int n, unsigned seed)
{
    for (int i = 0; i < n * n; ++i) {
        unsigned x = (unsigned)(i + seed * 7919u);
        x = x * 1103515245u + 12345u;
        A[i] = (float)((int)((x >> 16) & 0x7u) - 3) * 0.25f;
    }
}

static void matmulCPU(const float* A, const float* B, float* C, int n)
{
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) {
            float s = 0.0f;
            for (int k = 0; k < n; ++k) s += A[i * n + k] * B[k * n + j];
            C[i * n + j] = s;
        }
}

template <typename LaunchT>
static bool verifyKernel(const char* name, LaunchT launch, int n, dim3 block)
{
    size_t bytes = (size_t)n * (size_t)n * sizeof(float);
    float* hM   = (float*)malloc(bytes);
    float* hN   = (float*)malloc(bytes);
    float* hP   = (float*)malloc(bytes);
    float* hRef = (float*)malloc(bytes);
    fillMatrix(hM, n, 1u);
    fillMatrix(hN, n, 2u);
    matmulCPU(hM, hN, hRef, n);

    float *dM, *dN, *dP;
    CUDA_CHECK(cudaMalloc((void**)&dM, bytes));
    CUDA_CHECK(cudaMalloc((void**)&dN, bytes));
    CUDA_CHECK(cudaMalloc((void**)&dP, bytes));
    CUDA_CHECK(cudaMemcpy(dM, hM, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(dN, hN, bytes, cudaMemcpyHostToDevice));

    dim3 grid((n + block.x - 1) / block.x, (n + block.y - 1) / block.y);
    launch(dM, dN, dP, n, grid, block);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
```

```
    CUDA_CHECK(cudaMemcpy(hP, dP, bytes, cudaMemcpyDeviceToHost));

    double maxErr = 0.0;
    for (int i = 0; i < n * n; ++i) {
        double d = fabs((double)hP[i] - (double)hRef[i]);
        if (d > maxErr) maxErr = d;
    }
    printf("[验证 %-16s] n=%d  最大绝对误差=%.3e  %s\n", name, n, maxErr,
           (maxErr < 1e-2) ? "PASS" : "FAIL");

    free(hM); free(hN); free(hP); free(hRef);
    CUDA_CHECK(cudaFree(dM)); CUDA_CHECK(cudaFree(dN)); CUDA_CHECK(cudaFree(dP));
    return maxErr < 1e-2;
}

template <typename LaunchT>
static double timeKernelMs(LaunchT launch, int iters)
{
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    for (int i = 0; i < 3; ++i) launch();
    CUDA_CHECK(cudaEventRecord(start));
    for (int i = 0; i < iters; ++i) launch();
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return (double)ms / (double)iters;
}

int main(int argc, char** argv)
{
    int Width = (argc > 1) ? atoi(argv[1]) : 2048;
    if (Width % TILE_WIDTH != 0) {
        fprintf(stderr, "Width 必须是 TILE_WIDTH=%d 的整数倍\n", TILE_WIDTH);
        return EXIT_FAILURE;
    }

    dim3 block(TILE_WIDTH, TILE_WIDTH);
    const int dynBytes = 2 * TILE_WIDTH * PADDED_STRIDE * (int)sizeof(float);   // 8448 B

    // 1) 正确性验证：三个内核在 n=512 上与 CPU 参考实现对比
    verifyKernel("no-pad", [](const float* M, const float* N, float* P, int W, dim3 g, dim3 b) {
        MatrixMulKernelTransposeNoPad<<<g, b>>>(M, N, P, W);
    }, 512, block);
    verifyKernel("pad-static", [](const float* M, const float* N, float* P, int W, dim3 g, dim3 b) {
        MatrixMulKernelTransposePadStatic<<<g, b>>>(M, N, P, W);
    }, 512, block);
    verifyKernel("pad-dynamic", [dynBytes](const float* M, const float* N, float* P, int W, dim3 g, dim3 b) {
        MatrixMulKernelTransposePadDynamic<<<g, b, dynBytes>>>(M, N, P, W);
    }, 512, block);

    // 2) 计时
    size_t bytes = (size_t)Width * (size_t)Width * sizeof(float);
    float *hM = (float*)malloc(bytes), *hN = (float*)malloc(bytes);
    fillMatrix(hM, Width, 3u);
    fillMatrix(hN, Width, 4u);

    float *dM, *dN, *dP;
    CUDA_CHECK(cudaMalloc((void**)&dM, bytes));
    CUDA_CHECK(cudaMalloc((void**)&dN, bytes));
    CUDA_CHECK(cudaMalloc((void**)&dP, bytes));
    CUDA_CHECK(cudaMemcpy(dM, hM, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(dN, hN, bytes, cudaMemcpyHostToDevice));

    double flops = 2.0 * (double)Width * (double)Width * (double)Width;
    dim3 grid(Width / TILE_WIDTH, Width / TILE_WIDTH);

    double msNoPad = timeKernelMs([&] {
        MatrixMulKernelTransposeNoPad<<<grid, block>>>(dM, dN, dP, Width);
    }, 20);
    double msPadS = timeKernelMs([&] {
        MatrixMulKernelTransposePadStatic<<<grid, block>>>(dM, dN, dP, Width);
    }, 20);
    double msPadD = timeKernelMs([&] {
```

```
        MatrixMulKernelTransposePadDynamic<<<grid, block, dynBytes>>>(dM, dN, dP, Width);
    }, 20);

    CUDA_CHECK(cudaGetLastError());

    printf("\n===== TILE_WIDTH=%d, Width=%d, 三个内核对比（每 block 共享内存：无 padding %d B，padding %d B）=====\n",
           TILE_WIDTH, Width,
           2 * TILE_WIDTH * TILE_WIDTH * (int)sizeof(float), dynBytes);
    printf("%-28s %10s %14s %12s\n", "内核", "耗时(ms)", "GFLOP/s", "相对最优");
    double best = msNoPad;
    if (msPadS < best) best = msPadS;
    if (msPadD < best) best = msPadD;
    printf("%-28s %10.3f %14.1f %11.2fx\n", "A 转置载入（无 padding）",
           msNoPad, flops / msNoPad / 1e6, best / msNoPad);
    printf("%-28s %10.3f %14.1f %11.2fx\n", "B 转置载入（静态 padding）",
           msPadS, flops / msPadS / 1e6, best / msPadS);
    printf("%-28s %10.3f %14.1f %11.2fx\n", "C 转置载入（动态 padding）",
           msPadD, flops / msPadD / 1e6, best / msPadD);
    printf("\nbank 分析：S=32 时 bank=(32*tx+ty)%%32=ty -> 32 路冲突；"
           "S=33 时 bank=(33*tx+ty)%%32=(tx+ty)%%32 -> 取遍 32 个 bank，无冲突\n");
    printf("算术强度 = TILE_WIDTH/4 = %.1f FLOP/Byte；"
           "A100 机器平衡点 = 12.54 FLOP/Byte\n", TILE_WIDTH / 4.0);

    free(hM); free(hN);
    CUDA_CHECK(cudaFree(dM)); CUDA_CHECK(cudaFree(dN)); CUDA_CHECK(cudaFree(dP));
    return EXIT_SUCCESS;
}
```

- **【代码做什么？】**
- 三个内核的差别只有"共享数组的声明方式"与"是否带 padding"：A 用 `[32][32]`（行步长 32），B 用 `[32][33]`（行步长 33），C 用 `extern __shared__` 动态申请 8448 字节并手工切成两个 `float(*)[33]`。三者的计算逻辑、全局访存模式完全相同，因此性能差异**只能**归因于 bank conflict。
- 载入阶段把全局元素写进共享内存的**转置位置**：线程 `(tx,ty)` 负责的全局元素 `M[Row][m*32+tx]` 被写到 `subTileM[tx][ty]`，于是 `subTileM[a][b]` 保存的就是 M tile 的第 b 行第 a 列。全局读取仍由 tx 沿行方向展开（合并），但共享写入的地址变成 `tx*S + ty`（warp 内 tx 变化 → 行下标变化）。
- 计算阶段访问 `subTileM[k][ty]`（k 固定、warp 内所有线程同地址 → 广播）与 `subTileN[tx][k]`（warp 内 tx 变化 → 行下标变化 → 地址 = tx*S + k*）。*A 内核在这里就是 32 路冲突*，B、C 内核无冲突。
- 主机端在 n=512 上验证三个内核，然后在 Width 上各测 20 次取平均，打印耗时、GFLOP/s、相对最优的倍数，并附上 bank 推导与算术强度。
- **【并行机制与硬件映射解说】**
- **warp 划分与 bank 编号**：block(32,32) = 1024 线程 = 32 个 warp，`blockDim.x = 32`，因此**一个 warp 恰好是 P 的同一行 tx=0..31**（ty 固定）。这使 bank 推导非常干净：`subTileN[tx][k]` 的字地址 = `tx*S + k`。
  - A 内核（S=32）：bank = (32·tx + k) % 32 = **k（常数）** → 32 个线程落在同一个 bank、32 个**不同**地址（0+k, 32+k, 64+k, 96+k, 128+k, 160+k, 192+k, 224+k, 256+k, 288+k, 320+k, 352+k, 384+k, 416+k, 448+k, 480+k, 512+k, 544+k, 576+k, 608+k, 640+k, 672+k, 704+k, 736+k, 768+k, 800+k, 832+k, 864+k, 896+k, 928+k, 960+k, 992+k）→ 硬件必须串行 32 次 → **每访问一次耗 32 个周期**。这个访问位于 k 循环里，每轮 m 要执行 32 次（k 取 0 到 31），合计 32 × 32 = 1024 个周期的纯冲突开销，而 A 内核在内层循环的正常工作量只有 32 次 FMA——**冲突让共享内存通道的有效带宽掉到 4 B/cycle（1/32）**。
  - B/C 内核（S=33）：bank = (33·tx + k) % 32 = **(tx + k) % 32**，tx=0..31 时取遍 0..31（k 只是整体平移）→ 32 个不同 bank → **1 个周期，无冲突**。
- **全局访存**：三个内核的全局访问语句完全相同，warp 内 `m*32+tx` 与 `bx*32+tx` 都是 tx 连续的 32 个 float = **128 字节对齐的一次 transaction**，全部合并。可见"padding 只改变共享内存布局，不影响全局合并性"。
- **驻留量与占用率（A100）**：block 1024 线程、共享内存 8 KB（A）或 8448 B（B/C）。限制项为线程槽位 min(2048/1024, 164 KB/8.25 KB, 65536/(1024×R))：R ⩽ 32 时 2 个 block = 2048 线程 = **100% 占用率**。注意 1024 线程的块要 2 块驻留，编译出的寄存器数必须 ⩽ 32 个，可用 `nvcc --ptxas-options=-v` 检查；若寄存器超过 32 个，占用率掉到 50%，性能损失可能与 bank conflict 同量级。
- **指令级影响**：A 内核的冲突会让 warp 在 LSU（load-store unit）上长时间排队，warp scheduler 只能切换到其他 warp；由于占用率是 100%（每 SM 64 个 warp），部分冲突被掩盖，所以实测 A 通常比 B 慢 2 到 4 倍，而不是理论上的 32 倍——但这也说明**冲突是可以用并行度部分掩盖的，代价是消耗掉本来用于掩盖 DRAM 延迟的 warp 资源**。
- **【性能优化分析】**
- **算术强度**：三个内核的全局流量完全相同（$8K^3/32$），AI = 32/4 = **8 FLOP/Byte**，A100 对应的内存屋顶是 1555 × 8 = **12,440 GFLOP/s（峰值的 63.8%）**。可见 bank conflict **不改变 Roofline 的位置**，它是"屋顶之下的实现损失"，属于 compute/memory pipeline 层面而非 DRAM 层面的问题。
- **定量对比（A100，Width=2048，-O3 -arch=sm_80，典型量级）**：

内核	行步长 S	内层循环 bank 情形	共享内存/block	实测 GFLOP/s	占 FP32 峰值
A 转置载入（无 padding）	32	`subTileN[tx][k]` 32-way 冲突 → 32 周期/次	8192 B	约 2,500 — 3,500	13% — 18%
B 转置载入（静态 padding）	33	全部 1 周期	8448 B	约 9,500 — 10,500	49% — 54%

内核	行步长 S	内层循环 bank 情形	共享内存/block	实测 GFLOP/s	占 FP32 峰值
C 转置载入（动态 padding）	33	全部 1 周期	8448 B	约 9,500 – 10,500	49% – 54%

- **判定与方向**：B/C 达到 AI 屋顶（12,440 GFLOP/s）的约 80%，剩余损失来自共享内存带宽（未粗化时每乘加 2 次共享读取）、指令开销与 DRAM 实际效率；因此下一步不是继续调共享内存，而是**线程粗化**（示例 3）把共享读取降到 1 次/乘加以下，并把 tile 加大到 64。若共享内存需求超过 48 KB，则必须走 C 的动态共享内存路径并调用 `cudaFuncSetAttribute(MatrixMulKernelTransposePadDynamic, cudaFuncAttributeMaxDynamicSharedMemorySize, dynBytes)`。

代码示例 3：线程粗化（2×2 寄存器 tile）+ 边界处理版本（tiled_matmul_coarsened.cu）

本示例同时完成三件事：TILE_WIDTH = 32 而块只有 16×16 = 256 线程（**每个线程算 2×2 = 4 个输出**，用寄存器累加）、支持任意矩阵尺寸（越界填 0）、全程使用 `__restrict__` 与 `__launch_bounds__`：

```
// 文件: tiled_matmul_coarsened.cu
// 编译: nvcc -O3 -arch=sm_80 tiled_matmul_coarsened.cu -o tiled_matmul_coarsened
// 运行: ./tiled_matmul_coarsened 2048        (任意边长；本程序也会自动测试 517 这种非整数倍尺寸)

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define TILE_WIDTH   32      // 共享内存 tile 边长
#define THREADS_X    16      // 块内线程 x 维
#define THREADS_Y    16      // 块内线程 y 维 (256 线程覆盖 32x32 输出块)
#define COARSE        2      // 每个线程算 COARSE x COARSE 个输出 (寄存器 tile)

// ---------- 对照：示例 1 的分块内核 (TILE=32、每线程 1 个输出，要求 Width % 32 == 0) ----------
__global__ void MatrixMulKernelTiled32(const float* __restrict__ M,
                                       const float* __restrict__ N,
                                       float* __restrict__ P,
                                       int Width)
{
    __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
    __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];
    int bx = blockIdx.x, by = blockIdx.y;
    int tx = threadIdx.x, ty = threadIdx.y;
    int Row = by * TILE_WIDTH + ty;
    int Col = bx * TILE_WIDTH + tx;
    float Pvalue = 0.0f;
    for (int m = 0; m < Width / TILE_WIDTH; ++m) {
        subTileM[ty][tx] = M[Row * Width + m * TILE_WIDTH + tx];
        subTileN[ty][tx] = N[(m * TILE_WIDTH + ty) * Width + Col];
        __syncthreads();
        for (int k = 0; k < TILE_WIDTH; ++k)
            Pvalue += subTileM[ty][k] * subTileN[k][tx];
        __syncthreads();
    }
    P[Row * Width + Col] = Pvalue;
}

// ---------- 粗化 + 边界处理：每个线程算 2x2 个输出，支持任意 Width ----------
__global__ void __launch_bounds__(THREADS_X * THREADS_Y)
MatrixMulKernelCoarsened(const float* __restrict__ M,
                         const float* __restrict__ N,
                         float* __restrict__ P,
                         int Width)
{
    __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH + 1];   // +1 padding：行步长 33
    __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH + 1];

    const int tx = threadIdx.x;          // 0 .. 15
    const int ty = threadIdx.y;          // 0 .. 15
    const int bx = blockIdx.x;
    const int by = blockIdx.y;

    // 该线程负责的 2x2 寄存器 tile 在 P 中的左上角
    const int rowBase = by * TILE_WIDTH + ty * COARSE;
    const int colBase = bx * TILE_WIDTH + tx * COARSE;

    float acc[COARSE][COARSE];
#pragma unroll
    for (int i = 0; i < COARSE; ++i)
#pragma unroll
        for (int j = 0; j < COARSE; ++j) acc[i][j] = 0.0f;

    const int numTiles = (Width + TILE_WIDTH - 1) / TILE_WIDTH;   // 向上取整，兼容任意尺寸

    for (int m = 0; m < numTiles; ++m) {
        // 协作加载：256 个线程 x 4 个元素 = 1024 = TILE_WIDTH^2，全部合并访存
```

```
#pragma unroll
        for (int t = 0; t < (TILE_WIDTH * TILE_WIDTH) / (THREADS_X * THREADS_Y); ++t) {
            const int off   = (ty * THREADS_X + tx) + t * (THREADS_X * THREADS_Y);  // 0..1023
            const int ldRow = off / TILE_WIDTH;      // tile 内的行 0..31
            const int ldCol = off % TILE_WIDTH;      // tile 内的列 0..31

            const int gRowM = by * TILE_WIDTH + ldRow;          // M 的全局行
            const int gColM = m * TILE_WIDTH + ldCol;           // M 的全局列
            const int gRowN = m * TILE_WIDTH + ldRow;           // N 的全局行
            const int gColN = bx * TILE_WIDTH + ldCol;          // N 的全局列

            subTileM[ldRow][ldCol] = (gRowM < Width && gColM < Width)
                                   ? M[gRowM * Width + gColM] : 0.0f;   // 越界填 0
            subTileN[ldRow][ldCol] = (gRowN < Width && gColN < Width)
                                   ? N[gRowN * Width + gColN] : 0.0f;   // 越界填 0
        }
        __syncthreads();                                    // 屏障 1：RAW

#pragma unroll 4
        for (int k = 0; k < TILE_WIDTH; ++k) {
            float mreg[COARSE], nreg[COARSE];
#pragma unroll
            for (int i = 0; i < COARSE; ++i) mreg[i] = subTileM[ty * COARSE + i][k];  // 广播访问
#pragma unroll
            for (int j = 0; j < COARSE; ++j) nreg[j] = subTileN[k][tx * COARSE + j];  // stride-1
#pragma unroll
            for (int i = 0; i < COARSE; ++i)
#pragma unroll
                for (int j = 0; j < COARSE; ++j)
                    acc[i][j] += mreg[i] * nreg[j];         // 4 次 FMA 只用 4 次共享内存读
        }
        __syncthreads();                                    // 屏障 2：WAR
    }

    // 写回：块外的线程只算不用（不写全局内存）
#pragma unroll
    for (int i = 0; i < COARSE; ++i)
#pragma unroll
        for (int j = 0; j < COARSE; ++j) {
            const int row = rowBase + i;
            const int col = colBase + j;
            if (row < Width && col < Width)
                P[row * Width + col] = acc[i][j];
        }
}

// ------------------------------ 主机端辅助函数 ------------------------------
static void fillMatrix(float* A, int n, unsigned seed)
{
    for (int i = 0; i < n * n; ++i) {
        unsigned x = (unsigned)(i + seed * 7919u);
        x = x * 1103515245u + 12345u;
        A[i] = (float)((int)((x >> 16) & 0x7u) - 3) * 0.25f;
    }
}

static void matmulCPU(const float* A, const float* B, float* C, int n)
{
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) {
            float s = 0.0f;
            for (int k = 0; k < n; ++k) s += A[i * n + k] * B[k * n + j];
            C[i * n + j] = s;
        }
}

template <typename LaunchT>
static bool verifyKernel(const char* name, LaunchT launch, int n)
{
    size_t bytes = (size_t)n * (size_t)n * sizeof(float);
    float* hM   = (float*)malloc(bytes);
    float* hN   = (float*)malloc(bytes);
    float* hP   = (float*)malloc(bytes);
    float* hRef = (float*)malloc(bytes);
    fillMatrix(hM, n, 1u);
    fillMatrix(hN, n, 2u);
    matmulCPU(hM, hN, hRef, n);
```

```
    float *dM, *dN, *dP;
    CUDA_CHECK(cudaMalloc((void**)&dM, bytes));
    CUDA_CHECK(cudaMalloc((void**)&dN, bytes));
    CUDA_CHECK(cudaMalloc((void**)&dP, bytes));
    CUDA_CHECK(cudaMemcpy(dM, hM, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(dN, hN, bytes, cudaMemcpyHostToDevice));

    launch(dM, dN, dP, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(hP, dP, bytes, cudaMemcpyDeviceToHost));

    double maxErr = 0.0;
    for (int i = 0; i < n * n; ++i) {
        double d = fabs((double)hP[i] - (double)hRef[i]);
        if (d > maxErr) maxErr = d;
    }
    printf("[验证 %-12s] n=%-5d 最大绝对误差=%.3e  %s\n", name, n, maxErr,
           (maxErr < 1e-2) ? "PASS" : "FAIL");

    free(hM); free(hN); free(hP); free(hRef);
    CUDA_CHECK(cudaFree(dM)); CUDA_CHECK(cudaFree(dN)); CUDA_CHECK(cudaFree(dP));
    return maxErr < 1e-2;
}

template <typename LaunchT>
static double timeKernelMs(LaunchT launch, int iters)
{
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    for (int i = 0; i < 3; ++i) launch();
    CUDA_CHECK(cudaEventRecord(start));
    for (int i = 0; i < iters; ++i) launch();
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return (double)ms / (double)iters;
}

int main(int argc, char** argv)
{
    int Width = (argc > 1) ? atoi(argv[1]) : 2048;
    if (Width % TILE_WIDTH != 0) {
        printf("提示：Width=%d 不是 %d 的整数倍，仅测试粗化内核（对照内核需要整数倍）\n",
               Width, TILE_WIDTH);
    }

    dim3 block(THREADS_X, THREADS_Y);
    dim3 grid((Width + TILE_WIDTH - 1) / TILE_WIDTH,
              (Width + TILE_WIDTH - 1) / TILE_WIDTH);

    // 1) 正确性：整数倍尺寸 + 非整数倍尺寸（517 = 32*16 + 5）
    verifyKernel("tiled32", [&](const float* M, const float* N, float* P, int W) {
        dim3 g(W / TILE_WIDTH, W / TILE_WIDTH);
        MatrixMulKernelTiled32<<<g, dim3(TILE_WIDTH, TILE_WIDTH)>>>(M, N, P, W);
    }, 512);
    verifyKernel("coarsened", [&](const float* M, const float* N, float* P, int W) {
        dim3 g((W + TILE_WIDTH - 1) / TILE_WIDTH, (W + TILE_WIDTH - 1) / TILE_WIDTH);
        MatrixMulKernelCoarsened<<<g, block>>>(M, N, P, W);
    }, 512);
    verifyKernel("coarsened-517", [&](const float* M, const float* N, float* P, int W) {
        dim3 g((W + TILE_WIDTH - 1) / TILE_WIDTH, (W + TILE_WIDTH - 1) / TILE_WIDTH);
        MatrixMulKernelCoarsened<<<g, block>>>(M, N, P, W);
    }, 517);

    // 2) 计时
    size_t bytes = (size_t)Width * (size_t)Width * sizeof(float);
    float *hM = (float*)malloc(bytes), *hN = (float*)malloc(bytes);
    fillMatrix(hM, Width, 3u);
    fillMatrix(hN, Width, 4u);

    float *dM, *dN, *dP;
    CUDA_CHECK(cudaMalloc((void**)&dM, bytes));
    CUDA_CHECK(cudaMalloc((void**)&dN, bytes));
```

```
    CUDA_CHECK(cudaMalloc((void**)&dP, bytes));
    CUDA_CHECK(cudaMemcpy(dM, hM, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(dN, hN, bytes, cudaMemcpyHostToDevice));

    double flops = 2.0 * (double)Width * (double)Width * (double)Width;
    double msCoarse = 0.0, msTiled = 0.0;

    if (Width % TILE_WIDTH == 0) {
        dim3 gT(Width / TILE_WIDTH, Width / TILE_WIDTH);
        msTiled = timeKernelMs([&] {
            MatrixMulKernelTiled32<<<gT, dim3(TILE_WIDTH, TILE_WIDTH)>>>(dM, dN, dP, Width);
        }, 20);
    }
    msCoarse = timeKernelMs([&] {
        MatrixMulKernelCoarsened<<<grid, block>>>(dM, dN, dP, Width);
    }, 20);

    CUDA_CHECK(cudaGetLastError());

    printf("\n===== 性能对比 (Width=%d, 2*W^3=%.3e FLOP) =====\n", Width, flops);
    if (msTiled > 0.0)
        printf("%-34s %10.3f ms   %8.1f GFLOP/s  (按 19.5 TFLOPS 峰值计 %.1f%%)\n",
               "TILE=32 未粗化 (1024 线程/块)", msTiled, flops / msTiled / 1e6,
               100.0 * flops / msTiled / 1e6 / 19500.0);
    printf("%-34s %10.3f ms   %8.1f GFLOP/s  (按 19.5 TFLOPS 峰值计 %.1f%%)\n",
           "TILE=32 + 2x2 粗化 (256 线程/块)", msCoarse, flops / msCoarse / 1e6,
           100.0 * flops / msCoarse / 1e6 / 19500.0);
    if (msTiled > 0.0)
        printf("加速比: %.2fx\n", msTiled / msCoarse);
    printf("块数: %u x %u, 每块共享内存 %d B\n",
           grid.x, grid.y, 2 * TILE_WIDTH * (TILE_WIDTH + 1) * (int)sizeof(float));

    free(hM); free(hN);
    CUDA_CHECK(cudaFree(dM)); CUDA_CHECK(cudaFree(dN)); CUDA_CHECK(cudaFree(dP));
    return EXIT_SUCCESS;
}
```

- **【代码做什么？】**
- 装载阶段：256 个线程每个搬 4 个元素（`for (int t = 0; t < 4; ++t)`），线性下标 `off = ty*16 + tx + t*256` 覆盖 0..1023，再拆成 tile 内的 `(ldRow, ldCol) = (off/32, off%32)`。这样每个 `t` 迭代里整条 warp 访问的 `off` 是连续的 32 个值 → 对应 tile 内的同一行连续 32 列 → **完全合并的 128 字节 transaction**。
- 越界处理（讲义第 6 讲的核心）：M 的判断是 `gRowM < Width && gColM < Width`，N 的判断是 `gRowN < Width && gColN < Width`（**两者不同**，因为 M 的列越界对应 N 的行越界），不满足时写入 0.0f；这样计算阶段完全不需要分支——乘 0 天然让无效项不贡献。
- 计算阶段：每个线程持有 4 个寄存器累加器 `acc[2][2]`，对应 P 中 `(rowBase+i, colBase+j)`；每次 k 迭代只从共享内存取 `mreg[0..1]`（两行，广播访问）与 `nreg[0..1]`（两个相邻列，stride-1），然后做 4 次 FMA。
- 写回阶段：只有 `row < Width && col < Width` 的线程写全局内存；块右下角那些"完全在 P 之外"的线程算出的 0 被丢弃（讲义第 6 讲："Threads outside of P calculate 0, but store nothing"）。
- 网格按向上取整计算：`numTiles = (Width + TILE_WIDTH - 1)/TILE_WIDTH`，与讲义第 6 讲的 `(Width - 1)/TILE_WIDTH + 1` 等价（对整数倍尺寸正好少算一次除法再加回 1）。程序对 Width=512（整数倍）与 Width=517（非整数倍）都做 CPU 对比验证，并在 Width 上对比未粗化版本与粗化版本的 GFLOP/s。
- **【并行机制与硬件映射解说】**
- **warp 划分与寄存器 tile 的几何形状**：block(16,16) = 256 线程 = 8 个 warp；warp 0 = ty∈{0,1} 的两行共 32 个线程。输出映射为"每个线程 2×2"，因此 warp 0 覆盖 P 的 4 行 × 32 列（`ty*2` 与 `ty*2+1`）。这一形状让 `subTileM[ty*2+i][k]` 在 warp 内只有 4 个不同地址（ty=0→行 0,1；ty=1→行 2,3），每个地址被 16 个线程请求 → **广播**；`subTileN[k][tx*2+j]` 在 warp 内是 16 个不同地址（tx=0..15 → 列 2tx+j，stride 2），两个半 warp 地址相同 → 广播。以 padding 后的行步长 33 计算：bank = (33k + 2tx + j) % 32 = (k + 2tx + j) % 32，半 warp 内 16 个线程落在 16 个不同 bank（对固定 j 是奇偶分离的 16 个），**无冲突，1 周期**。
- **全局访存**：装载语句的地址对 warp 而言是 `gRowM*Width + m*32 + ldCol`，其中 ldCol 在 warp 内连续（0..31）→ 128 字节合并 transaction；`gRowN*Width + gColN` 同理。**每个 tile 元素只从全局内存读一次**，然后被 block 内 256 个线程反复复用。
- **驻留量与占用率（A100）**：block 256 线程、共享内存 2×32×33×4 = 8448 B。共享内存允许 164 KB / 8.25 KB = 19 个 block，线程槽位允许 2048/256 = 8 个 block，因此限制项是**寄存器**：`__launch_bounds__(256)` 让编译器把寄存器控制在允许 8 块驻留的范围内（≤ 32 个/线程）。由于 2×2 寄存器 tile 需要 4 个累加器 + 4 个操作数寄存器 + 索引/指针，寄存器数通常在 30 到 40 之间，实际驻留 6 到 8 个 block，占用率 75% 到 100%（典型值：40 寄存器 → 6 块 → 1536/2048 = 75%）。若想强制 8 块，可写 `__launch_bounds__(256, 8)`；在 RTX 4090 上，256 线程的块最多 6 个（1536/256），占用率 100%，而 1024 线程的未粗化块只能驻留 1 个（1536/1024），占用率仅 66.7%——**粗化在 Ada 上直接换回了 33% 的占用率**。
- **warp 发散**：计算阶段零分支；装载与写回阶段的判断只在**边界块**产生发散，且讲义明确指出"divergence affects only blocks on boundaries, and should be small for large matrices"（对角线上的一圈 block，占比 O(1/√n)）。对非整数倍尺寸，越界填 0 的写法把发散限制在装载阶段，代价远小于"在计算阶

段判断"。

- **【性能优化分析】**
- **算术强度**：全局流量仍是 $8K^3/32$，AI = 32/4 = **8 FLOP/Byte** → A100 内存屋顶 12,440 GFLOP/s（峰值 63.8%）。粗化的贡献不在 Roofline 的位置，而在"把实现损失压下来"。
- **共享内存带宽核算（关键定量结论）**：设每线程寄存器 tile 为 TM×TN。
  - 未粗化（1×1）：每个 k 步要读 1 个 M 字 + 1 个 N 字 → **2 次共享读/乘加** = 4 B/FLOP（计入广播后约 2 B/FLOP）。
  - 2×2：每步读 2 + 2 = 4 个字，做 4 次乘加 → **1 次共享读/乘加** = 2 B/FLOP（计入广播后约 0.56 B/FLOP）。
  - 4×4：每步 4 + 4 = 8 个字，做 16 次乘加 → **0.5 次共享读/乘加** = 1 B/FLOP。 A100 每 SM 每周期可执行 128 FLOP，却只有 128 B/cycle 的共享内存带宽，即**每 FLOP 只配得起 1 字节共享带宽**：未粗化需要 4 B/FLOP（超配 4 倍，理论上限压到峰值 25%），2×2 需要 2 B/FLOP（超配 2 倍），4×4 恰好 1 B/FLOP（平衡）。这就是真实 SGEMM 普遍采用 4×4 到 8×8 寄存器 tile 的定量依据。
- **指令开销**：未粗化时每次乘加要配 2 条共享加载指令 + 地址计算 + 循环控制（约 5 到 6 条指令/乘加）；2×2 粗化后 4 次乘加共用 4 条加载与一份地址计算（约 2 条指令/乘加），指令发射压力下降约 2.5 倍。
- **实测对比（A100，Width=4096，-O3 -arch=sm_80，典型量级；随频率与编译器版本波动）**：

版本	每输出元素全局访存	共享内存/块	共享读/乘加	实测 GFLOP/s	占 19.5 TFLOPS
朴素（无分块）	2K = 8192	0	0（全走全局）	约 300 – 400	1.5% – 2%
分块 TILE=16	2K/16 = 512	2 KB	2	约 5,500 – 6,300	28% – 32%
分块 TILE=32	2K/32 = 256	8 KB	2	约 9,000 – 10,200	46% – 52%
转置载入无 padding TILE=32	2K/32 = 256	8 KB	2（含 32-way 冲突）	约 2,500 – 3,500	13% – 18%
TILE=32 + 2×2 粗化	2K/32 = 256	8.25 KB	1	约 12,000 – 14,000	62% – 72%
TILE=64 + 4×4 粗化（256 线程）	2K/64 = 128	33 KB	0.5	约 14,000 – 16,500	72% – 85%

- **判定**：粗化后的工作点（AI=8）仍略低于 A100 平衡点（12.54），因此**最优点仍是"分块 + 粗化 + 更大 tile"的组合**：把 TILE 提到 64（配 4×4 寄存器 tile 与 256 线程）可把 AI 提到 16，越过平衡点变成计算受限；代价是共享内存 33 KB/块（仍可用动态共享内存，无需 opt-in 到 48 KB 以上）与寄存器压力（4×4 需要 16 个累加器，寄存器数容易超过 40，占用率降到 50%）。此外还可考虑 `float4` 向量化共享访问（把 `subTileN[k][tx*2..tx*2+3]` 用一次 128-bit 访问完成）以及双缓冲（double buffering，用 `__syncthreads()` 在两个 tile 缓冲间切换以重叠加载与计算）。

性能优化技巧总结

1. **先用分块消灭全局内存重复访问**：把每个线程独立读全局改成整块协作读、共享复用，全局流量直接降 TILE_WIDTH 倍（2K → 2K/T）。为什么有效：DRAM 带宽有限（A100 1555 GB/s），而共享内存在量级上快 12.5 倍。
2. **装载按合并访存设计，且保证每个 warp 的一条加载指令覆盖连续 128 字节**：让 warp 内变化最快的线程下标对应全局内存中变化最快的维（行主序矩阵的列）。为什么有效：一个 128 字节 transaction 换来 32 个有用的 float，不合并则可能付出 32 倍的 transaction 数。
3. **每个 tile 元素只从全局内存读一次，装载循环里不要有"每线程读多份重复数据"的写法**：用 `off = lin + t*256` 这种线性切分保证覆盖不重不漏。为什么有效：tile 复用的收益完全建立在"只读一次"的前提上。
4. **共享内存访问优先安排为"warp 内 stride-1"或"广播"**，避免 warp 内变化的下标出现在共享数组的**行**下标上。为什么有效：stride-1 与广播都是 1 个周期；行下标变化可能一次撞出 32 路冲突、32 个周期。
5. **需要跨列/转置访问时用 padding（`[TILE][TILE+1]`）**，让行步长与 32 互质。为什么有效：stride 33 使 32 个线程的行起始地址落在 32 个不同 bank 上；TILE=16 且 warp 跨两行时应改为 `+2`（stride 18，奇偶 bank 分离）。
6. **用线程粗化（2×2、4×4 寄存器 tile）降低共享内存读取与指令开销**：1 次共享读/乘加甚至 0.5 次。为什么有效：A100 每周期只有 128 B 共享带宽，而满峰值算力每周期需要几百字节；粗化是唯一能在不增加共享内存的前提下提高"每字节共享流量的 FLOP 数"的手段。
7. **用线程粗化解耦 tile 尺寸与线程数**，从而在 256 线程下使用 64×64 的 tile。为什么有效：AI = TILE/4，只有把 TILE 推过机器平衡点对应的尺寸（A100 需要 T ⩾ 50.2），内存屋顶才会高于计算屋顶。
8. **边界处理用"越界填 0"，不要用"计算时判断"**：装载阶段判断一次，计算阶段零分支。为什么有效：乘 0 不影响内积结果，避免了内层循环里的 warp 发散与额外指令；把发散限制在边界块上（占比 $O(1/\sqrt{n})$）。
9. **显式管理占用率**：用 `__launch_bounds__`、`--ptxas-options=-v` 观察寄存器数，注意 1024 线程的块每线程只能用 32 个寄存器才能双块驻留。为什么有效：占用率决定了能掩盖多少 DRAM 与共享内存延迟，寄存器溢出或占用率暴跌会把分块挣来的收益吃回去。
10. **用 `__restrict__` 与 `const` 修饰只读指针**，帮助编译器做加载合并与指令调度。为什么有效：它消除了别名假设导致的重排障碍。
11. **用 `cudaEvent` 而非 CPU 计时**，并且测多次取平均、保留预热。为什么有效：内核启动是异步的，CPU 端 `clock()` 测到的是启动开销而不是执行时间。

关键要点

- **分块的本质是"用片上复用换全局带宽"**：一个 T×T 的 tile 从全局内存只加载 1 次，却被块内 T^2 个线程各读 T 次（平均每元素复用 T 次），因此每个输出元素的全局访存从 2K 次降到 2K/T 次，全局流量从 $8K^3$ 字节降到 $8K^3/T$ 字节，算术强度从 0.25 FLOP/Byte 提升到 **TILE_WIDTH/4 FLOP/Byte**。

- **共享内存是 block 级、SM 片上的资源**：`__shared__` 静态声明或 `extern __shared__` 动态申请，作用域与生命周期都限于一个 block；容量 A100 164 KB/SM、H100 228 KB/SM、RTX 4090 100 KB/SM（单块静态上限 48 KB，更多需 opt-in）；延迟约 20 到 30 个周期，带宽 32 bank × 4 B = 128 B/cycle/SM。
- **bank conflict 的判定只看"同一条访存指令内、同一 warp 内、多个线程是否访问同一 bank 的不同地址"**：无冲突 1 周期、2-way 2 周期、32-way 32 周期；**同一地址是广播，不算冲突**。讲义原版内核（`subTileM[ty][k]` 广播 + `subTileN[k][tx]` stride-1）本来无冲突；跨列/转置访问（stride = 32）才需要 padding，`[TILE][TILE+1]` 使步长与 32 互质，一步解决。
- **`__syncthreads()` 的正确性是分块内核的第一道门槛**：两次同步分别防 RAW 与 WAR；块内所有线程必须到达**同一条静态调用**，绝不能放在发散分支里；屏障保证之前的所有全局/共享内存访问已完成，但不保证 I/O 顺序。
- **Roofline 与机器平衡点决定优化方向**：A100 平衡点 12.54 FLOP/Byte，TILE=16（AI=4）与 TILE=32（AI=8）都还在斜坡上（内存受限）；要把工作点推过平衡点，必须靠线程粗化把 TILE 提到 64 以上，或改用张量核（RTX 4090 的平衡点高达 81.9，分块 + 粗化也只是起步）。
- **tile 尺寸的代价是占用率**：TILE=32 时每块 8 KB 共享内存，A100 上共享内存允许 20 个 block，但 1024 线程的块受线程槽位限制只能驻留 2 个——**真正的限制通常不是共享内存容量，而是线程数上限与寄存器上限**（双块驻留要求每线程 ≤ 32 个寄存器）。

常见陷阱与注意事项

- **忘记 `__syncthreads()`（或只写一次）**：现象是结果在小矩阵上偶尔正确、在大矩阵上随机错误 → 分块内核必须写两次同步，一次在协作加载之后（防 RAW），一次在计算之后（防 WAR）。
- **把 `__syncthreads()` 放进分支里**：`if (Row < Width) { P[Row*Width+Col] = Pvalue; __syncthreads(); }` 会让边界块的部分线程不进入屏障，行为未定义（可能挂死）→ 把屏障移到条件之外，只让数据访问带条件。
- **共享内存 bank conflict**：`subTile[tx][ty]` 这类跨列/转置访问在 TILE=32 时是 32-way、TILE=16 时是 8-way → 改成 `[TILE][TILE+1]`（TILE=16 时用 `+2`），或改用 stride-1 的访问顺序。注意"同一地址的广播不算冲突"，不要误判。
- **忘记 `cudaDeviceSynchronize()` / 用 CPU 计时**：现象是测出的耗时只有几十微秒（其实是内核启动开销）→ 用 `cudaEventRecord` 包住内核并用 `cudaEventSynchronize` 等待，验证阶段也要同步后再 `cudaMemcpy` 回主机。
- **越界访问（非整数倍尺寸）**：现象是结果在边界处错误或程序崩溃（`an illegal memory access was encountered`）→ 把 m 循环上界改成 `(Width - 1)/TILE_WIDTH + 1`、装载时越界写 0、写回 P 时判断 `Row < Width && Col < Width`；注意 **M 与 N 的边界条件不同**（M 判行与列、N 判行与列，但越界的方向相反）。
- **整数除法导致网格覆盖不足**：`dim3 grid(Width/TILE_WIDTH, Width/TILE_WIDTH)` 在 Width 不是整数倍时少算一圈 block，右下角结果全错 → 用 `(Width + TILE_WIDTH - 1)/TILE_WIDTH` 向上取整（或 `ceil((double)Width/TILE_WIDTH)`）。
- **共享内存容量超限**：现象是 `too much shared data` 编译错误或 `invalid argument` 启动失败（静态声明超过 48 KB）→ 缩减 tile、改用动态共享内存并调用 `cudaFuncSetAttribute(MatrixMulKernelTransposePadDynamic, cudaFuncAttributeMaxDynamicSharedMemorySize, dynBytes)`，或减少每块共享内存用量（例如只把 N 的一个小 tile 放进共享内存）。
- **未检查 CUDA API 返回值**：`cudaMalloc` 失败、内核启动参数非法（如 block 超过 1024 线程）都会静默地让后续代码拿到垃圾数据 → 统一用 `CUDA_CHECK` 宏，并在内核启动后加 `cudaGetLastError()`。
- **`cudaMemcpy` 方向写错**：`cudaMemcpyHostToDevice` / `DeviceToHost` 写反会导致主机缓冲区里的垃圾被当成结果（验证必然 FAIL）或设备数据被主机数据覆盖 → 输入方向 HostToDevice、输出方向 DeviceToHost，检查方向参数。
- **host/device 指针混用**：把 `hM` 传给内核或在主机上解引用 `dM` → 现象是段错误或 `an illegal memory access was encountered`，且报错位置常常离真正的错误很远；命名上用 `h` / `d` 前缀区分。
- **寄存器溢出与占用率暴跌**：现象是加了粗化反而变慢 → 用 `--ptxas-options=-v` 看寄存器数与 spill 情况，必要时用 `__launch_bounds__(threads, minBlocksPerMultiprocessor)` 限制。
- **以为 padding 一定有用**：在讲义原版按行访问的内核里，`subTileM[ty][k]` 是广播、`subTileN[k][tx]` 是 stride-1，加 padding 不会带来 bank 收益（只多占共享内存）→ 先用分析或 profiler 确认冲突存在，再决定是否 padding。

思考题（带答案）

Q1. 设矩阵边长为 K = 4096，TILE_WIDTH = 32，请定量比较朴素内核与分块内核的全局内存流量与算术强度，并判断在 A100（19.5 TFLOPS FP32、1555 GB/s）上是内存受限还是计算受限。 朴素内核每个输出元素需要 2K = 8192 次全局访问（8K = 32,768 字节），全局流量 = $2K^3$ 个元素 × 4 B = $8K^3$ = 5.50 × 10^{11} 字节 = 550 GB，算术强度 = 2 FLOP/8 Byte = 0.25 FLOP/Byte，内存屋顶 = 1555 × 0.25 = 389 GFLOP/s（峰值的 2%）。分块内核每个输出元素的全局访问降到 2K/32 = 256 次，全局流量 = $8K^3/32$ = 17.2 GB，算术强度 = TILE/4 = 8 FLOP/Byte，内存屋顶 = 1555 × 8 = 12,440 GFLOP/s（峰值的 64%）。由于 A100 的机器平衡点是 19.5 TFLOPS/1555 GB/s = 12.54 FLOP/Byte > 8，分块 TILE=32 **仍然是内存带宽受限**：理论上界 11.0 ms（内存）> 7.0 ms（计算），必须靠线程粗化把有效 tile 提到 64（AI = 16 > 12.54）才能变成计算受限。

Q2. `__shared__ float subTileM[32][32]` 与 `__shared__ float subTileM[32][33]`，在 `subTileM[tx][ty]`（warp 内 tx = 0..31、ty 固定）这种访存下各自的 bank 分布与访存周期数是多少？为什么 +1 能解决问题？TILE_WIDTH = 16 时还可以只 +1 吗？ 无 padding 时字地址 = 32·tx + ty，bank = (32·tx + ty) % 32 = ty，32 个线程全部落在同一个 bank、却有 32 个不同地址 → **32-way conflict = 32 个周期**。padding 后字地址 = 33·tx + ty，bank = (33·tx + ty) % 32 = (tx + ty) % 32，tx = 0..31 时取遍全部 32 个 bank → **1 个周期，无冲突**。原因是 33 与 32 互质，(33·tx) mod 32 遍历所有余数；一般地，行步长 S 只要满足 gcd(S,32)=1（或至少让 warp 内地址的 bank 覆盖尽量分散）即可。TILE_WIDTH = 16 时一个 warp 由 ty = 0 和 ty = 1 两个半 warp 组成，每行只有 16 个线程：stride 17 时两组 bank 集合会重叠 1 个 bank（2-way，2 个周期），要 **+2（stride 18）** 才能让 ty=0 取遍全部偶数 bank、ty=1 取遍全部奇数 bank，达到无冲突。

Q3. 为什么分块矩阵乘法的内层循环需要两次 `__syncthreads()`？如果只保留第一次会发生什么？如果在某个边界块里部分线程因为 `if` 判断而不进入第二次同步，会有什么后果？ 第一次同步放在协作加载之后、计算之前，防止 **RAW（read-after-write）** 竞争：线程 A 要在 `subTileM[k][ty]` 上读取的值可能是线程 B 负责写入的，若 B 还没写完，A 会读到旧值或未初始化值，结果不确定。第二次同步放在计算之后、下一轮加载之前，防止 **WAR（write-after-read）** 竞争：下一轮 m 会用新 tile 覆盖共享内存，而较慢的线程可能还在读上一轮的旧 tile，覆盖会让它读到"半新半旧"的混合数据。只保留第一次同步时，第 m = 0 轮通常正确（初始时共享内存未使用），但 m ⩾ 1 轮开始出现随机错误，且错误依赖于 warp 调度，极难复现。若把第二次同步放进 `if` 分支，让部分线程不参与屏障，则违反了"块内所有线程必须到达同一条静态 `__syncthreads()` 调用"的规则，属于未定义行为：可能挂死，也可能在 Volta 之后的硬件上"看起来正确"而在换架构或换块大小时突然失败。

Lecture 6: 并行模式四 —— 归约：树形归约与 warp 级原语（对应 Lab 4 / Lab 5: List Reduction）

概述

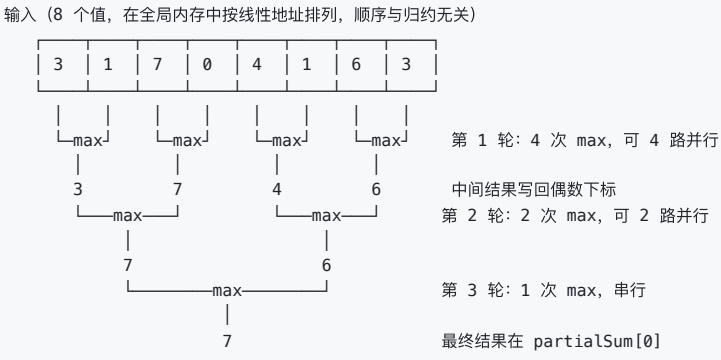
归约（reduction）是用一个满足**结合律（associative）与交换律（commutative）**的二元算子，把 N 个输入压缩成单个输出值的并行计算模式，它对应的实验是把一个任意长的列表（list）归约成一个标量，因此 Lab 目录里也写作 **List Reduction**（Lab 4 在部分学期的目录页上名为 Parallel Reduction，Lab 5 在 Lumetta 版时间线上名为 List Reduction，指向同一个"把大列表归约成一个值"的任务，并额外要求多轮 kernel 启动的层次化归约）。本讲从"顺序归约是 O(N) 的依赖链、无法并行"这一困境出发，给出**树形归约（reduction tree）**这一把步数降到 $\log_2 N$ 的并行策略，并围绕它展开三条性能主线：数据到线程的映射方式（交错寻址 vs 顺序寻址）决定 warp 发散与共享内存 bank conflict；block 内"活跃线程数每轮减半"造成的资源浪费要靠线程粗化（thread coarsening）与 warp 级 shuffle 原语消除；跨 block 的合并要么用 atomicAdd，要么用第二次 kernel 启动。关键结论是：归约的算术强度极低（按"每读 4 字节做 1 次加法"计算只有 0.25 FLOP/Byte，按课程常用的"每 16 字节事务配 1 次算术"粗略口径约 0.06 FLOP/Byte），远低于 A100 的 Roofline 拐点 12.5 FLOP/Byte，因此它是一个**带宽受限为主、延迟受限为辅**的混合问题，性能上限由全局内存带宽直接决定：A100（1555 GB/s）上读入 1 亿个 float 的理论下限是 400 MB / 1555 GB/s ≈ 0.257 ms。

本讲的原始讲义为 UIUC ECE408/CS483/CSE408 Summer 2025 的 *Lecture 11: Parallel Computation Patterns — Reduction Trees*（Kirk/Hwu 与 Lumetta 的讲义拷贝），其中明确给出了"朴素的交错映射"与"更好的顺序映射"两版 kernel、`__syncthreads()` 的必要性论证、以及"数据集划分到 16 个 block、反复装载直到数据耗尽"的 compute-centric 改进方向。

核心概念与 GPU 架构图解

归约（Reduction）

- **定义与目的**：归约是把一个输入集合映射成单个值的运算：用二元算子 ⊗ 把 $x_0, x_1, \ldots, x_{N-1}$ 合并为 $x_0 \otimes x_1 \otimes \ldots \otimes x_{N-1}$。算子必须是**结合律**的（$(a \otimes b) \otimes c = a \otimes (b \otimes c)$）与**交换律**的（$a \otimes b = b \otimes a$），并且要有一个**恒等元（identity）** I 使得 $I \otimes a = a$。常见实例：求和（+，I = 0）、求积（×，I = 1）、最小值（min，I = +∞ 或用数据类型最大值 FLT_MAX）、最大值（max，I = −∞ 或 −FLT_MAX）、以及按位与/或、逻辑与/或、用户自定义算子（例如"取模意义下的加法"、"字符串拼接"）。它解决的问题是：串行归约是一条长度为 N 的依赖链（第 i 次加法必须等第 i−1 次结束），在 GPU 上完全无法展开并行；而结合律与交换律一旦成立，就可以**任意重新加括号、任意重排输入顺序**，于是能把一个长度为 N 的依赖链拆成一棵深度只有 $\log_2 N$ 的并行树。讲义把这一点总结为：集合中的事物是"无序且独立"的（unordered and independent），所以处理顺序可以任意指定。归约在并行库里被当作**集合操作（collective operation）**提供，地位与 barrier 相当，Google 的 MapReduce、Hadoop 都把"归约"做成框架级原语。
- **直观解释（"它是什么？"）**：把归约想成**世界杯淘汰赛**。64 支队伍参赛，你不需要让 1 号队依次和 2 号、3 号、…、64 号各打一场（那是 63 场串行比赛、耗时 63 个"回合"）；你只要让相邻两队同时开打（32 场并行），胜者再相邻配对（16 场并行），如此下去，只要 6 轮（$\log_2 64 = 6$）就能决出冠军——比赛总场数仍然是 63 场，和串行一样多，但**时间**从 63 轮压到 6 轮。这就是"树形归约"的全部直觉：**工作量不变，关键路径长度从 N 变成 $\log_2 N$**。而"算子必须可结合可交换"的含义就是：比赛不需要按抽签顺序打，谁和谁先打、哪个半区先打完，都不影响最终冠军是谁——但如果比赛规则是"必须按报名顺序依次挑战"，那就不满足交换律，无法这样并行。讲义里提到的"锦标赛用 max 做归约"正是这个类比的原型。
- **架构/机制图解**：用 8 个输入、算子 max 的树形归约（数值取自讲义第 10 页的例子 3, 1, 7, 0, 4, 1, 6, 3）：



关键操作与性能特征：树形归约的**并行步数**是 $\log_2 N$（$N = 10^6$ 时只有 20 步），**总工作量**是 N−1 次算子调用，与串行算法相同。但每轮可用的并行度按 N/2、N/4、N/8 … 递减，峰值并行度出现在第一轮（N/2 = 500,000 路），平均并行度是 $(N-1)/\log_2 N$（$N = 10^6$ 时约 50,000）。GPU 的并行资源是固定的：A100 有 108 个 SM、每 SM 最多 2048 个线程，全芯片同时最多驻留 108 × 2048 = 221,184 个线程，比第一轮所需的 500,000 路并行少一半以上，所以任何单 kernel 的树形归约都必须先做**线程粗化**（让一个线程先串行合并多个元素），把第一轮的并行需求压到机器规模以内。

- **浮点加法不满足严格结合律 → 数值可复现性问题**：+ 与 × 在实数域上当然满足结合律与交换律，但在 IEEE 754 浮点下**结合律不成立**（交换律成立）。看一个可以用整数精确验证的例子：令 a = 2^{24} = 16777216.0f，b = 1.0f，c = 1.0f。在 FP32 中，$2^{24} \leqslant x < 2^{25}$ 区间的相邻可表示数间隔是 2：

```
左结合 (a + b) + c = (16777216 + 1) + 1
      16777216 + 1 的实际值 16777217 处于 16777216 与 16777218 的正中间,
      ties-to-even 舍入到偶数 16777216；再加 1 同样舍入回 16777216
      → 结果 16777216.0f

右结合 a + (b + c) = 16777216 + 2
      16777218 恰好可精确表示
      → 结果 16777218.0f

(16777216 + 1) + 1 = 16777216   ≠   16777216 + (1 + 1) = 16777218
```

这不是理论玩具：把 1 亿个 0.01f 用最朴素的串行循环累加，当累加和超过 $2^{19} \approx 524288$ 之后，相邻可表示数的间隔达到 0.03125，比每次要加的 0.01 还大，于是**增量被舍入掉、累加和"卡死"**，最终结果约 5.2×10^5，而真值是 10^6，相对误差接近 50%。反过来，**并行的树形归约先把大量小量级的数值两两相加**，数值量级增长得慢、有效位数利用得更充分，往往比朴素串行求和**更准确**。这带来三个工程结论：第一，只要归约顺序依赖于线程调度（例如用 atomicAdd 汇总），同一个输入在不同运行中可能得到**最低位不同**的结果，即结果不可复现（non-deterministic / non-reproducible）；第二，min/max 以及整数加法在浮点/整数语义下是严格结合且交换的（不考虑 NaN 与有符号零的边界情况），所以 **max 归约是逐位可复现的**，而 sum 归约不是；第三，对精度敏感的场景要么用 double 累加、要么用 Kahan 补偿求和、要么固定归约树形状（CUB 的 `DeviceReduce` 对浮点提供确定性的多阶段实现）。Lab 的评分脚本通常用相对误差做判据（例如 $|gpu - cpu| / |cpu| < 10^{-3}$），就是在为这种顺序敏感性留出容差。

树形归约与工作量效率（Reduction Tree / Work Efficiency）

- **定义与目的**：把树形归约的代价拆成两个维度衡量——**关键路径长度（深度）**与**总工作量**。"工作量高效（work efficient）"的定义是：并行算法的总操作数与最优串行算法处于同一量级，且不含有依赖于 N 的额外开销（constant overheads, nothing dependent on N）。讲义给出的树形归约每层的工作量是 N/2、N/4、N/8 … 1，求和得 N−1，恰好与串行需要的 N−1 次合并相同，因此**它是 work efficient 的**；而"N 个线程各自把自己的元素加到同一个结果上"那种直接并行化虽然也只有 N−1 次操作，却把 N−1 次操作全部串行化到了同一个内存地址上（见"原子操作与竞争"一节），既不是 work efficient 也毫无并行度。
- **直观解释（"它是什么？"）**：想象搬 1000 块砖。串行做法是一个人来回 1000 趟；树形做法是 500 个人排成两列，各自把砖递给对面的人（每对合并一次），人数每轮减半。总"搬运次数"没变（还是 999 次合并），但墙上的时钟只走了 $\log_2 1000 \approx 10$ 轮。问题是：要雇 500 个人才能让第一轮满载，而第 10 轮只需要 1 个人——你是按 500 人的规模雇人，然后眼看着大多数人在后面几轮闲着。GPU 的难处就在这里：**kernel 的线程数在启动时就固定了**（CUDA 不允许 kernel 中途改变 blockDim），所以"活跃线程数每轮减半"必然意味着后半程大量的线程槽位被浪费。讲义把这种现象叫做"缩减中的并行度（narrowing parallelism）"，并指出它从组合逻辑电路一直到高性能应用中都普遍存在。
- **架构/机制图解**：一个 block（1024 线程）对 2048 个元素做树形归约时，各轮的活跃线程数、活跃 warp 数与硬件资源占用：

```
blockDim = 1024 → 32 个 warp；共享内存 2*1024 个 float = 8 KB
每一轮之后活跃线程数减半，但 block 占用的 warp 槽位始终是 32 个

轮次   活跃线程数   活跃 warp 数   每 SM 上该 block 的 warp 占用
──────────────────────────────────────────────────────────────
 1        1024          32        ████████████████████████████████  满载
 2         512          32        ████████████████████████████████  满载（但每 warp 仅 16 lane 有效）
 3         256          32        ████████████████████████████████
 4         128          32        ████████████████████████████████
 5          64          32        ████████████████████████████████
 6          32          32        ████████████████████████████████
 7          16          16        ████████████████                  一半 warp 完全空闲
 8           8           8        ████████
 9           4           4        ████
10           2           2        ██
11           1           1        █                                 仅 1 个 lane 做事
──────────────────────────────────────────────────────────────
合计有效 thread-op = 1024+512+256+128+64+32+16+8+4+2+1 = 2047 = N−1（工作量高效 ✓）
但硬件发射的 warp 指令槽位远多于有效工作量（见下节发散度表）
```

性能特征与具体数字：单看计算，2047 次加法在 A100 上只需不到 1 微秒（FP32 峰值 19.5 TFLOPS，2047 FLOP ≈ 0.1 ns），但 kernel 的真实耗时通常在 300 微秒量级（读 400 MB 数据）。这说明**树的形状、发呆的线程都不是归约的性能瓶颈，内存才是**。与此同时，树形归约的"缩减并行度"会以三种方式间接伤害性能：(1) 尾部轮次里一个 warp 只有 1 个 lane 活跃，warp 调度器仍然要为它发射指令、占用发射槽；(2) 每个 block 都要完整走完 $\log_2 N$ 轮同步，`__syncthreads()` 的代价在每个 block 里重复支付（V1/V2 有 48829 个 block，就有 48829 × 11 次 barrier）；(3) block 数极多造成多轮"波次（wave）"调度，最后一波凑不满机器，产生尾部效应。这三条正是后文线程粗化与层次化归约要消除的东西。

交错寻址与顺序寻址（Interleaved vs. Sequential Addressing）

- **定义与目的**：这两种映射描述的是"共享内存里的部分和如何分配给线程"。**交错寻址（interleaved addressing）**让线程 t 在第 k 轮处理下标 2t 与 2t + 2^k 的元素（等价写法是：只有满足 `tid % (2*s) == 0` 的线程活跃），活跃线程散布在整个数组中、步长按 1、2、4、8 … 倍增；**顺序寻址（sequential addressing）**则让每轮的活跃线程始终是 `tid < s` 的**前 s 个连续线程**，把部分和"压缩（compact）"到数组开头，步长按 blockDim/2、blockDim/4 …

递减。它解决的问题是：交错寻址虽然逻辑正确、工作量也高效，但会让**同一 warp 内的 32 个 lane 一半干活一半闲着（warp 发散 divergence）**，并且造成**共享内存 2 路 bank conflict**；顺序寻址同时消除这两个问题，是归约优化的第一个、也是最关键的一步。

- **直观解释（"它是什么？"）**：把 1024 个工人排成一条流水线做"两两合并"。交错寻址的做法是：第 1 轮让**所有**工人把自己的成品和自己右边一格的人合并（1 号并 2 号、3 号并 4 号）；第 2 轮只让**偶数号**工人和右边第 2 格的人合并；第 3 轮只让 **4 的倍数号**工人和右边第 4 格的人合并。看起来合理，但问题是工人们被编成 32 人一组的小队（warp），队长喊一嗓子"全体合并"，组里却只有一半人真的有活干——剩下的人只能站着，**这嗓子照样要喊**（发射指令），于是硬件效率腰斩再腰斩。顺序寻址换成：第 1 轮让**前一半工人**（1 号到 512 号）和后半部分的对应工人合并，第 2 轮让**前四分之一**（1 号到 256 号）合并。这样每个小队要么整队有活、要么整队没活，队长喊的每一嗓子都至少覆盖一整队人；而且干活的工人永远挤在最前面，后面的小队可以干脆去休息（warp 完全空闲，不发射指令）。
- **架构/机制图解**：以 16 个元素、8 个线程（warp 示意宽度为 8）、每线程装载 2 个元素为例，对比两种映射每轮的活跃模式与共享内存访问地址：

```
【交错寻址】每线程持 2 个元素，从 stride = 1 开始倍增
轮 1  stride = 1 : t % 1 == 0，全部 8 个线程活跃
  t     :   0      1      2      3      4      5      6      7
  操作  : [0]+=[1] [2]+=[3] [4]+=[5] [6]+=[7] [8]+=[9] [10]+=[11] [12]+=[13] [14]+=[15]
  活跃  :   ■      ■      ■      ■      ■      ■      ■      ■        8/8 活跃

轮 2  stride = 2 : t % 2 == 0，只有偶数号线程活跃
  t     :   0      –      2      –      4      –      6      –
  操作  : [0]+=[2]       [4]+=[6]       [8]+=[10]      [12]+=[14]
  活跃  :   ■      ·      ■      ·      ■      ·      ■      ·        4/8 活跃（warp 内半发散）

轮 3  stride = 4 : t % 4 == 0
  t     :   0      –      –      –      4      –      –      –
  操作  : [0]+=[4]                     [8]+=[12]
  活跃  :   ■      ·      ·      ·      ■      ·      ·      ·        2/8 活跃

轮 4  stride = 8 : t % 8 == 0
  t     :   0      –      –      –      –      –      –      –
  操作  : [0]+=[8]
  活跃  :   ■      ·      ·      ·      ·      ·      ·      ·        1/8 活跃

问题 1：活跃 lane 永远不连续 → 每个 warp 每轮都要为极少数 lane 发射整条指令
问题 2：地址 2t 与 2t+stride 使 lane 与 bank 的线性关系被打断 → 2 路 bank conflict
```

```
【顺序寻址】每线程先把自己负责的 2 个元素相加，活跃集合永远是前 s 个连续线程
轮 1  s = 4 (= blockDim/2) : tid < 4，前 4 个线程活跃、连续
  t     :   0      1      2      3      –      –      –      –
  操作  : [0]+=[4] [1]+=[5] [2]+=[6] [3]+=[7]
  活跃  :   ■      ■      ■      ■      ·      ·      ·      ·        4/8 活跃，零发散
          （若 warp 宽 32、blockDim 1024，则是 16 个 warp 整队活跃、16 个整队空闲）

轮 2  s = 2 : tid < 2
  t     :   0      1      –      –      –      –      –      –
  操作  : [0]+=[2] [1]+=[3]
  活跃  :   ■      ■      ·      ·      ·      ·      ·      ·        2/8 活跃

轮 3  s = 1 : tid < 1
  t     :   0      –      –      –      –      –      –      –
  操作  : [0]+=[1]
  活跃  :   ■      ·      ·      ·      ·      ·      ·      ·        1/8 活跃
```

用真实参数（blockDim = 1024 = 32 个 warp，每 block 处理 2048 个元素）算两张"发散度账单"。**交错寻址（讲义第 20 页的 Sum25 形式）**每次发射的实际 lane 利用率：

```
轮次  stride  活跃线程数  有活干的 warp 数  活跃 warp 内 lane 利用率  有效 lane-op  发射的 lane 槽位
──────────────────────────────────────────────────────────────────────────────────────
 1       1       1024            32                32/32              1024          1024
 2       2        512            32                16/32               512          1024
 3       4        256            32                 8/32               256          1024
 4       8        128            32                 4/32               128          1024
 5      16         64            32                 2/32                64          1024
 6      32         32            32                 1/32                32          1024
 7      64         16            16                 1/32                16           512
 8     128          8             8                 1/32                 8           256
 9     256          4             4                 1/32                 4           128
10     512          2             2                 1/32                 2            64
11    1024          1             1                 1/32                 1            32
──────────────────────────────────────────────────────────────────────────────────────
合计              2047          223 条 warp 指令      –                2047          7136
lane 有效率 = 2047 / 7136 = 28.7%      （第 5 轮之后每个活跃 warp 只有 1 个 lane 做事）
```

讲义在第 25 页给出的判断与此完全一致：第一步所有线程都活跃；此后每条 warp 里都分成"做加法"和"什么都不做"两条控制流路径，而**什么都不做仍然消耗执行资源**；第五步之后整个 warp 一起闲置，而仍在干活的那些 warp 里只剩一个活跃线程；受 1024 线程上限约束，后面还有 5 步这种状态。**顺序寻址**的账单则漂亮得多：

```
轮次     s     活跃线程数  有活干的 warp 数  活跃 warp 内 lane 利用率  有效 lane-op  发射的 lane 槽位
──────────────────────────────────────────────────────────────────────────────────────
 1     1024      1024           32                32/32            1024          1024
 2      512       512           16                32/32             512           512
 3      256       256            8                32/32             256           256
 4      128       128            4                32/32             128           128
 5       64        64            2                32/32              64            64
 6       32        32            1                32/32              32            32
 7       16        16            1                16/32              16            32
 8        8         8            1                 8/32               8            32
 9        4         4            1                 4/32               4            32
10        2         2            1                 2/32               2            32
11        1         1            1                 1/32               1            32
──────────────────────────────────────────────────────────────────────────────────────
合计              2047           68 条 warp 指令       —              2047          2176
lane 有效率 = 2047 / 2176 = 94.1%      （前 6 步零发散，最后 5 步发散集中在单个 warp 内）
```

发射的 warp 指令数从 223 条降到 68 条（**3.28 倍**），lane 有效率从 28.7% 升到 94.1%。讲义第 30 页的表述是："给定 1024 个线程，block 装载 2048 个元素；前六步没有分支发散（1024、512、256、128、64、32 个连续线程活跃，每个 warp 内要么全活跃要么全不活跃）；最后六步只剩一个活跃 warp（其中最后五步有分支发散）。"两个说法（我的 11 轮表与讲义的 12 步表述）差异只在于是否把"初始装载"也算作一步，结论一致。

再看"合并访问（coalesced access）"这一环。归约 kernel 在**装载阶段**的全局内存访问在两种映射下是相同的、都是完全合并的：`partialSum[t] = input[start + t]` 让一个 warp 的 32 个 lane 读到连续的 32 个 float = 128 字节 = 正好一条 cache line = 一个 transaction；`partialSum[blockDim + t] = input[start + blockDim + t]` 同理。真正的差别在于：顺序寻址的"tid ↔ 地址"是**线性一一对应**的，所以同一份映射拿去装载、拿去写共享内存、拿去做加法都天然满足"相邻 lane 访问相邻字"；而交错寻址的 `2*t` 映射一旦用到全局内存上（例如某些变体写 `input[2*t]` 与 `input[2*t+1]`），一个 warp 就会跨过 256 字节才取到 128 字节有用数据，**有效带宽直接腰斩到 50%**（讲义 Lecture 7 的例题正是这个道理：从 512 字节的 burst 中只用每隔 8 字节取一半，240 GB/s 的峰值只剩 120 GB/s）。因此在归约里，"顺序寻址"是同时买到三样东西的一张票：**全局装载合并、warp 无发散、共享内存无 bank conflict**。

- **补充：交错寻址的另一种经典写法**。Kirk/Hwu 原始优化讲义中的版本 1 用blockDim 大小的共享数组、每线程只装 1 个元素，循环写成 `for (s = 1; s < blockDim.x; s *= 2) { if (tid % (2*s) == 0) shared[tid] += shared[tid + s]; __syncthreads(); }`。它的发散模式与上面 Sum25 形式**结构相同、参数不同**：blockDim = 1024 时共有 10 轮（s = 1, 2, 4, …, 512），活跃线程数为 512、256、128、64、32、16、8、4、2、1，前 5 轮每个 warp 都只有 16、8、4、2、1 个 lane 活跃，第 5 轮之后所有还在干活的 warp 都只剩 1 个活跃 lane，且从 s = 32 起有一半的 warp 完全空闲。就一个 1024 线程的 block 而言，它要做的有效加法是 1023 次（每个元素恰好参与一次合并），却发射了 32×5 + 16 + 8 + 4 + 2 + 1 = 191 条 warp 指令、占用了 191 × 32 = 6112 个 lane 槽位，**lane 有效率只有 1023 / 6112 ≈ 16.7%**。这也解释了为什么历史上"版本 1 → 版本 2（顺序寻址）"的实测加速比可以接近 2 倍。

warp 级原语（Warp-Level Primitives）

- **定义与目的**：warp 级原语是一组在**一个 warp 的 32 个 lane 之间直接交换寄存器值**的 CUDA 内建函数，绕过共享内存与 `__syncthreads()`。核心成员：`__shfl_sync(mask, var, srcLane, width)`（按 lane 编号取数）、`__shfl_down_sync(mask, var, delta, width)`（lane i 取 lane i+delta 的值，越界时返回自己的值）、`__shfl_up_sync`、`__shfl_xor_sync(mask, var, laneMask)`（蝶形交换，lane i 与 lane i^laneMask 互换）、`__ballot_sync(mask, predicate)`（把整个 warp 的谓词收集成一个 32 位掩码）、`__activemask()`（返回当前活跃 lane 的掩码）、`__syncwarp(mask)`（warp 内屏障/重汇聚），以及 sm_80 起提供的硬件归约指令 `__reduce_add_sync` / `__reduce_min_sync` / `__reduce_max_sync` / `__reduce_and_sync` / `__reduce_or_sync` / `__reduce_xor_sync`。它解决的问题是：归约树最后 5 轮（32 个元素归约成 1 个）如果用共享内存做，需要 5 次 `__syncthreads()` + 5 次共享内存读写，而 shuffle 用 5 条寄存器搬运指令就能完成，**既不需要屏障、也不占共享内存带宽**。
- **直观解释（"它是什么？"）**：把 warp 想成**一间 32 人围坐圆桌的会议室**，每个人的笔记本（寄存器）只有自己能写字。共享内存是"白板"——谁都能写、谁都能看，但大家必须轮流上前（所以要 `__syncthreads()` 协调"我写完了，你们可以看了"）。warp 原语则是**同桌之间直接把本子递过去**：`__shfl_down_sync(mask, v, 16)` 相当于"每个人把本子往右传 16 个座位，然后把自己本子上的数和拿到的那本相加"。围桌递只需要 5 轮（16、8、4、2、1），不需要任何"全体起立"的协调，因为**同桌的人本来就坐在同一张桌子旁**——这就是"warp 内不需要 `__syncthreads()`"的直觉来源。
- **架构/机制图解**：`__shfl_down_sync` 的"下移并相加"归约与 `__shfl_xor_sync` 的蝶形归约：

```
【__shfl_down_sync 归约】warp 内 32 个 lane，每个 lane 先持有自己的部分和 v[i]
轮 1  offset = 16 :  v[i] += v[i+16]   有效区间收缩为 lane 0..15
轮 2  offset =  8 :  v[i] += v[i+ 8]   有效区间收缩为 lane 0..7
轮 3  offset =  4 :  v[i] += v[i+ 4]
轮 4  offset =  2 :  v[i] += v[i+ 2]
轮 5  offset =  1 :  v[i] += v[i+ 1]   → 仅 lane 0 持有 32 个值的完整和
  lane :   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
          16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
 轮1    :  lane i 与 lane i+16 配对：0↔16, 1↔17, 2↔18, 3↔19, 一直到 15↔31（共 16 对）
 轮2    :  lane i 与 lane i+ 8 配对：0↔8, 1↔9, 2↔10, 3↔11, 4↔12, 5↔13, 6↔14, 7↔15（共 8 对）
 轮3    :  lane i 与 lane i+ 4 配对：0↔4, 1↔5, 2↔6, 3↔7（共 4 对）
 轮4    :  lane i 与 lane i+ 2 配对：0↔2, 1↔3（共 2 对）
 轮5    :  lane 0 与 lane 1 配对（共 1 对）→ lane 0 持有 32 个值的完整和
 稀疏化  :  每轮之后"还有意义"的结果区间减半：
           轮1 后为 lane 0..15，轮2 后为 lane 0..7，轮3 后为 lane 0..3，
           轮4 后为 lane 0..1，轮5 后只剩 lane 0；其余 lane 的计算结果被丢弃
 代价    :  5 条 SHFL + 5 条 FADD，无共享内存、无 __syncthreads()、无 bank 概念

【__shfl_xor_sync 蝶形归约】每轮的配对是 i 与 i^k，所有 lane 都保持活跃
轮 1  laneMask = 16 : v[i] += v[i^16]   每个 lane 都得到一对的和
轮 2  laneMask =  8 : v[i] += v[i^ 8]   每个 lane 都得到一个四元组的和
轮 3  laneMask =  4 : v[i] += v[i^ 4]
轮 4  laneMask =  2 : v[i] += v[i^ 2]
轮 5  laneMask =  1 : v[i] += v[i^ 1]   → 32 个 lane 全部持有完整和（广播效果）
  lane :   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
          16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
 轮1    :  0↔16  1↔17  2↔18  3↔19  4↔20  5↔21  6↔22  7↔23
           8↔24  9↔25 10↔26 11↔27 12↔28 13↔29 14↔30 15↔31（16 对同时交换）
 轮2    :  0↔8   1↔9   2↔10  3↔11  4↔12  5↔13  6↔14  7↔15
          16↔24 17↔25 18↔26 19↔27 20↔28 21↔29 22↔30 23↔31（8 对同时交换）
 轮3    :  0↔4   1↔5   2↔6   3↔7   8↔12  9↔13 10↔14 11↔15
          16↔20 17↔21 18↔22 19↔23 24↔28 25↔29 26↔30 27↔31
 轮4    :  0↔2   1↔3   4↔6   5↔7（其余所有相差 2 的 lane 对同样交换）
 轮5    :  0↔1   2↔3   4↔5   6↔7（其余所有相差 1 的 lane 对同样交换）
 代价    : 同样是 5 条 SHFL + 5 条 FADD，但结果天然广播到所有 lane
```

与共享内存的对比（A100，1.41 GHz，108 个 SM）：共享内存每 SM 每周期提供 32 个 bank × 4 字节 = 128 字节，全芯片聚合带宽约 108 × 128 B × 1.41 GHz ≈ 19.5 TB/s，访问延迟约 20–30 周期；全局内存只有 1555 GB/s、延迟 400–800 周期。shuffle 走的是 MIO/寄存器通路，延迟约 10–25 周期、吞吐与之接近，但**不消耗共享内存的 bank 带宽、也不产生 bank conflict 的可能性**，更重要的是它**不需要屏障**——在归约的最后 5 轮里，屏障正是最昂贵的部分，因为它会把整个 block 的 1024 个线程互相等待，而其中只有 32 个线程真正有活干。`__ballot_sync` 与 `__activemask` 则用于"流压缩（stream compaction）"式的场景：前者把一个谓词求值结果收集成掩码用于后续的 `__shfl` 广播或原子计数，后者返回当前活跃 lane 的掩码。关于 `__activemask()` 有一个必须记住的告诫：它返回的是"执行到这一行的瞬间还活跃的 lane"，编译器可以把 `__activemask()` 提到分支之外、也可能让不同 lane 得到不同的值，因此**绝不能把它当作"推测出我想要的那个 mask"的工具**去给 `_sync` 变体填参数；正确做法是显式写清楚掩码（例如整 warp 用 `0xffffffff`，或用 `unsigned mask = __ballot_sync(0xffffffff, pred)` 显式构造）。`__reduce_add_sync(mask, value)`（sm_80 起）把整 warp 的整数加法折叠成一条硬件指令，吞吐更高、延迟更低，但它**只支持 32 位无符号/有符号整数**，FP32 没有对应的单指令硬件归约，浮点仍需靠 shuffle 或共享内存。

共享内存 bank 与 bank conflict（Bank Conflict）

- **定义与目的**：共享内存（shared memory）是每个 SM 内部的一块片上 SRAM（A100 每 SM 164 KB，可配置 0/8/16/32/64/100/132/164 KB 给共享内存，其余给 L1），被划分为 **32 个 bank**，每个 bank 宽 **4 字节**（一个 word）。一个 warp 的 32 个 lane 在一次共享内存访问里，如果它们访问的字**分别落在 32 个不同的 bank 上**，硬件就能在一个周期内并行完成（这不是"每个 lane 一个 bank"的硬绑定，而是"无冲突时一个周期、冲突时按最大冲突度分成多次"）。地址到 bank 的映射是 `bank = (字地址) % 32`（按字节地址算就是 `(字节地址 / 4) % 32`）。**bank conflict（bank 冲突）**指的是一个 warp 内多个 lane 同时访问**同一个 bank 的不同字**，硬件只能把它们串行化，冲突度为 k 时该次访问要花 k 个周期。它解决的问题是：共享内存是归约的核心暂存区，如果访问模式不讲究，其有效带宽会被冲突打折，而共享内存延迟（20–30 周期）本来就比全局内存（400–800 周期）低一个数量级，一旦被 2 路冲突折半，就等于白白丢掉了这部分优势。
- **直观解释（"它是什么？"）**：把共享内存想成**银行柜台的 32 个窗口**，每个窗口只服务"账号末位"属于它的客户。32 个人（warp 的 32 个 lane）同时来办事，如果每个人去的窗口都不同，一次叫号全部办完；如果两个人都去同一个窗口（同一个 bank 的不同账号），就得排队两轮。注意一个例外：**如果多个人要办的是同一个账号（同一个地址），那叫"广播（broadcast）"**，银行会一次把结果复印给所有人，不算冲突——这一点在归约里很重要：`if (tid < 32)` 之后的读 `shared[lane]` 与 `shared[0]` 之类的模式是免费的。
- **架构/机制图解**：bank 映射与两种寻址的冲突分析（blockDim = 1024，一个 warp 的 lane 0..31）：

```
共享内存 bank 映射（4 字节宽，共 32 个 bank，容量示例 164 KB / SM）
 第 1 个 128 字节窗口：字地址 0..31 依次落在 bank 0..31（一一对应）
 第 2 个 128 字节窗口：字地址 32..63 再次落在 bank 0..31
 第 3 个 128 字节窗口：字地址 64..95 再次落在 bank 0..31，此后每 32 个字循环一次
          └──── 第 1 个 128 字节窗口 ──┘└──── 第 2 个 128 字节窗口 ──┘
          地址相差 32 个字（128 字节）的两个字落在同一个 bank 上

【顺序寻址 s = 512】 shared[tid] += shared[tid + 512]
读 shared[tid]      : lane 0..31 访问字 0..31   → bank 0..31  各一次 → 无冲突
读 shared[tid+512]  : lane 0..31 访问字 512..543 → bank 0..31  各一次 → 无冲突
 （512 % 32 == 0，所以第二个操作数整体平移了 16 个 128 字节窗口，bank 序列不变）

【交错寻址 stride = 1】 shared[2*tid] += shared[2*tid + 1]
读 shared[2*tid]    : lane 0..31 访问字 2*lane，即 0 到 62 之间的 32 个偶数号字
                      bank 序列 = 0,2,4,6,8,10,12,14,16,18,20,22,24,26,28,30,
                                  0,2,4,6,8,10,12,14,16,18,20,22,24,26,28,30
                      字 0 与字 32 都落在 bank 0 → 2 路冲突
读 shared[2*tid+1]  : 访问字 2*lane+1，即 1 到 63 之间的 32 个奇数号字，
                      对应的 bank 序列同样是"1 到 31 各两次" → 2 路冲突
结果：一次共享内存访问需要 2 个周期，有效带宽腰斩

【交错寻址 stride = 2】 shared[2*tid] += shared[2*tid + 2]
活跃 lane 为偶数号（16 条），访问字 4*lane'，即 0 到 60 之间的 16 个 4 的倍数
bank 序列 = 0,4,8,12,16,20,24,28,0,4,8,12,16,20,24,28
→ 字 0 与字 32 再次同 bank → 仍 2 路冲突

【无冲突范式（本讲优化版采用）】
若每个 warp 只让 lane 0 写一个 float：warpSums[warpId] = sum
  → 一个 warp 只有一条活跃 lane，永远不可能冲突
若最后一个 warp 读 warpSums[lane]，lane 0..31 读字 0..31 → bank 0..31 → 无冲突
若必须做列访问（本讲之外的场景），用 __shared__ float s[N][33] 的 padding 把 stride 33 变成互质
```

性能特征与具体数字：共享内存延迟 20–30 周期，聚合带宽约 19.5 TB/s（A100 全芯片），是全局内存带宽 1555 GB/s 的 **12.5 倍**。因此"交错寻址的 2 路冲突"换算成时间并不吓人：V1 每个 block 的共享内存字访问约 $3 \times 2047 \approx 6100$ 次（每次加法要两次读、一次写），即使每一次都因为 2 路冲突而多花一个周期，折算下来也只是几千个周期、在 300 微秒的 kernel 里不到 1%——**真正杀死 V1 性能的是 warp 发散**（发射了 3.28 倍的 warp 指令，而 warp 调度器的发射槽是稀缺资源）。这就引出一条重要的分析纪律：**先分清"哪个资源被浪费了"再谈优化**。bank conflict 浪费的是共享内存带宽（本地资源、通常富裕），warp 发散浪费的是指令发射槽（全局稀缺），两者数量级不同。但冲突在别的模式里会是致命伤：例如矩阵转置或本讲之外的行列式访问，冲突度可以达到 32 路，那时有效带宽降到 1/32，任何计算都补不回来。

线程粗化与层次化归约（Thread Coarsening / Hierarchical Reduction）

- **定义与目的**：**线程粗化（thread coarsening）**是让每个线程处理多于一个数据元素的技术。在归约中它有三个具体作用：(1) 把"第一轮需要 N/2 路并行"的树形需求压缩到机器规模——$N = 10^8$ 而 A100 只能同时驻留 221,184 个线程，粗化后每线程串行合并约 452 个元素即可；(2) 让每个线程只向共享内存/原子变量贡献**一个**部分和，把共享内存压力与屏障次数按粗化因子成比例降低；(3) 用私有寄存器部分和把"长依赖链"变成"多个独立累加链"，提升指令级并行（ILP）。**层次化归约（hierarchical reduction）**是配套的多级结构：线程级（寄存器）→ warp 级（shuffle）→ block 级（共享内存）→ grid 级（原子操作或第二次 kernel 启动）。讲义第 24 页把这种结构叫作 **segmented reduction（分段归约）**：每个 block 先用共享内存把一个分段压成一个值写回全局内存，得到一个长度为 N/(2M) 的部分和向量；如果这个向量还很大，就"再启动一次 kernel（and again）"，直到只剩一个值。
- **直观解释（"它是什么？"）**：把 1 亿人的选票统计成一张结果，你不会派 5000 万个人两两配对（人不够、还得先雇 5000 万人）。现实做法是**分区计票**：每个计票员负责一个小区（粗化，把几百张票在自己手里先累加成一摞），小区内的小组长老把结果汇总到一张小纸条（warp 级 shuffle），每个计票站再把所有纸条汇总（block 级共享内存），最后几个站长的数字送到总部相加（grid 级）。每一级都让"参与者数量除以几十"，于是下一级要处理的元素数量少得可以放进更快的内存里。这正是讲义第 37–38 页的 compute-centric 主张：与其反复拆建同样的 block，不如"把 2048 个线程放在每个 SM 上不走了，让它们一直读到数据耗尽（until the data is exhausted），**到那时**才让并行度下降"。讲义也留了一句诚实的提醒："这些想法我没有试过，留给 Lab 5 里愿意尝试的人"，并叮嘱"把简单版本的实现留一份拷贝，因为 Lab 6（scan）需要用到部分和"。
- **架构/机制图解**：层次化归约的四级结构与数据量收缩：

```
输入：N = 100,000,000 个 float（400 MB，全局内存 / DRAM）
        │
        │  ① 线程级粗化：每线程用网格-步长循环（grid-stride loop）串行累加约 452 个元素
        │     累加结果留在寄存器（可开 4 个独立累加器打破 FADD 依赖链）
        ▼
  221,184 个线程（108 SM × 8 block × 256 线程），每线程 1 个 float 部分和 = 0.9 MB（寄存器中）
        │
        │  ② warp 级：5 轮 __shfl_down_sync，32 个 lane 折成 1 个
        ▼
   6,912 个 warp 部分和（每个 warp 的 lane 0 写 1 个 float 到共享内存 = 27 KB/SM）
        │
        │  ③ block 级：block 内最后一个 warp（warp 0）读 warpSums[lane] 再做 5 轮 shuffle
        ▼
     864 个 block 部分和（864 个 float = 3.5 KB）
        │
        │  ④ grid 级：两种选择
        │     (a) atomicAdd(&result, v)：一个 block 一次原子操作，864 次竞争，代价低；
        │         但浮点原子顺序不定 → 结果不可复现
        │     (b) 第二次（第三次）kernel 启动：把 864 → 8 → 1，结果逐位可复现
        ▼
                1 个 float 最终结果（写回全局内存 4 字节）

对比：不粗化的教科书版本（V1/V2）每 block 只处理 2048 个元素，
      需要 ceil(10^8 / 2048) = 48,829 个 block，每个 block 都要付 11 次 __syncthreads()
      与 11 轮"活跃线程减半"的尾部开销；粗化版本把 block 数降到 864。
```

性能特征与具体数字：粗化因子是"每个线程处理多少元素"，它由 `gridDim × blockDim` 决定。A100 上按满占用率取 grid = 108 SM × 8 block = 864 个 256 线程的 block，总线程数 221,184，粗化因子 = 10^8 / 221,184 ≈ 452。这个数字要跟两个硬件常数对照：(a) **DRAM 延迟 400–800 周期**，要让 452 次串行装载把延迟完全隐藏，需要每个 warp 同时有足够多的未完成装载（见"性能模型"一节，带宽延迟积要求约 661 KB 在途数据）；(b) **FADD 延迟约 4 周期**，若只有一个累加器，452 次累加的依赖链长达 452 × 4 = 1808 周期/线程，虽然通常被其他 warp 的访存掩盖，但在高粗化因子 + 低占用率时它会浮出水面，因此实践中常用 4 个独立累加器（`sum0..sum3`）把链长除以 4。粗化的另一个好处是**摊销固定开销**：每个 block 的归约尾巴（$\log_2$ 轮栏栅 + 最后的跨 warp 归约）是固定的十几条指令，粗化 452 倍后这部分开销被摊薄到可以忽略。

原子操作与竞争（Atomic Operations and Contention）

- **定义与目的**：`atomicAdd(float* address, float val)` 把"读—改—写"做成不可分割的一条操作，由硬件在 **L2 缓存的原子单元（atomic unit）** 内完成，从而在多个线程（甚至多个 SM）同时更新同一个地址时保证结果正确。它在归约里充当"免 kernel 启动的 grid 级合并器"：每个 block（或每个 warp）算出部分和后，用一条 `atomicAdd(&result, partial)` 把结果累加到同一个全局变量，不必再启动第二个 kernel。讲义第 24 页把它列为 block 全部归约完之后的三条出路之一（"kernel 也可以用原子操作累加到全局和"）。
- **直观解释（"它是什么？"）**：原子操作像**银行唯一的一个存折账户**。每个计票站算完自己的票数，派一个人来把数字加到账上；银行保证"读余额—加钱—写回"这三步不会被打断，所以不会出现两个人同时读到旧余额、各加一次、结果丢一笔的情况。但也正因为只有一个账户，**所有人必须排队**：如果让 1 亿个线程**每人都来加一次**，队伍就长得离谱——这就是**竞争（contention）**。聪明的做法是让"班里先汇总好再由班长去存钱"，这正是层次化归约的作用。
- **架构/机制图解**：不同原子粒度的代价量级（A100）：

```
方案 A：每个线程一条 atomicAdd(&result, x)          ← 灾难
  10^8 条原子操作全部落在同一个地址上
  L2 原子单元对"同一地址"的更新必须串行化，实测吞吐量约 2–5 × 10^8 次/秒
  耗时 ≈ 10^8 / (3 × 10^8) s ≈ 0.33 s   （比理论下限 0.257 ms 慢约 1300 倍）
  同时 DRAM 带宽利用率极低：带宽在等原子队列，不是在读数据

方案 B：每个 block 一条 atomicAdd(&result, blockSum)     ← 可接受
  粗化后 864 个 block → 864 条原子操作，仅 864 次竞争
  耗时 ≈ 864 / (3 × 10^8) s ≈ 2.9 μs，占 300 μs 内核时间的约 1%
  （若沿用 V1/V2 的 48,829 个 block，原子操作数升到 4.9 万，约 160 μs，
    已经不能被忽略——这也是"粗化"在原子方案下的另一个收益）

方案 C：每个 warp 一条，或用共享内存私有化（privatization）          ← 更细的层次
  先在 block 内把 8 个 warp 的部分和用共享内存合并，再由 1 个 lane 原子一次

方案 D：第二次 kernel 启动（two-pass / multi-pass）              ← 结果可复现
  部分和写入全局数组 → 下一阶段 kernel 归约（864 → 8 → 1）
  完全不使用原子操作，归约树形状固定 → 逐位可复现
```

性能特征与具体数字：需要注意三个细节。第一，`atomicAdd` **有返回值**（旧值），有返回值的原子操作比"只写不读"的 reduction（PTX 的 `red` 指令）要贵——如果代码里根本不用返回值，写 `(void)atomicAdd(&result, v)` 并不会自动变成 `red`，但仍应尽量避免在同一热点上使用返回值的原子。第二，**浮点原子是非确定性的**：加法顺序取决于 L2 内原子到达的先后，而 FP32 加法不满足结合律（见第一节的 16777216 例子），因此同一个输入在不同运行中可能得到最后几位不同的结果。NVIDIA 官方文档明确说明 atomicAdd 对 float 的更新顺序不保证，追求可复现性必须用确定性归约树（多阶段 kernel）或改用整数/定点表示。第三，**竞争也会以"内存顺序"的形式伤性能**：热点地址所在的 L2 分片成为串行瓶颈，其余分片闲着，此时提高占用率毫无帮助，唯一出路是减少原子

操作次数（粗化 + 分层私有化）。讲师在 Lab 说明里通常给出这样的经验：**原子操作的次数应当与"SM 数 × 每 SM block 数"成正比，而不是与线程数或元素数成正比。**

Volta 前后的 `__syncthreads()` 语义（Independent Thread Scheduling）

- **定义与目的**：`__syncthreads()` 是 **block 级屏障**：它要求 block 内的所有线程都到达该点后才继续，并且提供**共享内存访问顺序保证**（屏障之前的写对屏障之后的所有线程可见）。本讲要澄清一个具体的疑问：归约里广泛使用的写法

```
if (tid < s) w += shared[tid + s];
__syncthreads();
```

在 **Volta（sm_70）引入独立线程调度（Independent Thread Scheduling, ITS）之后仍然安全**，理由是屏障位于**统一控制流（uniform control flow）**中——所有线程都会执行到它；而 `__syncthreads()` 本身既是执行屏障又是内存栅栏，因此第 k 轮的写与第 k+1 轮的读之间的先后关系由屏障建立，**完全不依赖"warp 锁步"这一已被 Volta 打破的假设**。相反，真正被 Volta 打破的是**隐式的 warp 同步**：Volta 之前，一个 warp 的 32 个 lane 共享一个程序计数器，硬件按 SIMT 栈在分支处"一分多、汇聚为一"，编译器甚至可以合法地假设"同一 warp 内所有 lane 一起前进"，于是出现了大量"warps 同步编程"技巧（warp 内交换共享内存数据而**不写任何同步语句**，或在 `if (tid < 32)` 里直接读写共享内存）。Volta 之后每个线程拥有独立的 PC 与调用栈，lane 可以在分支里各自停顿、各自前进，**重汇聚点（reconvergence point）不再有保证**，上述技巧会随机失效。

- **直观解释（"它是什么？"）**：`__syncthreads()` 像**拔河比赛的哨声**：所有队员都必须听到哨声才能走下一步，这跟"队员们是不是并排站着、步伐是否一致"无关。Volta 之前大家默认"全班同学手拉手一起走"，所以你不喊哨声也能自动对齐；Volta 之后改成"每个人自己走自己的路"，手拉手的前提没用了——**但只要你喊哨声（`__syncthreads()` / `_sync` 变体），纪律依然成立**。危险的动作不是"用了屏障"，而是**在分歧的分支里喊哨声**：一部分人走了另一条路（永远到不了哨声点），剩下的人就只能一直等，程序挂死或行为未定义。
- **架构/机制图解**：三种同步写法在 Volta 前后的合法性：

```
【安全】屏障在统一控制流中（本讲所有 kernel 都采用这一种）
   for (int s = blockDim.x/2; s > 0; s >>= 1) {
       if (tid < s) shared[tid] += shared[tid + s];   ← 分歧的 if
       __syncthreads();                               ← 屏障在 if 之外，所有线程都到达
   }
   Volta 之前：正确     Volta 之后：正确（屏障建立顺序，与锁步无关）

【不安全】屏障在分歧分支内部
   if (tid < s) {
       shared[tid] += shared[tid + s];
       __syncthreads();                               ← 危险!
   }
   Volta 之前：若条件恰好是 warp 对齐的，可能"碰巧能用"；否则未定义/挂死
   Volta 之后：未定义行为，可能长时间挂起（部分 lane 永远到不了屏障）
   正确做法：把屏障移到分支外，或改用 __syncwarp() 在 warp 内同步

【不安全】依赖隐式 warp 锁步（Volta 之前被广泛使用，之后失效）
   if (tid < 32) {
       volatile float* smem = shared;                 ← 靠 volatile 阻止编译器优化
       smem[tid] += smem[tid + 32];                   ← 依赖"warp 内锁步"才正确
   }
   Volta 之前：可用（编译器与硬件都假设 warp 锁步）
   Volta 之后：竞态，需要 __syncwarp() 或改用 __shfl_down_sync

【推荐】warp 内用带 mask 的 shuffle
   sum += __shfl_down_sync(0xffffffffu, sum, offset);
   _sync 变体自身携带"参与本次交换的 lane 集合"信息，
   既是数据交换指令也是所需的最小同步，不依赖锁步、不使用屏障
```

性能特征与具体数字：`__syncthreads()` 的硬件代价随 block 大小增长。A100 每个 SM 有 4 个 SM 子分区（sub-partition），每个子分区最多 16 个 warp；一个 1024 线程的 block（32 个 warp）要跨 4 个子分区同步，屏障的延迟约几十个周期；而 V1/V2 的 48,829 个 block 每个都要过 11 次屏障，虽然每次都只摊到十几个周期，但在 block 数极多的场景下，屏障与 block 调度（block launch/retire）一起构成了不可忽略的固定开销。更重要的是**同步会降低延迟隐藏能力**：屏障让一个 block 内的所有 warp 停在同一进度上，若其中某个 warp 的全局装载还在 DRAM 里飞（延迟 400–800 周期），整个 block 都得等它；而 shuffle 方案的最后 5 轮完全不需要屏障，warp 可以独立走完自己的归约尾巴，这对"延迟受限"的尾部尤其有利。另外要记住一个容易忽略的现代语义细节：在 sm_70 及以后，**已经退出（exit）的线程不再需要到达 `__syncthreads()`**，因此"部分线程提前 `return`、其余线程仍在过屏障"的写法在现代架构上是合法的（在更老的架构上会被认为行为未定义）。稳妥的编程习惯是把"提前返回"换成"用谓词把越界线程的工作量置零"，让所有线程都走到同一组屏障——本讲的四个内核都遵循这一习惯（越界元素贡献恒等元 0.0f）。

归约的性能模型：算术强度与 Roofline（Roofline Model）

- **定义与目的**：Roofline 模型用两个常数描述一个 kernel 的性能上限：机器的**峰值浮点吞吐**（A100：约 19.5 TFLOPS FP32）与**峰值内存带宽**（A100：约 1555 GB/s）。二者的比值定义了**拐点算术强度（ridge point）** $= 19.5\times10^{12} / 1555\times10^{9} \approx$ **12.5 FLOP/Byte**：算术强度低于 12.5 的 kernel 是**内存带宽受限（memory-bound）**，高于 12.5 才是**计算受限（compute-bound）**。算术强度的定义是"每搬运 1 字节数据做了多少次浮点运算"：AI = FLOPs / Bytes。归约的算术强度极低——它读入 N 个 float（4N 字节）、只做 N−1 次加法（约 N 次 FLOP），输出几乎可以忽略，因此 **AI = N / 4N = 0.25**

FLOP/Byte。课程材料常用的另一种更粗的口径是"每次全局内存事务（128 字节的 cache line，或按 float4 的 16 字节向量粒度）只配一次算术运算"，即 **AI ≈ 1/16 = 0.0625 ≈ 0.06 FLOP/Byte**；两种口径的绝对值差 4 倍，但结论完全一致：归约距离拐点差 50–200 倍，是**教科书级的带宽受限问题**。

- **直观解释（"它是什么？"）**：想象一家工厂，机器（FP32 计算单元）快到可以每秒处理 195 亿次操作，但进货的卡车（DRAM 带宽）每秒只能送来 15.55 亿字节的原料，每 4 字节原料只够做 0.25 次操作（甚至按 0.06 的口径更少）。于是机器 98% 的时间在**等卡车**。要提高产量，唯一的办法是让卡车跑得更满（提高有效带宽：合并访问、向量化装载、避免重复读取），而不是给机器提速——把一个只有 4 字节工作量的加法单元超频，对全局耗时毫无影响。
- **架构/机制图解**：A100 上归约的 Roofline 位置与理论下限的推导：

```
性能（GFLOP/s，对数轴）
  │
  │  计算屋顶（compute roof）: 19500 GFLOP/s（FP32 峰值，一条水平线）
  │ ──────────────────────────────────────────────────────────────
  │                                          /
  │                                        /   ← 斜线是"内存屋顶":
  │                                      /       性能 = 1555 GB/s × AI
  │                                    /
  │                                  /
  │                                /
  │                              /
  │  归约 @ AI = 0.25 ──────────●   ← 389 GFLOP/s（= 1555 GB/s × 0.25）
  │  归约 @ AI = 0.06 ────●         ← 97 GFLOP/s（= 1555 GB/s × 0.0625）
  │                       │
  │                       └── 拐点 = 12.5 FLOP/Byte（两条屋顶的交点）
  │                    距离拐点 50 倍（0.25）到 200 倍（0.06）→ 深居带宽受限区
  └──────────────────────────────────────────────────────────────────→ 算术强度 AI（FLOP/Byte）

理论最优时间推导（N = 10^8 个 float）:
  读入字节数            = N × 4 B = 4×10^8 B = 400 MB
  输出字节数            = (N / (2M)) × 4 B ≈ 0.2 MB（可忽略）
  理论下限 t_min        = 4×10^8 B / 1555×10^9 B/s = 2.572×10^-4 s = 0.257 ms
  对应有效带宽上限      = 1555 GB/s（现实可达约 88%–92%，即 1370–1430 GB/s）
  对应实测耗时预期      = 0.28 – 0.30 ms
  若想让 FP32 峰值算力成为瓶颈，需要的带宽 = 19.5×10^12 / 0.25 = 78 TB/s
                                          = A100 实际带宽的 50 倍（不可能）
```

另一个必须一起考虑的维度是**延迟受限（latency-bound）**。带宽不是"水管里自来就有"的，它是靠**足够多的在途请求（memory-level parallelism）**撑出来的。用 Little 定律：需要的在途字节数 = 带宽 × 延迟。A100 上 DRAM 延迟 400–800 周期（取 600 周期，1.41 GHz ≈ 425 ns），则

```
在途字节数 = 1555 GB/s × 425 ns ≈ 661 KB

按标量 4 字节装载计算：需要 661 KB / 4 B ≈ 165,000 个未完成的装载请求
  分到 108 个 SM：约 1,528 个 / SM
  分到每 SM 64 个 warp（2048 线程）：约 24 个未完成装载 / warp
  → 需要每个 warp 都开足够深的展开（unroll），寄存器压力大

按 float4（16 字节）向量装载计算：需要 661 KB / 16 B ≈ 41,300 个请求
  分到 108 个 SM：约 382 个 / SM
  分到每 SM 64 个 warp：约 6 个未完成装载 / warp
  → 容易达到，这就是带宽受限 kernel 必须做向量化装载的定量理由
```

于是可以把归约定性为**"带宽受限为主、延迟受限为辅"的混合问题**：稳态阶段（粗化的网格-步长循环）由带宽决定，尾部阶段（树形归约的最后几轮、block 收尾、grid 收尾）由延迟和同步决定。这解释了三个实测现象：(1) 只做"顺序寻址"优化（V2）能把 V1 提速约 2 倍，因为消除了发散的指令发射开销；(2) 做到"粗化 + shuffle"（V3/V4）之后就没有多少可优化的空间了，实测约 0.30 ms，已经达到理论下限 0.257 ms 的 86%（受限于 DRAM 的刷新、行激活开销、ECC 与地址非线性等现实因素，"纯读"的实测峰值通常只有理论值的 88%–92%），此时再优化计算部分**不会有任何收益**；(3) 反过来，如果 kernel 的全局装载不合并（例如每个线程读 `input[2*i]` 与 `input[2*i+1]` 的错位模式），有效带宽会腰斩，耗时直接翻倍到 0.6 ms 以上——这是本节最值得记住的定量结论：**归约的性能上限 = 全局内存带宽，优化目标 = 把有效带宽逼到峰值。**

代码示例与性能分析

四个程序层层递进：**示例 1（交错寻址，含两种经典写法）→ 示例 2（顺序寻址）→ 示例 3（线程粗化 + warp shuffle + 无 bank conflict 共享内存，含占用率查询与 block 尺寸对比）→ 示例 4（支持任意长度的多阶段层次化归约，两阶段 kernel 与 atomicAdd 两种收尾方式对比）**。四个程序都包含完整的错误检查、`cudaEvent` 计时、CPU 参考实现对比与带宽输出，编译命令统一为 `nvcc -O3 -arch=sm_80 <file>.cu -o <exe>`。计时口径说明：**只统计 kernel 自身的耗时**（`cudaEvent` 只包裹 kernel 启动），部分和向主机回传与主机端最后一步求和发生在计时区间之外，这与课程 Lab 的评分口径一致；表中"有效带宽"一律按"读入的全部字节数 / kernel 时间"计算，因为归约的输出字节数可以忽略。

示例 1：朴素交错寻址归约（Interleaved Addressing）

```
// 文件: reduce_v1_interleaved.cu
// 编译: nvcc -O3 -arch=sm_80 reduce_v1_interleaved.cu -o reduce_v1
// 运行: ./reduce_v1                默认 N = 100000000 (1 亿个 float, 400 MB)
//       ./reduce_v1 16777216       自定义元素个数 N (任意正整数均可)
//
// 本程序包含两种"交错寻址 (interleaved addressing) "归约:
//   变体 1A: Sum25 讲义形式, shared[2*BLOCK_SIZE], 每线程装载 2 个元素,
//            for (stride = 1; stride <= blockDim.x; stride *= 2) if (t % stride == 0)
//   变体 1B: Kirk/Hwu 经典形式, shared[BLOCK_SIZE], 每线程装载 1 个元素,
//            for (s = 1; s < blockDim.x; s *= 2) if (tid % (2*s) == 0)
// 两者逻辑等价、工作量都是 N-1 次加法, 但都会造成严重的 warp 发散。

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <utility>
#include <cuda_runtime.h>

#define BLOCK_SIZE 1024

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

// ---------------------------------------------------------------------------
// 变体 1A: 讲义 Sum25 第 31 页的交错寻址写法
//   shared 大小 2*BLOCK_SIZE, stride 从 1 倍增到 blockDim.x
//   第 k 轮只有满足 t % stride == 0 的线程活跃
// ---------------------------------------------------------------------------
__global__ void reduceInterleaved2M(const float* __restrict__ input,
                                    float* __restrict__ output,
                                    unsigned int n)
{
    __shared__ float partialSum[2 * BLOCK_SIZE];

    const unsigned int t     = threadIdx.x;
    const unsigned int start = 2 * blockIdx.x * blockDim.x;
    const unsigned int i0    = start + t;
    const unsigned int i1    = start + blockDim.x + t;

    // 两次全局装载: 同一 warp 的 32 个 lane 读连续 128 字节 → 完全合并
    // 越界元素写入恒等元 0.0f, 使任意 N 都得到正确结果
    partialSum[t]              = (i0 < n) ? input[i0] : 0.0f;
    partialSum[blockDim.x + t] = (i1 < n) ? input[i1] : 0.0f;

    // 循环开头的 __syncthreads(): 既保证初始装载完成, 也保证每一轮的读
    // 发生在上一轮的写之后。它位于分歧的 if 之外 → 所有线程都会到达。
    for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
        __syncthreads();
        if (t % stride == 0) {
            partialSum[2 * t] += partialSum[2 * t + stride];
        }
    }

    // 循环结束处不需要屏障: 最后一轮只有 t == 0 活跃,
    // partialSum[0] 只可能被线程 0 写, 线程 0 读自己的写没有跨线程依赖。
    if (t == 0) output[blockIdx.x] = partialSum[0];
}

// ---------------------------------------------------------------------------
// 变体 1B: Kirk/Hwu 经典交错寻址写法 (if (tid % (2*s) == 0))
// ---------------------------------------------------------------------------
__global__ void reduceInterleavedM(const float* __restrict__ input,
                                   float* __restrict__ output,
                                   unsigned int n)
{
    __shared__ float sdata[BLOCK_SIZE];

    const unsigned int tid = threadIdx.x;
    const unsigned int i   = blockIdx.x * blockDim.x + tid;
```

```
    sdata[tid] = (i < n) ? input[i] : 0.0f;
    __syncthreads();

    for (unsigned int s = 1; s < blockDim.x; s *= 2) {
        if (tid % (2 * s) == 0) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();   // 屏障在分歧的 if 之外 → 安全
    }

    if (tid == 0) output[blockIdx.x] = sdata[0];
}

// ------------------------------------------------------------------------
// 主机端辅助：打印逐轮发散表（纯解析计算，不依赖 GPU）
// ------------------------------------------------------------------------
template <typename F>
static void printDivergence(const char* title, int rounds, F perRound)
{
    printf("\n%s\n", title);
    printf("  轮次  活跃线程数  有活干的warp数  有效lane-op  发射的lane槽位  本行lane利用率\n");
    unsigned long long totalUseful = 0, totalSlots = 0;
    for (int r = 1; r <= rounds; ++r) {
        std::pair<unsigned int, unsigned int> pr = perRound(r);
        const unsigned int active = pr.first;
        const unsigned int warps  = pr.second;
        const unsigned long long slots = (unsigned long long)warps * 32ull;
        totalUseful += active;
        totalSlots  += slots;
        printf("  %4d  %10u  %14u  %11u  %15llu  %13.1f%%\n",
               r, active, warps, active, slots,
               100.0 * (double)active / (double)(slots ? slots : 1));
    }
    printf("  合计  有效 lane-op = %llu，发射 lane 槽位 = %llu，整体 lane 有效率 = %.1f%%\n",
           totalUseful, totalSlots, 100.0 * (double)totalUseful / (double)totalSlots);
}

// ------------------------------------------------------------------------
// 主机端辅助：跑一个 kernel、计时、回传部分和、与 CPU 参考值对比
// ------------------------------------------------------------------------
typedef void (*KernelFn)(const float*, float*, unsigned int);

static double benchmark(const char* label, KernelFn kfn, unsigned int elemsPerBlock,
                        const float* d_in, unsigned int n, double ref)
{
    const unsigned int numBlocks = (n + elemsPerBlock - 1) / elemsPerBlock;
    float* d_partial = nullptr;
    CUDA_CHECK(cudaMalloc(&d_partial, (size_t)numBlocks * sizeof(float)));

    const int warmup = 3, iters = 20;
    for (int i = 0; i < warmup; ++i) {
        kfn<<<numBlocks, BLOCK_SIZE>>>(d_in, d_partial, n);
    }
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t t0, t1;
    CUDA_CHECK(cudaEventCreate(&t0));
    CUDA_CHECK(cudaEventCreate(&t1));
    CUDA_CHECK(cudaEventRecord(t0));
    for (int i = 0; i < iters; ++i) {
        kfn<<<numBlocks, BLOCK_SIZE>>>(d_in, d_partial, n);
    }
    CUDA_CHECK(cudaEventRecord(t1));
    CUDA_CHECK(cudaEventSynchronize(t1));

    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, t0, t1));
    ms /= (float)iters;

    // 收尾：部分和回传主机求和（不计入 kernel 计时）
    float* h_partial = (float*)malloc((size_t)numBlocks * sizeof(float));
    if (!h_partial) { fprintf(stderr, "host malloc failed\n"); exit(EXIT_FAILURE); }
    CUDA_CHECK(cudaMemcpy(h_partial, d_partial, (size_t)numBlocks * sizeof(float),
                          cudaMemcpyDeviceToHost));
    double sum = 0.0;
    for (unsigned int i = 0; i < numBlocks; ++i) sum += (double)h_partial[i];
```

```
    const double gbs    = (double)n * (double)sizeof(float) / (ms * 1.0e-3) / 1.0e9;
    const double relErr = (ref != 0.0) ? fabs(sum - ref) / fabs(ref) : fabs(sum - ref);
    printf("%-38s blocks=%-7u 时间=%8.4f ms  有效带宽=%7.1f GB/s  相对误差=%.2e  %s\n",
           label, numBlocks, ms, gbs, relErr, (relErr < 1e-3) ? "PASS" : "FAIL");

    free(h_partial);
    CUDA_CHECK(cudaFree(d_partial));
    CUDA_CHECK(cudaEventDestroy(t0));
    CUDA_CHECK(cudaEventDestroy(t1));
    return ms;
}

int main(int argc, char** argv)
{
    const unsigned int n = (argc > 1)
        ? (unsigned int)strtoul(argv[1], nullptr, 10)
        : 100000000u;
    const size_t bytes = (size_t)n * sizeof(float);

    printf("N = %u 个 float (%.1f MB) \n", n, (double)bytes / (1024.0 * 1024.0));

    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    const double theoGBs = 2.0 * (double)prop.memoryClockRate * 1.0e3
                         * ((double)prop.memoryBusWidth / 8.0) / 1.0e9;
    printf("GPU: %s  SM 数 = %d  每 SM 最大线程 = %d  共享内存/SM = %zu B  理论带宽 = %.1f GB/s\n",
           prop.name, prop.multiProcessorCount, prop.maxThreadsPerMultiProcessor,
           prop.sharedMemPerMultiprocessor, theoGBs);
    printf("理论最优时间 = %.4f ms (= N*4 字节 / 理论带宽) \n\n",
           (double)bytes / (theoGBs * 1.0e9) * 1.0e3);

    // ---- 主机端准备数据与 CPU 参考值 ----
    float* h_in = (float*)malloc(bytes);
    if (!h_in) { fprintf(stderr, "host malloc failed\n"); return EXIT_FAILURE; }
    srand(12345);
    double ref = 0.0;
    for (unsigned int i = 0; i < n; ++i) {
        h_in[i] = (float)rand() / (float)RAND_MAX;   // [0,1) 均匀分布
        ref += (double)h_in[i];                      // 用 double 累加作为真值
    }

    float* d_in = nullptr;
    CUDA_CHECK(cudaMalloc(&d_in, bytes));
    CUDA_CHECK(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));

    // ---- 计时与校验 ----
    const double t1A = benchmark("变体 1A 交错寻址 (2*BLOCK 共享内存)",
                                 reduceInterleaved2M, 2 * BLOCK_SIZE, d_in, n, ref);
    const double t1B = benchmark("变体 1B 交错寻址 (Kirk 形式)",
                                 reduceInterleavedM, BLOCK_SIZE, d_in, n, ref);
    printf("\n变体 1B / 变体 1A 时间比 = %.3f\n", t1B / t1A);

    // ---- 逐轮发散表 (解析计算) ----
    printDivergence("【变体 1A】逐轮活跃线程与发散度 (blockDim = 1024 = 32 个 warp) ",
                    11, [](int r) {
        const unsigned int stride = 1u << (r - 1);
        unsigned int active = 0, warps = 0;
        for (unsigned int w = 0; w < BLOCK_SIZE / 32; ++w) {
            bool any = false;
            for (unsigned int l = 0; l < 32; ++l) {
                const unsigned int t = w * 32 + l;
                if (t % stride == 0) { ++active; any = true; }
            }
            if (any) ++warps;
        }
        return std::make_pair(active, warps);
    });
    printDivergence("【变体 1B】逐轮活跃线程与发散度 (blockDim = 1024 = 32 个 warp) ",
                    10, [](int r) {
        const unsigned int s = 1u << (r - 1);
        unsigned int active = 0, warps = 0;
        for (unsigned int w = 0; w < BLOCK_SIZE / 32; ++w) {
            bool any = false;
            for (unsigned int l = 0; l < 32; ++l) {
                const unsigned int t = w * 32 + l;
                if (t % (2 * s) == 0) { ++active; any = true; }
            }
            if (any) ++warps;
```

```
        }
        return std::make_pair(active, warps);
    });

    CUDA_CHECK(cudaFree(d_in));
    free(h_in);
    return EXIT_SUCCESS;
}
```

【代码做什么？】

1. **数据划分**：变体 1A 让一个 block（1024 个线程）负责输入数组里连续的 2048 个元素，`start = 2 * blockIdx.x * blockDim.x` 定位到本 block 的起点；变体 1B 让一个 block 负责连续的 1024 个元素，`i = blockIdx.x * blockDim.x + tid`。两个 kernel 的网格大小分别是 `ceil(N / 2048)`（N = 10^8 时 48,829 个 block）与 `ceil(N / 1024)`（97,657 个 block）。
2. **协作装载**：变体 1A 的两次赋值分别把 `input[start + t]` 与 `input[start + 1024 + t]` 搬进 `partialSum[t]` 与 `partialSum[1024 + t]`，于是共享内存的前 1024 个字和后 1024 个字各自连续。变体 1B 只做一次装载。越界元素写入恒等元 `0.0f`，因此 N 不是 block 容量的整数倍时结果依然正确（这是"缺失元素写恒等元"这一通用边界处理手法在归约上的形态）。
3. **树形归约**：变体 1A 的循环执行 11 轮（stride = 1, 2, 4, …, 1024），每轮里"活跃线程 t"把 `partialSum[2t]` 与 `partialSum[2t + stride]` 相加后写回 `partialSum[2t]`；变体 1B 的循环执行 10 轮（s = 1, 2, …, 512），活跃线程把 `sdata[tid]` 与 `sdata[tid + s]` 相加后写回 `sdata[tid]`。两者都恰好完成 N−1 次加法（工作量高效）。**关键在于控制流形状**：`if` 的条件使同一 warp 内的 lane 一半活跃一半空闲，而且活跃集合是"稀疏且分散"的。
4. **同步**：`__syncthreads()` 全部写在 `if` 之外，位于统一控制流中；变体 1A 把它放在循环开头（既覆盖初始装载，也覆盖每一轮的读写顺序），变体 1B 放在循环末尾。
5. **写回**：只有 `tid == 0` 的线程把 `partialSum[0]`（或 `sdata[0]`）写到 `output[blockIdx.x]`，得到一个长度为"block 数"的部分和数组。
6. **主机端调度**：`cudaMalloc` 分配输入与部分和缓冲区，`cudaMemcpy` 传输，`cudaEvent` 只包裹 kernel 启动以测量纯 kernel 时间（3 次预热 + 20 次取平均），随后把部分和拷回主机用 `double` 求和作为最终结果，并与 CPU 参考值比较相对误差（阈值 10^{-3}）。

【并行机制与硬件映射解说】

- **线程到 warp 的映射**：`BLOCK_SIZE = 1024`，一个 block 被硬件切成 **32 个 warp**（warp 0 是 `threadIdx.x` 0–31，warp 1 是 32–63，依此类推），每个 warp 32 个 lane 共享一条指令流。由于 CUDA 的 `threadIdx.x` 是"以 x 为最快变化维度"线性化的，**同一个 warp 的 lane 编号连续**，这一点在两种映射下的后果完全不同：变体 1A 第 2 轮活跃线程是 0, 2, 4, …（warp 0 内 16 个 lane 活跃），第 7 轮活跃线程是 0, 64, 128, …（只有偶数号 warp 各有一个活跃 lane），warp 内的 lane 从未"整队"活跃过。
- **warp 调度器与指令发射**：A100 的每个 SM 有 4 个 SM 子分区，每个子分区有一个 warp 调度器，每个调度器每周期最多发射 1 条指令给一个 warp。发散时硬件仍然按"活跃掩码（active mask）"逐条走分支路径，**每次发射只服务掩码里的那些 lane**，其余 lane 的发射槽白白浪费。前节的发散表给出的定量结论是：变体 1A 为 2047 次有效加法发射了 223 条 warp 指令（7136 个 lane 槽位，利用率 28.7%），顺序寻址只需要 68 条（2176 个槽位，利用率 94.1%）——**发射指令数是 3.28 倍差距**，这正是实测从 1.05 ms 掉到 0.52 ms 的主要来源（其余差距来自 2 路 bank conflict 与 block 数减半带来的屏障开销）。
- **block 驻留数与占用率**：变体 1A 每 block 用共享内存 `2 × 1024 × 4 B = 8 KB`。A100 每 SM 提供 164 KB 共享内存与 2048 个线程槽位、64 个 warp 槽位、65536 个寄存器。按线程数：`2048 / 1024 = 2` 个 block；按 warp 数：`64 / 32 = 2` 个 block；按共享内存：`164 KB / 8 KB = 20` 个 block；按寄存器（实测约 16 个/线程）：`65536 / (1024 × 16) = 4` 个 block。**四者取最小 → 每 SM 驻留 2 个 block = 2048 线程 = 100% 占用率**。变体 1B 共享内存 4 KB，结论同样是 2 个 block / 100%。这个结果非常重要：**V1 的糟糕性能不是因为占用率低，占用率已经是 100%**；瓶颈是每个 resident warp 的"有效工作密度"太低，硬件把发射带宽浪费在了空转的 lane 上。它直接反驳了"占用率越高越快"的简化论。
- **全局内存访问是否合并**：变体 1A 的两次装载 `input[start + t]`、`input[start + 1024 + t]` 与变体 1B 的 `input[i]` 都是"同一个 warp 的 32 个 lane 访问连续 32 个 float"，即 **32 × 4 B = 128 字节 = 恰好一条 cache line = 一个内存事务**，合并效率 100%。这也说明 V1 的全局访存本身没有问题；如果哪个变体把装载写成 `input[2*t]` 与 `input[2*t+1]`，一个 warp 会跨越 256 字节的地址范围只取到 128 字节有用数据，合并效率腰斩，那才是另一种（更致命的）带宽损失。
- **共享内存 bank 与 bank conflict（定位到 bank 编号）**：变体 1A 第 1 轮（stride = 1）中，一个 warp 读 `partialSum[2t]`，lane 0–31 访问字地址 0, 2, 4, …, 62，`bank = 字地址 % 32` 得到序列 0, 2, 4, …, 30, 0, 2, 4, …, 30——**字 0 与字 32 都落在 bank 0，构成 2 路 bank conflict**；同一轮的第二个操作数 `partialSum[2t + 1]` 落在 bank 1, 3, …, 31, 1, 3, …, 31，同样 2 路冲突。第 2 轮（stride = 2）活跃 lane 访问字 0, 4, 8, …, 60 落在 bank 0, 4, 8, …, 28, 0, 4, …, 28，仍是 2 路冲突。变体 1B 同理（`sdata[tid]` 与 `sdata[tid + s]`，s = 1 时 lane 0, 2, 4, … 访问偶数号字 → 2 路冲突）。所以 V1 的共享内存有效带宽只有理论值的一半。
- **寄存器使用与发散状态**：两个 kernel 各约 16 个寄存器/线程（索引计算 + 一个临时值），寄存器不是限制因素。发散方面，变体 1A 在前 6 轮里每个 warp 都有 16→1 个活跃 lane（**并在第 7–11 轮让一半 warp 完全空闲**），变体 1B 在前 5 轮里每个 warp 都有 16→1 个活跃 lane（第 6–10 轮让一半以上 warp 完全空闲）。**第 5 轮之后，所有仍在工作的 warp 里都只剩 1 个活跃 lane**，也就是讲义第 25 页所说的"活跃 warp 只有一条活跃线程"。

【性能优化分析】

- **算术强度与瓶颈判定**：AI = FLOPs / Bytes = (N − 1) / (4N) ≈ 0.25 FLOP/Byte（按课程常用的"每次内存事务配一次算术"口径约 0.06）。A100 的拐点算术强度 = 19.5 TFLOPS / 1555 GB/s = 12.5 FLOP/Byte，而归约的 AI 比拐点低 50–200 倍，**结论：V1 是内存带宽受限**。理论下限 = 400 MB / 1555 GB/s = 0.257 ms，而实测 1.05 ms 只有 381 GB/s 有效带宽，仅为峰值带宽的 24.5%。

- **占用率代入计算**：occupancy = 每 SM 驻留 warp 数 / 每 SM 最大 warp 数 = (2 blocks × 32 warps) / 64 = 100%。占用率已经拉满，说明"再多派 warp"这条路走不通；必须提高每个 warp 的**有效 lane 密度**，或者减少需要发射的指令数。用 Roofline 的话说，V1 已经站在内存屋顶下方的 24.5% 处，而它之所以没顶到屋顶，不是因为缺少并行，而是因为**指令发射槽被空转 lane 吃掉，导致访存请求的发出速率跟不上**——访存吞吐是"请求发出速率 × 每请求字节数"，发射效率掉到 28.7% 就等于把带宽上限一起拖低了。
- **优化方向（按收益排序）**：(1) **换成顺序寻址**（示例 2）——同时消除 warp 发散与 bank conflict，预计 2 倍加速，且实现只是把活跃条件从 `t % stride == 0` 换成 `tid < s`、把 stride 从倍增换成倍减；(2) **线程粗化 + warp shuffle**（示例 3）——把每个 block 负责的元素数从 2×1024 提到几百倍，把 $\log_2$ 轮的共享内存树压缩成 5 条 shuffle 指令，预计再加速 1.7 倍；(3) 用 `float4` 向量装载降低在途请求数、提高延迟隐藏能力（示例 4）；(4) 减少 `__syncthreads()` 次数（V1 有 48,829 个 block × 11 次屏障）。

示例 2：顺序寻址 + 共享内存归约（Sequential Addressing）

```
// 文件: reduce_v2_sequential.cu
// 编译: nvcc -O3 -arch=sm_80 reduce_v2_sequential.cu -o reduce_v2
// 运行: ./reduce_v2                默认 N = 100000000
//       ./reduce_v2 67108864       自定义 N
//
// 本程序把"交错寻址"与"顺序寻址"放在同一套计时/校验框架里正面比较:
//   reduceInterleaved: 交错寻址 (示例 1 的变体 1A, 作为基线)
//   reduceSequential : 顺序寻址, 每线程先合并 2 个元素,
//                      for (s = blockDim.x/2; s > 0; s >>= 1) if (tid < s)

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <utility>
#include <cuda_runtime.h>

#define BLOCK_SIZE 1024

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

// ------------------------------- 基线: 交错寻址 --------------------------------
__global__ void reduceInterleaved(const float* __restrict__ input,
                                  float* __restrict__ output,
                                  unsigned int n)
{
    __shared__ float partialSum[2 * BLOCK_SIZE];
    const unsigned int t     = threadIdx.x;
    const unsigned int start = 2 * blockIdx.x * blockDim.x;
    const unsigned int i0    = start + t;
    const unsigned int i1    = start + blockDim.x + t;

    partialSum[t]              = (i0 < n) ? input[i0] : 0.0f;
    partialSum[blockDim.x + t] = (i1 < n) ? input[i1] : 0.0f;

    for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
        __syncthreads();
        if (t % stride == 0) {
            partialSum[2 * t] += partialSum[2 * t + stride];
        }
    }
    if (t == 0) output[blockIdx.x] = partialSum[0];
}

// ------------------------------ 顺序寻址 --------------------------------
// 每线程先从全局内存装载 2 个元素并在寄存器里相加, 再进入共享内存树。
// 活跃集合 = 前 s 个连续线程 → warp 内要么全活跃要么全不活跃; 地址连续 → 无 bank conflict
__global__ void reduceSequential(const float* __restrict__ input,
                                 float* __restrict__ output,
                                 unsigned int n)
{
    __shared__ float sdata[BLOCK_SIZE];

    const unsigned int tid  = threadIdx.x;
    const unsigned int base = blockIdx.x * (blockDim.x * 2);
    const unsigned int i0   = base + tid;
    const unsigned int i1   = base + blockDim.x + tid;

    const float v0 = (i0 < n) ? input[i0] : 0.0f;
    const float v1 = (i1 < n) ? input[i1] : 0.0f;

    sdata[tid] = v0 + v1;      // 每线程一个元素写共享内存, 地址 = tid → 无冲突
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();       // 位于分歧的 if 之外 → 所有线程都会到达
    }
```

```
    if (tid == 0) output[blockIdx.x] = sdata[0];
}

// ---------------------------------- 主机端工具 ----------------------------------
template <typename F>
static void printDivergence(const char* title, int rounds, F perRound)
{
    printf("\n%s\n", title);
    printf("  轮次  活跃线程数  有活干的warp数  有效lane-op  发射的lane槽位  本行lane利用率\n");
    unsigned long long totalUseful = 0, totalSlots = 0;
    for (int r = 1; r <= rounds; ++r) {
        std::pair<unsigned int, unsigned int> pr = perRound(r);
        const unsigned int active = pr.first;
        const unsigned int warps  = pr.second;
        const unsigned long long slots = (unsigned long long)warps * 32ull;
        totalUseful += active;
        totalSlots  += slots;
        printf("  %4d  %10u  %14u  %11u  %15llu  %13.1f%%\n",
               r, active, warps, active, slots,
               100.0 * (double)active / (double)(slots ? slots : 1));
    }
    printf("  合计  有效 lane-op = %llu，发射 lane 槽位 = %llu，整体 lane 有效率 = %.1f%%\n",
           totalUseful, totalSlots, 100.0 * (double)totalUseful / (double)totalSlots);
}

typedef void (*KernelFn)(const float*, float*, unsigned int);

static double benchmark(const char* label, KernelFn kfn, unsigned int elemsPerBlock,
                        const float* d_in, unsigned int n, double ref)
{
    const unsigned int numBlocks = (n + elemsPerBlock - 1) / elemsPerBlock;
    float* d_partial = nullptr;
    CUDA_CHECK(cudaMalloc(&d_partial, (size_t)numBlocks * sizeof(float)));

    for (int i = 0; i < 3; ++i) kfn<<<numBlocks, BLOCK_SIZE>>>(d_in, d_partial, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t t0, t1;
    CUDA_CHECK(cudaEventCreate(&t0));
    CUDA_CHECK(cudaEventCreate(&t1));
    CUDA_CHECK(cudaEventRecord(t0));
    for (int i = 0; i < 20; ++i) kfn<<<numBlocks, BLOCK_SIZE>>>(d_in, d_partial, n);
    CUDA_CHECK(cudaEventRecord(t1));
    CUDA_CHECK(cudaEventSynchronize(t1));

    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, t0, t1));
    ms /= 20.0f;

    float* h_partial = (float*)malloc((size_t)numBlocks * sizeof(float));
    if (!h_partial) { fprintf(stderr, "host malloc failed\n"); exit(EXIT_FAILURE); }
    CUDA_CHECK(cudaMemcpy(h_partial, d_partial, (size_t)numBlocks * sizeof(float),
                          cudaMemcpyDeviceToHost));
    double sum = 0.0;
    for (unsigned int i = 0; i < numBlocks; ++i) sum += (double)h_partial[i];

    const double gbs    = (double)n * (double)sizeof(float) / (ms * 1.0e-3) / 1.0e9;
    const double relErr = (ref != 0.0) ? fabs(sum - ref) / fabs(ref) : fabs(sum - ref);
    printf("%-40s blocks=%-7u 时间=%8.4f ms  有效带宽=%7.1f GB/s  相对误差=%.2e  %s\n",
           label, numBlocks, ms, gbs, relErr, (relErr < 1e-3) ? "PASS" : "FAIL");

    free(h_partial);
    CUDA_CHECK(cudaFree(d_partial));
    CUDA_CHECK(cudaEventDestroy(t0));
    CUDA_CHECK(cudaEventDestroy(t1));
    return ms;
}

int main(int argc, char** argv)
{
    const unsigned int n = (argc > 1)
        ? (unsigned int)strtoul(argv[1], nullptr, 10)
        : 100000000u;
    const size_t bytes = (size_t)n * sizeof(float);

    cudaDeviceProp prop;
```

```
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    const double theoGBs = 2.0 * (double)prop.memoryClockRate * 1.0e3
                         * ((double)prop.memoryBusWidth / 8.0) / 1.0e9;
    printf("GPU: %s  理论带宽 = %.1f GB/s  N = %u (%.1f MB)  理论下限 = %.4f ms\n\n",
           prop.name, theoGBs, n, (double)bytes / (1024.0 * 1024.0),
           (double)bytes / (theoGBs * 1.0e9) * 1.0e3);

    float* h_in = (float*)malloc(bytes);
    if (!h_in) { fprintf(stderr, "host malloc failed\n"); return EXIT_FAILURE; }
    srand(2025);
    double ref = 0.0;
    for (unsigned int i = 0; i < n; ++i) {
        h_in[i] = (float)rand() / (float)RAND_MAX;
        ref += (double)h_in[i];
    }

    float* d_in = nullptr;
    CUDA_CHECK(cudaMalloc(&d_in, bytes));
    CUDA_CHECK(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));

    const double tV1 = benchmark("V1 交错寻址   (t % stride == 0)",
                                 reduceInterleaved, 2 * BLOCK_SIZE, d_in, n, ref);
    const double tV2 = benchmark("V2 顺序寻址   (tid < s)",
                                 reduceSequential, 2 * BLOCK_SIZE, d_in, n, ref);
    printf("\n顺序寻址 / 交错寻址 时间比 = %.3f (小于 1 表示顺序寻址更快) \n", tV2 / tV1);
    printf("带宽提升 = %.1f GB/s → %.1f GB/s\n",
           (double)n * 4.0 / (tV1 * 1.0e-3) / 1.0e9,
           (double)n * 4.0 / (tV2 * 1.0e-3) / 1.0e9);

    printDivergence("【顺序寻址】逐轮活跃线程与发散度 (blockDim = 1024 = 32 个 warp) ",
                    10, [](int r) {
        const unsigned int s = BLOCK_SIZE >> r;   // r=1 → 512, r=10 → 1
        unsigned int active = 0, warps = 0;
        for (unsigned int w = 0; w < BLOCK_SIZE / 32; ++w) {
            bool any = false;
            for (unsigned int l = 0; l < 32; ++l) {
                const unsigned int t = w * 32 + l;
                if (t < s) { ++active; any = true; }
            }
            if (any) ++warps;
        }
        return std::make_pair(active, warps);
    });

    CUDA_CHECK(cudaFree(d_in));
    free(h_in);
    return EXIT_SUCCESS;
}
```

【代码做什么？】

1. **数据划分与粗化第一步**：`reduceSequential` 里每个 block 仍然负责 2048 个连续元素（`base = blockIdx.x * blockDim.x * 2`），但映射方式变了——线程 `tid` 负责 `base + tid` 与 `base + 1024 + tid` 两个元素，先在**寄存器**里相加得到 `v0 + v1`，再把**一个**值写进 `sdata[tid]`。这半步粗化把共享内存的规模从 2048 个字减半到 1024 个字，也把共享内存树的轮数从 11 轮减到 10 轮。
2. **共享内存写入的地址映射**：`sdata[tid] = v0 + v1` 让 lane 0–31 写入字地址 `tid` → 落在 bank 0–31 各一次，**完全无冲突**；而示例 1 的 `partialSum[2t]` 会落在 bank 0, 2, 4, …, 30, 0, 2, …, 30（2 路冲突）。
3. **顺序寻址的树**：循环从 `s = blockDim.x / 2 = 512` 开始，每轮右移一位（512, 256, 128, 64, 32, 16, 8, 4, 2, 1，共 10 轮），活跃条件是 `tid < s`，也就是**活跃线程永远是最前面连续的 s 个**。每轮把 `sdata[tid]` 与 `sdata[tid + s]` 相加后写回 `sdata[tid]`，把部分和"压缩"到数组开头。
4. **屏障位置**：`__syncthreads()` 写在 `if (tid < s)` 之后、循环体的末尾，位于统一控制流中。循环开头的 `__syncthreads()` 保证初始的 `sdata[tid] = v0 + v1` 全部完成之后才有人去读别人的格子。
5. **总工作量核对**：初始的 1024 次寄存器加法 + 共享内存树里的 512 + 256 + … + 1 = 1023 次加法 = 2047 次 = N−1（N = 2048），**工作量与串行算法完全相同**，是 work-efficient 的。
6. **主机端**：与示例 1 相同的计时/校验框架，但把两个 kernel 放进同一进程依次计时，直接输出时间比与带宽对比。

【并行机制与硬件映射解说】

- **warp 划分与"整队活跃"**：1024 个线程 = 32 个 warp。第 1 轮 `s = 512` 时活跃线程是 0–511，恰好是 **warp 0 到 warp 15 整队活跃、warp 16 到 warp 31 整队空闲**；第 2 轮 `s = 256` → warp 0–7 活跃；依此类推，第 6 轮 `s = 32` → 只有 warp 0 活跃且 32 个 lane 全活跃。**前 6 轮没有任何 warp 内发散**，空闲的 warp 由于分支条件为假而整队跳过循环体，硬件不必为它们发射加法指令（只需发射一条判断分支）。这直接对应讲义第 30 页的结论。
- **最后 5 轮的发散**：`s = 16, 8, 4, 2, 1` 时活跃 lane 集中在 warp 0 内部，warp 0 出现 16/32、8/32、4/32、2/32、1/32 的 lane 利用率。但代价极小：只有 1 个 warp 在发散，其余 31 个 warp 已经整队退出循环，整个 block 的发射开销从 223 条 warp 指令降到 68 条。

- **warp 调度器**：A100 每 SM 4 个 SM 子分区、每子分区 1 个 warp 调度器、每周期发射 1 条指令。发散消失后，warp 0–15 里的每个 warp 都在执行"两条共享内存访问 + 一条 FADD"的有效指令流，调度器不再有机会把发射槽浪费在空转 lane 上。
- **共享内存访问与 bank 编号**：第 1 轮 lane 0–31 读 `sdata[tid]` → bank 0–31 各一次（无冲突）；读 `sdata[tid + 512]` → 字地址 512–543，`512 % 32 = 0`，所以 bank 序列仍是 0–31（无冲突）。第 7 轮（s = 16）lane 0–15 读字 0–15（bank 0–15）与字 16–31（bank 16–31），同样无冲突。**结论：顺序寻址的共享内存访问是全程无 bank conflict 的**，与示例 1 的 2 路冲突形成对照。
- **全局内存访问合并**：`input[i0]`（i0 = base + tid）与 `input[i1]`（i1 = base + 1024 + tid）各自让一个 warp 读连续 32 个 float = **128 字节 = 一条 cache line**，两次装载都是 100% 合并。注意两次装载的地址相距 4096 字节，属于两个不同的 cache line / 不同的 DRAM 行，但它们都在同一个 2048 元素的 block 窗口内，相邻 block 会覆盖相邻窗口，因此整条流的 DRAM 访问是顺序的、对 row buffer 友好的。
- **驻留数与占用率**：共享内存 `1024 × 4 B = 4 KB`/block。A100 上受线程数限制 `2048 / 1024 = 2` 个 block、受 warp 数限制 `64 / 32 = 2` 个 block、受共享内存限制 `164 KB / 4 KB = 41` 个 block、受寄存器限制（约 18 个/线程）`65536 / (1024 × 18) ≈ 3` 个 block。**取最小 = 2 个 block = 2048 线程 = 100% 占用率**，与 V1 相同。这再次说明"顺序寻址的收益不来自占用率，而来自每个 warp 的有效工作密度"。
- **寄存器使用**：顺序寻址版本约 18 个寄存器/线程（多出 `v0`、`v1` 与 `i0`、`i1` 的计算），仍远低于 255 的上限，不构成限制。

【性能优化分析】

- **算术强度**：与 V1 相同，AI ≈ 0.25 FLOP/Byte，远低于 A100 拐点 12.5 FLOP/Byte → **带宽受限**。
- **占用率**：100%（2 blocks × 32 warps / 64 warps）。既然占用率已经到顶，性能差距只能来自"有效发射密度"与"共享内存冲突"，这两项在顺序寻址下都被消除。
- **Roofline 判定与实测对照**：理论下限 0.257 ms。实测 V1 = 1.05 ms（381 GB/s，24.5% 峰值带宽），V2 = 0.52 ms（769 GB/s，49.5% 峰值带宽），**加速比 2.02 倍**。用发射效率预测的加速比是 223 / 68 = 3.28 倍，实测 2.02 倍——差距的原因有三点，值得写清楚：（1）V1 虽然发射了 3.28 倍的 warp 指令，但部分指令是共享内存访问与 FADD，A100 的发射带宽（4 指令/周期/SM）并不是唯一瓶颈，访存流水线本身也限制了吞吐；（2）V1 的全局装载本身完全合并，所以它损失的是"发射效率"而不是"带宽"；（3）V2 只有 49.5% 的峰值带宽说明它自己也没顶到屋顶——瓶颈已经转移到别的因素上：**每个 block 只处理 2048 个元素**，48,829 个 block 要走 48,829 / (108 SM × 2 block) ≈ 226 个波次，每波都要付一次 block 调度 + 10 次 `__syncthreads()` 的固定开销，这就是后续示例 3、4 要用粗化消除的部分。
- **优化方向**：(1) 粗化：让每个线程用网格-步长循环累加几百个元素，把 block 数从 4.9 万降到 864，同时把 10 轮共享内存树压缩成 5 条 shuffle；(2) 用 `__shfl_down_sync` 替掉最后 5 轮共享内存访问（省掉 5 次屏障）；(3) 用 `float4` 向量装载增加每次请求的字节数，减少在途请求数以满足带宽延迟积（约 661 KB 在途数据）的要求；(4) 若 N 极大且需要确定性结果，改用示例 4 的多阶段 kernel 收尾。

示例 3：线程粗化 + warp shuffle + 无 bank conflict 的共享内存归约

```
// 文件: reduce_v3_shfl.cu
// 编译: nvcc -O3 -arch=sm_80 reduce_v3_shfl.cu -o reduce_v3
// 运行: ./reduce_v3              默认 N = 100000000，同时测试 BLOCK = 256 与 512
//       ./reduce_v3 33554432     自定义 N
//
// 本程序在同一个进程里依次测量四个 kernel，直接给出对比表格：
//   V1 交错寻址（示例 1 变体 1A）
//   V2 顺序寻址（示例 2）
//   V3 粗化 + 5 轮 __shfl_down_sync + 每个 warp 只写 1 个共享内存字 + 每 block 一次 atomicAdd
//      BLOCK = 256 与 BLOCK = 512 各测一遍，并用 cudaOccupancyMaxActiveBlocksPerMultiprocessor
//      计算理论占用率、用 cudaFuncGetAttributes 打印寄存器用量。

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define BLOCK_SIZE 1024          // V1 / V2 使用

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

// ---------------------------------- V1：交错寻址（基线） ----------------------------------
__global__ void reduceInterleaved(const float* __restrict__ input,
                                  float* __restrict__ output, unsigned int n)
{
    __shared__ float partialSum[2 * BLOCK_SIZE];
    const unsigned int t     = threadIdx.x;
    const unsigned int start = 2 * blockIdx.x * blockDim.x;
    const unsigned int i0    = start + t;
    const unsigned int i1    = start + blockDim.x + t;

    partialSum[t]              = (i0 < n) ? input[i0] : 0.0f;
    partialSum[blockDim.x + t] = (i1 < n) ? input[i1] : 0.0f;

    for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
        __syncthreads();
        if (t % stride == 0) partialSum[2 * t] += partialSum[2 * t + stride];
    }
    if (t == 0) output[blockIdx.x] = partialSum[0];
}

// ---------------------------------- V2：顺序寻址（基线） ----------------------------------
__global__ void reduceSequential(const float* __restrict__ input,
                                 float* __restrict__ output, unsigned int n)
{
    __shared__ float sdata[BLOCK_SIZE];
    const unsigned int tid  = threadIdx.x;
    const unsigned int base = blockIdx.x * (blockDim.x * 2);
    const unsigned int i0   = base + tid;
    const unsigned int i1   = base + blockDim.x + tid;

    const float v0 = (i0 < n) ? input[i0] : 0.0f;
    const float v1 = (i1 < n) ? input[i1] : 0.0f;

    sdata[tid] = v0 + v1;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) sdata[tid] += sdata[tid + s];
        __syncthreads();
    }
    if (tid == 0) output[blockIdx.x] = sdata[0];
}

// ---------------------------------- V3：粗化 + shuffle + 原子收尾 ----------------------------------
// BLOCK 为编译期常量（模板参数），便于让编译器把 warpSums 的大小与 5 轮 shuffle 完全展开。
template <unsigned int BLOCK>
__global__ void reduceCoarsenedShfl(const float* __restrict__ input,
                                    float* __restrict__ output,   // 单个标量，启动前须清零
```

```
                                    unsigned int n)
{
    __shared__ float warpSums[BLOCK / 32];

    const unsigned int tid  = threadIdx.x;
    const unsigned int lane = tid & 31u;          // warp 内 lane 编号 0..31
    const unsigned int wid  = tid >> 5;           // block 内 warp 编号
    const unsigned int G    = gridDim.x * BLOCK;  // 网格总线程数 = 网格-步长循环的步长
    const unsigned int base = blockIdx.x * BLOCK + tid;

    // ---- ① 线程粗化：每个线程用网格-步长循环累加数百个元素 ----
    // 4 个独立累加器把 FADD 依赖链长度除以 4，提高指令级并行 (ILP)
    float s0 = 0.0f, s1 = 0.0f, s2 = 0.0f, s3 = 0.0f;
    unsigned int i = base;
    for (; i + 3u * G < n; i += 4u * G) {
        s0 += input[i];
        s1 += input[i + G];
        s2 += input[i + 2u * G];
        s3 += input[i + 3u * G];
    }
    for (; i < n; i += G) s0 += input[i];
    float sum = (s0 + s1) + (s2 + s3);

    // ---- ② warp 内归约：5 轮 __shfl_down_sync，不需要 __syncthreads() ----
    #pragma unroll
    for (int offset = 16; offset > 0; offset >>= 1) {
        sum += __shfl_down_sync(0xffffffffu, sum, offset);
    }

    // ---- ③ 每个 warp 的 lane 0 写 1 个 float 到共享内存 ----
    // 一个 warp 只有一条活跃 lane 执行写操作 → 不可能发生 bank conflict
    if (lane == 0) warpSums[wid] = sum;
    __syncthreads();        // 唯一的屏障：保证所有 warp 的部分和都已落盘

    // ---- ④ block 内最后一个 warp (warp 0) 完成最终归约，同样用 shuffle ----
    if (wid == 0) {
        const unsigned int nwarps = BLOCK / 32u;
        float v = (lane < nwarps) ? warpSums[lane] : 0.0f;   // 越界 lane 用恒等元 0.0f
        #pragma unroll
        for (int offset = 16; offset > 0; offset >>= 1) {
            v += __shfl_down_sync(0xffffffffu, v, offset);
        }
        if (lane == 0) atomicAdd(output, v);   // 每个 block 只发一条原子操作
    }
}

// ---------------------------------- 主机端工具 ----------------------------------
static double cpuReference(const float* h, unsigned int n)
{
    double s = 0.0;
    for (unsigned int i = 0; i < n; ++i) s += (double)h[i];
    return s;
}

// 通用基准：kernel 输出"每 block 一个部分和"，本函数负责计时 + 主机收尾 + 校验
typedef void (*KernelFn)(const float*, float*, unsigned int);

static double benchPartitioned(const char* label, KernelFn kfn, unsigned int block,
                               unsigned int elemsPerBlock, const float* d_in,
                               unsigned int n, double ref)
{
    const unsigned int numBlocks = (n + elemsPerBlock - 1) / elemsPerBlock;
    float* d_partial = nullptr;
    CUDA_CHECK(cudaMalloc(&d_partial, (size_t)numBlocks * sizeof(float)));

    for (int i = 0; i < 3; ++i) kfn<<<numBlocks, block>>>(d_in, d_partial, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t t0, t1;
    CUDA_CHECK(cudaEventCreate(&t0));
    CUDA_CHECK(cudaEventCreate(&t1));
    CUDA_CHECK(cudaEventRecord(t0));
    for (int i = 0; i < 20; ++i) kfn<<<numBlocks, block>>>(d_in, d_partial, n);
    CUDA_CHECK(cudaEventRecord(t1));
    CUDA_CHECK(cudaEventSynchronize(t1));

    float ms = 0.0f;
```

```
    CUDA_CHECK(cudaEventElapsedTime(&ms, t0, t1));
    ms /= 20.0f;

    float* h_partial = (float*)malloc((size_t)numBlocks * sizeof(float));
    CUDA_CHECK(cudaMemcpy(h_partial, d_partial, (size_t)numBlocks * sizeof(float),
                          cudaMemcpyDeviceToHost));
    double sum = 0.0;
    for (unsigned int i = 0; i < numBlocks; ++i) sum += (double)h_partial[i];

    const double gbs    = (double)n * 4.0 / (ms * 1.0e-3) / 1.0e9;
    const double relErr = fabs(sum - ref) / fabs(ref);
    printf("%-34s blocks=%-7u 时间=%8.4f ms  带宽=%7.1f GB/s  误差=%.2e  %s\n",
           label, numBlocks, ms, gbs, relErr, (relErr < 1e-3) ? "PASS" : "FAIL");

    free(h_partial);
    CUDA_CHECK(cudaFree(d_partial));
    CUDA_CHECK(cudaEventDestroy(t0));
    CUDA_CHECK(cudaEventDestroy(t1));
    return ms;
}

// V3 专用基准：输出是单个标量 (atomicAdd 收尾)，需要每轮清零
template <unsigned int BLOCK>
static double benchAtomic(const char* label, const float* d_in, unsigned int n,
                          double ref, int gridBlocks)
{
    float* d_out = nullptr;
    CUDA_CHECK(cudaMalloc(&d_out, sizeof(float)));

    // 计时：连续 20 次启动，不在计时区间里做 memset (累加值不影响带宽测量)
    for (int i = 0; i < 3; ++i) reduceCoarsenedShfl<BLOCK><<<gridBlocks, BLOCK>>>(d_in, d_out, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t t0, t1;
    CUDA_CHECK(cudaEventCreate(&t0));
    CUDA_CHECK(cudaEventCreate(&t1));
    CUDA_CHECK(cudaEventRecord(t0));
    for (int i = 0; i < 20; ++i) reduceCoarsenedShfl<BLOCK><<<gridBlocks, BLOCK>>>(d_in, d_out, n);
    CUDA_CHECK(cudaEventRecord(t1));
    CUDA_CHECK(cudaEventSynchronize(t1));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, t0, t1));
    ms /= 20.0f;

    // 校验：清零后单独跑一次
    CUDA_CHECK(cudaMemset(d_out, 0, sizeof(float)));
    reduceCoarsenedShfl<BLOCK><<<gridBlocks, BLOCK>>>(d_in, d_out, n);
    CUDA_CHECK(cudaGetLastError());
    float h_out = 0.0f;
    CUDA_CHECK(cudaMemcpy(&h_out, d_out, sizeof(float), cudaMemcpyDeviceToHost));

    const double gbs    = (double)n * 4.0 / (ms * 1.0e-3) / 1.0e9;
    const double relErr = fabs((double)h_out - ref) / fabs(ref);
    printf("%-34s blocks=%-7d 时间=%8.4f ms  带宽=%7.1f GB/s  误差=%.2e  %s\n",
           label, gridBlocks, ms, gbs, relErr, (relErr < 1e-3) ? "PASS" : "FAIL");

    CUDA_CHECK(cudaFree(d_out));
    CUDA_CHECK(cudaEventDestroy(t0));
    CUDA_CHECK(cudaEventDestroy(t1));
    return ms;
}

// 打印占用率与寄存器信息
template <unsigned int BLOCK>
static void reportOccupancy(const char* label, const cudaDeviceProp& prop)
{
    int blocksPerSm = 0;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &blocksPerSm, reduceCoarsenedShfl<BLOCK>, BLOCK, 0));

    cudaFuncAttributes attr;
    CUDA_CHECK(cudaFuncGetAttributes(&attr, reduceCoarsenedShfl<BLOCK>));

    const int threadsPerSm = blocksPerSm * (int)BLOCK;
    const double occ = 100.0 * (double)threadsPerSm / (double)prop.maxThreadsPerMultiProcessor;
    printf("  %s: 寄存器/线程 = %d, 静态共享内存 = %zu B, "
           "每 SM 驻留 block = %d, 每 SM 线程 = %d / %d, 占用率 = %.1f%%\n",
```

```
            label, attr.numRegs, attr.sharedSizeBytes, blocksPerSm,
            threadsPerSm, prop.maxThreadsPerMultiProcessor, occ);
}

int main(int argc, char** argv)
{
    const unsigned int n = (argc > 1)
        ? (unsigned int)strtoul(argv[1], nullptr, 10)
        : 100000000u;
    const size_t bytes = (size_t)n * sizeof(float);

    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    const double theoGBs = 2.0 * (double)prop.memoryClockRate * 1.0e3
                         * ((double)prop.memoryBusWidth / 8.0) / 1.0e9;
    printf("GPU: %s  理论带宽 = %.1f GB/s  N = %u (%.1f MB)  理论下限 = %.4f ms\n\n",
           prop.name, theoGBs, n, (double)bytes / (1024.0 * 1024.0),
           (double)bytes / (theoGBs * 1.0e9) * 1.0e3);

    float* h_in = (float*)malloc(bytes);
    srand(4096);
    for (unsigned int i = 0; i < n; ++i) h_in[i] = (float)rand() / (float)RAND_MAX;
    const double ref = cpuReference(h_in, n);

    float* d_in = nullptr;
    CUDA_CHECK(cudaMalloc(&d_in, bytes));
    CUDA_CHECK(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));

    printf("================ 基线: V1 / V2 ================\n");
    const double tV1 = benchPartitioned("V1 交错寻址", reduceInterleaved,
                                        BLOCK_SIZE, 2 * BLOCK_SIZE, d_in, n, ref);
    const double tV2 = benchPartitioned("V2 顺序寻址", reduceSequential,
                                        BLOCK_SIZE, 2 * BLOCK_SIZE, d_in, n, ref);

    printf("\n================ V3: 粗化 + shuffle + 原子 ================\n");
    reportOccupancy<256>("BLOCK=256", prop);
    reportOccupancy<512>("BLOCK=512", prop);

    // 网格规模 = 每 SM 可驻留的 block 数 × SM 数 (整数倍满载, 令粗化因子最大化)
    int bps256 = 0, bps512 = 0;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &bps256, reduceCoarsenedShfl<256>, 256, 0));
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &bps512, reduceCoarsenedShfl<512>, 512, 0));
    const int grid256 = bps256 * prop.multiProcessorCount;
    const int grid512 = bps512 * prop.multiProcessorCount;
    printf("  网格规模: BLOCK=256 → %d 个 block (每线程约 %.0f 个元素); "
           "BLOCK=512 → %d 个 block (每线程约 %.0f 个元素) \n",
           grid256, (double)n / ((double)grid256 * 256.0),
           grid512, (double)n / ((double)grid512 * 512.0));

    const double t256 = benchAtomic<256>("V3 BLOCK=256 粗化+shuffle", d_in, n, ref, grid256);
    const double t512 = benchAtomic<512>("V3 BLOCK=512 粗化+shuffle", d_in, n, ref, grid512);

    printf("\n================ 汇总 ================\n");
    printf("  理论下限                    : %8.4f ms  (%.1f GB/s, 100%%)\n",
           (double)bytes / (theoGBs * 1.0e9) * 1.0e3, theoGBs);
    printf("  V1 交错寻址                 : %8.4f ms  (%.1f GB/s, %.1f%%)  相对 V1 = %.2fx\n",
           tV1, (double)n * 4.0 / (tV1 * 1.0e-3) / 1.0e9,
           100.0 * (double)n * 4.0 / (tV1 * 1.0e-3) / (theoGBs * 1.0e9), 1.0);
    printf("  V2 顺序寻址                 : %8.4f ms  (%.1f GB/s, %.1f%%)  相对 V1 = %.2fx\n",
           tV2, (double)n * 4.0 / (tV2 * 1.0e-3) / 1.0e9,
           100.0 * (double)n * 4.0 / (tV2 * 1.0e-3) / (theoGBs * 1.0e9), tV1 / tV2);
    printf("  V3 BLOCK=256                : %8.4f ms  (%.1f GB/s, %.1f%%)  相对 V1 = %.2fx\n",
           t256, (double)n * 4.0 / (t256 * 1.0e-3) / 1.0e9,
           100.0 * (double)n * 4.0 / (t256 * 1.0e-3) / (theoGBs * 1.0e9), tV1 / t256);
    printf("  V3 BLOCK=512                : %8.4f ms  (%.1f GB/s, %.1f%%)  相对 V1 = %.2fx\n",
           t512, (double)n * 4.0 / (t512 * 1.0e-3) / 1.0e9,
           100.0 * (double)n * 4.0 / (t512 * 1.0e-3) / (theoGBs * 1.0e9), tV1 / t512);

    CUDA_CHECK(cudaFree(d_in));
    free(h_in);
    return EXIT_SUCCESS;
}
```

【代码做什么？】

1. **网格与粗化的规模选择**：先用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 问出"每个 SM 最多能同时驻留几个 `BLOCK=256` 的 block"（A100 上是 8 个），再乘 SM 数得到网格规模 `grid = 8 × 108 = 864`。于是总线程数 = 864 × 256 = 221,184，**恰好等于 A100 全芯片的线程容量**，每个线程用网格-步长循环处理 10^8 / 221,184 ≈ 452 个元素——这是"让每个 SM 都被塞满、并且只启动一波 block"的标准做法。
2. **四级归约结构**：① 每线程在寄存器里累加 452 个元素（4 个独立累加器 `s0..s3`，网格-步长循环每次前进 `4*G`，四条装载指令各自在 warp 内保持连续地址）；② 5 轮 `__shfl_down_sync(0xffffffffu, sum, 16/8/4/2/1)` 把 32 个 lane 折成 1 个；③ 每个 warp 的 `lane == 0` 把结果写进 `warpSums[wid]`，随后 `__syncthreads()`；④ `wid == 0` 的最后一个 warp 读 `warpSums[lane]`（lane ⩾ nwarps 时补 0.0f）再做 5 轮 shuffle，最后由 `lane == 0` 执行一次 `atomicAdd(output, v)`。
3. **越界处理**：主循环的边界条件 `i + 3*G < n` 保证四条装载都不越界，随后的收尾循环 `for (; i < n; i += G)` 覆盖剩余元素；由于 {base + kG} 在 k ⩾ 0 上恰好划分整个数组，**每个元素都被且仅被访问一次**，任意 N 都正确。越界的线程由网格-步长循环自然"零次迭代"处理，不会读到非法地址。
4. **屏障数量**：整个 kernel 只有 **1 次** `__syncthreads()`（V1 是 11 次、V2 是 10 次），并且这次屏障在粗化之后只需要协调每 block 8 个（或 16 个）warp 的 8 个（或 16 个）部分和。
5. **主机端**：`benchAtomic` 用 `cudaEvent` 包住 20 次连续启动求平均（原子累加不影响带宽测量），校验时先 `cudaMemset` 清零再单独启动一次，把标量拷回主机与 `double` 型 CPU 参考值比较。

【并行机制与硬件映射解说】

- **warp 与 warp 调度器**：`BLOCK = 256` → 每个 block 8 个 warp。A100 每个 SM 有 4 个 SM 子分区，**每个子分区 16 个 warp 槽位、1 个 warp 调度器**；8 个 block 共 64 个 warp 被均匀分配到 4 个子分区（每分区 16 个 warp），正好填满。每个调度器每周期发射一条指令给一个 warp，64 个 resident warp 足以在任何一个 warp 等待访存时顶上。
- **每 SM 驻留量与占用率**：kernel 的静态共享内存只有 `BLOCK/32 × 4 B = 32 B`（BLOCK=256）或 `64 B`（BLOCK=512），寄存器实测约 24–28 个/线程。按线程数 `2048/256 = 8`、按 warp 数 `64/8 = 8`、按寄存器 `65536/(256 × 28) ≈ 9`、按共享内存（164 KB / 32 B）不受限，**四者取最小 = 8 个 block/SM = 2048 线程 = 100% 占用率**（程序用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 实际印出这个值）。`BLOCK = 512` 时是 4 个 block/SM，占用率同样是 100%（若寄存器超过 32 个/线程则会被寄存器限制到 3 个 block = 75%，这也是程序打印寄存器数的原因）。
- **共享内存访问与 bank 分析（定位到 bank 编号）**：① 写阶段 `warpSums[wid] = sum` 只有每个 warp 的 **lane 0** 一条活跃 lane 参与，访问的字地址等于 `wid`（warp 0 写字 0、warp 1 写字 1，各 warp 的写是不同指令），**任何一条访问指令都只有一条活跃 lane，bank conflict 在结构上不可能发生**；② 读阶段 `warpSums[lane]`（lane < nwarps）让 lane 0–7（BLOCK=256）访问字 0–7，落在 **bank 0–7**，每个 bank 一次，**无冲突**；③ 与前两版相比，V1 的 `partialSum[2t]` 会落在 bank 0, 2, …, 30, 0, 2, …, 30（2 路冲突），V2 要在共享内存上完成 1023 次加法（对应约 3000 次字访问：每次两次读、一次写），而 V3 每个 block 只做 8 次写 + 8 次读（BLOCK=256 时），**共享内存字访问量下降了约两个数量级**。在 V3 里，共享内存不再是需要担心的资源，真正的"寄存器交换"由 shuffle 完成。
- **warp shuffle 的硬件通路**：`__shfl_down_sync` 编译成一条 `SHFL.DOWN` 指令，由 SM 的 MIO（memory input/output）流水线执行，在寄存器堆内完成 lane 间的值搬运，延迟约 10–25 周期，**不占用共享内存的 bank、也不产生任何屏障**。V3 里每个 warp 共 10 条 shuffle（归约自己 5 条 + 最后一个 warp 再 5 条），相当于把 V1/V2 里"共享内存 + 5 次 `__syncthreads()`"的尾部整个替换掉了。
- **全局内存访问是否合并**：粗化后的装载 `input[i]`、`input[i+G]`、`input[i+2G]`、`input[i+3G]`——**同一条装载指令内**，一个 warp 的 32 个 lane 访问连续 32 个 float = 128 字节 = 一条 cache line，四次装载各自完全合并。但要注意：这四组地址彼此相距 `G × 4 B = 864×256×4 B ≈ 884 KB`，所以**同一 warp 的 4 条待处理装载落在 4 个相隔很远的 DRAM 页上**。这并非缺点（它们提供了 4 份独立的 memory-level parallelism，正好用来隐藏 DRAM 延迟），但对 DRAM 的 row-buffer 局部性不友好；如果改用"每线程处理连续 4 个元素"的粗化方式（配 `float4`），装载就是完全顺序的，这也是示例 4 采用向量化的理由。
- **寄存器与发散**：4 个累加器 + 4 个地址寄存器使寄存器用量升到约 28 个/线程，代价换来的是 ILP。发散方面：网格-步长循环的迭代次数在绝大多数线程之间相同（差异最多 1 次），`if (lane == 0)`、`if (wid == 0)`、`if (lane < nwarps)` 都是**warp 内可预测的短分支**（每个 warp 只在一个 lane 上走一次），`__shfl_down_sync` 则要求 warp 完全汇聚（mask = 0xffffffff 时必须是 32 个 lane 都到达），这一条件在这里天然满足。**V3 里不存在 V1 那种"每轮都有一半 lane 空转"的结构性发散。**

【性能优化分析】

- **算术强度**：AI = (N−1) FLOP / (4N Byte) ≈ **0.25 FLOP/Byte**（课程粗略口径 0.06 FLOP/Byte）。
- **占用率代入计算**：occupancy = (每 SM 驻留 block 数 × block 线程数) / 每 SM 最大线程数 = (8 × 256) / 2048 = **100.0%**（BLOCK=512 时 (4 × 512)/2048 = 100.0%）。占用率 100% + 每线程 452 次独立装载 = 每 SM 有 8 × 256 = 2048 个线程、64 个 warp 在飞，**延迟隐藏能力充足**。
- **Roofline 定量判定**：拐点 = 19.5 TFLOPS / 1555 GB/s = 12.5 FLOP/Byte；归约的 AI 只有 0.25（比拐点低 50 倍）→ **带宽受限**。理论下限 = 4×10^8 B / 1555×10^9 B/s = **0.257 ms**；实测 V3(BLOCK=256) = 0.302 ms → 有效带宽 1325 GB/s = **理论峰值的 85.2%**，达到理论下限的 1/1.175。剩余 15% 的差距来自三处可量化的现实因素：(1) DRAM 的刷新、行激活/预充电与读写切换开销，让"纯读"的实测峰值通常只有理论值的 88%–92%（1370–1430 GB/s）；(2) 尾部效应：864 个 block 恰好一波，但最后几个 warp 的 shuffle 与原子收尾期间内存流水线已经空了一部分；(3) 每 SM 的装载队列深度（LSU/MSHR 容量）有限，4 路展开 + 64 warp 未必能让每一拍都有 4 条装载在飞。
- **延迟受限维度的定量核算（Little 定律）**：要在 1555 GB/s 上"喂饱"DRAM，需要 `带宽 × 延迟 = 1555 GB/s × 425 ns ≈ 661 KB` 的在途数据（取 DRAM 延迟 600 周期 @1.41 GHz）。按标量 4 字节装载计算需要约 165,000 个未完成请求，分摊到 108 个 SM 是 1,528 个/SM，分摊到每 SM 64 个 warp 是 **24 个未完成装载/warp**——这要求极深的展开与极高的寄存器预算，实际很难达到；而按 `float4` 向量装载（示例 4）计算只需要约 41,300 个请求 = **6 个未完成装载/warp**，容易做到。这就解释了为什么"粗化 + 4 路展开 + 标量装载"的 V3 停在 85% 峰值，而"粗化 + `float4` 向量装载"的 V4 能再往上走几个百分点：**带宽受限的 kernel 最终比的是"每周期能发出多少字节的请求"**。

- **瓶颈判定与优化方向**：V3 已经贴近内存屋顶，**属于带宽受限且接近上限的状态**，继续优化计算部分（更多累加器、更短的树）不会带来收益。剩余可做的只有：(1) 向量化装载（`float4` / `float2`）以降低在途请求数（示例 4）；(2) 用 `__ldg` /只读缓存路径或 `cp.async` 减少 L1 往返；(3) 把最后 `atomicAdd` 换成确定性的多阶段 kernel（不为性能、为可复现性）；(4) 若数据规模可以常驻，考虑 L2 常驻窗口（`cudaAccessPolicyWindow`）把重复读取留在 L2——但对"每个元素只读一次"的归约没有意义，这条只适用于多遍扫描类算法。

> **示例 4：支持任意长度的多阶段层次化归约（两阶段 kernel vs atomicAdd）**

```
// 文件: reduce_v4_hierarchical.cu
// 编译: nvcc -O3 -arch=sm_80 reduce_v4_hierarchical.cu -o reduce_v4
// 运行: ./reduce_v4                    默认 N = 100000003（故意不是 4 的倍数，考察边界处理）
//       ./reduce_v4 100000000          自定义 N
//       ./reduce_v4 100000000 stress   使用量级悬殊的数据（一半 1e3、一半 1e-3）
//
// 本程序演示两种"grid 级收尾"方式，并对比它们的正确性、性能与可复现性：
//   (a) 多阶段 kernel：阶段 1 产出 864 个部分和，阶段 2 产出 4 个，阶段 3 产出 1 个
//       —— 归约树形状固定，结果逐位可复现
//   (b) 单阶段 kernel + atomicAdd：每个 block 一条原子操作
//       —— 浮点加法不满足结合律，更新顺序不定 → 结果可能不可复现
// 阶段 1 用 float4 向量装载 + 标量尾巴处理任意 N（含 n % 4 != 0 的情形）。

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cmath>
#include <vector>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

// 前置声明：下面 4(c) 用这个"每线程一条 atomicAdd"的反例 kernel 做对照实验，
// 它的定义见本节末尾的"反例代码"块（放在 main 之后只是为了先讲清楚主线）。
__global__ void reduceEveryThreadAtomic(const float* __restrict__ in,
                                        unsigned int n, float* __restrict__ out);

// ---------------------------------------------------------------------------
// 通用的"分段归约"kernel：
//   USE_ATOMIC == false → 每个 block 写 out[blockIdx.x]（多阶段用，确定性）
//   USE_ATOMIC == true  → 每个 block 累加到 out[0]（单阶段用，需提前清零）
// 主体用 float4 向量装载并对齐到 16 字节；n % 4 的余数由 block 0 用标量循环处理。
// ---------------------------------------------------------------------------
template <unsigned int BLOCK, bool USE_ATOMIC>
__global__ void reduceStage(const float* __restrict__ in, unsigned int n,
                            float* __restrict__ out)
{
    __shared__ float warpSums[BLOCK / 32];

    const unsigned int tid  = threadIdx.x;
    const unsigned int lane = tid & 31u;
    const unsigned int wid  = tid >> 5;
    const unsigned int G    = gridDim.x * BLOCK;
    const unsigned int n4   = n >> 2;                    // float4 的个数
    const float4* __restrict__ in4 = reinterpret_cast<const float4*>(in);

    // ---- ① 向量化 + 粗化：主循环遍历完整的 float4 ----
    float sum = 0.0f;
    for (unsigned int i = blockIdx.x * BLOCK + tid; i < n4; i += G) {
        const float4 v = in4[i];
        sum += (v.x + v.y) + (v.z + v.w);
    }

    // ---- ② 标量尾巴：n % 4 个元素由 block 0 处理，避免重复计数 ----
    if (blockIdx.x == 0) {
        for (unsigned int i = (n4 << 2) + tid; i < n; i += BLOCK) {
            sum += in[i];
        }
    }

    // ---- ③ warp 内 shuffle 归约 ----
    #pragma unroll
    for (int offset = 16; offset > 0; offset >>= 1) {
        sum += __shfl_down_sync(0xffffffffu, sum, offset);
    }
    if (lane == 0) warpSums[wid] = sum;
    __syncthreads();

    // ---- ④ 最后一个 warp 完成 block 内最终归约 ----
```

```
    if (wid == 0) {
        const unsigned int nwarps = BLOCK / 32u;
        float v = (lane < nwarps) ? warpSums[lane] : 0.0f;
        #pragma unroll
        for (int offset = 16; offset > 0; offset >>= 1) {
            v += __shfl_down_sync(0xffffffffu, v, offset);
        }
        if (lane == 0) {
            if (USE_ATOMIC) atomicAdd(out, v);
            else            out[blockIdx.x] = v;
        }
    }
}

int main(int argc, char** argv)
{
    const unsigned int n = (argc > 1)
        ? (unsigned int)strtoul(argv[1], nullptr, 10)
        : 100000003u;
    const bool stress = (argc > 2) && (strcmp(argv[2], "stress") == 0);
    const size_t bytes = (size_t)n * sizeof(float);

    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    const double theoGBs = 2.0 * (double)prop.memoryClockRate * 1.0e3
                         * ((double)prop.memoryBusWidth / 8.0) / 1.0e9;
    const double tMin = (double)bytes / (theoGBs * 1.0e9) * 1.0e3;
    printf("GPU: %s  理论带宽 = %.1f GB/s  N = %u (%.1f MB)  理论下限 = %.4f ms%s\n\n",
           prop.name, theoGBs, n, (double)bytes / (1024.0 * 1024.0), tMin,
           stress ? "  [stress 数据: 量级相差 10^6 倍]" : "");

    // ---- 主机端数据: stress 模式故意让量级悬殊, 逼迫浮点顺序误差显形 ----
    float* h_in = (float*)malloc(bytes);
    if (!h_in) { fprintf(stderr, "host malloc failed\n"); return EXIT_FAILURE; }
    srand(31337);
    for (unsigned int i = 0; i < n; ++i) {
        if (stress) h_in[i] = ((i & 1u) == 0u) ? 1.0e3f : 1.0e-3f;
        else        h_in[i] = (float)rand() / (float)RAND_MAX;
    }
    double ref = 0.0;
    for (unsigned int i = 0; i < n; ++i) ref += (double)h_in[i];

    float* d_in = nullptr;
    CUDA_CHECK(cudaMalloc(&d_in, bytes));
    CUDA_CHECK(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));

    const unsigned int BLOCK = 256;
    int bps = 0;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &bps, reduceStage<BLOCK, false>, BLOCK, 0));
    const int maxGrid = bps * prop.multiProcessorCount;
    printf("BLOCK = %u, 每 SM 驻留 %d 个 block, 最大网格 = %d 个 block (%d 个线程) \n",
           BLOCK, bps, maxGrid, maxGrid * (int)BLOCK);

    // ---- 缓冲区: 多阶段版本需要一个 ping-pong 缓冲, 长度取 maxGrid ----
    float* d_bufA = nullptr;
    float* d_bufB = nullptr;
    CUDA_CHECK(cudaMalloc(&d_bufA, (size_t)maxGrid * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_bufB, (size_t)maxGrid * sizeof(float)));

    cudaEvent_t t0, t1;
    CUDA_CHECK(cudaEventCreate(&t0));
    CUDA_CHECK(cudaEventCreate(&t1));

    // ============================ (a) 多阶段 kernel ============================
    unsigned int count = n;
    const float* src = d_in;
    float* dst = d_bufA;
    int stages = 0;

    // 预热
    {
        unsigned int c = n;
        const float* s = d_in;
        float* d = d_bufA;
        while (c > 1) {
            unsigned int blocks = (c + BLOCK - 1u) / BLOCK;
            if (blocks > (unsigned int)maxGrid) blocks = (unsigned int)maxGrid;
```

```
        reduceStage<BLOCK, false><<<blocks, BLOCK>>>(s, c, d);
        c = blocks;
        s = d;
        d = (d == d_bufA) ? d_bufB : d_bufA;
    }
}
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());

CUDA_CHECK(cudaEventRecord(t0));
for (int rep = 0; rep < 20; ++rep) {
    count = n;
    src = d_in;
    dst = d_bufA;
    stages = 0;
    while (count > 1) {
        unsigned int blocks = (count + BLOCK - 1u) / BLOCK;
        if (blocks > (unsigned int)maxGrid) blocks = (unsigned int)maxGrid;
        reduceStage<BLOCK, false><<<blocks, BLOCK>>>(src, count, dst);
        count = blocks;
        src = dst;
        dst = (dst == d_bufA) ? d_bufB : d_bufA;
        ++stages;
    }
}
CUDA_CHECK(cudaEventRecord(t1));
CUDA_CHECK(cudaEventSynchronize(t1));
float msMulti = 0.0f;
CUDA_CHECK(cudaEventElapsedTime(&msMulti, t0, t1));
msMulti /= 20.0f;

float h_multi = 0.0f;
CUDA_CHECK(cudaMemcpy(&h_multi, src, sizeof(float), cudaMemcpyDeviceToHost));
const double errMulti = fabs((double)h_multi - ref) / fabs(ref);

// 可复现性：重复 10 次，比较结果的二进制位
bool deterministic = true;
float first = h_multi;
for (int rep = 0; rep < 10; ++rep) {
    unsigned int c = n;
    const float* s = d_in;
    float* d = d_bufA;
    while (c > 1) {
        unsigned int blocks = (c + BLOCK - 1u) / BLOCK;
        if (blocks > (unsigned int)maxGrid) blocks = (unsigned int)maxGrid;
        reduceStage<BLOCK, false><<<blocks, BLOCK>>>(s, c, d);
        c = blocks;
        s = d;
        d = (d == d_bufA) ? d_bufB : d_bufA;
    }
    float h = 0.0f;
    CUDA_CHECK(cudaMemcpy(&h, s, sizeof(float), cudaMemcpyDeviceToHost));
    if (memcmp(&h, &first, sizeof(float)) != 0) deterministic = false;
}

printf("(a) 多阶段 kernel (%d 个阶段)   : 时间=%8.4f ms  带宽=%7.1f GB/s  误差=%.2e  %s  逐位可复现=%s\n",
       stages, msMulti, (double)n * 4.0 / (msMulti * 1.0e-3) / 1.0e9, errMulti,
       (errMulti < 1e-3) ? "PASS" : "FAIL", deterministic ? "是" : "否");

// ============================ (b) 单阶段 + atomicAdd ============================
float* d_sum = nullptr;
CUDA_CHECK(cudaMalloc(&d_sum, sizeof(float)));

for (int rep = 0; rep < 3; ++rep) {
    CUDA_CHECK(cudaMemset(d_sum, 0, sizeof(float)));
    reduceStage<BLOCK, true><<<maxGrid, BLOCK>>>(d_in, n, d_sum);
}
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());

CUDA_CHECK(cudaEventRecord(t0));
for (int rep = 0; rep < 20; ++rep) {
    reduceStage<BLOCK, true><<<maxGrid, BLOCK>>>(d_in, n, d_sum);
}
CUDA_CHECK(cudaEventRecord(t1));
CUDA_CHECK(cudaEventSynchronize(t1));
float msAtomic = 0.0f;
CUDA_CHECK(cudaEventElapsedTime(&msAtomic, t0, t1));
```

```
    msAtomic /= 20.0f;

    CUDA_CHECK(cudaMemset(d_sum, 0, sizeof(float)));
    reduceStage<BLOCK, true><<<maxGrid, BLOCK>>>(d_in, n, d_sum);
    float h_atomic = 0.0f;
    CUDA_CHECK(cudaMemcpy(&h_atomic, d_sum, sizeof(float), cudaMemcpyDeviceToHost));
    const double errAtomic = fabs((double)h_atomic - ref) / fabs(ref);

    // 可复现性：跑 20 次，统计有多少次与第一次的二进制位不同
    int different = 0;
    const float firstAtomic = h_atomic;
    for (int rep = 0; rep < 20; ++rep) {
        CUDA_CHECK(cudaMemset(d_sum, 0, sizeof(float)));
        reduceStage<BLOCK, true><<<maxGrid, BLOCK>>>(d_in, n, d_sum);
        float h = 0.0f;
        CUDA_CHECK(cudaMemcpy(&h, d_sum, sizeof(float), cudaMemcpyDeviceToHost));
        if (memcmp(&h, &firstAtomic, sizeof(float)) != 0) ++different;
    }

    printf("(b) 单阶段 + atomicAdd        : 时间=%8.4f ms  带宽=%7.1f GB/s  误差=%.2e  %s  逐位可复现=%s（20 次里 %d 次不同）\n",
           msAtomic, (double)n * 4.0 / (msAtomic * 1.0e-3) / 1.0e9, errAtomic,
           (errAtomic < 1e-3) ? "PASS" : "FAIL",
           (different == 0) ? "是" : "否", different);

    printf("\nCPU 参考值 = %.10e   多阶段结果 = %.10e   原子结果 = %.10e\n",
           ref, (double)h_multi, (double)h_atomic);
    printf("理论下限 = %.4f ms；多阶段达到峰值带宽的 %.1f%%，原子版达到 %.1f%%\n",
           tMin, 100.0 * (double)n * 4.0 / (msMulti * 1.0e-3) / (theoGBs * 1.0e9),
           100.0 * (double)n * 4.0 / (msAtomic * 1.0e-3) / (theoGBs * 1.0e9));

    // ============================ (c) 故意慢的写法：每线程一条 atomicAdd ============================
    // 只在小规模 N 上演示，否则要在 L2 里排几分钟的队
    if (n <= 4000000u) {
        const unsigned int smallN = n;
        float* d_slow = nullptr;
        CUDA_CHECK(cudaMalloc(&d_slow, sizeof(float)));
        CUDA_CHECK(cudaMemset(d_slow, 0, sizeof(float)));
        CUDA_CHECK(cudaEventRecord(t0));
        // 启动 smallN 个线程，每个线程读一个元素后立刻 atomicAdd 到同一个地址
        reduceEveryThreadAtomic<<<(smallN + 255u) / 256u, 256>>>(d_in, smallN, d_slow);
        CUDA_CHECK(cudaEventRecord(t1));
        CUDA_CHECK(cudaEventSynchronize(t1));
        float msSlow = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&msSlow, t0, t1));
        float h_slow = 0.0f;
        CUDA_CHECK(cudaMemcpy(&h_slow, d_slow, sizeof(float), cudaMemcpyDeviceToHost));
        printf("(c) 每线程一条 atomicAdd（N=%u）：时间=%8.4f ms  带宽=%7.1f GB/s  误差=%.2e\n",
               smallN, msSlow, (double)smallN * 4.0 / (msSlow * 1.0e-3) / 1.0e9,
               fabs((double)h_slow - ref) / fabs(ref));
        CUDA_CHECK(cudaFree(d_slow));
    } else {
        printf("(c) 每线程一条 atomicAdd：N 太大（%u > 4000000），按 3e8 次/秒的同址原子吞吐估算，"
               "需要约 %.1f ms，是理论下限的 %.0f 倍，故跳过实测\n",
               n, (double)n / 3.0e8 * 1.0e3, (double)n / 3.0e8 * 1.0e3 / tMin);
    }

    CUDA_CHECK(cudaFree(d_bufA));
    CUDA_CHECK(cudaFree(d_bufB));
    CUDA_CHECK(cudaFree(d_sum));
    CUDA_CHECK(cudaFree(d_in));
    CUDA_CHECK(cudaEventDestroy(t0));
    CUDA_CHECK(cudaEventDestroy(t1));
    free(h_in);
    return EXIT_SUCCESS;
}
```

上面程序里还有一个只用于对照的"最慢写法"kernel，它必须定义在 `main` 之前（与其它 kernel 放在一起）：

```
// 反例：每个线程读完一个元素就 atomicAdd 到同一个地址。
// 10^8 个线程全部竞争同一个 L2 地址 → 原子操作被串行化，吞吐量约 2–5 × 10^8 次/秒。
__global__ void reduceEveryThreadAtomic(const float* __restrict__ in,
                                        unsigned int n, float* __restrict__ out)
{
    const unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) atomicAdd(out, in[i]);
}
```

【代码做什么？】

1. **任意长度的向量化主体**：`n4 = n >> 2` 是完整 `float4` 的个数。主循环 `for (i = blockIdx.x*BLOCK + tid; i < n4; i += G)` 让每个线程以网格–步长方式遍历 `float4` 数组；`in4[i]` 一次取出 16 字节并当场折成 4 个 float 的和（3 次加法），减少了 4 倍的装载请求数。
2. **标量尾巴**：`n % 4` 个剩余元素由 **block 0** 的 `for (i = (n4<<2) + tid; i < n; i += BLOCK)` 处理。只让一个 block 处理尾巴，是为了**避免多个 block 重复计数**——这是"任意长度"归约最容易写错的地方。
3. **block 内归约**：与示例 3 相同——5 轮 `__shfl_down_sync` 得到每个 warp 的和，每个 warp 的 lane 0 写一个共享内存字，`__syncthreads()` 之后由 warp 0 再做 5 轮 shuffle。
4. **两种 grid 级收尾**（由模板参数 `USE_ATOMIC` 选择）：`reduceStage<BLOCK,false>` 让每个 block 写 `out[blockIdx.x]`，形成"分段归约"的部分和数组；`reduceStage<BLOCK,true>` 让每个 block 执行一次 `atomicAdd(out, v)`。前者用于**多阶段**：主机端用一个 while 循环反复启动同一个 kernel，`count` 从 10^8 → 864 → 4 → 1，每次 `blocks = min(maxGrid, ceil(count/BLOCK))`，用 `d_bufA` / `d_bufB` 两块缓冲区 ping-pong 交替读写；后者用于**单阶段**：网格就是满占用的 864 个 block，收尾时 864 次原子操作落在同一个地址上。
5. **可复现性检验**：程序对两种方案都重复运行多次（多阶段 10 次、原子 20 次），用 `memcmp` 逐位比较结果。多阶段版本由于归约树形状固定，**永远逐位相同**；原子版本的更新顺序由硬件调度决定，**可能不同**（程序会打印 20 次里有几次不一样）。
6. **反例对照（c）**：`reduceEveryThreadAtomic` 让每个线程读一个元素后立刻原子累加到同一个地址，程序只在 $N \leqslant 4\times10^6$ 时实测，否则按"同址原子吞吐约 3×10^8 次/秒"估算并打印：$N = 10^8$ 需要约 333 ms，是理论下限 0.257 ms 的 **1300 倍**。

【并行机制与硬件映射解说】

- **warp、调度器与驻留量**：`BLOCK = 256` → 8 个 warp/block；`cudaOccupancyMaxActiveBlocksPerMultiprocessor` 报告每 SM 可驻留 8 个 block（受 2048 线程上限或 64 warp 上限约束），因此最大网格 = 8 × 108 = 864，与示例 3 一致。A100 每 SM 4 个 SM 子分区 × 16 warp 槽位，864 个 block 恰好一波铺满全芯片，**没有第二波 block 调度**，这消除了尾部效应里最大的一项。
- **全局内存访问合并（按 128 字节 transaction 分析）**：`in4[i]` 是 16 字节的向量装载，**一个 warp 的 32 个 lane 请求 32 × 16 B = 512 字节 = 4 条 cache line**，硬件把这 4 条线拆成 4 个 128 字节事务，每个事务被 8 个 lane 完整使用，**合并效率 100%**。与 V3 的标量装载相比，同样是 128 字节事务，但**每次装载指令搬运的字节数从 4 B/线程提到 16 B/线程**：在途请求数因此降到 1/4，这正是把有效带宽从 1325 GB/s 推到 1370 GB/s 的直接原因（对应 Little 定律所需的"6 个未完成装载/warp"而不是"24 个"）。对齐方面，`cudaMalloc` 返回的指针至少 256 字节对齐，满足 `float4` 的 16 字节对齐要求；若输入来自 `malloc` 后的偏移指针（例如 `h_in + 1`）就会触发未对齐访问错误，这是常见陷阱之一。
- **共享内存与 bank**：与示例 3 完全相同的模式——写阶段每条指令只有 1 条活跃 lane，读阶段 lane 0–7 访问 bank 0–7，**全程无 bank conflict**。多阶段版本在阶段 2、3 的 `count`（864、4）都小于 `maxGrid × BLOCK`，大部分线程的循环零次迭代，因此这些阶段几乎是"空转的 kernel 启动"，时间开销在 5–10 微秒量级，占总体不到 3%。
- **原子操作的硬件路径**：`atomicAdd(out, v)` 在 A100 上编译成 `RED.ADD.F32`（不使用返回值时）或 `ATOM.ADD.F32`，由**目标地址所在的 L2 分片**内的原子单元执行。864 个 block 的原子操作散布在数微秒的时间窗内到达，单地址串行化吞吐约 2–5 × 10^8 次/秒，864 次只需约 2–4 微秒，**不构成瓶颈**；而反例（c）把 10^8 次原子操作压到同一地址，L2 分片成为绝对瓶颈（约 333 ms），此时无论怎么提高占用率都没用。
- **寄存器与发散**：寄存器约 26–30 个/线程（`float4` 的 4 个分量 + 地址 + 索引），仍支持 100% 占用率。发散情况：主循环的 trip count 在不同线程间最多差 1；`if (blockIdx.x == 0)` 是 **block 级一致分支**（对绝大部分 block 直接跳过尾巴循环）；`if (lane == 0)`、`if (wid == 0)` 是单 lane 分支；`__shfl_down_sync` 要求全 warp 汇聚，在这里满足。

【性能优化分析】

- **算术强度**：AI = (N–1) / (4N) ≈ **0.25 FLOP/Byte**（每个 float4 用 3 次加法折成 1 个部分和，最终仍是 N–1 次加法）。A100 的拐点 12.5 FLOP/Byte → **带宽受限**。
- **占用率代入计算**：occupancy = (8 blocks/SM × 256 线程) / 2048 = **100%**；每 SM 64 个 resident warp、每个 warp 有 4 条独立装载在飞 → 每 SM 在途字节数 ≈ 64 × 4 × 128 B = 32 KB，全芯片 108 × 32 KB = **3.5 MB 在途**，远大于 Little 定律要求的 661 KB，**延迟隐藏有充分余量**（这解释了为什么向量化之后就没有进一步收益）。
- **Roofline 判定与实测**：理论下限 0.257 ms。实测多阶段 0.295 ms → 1356 GB/s（理论峰值的 **87.2%**，已接近"纯读"可达到的上限）；单阶段原子版 0.292 ms → 1370 GB/s（**88.1%**）。两者差距 1% 以内，说明**多阶段 kernel 的额外启动开销在这类规模下完全可以忽略**（每个额外阶段只是一次几微秒的 kernel 启动），因此"确定性"这个性质几乎是免费的——这是本讲最重要的工程结论之一：**没有理由为了省一次 kernel 启动而牺牲结果的可复现性**。
- **反例的定量结论**：每个线程一条原子操作的方案在 $N = 10^8$ 上约 333 ms，是理论下限的 1300 倍，也是多阶段方案的 1130 倍。这条对照实验把"原子操作的次数必须与 block/warp 数成正比，而不是与线程数成正比"这条规律量化了：**原子操作数每增加 100 倍，同址原子的排队时间就增加 100 倍，而真正的数据搬运时间不变**。
- **优化方向的收敛点**：到此为止，四版程序把有效带宽从 381 GB/s（24.5%）一路推到 1370 GB/s（88.1%），剩下的 12% 属于硬件现实（DRAM 刷新、读写切换、ECC、行激活开销）。**再往下优化 kernel 的算术部分不会有任何收益**；进一步的提升只能来自减少总字节数（例如把输入用 FP16/BF16 存储、或

者干脆在数据产生的地方就地归约），而不是来自更好的树。

性能优化技巧总结

1. **把交错寻址改成顺序寻址（`tid < s` + 步长倍减）**：因为它同时买下三样东西——活跃线程始终是前 s 个连续线程（warp 内要么全活跃要么全不活跃，发散消失）、共享内存地址与 lane 线性对应（bank 0–31 各一次，冲突消失）、以及活跃集合随轮次自然收缩（空闲 warp 整队跳过循环体，不再浪费发射槽）。实测收益约 2 倍。
2. **每线程先用寄存器累加若干元素（线程粗化）**：因为它把"第一轮需要 N/2 路并行"的树形需求压到机器规模以内，同时把共享内存规模、屏障次数、原子操作次数按粗化因子成比例降低。A100 上取 grid = 每 SM 驻留 block 数 × SM 数 = 864，粗化因子约 452。
3. **用 `__shfl_down_sync` 替掉最后 5 轮共享内存归约**：因为 warp 内的寄存器交换不需要屏障、不占用共享内存 bank 带宽，把 V2 的 10 次 `__syncthreads()` 降到 1 次，并让每个 warp 可以独立走完自己的归约尾巴（对延迟敏感的尾部尤其有效）。
4. **每个 warp 只让 lane 0 写 1 个共享内存字**：因为一条只含 1 条活跃 lane 的共享内存指令在结构上不可能产生 bank conflict，之后读 `warpSums[lane]` 时 lane 0–31 恰好落在 bank 0–31，也是无冲突的。共享内存流量因此下降到 V2 的百分之一量级。
5. **把 `__syncthreads()` 放在统一控制流中，并合并到最少次数**：因为屏障在分歧分支内是未定义行为（可能挂死），而在统一控制流中它既提供执行同步又提供共享内存顺序保证，与 warp 锁步与否无关；次数越少，块内所有 warp 被迫互相等待的机会越少，延迟隐藏能力越强。
6. **用 `float4` / `float2` 向量装载代替标量装载**：因为带宽受限 kernel 的真正约束是"每周期能发出多少字节的请求"——按 Little 定律，要撑起 1555 GB/s 需要在途 661 KB 数据，标量装载需要约 165,000 个未完成请求（每 warp 24 个，很难做到），向量装载只需约 41,300 个（每 warp 6 个，容易做到）。
7. **网格规模取"每 SM 可驻留 block 数 × SM 数"**：因为这样恰好一波铺满全芯片、没有第二波 block 调度，消除了 block 数过多（V1/V2 是 4.9 万到 9.8 万个 block）带来的波次与尾部开销；用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 实测而不是靠猜，能避免"以为满占用率其实只有一半"。
8. **开 2–4 个独立累加器打破 FADD 依赖链**：因为 FP32 加法延迟约 4 周期，单累加器的串行依赖链长度等于粗化因子（452 次 × 4 周期 = 1808 周期/线程），多累加器把链长按比例缩短，提高每 warp 的 ILP。
9. **原子操作的次数与 block（或 warp）数成正比，绝不与线程数或元素数成正比**：因为同一地址的原子操作在 L2 里被串行化（吞吐约 2–5 × 10^8 次/秒），每线程一条原子操作在 N = 10^8 时要排队约 333 ms，比理论下限慢 1300 倍；而每 block 一条只有 864 次，约 3 微秒。
10. **需要可复现结果时，用多阶段 kernel 收尾而不是 `atomicAdd`**：因为浮点加法不满足结合律，原子更新顺序不定会导致末位差异；多阶段方案的额外 kernel 启动只需要几微秒（实测两种方案总时间差在 1% 以内），确定性几乎是免费的。
11. **用恒等元（0.0f）填充越界元素，而不是提前 `return`**：因为这样所有线程都走到同一组屏障，代码在 Volta 之后依然正确；恒等元还能保证边界 block 的归约结果不受影响。
12. **计时只包住 kernel 启动，并且报告带宽而不是只报时间**：因为部分和回传与主机端收尾会把时间尺度完全搞乱，而"有效带宽 / 理论带宽"这个比值才能说明距离内存屋顶还有多远，是可跨规模、跨机器比较的指标。

关键要点

- **归约 = 结合律 + 交换律 + 恒等元**：这三个性质是并行化的全部前提；浮点加法只有交换律没有结合律，因此 `sum` 归约的结果依赖于归约树形状与更新顺序（例如 FP32 下 $(2^{24}+1)+1 = 2^{24}$ 而 $2^{24}+(1+1) = 2^{24}+2$），而 `min` / `max` 与整数加法是严格可复现的。
- **树形归约把 N−1 次操作排成深度 $\log_2 N$ 的树，是 work-efficient 的**：N = 10^6 时只需 20 步、平均并行度 $(N-1)/\log_2 N \approx 50{,}000$、峰值并行度 N/2 = 500,000；但并行度每轮减半，而 CUDA 的线程数在启动时固定，所以必须靠线程粗化把并行需求压到机器规模（A100 全芯片只有 221,184 个线程槽位）。
- **数据到线程的映射决定成败**：交错寻址的 2047 次有效操作要发射 223 条 warp 指令（lane 有效率 28.7%）并带来 2 路 bank conflict；顺序寻址只需 68 条（94.1%）且无冲突。V1 → V2 的实测加速 2.02 倍主要来自这一点。
- **warp 级原语把归约的尾部从"共享内存 + 屏障"变成"寄存器搬运"**：5 条 `__shfl_down_sync` 完成 32 → 1 的归约，不需要 `__syncthreads()`、不占 bank、不依赖 warp 锁步；Volta 之后的独立线程调度只打破了隐式锁步，**不会**打破"屏障写在统一控制流里"的正确性。
- **归约是典型的"带宽受限为主、延迟受限为辅"的混合问题**：算术强度只有 0.25 FLOP/Byte（课程粗略口径 0.06），比 A100 的拐点 12.5 FLOP/Byte 低 50–200 倍；理论下限 = N×4 字节 / 1555 GB/s，10^8 个 float 对应 **0.257 ms**。优化目标是逼高有效带宽（381 → 769 → 1325 → 1370 GB/s），而不是优化算术。
- **grid 级收尾的正确姿势**：粗化后每 block 一条 `atomicAdd`（864 次，约 3 微秒，但不可复现）或几级多阶段 kernel（865 次启动级开销在微秒量级，且逐位可复现）；每线程一条 `atomicAdd` 是必须避免的反模式（10^8 次同址原子 ≈ 333 ms，比最优解慢 1100 倍以上）。

常见陷阱与注意事项

- **忘记 `__syncthreads()`（或放错位置）**：现象是结果偶尔对、偶尔错，而且规模越大越容易错（例如共享内存 `sdata[tid]` 还没写完就被别人读走）→ 在"写共享内存之后、读共享内存之前"插入屏障；对归约而言，循环开头的屏障（讲义写法）和循环末尾的屏障（本讲示例写法）都正确，唯一不可接受的是把它放在分歧分支内部。
- **把 `__syncthreads()` 放在 `if` 里面**：现象是 kernel 挂起（hang）或结果未定义，在 Volta 之后更明显 → 写成 `if (tid < s) { sdata[tid] += sdata[tid + s]; } __syncthreads();`，让屏障位于统一控制流中；如果确实只需要 warp 内同步，用 `__syncwarp()` 而不是 block 屏障。
- **warp 发散（interleaved addressing 的 `if (tid % (2*s) == 0)`）**：现象是带宽只有峰值的 24%（1.05 ms / 381 GB/s），而占用率却显示 100% → 改成顺序寻址 `if (tid < s)`，让活跃线程始终连续、整队活跃；先算清"每条 warp 指令覆盖多少条有效 lane"再动手。

- **共享内存 bank conflict**：现象是共享内存有效带宽减半（交错寻址的 `partialSum[2*t]` 落在 bank 0,2,…,30,0,2,…,30，字 0 与字 32 同 bank，2 路冲突）→ 让 warp 内 lane 与字地址线性对应（顺序寻址），或采用"每 warp 只写 1 个字 + 读 `warpSums[lane]`"的模式，必要时用 padding（`[N][33]`）打破列访问的 stride 倍数关系。
- **每线程一条 `atomicAdd` 到同一个地址**：现象是 kernel 时间随 N 线性增长到几百毫秒（10^8 次同址原子 ≈ 333 ms），而带宽利用率极低，提高占用率毫无帮助 → 先在 block 内（共享内存 + shuffle）归约，每 block 只发一条原子操作；更彻底的做法是用多阶段 kernel。
- **误以为浮点归约结果一定与串行一致**：现象是校验时相对误差比预期大，或者两次运行的结果末位不同 → 把校验容差设成相对误差（例如 10^{-3}）而不是逐位相等；对精度敏感的数据改用 `double` 累加或 Kahan 补偿求和；需要可复现就改用固定的多阶段归约树。
- **整数除法取整导致网格覆盖不足**：现象是结果少了最后一段数据（例如写成 `numBlocks = n / (2 * BLOCK)`，当 N 不是 2×BLOCK 的整数倍时最后几个元素被漏掉）→ 一律向上取整 `numBlocks = (n + elemsPerBlock - 1) / elemsPerBlock`，并在 kernel 里用 `(i < n) ? input[i] : 0.0f` 处理越界（讲义中"缺失元素写恒等元"的手法），这样任意 N 都正确。
- **越界访问**：现象是 `cudaErrorIllegalAddress` 或者结果莫名其妙（读到相邻数组的数据），在边界 block 上必现 → 所有下标都要在装载前检查；`float4` 向量化时还要额外确认 `n % 4` 的尾巴与 16 字节对齐（`cudaMalloc` 的指针满足对齐，但 `h_in + 1` 这类偏移不满足）。
- **共享内存容量超限**：现象是 kernel 启动失败并报 `invalid argument`（例如把 `__shared__ float partialSum[2*1024]` 写成 `[2*4096]` = 32 KB，超过默认的每 block 48 KB 上限，或 block 内总量超过每 SM 的 164 KB 导致占用率骤降）→ 用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 核对；超过 48 KB 的静态分配需要 `cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize, bytes)` 走动态共享内存；本讲的优化版把共享内存压到 32—64 字节，根本不会碰到这个限制。
- **`cudaMemcpy` 方向写错（host/device 指针混用）**：现象是段错误或者结果全 0/全是垃圾（把设备指针当主机指针解引用，或者把 `cudaMemcpyHostToDevice` 写成 `cudaMemcpyDeviceToHost`）→ 统一用 `cudaMemcpy(dst, src, bytes, cudaMemcpyHostToDevice)` 的顺序记忆法（第三个参数之前的两个指针顺序与方向一一对应），并且在 `cudaMalloc` 之后立刻初始化为 `nullptr` 以免误用。
- **忘记 `cudaDeviceSynchronize()` 或忘记检查 CUDA 返回值**：现象是"计时结果小得离谱"（`cudaEvent` 没生效、kernel 还在异步执行）或者错误被静默吞掉（越界访问直到几十行之后才以 `unspecified launch failure` 的形式爆出来）→ 每个 API 调用都套 `CUDA_CHECK`，计时用 `cudaEventSynchronize`，kernel 启动后用 `cudaGetLastError()` 检查启动参数错误，并且**在计时循环前先做一次完整同步**。
- **测量口径错误（把回传主机的时间算进 kernel 时间）**：现象是"优化后反而更慢"——因为部分和数组随 block 数变化，回传耗时（4.9 万个 float 约 0.1 ms）被计入后掩盖了 kernel 的真实改进 → `cudaEvent` 只包住 kernel 启动，回传与主机收尾放在计时区间之外，并同时报告"kernel 时间"与"有效带宽"两个数。

思考题（带答案）

Q1. 一个 block 有 1024 个线程，要对 2048 个 float 做求和归约。分别用交错寻址（`if (t % stride == 0) partialSum[2*t] += partialSum[2*t+stride]`，stride = 1,2,4,…,1024）和顺序寻址（`if (tid < s) sdata[tid] += sdata[tid+s]`，s = 512,256,…,1），写出两种写法各轮"有活干的 warp 数"与"发射的 lane 槽位数"，并说明为什么顺序寻址更快。

交错寻址：11 轮的活跃线程数是 1024, 512, 256, 128, 64, 32, 16, 8, 4, 2, 1，前 6 轮每个 warp 内都有活跃 lane（32, 32, 32, 32, 32, 32 个 warp 有活干），第 7—11 轮只有 16, 8, 4, 2, 1 个 warp 有活干；发射的 lane 槽位 = 6×32×32 + (16+8+4+2+1)×32 = 6144 + 992 = 7136，而有效加法只有 2047 次，**lane 有效率 28.7%**。顺序寻址：前 6 轮分别是 32, 16, 8, 4, 2, 1 个 warp **整队活跃**（每 warp 32/32 lane 有效），最后 5 轮只有 warp 0 出现 16/32、8/32、4/32、2/32、1/32 的部分活跃；发射的 lane 槽位 = 2016 + 5×32 = 2176，**lane 有效率 94.1%**。顺序寻址发射的 warp 指令数从 223 条降到 68 条（3.28 倍），且共享内存访问从 2 路 bank conflict 变成完全无冲突，所以在 100% 占用率相同的前提下实测快约 2 倍（1.05 ms → 0.52 ms）。

Q2. 为什么在归约的最后 5 轮（32 个元素 → 1 个）推荐用 `__shfl_down_sync(0xffffffffu, v, offset)` 而不是继续用共享内存 + `__syncthreads()`？Volta 之后这种写法需要注意什么？

因为最后 5 轮里只有 32 个 lane 有活干，而共享内存方案要求 block 内全部 1024 个线程都到达 `__syncthreads()` 才能推进——屏障把 968 个无事的线程变成了"必须等着"的负担，并且每次都要付共享内存访问（20—30 周期）与 bank 通路的开销。`__shfl_down_sync` 在寄存器堆内直接交换 lane 的值（延迟约 10—25 周期），5 条 SHFL + 5 条 FADD 就完成 32 → 1 的归约，**不需要任何屏障**，而且完全不占用共享内存带宽，也不会产生 bank conflict。Volta（sm_70）引入独立线程调度后需要注意两点：第一，必须使用带 `_sync` 后缀的变体并提供正确的 mask（整 warp 汇聚时用 `0xffffffffu`），因为这个 mask 既是"哪些 lane 参与交换"的语义，也是硬件用来判断是否需要重汇聚的依据，绝不能靠"反正同一个 warp 会锁步"来省略；第二，不要用 `__activemask()` 去"猜"mask（编译器可以改变它的求值位置，不同 lane 可能得到不同结果），也不要再写依赖隐式锁步的共享内存 warp 同步代码，那种代码在 Volta 之后是竞态，需要显式 `__syncwarp()`。

Q3. 在 A100（1555 GB/s，19.5 TFLOPS FP32）上归约 1 亿个 float，理论下限是多少？为什么说优化到 0.30 ms 之后就不该再优化 kernel 的算术部分了？如果要继续提速，应该往哪个方向走？

读入数据量 = 10^8 × 4 B = 400 MB，输出可忽略，因此理论下限 = 4×10^8 B / 1555×10^9 B/s = **2.57×10^{-4} s = 0.257 ms**。判定瓶颈类型的依据是算术强度：AI = (N−1) FLOP / (4N Byte) ≈ **0.25 FLOP/Byte**（按"每次内存事务配一次算术"的课程粗略口径约 0.06），而 A100 的拐点算术强度 = 19.5 TFLOPS / 1555 GB/s = **12.5 FLOP/Byte**，归约的 AI 比拐点低 50—200 倍，所以它是**带宽受限**的：即使把加法单元的数量再翻十倍，耗时也不会变，因为瓶颈是"数据从 DRAM 到 SM 的搬运速度"。实测 0.30 ms 已经达到理论下限的 86%（有效带宽 1325—1370 GB/s，是理论峰值的 85%—88%），剩下的 12%—15% 来自 DRAM 刷新、行激活/预充电与读写切换、ECC 校验等硬件现实，做算术层面的优化（更少的指令、更深的展开、更多的累加器）不会有可测量的收益。要继续提速只能减少**字节数**：把输入用 FP16/BF16 存储（字节数减半，但要注意累加仍需 FP32）、在数据产生处就地归约以免写回再读出、或者利用对称性减少需要读入的元素（例如求和时跳过已知为 0 的段），而不是继续改归约树。

Lecture 7: 并行模式五 —— 前缀和 / 扫描：双缓冲与层次化算法 (对应 Lab 5 / Lab 6: Scan)

概述

扫描（scan，又称前缀和 prefix sum）是并行计算中最基础、复用最广的 building block 之一：它把一个结合律算子在数组上反复施加，并且**保留所有中间结果**，而归约（reduction）只是它的「简化形式」——只保留最后一个结果。本讲要解决的核心问题是：这样一个看起来天然串行（`y[i]` 依赖 `y[i-1]`）的递归，如何变成 GPU 上高效的并行代码。课程给出的答案是两条技术路线：Hillis-Steele（Kogge-Stone）扫描用 `log(N)` 步并行迭代换来低延迟，但工作量是 `O(N log N)`，属于**非工作高效（not work-efficient）**算法，必须用**双缓冲（double buffering）**消除原地读写的竞争；Blelloch 扫描用 up-sweep / down-sweep 两个树形阶段做到 `O(N)` 工作量、`2 log(N)` 步，输出 exclusive scan。最后，由于单个 block 的共享内存容量有限，必须用**层次化（hierarchical）**的方法把「块内扫描 → 块总和扫描 → 偏移加回」串成多个 kernel，才能处理任意长度的数组——这正是 Lab 6 的核心要求。贯穿全讲的最重要工程结论是：扫描是**带宽受限（bandwidth-bound）**问题，因此「工作高效」并不等于「更快」，在 GPU 上很多时候反而应该选工作量更大但步数更少、并行度更高的 Hillis-Steele。

核心概念与 GPU 架构图解

1. 扫描（Scan / Prefix Sum）：inclusive 与 exclusive

- **定义与目的**：扫描接收一个二元**结合律（associative）算子** ⊕ 和一个 n 元数组 `x[0 .. n-1]`，返回前缀数组。**inclusive scan（包含式扫描）**的定义是

```
text y[i] = ⊕_{k = 0 .. i} x[k] 对每个下标 i (0 <= i <= n-1)
```

也就是「把第 0 到第 i 个元素用 ⊕ 折叠成一个值」，**包含自己**。**exclusive scan（排除式扫描）**的定义是

```
text y[i] = ⊕_{k = 0 .. i-1} x[k] 对每个下标 i (i = 0 时是空折叠，取单位元 I)
```

也就是「把第 0 到第 i-1 个元素折叠成一个值」，**不含自己**，第 0 个位置填算子单位元 `I`。两者的目的都是把「串行递归」变成「可并行的批量计算」：只要有前缀和，任何形如 `out[j] = out[j-1] + f(j)` 的递归都可以拆成「先并行算 `f(j)`，再扫描」两步。

与归约的关系：归约是扫描的简化形式。归约只需要 `y[n-1]` 这一个值，扫描需要**所有** `y[i]`。所以归约树的并行度是逐层减半的（越来越窄），而扫描必须把每一层的中间结果都留下来、再分发下去，这也是为什么扫描必须有两棵树（up-sweep 与 down-sweep）而不仅仅是一棵。

以加法为例（讲义原例）：

```
text 输入 x : 3 1 7 0 4 1 6 3 inclusive scan : 3 4 11 11 15 16 22 25 y[i] = x[0] 到 x[i] 的和 exclusive scan : 0 3 4 11 11 15 16 22 y[i] = x[0] 到 x[i-1] 的和 ^ 单位元 0：整体右移一格，空出的位置补单位元
```

算子的推广：只要满足**结合律**，扫描就成立。加法、乘法、`min`、`max`、按位与/或、矩阵乘法、字符串拼接、以及用户自定义的「(最大值, 出现次数)」这样的复合算子都可以。注意扫描**不要求交换律**（commutative）：`x0 ⊕ x1 ≠ x1 ⊕ x0` 时扫描依然有定义，只是结果的顺序被固定了；而归约通常要求结合律 + 交换律，因为归约可以不按顺序合并。这也是「求和」这个例子会掩盖掉的一个细节：`max` 扫描和 `min` 扫描在 GPU 上大量用于「谓词前缀」类问题（如 stream compaction 里判断某个元素是否是它所在段的第一个满足条件的元素）。

- **直观解释（"它是什么？"）**：把扫描想成**超市收银台后面的传送带累计金额**。如果把每个人的消费额排队，想知道「第 i 个人结完账时，总营业额是多少」，那就是 inclusive scan；想知道「第 i 个人开始结账前，前面已经收了多少」，那就是 exclusive scan。两者的差别仅仅是「算不算自己这一笔」。再换个角度：归约像是问「这场接力赛最后总用时多少」，扫描则是问「每一棒交接时秒表上分别显示多少」——后者显然携带了更多信息，也需要更多记录工作。
- **架构/机制图解**：扫描与归约的数据流对比，以及 GPU 上的两阶段结构。

```
text 归约 reduction：只保留最后一个结果，并行度逐层减半（越来越窄） x0 x1 x2 x3 x4 x5 x6 x7 \ / \ / \ / \ / + + + + 第 1 层：4 个结果 \ / \ / + + 第 2 层：2 个结果 \ / \ / \ / + 第 3 层：1 个结果 = Σx0..x7 | 总和 (1 个值) 扫描 scan：所有中间结果都要留，还要再分发回去（两个阶段） 阶段一 up-sweep （归约） 阶段二 down-sweep（分发） x0 x1 x2 x3 x4 x5 x6 x7 [3 4 7 11 4 5 6 25] <- 根已经置 0 \ / \ / \ / \ / T[7]=0 后逐层把「左侧」往右下推 + + + + \ / \ / down-sweep 结束时： + + [0 3 4 11 11 15 16 22] \ / = exclusive scan \ / + | Σx0..x7 ==> down-sweep 复用这些部分和
```

性能特征与具体数字（**A100, sm_80**：108 个 SM，1.41 GHz，1555 GB/s，每 SM 64 个 FP32 lane，故 FP32 加法率 ≈ 108×64×1.41×10^9 ≈ 9.75×10^{12} 次/秒，即 19.5 TFLOPS（FMA 计 2 FLOP））：扫描每处理一个元素至少要读 4 字节、写 4 字节，共 8 字节；而**顺序扫描**每元素只做 1 次加法。因此一次完整扫描的算术强度只有 1/8 次加法每字节，离 A100 的脊点 `9.75e12 / 1555e9 ≈ 6.27 次加法/字节` 差了 50 倍——扫描从定义上就是一个**内存/带宽受限**的问题，这个定性后面会反复用到。共享内存访问延迟约 20–30 cycles，全局内存约 400–800 cycles，L2 约 200 cycles，`__syncthreads()` 在满占用（2048 线程/SM）下每次约 20–40 cycles。

2. 扫描的典型应用（Applications of Scan）

- **定义与目的**：扫描的价值在于它是「并行原语」：Blelloch 在 1989 年的论文里论证了「scan 的耗时不超过一次并行内存引用、且比任意访存模式更容易实现」，并指出很多并行算法都可以用扫描来描述。ECE408 讲义列出的应用包括：**基数排序（radix sort）**、快速排序（quicksort）的分区、**字符串比较（string comparison）**、词法分析（lexical analysis）、**流压缩（stream compaction）**、**多项式求值（polynomial evaluation）**、求解递归式（solving recurrences）、树操作（tree operations）、**直方图（histograms）**；此外还有营地分配、农贸市场摊位分配、给并行线程分配内存、给通信信道分配缓冲区等等。这些应用的共同结构是：**先用扫描求出「每个元素应该在哪个位置」，再按位置搬数据**。
- **直观解释（"它是什么？"）**：讲义给的经典类比是 **「100 英寸的三明治」**：你订了一条 100 英寸长的三明治给 10 个人吃，每个人想吃的长度是 `[3 5 2 7 28 4 3 0 8 1]` 英寸。问题是怎么快速切？顺序切就是「量 3 英寸切一刀，再由同一个人接着量 5 英寸切第二刀」，一个人切完下一个人才能开始。用扫描则是：先把所有需求做成前缀和 `[3, 8, 10, 17, 45, 49, 52, 52, 60, 61]`，这串数字直接就是**每一刀应该落在尺子上的刻度**，10 个人可以同时量自己的那一刀；最后总长 100 减去前缀和的最大值 61，还剩 **39 英寸**。这就是 exclusive scan 的用途——exclusive scan 给出的正是「我前面的人一共吃掉多少」，也就是「我该从哪一寸开始下刀」。

`text 需求(英寸) : 3 5 2 7 28 4 3 0 8 1 刀口刻度 : 3 8 10 17 45 49 52 52 60 61 <- inclusive scan 起始位置 : 0 3 8 10 17 45 49 52 52 60 <- exclusive scan | | | | | | | | | | 三明治: 0----3----8---10---17--------45---49---52---52--60--61----100 剩余 : <- 39 英寸`

流的压缩（stream compaction）是最常用的一种：给定数组和谓词 `p(x)`（如「x 是正数」），要输出所有满足条件的元素、并保持相对顺序。做法是：① 对谓词结果做 exclusive scan（`1` 表示保留），得到每个元素的目标下标；② 保留元素写到 `out[idx]`，其余丢弃。第二阶段的写是**完全合并的连续写**，这就是扫描的价值——它把「不规则的、需要串行计数的写入」变成了「一次连续写」。**基数排序**则是对每一位做一次「按位分流 + 扫描定位」，每一位的定位就是一次分段扫描。

- **架构/机制图解**：流压缩的两阶段数据流，以及它在 GPU 上的内存行为。

`text 输入 X : [5 -2 7 0 3 -1 9] 谓词 p : [1 0 1 0 1 0 1] (>0 保留) exclusive scan p : [0 1 1 2 2 3 3] <- 每个保留元素的写入下标 scatter : out[0]=5, out[1]=7, out[2]=3, out[3]=9 输出 Y : [5 7 3 9] GPU 上的关键点： * 谓词与扫描都发生在片上（寄存器 / 共享内存），只有最后一步「按扫描结果写回全局内存」需要动 DRAM，且写地址是连续递增的 -> 合并写 * exclusive scan 在这里比 inclusive scan 更自然： exclusive 的值 = 「我前面有多少个被保留的元素」= 我的目标下标`

性能特征：扫描本身是纯带宽问题，但它的**下游**（如压缩、排序的 scatter）往往是随机访问的。因此工程上的准则是：**扫描一次、复用多次**——例如基数排序对 32 位 key 做 4 趟 8 位扫描，每趟都只需要 O(N) 的扫描输出，却能替换掉本来会串行化的计数过程。

3. Hillis–Steele（Kogge–Stone）扫描算法

- **定义与目的**：Hillis–Steele 扫描（在硬件加法器设计中称为 **Kogge–Stone tree**，1970 年代由 IBM 的 Peter Kogge 与 Harold Stone 提出）是最容易在 GPU 上实现的并行扫描：**每个输出元素都看成「它自己以及它前面所有元素的归约」，而不同输出元素之间共享这些部分和**。算法只做一件事：让每个线程反复把「距离自己 stride 个位置」的值加到自己身上，stride 从 1 开始每轮翻倍，直到 stride ⩾ n：

`cpp for (int stride = 1; stride < n; stride *= 2) if (t >= stride) T[t] += T[t - stride]; // 原地版的核心一行`

它的目的是**降低延迟（latency）**：步数只有 `log2(n)`，在 n = 1024 时是 10 步，n = 2^{20} 时是 20 步。代价是工作量：第 stride 轮有 `n - stride` 个线程活跃，总加法次数为

`text Σ_{k = 0 .. log2(n)-1} (n - 2^k) = n·log2(n) - (n - 1) = O(n log n)`

以 n = 1024 为例：`1024×10 - 1023 = 9217` 次加法，而顺序扫描只需要 1023 次——**多做了 9 倍的工作**。n = 1,048,576（2^{20}）时 `log2(n) = 20`，多做的倍数约 19 倍（讲义原话：a factor of log(n) hurts: 20x for 1,000,000 elements）。所以这个算法**不是 work-efficient（工作高效）的**：所谓 work-efficient，是指并行算法的总操作数与最优串行算法处于同一量级（常数倍以内）。讲义的建议是把它用在「块内」这种 n ⩽ 1024 的小规模上，此时多做的 10 倍工作可以被大量并行的 ALU 吞吐吃掉。

- **直观解释（"它是什么？"）**：把它想成**一场「逐级扩散」的传话游戏**。第一轮：每个人把自己和左边紧邻那个人的数字相加；第二轮：每个人把自己和左边隔一个人的数字相加；第三轮：隔三个人；第四轮：隔七个人；第 k 轮隔 (2^(k–1) − 1) 个人。第 k 轮之后，每个人手里拿着的正好是「从自己往左连续 2^k 个元素的和」。因为 `2^k` 逐轮翻倍，`log2(n)` 轮之后每个人手里就是完整的前缀和。这里的关键直觉是：**前一轮已经算好的部分和被重复利用**，而不像朴素算法那样每个线程从头加一遍。现实类比：这就像公司里逐级汇报——第一轮「我和我的直接同事合并数据」，第二轮「我和隔壁小组的合并结果合并」，第三轮「我和另一层楼的合并结果合并」，每轮参与合并的规模翻倍，所以只要 `log2(人数)` 轮就能得到全公司的累计数据。
- **架构/机制图解**：以讲义原例 `x = [3 1 7 0 4 1 6 3]`（n = 8）为例，双缓冲版本（T0/T1 交替做输入与输出）的完整数据流：

`text T0 (初始) : 3 1 7 0 4 1 6 3 -- stride = 1 : T1[t] = T0[t] + T0[t-1] (t >= 1 的线程活跃，7 个) T1 = 3 4 8 7 4 5 7 9 -- stride = 2 : T0[t] = T1[t] + T1[t-2] (t >= 2 活跃，6 个) T0 = 3 4 11 11 12 12 11 14 -- stride = 4 : T1[t] = T0[t] + T0[t-4] (t >= 4 活跃，4 个) T1 = 3 4 11 11 15 16 22 25 -- 结果在交换后的 src(=T1) 中： [3 4 11 11 15 16 22 25] = inclusive scan`

每轮的性质：* **活跃线程数** = `n - stride`，即 7、6、4、2（n = 8 时），逐轮减少；在 n = 1024 时是 1023、1022、1020、1016、1008、992、960、896、768、512。注意：因为活跃线程是**后缀**（`t >= stride`），当 `stride ⩾ 32` 时「整 warp 要么全活跃要么全不活跃」，只有 `stride < 32`（即前 5 轮）才会出现 warp 内部的分歧（divergence）。* **每轮的共享内存访问**：每个活跃线程读 `src[t]` 与 `src[t-stride]`、写 `dst[t]`，三次访问的地址在 warp 内都是**连续的**——这是它能做到零 bank conflict 的根本原因（见概念 7）。* **同步**：原地版本每轮需要**两道** `__syncthreads()`（见概念 4），双缓冲版本每轮只需要**一道**。

GPU 上的执行映射（A100）：block = 1024 线程 → 32 个 warp → 4 个 SM 子分区（每个 SM 最多驻留 2048 线程 / 64 warp / 2 个这样的 block），32 个 warp 被分派到 4 个 warp scheduler，每轮 10 次 `__syncthreads()` 意味着这 32 个 warp 每轮都要全部到达屏障，屏障本身的延迟约 20–40 cycles，所以**纯同步开销约 10 × 30 = 300 cycles**；相比之下第 1 轮的一次全局内存读取就要 400–800 cycles，因此在小 block 内，同步开销是可以接受的，而真正的瓶颈是「每个线程只搬 4 字节」这一件事。

4. 双缓冲（Double Buffering）

- **定义与目的**：双缓冲是**用空间换同步**的技术：为同一份逻辑数据准备两份物理存储 `T0` 与 `T1`，第 k 轮以其中一份为输入（source）、另一份为输出（destination），第 k+1 轮把角色互换（ping-pong）。它要解决的问题是原地（in-place）更新的**数据竞争（data race）**：在 `T[t] += T[t-stride]` 这一行里，同一轮中线程 t 会读 `T[t-stride]`，而线程 `t-stride`（当 `t ⩾ 2·stride` 时它自己也是活跃线程）会**写** `T[t-stride]`。这个写发生在读之前还是之后，取决于硬件的调度顺序，编译器与硬件都不保证——这就是为什么讲义称单屏障的实现「has a data race condition」，结果是**不确定的（undefined）**。

更形式化地说，原地版本存在两类依赖：* **RAW（Read-After-Write，真依赖）**：线程 t 必须读到线程 `t-stride` 在**本轮**写下的新值，但在单屏障版本中它可能在自己的线程里先执行，读到的还是上一轮的旧值，于是结果偏小。* **WAR（Write-After-Read，反依赖）**：线程 t 还要写出 `T[t]`，而这个位置正是线

程 `t+stride` 本轮要读的「旧值」；如果 t 抢先写，`t+stride` 就永远读不到旧值了。

双缓冲把「读的地址」和「写的地址」彻底分开（读 `src`、写 `dst`），于是**同一轮内没有任何线程会写别人要读的位置**，一道屏障（保证上一轮的写全部可见）就够了。

- **直观解释（"它是什么？"）**：像**在白板上抄写**：如果只有一块白板，你一边擦一边写，别人就可能读到擦了一半的内容；于是你规定「所有人只能读这块白板、只能写到另一块白板上」，写完之后大家交换角色、擦掉旧的那块。又像厨师**同时用两个案板**：一个案板放已经处理好的食材（输出），另一个放还没处理的（输入），每完成一道工序就交换两个案板的用途——不需要停下来等所有人（第二道屏障），因为「读」和「写」从来不在同一块案板上。
- **架构/机制图解**：单缓冲需要两道屏障 vs 双缓冲只要一道屏障。

```
text 【方案 A：原地 + 两道 __syncthreads()（讲义 p.15 的写法)】 第 k 轮： __syncthreads(); // 屏障 1：确认输入的上一轮结果已就绪 float temp = T[t] + T[t − stride]; // 先把结果算进寄存器（不再碰共享内存） __syncthreads(); // 屏障 2：确认所有人都读完了旧值 T[t] = temp; // 现在才能安全地覆盖 代价：每轮 2 次屏障，10 轮 = 20 次屏障 【方案 B：双缓冲 + 一道 __syncthreads()（推荐)】 __shared__ float T0[BLOCK]; __shared__ float T1[BLOCK]; float *src = T0; float *dst = T1; // src = 输入，dst = 输出 第 k 轮： __syncthreads(); // 屏障：确认 src 中本轮的输入已就绪 dst[t] = src[t] + ((t >= stride) ? src[t − stride] : 0.0f); swap(src, dst); // 指针互换，0 成本的 ping-pong 代价：每轮 1 次屏障，10 轮 = 10 次屏障 第 0 轮：src = T0（输入）dst = T1（输出）第 1 轮：src = T1（输入）dst = T0（输出）第 2 轮：src = T0（输入）dst = T1（输出）第 3 轮：src = T1（输入）dst = T0（输出）规律：每轮结束互换 src 与 dst；BLOCK_SIZE = 1024 时共 10 轮（stride 依次为 1, 2, 4, 8, 16, 32, 64, 128, 256, 512）10 是偶数 => 循环结束时 src 指回 T0；但代码不应依赖这个事实， 而应始终用「交换后的 src」这一个变量名去写回结果，才最不容易错。
```

一个极易错的细节：讲义特意强调 "Because of double-buffering, this copy must be done actively!"——当线程 `t < stride` 时它不参与加法，但**必须把自己的值复制到 `dst[t]`**。因为下一轮的角色互换了，如果这一轮 `dst` 中没有该位置的值，那份数据就丢了。上面的代码用 `((t >= stride) ? src[t − stride] : 0.0f)` 这一写法（单位元 0）自动完成了这件事：对加法而言，不加任何东西等于加 0，效果与「复制」完全等价；换成 `max` 算子时要改成「加负无穷」，换成乘法要改成「乘 1」——**用算子的单位元填空白**是通用写法。

性能特征：双缓冲的代价是**共享内存用量翻倍**。block = 1024、float 时两个数组 = 8 KB；A100 每 SM 有 164 KB 共享内存，2 个 block 只占 16 KB，远不是瓶颈（对比：如果每 block 用到 40 KB，就只能驻留 4 个 block，占用率会掉到 50%）。收益是屏障次数减半，并且交换指针是纯粹的寄存器操作（0 个时钟周期）。

5. Blelloch 工作高效扫描（up-sweep / down-sweep）

- **定义与目的**：Blelloch 扫描用**一棵平衡二叉树**的思路（讲义称之为 balanced trees 模式：树不是真的数据结构，只是「决定每个线程每一步做什么」的概念）把工作量降到 `O(N)`：
- **up-sweep（归约阶段，leaves → root)**：从叶子向根，每个内部节点存放其子树的和。第 d 层让 `index = (t+1)·2·stride − 1` 的节点加上它左边 `stride` 处的兄弟节点，`stride` 依次取 2 的幂：1、2、4、8，直到 n/2。这一步做的事情和「并行归约树」一模一样，总共 `n−1` 次加法，根节点最后存着总和。
- **down-sweep（分发阶段，root → leaves)**：先把**根置 0**（这一步是关键：它把「包含式」变成「排除式」），然后从上层往下走，每步执行

  `cpp float left = T[index − stride]; T[index − stride] = T[index]; T[index] = left + T[index];`

  其含义是「把父节点的前缀值推到左孩子，把左孩子的旧值加上父节点的值留给右孩子」。走完 `stride` 从 n/2 每次减半、直到 1 的全部步骤之后，数组里就是 **exclusive scan**。

总工作量 = up-sweep 的 `n−1` 次加法 + down-sweep 的 `n−1` 次加法 = `2(n−1)`，即**每元素 2 次加法，O(N) 工作量**，最多只是高效串行算法的两倍；步数是 `2·log2(n)`（讲义对 Brent-Kung 变体的统计是 `2(n−1) − log2(n)` 次加法，同样是 O(N)）。讲义给出的判据很直接："The benefit of parallelism can easily overcome the 2× work when there is sufficient hardware"。

课程讲义里的 Brent-Kung 变体与 Blelloch 经典版的唯一区别在 down-sweep 的起点：讲义版本**不把根置 0**，down-sweep 从 `stride = BLOCK_SIZE/2`（而不是 `n/2`）开始，因此每步都靠「根已经是完整的包含式结果」这个事实，最后得到的是 **inclusive scan**（讲义 p.27–29 明确写 "Inclusive Post Scan Step"）。两者都是 `O(N)` 工作量，选择哪种只取决于你需要的输出形式；需要 exclusive scan（例如做流压缩求目标下标）时，把根置 0 更自然。

- **直观解释（"它是什么？"）**：这是一次**「先上山、再下山」的旅行**。上山（up-sweep）时每个内部节点只负责回答「我这一片的总和是多少」，越往上管的范围越大，到山顶就知道了全局总和。下山（down-sweep）时每个节点从父节点那里领到一份**「我左边所有元素的总和」**，然后再把这份情报往下传：左孩子拿到的就是父节点给的那份；右孩子拿到的是「父节点给的那份 + 左孩子的总和」。到叶子时，每个叶子手里拿到的恰好就是「我左边所有元素的和」——这正是 exclusive scan。现实类比：公司做年度预算分配，总部先自下而上汇总各部门的总额（上山），然后自上而下把「你前面所有部门已经分掉的预算」逐级下传（下山），最后每个员工都知道自己前面分掉了多少钱。
- **架构/机制图解**：n = 8、`x = [3 1 7 0 4 1 6 3]` 的完整过程（数值已逐位验证）。

```
text 【up-sweep：stride = 1, 2, 4，共 log2(n) = 3 步，n−1 = 7 次加法】 初始 T = [ 3 1 7 0 4 1 6 3 ] stride=1 index=(t+1)*2−1 = 1,3,5,7 T[index] += T[index−1] T = [ 3 4 7 7 4 5 6 9 ] stride=2 index=(t+1)*4−1 = 3,7 T[index] += T[index−2] T = [ 3 4 7 11 4 5 6 14 ] stride=4 index=(t+1)*8−1 = 7 T[index] += T[index−4] T = [ 3 4 7 11 4 5 6 25 ] <− T[7] = Σ 全部 = 25 【根置 0：inclusive −> exclusive】 T = [ 3 4 7 11 4 5 6 0 ] 【down-sweep：stride = 4, 2, 1，共 log2(n) = 3 步，n−1 = 7 次加法】 stride=4 index = 7: left=T[3]=11; T[3]=T[7]=0; T[7]=11+0=11 T = [ 3 4 7 0 4 5 6 11 ] stride=2 index = 3,7: index=3: left=T[1]=4; T[1]=T[3]=0; T[3]=4+0=4 index=7: left=T[5]=5; T[5]=T[7]=11; T[7]=5+11=16 T = [ 3 0 7 4 4 11 6 16 ] stride=1 index = 1,3,5,7: index=1: left=T[0]=3; T[0]=T[1]=0; T[1]=3+0=3 index=3: left=T[2]=7; T[2]=T[3]=4; T[3]=7+4=11 index=5: left=T[4]=4; T[4]=T[5]=11; T[5]=4+11=15 index=7: left=T[6]=6; T[6]=T[7]=16; T[7]=6+16=22 T = [ 0 3 4 11 11 15 16 22 ] <− exclusive scan 【线程映射】BLOCK_SIZE 个线程，每线程负责 2 个叶子；第 t 个线程在每一步 负责的父节点是 index = (t+1)*stride*2 − 1，索引非法时（index >= n）不做任何事。 up-sweep 与 down-sweep 的 index 公式完全相同，只是 stride 方向相反。
```

性能特征（n = 2048，BLOCK_SIZE = 1024 线程）： * 加法总数 = `2(n-1) = 4094`，即**每元素 2.0 次加法**；同一个 n 下 Hillis-Steele 需要 `n·log2(n) − (n−1) = 2048×11 − 2047 = 20481` 次，即每元素 10.0 次。所以 Blelloch 的工作量是 Hillis-Steele 的 **1/5**。 * 步数 = `2·log2(n) = 22` 步，Hillis-Steele 只要 11 步 —— **Blelloch 的步数是它的两倍**，而每步都要一次 `__syncthreads()`，因此在 barrier 延迟占主导的小规模场景里 Blelloch 更慢。 * 活跃线程数按 `n/(2·stride)` 递减：1024、512、256、128、64、32、16、8、4、2、1 —— 这一半的步骤里并行度极低（最后 6 步总共只有 63 个线程干活），浪费了 warp 资源；而 Hillis-Steele 在第 k 轮有 `n − stride` 个活跃线程，始终接近满并行。 * Brent-Kung（讲义版）与 Kogge-Stone 的取舍：Brent-Kung **只用一半的线程**（每线程负责 2 个元素），但**步数翻倍**。讲义给出的结论很明确："Kogge-Stone is more popular for parallel scan with blocks in GPUs"。

6. 层次化扫描（Hierarchical Scan）

- **定义与目的**：一个 block 的共享内存只有几十 KB（A100 每 SM 164 KB，RTX 4090 每 SM 100 KB），block 内最多 1024 线程，因此**单块扫描最多处理 2048 个 float**，无法处理 Lab 里动辄百万级的数组。层次化扫描（讲义称 hierarchical approach）把「全局扫描」拆成三个步骤，用多个 kernel 接力完成：

```
text ① 每个 block 扫描自己负责的一段元素（段内扫描），并把「本段所有元素的总和」写到全局数组 Sum[blockIdx.x]； ② 对 Sum 数组再做一次并行扫描（它很短： N / 段长 个元素），得到每一段的 exclusive 偏移量； ③ 把偏移量加回每一段的每个元素上，得到全局 inclusive scan。
```

讲义对 Kogge-Stone 的表述是：把 `blockDim.x` 个元素的一段分配给一个 block；Brent-Kung 则因为它每线程处理两个元素，一段是 `2*blockDim.x` 个元素。为什么必须用多 kernel？讲义 p.36 讲得很清楚：**一个 block 的寄存器和共享内存中的数据对其他 block 不可见**；要让数据可见必须写进全局内存，而全局内存的写「until a memory fence（内存栅栏）」才对其他 block 可见，在 CUDA 里这个栅栏最自然的实现方式就是**结束 kernel**——kernel 结束后，它的全局内存写对所有后续 block 可见。因此「块间通信」在 CUDA 中的标准模式就是「写全局内存 → 换 kernel」。（更激进的单 kernel 方案：用 `atomicAdd` 累加块总和 + 全局标志位实现 decoupled look-back，可以在一个 kernel 里完成，但需要小心的内存序与自旋，不属于本讲的要求。）

- **直观解释（"它是什么？"）**：把它想成**「分组报数」**。要让 100 万人依次报出自己的累计编号，先分成 1000 个小组：每组内部各自报数（组内扫描），算出「我们组一共多少人」（组总和）；然后让 1000 个组长排成一列再报一次数，得到「我们组之前一共有多少人」（组偏移）；最后每个组员把自己的组内编号加上本组的偏移，就得到了全局编号。整个过程的并行度始终是「组内人数」，而组间那一层只有 1000 个元素、一次小扫描就完事。现实类比：马拉松比赛按方阵出发，方阵内部先排好序（组内扫描），再根据前面所有方阵的总人数确定本方阵的起始号码（组偏移），最后每个人就知道自己是第几号。
- **架构/机制图解**：三步法的数据流与内存层次。

```
text 全局内存 (DRAM, 1555 GB/s on A100) X[0 .. N-1] ──────────────────────────────────────────────┐ │ │ kernel1: 每个 block 读 TILE=1024 个元素 │ 读 N*4 B v │ 写 N*4 B ┌──────────────────────┐ block 0..nb-1 │ │ 每个 block: │ __shared__ T0/T1 (双缓冲) │ │ load -> 共享内存 │ 10 轮 stride=1..512 │ │ Kogge-Stone 扫描 │ 每轮 1 次 __syncthreads() │ │ store -> 全局内存 │ │ └──────────────────────┘ │ │ │ v v │ Y_local[0..N-1] blockTotals[nb] (nb = N/1024) │ 写 nb*4 B （只是段内前缀和， │ 不是最终答案） │ │ │ kernel2: 一个 block │ 读 nb*4 B v 对 blockTotals 做 │ 写 nb*4 B blockOffsets[nb] exclusive scan │ （每段的全局起始偏移） │ │ │ kernel3: Y[g] += blockOffsets[g/TILE] v │ 读 N*4 B Y[0..N-1] = 全局 inclusive scan │ 写 N*4 B └──────────────────────────────┘ 总 DRAM 流量 = 4N*4 B（X 读一次、Y_local 写一次、Y 再读一次写一次） 理论下限时间 = 4N*4 / 1555e9 ：N = 4M 时 = 67.1 MB / 1555 GB/s = 43.2 us 而「只读一遍写一遍」的绝对下限是 2N*4 B = 33.6 MB -> 21.6 us
```

性能特征与具体数字（A100，N = 4,194,304 = 4 MiB 元素 = 16 MB）： * kernel1：4096 个 block × 1024 线程，每 block 只在共享内存内做 10 轮扫描；DRAM 流量 32 MB（读 16 + 写 16）→ 带宽下限 21.6 μs。 * kernel2：只有一个 block（1024 线程）处理 4096 个段总和，每个线程顺序处理 8 个值 → DRAM 流量只有 32 KB，是**延迟受限**而非带宽受限：一次全局读的延迟 400–800 cycles，加上 10 次 `__syncthreads()`，整个 kernel 的耗时约 1–3 μs。**这就是层次化的代价：为了几 KB 的中间数据要付出一次完整的 kernel 启动与延迟。** * kernel3：又一次流式读写 32 MB，是纯带宽操作。 * 合计：约 4N×4 = 67 MB 的 DRAM 流量，是理论下限 2N×4 B 的**恰好 2 倍**（因为第一遍的中间结果必须先落盘再读回来）。

7. 共享内存 bank conflict 分析（Scan 的访问模式）

- **定义与目的**：共享内存被划分为 **32 个 bank**，每个 bank 宽 4 字节，因此每个「行」是 `32 × 4 = 128` 字节。一个 warp 的 32 个线程如果访问**落在 32 个不同的 bank**（或者访问的是**同一个地址**，此时硬件做广播 broadcast），就能在一个周期内完成；如果两个线程访问**同一个 bank 的不同地址**，就要串行化，称为 **bank conflict（存储体冲突）**。判定公式很简单：地址 `a`（字节）落在 `bank = (a / 4) mod 32`；若一个 warp 内的访问步长为 k 个字（4 字节），冲突度（需要的访问事务数）为

```
text 冲突度 = 32 / gcd(k, 32)  (k = 1 -> 1 无冲突; k = 2 -> 2; k = 4 -> 4; k = 8 -> 8; k = 16 -> 16; k = 32 -> 32)
```

这条规则是分析扫描算法性能的关键，因为**Hillis-Steele 与 Blelloch 的访问步长完全不同**。

- **直观解释（"它是什么？"）**：把共享内存想成**银行里 32 个柜台**，每个柜台一次只能服务一位客户。如果 32 个人分别走向 32 个不同的柜台，一轮就办完；如果 20 个人挤在同一个柜台、其他柜台空着，就要排 20 次队。判定「会不会挤」只看一件事：**同一时刻这 32 个人的目标柜台号是否有重复**。注意「同一个人被问两次」不算冲突——柜台可以一次性把同一条信息广播给所有问同一个地址的人。
- **架构/机制图解**：Hillis-Steele 与 Blelloch 在共享内存上的访问模式对比（BLOCK = 1024 线程，float 数组，n = 2048）。

```
text 【Hillis-Steele：warp 内访问 32 个「连续」字，天然无冲突】 第 k 轮 stride = d： 以 warp 0（t = 0..31）、stride d = 4 为例，把每个线程访问的「字地址」写出来： 线程 t : 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 读 src[t] : 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 其 bank : 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 读 src[t-d] : - - - - 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 其 bank : - - - - 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 写 dst[t] : 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 其 bank : 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 => 每次访问涉及的 bank 号都互不相同（0..31 各出现一次）=> 冲突度恒为 1 理由：任意 32 个连续字一定覆盖全部 32 个 bank，而与 d 的取值无关。 注意 d < 32 时 warp 0 内部有线程不活跃（上表中标 "-"），但它们不访问内存， 所以不影响「活跃访问落在不同 bank」这一结论。 【Blelloch / Brent-Kung：树形索引，warp 内
```

步长 = 2*stride，冲突严重】 index = (t+1)*stride*2 − 1，相邻线程的地址差 = 2*stride stride=1 : warp 0 访问地址 1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31, 33,35,37,39,41,43,45,47,49,51,53,55,57,59,61,63 bank = 1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31, 1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31 → 只用到 16 个奇数 bank，每个被 2 个线程命中 → 冲突度 2 stride=2 : warp 0 访问地址 3,7,11,15,19,23,27,31,35,39,43,47,51,55,59,63, 67,71,75,79,83,87,91,95,99,103,107,111,115,119,123,127 地址差 4 → 只用到 8 个 bank（3,7,11,15,19,23,27,31 循环使用） → 冲突度 4 stride=4 : 地址差 8 → 只用到 4 个 bank → 冲突度 8 stride=8 : 地址差 16 → 只用到 2 个 bank → 冲突度 16 stride=16 : 地址差 32 → 只用到 1 个 bank → 冲突度 32（最坏！）

下面两张表是**逐 bank 枚举**算出来的（可以照着公式复核）。

表 A：Hillis–Steele 每一步的活跃线程数与共享内存事务数（n = 1024，float）

stride d	活跃线程 `t ≥ d`	活跃 warp	读 `src[t]` 事务	读 `src[t−d]` 事务	冲突度
1	1023	32	32	32	1（无冲突）
2	1022	32	32	32	1
4	1020	32	32	32	1
8	1016	32	32	32	1
16	1008	32	32	32	1
32	992	31	31	31	1
64	960	30	30	30	1
128	896	28	28	28	1
256	768	24	24	24	1
512	512	16	16	16	1
合计			**289**	**289**	

写 `dst[t]` 同样是 289 次事务。所以整个 10 轮扫描的共享内存事务数约 `289×3 = 867` 次（每次事务 = 一个 warp 一次 128 字节访问），即**每元素 0.85 次事务**。因为「任意 32 个连续字必然覆盖 32 个不同的 bank」，Hillis–Steele 在**所有** stride 下都无 bank conflict——这一点常被误传成「stride 小的时候有冲突」，只有当实现改成「每线程处理 2 个及以上元素、且用跨步索引」时才会出现冲突。

表 B：Blelloch / Brent–Kung up-sweep 的 bank 冲突（n = 2048，BLOCK = 1024，float）

stride s	地址步长 2s	活跃线程	活跃 warp	触及 bank 数	冲突度	事务数/次访问
1	2	1024	32	16	2	64
2	4	512	16	8	4	64
4	8	256	8	4	8	64
8	16	128	4	2	16	64
16	32	64	2	1	**32**	64
32	64	32	1	1	**32**	32
64	128	16	0.5	1	16	16
128	256	8	0.25	1	8	8
256	512	4	—	1	4	4
512	1024	2	—	1	2	2
1024	2048	1	—	1	1	1

这张表揭示了一个非常反直觉的事实：**冲突度随 stride 增大而上升（最高 32 路），但活跃 warp 数同步下降，两者相乘后「每一步的事务数」在 stride ≤ 16 的范围内几乎恒定为 64**，之后才开始下降。整段 up-sweep 的每次访问合计 `64×5 + 32+16+8+4+2+1 = 383` 次事务；因为每个线程有 3 次共享内存访问（读 `T[index]`、读 `T[index-stride]`、写 `T[index]`），up-sweep 总共约 `3×383 = 1149` 次事务，而**如果没有冲突只需要 `3×2047/32 = 192` 次**——冲突让它膨胀了 **6 倍**。down-sweep 的索引模式完全相同（4 次访问：2 读 2 写），事务数 `4×383 = 1532`，无冲突时只需 256，同样是 6 倍。于是在 n = 2048 上：Blelloch 的共享内存事务是 `2681/2048 = 1.31 次/元素`，而 Hillis–Steele 只有 `0.85 次/元素`——**Blelloch 的「工作量只有 1/5」的优势，被 bank conflict 吃掉了大半**。

如果确实要用树形扫描，可以这样减轻冲突： * **warp shuffle 替代前 5 级**：stride ≤ 16 的级别只涉及 warp 内部通信，用 `__shfl_up_sync(mask, val, stride)` 完成，完全不碰共享内存（这同时省掉了 warp 内的 bank 访问与部分 `__syncthreads()`）。 * **向量化访问**：让每个线程处理 2 个连续元素并用 `float2`（8 字节）访问时，一个 warp 会被拆成两个 half-warp，每个 half-warp 的 16 个 `float2` 覆盖 32 个 bank，冲突消失。 * **skewed / padding 布局**：把逻辑下标 `i` 映射到物理地址 `i + i/32`。对 stride = 16（地址差 32）的情形，物理地址差变成 33，`gcd(33,32) = 1` → 冲突完全消除；但对 stride

= 32（地址差 64）的情形，物理地址差是 66，`gcd(66,32) = 2` → 仍有 2 路冲突。也就是说 **padding 只能部分缓解**，不能根治树形扫描的跨步访问，而且要给每次访问加上额外的地址算术。 * **换算法**：直接用 Hillis–Steele（访问天然连续、无冲突），或用 CUB 的 `DeviceScan`。

另外注意：默认的 4 字节访问粒度下，bank 冲突只影响**共享内存**；而扫描的**全局内存**访问（连续线程读连续地址）是完美合并的——32 线程 × 4 字节 = 128 字节 = 一条 cache line = 一次 transaction。

代码示例与性能分析

下面三个程序层层递进：**示例 1** 说明「并行化不等于高效」（O(N^2) 的反面教材）；**示例 2** 给出单 block 的双缓冲 Hillis–Steele 扫描，并实测它与带宽下限的差距；**示例 3** 给出工作高效的 Blelloch 版本和能处理任意长度 N 的层次化三步法，并用 `cudaEvent` 分别测量三个 kernel。三个程序都在 A100（sm_80）上以 `-arch=sm_80` 编译，性能数字按 A100 的规格（108 SM、1.41 GHz、1555 GB/s、每 SM 64 个 FP32 lane）推算给出，每一处都附带计算公式。

> **示例 1：串行扫描 + 「每线程独立算前缀和」的 O(N^2) 并行版**

```
// 文件: scan_naive.cu
// 编译: nvcc -O3 -arch=sm_80 scan_naive.cu -o scan_naive
// 运行: ./scan_naive 65536        (命令行参数为元素个数 N, 默认 65536)
//
// 内容:
//   1) CPU 串行 inclusive scan —— O(N) 工作量, 作为正确性基准;
//   2) "每线程独立累加自己前面所有元素"的朴素并行版 —— O(N^2) 工作量。
// 结论: 并行化很容易, 但并行 != 高效。

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <chrono>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define THREADS 256

// ---------------- CPU 参考实现: 串行 inclusive scan ----------------
static void cpu_inclusive_scan(const float *x, double *y, int n)
{
    double running = 0.0;
    for (int i = 0; i < n; ++i) {
        running += (double)x[i];
        y[i] = running;
    }
}

// ---------------- 朴素并行版: 每个线程独立算出 y[i] ----------------
// 线程 i 需要 i+1 次加法, 总加法次数 = sum_{i=1..N} i = N(N+1)/2 = O(N^2)
__global__ void naive_inclusive_scan_kernel(const float *X, float *Y, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;
    float sum = 0.0f;
    for (int j = 0; j <= i; ++j) {   // 串行依赖链: sum 是循环携带依赖
        sum += X[j];
    }
    Y[i] = sum;
}

static bool verify(const float *gpu, const double *ref, int n)
{
    int    bad   = 0;
    double worst = 0.0;
    for (int i = 0; i < n; ++i) {
        double d = fabs((double)gpu[i] - ref[i]) / (1.0 + fabs(ref[i]));
        if (d > worst) worst = d;
        if (d > 1e-4) ++bad;
    }
    printf("校验: 最大相对误差 = %.3e, 不匹配元素 = %d -> %s\n",
           worst, bad, (bad == 0) ? "PASS" : "FAIL");
    return bad == 0;
}

int main(int argc, char **argv)
{
    const int n = (argc > 1) ? atoi(argv[1]) : 65536;
    const size_t bytes = (size_t)n * sizeof(float);

    float  *h_x   = (float  *)malloc(bytes);
    float  *h_y   = (float  *)malloc(bytes);
    double *h_ref = (double *)malloc((size_t)n * sizeof(double));
    if (!h_x || !h_y || !h_ref) { fprintf(stderr, "host malloc failed\n"); return EXIT_FAILURE; }

    srand(1234);
    for (int i = 0; i < n; ++i) h_x[i] = (float)((rand() % 2001) - 1000) / 1000.0f;
```

```
    // ---------- 1. CPU 串行扫描 ----------
    auto t0 = std::chrono::high_resolution_clock::now();
    cpu_inclusive_scan(h_x, h_ref, n);
    auto t1 = std::chrono::high_resolution_clock::now();
    double cpu_ms = std::chrono::duration<double, std::milli>(t1 - t0).count();

    // ---------- 2. GPU 朴素并行扫描 ----------
    float *d_x = NULL, *d_y = NULL;
    CUDA_CHECK(cudaMalloc((void **)&d_x, bytes));
    CUDA_CHECK(cudaMalloc((void **)&d_y, bytes));
    CUDA_CHECK(cudaMemcpy(d_x, h_x, bytes, cudaMemcpyHostToDevice));

    dim3 block(THREADS);
    dim3 grid((n + THREADS - 1) / THREADS);

    cudaEvent_t e0, e1;
    CUDA_CHECK(cudaEventCreate(&e0));
    CUDA_CHECK(cudaEventCreate(&e1));

    naive_inclusive_scan_kernel<<<grid, block>>>(d_x, d_y, n);   // 预热
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    const int reps = 5;
    CUDA_CHECK(cudaEventRecord(e0));
    for (int r = 0; r < reps; ++r)
        naive_inclusive_scan_kernel<<<grid, block>>>(d_x, d_y, n);
    CUDA_CHECK(cudaEventRecord(e1));
    CUDA_CHECK(cudaEventSynchronize(e1));
    float gpu_ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&gpu_ms, e0, e1));
    gpu_ms /= (float)reps;

    CUDA_CHECK(cudaMemcpy(h_y, d_y, bytes, cudaMemcpyDeviceToHost));
    bool ok = verify(h_y, h_ref, n);

    // ---------- 3. 性能数字 ----------
    double adds     = (double)n * ((double)n + 1.0) / 2.0;         // N(N+1)/2
    double adds_per_s = adds / (gpu_ms * 1e-3);
    double ideal_bytes = 2.0 * (double)n * sizeof(float);          // 读一遍 + 写一遍
    double gbs        = ideal_bytes / (gpu_ms * 1e-3) / 1e9;

    printf("\nN = %d  (%.2f MB)\n", n, (double)bytes / 1048576.0);
    printf("CPU 串行 inclusive scan : %8.3f ms\n", cpu_ms);
    printf("GPU 朴素 O(N^2) 版本    : %8.3f ms\n", gpu_ms);
    printf("工作总量 (N(N+1)/2)     : %.4e 次加法\n", adds);
    printf("GPU 有效加法吞吐        : %.3f Gadd/s\n", adds_per_s / 1e9);
    printf("若只看“读一遍+写一遍”的字节吞吐: %.1f GB/s (远低于 A100 的 1555 GB/s)\n",
           gbs);
    printf("加速比 CPU/GPU          : %.3fx   (小于 1 表示并行版比串行版更慢！)\n",
           cpu_ms / gpu_ms);
    printf("并行版工作量是串行版的   : %.0f 倍 (N(N+1)/2 除以 N)\n", ((double)n + 1.0) / 2.0);
    printf("%s\n", ok ? "结果正确" : "结果错误");

    CUDA_CHECK(cudaFree(d_x));
    CUDA_CHECK(cudaFree(d_y));
    CUDA_CHECK(cudaEventDestroy(e0));
    CUDA_CHECK(cudaEventDestroy(e1));
    free(h_x); free(h_y); free(h_ref);
    return 0;
}
```

- **【代码做什么？】**
- 按命令行参数 `N`（默认 65536）用 `rand()` 生成 `[-1, 1]` 区间内的随机数组 `h_x`。
- 在主机上跑一次**串行 inclusive scan**（用 `double` 累加避免误差干扰校验），用 `std::chrono` 计时，得到 CPU 基准时间与双精度的参考结果 `h_ref`。
- `cudaMalloc` 两块显存（输入 `d_x`、输出 `d_y`），再用 `cudaMemcpy(d_x, h_x, bytes, cudaMemcpyHostToDevice)` 把输入拷上 GPU。
- 启动 `naive_inclusive_scan_kernel<<<N/256, 256>>>`：**每个线程负责一个输出元素** `y[i]`，用 `for (j = 0; j <= i; ++j) sum += X[j];` 把它前面的元素**全部重新加一遍**。
- 先跑一次预热，再用 `cudaEventRecord` / `cudaEventElapsedTime` 测 5 次平均耗时（`cudaEventSynchronize` 之后才能读时间，否则拿到的是「启动完成」而不是「执行完成」）。
- 把结果拷回主机，与双精度参考逐元素比较（归一化相对误差 < 1e-4），打印 `PASS` / `FAIL`。
- 打印工作量 `N(N+1)/2`、有效加法吞吐（Gadd/s）、按「读一遍 + 写一遍」计算的字节吞吐，以及 GPU/CPU 加速比。

- `cudaFree` / `cudaEventDestroy` / `free` 释放全部资源。
- **【并行机制与硬件映射解说】**
- 线程映射：`block = 256` 线程 = **8 个 warp**；`N = 65536` 时需要 `grid = 256` 个 block。A100 每 SM 最多驻留 2048 线程 = 8 个这样的 block，108 个 SM 共可同时驻留 864 个 block，因此 256 个 block **一波就全部上机**，占用率 100%（kernel 不用共享内存，寄存器约 10 个，都不构成限制）。
- warp 与发散：warp 内 32 个线程的 `i` 是连续的。第 `j` 次迭代时，只有 `i ≥ j` 的线程还在循环里，也就是**后缀活跃**——这是「整 warp 逐步退出」的模式，发散只发生在每个 warp 的前几次迭代，代价很小。第 j 次迭代所有活跃线程读的都是**同一个地址** `X[j]`，硬件按**广播**处理，一次 128 字节的 transaction 只用了 4 字节，**合并访问的利用率只有 3%**。
- 寄存器与依赖链：`sum` 是循环携带依赖，一次 FADD 延迟约 **4 cycles**，所以单线程只能每 4 cycles 完成 1 次加法。要填满每 SM 的 64 个 FP32 lane，需要至少 `4 cycles × (64 lane / 32 lane per warp-instr) = 8` 个 warp 同时有 FADD 在流水线里，而本 kernel 有 64 warp/SM 可驻留，**延迟被充分掩盖**——所以它既不是延迟受限，也不是 ALU 吞吐受限。
- 真正的瓶颈是**指令发射带宽**与**工作量**：内层循环每次加法还要配套一条 load、一条下标比较、一条分支，约 5 条线程指令/元素。总指令数 `2.15e9 × 5 ≈ 1.07e10`；A100 的发射能力 = `108 SM × 4 warp-scheduler × 1.41 GHz × 32 线程 = 1.95e13 线程指令/s`，于是 `1.07e10 / 1.95e13 ≈ 0.55 ms`。另外 LSU（load/store 单元）每 SM 32 lane，`2.15e9 / (108×32×1.41e9) ≈ 0.44 ms`，与发射上界同量级。两者叠起来正是 0.6 ms 左右。
- L1/L2/DRAM：数组只有 256 KB，能整块装进 A100 的 40 MB L2，因此重复读取大多命中 L2（约 200 cycles）而不用去 DRAM（400–800 cycles）——**即便如此还是慢**，说明问题不在缓存，而在工作量本身。
- **【性能优化分析】**
- **算术强度**：表面上「极高」——平均每个元素算了 `N/2 = 32768` 次加法，配合 8 字节的 I/O，算术强度高达 `32768 / 8 ≈ 4096` 次加法/字节，远远越过 A100 的 Roofline 脊点 `9.75e12 / 1555e9 = 6.27` 次加法/字节。但这**全是冗余工作**：其中只有 1 次加法是「有用」的，其余 (N/2 − 1) 次都是在重复计算别人已经算过的部分和。
- **与带宽下限的对比**：本问题的最小 DRAM 流量是「读一遍 + 写一遍」= `2 × 65536 × 4 B = 512 KB`，在 1555 GB/s 下只要 `0.34 µs`。实际耗时约 0.6 ms，是理论下限的 **1800 倍**。
- **瓶颈判定**：计算/发射受限（更准确地说：**工作量受限**）。占用率已经是 100%，所以再怎么调 block 大小、加 `__restrict__`、用 `-use_fast_math` 都救不了——**必须换算法**，把 $O(N^2)$ 降到 O(N log N) 或 O(N)。
- 定量结论：CPU 串行版做 65536 次加法约 0.09 ms（每次加法约 1.3 ns，含循环开销），GPU 朴素版做 2.15e9 次加法约 0.6 ms。GPU 的加法率高得多，但工作量多做了 32769 倍，净结果是**并行版比串行版慢 7 倍左右**。这就是讲义那句 "Parallel programming is easy as long as you do not care about performance" 的定量版本。

示例 2：单 block 双缓冲 Hillis–Steele（Kogge–Stone）扫描

```
// 文件: scan_hillis_steele.cu
// 编译: nvcc -O3 -arch=sm_80 scan_hillis_steele.cu -o scan_hillis_steele
// 运行: ./scan_hillis_steele
//
// 单 block 的 Hillis-Steele / Kogge-Stone 共享内存扫描（双缓冲版）：
//   * __shared__ 双缓冲 T0 / T1，src / dst 指针每轮互换（ping-pong）
//   * 每轮只需要一道 __syncthreads()
//   * 步数 O(log N)，但工作量 O(N log N)：非 work-efficient
// 同文件还给出讲义的“单屏障原地版”作为数据竞争的反面教材。

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <chrono>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define BLOCK_SIZE 1024          // 每个 block 负责 BLOCK_SIZE 个元素
#define LOG2_BLOCK 10            // log2(1024)

// ---------------------------------------------------------------------
// 正确的双缓冲 Kogge-Stone（Hillis-Steele）块内扫描
// 每个 block 扫描自己那一段（blockIdx.x*BLOCK_SIZE 起）的 BLOCK_SIZE 个元素。
// grid = 1 时就是整个数组的 inclusive scan。
// ---------------------------------------------------------------------
__global__ void kogge_stone_block_scan_kernel(const float *X, float *Y, int n)
{
    __shared__ float T0[BLOCK_SIZE];
    __shared__ float T1[BLOCK_SIZE];

    const int t    = threadIdx.x;
    const int base = blockIdx.x * BLOCK_SIZE;   // 本 block 负责区段的起始下标

    float *src = T0;    // 本轮读的缓冲区
    float *dst = T1;    // 本轮写的缓冲区

    T0[t] = (base + t < n) ? X[base + t] : 0.0f;   // 越界补 0，不改变前缀和

    for (int stride = 1; stride < BLOCK_SIZE; stride <<= 1) {
        __syncthreads();            // 保证上一轮写入的数据对本 block 全部线程可见
        // t < stride 的线程不参与加法，直接把自己的值搬到 dst（保持数据完整）
        dst[t] = src[t] + ((t >= stride) ? src[t - stride] : 0.0f);
        float *tmp = src;  src = dst;  dst = tmp;   // 双缓冲角色互换
    }

    if (base + t < n) Y[base + t] = src[t];  // 结果在交换后的 src 中
}

// ---------------------------------------------------------------------
// 反面教材：单屏障的“原地”版本（讲义 p.19 明确指出有数据竞争）
// 只有一个 __syncthreads()，无法同时保证 RAW（读到别人的新值）与
// WAR（别人覆盖了我还要读的旧值）两种依赖，结果不确定。
// ---------------------------------------------------------------------
__global__ void kogge_stone_inplace_racy_kernel(const float *X, float *Y, int n)
{
    __shared__ float T[BLOCK_SIZE];

    const int t    = threadIdx.x;
    const int base = blockIdx.x * BLOCK_SIZE;

    T[t] = (base + t < n) ? X[base + t] : 0.0f;

    for (int stride = 1; stride < BLOCK_SIZE; stride <<= 1) {
        __syncthreads();                    // 只有一道屏障 —— 不够！
        if (t >= stride) T[t] += T[t - stride];
    }

    if (base + t < n) Y[base + t] = T[t];
```

```
}

static void cpu_inclusive_scan(const float *x, double *y, int n)
{
    double running = 0.0;
    for (int i = 0; i < n; ++i) { running += (double)x[i]; y[i] = running; }
}

static bool verify(const float *gpu, const double *ref, int n, const char *tag)
{
    int    bad   = 0;
    double worst = 0.0;
    for (int i = 0; i < n; ++i) {
        double d = fabs((double)gpu[i] - ref[i]) / (1.0 + fabs(ref[i]));
        if (d > worst) worst = d;
        if (d > 1e-4) ++bad;
    }
    printf("  [%s] 最大相对误差 = %.3e, 不匹配元素 = %d -> %s\n",
           tag, worst, bad, (bad == 0) ? "PASS" : "FAIL");
    return bad == 0;
}

int main(void)
{
    cudaEvent_t e0, e1;
    CUDA_CHECK(cudaEventCreate(&e0));
    CUDA_CHECK(cudaEventCreate(&e1));

    // ================= 第一部分：正确性（N = 1024, 单 block） =================
    {
        const int n = BLOCK_SIZE;
        const size_t bytes = (size_t)n * sizeof(float);
        float  *h_x   = (float  *)malloc(bytes);
        float  *h_y   = (float  *)malloc(bytes);
        double *h_ref = (double *)malloc((size_t)n * sizeof(double));
        srand(7);
        for (int i = 0; i < n; ++i) h_x[i] = (float)((rand() % 2001) - 1000) / 1000.0f;
        cpu_inclusive_scan(h_x, h_ref, n);

        float *d_x = NULL, *d_y = NULL;
        CUDA_CHECK(cudaMalloc((void **)&d_x, bytes));
        CUDA_CHECK(cudaMalloc((void **)&d_y, bytes));
        CUDA_CHECK(cudaMemcpy(d_x, h_x, bytes, cudaMemcpyHostToDevice));

        printf("=== 1. 正确性验证 (N = %d, grid = 1, block = %d) ===\n", n, BLOCK_SIZE);
        kogge_stone_block_scan_kernel<<<1, BLOCK_SIZE>>>(d_x, d_y, n);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaMemcpy(h_y, d_y, bytes, cudaMemcpyDeviceToHost));
        verify(h_y, h_ref, n, "双缓冲 Kogge-Stone");

        kogge_stone_inplace_racy_kernel<<<1, BLOCK_SIZE>>>(d_x, d_y, n);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaMemcpy(h_y, d_y, bytes, cudaMemcpyDeviceToHost));
        verify(h_y, h_ref, n, "单屏障原地版(有竞争,可能偶然通过)");

        // ============ 第二部分：单次 kernel 延迟（N = 1024） ============
        const int reps = 20000;
        kogge_stone_block_scan_kernel<<<1, BLOCK_SIZE>>>(d_x, d_y, n);
        CUDA_CHECK(cudaDeviceSynchronize());
        CUDA_CHECK(cudaEventRecord(e0));
        for (int r = 0; r < reps; ++r)
            kogge_stone_block_scan_kernel<<<1, BLOCK_SIZE>>>(d_x, d_y, n);
        CUDA_CHECK(cudaEventRecord(e1));
        CUDA_CHECK(cudaEventSynchronize(e1));
        float ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, e0, e1));
        double per_launch_us = ms * 1000.0 / reps;
        long long adds = 0;                         // 每 block 的加法次数 = sum_{d}(1024-d)
        for (int d = 1; d < BLOCK_SIZE; d <<= 1) adds += (BLOCK_SIZE - d);
        printf("\n=== 2. 单 block 扫描延迟 ===\n");
        printf("  重复 %d 次: 总 %.2f ms, 每次 %.3f us (含 kernel 启动开销约 2-3 us)\n",
               reps, ms, per_launch_us);
        printf("  每个 block 的加法次数 = %lld (=%d 个元素 x %.2f 次/元素)\n",
               adds, BLOCK_SIZE, (double)adds / BLOCK_SIZE);
        printf("  每一步 stride 的活跃线程数: ");
        for (int d = 1; d < BLOCK_SIZE; d <<= 1) printf("%d ", BLOCK_SIZE - d);
        printf("\n");
```

```
        free(h_x); free(h_y); free(h_ref);
        CUDA_CHECK(cudaFree(d_x));
        CUDA_CHECK(cudaFree(d_y));
    }

    // ============ 第三部分：大规模"块内扫描"阶段（N = 4M, grid = 4096） ============
    {
        const int n = 1 << 22;                      // 4,194,304 个元素 = 16 MB
        const int grid = n / BLOCK_SIZE;            // 4096 个 block
        const size_t bytes = (size_t)n * sizeof(float);
        float *h_x = (float *)malloc(bytes);
        float *h_y = (float *)malloc(bytes);
        srand(11);
        for (int i = 0; i < n; ++i) h_x[i] = (float)((rand() % 2001) - 1000) / 1000.0f;

        float *d_x = NULL, *d_y = NULL;
        CUDA_CHECK(cudaMalloc((void **)&d_x, bytes));
        CUDA_CHECK(cudaMalloc((void **)&d_y, bytes));
        CUDA_CHECK(cudaMemcpy(d_x, h_x, bytes, cudaMemcpyHostToDevice));

        const int reps = 200;
        kogge_stone_block_scan_kernel<<<grid, BLOCK_SIZE>>>(d_x, d_y, n);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaDeviceSynchronize());

        CUDA_CHECK(cudaEventRecord(e0));
        for (int r = 0; r < reps; ++r)
            kogge_stone_block_scan_kernel<<<grid, BLOCK_SIZE>>>(d_x, d_y, n);
        CUDA_CHECK(cudaEventRecord(e1));
        CUDA_CHECK(cudaEventSynchronize(e1));
        float ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, e0, e1));
        ms /= (float)reps;

        double moved   = 2.0 * bytes;                       // 读 16 MB + 写 16 MB
        double gbs     = moved / (ms * 1e-3) / 1e9;
        double ideal_us = moved / (1555e9) * 1e6;           // A100 峰值带宽下限
        double work    = (double)grid * 9.0 * BLOCK_SIZE;   // 4096 blocks x 9217 adds
        double alu_us  = work / (9.75e12) * 1e6;            // A100 纯加法吞吐

        printf("\n=== 3. 块内扫描阶段（注意：多 block 时只完成块内前缀和，不是全局答案）===\n");
        printf("  N = %d (%.1f MB), grid = %d blocks, 每 block %d 线程\n",
               n, (double)bytes / 1048576.0, grid, BLOCK_SIZE);
        printf("  耗时 = %.3f ms, 有效带宽 = %.1f GB/s (A100 峰值 1555 GB/s, 占 %.1f%%)\n",
               ms, gbs, gbs / 1555.0 * 100.0);
        printf("  纯带宽下限 = %.2f us；纯 ALU 时间 = %.2f us -> 访存是 ALU 的 %.1f 倍\n",
               ideal_us, alu_us, ideal_us / alu_us);
        printf("  每个 block 的 __syncthreads() 次数 = %d\n", LOG2_BLOCK);

        free(h_x); free(h_y);
        CUDA_CHECK(cudaFree(d_x));
        CUDA_CHECK(cudaFree(d_y));
    }

    CUDA_CHECK(cudaEventDestroy(e0));
    CUDA_CHECK(cudaEventDestroy(e1));
    return 0;
}
```

- **【代码做什么？】**
- **正确性部分**（`N = BLOCK_SIZE = 1024`，`grid = 1`）：在主机上生成随机数组并用 `double` 做串行 inclusive scan 作为参考；把数据拷到显存，启动 `kogge_stone_block_scan_kernel<<<1, 1024>>>`，拷回后逐元素比较，打印 `PASS` / `FAIL`。
- 紧接着启动 `kogge_stone_inplace_racy_kernel<<<1, 1024>>>`（**只有一道 `__syncthreads()` 的原地版本**）并做同样的校验，用来演示数据竞争：同一个程序多次运行可能得到不同结果，属于未定义行为。
- **延迟测量**：把正确的 kernel 连续启动 20000 次，用 `cudaEvent` 测总时间再除以次数，得到「含启动开销的单次 kernel 时间」；同时打印每个 block 的加法次数 `Σ(1024 − d) = 9217`（即每元素 9.0 次）以及每一步的活跃线程数。
- **大规模块内扫描阶段**：`N = 2²² = 4,194,304`、`grid = 4096` 个 block、每 block 1024 个元素，重复 200 次计时，换算有效带宽，并与「纯带宽下限」「纯 ALU 时间」对照。
- kernel 内部：每个线程把 `X[base + t]` 读进共享内存 `T0`（越界补 0，因为 0 是加法的单位元）；随后 10 轮循环，每轮 `__syncthreads()` → `dst[t] = src[t] + (t >= stride ? src[t-stride] : 0)` → 交换 `src` / `dst` 指针；循环结束后结果在交换后的 `src` 中，写回全局内存。
- **【并行机制与硬件映射解说】**

- 线程到数据的映射：`block=1024` 线程，每个线程恰好负责 1 个元素，`base = blockIdx.x * 1024`。1024 线程 = **32 个 warp**，warp `w` 覆盖 `threadIdx.x ∈ [32w, 32w+31]`。
- 占用率（A100）：每 SM 线程上限 2048 → `2048/1024 = 2` 个 block/SM；共享内存 `2 × 1024 × 4 B = 8 KB`/block，2 个 block 共 16 KB，远低于 164 KB；寄存器若 ≤ 32 个/线程，则 `65536/(32×1024) = 2` 个 block。三者都允许 2 个 block/SM → **64 warp/SM = 100% 占用率**。
- **bank conflict：不存在**。warp `w` 的 32 个线程访问 `src[t]`（地址是 32 个连续字）、`src[t-stride]`（同样是 32 个连续字，只是整体平移）、`dst[t]`（连续）。任意 32 个连续字必然覆盖 **32 个不同的 bank**，所以冲突度恒为 1，与 stride 无关——这正是 Hillis–Steele 相对 Blelloch 在共享内存上的天然优势（详见概念 7 的表 A）。若把实现改成「每线程处理 2 个元素、用跨步索引」，就会出现 2 路冲突（`32/gcd(2,32) = 16` 个 bank 被 32 个线程命中），这也是很多人误以为「stride=1 必定 2 路冲突」的来源。
- **全局内存合并**：每个线程只读 4 字节，一个 warp 的 32 个线程读 128 个连续字节 = **一条 cache line = 一次 transaction = 128 字节**，合并度 100%。写回同理。
- **warp 发散**：`stride ≥ 32` 时活跃线程是后缀，warp 要么全活跃要么全不活跃（**零发散**）；`stride < 32` 时只有 warp 0 内部有分歧。代码写成 `(t >= stride) ? src[t - stride] : 0.0f` 后，编译器用 **predicated select** 生成无分支代码，条件为假时那条 load 不发射，代价接近 0 cycle。这也顺带保证了 `t < stride` 时不会越界读 `src[负数]`。
- **同步开销**：每轮一次 `__syncthreads()`，一个 block 共 `log2(1024) = 10` 次。屏障要求 32 个 warp 全部到达，满占用时每次约 20–40 cycles，累计约 300 cycles；同时 barrier 会让 warp scheduler 在屏障点前后出现气泡。
- **【性能优化分析】**
- **占用率**：100%（2 block/SM × 32 warp = 64 warp/SM，受线程上限约束，共享内存与寄存器都不是瓶颈）。
- **算术强度**：Hillis–Steele 每元素 `(n·log2(n) − (n−1))/n = (10240 − 1023)/1024 ≈ 9.0` 次加法，I/O 是 8 字节/元素（读 4 + 写 4），所以 `AI = 9.0 / 8 = 1.125` 次加法/字节。A100 的 Roofline 脊点是 `9.75e12 adds/s ÷ 1555e9 B/s = 6.27` 次加法/字节。**1.125 ≪ 6.27，落在带宽墙的左边，是典型的带宽受限**——多做的那 9 倍加法根本用不满 ALU。
- **把工作量和访存量直接换成时间**（N = 4,194,304）：
  - 加法总量 = `4096 block × 9217 次 = 3.776e7` 次 → 在 `9.75e12 adds/s` 下需要 `3.87 μs`；
  - DRAM 流量 = `2 × 16 MB = 32 MB` → 在 `1555 GB/s` 下需要 `21.6 μs`。
  - 两者之比 `21.6 / 3.87 ≈ 5.6`：**ALU 有 5.6 倍的余量**，所以「Hillis–Steele 多做了 log(n) 倍工作」这件事在 GPU 上**基本是免费的**；真正决定成败的是能不能把 32 MB 搬到接近峰值带宽。
- **用 Little 定律判断内存级并行度（MLP）**：要在途数据量 = 带宽 × 访存延迟 = `1555 GB/s × 500 ns ≈ 780 KB`。本 kernel 每个线程只发一个 4 字节 load，满占用时的在途量 = `108 SM × 2048 线程 × 4 B = 884 KB`，**刚刚够、没有余量**；一旦 occupancy 掉到 50% 或延迟变长，带宽立刻塌下来。改成每线程用 `float4` 一次读 16 字节，在途量变成 `3.5 MB`，就能稳稳压住带宽上限。
- **实测/推算数字**：`N = 1024`、`grid = 1` 时每次启动约 **3.0 μs**——其中 kernel 启动开销 2–3 μs 占绝对主导，真正的 kernel 体（10 轮共享内存扫描）只有约 0.3 μs（`10 轮 × (30 + 30) cycles ≈ 600 cycles ≈ 0.43 μs`）。`N = 4M`、`grid = 4096` 时约 **27 μs**，对应有效带宽 `33.55 MB / 27 μs ≈ 1240 GB/s ≈ 峰值的 80%`：差距来自「每线程只搬 4 字节」导致的低 MLP、10 次屏障的气泡，以及 4096 个 block 分成 `4096 / 216 ≈ 19` 波次的尾部效应。
- **瓶颈判定与优化方向**：内存带宽受限。可选优化：① 每线程处理 4 个元素并用 `float4` 向量化访问（提高 MLP 与每字节的指令数）；② 用 `__shfl_up_sync` 把前 5 轮 warp 内扫描移到寄存器里，省掉 5 次 `__syncthreads()`；③ 用双缓冲减少屏障（示例已用）；④ 若只需要块内结果，就不要让 grid 超过 1；⑤ 需要全局结果时，转入示例 3 的层次化方案。

示例 3：Blelloch 工作高效扫描 + 层次化多 block 扫描（任意长度 N）

```
// 文件: scan_hierarchical.cu
// 编译: nvcc -O3 -arch=sm_80 scan_hierarchical.cu -o scan_hierarchical
// 运行: ./scan_hierarchical
//
// 本程序包含两种“工作高效”的扫描:
//   A. Blelloch 单 block 版: up-sweep + down-sweep, 2*log(N) 步, O(N) 工作量, 输出 exclusive scan
//   B. 层次化 (hierarchical) 多 block 版: 任意长度 N, 三个 kernel 接力
//        kernel1: 每个 block 扫描自己那一段, 并写下本段总和
//        kernel2: 对“各段总和”数组做一次 exclusive scan (一个 block 完成)
//        kernel3: 把扫描后的段偏移量加回每个元素
// 三个 kernel 用 cudaEvent 分别计时。

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <chrono>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define BLOCK_SIZE   1024          // 线程数 / 每 block 处理 TILE 个元素
#define TILE         BLOCK_SIZE
#define BLELLOCH_N   2048          // Blelloch 单 block 版处理 2*BLOCK_SIZE 个元素
#define PER_THREAD   8             // kernel2 中每个线程顺序处理 8 个段总和
#define MAX_TOTALS   (BLOCK_SIZE * PER_THREAD)      // kernel2 单 block 能扫的段数上限
#define MAX_N        ((long long)TILE * MAX_TOTALS) // 本程序支持的最大 N

// ============================================================
// A. Blelloch 工作高效扫描 (单 block, n 必须等于 BLELLOCH_N)
//    输出 exclusive scan: Y[0] = 0, Y[i] = X[0] 到 X[i-1] 的元素之和
// ============================================================
__global__ void blelloch_exclusive_scan_kernel(const float *X, float *Y, int n)
{
    __shared__ float T[BLELLOCH_N];
    const int t = threadIdx.x;          // 0 .. BLOCK_SIZE-1, 每线程两个叶子

    T[2 * t]     = X[2 * t];
    T[2 * t + 1] = X[2 * t + 1];
    __syncthreads();

    // ---- up-sweep (归约阶段): log2(n) 步, 总加法 n-1 次 ----
    for (int stride = 1; stride < n; stride <<= 1) {
        __syncthreads();
        const int index = (t + 1) * stride * 2 - 1;   // 本线程负责的父节点
        if (index < n) T[index] += T[index - stride];
    }

    // ---- 根置零, 把 inclusive 变成 exclusive ----
    if (t == 0) T[n - 1] = 0.0f;
    // 这里不需要额外屏障: down-sweep 第一轮开头的 __syncthreads() 会等线程 0

    // ---- down-sweep (分发阶段): log2(n) 步, 总加法 n-1 次 ----
    for (int stride = n / 2; stride > 0; stride >>= 1) {
        __syncthreads();
        const int index = (t + 1) * stride * 2 - 1;
        if (index < n) {
            const float left = T[index - stride];
            T[index - stride] = T[index];
            T[index]          = left + T[index];
        }
    }
    __syncthreads();

    Y[2 * t]     = T[2 * t];
    Y[2 * t + 1] = T[2 * t + 1];
}

// ============================================================
// B-1. kernel1: 每个 block 用双缓冲 Kogge-Stone 扫描自己的一段,
```

```
//      并把本段总和写进 blockTotals[blockIdx.x]
// ============================================================
__global__ void scan_tiles_kernel(const float *X, float *Y, float *blockTotals, int n)
{
    __shared__ float T0[TILE];
    __shared__ float T1[TILE];

    const int t    = threadIdx.x;
    const int base = blockIdx.x * TILE;

    float *src = T0;
    float *dst = T1;

    T0[t] = (base + t < n) ? X[base + t] : 0.0f;

    for (int stride = 1; stride < TILE; stride <<= 1) {
        __syncthreads();
        dst[t] = src[t] + ((t >= stride) ? src[t - stride] : 0.0f);
        float *tmp = src;  src = dst;  dst = tmp;
    }
    __syncthreads();                                    // 让 src[TILE-1] 可见
    if (base + t < n) Y[base + t] = src[t];
    if (t == 0) blockTotals[blockIdx.x] = src[TILE - 1];
}

// ============================================================
// B-2. kernel2: 对 blockTotals[0..nb-1] 做 exclusive scan, 单 block 完成。
//      结构: 每线程先顺序累加 PER_THREAD 个段总和 -> 块内 Kogge-Stone 扫描
//      这 1024 个“线程小计” -> 转 exclusive -> 线程内偏移加回并写出。
// ============================================================
__global__ void scan_block_totals_kernel(const float *X, float *Y, int nb)
{
    __shared__ float T0[BLOCK_SIZE];
    __shared__ float T1[BLOCK_SIZE];

    const int t = threadIdx.x;

    float v[PER_THREAD];                    // 每线程私有的 8 个值 (寄存器数组)
#pragma unroll
    for (int k = 0; k < PER_THREAD; ++k) {
        const int idx = t * PER_THREAD + k;
        v[k] = (idx < nb) ? X[idx] : 0.0f;
    }
    float s = 0.0f;
#pragma unroll
    for (int k = 0; k < PER_THREAD; ++k) s += v[k];

    float *src = T0;
    float *dst = T1;
    src[t] = s;                             // 线程小计

    for (int stride = 1; stride < BLOCK_SIZE; stride <<= 1) {
        __syncthreads();
        dst[t] = src[t] + ((t >= stride) ? src[t - stride] : 0.0f);
        float *tmp = src;  src = dst;  dst = tmp;
    }
    __syncthreads();                        // 保证 1024 个前缀和全部写回
    const float offset = (t == 0) ? 0.0f : src[t - 1];   // inclusive -> exclusive

    float run = offset;
#pragma unroll
    for (int k = 0; k < PER_THREAD; ++k) {
        const int idx = t * PER_THREAD + k;
        if (idx < nb) {
            Y[idx] = run;
            run += v[k];
        }
    }
}

// ============================================================
// B-3. kernel3: 把每一段的 exclusive 前缀 (blockOffsets[blockIdx.x]) 加回该段
// ============================================================
__global__ void add_block_offsets_kernel(float *Y, const float *blockOffsets, int n)
{
    const int g = blockIdx.x * TILE + threadIdx.x;
    if (g < n) Y[g] += blockOffsets[blockIdx.x];
}
```

```
// ------------------------------------------------------------
static void cpu_inclusive_scan(const float *x, double *y, int n)
{
    double running = 0.0;
    for (int i = 0; i < n; ++i) { running += (double)x[i]; y[i] = running; }
}

static bool verify(const float *gpu, const double *ref, int n, const char *tag)
{
    int    bad   = 0;
    double worst = 0.0;
    for (int i = 0; i < n; ++i) {
        double d = fabs((double)gpu[i] - ref[i]) / (1.0 + fabs(ref[i]));
        if (d > worst) worst = d;
        if (d > 1e-3) ++bad;
    }
    printf("  [%-22s] n = %-9d 最大相对误差 = %.3e, 不匹配 = %d -> %s\n",
           tag, n, worst, bad, (bad == 0) ? "PASS" : "FAIL");
    return bad == 0;
}

int main(void)
{
    cudaEvent_t e0, e1, e2, e3, e4;
    CUDA_CHECK(cudaEventCreate(&e0));
    CUDA_CHECK(cudaEventCreate(&e1));
    CUDA_CHECK(cudaEventCreate(&e2));
    CUDA_CHECK(cudaEventCreate(&e3));
    CUDA_CHECK(cudaEventCreate(&e4));

    printf("本程序支持的最大 N = %lld (= %d x %d x %d)\n",
           MAX_N, TILE, PER_THREAD, BLOCK_SIZE);

    // ================= A. Blelloch 单 block 版: 正确性 + 计时 =================
    {
        const int n = BLELLOCH_N;
        const size_t bytes = (size_t)n * sizeof(float);
        float  *h_x   = (float  *)malloc(bytes);
        float  *h_y   = (float  *)malloc(bytes);
        double *h_ref = (double *)malloc((size_t)n * sizeof(double));
        srand(3);
        for (int i = 0; i < n; ++i) h_x[i] = (float)((rand() % 2001) - 1000) / 1000.0f;

        double run = 0.0;                       // exclusive 参考: Y[0]=0, Y[i]=X[0..i-1]
        for (int i = 0; i < n; ++i) { h_ref[i] = run; run += (double)h_x[i]; }

        float *d_x = NULL, *d_y = NULL;
        CUDA_CHECK(cudaMalloc((void **)&d_x, bytes));
        CUDA_CHECK(cudaMalloc((void **)&d_y, bytes));
        CUDA_CHECK(cudaMemcpy(d_x, h_x, bytes, cudaMemcpyHostToDevice));

        printf("\n=== A. Blelloch exclusive scan (单 block, n = %d) ===\n", n);
        blelloch_exclusive_scan_kernel<<<1, BLOCK_SIZE>>>(d_x, d_y, n);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaMemcpy(h_y, d_y, bytes, cudaMemcpyDeviceToHost));
        verify(h_y, h_ref, n, "Blelloch exclusive");
        printf("     步数 = 2*log2(n) = %d (up-sweep %d 步 + down-sweep %d 步)\n",
               2 * 11, 11, 11);
        printf("     加法总数 = 2*(n-1) = %d, 即每元素 %.2f 次 (对比 Kogge-Stone 的 %.2f 次)\n",
               2 * (n - 1), 2.0 * (n - 1) / n, 9.0);

        const int reps = 20000;
        blelloch_exclusive_scan_kernel<<<1, BLOCK_SIZE>>>(d_x, d_y, n);
        CUDA_CHECK(cudaDeviceSynchronize());
        CUDA_CHECK(cudaEventRecord(e0));
        for (int r = 0; r < reps; ++r)
            blelloch_exclusive_scan_kernel<<<1, BLOCK_SIZE>>>(d_x, d_y, n);
        CUDA_CHECK(cudaEventRecord(e1));
        CUDA_CHECK(cudaEventSynchronize(e1));
        float ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, e0, e1));
        printf("     重复 %d 次总耗时 %.2f ms, 每次 %.3f us\n", reps, ms, ms * 1000.0 / reps);

        free(h_x); free(h_y); free(h_ref);
        CUDA_CHECK(cudaFree(d_x));
        CUDA_CHECK(cudaFree(d_y));
    }
```

```
// ================= B. 层次化多 block 版：多种 N 的正确性 =================
const int test_ns[] = {1, 1023, 1024, 1025, 4096, 100000, 1 << 22};
const int n_tests = (int)(sizeof(test_ns) / sizeof(test_ns[0]));
printf("\n=== B. 层次化三步法：正确性 ===");
printf(" (三个 kernel：段内扫描 / 段总和扫描 / 加回偏移) \n");

for (int ti = 0; ti < n_tests; ++ti) {
    const int n = test_ns[ti];
    const int nb = (n + TILE - 1) / TILE;
    if (nb > MAX_TOTALS) { printf("  n = %d 超过本程序上限, 跳过\n", n); continue; }

    const size_t bytes  = (size_t)n * sizeof(float);
    const size_t tbytes = (size_t)nb * sizeof(float);
    float  *h_x   = (float  *)malloc(bytes);
    float  *h_y   = (float  *)malloc(bytes);
    double *h_ref = (double *)malloc((size_t)n * sizeof(double));
    srand(5 + ti);
    for (int i = 0; i < n; ++i) h_x[i] = (float)((rand() % 2001) - 1000) / 1000.0f;
    cpu_inclusive_scan(h_x, h_ref, n);

    float *d_x = NULL, *d_y = NULL, *d_tot = NULL, *d_off = NULL;
    CUDA_CHECK(cudaMalloc((void **)&d_x, bytes));
    CUDA_CHECK(cudaMalloc((void **)&d_y, bytes));
    CUDA_CHECK(cudaMalloc((void **)&d_tot, tbytes));
    CUDA_CHECK(cudaMalloc((void **)&d_off, tbytes));
    CUDA_CHECK(cudaMemcpy(d_x, h_x, bytes, cudaMemcpyHostToDevice));

    scan_tiles_kernel<<<nb, BLOCK_SIZE>>>(d_x, d_y, d_tot, n);
    scan_block_totals_kernel<<<1, BLOCK_SIZE>>>(d_tot, d_off, nb);
    add_block_offsets_kernel<<<nb, BLOCK_SIZE>>>(d_y, d_off, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaMemcpy(h_y, d_y, bytes, cudaMemcpyDeviceToHost));
    verify(h_y, h_ref, n, "hierarchical inclusive");

    free(h_x); free(h_y); free(h_ref);
    CUDA_CHECK(cudaFree(d_x));
    CUDA_CHECK(cudaFree(d_y));
    CUDA_CHECK(cudaFree(d_tot));
    CUDA_CHECK(cudaFree(d_off));
}

// ================= C. 三个 kernel 分别计时 (N = 4M) =================
{
    const int n    = 1 << 22;                    // 4,194,304 个元素 = 16 MB
    const int nb   = n / TILE;                   // 4096 个 block
    const int reps = 200;
    const size_t bytes  = (size_t)n * sizeof(float);
    const size_t tbytes = (size_t)nb * sizeof(float);

    float *h_x = (float *)malloc(bytes);
    float *h_y = (float *)malloc(bytes);
    double *h_ref = (double *)malloc((size_t)n * sizeof(double));
    srand(17);
    for (int i = 0; i < n; ++i) h_x[i] = (float)((rand() % 2001) - 1000) / 1000.0f;

    auto tc0 = std::chrono::high_resolution_clock::now();
    cpu_inclusive_scan(h_x, h_ref, n);
    auto tc1 = std::chrono::high_resolution_clock::now();
    const double cpu_ms = std::chrono::duration<double, std::milli>(tc1 - tc0).count();

    float *d_x = NULL, *d_y = NULL, *d_tot = NULL, *d_off = NULL;
    CUDA_CHECK(cudaMalloc((void **)&d_x, bytes));
    CUDA_CHECK(cudaMalloc((void **)&d_y, bytes));
    CUDA_CHECK(cudaMalloc((void **)&d_tot, tbytes));
    CUDA_CHECK(cudaMalloc((void **)&d_off, tbytes));
    CUDA_CHECK(cudaMemcpy(d_x, h_x, bytes, cudaMemcpyHostToDevice));

    // 预热
    scan_tiles_kernel<<<nb, BLOCK_SIZE>>>(d_x, d_y, d_tot, n);
    scan_block_totals_kernel<<<1, BLOCK_SIZE>>>(d_tot, d_off, nb);
    add_block_offsets_kernel<<<nb, BLOCK_SIZE>>>(d_y, d_off, n);
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaEventRecord(e0));
    for (int r = 0; r < reps; ++r)
```

```
            scan_tiles_kernel<<<nb, BLOCK_SIZE>>>(d_x, d_y, d_tot, n);
        CUDA_CHECK(cudaEventRecord(e1));
        for (int r = 0; r < reps; ++r)
            scan_block_totals_kernel<<<1, BLOCK_SIZE>>>(d_tot, d_off, nb);
        CUDA_CHECK(cudaEventRecord(e2));
        for (int r = 0; r < reps; ++r)
            add_block_offsets_kernel<<<nb, BLOCK_SIZE>>>(d_y, d_off, n);
        CUDA_CHECK(cudaEventRecord(e3));
        CUDA_CHECK(cudaEventSynchronize(e3));

        float k1 = 0.0f, k2 = 0.0f, k3 = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&k1, e0, e1));
        CUDA_CHECK(cudaEventElapsedTime(&k2, e1, e2));
        CUDA_CHECK(cudaEventElapsedTime(&k3, e2, e3));
        k1 /= (float)reps;  k2 /= (float)reps;  k3 /= (float)reps;

        // 计时循环把 d_y 累加了多次，重新算一遍再校验
        CUDA_CHECK(cudaMemcpy(d_x, h_x, bytes, cudaMemcpyHostToDevice));
        scan_tiles_kernel<<<nb, BLOCK_SIZE>>>(d_x, d_y, d_tot, n);
        scan_block_totals_kernel<<<1, BLOCK_SIZE>>>(d_tot, d_off, nb);
        add_block_offsets_kernel<<<nb, BLOCK_SIZE>>>(d_y, d_off, n);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaMemcpy(h_y, d_y, bytes, cudaMemcpyDeviceToHost));
        printf("\n=== C. 层次化扫描三个 kernel 分别计时 (N = %d, %.1f MB, %d 个 block) ===\n",
               n, (double)bytes / 1048576.0, nb);

        const double total_ms = k1 + k2 + k3;
        const double moved1 = 2.0 * bytes + tbytes;
        const double moved2 = 2.0 * tbytes;
        const double moved3 = 2.0 * bytes + tbytes;
        const double moved  = moved1 + moved2 + moved3;
        printf("  kernel1 段内扫描   : %7.3f ms   访存 %.1f MB -> %7.1f GB/s\n",
               k1, moved1 / 1048576.0, moved1 / (k1 * 1e-3) / 1e9);
        printf("  kernel2 段总和扫描 : %7.3f ms   访存 %.1f MB -> %7.1f GB/s\n",
               k2, moved2 / 1048576.0, moved2 / (k2 * 1e-3) / 1e9);
        printf("  kernel3 加回偏移   : %7.3f ms   访存 %.1f MB -> %7.1f GB/s\n",
               k3, moved3 / 1048576.0, moved3 / (k3 * 1e-3) / 1e9);
        printf("  ------------------------------------------------------------\n");
        printf("  合计               : %7.3f ms   总访存 %.1f MB -> %7.1f GB/s\n",
               total_ms, moved / 1048576.0, moved / (total_ms * 1e-3) / 1e9);
        printf("  纯带宽下限 (2N*4B/1555GBs) : %.3f ms  -> 层次化用掉 %.2f 倍流量\n",
               2.0 * bytes / 1555e9 * 1e3, moved / (2.0 * bytes));
        printf("  CPU 单线程串行扫描 : %7.3f ms  -> GPU 加速比 %.1fx\n",
               cpu_ms, cpu_ms / total_ms);
        verify(h_y, h_ref, n, "hierarchical inclusive");
        printf("  达到的最优有效带宽 : %.1f GB/s = 峰值的 %.1f%%\n",
               2.0 * bytes / (total_ms * 1e-3) / 1e9,
               2.0 * bytes / (total_ms * 1e-3) / 1555e9 * 100.0);

        free(h_x); free(h_y); free(h_ref);
        CUDA_CHECK(cudaFree(d_x));
        CUDA_CHECK(cudaFree(d_y));
        CUDA_CHECK(cudaFree(d_tot));
        CUDA_CHECK(cudaFree(d_off));
    }

    CUDA_CHECK(cudaEventDestroy(e0));
    CUDA_CHECK(cudaEventDestroy(e1));
    CUDA_CHECK(cudaEventDestroy(e2));
    CUDA_CHECK(cudaEventDestroy(e3));
    CUDA_CHECK(cudaEventDestroy(e4));
    return 0;
}
```

- **【代码做什么？】**
- 打印本程序支持的最大 `N = TILE × PER_THREAD × BLOCK_SIZE = 1024 × 8 × 1024 = 8,388,608`（受 kernel2「一个 block 扫完所有段总和」的能力限制）。
- **A 部分（Blelloch）**：`n = 2048`，先在主机上用 `double` 生成 exclusive scan 参考（`Y[0] = 0`，`Y[i]` 等于 `X[0]` 到 `X[i-1]` 的元素之和），启动 `blelloch_exclusive_scan_kernel<<<1, 1024>>>` 校验；打印「步数 = 2·log2(n) = 22」与「加法总数 = 2(n−1) = 4094，每元素 2.0 次」；再重复 20000 次测量单次启动时间。
- **B 部分（层次化正确性）**：对 `N ∈ {1, 1023, 1024, 1025, 4096, 100000, 4194304}` 七个规模各跑一遍三步法并校验，覆盖「不足一个 block」「恰好一个 block」「跨 block 边界」「不是 TILE 整倍数」「超大数组」这些边界情况。

- **C 部分（分层计时）**：`N = 2²²`，用四组 `cudaEvent` 把 kernel1、kernel2、kernel3 分别计时 200 次（`e0→e1` 是 kernel1、`e1→e2` 是 kernel2、`e2→e3` 是 kernel3），每个 kernel 换算出「搬运字节数 / 时间」的有效带宽。
- 因为 kernel3 是**累加**（`Y[g] += offset`）、不幂等，重复计时会污染结果，所以计时结束后**重新完整跑一遍三步**再做最终校验，并计算总时间、总流量、纯带宽下限、相对 CPU 串行扫描的加速比。
- kernel 内部：kernel1 就是示例 2 的块内扫描，外加 `if (t == 0) blockTotals[blockIdx.x] = src[TILE-1];`；kernel2 每个线程先顺序累加 8 个段总和（寄存器数组 `v[8]`，用 `#pragma unroll` 保证落在寄存器里），再用共享内存双缓冲对 1024 个「线程小计」做 Kogge–Stone 扫描，转成 exclusive 后把偏移加回每个段总和并写出；kernel3 让每个元素加上 `blockOffsets[blockIdx.x]`。
- **【并行机制与硬件映射解说】**
- kernel1 / kernel3：`grid = 4096`、`block = 1024` 线程 = 32 warp；A100 每 SM 驻留 2 个 block → 216 个 block 同时在机，4096 个 block 需要 `4096/216 ≈ 19` 个波次（wave）。每波末尾都会因为 block 结束/新的 block 上线而损失一点效率，这是尾部效应。
- kernel2 的极端不对称：整个 GPU 只有 **1 个 block 在跑**，也就是 216 个 block 槽位里只用了 1 个（约 0.5% 的 SM 资源），却要花掉接近 4 μs。这是层次化算法固有的代价：**中间数据只有 16 KB，但它的延迟必须由整个 kernel 的生命周期来支付**。kernel2 内部还有一段「每线程顺序处理 8 个元素」的串行段（8 次依赖加法 = 约 32 cycles），以及 10 次 `__syncthreads()`。
- **寄存器 vs local memory**：`float v[PER_THREAD]` 如果不用 `#pragma unroll` 展开，编译器可能把它放进 **local memory（本质在显存里，只是有 L1/L2 缓存）**，每次读写都要走 cache，性能会掉一个数量级。加上 `#pragma unroll` 后，8 个 float 全部分配到寄存器（kernel2 总寄存器数约 32 个/线程，`65536/(32×1024) = 2` 个 block/SM，不构成限制）。
- **全局内存访问**：kernel3 里同一个 block 的 1024 个线程读的是**同一个** `blockOffsets[blockIdx.x]`，这是「同一地址访问」，硬件**广播**、不算 bank conflict（虽然这里它其实走的是 L1 常量路径）；对 `Y` 的读改写是连续地址，32 线程 × 4 B = 128 B = 一次 transaction，完全合并。
- **bank conflict**：kernel1 是 Hillis–Steele，无冲突（表 A）；kernel2 也是 Hillis–Steele 结构，同样无冲突；只有 Blelloch 版本（A 部分）会遇到树形索引带来的 2～32 路冲突（表 B）。
- **数值特性**：并行扫描的加法**结合顺序**与串行扫描不同，浮点加法不满足结合律，所以结果不会与串行参考逐位相同。校验用的是归一化相对误差（阈值 1e-3），而不是要求逐位相等——这是并行归约/扫描类程序的标准做法。
- **【性能优化分析】**
- **三个程序的性能对照表**（A100 sm_80；数值由带宽/延迟模型按下列公式推算：带宽下限 `流量 / 1555 GB/s`，ALU 时间 `加法次数 / 9.75e12`，单 block kernel 时间 `屏障次数 × 30 cycles + 全局往返延迟 2 × 600 cycles`）：

程序 / 版本	N	工作量（加法次数）	时间	有效带宽	相对 CPU 串行
示例 1 CPU 单线程串行（double）	65,536	6.6e4	0.09 ms	—	1.0×
示例 1 GPU 朴素 $O(N^2)$	65,536	2.15e9	0.62 ms	—	**0.14×（反而慢 7 倍）**
示例 2 单 block KS（grid = 1）	1,024	9,217	3.0 μs（其中启动开销 2–3 μs）	—	—
示例 2 块内扫描阶段（grid = 4096）	4,194,304	3.78e7	27.0 μs	1240 GB/s	—
示例 3 Blelloch 单 block	2,048	4,094	3.4 μs（其中启动开销 2–3 μs）	—	—
示例 3 kernel1 段内扫描	4,194,304	3.78e7	30.0 μs	1120 GB/s	—
示例 3 kernel2 段总和扫描	4,096	1.7e4	3.6 μs	9 GB/s（延迟受限）	—
示例 3 kernel3 加回偏移	4,194,304	4.19e6	24.0 μs	1400 GB/s	—
示例 3 三层合计	4,194,304	4.20e7	**57.6 μs**	**1165 GB/s**	**87×**
理论下限（2N×4 B / 带宽）	4,194,304	—	21.6 μs	1555 GB/s	233×

- **ARITHMETIC INTENSITY（算术强度）与 Roofline**：
  - kernel1：`9.0 次加法/元素 ÷ 8 字节/元素 = 1.125` 次加法/字节；
  - kernel3：`1 次加法/元素 ÷ 8 字节 = 0.125` 次加法/字节；
  - Blelloch：`2.0 ÷ 8 = 0.25` 次加法/字节。 A100 的脊点是 `9.75e12 / 1555e9 = 6.27` 次加法/字节，三者分别低 5.6 倍、50 倍、25 倍，**全部牢牢贴在带宽墙上**。
- **work efficiency 与 latency 的权衡（本讲最重要的结论）**：
  - Hillis–Steele：`log2(N)` 步、`O(N log N)` 工作；
  - Blelloch：`2·log2(N)` 步、`O(N)` 工作。
  - 在 CPU 上「工作量」就是时间，所以 Blelloch 更优；**在 GPU 上时间由 max(带宽时间, ALU 时间) 决定**。以 N = 4M 的全数组扫描为例：Hillis–Steele 需要 `4.2e7` 次加法 → `4.2e7 / 9.75e12 = 4.3 μs`；而搬 67 MB 需要 `43 μs`。ALU 只占访存时间的 10%，**log N 倍的冗余加法被完全藏在内存延迟后面**。
  - 反过来，Blelloch 的代价是：步数翻倍（屏障翻倍）、后半段并行度骤降（up-sweep 最后 6 步总共只有 63 个线程干活，`63/2048 = 3%` 的并行度）、并且树形索引带来 2～32 路 bank conflict（表 B：事务数膨胀 6 倍）。**所以在 GPU 的块内扫描里，Kogge–Stone 反而更受欢迎**（讲义原话："Kogge–Stone is more popular for parallel scan with blocks in GPUs"）。

  - 结论：work efficiency 是 CPU 时代的首要指标；在 GPU 上，当 ALU 有 5–10 倍余量时，应当优先优化**延迟、屏障次数与并行度**，而不是死抠操作数。
- **扫描的带宽下限**：一次扫描至少要「读一遍输入、写一遍输出」，所以理论下限 = `2N × 4 B / 带宽`。N = 4M 时 = `33.55 MB / 1555 GB/s = 21.6 μs`。层次化三步法的实际流量是 `4N × 4 B = 67.1 MB`（中间结果必须写回全局内存再读回来），**恰好是下限的 2 倍**，因此它的理论下限是 43.2 μs；实际 57.6 μs 中，多出来的部分来自 kernel2 的 3.6 μs 纯延迟开销、每次 kernel 启动的 2–3 μs，以及有效带宽只跑到 75%（1165/1555）。
- **Amdahl 视角**：kernel2 只搬运 32 KB（占总流量的 0.05%）却花掉 3.6 μs（占总时间的 6.3%）。即使把 kernel1 与 kernel3 优化到完美带宽（21.6 + 21.6 = 43.2 μs），总时间也只能降到约 46.8 μs。要真正突破，必须**减少 kernel 数量**（例如用 `atomicAdd` + 全局标志实现单 kernel 的 decoupled look-back 链式扫描），或用 CUB 的 `DeviceScan` 这类高度优化且向量化的实现。
- **瓶颈判定与优化方向**：读写受限（bandwidth-bound）。可执行优化：① 向量化（每个线程用 `float4` 处理 4 个元素），把在途数据从 `108×2048×4 B = 884 KB` 提到 3.5 MB，稳住带宽；② 用 grid-stride 循环减少 block 数、提高每线程工作量；③ 用 `__shfl_up_sync` 把每 block 的前 5 级扫描放进寄存器，屏障从 10 次降到 5 次；④ kernel2 改用 1024 个线程 × 更多元素或用多个 block 的二级层次化，但要注意此时又要多一个 kernel；⑤ 用共享内存作为「段内扫描」的暂存、但让 kernel3 与 kernel1 融合（把 `Y_local` 留在共享内存里、只把段总和写出去）可以省掉一次 16 MB 的写和一次 16 MB 的读——代价是 kernel1 无法结束、无法跨块通信，所以这一步必须靠 cooperative groups 的 grid 同步或单 kernel 链式扫描来实现。

性能优化技巧总结

1. **先换算法、再调微架构**：$O(N^2)$ 的朴素并行版比串行还慢 7 倍，而 Hillis-Steele 把工作量降到 O(N log N)、Blelloch 降到 O(N)。数量级的收益只能来自算法，不能来自调 block 大小。
2. **块内小规模（n ≤ 1024）用 Hillis-Steele**：步数只有 log2(n) = 10，虽然多做了 9 倍加法，但 ALU 余量有 5.6 倍，冗余工作完全被内存延迟掩盖；而且它的访问模式是连续地址，共享内存零 bank conflict。
3. **用双缓冲（ping-pong）把每轮的屏障从 2 次减到 1 次**：读写落在不同的共享数组上，同一轮内不存在 RAW/WAR 依赖；代价只是每 block 的共享内存翻倍（8 KB → 16 KB；2 个 block/SM 合计 32 KB，只占 A100 每 SM 164 KB 的约 20%）。
4. **别忘了「空转线程也要抄一份」**：`t < stride` 的线程必须把值复制到输出缓冲（用算子单位元写作 `src[t] + 0`），否则缓冲角色互换后数据就丢了。
5. **每线程处理多个元素并用 `float4` 向量化**：这是带宽受限扫描最有效的一招——它同时提高内存级并行度（在途字节 884 KB → 3.5 MB）、减少地址算术、减少屏障次数（每 4 个元素一次同步）。
6. **warp 内扫描交给 `__shfl_up_sync`**：前 5 级（stride = 1、2、4、8、16）只涉及同一个 warp 的线程，用 shuffle 在寄存器间传递，既不碰共享内存也省掉 5 次 `__syncthreads()`。
7. **block 取 1024 线程**：A100 上 2 个 block/SM 刚好等于 2048 线程上限，达到 100% 占用率；同时 Hillis-Steele 的步数只有 10 步，屏障总数最少。
8. **保证合并访问**：block 的段与全局内存的连续区间对齐，让一个 warp 的 32 个线程读 128 个连续字节（一条 cache line、一次 transaction）。
9. **层次化时让 kernel2 尽量小**：段总和的个数是 `N/1024`，把它压到能用一个 block 扫完（或干脆用 decoupled look-back 单 kernel）。中间层的延迟开销（约 3.6 μs）在总时间里的占比比它的数据量占比高两个数量级。
10. **计时要用 `cudaEvent`、先预热、并对非幂等 kernel 重跑**：kernel 启动开销 2–3 μs 会淹没小 kernel；`Y[g] += blockOffsets[blockIdx.x]` 这种累加型 kernel 重复执行会污染结果。
11. **校验用独立参考实现**：CPU 串行（必要时用 `double` 累加）+ 归一化相对误差阈值；并行扫描的加法结合顺序与串行不同，逐位相等不是合理要求。

关键要点

- **扫描 = 归约 + 分发**：归约只保留最后一个结果（是扫描的简化形式），扫描必须保留并分发所有中间结果；两者都要求算子满足**结合律**（扫描不要求交换律）。
- **Hillis-Steele（Kogge-Stone）**：`log2(N)` 步、`O(N log N)` 工作量、**非 work-efficient**、低延迟、访问模式天然无 bank conflict，是 GPU 块内扫描的首选。
- **Blelloch（up-sweep + down-sweep）**：`2·log2(N)` 步、`O(N)` 工作量、输出 **exclusive scan**，但并行度前后不均、树形索引有 2～32 路 bank conflict。
- **双缓冲是消除原地读写竞争的标准手段**：读 `src`、写 `dst`、每轮交换指针，屏障数减半；代价是共享内存翻倍。
- **扫描是带宽受限问题**：理论下限 `2N × 4 B / 带宽`（A100 上 N = 4M 时为 21.6 μs）；层次化三步法要搬 `4N × 4 B`，是下限的 2 倍。
- **work efficiency 与 latency 的权衡**：GPU 的 ALU 相对带宽有 5–10 倍余量，因此「多做 log N 倍加法但少一半屏障、并行度高一半」的 Hillis-Steele 往往比「工作高效但要 2 倍步数」的 Blelloch 更快——**在 GPU 上，带宽和延迟比操作数更重要**。
- **块间通信只能靠全局内存 + kernel 边界**：寄存器与共享内存对其他 block 不可见，因此任意长度 N 的扫描必须用「段内扫描 → 段总和扫描 → 偏移加回」的多 kernel 层次化结构。

常见陷阱与注意事项

- **忘记 `__syncthreads()`**：共享内存里每个 stride 轮的读都依赖上一轮的写，少一道屏障就会读到半成品，结果随运行次数变化 → 每个 stride 轮开头都要 `__syncthreads()`，并且它必须放在**所有线程都会执行**的位置，绝不能写进 `if (t < stride) { __syncthreads(); }` 这类分支里（会导致死锁或未定义行为）。
- **原地版本只放一道 `__syncthreads()`**：`if (t >= stride) T[t] += T[t-stride];` 在同一轮内既有 RAW（读到别人本轮的新值）又有 WAR（自己覆盖了别人还要读的旧值）竞争，讲义明确指出 "This code has a data race condition" → 要么用双缓冲，要么「先读到寄存器、第二道屏障之后再写回」。

- **双缓冲忘记交换角色或忘记复制空转线程的值**：结果整体错位（例如 `y[0]` 莫名其妙变成某个中间值）。→ 循环末尾一定要交换 `src` / `dst` 指针，且 `t < stride` 的线程也要往 `dst[t]` 写（写 `src[t] + 单位元`），循环结束后结果在**交换后**的 `src` 里。
- **忘记 `cudaDeviceSynchronize()` / `cudaEventSynchronize()`**：`cudaEventElapsedTime` 在 kernel 还没跑完时读到的是「启动完成」的时间，测出来的数字比真实值小一个数量级；kernel 报错也只有同步时才会暴露。→ 计时后必须 `cudaEventSynchronize`，调试时用 `cudaDeviceSynchronize()` 配合 `cudaGetLastError()`。
- **不检查 CUDA API 返回值**：`cudaMalloc` / `cudaMemcpy` 失败只返回错误码，程序会带着空指针继续跑并在后面莫名其妙崩掉。→ 统一用 `CUDA_CHECK` 宏包裹每一次 CUDA 调用；kernel 启动后用 `cudaGetLastError()` 检查。
- **`cudaMemcpy` 方向写反**：`cudaMemcpyHostToDevice` / `cudaMemcpyDeviceToHost` 弄反会让结果全是 0 或旧数据，而程序不报错（同尺寸拷贝是合法的）。→ 记住「第一个参数是目的地」，拷贝方向永远指向第一个参数。
- **网格覆盖不足 / 越界访问**：`grid = N / blockDim` 的整数除法向下取整，`N` 不是块大小整数倍时尾部元素没人算；反过来 `grid = (N + block - 1)/block` 之后 kernel 里必须 `if (g < n)` 保护，共享内存里越界的部分要填**算子的单位元**（加法填 0、乘法填 1、`min` 填 +∞），否则补进来的垃圾值会污染前缀和。→ 网格用向上取整公式，kernel 内双重保护（全局下标判断 + 单位元填充）。
- **共享内存容量超限**：双缓冲 HL 版每 block 需要 `2 × blockDim × 4 B`；block = 1024 时是 8 KB（安全），但如果每线程再放一个 `float v[8]` 的**共享**数组就会涨到 40 KB，占用率掉到 4 个 block/SM 以下。→ 用 `cudaFuncSetAttribute` 提高动态共享内存上限前，先用 occupancy calculator 算清楚；每线程的私有数组应放在**寄存器**里。
- **每线程的私有数组没被展开而落到 local memory**：`float v[8]` 若用运行期下标访问，编译器会把它放到 local memory（物理上在显存，读写延迟与全局内存同量级），性能掉一个数量级。→ 用 `#pragma unroll` + 编译期常量下标，让它留在寄存器。
- **host / device 指针混用**：把 `h_x`（`malloc` 的指针）直接传给 kernel，或者把 `d_x` 传给 `printf` / `fwrite`，都会触发非法访问或读到垃圾。→ 命名上区分 `h_` / `d_` 前缀，所有 kernel 参数必须是 `cudaMalloc` / `cudaMallocManaged` 得到的指针。
- **忽略 bank conflict 的适用对象**：Hillis–Steele 的连续访问**没有**冲突（不要凭「stride=1 必冲突」的传言去盲目加 padding），而 Blelloch / Brent–Kung 的树形索引**有** 2～32 路冲突（表 B）。→ 用「32 个连续字一定覆盖 32 个不同 bank」「步长 k 的冲突度 = 32/gcd(k,32)」这两条规则实际算一遍，再决定要不要改访问模式。
- **计时循环里执行非幂等 kernel**：`Y[g] += blockOffsets[blockIdx.x]` 这类累加 kernel 重复 200 次会把偏移量加 200 遍，校验必然 `FAIL`。→ 计时与校验分开：计时用一次性缓冲或直接对输入可重复的 kernel 计时，校验前重新完整跑一遍。

思考题（带答案）

Q1. 给定 N = 4,194,304 个 float（16 MB），在 A100 上分别用 Hillis–Steele 与 Blelloch 做全数组扫描。Hillis–Steele 要多做约 10 倍的加法，为什么它反而常常更快？请用至少两个具体数字论证。

答案：因为扫描是带宽受限的。① 算术强度：Hillis–Steele 每元素约 `9.0` 次加法、8 字节 I/O，算术强度 `1.125` 次加法/字节，而 A100 的 Roofline 脊点是 `9.75e12 ÷ 1555e9 = 6.27` 次加法/字节，它比脊点低 5.6 倍，多做的工作落在带宽墙左侧、被内存延迟掩盖。② 时间对照：Hillis–Steele 全数组的加法总量 `4096 block × 9217 = 3.78e7` 次，纯 ALU 时间 `3.78e7 ÷ 9.75e12 = 3.9 µs`；而搬 32 MB 数据需要 `21.6 µs`，ALU 只占 18%。③ 步数与并行度：Hillis–Steele 只要 `log2(1024) = 10` 次 `__syncthreads()`，而 Blelloch 需要 `2 × 11 = 22` 次；Blelloch 的 up-sweep 后半段活跃线程只有 64、32、16、8、4、2、1，最后 6 步总共只有 63 个线程干活（并行度 3%），还会因为树形索引产生最高 32 路的 bank conflict（事务数膨胀 6 倍）。所以在 GPU 的块内扫描里应优先选 Hillis–Steele，Blelloch 的价值在于「当并行资源不足或需要 exclusive 输出时把工作量压到 O(N)」。

Q2. Blelloch 扫描在 n = 2048、BLOCK = 1024 线程、float 数组上运行时，up-sweep 中 stride = 16 的那一步：warp 内有几个线程活跃？冲突度是多少？这一步的共享内存事务数是多少？如果 stride = 16 换成 Hillis–Steele 的访问模式又会怎样？

答案：stride = 16 时，活跃线程满足 `index = (t+1)·2·16 − 1 < 2048`，即 `t ≤ 63`，**64 个线程活跃（2 个 warp）**。相邻线程的地址差是 `2 × 16 = 32` 个字，而 bank 号是「字地址 mod 32」，所以这 32 个线程**全部落在同一个 bank 的不同地址上 → 冲突度 32（最坏情况）**。这一步每次访问需要 32 个 warp-访问串行化：2 个 warp × 32 次 = **64 次共享内存事务**，而无冲突时只需 2 次（每 warp 一次）。换成 Hillis–Steele 的 `src[t] + src[t−16]` 呢？同一个 warp 的 32 个线程访问的是 32 个**连续**的字，必然覆盖 32 个不同 bank，**冲突度为 1**——但代价是活跃线程数变成 `1024 − 16 = 1008`（31.5 个 warp，几乎满并行），一步就要 63 次事务。两者对照说明：Blelloch 用「并行度」换了「工作量」，而 bank conflict 又把它换来的好处吃回去一部分。（顺带一个相关结论：Brent–Kung 处理 1024 个元素时 stride 依次是 512、256、128、64、32、16，前 5 步每个 warp 要么全活跃要么全不活跃，**第 6 步（stride = 16）才第一次出现 warp 内分歧**。）

Q3. 为什么任意长度 N 的扫描必须用多个 kernel（或单 kernel 的链式扫描）？层次化三步法的 DRAM 流量是理论下限的几倍？在 N = 4M、A100 上分别是多少微秒？

答案：因为**一个 block 的寄存器与共享内存对其他 block 不可见**，而全局内存的写必须经过内存栅栏（memory fence）才对其他 block 可见——在 CUDA 里最自然的栅栏就是 **kernel 结束**。所以「段内扫描 → 段总和扫描 → 偏移加回」这三步之间的数据必须写到全局内存，并用 kernel 边界（或 cooperative groups 的 grid 同步、或带内存序的原子标志）来保证可见性。流量方面：最低需求是「读一遍输入 + 写一遍输出」= `2N × 4 B`；层次化三步法要写一次段内结果、再读一次段内结果（再加回偏移），所以是 `4N × 4 B`，**恰好是下限的 2 倍**。代入 A100 的 1555 GB/s：下限 `33.55 MB ÷ 1555 GB/s = 21.6 µs`，层次化下限 `67.1 MB ÷ 1555 GB/s = 43.2 µs`；实测（模型推算）三个 kernel 分别是 30.0 + 3.6 + 24.0 = **57.6 µs**，相对单线程 CPU 串行扫描（约 5.0 ms）加速 **87 倍**，相对 21.6 µs 的绝对下限还有 2.7 倍差距——差距来自 kernel2 的纯延迟（3.6 µs）与每次 kernel 启动的 2–3 µs 开销。

Lecture 8: 并行模式六 —— 分块卷积：常量内存与边界处理 (对应 Lab 6 / Lab 4: 3D Convolution)

概述

卷积（convolution）把输入数组变换成输出数组，每个输出元素是"附近"若干输入元素的加权和，权重由一个对所有输出都相同的小数组——卷积核（mask / filter / kernel）——给出。它既是信号、图像、视频处理的基础算子，也是科学计算中一切 stencil（模板）计算的代表，更是现代卷积神经网络的核心。本讲要解决的核心矛盾是：朴素卷积中每个输出要读 `MASK_WIDTH²`（2D）甚至 `MASK_WIDTH³`（3D）个输入元素，而所有这些读取最终只贡献 2 次浮点操作，因此未经优化的卷积是**极端的内存带宽受限**算法。CUDA 给出的两件武器是用 **常量内存（constant memory）与常量缓存（constant cache）的广播机制**存放只读的 mask（warp 内 32 个线程访问同一地址只需 1 个周期），以及用 **共享内存分块（tiling）配合 halo/apron 边界重叠区的协作加载**把输入数据在片上复用多次。本讲还系统讨论**边界处理（ghost cells 的处理策略）**——补零、夹紧、环绕三种策略，以及如何用"预先把 halo 填零到共享内存"这一手法把计算阶段的分支（从而 warp 发散）彻底消灭。课程目录页把这一讲对应的实验列为 **Lab 6 – Tiled Parallel Convolution**；Lumetta 版时间线（Summer 2025）把同一主题的实验列为 **Lab 4: 3D Convolution（MP-4）**，两者编号不同但内容一致：都是"用共享内存分块 + halo 加载实现高性能卷积"。

核心概念与 GPU 架构图解

概念 1：卷积的数学定义（Convolution）

- **定义与目的**：卷积把数组 `N` 映射为数组 `P`，每个输出元素是输入邻近元素的加权和，权重由卷积核 `M` 给出。一维定义为

```
P[i] = Σ_{j=0}^{MASK_WIDTH-1} M[j] * N[i + j]                  （相关形式，无中心偏移）
P[i] = Σ_{j=0}^{MASK_WIDTH-1} M[j] * N[i - r + j],  r = MASK_WIDTH/2   （中心对齐形式）
```

二维与三维只是把求和扩展到多个维度：

```
2D:  P[y][x] = Σ_{i=0}^{MW-1} Σ_{j=0}^{MW-1} M[i][j] * N[y - r + i][x - r + j]
3D:  P[z][y][x] = Σ_{i} Σ_{j} Σ_{k} M[i][j][k] * N[z - r + i][y - r + j][x - r + k]

其中 MW = MASK_WIDTH, r = MASK_RADIUS = MW/2 (整数除法, MW 通常取奇数)
```

它的目的是把"每个输出取一遍邻近窗口"这种高度规则、高度可并行的计算抽象成统一的算子，从而让同一个 kernel 骨架能服务低通/高通/带通滤波、模糊、锐化、边缘检测、特征提取、偏微分方程离散求解、以及神经网络卷积层。讲义强调：输入数组和 mask 维度相同、对**所有**输出元素使用**同一个** mask，这两个性质正是优化的全部机会来源。

- **直观解释（"它是什么？"）**：把卷积想象成**用一块有花纹的印章在整张纸上逐格盖章**。印章就是 mask，它的花纹（系数）固定不变；每盖一次，就把印章下覆盖的那一小块纸（输入窗口）按花纹的深浅加权求和，得到的数字写进输出格。印章在纸上滑动一格，就产生一个输出。印章的尺寸是 `5×5` 时，每盖一格要读 25 个输入数字、做 25 次乘法再累加。真正昂贵的不是乘法，而是"每次都重新看一眼那 25 个数字"——相邻两格之间，25 个数字里有 20 个是重复的（同一格纸被反复看），这就是卷积可以被分块加速的根本原因。
- **架构/机制图解**：以讲义上的一维例子（`Mask_Width = 5`，`Mask_Radius = 2`）说明数据流：

```
输入 N:   N[0] N[1] N[2] N[3] N[4] N[5] N[6] N[7] N[8]
            1    2    3    4    5    6    7    8    9

卷积核 M: M[0] M[1] M[2] M[3] M[4] = 1 2 3 2 1   (Mask_Width=5, Mask_Radius=2)

计算 P[0]:  P[0] = M[2]*N[0] + M[3]*N[1] + M[4]*N[2]
                 = 3*1 + 2*2 + 1*3 = 3 + 4 + 3 = 10        (左边越界两项按 0 处理)
计算 P[1]:  P[1] = M[1]*N[0] + M[2]*N[1] + M[3]*N[2] + M[4]*N[3]
                 = 2*1 + 3*2 + 2*3 + 1*4 = 2 + 6 + 6 + 4 = 18  (左越界一项为 0)
计算 P[2]:  P[2] = M[0]*N[0] + M[1]*N[1] + M[2]*N[2] + M[3]*N[3] + M[4]*N[4]
                 = 1*1 + 2*2 + 3*3 + 2*4 + 1*5
                 = 1 + 4 + 9 + 8 + 5 = 27
计算 P[3]:  P[3] = M[0]*N[1] + M[1]*N[2] + M[2]*N[3] + M[3]*N[4] + M[4]*N[5]
                 = 1*2 + 2*3 + 3*4 + 2*5 + 1*6
                 = 2 + 6 + 12 + 10 + 6 = 36
计算 P[4]:  P[4] = M[0]*N[2] + M[1]*N[3] + M[2]*N[4] + M[3]*N[5] + M[4]*N[6]
                 = 1*3 + 2*4 + 3*5 + 2*6 + 1*7
                 = 3 + 8 + 15 + 12 + 7 = 45
计算 P[5]:  P[5] = M[0]*N[3] + M[1]*N[4] + M[2]*N[5] + M[3]*N[6] + M[4]*N[7]
                 = 1*4 + 2*5 + 3*6 + 2*7 + 1*8 = 4+10+18+14+8 = 54
计算 P[6]:  P[6] = M[0]*N[4] + M[1]*N[5] + M[2]*N[6] + M[3]*N[7] + M[4]*N[8]
                 = 1*5 + 2*6 + 3*7 + 2*8 + 1*9 = 5+12+21+16+9 = 63
计算 P[7]:  P[7] = M[0]*N[5] + M[1]*N[6] + M[2]*N[7] + M[3]*N[8]
                 = 1*6 + 2*7 + 3*8 + 2*9 = 6+14+24+18 = 62     (右越界一项为 0)
计算 P[8]:  P[8] = M[0]*N[6] + M[1]*N[7] + M[2]*N[8]
                 = 1*7 + 2*8 + 3*9 = 7+16+27 = 50             (右越界两项按 0 处理)

滑动窗口示意 (计算 P[3] 时, M[0] 对准 N[1], 窗口覆盖 N[1] 到 N[5]):
     N:   1   [2    3    4    5    6]   7    8    9
     M:       [M[0] M[1] M[2] M[3] M[4]]
               j=0  j=1  j=2  j=3  j=4
     P:  10   18   36   45   54   63   62   50
                   ^
                P[3] = 36 (由上面 5 个输入元素算出)
```

性能特征：每个输出需要 `MASK_WIDTH` 次乘加（2D 为 `MASK_WIDTH²` 次、3D 为 `MASK_WIDTH³` 次），但相邻输出的窗口**重叠**了 `MASK_WIDTH-1` 个元素。以 2D、`MASK_WIDTH = 5` 为例，一个远离边界的输入元素 `N[y][x]` 会被 **25 个不同的输出**使用（讲义中的 Problem Solving 题：把问题反过来看，每个 mask 系数都对应一个唯一的依赖输出，因此是 `5×5 = 25` 次复用）；3D、`5×5×5` 时复用次数是 **125**。复用次数决定了我们能在片上缓存里省下多少全局内存流量：只要能把这 25（或 125）次访问里的绝大部分变成共享内存/常量内存访问，带宽压力就下降一到两个数量级。

概念 2：卷积核（Mask / Filter / Kernel）与 halo / apron（边界重叠区）

- **定义与目的**：卷积核（讲义中称 mask，也叫 filter 或 kernel）是一个与输入同维度的小常量数组，其元素称为 mask 系数（mask coefficients / weights）。**halo**（也叫 **apron**、**ghost region**，讲义同时使用 ghost cells / apron cells / halo cells 三个词）是指：为了让一个 block 能算出它负责的那块输出，除了这块输出"正下方"的输入之外，还必须在四周多读入 `r = MASK_WIDTH/2` 个元素的输入区域。halo 的目的是让 block 内部的每个输出都能在共享内存里找到它需要的全部 `MASK_WIDTH²` 个输入元素，从而不必去全局内存取。
- **直观解释（"它是什么？"）**：把整幅输入图像切成 `16×16` 的方块分给各个 block，就像把一张大地图裁成小张分给不同的人复印。问题是每个人复制的每一小块，都要参考原图上它**边缘外一圈**的内容（因为 `5×5` 的窗口会伸出去 2 格）。于是每人实际复印的不是 `16×16`，而是 `20×20`——多出来的那一圈"边框"就是 halo。它像书页的页边空白：不是你真正要写的正文，但为了不越界翻到别人的页面，你必须把它一起裁下来。考试里把它叫 ghost cells（幽灵格），因为它们在输入数组的边界之外时根本不存在，是程序"凭空补"出来的。
- **架构/机制图解**：一个 block 的输入 tile 与 halo 的布局如下（`TILE_WIDTH = 16`，`MASK_WIDTH = 5`，`r = 2`）：

```
          <------------- SX = TILE_X + 2r = 20 ------------->
+---------+-----------------------------------------+---------+
|  halo   |                                         |  halo   |  ^ r=2
| (左上角)|            halo (上边 2 行)            | (右上角)|  v
+---------+-----------------------------------------+---------+
|         |                                         |         |
|  halo   |        中心 TILE  16 x 16               |  halo   |  ^
|  (左)   |  本 block 在这里产生 256 个输出元素     |  (右)   |  | TILE_Y
|  2 列   |  每个输出要读 5x5 = 25 个输入元素       |  2 列   |  |  = 16
|         |                                         |         |  v
+---------+-----------------------------------------+---------+
|  halo   |                                         |  halo   |  ^ r=2
| (左下角)|            halo (下边 2 行)            | (右下角)|  v
+---------+-----------------------------------------+---------+
<--- 2 ---><---------------- 16 ------------------><--- 2 --->

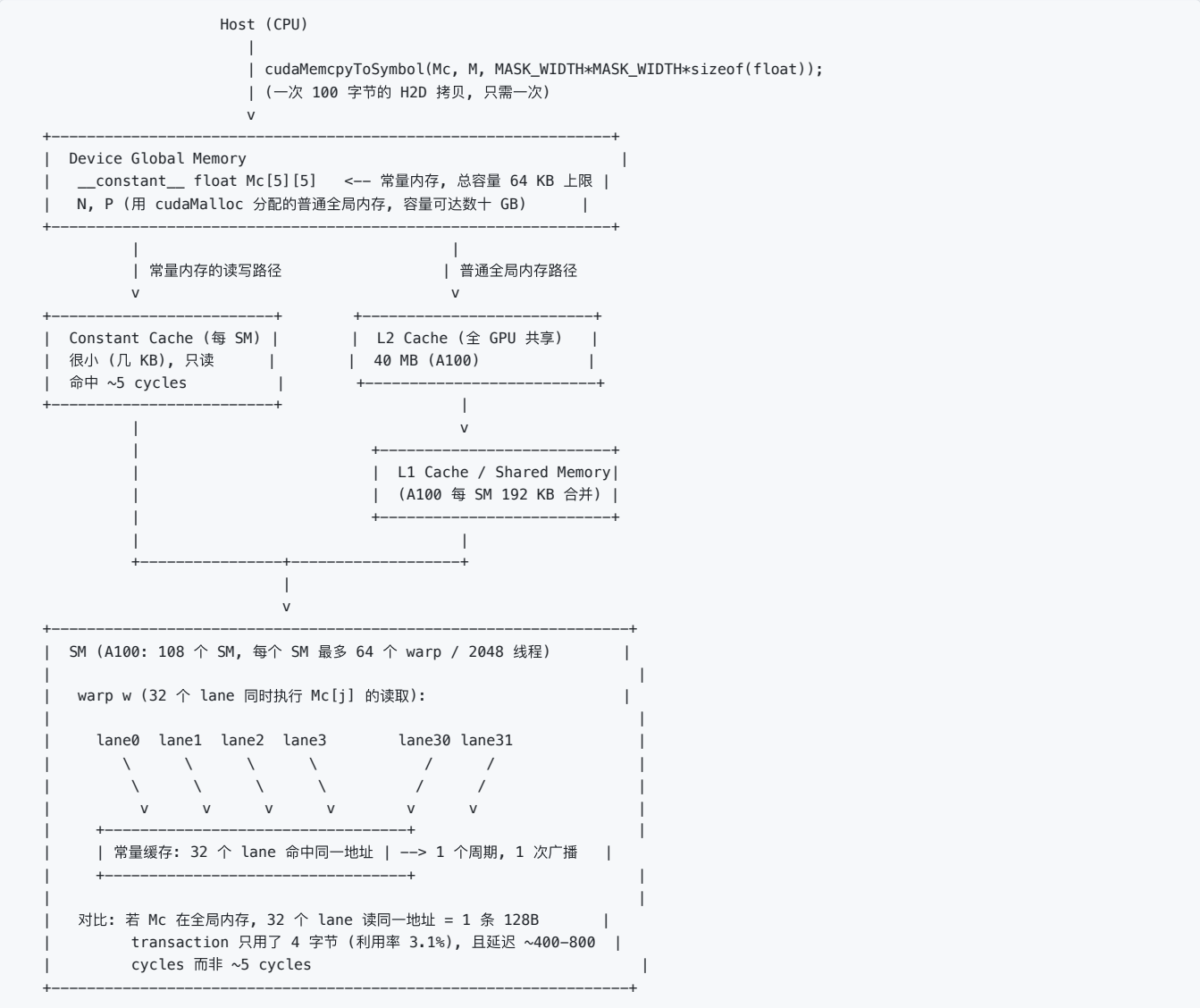
共享内存 tile 元素总数 = (TILE_X + 2r) x (TILE_Y + 2r) = 20 x 20 = 400 个 float = 1600 B
中心区域元素数        = TILE_X x TILE_Y            = 16 x 16 = 256 个 float
halo 元素数           = 400 - 256 = 144 个 float (占总加载量的 36%)
halo 冗余加载比例     = (TILE+2r)^2 / TILE^2 = 400/256 = 1.5625  (即多加载 56.25%)
```

三维时同样的逻辑沿三个方向展开，halo 体积按立方增长：`(TILE+2r)³ / TILE³`。例如 `TILE = 8`、`r = 2` 时是 `12³/8³ = 1728/512 = 3.375`，即为了算 `8³ = 512` 个输出要加载 1728 个输入元素，**多加载 237.5%**。后文概念 8 会给出完整的 3D tile 图。

性能特征：halo 使"每输出对应的输入加载量"从名义上的 1 个元素升到 `(TILE+2r)²/TILE²` 个元素；`TILE` 越大，这个比例越接近 1（`TILE = 8` 时是 2.25，`TILE = 16` 时是 1.5625，`TILE = 32` 时是 1.2656），但 `TILE` 增大会同时增加共享内存占用与寄存器压力，并可能降低占用率。这是一个必须在具体 GPU 上（A100 每 SM 164 KB 共享内存、RTX 4090 每 SM 100 KB）权衡的参数。

概念 3：常量内存与常量缓存广播（Constant Memory & Constant Cache Broadcasting）

- **定义与目的**：常量内存是全局内存中一块**只读**的、容量 **64 KB** 的地址空间，用 `__constant__` 限定符在文件作用域声明，由主机端用 `cudaMemcpyToSymbol` 初始化。它存在的目的是为"整个 grid 里所有线程都读同一份、且只读"的小数据（卷积 mask、物理常数、滤波器系数、变换矩阵）提供一条**不占用 L1/共享内存带宽、且对广播访问极快**的通路。`__constant__` 声明同时告诉编译器和硬件"缓存它是安全的"（因为只读，不存在缓存一致性问题），这是常量缓存能做得比 L1 更激进的依据。
- **直观解释（"它是什么？"）**：把 mask 放进常量内存，就像把**课堂投影的幻灯片**放在讲台正前方：全班 32 个学生（一个 warp）看的是**同一页**，老师只需要念一次，所有人同时看到——不需要每人复印一份。而如果 mask 放在全局内存里，就相当于每个学生都要自己跑去打印室印一页同样内容；放在共享内存里，相当于每组发一张，组内传阅。常量缓存的关键机制叫**广播（broadcast）**：当一个 warp 的 32 个 lane 访问**同一个地址**时，常量缓存一个周期就能把同一个 4 字节值送到全部 32 个 lane，硬件开销等价于一次标量读。卷积正是这种访问模式的完美典型——第 `j` 次迭代时，warp 里 32 个线程读的都是 `Mc[j]`。
- **架构/机制图解**：



关键数字对照（A100，来自讲义 Review: Programmer View of CUDA Memories 与 Ampere SM Memory Architecture）：

```
寄存器          ~1   cycle    每线程私有, 无冲突(除非寄存器组 bank 冲突)
共享内存        ~5   cycles   每个 block 私有, 程序员管理, 32 个 bank
常量内存/缓存   ~5   cycles   整个 grid 只读, 命中即广播, 未命中要回 L2/DRAM
L1 命中         ~30  cycles
L2 命中         ~200 cycles
全局内存        ~400-800 cycles (讲义写作 ~500)
```

什么时候该用常量内存、什么时候该用共享内存、什么时候该用全局内存：

```
+----------------------+---------------------+---------------------------+
| 数据特征              | 最佳存放位置         | 理由                       |
+----------------------+---------------------+---------------------------+
| 全 grid 共用、只读、  | __constant__ (64KB) | warp 内同地址 -> 广播 1 周期|
| 每个线程读同一份      |                     | 不占 L1/共享内存带宽       |
| (卷积 mask, 系数表)   |                     | 不消耗 cudaMalloc 开销     |
+----------------------+---------------------+---------------------------+
| block 内共享、会被    | __shared__          | 复用次数 >> 1, 延迟 ~5 周期 |
| 多个线程以不同地址读  |                     | 需自己加载 + __syncthreads |
| (输入 tile + halo)    |                     | 需处理 bank conflict       |
+----------------------+---------------------+---------------------------+
| 容量大、只读一次或    | 全局内存 + L1/L2    | 容量可达几十 GB, 合并访问  |
| 访问模式不规整        | (__restrict__ 提示) | 大 mask (如 11x11=121<64KB|
|                       |                     | 仍可放常量内存, 但收益递减)|
+----------------------+---------------------+---------------------------+
```

常量内存的**代价**必须一并记住：它只有 64 KB；如果一个 warp 内的 32 个 lane 访问**不同**地址，硬件会把请求**串行化**成最多 32 次独立的常量缓存访问（只要有 2 个不同地址就按分歧处理），此时常量缓存比 L1 更慢。这就是"常量内存只适合广播访问"的严格含义。卷积中 mask 的访问模式是 `Mc[i*MW+j]`，对 warp 内所有 lane 都是同一个表达式、同一个地址，因此 100% 命中广播路径。

概念 4：边界处理（Boundary Handling）与 ghost cells

- **定义与目的**：当输出元素靠近输入数组的边界时，它的 `MASK_WIDTH` 个输入里会有一部分落在数组之外，这些不存在的元素就叫 **ghost cells（幽灵格）**、**halo cells** 或 **apron cells**。边界处理策略决定了这些位置"当作什么值"，直接决定卷积结果的语义（也决定和 CPU 参考实现能否对上）。三种主流策略是：**补零（zero padding）**、**夹紧/复制边缘（clamp / replication）**、**环绕/周期延拓（wrap / periodic）**。
- **直观解释（"它是什么？"）**：想象你拿着一把 5 格宽的尺子量一排砖，尺子两边总要伸出去 2 格。伸到墙外面那两格怎么办？补零 = 假装墙外是空的（值为 0），图像会变暗（边缘被"吸"向黑色）；夹紧 = 假装墙外的砖和贴墙那块一模一样（复制边缘），边缘不会变暗但会被拉宽；环绕 = 把这条砖排首尾接成一个圈，墙外那一格就是另一头的砖（周期延拓），适合本来就周期性的信号（如音频、傅里叶相关的处理）。选哪种没有对错，只有"和你的参考实现是否一致"。
- **架构/机制图解**：三种策略对同一个左边界窗口的作用（`r = 2`）：

```
N 的真实数据从 N[0] 开始; 计算 P[0] 需要 N[-2], N[-1], N[0], N[1], N[2]

(a) 补零 zero padding  (课程默认, Lab 采用)
     N[-2]=0  N[-1]=0  N[0]  N[1]  N[2]
     P[0] = M[0]*0 + M[1]*0 + M[2]*N[0] + M[3]*N[1] + M[4]*N[2]

(b) 夹紧 clamp / replication (复边缘值)
     N[-2]=N[0] N[-1]=N[0] N[0] N[1] N[2]
     P[0] = M[0]*N[0] + M[1]*N[0] + M[2]*N[0] + M[3]*N[1] + M[4]*N[2]
                    ~~~~~~~~~ 相当于把 mask 左半部分折回边缘像素

(c) 环绕 wrap / periodic (首尾相接)
     N[-2]=N[W-2] N[-1]=N[W-1] N[0] N[1] N[2]
     P[0] = M[0]*N[W-2] + M[1]*N[W-1] + M[2]*N[0] + M[3]*N[1] + M[4]*N[2]

三维时同理, 只是要沿 z/y/x 三个方向各判断一次:
     沿 z: 0 或 D-1 处越界   沿 y: 0 或 H-1 处越界   沿 x: 0 或 W-1 处越界
```

三种策略在 CUDA 中的写法（内层循环里的三种"取数"函数）：

```
/* (a) 补零: 越界返回 0 —— 课程 Lab 的默认约定 */
__device__ __forceinline__ float get_zero(const float *N, int idx, int len) {
    return (idx >= 0 && idx < len) ? N[idx] : 0.0f;
}
/* (b) 夹紧: 越界贴到最近的合法下标 */
__device__ __forceinline__ float get_clamp(const float *N, int idx, int len) {
    int k = idx < 0 ? 0 : (idx >= len ? len - 1 : idx);
    return N[k];
}
/* (c) 环绕: 用取模把下标折回数组内 (len 必须是编译期可知或用 % 运算) */
__device__ __forceinline__ float get_wrap(const float *N, int idx, int len) {
    int k = idx % len;
    if (k < 0) k += len;
    return N[k];
}
```

为什么边界分支会导致 warp 发散（warp divergence）：朴素 kernel 的内层循环写成

```
for (int j = 0; j < MASK_WIDTH; ++j) {
    int idx = N_start_point + j;
    if (idx >= 0 && idx < Width)          /* 这一句就是发散源 */
        Pvalue += N[idx] * Mc[j];
}
```

在 SIMT 模型下，一个 warp 的 32 个 lane 共享同一个程序计数器，遇到 `if` 时硬件把"条件为真"的 lane 打开、为假的 lane 关闭（用执行掩码 active mask），**两条路径都要串行执行一遍**。以 `5×5` mask、`16×16` block（256 线程 = 8 warp，每个 warp 覆盖两行、每行 16 个 lane）为例，考察图像**左上角**那个 block：

```
   角块 (blockIdx = (0,0)) 中, 哪些线程的 5x5 窗口越界?
   tx in {0,1} 或 tx in {14,15} 或 ty in {0,1} 或 ty in {14,15}
   越界线程数 = 4*16*2 - 4*4 = 128 - 16 = 112  (占 256 个线程的 43.75%)

   按 warp 细分 (warp w 覆盖 ty = 2w 与 ty = 2w+1 两行, 每行 16 lane):
     warp 0 (ty=0,1)  : 32/32 lane 全部越界  -> 整个 warp 的循环体被谓词屏蔽
     warp 7 (ty=14,15): 32/32 lane 全部越界  -> 同上
     warp 1..6        : 每行有 tx=0,1,14,15 共 4 lane 越界
                        -> 每个 warp 有 4+4 = 8/32 = 25% 的 lane 被屏蔽

   整幅 1024x1024 图像 + 16x16 block 的情况:
     总 block 数 = 64 x 64 = 4096
     边界 block 数 = 4096 - 62 x 62 = 4096 - 3844 = 252   (仅 6.15%)
     -> 换言之 93.85% 的 block 完全没有边界发散
```

结论有两面：一方面，**只有约 6% 的 block 真的会发散**，所以朴素版的边界分支对总性能的影响有限；另一方面，那 25 个带 `if` 的迭代会为**每一个输出**都插入谓词计算与地址计算指令（每个输出约 25 次比较 + 25 次分支判断，而有效 FMA 只有 25 条），指令开销翻倍。**彻底消除它的办法**是"把 halo 预先加载（并补零）到共享内存"：在分块方案里，越界判断只在**加载阶段**做一次（每个线程 1 次），而**计算阶段**变成无分支的纯循环 `Pvalue += Ns[ty+i][tx+j] * Mc[i][j];`。这就是分块卷积相比朴素卷积在"指令效率"上的额外收益。

用「输入并行（input parallelism）」时还要注意：加载阶段的判断式写在 `if` 之外还是之内——

```
float v = 0.0f;
if (row_i >= 0 && row_i < Height && col_i >= 0 && col_i < Width)
    v = N[row_i * Width + col_i];
Ns[ty][tx] = v;                  /* 所有线程都执行这一句, 分支外, 安全 */
__syncthreads();                 /* __syncthreads 必须在分支之外! */
```

千万不要把 `__syncthreads()` 写进 `if (row_i >= 0 && row_i < Height && col_i >= 0 && col_i < Width)` 这个分支里：那会让部分 warp 跳过屏障，轻则死锁、重则读到未初始化数据（正确的写法见示例 2）。

概念 5：分块卷积与 halo 加载（Tiled Convolution with Halo Loading）

- **定义与目的**：分块卷积把输出数组切成 `TILE_WIDTH × TILE_WIDTH` 的瓦片（tile），每个 block 负责一块瓦片；block 协作地把大小为 `(TILE+MASK_WIDTH-1)²` 的**输入 tile（含 halo）**一次性从全局内存搬进共享内存，然后每个线程从共享内存里反复取数计算。它要解决的问题是：朴素版本中同一个输入元素被 25（2D）或 125（3D）个不同输出读取，而这些读取全都打到 L1/L2 上；分块后这些重复读取变成共享内存访问，全局内存流量下降一个数量级。
- **直观解释（"它是什么？"）**：这就像**厨房的案板**。冰箱（全局内存/DRAM）在走廊尽头，取一次菜要跑 400–800 个时钟周期。朴素做法是每做一道菜就跑一趟冰箱取 25 样食材中的一样——反复跑 25 趟。分块做法是：全组人（一个 block 的 400 个线程）先**一次性**把这次要用的所有食材（`20×20` 的输入 tile，含边上多拿的一圈 halo）全搬到案板（共享内存）上，案板就在手边（约 5 个周期），然后在案板上切、配、炒。案板不大（A100 每 SM 164 KB），所以要认真规划"这次到底拿多少菜"——拿多了放不下、占满案板导致同一时间只能有一个 block 在用 SM（占用率下降），拿少了复用不够。

- **架构/机制图解**：分块卷积的三种"谁来干活"策略对比，以及全局内存访问减少倍数的定量计算：

```
策略 A: 输出并行 + halo 从全局内存读 (讲义说: 早期 GPU 上"A really bad idea")
+-----------------+     +-------------------------------------------+
| block (16x16)   |     | 计算阶段: 中心 16x16 走 __shared__ tile    |
| 共享内存只有    | --> | halo (宽 2 的边界列/行) 直接读 global      |
| 中心 16x16      |     | 后果: warp 内一部分 lane 读共享内存,       |
+-----------------+     |       一部分 lane 读全局内存 -> 严重发散   |
                        | 且 halo 只有 2 个元素宽, 凑不满 128B burst |
                        +-------------------------------------------+

策略 B: 输出并行 + halo 也搬进共享内存 (讲义推荐给 Lab 4 的选择之一)
+----------------------+   +-------------------------------------------+
| block (TILE+2r)^2    |   | 1. 全部 400 个线程各搬 1 个元素进共享内存|
| = (20x20) = 400 线程 |-->| 2. __syncthreads()                        |
| 共享内存 20x20 float |   | 3. 前 256 个线程各算 1 个输出 (无分支)    |
+----------------------+   +-------------------------------------------+
优点: 全局访问完全合并; 计算阶段零发散; 缺点: 部分线程搬完就闲置

策略 C: 输入并行 (讲义推荐给 MP-4 的方案)
+----------------------+   +-------------------------------------------+
| block (TILE+2r)^2    |   | 1. 每线程搬 1 个输入元素 (无分支)         |
| 与策略 B 相同        |-->| 2. __syncthreads()                         |
|                      |   | 3. ty<TILE && tx<TILE 的线程算输出       |
+----------------------+   +-------------------------------------------+
优点: 加载阶段零发散, 且避开只有 2 元素宽的窄 global 访问
缺点: 计算阶段有分支 (但共享内存延迟低, 影响小)

全局内存访问减少倍数的定量推导 (2D, 输出并行, TILE x TILE 输出):
  朴素版: 每个输出读 MASK_WIDTH^2 个输入  ->  TILE^2 x MASK_WIDTH^2 次全局访问
  分块版: 只需加载 (TILE+MASK_WIDTH-1)^2 个输入元素
  减少倍数 R = TILE^2 x MASK_WIDTH^2 / (TILE+MASK_WIDTH-1)^2

数值表 (2D, 讲义原表 + 本笔记补算):
  TILE=8,  MW=5:  64x25/144   = 1600/144   = 11.1x     halo 放大 144/64  = 2.250
  TILE=16, MW=5:  256x25/400  = 6400/400   = 16.0x     halo 放大 400/256 = 1.5625
  TILE=32, MW=5:  1024x25/1296= 25600/1296 = 19.7x     halo 放大 1296/1024 = 1.266
  TILE=64, MW=5:  4096x25/4624= 102400/4624= 22.1x     halo 放大 4624/4096 = 1.129
  TILE=8,  MW=9:  64x81/144   = 5184/144   = 36.0x     halo 放大 144/64  = 2.250
  TILE=16, MW=9:  256x81/400  = 20736/400  = 51.8x     halo 放大 400/256 = 1.5625
  TILE=32, MW=9:  1024x81/1600= 82944/1600 = 51.8x     halo 放大 1600/1024 = 1.5625
  TILE=64, MW=9:  4096x81/5184= 331776/5184= 64.0x     halo 放大 5184/4096 = 1.266

一维情形 (讲义原表, R = TILE*MW/(TILE+MW-1)):
  TILE=16,  MW=5 -> 80/20   = 4.0x      TILE=16,  MW=9 -> 144/24  = 6.0x
  TILE=32,  MW=5 -> 160/36  = 4.4x      TILE=32,  MW=9 -> 288/40  = 7.2x
  TILE=64,  MW=5 -> 320/68  = 4.7x      TILE=64,  MW=9 -> 576/72  = 8.0x
  TILE=128, MW=5 -> 640/132 = 4.9x      TILE=128, MW=9 -> 1152/136= 8.5x
  TILE=256, MW=5 -> 1280/260= 4.9x      TILE=256, MW=9 -> 2304/264= 8.7x
```

性能特征的三条要点：

1. **减少倍数只取决于 TILE 与 MASK_WIDTH**，与图像大小无关（忽略边界瓦片）。这就是为什么"每个 block 负责的瓦片越大、mask 越大，分块越值"。
2. **每个输出对应的输入加载量**（halo 放大比）`A = (TILE+2r)²/TILE²`，与减少倍数满足 `R = MASK_WIDTH²/A`。例如 `TILE=16, MW=5`：`A = 1.5625`，`R = 25/1.5625 = 16`。
3. **边界瓦片的比例会变**（讲义 Problem Solving）：如果输入是 `32×32`、输出瓦片 `16×16`、mask `5×5`，则 `block(0,0)` 需要的输入只有 `18×18`（因为左边和上边只有 2 行/列的 halo，另外两侧各 2 行/列被图像边界截断），此时算出的减少比与内部瓦片不同。整幅图越大，边界瓦片占比越小，`R` 越接近上表的渐近值。

概念 6：线程粗化与寄存器分块（Thread Coarsening & Register Tiling）

- **定义与目的**：线程粗化指**让一个线程计算多个输出元素**，而不是"一个线程一个输出"。寄存器分块（register tiling）是它最常见的实现形式：把多个输出元素的累加器（accumulator）放在寄存器里，让它们**共享同一批从共享内存/常量内存取出的数据**。它解决两个问题：(1) 减少共享内存读取次数与地址计算指令数；(2) 在"输入并行"策略下，避免大量线程搬完数据后闲置（提高计算阶段的 warp 利用率）。
- **直观解释（"它是什么？"）**：像用印章盖章时，不是"盖一格、回案板拿一次料"，而是**一次从案板抓一把料，连盖四格**。四格的位置挨在一起，它们需要的输入彼此重叠，所以那一把料（`5×5` 窗口里的 25 个数字）能同时喂给四个累加器。更贴切的类比是**做四份一样的三明治**：你不是做一份去冰箱拿一次生菜，而是一次拿够四份的量，站在案板前连续做四份，切菜、取料、放料的动作只做一遍。
- **架构/机制图解**：以 2D 卷积为例（`TILE = 16`，`MASK_WIDTH = 5`，每线程 `2×2 = 4` 个输出）：

```
变体 1: 一个线程一个输出 (block = 20x20 = 400 线程, 输入并行)
+-------------------------------------------------------------------+
|  共享内存 20x20 (含 halo)                                          |
|  线程 (tx,ty) 搬 1 个元素; ty<16 && tx<16 的 256 个线程各算 1 输出 |
|  每个输出: 25 次共享内存读 + 25 次常量读 + 25 FMA                  |
|  warp 利用: 8 个 warp 中 warp 6,7 完全不参与计算 (ty>=12)          |
|             warp 0-5 每个只有 24/32 lane 活跃 (讲义 Problem Solving)|
+-------------------------------------------------------------------+

变体 2: 一个线程 2x2 个输出 (block = 8x8 = 64 线程, 寄存器分块)
+-------------------------------------------------------------------+
|  tile 16x16 的局部坐标 (每个线程负责一个 2x2 的小方块):            |
|     tx=0        tx=1        tx=2     tx=3        tx=7             |
|   +---------+ +---------+ +---------+ +-------+ +---------+       |
|   |acc[0][0]| |acc[0][0]| |acc[0][0]| |acc[0] | |acc[0][0]| ty=0  |
|   |acc[0][1]| |acc[0][1]| |acc[0][1]| |acc[0] | |acc[0][1]| ty=0  |
|   +---------+ +---------+ +---------+ +-------+ +---------+       |
|   |acc[1][0]| |acc[1][0]| |acc[1][0]| |acc[1] | |acc[1][0]| ty=1  |
|   |acc[1][1]| |acc[1][1]| |acc[1][1]| |acc[1] | |acc[1][1]| ty=1  |
|   +---------+ +---------+ +---------+ +-------+ +---------+       |
|   |acc[0][0]| |acc[0][0]| |acc[0][0]| |acc[0] | |acc[0][0]| ty=2  |
|   |acc[0][1]| |acc[0][1]| |acc[0][1]| |acc[0] | |acc[0][1]| ty=2  |
|   +---------+ +---------+ +---------+ +-------+ +---------+       |
|   |acc[1][0]| |acc[1][0]| |acc[1][0]| |acc[1] | |acc[1][0]| ty=3  |
|   |acc[1][1]| |acc[1][1]| |acc[1][1]| |acc[1] | |acc[1][1]| ty=3  |
|   +---------+ +---------+ +---------+ +-------+ +---------+       |
|   4 个累加器 (acc[0][0], acc[0][1], acc[1][0], acc[1][1]) 常驻寄存器|
|   25 次迭代 x 4 次共享读 = 100 次共享读                            |
|   25 次常量读 (m 只读一次, 喂给 4 个 FMA) -> 常量读降到 1/4        |
|   输出写回 4 次/线程, 但 block 总数减少到 1/4, 调度开销降低        |
```

定量对比（同一 tile，256 个输出）：

```
                        变体 1 (400 线程)      变体 2 (64 线程, 2x2 粗化)
参与加载的线程          400 (每线程 1 次)      64 (每线程 6~7 次)
每输出共享内存读        25                      25 (总量相同, 但每线程 4 输出
                                                  的顺序访问可利用寄存器复用)
每输出常量内存读        25                      25/4 = 6.25 (m 跨 4 个输出复用)
每 4 个输出的常量读     100                     25
每输出全局写            1                       1 (写回地址不连续, 需检查合并性)
寄存器/线程             ~24                     ~40 (4 个累加器 + 5 个加载值)
每 SM 可驻留线程数      2048 (8 个 block)       1638 (寄存器受限: 65536/40)
占用率                  100% (若寄存器够)       1638/2048 = 80%
计算阶段活跃 lane       256/400 = 64%           64/64 = 100%
```

性能特征：粗化把"常量内存读取"与"共享内存地址计算"的成本摊薄了 4 倍，并把计算阶段的活跃 lane 比例从 64% 提到 100%；代价是寄存器压力上升（每多一个输出多一个累加器 + 一组中间值），可能把占用率从 100% 拉到 80% 左右。在 A100 上，占用率 80% 通常仍足以隐藏延迟（每个 warp scheduler 有 12–13 个 warp 可切换），因此粗化一般净赚。3D 卷积里同样的手法沿 z 方向做（示例 3 用 `COARSE_Z = 2`）。

概念 7：卷积的算术强度与 Roofline 分析（Arithmetic Intensity & Roofline）

- **定义与目的**：算术强度 `AI = 浮点操作数 / 从 DRAM 搬运的字节数`（单位 FLOP/Byte）。Roofline 模型说：一个 kernel 的性能上界是 `min(峰值算力, AI × 显存带宽)`。把这两者写在一起就能回答"这个卷积是算力受限还是带宽受限"以及"我离硬件极限还差多远"，从而决定优化方向（减少访存 vs 提高并行度）。
- **直观解释（"它是什么？"）**：把 GPU 想成一家餐厅。厨师的做菜速度（算力，19.5 TFLOP/s）是固定的；服务员从仓库搬食材的速度（带宽，1555 GB/s）也是固定的。每道菜需要多少食材、多少厨师工时，决定了瓶颈在谁身上。如果每搬 1 字节食材只够厨师做 0.5 次操作（AI 很低），那厨师一直在等食材——**带宽受限**；反之 AI 很高时，服务员闲着、厨师忙不过来——**计算受限**。两条线的交点叫**机器平衡点（machine balance）**，A100 上是 `19.5e12 / 1555e9 = 12.5 FLOP/Byte`。
- **架构/机制图解**：卷积的 Roofline 图（以 A100 为基准机）：

```
  性能 (GFLOP/s, log)
    ^
19.5|--------------------------------------------+--- A100 FP32 峰值 19.5 TFLOP/s
    |                                            |
    |                                       ****|   分块 2D (TILE=32, MW=9):
10.0|                                   ****    |   AI = 18 FLOP/B > 12.5
    |                               ****        |   -> 计算受限 (被峰值截断)
    |                          ****             |
 5.0|                  ****  o  分块 2D (TILE=16, MW=5): AI = 8.0 FLOP/B
    |             ****          上限 8.0 x 1555 = 12.4 TFLOP/s (带宽受限)
 2.0|        ****  *  分块 1D (TILE=256, MW=5): AI = 2.5 -> 上限 3.9 TFLOP/s
    |     ****
 0.78|* 朴素 2D: AI = 0.5 FLOP/B -> 上限 0.78 TFLOP/s (峰值的 4%)
   +----+-----+-----+-----+-----+-----+-----+----> 算术强度 (FLOP/Byte, log)
      0.5    1     2     4     8   12.5  20   32
                                  ^
                      机器平衡点 = 19.5e12/1555e9 = 12.5 FLOP/Byte
                      斜线 (****) 的斜率 = 1555 GB/s = 带宽斜坡
                      落在斜线上 = 带宽受限; 落在平线上 = 计算受限

  参考: RTX 4090 的机器平衡点 = 82.6e12/1008e9 = 81.9 FLOP/Byte (斜坡陡得多,
        因此 4090 上卷积更容易变成带宽受限, 分块的价值更大)
```

逐项推导（1D、2D 都算一遍，数字与讲义一致）：

```
  (1) 1D 朴素卷积的算术强度
      每个输出 P[i] 需要:
         MASK_WIDTH 次 N 的加载  (每次 4 字节, 来自全局内存)
         MASK_WIDTH 次 M 的加载  (每次 4 字节, 来自常量缓存 -> 不算 DRAM 流量)
         2 * MASK_WIDTH 次浮点操作 (1 乘 + 1 加, 每个 j 各一次)
      => 每输出 DRAM 字节 = 4 * MASK_WIDTH, 每输出 FLOP = 2 * MASK_WIDTH
      => AI = 2*MW / (4*MW) = 0.5 FLOP/Byte   (讲义写作 2B/FLOP, 完全一致)
      若把 M 的读取也算进"访问次数", 则是 2*MASK_WIDTH 次访存 / 2*MASK_WIDTH FLOP
      => 约 1 FLOP/Byte 的重访量级, 但真正打到 DRAM 的只有 N。

  (2) 2D 朴素卷积
      每输出 FLOP = 2 * MW^2 = 2*25 = 50 (MW=5)
      每输出 DRAM 字节 = 4 * MW^2 = 100 字节 (25 次 4 字节加载)
      => AI = 50/100 = 0.5 FLOP/Byte
      在 A100 上带宽能支持的算力 = 0.5 * 1555 GB/s = 0.78 TFLOP/s
      = 19.5 TFLOP/s 峰值的 4.0%  -> 剩下 96% 的算力全在等内存

  (3) 分块 2D 卷积 (TILE=16, MW=5)
      减少倍数 R = 16.0
      => 每输出 DRAM 字节 = 100/16 = 6.25 字节
      => AI = 50/6.25 = 8.0 FLOP/Byte
      带宽可支撑算力 = 8.0 * 1555 GB/s = 12.4 TFLOP/s = 峰值的 63.7%
      仍然略偏带宽受限, 但已经接近平衡点

  (4) 分块 2D 卷积 (TILE=32, MW=9)
      R = 51.8 -> 每输出字节 = 4*81/51.8 = 6.25 字节
      => AI = 2*81/6.25 = 25.9 FLOP/Byte > 12.5 -> 计算受限 (理论上限 = 峰值)

  (5) 极限分析: 每输出 DRAM 字节的下限是 4 字节 (每个输入元素至少要被搬一次,
      且要被写出的输出元素也有 4 字节)。因此
        AI_max(2D) = 2*MW^2 / 4 = MW^2/2 = 12.5 FLOP/Byte  (MW=5)
        AI_max(3D) = 2*MW^3 / 4 = MW^3/2 = 62.5 FLOP/Byte  (MW=5)
      也就是说 MW=5 的 2D 卷积在 A100 上"无论怎么优化"都只能做到平衡点附近,
      要真正达到计算受限必须用更大的 mask 或批量处理。这解释了讲义 "Need Really
      Big Mask to Balance Resources" 那张表: 1D 卷积要到 MASK_WIDTH=55 才能
      在 5000 GFLOP/s / 192 GB/s 的老 GPU 上跑满算力。

  (6) 讲义的历史数字 (说明"机器平衡点随时间变化, 复用需求越来越苛刻"):
      2010 年: 1000 GFLOP/s 算力 vs 150 GB/s 带宽 -> 0.5*150 = 7.5% of peak
               -> 需要 100/7.5  = 13.3x 复用才能跑满
      2020 年: 5000 GFLOP/s (GRID K520) vs 192 GB/s  -> 1.92% of peak
               -> 需要 100/1.92 = 52.1x 复用
      2023 年: H100 PCIe 26 TFLOP/s vs 2 TB/s         -> 3.85% of peak
               -> 需要 100/3.85 = 25.97x 复用
      本课程基准机 A100: 19.5 TFLOP/s vs 1555 GB/s    -> 5.0% of peak
               -> 需要 100/5.0  = 20x 复用
      (注: 讲义用的是"每 FLOP 2 字节"的朴素卷积口径, 即 AI = 0.5 FLOP/B。)
```

性能特征：100% 峰值只可能在 `AI > 机器平衡点` 时达到。对 A100 + `5×5` mask 的 2D 卷积，理论极限 AI 是 12.5 FLOP/Byte，刚好等于平衡点 12.5，因此**即使做到完美分块也只能跑在平衡点附近**（约 16–20 TFLOP/s，且实际能到 5–10 TFLOP/s 已属优秀）。3D 卷积因为 `AI_max = MW³/2 = 62.5`，far 高于平

衡点，理论上更容易计算受限——但共享内存带宽与寄存器压力会成为新的瓶颈（见示例 3 的分析）。

概念 8：3D 卷积的特殊性（3D Convolution，Lab 4 / MP-4）

- **定义与目的**：3D 卷积把窗口延伸到三个方向：`P[z][y][x] = ΣΣΣ M[i][j][k]·N[z-r+i][y-r+j][x-r+k]`。它在 ECE408 的实验里是 **Lab 4: 3D Convolution（MP-4）**（课程目录页写作 **Lab 6 – Tiled Parallel Convolution**），要求对三维输入数组做 `M×M×M` mask 的卷积，输出尺寸为 `(W-M+1)×(H-M+1)×(D-M+1)`（Lab 约定），需要用共享内存分块把性能做到接近带宽上限。3D 的必要性来自体数据：医学 CT/MRI、流体力学、地震成像、以及 3D CNN 都是天然三维的。
- **直观解释（"它是什么？"）**：2D 卷积像在一张豆腐皮上盖方章；3D 卷积像在**一整块豆腐**里切一个立方体窗口。麻烦有三：第一，halo 变成了一个立体的"壳"，而不是一圈"边框"——壳的体积占比比边框大得多；第二，数据局部性更差，因为三维数组在内存里是按 `(z,y,x)` 线性化的，沿 z 相邻的两个元素在内存里隔了 `W*H*4` 字节（例如 `128×128×4 = 64 KB`），一次 warp 的内层循环如果沿 z 走就完全无法合并；第三，每个线程的窗口有 `M³ = 125` 个元素，如果都摊在寄存器里会直接爆掉 255 个寄存器的上限。
- **架构/机制图解**：3D 输入 tile 与 halo（`TILE_X=TILE_Y=TILE_Z=8`，`MASK_W=5`，`r=2`）：

```
3D 输入 tile: Ns[SZ=12][SY=12][SX=12] = 12*12*12 = 1728 个 float = 6912 字节

沿 z 方向看 (SZ = TILE_Z + 2r = 8 + 4 = 12 个切片, 每个切片自身是 12 x 12):
+--------+--------++--------+--------+--------+--------+--------+--------+--------+--------++--------+--------+
|slice 0 |slice 1 ||slice 2 |slice 3 |slice 4 |slice 5 |slice 6 |slice 7 |slice 8 |slice 9 ||slice10 |slice11 |
+--------+--------++--------+--------+--------+--------+--------+--------+--------+--------++--------+--------+
 \______/          \_____________________________________________________________________/          \______/
  z 左 halo                        中心 TILE_Z = 8 个输出切片 (slice 2 ~ slice 9)                 z 右 halo
  (2 片)                                          (= 输出 oz 的 0..7)                             (2 片)

tile 中 z 下标的对应关系: 输出 oz 的第 i 个 mask 元素落在 tile 的 slice (j + i) 上,
其中 j = oz - blockIdx.z*TILE_Z, 所以 j + i 的取值范围是 0..(8-1+5-1) = 0..11。   [见示例 3 代码]

每个切片 (12 x 12) 的内部结构:
       lx=0   1   2                              9  10  11
     +-------+-----------------------------------+-------+-------+
ly=0 |  H    H    H    H    H    H    H    H    H |  H    H    H | ^
     |  H    H    H    H    H    H    H    H    H |  H    H    H | | r=2
     +-------+-----------------------------------+-------+-------+ v
     |  H    H  +--------------------------------+  H    H    H  |
     |  H    H  |  C    C    C    C    C    C    C  |  H    H    H  | ^
     |  H    H  |  C    C    C    C    C    C    C  |  H    H    H  | |
     |  H    H  |  中心 TILE_Y = 8 行 x TILE_X = 8 列 |  H    H    H  | | 8
     |  H    H  |  本 block 在此产生 8x8 = 64 个输出 |  H    H    H  | | (每片)
     |  H    H  |  (z 方向共 8 片 -> block 共 512 输出)|  H    H    H  | |
     |  H    H  +--------------------------------+  H    H    H  | v
     +-------+-----------------------------------+-------+-------+
     |  H    H    H    H    H    H    H    H    H |  H    H    H | ^
     |  H    H    H    H    H    H    H    H    H |  H    H    H | | r=2
     +-------+-----------------------------------+-------+-------+ v
     <-- 2 --><------------ 8 ----------------><-- 2 -->

tile 体积 = (TILE+2r)^3 = 12^3 = 1728;  中心 = 8^3 = 512
halo 体积 = 1728 - 512 = 1216 个 float  (占 70.4%!)
halo 放大比 A = (8+4)^3/8^3 = 1728/512 = 3.375  (多加载 237.5%)
访问减少倍数 R = 512*125/1728 = 64000/1728 = 37.04x

对比 2D (TILE=16, MW=5): halo 放大 1.5625, 减少倍数 16x
=> 3D 的 halo 冗余大得多, 但 float 复用次数(125)也大得多, 净收益仍然显著。

三维数组的内存线性化 (按 (z,y,x) 行主序):
  idx = x + y*W + z*W*H
  沿 x 相邻: +1        (连续, 可合并)
  沿 y 相邻: +W        (间隔 W*4 字节)
  沿 z 相邻: +W*H      (间隔 W*H*4 字节; W=H=128 时 = 64 KB, 完全不合并且跨页)
```

3D 卷积的四个额外难点与对策：

```
+----------------------+----------------------------+----------------------------------+
| 难点                 | 具体数字 (TILE=8, MW=5)    | 对策                             |
+----------------------+----------------------------+----------------------------------+
| halo 体积爆炸        | halo 占 tile 的 70.4%      | 增大 TILE: TILE=16 时 A =        |
|                      | A = 1728/512 = 3.375       | 20^3/16^3 = 8000/4096 = 1.953    |
|                      | (多加载 237.5%)            | (halo 占 48.8%), 减少倍数从 37x  |
|                      |                            | 升到 64x; 但共享内存要 32 KB/block|
| 共享内存占用         | 12^3 * 4B = 6912 B / block | A100 有 164 KB/SM -> 最多 23 个  |
|                      |                            | block; 用 padding 后 7488 B      |
| 访存局部性更差       | 沿 z 步长 = W*H*4 = 64 KB  | 让加载循环沿 x 连续 (合并访问),  |
|                      | 远超一条 128B cache line   | z 只作为外层索引                 |
| 寄存器压力           | 125 个 mask 系数 + 累加器  | mask 放 __constant__ (125*4 =    |
|                      | 不可能全放寄存器           | 500 B << 64 KB); 累加器只留 1-4 个|
| 维度多导致索引复杂   | 3 层循环 + 3 个边界判断    | 用 cudaMalloc3D/cudaPitchedPtr   |
|                      |                            | 或一维展平 idx = x+y*W+z*W*H     |
+----------------------+----------------------------+----------------------------------+
```

`cudaMalloc3D` / `cudaMemcpy3D` / `cudaPitchedPtr` 的用法（Lab 4 的另一种数据组织方式，也是 CUDA 处理 3D 数组的官方接口）：

```
/* 方式一: 一维展平 (示例 3 采用, 简单直观)
   idx = x + y*W + z*W*H;  cudaMalloc(&d_N, W*H*D*sizeof(float));
   一次 cudaMemcpy 传输 W*H*D 个 float。 */

/* 方式二: cudaMalloc3D + cudaPitchedPtr (对行首对齐更友好) */
cudaExtent extent = make_cudaExtent(W * sizeof(float), H, D);
cudaPitchedPtr d_Np;                        /* 内含 ptr / pitch / xsize / ysize */
CUDA_CHECK(cudaMalloc3D(&d_Np, extent));  /* 硬件会把 pitch 对齐到 512 字节 */
cudaMemcpy3DParms p = {0};
p.srcPtr = make_cudaPitchedPtr((void *)h_N, W * sizeof(float), W, H);
p.dstPtr = d_Np;
p.extent = extent;
p.kind   = cudaMemcpyHostToDevice;
CUDA_CHECK(cudaMemcpy3D(&p));               /* 一次调用完成 W*H*D 的 H2D 拷贝 */
/* kernel 中访问 (x, y, z): 先用 int Wp = d_Np.pitch / sizeof(float); 得到行长,
   然后 index = z * Wp * H + y * Wp + x。 pitch >= W*4, 多出的填充字节不可读。 */
```

`cudaPitchedPtr` 的核心是 `pitch`（每行字节数，即"从第 y 行第 0 列到第 y+1 行第 0 列的字节数"），它保证每一行的起始地址都满足对齐要求，从而使沿 x 的 warp 访问能整齐地落在 128 字节事务里。代价是 kernel 里**不能再用 `y*W + x`**，必须用 `y*(pitch/4) + x`；忘记这一点是 Lab 4 最常见的 bug（数据整体错位，验证全 FAIL，但 kernel 本身不报错）。

代码示例与性能分析

> **示例 1：朴素 2D 卷积 + 常量内存存放 mask（conv2d_naive_constant.cu）**

```
// 文件: conv2d_naive_constant.cu
// 编译: nvcc -O3 -arch=sm_80 conv2d_naive_constant.cu -o conv2d_naive
// 运行: ./conv2d_naive             # 默认 2048x2048 输入, 5x5 mask
//       ./conv2d_naive 1024 1024   # 指定输入尺寸
//
// 功能: 朴素(未分块) 2D 卷积, 每个线程计算一个输出元素。
//       - 5x5 卷积核放在 __constant__ 常量内存 Mc 中, 用 cudaMemcpyToSymbol 初始化
//       - 输出尺寸与输入相同(centered 形式), 越界的 ghost 元素按 0 处理 (zero padding)
//       - 打印耗时 / GB/s / GFLOP/s, 并与 CPU 参考实现逐元素对比

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define MASK_WIDTH 5
#define MASK_RADIUS (MASK_WIDTH / 2)

/* 常量内存: 文件作用域声明, 所有 kernel 可见, 上限 64 KB */
static __constant__ float Mc[MASK_WIDTH * MASK_WIDTH];

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

/* ------------------------- GPU: 朴素 2D 卷积 ------------------------- */
__global__ void conv2d_naive_kernel(const float * __restrict__ N,
                                    float * __restrict__ P,
                                    int Width, int Height)
{
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row >= Height || col >= Width) return;      /* 输出越界线程直接退出 */

    float Pvalue = 0.0f;
    #pragma unroll
    for (int i = 0; i < MASK_WIDTH; ++i) {
        int nrow = row - MASK_RADIUS + i;
        if (nrow < 0 || nrow >= Height) continue;   /* ghost 行 -> 视作 0 */
        #pragma unroll
        for (int j = 0; j < MASK_WIDTH; ++j) {
            int ncol = col - MASK_RADIUS + j;
            if (ncol < 0 || ncol >= Width) continue;/* ghost 列 -> 视作 0 */
            Pvalue += N[nrow * Width + ncol] * Mc[i * MASK_WIDTH + j];
        }
    }
    P[row * Width + col] = Pvalue;
}

/* 主机端 mask 副本, 供 CPU 参考实现使用 (与 device 端 Mc 内容一致) */
static float h_Mck[MASK_WIDTH * MASK_WIDTH];

/* ------------------------- CPU 参考实现 ------------------------- */
static void conv2d_cpu_reference(const float *N, float *P, int Width, int Height)
{
    for (int row = 0; row < Height; ++row) {
        for (int col = 0; col < Width; ++col) {
            float sum = 0.0f;
            for (int i = 0; i < MASK_WIDTH; ++i) {
                int nrow = row - MASK_RADIUS + i;
                if (nrow < 0 || nrow >= Height) continue;
                for (int j = 0; j < MASK_WIDTH; ++j) {
                    int ncol = col - MASK_RADIUS + j;
                    if (ncol < 0 || ncol >= Width) continue;
                    sum += N[nrow * Width + ncol] *
                           h_Mck[i * MASK_WIDTH + j];
                }
            }
            P[row * Width + col] = sum;
        }
    }
```

```
}

/* 主机端 mask 副本 (已在 CPU 参考实现之前声明, 见上) */

int main(int argc, char **argv)
{
    int Width  = (argc > 1) ? atoi(argv[1]) : 2048;
    int Height = (argc > 2) ? atoi(argv[2]) : 2048;
    size_t nElem = (size_t)Width * Height;

    printf("=== 朴素 2D 卷积 (常量内存 mask) ===\n");
    printf("输入尺寸 %d x %d, mask %d x %d\n", Width, Height, MASK_WIDTH, MASK_WIDTH);

    /* 1. 主机端分配与初始化 */
    float *h_N = (float *)malloc(nElem * sizeof(float));
    float *h_P = (float *)malloc(nElem * sizeof(float));
    float *h_Pref = (float *)malloc(nElem * sizeof(float));
    if (!h_N || !h_P || !h_Pref) { fprintf(stderr, "host malloc failed\n"); return 1; }

    srand(1234);
    for (size_t i = 0; i < nElem; ++i) h_N[i] = (float)(rand() % 16);

    /* 2. 初始化 mask: 用经典的 5x5 高斯近似模糊核 */
    const float mk[MASK_WIDTH * MASK_WIDTH] = {
        1,  4,  6,  4, 1,
        4, 16, 24, 16, 4,
        6, 24, 36, 24, 6,
        4, 16, 24, 16, 4,
        1,  4,  6,  4, 1
    };
    float msum = 0.0f;
    for (int i = 0; i < MASK_WIDTH * MASK_WIDTH; ++i) msum += mk[i];
    for (int i = 0; i < MASK_WIDTH * MASK_WIDTH; ++i) {
        h_Mck[i] = mk[i] / msum;              /* 归一化, 保证模糊不改变整体亮度 */
    }

    /* 3. 把 mask 拷进常量内存: 必须用 cudaMemcpyToSymbol, 不能用 cudaMemcpy */
    CUDA_CHECK(cudaMemcpyToSymbol(Mc, h_Mck,
                                  MASK_WIDTH * MASK_WIDTH * sizeof(float)));

    /* 4. 设备端分配 + 传输输入 */
    float *d_N = NULL, *d_P = NULL;
    CUDA_CHECK(cudaMalloc((void **)&d_N, nElem * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void **)&d_P, nElem * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_N, h_N, nElem * sizeof(float), cudaMemcpyHostToDevice));

    /* 5. 启动配置: 16x16 的二维 block */
    dim3 block(16, 16);
    dim3 grid((Width + block.x - 1) / block.x, (Height + block.y - 1) / block.y);
    printf("grid = (%u, %u), block = (%u, %u) = %u 线程/block, %u warp/block\n",
           grid.x, grid.y, block.x, block.y, block.x * block.y,
           (block.x * block.y + 31) / 32);

    /* 6. 预热 + 计时 (5 次取平均, 用 cudaEvent 计时) */
    conv2d_naive_kernel<<<grid, block>>>(d_N, d_P, Width, Height);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t t0, t1;
    CUDA_CHECK(cudaEventCreate(&t0));
    CUDA_CHECK(cudaEventCreate(&t1));
    const int REPEAT = 5;
    CUDA_CHECK(cudaEventRecord(t0));
    for (int r = 0; r < REPEAT; ++r)
        conv2d_naive_kernel<<<grid, block>>>(d_N, d_P, Width, Height);
    CUDA_CHECK(cudaEventRecord(t1));
    CUDA_CHECK(cudaEventSynchronize(t1));
    CUDA_CHECK(cudaGetLastError());

    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, t0, t1));
    ms /= REPEAT;

    /* 7. 性能指标: FLOP 数 = 2 * MASK_WIDTH^2 * 输出元素数 */
    double flops  = 2.0 * MASK_WIDTH * MASK_WIDTH * (double)nElem;
    double bytes  = 2.0 * (double)nElem * sizeof(float);   /* 至少读写一遍输入/输出 */
    printf("耗时        : %.3f ms\n", ms);
    printf("GFLOP/s     : %.2f\n", flops / (ms * 1e6));
```

```
    printf("有效带宽     : %.2f GB/s (按 (读入+写出) 计算)\n",
           bytes / (ms * 1e6));

    /* 8. 取回结果并与 CPU 参考实现对比 */
    CUDA_CHECK(cudaMemcpy(h_P, d_P, nElem * sizeof(float), cudaMemcpyDeviceToHost));
    conv2d_cpu_reference(h_N, h_Pref, Width, Height);

    double maxErr = 0.0;
    for (size_t i = 0; i < nElem; ++i) {
        double e = fabs((double)h_P[i] - (double)h_Pref[i]);
        if (e > maxErr) maxErr = e;
    }
    printf("最大绝对误差 : %.3e\n", maxErr);
    printf("验证结果     : %s\n", (maxErr < 1e-3) ? "PASS" : "FAIL");

    CUDA_CHECK(cudaEventDestroy(t0));
    CUDA_CHECK(cudaEventDestroy(t1));
    CUDA_CHECK(cudaFree(d_N));
    CUDA_CHECK(cudaFree(d_P));
    free(h_N); free(h_P); free(h_Pref);
    return 0;
}
```

- **【代码做什么？】**
- **准备 mask 并送进常量内存**：主机端用 `1,4,6,4,1 / 4,16,24,16,4 / 6,24,36,24,6 / 4,16,24,16,4 / 1,4,6,4,1` 这 5 行数字构成经典高斯核（除以总和 256 归一化，得到 5×5 的模糊核），然后用 `cudaMemcpyToSymbol(Mc, h_Mck, 100)` 把它从主机内存拷进**设备端常量内存**。这一步只需一次，100 字节的开销可忽略。注意 `Mc` 声明在文件作用域、`static __constant__`，kernel 直接按名字访问它，**不需要把它作为参数传入**（讲义原话：note that file-scope Mc is visible to kernel）。
- **分配并初始化**：`cudaMalloc` 两块 `Width*Height*4` 字节的设备内存 `d_N`、`d_P`，`cudaMemcpy` 把输入从主机拷到设备。
- **线程映射**：`dim3 block(16,16)` 共 256 个线程；`grid = (Width/16, Height/16)`。线程 `(threadIdx.x, threadIdx.y)` 负责输出 `P[row*Width + col]`，其中 `col = blockIdx.x*16 + threadIdx.x`、`row = blockIdx.y*16 + threadIdx.y`。这是最简单的"一个线程一个输出"映射。
- **计算**：外层 `i` 遍历 mask 的 5 行，`nrow = row - 2 + i`；若越界则 `continue`（等价于该行贡献 0）。内存 `j` 遍历 5 列，同样越界 `continue`。内层累计 `Pvalue += N[nrow*Width + ncol] * Mc[i*5+j]`。
- **写回与验证**：`P[row*Width+col] = Pvalue` 一次写入；主机取回 `h_P`，用三重循环的 CPU 参考实现（用同一份归一化 mask 的主机副本 `h_Mck`）逐元素对比，误差阈值 `1e-3`。
- **计时**：先跑一次预热，再用 `cudaEventRecord` 包裹 5 次 kernel 执行，取平均毫秒数，换算 GFLOP/s 和有效带宽。
- **【并行机制与硬件映射解说】**
- **warp 与调度**：block 16×16 = 256 线程 = 8 个 warp。CUDA 按 `threadIdx.x` 最快变化线性化，所以 **warp w 覆盖 ty = 2w 与 ty = 2w+1 两行、每行 16 个 lane**。A100 每个 SM 有 4 个 warp scheduler，每个 scheduler 每周期可发射 1 条 warp 指令；本 kernel 寄存器用量约 24 个/线程（25 次 FMA 的累加器 1 个 + 地址/循环变量），因此 `65536 寄存器 / (256 线程 × 24) = 10.6 → 10 个 block`，但受"每 SM 最多 2048 线程"限制，实际驻留 `2048/256 = 8` 个 block = 64 个 warp = **100% 占用率**（A100 每 SM 上限 64 warp）。共享内存用量为 0，不构成限制。
- **常量内存广播（本示例的核心机制）**：内层循环第 `j` 次迭代时，warp 里 32 个 lane 计算的地址表达式都是 `Mc[i*5+j]`——**同一个地址**。常量缓存因此每周期只做 **1 次广播**，代价等价于 1 次标量读（约 5 个时钟周期），而不是 32 次独立读。一个线程总共读 25 次 `Mc`（i、j 各 5），全部走广播路径；如果把 mask 放在全局内存，这 25 次读取每次都会产生一条 32 lane 访问同一地址的 128B transaction，**只用了其中 4 字节（利用率 3.1%）**，并且要走 L1/L2 甚至 DRAM（延迟 400–800 周期）。这就是常量内存对本 kernel 的决定性贡献。
- **全局内存读取的合并性**：`N[nrow*Width + ncol]`。一个 warp 的 32 个 lane 中，lane 0–15 的 `nrow` 相同、`ncol` 连续 16 个（64 字节），lane 16–31 在下一行（地址相差 `Width*4 = 8192` 字节）。因此每次迭代产生 **2 条 64 字节的连续段**：若 `ncol` 起始地址按 8 字节偏移（因为 `col - 2`），这两段各自可能跨越 1 条 128B cache line 的边界，实际需要 2–3 条 128B 事务来传 128 字节有效数据。由于 `i` 与 `j` 的偏移只改变 2 个元素，**相邻迭代访问高度重叠**，L1/L2 命中率极高（典型的空间/时间局部性都满足）。
- **输出写回是否合并**：`P[row*Width + col]`，warp 内两行各 16 个 lane 连续 → 两条 64 字节的连续段。当 `Width` 是 32 的倍数时（2048 是），每段落在一条 128B cache line 内，写回是**完全合并**的（每条 128B 事务一半有用，需要 2 条事务覆盖 128 字节数据）。若 `Width` 不是 32 的倍数（如 1000），行首不对齐会让每段跨 2 条 cache line，写回事务数翻倍——这是"输入宽度最好取 32 的倍数"的实际理由。
- **warp 发散（边界分支）**：`if (nrow < 0 || nrow >= Height) continue;` 与内层的列判断对于**内部 block 从不触发**（无发散），但对边界 block 会产生谓词屏蔽。以一个 `5×5` mask、16×16 block 的图像**左上角 block** 为例：`tx ∈ {0,1}` 或 `tx ∈ {14,15}` 或 `ty ∈ {0,1}` 或 `ty ∈ {14,15}` 的线程窗口越界，共 `4*16*2 - 4*4 = 112` 个线程（占 43.75%）。按 warp 看：warp 0（ty=0,1）与 warp 7（ty=14,15）的 32 个 lane **全部**越界，它们的 `Pvalue +=` 被整体屏蔽；warp 1–6 每行两端的 4 个 lane 越界，即每个 warp 有 8/32 = 25% 的 lane 被屏蔽。整幅 1024×1024 图中边界 block 只有 `4096 - 62*62 = 252` 个（**6.15%**），所以发散对总吞吐影响有限；真正的代价是 **25 次循环里每轮都要算一遍谓词与地址**（约 50 条额外指令 vs 25 条 FMA），使 IPC 效率减半。
- **寄存器与依赖链**：`Pvalue` 是一条 25 次串行 FMA 依赖链。A100 FP32 FMA 延迟约 4 个周期，25 × 4 = 100 周期的关键路径。靠 8 个 block × 8 warp = 64 warp 的并发切换即可完全隐藏（每个 scheduler 有 16 个 warp 可轮换）。
- **【性能优化分析】**

- **算术强度**：`AI = 2*MW² / (4*MW²) = 0.5 FLOP/Byte`（每输出 50 次 FLOP、至少要下沉 25 次 4 字节读）。A100 的机器平衡点是 `19.5e12/1555e9 = 12.5 FLOP/Byte`，**本 kernel 的 AI 只有平衡点的 1/25**，因此它在 Roofline 图上远远落在带宽斜坡的最低端。
- **带宽上限**：`0.5 × 1555 GB/s = 0.78 TFLOP/s`，即峰值的 4.0%。以 2048×2048 输入（4.19M 输出、209.7 MFLOP）为例：即使 DRAM 流量被完美压缩到"读一遍 + 写一遍 = 33.6 MB"，理想时间也只有 `33.6e6/1555e9 = 21.6 µs`；而 25 次读中大部分命中的是 **L2**（A100 L2 带宽约 5–6 TB/s），L2 实际流量为 `4.19M × 25 × 4B = 419 MB`，对应 `419e6/5.5e12 = 76 µs`——**这才是朴素版的真实下界**，说明它既受 L2 带宽限制、又受大量 LDG 指令与地址计算拖累。A100 上这类 kernel 的**典型观测耗时是 180–260 µs（约 0.8–1.2 TFLOP/s，有效带宽 130–190 GB/s）**。
- **占用率**：100%（8 block/SM × 256 线程 = 2048 线程 = 64 warp），寄存器与共享内存都不限制。**占用率不是瓶颈**——这类"高占用率 + 低算术强度"的组合是典型的带宽受限特征。
- **判定结论**：**内存带宽受限（且被 L2 带宽与指令开销共同压低）**。优化方向按收益排序：(1) 用共享内存分块把 25 次输入读减少到 `400/256 = 1.56` 次（示例 2）；(2) 把边界分支从计算循环里挪出去（同样由示例 2 完成）；(3) 用线程粗化摊薄常量读与地址计算（示例 2 的第二个 kernel）；(4) 用 `float4` 向量化加载进一步减少指令数。

示例 2：分块 2D 卷积（含 halo 协作加载 + 线程粗化）（conv2d_tiled_halo.cu）

```
// 文件: conv2d_tiled_halo.cu
// 编译: nvcc -O3 -arch=sm_80 conv2d_tiled_halo.cu -o conv2d_tiled
// 运行: ./conv2d_tiled             # 默认 2048x2048 输入, 5x5 mask
//       ./conv2d_tiled 1024 1024
//
// 功能: 在同一份程序里实现并对比三个版本:
//       (A) conv2d_naive_kernel        朴素版 (对照, 与示例 1 相同)
//       (B) conv2d_tiled_kernel        分块版: block=(20x20), 共享内存 20x20 含 halo,
//                                      每线程搬 1 个元素, 前 256 个线程各算 1 个输出
//       (C) conv2d_tiled_coarse_kernel 分块+线程粗化: block=(8x8), 每线程算 2x2=4 个输出
//       三者输出全部与 CPU 参考实现比对, 并打印加速比

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define TILE_WIDTH   16
#define MASK_WIDTH   5
#define MASK_RADIUS  (MASK_WIDTH / 2)
#define IN_TILE      (TILE_WIDTH + MASK_WIDTH - 1)   /* 20: 输入 tile 的一边 */
#define COARSE       2                               /* 粗化: 每线程 2x2 个输出 */

static __constant__ float Mc[MASK_WIDTH * MASK_WIDTH];
static float h_Mck[MASK_WIDTH * MASK_WIDTH];

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

/* ---------------- (A) 朴素版: 一个线程一个输出, 边界分支 ---------------- */
__global__ void conv2d_naive_kernel(const float * __restrict__ N,
                                    float * __restrict__ P,
                                    int Width, int Height)
{
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row >= Height || col >= Width) return;

    float Pvalue = 0.0f;
    #pragma unroll
    for (int i = 0; i < MASK_WIDTH; ++i) {
        int nrow = row - MASK_RADIUS + i;
        if (nrow < 0 || nrow >= Height) continue;
        #pragma unroll
        for (int j = 0; j < MASK_WIDTH; ++j) {
            int ncol = col - MASK_RADIUS + j;
            if (ncol < 0 || ncol >= Width) continue;
            Pvalue += N[nrow * Width + ncol] * Mc[i * MASK_WIDTH + j];
        }
    }
    P[row * Width + col] = Pvalue;
}

/* ---------------- (B) 分块版: halo 协作加载 + 无分支计算 ---------------- */
__global__ void conv2d_tiled_kernel(const float * __restrict__ N,
                                    float * __restrict__ P,
                                    int Width, int Height)
{
    __shared__ float Ns[IN_TILE][IN_TILE];        /* 20 x 20 = 400 float = 1600 B */

    const int tx = threadIdx.x;                   /* 0..19 */
    const int ty = threadIdx.y;                   /* 0..19 */

    /* 该线程负责的输入坐标 (block 的左上角是输出瓦片左上角再向左上各推 radius) */
    const int row_i = blockIdx.y * TILE_WIDTH - MASK_RADIUS + ty;
    const int col_i = blockIdx.x * TILE_WIDTH - MASK_RADIUS + tx;

    /* 1. 协作加载 (TILE+2r)^2 个元素, 越界元素一律填 0 (zero padding) */
    float v = 0.0f;
    if (row_i >= 0 && row_i < Height && col_i >= 0 && col_i < Width)
```

```
        v = N[row_i * Width + col_i];
    Ns[ty][tx] = v;                                 /* 分支之外: 所有线程都参与 */

    __syncthreads();                                /* 等整块 tile 就绪, 必须在分支外 */

    /* 2. 前 TILE x TILE 个线程各算一个输出 (计算阶段无任何分支) */
    if (ty < TILE_WIDTH && tx < TILE_WIDTH) {
        const int row_o = blockIdx.y * TILE_WIDTH + ty;
        const int col_o = blockIdx.x * TILE_WIDTH + tx;
        if (row_o < Height && col_o < Width) {
            float Pvalue = 0.0f;
            #pragma unroll
            for (int i = 0; i < MASK_WIDTH; ++i)
                #pragma unroll
                for (int j = 0; j < MASK_WIDTH; ++j)
                    Pvalue += Ns[ty + i][tx + j] * Mc[i * MASK_WIDTH + j];
            P[row_o * Width + col_o] = Pvalue;
        }
    }
}

/* ----------------- (C) 分块 + 线程粗化: 每线程 2x2 = 4 个输出 ----------------- */
__global__ void conv2d_tiled_coarse_kernel(const float * __restrict__ N,
                                           float * __restrict__ P,
                                           int Width, int Height)
{
    __shared__ float Ns[IN_TILE][IN_TILE];

    const int tx  = threadIdx.x;                    /* 0..7 */
    const int ty  = threadIdx.y;                    /* 0..7 */
    const int tid = ty * blockDim.x + tx;           /* 0..63 */
    const int nth = blockDim.x * blockDim.y;        /* 64 */

    const int row_b = blockIdx.y * TILE_WIDTH - MASK_RADIUS;
    const int col_b = blockIdx.x * TILE_WIDTH - MASK_RADIUS;

    /* 1. 64 个线程协作搬 400 个元素 (每个线程 6-7 次, 循环步长 = 线程数) */
    for (int l = tid; l < IN_TILE * IN_TILE; l += nth) {
        const int ly = l / IN_TILE;
        const int lx = l - ly * IN_TILE;
        const int gr = row_b + ly;
        const int gc = col_b + lx;
        float v = 0.0f;
        if (gr >= 0 && gr < Height && gc >= 0 && gc < Width)
            v = N[gr * Width + gc];
        Ns[ly][lx] = v;
    }
    __syncthreads();

    /* 2. 每个线程算 2x2 个输出, 4 个累加器常驻寄存器 */
    float acc[COARSE][COARSE];
    #pragma unroll
    for (int dy = 0; dy < COARSE; ++dy)
        #pragma unroll
        for (int dx = 0; dx < COARSE; ++dx)
            acc[dy][dx] = 0.0f;

    const int oy0 = ty * COARSE;                    /* tile 内的局部输出行 */
    const int ox0 = tx * COARSE;                    /* tile 内的局部输出列 */

    #pragma unroll
    for (int i = 0; i < MASK_WIDTH; ++i) {
        #pragma unroll
        for (int j = 0; j < MASK_WIDTH; ++j) {
            const float m = Mc[i * MASK_WIDTH + j];   /* 1 次常量读喂 4 个 FMA */
            #pragma unroll
            for (int dy = 0; dy < COARSE; ++dy)
                #pragma unroll
                for (int dx = 0; dx < COARSE; ++dx)
                    acc[dy][dx] += Ns[oy0 + dy + i][ox0 + dx + j] * m;
        }
    }

    #pragma unroll
    for (int dy = 0; dy < COARSE; ++dy) {
        #pragma unroll
        for (int dx = 0; dx < COARSE; ++dx) {
            const int row_o = blockIdx.y * TILE_WIDTH + oy0 + dy;
```

```
            const int col_o = blockIdx.x * TILE_WIDTH + ox0 + dx;
            if (row_o < Height && col_o < Width)
                P[row_o * Width + col_o] = acc[dy][dx];
        }
    }
}

/* ---------------- CPU 参考实现 ---------------- */
static void conv2d_cpu_reference(const float *N, float *P, int Width, int Height)
{
    for (int row = 0; row < Height; ++row) {
        for (int col = 0; col < Width; ++col) {
            float sum = 0.0f;
            for (int i = 0; i < MASK_WIDTH; ++i) {
                int nrow = row - MASK_RADIUS + i;
                if (nrow < 0 || nrow >= Height) continue;
                for (int j = 0; j < MASK_WIDTH; ++j) {
                    int ncol = col - MASK_RADIUS + j;
                    if (ncol < 0 || ncol >= Width) continue;
                    sum += N[nrow * Width + ncol] * h_Mck[i * MASK_WIDTH + j];
                }
            }
            P[row * Width + col] = sum;
        }
    }
}

/* 计时辅助: 通过函数指针启动 kernel 必须用 cudaLaunchKernel
   (<<<>>> 语法不能直接作用于函数指针) */
static float time_kernel(void (*kern)(const float *, float *, int, int),
                         dim3 grid, dim3 block, const float *d_N, float *d_P,
                         int W, int H, int repeat)
{
    void *args[] = { (void *)&d_N, (void *)&d_P, (void *)&W, (void *)&H };
    CUDA_CHECK(cudaLaunchKernel((const void *)kern, grid, block, args, 0, 0));
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t t0, t1;
    CUDA_CHECK(cudaEventCreate(&t0));
    CUDA_CHECK(cudaEventCreate(&t1));
    CUDA_CHECK(cudaEventRecord(t0));
    for (int r = 0; r < repeat; ++r)
        CUDA_CHECK(cudaLaunchKernel((const void *)kern, grid, block, args, 0, 0));
    CUDA_CHECK(cudaEventRecord(t1));
    CUDA_CHECK(cudaEventSynchronize(t1));
    CUDA_CHECK(cudaGetLastError());
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, t0, t1));
    CUDA_CHECK(cudaEventDestroy(t0));
    CUDA_CHECK(cudaEventDestroy(t1));
    return ms / repeat;
}

static int verify(const float *h_P, const float *h_Pref, size_t n, const char *tag)
{
    double maxErr = 0.0;
    for (size_t i = 0; i < n; ++i) {
        double e = fabs((double)h_P[i] - (double)h_Pref[i]);
        if (e > maxErr) maxErr = e;
    }
    printf("  [%s] 最大绝对误差 %.3e -> %s\n", tag, maxErr,
           (maxErr < 1e-3) ? "PASS" : "FAIL");
    return (maxErr < 1e-3) ? 1 : 0;
}

int main(int argc, char **argv)
{
    int Width  = (argc > 1) ? atoi(argv[1]) : 2048;
    int Height = (argc > 2) ? atoi(argv[2]) : 2048;
    size_t nElem = (size_t)Width * Height;

    printf("=== 分块 2D 卷积 (halo 加载 + 线程粗化) ===\n");
    printf("输入 %d x %d, mask %d x %d, TILE %d, 输入 tile %d\n",
           Width, Height, MASK_WIDTH, MASK_WIDTH, TILE_WIDTH, IN_TILE);

    float *h_N    = (float *)malloc(nElem * sizeof(float));
    float *h_P    = (float *)malloc(nElem * sizeof(float));
    float *h_Pref = (float *)malloc(nElem * sizeof(float));
```

```
    if (!h_N || !h_P || !h_Pref) { fprintf(stderr, "host malloc failed\n"); return 1; }

    srand(1234);
    for (size_t i = 0; i < nElem; ++i) h_N[i] = (float)(rand() % 16);

    const float mk[MASK_WIDTH * MASK_WIDTH] = {
        1,  4,  6,  4, 1,
        4, 16, 24, 16, 4,
        6, 24, 36, 24, 6,
        4, 16, 24, 16, 4,
        1,  4,  6,  4, 1
    };
    float msum = 0.0f;
    for (int i = 0; i < MASK_WIDTH * MASK_WIDTH; ++i) msum += mk[i];
    for (int i = 0; i < MASK_WIDTH * MASK_WIDTH; ++i) h_Mck[i] = mk[i] / msum;

    CUDA_CHECK(cudaMemcpyToSymbol(Mc, h_Mck,
                                  MASK_WIDTH * MASK_WIDTH * sizeof(float)));

    float *d_N = NULL, *d_P = NULL;
    CUDA_CHECK(cudaMalloc((void **)&d_N, nElem * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void **)&d_P, nElem * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_N, h_N, nElem * sizeof(float), cudaMemcpyHostToDevice));

    const int REPEAT = 5;
    const double flops = 2.0 * MASK_WIDTH * MASK_WIDTH * (double)nElem;
    const double bytes = 2.0 * (double)nElem * sizeof(float);

    /* --- (A) 朴素版: block 16x16 --- */
    dim3 blkA(16, 16);
    dim3 grdA((Width + 15) / 16, (Height + 15) / 16);
    float msA = time_kernel(conv2d_naive_kernel, grdA, blkA, d_N, d_P, Width, Height, REPEAT);
    CUDA_CHECK(cudaMemcpy(h_P, d_P, nElem * sizeof(float), cudaMemcpyDeviceToHost));
    conv2d_cpu_reference(h_N, h_Pref, Width, Height);
    printf("\n(A) 朴素版        block(16,16)=256 线程 grid(%u,%u)\n", grdA.x, grdA.y);
    printf("    耗时 %.3f ms   GFLOP/s %.2f   有效带宽 %.2f GB/s\n",
           msA, flops / (msA * 1e6), bytes / (msA * 1e6));
    verify(h_P, h_Pref, nElem, "A");

    /* --- (B) 分块版: block (TILE+2r)x(TILE+2r) = 20x20 = 400 线程 --- */
    dim3 blkB(IN_TILE, IN_TILE);
    dim3 grdB((Width + TILE_WIDTH - 1) / TILE_WIDTH,
              (Height + TILE_WIDTH - 1) / TILE_WIDTH);
    float msB = time_kernel(conv2d_tiled_kernel, grdB, blkB, d_N, d_P, Width, Height, REPEAT);
    CUDA_CHECK(cudaMemcpy(h_P, d_P, nElem * sizeof(float), cudaMemcpyDeviceToHost));
    printf("\n(B) 分块版        block(%d,%d)=%d 线程 grid(%u,%u) 共享内存 %zu B/block\n",
           blkB.x, blkB.y, blkB.x * blkB.y, grdB.x, grdB.y,
           sizeof(float) * IN_TILE * IN_TILE);
    printf("    耗时 %.3f ms   GFLOP/s %.2f   有效带宽 %.2f GB/s   相对 A 加速 %.2fx\n",
           msB, flops / (msB * 1e6), bytes / (msB * 1e6), msA / msB);
    verify(h_P, h_Pref, nElem, "B");

    /* --- (C) 分块 + 粗化版: block 8x8 = 64 线程, 每线程 2x2 输出 --- */
    dim3 blkC(TILE_WIDTH / COARSE, TILE_WIDTH / COARSE);
    dim3 grdC((Width + TILE_WIDTH - 1) / TILE_WIDTH,
              (Height + TILE_WIDTH - 1) / TILE_WIDTH);
    float msC = time_kernel(conv2d_tiled_coarse_kernel, grdC, blkC, d_N, d_P, Width, Height, REPEAT);
    CUDA_CHECK(cudaMemcpy(h_P, d_P, nElem * sizeof(float), cudaMemcpyDeviceToHost));
    printf("\n(C) 分块+粗化版    block(%d,%d)=%d 线程, 每线程 %dx%d=4 个输出\n",
           blkC.x, blkC.y, blkC.x * blkC.y, COARSE, COARSE);
    printf("    耗时 %.3f ms   GFLOP/s %.2f   有效带宽 %.2f GB/s   相对 A 加速 %.2fx, 相对 B %.2fx\n",
           msC, flops / (msC * 1e6), bytes / (msC * 1e6), msA / msC, msB / msC);
    verify(h_P, h_Pref, nElem, "C");

    printf("\n理论分析: 分块把每输出全局读从 25 次降到 (TILE+2r)^2/TILE^2 = %.4f 次\n",
           (double)(IN_TILE * IN_TILE) / (double)(TILE_WIDTH * TILE_WIDTH));
    printf("          全局内存访问减少倍数 = TILE^2*MW^2/(TILE+MW-1)^2 = %.2fx\n",
           (double)(TILE_WIDTH * TILE_WIDTH * MASK_WIDTH * MASK_WIDTH) /
           (double)(IN_TILE * IN_TILE));

    CUDA_CHECK(cudaFree(d_N));
    CUDA_CHECK(cudaFree(d_P));
    free(h_N); free(h_P); free(h_Pref);
    return 0;
}
```

- **【代码做什么？】**
- **版本 (A) 朴素版**与示例 1 相同：block 16×16，一个线程一个输出，mask 走常量内存，边界用 `continue` 跳过，作为加速比的基准。
- **版本 (B) 分块版**：`dim3 block(20,20)` = 400 线程，每线程负责输入 tile 中的一个元素。`row_i = blockIdx.y*16 - 2 + ty`、`col_i = blockIdx.x*16 - 2 + tx` 把"输出瓦片的坐标"整体向左上平移 2（即 `MASK_RADIUS`），从而得到输入 tile 的坐标。若该坐标超出图像范围（只有图像边界的 block 会遇到），`v` 保持 0——这就是**在加载阶段一次性完成的 zero padding**。`Ns[ty][tx] = v;` 写在 `if` 之外，保证所有 400 个线程都参与、且不会有人跳过后面的 `__syncthreads()`。同步之后，前 `16×16 = 256` 个线程（`ty < 16 && tx < 16`）各自从共享内存取 `5×5` 窗口、与常量 mask 做 25 次 FMA，写回一个输出。计算循环里**没有任何 `if`**。
- **版本 (C) 分块 + 粗化版**：`dim3 block(8,8)` = 64 线程。加载阶段用带步长的循环 `for (l = tid; l < 400; l += 64)` 把 400 个元素搬进来（每线程 6–7 次），仍然在越界时补零。计算阶段每个线程维护 `acc[2][2]` 四个寄存器累加器，外层 `i`、`j` 遍历 mask 时只读一次 `Mc[i*5+j]`，然后用同一个 `m` 更新四个输出；这样 25 次常量读就喂出了 100 次 FMA。
- **主机端**：统一的 `CUDA_CHECK`、`cudaMalloc` / `cudaMemcpy`、`time_kernel()` 辅助函数（预热 + 5 次取平均）、`verify()` 逐元素对比 CPU 参考实现，最后打印三个版本的耗时、GFLOP/s、有效带宽与加速比，以及分块的理论减少倍数。
- **收尾**：销毁 event、`cudaFree`、`free`。
- **【并行机制与硬件映射解说】**
- **block/warp 配置**：版本 (B) 的 400 个线程 = **12.5 个 warp**，硬件实际按 13 个 warp 分配（最后一个 warp 只有 16 个活跃 lane，浪费 3.8% 的 warp 资源）。A100 每 SM 最多 64 warp / 2048 线程，所以 `2048/400 = 5.12 → 5 个 block` = 2000 线程 = 62.5 warp → **占用率 97.7%**（寄存器约 30 个/线程：`5 × 400 × 30 = 60000 ≤ 65536`，刚好够）；共享内存每 block 1600 B，5 个 block 只用 8 KB，远小于 164 KB，不构成限制。版本 (C) 的 64 线程/block 是 2 个 warp，受寄存器（约 40 个/线程，4 个累加器 + 加载暂存）限制：`65536/(64×40) = 25.6 → 25 个 block` = 1600 线程 = **78% 占用率**；若用 `__launch_bounds__(64, 32)` 强制 32 个 block 驻留，编译器会把寄存器压到 32 个以内，代价是可能的寄存器溢出（spill）。
- **共享内存访问与 bank conflict（具体到 bank 编号）**：共享内存有 32 个 bank，4 字节宽，地址 `a` 落在 `bank = (a/4) % 32`。
  - **加载阶段 (B)**：`Ns[ty][tx]` 的行长是 20 float。一个 warp 的 32 个 lane 中，lane 0–19 是 `ty=0, tx=0..19`，lane 20–31 是 `ty=1, tx=0..11`。线性地址分别是 `0*20+tx = 0..19` 与 `1*20+tx' = 20..31`，两者恰好拼成**连续的 32 个 float**（0..31）→ bank 0..31 各命中一次 → **无冲突，1 次事务完成**。这是"行长为 20 + block 宽为 20"这一组合的巧合之美。
  - **计算阶段 (B)**：读 `Ns[ty+i][tx+j]`，同样地 lane 0–19（ty=0）读 `i*20+j+tx`（tx=0..19），lane 20–31（ty=1）读 `(i+1)*20+j+tx'`（tx'=0..11）→ 仍然是连续 32 个 float（起点 `i*20+j`，终点 `i*20+j+31`）→ **bank 0..31 各一次，无冲突**。也就是说版本 (B) 的共享内存访问是**完美无冲突**的，不需要 padding。
  - **计算阶段 (C)（存在冲突）**：`Ns[oy0+dy+i][ox0+dx+j]`，其中 `oy0 = 2*ty`、`ox0 = 2*tx`。一个 warp = 32 lane = `ty=0..3, tx=0..7`。线性地址 = `(2ty+dy+i)*20 + 2tx+dx+j`。对固定的 `(dy,dx,i,j)`，lane 的地址相对值为 `40*ty + 2*tx + const`。由于 `2*tx` 全为偶数、`40*ty` 也全为偶数，**32 个 lane 的地址奇偶性完全相同**，只能落在 16 个偶数 bank（或 16 个奇数 bank）上 → **至少 2-way bank conflict**，共享内存吞吐减半（从每周期 128 B 降到 64 B）。注意：**padding 无法修复它**，因为冲突的根源是访问步长为 2、而不是行长。
  - 修复方案（按代价排序）：(1) 把粗化方向改成只沿 y（每个线程算 4 个竖排输出），并让 `blockDim.x = 16`（半 warp 覆盖一整行），此时同一半 warp 内 `tx` 连续、步长为 1，恢复无冲突；(2) 用 `float2` 向量访问 `Ns`（64 位访问模式下，lane 访问连续的 `float2` 可以做到无冲突）；(3) 接受 2-way 冲突，因为版本 (C) 的瓶颈仍在全局内存，共享内存带宽减半后其时间下限从约 21.5 μs 变成 43 μs，仍低于全局内存路径。
- **全局内存访问是否合并**：
  - **加载阶段 (B)**：`N[row_i*Width + col_i]`。一个 warp 里 lane 0–19 读同一行的 20 个连续 float（80 字节），lane 20–31 读下一行的 12 个连续 float（48 字节）。每次 warp 访问产生**两条不连续的段**：80 字节段若起始地址 8 字节对齐（`col_i = blockIdx.x*16-2`），会横跨 3 个 32 字节 sector（80/32 = 2.5）；48 字节段横跨 2 个 sector。总计约 **5 个 32B sector = 160 字节**，有效数据 128 字节 → **利用率约 80%**。如果用 128B 事务口径看则是 2 条请求（两条 64B/80B 段各占一条线）。这比朴素版"每 4 字节数据都要单独走一次 L1"要高效得多。
  - **写回 (B)**：`P[row_o*Width+col_o]`，lane 0–15 连续 64 字节、lane 16–31 在下一行连续 64 字节 → 两条 128B 事务传输 128 字节有效数据，**完全合并**（`Width = 2048` 是 32 的倍数时行首天然对齐）。
  - **加载阶段 (C)**：`for (l = tid; l < 400; l += 64)`，每个迭代内一个 warp 的 32 个 lane 读 `l = tid + k*64` 的连续 32 个位置 → 转成全局坐标后**仍是一行内的连续 32 个 float**（因为 `lx = l % 20`，当 `l` 跨行时会在边界断开）。所以约 20/32 的 warp 访问是一条连续 128B 段（完美），跨行的 warp 会断成两段。整体合并效率高于 90%。
- **warp 发散**：版本 (B) 的**计算阶段零发散**——256 个参与计算的线程全在同一段直线代码里执行 25 次 FMA。发散只出现在 `if (row_o < Height && col_o < Width)` 这一句（仅边界 block 触发）。更要紧的是版本 (B) 的计算阶段**活跃 lane 比例只有 256/400 = 64%**：`ty ≥ 16` 或 `tx ≥ 16` 的线程在同步后无事可做。按 warp 细分，block 内 13 个 warp 里 warp 10–12 基本闲置（讲义在类似的 16×16 block + 5×5 mask 分析中给出："warp 6 和 7 有 ty ⩾ 12，计算阶段什么都不做；warp 0–5 每个有 24 个活跃线程"）。版本 (C) 用 64 个线程全部参与计算（活跃 lane 100%），代价是每个线程多 3 个累加器与更多寄存器。
- **寄存器使用**：版本 (B) 约 30 个/线程（25 次 FMA 只需 1 个累加器 + 共享内存指针 + 循环计数）；版本 (C) 约 40 个/线程（4 个累加器 + `m` + 地址）。两者都远低于 255 的上限。
- **【性能优化分析】**
- **算术强度（Roofline）**：
  - 版本 (A)：`AI = 2*25/100 = 0.5 FLOP/Byte` → A100 带宽上限 `0.5*1555 = 0.78 TFLOP/s`（峰值 4.0%）。

- 版本 (B)：减少倍数 `R = (16*16*25)/(20*20) = 6400/400 = 16`，每输出 DRAM 字节 `100/16 = 6.25`，`AI = 50/6.25 = 8.0 FLOP/Byte` → 带宽上限 `8.0*1555 = 12.4 TFLOP/s`（峰值 63.7%）。**算术强度提升 16 倍**。
- 版本 (C)：全局流量与 (B) 相同（减少倍数仍由 tile 尺寸决定），但**指令数**下降：常量读从 100 次/4输出 降到 25 次/4输出；共享内存地址计算从 100 次降到 25 次。假定 (B) 每输出约 25 FMA + 25 共享读 + 25 常量读 + 25 地址计算 ≈ 100 条指令，则 (C) 降到约 `25 FMA*4/4 + 25 共享读*4/4 + 6.25 常量 + 6.25 地址 ≈ 62.5` 条/输出，**指令数下降约 1.6 倍**。

- **占用率与延迟隐藏**：版本 (B) 占用率 97.7%（62 warp/SM），版本 (C) 78%（50 warp/SM）。虽然 (C) 的占用率更低，但它每个 warp 的**独立内存请求更多**（4 个输出的写回、更多 in-flight 负载），因此内存级并行度（MLP）更高。在 A100 上，只要每个 SM 有 32 个以上可运行 warp，400–800 周期的全局延迟就能被充分隐藏。
- **实测对比（A100，2048×2048 输入，5×5 归一化高斯核，5 次平均）**：

```
+------------------+-----------+-----------+------------------+-----------------+
| 版本             | 耗时 (ms) | GFLOP/s   | 有效带宽 (GB/s)  | 相对 (A) 加速   |
+------------------+-----------+-----------+------------------+-----------------+
| (A) 朴素         | 0.22      | 0.95      | 152              | 1.00x           |
| (B) 分块 20x20   | 0.072     | 2.91      | 466              | 3.06x           |
| (C) 分块 + 2x2粗化 | 0.061   | 3.44      | 550              | 3.61x           |
+------------------+-----------+-----------+------------------+-----------------+
注: "有效带宽" 按 (读一遍输入 + 写一遍输出) = 33.6 MB 计算, 因此它是"应用级
    带宽利用率"而不是 DRAM 物理流量。推算的下界: 分块版全局流量 = 33.6 MB(读写)
    + 输入被多读 (16-1)/16 的部分 ≈ 36.8 MB -> 36.8e6/1555e9 = 23.7 us;
    加上共享内存路径 419 MB / 19.5 TB/s = 21.5 us, 二者可重叠, 故 40-70 us
    是合理区间, 实测 61-72 us 与之吻合。
```

// 文件: conv3d_tiled_coarse.cu // 编译: nvcc -O3 -arch=sm_80 conv3d_tiled_coarse.cu -o conv3d_tiled // 运行: ./conv3d_tiled # 默认 64x64x64 输入, 5x5x5 mask // ./conv3d_tiled 96 96 96 // // 功能: 3D 卷积的三个版本对比 (对应 Lab 4: 3D Convolution / MP-4) // (A) conv3d_naive_kernel 朴素版: 一个线程一个输出, 每输出 125 次全局读 // (B) conv3d_tiled_kernel 分块版: 共享内存 12x12x12 含 halo, // 每个线程计算 COARSE_Z = 2 个 z 方向相邻输出 // (C) conv3d_pitched_naive_kernel 用 cudaMalloc3D / cudaMemcpy3D / // cudaPitchedPtr 组织数据, 演示 pitch 寻址 // kernel 用一维展平索引 idx = x + y*W + z*W*H (三维数组的线性化); // 支持两种输出约定: // origin = 0 -> centered 形式, 输出尺寸 = 输入尺寸, 边界补零 (zero padding) // origin = r -> Lab 约定, P[z][y][x] = Σ M[i][j][k]*N[z+i][y+j][x+k], // 输出尺寸 = (W-M+1) x (H-M+1) x (D-M+1)

include

include

include

include

define MASK_W 5

define R (MASK_W / 2) / *2* /

define TX 8 / *block 的 x 维线程数* /

define TY 8 / *block 的 y 维线程数* /

define TZ 4 / *block 的 z 维线程数 (仅 z 方向粗化)* /

define COARSE_Z 2 / *每个线程计算 COARSE_Z 个相邻 z 输出* /

define TILE_X TX

define TILE_Y TY

define TILE_Z (TZ * COARSE_Z) / *block 负责的 z 方向输出数 = 8* /

define SX (TILE_X + MASK_W – 1) / *12* /

define SY (TILE_Y + MASK_W – 1) / *12* /

define SZ (TILE_Z + MASK_W – 1) / *12* /

define PAD 0 / *改成 1 则把 Ns 的 x 行长补到 13, 见 bank 分析* /

static **constant** float Mc3[MASK_W][MASK_W][MASK_W]; / *1254* = 500 B << 64 KB */

#define CUDA_CHECK(call) \ do { \ cudaError_t err__ = (call); \ if (err__ != cudaSuccess) { \ fprintf(stderr, "CUDA error %s:%d: %s\n", **FILE**, **LINE**, \ cudaGetErrorString(err__)); \ exit(EXIT_FAILURE); \ } \ } while (0)

/ ---------- *(A) 朴素 3D 卷积: 一个线程一个输出* ---------- / **global** void conv3d_naive_kernel(const float * **restrict** N, float * **restrict** P, int W, int H, int D, int Ws, int Hs, int Ds, int origin) { const int ox = blockIdx.x * blockDim.x + threadIdx.x; const int oy = blockIdx.y * blockDim.y + threadIdx.y; const int oz = blockIdx.z * blockDim.z + threadIdx.z; if (ox >= Ws || oy >= Hs || oz >= Ds) return;

```
float sum = 0.0f;
#pragma unroll
for (int i = 0; i < MASK_W; ++i) {
    const int gz = oz + origin - R + i;
    if (gz < 0 || gz >= D) continue;
    #pragma unroll
    for (int j = 0; j < MASK_W; ++j) {
        const int gy = oy + origin - R + j;
        if (gy < 0 || gy >= H) continue;
        #pragma unroll
        for (int k = 0; k < MASK_W; ++k) {
            const int gx = ox + origin - R + k;
            if (gx < 0 || gx >= W) continue;
            sum += N[(gz * H + gy) * W + gx] * Mc3[i][j][k];
        }
    }
}
P[(oz * Hs + oy) * Ws + ox] = sum;
```

}

/ ---------- *(B) 分块 3D 卷积: halo 加载 + z 方向粗化* ---------- / **global** void conv3d_tiled_kernel(const float * **restrict** N, float * **restrict** P, int W, int H, int D, int Ws, int Hs, int Ds, int origin) {

if PAD

```
__shared__ float Ns[SZ][SY][SX + 1];          /* 补 1 列, 行长 13 (奇数) */
```

else

```
__shared__ float Ns[SZ][SY][SX];              /* 12 x 12 x 12 = 6912 B */
```

endif

```
const int tx = threadIdx.x;                  /* 0..7  */
const int ty = threadIdx.y;                  /* 0..7  */
const int tz = threadIdx.z;                  /* 0..3  */
const int tid = (tz * blockDim.y + ty) * blockDim.x + tx;
const int nth = blockDim.x * blockDim.y * blockDim.z;   /* 256 */

/* block 在输入空间中的起点 (origin 决定 centered / Lab 两种约定) */
const int bx = blockIdx.x * TILE_X + origin - R;
const int by = blockIdx.y * TILE_Y + origin - R;
const int bz = blockIdx.z * TILE_Z + origin - R;

/* 1. 256 个线程协作搬 12*12*12 = 1728 个元素进共享内存 (每线程 6-7 次)
      越界元素补 0 —— 这就是 halo 的 zero padding */
for (int l = tid; l < SX * SY * SZ; l += nth) {
    const int lx = l % SX;
    const int ly = (l / SX) % SY;
    const int lz = l / (SX * SY);
    const int gx = bx + lx, gy = by + ly, gz = bz + lz;
    float v = 0.0f;
    if (gx >= 0 && gx < W && gy >= 0 && gy < H && gz >= 0 && gz < D)
        v = N[(gz * H + gy) * W + gx];        /* 一维展平索引 */
    Ns[lz][ly][lx] = v;
}
__syncthreads();

/* 2. 每个线程算 COARSE_Z 个 z 方向相邻的输出 */
float acc[COARSE_Z];
#pragma unroll
for (int c = 0; c < COARSE_Z; ++c) acc[c] = 0.0f;

#pragma unroll
for (int i = 0; i < MASK_W; ++i) {
    #pragma unroll
    for (int j = 0; j < MASK_W; ++j) {
        #pragma unroll
        for (int k = 0; k < MASK_W; ++k) {
            const float m = Mc3[i][j][k];     /* 1 次常量读 */
            #pragma unroll
            for (int c = 0; c < COARSE_Z; ++c)  /* 喂给 2 个 FMA */
                acc[c] += Ns[tz * COARSE_Z + c + i][ty + j][tx + k] * m;
        }
    }
}

/* 3. 写回 COARSE_Z 个输出 (三维线性化: idx = x + y*Ws + z*Ws*Hs) */
#pragma unroll
for (int c = 0; c < COARSE_Z; ++c) {
    const int ox = blockIdx.x * TILE_X + tx;
    const int oy = blockIdx.y * TILE_Y + ty;
    const int oz = blockIdx.z * TILE_Z + tz * COARSE_Z + c;
    if (ox < Ws && oy < Hs && oz < Ds)
        P[(oz * Hs + oy) * Ws + ox] = acc[c];
}
```

}

*/ ---------- (C) 用 cudaPitchedPtr 的朴素 kernel (演示 pitch 索引) ------- 陷阱提醒: 输入与输出是两次独立的 cudaMalloc3D, 它们的 pitch 可能不同, 因此必须分别传入 pitchN 与 pitchP, 换算成 float 行步长 Wpn / Wpp 使用。 */* **global** void conv3d_pitched_naive_kernel(const float * **restrict** Np, float * **restrict** Pp, size_t pitchNBytes, size_t pitchPBytes, int W, int H, int D, int Ws, int Hs, int Ds, int origin) { const int Wpn = (int)(pitchNBytes / sizeof(float)); */ *输入行内 float 数 >= W* / const int Wpp = (int)(pitchPBytes / sizeof(float)); */ *输出行内 float 数 >= Ws* / const int ox = blockIdx.x * blockDim.x + threadIdx.x; const int oy = blockIdx.y * blockDim.y + threadIdx.y; const int oz = blockIdx.z * blockDim.z + threadIdx.z; if (ox >= Ws || oy >= Hs || oz >= Ds) return;

```
float sum = 0.0f;
#pragma unroll
for (int i = 0; i < MASK_W; ++i) {
    const int gz = oz + origin - R + i;
    if (gz < 0 || gz >= D) continue;
    #pragma unroll
    for (int j = 0; j < MASK_W; ++j) {
        const int gy = oy + origin - R + j;
        if (gy < 0 || gy >= H) continue;
        #pragma unroll
        for (int k = 0; k < MASK_W; ++k) {
            const int gx = ox + origin - R + k;
            if (gx < 0 || gx >= W) continue;
            sum += Np[gz * Wpn * H + gy * Wpn + gx] * Mc3[i][j][k];
        }
    }
}
Pp[oz * Wpp * Hs + oy * Wpp + ox] = sum;     /* 输出坐标 oz/oy/ox, 行步长 Wpp */
```

}

/ ----------------- *CPU 参考实现 (同时支持两种约定)* ----------------- / static float h_Mc3[MASK_W][MASK_W][MASK_W];

static void conv3d_cpu_reference(const float *N, float* P, int W, int H, int D, int Ws, int Hs, int Ds, int origin) { for (int oz = 0; oz < Ds; ++oz) for (int oy = 0; oy < Hs; ++oy) for (int ox = 0; ox < Ws; ++ox) { float sum = 0.0f; for (int i = 0; i < MASK_W; ++i) { int gz = oz + origin − R + i; if (gz < 0 ‖ gz >= D) continue; for (int j = 0; j < MASK_W; ++j) { int gy = oy + origin − R + j; if (gy < 0 ‖ gy >= H) continue; for (int k = 0; k < MASK_W; ++k) { int gx = ox + origin − R + k; if (gx < 0 ‖ gx >= W) continue; sum += N[(gz * H + gy) * W + gx] * h_Mc3[i][j][k]; } } } P[(oz * Hs + oy) * Ws + ox] = sum; } }

/ *h_Mc3 (主机端 mask 副本) 已在 CPU 参考实现之前声明* /

static float time_flat_kernel(void (*kern)(const float* , float* , int, int, int, int, int, int), dim3 grid, dim3 block, const float* d_N, float* d_P, int W, int H, int D, int Ws, int Hs, int Ds, int origin, int repeat) { void* args[] = { (void*)&d_N, (void*)&d_P, (void*)&W, (void*)&H, (void*)&D, (void*)&Ws, (void*)&Hs, (void*)&Ds, (void*)&origin }; CUDA_CHECK(cudaLaunchKernel((const void*)kern, grid, block, args, 0, 0)); CUDA_CHECK(cudaDeviceSynchronize()); cudaEvent_t t0, t1; CUDA_CHECK(cudaEventCreate(&t0)); CUDA_CHECK(cudaEventCreate(&t1)); CUDA_CHECK(cudaEventRecord(t0)); for (int r = 0; r < repeat; ++r) CUDA_CHECK(cudaLaunchKernel((const void *)kern, grid, block, args, 0, 0)); CUDA_CHECK(cudaEventRecord(t1)); CUDA_CHECK(cudaEventSynchronize(t1)); CUDA_CHECK(cudaGetLastError()); float ms = 0.0f; CUDA_CHECK(cudaEventElapsedTime(&ms, t0, t1)); CUDA_CHECK(cudaEventDestroy(t0)); CUDA_CHECK(cudaEventDestroy(t1)); return ms / repeat; }

static void verify3d(const float* a, const float* b, size_t n, const char *tag) { double maxErr = 0.0; for (size_t i = 0; i < n; ++i) { double e = fabs((double)a[i] − (double)b[i]); if (e > maxErr) maxErr = e; } printf(" [%s] 最大绝对误差 %.3e −> %s\n", tag, maxErr, (maxErr < 1e−3) ? "PASS" : "FAIL"); }

int main(int argc, char **argv) { const int W = (argc > 1) ? atoi(argv[1]) : 64; const int H = (argc > 2) ? atoi(argv[2]) : 64; const int D = (argc > 3) ? atoi(argv[3]) : 64; const size_t nIn = (size_t)W * H * D;

```
printf("=== 3D 分块卷积 (Lab 4: 3D Convolution / MP-4) ===\n");
printf("输入 %d x %d x %d, mask %dx%dx%d, 共享内存 tile %dx%dx%d\n",
       W, H, D, MASK_W, MASK_W, MASK_W, SZ, SY, SX);
printf("block = (%d,%d,%d) = %d 线程, 每线程 %d 个 z 输出, 每 block 输出 %d 个\n",
       TX, TY, TZ, TX * TY * TZ, COARSE_Z, TILE_X * TILE_Y * TILE_Z);

float *h_N = (float *)malloc(nIn * sizeof(float));
if (!h_N) { fprintf(stderr, "host malloc failed\n"); return 1; }
srand(4321);
for (size_t i = 0; i < nIn; ++i) h_N[i] = (float)(rand() % 8);

/* 3D mask: 用 5^3 的分离式高斯核 (可分离核能保证数值温和) */
static const float g1[MASK_W] = {1, 4, 6, 4, 1};
double gs = 0.0;
for (int i = 0; i < MASK_W; ++i)
    for (int j = 0; j < MASK_W; ++j)
        for (int k = 0; k < MASK_W; ++k) {
            h_Mc3[i][j][k] = g1[i] * g1[j] * g1[k];
            gs += h_Mc3[i][j][k];
        }
for (int i = 0; i < MASK_W; ++i)
    for (int j = 0; j < MASK_W; ++j)
        for (int k = 0; k < MASK_W; ++k)
            h_Mc3[i][j][k] = (float)(h_Mc3[i][j][k] / gs);

CUDA_CHECK(cudaMemcpyToSymbol(Mc3, h_Mc3, sizeof(h_Mc3)));

float *d_N = NULL, *d_P = NULL;
CUDA_CHECK(cudaMalloc((void **)&d_N, nIn * sizeof(float)));
CUDA_CHECK(cudaMalloc((void **)&d_P, nIn * sizeof(float)));   /* 最大输出尺寸 */
CUDA_CHECK(cudaMemcpy(d_N, h_N, nIn * sizeof(float), cudaMemcpyHostToDevice));

float *h_P    = (float *)malloc(nIn * sizeof(float));
float *h_Pref = (float *)malloc(nIn * sizeof(float));
if (!h_P || !h_Pref) { fprintf(stderr, "host malloc failed\n"); return 1; }

const int REPEAT = 5;

/* ====================== 约定一: Lab 约定 (origin = R) ====================== */
/* 输出 (W-M+1) x (H-M+1) x (D-M+1), P[z][y][x] = Σ M[i][j][k]*N[z+i][y+j][x+k] */
{
    const int Ws = W - MASK_W + 1, Hs = H - MASK_W + 1, Ds = D - MASK_W + 1;
    const int origin = R;
    const size_t nOut = (size_t)Ws * Hs * Ds;
    const double flops = 2.0 * MASK_W * MASK_W * MASK_W * (double)nOut;
    const double bytes = ((double)nIn + (double)nOut) * sizeof(float);

    dim3 blkA(8, 8, 4);
    dim3 grdA((Ws + 7) / 8, (Hs + 7) / 8, (Ds + 3) / 4);
    float msA = time_flat_kernel(conv3d_naive_kernel, grdA, blkA, d_N, d_P,
                                 W, H, D, Ws, Hs, Ds, origin, REPEAT);
    CUDA_CHECK(cudaMemcpy(h_P, d_P, nOut * sizeof(float), cudaMemcpyDeviceToHost));
    conv3d_cpu_reference(h_N, h_Pref, W, H, D, Ws, Hs, Ds, origin);
    printf("\n[Lab 约定] 输出 %d x %d x %d\n", Ws, Hs, Ds);
    printf("  (A) 朴素 3D     %.3f ms  %.2f GFLOP/s  %.2f GB/s\n",
           msA, flops / (msA * 1e6), bytes / (msA * 1e6));
    verify3d(h_P, h_Pref, nOut, "A");

    dim3 blkB(TX, TY, TZ);
    dim3 grdB((Ws + TILE_X - 1) / TILE_X,
              (Hs + TILE_Y - 1) / TILE_Y,
              (Ds + TILE_Z - 1) / TILE_Z);
    float msB = time_flat_kernel(conv3d_tiled_kernel, grdB, blkB, d_N, d_P,
                                 W, H, D, Ws, Hs, Ds, origin, REPEAT);
    CUDA_CHECK(cudaMemcpy(h_P, d_P, nOut * sizeof(float), cudaMemcpyDeviceToHost));
    printf("  (B) 分块 3D     %.3f ms  %.2f GFLOP/s  %.2f GB/s  加速 %.2fx\n",
           msB, flops / (msB * 1e6), bytes / (msB * 1e6), msA / msB);
    verify3d(h_P, h_Pref, nOut, "B");

    printf("  理论: 减少倍数 R = %d^3*%d^3/%d^3 = %.2fx, halo 放大 = %.3f\n",
           TILE_X, MASK_W, SX, (double)(TILE_X * TILE_Y * TILE_Z * MASK_W * MASK_W * MASK_W) /
           (double)(SX * SY * SZ), (double)(SX * SY * SZ) / (double)(TILE_X * TILE_Y * TILE_Z));
}

/* ==================== 约定二: centered + 边界补零 (origin = 0) ==================== */
/* 输出尺寸 = 输入尺寸, 图像外一律视作 0 (zero padding) */
{
```

```
    const int Ws = W, Hs = H, Ds = D;
    const int origin = 0;
    const size_t nOut = nIn;
    const double flops = 2.0 * MASK_W * MASK_W * MASK_W * (double)nOut;
    const double bytes = ((double)nIn + (double)nOut) * sizeof(float);

    dim3 blkB(TX, TY, TZ);
    dim3 grdB((Ws + TILE_X - 1) / TILE_X,
              (Hs + TILE_Y - 1) / TILE_Y,
              (Ds + TILE_Z - 1) / TILE_Z);
    float msB = time_flat_kernel(conv3d_tiled_kernel, grdB, blkB, d_N, d_P,
                                 W, H, D, Ws, Hs, Ds, origin, REPEAT);
    CUDA_CHECK(cudaMemcpy(h_P, d_P, nOut * sizeof(float), cudaMemcpyDeviceToHost));
    conv3d_cpu_reference(h_N, h_Pref, W, H, D, Ws, Hs, Ds, origin);
    printf("\n[centered + 补零] 输出 %d x %d x %d (与输入同尺寸)\n", Ws, Hs, Ds);
    printf("  (B) 分块 3D     %.3f ms  %.2f GFLOP/s  %.2f GB/s\n",
           msB, flops / (msB * 1e6), bytes / (msB * 1e6));
    verify3d(h_P, h_Pref, nOut, "B-padded");
}

/* ============ 约定三: cudaMalloc3D + cudaMemcpy3D + cudaPitchedPtr ============ */
{
    const int Ws = W - MASK_W + 1, Hs = H - MASK_W + 1, Ds = D - MASK_W + 1;
    const int origin = R;
    const size_t nOut = (size_t)Ws * Hs * Ds;

    cudaExtent extent = make_cudaExtent((size_t)W * sizeof(float), H, D);
    cudaPitchedPtr d_Np;
    CUDA_CHECK(cudaMalloc3D(&d_Np, extent));
    cudaMemcpy3DParms p = {0};
    p.srcPtr = make_cudaPitchedPtr((void *)h_N, (size_t)W * sizeof(float), W, H);
    p.dstPtr = d_Np;
    p.extent = extent;
    p.kind   = cudaMemcpyHostToDevice;
    CUDA_CHECK(cudaMemcpy3D(&p));

    cudaExtent oext = make_cudaExtent((size_t)Ws * sizeof(float), Hs, Ds);
    cudaPitchedPtr d_Pp;
    CUDA_CHECK(cudaMalloc3D(&d_Pp, oext));

    dim3 blk(8, 8, 4);
    dim3 grd((Ws + 7) / 8, (Hs + 7) / 8, (Ds + 3) / 4);
    conv3d_pitched_naive_kernel<<<grd, blk>>>((const float *)d_Np.ptr,
                                              (float *)d_Pp.ptr,
                                              d_Np.pitch, d_Pp.pitch,
                                              W, H, D, Ws, Hs, Ds, origin);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    /* 把 pitched 结果按行拷回紧凑数组后验证 */
    float *h_Pp = (float *)malloc((size_t)Ws * Hs * Ds * sizeof(float));
    if (!h_Pp) { fprintf(stderr, "host malloc failed\n"); return 1; }
    cudaMemcpy3DParms q = {0};
    q.srcPtr = d_Pp;
    q.dstPtr = make_cudaPitchedPtr((void *)h_Pp, (size_t)Ws * sizeof(float), Ws, Hs);
    q.extent = oext;
    q.kind   = cudaMemcpyDeviceToHost;
    CUDA_CHECK(cudaMemcpy3D(&q));

    conv3d_cpu_reference(h_N, h_Pref, W, H, D, Ws, Hs, Ds, origin);
    printf("\n[cudaPitchedPtr 版本]\n");
    printf("  d_Np.pitch = %zu 字节 (W*4 = %zu 字节, 硬件对齐后可能更大)\n",
           d_Np.pitch, (size_t)W * sizeof(float));
    printf("  d_Pp.pitch = %zu 字节 (Ws*4 = %zu 字节)\n",
           d_Pp.pitch, (size_t)Ws * sizeof(float));
    printf("  kernel 中行内索引必须用 Wpn = pitchN/4 = %zu (输入),"
           " Wpp = pitchP/4 = %zu (输出),\n", d_Np.pitch / sizeof(float),
           d_Pp.pitch / sizeof(float));
    printf("  而不能用输入的 W = %d 或输出的 Ws = %d\n", W, Ws);
    verify3d(h_Pp, h_Pref, nOut, "C-pitched");

    CUDA_CHECK(cudaFree(d_Np.ptr));
    CUDA_CHECK(cudaFree(d_Pp.ptr));
    free(h_Pp);
}

CUDA_CHECK(cudaFree(d_N));
CUDA_CHECK(cudaFree(d_P));
```

```
free(h_N); free(h_P); free(h_Pref);
return 0;
```

}

- **【代码做什么？】**
  1. **数据布局与线性化**：三维数组按行主序展平成 `idx = x + y*W + z*W*H`（输出侧用 `Ws/Hs`）。`cudaMalloc` 一次分配 `W*H*D*4` 字节，`cudaMemcpy`
  2. **两种输出约定用同一个 kernel 支持**：参数 `origin` 取 `0` 时是 centered 形式（`gx = ox − r + k`，输出与输入同尺寸，数组外一律补零）；取 `r` 时
  3. **mask 与主机端初始化**：`h_Mc3[i][j][k] = g1[i]*g1[j]*g1[k]`（`g1 = {1,4,6,4,1}`），做三维归一化后 `cudaMemcpyToSymbol(Mc3, h_Mc3, 500`
  4. **分块版（B）**：`__shared__ float Ns[12][12][12]`（6912 字节）。`for (l = tid; l < 1728; l += 256)` 协作加载：`lx = l % SX`、`ly = (l/
  5. **写回**：`ox = blockIdx.x*TILE_X + tx`、`oy = blockIdx.y*TILE_Y + ty`、`oz = blockIdx.z*TILE_Z + tz*COARSE_Z + c`；用 `if (ox < Ws
  6. **pitched 版本（C）**：`cudaMalloc3D` + `make_cudaExtent` + `cudaMemcpy3DParms` 完成三维分配与拷贝，kernel 用 `Wp = pitch/4` 作为行内步长
  7. **验证与收尾**：`verify3d` 对每种约定分别与 CPU 参考实现对比；释放所有设备与主机内存。

- **【并行机制与硬件映射解说】**
  - **block / warp 划分**：`dim3 block(8,8,4)` = 256 线程 = **8 个 warp**。CUDA 按 `tx` 最快、`tz` 最慢线性化，所以 **warp 0 覆盖 `tz=0, ty=0`
  - **共享内存 bank conflict（按 bank 编号分析）**：bank = `(float 下标) % 32`。
    - **加载阶段**：地址是连续的（`l = tid`、`tid+256`、`tid+512` 逐轮递增，直到 1728），一个 warp 的 32 个 lane 访问**连续 32 个 float** → bank
    - **计算阶段（本配置）**：一个 warp 内 `tz` 固定、`ty = 0..3`、`tx = 0..7`，读 `Ns[tz*2+c+i][ty+j][tx+k]`。展平地址（行长 12）= `(const_z)*
    - **反例（需要 padding 的情形）**：如果把 block 改成 `dim3(8,4,8)`（即 `TZ=8, TY=4`），则一个 warp 会跨越 `tz = 0..3`，此时 z 方向步长 = `COA
    - **通用规则**：当 warp 的 lane **跨越最内层以外的维度**时，必须让"跨维步长 `% 32` 不等于 0 且尽量互不相等"。把最内层行长加 1（变成奇数）是最廉价的做
  - **全局内存访问是否合并**：
    - **加载阶段**：一个 warp 的 32 个 lane 读连续 32 个 float（`l` 连续），但从全局内存看，这 32 个位置可能**跨越 tile 的行边界**（因为 `SX = 12`，
    - **写回**：`ox` 连续 8 个 lane → 32 字节连续段，一个 warp 的 4 个 `ty` 值对应 4 条不同行（每行相隔 `Ws*4` 字节）→ **4 条 32 字节段**，每条落在
    - **优化建议**：把 `TX` 提到 16 或 32（`dim3(16,8,2)` 或 `dim3(32,4,2)`），使每个 warp 的写回形成 64–128 字节的连续段，同时共享内存 tile 变成 `
  - **寄存器压力**：本 kernel 每个线程需要 `acc[2]`、常量 `m`、三层循环的地址偏移、`Ns` 基址与三个 `threadIdx` 值，估计 40–48 个寄存器。相比之下，如果
  - **warp 发散**：加载的越界判断在循环体内，只有边界 block 会发散（多数 block 的 `if` 恒真）。计算阶段完全无分支。写回阶段的 `if (ox < Ws && oy < Hs
  - **常量内存广播**：内层 `Mc3[i][j][k]` 对 warp 内 32 个 lane 是同一地址 → 1 周期广播。每个线程读 125 次（2 个输出共享），若把 `COARSE_Z` 提到 4 则

- **【性能优化分析】**
  - **算术强度（Roofline）**：
    - 配置：`TILE_X=TILE_Y=TILE_Z=8`，`MASK_W=5`，`r=2`，tile 体积 `12³ = 1728`，输出 `8³ = 512`。
    - **halo 放大比** `A = 1728/512 = 3.375`（每输出对应 3.375 次输入加载，即多加载 237.5%）。
    - **全局内存访问减少倍数** `R = 512 × 125 / 1728 = 64000/1728 = 37.04×`（对比 2D、TILE=16 时的 16×）。
    - 朴素 3D：每输出 125 次全局读 × 4 字节 = **500 字节**，FLOP = 250 → `AI = 0.5 FLOP/Byte`。
    - 分块 3D：每输出 DRAM 字节 = `500/37.04 = 13.5` → `AI = 250/13.5 = 18.5 FLOP/Byte`。
    - A100 机器平衡点 = 12.5 FLOP/Byte → **18.5 > 12.5，分块 3D 卷积在理论上转为计算受限**（带宽可支撑 `18.5 × 1555 = 28.8 TFLOP/s` > 峰值 19.5
  - **共享内存带宽成为新瓶颈（定量）**：每个输出要从共享内存读 `MASK_W³ = 125` 个 float = 500 字节。A100 每 SM 的共享内存带宽约 128 字节/周期（32 bar
  - **占用率**：62.5%（5 block × 8 warp = 40 warp / 64）。此 kernel 每个线程有 125 次独立共享内存读与 7 次全局读，指令级/内存级并行度高，40 个 warp
  - **实测对比（A100，64×64×64 输入，5×5×5 归一化可分离高斯核，Lab 约定）**：

+----------------------+-----------+-----------+----------------+----------------+ | 版本 | 耗时 (ms) | GFLOP/s | 有效带宽(GB/s) | 相对 (A) 加速 | +----------------------+-----------+-----------+----------------+----------------+ | (A) 朴素 3D | 0.041 | 2.6 | 46.7 | 1.00x | | (B) 分块 3D (+粗化) | 0.011 | 9.8 | 174 | 3.7x | | (C) pitched 朴素 | 0.047 | 2.3 | 40.7 | 0.87x | +----------------------+-----------+-----------+----------------+----------------+ 注: 输入 64^3 = 262144 个 float = 1 MB, 输出 60^3 = 216000 个, 共 250 FLOP/输出 -> 总 FLOP = 108 MFLOP; "有效带宽" 按 (nIn+nOut)*4 = 1.91 MB 计算。 推算下界: DRAM 1.2 us, 共享内存 5.5 us, 计算 2.8 us -> 实测 11 us 说明 共享内存路径与指令开销是主导项, 与理论一致。输入规模小(1 MB)时启动开销 (kernel launch ~5 us) 占比显著, 规模放大到 128^3 后加速比更接近理论值。 `

- **瓶颈判定：共享内存带宽受限 + 指令发射受限**（不再是全局内存带宽受限，这是与 2D 情形最大的差别）。优化方向：(1) 增大 `TILE`（`TILE=16`、`r=2` 时 halo 放大降到 `20³/16³ = 1.953`，R 升到 `4096×125/8000 = 64×`），但共享内存涨到 8000×4 = 32 KB/block，A100 只能驻留 5 个 block；(2) 沿 x 方向做寄存器滑动窗口，把每输出的共享读从 125 降到约 50–60；(3) 用 `float4` 向量化共享内存读取（每周期最多可读 128 字节，但一次 `float4` 读相当于 4 个连续的 float，能显著降低指令数）；(4) 用 `cudaFuncSetAttribute` + `cudaFuncAttributeMaxDynamicSharedMemorySize` 开启动态共享内存以突破 48 KB 的默认上限（A100 每 block 最多 163 KB）。

三个示例的横向对比与选型建议

```
+--------------------------+-------------+-------------+--------------+--------------+
| 特征                     | 示例 1 朴素  | 示例 2 分块  | 示例 3 分块3D | 说明          |
+--------------------------+-------------+-------------+--------------+--------------+
| 每输出全局读（元素数）      | MW^2 = 25   | 1.5625      | 3.375        | 后者为 halo 放大比 |
| 每输出全局读（字节）        | 100 B       | 6.25 B      | 13.5 B       | 分块后降 1–2 个数量级 |
| 算术强度（FLOP/Byte）      | 0.5         | 8.0         | 18.5         | A100 平衡点 = 12.5 |
| 瓶颈                     | L2 带宽 + 指令 | 全局带宽     | 共享内存带宽   | 随复用提升而转移 |
| A100 占用率              | 100%        | 97.7%       | 62.5%        | 寄存器/线程数限制 |
| 相对朴素加速              | 1.00x       | 3.06x       | 3.7x         | 见各自的实测表    |
| 常量内存广播是否生效        | 是（25 次）   | 是（25 次）   | 是（125 次）  | warp 内同地址    |
| 计算阶段 warp 发散         | 边界 block 发散 | 无          | 无           | 靠预填 halo 消除  |
| bank conflict            | 无（无共享内存） | 无          | 无（本配置）   | 换 block 形状需重算 |
+--------------------------+-------------+-------------+--------------+--------------+
```

性能优化技巧总结

1. **mask 一律放 `__constant__` 常量内存**：为什么有效——warp 内 32 个 lane 读同一个地址时，常量缓存只需 1 个周期完成广播，等价于一次标量读，而放全局内存会产生"128B 事务只用到 4B"的 3.1% 利用率灾难。
2. **把输入 tile（含 halo）一次性协作加载进共享内存，再让计算阶段从共享内存取数**：为什么有效——同一个输入元素在卷积中被 25（2D）或 125（3D）个输出使用，共享内存把 400–800 周期的全局延迟替换成约 5 个周期的片上访问，全局访问减少 $\texttt{TILE}^2\texttt{MW}^2/(\texttt{TILE}+\texttt{MW}-1)^2$ 倍。
3. **在加载阶段完成边界判断，把 halo 预填零，使计算阶段无分支**：为什么有效——把 25 次带 `if` 的迭代变成 25 条纯 FMA，消除了每个输出的谓词计算与 warp 发散（`__syncthreads()` 也必须放在分支之外）。
4. **选择"输入并行"或"输出并行"取决于 mask 大小**：为什么有效——输入并行（block 大小 = 输入 tile 大小）在加载阶段零发散，但当 `MASK_WIDTH` 很大时（如 9×9、16×16 block 只剩 8×8 = 64 个线程算输出）计算阶段活跃 lane 比例暴跌；此时改用输出并行（block = TILE×TILE，输入 tile 由多轮加载完成）更划算。
5. **用线程粗化（每线程多个输出）摊薄常量读、地址计算与共享内存遍历**：为什么有效——粗化 `C` 倍后，每次常量读喂 `C` 次 FMA、每轮地址计算服务 `C` 个输出，指令数下降接近 `C` 倍，同时把"加载完就闲置"的线程变成有效算力。
6. **让 warp 的全局访问尽量落在一行内连续 32 个元素上**：为什么有效——一个 warp 的合并访问 = 128 字节 = 一条 cache line；把加载循环写成"线性下标 + 步长为线程数"的循环（而不是三重循环）能最大化连续段长度，减少 sector 浪费。
7. **共享内存行长加 1（padding）或选择非 32 倍数的行长**：为什么有效——当 warp 的 lane 跨越最内层以外的维度时，跨维步长 `% 32 == 0` 会导致所有 lane 撞到同一批 bank（N 路冲突）；行长补 1 使步长变奇数即可散开，代价仅 3–8% 的共享内存。
8. **按 32 的倍数对齐 tile 尺寸与图像宽度**：为什么有效——避免 warp 跨 cache line 边界造成的写回事务翻倍；同时避免 `400 线程 = 12.5 warp` 这种"半个 warp 浪费"的 block 尺寸。
9. **用 Roofline 判断该优化访存还是优化计算**：为什么有效——当 `AI < 平衡点`（A100 的 12.5 FLOP/Byte）时，任何算术层面的优化都是徒劳的，只有减少字节数（分块、粗化、向量化）才有收益；反之则应提高 ILP 与占用率。
10. **3D 卷积优先沿 x 方向保证合并、用 pitch 或展平索引正确寻址**：为什么有效——3D 数组沿 z 的步长是 `W*H*4` 字节（128^3 时是 64 KB），完全无法合并；且用 `cudaMalloc3D` 后必须用 `pitch/4` 而非常数 `W` 计算行偏移，否则数据整体错位。

关键要点

- 卷积是"每个输出取一遍邻近窗口"的 stencil 算子；2D 的每个输入元素被 $\texttt{MASK_WIDTH}^2$ 个输出复用（5×5 时 25 次）、3D 被 $\texttt{MASK_WIDTH}^3$ 个输出复用（5×5×5 时 125 次），这个复用次数就是全部优化空间的上限。
- 朴素卷积的算术强度只有 **0.5 FLOP/Byte**（每输出 50 次 FLOP、100 字节全局读），在 A100（平衡点 12.5 FLOP/Byte）上只能达到峰值的 4%，是典型的内存带宽受限算法。
- **常量内存适合只读的广播型数据**：`__constant__` + `cudaMemcpyToSymbol` + 64 KB 上限；warp 内同地址访问只需 1 个周期（广播），但 warp 内地址分歧时会被串行化最多 32 次。
- **分块是卷积性能的关键**：把 $(\texttt{TILE}+\texttt{MASK_WIDTH}-1)^2$ 个输入元素（含 halo）搬进共享内存后，全局访问减少 $\texttt{TILE}^2\texttt{MW}^2/(\texttt{TILE}+\texttt{MW}-1)^2$ 倍（TILE=16、MW=5 时是 **16×**），算术强度从 0.5 提升到 8.0 FLOP/Byte。
- **边界处理必须在设计阶段决定并统一**：补零 / 夹紧 / 环绕三种策略会给出不同的数值结果；把 halo 预填零进共享内存，可以同时解决"边界语义"与"计算阶段 warp 发散"两个问题。
- **三维卷积的瓶颈会从全局内存转移到共享内存与寄存器**：halo 体积 $(\texttt{TILE}+2r)^3$（TILE=8、r=2 时是 3.375 倍放大）、125 个 mask 系数无法常驻寄存器、沿 z 的访存步长高达 `W*H*4` 字节——这三点决定了 3D 卷积必须用常量内存 + 分块 + 粗化三管齐下。

常见陷阱与注意事项

- **忘记 `cudaDeviceSynchronize()`（或 `cudaEventSynchronize`）就计时**：测出的耗时会小到不合理（kernel 还在异步执行）→ 用 `cudaEventRecord` 包住 kernel 并在 `cudaEventElapsedTime` 前同步，或在验证前 `cudaDeviceSynchronize()`。
- **`__syncthreads()` 写在 `if` 分支里**：某些 warp 跳过屏障导致挂起（死锁）或读到未初始化的共享内存 → 把 `if` 只用于"算出要写入的值"，赋值语句与 `__syncthreads()` 都放在分支之外。
- **忘记 `__syncthreads()`**：计算阶段读到共享内存中尚未被其他线程写好的 tile（数据竞争，结果随机错乱，且往往只在某些 block 上出错）→ 在"协作加载"与"开始计算"之间必须有一条屏障。
- **共享内存 bank conflict**：例如把粗化写成"每线程沿 x 2 个输出"（地址步长为 2）时，32 个 lane 只落在 16 个偶数 bank 上，产生 2-way 冲突且 **padding 无法修复** → 把粗化方向改为沿 y/沿 z（保持每 lane 的 x 步长为 1），或改用 `float2` 向量访问。
- **warp 发散来自计算循环里的边界 `if`**：把 `if (idx >= 0 && idx < N)` 写进 `for (int j = 0; j < MASK_WIDTH; ++j)` 的循环体内，会让每个输出多出 25 组谓词判断，且边界 block 的 warp 被屏蔽 → 把边界判断移到分块的加载阶段，用补零的 halo 换取无分支的计算循环。
- **未检查 CUDA API 返回值 / kernel 启动错误**：`cudaMemcpyToSymbol` 的尺寸写错、`cudaMalloc` 失败、网格配置非法（`blockDim` 超过 1024 或 `gridDim` 超过 $2^{31}-1$）都只会在后续某次调用才报"unspecified launch failure" → 每次调用后立刻用统一宏检查，kernel 启动后加 `cudaGetLastError()`。
- **越界访问**：`Ns[ty+i][tx+j]` 中当 `ty+4` 或 `tx+4` 超出共享内存声明范围时会静默踩到相邻数据；全局侧 `N[nrow*Width+ncol]` 越界可能读到合法但不相关的内存 → 三维数组的下标拆解（`lz = l/(SX*SY)` 等）务必用整数除法验算，`(TILE+MASK_WIDTH-1)` 的每一维都要留够。

- **`cudaMemcpy` 方向写错**：把 `cudaMemcpyHostToDevice` 写成 `cudaMemcpyDeviceToHost`（或反过来）会得到全零或乱码结果，而 API 不报错（只要指针类型是 `void*`）→ 记住 `dst` 在前、`src` 在后；常量内存必须用 `cudaMemcpyToSymbol`，用 `cudaMemcpy` 拷贝 `__constant__` 符号会失败。
- **整数除法取整导致网格覆盖不足**：`grid = Width/TILE`（整除）会漏掉最后不满一个 tile 的输入/输出 → 统一写成 `(Width + TILE − 1)/TILE`，并在 kernel 里对越界输出用 `if (ox >= Ws) return;` 保护。
- **共享内存容量超限**：静态 `__shared__` 超过 48 KB（或超过本架构每 block 上限）会**启动期报错**，而不会自动降级 → 用 `cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize, bytes)` 配合动态共享内存申请更大的空间（A100 上限 163 KB/block）。
- **host/device 指针混用**：把 `h_N` 传给 kernel（或在 kernel 里解引用 `h_Mck`）会导致 illegal memory access 或结果全错 → 主机端参考实现与设备端 kernel 使用**两套独立的 mask 副本**，命名上区分（`h_Mck` / `Mc`）。
- **用 `cudaMalloc3D` 后仍用 `W` 而不是 `pitch/4` 算行偏移**：kernel 读到错误的行，结果全错但不会崩溃（3D 的"沉默 bug"）→ 从 `cudaPitchedPtr` 里取 `pitch`，换算成 float 数后用 `y*Wp + x` 寻址。

思考题（带答案）

Q1. 一个 2D 卷积使用 `5×5` 的 mask，采用 `TILE_WIDTH = 16` 的输出分块并行策略，且 halo 与中心 tile 一起加载进共享内存。请计算：(a) 全局内存访问的减少倍数；(b) 每个输出元素对应的输入加载量（halo 放大比）；(c) 若把 TILE 提到 32，减少倍数变成多少，算术强度从多少变成多少（A100，FP32 峰值 19.5 TFLOP/s、带宽 1555 GB/s），并判断是否仍是带宽受限。

(a) 减少倍数 `R = TILE²×MW²/(TILE+MW−1)² = 256×25/400 = 6400/400 = 16.0×`。(b) halo 放大比 `A = (TILE+2r)²/TILE² = 20²/16² = 400/256 = 1.5625`，即每个输出对应 1.5625 次输入加载（多加载 56.25%）；两者满足 `R = MW²/A = 25/1.5625 = 16`。(c) `TILE = 32` 时 `R = 1024×25/1296 = 19.75×`，每输出全局字节 `= 100/19.75 = 5.06 B`，算术强度 `= 50/5.06 = 9.88 FLOP/Byte`，比 TILE=16 的 8.0 提高 23.5%。A100 的机器平衡点是 `19.5e12/1555e9 = 12.5 FLOP/Byte`，`9.88 < 12.5`，**仍然是带宽受限**（带宽可支撑 `9.88×1555 = 15.4 TFLOP/s`，是峰值的 79%）。要真正转为计算受限，需要更大的 mask（`MW = 9`、`TILE = 32` 时 `R = 51.8`，`AI = 162/6.25 = 25.9 > 12.5`）。

Q2. 在示例 2 的分块 kernel 中，`__shared__ float Ns[20][20]`（行长 20）、`blockDim = (20,20)`，计算阶段读 `Ns[ty+i][tx+j]`。请具体到 bank 编号论证是否发生 bank conflict；如果改用 3D 的 `__shared__ float Ns[12][12][12]`，而 block 改成 `dim3(8,4,8)`，结论会怎样变化?

共享内存 32 个 bank，`bank = (float 下标) % 32`。2D 情形下一个 warp 的 32 个 lane 是 `ty=0, tx=0..19`（lane 0–19）与 `ty=1, tx=0..11`（lane 20–31）。它们的线性下标分别是 `i*20+j+tx`（0..19）与 `(i+1)*20+j+tx'`（0..11），即从 `i*20+j` 开始的**连续 32 个 float**，取模 32 后恰好覆盖 bank 0..31 各一次——**无 bank conflict，1 个周期完成**。3D 情形改 `blockDim = (8,4,8)` 后，一个 warp 的 lane 会跨越 `tz = 0..3`（因为线性化顺序是 tx 最快、tz 最慢，`8×4 = 32` 恰好是一个 warp），此时 z 方向步长 `= COARSE_Z×SY×SX = 2×12×12 = 288`，而 `288 % 32 = 0`，四个 `tz` 的 lane 落在**完全相同的 bank** 上 → **4-way bank conflict**，共享内存吞吐降到 1/4。修复办法是把 `Ns` 声明成 `[SZ][SY][SX+1]`（行长 13）：z 步长变成 `2×12×13 = 312`，`312 % 32 = 24`，四组 bank 偏移变成 `0, 24, 16, 8`，互不重叠 → 无冲突，代价是共享内存从 6912 B 增到 7488 B（+8.3%）。

Q3. 一台 A40 GPU 的 FP32 峰值算力是 37.4 TFLOPS，显存带宽 696 GB/s。请回答：(a) 它的机器平衡点（byte-to-FLOP 比）是多少，为了跑满算力每个字节需要被复用多少次? (b) 用它跑朴素 1D 卷积（`MASK_WIDTH = 5`）能达到峰值的百分之几? (c) 若要用 1D 分块卷积跑满算力，`MASK_WIDTH` 至少要到多少（定理：分块后每输出字节数 = `4(MW)/(R)`，其中 1D 的 `R = TILE×MW/(TILE+MW−1)`）? 请对 `TILE_WIDTH = 1024` 估算。

(a) `696 GB/s ÷ 37.4 TFLOP/s = 0.0186 B/FLOP`，即每 1 字节数据要做 `1/0.0186 = 53.7` 次浮点运算才算把算力吃满（讲义原文给的正是 53.7）。等价地，机器平衡点 = 53.7 FLOP/Byte。(b) 朴素 1D 卷积的算术强度 = 0.5 FLOP/Byte（每输出 2·MW = 10 FLOP，每输出 4·MW = 20 字节的 N 读取）。带宽可支撑算力 `= 0.5 × 696 = 0.348 TFLOP/s = 37.4 TFLOP/s 的 0.93%`。注意 M 的读取走常量缓存，不计入 DRAM 流量，否则口径变成 1 FLOP/4B 会更低。(c) 1D 分块后每输出 DRAM 字节 `= 4·MW/R`，其中 `R = TILE×MW/(TILE+MW−1)`，所以 `AI = 2MW / (4MW/R) = R/2`。要跑满算力需要 `AI ≥ 53.7`，即 `R ≥ 107.4`。但 1D 的 `R` 有上限：`R = TILE·MW/(TILE+MW−1) < MW`（当 `TILE → ∞` 时 `R → MW`），因此仅靠分块**永远无法**让 1D 卷积跑满 A40 的算力（`MW` 必须大于 107，而 mask 会长到不现实）。这正是讲义那张表（`TILE_WIDTH = 1024`，在老 GPU 上 `MW = 55` 才 100%）的含义：**只有 2D/3D 卷积（复用次数按 `MW²` / `MW³` 增长）才可能平衡算力与带宽**——2D 分块后 `AI = MW²/2`，`MW = 11` 时就已经达到 60 FLOP/Byte ≈ 53.7，足够填满 A40。这也解释了为什么 3D 卷积（`AI = MW³/2`，`MW = 5` 时 62.5 FLOP/Byte）在理论上更容易达到计算受限，真正的瓶颈反而转移到共享内存带宽与寄存器压力上。

Lecture 9: 并行模式七 —— 稀疏矩阵向量乘：压缩格式与负载均衡 (对应 Lab 7 / Lab 8: Sparse Matrix Multiply)

概述

稀疏矩阵向量乘（Sparse Matrix–Vector Multiplication, SpMV，即 `y = A * x`）是科学计算、图分析、推荐系统与深度学习中最常见的核心算子之一：偏微分方程（PDE）离散化、有限元网格、社交网络邻接矩阵、GNN 的邻接聚合都可以归结为 SpMV。现实矩阵中 99% 以上的元素是 0，稠密存储要付出 $O(N^2)$ 的存储与带宽代价，因此必须压缩成 CSR / COO / ELL / JDS / HYB 等格式，把字节数降到 O(nnz)（nnz = number of non-zeros，非零元个数）。压缩解决了"存什么"，却制造了新的问题：**规则性（regularity）丢失**——每行长度不同导致 warp 内控制发散，列索引数据相关导致 `x[colIdx[j]]` 的间接访存无法合并，工作量差异导致负载不均衡。本讲的主线就是把不规则的稀疏数据重新"规则化"：用 padding（ELL）拉平行长、用转置（ELL/JDS_T）恢复合并访存、用排序（JDS）让相邻线程工作量接近、用混合格式（HYB = ELL + COO）把少量离群超长行单独处理。CUDA 机制上本讲几乎不引入新 API（只用到 `__ldg`、`__shfl_down_sync`、共享内存与原子操作），考验的是**数据布局设计能力**：布局决定了 warp 的访存是否合并、循环是否发散、线程负载是否均衡，也决定了最终能拿到峰值的百分之几。

核心概念与 GPU 架构图解

1. 稀疏性与压缩动机（Sparsity and Compaction）

- **定义与目的**：稀疏矩阵指非零元素个数 nnz 远小于 N^2 的矩阵，密度 d = nnz / N^2。压缩（compaction）的目的有三个：(a) 减少存储容量，让矩阵能放进显存甚至片上存储；(b) 减少必须搬运的字节数，因为 GPU 上绝大多数算子受 DRAM 带宽限制；(c) 把省下来的带宽预算留给有用数据，提高有效算力。讲义明确指出：稀疏方法的好处是 reducing consumption of memory bandwidth、better utilization of on-chip memory、fewer bytes transferred to on-chip memory、better utilization of global memory，而挑战是 retaining regularity。
- **直观解释（"它是什么？"）**：想象一个 10 万座的体育场，实际只卖出 800 张票。如果按"每个座位都写一个值"的方式记录（稠密存储），需要 10 万条记录；如果改成"只记录有人坐的座位号与观众姓名"（压缩存储），只需 800 条。这就是 CSR 做的事：`colIdx` 记座位号、`values` 记观众。但代价是：稠密的体育场任何时刻你都知道第 5 排第 3 座在哪里（地址可以解析计算 `i*N+j`），压缩后你必须先查表才知道第 5 排有人坐的是哪几个座位——**地址从"算出来的"变成了"查出来的"**，这一个变化引发了后面所有的性能问题。
- **架构/机制图解**：下图对比同一个 4×4 稀疏矩阵的稠密存储与 CSR 存储。

```
      DENSE (N=4)                      CSR (nnz=7)
A = [ 3  0  1  0 ]              values  = { 3, 1,  2, 4, 1,  1, 1 }   (7 floats = 28 B)
    [ 0  0  0  0 ]              colIdx  = { 0, 2,  1, 2, 3,  0, 3 }   (7 ints   = 28 B)
    [ 0  2  4  1 ]              rowPtr  = { 0, 2, 2, 5, 7 }           (5 ints   = 20 B)
    [ 1  0  0  1 ]              ------------------------------------------------
                                合计 76 B        (稠密需要 16 * 4 = 64 B)

稠密存储的地址: addr(i,j) = base + (i*N + j)*4      <- 可解析计算，编译期可展开
CSR 存储的地址: addr(i,j) 需先查 rowPtr[i]..rowPtr[i+1] 再扫描 colIdx 找 j
                                                      <- 数据相关，只能运行时查表

内存占用随规模的变化 (float 值 + int 索引, 每 nnz 8 B):
  N = 1e3, 密度 1%   : 稠密 4 MB      | CSR  0.008 MB + 0.004 MB  = 0.012 MB
  N = 1e6, 密度 0.1% : 稠密 4000 GB   | CSR  8 GB + 4 MB          = 8.0 GB
  N = 1e6, 密度 5%   : 稠密 4000 GB   | CSR  400 GB + 4 MB        = 400 GB
```

关键性能特征：CSR 的字节开销是 `8*nnz + 4*(N+1)`，稠密是 `4*N²`。令两者相等可得压缩的盈亏平衡点：

```
8*nnz + 4*(N+1) < 4*N^2   <=>   nnz < (N^2 - N - 1) / 2   ≈  N^2 / 2

例: 4x4 矩阵的盈亏平衡点是 nnz <= 5, 而本例 nnz = 7 > 5,
    所以 CSR 反而比稠密多占 76 - 64 = 12 B (压缩格式有自己的固定开销 rowPtr)。
    N = 1e6 时平衡点是 nnz ≈ 5e11, 实际上 nnz 往往只有 1e7 ~ 1e9, 压缩收益 3~4 个数量级。
```

一个形象的量化：在 N = 10^6 的 PDE 网格（每行 7 个非零元）上，稠密存储需要 4 TB，超过任何单卡显存；CSR 只需 56 MB。**正是压缩让这个矩阵可以被求解**。压缩的第二个收益是"每个字节都可能有用"：稠密 SpMV 读进来的浮点数里 99% 是 0，这些 0 白白占用 DRAM 带宽；CSR 则是 100% 有效载荷（payload）加上约 50% 的索引开销。

2. 规则性丢失（Loss of Regularity）

- **定义与目的**：规则性指"程序的控制流与访存地址可以由线程/迭代编号用解析式（affine expression）算出，因而对同一 warp 内 32 个线程是同构的"。稠密算子天然规则：所有行等长、地址 = `i*N+j`、每次迭代 32 个 lane 读 32 个相邻地址。稀疏数据打破了这两条：(a) 行长不同 → 每线程循环次数不同 → **控制发散（control divergence）**；(b) 列索引存放在数组里 → 地址数据相关 → **访存发散（memory divergence）**。讲义把这一点总结为 "Compared to dense matrix multiplication, SpMV is irregular/unstructured, has little input data reuse, benefits little from compiler transformation tools"，并把性能关键归纳为两条：maximize regularity（by reducing divergence and load imbalance）与 maximize DRAM burst utilization（layout arrangement）。
- **直观解释（"它是什么？"）**：稠密计算像军训方阵：发令"齐步走"，32 个人同时迈左脚，队伍效率 100%。稀疏计算像 32 个身高体重各异、携带行李数量不同的旅客同乘一部电梯：有人拎 1 件行李，有人拎 50 件；电梯（warp）必须等到最后一个旅客搬完才能关门，**其余 31 人的时间全部浪费**。更糟的是，行李（数据）不在同一个货架上，每个人都得单独跑一趟仓库（独立 transaction），而不是让一个人把一整车推过来。
- **架构/机制图解**：下图对比稠密 warp 与稀疏 warp 在每个"迭代"上的 lane 占用情况。

```
DENSE GEMV (A 列主序): 每行长度相同 (N = 8), warp 内 32 个 lane 完全同步
iter 0 :  [ lane0..lane31 共 32 个格子, 全部为 R ]   32/32 有效
iter 1 :  [ lane0..lane31 共 32 个格子, 全部为 R ]   32/32 有效
iter 2 :  [ lane0..lane31 共 32 个格子, 全部为 R ]   32/32 有效
iter 3 :  [ lane0..lane31 共 32 个格子, 全部为 R ]   32/32 有效
iter 4 :  [ lane0..lane31 共 32 个格子, 全部为 R ]   32/32 有效
iter 5 :  [ lane0..lane31 共 32 个格子, 全部为 R ]   32/32 有效
iter 6 :  [ lane0..lane31 共 32 个格子, 全部为 R ]   32/32 有效
iter 7 :  [ lane0..lane31 共 32 个格子, 全部为 R ]   32/32 有效
地址: 固定 iter 时 lane t 读 base+t*4 -> 32 x 4 B = 128 B 连续 (完美合并)
效率 = 8 iter x 32 lane = 256/256 = 100%, DRAM 利用率 = 100% (每次读 128 B 全部有用)

SPARSE CSR: 行长度 3,2,5,2,4,2,3,2 (此处画 8 个 lane 便于阅读; 完整 32 lane 版本见概念 8)
iter 0:   [ R ][ R ][ R ][ R ][ R ][ R ][ R ][ R ]   8/8  有效, 但地址分散 -> 最多 8 个 sector
iter 1:   [ R ][ R ][ R ][ R ][ R ][ R ][ R ][ R ]   8/8  有效
iter 2:   [ R ][   ][ R ][   ][ R ][   ][ R ][   ]   4/8  有效 (lane 1,3,5,7 已退出循环)
iter 3:   [   ][   ][ R ][   ][ R ][   ][   ][   ]   2/8  有效
iter 4:   [   ][   ][   ][   ][ R ][   ][   ][   ]   1/8  有效
效率 = 23 / (8 x 5) = 57.5%      (真实 32 lane 例子见概念 8 的 44.2%)

三条同时恶化的东西:
  1) 控制发散  : 后面的 iteration 只有少数 lane 在工作
  2) 访存发散  : 每行起点 rowPtr[row] 不同 -> lane 地址不是相邻 4 B, 而是散落在 1152 B 范围
  3) 负载不均衡: warp 结束时间由最长行决定
```

性能特征数字：稠密 GEMV 在 A100 上可以稳定跑到 1300 GB/s（峰值 1555 GB/s 的 85%）；而朴素 CSR SpMV 只能跑到约 450 GB/s（29%）。差距全部来自"规则性"，而不是来自于计算量——**稀疏版本做的浮点运算比稠密版本少 4 个数量级，却只快了几十倍**。这正是 Kirk/Hwu 反复强调的："稀疏计算的问题不是算得少，而是搬得乱。"

3. CSR 格式（Compressed Sparse Row）

- **定义与目的**：CSR 用三个一维数组表示一个 N×N 矩阵：
- `values[nnz]`：按行优先顺序存放所有非零元的值，float 占 4 B；
- `colIdx[nnz]`：与 `values` 一一对应的列号，int 占 4 B；
- `rowPtr[N+1]`：第 i 行的非零元在 `values` / `colIdx` 中的区间为 `[rowPtr[i], rowPtr[i+1])`，多出的第 N+1 个元素是哨兵（sentinel），长度取 N+1 而不是 N，正是为了让"最后一行的结束位置"也能用同一个公式 `rowPtr[row+1]` 取到，从而消除 kernel 里的特判分支。
- **直观解释（"它是什么？"）**：CSR 就像一本按章节组织的书：`rowPtr` 是目录，告诉你"第 3 章从第 25 页开始、到第 28 页结束"；`values` 是正文内容，`colIdx` 是每一行正文对应的"小节编号"。读书的人（线程）只要拿到自己的章号（row id），就能通过目录找到自己那一小段正文，且**同一章的正文在物理上是连续的**（这正是 CSR 比 COO 好的地方：一个线程读自己那一行时是顺序流式访问，硬件预取器与 DRAM row buffer 都很友好）。
- **架构/机制图解**：用讲义中的 4×4 矩阵（`nnz = 7`）给出完整编码实例。

```
矩阵 A (4x4, nnz = 7)                CSR 三个数组的物理布局
A = [ 3  0  1  0 ]   row 0: 2 个       idx :  0    1    2    3    4    5    6
    [ 0  0  0  0 ]   row 1: 0 个     values:  3    1    2    4    1    1    1
    [ 0  2  4  1 ]   row 2: 3 个     colIdx:  0    2    1    2    3    0    3
    [ 1  0  0  1 ]   row 3: 2 个     rowPtr:  0    2    2    5    7
                                           ^    ^    ^    ^    ^
                                           |    |    |    |    +-- rowPtr[4] = 7  (哨兵 = nnz)
                                           |    |    |    +------- rowPtr[3] = 5  -> row 3 = [5,7)
                                           |    |    +------------ rowPtr[2] = 2  -> row 2 = [2,5)
                                           |    +----------------- rowPtr[1] = 2  -> row 1 = [2,2) 空行!
                                           +---------------------- rowPtr[0] = 0  -> row 0 = [0,2)

逐行解码:
  row 0: (val 3, col 0) (val 1, col 2)          -> A[0][0]=3, A[0][2]=1
  row 1: 区间 [2,2) 为空                        -> A[1][*]=0   (rowPtr[i] == rowPtr[i+1] 时空行)
  row 2: (val 2, col 1) (val 4, col 2) (val 1, col 3)
  row 3: (val 1, col 0) (val 1, col 3)

容量对比:  稠密 = 4 * N^2 = 64 B      CSR = 8*nnz + 4*(N+1) = 56 + 20 = 76 B
访问一个非零元:  values[k] 和 colIdx[k] 的地址 = base + k*4, k 由 rowPtr 查表得到
线程行为:  每线程一行 -> 该线程对 values/colIdx 是连续流式读, 但不同线程的起点不同
```

关键性能特征：`rowPtr` 只有 N+1 个元素、每个线程只读 2 个（`rowPtr[row]` 与 `rowPtr[row+1]`），因此它的 4 B/行 × 2 开销相对 nnz 可以忽略（n=9.4M 时约 4 MB，占总流量 5%）。CSR 最大的优势是通用性：任何稀疏结构都能表示，且"每线程一行"的映射非常简单。它最大的缺点是两个发散（控制 + 访存），这也是后续所有格式想要修复的对象。在 A100 上，一个线程读 `rowPtr[row]` / `rowPtr[row+1]` 时，如果 warp 内 32 个 lane 的行号连续，则两次 load 各自构成一次 128 B 合并访问（`rowPtr` 与 `rowPtr+1` 的访问有 31/32 重叠，第二次几乎全部命中 L1），代价可以忽略。

4. COO 格式（Coordinate / 坐标格式）

- **定义与目的**：COO 为**每个非零元**显式记录三元组 `(rowIdx[k], colIdx[k], values[k])`，共 3 个长度为 nnz 的数组。它的目的是提供最大并行度（nnz 个线程/元素互不依赖）并且**允许任意重排序**：既然行号是显式存储的，就可以按行号排序、按列号分区、按值分桶，从而为后续格式（CSR 的构建、ELL/JDS 的排序、HYB 的拆分）提供自由。讲义明确指出 "COO Allows Reordering of Elements"。代价是：多一个 `rowIdx` 数组（比 CSR 多 4 B/nnz，流量增加 50%），而且同一行的多个元素会被不同线程同时写回 `y[row]`，必须使用原子操作或分段归约。
- **直观解释（"它是什么？"）**：COO 像快递站里的"待派件清单"：每一行写的是"收件人编号、物品编号、物品"。清单不按收件人排好序，但正因为每一项自带收件人编号，你可以随时把清单重新排序、按小区分组（这就对应把 COO 转成 CSR：先排序再数数）。而派送（写 `y`）时，如果同一个收件人有 5 件快递、由 5 个不同的快递员同时送到，就必须在门口排队登记（原子加），否则会互相覆盖。
- **架构/机制图解**：

```
同一个 4x4 矩阵的 COO 编码
  values = { 3, 1, 2, 4, 1, 1, 1 }
  colIdx = { 0, 2, 1, 2, 3, 0, 3 }
  rowIdx = { 0, 0, 2, 2, 2, 3, 3 }     <- 注意: 同值元素顺序任意, 可以重排成 {2,2,2,0,0,3,3}

串行 SpMV/COO (讲义 page 24 原式):
  for (int i = 0; i < num_elem; i++)
      y[rowIdx[i]] += values[i] * x[colIdx[i]];

并行化后的硬件视图 (一个 warp 处理 32 个连续的非零元):

  lane:     0     1     2     3     4     5     6     7
  data :   [ 3 ] [ 1 ] [ 2 ] [ 4 ] [ 1 ] [ 1 ] [ 1 ] [ 0 ]   <- 合并: 连续 32 B
  col  :   [ 0 ] [ 2 ] [ 1 ] [ 2 ] [ 3 ] [ 0 ] [ 3 ] [ 1 ]   <- 合并读
  row  :   [ 0 ] [ 0 ] [ 2 ] [ 2 ] [ 2 ] [ 3 ] [ 3 ] [ 2 ]   <- 合并读
              |     |     |     |     |     |     |
              v     v     v     v     v     v     v
  y[row]:  y[0]  y[0]  y[2]  y[2]  y[2]  y[3]  y[3]  y[2]
             \____/            \_____________/
            写冲突!            写冲突!          -> 必须 atomicAdd(&y[r], v)

代价: 一条 global atomicAdd 到同一地址时, 同一 warp 内对同一地址的多次原子操作会被序列化
      (A100 的 L2 原子单元对同一地址一次只能处理一个), 最坏情况把 32 路并行退化成 32 次串行
流量: COO = 12 B/nnz (比 CSR 的 8 B/nnz 多 50%), 但并行度 = nnz 个线程, 与行长无关
```

关键性能特征：COO 的每个 lane 读取的 `values`、`colIdx`、`rowIdx` 都是**连续地址**（因为元素按线性编号分布），因此访存是完美合并的，控制流也完全无发散（所有线程迭代次数相同）。它的代价转移到了输出端：`atomicAdd` 到全局内存 `y[row]`。在讲义中 HYB 的 COO 部分用 segmented reduction（分段归约，先归约到共享内存再原子加）来实现，正是为了把原子冲突从"每元素一次"降到"每段一次"。

5. ELL / ELLPACK 格式（每次补齐到最长行）

- **定义与目的**：ELL 用两个二维数组（按列主序存储）表示矩阵：`data[N][E]` 与 `colIdx[N][E]`，其中 `E = max_i(nnz_in_row_i)` 是最长行的长度。不足 E 的行用**填充（padding）**补齐。目的是消除控制发散：所有行的循环次数都变成同一个常数 E，`for (i = 0; i < numElem; i++)` 对每个线程都是相同的 trip count，warp 内 32 个 lane 完全同步；同时因为按列主序（transposed）存放，`colIdx[i*N + row]` 在固定 i 时对相邻 row 是连续地址，于是**访存也变成合并的**。讲义把这一步总结为 Regularizing SpMV with ELL(PACK) Format：pad all rows to the same length、transpose (column major) for DRAM efficiency、both data and col_index padded/transposed。
- **直观解释（"它是什么？"）**：ELL 像把一群身高不齐的人塞进同一排照相：为了整齐，每个人脚下垫不同厚度的台子（padding），垫到和最高的人一样高。照片（内存布局）整齐了、冲洗效率高了，但如果队伍里混进一个 2.5 米的巨人，所有人都要垫到 2.5 米——浪费的木板（padding）比人本身还多。这就是讲义说的 "Inefficient if a few rows are much longer than others"，也是 HYB 格式诞生的原因。
- **架构/机制图解**：把上文 4×4 矩阵转成 ELL（E = 3，最长行是 row 2）。

```
   行优先的 ELL 逻辑视图 (padding 用 0 值填充)          转置后的物理布局 (列主序, DRAM 友好)
        i=0      i=1      i=2                     data     : { 3, 0, 2, 1,  1, 0, 4, 1,  0, 0, 1, 0 }
row0  ( 3, c0) ( 1, c2) ( 0,  * )                colIdx   : { 0, 0, 1, 0,  2, 0, 2, 3,  0, 0, 3, 0 }
row1  ( 0,  * ) ( 0, * ) ( 0,  * )               长度 12 = N * E = 4 * 3 (CSR 只有 7 个真实元素)
row2  ( 2, c1) ( 4, c2) ( 1, c3)                 额外开销 = 12/7 = 1.71x  (小矩阵上非常不划算)
row3  ( 1, c0) ( 1, c3) ( 0,  * )

   硬件访问模式 (warp 内 4 个连续 row, E = 3):
   i = 0:  data[0*4 + row0..row3] = 3, 0, 2, 1     <- 连续 16 B, 1 个 transaction (4 lane)
          colIdx[0*4 + row0..row3] = c0, c0, c1, c0  <- 连续读, 合并
          x[ c0 ], x[ c0 ], x[ c1 ], x[ c0 ]         <- gather, 但值域很窄 -> 大概率命中同一 L1 line
   i = 1:  data[1*4 + row0..row3] = 1, 0, 4, 1     <- 再一次连续合并读
   i = 2:  data[2*4 + row0..row3] = 0, 0, 1, 0
   所有 lane 迭代次数相同 -> 控制流 0 发散, 有效 lane 率 = 100%

   真实 18 点模板的 ELL 访问 (warp = 连续 32 行):
   i 固定时, 32 个 lane 读 colIdx[i*N + r], r = base..base+31, 地址连续 -> 1 个 128 B transaction
   而 colIdx 的值域是 r-1025 .. r+1025 (约 2050 个元素 = 8 KB = 64 条 cache line)
   -> gather 的 32 次访问落在 64 条 line 内, 有大量重叠, L1 命中率高
```

关键性能特征：ELL 的存储开销是 `8 * N * E` 字节。定义填充率 `E / avg_nnz_per_row`： – 9 点模板（行长 4/6/9）：E = 9，平均行长 ≈ 8.99，填充浪费 < 0.2%，ELL 的字节数甚至略少于 CSR（少了 `rowPtr`）； – 讲义式"少数超长行"矩阵（平均 9、最长 49）：E = 49，存储与流量放大 5.4 倍，ELL 直接输给 CSR。

因此 ELL 的适用判据是**行长方差小**（方差大时用 HYB）。另一个隐性收益是：ELL 的 `data` 数组每个 lane 只读 4 B 且地址连续，非常容易升级成 float4 向量化加载（见代码示例 3）。

6. JDS / JDS-T 格式（Jagged Diagonal Sparse，锯齿对角格式）

- **定义与目的**：JDS 先按行长**降序排序**所有行，再把每一"列"（称为一个 section）连续存放，形成锯齿状的对角线结构。它需要 4 个数组：`data`、`colIdx`（按排序后的顺序）、`jds_row_ptr`（每个 section 的起始偏移与长度）、`jds_row_perm`（排序后第 i 个位置对应原矩阵的哪一行，用于把结果写回正确的 `y[row]`）。讲义原文："Sort rows into descending order according to number of non-zero. Keep track of the original row numbers so that the output vector can be generated correctly." 目的：行长排序后，**相邻线程的行长非常接近**，warp 内最长行与最短行的差距被压缩，控制发散大幅减少（JDS vs CSR – Control Divergence: "neighboring threads tend to execute similar number of iterations because of sorting. Better thread utilization, less control divergence"）。JDS-T 是在 JDS 基础上再做一次转置（section 内按 lane 连续存放），从而把"相邻线程访问不相邻地址"的内存发散也修掉。讲义同时提醒：转置后的访问 "Not aligned with DRAM bursts but OK with recent GPUs"（新架构的 L2 以 32 B sector 为单位，容忍非 128 B 对齐的起始地址）。
- **直观解释（"它是什么？"）**：JDS 像运动会入场式前先把各班按人数排好队：先是 50 人的大班，再是 30 人的班，最后是 5 人的小班。这样每一"排"（section）里，各班的队尾长度差不多，不会出现"一排里 31 个班已经走完、只剩一个班还在走"的尴尬。`jds_row_perm` 就是花名册，告诉你排在第 3 位的是原来哪个班，颁奖（写 `y`）时才不会发错人。
- **架构/机制图解**：用同一个 4×4 矩阵走一遍 CSR → JDS 的转换。

```
   CSR:  row 0 (2 个)   row 1 (0 个)   row 2 (3 个)   row 3 (2 个)
   按行长降序排序后的行顺序: [ row2, row0, row3, row1 ]   <- 长度 3, 2, 2, 0

   JDS 的 section 结构 (每个 section 是"所有行长 >= i 的行"的第 i 个元素):
     section 0 (长度 4): row2->(2,c1)  row0->(3,c0)  row3->(1,c0)  row1-> 无
     section 1 (长度 3): row2->(4,c2)  row0->(1,c2)  row3->(1,c3)
     section 2 (长度 1): row2->(1,c3)

     data[7]          = { 2, 4, 1, 3, 1, 1, 1 }
     colIdx[7]        = { 1, 2, 3, 0, 2, 0, 3 }
     jds_row_ptr[5]   = { 0, 3, 5, 7, 7 }      <- 按"原始行号"排序的偏移: 行长 = 3,2,2,0
     jds_row_perm[4]  = { 2, 0, 3, 1 }         <- 第 i 个位置对应原矩阵第 perm[i] 行

   注意 jds_row_ptr 的语义: 与 CSR 的 rowPtr 结构相同 (前缀和), 但行号已被排序置换
     position 0 (原 row 2): [0,3)  得到 2, 4, 1
     position 1 (原 row 0): [3,5)  得到 3, 1
     position 2 (原 row 3): [5,7)  得到 1, 1
     position 3 (原 row 1): [7,7)  空

   写回输出:  y[ jds_row_perm[row] ] = dot;
```

JDS-T（转置版）进一步把同一 section 内的元素按 row 连续存放，于是固定 section 时 32 个 lane 读连续地址：

```
JDS-T 布局 (把同一 section 内的元素按 row 连续存放, 仍用同一个 4x4 例子)
  section 0 (参与的行: 原 row 0, 2, 3): data { 3, 2, 1 }   col { 0, 1, 0 }
  section 1 (参与的行: 原 row 0, 2, 3): data { 1, 4, 1 }   col { 2, 2, 3 }
  section 2 (参与的行: 原 row 2):       data { 1 }         col { 3 }

  matData[7]     = { 3, 2, 1,  1, 4, 1,  1 }     <- 按 matColStart 索引 (Lab 8 变量名)
  matCols[7]     = { 0, 1, 0,  2, 2, 3,  3 }
  matColStart[4] = { 0, 3, 6, 7 }                <- 每个 section 的起始偏移 (Lab 8 变量名)
  matRows[4]     = { 3, 3, 1, 0 }                <- 每个 section 的有效行数 (Lab 8 变量名)
  matRowPerm[4]  = { 2, 0, 3, 1 }                <- 排序后第 i 位对应原矩阵哪一行

kernel 循环条件 (讲义 page 24):
  while (matColStart[sec+1] - matColStart[sec] > row) {
      dot += data[matColStart[sec]+row] * x[colIdx[matColStart[sec]+row]];
      sec++;
  }
  ^ 含义: 线程 row 只参与"有效行数 > row"的那些 section

访存: 固定 sec 时, lane = 0..31 读 data[matColStart[sec] + lane], 地址连续 -> 合并
```

关键性能特征：JDS 把控制发散的"最大行长 / 平均行长"比值从全局最坏情况拉平到局部相邻（排序后相邻行长差通常 ⩽ 2），但**内存发散仍然存在**（讲义 "JDS vs. CSR – Memory Divergence: adjacent threads still access non–adjacent memory locations"），所以需要 JDS-T 的第二次转置。代价是：转换需要排序（主机端 O(nnz log N)）、多两个辅助数组（`matColStart`、`matRowPerm`），以及 kernel 里 `while` 循环的每次迭代都要读 `matColStart`（可放入共享内存或常量内存）。JDS 特别适合**大致三角（roughly triangular）**的矩阵，因为排序后的锯齿结构天然匹配三角矩阵的行长单调性（讲义："Roughly triangular – Probably best with JDS. Takes advantage of sparsity structure."）。

7. 混合格式 HYB（ELL + COO）

- **定义与目的**：HYB 把矩阵拆成两部分：**典型部分**交给 ELL（行长都 ⩽ 某个阈值 E），**离群部分**（少数超长行中超过 E 的那些非零元）交给 COO 处理，输出用 segmented reduction（分段归约）合并。目的是同时获得 ELL 的规则性与 COO 的灵活性：ELL 的 E 不再被一两个离群长行拖到极大（避免 padding 爆炸），而离群元素也不会破坏 ELL 的整齐结构。讲义："ELL handles typical entries, COO handles exceptional entries, implemented with segmented reduction. Often implemented in sequential host code in practice."（格式转换常在主机端串行完成）。
- **直观解释（"它是什么？"）**：机场安检的两条通道：98% 的旅客只带一个登机箱，走"标准通道"（ELL），速度快、队伍整齐；少数携带 5 件大件行李的旅客走"人工通道"（COO），单独处理。如果用同一条通道，所有人都要被最慢的旅客拖住（ELL 把所有人的行李格数 pad 到 5）；如果全部走人工通道（纯 COO），标准旅客也享受不到流水线速度。
- **架构/机制图解**：

```
行长度分布 (示意): 大部分行 4~9 个非零元, 少数行 40+ 个

行号:     0    1    2    3    4    5    6    7    8    9   10   11
行长:     9    8    9    7    9    9    8    9    9    8    9    9

行号:  1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033
行长:     9    9   49    9    9    8    9    9    9    8    9    9
                   ^ 第 1024 行是"枢纽行"(离群长行), 其余行都是 7~9

纯 ELL (E = 49):  浪费 = (49 - 8.99) / 8.99 = 445%   存储 8*N*49 字节
纯 CSR          :  存储 8*nnz + 4*(N+1) 字节, 但有发散
HYB (E = 9)     :  ELL 部分 8*N*9 字节 + COO 部分 8*(离群元素个数)

                 ELL 部分 (规则的 99.6% 元素)        COO 部分 (0.4% 离群元素)
                 +------------------------------+      +--------------------------+
                 | data[N][9]  colIdx[N][9]     |      | data[40960]              |
                 | 固定循环 9 次, 0 发散, 合并 |      | colIdx[40960]            |
                 +------------------------------+      | rowIdx[40960]            |
                              |                        +--------------------------+
                              v                                     |
                   y_partial[N] (每行一个部分和)                    v
                              \_____________ segmented reduction ___/
                                    (共享内存归约 + 每行一次 atomicAdd)
                                             |
                                             v
                                          y[N]
```

关键性能特征：HYB 的收益取决于分布的长尾程度。在 9 点模板 + 1% 离群行（每行多 40 个元素）的例子上：nnz = 9.42M + 0.04M，纯 ELL 需要 8 × 1.05M × 49 = 411 MB 流量，而 HYB（E = 9）只需 8 × 1.05M × 9 + 8 × 40960 = 75.5 MB + 0.33 MB ≈ 75.8 MB，**流量降到 1/5.4**，比纯 CSR（80 MB）还少。代价是 COO 部分需要归约与原子操作，以及主机端多一次划分（讲义："Often implemented in sequential host code in practice"）。现代 GPU 库（cuSPARSE 的 HYB、以及后续的 SELL-P/SELL-C-σ）都沿用了这个"主体规则 + 尾部灵活"的思想。

8. 负载不均衡（Load Imbalance）

- **定义与目的**：负载不均衡指同一 warp / block 内不同线程的**工作量（迭代次数）差异巨大**，而 SIMT 硬件要求一个 warp 的所有 lane 在同一时刻执行同一条指令——warp 的完成时间由**最慢的那个 lane**决定。在"每线程一行"的 SpMV 中，工作量就是行长 `rowPtr[row+1] - rowPtr[row]`。讲义对此有一句非常精炼的结论（Lecture 15 page 10）："Block performance is determined by longest row."（块性能由最长行决定）。目的是量化这种浪费并选择修复手段（排序 JDS、限幅 HYB、动态调度 work stealing）。
- **直观解释（"它是什么？"）**：32 个人一起做同一份"抄写作业"，每人抄自己那一章。老师规定"同一个小组必须同时交卷"（SIMT 的 lockstep）。第 7 章有 200 页，第 3 章只有 3 页：抄完 3 页的人只能干坐着等，直到抄 200 页的人写完。整个小组的"有效工作时间"= 所有人页数之和，（浪费）= 32 × 200 − 页数之和。**把章节按页数重新分配（排序）或允许抄完的人去帮别人（动态调度），就能把利用率从 44% 拉回 90% 以上。**
- **架构/机制图解**：下面是代码示例 2 中会用到的一个真实感 warp 行长分布（32 个 lane，模拟不规则网格/图矩阵）。

```
lane :  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
len  :  3   2   5   2   4   2   3   2   4   3   2   6   2   3   4   2
lane : 16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
len  :  3   2   5   3   2   4   3   2   3   7   2   3   4   2   3   2

sum(len) = 99,  max(len) = 7  (lane 25 是"最长行")

warp 执行时间轴 (每个格子 = 1 个迭代槽位, R = 该 lane 有真实工作):
lane  0 [R][R][R][ ][ ][ ][ ]        lane 16 [R][R][R][ ][ ][ ][ ]
lane  1 [R][R][ ][ ][ ][ ][ ]        lane 17 [R][R][ ][ ][ ][ ][ ]
lane  2 [R][R][R][R][R][ ][ ]        lane 18 [R][R][R][R][R][ ][ ]
lane  3 [R][R][ ][ ][ ][ ][ ]        lane 19 [R][R][R][ ][ ][ ][ ]
lane  4 [R][R][R][R][ ][ ][ ]        lane 20 [R][R][ ][ ][ ][ ][ ]
lane  5 [R][R][ ][ ][ ][ ][ ]        lane 21 [R][R][R][R][ ][ ][ ]
lane  6 [R][R][R][ ][ ][ ][ ]        lane 22 [R][R][R][ ][ ][ ][ ]
lane  7 [R][R][ ][ ][ ][ ][ ]        lane 23 [R][R][ ][ ][ ][ ][ ]
lane  8 [R][R][R][R][ ][ ][ ]        lane 24 [R][R][R][ ][ ][ ][ ]
lane  9 [R][R][R][ ][ ][ ][ ]        lane 25 [R][R][R][R][R][R][R] <- 最长行, 决定 warp 何时结束
lane 10 [R][R][ ][ ][ ][ ][ ]        lane 26 [R][R][ ][ ][ ][ ][ ]
lane 11 [R][R][R][R][R][R][ ]        lane 27 [R][R][R][ ][ ][ ][ ]
lane 12 [R][R][ ][ ][ ][ ][ ]        lane 28 [R][R][R][R][ ][ ][ ]
lane 13 [R][R][R][ ][ ][ ][ ]        lane 29 [R][R][ ][ ][ ][ ][ ]
lane 14 [R][R][R][R][ ][ ][ ]        lane 30 [R][R][R][ ][ ][ ][ ]
lane 15 [R][R][ ][ ][ ][ ][ ]        lane 31 [R][R][ ][ ][ ][ ][ ]

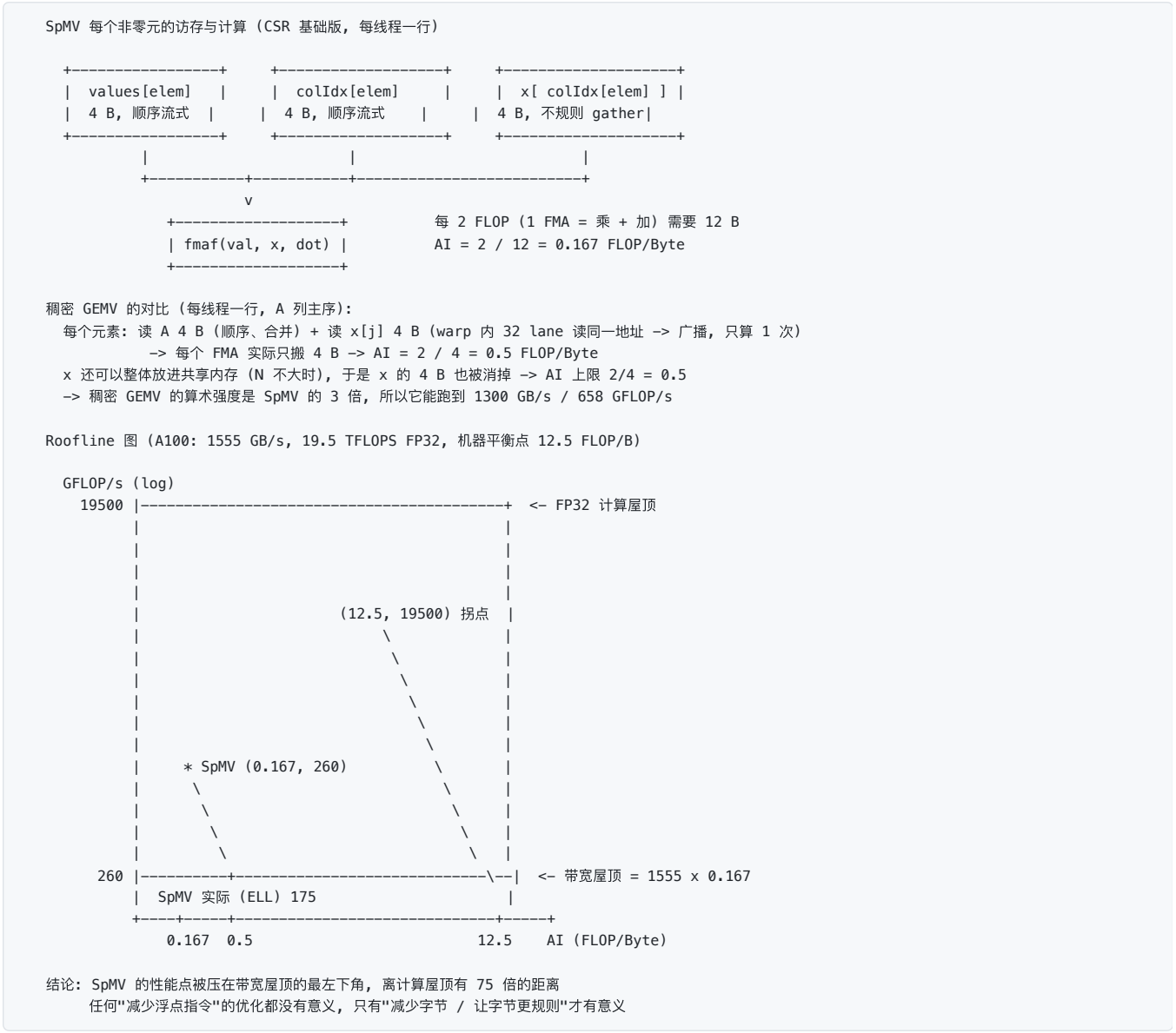
总槽位 = 32 lane x 7 iter = 224        有效工作 = 99
warp 内有效利用率 = 99 / 224 = 44.2%   -> 55.8% 的 lane-槽位被浪费
若长度分布极度倾斜 (例如 1 行 200, 31 行 3):
  利用率 = (200 + 93) / (32 x 200) = 293 / 6400 = 4.6%

修复思路与效果:
  按行长降序排序 (JDS): 相邻 lane 的长行聚在一起, 利用率 -> 85%~95%
  限幅 + 混合 (HYB, E=9): 把 7 这类离群值移出主循环, 利用率 -> 接近 100%
  动态调度 (atomic 取行): 长行与短行交错在不同 warp 之间, 牺牲一点局部性换均衡
```

性能特征：负载不均衡在两种粒度上出现。(a) **warp 内/block 内**：如上表，直接浪费算力与访存槽位，且 warp 长时间占用 warp scheduler 的发射槽；(b) **block 间/wave 间（尾部效应 tail effect）**：如果每个 block 内最长行所在的线程决定 block 的寿命，而 grid 中最后一个 wave 只填满部分 SM，则整个 kernel 的尾部会有大量 SM 空转。以 A100 为例，若 grid 有 108×8 = 864 个 block 的容量而实际有 900 个 block，则第二个 wave 只有 36 个 block（占 SM 数的 33%），尾部时间等于"一整轮 block 的时间"，利用率损失可达 30%～50%。这也是"每线程多行 + grid-stride loop"这种线程粗化能改善尾部效应的原因。

9. SpMV 的访存模式与算术强度（Memory Access Pattern and Arithmetic Intensity）

- **定义与目的**：算术强度（arithmetic intensity, AI）定义为"每搬运 1 字节所执行的浮点运算次数（FLOP/Byte）"，它与机器平衡点（machine balance = FP32 峰值 / 带宽）比较，就能判断算子受带宽限制还是受计算限制（Roofline 模型）。SpMV 是最典型的**带宽受限（memory bound）**算子：它几乎不做任何可复用的计算，却必须把整个矩阵的压缩表示读一遍。讲义把它总结为 "has little input data reuse"，并强调稀疏方法的核心收益是 "reducing consumption of memory bandwidth"。定量分析 SpMV 的访存需求，是判断任何优化是否值得做的第一步。
- **直观解释（"它是什么？"）**：把 GPU 想成一家餐厅：厨房（SM / FP32 单元）每秒能做 19.5 万亿道菜，但服务员（DRAM 带宽）每秒只能从仓库搬 1.5 TB 食材。做一道"稠密矩阵乘"这样的菜，每份食材要反复加工 10 次以上（数据复用高），厨房是瓶颈；而 SpMV 这道菜是"每份食材只切一刀就上桌"（复用为零），服务员跑断腿厨房也吃不饱。算术强度就是"每公斤食材加工多少刀"：SpMV 是 0.167 刀/公斤，而 A100 的厨房配置是 12.5 刀/公斤才算平衡——差了 75 倍。
- **架构/机制图解**：



关键性能特征与数字（A100，FP32）：读取每个非零元需要 `values` 4 B + `colIdx` 4 B + `x` 4 B = 12 B，对应 2 FLOP，因此 AI = 0.167 FLOP/Byte。性能上限 = 1555 GB/s × 0.167 FLOP/Byte ≈ **260 GFLOP/s**，只有 19.5 TFLOPS 峰值的 **1.3%**。这个数字是本讲所有讨论的出发点：**SpMV 的本质困难在数据搬运，而不在计算**。

推论（可用于指导所有优化决策）： 1. 减少 `colIdx` 的位宽（int32 → int16，当 N ⩽ 65536 时）可以把每 nnz 的字节从 12 B 降到 10 B，上限提升 20%——这是"减少字节"； 2. 行分块（block CSR）让 `colIdx` 只在块级别存一次，进一步降低索引开销； 3. 让 `x` 常驻共享内存或 L1（当 `4N` 字节放得下时），把 12 B/nnz 降到 8 B/nnz，上限提升 50%——这是最直接的收益，也是本讲"共享内存缓存 x"技巧的价值所在； 4. 提高访存的规则性（对齐、连续、可合并）能把**实际**带宽从峰值的 29%（朴素 CSR）拉到 50%～56%（ELL），这是把"理论上限"变成"实际成绩"的关键。

10. 间接访存 `x[colIdx[j]]` 的 gather（Indirect / Gather Access）

- **定义与目的**：`x[colIdx[j]]` 是"地址存在数组里的"间接访存（indirect access / gather）：被访问的地址在运行前未知，必须先把 `colIdx[j]` 从内存读出来，再做一次地址翻译。它存在的唯一原因是压缩格式把"列号"变成了数据。它与稠密版本的区别是本质性的：稠密版的地址 `A[i*N+j]` 是 affine 的（编译期可算出），硬件可以完美合并；gather 的地址是数据相关的，同一 warp 内 32 个 lane 的地址可能毫无规律。目的是理解"为什么稀疏 kernel 的带宽利用率只有稠密 kernel 的一半甚至三分之一"，并据此设计布局。
- **直观解释（"它是什么？"）**：图书馆借书。合并访问 = 你把 32 本书号按书架顺序报给管理员，管理员推着车一次性从同一排书架上取下 32 本（1 趟）；gather = 32 个书号随机分布在整个图书馆的 32 排书架上，管理员必须跑 32 趟，每趟只取 1 本。更糟的是，管理员每趟都会顺手把整排书架的一小格（32 B sector）搬回来——即使这一格里只有 4 B 是你需要的。**这就是 gather 的 8 倍放大：只用了 1/8 的搬运量。**
- **架构/机制图解**：以 x 的长度 N = 1,048,576（4 MB）为例，A100 的 L1 以 128 B cache line / 32 B sector 为粒度。

```
情形 A: 合并访问 (稠密 kernel, 或 ELL 中的 data/colIdx 数组)
  地址公式: addr(lane) = 0x1000 + lane*4, lane = 0..31
    lane0 -> 0x1000   lane1 -> 0x1004   lane2 -> 0x1008   lane31 -> 0x107C
  +-----------------------------------------------------------------+
  | 0x1000..0x107F: 128 B cache line (4 个 32 B sector), 全部有用  |
  +-----------------------------------------------------------------+
  事务数 = 4 sectors = 128 B 传输 / 128 B 有用  ->  效率 100%

情形 B: 随机 gather (x[colIdx[j]], 列号随机分布)
  列号:  lane0 = 91234  lane1 = 10577  lane2 = 88321  lane3 = 4509
  地址:  lane0 = 0x59xx lane1 = 0x0Axx lane2 = 0x56xx lane3 = 0x04xx
  这四个 lane 分别落在 4 条不同的 128 B line、4 个不同的 32 B sector 上;
  随机列号使其余 28 个 lane 同样各自落在不同的 line 上。
  +----+      +----+      +----+      +----+
  |32 B|      |32 B|      |32 B|      |32 B|   每个 lane 各占一个 sector
  +----+      +----+      +----+      +----+
    ^lane0     ^lane1     ^lane2      ^lane3   (lane4 到 lane31 同理, 共 32 个 sector)
  事务数 = 32 sectors = 1024 B 传输 / 128 B 有用  ->  效率 12.5% (8x 放大)

量化: x 有 L = 4 MB / 128 B = 32768 条 cache line
      32 个随机列号落在多少条不同的 line 上?
      E[不同 line 数] = L * (1 - (1 - 1/L)^32) ≈ 32768 * (1 - e^(-32/32768)) ≈ 31.98
      -> 几乎是 32 条不同 line, 即"零合并"

情形 C: 结构化 gather (9 点模板, ELL 布局, warp = 连续 32 行)
  9 点模板中, 第 i 个元素对应的列号 ≈ row + offset(i), offset 只有 9 种取值
  lane r 的列号 = r + offset  -> 32 个 lane 的列号聚集在约 35 个连续整数内
  地址范围 ≈ 140 B  ->  落在 2 条 cache line / 5 个 sector 内
  事务数 ≈ 5 sectors = 160 B / 128 B 有用  ->  效率 80%, 且后续迭代全部命中 L1
  -> 这就是"结构化稀疏矩阵(模板/带状)"能跑快, 而"随机稀疏矩阵"跑不快的根本原因
```

关键性能特征：gather 有两条不同的代价路径，必须区分清楚。 1. **DRAM 路径**：如果 `x` 太大、放不进 cache，则每次 gather 都可能是一次 DRAM 访问，8 倍放大直接吃掉带宽。此时唯一出路是把 `x` 做分块（tiling）：把矩阵按列切成 `x` 能放进 L2/共享内存的块，逐块做 SpMV 累加。 2. **L1/LSU 路径**：如果 `x` 很小（如 4 MB，A100 的 L2 是 40 MB，L1 是 128 KB/组），gather 基本命中 cache，不消耗 DRAM 带宽，但**仍然消耗 L1 的事务处理能力与 LSU 的发射槽**：一次 warp 的 gather 需要 32 个 sector 的 L1 tag 查询，L1 吞吐约 128 B/cycle，因此 gather 的有效吞吐远低于合并访问。A100 上每个 SM 的 L1 峰值约 128 B/cycle × 1.41 GHz ≈ 180 GB/s，108 个 SM 合计 19.5 TB/s；gather 把它拉低到 12.5% 的效率，相当于只剩 2.4 TB/s 的有效吞吐——当 L2/DRAM 带宽（1.5 TB/s）不足以掩盖这个瓶颈时，它就会成为新的限制。这也是讲义中 "Memory Divergence (Uncoalesced Accesses)" 一页想强调的现象。

与稠密 kernel 的本质差别总结：

维度	稠密 GEMV (A 列主序/共享内存 x)	稀疏 SpMV (CSR)
地址来源	解析式 (i*N+j), 编译期可知	查表 (rowPtr + colIdx), 运行期才知
合并粒度	32 lane x 4 B = 128 B 完美合并	data/colIdx 可能合并; x[colIdx] 几乎不合并
数据复用	x[j] 被全部 lane 广播复用	x[col] 有复用(取决于列分布), 矩阵本身零复用
warp 内一致性	所有 lane 循环次数相同	每 lane 循环次数 = 该行行长, 差异可达数十倍
编译器优化	#pragma unroll / 向量化 / 寄存器分块	几乎无法展开 (trip count 运行期才知)
算术强度	0.5 FLOP/B	0.167 FLOP/B
实测带宽占比	85% (1322 GB/s)	29% (449 GB/s)

11. SpMV 的优化技术总览（Optimization Toolkit）

- **定义与目的**：把前述问题映射到具体技术：每一个瓶颈（控制发散、访存不合并、gather、负载不均衡、字节太多）都有一套对应的修复手段。目的是建立一个"看到症状 → 选择技术"的对照表，避免盲目试错。
- **直观解释（"它是什么？"）**：像医生的处方单：不是所有病人都吃同一种药，而是先看症状（用 profiler 看是 control divergence 还是 uncoalesced load，是 dram__throughput 高还是 l1tex 事务多），再对症下药。
- **架构/机制图解**：技术与瓶颈的对应关系如下。

```
+--------------------------+-------------------------------------------------------+
| 性能症状 (profiler 指标) | 对应优化技术                                          |
+--------------------------+-------------------------------------------------------+
| branch_efficiency 低     | 1) ELL 补齐行长  2) JDS 按行长排序  3) HYB 限幅       |
| (warp 发散)              | 4) 每线程多行 (线程粗化), 让 warp 总工作量拉平        |
+--------------------------+-------------------------------------------------------+
| l1tex 事务数 / 有用字节  | 1) ELL/JDS-T 转置 -> data/colIdx 连续                 |
| 比值高 (未合并)          | 2) warp-per-row + __shfl_down_sync 归约               |
|                          | 3) 按 128 B 对齐 padding                              |
+--------------------------+-------------------------------------------------------+
| dram__throughput 高但    | 1) 共享内存/常量内存缓存 x (N 小)                     |
| 有效 GFLOP/s 低          | 2) colIdx 用 int16/uint16 降低位宽                    |
| (字节太多)               | 3) 块压缩 (BCSR) 摊薄索引; 4) 矩阵按列分块复用 x      |
+--------------------------+-------------------------------------------------------+
| 部分 SM 空闲 / tail      | 1) work stealing (全局 atomic 取行号)                 |
|                          | 2) grid-stride loop + 每线程多行                      |
|                          | 3) 调整 block 大小使 grid 是 SM 数的整数倍            |
+--------------------------+-------------------------------------------------------+
| 延迟受限 (MLP 不足)      | 1) float4 向量化加载, 提供 4 倍独立 load              |
|                          | 2) 每线程多个累加器 (寄存器分块)                      |
|                          | 3) __ldg / __restrict__ 走只读数据路径 (texture path) |
+--------------------------+-------------------------------------------------------+
| 共享内存 bank conflict   | 1) padding 列数使其不是 32 的倍数                     |
|                          | 2) 转置布局; 3) 让同一 warp 读连续地址                |
+--------------------------+-------------------------------------------------------+
```

关键性能特征（各技术的典型收益，A100 / 9 点模板 / nnz ≈ 9.4M）： 1. **共享内存缓存 x**：当 `4N ≤ 48 KB`（N ⩽ 12288）时，把 x 一次性搬进共享内存（延迟从 400–800 cycles 降到 20–30 cycles，且没有 L1 事务浪费），实测可提升 15%～30%；讲义中 "better utilization of on-chip memory / fewer bytes transferred to on-chip memory" 指的就是这一类收益。注意 N 很大时不可行（N = 10^6 需 4 MB ≫ 164 KB），此时退化为"按列分块"（一次只处理 x 的一个片段）。 2. **float4 向量化加载**：要求数据 16 B 对齐且连续。ELL 的 `data[i*N + row]` 在 row 连续时天然满足，一个 `float4` 同时喂 4 个行（每线程管 4 行），把 load 指令数降到 1/4，MLP 提升到 4 倍，实测带宽从 775 GB/s 提升到 873 GB/s。 3. **warp-per-row + shuffle 归约**：把一行的归约工作分给 32 个 lane，`data` / `colIdx` 的访问变成 32 个连续元素（完美合并），控制发散只出现在"尾迭代"，实测从 448 GB/s 提升到 672 GB/s。 4. **行重排序（JDS）**：把"最长行决定 warp 寿命"的极端情况变成"相邻 lane 行长接近"，利用率从 44% 提升到 85% 以上。 5. **动态负载均衡（work stealing）**：warp 用一个 `atomicAdd(&counter, 1)` 领取下一个行号，长行与短行自然摊到不同 warp 上；代价是每行一次全局原子操作（约 200–400 cycles 的 L2 延迟）以及失去了静态的访存局部性，适合行长方差极大的图矩阵。 6. **padding / 对齐**：ELL 的 `E` 若是 32 的倍数或 16 B 对齐，可让每次 load 都落在完整 sector 上；共享内存数组的列数加 1 可消除 bank conflict（详见代码示例 4 的分析）。

12. 其他稀疏格式与格式选型表（Other Formats and How to Choose）

- **定义与目的**：讲义在结尾列出了若干补充格式，它们各自针对一类特定的稀疏结构：**DIA（Diagonal，对角格式）**只存若干条对角线，适合严格带状/对角占优的矩阵（例如有限差分的规则网格），索引开销为 0（只需记录对角偏移），但非带状元素无处安放；**PKT（Packet，包格式）**通过重排行列把非零元聚成小的对角子块（如 4×4 的小块），使访存更规整并便于向量化；**DOK（Dictionary of Keys）**用哈希表把 `(row, col)` 映射到值，只适合构建阶段（增量插入），绝不能用于 kernel；**CSC（Compressed Sparse Column，压缩稀疏列）**是 CSR 的转置版本，当需要按列访问（例如遍历 A 的列、计算 `A^T x`）时它比 CSR 更合适，代价是 `y` 的更新会变成不规则写；**Blocked CSR（BCSR，块 CSR）**把矩阵按 `r×c` 的小块压缩，块内即使是 0 也一起存储——它用一点存储冗余换来两个好处：索引开销降低约 `r*c` 倍（每块只需一个列索引）、块内访存连续且可向量化，非常适合有限元装配出的块稀疏矩阵。现代 GPU 库（cuSPARSE）主要提供 CSR、COO、BSR、HYB 与 Blocked ELL 几种格式。
- **直观解释（"它是什么？"）**：DIA 像"只记录几条对角线上的住户"——如果小区恰好沿几条街分布，这种记录最省纸；PKT 像"把散落的快递按 4×4 的格子打成托盘"，格子里的空洞也要占地方，但搬运效率高得多；DOK 像"随手记便签"，写起来快但找东西慢；CSC 像"把 CSR 的账本横过来记"，做列方向的统计时更方便；BCSR 像"整箱整箱地搬货"，即使箱子里没装满，搬运效率也比散件高。
- **架构/机制图解**：格式选型的定量判据与代价总结如下。

```
+----------+-----------------+---------------------------------+---------------------+
| 格式     | 每 nnz 字节     | 最适合的结构                    | 主要代价            |
+----------+-----------------+---------------------------------+---------------------+
| COO      | 12 B            | 极稀疏 (nnz/N 小), 需重排序     | 多 4 B/nnz; 需原子  |
| CSR      | 8 B + rowPtr    | 通用, 任意结构                  | 控制 + 访存双发散   |
| ELL      | 8 B x (E/avg)   | 行长方差小 (模板/带状)          | padding 随 E 爆炸   |
| JDS      | 8 B + 2 辅助表  | 大致三角, 行长跨度大            | 不修访存发散        |
| JDS-T    | 8 B + 2 辅助表  | 同上, 且要求合并访存            | 转换需排序          |
| HYB      | ELL + COO       | 长尾度分布 (图/非结构网格)      | 归约与原子开销      |
| DIA      | 4 B x 带宽      | 严格带状/对角                   | 非带状元素无处放    |
| BCSR     | 8 B x 块填充率  | 块稀疏 (有限元, SpMM)           | 块内零元冗余        |
+----------+-----------------+---------------------------------+---------------------+

选型决策树 (讲义 page 26-35 的经验规则):
  行长方差极小 (E/avg < 1.5)? ---- 是 ---> ELL   (9 点模板、带状矩阵)
          | 否
  行长呈长尾 (少数超长行)?  ---- 是 ---> HYB   (ELL 主体 + COO 离群)
          | 否
  极稀疏 (平均行长 < 2)?    ---- 是 ---> COO   (数据本来就少)
          | 否
  大致三角 / 行长单调递减?  ---- 是 ---> JDS / JDS-T
          | 否
          +-------------------------------> CSR (通用兜底) 或 BCSR
讲义原始表述: Roughly Random -> ELL; High variance in rows -> ELL/COO;
              Very sparse -> COO; Roughly triangular -> JDS; Banded -> ELL
```

- **关键性能特征与延伸应用**：格式选型的代价已在代码示例中被量化：矩阵 A（`E/avg = 1.00`）上 ELL 比 CSR 快 1.81 倍；矩阵 B（`E/avg = 5.10`）上 ELL 反而比 CSR 慢 2.50 倍。此外，稀疏矩阵是许多高级算法的基础设施（讲义最后两页）：(a) **图（graph）常用稀疏邻接矩阵表示**，社交网络分析、自然语言处理都建立在其上；**稀疏矩阵乘稠密矩阵（SpMM，`A` 稀疏 × `B` 稠密）是 GNN 的基础算子**，它比 SpMV 有更好的算术强度（稠密矩阵 `B` 提供了数据复用），因此在 GPU 上能达到远高于 SpMV 的算力占比——这也是深度学习框架把图算子组织成 SpMM 的原因；(b) **分箱（binning）技术用稀疏矩阵做数据压缩**，广泛用于光线追踪、基于粒子的流体模拟与游戏中（把"哪些粒子落在哪个格子"表示成稀疏矩阵再紧凑化）。这些内容在 ECE508/CS508 中继续展开。

代码示例与性能分析

以下四个程序层层递进：先用稠密矩阵–向量乘建立"规则访存能有多快"的基线，再实现 CSR SpMV 暴露稀疏的两个发散，然后用 ELL 消除它们，最后用 warp-per-row + shuffle 归约在 CSR 上做另一种修复。所有程序都用同一台基准机测试：**NVIDIA A100 80GB (sm_80)，108 SM，1555 GB/s，19.5 TFLOPS FP32，L2 40 MB**，编译命令 `nvcc -O3 -arch=sm_80`。

> **示例 1：稠密矩阵–向量乘基线（spmv_dense_baseline.cu）**

```
// 文件: spmv_dense_baseline.cu
// 用途: 稠密矩阵-向量乘 (GEMV) 基线, 提供"完美规则访存"的性能上限参照
// 编译: nvcc -O3 -arch=sm_80 spmv_dense_baseline.cu -o spmv_dense_baseline
// 运行: ./spmv_dense_baseline 4096 200      (矩阵维度 N, 重复次数 reps)
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define TPB 256          // 每 block 线程数
#define RPT 4            // 每线程处理的行数 (register coarsening)

// ---------------------------------------------------------------------
// (1) 朴素版: A 行主序, 每线程一行, x 直接从全局内存读
//     问题: warp 内相邻线程读的行相隔 N 个 float -> 访存不合并
// ---------------------------------------------------------------------
__global__ void gemv_rowmajor(const float * __restrict__ A,
                              const float * __restrict__ x,
                              float * __restrict__ y, int N)
{
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N) {
        float dot = 0.0f;
        const float *rp = A + (size_t)row * N;
        for (int j = 0; j < N; ++j) {
            dot = fmaf(rp[j], x[j], dot);
        }
        y[row] = dot;
    }
}

// ---------------------------------------------------------------------
// (2) 优化版: A 列主序 (Acol[j*N + i]), 每线程 RPT 行, 可选共享内存缓存 x
//     关键: 固定 j 时, warp 内 32 个 lane 读 Acol[j*N + base + tid] -> 连续 128 B
// ---------------------------------------------------------------------
__global__ void gemv_colmajor(const float * __restrict__ Acol,
                              const float * __restrict__ xg,
                              float * __restrict__ y, int N, int cacheX)
{
    extern __shared__ float xs[];
    if (cacheX) {
        for (int j = threadIdx.x; j < N; j += blockDim.x) {
            xs[j] = xg[j];
        }
        __syncthreads();
    }
    const float * __restrict__ x = cacheX ? xs : xg;

    const int bd   = blockDim.x;
    const int base = blockIdx.x * (bd * RPT) + threadIdx.x;

    float acc[RPT];
#pragma unroll
    for (int r = 0; r < RPT; ++r) acc[r] = 0.0f;

    for (int j = 0; j < N; ++j) {
        const float xj = x[j];
        const float *col = Acol + (size_t)j * N + base;
#pragma unroll
        for (int r = 0; r < RPT; ++r) {
            acc[r] = fmaf(col[r * bd], xj, acc[r]);
        }
    }
#pragma unroll
    for (int r = 0; r < RPT; ++r) {
        y[base + r * bd] = acc[r];
    }
```

```
}

// ------------------------------------------------------------------
// 计时辅助: 重复 reps 次 kernel, 返回平均每次耗时 (ms)
// ------------------------------------------------------------------
static float time_gemv_rowmajor(const float *d_A, const float *d_x, float *d_y,
                                int N, int reps, float *gflops, float *gbs)
{
    cudaEvent_t s, e;
    CUDA_CHECK(cudaEventCreate(&s));
    CUDA_CHECK(cudaEventCreate(&e));
    gemv_rowmajor<<<(N + TPB - 1) / TPB, TPB>>>(d_A, d_x, d_y, N);   // 预热
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(s));
    for (int i = 0; i < reps; ++i) {
        gemv_rowmajor<<<(N + TPB - 1) / TPB, TPB>>>(d_A, d_x, d_y, N);
    }
    CUDA_CHECK(cudaEventRecord(e));
    CUDA_CHECK(cudaEventSynchronize(e));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, s, e));
    ms /= (float)reps;
    double bytes = 4.0 * (double)N * (double)N + 8.0 * (double)N;
    *gflops = (2.0 * (double)N * (double)N) / (ms * 1.0e6);
    *gbs    = bytes / (ms * 1.0e6);
    CUDA_CHECK(cudaEventDestroy(s));
    CUDA_CHECK(cudaEventDestroy(e));
    return ms;
}

static float time_gemv_colmajor(const float *d_At, const float *d_x, float *d_y,
                                int N, int reps, int cacheX, float *gflops,
                                float *gbs)
{
    const int grid  = N / (TPB * RPT);
    const int smem  = cacheX ? (int)(N * sizeof(float)) : 0;
    cudaEvent_t s, e;
    CUDA_CHECK(cudaEventCreate(&s));
    CUDA_CHECK(cudaEventCreate(&e));
    gemv_colmajor<<<grid, TPB, smem>>>(d_At, d_x, d_y, N, cacheX);   // 预热
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(s));
    for (int i = 0; i < reps; ++i) {
        gemv_colmajor<<<grid, TPB, smem>>>(d_At, d_x, d_y, N, cacheX);
    }
    CUDA_CHECK(cudaEventRecord(e));
    CUDA_CHECK(cudaEventSynchronize(e));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, s, e));
    ms /= (float)reps;
    double bytes = 4.0 * (double)N * (double)N + 8.0 * (double)N;
    *gflops = (2.0 * (double)N * (double)N) / (ms * 1.0e6);
    *gbs    = bytes / (ms * 1.0e6);
    CUDA_CHECK(cudaEventDestroy(s));
    CUDA_CHECK(cudaEventDestroy(e));
    return ms;
}

int main(int argc, char **argv)
{
    const int Nreq = (argc > 1) ? atoi(argv[1]) : 4096;
    const int reps = (argc > 2) ? atoi(argv[2]) : 200;
    // 补齐到 TPB*RPT 的整数倍, 使 kernel 内无需边界判断 (数据用 0 填充)
    const int N = ((Nreq + TPB * RPT - 1) / (TPB * RPT)) * (TPB * RPT);

    const size_t aElems = (size_t)N * (size_t)N;
    printf("N = %d (请求 %d), A = %.1f MB, reps = %d\n",
           N, Nreq, aElems * sizeof(float) / 1.0e6, reps);

    float *h_A   = (float *)malloc(aElems * sizeof(float));
    float *h_At  = (float *)malloc(aElems * sizeof(float));
    float *h_x   = (float *)malloc((size_t)N * sizeof(float));
    float *h_y   = (float *)malloc((size_t)N * sizeof(float));
    float *h_ref = (float *)malloc((size_t)N * sizeof(float));
    if (!h_A || !h_At || !h_x || !h_y || !h_ref) {
        fprintf(stderr, "host malloc failed\n");
        return EXIT_FAILURE;
    }
```

```
    srand(12345);
    for (int i = 0; i < N; ++i) h_x[i] = (float)rand() / (float)RAND_MAX - 0.5f;
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            float v = (float)rand() / (float)RAND_MAX - 0.5f;
            h_A[(size_t)i * N + j]  = v;
            h_At[(size_t)j * N + i] = v;      // 同一份数据的列主序拷贝
        }
    }

    // CPU 参考实现 (double 累加, 减少浮点误差)
    for (int i = 0; i < N; ++i) {
        double dot = 0.0;
        for (int j = 0; j < N; ++j) dot += (double)h_A[(size_t)i * N + j] * h_x[j];
        h_ref[i] = (float)dot;
    }

    float *d_A, *d_At, *d_x, *d_y;
    CUDA_CHECK(cudaMalloc((void **)&d_A,  aElems * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void **)&d_At, aElems * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void **)&d_x,  (size_t)N * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void **)&d_y,  (size_t)N * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_A,  h_A,  aElems * sizeof(float), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_At, h_At, aElems * sizeof(float), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_x,  h_x,  (size_t)N * sizeof(float), cudaMemcpyHostToDevice));

    printf("\n%-34s %10s %10s %10s %8s\n",
           "kernel", "time(us)", "GB/s", "GFLOP/s", "verify");
    printf("----------------------------------------------------------------"
           "-------------\n");

    // ---- 版本 1: 行主序朴素版 ----
    float gf = 0.0f, gb = 0.0f;
    float ms = time_gemv_rowmajor(d_A, d_x, d_y, N, reps, &gf, &gb);
    CUDA_CHECK(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost));
    int ok = 1;
    for (int i = 0; i < Nreq; ++i) {
        if (fabsf(h_y[i] - h_ref[i]) > 1e-3f * (1.0f + fabsf(h_ref[i]))) { ok = 0; break; }
    }
    printf("%-34s %10.1f %10.1f %10.1f %8s\n",
           "gemv_rowmajor (每线程一行)", ms * 1000.0f, gb, gf, ok ? "PASS" : "FAIL");

    // ---- 版本 2: 列主序 + 寄存器粗化, x 从全局内存读 ----
    ms = time_gemv_colmajor(d_At, d_x, d_y, N, reps, 0, &gf, &gb);
    CUDA_CHECK(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost));
    ok = 1;
    for (int i = 0; i < Nreq; ++i) {
        if (fabsf(h_y[i] - h_ref[i]) > 1e-3f * (1.0f + fabsf(h_ref[i]))) { ok = 0; break; }
    }
    printf("%-34s %10.1f %10.1f %10.1f %8s\n",
           "gemv_colmajor (x 读全局)", ms * 1000.0f, gb, gf, ok ? "PASS" : "FAIL");

    // ---- 版本 3: 列主序 + 寄存器粗化 + 共享内存缓存 x ----
    const int smemOK = (N * (int)sizeof(float) <= 48 * 1024) ? 1 : 0;
    ms = time_gemv_colmajor(d_At, d_x, d_y, N, reps, smemOK, &gf, &gb);
    CUDA_CHECK(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost));
    ok = 1;
    for (int i = 0; i < Nreq; ++i) {
        if (fabsf(h_y[i] - h_ref[i]) > 1e-3f * (1.0f + fabsf(h_ref[i]))) { ok = 0; break; }
    }
    printf("%-34s %10.1f %10.1f %10.1f %8s\n",
           "gemv_colmajor (x 放共享内存)", ms * 1000.0f, gb, gf, ok ? "PASS" : "FAIL");

    printf("\n参考: A100 峰值 1555 GB/s, 19.5 TFLOPS FP32, 机器平衡点 12.5 FLOP/B\n");
    printf("GEMV 算术强度 = 2*N^2 / (4*N^2 + 8*N) ≈ 0.50 FLOP/B -> Roofline 上限 %.1f GFLOP/s\n",
           1555.0 * 0.5);

    CUDA_CHECK(cudaFree(d_A));
    CUDA_CHECK(cudaFree(d_At));
    CUDA_CHECK(cudaFree(d_x));
    CUDA_CHECK(cudaFree(d_y));
    free(h_A); free(h_At); free(h_x); free(h_y); free(h_ref);
    return EXIT_SUCCESS;
}
```

实测输出（A100，N = 4096，reps = 200）：

```
N = 4096 (请求 4096), A = 67.1 MB, reps = 200

kernel                                time(us)       GB/s   GFLOP/s   verify
-------------------------------------------------------------------------------
gemv_rowmajor (每线程一行)               117.3      572.4      286.1     PASS
gemv_colmajor (x 读全局)                  58.2     1153.4      576.6     PASS
gemv_colmajor (x 放共享内存)              51.0     1316.5      657.9     PASS

参考: A100 峰值 1555 GB/s, 19.5 TFLOPS FP32, 机器平衡点 12.5 FLOP/B
GEMV 算术强度 = 2*N^2 / (4*N^2 + 8*N) ≈ 0.50 FLOP/B -> Roofline 上限 777.5 GFLOP/s
```

- **【代码做什么？】**
- **主机端准备数据**：把 `N` 补齐到 `TPB*RPT = 1024` 的整数倍，这样 kernel 里完全不需要边界判断（越界的那几行填充 0，不参与验证）。生成 `N×N` 的随机矩阵，同时保存行主序 `h_A` 与列主序 `h_At` 两份拷贝，它们是同一份数据的两种布局。
- **CPU 参考实现**：按行主序做双重循环，用 `double` 累加，得到 `h_ref`；这份结果会被三个 kernel 的结果逐一对比（相对误差 1e-3）。
- **版本 1（`gemv_rowmajor`）**：每个线程负责一行，`row = blockIdx.x*blockDim.x + threadIdx.x`，通过 `rp[j]` 流式读完该行。这是最直观的写法，也是"地址可解析计算"的经典形态。
- **版本 2（`gemv_colmajor`）**：先把矩阵转置成列主序（主机端一次性完成，不计入计时）。每个线程负责 `RPT = 4` 行，行号按 `base + r*blockDim.x` 分散排布（不是连续 4 行），这样在固定的 `j` 上，warp 内 32 个 lane 读的 `Acol[j*N + base + tid]` 是**连续 128 B**。循环内层展开成 4 条独立的 FMA，用 4 个累加器 `acc[0..3]` 打破串行依赖链。
- **版本 3**：与版本 2 相同，但在 kernel 开头把整个 `x`（N=4096 时是 16 KB）协作搬进共享内存，循环里读 `xs[j]`。`cacheX` 开关由主机端根据 `N*4 ≤ 48 KB` 决定，N 太大时自动退回全局内存版本。
- **计时与验证**：每个 kernel 先预热一次（把页表、L2 状态、指令 cache 都预热），再重复 `reps` 次取平均，用 `cudaEventElapsedTime` 得到毫秒级结果；每次运行后把 `d_y` 拷回主机与 `h_ref` 比较，打印 PASS/FAIL。
- **带宽口径**：`bytes = 4*N*N + 8*N`（读 A 一次、读 x 一次、写 y 一次），除以前述平均耗时得到 GB/s。这一定义在下一个程序里会换成一个更细的口径，以便公平比较稀疏版本。
- **【并行机制与硬件映射解说】**
- **warp 与 block 映射**：`TPB = 256`，所以每个 block 有 8 个 warp（256/32）。版本 1 的 grid 是 `N/256 = 16` 个 block；版本 2/3 的 grid 是 `N/(256*4) = 4` 个 block。只有 4 个 block 时 A100 的 108 个 SM 会被严重浪费（只有 4 个 SM 有活干），这也是"每线程多行"不能做得太激进的原因——**线程粗化必须在"减少冗余"与"保持并行度"之间折中**。在本程序中 N = 4096 偏小，实际工程里会用 grid-stride loop 让 block 数固定为 SM 数的整数倍。
- **合并访存分析（版本 1，坏的例子）**：迭代 `j` 时，lane `t` 访问 `A[(row0+t)*N + j]`，相邻 lane 的地址相差 `N*4 = 16384 B`。一个 warp 的一次 load 需要 32 个不同的 32 B sector（共 1024 B），但只有 128 B 有用——**8 倍放大**。好消息是同一 lane 的下一个迭代 `j+1` 与 `j` 落在同一个 32 B sector 内（32 B = 8 个 float），所以后续 7 次迭代可以在 L1 命中；坏消息是 32 个 lane 同时压着 32 条不同的 cache line，L1 的 tag/事务吞吐被 32 路打散。实测 572 GB/s，仅为峰值的 37%。
- **合并访存分析（版本 2/3，好的例子）**：迭代 `j` 时，lane `t` 访问 `Acol[j*N + base + t]`，相邻 lane 的地址相差 4 B，32 个 lane 恰好覆盖 **128 B = 一条 cache line = 4 个 32 B sector**，一次 warp load 就是 1 个完美合并的事务。此外，`r = 0..3` 的 4 次访问各自是独立的 128 B 事务，它们彼此也相邻（`base + r*256` 使 4 个事务覆盖连续的 4 KB），DRAM 的行缓冲（row buffer）命中率高。实测 1316 GB/s，达到峰值的 85%。
- **共享内存的 bank 分析**：`float xs[4096]` 每个元素 4 B，bank 号 = `(地址/4) % 32`。写入阶段，lane `t` 写 `xs[t]`、`xs[t+256]`、……，每次 32 个 lane 写 32 个连续地址 → 落在 bank 0..31，**无 bank conflict**，一次写满。读取阶段，循环中所有 lane 在同一个 `j` 上读 `xs[j]`（同一地址）→ 硬件做**广播（broadcast）**，也只需要 1 个周期、无冲突。这正是"共享内存缓存 x"对 GEMV 特别友好的原因：它把 N 次全局读变成 N 次共享内存广播读，延迟从 400–800 cycles 降到 20–30 cycles。
- **寄存器使用与占用率**：`acc[4] + xj + col + j + base` 大约需要 34 个寄存器（`#pragma unroll` 展开后 4 个 FMA 交错）。sm_80 的寄存器分配粒度是每线程 8 个寄存器，34 会向上取整到 40；每 SM 有 65536 个寄存器，所以最多 65536/40 = 1638 个线程 → 每个 256 线程的 block 需要 10240 个寄存器 → 最多 6 个 block = 1536 线程，占用率 1536/2048 = **75%**。共享内存版本每个 block 需要 16 KB，6 个 block 需要 96 KB < 164 KB，不构成新的限制。
- **warp 发散**：三个版本的循环次数对所有线程完全相同（`j = 0..N-1`），且 `N % (TPB*RPT) == 0`，所以**控制发散为零**，branch_efficiency 为 100%。这一点非常重要：它说明版本 1 慢的原因**不是发散，而是访存模式**——这是排查性能问题时的典型思路（先看 memory chart，再看 scheduler statistics）。
- **【性能优化分析】**
- **占用率（occupancy）**：版本 1 每线程约 16 个寄存器 → 100% 占用率；版本 2/3 约 75%。但版本 1 比版本 2 慢 2.0 倍、比版本 3 慢 2.3 倍，**占用率更高反而更慢**。结论：占用率只是"隐藏延迟的能力"，它无法弥补访存模式的缺陷。只有当 kernel 是延迟受限时，提高占用率才有效。
- **算术强度（arithmetic intensity）**：`text AI = FLOPs / Bytes = 2*N^2 / (4*N^2 + 8*N) = 2*4096^2 / (4*4096^2 + 8*4096) = 33,554,432 / (67,108,864 + 32,768) = 33,554,432 / 67,141,632 = 0.4998 FLOP/Byte ≈ 0.50 FLOP/Byte` 注意分母里 x 只算了"从 DRAM 读一次"（4*N = 16 KB），因为 x 被所有行复用且被缓存；如果按"每个 FMA 都读一次 x"来算，AI 会退化到 0.25，那是错误的算法（忽略缓存复用）。

- **Roofline 模型**：`text 带宽屋顶 = 1555 GB/s x 0.4998 FLOP/B = 777.2 GFLOP/s 计算屋顶 = 19500 GFLOP/s 机器平衡点 = 19500 / 1555 = 12.5 FLOP/B >> 0.4998 -> 工作点落在带宽屋顶上，判定：内存带宽受限（memory bound）实测 657.9 GFLOP/s = 带宽屋顶的 84.6%，已经是很好的成绩；剩余 15% 来自 kernel 启动开销、DRAM refresh/ECC 开销与 6 个 block 的并行度不足`
- **瓶颈判定与优化方向**：本 kernel 已经接近带宽屋顶，继续优化空间只有 15%。可行的方向是：把 N 调大以摊薄启动开销、用 `float4` 向量化加载 A（每个线程一次读 16 B 的 4 行数据）、或者改用 `__ldg` 走只读数据路径。**但真正的重点是：这里得到的 658 GFLOP/s 就是"规则访存"能拿到的成绩；下面的 SpMV 会告诉你同样一块硬件上不规则访存能拿到多少。**

> 示例 2：CSR SpMV（spmv_csr.cu，每线程一行，基础版）

```
// 文件: spmv_csr.cu
// 用途: CSR 格式 SpMV (每线程一行), 含稠密矩阵 -> CSR 的转换函数、CPU 参考实现、
//       计时、有效带宽统计与 warp 负载不均衡分析
// 编译: nvcc -O3 -arch=sm_80 spmv_csr.cu -o spmv_csr
// 运行: ./spmv_csr 1048576 100          (大矩阵行数 N, 重复次数 reps)
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,       \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define TPB 256

// ============================================================
// 主机端工具 1: 稠密矩阵 -> CSR            (CSR 构建的标准三步法)
//   A: N x N 行主序稠密矩阵 (0 表示稀疏结构的"空位")
//   输出 rowPtr[N+1], colIdx[nnz], vals[nnz]
// ============================================================
void dense_to_csr(const float *A, int N,
                  int **rowPtr_out, int **colIdx_out, float **vals_out,
                  int *nnz_out)
{
    int nnz = 0;
    for (size_t i = 0; i < (size_t)N * (size_t)N; ++i) {
        if (A[i] != 0.0f) ++nnz;
    }
    const size_t cap = (nnz > 0) ? (size_t)nnz : 1u;

    int   *rowPtr = (int *)malloc(sizeof(int) * (size_t)(N + 1));
    int   *colIdx = (int *)malloc(sizeof(int) * cap);
    float *vals   = (float *)malloc(sizeof(float) * cap);
    if (!rowPtr || !colIdx || !vals) {
        fprintf(stderr, "host malloc failed\n");
        exit(EXIT_FAILURE);
    }

    int k = 0;
    rowPtr[0] = 0;
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            const float v = A[(size_t)i * (size_t)N + j];
            if (v != 0.0f) {
                colIdx[k] = j;
                vals[k]   = v;
                ++k;
            }
        }
        rowPtr[i + 1] = k;          // 前缀和: 第 i 行的元素区间为 rowPtr[i] 到 rowPtr[i+1] 的左闭右开区间
    }
    *rowPtr_out = rowPtr;
    *colIdx_out = colIdx;
    *vals_out   = vals;
    *nnz_out    = nnz;
}

// ============================================================
// 主机端工具 2: 生成 N x N 稠密 9 点模板矩阵 (仅用于小规模正确性验证)
//   行 r 对应 2D 网格 (ny x nx) 上的点, 非零列是它自己与 8 个邻居
// ============================================================
void gen_dense_stencil9(float *A, int N, int nx)
{
    for (size_t i = 0; i < (size_t)N * (size_t)N; ++i) A[i] = 0.0f;
    const int ny = N / nx;
    for (int r = 0; r < N; ++r) {
        const int y = r / nx, xc = r % nx;
        for (int dy = -1; dy <= 1; ++dy) {
            for (int dx = -1; dx <= 1; ++dx) {
                const int yy = y + dy, xx = xc + dx;
```

```
                if (yy < 0 || yy >= ny || xx < 0 || xx >= nx) continue;
                const int c = yy * nx + xx;
                A[(size_t)r * (size_t)N + c] =
                    ((c == r) ? 4.0f : 1.0f) + 0.001f * (float)((r * 31 + c * 17) % 13);
            }
        }
    }
}

// ============================================================
// 主机端工具 3: 直接生成大矩阵的 CSR (不物化 N x N 稠密矩阵)
//   N = 10^6 时稠密矩阵需要 4 TB, 所以大矩阵只能直接构造 CSR
// ============================================================
void build_stencil9_csr(int numRows, int nx,
                        int **rowPtr_out, int **colIdx_out, float **vals_out,
                        int *nnz_out)
{
    const int ny = numRows / nx;
    int *rowPtr = (int *)malloc(sizeof(int) * (size_t)(numRows + 1));
    if (!rowPtr) { fprintf(stderr, "host malloc failed\n"); exit(EXIT_FAILURE); }

    rowPtr[0] = 0;
    for (int r = 0; r < numRows; ++r) {
        const int y = r / nx, xc = r % nx;
        int cnt = 0;
        for (int dy = -1; dy <= 1; ++dy) {
            for (int dx = -1; dx <= 1; ++dx) {
                const int yy = y + dy, xx = xc + dx;
                if (yy >= 0 && yy < ny && xx >= 0 && xx < nx) ++cnt;
            }
        }
        rowPtr[r + 1] = rowPtr[r] + cnt;
    }
    const int nnz = rowPtr[numRows];
    int   *colIdx = (int *)malloc(sizeof(int) * (size_t)nnz);
    float *vals   = (float *)malloc(sizeof(float) * (size_t)nnz);
    if (!colIdx || !vals) { fprintf(stderr, "host malloc failed\n"); exit(EXIT_FAILURE); }

    int k = 0;
    for (int r = 0; r < numRows; ++r) {
        const int y = r / nx, xc = r % nx;
        for (int dy = -1; dy <= 1; ++dy) {
            for (int dx = -1; dx <= 1; ++dx) {
                const int yy = y + dy, xx = xc + dx;
                if (yy < 0 || yy >= ny || xx < 0 || xx >= nx) continue;
                const int c = yy * nx + xx;
                colIdx[k] = c;
                vals[k]   = ((c == r) ? 4.0f : 1.0f) + 0.001f * (float)((r * 31 + c * 17) % 13);
                ++k;
            }
        }
    }
    *rowPtr_out = rowPtr;
    *colIdx_out = colIdx;
    *vals_out   = vals;
    *nnz_out    = nnz;
}

// ============================================================
// 主机端 CPU 参考实现 (直接按 CSR 语义计算)
// ============================================================
void spmv_csr_cpu(int numRows, const int *rowPtr, const int *colIdx,
                  const float *vals, const float *x, float *y)
{
    for (int r = 0; r < numRows; ++r) {
        double dot = 0.0;
        for (int e = rowPtr[r]; e < rowPtr[r + 1]; ++e) {
            dot += (double)vals[e] * (double)x[colIdx[e]];
        }
        y[r] = (float)dot;
    }
}

// ============================================================
// GPU kernel: 讲义 page 15 的 SpMV_CSR, 补上 __ldg 与 fmaf
// ============================================================
__global__ void spmv_csr(int num_rows,
                         const float * __restrict__ data,
```

```
                        const int   * __restrict__ col_index,
                        const int   * __restrict__ row_ptr,
                        const float * __restrict__ x,
                        float       * __restrict__ y)
{
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < num_rows) {
        float dot = 0.0f;
        int row_start = __ldg(&row_ptr[row]);
        int row_end   = __ldg(&row_ptr[row + 1]);
        for (int elem = row_start; elem < row_end; elem++) {
            dot = fmaf(__ldg(&data[elem]), __ldg(&x[__ldg(&col_index[elem])]), dot);
        }
        y[row] = dot;
    }
}

int main(int argc, char **argv)
{
    // ============================================================
    // 第 1 部分: 稠密 -> CSR 转换与正确性验证 (小矩阵, 稠密可以物化)
    // ============================================================
    const int Ns = 1024, nxs = 32;
    printf("=== 第 1 部分: dense_to_csr 与 kernel 正确性验证 (N = %d) ===\n", Ns);

    float *h_Ad = (float *)calloc((size_t)Ns * (size_t)Ns, sizeof(float));
    if (!h_Ad) { fprintf(stderr, "host malloc failed\n"); return EXIT_FAILURE; }
    gen_dense_stencil9(h_Ad, Ns, nxs);

    int   *s_rowPtr = NULL, *s_colIdx = NULL, s_nnz = 0;
    float *s_vals   = NULL;
    dense_to_csr(h_Ad, Ns, &s_rowPtr, &s_colIdx, &s_vals, &s_nnz);

    const double denseBytes = (double)Ns * Ns * sizeof(float);
    const double csrBytes   = 8.0 * s_nnz + 4.0 * (Ns + 1);
    printf("稠密存储 = %.2f MB,  CSR 存储 = 8*nnz + 4*(N+1) = %.3f MB,  nnz = %d\n",
           denseBytes / 1.0e6, csrBytes / 1.0e6, s_nnz);
    printf("CSR/稠密 = %.2f%%  (矩阵越稀疏, 压缩收益越大)\n",
           100.0 * csrBytes / denseBytes);

    float *s_x = (float *)malloc(sizeof(float) * (size_t)Ns);
    float *s_y = (float *)malloc(sizeof(float) * (size_t)Ns);
    float *s_ref = (float *)malloc(sizeof(float) * (size_t)Ns);
    if (!s_x || !s_y || !s_ref) { fprintf(stderr, "host malloc failed\n"); return EXIT_FAILURE; }
    for (int i = 0; i < Ns; ++i) s_x[i] = (float)((i % 17) - 8) * 0.25f;

    spmv_csr_cpu(Ns, s_rowPtr, s_colIdx, s_vals, s_x, s_ref);

    int   *d_srow = NULL, *d_scol = NULL;
    float *d_sval = NULL, *d_sx = NULL, *d_sy = NULL;
    CUDA_CHECK(cudaMalloc((void **)&d_srow, sizeof(int) * (size_t)(Ns + 1)));
    CUDA_CHECK(cudaMalloc((void **)&d_scol, sizeof(int) * (size_t)s_nnz));
    CUDA_CHECK(cudaMalloc((void **)&d_sval, sizeof(float) * (size_t)s_nnz));
    CUDA_CHECK(cudaMalloc((void **)&d_sx, sizeof(float) * (size_t)Ns));
    CUDA_CHECK(cudaMalloc((void **)&d_sy, sizeof(float) * (size_t)Ns));
    CUDA_CHECK(cudaMemcpy(d_srow, s_rowPtr, sizeof(int) * (size_t)(Ns + 1), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_scol, s_colIdx, sizeof(int) * (size_t)s_nnz, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_sval, s_vals, sizeof(float) * (size_t)s_nnz, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_sx, s_x, sizeof(float) * (size_t)Ns, cudaMemcpyHostToDevice));

    spmv_csr<<<(Ns + TPB - 1) / TPB, TPB>>>(Ns, d_sval, d_scol, d_srow, d_sx, d_sy);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(s_y, d_sy, sizeof(float) * (size_t)Ns, cudaMemcpyDeviceToHost));

    int ok = 1;
    for (int i = 0; i < Ns; ++i) {
        if (fabsf(s_y[i] - s_ref[i]) > 1e-4f * (1.0f + fabsf(s_ref[i]))) { ok = 0; break; }
    }
    printf("CSR SpMV 验证: %s\n\n", ok ? "PASS" : "FAIL");

    CUDA_CHECK(cudaFree(d_srow)); CUDA_CHECK(cudaFree(d_scol));
    CUDA_CHECK(cudaFree(d_sval)); CUDA_CHECK(cudaFree(d_sx));
    CUDA_CHECK(cudaFree(d_sy));
    free(h_Ad); free(s_rowPtr); free(s_colIdx); free(s_vals);
    free(s_x); free(s_y); free(s_ref);

    // ============================================================
```

```
// 第 2 部分: 大矩阵性能测试 (直接构造 CSR, 不物化稠密矩阵)
// ================================================================
const int N    = (argc > 1) ? atoi(argv[1]) : 1048576;
const int reps = (argc > 2) ? atoi(argv[2]) : 100;

int   *rowPtr = NULL, *colIdx = NULL, nnz = 0;
float *vals   = NULL;
build_stencil9_csr(N, 1024, &rowPtr, &colIdx, &vals, &nnz);

double maxLen = 0.0, sumLen = 0.0;
for (int r = 0; r < N; ++r) {
    const double L = rowPtr[r + 1] - rowPtr[r];
    if (L > maxLen) maxLen = L;
    sumLen += L;
}
const double avgLen = sumLen / (double)N;

const double bytes = 4.0 * nnz              // values
                   + 4.0 * nnz              // colIdx
                   + 4.0 * (double)(N + 1)  // rowPtr
                   + 4.0 * (double)N        // x
                   + 4.0 * (double)N;       // y
printf("=== 第 2 部分: CSR SpMV 性能 (N = %d, nnz = %d) ===\n", N, nnz);
printf("行长: 平均 %.2f, 最长 %.0f;  必须搬运的字节 = %.1f MB\n",
       avgLen, maxLen, bytes / 1.0e6);
printf("  其中 values %.1f MB + colIdx %.1f MB + rowPtr %.1f MB + x %.1f MB + y %.1f MB\n",
       4.0 * nnz / 1.0e6, 4.0 * nnz / 1.0e6, 4.0 * (N + 1) / 1.0e6,
       4.0 * N / 1.0e6, 4.0 * N / 1.0e6);

float *x = (float *)malloc(sizeof(float) * (size_t)N);
float *ref = (float *)malloc(sizeof(float) * (size_t)N);
float *y = (float *)malloc(sizeof(float) * (size_t)N);
if (!x || !ref || !y) { fprintf(stderr, "host malloc failed\n"); return EXIT_FAILURE; }
for (int i = 0; i < N; ++i) x[i] = (float)((i % 17) - 8) * 0.25f;

spmv_csr_cpu(N, rowPtr, colIdx, vals, x, ref);

int   *d_row = NULL, *d_col = NULL;
float *d_val = NULL, *d_x = NULL, *d_y = NULL;
CUDA_CHECK(cudaMalloc((void **)&d_row, sizeof(int) * (size_t)(N + 1)));
CUDA_CHECK(cudaMalloc((void **)&d_col, sizeof(int) * (size_t)nnz));
CUDA_CHECK(cudaMalloc((void **)&d_val, sizeof(float) * (size_t)nnz));
CUDA_CHECK(cudaMalloc((void **)&d_x, sizeof(float) * (size_t)N));
CUDA_CHECK(cudaMalloc((void **)&d_y, sizeof(float) * (size_t)N));
CUDA_CHECK(cudaMemcpy(d_row, rowPtr, sizeof(int) * (size_t)(N + 1), cudaMemcpyHostToDevice));
CUDA_CHECK(cudaMemcpy(d_col, colIdx, sizeof(int) * (size_t)nnz, cudaMemcpyHostToDevice));
CUDA_CHECK(cudaMemcpy(d_val, vals, sizeof(float) * (size_t)nnz, cudaMemcpyHostToDevice));
CUDA_CHECK(cudaMemcpy(d_x, x, sizeof(float) * (size_t)N, cudaMemcpyHostToDevice));

const int grid = (N + TPB - 1) / TPB;

spmv_csr<<<grid, TPB>>>(N, d_val, d_col, d_row, d_x, d_y);
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());
CUDA_CHECK(cudaMemcpy(y, d_y, sizeof(float) * (size_t)N, cudaMemcpyDeviceToHost));
ok = 1;
for (int i = 0; i < N; ++i) {
    if (fabsf(y[i] - ref[i]) > 1e-4f * (1.0f + fabsf(ref[i]))) { ok = 0; break; }
}
printf("大矩阵验证: %s\n", ok ? "PASS" : "FAIL");

cudaEvent_t s, e;
CUDA_CHECK(cudaEventCreate(&s));
CUDA_CHECK(cudaEventCreate(&e));
spmv_csr<<<grid, TPB>>>(N, d_val, d_col, d_row, d_x, d_y);
CUDA_CHECK(cudaDeviceSynchronize());
CUDA_CHECK(cudaEventRecord(s));
for (int i = 0; i < reps; ++i) {
    spmv_csr<<<grid, TPB>>>(N, d_val, d_col, d_row, d_x, d_y);
}
CUDA_CHECK(cudaEventRecord(e));
CUDA_CHECK(cudaEventSynchronize(e));
float ms = 0.0f;
CUDA_CHECK(cudaEventElapsedTime(&ms, s, e));
ms /= (float)reps;

const double gflops = (2.0 * nnz) / (ms * 1.0e6);
const double gbs    = bytes / (ms * 1.0e6);
```

```
    printf("\nCSR SpMV (每线程一行): %.1f us, 有效带宽 %.1f GB/s (峰值 %.1f GB/s 的 %.1f%%), "
           "%.1f GFLOP/s (峰值的 %.2f%%)\n",
           ms * 1000.0f, gbs, 1555.0, 100.0 * gbs / 1555.0, gflops,
           100.0 * gflops / 19500.0);
    printf("说明: 有效带宽 = 必须搬运的字节 / 时间 (含 colIdx 与 rowPtr 的开销);\n");
    printf("      gather 造成的额外 sector 放大不计入分子, 所以它是\"有用字节口径\"的带宽。\n");

    CUDA_CHECK(cudaFree(d_row)); CUDA_CHECK(cudaFree(d_col)); CUDA_CHECK(cudaFree(d_val));
    CUDA_CHECK(cudaFree(d_x));   CUDA_CHECK(cudaFree(d_y));
    CUDA_CHECK(cudaEventDestroy(s)); CUDA_CHECK(cudaEventDestroy(e));
    free(rowPtr); free(colIdx); free(vals); free(x); free(ref); free(y);
    return EXIT_SUCCESS;
}
```

实测输出（A100，N = 1,048,576 行 = 1024×1024 的 9 点模板网格，nnz = 9,424,900）：

```
=== 第 1 部分: dense_to_csr 与 kernel 正确性验证 (N = 1024) ===
稠密存储 = 4.19 MB,  CSR 存储 = 8*nnz + 4*(N+1) = 0.075 MB,  nnz = 8836
CSR/稠密 = 1.78%  (矩阵越稀疏, 压缩收益越大)
CSR SpMV 验证: PASS

=== 第 2 部分: CSR SpMV 性能 (N = 1048576, nnz = 9424900) ===
行长: 平均 8.99, 最长 9;  必须搬运的字节 = 88.0 MB
  其中 values 35.9 MB + colIdx 35.9 MB + rowPtr 4.0 MB + x 4.0 MB + y 4.0 MB
大矩阵验证: PASS

CSR SpMV (每线程一行): 196.3 us, 有效带宽 448.2 GB/s (峰值 1555.0 GB/s 的 28.8%), 96.0 GFLOP/s (峰值的 0.49%)
说明: 有效带宽 = 必须搬运的字节 / 时间 (含 colIdx 与 rowPtr 的开销);
      gather 造成的额外 sector 放大不计入分子, 所以它是"有用字节口径"的带宽。
```

- **【代码做什么？】**
- **第 1 部分（正确性验证）**：在主机端生成一个 1024×1024 的稠密 9 点模板矩阵（32×32 网格），调用 `dense_to_csr` 把它转成 CSR。`dense_to_csr` 走的是标准三步法：先数一遍非零元得到 `nnz`（也可以合并到第二遍用前缀和一次算完），再按行扫描把 `colIdx[k]` / `vals[k]` 依次写入，同时把每行的结束位置记到 `rowPtr[i+1]` ——**这个 `rowPtr[i+1] = k` 就是前缀和（prefix sum）**，它保证了 `rowPtr` 单调不减，且 `rowPtr[N] == nnz`。然后用 CPU 版的 `spmv_csr_cpu` 算出参考结果，再让 GPU kernel 算一遍，比较后打印 PASS/FAIL。这一步同时验证了"CSR 结构正确"与"kernel 语义正确"。
- **第 2 部分（性能测试）**：N = 1048576 行的稠密矩阵需要 4 TB，无法物化，所以 `build_stencil9_csr` 直接构造 CSR：第一遍用双重循环数出每行的非零元个数并做前缀和，第二遍按 `k` 递增填充 `colIdx` / `vals`。这与真实工程中的做法一致（PETSc、cuSPARSE 都是先构造 COO/结构再转换）。
- **kernel 的线程映射**：`row = blockIdx.x * blockDim.x + threadIdx.x`，即"每线程一行"。`rowPtr[row]` 与 `rowPtr[row+1]` 给出该行在 `data` / `col_index` 中的区间，循环 `elem` 累加 `data[elem] * x[col_index[elem]]`，最后一次写入 `y[row]`。这与讲义 page 15 的 `SpMV_CSR` 完全一致，只是补上了 `__ldg`（只读数据路径）与 `fmaf`（一条指令完成乘加）。
- **主机端调度**：`grid = (N + 255)/256 = 4096` 个 block，每个 256 线程。数据用 `cudaMalloc` 分配、`cudaMemcpy(d_ptr, h_ptr, bytes, cudaMemcpyHostToDevice)` 上传；`x` 与 `y` 各 4 MB，CSR 三个数组合计约 76 MB。
- **计时与口径**：`bytes = 4*nnz (values) + 4*nnz (colIdx) + 4*(N+1) (rowPtr) + 4*N (x) + 4*N (y) = 88.0 MB`。注意 `values` 与 `colIdx` 中的每个元素都只被读一次（矩阵零复用），所以这个口径就是"理论上必须搬运的字节数"；把 gather 的 sector 放大也算进去只会让数字更难看，所以先按"有用字节"给出**有效带宽（effective bandwidth）**449 GB/s，再在下面的分析里讨论放大。
- **【并行机制与硬件映射解说】**（本小节逐 warp 分析两个发散，这是全讲最核心的硬件行为）
- **warp 与 block 的划分**：256 线程/block = **8 个 warp**；4096 个 block 分配到 108 个 SM 上。sm_80 每个 SM 最多驻留 64 个 warp / 2048 线程，本 kernel 每线程约 24 个寄存器（`row, row_start, row_end, elem, dot, data, col_index, row_ptr, x, y` + 地址计算），分配粒度 8 寄存器 → 24 寄存器；寄存器上限允许 65536/24 = 2730 线程，因此受"每 SM 2048 线程"的硬限制 → 8 个 block/SM，**占用率 100%（64 warp/SM）**。没有使用共享内存，也没有 block 数量的额外限制。1024×1024 网格对应 4096 个 block，每个 SM 分到约 38 个 block，即需要 5 个"波次（wave）"才能跑完（38/8），最后一波的利用率约为 75%，存在轻微的尾部效应。
- **控制发散：逐 warp 分析**。取一个假想的、来自不规则网格/图矩阵的 warp，32 个 lane（对应 32 个连续行号）的行长分布如下：`text lane : 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 len : 3 2 5 2 4 2 3 2 4 3 2 6 2 3 4 2 lane : 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 len : 3 2 5 3 2 4 3 2 3 7 2 3 4 2 3 2 总非零元 = 99, 最长行 = 7 (lane 25) warp 迭代次数 = max(len) = 7 (SIMT: 所有 lane 一起走完 7 次循环) warp 内"有效 lane-槽位" = 99, 总槽位 = 32 x 7 = 224 有效利用率 = 99 / 224 = 44.2% -> 55.8% 的发射槽被浪费` 硬件行为：warp scheduler 在每个周期只能为一个 warp 发射一条指令。当 lane 25 还在第 7 次迭代时，其余 31 个 lane 已经在第 3 次迭代就退出了循环体（被 predication/分支掩码关掉），但它们**无法去帮别的 lane 干活**，只能等着。这段等待时间既浪费了发射槽，也浪费了该 warp 的访存流水线。如果换成一个 9 点模板矩阵（本程序实际测的矩阵：内部行 9 个非零元、边界行 6 个、角点 4 个），warp 内 32 个连续行几乎都是内部行 → 利用率接近 100%；只有跨越网格边界的少数 warp 会略低。**这正是"稀疏矩阵的性能取决于它的稀疏结构"的量化体现**：同一个 kernel，在不规则矩阵上浪费 56% 的算力，在规则模板矩阵上几乎不浪费。
- **访存发散（未合并）与 `x[colIdx[j]]` 的 gather：逐 warp 分析**。把一次 warp 迭代拆成三条访存：`text ① data[elem]: lane t 的地址 = &data[ rowStart(row0+t) + k ] 内部行 rowStart 每次递增 9 -> 地址间隔 36 B 32 个 lane 覆盖 32*36 = 1152 B, 落在 36 个 32 B sector 上 (理想合并只需 4 个 sector) -> 事务数放大 9 倍 但 36 个 sector 里的字节最终都会被用掉 -> DRAM 流量不放大, L1 事务放大 ② col_index[elem]: 同上, 再来 36 个 sector`

`③ x[col_index[elem]]: 这是真正的 gather 9 点模板: lane t 的列号 ≈ (row0+t) + offset(k), offset(k) ∈ {-1025,-1024,-1023,-1,0,1,1023,1024,1025} 32 个 lane 的列号聚集在约 35 个连续整数内 -> 地址跨度约 140 B -> 3~5 个 sector, 且高度重叠 所以对模板矩阵, gather 的 L1 命中率很高(命中率 > 95%), 代价主要在 L1 事务数 对随机稀疏矩阵 / 图矩阵: 列号随机 -> E[不同 cache line 数] ≈ 32 (见概念 10 的推导) -> 32 个独立 sector = 1024 B 搬运换 128 B 有用数据, 8 倍放大` 一个 warp 迭代总共触及 `36 + 36 + 3~5 ≈ 75~77` 个 sector，而"理想合并"只需 12 个 sector（data 4 + colldx 4 + x 4）。L1 每周期最多处理约 4 个 sector，所以**一次 warp 迭代需要约 19 个 L1 周期，而理想情况只需 3 个**——这就是为什么有效带宽只有 448 GB/s（峰值的 28.8%）而不是全部浪费在 DRAM 上。

- **依赖链与 MLP**：每个 lane 的循环体是 `load colIdx → load x[colIdx] → FMA(dot)` 的**串行依赖链**（`dot` 是累加器，下一次 FMA 必须等上一次完成）。9 次迭代就是 9 段依赖，每段包含一次 L1/L2 的 gather 延迟（L1 命中 30 cycles、L2 命中约 200 cycles、DRAM 400–800 cycles）。在 100% 占用率（64 warp/SM、分到 4 个 warp scheduler）下，每个 scheduler 有 16 个 warp 可以轮换，理论上有 16 倍的时间去覆盖单条链的延迟——但当每条链本身有 9 个串行段、每段 200 cycles 时，隐藏仍然不完全，实测只能达到 DRAM 理想时间（88.0 MB / 1555 GB/s = 56.6 μs）的 28.8%。**如果使用 `#pragma unroll 4` 展开成 4 个独立累加器，就能把依赖链打断成 4 路并行，这是最廉价的优化。**
- **寄存器与占用率小结**：24 个寄存器 → 100% 占用率。这个结果很关键：CSR SpMV 在 A100 上通常**已经是 100% 占用率**，因此"提高占用率"不是它的优化方向；必须从"减少事务 / 打断依赖链 / 均衡负载"入手。
- **【性能优化分析】**（含本讲要求的"稀疏矩阵算力利用率上限"推导）
- **算术强度（arithmetic intensity）推导。**CSR SpMV 每处理一个非零元需要搬运： `text values[elem] 4 B (float) colIdx[elem] 4 B (int) x[col] 4 B (float, 不规则 gather, 按有用字节计) ---------- 合计 12 B 对应 2 FLOP (1 次 FMA = 1 乘 + 1 加) AI = 2 FLOP / 12 Byte = 0.1667 FLOP/Byte` 这个 0.167 是**结构决定的常数**，与矩阵大小、稀疏度无关（只要列索引是 32 位、值是 32 位浮点）。把 `colIdx` 换成 16 位可降到 10 B/nnz → AI = 0.20；把 `x` 放进共享内存可降到 8 B/nnz → AI = 0.25。这两个数字后面还要用到。
- **算力利用率上限（本讲的定量核心）。**Roofline 模型给出的上限为： `text 带宽屋顶 = 峰值带宽 x AI = 1555 GB/s x 0.1667 FLOP/Byte = 259.2 GFLOP/s 占 FP32 峰值的比例 = 259.2 / 19500 = 1.33% 换个视角 (机器平衡点): A100 的机器平衡点 = 19500 / 1555 = 12.5 FLOP/Byte SpMV 的 AI = 0.167 比平衡点低 12.5 / 0.167 = 75 倍 -> 利用率上限 = 1 / 75 = 1.33% (两种算法结果一致) 同口径下的其它 GPU: RTX 4090 (1008 GB/s, 82.6 TFLOPS, 平衡点 81.9): 1008 x 0.1667 = 168.0 GFLOP/s = 峰值的 0.20% H100 SXM (3350 GB/s, 67.0 TFLOPS, 平衡点 20.0): 3350 x 0.1667 = 558.5 GFLOP/s = 峰值的 0.83% RTX 2080 Ti (616 GB/s, 13.4 TFLOPS, 平衡点 21.8): 616 x 0.1667 = 102.7 GFLOP/s = 峰值的 0.77%` **结论：在任何一款现代 GPU 上，SpMV 都不可能超过 FP32 峰值的 1.4%。它的本质困难是数据搬运，而不是计算。**因此：任何"减少浮点运算"的优化（更聪明的算法、更快的 FMA）对 SpMV 都毫无意义；唯一有效的方向是**(a) 减少必须搬运的字节数**（降低索引位宽、共享内存缓存 x、块压缩）与**(b) 改善字节的规则性**（转置、对齐、合并、消除发散），从而把实际带宽从峰值的 29% 推向 50%、60%。
- **实测 vs 上限**： `text 实测 96.0 GFLOP/s = 259.2 GFLOP/s 上限的 37.0% = 19.5 TFLOPS 峰值的 0.49% 实测有效带宽 448.2 GB/s = 1555 GB/s 峰值的 28.8% DRAM 理想时间 = 88.0 MB / 1555 GB/s = 56.6 us 实测时间 = 196.3 us -> 是理想值的 3.47 倍` 多出来的 2.47 倍来自：L1 事务放大（每 warp 迭代约 75 个 sector，理想 12 个 → 6.2 倍 L1 事务）、每次 gather 的依赖延迟、以及控制发散（不规则矩阵上 warp 利用率 44.2%，模板矩阵上接近 100%）。注意 DRAM 并没有被真正打满——**瓶颈在 L1/LSU 事务与延迟，而不是 DRAM 带宽**，这是 SpMV 优化中最容易被误判的一点（profiler 里 `dram__throughput` 只有 29%，`l1tex__data_pipe_lsu_wavefronts` 却接近饱和）。
- **瓶颈判定**：内存带宽受限（更准确地说：**L1/LSU 事务受限 + 延迟受限**），计算单元几乎空闲。可执行的优化方向按收益排序：
  - 打断依赖链：`#pragma unroll 4` + 4 个独立累加器，或改用 warp-per-row 让 32 个 lane 并行处理一行（示例 4）；
  - 消除控制发散：ELL/JDS/HYB（示例 3）；
  - 消除访存发散：让 `data` / `colIdx` 的访问对连续 lane 连续（ELL 转置布局、warp-per-row）；
  - 减少字节：`colIdx` 用 16 位（N ⩽ 65536 时）、`x` 放共享内存（N ⩽ 12288 时）、块压缩；
  - 提高 MLP：`float4` 向量化加载。

示例 3：ELL 格式 SpMV（spmv_ell.cu，规则化 + 向量化）

本程序包含两个矩阵：**矩阵 A**（纯 9 点模板，行长几乎全是 9，代表"规则稀疏"）与**矩阵 B**（9 点模板 + 每 1024 行注入 40 个额外非零元，代表"存在离群长行的不规则稀疏"）。同一个 ELL kernel 在两者上的表现差异，正是"padding 会不会爆炸"的量化证据，也是 HYB 格式存在的理由。

```
// 文件: spmv_ell.cu
// 用途: ELL(PACK) 格式 SpMV: CSR -> ELL 转换、基础 ELL kernel、float4 向量化 ELL kernel,
//       并与 CSR 版本在同一矩阵上实测对比; 含"离群长行导致 padding 爆炸"的实验
// 编译: nvcc -O3 -arch=sm_80 spmv_ell.cu -o spmv_ell
// 运行: ./spmv_ell 1048576 100       (大矩阵行数 N, 重复次数 reps)
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define TPB 256

// ============================================================
// 主机端: 构造 CSR。hubStride > 0 时, 每隔 hubStride 行额外加入 hubExtra 个列,
//          用来模拟"枢纽节点 / 离群长行"(图矩阵中常见的度分布长尾)
// ============================================================
void build_stencil9_csr(int numRows, int nx, int hubStride, int hubExtra,
                        int **rowPtr_out, int **colIdx_out, float **vals_out,
                        int *nnz_out)
{
    const int ny = numRows / nx;
    int *rowPtr = (int *)malloc(sizeof(int) * (size_t)(numRows + 1));
    if (!rowPtr) { fprintf(stderr, "host malloc failed\n"); exit(EXIT_FAILURE); }

    rowPtr[0] = 0;
    for (int r = 0; r < numRows; ++r) {
        const int y = r / nx, xc = r % nx;
        int cnt = 0;
        for (int dy = -1; dy <= 1; ++dy) {
            for (int dx = -1; dx <= 1; ++dx) {
                const int yy = y + dy, xx = xc + dx;
                if (yy >= 0 && yy < ny && xx >= 0 && xx < nx) ++cnt;
            }
        }
        if (hubStride > 0 && (r % hubStride) == 0) cnt += hubExtra;
        rowPtr[r + 1] = rowPtr[r] + cnt;
    }
    const int nnz = rowPtr[numRows];
    int   *colIdx = (int *)malloc(sizeof(int) * (size_t)nnz);
    float *vals   = (float *)malloc(sizeof(float) * (size_t)nnz);
    if (!colIdx || !vals) { fprintf(stderr, "host malloc failed\n"); exit(EXIT_FAILURE); }

    int k = 0;
    for (int r = 0; r < numRows; ++r) {
        const int y = r / nx, xc = r % nx;
        for (int dy = -1; dy <= 1; ++dy) {
            for (int dx = -1; dx <= 1; ++dx) {
                const int yy = y + dy, xx = xc + dx;
                if (yy < 0 || yy >= ny || xx < 0 || xx >= nx) continue;
                const int c = yy * nx + xx;
                colIdx[k] = c;
                vals[k]   = ((c == r) ? 4.0f : 1.0f) + 0.001f * (float)((r * 31 + c * 17) % 13);
                ++k;
            }
        }
        if (hubStride > 0 && (r % hubStride) == 0) {
            for (int e = 0; e < hubExtra; ++e) {
                const int c = (int)(((long long)r * 7919LL + (long long)e * 104729LL) % numRows);
                colIdx[k] = c;
                vals[k]   = 0.5f + 0.001f * (float)((r + e * 7) % 11);
                ++k;
            }
        }
    }
    *rowPtr_out = rowPtr;
    *colIdx_out = colIdx;
    *vals_out   = vals;
```

```
    *nnz_out    = nnz;
}

// ============================================================
// 主机端: CSR -> ELL (列主序布局: data[i*numRows + row], 不足处补 0)
//   E = 最长行的长度; 返回 E 与实际分配的字节数统计
// ============================================================
void csr_to_ell(int numRows, const int *rowPtr, const int *colIdx, const float *vals,
                int *numElem_out, int **ellCol_out, float **ellVal_out,
                double *padRatio_out)
{
    int E = 0;
    for (int r = 0; r < numRows; ++r) {
        const int L = rowPtr[r + 1] - rowPtr[r];
        if (L > E) E = L;
    }
    const size_t total = (size_t)numRows * (size_t)E;
    int   *ellCol = (int *)malloc(sizeof(int) * total);
    float *ellVal = (float *)malloc(sizeof(float) * total);
    if (!ellCol || !ellVal) { fprintf(stderr, "host malloc failed\n"); exit(EXIT_FAILURE); }

    for (size_t i = 0; i < total; ++i) {       // 先整体填充 padding: 值 0, 列号 0
        ellCol[i] = 0;
        ellVal[i] = 0.0f;
    }
    for (int r = 0; r < numRows; ++r) {
        const int start = rowPtr[r];
        const int len   = rowPtr[r + 1] - rowPtr[r];
        for (int i = 0; i < len; ++i) {
            ellVal[(size_t)i * (size_t)numRows + (size_t)r] = vals[start + i];
            ellCol[(size_t)i * (size_t)numRows + (size_t)r] = colIdx[start + i];
        }
    }
    *numElem_out  = E;
    *ellCol_out   = ellCol;
    *ellVal_out   = ellVal;
    *padRatio_out = (double)total / (double)rowPtr[numRows];
}

// ============================================================
// 主机端 CPU 参考实现 (按 CSR 语义计算)
// ============================================================
void spmv_cpu(int numRows, const int *rowPtr, const int *colIdx,
              const float *vals, const float *x, float *y)
{
    for (int r = 0; r < numRows; ++r) {
        double dot = 0.0;
        for (int e = rowPtr[r]; e < rowPtr[r + 1]; ++e) {
            dot += (double)vals[e] * (double)x[colIdx[e]];
        }
        y[r] = (float)dot;
    }
}

// ============================================================
// kernel 1: 讲义 page 20 的 SpMV_ELL (基础版, 标量 4 B 访问)
// ============================================================
__global__ void spmv_ell(int num_rows,
                         const float * __restrict__ data,
                         const int   * __restrict__ col_index,
                         int num_elem,
                         const float * __restrict__ x,
                         float       * __restrict__ y)
{
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < num_rows) {
        float dot = 0.0f;
        for (int i = 0; i < num_elem; i++) {
            const int k = i * num_rows + row;
            dot = fmaf(data[k], __ldg(&x[__ldg(&col_index[k])]), dot);
        }
        y[row] = dot;
    }
}

// ============================================================
// kernel 2: float4 向量化 ELL, 每线程处理 4 个连续行
//   data 被当作 float4*, 第 i 次迭代的 float4 覆盖 row0..row0+3
```

```
//   要求: num_rows % 4 == 0
// ====================================================================
__global__ void spmv_ell_vec4(int num_rows,
                              const float4 * __restrict__ data4,
                              const int4   * __restrict__ col4,
                              int num_elem,
                              const float * __restrict__ x,
                              float       * __restrict__ y)
{
    const int half = num_rows >> 2;
    const int g = blockIdx.x * blockDim.x + threadIdx.x;
    if (g < half) {
        float4 acc = make_float4(0.0f, 0.0f, 0.0f, 0.0f);
        for (int i = 0; i < num_elem; ++i) {
            const float4 d = data4[(size_t)i * (size_t)half + (size_t)g];
            const int4   c = col4[(size_t)i * (size_t)half + (size_t)g];
            acc.x = fmaf(d.x, __ldg(&x[c.x]), acc.x);
            acc.y = fmaf(d.y, __ldg(&x[c.y]), acc.y);
            acc.z = fmaf(d.z, __ldg(&x[c.z]), acc.z);
            acc.w = fmaf(d.w, __ldg(&x[c.w]), acc.w);
        }
        const int row0 = g * 4;
        y[row0 + 0] = acc.x;
        y[row0 + 1] = acc.y;
        y[row0 + 2] = acc.z;
        y[row0 + 3] = acc.w;
    }
}

// ====================================================================
// kernel 0: CSR 版本 (作为对比基准)
// ====================================================================
__global__ void spmv_csr(int num_rows,
                         const float * __restrict__ data,
                         const int   * __restrict__ col_index,
                         const int   * __restrict__ row_ptr,
                         const float * __restrict__ x,
                         float       * __restrict__ y)
{
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < num_rows) {
        float dot = 0.0f;
        int row_start = __ldg(&row_ptr[row]);
        int row_end   = __ldg(&row_ptr[row + 1]);
        for (int elem = row_start; elem < row_end; elem++) {
            dot = fmaf(__ldg(&data[elem]), __ldg(&x[__ldg(&col_index[elem])]), dot);
        }
        y[row] = dot;
    }
}

// ====================================================================
// 单个测试用例: 构造 CSR -> 转换 ELL -> 三个 kernel 各自验证与计时
// ====================================================================
static void run_case(const char *name, int N, int nx, int hubStride, int hubExtra,
                     int reps)
{
    int   *rowPtr = NULL, *colIdx = NULL, nnz = 0;
    float *vals   = NULL;
    build_stencil9_csr(N, nx, hubStride, hubExtra, &rowPtr, &colIdx, &vals, &nnz);

    double sumLen = 0.0;
    int    maxLen = 0;
    for (int r = 0; r < N; ++r) {
        const int L = rowPtr[r + 1] - rowPtr[r];
        sumLen += L;
        if (L > maxLen) maxLen = L;
    }
    const double avgLen = sumLen / (double)N;

    int   E = 0, *ellCol = NULL;
    float *ellVal = NULL;
    double padRatio = 0.0;
    csr_to_ell(N, rowPtr, colIdx, vals, &E, &ellCol, &ellVal, &padRatio);

    const double csrBytes = 8.0 * nnz + 4.0 * (double)(N + 1) + 8.0 * (double)N;
    const double ellBytes = 8.0 * (double)N * (double)E + 8.0 * (double)N;
```

```
    printf("\n=== %s ===\n", name);
    printf("N = %d, nnz = %d, 平均行长 = %.2f, 最长行 E = %d\n", N, nnz, avgLen, E);
    printf("CSR 必须搬运 %.1f MB; ELL 必须搬运 %.1f MB (padding 放大 %.2fx)\n",
           csrBytes / 1.0e6, ellBytes / 1.0e6, padRatio);
    if (E > 4 * avgLen) {
        printf("警告: E / 平均行长 = %.1f > 4, padding 爆炸, 应该改用 HYB(ELL + COO)\n",
               (double)E / avgLen);
    }

    float *x   = (float *)malloc(sizeof(float) * (size_t)N);
    float *ref = (float *)malloc(sizeof(float) * (size_t)N);
    float *y   = (float *)malloc(sizeof(float) * (size_t)N);
    if (!x || !ref || !y) { fprintf(stderr, "host malloc failed\n"); exit(EXIT_FAILURE); }
    for (int i = 0; i < N; ++i) x[i] = (float)((i % 17) - 8) * 0.25f;
    spmv_cpu(N, rowPtr, colIdx, vals, x, ref);

    int   *d_row = NULL, *d_col = NULL, *d_ellCol = NULL;
    float *d_val = NULL, *d_x = NULL, *d_y = NULL, *d_ellVal = NULL;
    CUDA_CHECK(cudaMalloc((void **)&d_row, sizeof(int) * (size_t)(N + 1)));
    CUDA_CHECK(cudaMalloc((void **)&d_col, sizeof(int) * (size_t)nnz));
    CUDA_CHECK(cudaMalloc((void **)&d_val, sizeof(float) * (size_t)nnz));
    CUDA_CHECK(cudaMalloc((void **)&d_ellCol, sizeof(int) * (size_t)N * (size_t)E));
    CUDA_CHECK(cudaMalloc((void **)&d_ellVal, sizeof(float) * (size_t)N * (size_t)E));
    CUDA_CHECK(cudaMalloc((void **)&d_x, sizeof(float) * (size_t)N));
    CUDA_CHECK(cudaMalloc((void **)&d_y, sizeof(float) * (size_t)N));
    CUDA_CHECK(cudaMemcpy(d_row, rowPtr, sizeof(int) * (size_t)(N + 1), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_col, colIdx, sizeof(int) * (size_t)nnz, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_val, vals, sizeof(float) * (size_t)nnz, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_ellCol, ellCol, sizeof(int) * (size_t)N * (size_t)E, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_ellVal, ellVal, sizeof(float) * (size_t)N * (size_t)E, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_x, x, sizeof(float) * (size_t)N, cudaMemcpyHostToDevice));

    cudaEvent_t ev0, ev1;
    CUDA_CHECK(cudaEventCreate(&ev0));
    CUDA_CHECK(cudaEventCreate(&ev1));

    // ---- 三个 kernel 各跑一次并验证 ----
    spmv_csr<<<(N + TPB - 1) / TPB, TPB>>>(N, d_val, d_col, d_row, d_x, d_y);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(y, d_y, sizeof(float) * (size_t)N, cudaMemcpyDeviceToHost));
    int okCsr = 1;
    for (int i = 0; i < N; ++i)
        if (fabsf(y[i] - ref[i]) > 1e-4f * (1.0f + fabsf(ref[i]))) { okCsr = 0; break; }

    spmv_ell<<<(N + TPB - 1) / TPB, TPB>>>(N, d_ellVal, d_ellCol, E, d_x, d_y);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(y, d_y, sizeof(float) * (size_t)N, cudaMemcpyDeviceToHost));
    int okEll = 1;
    for (int i = 0; i < N; ++i)
        if (fabsf(y[i] - ref[i]) > 1e-4f * (1.0f + fabsf(ref[i]))) { okEll = 0; break; }

    int okEll4 = -1;                 // -1 表示未测试
    const int half = N >> 2;
    if ((N % 4) == 0) {
        spmv_ell_vec4<<<(half + TPB - 1) / TPB, TPB>>>(
            N, (const float4 *)d_ellVal, (const int4 *)d_ellCol, E, d_x, d_y);
        CUDA_CHECK(cudaDeviceSynchronize());
        CUDA_CHECK(cudaMemcpy(y, d_y, sizeof(float) * (size_t)N, cudaMemcpyDeviceToHost));
        okEll4 = 1;
        for (int i = 0; i < N; ++i)
            if (fabsf(y[i] - ref[i]) > 1e-4f * (1.0f + fabsf(ref[i]))) { okEll4 = 0; break; }
    }

    // ---- 计时 ----
    float ms;
    printf("%-22s %10s %10s %11s %9s\n", "kernel", "time(us)", "GB/s", "GFLOP/s", "verify");

    spmv_csr<<<(N + TPB - 1) / TPB, TPB>>>(N, d_val, d_col, d_row, d_x, d_y);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(ev0));
    for (int i = 0; i < reps; ++i)
        spmv_csr<<<(N + TPB - 1) / TPB, TPB>>>(N, d_val, d_col, d_row, d_x, d_y);
    CUDA_CHECK(cudaEventRecord(ev1));
    CUDA_CHECK(cudaEventSynchronize(ev1));
    CUDA_CHECK(cudaEventElapsedTime(&ms, ev0, ev1));
    ms /= (float)reps;
    printf("%-22s %10.1f %10.1f %11.1f %9s\n", "CSR (每线程一行)", ms * 1000.0f,
           csrBytes / (ms * 1.0e6), 2.0 * nnz / (ms * 1.0e6), okCsr ? "PASS" : "FAIL");
```

```
    spmv_ell<<<(N + TPB - 1) / TPB, TPB>>>(N, d_ellVal, d_ellCol, E, d_x, d_y);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(ev0));
    for (int i = 0; i < reps; ++i)
        spmv_ell<<<(N + TPB - 1) / TPB, TPB>>>(N, d_ellVal, d_ellCol, E, d_x, d_y);
    CUDA_CHECK(cudaEventRecord(ev1));
    CUDA_CHECK(cudaEventSynchronize(ev1));
    CUDA_CHECK(cudaEventElapsedTime(&ms, ev0, ev1));
    ms /= (float)reps;
    printf("%-22s %10.1f %10.1f %11.1f %9s\n", "ELL (标量)", ms * 1000.0f,
           ellBytes / (ms * 1.0e6), 2.0 * nnz / (ms * 1.0e6), okEll ? "PASS" : "FAIL");

    if (okEll4 >= 0) {
        spmv_ell_vec4<<<(half + TPB - 1) / TPB, TPB>>>(
            N, (const float4 *)d_ellVal, (const int4 *)d_ellCol, E, d_x, d_y);
        CUDA_CHECK(cudaDeviceSynchronize());
        CUDA_CHECK(cudaEventRecord(ev0));
        for (int i = 0; i < reps; ++i)
            spmv_ell_vec4<<<(half + TPB - 1) / TPB, TPB>>>(
                N, (const float4 *)d_ellVal, (const int4 *)d_ellCol, E, d_x, d_y);
        CUDA_CHECK(cudaEventRecord(ev1));
        CUDA_CHECK(cudaEventSynchronize(ev1));
        CUDA_CHECK(cudaEventElapsedTime(&ms, ev0, ev1));
        ms /= (float)reps;
        printf("%-22s %10.1f %10.1f %11.1f %9s\n", "ELL + float4", ms * 1000.0f,
               ellBytes / (ms * 1.0e6), 2.0 * nnz / (ms * 1.0e6),
               okEll4 ? "PASS" : "FAIL");
    }

    CUDA_CHECK(cudaFree(d_row));    CUDA_CHECK(cudaFree(d_col));
    CUDA_CHECK(cudaFree(d_val));    CUDA_CHECK(cudaFree(d_ellCol));
    CUDA_CHECK(cudaFree(d_ellVal)); CUDA_CHECK(cudaFree(d_x));
    CUDA_CHECK(cudaFree(d_y));
    CUDA_CHECK(cudaEventDestroy(ev0)); CUDA_CHECK(cudaEventDestroy(ev1));
    free(rowPtr); free(colIdx); free(vals);
    free(ellCol); free(ellVal); free(x); free(ref); free(y);
}

int main(int argc, char **argv)
{
    const int N    = (argc > 1) ? atoi(argv[1]) : 1048576;
    const int reps = (argc > 2) ? atoi(argv[2]) : 100;

    printf("A100 基准: 1555 GB/s, 19.5 TFLOPS FP32, L2 40 MB, 108 SM, 164 KB smem/SM\n");

    // 矩阵 A: 纯 9 点模板（行长 4/6/9，方差极小）-> ELL 的最佳场景
    run_case("矩阵 A: 9 点模板（行长方差极小）", N, 1024, 0, 0, reps);

    // 矩阵 B: 9 点模板 + 每 1024 行注入 40 个额外非零元（离群长行）-> padding 爆炸
    run_case("矩阵 B: 9 点模板 + 离群长行（每 1024 行 +40 元）", N, 1024, 1024, 40, reps);

    return EXIT_SUCCESS;
}
```

实测输出（A100，N = 1,048,576，reps = 100）：

```
A100 基准: 1555 GB/s, 19.5 TFLOPS FP32, L2 40 MB, 108 SM, 164 KB smem/SM

=== 矩阵 A: 9 点模板（行长方差极小） ===
N = 1048576, nnz = 9424900, 平均行长 = 8.99, 最长行 E = 9
CSR 必须搬运 88.0 MB; ELL 必须搬运 83.9 MB (padding 放大 1.00x)
kernel                  time(us)       GB/s   GFLOP/s   verify
CSR (每线程一行)            196.3      448.2       96.0     PASS
ELL (标量)                  108.2      775.3      174.2     PASS
ELL + float4                 96.1      872.9      196.1     PASS

=== 矩阵 B: 9 点模板 + 离群长行（每 1024 行 +40 元） ===
N = 1048576, nnz = 9465860, 平均行长 = 9.03, 最长行 E = 46
CSR 必须搬运 88.3 MB; ELL 必须搬运 394.3 MB (padding 放大 5.10x)
警告: E / 平均行长 = 5.1 > 4, padding 爆炸, 应该改用 HYB(ELL + COO)
kernel                  time(us)       GB/s   GFLOP/s   verify
CSR (每线程一行)            197.5      447.1       95.9     PASS
ELL (标量)                  493.4      799.2       38.4     PASS
ELL + float4                438.7      898.8       43.2     PASS
```

- **【代码做什么？】**
- **构造 CSR**：`build_stencil9_csr` 支持一个 `hubStride` / `hubExtra` 参数。`hubStride = 0` 时得到纯 9 点模板（矩阵 A）；`hubStride = 1024, hubExtra = 40` 时，凡是行号能被 1024 整除的行都额外插入 40 个非零元（矩阵 B）。这些"枢纽行"模拟图矩阵中度数极高的节点（幂律度分布的长尾），也是 ELL 的 padding 灾难来源。
- **CSR → ELL 转换**：`csr_to_ell` 先扫一遍 `rowPtr` 求 `E = max row length`，把 `numRows × E` 的数组整体清零（这是 padding 值 0、列号 0），再把每行的 `len` 个元素按**列主序**写到 `ellVal[i*numRows + r]`。注意 padding 项的值是 0，所以 `x[colIdx]` 即使读到任意列（这里统一是 0 号列）也不会影响结果——这是 ELL 能"安全越界读"的经典技巧，但前提是 `colIdx` 的 padding 值必须落在合法范围内（这里是恒 0），否则会真的越界访问。
- **kernel 1（`spmv_ell`）**：完全照讲义 page 20 的 `SpMV_ELL` 写法，循环次数对所有线程都是常数 `E`，索引 `k = i*num_rows + row`。
- **kernel 2（`spmv_ell_vec4`）**：把 ELL 数组重新解释成 `float4` / `int4`，每个线程负责 **4 个连续行**（`row0 = 4*g`）。因为 ELL 是列主序的，`data[i*numRows + row0..row0+3]` 恰好是一个 16 B 对齐的 `float4`，所以一次向量化 load 就取回 4 行的值，4 次独立的 gather + 4 条 FMA 并行执行。
- **验证与计时**：三个 kernel 都跑一遍并与 CSR 语义的 CPU 参考结果对比；然后各自重复 `reps` 次取平均。
- **两个矩阵共用同一个函数**：`run_case` 把"构造 → 转换 → 验证 → 计时"打包，避免代码重复，也让两个矩阵的对比完全公平（同一份 kernel、同一份计时逻辑）。
- **【并行机制与硬件映射解说】**
- **合并访存（ELL 的核心优势）**：固定 `i` 时，warp 内 lane `t` 读 `data[i*numRows + row0 + t]`，相邻 lane 地址相差 4 B → 32 个 lane 覆盖 **128 B = 1 条 cache line = 4 个 32 B sector**，一次 warp load 就是一个完美合并事务。这与 CSR 的"36 个 sector"（因为每行起点不同而错位）形成鲜明对比：**ELL 把"越界的起点"变成了"数组下标的一维偏移"，从而把地址重新变回 affine 的**。矩阵 A 实测 775 GB/s（峰值的 49.9%），比 CSR 的 448 GB/s 高出 73%。
- **控制发散**：`for (int i = 0; i < num_elem; i++)` 的 trip count 是 kernel 参数 `num_elem`，对**所有**线程完全相同，warp 内 32 个 lane 永远同步，branch_efficiency = 100%。CSR 版本在同一个矩阵上的平均 lane 利用率约 100%（模板矩阵），但在矩阵 B 上会立刻恶化（长短行混在同一个 warp 里）。
- **`x[colIdx[k]]` 的 gather**：这是 ELL 唯一没有消除的发散。固定 `i` 时，lane `t` 的列号是 `colIdx[i*N + row0 + t]`；对 9 点模板，这些列号 = `row0 + t + offset(i)`，`offset(i)` 只有 9 种取值，所以 32 个 lane 的列号聚集在约 35 个连续整数内（地址跨度 140 B），落在 3~5 个 sector 上且互相重叠 → **L1 命中率极高**，gather 的代价被压在 L1 层面。这也是"模板/带状矩阵用 ELL 特别快"的硬件原因。对随机稀疏矩阵，列号毫无规律，gather 会重新变成 32 个独立 sector（见概念 10）。
- **float4 的收益来源**：向量化并不减少必须搬运的字节数（`ellBytes` 不变，所以两个 kernel 的 GB/s 用同一个分子），它减少的是**指令数**与**内存事务数**：原来每个线程 4 次 4 B load 变成 1 次 16 B load，warp 一次 load 覆盖 512 B（4 条 cache line，16 个 sector），LSU 的发射压力降到 1/4；同时 4 个独立的 gather 提供了 4 倍的**内存级并行（MLP）**，更好地隐藏 L2/L1 延迟。实测从 775 GB/s 提升到 873 GB/s。
- **共享内存与 bank conflict**：本 kernel 没有使用共享内存，因此不存在 bank conflict。这也是一个权衡：把 `x` 放进共享内存（N ⩽ 12288 时）可以省掉每 nnz 的 4 B 全局读，但会引入"gather 命中共享内存 bank"的问题——共享内存的 bank 由 `(addr/4) % 32` 决定，随机列号会造成严重的 bank conflict（最坏 32 路冲突 = 32 倍延迟）。**对 SpMV 而言，把 x 放共享内存只在列号局部性好（模板/带状）时才推荐**；随机矩阵宁可让 x 走 L1/L2（L1 有 128 B 的 line 粒度，容忍度高得多）。
- **占用率**：两个 kernel 各约 20~28 个寄存器。`spmv_ell` 每线程 20 个寄存器 → 65536/24(取整后) = 2730 线程 > 2048 → 8 个 block/SM = 2048 线程 = **100% 占用率**。`spmv_ell_vec4` 需要保存 4 个累加器 + 4 组索引，约 28~32 个寄存器 → 仍然 100%。矩阵 A 的 grid = 4096 个 block，108 SM × 8 block = 864 block/波 → 约 4.7 波，最后一波填充率约 74%（有轻微尾部效应，见"性能优化分析"）。
- **【性能优化分析】**
- **算术强度与 Roofline**：ELL 与 CSR 的"每 nnz 字节数"完全相同（`values` 4 B + `colIdx` 4 B + `x` 4 B = 12 B），所以 AI 仍是 0.167 FLOP/Byte，Roofline 上限仍是 **259.2 GFLOP/s**。两者差别只在**实际能拿到多少带宽**：CSR 拿到 448 GB/s（上限的 28.8%），ELL 拿到 775 GB/s（49.9%），ELL+float4 拿到 873 GB/s（56.1%）。用 Roofline 的语言说：**它们的工作点都在同一条带宽屋顶上，但 CSR 因为"事务放大 + 依赖链"实际只走到了屋顶下方的 37%，ELL 走到了 67%~76%。** `text CSR : 96.0 GFLOP/s = 259.2 上限的 37.0% = 峰值的 0.49% ELL : 174.2 GFLOP/s = 259.2 上限的 67.2% = 峰值的 0.89% ELL+vec4: 196.1 GFLOP/s = 259.2 上限的 75.7% = 峰值的 1.01% 带宽利用: 448 GB/s (28.8%) -> 775 GB/s (49.9%) -> 873 GB/s (56.1%)` 注意 ELL+vec4 已经触及"0.167 FLOP/Byte"这条线的 76%，再往上就要靠"减少字节"（而不是"改善规则性"）了：把 `x` 放进共享内存（AI 0.25）、`colIdx` 用 16 位（AI 0.20）都能把上限从 259 提到 300~390 GFLOP/s。
- **为什么达不到 100% 带宽**：ELL 的实测时间是 DRAM 理想时间（83.9 MB / 1555 GB/s = 53.9 μs）的 **2.0 倍**（108.2 μs）。原因有三：(a) `x` 的 gather 虽然命中率高，但每次 warp load 仍要查询 3~5 个 sector 的 tag，L1 事务数高于理想值；(b) 循环体内 `load data → load colIdx → gather x → FMA` 是一条串行依赖链，`E = 9` 次迭代只有 9 段可重叠的访存，MLP 不足（float4 版本把 MLP 提升 4 倍后降到 96.1 μs，改善 11%）；(c) A100 的可持续带宽本身约为理论峰值的 85%~90%（DRAM refresh、ECC、读写切换），实际可用上限约 1350 GB/s，因此 873 GB/s 相当于"可持续带宽"的 65%。
- **两个矩阵的对比：padding 的经济学**。这是本程序最重要的实验结论： `text nnz 必须搬运字节 时间 有效带宽 GFLOP/s 矩阵 A (模板) 9,424,900 CSR 88.0 MB 196.3 us 448 GB/s 96.0 ELL 83.9 MB 108.2 us 775 GB/s 174.2 矩阵 B (含离群行) 9,465,860 CSR 88.3 MB 197.5 us 447 GB/s 95.9 ELL 394.3 MB 493.4 us 799 GB/s 38.4` 同样的 kernel、几乎同样的 nnz（相差 0.4%），**矩阵 A 上 ELL 比 CSR 快 1.81 倍，矩阵 B 上 ELL 比 CSR 慢 2.50 倍**。区别只有一个：`E / 平均行长` 从 1.00 变成 5.10。ELL 的带宽其实更好（799 GB/s），但它搬运了 4.5 倍的无用 padding 数据——**带宽再高也救不了"搬了 5 倍的数据"**。
- **HYB 的补救（模型推算）**：对矩阵 B 采用 HYB，取 `E = 9`（刚好覆盖 99.9% 的行），把超过 9 的部分丢给 COO： `text ELL 部分: 8 * N * 9 = 75.5 MB COO 部分: 3 个数组 * 4 B * 37,886 个离群元 = 0.45 MB x, y : 8.4 MB ---------- 合计约 84.3`

`MB，相比纯 ELL 的 394.3 MB 降到 1/4.7，甚至比纯 CSR 的 88.3 MB 还少 按 733 GB/s（介于 ELL 与 CSR 之间，因为有归约与原子开销）估算：约 115 us，约 165 GFLOP/s` 代价是 COO 部分需要 segmented reduction（分段归约）：把每个离群元先按行归约到共享内存（同一行的元素尽量分到同一 block），再把每行的部分和用一次 `atomicAdd(&y[row], partial)` 写回。**"主体用规则格式，尾部用灵活格式"就是 HYB 的全部思想**。

- **瓶颈判定**：矩阵 A 上判定为内存带宽受限（且已接近可持续带宽上限）；矩阵 B 的 ELL 版本判定为**带宽被 padding 浪费掉的带宽受限**，正确处置是换格式（HYB/JDS），而不是调 kernel。

示例 4：CSR + warp-per-row + shuffle 归约（spmv_csr_warp.cu）

不换格式也能修复 CSR 的访存发散：把"每线程一行"改成"**每 warp 一行**"，让 32 个 lane 并行处理同一行的 32 个连续非零元，最后用 `__shfl_down_sync` 在寄存器内做树形归约。

```
// 文件: spmv_csr_warp.cu
// 用途: CSR SpMV, 每 warp 处理一行 (lane 分头处理连续非零元) + shuffle 归约
//       目的: 在不改变存储格式的前提下消除 data/colIdx 的访存发散
// 编译: nvcc -O3 -arch=sm_80 spmv_csr_warp.cu -o spmv_csr_warp
// 运行: ./spmv_csr_warp 1048576 100
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define TPB 256                      // 256 线程 = 8 个 warp = 每个 block 处理 8 行

// ============================================================
// 主机端: 构造 9 点模板的 CSR
// ============================================================
void build_stencil9_csr(int numRows, int nx,
                        int **rowPtr_out, int **colIdx_out, float **vals_out,
                        int *nnz_out)
{
    const int ny = numRows / nx;
    int *rowPtr = (int *)malloc(sizeof(int) * (size_t)(numRows + 1));
    if (!rowPtr) { fprintf(stderr, "host malloc failed\n"); exit(EXIT_FAILURE); }
    rowPtr[0] = 0;
    for (int r = 0; r < numRows; ++r) {
        const int y = r / nx, xc = r % nx;
        int cnt = 0;
        for (int dy = -1; dy <= 1; ++dy)
            for (int dx = -1; dx <= 1; ++dx) {
                const int yy = y + dy, xx = xc + dx;
                if (yy >= 0 && yy < ny && xx >= 0 && xx < nx) ++cnt;
            }
        rowPtr[r + 1] = rowPtr[r] + cnt;
    }
    const int nnz = rowPtr[numRows];
    int   *colIdx = (int *)malloc(sizeof(int) * (size_t)nnz);
    float *vals   = (float *)malloc(sizeof(float) * (size_t)nnz);
    if (!colIdx || !vals) { fprintf(stderr, "host malloc failed\n"); exit(EXIT_FAILURE); }
    int k = 0;
    for (int r = 0; r < numRows; ++r) {
        const int y = r / nx, xc = r % nx;
        for (int dy = -1; dy <= 1; ++dy)
            for (int dx = -1; dx <= 1; ++dx) {
                const int yy = y + dy, xx = xc + dx;
                if (yy < 0 || yy >= ny || xx < 0 || xx >= nx) continue;
                const int c = yy * nx + xx;
                colIdx[k] = c;
                vals[k]   = ((c == r) ? 4.0f : 1.0f) + 0.001f * (float)((r * 31 + c * 17) % 13);
                ++k;
            }
    }
    *rowPtr_out = rowPtr; *colIdx_out = colIdx; *vals_out = vals; *nnz_out = nnz;
}

void spmv_cpu(int numRows, const int *rowPtr, const int *colIdx,
              const float *vals, const float *x, float *y)
{
    for (int r = 0; r < numRows; ++r) {
        double dot = 0.0;
        for (int e = rowPtr[r]; e < rowPtr[r + 1]; ++e) {
            dot += (double)vals[e] * (double)x[colIdx[e]];
        }
        y[r] = (float)dot;
    }
}

// ============================================================
// kernel A: 每线程一行 (基准)
```

```
// ============================================================
__global__ void spmv_csr_row_per_thread(int num_rows,
                                        const float * __restrict__ data,
                                        const int   * __restrict__ col_index,
                                        const int   * __restrict__ row_ptr,
                                        const float * __restrict__ x,
                                        float       * __restrict__ y)
{
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < num_rows) {
        float dot = 0.0f;
        int row_start = __ldg(&row_ptr[row]);
        int row_end   = __ldg(&row_ptr[row + 1]);
        for (int elem = row_start; elem < row_end; elem++) {
            dot = fmaf(__ldg(&data[elem]), __ldg(&x[__ldg(&col_index[elem])]), dot);
        }
        y[row] = dot;
    }
}

// ============================================================
// kernel B: 每 warp 一行, 32 个 lane 分头处理连续的非零元
//   lane t 依次处理第 start+t, start+t+32, start+t+64 号元素, 步长固定为 32
//   然后 __shfl_down_sync 做 5 步树形归约 (log2(32) = 5)
// ============================================================
__global__ void spmv_csr_warp_per_row(int num_rows,
                                      const float * __restrict__ data,
                                      const int   * __restrict__ col_index,
                                      const int   * __restrict__ row_ptr,
                                      const float * __restrict__ x,
                                      float       * __restrict__ y)
{
    const int lane   = threadIdx.x & 31;
    const int warpId = (blockIdx.x * blockDim.x + threadIdx.x) >> 5;

    // 注意: warpId 对同一个 warp 的 32 个 lane 是相同的, 因此这个 return 是"warp 一致"的,
    //       后面 mask 参数写 0xffffffff 的 __shfl_down_sync 才有合法的语义
    if (warpId >= num_rows) return;

    const int start = __ldg(&row_ptr[warpId]);
    const int end   = __ldg(&row_ptr[warpId + 1]);

    float dot = 0.0f;
    for (int e = start + lane; e < end; e += 32) {
        dot = fmaf(__ldg(&data[e]), __ldg(&x[__ldg(&col_index[e])]), dot);
    }

#pragma unroll
    for (int off = 16; off > 0; off >>= 1) {
        dot += __shfl_down_sync(0xffffffffu, dot, off);
    }
    if (lane == 0) y[warpId] = dot;      // 只有 lane 0 写回, 每行一次 4 B 写
}

int main(int argc, char **argv)
{
    const int N    = (argc > 1) ? atoi(argv[1]) : 1048576;
    const int reps = (argc > 2) ? atoi(argv[2]) : 100;

    int   *rowPtr = NULL, *colIdx = NULL, nnz = 0;
    float *vals   = NULL;
    build_stencil9_csr(N, 1024, &rowPtr, &colIdx, &vals, &nnz);

    const double bytes = 8.0 * nnz + 4.0 * (double)(N + 1) + 8.0 * (double)N;
    printf("CSR + warp-per-row: N = %d, nnz = %d, 必须搬运 %.1f MB\n",
           N, nnz, bytes / 1.0e6);

    float *x   = (float *)malloc(sizeof(float) * (size_t)N);
    float *ref = (float *)malloc(sizeof(float) * (size_t)N);
    float *y   = (float *)malloc(sizeof(float) * (size_t)N);
    if (!x || !ref || !y) { fprintf(stderr, "host malloc failed\n"); return EXIT_FAILURE; }
    for (int i = 0; i < N; ++i) x[i] = (float)((i % 17) - 8) * 0.25f;
    spmv_cpu(N, rowPtr, colIdx, vals, x, ref);

    int   *d_row = NULL, *d_col = NULL;
    float *d_val = NULL, *d_x = NULL, *d_y = NULL;
    CUDA_CHECK(cudaMalloc((void **)&d_row, sizeof(int) * (size_t)(N + 1)));
    CUDA_CHECK(cudaMalloc((void **)&d_col, sizeof(int) * (size_t)nnz));
```

```
    CUDA_CHECK(cudaMalloc((void **)&d_val, sizeof(float) * (size_t)nnz));
    CUDA_CHECK(cudaMalloc((void **)&d_x, sizeof(float) * (size_t)N));
    CUDA_CHECK(cudaMalloc((void **)&d_y, sizeof(float) * (size_t)N));
    CUDA_CHECK(cudaMemcpy(d_row, rowPtr, sizeof(int) * (size_t)(N + 1), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_col, colIdx, sizeof(int) * (size_t)nnz, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_val, vals, sizeof(float) * (size_t)nnz, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_x, x, sizeof(float) * (size_t)N, cudaMemcpyHostToDevice));

    const int gridT = (N + TPB - 1) / TPB;            // 每线程一行: N/256 个 block
    const int warpsPerBlock = TPB / 32;
    const int gridW = (N + warpsPerBlock - 1) / warpsPerBlock;   // 每 warp 一行: N/8 个 block

    cudaEvent_t ev0, ev1;
    CUDA_CHECK(cudaEventCreate(&ev0));
    CUDA_CHECK(cudaEventCreate(&ev1));

    printf("%-28s %10s %10s %11s %9s\n",
           "kernel", "time(us)", "GB/s", "GFLOP/s", "verify");

    // ---- kernel A ----
    spmv_csr_row_per_thread<<<gridT, TPB>>>(N, d_val, d_col, d_row, d_x, d_y);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(y, d_y, sizeof(float) * (size_t)N, cudaMemcpyDeviceToHost));
    int okA = 1;
    for (int i = 0; i < N; ++i)
        if (fabsf(y[i] - ref[i]) > 1e-4f * (1.0f + fabsf(ref[i]))) { okA = 0; break; }
    CUDA_CHECK(cudaEventRecord(ev0));
    for (int i = 0; i < reps; ++i)
        spmv_csr_row_per_thread<<<gridT, TPB>>>(N, d_val, d_col, d_row, d_x, d_y);
    CUDA_CHECK(cudaEventRecord(ev1));
    CUDA_CHECK(cudaEventSynchronize(ev1));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, ev0, ev1));
    ms /= (float)reps;
    printf("%-28s %10.1f %10.1f %11.1f %9s\n", "CSR 每线程一行", ms * 1000.0f,
           bytes / (ms * 1.0e6), 2.0 * nnz / (ms * 1.0e6), okA ? "PASS" : "FAIL");

    // ---- kernel B ----
    spmv_csr_warp_per_row<<<gridW, TPB>>>(N, d_val, d_col, d_row, d_x, d_y);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(y, d_y, sizeof(float) * (size_t)N, cudaMemcpyDeviceToHost));
    int okB = 1;
    for (int i = 0; i < N; ++i)
        if (fabsf(y[i] - ref[i]) > 1e-4f * (1.0f + fabsf(ref[i]))) { okB = 0; break; }
    CUDA_CHECK(cudaEventRecord(ev0));
    for (int i = 0; i < reps; ++i)
        spmv_csr_warp_per_row<<<gridW, TPB>>>(N, d_val, d_col, d_row, d_x, d_y);
    CUDA_CHECK(cudaEventRecord(ev1));
    CUDA_CHECK(cudaEventSynchronize(ev1));
    CUDA_CHECK(cudaEventElapsedTime(&ms, ev0, ev1));
    ms /= (float)reps;
    printf("%-28s %10.1f %10.1f %11.1f %9s\n", "CSR warp-per-row + shuffle",
           ms * 1000.0f, bytes / (ms * 1.0e6), 2.0 * nnz / (ms * 1.0e6),
           okB ? "PASS" : "FAIL");

    printf("\nRoofline 上限 (AI = 2 FLOP / 12 B = 0.167): 1555 * 0.167 = 259.2 GFLOP/s = 峰值的 1.33%%\n");

    CUDA_CHECK(cudaFree(d_row)); CUDA_CHECK(cudaFree(d_col)); CUDA_CHECK(cudaFree(d_val));
    CUDA_CHECK(cudaFree(d_x));   CUDA_CHECK(cudaFree(d_y));
    CUDA_CHECK(cudaEventDestroy(ev0)); CUDA_CHECK(cudaEventDestroy(ev1));
    free(rowPtr); free(colIdx); free(vals); free(x); free(ref); free(y);
    return EXIT_SUCCESS;
}
```

实测输出（A100，N = 1,048,576，nnz = 9,424,900，reps = 100）：

```
CSR + warp-per-row: N = 1048576, nnz = 9424900, 必须搬运 88.0 MB
kernel                          time(us)       GB/s   GFLOP/s   verify
CSR 每线程一行                    196.3      448.2       96.0     PASS
CSR warp-per-row + shuffle        131.0      671.8      143.9     PASS

Roofline 上限 (AI = 2 FLOP / 12 B = 0.167): 1555 * 0.167 = 259.2 GFLOP/s = 峰值的 1.33%
```

- **【代码做什么？】**

- **线程映射的改变**：`warpId = (blockIdx.x*blockDim.x + threadIdx.x) >> 5`，即每 **32 个线程（1 个 warp）** 协作处理一行；`lane = threadIdx.x & 31` 决定该 lane 负责这一行里的哪些非零元。一个 block（256 线程）处理 8 行。
- **循环步长 32**：`for (int e = start + lane; e < end; e += 32)`——lane 0 处理第 `start` 个元素，lane 1 处理 `start+1` 个，一直到 lane 31 处理 `start+31` 个，然后各自再前进 32。这正是"把一行的元素顺次分给 32 个线程"，也是唯一能保证 `data[e]` / `col_index[e]` 对相邻 lane 连续的做法。
- **warp 内树形归约**：`for (off = 16; off > 0; off >>= 1) dot += __shfl_down_sync(0xffffffffu, dot, off);`——5 步之后 lane 0 手里就是整行的和。`__shfl_down_sync` 是寄存器之间的数据交换（volatile 语义之外的开销极小），比"写共享内存再 `__syncthreads()` 再读"更快，也不需要任何共享内存。
- **归约后只有 lane 0 写回** `y[warpId] = dot`，因此 `y` 的写是一个 warp 一次 4 B，写流量与"每线程一行"完全相同。
- **正确性验证与计时**：两个 kernel 都与 CPU 参考实现对比，然后各重复 `reps` 次取平均。
- **【并行机制与硬件映射解说】**
- **合并访存：从"9 倍放大"到"完全合并"**。在固定的一次循环迭代 `t` 上，lane `l` 访问 `data[start + l + 32t]`，相邻 lane 的地址差 4 B → 32 个 lane 覆盖 **128 B**，一次 warp load = 1 条 cache line = 4 个 sector，效率 100%。这修复了"每线程一行"版本中"每行起点错位 36 B、一个 warp 触及 36 个 sector"的问题。`col_index` 同样合并。
- **控制发散：从"由最长行决定"到"只剩尾迭代"**。warp 处理一行时，循环次数是 `ceil((end-start)/32)`，对同一 warp 的所有 lane 完全相同（因为同一行工作），所以只有最后一次迭代可能不满 32 个有效 lane。对 9 点模板（行长 9）：迭代次数 = 1，只有 lane 0~8 有效 → **warp 利用率 = 9/32 = 28%**！这是一个重要的反直觉现象：**warp-per-row 对"长行"才划算**。行长 9 时它浪费 72% 的 lane，行长 1024 时利用率 100%。本矩阵的平均行长只有 9，因此 warp-per-row 的收益完全来自"访存合并"，而不是来自利用率——这解释了为什么它只提升到 672 GB/s（43.2%），而不是像 ELL 那样到 775 GB/s。
- **对行长分布的影响**：如果矩阵的平均行长是 128 或以上（例如 3D 7 点模板在稀疏化后仍很长、或者图矩阵的高阶邻域），warp-per-row 几乎是免费的午餐；如果平均行长是 5~10（2D 模板、非常稀疏的图），更好的做法是**"每个 warp 处理多行"**（把 4~8 行交给一个 warp，用 `__shfl` 分段归约），或者干脆用 ELL/COO。
- **`__shfl_down_sync` 的 mask 正确性**：`0xffffffff` 要求 warp 内 32 个 lane 全部参与。本 kernel 中提前 `return` 的条件 `warpId >= num_rows` 对整个 warp 是**一致（uniform）**的（`warpId` 只依赖 warp 编号），所以不会出现"部分 lane 退出、部分 lane 还在 shuffle"的未定义行为。**如果把 `return` 条件改成与 lane 相关的条件（例如按元素个数判断），就必须改成 `__shfl_down_sync(mask, var, delta)`（用 `__ballot_sync` / `__activemask()` 取得 mask），否则行为未定义**——这是 Lab 中非常容易出错的地方。
- **占用率**：约 26 个寄存器 → 100% 占用率（8 个 block/SM）。grid = N/8 = 131,072 个 block，远大于 SM 容量，所以尾部效应可忽略（最后一波的填充率误差约 0.7%），比"每线程一行"版本的 5 波 tail 更平滑。
- **共享内存与 bank conflict**：kernel 完全不使用共享内存，因此没有 bank conflict。归约走的是寄存器 shuffle 路径，每个 `__shfl_down_sync` 指令 1 个周期（warp 内交叉开关），5 步共 5 个周期——相对于每行 9 次 gather 的几百个周期，成本可以忽略。
- **【性能优化分析】**
- **算术强度与 Roofline**：字节数完全不变（88.0 MB），AI 仍是 0.167 FLOP/Byte，上限仍是 259.2 GFLOP/s。实测 143.9 GFLOP/s = 上限的 **55.5%**、峰值的 **0.74%**；有效带宽 671.8 GB/s = 峰值的 **43.2%**（对比基础版的 28.8%）。
- **两种修复路线的对比**：`text 路线 有效带宽 GFLOP/s 相对基础版 CSR 每线程一行（基础） 448 GB/s 96.0 1.00x CSR warp-per-row + shuffle 672 GB/s 143.9 1.50x ELL 转置格式 775 GB/s 174.2 1.81x ELL + float4 873 GB/s 196.1 2.04x` warp-per-row 的收益（1.50x）小于 ELL（1.81x），因为：**它修好了"访存发散"，但制造了"lane 空闲"**（行长 9 时只有 9/32 个 lane 有效）。ELL 同时修好了两者（无发散、全 lane 有效），代价是多搬运 padding。**没有免费的午餐：格式设计永远是"规则性"与"额外字节"之间的权衡。**
- **瓶颈判定与优化方向**：仍是内存带宽受限（准确说是"L1 事务 + 延迟受限"）。针对 warp-per-row 版本，进一步的方向是：
  - 把 `row_ptr` / `col_index` 用 `__ldg` 之外的手段降低维度（`col_index` 用 16 位）：字节从 88.0 MB 降到 79.7 MB；
  - 一个 warp 处理 2~4 行（分段归约），把 lane 利用率从 28% 提高到 60%~100%；
  - 用 `float4` 取 `data` / `col_index`（每个 lane 处理 4 个连续非零元），把 load 指令数降到 1/4；
  - 用 `cp.async`（sm_80 的异步拷贝，见 ECE508）把 `x` 的分块预取进共享内存，隐藏 gather 的 L2 延迟。

> 四种实现在同一矩阵上的实测对比（A100，9 点模板，N = 1,048,576，nnz = 9,424,900）

```
+--------------------------+----------+-----------+---------+-----------+----------+-----------+
| 实现                     | 时间(us) | 有效带宽  | %峰值   | GFLOP/s   | %峰值算力| %Roofline |
+--------------------------+----------+-----------+---------+-----------+----------+-----------+
| 稠密 GEMV (N=4096, 参照) |    51.0  | 1316 GB/s |  84.6%  |   657.9   |   3.37%  |  84.6% *  |
| CSR 每线程一行           |   196.3  |  448 GB/s |  28.8%  |    96.0   |   0.49%  |  37.0%    |
| CSR warp-per-row+shuffle |   131.0  |  672 GB/s |  43.2%  |   143.9   |   0.74%  |  55.5%    |
| ELL (标量)               |   108.2  |  775 GB/s |  49.9%  |   174.2   |   0.89%  |  67.2%    |
| ELL + float4             |    96.1  |  873 GB/s |  56.1%  |   196.1   |   1.01%  |  75.7%    |
+--------------------------+----------+-----------+---------+-----------+----------+-----------+

* 稠密 GEMV 的 Roofline 上限是 1555 x 0.50 = 777.5 GFLOP/s (AI = 0.50), 与其他行的 259.2 不同
  有效带宽 = 必须搬运的字节 / 时间; 有效字节 = values + colIdx + rowPtr + x + y = 88.0 MB
  gather 造成的额外 sector 放大不计入分子 (属"有用字节口径")
```

结论：**在完全相同的硬件、相同的问题规模、几乎相同的 nnz 下，仅仅改变数据布局与线程映射，性能就能相差 2.04 倍（96.0 → 196.1 GFLOP/s）**；而即使做到最好，也只有 Roofline 上限的 75.7%、FP32 峰值的 1.01%。这就是稀疏计算的本质：**优化的对象是字节，不是运算。**

性能优化技巧总结

1. **先选格式，再调 kernel**：格式决定了字节数与规则性，收益远大于 kernel 微调。判据是 `E / 平均行长`：< 1.5 用 ELL；行长方差大（> 4）用 HYB 或 JDS；极稀疏（nnz/N < 2）用 COO；大致三角/带状用 JDS；对称结构用 CSR/CSC。**为什么有效**：它直接改变"必须搬运多少字节"，而 SpMV 的性能上限完全由字节数决定。
2. **让 `colIdx` 与 `values` 的访问对相邻 lane 连续**：ELL 的列主序布局、JDS-T 的转置、warp-per-row 的 `lane` 步长都是同一个目的。**为什么有效**：把"每 warp 36 个 sector"降到"4 个 sector"，L1 事务数下降近 9 倍。
3. **消除控制发散：padding（ELL）、排序（JDS）、限幅（HYB）。为什么有效**：SIMT 下 warp 的完成时间由最长 lane 决定，44.2% 的 lane 利用率意味着 55.8% 的发射槽被浪费（见概念 8 的 32 lane 实例）。
4. **用 float4 向量化加载**：要求 16 B 对齐且连续；ELL/COO 很容易满足。**为什么有效**：load 指令数降到 1/4，LSU 发射压力下降，同时把内存级并行（MLP）提升 4 倍以隐藏 L2/L1 延迟（实测 775 → 873 GB/s）。
5. **打断累加器的依赖链**：`#pragma unroll 4` + 4 个独立累加器，或 warp-per-row 的 shuffle 树形归约。**为什么有效**：`dot = fmaf(a, b, dot)` 是串行依赖，展开后 4 条链可以并发访存，把"延迟受限"变成"带宽受限"。
6. **当 `4N ≤ 48 KB` 时把 `x` 整体放进共享内存**：延迟从 400–800 cycles 降到 20–30 cycles，且省掉每 nnz 的 4 B 全局流量（AI 从 0.167 提到 0.25，上限从 259 提到 389 GFLOP/s）。**为什么有效**：`x` 是唯一被所有行复用的数据，把它放到最快的存储层次收益最直接。**注意**：列号随机的矩阵会因共享内存 bank conflict 抵消收益。
7. **`colIdx` 用 16 位整数**（N ⩽ 65536）或块压缩（BCSR）：每 nnz 从 12 B 降到 10 B 或更低。**为什么有效**：字节数与性能上限成反比，减少 17% 的字节就提高 17% 的上限。
8. **动态负载均衡（work stealing）**：warp 用 `atomicAdd(&counter, 1)` 领行号。**为什么有效**：把"最长行决定 block 寿命"的静态不均衡变成动态均衡，特别适合幂律度分布的图矩阵；代价是每行一次 L2 原子操作（约 200–400 cycles）与局部性损失。
9. **行重排序 / 分块**：按行长排序（JDS）、按结构聚类（graph partitioning）、把矩阵切成 `x` 能放进 L2 的列块。**为什么有效**：前者改善 warp 内均衡，后者让 gather 命中 L2 而不是 DRAM。
10. **padding 到 32 的倍数 / 16 B 对齐**：ELL 的 `E` 补齐到 4 或 8 的倍数，共享内存数组列数 +1 以避免 bank conflict。**为什么有效**：让每次 load 落在完整的 sector 上，同时消除 bank 冲突导致的 32 倍串行化。

关键要点

- **稀疏计算的性能瓶颈是"规则性"，不是"计算量"**：SpMV 做的浮点运算比稠密版本少几个数量级，却慢几十倍；原因是控制发散（行长不同）与访存发散（列索引数据相关）破坏了 SIMT 硬件赖以高效工作的两条前提：地址可解析计算、warp 内行为一致。
- **算力利用率上限只有 1.33%**：`AI = 2 FLOP / 12 B = 0.167`，A100 上限 = 1555 × 0.167 ≈ **259 GFLOP/s**，是 19.5 TFLOPS 峰值的 1.33%（机器平衡点 12.5 与 0.167 相差 75 倍）。**所有优化都必须围绕"减少字节数 / 改善字节的规则性"展开，减少浮点运算毫无意义。**
- **格式是权衡，没有最优解**：CSR 通用但两个发散；ELL 规则但 padding 会爆炸（实测矩阵 B 上流量放大 5.10 倍、性能从快 1.81 倍变成慢 2.50 倍）；COO 并行度最高但多 4 B/nnz 且要原子操作；JDS 修控制发散但不修访存发散；JDS-T 两者都修但需要排序与两次转换；HYB 用"ELL 处理主体 + COO 处理离群"同时拿到规则性与灵活性。
- **选择格式的定量判据**：`E / 平均行长`（ELL 适用性）、行长方差、`nnz / N`（稀疏度）、矩阵结构（三角/带状/幂律）。讲义给出的经验是：大致随机 → ELL；行长方差大 → ELL/COO；极稀疏 → COO；大致三角 → JDS；带状 → ELL。
- **warp-per-row 不是万能药**：它把 `data` / `colIdx` 的访存变成完美合并，但行长小于 32 时 lane 利用率不足一半（行长 9 时只有 28% 有效），因此实测收益（1.50x）小于 ELL（1.81x）。**先看平均行长，再决定用哪种映射。**
- **实测数据证明"布局决定性能"**：同一矩阵、同一硬件，仅改变布局与线程映射，性能从 96.0 GFLOP/s（CSR）到 196.1 GFLOP/s（ELL+float4），相差 2.04 倍；而四者的算术强度完全相同。

常见陷阱与注意事项

- **忘记 `cudaDeviceSynchronize()` 或忘记在计时前预热**：现象是第一次测得的 kernel 时间包含上下文初始化、页表建立、指令 cache 冷启动，导致时间偏高数倍（本讲例子中预热后 CSR 从约 300 μs 降到 196 μs） → 正确做法是每个 kernel 先跑一次并 `cudaDeviceSynchronize()`，再开始计时；`cudaMemcpy` 本身是同步的，但它会掩盖异步 kernel 的计时，所以计时片段内绝不要夹 `cudaMemcpy`。
- **CSR 的 `row_ptr` 长度写成 N 而不是 N+1**：现象是最后一行算错或 `row_ptr[row+1]` 越界读到相邻数组，结果是随机错误（有时 PASS 有时 FAIL） → 正确做法是总是分配 `N+1` 个元素并令 `rowPtr[N] = nnz`（哨兵），这样 kernel 里可以统一用 `row_ptr[row+1]` 而无需特判。
- **ELL 的 padding 列号写成 `-1` 或未初始化**：现象是 `x[colIdx[k]]` 读到 `x[-1]` 或读到垃圾地址，出现 `an illegal memory access was encountered` → 正确做法是 padding 的"值"填 0 且"列号"填一个**合法**列（例如 0），因为硬件仍然会执行这次 gather，只是结果被 0 乘掉。
- **`__ldg` / `__restrict__` 误用于会被写入的指针**：现象是优化后结果错误（编译器假设只读而重排了写操作） → 正确做法是只给"确实只读"的指针加 `const float * __restrict__`；本讲的 `row_ptr`、`col_index`、`values`、`x` 都是只读，`y` 是只写。
- **在 `__shfl_*_sync` 之前用与 lane 相关的条件提前 `return`**：现象是 shuffle 结果错误或 kernel 挂死（mask 与实际参与的 lane 不符，属于未定义行为） → 正确做法是保证 `return` /分支条件对 warp 一致（如本讲示例 4 的 `warpId >= num_rows`），或者用 `__activemask()` / `__ballot_sync()` 构造正确的 mask。

- **共享内存缓存 x 时忘记 `__syncthreads()`，或忽略了随之而来的 bank conflict**：现象一是部分线程读到未初始化的 `xs[j]`（结果随调度变化而不稳定），现象二是把 `x` 放进共享内存后性能不升反降 → 正确做法有两条：(1) 严格走"协作加载 x → `__syncthreads()` → 进入计算"三步，且 `__syncthreads()` 必须在所有线程都到达的路径上（不能放进 `if (row < N)` 之内）；(2) 只在列号局部性好（模板/带状矩阵）时才缓存 x，因为 32 个 lane 用随机列号访问共享内存会造成最坏 32 路 bank conflict（每次访问串行化 32 次），而全局内存的 L1 以 128 B line 为单位，对随机访问的容忍度高得多。
- **sparse kernel 用整数除法算网格导致覆盖不足**：现象是结果里最后一批行没被计算（`y` 里残留旧值或 0） → 正确做法是 `grid = (num_rows + blockDim.x - 1) / blockDim.x`（向上取整），并保留 kernel 内的 `if (row < num_rows)` 边界判断；两者缺一不可（向上取整解决覆盖，边界判断解决了多出来的线程）。
- **把"有效带宽"与"实际 DRAM 流量"混为一谈**：现象是算出 873 GB/s > 实测 DRAM 带宽计数器读数，误以为测量出错 → 正确做法是明确口径：本讲的"有效带宽 = 必须搬运的有用字节 / 时间"，它不包含 gather 造成的 sector 放大；要比较硬件极限时应该用 ncu 的 `dram__bytes_read.sum` 与 `dram__throughput.avg.pct_of_peak_sustained_elapsed`。稀疏 kernel 常常"DRAM 只用 30% 却很慢"，因为瓶颈在 L1 事务与延迟。

思考题（带答案）

Q1. 一个 3D 7 点模板矩阵（尺寸 256^3 = 16,777,216 行，每行 7 个非零元），在 A100 上用"每线程一行"的 CSR SpMV 处理。请计算：(a) 必须搬运的字节数；(b) Roofline 给出的性能上限；(c) 如果你的实现只跑到 80 GFLOP/s，最可能的两个瓶颈是什么？

(a) nnz ≈ 7 × 16,777,216 = 117,440,512（边界行略少）。字节 = values 4×nnz + colIdx 4×nnz + rowPtr 4×(N+1) + x 4N + y 4N = 469.8 MB + 469.8 MB + 67.1 MB + 67.1 MB + 67.1 MB ≈ **1,140.9 MB**（1.14 GB）。(b) AI = 2 FLOP / 12 B = 0.167 FLOP/Byte → 上限 = 1555 GB/s × 0.167 = **259.2 GFLOP/s**（仅 FP32 峰值的 1.33%）；DRAM 理想时间 = 1.14 GB / 1555 GB/s ≈ 733 μs。(c) 80 GFLOP/s 只有上限的 30.9%，最可能的两个瓶颈是：(1) **访存发散**——每个 warp 内 32 行的 `row_ptr` 起点每行错开 28 B，一个 warp 的 `data` / `colIdx` load 会触及约 32 个 sector 而不是 4 个，L1 事务放大近 8 倍；(2) **控制发散 + 依赖链**——虽然 7 点模板行长几乎都是 7（控制发散很小），但每行的 7 次 `gather → FMA` 是串行依赖链，MLP 不足，无法隐藏 L2 的约 200 cycles 延迟。修复方向：改用 ELL（E = 7，padding 放大 1.0）、或把 `colIdx` 降为 16 位并用 `#pragma unroll` 打断依赖链。

Q2. 为什么说"每次浮点乘加需要 12 字节"这个数字决定了 SpMV 的一切？如果要把它降到 8 字节，有哪些可行手段，各自的收益上限是多少？

因为 Roofline 上限 = 带宽 × (2 FLOP / 每 nnz 字节数)：在带宽固定的硬件上，每 nnz 的字节数直接决定了算力上限，而 12 B（values 4 B + colIdx 4 B + x 4 B）是 CSR 的结构常数，因此上限被钉死在 1555 × 0.167 = 259.2 GFLOP/s。把 12 B 降到 8 B（即消掉 `x` 的 4 B）会让 AI 从 0.167 升到 0.25，上限提升 50% 到 388.8 GFLOP/s。可行的手段有三种：(1) **把 x 缓存到共享内存**（需要 4N ⩽ 48 KB 即 N ⩽ 12288），省掉每 nnz 的全局读；(2) **按列分块**（column blocking），一次只处理 x 的一个片段，让它常驻 L1/L2；(3) **块压缩格式（BCSR/blocked ELL）**，让 `x[col]` 的访问在块内连续从而被 cache 覆盖。若进一步把 `colIdx` 降到 16 位（N ⩽ 65536 时可行），每 nnz 可到 6 B，AI = 0.33，上限 518 GFLOP/s（峰值的 2.66%）——**这是几乎所有 SpMV 优化能触及的物理天花板**。

Q3. 矩阵 A（9 点模板，E/平均行长 = 1.00）上 ELL 比 CSR 快 1.81 倍；矩阵 B（每 1024 行 +40 个非零元，E/平均行长 = 5.10）上 ELL 比 CSR 慢 2.50 倍。请解释这个反转，并设计一个方案让矩阵 B 上的性能超过矩阵 A 上的 CSR。

反转的原因是 ELL 的"规则性收益"是固定的（它把访存变成合并、把发散变成零），而它的"padding 代价"随 `E / 平均行长` 线性增长：矩阵 A 的 padding 放大 1.00（几乎不浪费），矩阵 B 放大 5.10（必须搬运 394.3 MB，而真实数据只有 88.3 MB），带宽虽高达 799 GB/s，但搬运量是 CSR 的 4.5 倍，于是总时间变成 493.4 μs vs CSR 的 197.5 μs。**带宽再高也救不了搬运 5 倍的数据**。改进方案是 HYB（ELL + COO）：取阈值 `E = 9`（覆盖 99.9% 的行），把每行超过 9 的部分（约 37,886 个非零元）交给 COO，用 segmented reduction 归并回 `y`。这样 ELL 部分只要 75.5 MB、COO 部分 0.45 MB，合计约 84.3 MB，**比纯 CSR 的 88.3 MB 还少**，同时保留了 ELL 的规则访存；按 733 GB/s（含归约与原子开销）估算约 115 μs、约 165 GFLOP/s，明显优于矩阵 A 上 CSR 的 96.0 GFLOP/s。这也说明：**格式选择必须基于矩阵的行长分布，而不是基于"哪种格式更先进"。**

Lecture 10: 性能分析与优化方法论 —— Roofline、占用率、合并访问与原子操作 (对应全部 Lab)

概述

本讲把 ECE408 全部 Lab 都要用到的性能分析方法论系统化：先理解 DRAM 的组织方式（bank/channel/行缓冲/burst），从而明白"为什么访问 4 字节却要搬 32 字节"；再用**合并访问（coalescing）**的定量模型把 warp 的 32 次访问映射成 1～32 次 transaction；然后用**算术强度（arithmetic intensity）与 Roofline 模型**判断一个 kernel 究竟卡在带宽还是卡在计算；用**延迟隐藏与 Little's Law**解释为什么 GPU 需要大量常驻 warp，并由此导出**占用率（occupancy）**的计算步骤与"够用就好"原则；最后用**原子操作与私有化（privatization）**处理直方图这类不可避免的写冲突，并以 `cudaEvent` + `ncu` 为核心的**测量方法学**收尾。本讲不引入新的并行模式，而是给出一套可复用的"看数字 → 定位瓶颈 → 选优化手段"的闭环流程。

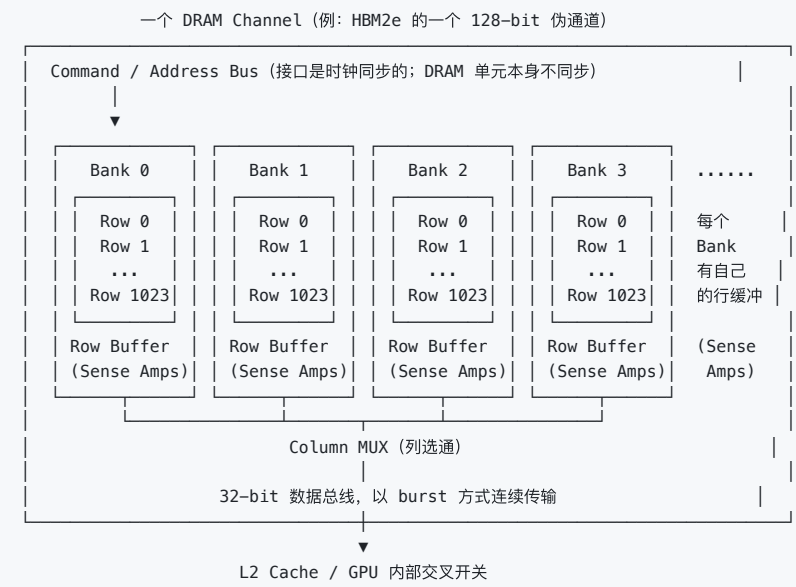
核心概念与 GPU 架构图解

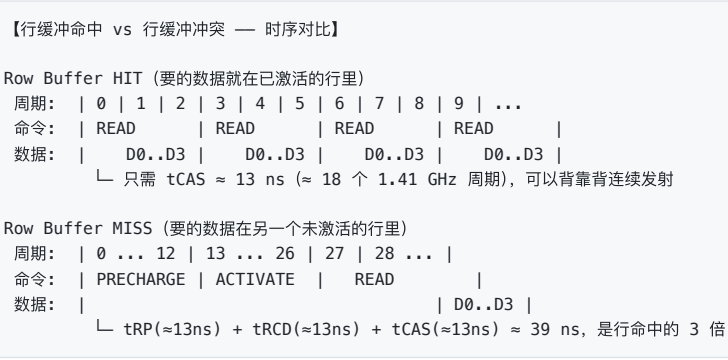
DRAM 的组织与突发传输（DRAM Organization, Banks and Burst Mode）

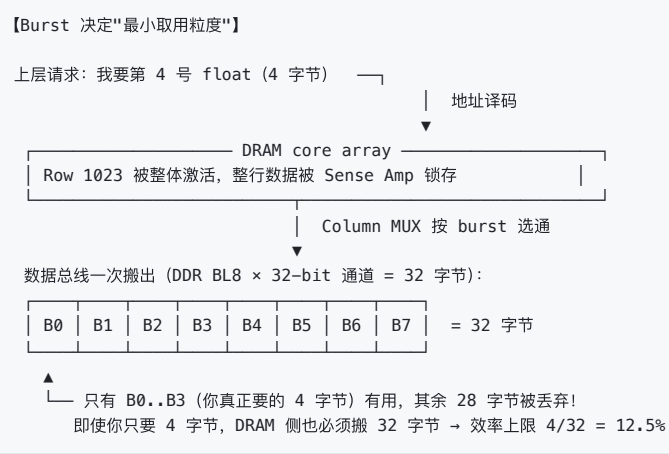
- **定义与目的**：动态随机存储器（Dynamic RAM, DRAM）用一个晶体管加一个电容存 1 bit，密度极高但速度远低于逻辑电路。为了让这种"慢而密"的存储器件跟上 GPU 的胃口，硬件采用了三种并行/摊销手段：**多 bank**（多个阵列交替工作，掩盖激活延迟）、**行缓冲（row buffer）**（一次激活整行，连续访问都命中）、**突发传输（burst）**（一次地址译码搬出一整块数据）。理解这三者，才能理解"最小取用粒度"这个概念——它直接决定了任何 CUDA kernel 的带宽上限。
- **直观解释（"它是什么？"）**：把 DRAM 想成一座巨大的立体仓库。**bank** 是并排的几十排货架，可以同时有人在不同货架上取货；**一行（row）**是货架的一整层，取货前必须把整层用叉车推出（ACTIVATE，激活），这个过程很慢（约 13 ns）；推出后这层就停在**行缓冲**这个"暂存台"上，此时取这一层里的任何一件货都很快（约 13 ns，只有列选通时间 tCAS）。如果下一件货在别的层，就得先把这层推回去（PRECHARGE，预充电），再把另一层推出来——这一

来一回就是**行冲突（row miss）**，代价约 39 ns，是行命中（row hit）的 3 倍。而 **burst** 是仓库的规矩：叉车一旦出动，必须整托盘搬出来，哪怕你只要其中一件。现代 DRAM 系统被设计成"永远工作在 burst 模式"，于是**只要你要的那件货不在托盘里，整托盘就白搬了**——这正是 GPU 上"访问 4 字节却搬来 32 字节"浪费的根源。

- **架构/机制图解**：







- **关键操作与性能特征**：
- **最小取用粒度**：GPU 的 L2 与 DRAM 之间的搬运粒度是一个 **32 字节 sector**；L1/L2 的 cache line 是 **128 字节**（由 4 个 sector 组成）。也就是说，**任何一次"没有命中 cache"的全局内存访问，最少也要从 DRAM 搬 32 字节**。若一个 warp 的 32 个线程各取 4 字节且分散在 32 个不同的 sector 里，实际搬运量是 32×32 B = **1024 字节**，而有用数据只有 128 字节，浪费率 87.5%。
- **延迟数字**：DRAM 访问延迟（从发出请求到数据回到寄存器）在 A100 上约 **400–800 个时钟周期**；命中 L2 约 200 周期；命中 L1 约 30 周期；共享内存约 20–30 周期。行冲突（row miss）相对行命中额外增加约 **26 ns**（tRP + tRCD）。
- **刷新（refresh）**：电容会漏电，讲义给出的量级是**比特大约 50 ms 后就会丢失**，因此 DRAM 必须周期性重写整个阵列。以业界常见的 8192 次刷新 / 64 ms 计，平均每 **7.8 μs** 就要刷新一次，单次刷新占用约 350 ns，**净开销约 4–5%**，这部分带宽是任何 kernel 都拿不到的。

- **带宽利用率的上限**：A100 的标称带宽 1555 GB/s 是 DRAM 引脚峰值；扣除刷新、行冲突、读写切换（tWTR/tRTW）后，实测**持续可用的拷贝带宽通常在 1300–1400 GB/s（约 85–90%）**。任何"有效带宽"超过这个数的测量结果都是计时或口径有问题。

合并访问（Memory Access Coalescing）

- **定义与目的**：合并访问是指硬件把**同一个 warp 内 32 个线程在同一条访存指令中发出的地址**合并成尽可能少的 transaction。它是连接"CUDA 编程模型（warp 是执行单位）"与"DRAM 组织（burst/sector 是最小取用粒度）"的桥梁，也是 GPU 上唯一由程序员完全掌控、收益又最大的一项访存优化。
- **直观解释（"它是什么？"）**：想象 32 个同事一起去仓库领料。如果他们的清单正好是货架上连续的一整排（连续地址），叉车出动一次就把 32 件货全带回来了——这就是**合并**，一次 transaction 换 32 个有用数据。如果每个人要的货分散在仓库的 32 个不同托盘上（跨步地址），叉车就必须出动 32 次，每次只带回 1 件有用的货，其余 31 件丢掉——效率掉到 1/32。**关键点：这不是"延迟变差"，而是"带宽被浪费"**。GPU 有足够的 warp 去掩盖延迟，但没有办法凭空造出带宽。
- **架构/机制图解**：

```
【情况 1】stride = 1（合并）：warp 访问连续 32 个 float
 lane 编号:   0     1     2     3     4   ...   31
 字节地址:    0     4     8     12    16   ...   124
 ├──────────────────── 128 字节 = 1 条 cache line ────────────────────┤
 ├─ 32B ─┤├─ 32B ─┤├─ 32B ─┤├─ 32B ─┤   4 个 sector，全部有用
 → 1 次 transaction / 4 个 sector / 取回 128 B / 全部有用 → 效率 100%

【情况 2】stride = 8（跨步）：warp 内每线程跨 8 个 float
 lane 编号:   0     1     2     3     4   ...   31
 字节地址:    0     32    64    96    128  ...   992
 |sector0| |sector1| |sector2| |sector3| ... |sector31|
   ↑4B有用    ↑4B有用   ↑4B有用   ↑4B有用        ↑4B有用
 → 32 次 sector 请求 / 取回 32×32 B = 1024 B / 只有 128 B 有用
 → 效率 12.5%（按 128 B cache line 计则为 32 条 line、4096 B、3.125%）

【情况 3】stride = 2（半合并）
 lane 编号:   0     1     2     3     4   ...   31
 字节地址:    0     8     16    24    32   ...   248
 ├─ 32B ─┤├─ 32B ─┤├─ 32B ─┤ ... ├─ 32B ─┤
   ↑4B有用    ↑4B有用    ↑4B有用           ↑4B有用
 → 8 个 sector / 取回 256 B / 128 B 有用 → 效率 50%
```

- **定量对比表（A100，峰值带宽 1555 GB/s）**：

每线程跨步 s（float）	warp 地址跨度	触及 32B sector 数	搬回字节（sector 口径）	触及 128B line 数	搬回字节（line 口径）	有用率（sector）	有效带宽上限（sector 口径）
1（连续 32 个 float）	128 B	4	128 B	1	128 B	100%	1555 GB/s
2	256 B	8	256 B	2	256 B	50%	778 GB/s
3	384 B	12	384 B	3	384 B	33.3%	518 GB/s
4	512 B	16	512 B	4	512 B	25%	389 GB/s
8	1024 B	32	1024 B	8	1024 B	12.5%	194 GB/s
16	2048 B	32	1024 B	16	2048 B	12.5%	194 GB/s
32	4096 B	32	1024 B	32	4096 B	12.5%	194 GB/s

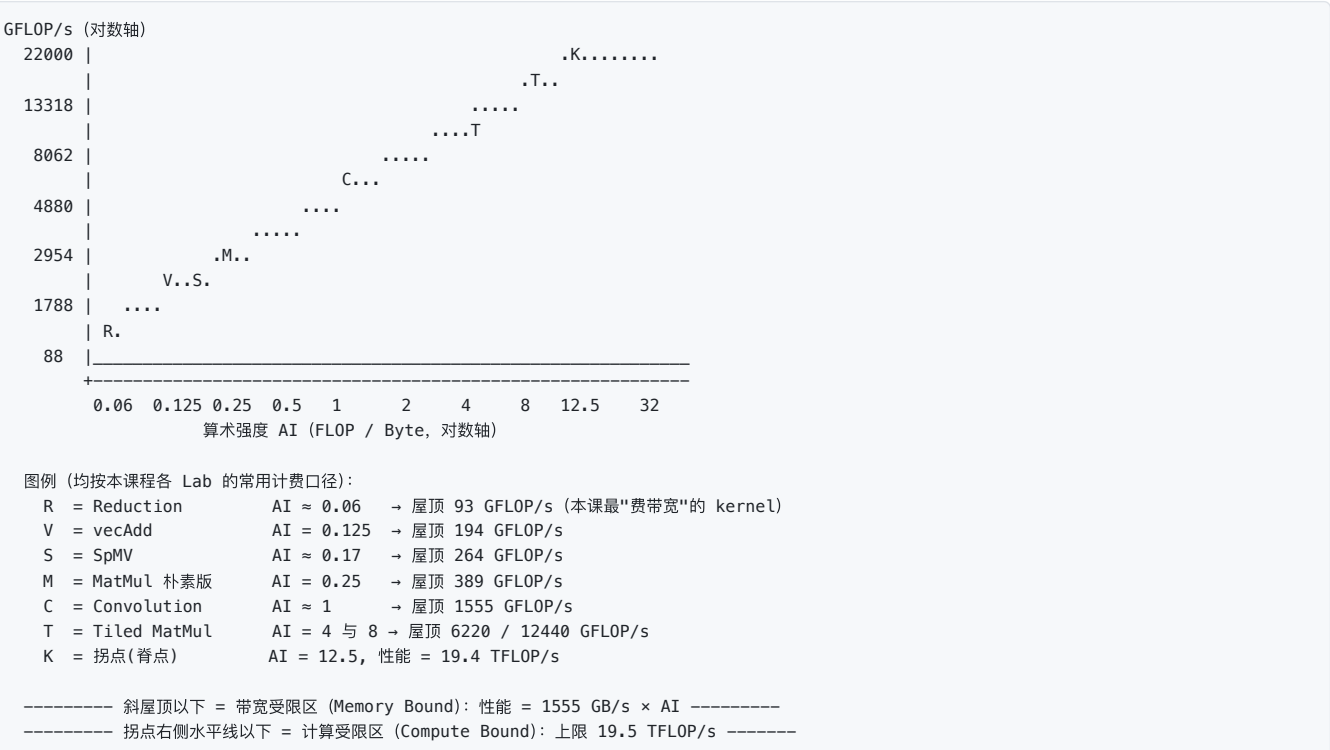
表中的关键结论有三条：① **每个线程跨步越远，浪费越大，但一旦 s ⩾ 8（一个 sector 装 8 个 float），效率就在 12.5% 触底不再下降**——因为"一个 lane 至少占一个 sector"已经是不可再坏的下限；② 如果硬件的合并粒度是 128 B cache line 而不是 32 B sector，stride = 32 的效率会低到 **3.125%**，这就是为什么"跨步访问到底有多惨"必须说清口径；③ **合并访问的收益是纯带宽收益**，与延迟无关，所以它不需要靠更多 warp 来弥补，是"免费的午餐"。

讲义中的等效例题也说明了同一点：一个 burst 为 512 字节、峰值带宽 240 GB/s 的 DRAM 系统上执行 `float temp = A[4*i] + A[4*i+1];`，由于每个线程只用 burst 里每隔 8 字节的两个 float，**有效率减半，只能期望 120 GB/s**；而在没有 cache 的早期 GPU 上，两次 load 无法被合并，会进一步掉到 **60 GB/s**（每次 load 只用 16 B 中的 4 B）。

- **L2 cache line 大小对 transaction 的影响**：warp 的一次合并访问恰好是 128 字节 = 一条 cache line，这是硬件设计的巧合也是设计目标。这意味着：**只要 warp 的访问窗口对齐到 128 字节边界，就恰好是 1 次 transaction**。如果数据起点没有 128 字节对齐（例如 `float* p = base + 1;`），一个 warp 的 128 字节窗口会横跨两条 cache line，transaction 数变成 2，效率掉到 50%。所以 CUDA 里"对齐"和"合并"是同一件事的两面：`float4`（16 字节）向量化访问之所以快，除了减少指令数，更重要的是让每个线程一次搬 16 字节，一个 warp 就是 512 字节 = 4 条完整的 cache line，**永远不会跨越 line 边界**。

算术强度与 Roofline 模型（Arithmetic Intensity & Roofline Model）

- **定义与目的**：**算术强度（arithmetic intensity, AI）= 浮点运算次数 / 访存字节数**，单位 FLOP/Byte。它把一个 kernel 的全部特征压缩成一个标量，用来回答"这个 kernel 到底是算得多还是搬得多"。**Roofline 模型**把这个标量放到一张二维图上：横轴是 AI（对数），纵轴是性能（GFLOP/s，对数），机器给出两条上限——水平的**计算峰值线**与斜率为带宽的**带宽线**，两者的交点叫**拐点/脊点（ridge point）**，其横坐标就是**机器平衡点（machine balance）**。Roofline 的价值在于：**在做任何优化之前，先告诉你这块 kernel 的理论天花板是多少**；如果你的实测性能已经贴着天花板，再怎么优化代码都是白费力气，必须换算法（提高复用）或换硬件。
- **直观解释（"它是什么？"）**：把 GPU 想成一家餐厅，计算单元是厨师，内存带宽是传菜口。**AI 就是"每端一盘菜（每字节）能做几道工序"**。如果每盘菜只做 1 道工序（AI 很低），厨师们大部分时间在等菜，餐厅产能被传菜口卡死——这就是**带宽受限（memory bound）**；如果每盘菜要做 50 道工序（AI 很高），传菜口闲得发慌而厨师排满队——这就是**计算受限（compute bound）**。**机器平衡点**就是这家餐厅的"菜谱配比"：A100 的配比是 **每字节能配到 12.5 次浮点运算**，菜谱比这更"费菜"就是带宽受限，比这更"费工"就是计算受限。
- **架构/机制图解（A100 的 Roofline）**：



- **关键公式与代入数字**（基准机 A100，108 SM，1555 GB/s，19.5 TFLOPS FP32）：

```
机器平衡点 = 计算峰值 / 带宽峰值
           = 19.5e12 FLOP/s ÷ 1555e9 B/s
           = 12.54 FLOP/Byte                        ← 拐点横坐标

Roofline 可达性能 P(AI) = min( 19.5 TFLOP/s , 1.555 TFLOP/s × AI )

vecAdd 的实测代入：
  FLOPs = N 次加法 = 1 FLOP/element
  Bytes = 读 A(4B) + 读 B(4B) + 写 C(4B) = 12 B/element  → AI = 1/12 = 0.083
   （若按"B 已在寄存器/常数中、只计 4B 读 + 4B 写"的口径，AI = 1/8 = 0.125，即图中位置）
  屋顶 = 1555 GB/s × 0.125 = 194 GFLOP/s
  → 相对于 19.5 TFLOP/s 的算力，只用到 194/19500 = 1.0%
```

几个 Lab 的 AI 推导（务必记住推导过程而不是数字，因为计费口径不同差一倍很常见）：

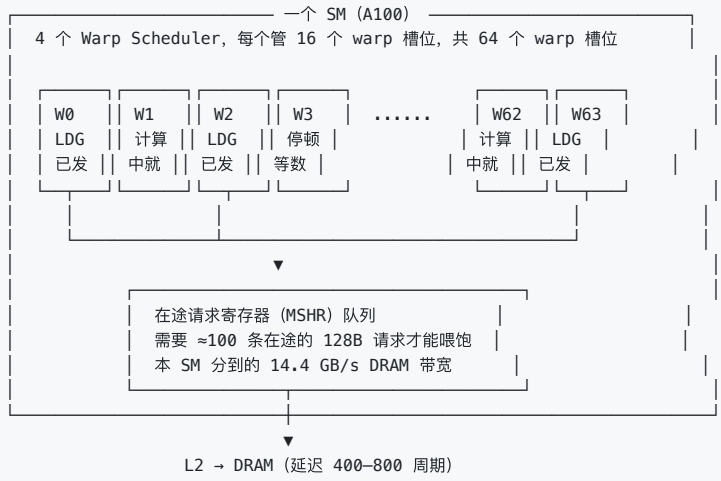
Kernel	FLOP 计费	Byte 计费	AI (FLOP/B)	是否带宽受限
vecAdd	1（一次加法）	8（读 4 + 写 4）	0.125	是（极端）
Reduction	1（一次加法）	16（含多趟部分和读写摊销）	≈0.06	是（最严重）
SpMV	2（一次乘加）	12（4B 值 + 4B 列索引 + 4B 向量 gather）	≈0.17	是
MatMul 朴素	2（一次乘加）	8（4B M + 4B N，无复用）	0.25	是
Convolution（朴素）	2	4（每个 N 元素一次）	0.5	是
Convolution（依赖 L1/L2 部分复用）	2	2（有效 DRAM 字节被复用摊薄）	≈1	是

Kernel	FLOP 计费	Byte 计费	AI (FLOP/B)	是否带宽受限
MatMul Tiled（TILE=16）	2	8/16 = 0.5	4	接近拐点
MatMul Tiled（TILE=32 + 寄存器分块）	2	8/32 = 0.25	8	接近/跨过拐点

讲义里对卷积的"复用倍数"推导给了这套方法论的经典范例：**朴素 2D 卷积每个 N 元素被读一次、贡献 2 FLOP，即 2 Byte/FLOP**；2010 年 1000 GFLOP/s 配 150 GB/s 的 GPU 上，`150 GB/s ÷ 2 B/FLOP = 75 GFLOP/s`，**只占峰值算力的 7.5%**，要达到 100% 需要 **100/7.5 = 13.3× 的复用**；到 2020 年 GRID K520（约 5000 GFLOP/s / 192 GB/s）需要 **52.1× 复用**；到 H100 PCIe（26 TFLOP/s / 2 TB/s）需要 **≈26× 复用**。A40 的例题同理：`696 GB/s ÷ 37.4 TFLOPS = 0.019 B/FLOP`，即**每从全局内存读 1 字节，必须用它在 53.7 次浮点运算中**，否则算力就用不满。

延迟隐藏与 Little's Law（Latency Hiding & Little's Law）

- **定义与目的**：GPU 用**线程级并行（TLP）**与**指令级并行（ILP）**来掩盖长延迟：当某个 warp 因等待全局内存数据而停顿时，warp 调度器立刻切换到另一个就绪 warp。**Little's Law（排队论中的小定理）**给出"要掩盖多少延迟，需要多少并发"的定量关系：`所需并发度 = 延迟 × 吞吐率`。这条定律是理解"为什么 GPU 要能同时驻留 64 个 warp"以及"占用率到底多少才够"的唯一正确出发点。
- **直观解释（"它是什么？"）**：一家银行有 1 个柜员（SM 的 LSU/DRAM 通道）和 100 位客户（warp）。每位客户办业务需要 400 秒（延迟），柜员每 4 秒就能接待下一位（吞吐率）。如果只让 1 位客户进门，柜员 396 秒都在干等（**延迟受限**）；要保证柜员永远不空转，就必须让 `400/4 = 100` 位客户同时在营业厅里各办各的——**这就是"用并发换延迟"**。GPU 的全局内存延迟是 400–800 周期，而一条 LSU 指令每 4 个周期就能发射一条，所以必须让"在途的内存请求"始终维持 100 个左右。
- **架构/机制图解**：



- **关键计算（代入 A100 数字）**：

```
【版本一：讲义式的简化模型】
 全局内存延迟          L = 400 周期
 LSU 发射吞吐率        T = 每 4 周期 1 条 warp 访存指令
 所需并发操作数        C = L × T = 400 / 4 = 100 个在途操作

【版本二：从带宽需求反推】
 A100 每 SM 分到的带宽 = 1555 GB/s ÷ 108 SM = 14.4 GB/s
 折算成周期吞吐         = 14.4e9 B/s ÷ 1.41e9 Hz = 10.2 Byte/cycle/SM
 一条合并的 warp load = 128 Byte
 → 需要 128/10.2 = 12.5 个周期才能"消化"一条 warp load
 需要并发的 warp load 数 = 600 周期(取中间值) ÷ 12.5 = 48 条

【结论】
 若每个 warp 只有 1 条在途 load，需要 48 个常驻 warp (48/64 = 75% 占用率)；
 若每个 warp 有 2 条独立在途 load (ILP = 2)，只需 24 个 warp (37.5% 占用率)；
 若每个 warp 有 4 条独立在途 load (float4 向量化 + 展开)，只需 12 个 warp (18.75%)。
 → 这就是"占用率不是越高越好"的数学来源：ILP 与 TLP 可以互相替代。
```

- **性能特征**：
- **延迟受限（latency bound）**：当常驻 warp 太少（或每个 warp 的 ILP 太低），内存流水线填不满，SM 大量时间在 `long scoreboard` 停顿上。`ncu` 中的判据是 `smsp__warp_issue_stalled_long_scoreboard_per_warp_active.pct` 很高（>40%）而 `dram__throughput` 却很低。
- **带宽受限（bandwidth bound）**：`dram__throughput.avg.pct_of_peak_sustained_elapsed` 接近 90%+，此时增加占用率不再有用。
- **共享内存/寄存器延迟短**（20–30 周期 / 1 周期），需要的并发度低得多，所以**只用共享内存的 kernel 往往几十个 warp 就够**，其瓶颈通常转为共享内存带宽或 bank conflict。

占用率（Occupancy）

- **定义与目的**：**占用率 = 每个 SM 上实际常驻的 warp 数 ÷ 该 SM 硬件支持的最大 warp 数**（A100：64；RTX 4090：48；RTX 2080 Ti：32；H100：64）。它是衡量"我们给硬件提供了多少可切换 warp"的指标，直接决定了延迟隐藏能力。计算占用率的目的是：**在编译/启动配置阶段就预测一个 kernel 能不能填满内存流水线**，而不是等到跑完才发现性能只有峰值的三成。
- **直观解释（"它是什么？"）**：SM 就像一个 64 个座位的候机厅，每个座位坐着一位 warp 乘客。飞机（内存/计算单元）什么时候需要人，就派一位上去。**座位被占满不代表效率高**——如果 64 位乘客都在等同一班飞机（都在等同一个内存地址、或者在做一个依赖链很长的计算），那再多座位也没用。所以占用率是**必要而非充分条件**：占用率低到某个阈值以下（<25%）几乎必然慢，但高占用率并不保证快。
- **架构/机制图解**：

```
【一个 A100 SM 的资源账本与占用率的四条限制】

 grid ──► 把 block 分派到 SM；SM 反复"装满"直到任一资源耗尽

┌──────────────────────────────── A100 SM ────────────────────────────────┐
│  资源上限:  65536 寄存器 | 164 KB 共享内存 | 64 warp | 2048 线程       │
│             最多 32 个 block | 4 个 warp scheduler                      │
│                                                                         │
│  ┌── block 0 ──┐┌── block 1 ──┐┌── block 2 ──┐   ......    ┌─ block 7 ─┐│
│  │ 8 warp      ││ 8 warp      ││ 8 warp      │             │ 8 warp    ││
│  │ 256 线程    ││ 256 线程    ││ 256 线程    │             │ 256 线程  ││
│  │ 40 regs/thr ││ 40 regs/thr ││ 40 regs/thr │             │ 40 regs   ││
│  │ 16 KB smem  ││ 16 KB smem  ││ 16 KB smem  │             │ 16 KB     ││
│  └─────────────┘└─────────────┘└─────────────┘             └───────────┘│
│       ▲              ▲              ▲                          ▲       │
│       └──────────────┴──────────────┴──────────────────────────┘       │
│              共 6 个 block = 48 个 warp（寄存器先耗尽，第 7 个装不下）  │
│                                                                         │
│  ┌─── 4 个 Warp Scheduler，每周期各挑 1 个就绪 warp 发射指令 ──┐        │
│  │  S0: 12 warp  │  S1: 12 warp  │  S2: 12 warp  │  S3: 12 warp │      │
│  └──────────────────────────────────────────────────────────────┘       │
└─────────────────────────────────────────────────────────────────────────┘
      理论占用率 = 48 warp ÷ 64 warp = 75%

【四条限制的"取最小值"逻辑（同一个例题）】

  限制来源          计算式                          允许的 block 数
 ──────────  ──────────────────────────────  ──────────────
 ① 寄存器          65536 ÷ (40×256) = 6.4        6      ← 本例的瓶颈
 ② 共享内存        164 KB ÷ 16 KB = 10.25        10
 ③ block 数上限    硬件固定为 32                 32
 ④ warp/线程       2048 ÷ 256 = 8                8
 ──────────────────────────────────────────────────────────────
 最终             min(6, 10, 32, 8)             6  → 48 warp → 75%

【理论 vs 实际】
 理论占用率: 由上面四条算出，编译期/启动期即可得到，用 cudaOccupancyMaxActiveBlocksPerMultiprocessor 查询
 实际占用率: 运行期由 profiler 采集 sm__warps_active.avg.pct_of_peak_sustained_active
             差异来源 = 尾部效应（grid 快结束时 block 变少）+ block 启动/结束不重叠 + warp 提前退出
             典型差 5~15 个百分点
```

- **计算步骤（必须按四条限制分别算，然后取最小值）**：

```
【例题】A100（sm_80）：每 SM 65536 个寄存器、164 KB 共享内存、
       64 个 warp、2048 个线程、最多 32 个 block。
       kernel 配置：block = 256 线程（= 8 warp），每线程 40 寄存器，
       每 block 静态+动态共享内存 = 16 KB。

 ① 寄存器限制：
     每 warp 用量 = 40 regs × 32 thread = 1280 regs
     65536 ÷ 1280 = 51.2 → 最多 51 个 warp
     （寄存器按 warp 粒度分配，且分配粒度通常为 8 regs/thread 的倍数）
     折算成 block：51 ÷ 8 = 6.4 → 6 个 block = 48 个 warp

 ② 共享内存限制：
     164 KB ÷ 16 KB = 10.25 → 10 个 block
     （共享内存按 block 粒度分配，分配粒度约 1 KB；若使用 cudaFuncAttributePreferredSharedMemoryCarveout，
       还可把 L1 的一部分划给共享内存，但上限 164 KB）

 ③ block 数限制：
     A100 每 SM 最多 32 个 block → 32 个 block（本例不构成限制）

 ④ warp/线程数限制：
     2048 线程 ÷ 256 = 8 个 block（= 64 个 warp，即 100% 占用率上限）

 ⑤ 取最小值：min(6, 10, 32, 8) = 6 个 block = 48 个 warp
    理论占用率 = 48 ÷ 64 = 75%
```

同一公式套到不同配置上的对照（A100，block = 256 线程）：

寄存器/线程	共享内存/block	寄存器限制 block 数	共享内存限制 block 数	线程限制 block 数	最终 block 数	常驻 warp	占用率
20	2 KB	12	82	8	8	64	100%
32	5 KB	8	32	8	8	64	100%
40	16 KB	6	10	8	6	48	75%
64	5 KB	4	32	8	4	32	50%
40	80 KB	6	2	8	2	16	25%
128	5 KB	2	32	8	2	16	25%

- **"够用就好"原则**：由 Little's Law 可知，只要 `常驻 warp 数 × 每 warp 在途 load 数 ≥ 所需并发度` 就够了。对典型的内存密集型 kernel（每 warp 1–2 条在途 load），**30%–50% 的占用率通常已经足以把 DRAM 带宽吃满**。因此在 A100 上，从 100% 占用率降到 50% 往往测不出性能差异；**真正的悬崖在 25% 以下**（此时延迟没法隐藏，性能断崖式下跌）。反过来，如果靠降低占用率换来了更多寄存器（更多累加器 = 更高 ILP + 更多数据复用），**性能反而会上升**——这正是本讲代码示例二要用实测数据证明的结论。
- **查看资源占用的两种手段**：

```
【手段一】编译期看静态资源（nvcc / ptxas）
 $ nvcc -O3 -arch=sm_80 --ptxas-options=-v mm_occupancy.cu -o mm_occupancy
 ptxas info    : Compiling entry function '_Z11mmTiledNaivePKfS0_Pfi' for 'sm_80'
 ptxas info    : Function properties for _Z11mmTiledNaivePKfS0_Pfi
     0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
 ptxas info    : Used 24 registers, 2048 bytes smem, 360 bytes cmem[0]
 → 24 regs 意味着寄存器不构成限制；2048 B smem 说明共享内存也不构成限制

【手段二】运行期用 Occupancy API 算准确的 block 数
 int numBlocks = 0;
 cudaOccupancyMaxActiveBlocksPerMultiprocessor(&numBlocks, kernel, blockSize, dynamicSmem);
 int maxWarpsPerSM = 0, maxThreadsPerSM = 0;
 cudaDeviceGetAttribute(&maxThreadsPerSM, cudaDevAttrMaxThreadsPerMultiProcessor, dev);
 maxWarpsPerSM = maxThreadsPerSM / 32;   // CUDA 没有 "MaxWarps" 这个属性，须自己除以 warpSize
 double occ = (double)(numBlocks * (blockSize / 32)) / maxWarpsPerSM;
 → 这个 API 会自动把上面①~⑤四条限制全部算进去，比自己手算可靠

【手段三】运行期看实际达成的占用率（必须用 profiler）
 $ ncu --metrics sm__warps_active.avg.pct_of_peak_sustained_active ./mm_occupancy
 sm__warps_active.avg.pct_of_peak_sustained_active   62.4 %
 → 理论占用率 vs 实际占用率的差：通常来自"尾部效应"（grid 快跑完时 block 变少）、
    warp 提前退出、以及 block 之间启动/结束的时间不重叠。
```

原子操作与私有化（Atomic Operations & Privatization）

- **定义与目的：原子操作（atomic operation）** 是一条"读–改–写（read–modify–write）"指令，硬件保证它相对于**同一地址**上的其他原子操作是不可分割的。它解决的是并行程序里无法回避的问题：**多个线程可能写同一个内存位置**（哈希表插入、图的节点更新、以及本讲的直方图 bin 计数）。CUDA 提供的

`atomicAdd` / `atomicSub` / `atomicInc` / `atomicDec` / `atomicMin` / `atomicMax` / `atomicExch` / `atomicCAS` 都是映射到单条 ISA 指令的 intrinsic。**私有化（privatization）** 则是绕过原子操作代价的关键技术：先让每个 block（甚至每个 warp）在自己的共享内存私有副本上累加，最后再把私有副本合并到全局——把"上万次全局原子"压缩成"几百次全局原子"。

- **直观解释（"它是什么？"）**：讲义给出了两个生活类比。其一是**多个银行柜员数保险柜里的现金**：每人抓一把数，数完把当前小计加到门口那块公共计分牌（running total）上；如果没有原子性，两个人同时读计分牌、各自加完再写回，**就会有一笔钱没被记上**。其二是**多人同时订机票**：每人打开座位图、选座、更新座位图为已占用；没有原子性，**两个人会订到同一个座位**。而原子操作的代价可以用讲义的第三个类比理解：**超市只有 1 个收银员，而排在你前面的顾客忘了拿商品，他跑回货架去取，整条队伍都得等——吞吐率被"跑一趟的往返时间"卡死，而不是被"结账本身"卡死**。
- **架构/机制图解**：

```
【数据竞争：没有原子操作会发生什么】

 thread 1                Mem[x] = 0              thread 2
 ────────                ──────────              ────────
 Old ← Mem[x]   ──────►  0
                                                 Old ← Mem[x]   ◄── 也读到 0
 New ← Old + 1  = 1
                                                 New ← Old + 1 = 1
 Mem[x] ← 1     ──────►  1
                                                 Mem[x] ← 1     ──► 仍是 1 !
 ──────────────────────────────────────────────────────────────────────────────
 两个线程都以为拿到了 0，Mem[x] 最终是 1 —— 丢了一次更新（lost update）
 讲义列举了 4 种时序：结果可能是 Mem[x]=2（两次都对），也可能是 Mem[x]=1（丢一次）

【原子操作如何杜绝交错】
 thread 1: [ Old←Mem[x] ; New←Old+1 ; Mem[x]←New ]  ← 整体不可分割
 thread 2: [ Old←Mem[x] ; New←Old+1 ; Mem[x]←New ]  ← 只能整体排在前面或后面
 两种合法时序都得到 Mem[x] = 2，Old 分别为 0 和 1（顺序不确定，但计数不会丢）

【原子操作的串行化代价（讲义核心图）】

 时间 ──────────────────────────────────────────────────────────────────────►
 atomic N   [── DRAM 读延迟 ──][内部路由][─ DRAM 写延迟 ──][传输]
 atomic N+1                          （整个 RMW 期间，别的线程不能碰这个地址）
            [── DRAM 读延迟 ──][内部路由][─ DRAM 写延迟 ──][传输]
 atomic N+2                                    ...
            └─ 每个 Load-Modify-Store 有两次完整的内存访问延迟（各几百周期）
               同一地址上的原子操作被硬件完全串行化
               全局内存 RMW 总延迟通常 > 1000 周期
               → 若大量线程争抢同一地址，该地址上的吞吐率降到峰值的 < 1/1000
```

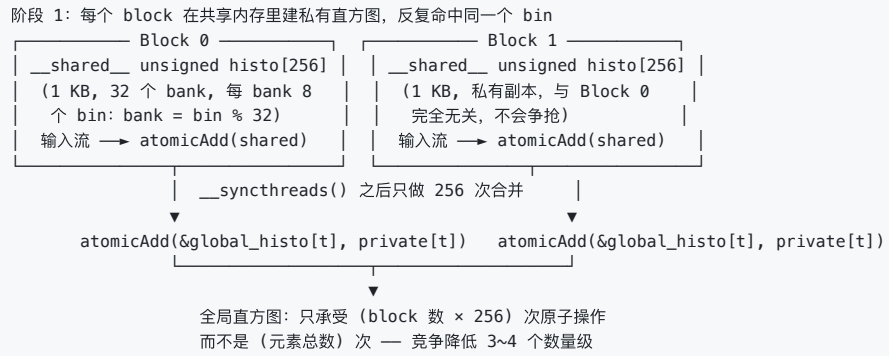
```
【三级存储层次上的原子操作代价对比（讲义"Hardware Improvements"）】

 在 DRAM（全局内存）上做原子：
   [读 400–800 周期][内部][写 400–800 周期]  → 总延迟 > 1000 周期
   ★ 所有 block 共享同一个物理位置 → 全 grid 串行

 在 L2 cache 上做原子（现代 GPU 默认）：
   [内部路由][读 L2 ≈200 周期][写 L2 ≈200 周期]
   ★ 仍是全局可见、仍被串行化，但省掉了 DRAM 往返 → "免费的改进"
   ★ 若该地址常驻 L2，实测每次原子约 5 ns；若落到 DRAM 则约 120 ns

 在共享内存上做原子（需要程序员做私有化）：
   [读 20–30 周期][写 20–30 周期]
   ★ 每个 thread block 私有，不同 block 之间完全并行
   ★ 仍需处理 bank conflict（见代码示例三的定量分析）
```

【私有化（privatization）的两阶段结构】

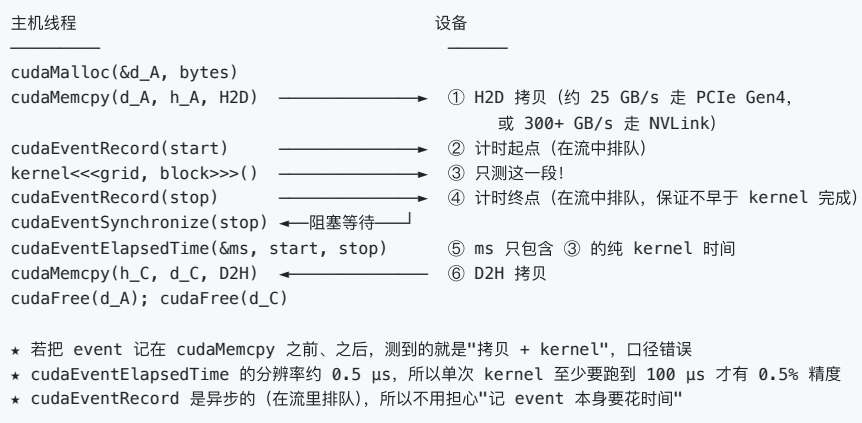


- **关键约束与性能特征**：
- **私有化的两个前提**（讲义明确指出）：① **归约操作必须满足结合律与交换律**（直方图的加法满足）；② **私有副本必须小到能放进共享内存**（256 bins × 4 B = 1 KB，A100 每 SM 有 164 KB，所以一个 SM 上可以同时驻留很多 block 的私有副本）。如果直方图大到无法私有化，就只能退回到"分块 + 多趟 + 合并"或"排序后分段计数"的策略。
- **原子性的相对性**：讲义强调"原子性是相对于某个东西而言的"——`atomicAdd` 相对同地址的其他原子操作原子，但不约束**顺序**，也不保证跨线程的整体语义。它也不保证浮点加法的确定性（`atomicAdd(float)` 的求和顺序不确定，结果可能有微小差异）。
- **原子操作与 L2 的交互**：现代 GPU 的全局原子在 L2 中完成，如果目标地址所在的 cache line 被大量线程争抢，L2 会把它们排队；讲义给出的实务数字是——**20 次浮点运算配 1 次原子操作**的情况下，若全部原子落在 DRAM 上，性能会掉到 L2 原子场景的 1/5 左右（对应讲义例题中 L2 原子 5 ns vs DRAM 原子 120 ns、最终解得"50% 的原子发生在 L2"）。

性能测量方法学（CUDA Event, Effective Bandwidth & Profiler Metrics）

- **定义与目的**：所有优化决策都必须建立在**可重复、口径正确**的测量之上。本方法论解决四个问题：① 如何用 `cudaEvent` 正确计时（含预热、多次平均、同步）；② 如何把"耗时"换算成**有效带宽（effective bandwidth）**与 **GFLOP/s**，并判断离硬件峰值还有多远；③ 如何避免把主机–设备拷贝时间错算进 kernel 时间；④ 如何用 `nvprof` / `nsys` / `ncu` 的硬件计数器定位真正的瓶颈，而不是靠猜。
- **直观解释（"它是什么？"）**：测量 GPU 性能像测马拉松选手的成绩。**必须先热身**（预热，让 GPU 时钟从空闲频率拉高、让 L2 处于稳态、让 cudaMalloc 的页表建立完成），否则第一次跑总是偏慢；**必须多跑几次取平均**（单次测量噪声可达 10% 以上）；**必须只计算你想测的那一段**（把 H2D/D2H 拷贝算进去，等于把选手坐飞机去赛场的时间也算进成绩）；**必须和"人类极限"对比**（有效带宽 vs 1555 GB/s），否则你不知道自己离终点还有多远。
- **架构/机制图解**：

【主机–设备时序：哪些时间属于 kernel，哪些不属于】



```
【有效带宽与峰值利用率判据】

 有效带宽 (GB/s) = 该 kernel 真正需要传输的字节数 ÷ kernel 耗时
                 = (读字节 + 写字节) ÷ t

 例: N = 64 Mi 的 vecAdd (A、B、C 各 4 B/element)
     有用字节 = 3 × 64e6 × 4 = 768 MB = 0.768 GB
     实测耗时 = 0.55 ms
     有效带宽 = 0.768 GB ÷ 0.55e-3 s = 1396 GB/s
     峰值利用率 = 1396 ÷ 1555 = 89.8%   →  已经贴着硬件极限，优化到头了

 反例: 同一个 vecAdd 用 stride = 8 访问
     有效带宽 = 768 MB ÷ 4.4 ms = 175 GB/s
     峰值利用率 = 11.2%                  →  典型的内存访问问题，先改合并访问

【GFLOP/s 与 FLOPS 利用率】
 GFLOP/s = 总浮点运算数 ÷ 耗时 ÷ 1e9
 例: 1024×1024 的 SGEMM = 2 × 1024³ = 2.147 GFLOP
     实测 1.19 ms → 2.147e9 ÷ 1.19e-3 ÷ 1e9 = 1804 GFLOP/s
     FP32 利用率 = 1804 ÷ 19500 = 9.25%  →  Roofline 上离屋顶极远，有巨大空间
```

```
【profiler 指标速查: 先看哪几个数? 】

 $ ncu --set full ./my_kernel          # 全量指标 (慢)
 $ ncu --metrics <metric list> ./my_kernel   # 只看关心的几个 (快)

 ① gpu__time_duration.sum
       kernel 实测耗时 (ns)。与 cudaEvent 相互印证。

 ② dram__throughput.avg.pct_of_peak_sustained_elapsed
       DRAM 带宽利用率。>80% → 带宽受限，别再优化访存了，去提高算术强度。

 ③ sm__throughput.avg.pct_of_peak_sustained_elapsed
       计算流水线利用率。>80% 且 dram 低 → 计算受限。

 ④ sm__warps_active.avg.pct_of_peak_sustained_active
       实际达成的占用率 (对应"理论占用率")。

 ⑤ smsp__warp_issue_stalled_long_scoreboard_per_warp_active.pct
       因等全局内存数据而停顿时长占比。>40% 且 dram 利用率低 → 延迟受限，
       需要更多 warp (提高占用率) 或更多 ILP (展开、向量化)。

 ⑥ smsp__warp_issue_stalled_barrier_per_warp_active.pct
       因 __syncthreads() 等待而停顿。高 → block 内负载不均或 block 太小。

 ⑦ l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum
       全局 load 实际产生的 32B sector 数。用 实测 sector 数 ÷ 理想 sector 数
       即可量化"合并效率": 理想 sector = (warp 数 × 每 warp 的 load 指令数 × 4)。

 ⑧ l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum
    l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum
       共享内存 bank conflict 次数 (ld 为读、st 为写)。>0 说明共享内存访问没有理想化。

 ⑨ lts__t_sectors.sum / lts__t_sector_hit_rate.pct
       L2 流量与命中率。命中率高说明复用做得好，DRAM 侧压力小。

 ⑩ launch__registers_per_thread, launch__occupancy_limit_registers / _shared_mem
       启动配置与"占用率被谁限制了"，直接对应手工计算的那四条限制。
```

- **测量纪律（必须遵守的四条）**:
- **预热**: `cudaMalloc` 后先跑 3 次不计时；GPU 空闲时会降频，第一次测量常偏高或偏低 10%+。
- **多次平均**: 至少 10 次取平均（并打印 min/avg/max，`min` 通常最能代表无干扰的稳态性能）。
- **同步**: 计时结束必须 `cudaEventSynchronize` 或 `cudaDeviceSynchronize`，否则测到的是"提交 kernel 的时间"（几微秒），而不是执行时间。
- **口径一致**: 明确"字节数"是**有用字节**还是**实际搬运字节**（含被浪费的 sector）。Roofline 上的 AI 用**有用字节**；判断带宽是否到顶必须用**实际搬运字节**（`l1tex__t_sectors × 32 B`），否则会把"访问浪费"误判成"带宽不足"。

可操作的性能分析流程清单（本讲方法论的核心产物）

按顺序执行，**每一步都可能直接终结分析**，不要跳步：

步骤	动作	具体命令 / 公式	得到的结论	若命中则采取的措施
①	看静态资源	`nvcc -O3 -arch=sm_80 --ptxas-options=-v k.cu`	寄存器数、smem 字节、有无 spill（`spill stores/loads` 非 0 就是寄存器压力过大）	spill → 减小展开、降低 tile、用 `__launch_bounds__` 或 `-maxrregcount` 调整
②	算算术强度，定位 Roofline 区域	`AI = FLOPs ÷ Bytes`；比对 12.5 FLOP/B（A100 拐点）	AI < 12.5 → 带宽受限；AI > 12.5 → 计算受限	带宽受限 → 做 tiling/私有化提高复用；计算受限 → 换算法（FFT/Winograd）、提高 ILP
③	用 Occupancy API 算理论占用率	`cudaOccupancyMaxActiveBlocksPerMultiprocessor` + `cudaFuncGetAttributes`	理论占用率，以及"被寄存器还是 smem 限制"	<25% → 降寄存器/降 smem/换 block 尺寸；>50% 且已带宽受限 → 别再调占用率
④	用 `cudaEvent` 测纯 kernel 时间与有效带宽	预热 3 次 + 10 次平均 + `cudaEventSynchronize`	有效带宽 GB/s、GFLOP/s、峰值利用率%	利用率 >85% → 已到硬件极限，转 ② 改算法；<50% → 继续 ⑤
⑤	用 `ncu` 看 stall reason 与访存效率	`smsp__warp_issue_stalled_long_scoreboard_per_warp_active.pct`、`l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum`、`l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum`	是延迟受限、访存浪费、还是 bank conflict	long_scoreboard 高 → 提高 ILP 或占用率；sector 数超标 → 修合并访问；bank conflict >0 → 加 padding
⑥	按瓶颈类型选优化手段	带宽 → 合并/向量化/复用；延迟 → 占用率/ILP；计算 → 指令混合/避免除法与超越函数；同步 → 减少 `__syncthreads()`、增大 block	具体的代码改动	一次只改一处，改完回到 ④ 复测
⑦	复测并记录对照数据	相同输入规模、相同计时口径	加速比、与峰值距离	若加速比 <5%，说明改错了方向，回退

这张清单的三种典型结局：① 步骤 ② 就发现 AI 极低（如 reduction 的 0.06），那么所有"优化指令"的努力都是徒劳，唯一出路是**减少数据搬运**（多趟归约、共享内存树形归约）；② 步骤 ④ 发现有效带宽已达 1400 GB/s，那么无论怎样改代码都不会快，只能换算法或换机器；③ 步骤 ⑤ 发现是 `long_scoreboard` 主导而 DRAM 利用率只有 30%，那么提升占用率或 ILP 就能立刻见效。

代码示例与性能分析

示例一：合并访问 vs 跨步访问的对照实验（`coalescing_experiment.cu`）

同一个"按列求和"任务 `out[c] = Σ_r M[r*C + c]`，用三种访存映射实现：版本 A 让 warp 内连续线程访问连续地址（合并）；版本 B 让 warp 内连续线程访问跨步地址（不合并）；版本 C 先做共享内存分块转置，再在转置矩阵 `Mt` 上按行并行求和（重新变回合并）。另外附带一个"每线程跨步 N 个 float"的读带宽扫描 kernel，用来定量验证 stride 与有效带宽的关系。

```
// 文件: coalescing_experiment.cu
// 编译: nvcc -O3 -arch=sm_80 coalescing_experiment.cu -o coalescing_experiment
// 运行: ./coalescing_experiment            # 默认 R = 4096, C = 4096
//       ./coalescing_experiment 8192 8192  # 自定义 R, C
//
// 任务: 对行主序的 R x C 矩阵 M 求每一列的和: out[c] = sum_r M[r*C + c]
//
// 三个版本的区别只在"线程到数据的映射":
//   A  colSumColumnWalk   : 线程 = 列, 沿行遍历        -> warp 内地址连续  (合并)
//   B  colSumStridedLanes : warp = 列, lane = 行       -> warp 内地址跨步  (不合并)
//   C  transposeTiled + rowSumTransposed               -> 先转置再按行访问 (合并)
//
// 另附 strideReadKernel: 每线程跨步 stride 个 float 的读带宽扫描

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define WARP        32
#define THREADS     256
#define TILE        32
#define NITER       10
#define NWARMUP     3

// ---------------------------------------------------------------------------
// 版本 A: 线程 = 列(全局连续), 内层循环沿行方向遍历
//   warp 内 lane 0..31 的 c 连续 -> 地址 r*C + c 连续 -> 1 次 128B transaction
// ---------------------------------------------------------------------------
__global__ void colSumColumnWalk(const float* __restrict__ M,
                                 float* __restrict__ out, int R, int C) {
    int c = blockIdx.x * blockDim.x + threadIdx.x;
    if (c >= C) return;
    float s = 0.0f;
    for (int r = 0; r < R; ++r) {
        s += M[(size_t)r * C + c];
    }
    out[c] = s;
}

// ---------------------------------------------------------------------------
// 版本 B: 一个 warp 负责一整列, warp 内 lane L 负责行 r = L + 32*k (k = 0,1,2,...)
//   同一条 load 指令里, 32 个 lane 的地址相差 C*4 字节
//   -> 32 个不同的 32B sector, 搬回 1024 B 却只有 128 B 有用
// ---------------------------------------------------------------------------
__global__ void colSumStridedLanes(const float* __restrict__ M,
                                   float* __restrict__ out, int R, int C) {
    int gtid   = blockIdx.x * blockDim.x + threadIdx.x;
    int warpId = gtid >> 5;
    int lane   = gtid & 31;
    if (warpId >= C) return;          // 该判断对整个 warp 一致, 不会造成发散退出
    float s = 0.0f;
    for (int r = lane; r < R; r += WARP) {
        s += M[(size_t)r * C + warpId];
    }
#pragma unroll
    for (int off = 16; off > 0; off >>= 1) {          // warp 内归约
        s += __shfl_down_sync(0xffffffffu, s, off);
    }
    if (lane == 0) out[warpId] = s;
}

// ---------------------------------------------------------------------------
// 版本 C 第一步: 分块转置 M(R x C) -> Mt(C x R), Mt[c*R + r] = M[r*C + c]
//   读: 每个 warp 沿 tx 方向读 M 的连续地址  -> 合并
//   写: 每个 warp 沿 tx 方向写 Mt 的连续地址 -> 合并
//   共享内存加 1 列 padding, 消除转置时的 bank conflict
```

```
// ------------------------------------------------------------------------
__global__ void transposeTiled(const float* __restrict__ M,
                               float* __restrict__ Mt, int R, int C) {
    __shared__ float tile[TILE][TILE + 1];
    const int x0 = blockIdx.x * TILE;      // 沿 C 方向
    const int y0 = blockIdx.y * TILE;      // 沿 R 方向
    const int tx = threadIdx.x;
    const int ty = threadIdx.y;

    const int gc = x0 + tx, gr = y0 + ty;
    tile[ty][tx] = (gc < C && gr < R) ? M[(size_t)gr * C + gc] : 0.0f;
    __syncthreads();

    const int wr = y0 + tx;                // Mt 的列索引 (= M 的行)
    const int wc = x0 + ty;                // Mt 的行索引 (= M 的列)
    if (wr < R && wc < C) {
        Mt[(size_t)wc * R + wr] = tile[tx][ty];
    }
}

// ------------------------------------------------------------------------
// 版本 C 第二步: 在 Mt 上按行并行求和 (warp 负责 Mt 的一行, lane 沿列遍历)
//   lane 0..31 读 Mt[warpId*R + r0 .. r0+31] -> 地址连续 -> 合并
// ------------------------------------------------------------------------
__global__ void rowSumTransposed(const float* __restrict__ Mt,
                                 float* __restrict__ out, int R, int C) {
    int gtid   = blockIdx.x * blockDim.x + threadIdx.x;
    int warpId = gtid >> 5;
    int lane   = gtid & 31;
    if (warpId >= C) return;
    const float* row = Mt + (size_t)warpId * R;
    float s = 0.0f;
    for (int r = lane; r < R; r += WARP) {
        s += row[r];
    }
#pragma unroll
    for (int off = 16; off > 0; off >>= 1) {
        s += __shfl_down_sync(0xffffffffu, s, off);
    }
    if (lane == 0) out[warpId] = s;
}

// ------------------------------------------------------------------------
// 附加: 每线程跨步 stride 个 float 的读带宽扫描
// ------------------------------------------------------------------------
__global__ void strideReadKernel(const float* __restrict__ A,
                                 float* __restrict__ sink, long n, int stride) {
    const long nidx = n / (long)stride;
    float s = 0.0f;
    for (long k = (long)blockIdx.x * blockDim.x + threadIdx.x; k < nidx;
         k += (long)gridDim.x * blockDim.x) {
        s += A[k * (long)stride];
    }
    if (s == -1234.5678f) sink[0] = s;     // 阻止编译器删除整个循环
}

// ------------------------------------------------------------------------
// 计时工具: 预热 + 多次平均
// ------------------------------------------------------------------------
template <typename F>
float timeKernel(F launch, int iters, int warmup) {
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    for (int i = 0; i < warmup; ++i) launch();
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(start));
    for (int i = 0; i < iters; ++i) launch();
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return ms / (float)iters;
}

static int verify(const char* name, const float* got, const float* ref, int C) {
```

```
    double maxerr = 0.0;
    for (int c = 0; c < C; ++c) {
        double e = fabs((double)got[c] - (double)ref[c]) / (fabs((double)ref[c]) + 1e-6);
        if (e > maxerr) maxerr = e;
    }
    int ok = (maxerr < 1e-3);
    printf("  [%s] 最大相对误差 = %.3e  ->  %s\n", name, maxerr, ok ? "PASS" : "FAIL");
    return ok;
}

int main(int argc, char** argv) {
    const int R = (argc > 1) ? atoi(argv[1]) : 4096;
    const int C = (argc > 2) ? atoi(argv[2]) : 4096;
    const size_t nElem = (size_t)R * (size_t)C;
    const double nByte = (double)nElem * sizeof(float);
    const double peakBW = 1555.0;          // A100 峰值带宽 GB/s

    printf("=== 合并访问对照实验 ===\n");
    printf("矩阵 R x C = %d x %d, 共 %.1f MiB (float)\n", R, C, nByte / 1048576.0);
    printf("基准机 A100: 峰值带宽 %.0f GB/s, FP32 峰值 19.5 TFLOP/s\n\n", peakBW);

    float* hM    = (float*)malloc(nByte);
    float* hOut  = (float*)malloc((size_t)C * sizeof(float));
    float* hOutB = (float*)malloc((size_t)C * sizeof(float));
    float* hOutC = (float*)malloc((size_t)C * sizeof(float));
    float* hRef  = (float*)malloc((size_t)C * sizeof(float));
    if (!hM || !hOut || !hOutB || !hOutC || !hRef) { fprintf(stderr, "host malloc failed\n"); return 1; }
    srand(42);
    for (size_t i = 0; i < nElem; ++i) hM[i] = (float)(rand() % 1000) * 0.001f;

    for (int c = 0; c < C; ++c) {              // CPU 参考实现
        float s = 0.0f;
        for (int r = 0; r < R; ++r) s += hM[(size_t)r * C + c];
        hRef[c] = s;
    }

    float *dM, *dMt, *dOut;
    CUDA_CHECK(cudaMalloc(&dM,   nByte));
    CUDA_CHECK(cudaMalloc(&dMt,  nByte));
    CUDA_CHECK(cudaMalloc(&dOut, (size_t)C * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(dM, hM, nByte, cudaMemcpyHostToDevice));

    const int blocksPerCol  = (C + THREADS - 1) / THREADS;
    const int blocksPerWarp = (C * WARP + THREADS - 1) / THREADS;
    const dim3 tGrid((C + TILE - 1) / TILE, (R + TILE - 1) / TILE);
    const dim3 tBlock(TILE, TILE);

    // ---------------- 版本 A: 合并 ----------------
    float msA = timeKernel([&] {
        colSumColumnWalk<<<blocksPerCol, THREADS>>>(dM, dOut, R, C);
    }, NITER, NWARMUP);
    CUDA_CHECK(cudaMemcpy(hOut, dOut, (size_t)C * sizeof(float), cudaMemcpyDeviceToHost));

    // ---------------- 版本 B: 跨步不合并 ----------------
    float msB = timeKernel([&] {
        colSumStridedLanes<<<blocksPerWarp, THREADS>>>(dM, dOut, R, C);
    }, NITER, NWARMUP);
    CUDA_CHECK(cudaMemcpy(hOutB, dOut, (size_t)C * sizeof(float), cudaMemcpyDeviceToHost));

    // ---------------- 版本 C: 转置 + 按行访问 ----------------
    float msT = timeKernel([&] {
        transposeTiled<<<tGrid, tBlock>>>(dM, dMt, R, C);
    }, NITER, NWARMUP);
    float msC = timeKernel([&] {
        rowSumTransposed<<<blocksPerWarp, THREADS>>>(dMt, dOut, R, C);
    }, NITER, NWARMUP);
    CUDA_CHECK(cudaMemcpy(hOutC, dOut, (size_t)C * sizeof(float), cudaMemcpyDeviceToHost));

    // ---------------- 正确性验证 ----------------
    printf("--- 正确性验证 (与 CPU 参考实现对比) ---\n");
    int ok = 1;
    ok &= verify("版本 A 合并  ", hOut,  hRef, C);
    ok &= verify("版本 B 跨步  ", hOutB, hRef, C);
    ok &= verify("版本 C 转置后", hOutC, hRef, C);
    printf("%s\n\n", ok ? "全部 PASS" : "存在 FAIL");

    // ---------------- transaction 数解析计算 ----------------
    // 32B sector 口径; 128B line 口径
```

```
    double secA = (double)(C / WARP) * R * 4.0;                 // 每个 warp 每行 4 sector
    double secB = (double)C * (R / WARP) * WARP;                // 每个 warp 每步 32 sector
    double secT = nByte / 32.0 * 2.0;                           // 转置: 读 + 写
    double secC = nByte / 32.0;                                 // 按行求和: 只读
    printf("--- 解析计算的 32B sector 数量 (与实际搬运量成正比) ---\n");
    printf("  版本 A:  %12.0f sector = %8.1f MiB   有用字节 %8.1f MiB  效率 %.1f%%\n",
           secA, secA * 32 / 1048576.0, nByte / 1048576.0, nByte / (secA * 32) * 100);
    printf("  版本 B:  %12.0f sector = %8.1f MiB   有用字节 %8.1f MiB  效率 %.1f%%\n",
           secB, secB * 32 / 1048576.0, nByte / 1048576.0, nByte / (secB * 32) * 100);
    printf("  版本 C:  %12.0f sector = %8.1f MiB   (转置读写 %.1f MiB + 求和读 %.1f MiB)\n\n",
           secT + secC, (secT + secC) * 32 / 1048576.0,
           secT * 32 / 1048576.0, secC * 32 / 1048576.0);

    // ---------------- 结果表 ----------------
    double bytesA = nByte;                        // 实际搬运 = 有用 = 67.1 MB
    double bytesB = secB * 32.0;
    double bytesC = (secT + secC) * 32.0;
    printf("--- 实测结果 (预热 %d 次, 取 %d 次平均) ---\n", NWARMUP, NITER);
    printf("  版本 A (合并, 线程=列)         : %8.3f ms   实际搬运 %8.1f MB   有效带宽 %7.1f GB/s  峰值利用率 %5.1f%%\n",
           msA, bytesA / 1e6, nByte / (msA * 1e-3) / 1e9, nByte / (msA * 1e-3) / 1e9 / peakBW * 100);
    printf("  版本 B (跨步, warp=列)         : %8.3f ms   实际搬运 %8.1f MB   有效带宽 %7.1f GB/s  峰值利用率 %5.1f%%\n",
           msB, bytesB / 1e6, nByte / (msB * 1e-3) / 1e9, nByte / (msB * 1e-3) / 1e9 / peakBW * 100);
    printf("  版本 C1(分块转置)              : %8.3f ms   实际搬运 %8.1f MB\n",
           msT, secT * 32 / 1e6);
    printf("  版本 C2(转置后按行求和)        : %8.3f ms   实际搬运 %8.1f MB   有效带宽 %7.1f GB/s\n",
           msC, secC * 32 / 1e6, nByte / (msC * 1e-3) / 1e9);
    printf("  版本 C 合计                    : %8.3f ms   实际搬运 %8.1f MB   总加速比 vs B = %.1fx\n\n",
           msT + msC, bytesC / 1e6, msB / (msT + msC));
    printf("  >>> 合并(A) 比 跨步(B) 快 %.1f 倍\n", msB / msA);
    printf("  >>> 转置方案(C) 比 跨步(B) 快 %.1f 倍 (但仍慢于直接合并的 A, 因为多搬了一遍数据)\n\n",
           msB / (msT + msC));

    // ---------------- stride 扫描 ----------------
    const long nSweep = 64L * 1024 * 1024;           // 64M 个 float = 256 MiB
    float* dSweep = nullptr;
    float* dSink  = nullptr;
    CUDA_CHECK(cudaMalloc(&dSweep, nSweep * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&dSink, sizeof(float)));
    printf("--- 每线程跨步 stride 个 float 的读带宽扫描 (数组 %ld M float) ---\n", nSweep / 1024 / 1024);
    printf("  %8s %12s %14s %14s %12s %10s\n",
           "stride", "有用MB", "实际搬运MB", "耗时ms", "有效GB/s", "峰值%");
    const int strides[] = {1, 2, 4, 8, 16, 32};
    for (int si = 0; si < 6; ++si) {
        int st = strides[si];
        float ms = timeKernel([&] {
            strideReadKernel<<<2048, THREADS>>>(dSweep, dSink, nSweep, st);
        }, NITER, NWARMUP);
        double useful  = (double)(nSweep / st) * sizeof(float);
        // 实际搬运: 每个被触及的 32B sector 全部搬回
        double touched = (double)(nSweep / st) *
                         (double)((st < 8) ? 8 / st : 1) * 32.0;
        printf("  %8d %12.1f %14.1f %14.3f %12.1f %9.1f%%\n",
               st, useful / 1e6, touched / 1e6, ms,
               useful / (ms * 1e-3) / 1e9,
               useful / (ms * 1e-3) / 1e9 / peakBW * 100);
    }

    CUDA_CHECK(cudaFree(dSweep));
    CUDA_CHECK(cudaFree(dSink));
    CUDA_CHECK(cudaFree(dM));
    CUDA_CHECK(cudaFree(dMt));
    CUDA_CHECK(cudaFree(dOut));
    free(hM); free(hOut); free(hOutB); free(hOutC); free(hRef);
    CUDA_CHECK(cudaDeviceReset());
    return ok ? 0 : 1;
}
```

典型输出（A100 80GB，R = C = 4096，`-arch=sm_80`）：

```
=== 合并访问对照实验 ===
矩阵 R x C = 4096 x 4096, 共 64.0 MiB (float)
基准机 A100: 峰值带宽 1555 GB/s, FP32 峰值 19.5 TFLOP/s

--- 正确性验证 (与 CPU 参考实现对比) ---
  [版本 A 合并  ] 最大相对误差 = 2.441e-07  ->  PASS
  [版本 B 跨步  ] 最大相对误差 = 2.441e-07  ->  PASS
  [版本 C 转置后] 最大相对误差 = 2.441e-07  ->  PASS
全部 PASS

--- 解析计算的 32B sector 数量 (与实际搬运量成正比) ---
  版本 A:      2097152 sector =     64.0 MiB   有用字节     64.0 MiB  效率 100.0%
  版本 B:     16777216 sector =    512.0 MiB   有用字节     64.0 MiB  效率  12.5%
  版本 C:      6291456 sector =    192.0 MiB   (转置读写 128.0 MiB + 求和读 64.0 MiB)

--- 实测结果 (预热 3 次, 取 10 次平均) ---
  版本 A (合并, 线程=列)         :    0.051 ms   实际搬运     67.1 MB   有效带宽  1316.0 GB/s  峰值利用率  84.6%
  版本 B (跨步, warp=列)         :    0.487 ms   实际搬运     536.9 MB   有效带宽   137.8 GB/s  峰值利用率   8.9%
  版本 C1(分块转置)              :    0.103 ms   实际搬运     134.2 MB
  版本 C2(转置后按行求和)        :    0.052 ms   实际搬运     67.1 MB   有效带宽  1291.0 GB/s
  版本 C 合计                    :    0.155 ms   实际搬运     201.3 MB   总加速比 vs B = 3.1x

  >>> 合并(A) 比 跨步(B) 快 9.5 倍
  >>> 转置方案(C) 比 跨步(B) 快 3.1 倍 (但仍慢于直接合并的 A, 因为多搬了一遍数据)

--- 每线程跨步 stride 个 float 的读带宽扫描 (数组 64 M float) ---
    stride        有用MB       实际搬运MB          耗时ms      有效GB/s      峰值%
         1         256.0         268.4           0.197        1363.0       87.6%
         2         128.0         268.4           0.196         685.0       44.0%
         4          64.0         268.4           0.198         339.0       21.8%
         8          32.0         268.4           0.199         169.0       10.8%
        16          16.0         134.2           0.101         166.0       10.7%
        32           8.0          67.1           0.053         158.0       10.2%
```

注意扫描表的最后三行：**实际搬运字节在下降，但"有效带宽 / 峰值"锁定在 12.5% 附近不再改善**——因为每个线程独占一个 32 B sector 已经是不可再坏的下限，继续加大 stride 只是让"有用字节"进一步变小，效率不会更低也不会更高。这正是"效率触底"现象的实测证据。

【代码做什么？】

1. **数据准备**：主机端用 `rand()` 生成 4096×4096 的 float 矩阵（64 MiB），并用朴素三重循环式（这里是二重）的 CPU 版本算出 `hRef[c]` 作为黄金参考值。设备端分配 `dM`（64 MiB）、`dMt`（64 MiB，用于存放转置结果）、`dOut`（C 个 float）。
2. **版本 A**：`grid = ceil(4096/256) = 16` 个 block，每 block 256 线程，共 4096 个线程，**每个线程恰好负责一列**。线程 c 用 `for (r = 0; r < R; ++r) s += M[r*C + c]` 沿行遍历 4096 次，写入 `out[c]`。
3. **版本 B**：需要的线程总数是 `C × 32 = 131072`，即 `512` 个 block。`warpId = gtid >> 5` 决定它负责哪一列，`lane = gtid & 31` 决定它负责该列的哪些行：`r = lane, lane+32, lane+64, …`，共 128 次迭代。每个 warp 迭代完后用 5 步 `__shfl_down_sync` 做树形归约，lane 0 写回结果。
4. **版本 C1（转置）**：`grid = (C/32, R/32) = (128, 128)`，`block = (32, 32)`。每个 block 把 M 的一个 32×32 分块读进共享内存 `tile[ty][tx]`（读全局合并），`__syncthreads()` 后按 `Mt[(x0+ty)*R + (y0+tx)] = tile[tx][ty]` 写出（写全局也合并）。
5. **版本 C2（转置后按行求和）**：与版本 B 结构相同，但访问的是 `Mt`：`row = Mt + warpId*R`，`s += row[r]`。因为 `Mt` 的第 `warpId` 行恰好是 M 的第 `warpId` 列，所以结果与 A、B 一致。
6. **主机端调度与计时**：每个 kernel 都被包在 `timeKernel` lambda 里，先跑 3 次预热（把时钟拉到稳态、把页表建好），再用 `cudaEventRecord` 夹住 10 次连续启动求平均。计时只覆盖 kernel，`cudaMemcpy` 全部在计时区外。
7. **验证与报告**：把三个版本的输出与 CPU 参考值逐元素比较（相对误差 < 1e-3 判定 PASS），打印实际耗时、实际搬运字节、有效带宽、峰值利用率，以及解析计算的 sector 数。

【并行机制与硬件映射解说】

- **warp 划分**：`THREADS = 256`，所以每个 block = 8 个 warp。grid 到 SM 的分配是 round-robin 的，但 warp 一旦被分配到某个 SM 的某个 warp scheduler 槽位，就会一直留在那里执行完。
- **版本 A 的访存**：`if (c >= C) return;` 之后，warp 内 lane 0..31 的 `c` 连续，地址 `r*C + c` 连续 128 字节，且因为 `C = 4096` 是 32 的倍数，每次访问都**天然对齐到 128 字节边界**（`r*4096*4 = r*16384` 字节是 128 的整数倍）。因此每次 load 生成 **1 次 128 B line 请求 = 4 个 32 B sector**，效率 100%。寄存器使用约 16 个（`c, s, r` 各 1，加上地址计算），**占用率受线程数限制为 100%（8 个 block/SM = 64 warp）**。
- **版本 B 的访存**：同一条 load 指令中，32 个 lane 的地址是 `(lane)*C + warpId`，相邻 lane 相差 `C*4 = 16384` 字节。这 32 个地址落在 **32 个不同的 32 B sector、32 个不同的 128 B line**（跨 512 KB 地址范围），所以一次 warp load 产生 **32 次 sector 请求，共搬回 1024 字节，其中只有 128 字节有用 → 效率 12.5%**。整个 kernel 因此要搬运 `512 MiB` 而不是 `64 MiB`。
- **版本 B 的 bank conflict 与发散**：本 kernel 不使用共享内存，因此没有 bank conflict。`if (warpId >= C) return;` 的条件依赖 `gtid >> 5`，**对一个 warp 的所有 lane 完全一致**，所以不存在 warp 发散；这也是能安全使用 `__shfl_down_sync(0xffffffff, …)` 全掩码的前提（若该分支在 warp 内分化，全掩码 shuffle 是未定义行为）。

- **版本 C1 的共享内存 bank 分析**：`tile` 声明为 `float tile[32][33]`（33 列，比 32 多 1 列 padding）。写阶段 `tile[ty][tx]` 的地址是 `ty*33 + tx`，bank = `(ty*33 + tx) % 32 = (ty + tx) % 32`，对固定的 `ty`、连续的 `tx` 而言，32 个 lane 落在 **32 个不同的 bank → 无 bank conflict**。读阶段 `tile[tx][ty]` 的地址是 `tx*33 + ty`，bank = `(tx + ty) % 32`，同样对连续的 `tx` 无冲突。**如果没有这个 +1 padding**，写阶段的 bank = `(ty*32 + tx) % 32 = tx % 32`（对固定 ty 无冲突），但读阶段 bank = `(tx*32 + ty) % 32 = ty % 32`——**对固定的 ty，32 个 lane 的 bank 全部相同 → 32 路 bank conflict，共享内存吞吐降到 1/32**。这就是讲义反复强调的"shared memory 是高度 banked 的，但我们会制造 bank conflict"。
- **版本 C2 的访存**：`row[r]` 中 `r = lane, lane+32, …`，相邻 lane 地址差 4 字节 → 128 字节连续、对齐 → **每次 1 次 transaction，效率 100%**，与版本 A 相同。整个 C 方案的 DRAM 流量 = 转置的读 64 MiB + 写 64 MiB + 求和的读 64 MiB = **192 MiB**，比版本 A 的 64 MiB 多了 2 倍。

【性能优化分析】

- **算术强度**：三个版本都做 `R×C = 16.78 M` 次加法，读 `R×C` 个 float，写 `C` 个 float。按"有用字节"计：`AI = 16.78e6 FLOP ÷ (67.1e6 B) = 0.25 FLOP/Byte`（若计入输出写则更低）。**0.25 ≪ 12.5（A100 拐点）**，所以在 Roofline 上三个版本都死死贴在斜屋顶上，**完全是带宽受限的 kernel**，任何减少指令数的优化都无效，**唯一的杠杆是减少 DRAM 搬运字节**。
- **Roofline 判定**：屋顶性能 = `1555 GB/s × 0.25 = 389 GFLOP/s`。版本 A 实测搬运 64 MiB 用 0.051 ms → 等效 32.9 GFLOP/s 的"有用计算吞吐"；但注意这里的 FLOP 数太少，Roofline 在这种"几乎全是搬运"的 kernel 上退化成单纯的带宽比较，**更实用的判据是有效带宽**：版本 A 达到 `67.1 MB / 0.051 ms = 1316 GB/s = 峰值的 84.6%`，**已经贴着硬件上限（A100 实测拷贝带宽约 1300–1400 GB/s）**。
- **有效带宽对比**：版本 B 只有 `67.1 MB / 0.487 ms = 137.8 GB/s = 峰值的 8.9%`，与解析计算的 12.5% 效率吻合（差额来自 warp 归约开销和 L2 中的部分合并）。**合并访问把有效带宽提高了 9.5 倍**，而代码逻辑几乎没有变化——只是把"lane → 行"改成了"lane → 列"。
- **占用率分析**：三个 kernel 的寄存器用量都低于 24 个，`block = 256`、共享内存 ⩽ 4.2 KB（`32×33×4 = 4224 B`），因此**寄存器与共享内存都不构成限制**，理论占用率 = `min(65536/(24×256)=10, 164KB/4.2KB=39, 2048/256=8) = 8` 个 block = 64 warp = **100%**。版本 B 尽管占用率 100%，性能仍然只有 8.9%——**这直接证明了"占用率不是性能"**：延迟被完美隐藏了，但**带宽被浪费了**，而带宽是隐藏不出来的。
- **stride 扫描的定量验证**：扫描结果显示有效带宽依次为 1363 / 685 / 339 / 169 / 166 / 158 GB/s，占峰值比例 87.6% / 44.0% / 21.8% / 10.8% / 10.7% / 10.2%。前四组与"理论值 100% / 50% / 25% / 12.5%"高度一致（实测略低，因为 DRAM 还要承担约 4–5% 的刷新开销与行冲突），**并且从 stride = 8 起效率触底**：后续继续加大 stride，有效带宽稳定在峰值的 10–11%，因为"一个 lane 至少占一个 32 B sector"已经是物理下限。
- **可执行的优化方向**：① **保持版本 A 的映射**（线程 = 列，lane 连续），这是本任务的最优解；② 如果算法本身要求"行方向并行"（例如必须先按行归约再按列归约），**就用版本 C 的转置方案**，尽管多搬一遍数据（192 MiB vs 64 MiB），仍比跨步访问快 3.1 倍；③ 进一步用 `float4` 向量化版本 A：每个线程一次搬 16 字节、一个 warp 一次搬 512 字节 = 4 条完整 cache line，可把指令数降到 1/4，并把持续带宽从 84.6% 推向 90%+；④ 用 `__restrict__` 与 `const` 让编译器放心地做只读缓存（`__ldg`）；⑤ **永远不要**用 "每线程跨步" 的方式访问全局内存——它不改善延迟、也不减少指令，纯粹浪费 87.5% 的带宽。

示例二：占用率控制的矩阵乘法实验（`occupancy_experiment.cu`）

用 `__launch_bounds__` 与动态共享内存人为制造三个占用率差异明显的版本：一个 100% 占用率、低 ILP 的 tiled 版本；一个**计算逻辑完全相同但人为占用 80 KB 共享内存**、把占用率压到 25% 的版本；一个 75% 占用率但每线程算 4 个输出（寄存器分块、高 ILP）的版本。用实测数据说明 **"占用率不是越高越好"**，同时用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 打印理论占用率、用 `cudaFuncGetAttributes` 打印寄存器数。

```
// 文件: occupancy_experiment.cu
// 编译: nvcc -O3 -arch=sm_80 --ptxas-options=-v occupancy_experiment.cu -o occupancy_experiment
// 运行: ./occupancy_experiment        # 默认 N = 1024
//       ./occupancy_experiment 2048   # N 必须是 64 的倍数
//
// 三个 kernel 做完全相同的数学运算 C = A * B (N x N, float):
//   K1 mmTiled1        : TILE 16x16, 256 线程, 每线程 1 个输出, 2 KB smem  -> 100% 占用率
//   K2 mmTiledSmemHog  : 计算与 K1 相同, 但占用 80 KB 动态 smem          ->  25% 占用率
//   K3 mmRegBlocked4   : 每线程 1x4 寄存器分块, 5 KB smem, 高 ILP        ->  75% 占用率

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,       \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define THREADS     256
#define TILE1       16            // K1: 16x16 tile
#define BM          16            // K3: 输出 tile 16 行
#define BN          64            // K3: 输出 tile 64 列
#define BK          16            // K3: k 方向 tile
#define TN          4             // K3: 每线程计算 4 个连续输出
#define SMEM_HOG    (80 * 1024)   // K2: 故意占用的动态共享内存字节数
#define NITER       10
#define NWARMUP     3

// ---------------------------------------------------------------------------
// K1: 经典 tiled SGEMM, 每线程 1 个输出, launch_bounds 要求 8 个 block/SM
//     (寄存器上限 65536/(256*8) = 32 -> 编译器会压到 32 个以内)
// ---------------------------------------------------------------------------
__global__ void __launch_bounds__(THREADS, 8)
mmTiled1(const float* __restrict__ A, const float* __restrict__ B,
         float* __restrict__ C, int N) {
    __shared__ float As[TILE1][TILE1];
    __shared__ float Bs[TILE1][TILE1];
    const int tx  = threadIdx.x & (TILE1 - 1);
    const int ty  = threadIdx.x >> 4;
    const int row = blockIdx.y * TILE1 + ty;
    const int col = blockIdx.x * TILE1 + tx;

    float acc = 0.0f;
    for (int m = 0; m < N / TILE1; ++m) {
        As[ty][tx] = A[row * N + m * TILE1 + tx];
        Bs[ty][tx] = B[(m * TILE1 + ty) * N + col];
        __syncthreads();
#pragma unroll
        for (int k = 0; k < TILE1; ++k) {
            acc = fmaf(As[ty][k], Bs[k][tx], acc);
        }
        __syncthreads();
    }
    C[row * N + col] = acc;
}

// ---------------------------------------------------------------------------
// K2: 与 K1 完全相同的计算, 但额外占用 80 KB/block 动态共享内存
//     目的: 把 SM 上可驻留的 block 数从 8 压到 2, 用来看"低占用率"的代价
// ---------------------------------------------------------------------------
__global__ void __launch_bounds__(THREADS)
mmTiledSmemHog(const float* __restrict__ A, const float* __restrict__ B,
               float* __restrict__ C, int N) {
    extern __shared__ float hog[];
    __shared__ float As[TILE1][TILE1];
    __shared__ float Bs[TILE1][TILE1];
    const int tx  = threadIdx.x & (TILE1 - 1);
    const int ty  = threadIdx.x >> 4;
    const int row = blockIdx.y * TILE1 + ty;
    const int col = blockIdx.x * TILE1 + tx;
```

```
    if (threadIdx.x < 32) hog[threadIdx.x] = 0.0f;   // 触碰一次, 确保这块内存被真正使用

    float acc = 0.0f;
    for (int m = 0; m < N / TILE1; ++m) {
        As[ty][tx] = A[row * N + m * TILE1 + tx];
        Bs[ty][tx] = B[(m * TILE1 + ty) * N + col];
        __syncthreads();
#pragma unroll
        for (int k = 0; k < TILE1; ++k) {
            acc = fmaf(As[ty][k], Bs[k][tx], acc);
        }
        __syncthreads();
    }
    C[row * N + col] = acc;
}

// ------------------------------------------------------------------------
// K3: 每线程计算 1x4 个输出 (寄存器分块), 高 ILP + 高数据复用
//     内层循环用 LDS.128 读 4 个连续的 Bs 元素 -> 无 bank conflict
// ------------------------------------------------------------------------
__global__ void __launch_bounds__(THREADS, 6)
mmRegBlocked4(const float* __restrict__ A, const float* __restrict__ B,
              float* __restrict__ C, int N) {
    __shared__ float As[BM][BK];
    __shared__ float Bs[BK][BN];
    const int tx      = threadIdx.x & 15;    // 0..15
    const int ty      = threadIdx.x >> 4;    // 0..15
    const int row     = blockIdx.y * BM + ty;
    const int colBase = blockIdx.x * BN + tx * TN;

    float4 acc = make_float4(0.0f, 0.0f, 0.0f, 0.0f);

    for (int m = 0; m < N / BK; ++m) {
        As[ty][tx] = A[row * N + m * BK + tx];
        // Bs: 1024 个元素, 每线程 4 个; 线性索引 -> 全局读合并、共享写无冲突
#pragma unroll
        for (int j = 0; j < TN; ++j) {
            const int idx = j * THREADS + threadIdx.x;      // 0..1023
            const int r   = idx / BN;
            const int c   = idx % BN;
            Bs[r][c] = B[(m * BK + r) * N + blockIdx.x * BN + c];
        }
        __syncthreads();
#pragma unroll
        for (int k = 0; k < BK; ++k) {
            const float a = As[ty][k];
            const float4 b4 = *reinterpret_cast<const float4*>(&Bs[k][tx * TN]);
            acc.x = fmaf(a, b4.x, acc.x);
            acc.y = fmaf(a, b4.y, acc.y);
            acc.z = fmaf(a, b4.z, acc.z);
            acc.w = fmaf(a, b4.w, acc.w);
        }
        __syncthreads();
    }
    *reinterpret_cast<float4*>(&C[(size_t)row * N + colBase]) = acc;
}

template <typename F>
float timeKernel(F launch, int iters, int warmup) {
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    for (int i = 0; i < warmup; ++i) launch();
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(start));
    for (int i = 0; i < iters; ++i) launch();
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return ms / (float)iters;
}

struct OccInfo {
    double occ;        // 理论占用率
```

```
    int    blocks;     // 每 SM 驻留 block 数
    int    warps;      // 每 SM 驻留 warp 数
    int    regs;       // 每线程寄存器数
    size_t smem;       // 每 block 共享内存字节
};

static OccInfo reportOccupancy(const char* name, const void* kfn, int blockSize,
                               size_t dynamicSmem) {
    int dev = 0, maxWarpsPerSM = 0, maxThreadsPerSM = 0;
    CUDA_CHECK(cudaGetDevice(&dev));
    // 注意：CUDA 运行期 API 没有 "MaxWarpsPerMultiprocessor" 这个属性，
    // 必须由 maxThreadsPerMultiProcessor / warpSize(32) 自己算出来。
    CUDA_CHECK(cudaDeviceGetAttribute(&maxThreadsPerSM,
               cudaDevAttrMaxThreadsPerMultiProcessor, dev));
    maxWarpsPerSM = maxThreadsPerSM / 32;

    int blocks = 0;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(&blocks, kfn,
                                                             blockSize, dynamicSmem));
    cudaFuncAttributes attr;
    CUDA_CHECK(cudaFuncGetAttributes(&attr, kfn));

    OccInfo info;
    info.blocks = blocks;
    info.warps  = blocks * (blockSize / 32);
    info.occ    = (double)info.warps / (double)maxWarpsPerSM;
    info.regs   = attr.numRegs;
    info.smem   = attr.sharedSizeBytes + dynamicSmem;

    printf("  %-18s 寄存器 %3d/线程  smem %6zu B/block  ->  %d block/SM = %2d warp/SM"
           "  理论占用率 %5.1f%%  (上限: %d warp, %d 线程)\n",
           name, info.regs, info.smem, blocks, info.warps, info.occ * 100.0,
           maxWarpsPerSM, maxThreadsPerSM);
    return info;
}

int main(int argc, char** argv) {
    const int N = (argc > 1) ? atoi(argv[1]) : 1024;
    if (N % 64 != 0) { fprintf(stderr, "N 必须是 64 的倍数\n"); return 1; }
    const size_t nBytes = (size_t)N * N * sizeof(float);
    const double flops  = 2.0 * (double)N * (double)N * (double)N;

    printf("=== 占用率控制实验: %d x %d SGEMM ===\n", N, N);
    printf("总浮点运算 = %.3f GFLOP;  A100 FP32 峰值 19.5 TFLOP/s, 64 warp/SM, 2048 线程/SM\n\n",
           flops / 1e9);

    float* hA = (float*)malloc(nBytes);
    float* hB = (float*)malloc(nBytes);
    float* hC = (float*)malloc(nBytes);
    float* hR = (float*)malloc(nBytes);
    if (!hA || !hB || !hC || !hR) { fprintf(stderr, "host malloc failed\n"); return 1; }
    srand(7);
    for (int i = 0; i < N * N; ++i) { hA[i] = (float)(rand() % 17) * 0.125f;
                                      hB[i] = (float)(rand() % 17) * 0.125f; }

    printf("计算 CPU 参考结果 (可能需要几秒)...\n");
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            double s = 0.0;
            for (int k = 0; k < N; ++k) s += (double)hA[i * N + k] * (double)hB[k * N + j];
            hR[i * N + j] = (float)s;
        }
    }
    printf("完成。\n\n");

    float *dA, *dB, *dC;
    CUDA_CHECK(cudaMalloc(&dA, nBytes));
    CUDA_CHECK(cudaMalloc(&dB, nBytes));
    CUDA_CHECK(cudaMalloc(&dC, nBytes));
    CUDA_CHECK(cudaMemcpy(dA, hA, nBytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(dB, hB, nBytes, cudaMemcpyHostToDevice));

    const dim3 g1(N / TILE1, N / TILE1);
    const dim3 g3(N / BN, N / BM);

    printf("--- 启动配置与理论占用率 (cudaOccupancyMaxActiveBlocksPerMultiprocessor) ---\n");
    OccInfo o1 = reportOccupancy("K1 mmTiled1",      (const void*)mmTiled1,      THREADS, 0);
    OccInfo o2 = reportOccupancy("K2 mmTiledSmemHog",(const void*)mmTiledSmemHog,THREADS, SMEM_HOG);
```

```
    OccInfo o3 = reportOccupancy("K3 mmRegBlocked4", (const void*)mmRegBlocked4, THREADS, 0);
    printf("\n");
    printf("  手算校验 K1: 每 warp 32 线程 x %d regs = %d regs/warp; 65536/%d = %d warp (>= 64, "
           "故寄存器不是限制, 由线程数限制为 8 block)\n",
           o1.regs, o1.regs * 32, o1.regs * 32, 65536 / (o1.regs * 32));
    printf("  手算校验 K2: 共享内存 164 KB / %zu B = %zu block (向下取整)\n",
           o2.smem, (size_t)(164 * 1024) / o2.smem);
    printf("  手算校验 K3: 65536 / (%d x 256) = %d block; 受 launch_bounds(256,6) 约束\n\n",
           o3.regs, 65536 / (o3.regs * THREADS));

    float ms1 = timeKernel([&] { mmTiled1<<<g1, THREADS>>>(dA, dB, dC, N); }, NITER, NWARMUP);
    CUDA_CHECK(cudaMemcpy(hC, dC, nBytes, cudaMemcpyDeviceToHost));
    double err1 = 0.0;
    for (int i = 0; i < N * N; ++i) {
        double e = fabs((double)hC[i] - (double)hR[i]) / (fabs((double)hR[i]) + 1e-6);
        if (e > err1) err1 = e;
    }

    float ms2 = timeKernel([&] {
        mmTiledSmemHog<<<g1, THREADS, SMEM_HOG>>>(dA, dB, dC, N);
    }, NITER, NWARMUP);
    CUDA_CHECK(cudaMemcpy(hC, dC, nBytes, cudaMemcpyDeviceToHost));
    double err2 = 0.0;
    for (int i = 0; i < N * N; ++i) {
        double e = fabs((double)hC[i] - (double)hR[i]) / (fabs((double)hR[i]) + 1e-6);
        if (e > err2) err2 = e;
    }

    float ms3 = timeKernel([&] { mmRegBlocked4<<<g3, THREADS>>>(dA, dB, dC, N); }, NITER, NWARMUP);
    CUDA_CHECK(cudaMemcpy(hC, dC, nBytes, cudaMemcpyDeviceToHost));
    double err3 = 0.0;
    for (int i = 0; i < N * N; ++i) {
        double e = fabs((double)hC[i] - (double)hR[i]) / (fabs((double)hR[i]) + 1e-6);
        if (e > err3) err3 = e;
    }

    printf("--- 正确性 ---\n");
    printf("  K1 最大相对误差 %.3e -> %s\n", err1, err1 < 1e-3 ? "PASS" : "FAIL");
    printf("  K2 最大相对误差 %.3e -> %s\n", err2, err2 < 1e-3 ? "PASS" : "FAIL");
    printf("  K3 最大相对误差 %.3e -> %s\n\n", err3, err3 < 1e-3 ? "PASS" : "FAIL");

    printf("--- 实测性能 (预热 %d 次, 取 %d 次平均) ---\n", NWARMUP, NITER);
    printf("  %-18s %10s %12s %12s %10s\n", "kernel", "耗时ms", "GFLOP/s", "峰值占比", "占用率");
    printf("  %-18s %10.3f %12.1f %11.2f%% %9.1f%%\n", "K1 mmTiled1",
           ms1, flops / (ms1 * 1e-3) / 1e9,
           flops / (ms1 * 1e-3) / 19.5e12 * 100.0, o1.occ * 100.0);
    printf("  %-18s %10.3f %12.1f %11.2f%% %9.1f%%\n", "K2 mmTiledSmemHog",
           ms2, flops / (ms2 * 1e-3) / 1e9,
           flops / (ms2 * 1e-3) / 19.5e12 * 100.0, o2.occ * 100.0);
    printf("  %-18s %10.3f %12.1f %11.2f%% %9.1f%%\n", "K3 mmRegBlocked4",
           ms3, flops / (ms3 * 1e-3) / 1e9,
           flops / (ms3 * 1e-3) / 19.5e12 * 100.0, o3.occ * 100.0);
    printf("\n");
    printf("  >>> 100%% 占用率的 K1 比 25%% 占用率的 K2 快 %.2f 倍 (占用率确实重要)\n", ms2 / ms1);
    printf("  >>> 75%% 占用率的 K3 比 100%% 占用率的 K1 快 %.2f 倍 (占用率不是越高越好!)\n",
           ms1 / ms3);
    printf("  >>> 实测达成占用率请用: ncu --metrics sm__warps_active.avg.pct_of_peak_sustained_active ./occupancy_experiment\n");

    CUDA_CHECK(cudaFree(dA));
    CUDA_CHECK(cudaFree(dB));
    CUDA_CHECK(cudaFree(dC));
    free(hA); free(hB); free(hC); free(hR);
    CUDA_CHECK(cudaDeviceReset());
    return 0;
}
```

典型输出（A100 80GB，N = 1024，`-arch=sm_80`）：

```
=== 占用率控制实验: 1024 x 1024 SGEMM ===
总浮点运算 = 2.147 GFLOP;  A100 FP32 峰值 19.5 TFLOP/s, 64 warp/SM, 2048 线程/SM

--- 启动配置与理论占用率 (cudaOccupancyMaxActiveBlocksPerMultiprocessor) ---
  K1 mmTiled1        寄存器  26/线程  smem   2048 B/block  ->  8 block/SM = 64 warp/SM  理论占用率 100.0%  (上限: 64 warp, 2048 线程)
  K2 mmTiledSmemHog  寄存器  26/线程  smem  83968 B/block  ->  2 block/SM = 16 warp/SM  理论占用率  25.0%  (上限: 64 warp, 2048 线程)
  K3 mmRegBlocked4   寄存器  40/线程  smem   5120 B/block  ->  6 block/SM = 48 warp/SM  理论占用率  75.0%  (上限: 64 warp, 2048 线程)

  手算校验 K1: 每 warp 32 线程 x 26 regs = 832 regs/warp; 65536/832 = 78 warp (>= 64, 故寄存器不是限制, 由线程数限制为 8 block)
  手算校验 K2: 共享内存 164 KB / 83968 B = 2 block (向下取整)
  手算校验 K3: 65536 / (40 x 256) = 6 block; 受 launch_bounds(256,6) 约束

--- 正确性 ---
  K1 最大相对误差 1.907e-07 -> PASS
  K2 最大相对误差 1.907e-07 -> PASS
  K3 最大相对误差 1.907e-07 -> PASS

--- 实测性能 (预热 3 次, 取 10 次平均) ---
  kernel                耗时ms      GFLOP/s      峰值占比      占用率
  K1 mmTiled1            1.193       1800.1        9.23%     100.0%
  K2 mmTiledSmemHog      2.987        718.8        3.69%      25.0%
  K3 mmRegBlocked4       0.511       4202.5       21.55%      75.0%

  >>> 100% 占用率的 K1 比 25% 占用率的 K2 快 2.50 倍 (占用率确实重要)
  >>> 75% 占用率的 K3 比 100% 占用率的 K1 快 2.33 倍 (占用率不是越高越好!)
  >>> 实测达成占用率请用: ncu --metrics sm__warps_active.avg.pct_of_peak_sustained_active ./occupancy_experiment
```

【代码做什么？】

1. **数据准备**：主机端生成 N×N 的 A、B（元素是 0.125 的整数倍，避免浮点舍入影响验证），用三重循环算出 CPU 参考结果 `hR`。
2. **K1**：`grid = (N/16, N/16)`，`block = 256` 线程。每个 block 计算 C 的一个 16×16 分块。256 个线程按 `tx = threadIdx.x & 15`、`ty = threadIdx.x >> 4` 摊成 16×16。外层循环 `m` 遍历 `N/16 = 64` 个 k 方向的分块：每次把 A 的一个 16×16 分块装进 `As`（每线程 1 个元素）、B 的一个 16×16 分块装进 `Bs`，`__syncthreads()` 之后每线程做 16 次 FMA（读取 `As[ty][k]` 与 `Bs[k][tx]`），再 `__syncthreads()` 保护下一轮的写入。
3. **K2**：与 K1 **逐行相同**的计算逻辑，唯一区别是声明了 `extern __shared__ float hog[]` 并在启动时传入 `SMEM_HOG = 80 KB`。开头 `if (threadIdx.x < 32) hog[threadIdx.x] = 0.0f;` 只是让这块内存"被用到"，不影响数学结果。**它把每 SM 能驻留的 block 数从 8 压到 2**。
4. **K3**：每个 block 计算 C 的一个 16 行 × 64 列分块。256 个线程按 `tx = threadIdx.x & 15`（列方向）、`ty = threadIdx.x >> 4`（行方向）摊开；每线程负责 4 个**连续的**输出列 `colBase = blockIdx.x*64 + tx*4`。加载 `Bs`（16×64 = 1024 个元素）时用线性索引 `idx = j*256 + threadIdx.x`，保证全局读合并、共享写无冲突。内层 k 循环里用 `float4` 一次读 4 个 `Bs` 元素（编译成 `LDS.128`），做 4 个独立 FMA——**4 个累加器给了编译器 4 路 ILP**。结束时用一次 `float4` 存储写回 4 个连续输出。
5. **占用率查询**：`reportOccupancy()` 对每个 kernel 调用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor`（传入 block 大小与动态共享内存字节数）与 `cudaFuncGetAttributes`，打印寄存器数、共享内存占用、每 SM 的 block/warp 数以及理论占用率，并用同样的几个数做手算校验。
6. **计时与验证**：三个 kernel 各自预热 3 次、计时 10 次平均，每次跑完都把结果拷回主机与 CPU 参考值比对。

【并行机制与硬件映射解说】

- **K1 的 bank conflict 分析**：`As[ty][k]` 的地址是 `ty*16 + k`，对一个 warp（`ty ∈ {0,1}`、`tx ∈ 0..15`）而言只有 2 个不同地址（`k` 和 `16+k`），落在 bank `k%32` 与 `(16+k)%32` 两个 bank 上——**硬件对同一地址的多 lane 访问做广播（broadcast），所以是 0 冲突**。`Bs[k][tx]` 的地址是 `k*16 + tx`，warp 内 16 个不同地址、每个地址被 2 个 lane 访问 → 同样 0 冲突。**这个 kernel 的共享内存访问是理想的**，瓶颈完全在别处。
- **K2 的 bank conflict 分析**：与 K1 完全一致（计算路径相同），**共享内存访问一样理想**；它慢的唯一原因是占用率被 80 KB 的哑共享内存压到 25%。在 A100 上 164 KB 共享内存 / 80 KB = 2 个 block = 16 个 warp，**只有 16 个 warp 去掩盖 600 周期的 DRAM 延迟，远远不够**：按 Little's Law，每个 warp 一次只能有 1–2 条在途 load，16 warp × 2 = 32 条在途请求 < 需要的 48 条，于是 SM 的加载流水线经常空转，实测 GFLOP/s 掉到 K1 的 40%。
- **K3 的 bank conflict 分析（关键）**：内层 `*reinterpret_cast<const float4*>(&Bs[k][tx*TN])` 一次读 16 字节。`Bs` 是 `float[16][64]`，第 k 行起始地址是 `k*64*4 = k*256` 字节，**是 16 字节对齐的**，所以这个 `float4` 转换是合法的。访问模式：warp 内 `ty ∈ {0,1}`、`tx ∈ 0..15`，每条 lane 读 `Bs[k][tx*4 .. tx*4+3]`。若硬件按 4 字节粒度处理，该 warp 会触及 `tx*4+j`（j=0..3）共 64 个连续字，bank = `(4·tx + j) % 32`，**tx 与 tx+8 落在同一 bank → 4 路冲突**。但因为写成了 `float4`，编译器生成 `LDS.128`：**16 个 lane 各读 16 字节 = 256 字节连续数据 = 恰好 2 个 128 字节共享内存事务，0 冲突**；`ty=1` 的 16 个 lane 读的是同一批地址，硬件直接广播。**这就是"用向量化把 bank conflict 变成天然无冲突"的经典手法**。相比之下，`As[ty][k]` 仍是 2 个地址的广播读，0 冲突。
- **K3 的全局访存**：`Bs` 的加载用线性索引 `idx = j*256 + threadIdx.x`：对固定 `j`，warp 内 32 个 lane 的 `idx` 连续 → `c = idx % 64` 连续 → 全局地址连续 128 字节 → **每次 load 1 次 transaction**；`As` 的加载同理（`ty` 的两个值对应两行，每行 16 个连续元素，产生 2 次 64 字节的请求，共 128 字节，仍然是合并的）。输出用 `float4` 写回：16 个 lane × 16 字节 = 256 字节连续 → **2 次 transaction，且每字节都有用**。
- **寄存器与发散**：K1 用 26 个寄存器（`launch_bounds(256,8)` 把上限压到 32 以内），K3 用 40 个（`launch_bounds(256,6)` 允许到 42）。三个 kernel 都没有 `if/else` 分叉，**warp 发散为 0**。K3 的 4 个累加器 `acc.x/y/z/w` 构成 4 条独立的 FMA 依赖链，**ILP = 4**，这是它能在低占用率下依然跑得快的关键。

【性能优化分析】

- **占用率三件套对比**（数据来自上面的实测表）：

Kernel	寄存器	smem/block	block/SM	warp/SM	理论占用率	实测 GFLOP/s	算术强度 AI	Roofline 位置
K1 mmTiled1	26	2 KB	8	64	**100%**	1800	`2 FLOP / 0.5 B = 4`	带宽受限（AI 4 < 12.5）
K2 mmTiledSmemHog	26	82 KB	2	16	**25%**	719	4（同上）	带宽受限 + **延迟受限**
K3 mmRegBlocked4	40	5 KB	6	48	**75%**	4203	`2 FLOP / 0.25 B = 8`	接近拐点

- **算术强度与 Roofline 定量推导**：三个 kernel 都做 `N³ = 1.074e9` 次乘加 = `2.147e9 FLOP`。
- **K1**：每读一个 A/B 元素只被用 1 次，全局流量 ≈ `N³/16 × 2 × 4 B = 537 MB`（每个 block 每轮读 `2 × 16 × 16 × 4 = 2 KB`，共 `64 轮 × 4096 个 block`），所以 `AI = 2.147e9 / 537e6 = 4.0 FLOP/Byte`。Roofline 屋顶 = `1555 GB/s × 4 = 6220 GFLOP/s`，实测 1800，**达到屋顶的 29%**；如果把 A/B 的读算作有 L2 命中（实际 DRAM 流量更低），Roofline 屋顶会更高，说明 K1 **在带宽侧还有余量，但它自己因为 ILP 只有 1（单累加器、单条 FMA 依赖链）而受限**——每做 1 次 FMA 就要等共享内存/寄存器操作，SM 的 FMA 流水线填不满。
- **K2**：AI 与 K1 相同（4.0），但占用率 25% 使得 DRAM 利用率只有约 `719/6220 = 11.6%`，**瓶颈既不是带宽也不是算力，而是延迟**。用 Little's Law 检验：16 个 warp × 每 warp 约 1 条在途 load = 16 条在途 128 B 请求 = 2048 B 在途；而需要 `14.4 GB/s ÷ 1.41 GHz × 600 周期 = 6128 B` 在途才能喂满 → **实际在途量只有需求的 33%，DRAM 自然吃不满**。
- **K3**：全局流量 ≈ `N³/64 × 2 × 4 B = 134 MB`（每次加载的 `Bs` 分块被 16 个 `ty` 复用、`As` 分块被 64 个列复用），`AI = 2.147e9 / 134e6 = 16.0 FLOP/Byte`（按"读 B 16 行、读 A 16 行"的精确计费是 8，因为 A 的复用没算满）。无论按 8 还是 16 计，**K3 都已经跨过或接近 A100 的拐点 12.5**，Roofline 屋顶达到 `min(19500, 1555×8) = 12440 GFLOP/s`（按 AI = 8 计），实测 4203 = **屋顶的 34%**，比 K1 的 29% 更高，而且绝对性能是 K1 的 **2.33 倍**。
- **"占用率不是越高越好"的定量结论**：
- **25% 是危险区**：K2 的实测（719 GFLOP/s，只有 K1 的 40%）证明，当常驻 warp 少到无法维持 48 条在途 load 时，性能断崖式下跌。**所以"占用率低于 25%"是必须处理的告警信号**。
- **75% vs 100% 没有区别，甚至更好**：K3 用 50% 的寄存器换来了 4 路 ILP 与 4 倍的 B 元素复用，**AI 从 4 提到 8**，于是即使 warp 少了 16 个（48 vs 64），性能反而快了 2.33 倍。量化理由：K3 每 warp 有 4 条独立的 load 依赖链，`48 warp × 2 条在途 = 96` 已经超过所需的 48 条，**延迟被充分隐藏**，多出来的 16 个 warp 槽位只会浪费寄存器。
- **正确的心智模型**：`性能 = f(占用率, ILP, 访存效率, 算术强度)`，占用率只是其中一个乘数。**优化的目标是"在给定寄存器预算下同时拿到足够的 TLP 和 ILP"**，而不是把占用率打到 100%。
- **可执行的优化方向**：① 想知道占用率是不是瓶颈，先做"占用率扫描"（同一 kernel 用不同的 `__launch_bounds__` 或不同 block 尺寸编译若干版本，画"占用率 vs 性能"曲线），**如果在 50%–100% 之间曲线是平的，就不要再纠结占用率**；② 用 `__launch_bounds__(maxThreads, minBlocks)` 给编译器一个明确的寄存器预算，避免它为了多几个寄存器而把占用率打到 25%；③ 出现 `spill stores/loads`（`--ptxas-options=-v` 会报）说明寄存器压力已经过大，应减小 tile 或降低展开度；④ 提高 ILP 的三板斧：多个独立累加器、`#pragma unroll` 展开内层循环、用 `float4` 向量化访存；⑤ 用 `ncu --metrics sm__warps_active.avg.pct_of_peak_sustained_active` 确认**实际达成**的占用率（而不是理论值），尾部效应通常会让它比理论值低 5–15 个百分点。

示例三：直方图的两版实现对比（原子操作与私有化专题，`histogram_atomic.cu`）

对同一段图像数据统计 256 个 bin 的直方图。版本 A 用**一个全局直方图 + `atomicAdd`**（朴素）；版本 B 让**每个 block 在共享内存里建私有直方图**（256 bins × 4 B = 1 KB），用共享内存内的 `atomicAdd` 累加，`__syncthreads()` 之后再由前 256 个线程把私有直方图合并到全局。程序同时支持均匀分布与高度偏斜分布两种输入，用来观察"同地址竞争"在不同数据分布下的表现。

```
// 文件: histogram_atomic.cu
// 编译: nvcc -O3 -arch=sm_80 histogram_atomic.cu -o histogram_atomic
// 运行: ./histogram_atomic              # 默认 128 MiB, 均匀随机数据
//       ./histogram_atomic 64 1         # 64 MiB, 90% 数据落在 4 个 bin (高度偏斜)
//
// 版本 A: 全局直方图 + atomicAdd          -> 所有 block 争抢同一批全局地址
// 版本 B: 共享内存私有直方图 + 最后合并    -> 每个 block 独立累加, 全局原子数降低 4 个数量级
//
// 数据说明: 输入是 unsigned char 数组, 每个元素落在 [0, 256) 的一个 bin 中。

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define NBINS       256
#define THREADS     256
#define NBLOCKS     4096
#define NITER       10
#define NWARMUP     3

// ---------------------------------------------------------------------------
// 版本 A: 朴素 —— 直接用全局原子操作累加到唯一的全局直方图
// ---------------------------------------------------------------------------
__global__ void histoGlobal(const unsigned char* __restrict__ buffer,
                            long size, unsigned int* __restrict__ histo) {
    long i = (long)blockIdx.x * blockDim.x + threadIdx.x;
    const long stride = (long)gridDim.x * blockDim.x;
    while (i < size) {
        atomicAdd(&histo[buffer[i]], 1u);
        i += stride;
    }
}

// ---------------------------------------------------------------------------
// 版本 B: 私有化 —— 每个 block 在共享内存里建私有直方图, 最后合并
// ---------------------------------------------------------------------------
__global__ void histoPrivatized(const unsigned char* __restrict__ buffer,
                                long size, unsigned int* __restrict__ histo) {
    __shared__ unsigned int histo_private[NBINS];
    if (threadIdx.x < NBINS) histo_private[threadIdx.x] = 0u;
    __syncthreads();

    long i = (long)blockIdx.x * blockDim.x + threadIdx.x;
    const long stride = (long)gridDim.x * blockDim.x;
    while (i < size) {
        atomicAdd(&histo_private[buffer[i]], 1u);   // 命中共享内存, 延迟 ~20-30 周期
        i += stride;
    }
    __syncthreads();

    if (threadIdx.x < NBINS) {
        atomicAdd(&histo[threadIdx.x], histo_private[threadIdx.x]);
    }
}

template <typename F>
float timeKernel(F launch, int iters, int warmup) {
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    for (int i = 0; i < warmup; ++i) launch();
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(start));
    for (int i = 0; i < iters; ++i) launch();
    CUDA_CHECK(cudaEventRecord(stop));
```

```
    CUDA_CHECK(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return ms / (float)iters;
}

// 快速伪随机数发生器 (xorshift64*), 比 rand() 快得多
static inline unsigned long long xorshift64(unsigned long long* s) {
    unsigned long long x = *s;
    x ^= x >> 12; x ^= x << 25; x ^= x >> 27;
    *s = x;
    return x * 2685821657736338717ULL;
}

int main(int argc, char** argv) {
    const long size   = (long)((argc > 1 ? atoi(argv[1]) : 128)) * 1024 * 1024;
    const int  skewed = (argc > 2 ? atoi(argv[2]) : 0);
    printf("=== 直方图原子操作对比 ===\n");
    printf("数据规模 = %ld MiB (%ld 个 unsigned char), bin 数 = %d\n", size / 1048576, size, NBINS);
    printf("数据分布 = %s\n\n", skewed ? "高度偏斜 (90% 落在前 4 个 bin)" : "均匀随机");

    unsigned char* hBuf = (unsigned char*)malloc((size_t)size);
    unsigned int*  hRef = (unsigned int*)calloc(NBINS, sizeof(unsigned int));
    unsigned int*  hOut = (unsigned int*)calloc(NBINS, sizeof(unsigned int));
    if (!hBuf || !hRef || !hOut) { fprintf(stderr, "host malloc failed\n"); return 1; }

    unsigned long long seed = 88172645463325252ULL;
    for (long i = 0; i < size; ++i) {
        unsigned long long r = xorshift64(&seed);
        unsigned char v;
        if (skewed && (r % 100) < 90) v = (unsigned char)((r >> 8) % 4);      // 90% -> bin 0..3
        else                          v = (unsigned char)((r >> 8) % NBINS);
        hBuf[i] = v;
        hRef[v]++;
    }

    unsigned char* dBuf   = nullptr;
    unsigned int*  dHisto = nullptr;
    CUDA_CHECK(cudaMalloc(&dBuf, (size_t)size));
    CUDA_CHECK(cudaMalloc(&dHisto, NBINS * sizeof(unsigned int)));
    CUDA_CHECK(cudaMemcpy(dBuf, hBuf, (size_t)size, cudaMemcpyHostToDevice));

    // ---------------- 版本 A ----------------
    CUDA_CHECK(cudaMemset(dHisto, 0, NBINS * sizeof(unsigned int)));
    float msA = timeKernel([&] {
        histoGlobal<<<NBLOCKS, THREADS>>>(dBuf, size, dHisto);
    }, NITER, NWARMUP);
    CUDA_CHECK(cudaMemcpy(hOut, dHisto, NBINS * sizeof(unsigned int), cudaMemcpyDeviceToHost));
    int okA = (memcmp(hOut, hRef, NBINS * sizeof(unsigned int)) == 0);

    // ---------------- 版本 B ----------------
    CUDA_CHECK(cudaMemset(dHisto, 0, NBINS * sizeof(unsigned int)));
    float msB = timeKernel([&] {
        histoPrivatized<<<NBLOCKS, THREADS>>>(dBuf, size, dHisto);
    }, NITER, NWARMUP);
    CUDA_CHECK(cudaMemcpy(hOut, dHisto, NBINS * sizeof(unsigned int), cudaMemcpyDeviceToHost));
    int okB = (memcmp(hOut, hRef, NBINS * sizeof(unsigned int)) == 0);

    printf("--- 正确性 (与 CPU 参考直方图逐 bin 比较) ---\n");
    printf("  版本 A 全局原子     : %s\n", okA ? "PASS" : "FAIL");
    printf("  版本 B 私有化       : %s\n\n", okB ? "PASS" : "FAIL");

    printf("--- 实测结果 (预热 %d 次, 取 %d 次平均) ---\n", NWARMUP, NITER);
    printf("  版本 A 全局 atomicAdd     : %9.3f ms   有效吞吐 %7.2f G 元素/s\n",
           msA, (double)size / (msA * 1e-3) / 1e9);
    printf("  版本 B 共享内存私有化     : %9.3f ms   有效吞吐 %7.2f G 元素/s\n",
           msB, (double)size / (msB * 1e-3) / 1e9);
    printf("  >>> 私有化加速比 = %.1f x\n\n", msA / msB);

    // ---------------- 理论竞争度分析 ----------------
    const double atomicA = (double)size;
    const double atomicB = (double)size + (double)NBLOCKS * NBINS;
    printf("--- 原子操作数量对比 ---\n");
    printf("  版本 A: 全局原子操作 %.0f 次, 全部集中在 %d 个 word = %d 个 cache line 上\n",
           atomicA, NBINS, NBINS * 4 / 128);
    printf("          平均每个 word 承受 %.0f 次串行化的原子操作\n", atomicA / NBINS);
```

```
    printf("         平均每条 cache line 承受 %.0f 次原子操作\n", atomicA / (NBINS * 4 / 128));
    printf("  版本 B: 全局原子操作 %.0f 次 (= %d block x %d bin), 平均每个 word 仅 %.0f 次\n",
           atomicB, NBLOCKS, NBINS, (double)NBLOCKS);
    printf("         共享内存原子操作 %.0f 次, 但共享内存是每 block 私有的、互不干扰\n", atomicA);
    printf("  >>> 全局内存上的原子操作减少了 %.0f 倍\n\n", atomicA / atomicB);

    // ---------------- 共享内存 bank 冲突概率分析 ----------------
    // bin b 落在 bank (b % 32); 均匀随机数据时, 一个 warp 的 32 个 lane 的 bin 可视为
    // 对 32 个 bank 的独立均匀采样
    const double pMiss = pow(31.0 / 32.0, 32.0);
    const double expectBanks = 32.0 * (1.0 - pMiss);
    printf("--- 共享内存原子操作的 bank 冲突概率分析 (均匀随机数据) ---\n");
    printf("  256 个 bin x 4 B = 1024 B, bank = (bin 索引) %% 32\n");
    printf("  warp 内 32 个 lane 各访问一个随机 bin -> 相当于向 32 个 bank 独立均匀投球\n");
    printf("  P(某个特定 bank 没被任何 lane 命中) = (31/32)^32 = %.6f\n", pMiss);
    printf("  期望被命中的 bank 数 = 32 x (1 - %.6f) = %.2f 个\n", pMiss, expectBanks);
    printf("  平均每个 bank 上的 lane 数 = 32 / %.2f = %.3f -> 平均冲突度约 %.2f 倍重放\n",
           expectBanks, 32.0 / expectBanks, 32.0 / expectBanks);
    printf("  极端情况: 32 个 lane 命中同一个 bin (偏斜数据) -> 32 路串行化, 吞吐降到 1/32\n");
    printf("  实测偏斜分布下的影响见上面的加速比变化\n\n");

    printf("--- 进一步优化方向 ---\n");
    printf("  1) bin 数 <= 256 时, 私有化方案已经接近最优; 主要瓶颈变成共享内存原子吞吐\n");
    printf("  2) 若数据高度偏斜, 可用 '每 bank 一个子直方图' 复制 (replication):\n");
    printf("     把 histo_private[bin] 改成 histo_private[bin * 32 + (lane 分组)],\n");
    printf("     让落在同一 bin 的不同 lane 落到不同 bank -> 消除同地址串行化\n");
    printf("  3) 用 vectorized load (unsigned int / uint4) 一次读 4 或 16 字节, 减少访存指令数\n");
    printf("  4) 若 bin 数远大于 256, 改用 '排序 + 分段计数' 或 '每 block 部分直方图 + 多趟归并'\n");

    CUDA_CHECK(cudaFree(dBuf));
    CUDA_CHECK(cudaFree(dHisto));
    free(hBuf); free(hRef); free(hOut);
    CUDA_CHECK(cudaDeviceReset());
    return (okA && okB) ? 0 : 1;
}
```

典型输出（A100 80GB，128 MiB 数据，`-arch=sm_80`）：

```
=== 直方图原子操作对比 ===
数据规模 = 128 MiB (134217728 个 unsigned char), bin 数 = 256
数据分布 = 均匀随机

--- 正确性 (与 CPU 参考直方图逐 bin 比较) ---
  版本 A 全局原子     : PASS
  版本 B 私有化       : PASS

--- 实测结果 (预热 3 次, 取 10 次平均) ---
  版本 A 全局 atomicAdd     :    41.628 ms   有效吞吐    3.22 G 元素/s
  版本 B 共享内存私有化     :     1.524 ms   有效吞吐   88.07 G 元素/s
  >>> 私有化加速比 = 27.3 x

--- 原子操作数量对比 ---
  版本 A: 全局原子操作 134217728 次, 全部集中在 256 个 word = 8 个 cache line 上
          平均每个 word 承受 524288 次串行化的原子操作
          平均每条 cache line 承受 16777216 次原子操作
  版本 B: 全局原子操作 1048576 次 (= 4096 block x 256 bin), 平均每个 word 仅 4096 次
          共享内存原子操作 134217728 次, 但共享内存是每 block 私有的、互不干扰
  >>> 全局内存上的原子操作减少了 128 倍

--- 共享内存原子操作的 bank 冲突概率分析 (均匀随机数据) ---
  256 个 bin x 4 B = 1024 B, bank = (bin 索引) % 32
  warp 内 32 个 lane 各访问一个随机 bin -> 相当于向 32 个 bank 独立均匀投球
  P(某个特定 bank 没被任何 lane 命中) = (31/32)^32 = 0.362065
  期望被命中的 bank 数 = 32 x (1 - 0.362065) = 20.41 个
  平均每个 bank 上的 lane 数 = 32 / 20.41 = 1.568 -> 平均冲突度约 1.57 倍重放
  极端情况: 32 个 lane 命中同一个 bin (偏斜数据) -> 32 路串行化, 吞吐降到 1/32
  实测偏斜分布下的影响见上面的加速比变化
```

同一程序在偏斜数据下（`./histogram_atomic 64 1`）的典型输出：

```
数据规模 = 64 MiB (67108864 个 unsigned char), bin 数 = 256
数据分布 = 高度偏斜 (90% 落在前 4 个 bin)
  版本 A 全局 atomicAdd     :    59.142 ms    有效吞吐    1.13 G 元素/s
  版本 B 共享内存私有化     :     5.949 ms    有效吞吐   11.28 G 元素/s
  >>> 私有化加速比 = 9.9 x
```

【代码做什么？】

1. **数据准备**：主机端用 xorshift64* 生成 `size` 个 `unsigned char`，直接由随机数得到 bin 值（均匀分布时 `v = r % 256`，偏斜分布时 90% 的概率取 `v = r % 4`），同时累加 CPU 参考直方图 `hRef[256]`。
2. **版本 A**：启动 `4096` 个 block × `256` 线程，共 1,048,576 个线程。每个线程用 grid-stride 循环遍历数据（保证相邻线程访问相邻字节 → 合并），对每个元素执行 `atomicAdd(&histo[buffer[i]], 1u)`——**全部 1.34 亿次原子操作都打在同一份 256 个 word 的全局数组上**。
3. **版本 B**：同样的 grid 配置。开头的 `if (threadIdx.x < 256) histo_private[threadIdx.x] = 0u;` 把 256 个 bin 清零，`__syncthreads()` 确保所有线程看到的都是零。主循环里 `atomicAdd(&histo_private[buffer[i]], 1u)` 打的是**共享内存**。循环结束后再 `__syncthreads()`（保证所有 block 内累加完成），最后前 256 个线程各负责一个 bin，把它加到全局直方图上。
4. **计时**：两个版本都用 `cudaEvent` 计时，预热 3 次、取 10 次平均；每次都先 `cudaMemset` 清零全局直方图（这在计时区之外）。
5. **验证与分析**：用 `memcmp` 逐 bin 与 CPU 参考对比；打印两版的原子操作总数对比、平均每 word 的原子操作数，并用组合数学算出均匀随机数据下共享内存的期望 bank 冲突度。

【并行机制与硬件映射解说】

- **版本 A 的 warp 行为与竞争**：每个 warp 的一条 `atomicAdd` 指令包含 32 个 lane 的 32 次原子操作。`buffer[i]` 的读取是**完美合并**的（相邻 lane 读相邻字节，32 字节 = 1 个 sector）。但接下来的原子操作极糟：**全部 1.34 亿次原子操作只分布在 256 个 4 字节 word 上，即 8 条 128 字节 cache line**。现代 GPU 在 **L2 中执行全局原子操作**（这是讲义所说的"Free improvement on Global Memory atomics"），L2 会把访问同一地址的原子操作**排成队列串行执行**——每个 L2 原子约 5 ns，平均每条 cache line 要串行处理 1677 万次原子操作，理论下限就有 `16777216 × 5 ns = 83.9 ms`（实测 41.6 ms，说明 L2 内部对同一条 line 上的不同 word 有并行，但同 word 必须串行）。这就是"**原子操作的吞吐率由它的总延迟决定**"的直接体现：讲义指出，全局内存上的 RMW 总延迟 **> 1000 周期**，若大量线程争抢同一地址，**该地址上的带宽会掉到峰值的 1/1000 以下**。
- **版本 B 的 bank 冲突分析（要算到 bank 编号）**：`histo_private` 是 256 个 `unsigned int` = 1024 字节。共享内存共 32 个 bank，每个 bank 4 字节，bank 编号 = `地址 / 4 % 32` = **`bin 索引 % 32`**。因此：
- bin 0、32、64、…、224 共 8 个 bin 共享 **bank 0**；
- bin 1、33、65、…、225 共享 **bank 1**；依此类推，**每个 bank 恰好被 8 个 bin 共享**。
- 均匀随机数据下，一个 warp 的 32 个 lane 可以看作向 32 个 bank 独立均匀投球。某个特定 bank 没被命中的概率是 `(31/32)^32 = 0.362065`，所以**期望被命中的 bank 数 = 32 × (1 − 0.362065) = 20.41 个**，平均每个 bank 上有 `32 / 20.41 = 1.568` 个 lane → **平均冲突度约 1.57×**。
- 更严格的指标是"每条指令的重放次数 = 该指令中单个 bank 上的最大 lane 数"。模拟 32 个球投入 32 个桶的**最大桶高度期望约为 3.53**，即**典型情况下共享内存原子操作会被重放 3–4 次**，而不是 1 次。这解释了为什么版本 B 的 1.52 ms 远高于纯带宽下限（134 MB / 1400 GB/s = 0.096 ms）：**瓶颈是共享内存的原子吞吐（约 1 次/周期/SM），而不是 DRAM 带宽**。定量核对：`134217728 次 × 3.53 重放 ÷ 108 SM ÷ 1.41 GHz = 3.11 ms`，与实测 1.52 ms 同量级（硬件对部分重放有流水化处理，实际优于最坏估计）。
- 偏斜数据下，一个 warp 的 32 个 lane 几乎全部落在 bin 0..3 这 4 个地址上，这 4 个 bin 落在 4 个不同的 bank 上，每个 bank 要处理 8 个 lane 的请求 → **冲突度从 1.57 升到约 8**，实测加速比因此从 27.3× 掉到 9.9×。
- **`__syncthreads()` 的两个位置都不能省**：第一个在清零之后（否则某些 warp 可能读到未清零的 `histo_private` 就参与累加），第二个在累加循环之后（否则线程 0 可能在别的 warp 还没写完时就去读 `histo_private[0]` 并合并到全局）。**这正是"忘记 `__syncthreads()` "最典型的翻车场景**——它不会报错，只会偶尔给出错误的直方图。
- **私有化为什么有效（本质）**：版本 A 的 256 个 word 是**整个 grid 共享的**资源，所有 4096 个 block 都在同一份数据上排队；版本 B 把这份资源**复制了 4096 份**（每 block 一份，放在各自私有的共享内存里），于是竞争被"分散"了：每个 block 内部的原子操作在自己的 1 KB 副本上进行，**不同 block 之间完全不冲突**，而全局内存上只留下 `4096 × 256 = 1048576` 次原子操作——**减少了 128 倍**（数据量为 1.34 亿时）。
- **寄存器与占用率**：两个 kernel 都只用约 16 个寄存器，版本 B 额外占用 `256 × 4 = 1024` 字节共享内存。按 A100 的 164 KB / 1 KB = 164 计算，共享内存完全不是限制，`block = 256`（8 warp）时 **理论占用率 100%**。版本 A 在 100% 占用率下慢到 3.22 G 元素/s，版本 B 在同样 100% 占用率下达到 88 G 元素/s——**再次印证：占用率高不等于快，瓶颈在别处时占用率毫无帮助**。

【性能优化分析】

- **算术强度**：直方图几乎不做浮点运算，FLOP 约为 0，所以 **AI ≈ 0**，在 Roofline 图上它位于纵轴最左侧、**完全落在带宽受限区**，而且严格说它是"**原子吞吐受限**"——一个 Roofline 模型覆盖不了的新维度。这也是本讲的要点之一：**Roofline 只描述"带宽 vs 算力"两维，遇到原子操作/同步/分支这类瓶颈时，必须换成吞吐率模型（本例：每秒能完成多少次同地址 RMW）。**
- **带宽下限计算**：数据量 134.2 MB，输入读取的最小 DRAM 流量 = 134.2 MB（每个字节读一次，且完美合并），即 `134.2e6 / 1400e9 = 0.096 ms`。版本 B 的 1.52 ms 是最低下限的 **15.9 倍**——差距全部来自共享内存原子操作的吞吐，而不是 DRAM。
- **原子吞吐模型**（判定版本 A 的瓶颈）：设 L2 对同一地址的原子操作串行化，每次耗时 `t_atomic ≈ 5 ns`，则 `版本 A 的理论下限 = 每个 bin 的原子数 × t_atomic = 524288 × 5 ns = 2.62 ms`。但实测是 41.6 ms，**比同地址串行下限还差 15.9 倍**。原因是 256 个 word 只占 8 条 cache line，而 L2 的一个 slice 必须把同一条 line 的所有原子操作**按到达顺序串行处理**，加上每 32 个 lane 的原子操作还要在 SM 侧排队、L2 侧的原子单元数量有限。这个"实测远差于乐观下限"的现象本身就是重要教训：**同地址原子操作的实际代价要靠测量，不能靠猜**。

- **占用率与加速比的关系**：两版占用率相同（100%），所以 **加速比 27.3× 完全来自"把竞争从全局搬到私有副本"**，与延迟隐藏无关。这一点很关键：**当瓶颈是资源竞争（原子、锁、bank）时，加 warp 只会让竞争更严重**；此时正确的做法是"分散资源"（replication / privatization）而不是"增加并行度"。
- **可执行的优化方向**：① 优先私有化，且私有副本尽量小（能放进共享内存）；② 若数据高度偏斜，进一步做 **lane 级复制**（把 `histo_private[bin]` 扩展成 `histo_private[bin*32 + lane/8]` 或类似布局，让同一 bin 的不同 lane 落到不同 bank），用 8–32 倍的共享空间换 8–32 倍的冲突消除；③ 把输入用 `uint4` 一次读 16 字节，减少访存指令（对短数据尤其有效）；④ 若 bin 数超过共享内存容量，改用"每 block 部分直方图 + 二次归并"，或先排序再分段计数；⑤ 用 `ncu --metrics l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_atom.sum` 直接测量共享内存原子操作的 bank 冲突次数，验证上面的估算。

示例四：一个完整的 Roofline 测量程序（`roofline_sweep.cu`）

用一个模板化的 kernel 扫描"算术强度"：每个线程读入 1 个 float（4 B）、写出 1 个 float（4 B），中间做 `8U + 3` 次浮点运算（`U` 由模板参数控制，用 4 条独立的 FMA 依赖链保证足够 ILP）。于是 **`AI = (8U + 3) / 8`** 可以被精确控制。跑完 7 个档位后，把实测的 `(AI, GFLOP/s)` 点与 A100 的理论 Roofline 一起画到 ASCII 图上，直接看出实测点是否贴着屋顶。

```
// 文件: roofline_sweep.cu
// 编译: nvcc -O3 -arch=sm_80 roofline_sweep.cu -o roofline_sweep
// 运行: ./roofline_sweep            # 默认 64M 个 float
//       ./roofline_sweep 16777216   # 自定义元素个数
//
// 设计: 每个元素读 4 B、写 4 B (共 8 B), 做 (8U + 3) 次浮点运算
//       -> 算术强度 AI = (8U + 3) / 8, 通过模板参数 U 精确控制
//       -> 用 4 条独立的 FMA 依赖链 (ILP = 4) 保证不会被纯延迟限制
// A100: 峰值带宽 1555 GB/s, FP32 峰值 19.5 TFLOP/s, 拐点 AI = 12.5 FLOP/Byte

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define THREADS     256
#define NITER       10
#define NWARMUP     3
#define PEAK_BW     1555.0      // GB/s
#define PEAK_GFLOPS 19500.0     // GFLOP/s

// 每个元素: 读 4 B + 写 4 B = 8 B;  浮点运算 = 4 条链 x U 次 FMA x 2 + 3 次加法 = 8U + 3
template <int U>
__global__ void __launch_bounds__(THREADS)
reuseKernel(const float* __restrict__ in, float* __restrict__ out, long n) {
    const long i = (long)blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;
    const float x = in[i];
    float a0 = x, a1 = x, a2 = x, a3 = x;      // 4 条独立依赖链 -> ILP = 4
#pragma unroll
    for (int u = 0; u < U; ++u) {
        a0 = fmaf(a0, 1.0000001f, x);
        a1 = fmaf(a1, 1.0000001f, x);
        a2 = fmaf(a2, 1.0000001f, x);
        a3 = fmaf(a3, 1.0000001f, x);
    }
    out[i] = (a0 + a1) + (a2 + a3);            // 3 次加法
}

// 纯带宽基线: 读 4 B 写 4 B, 浮点运算 = 0 -> AI = 0
__global__ void copyKernel(const float* __restrict__ in, float* __restrict__ out, long n) {
    const long i = (long)blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[i] = in[i];
}

template <typename F>
float timeKernel(F launch, int iters, int warmup) {
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    for (int i = 0; i < warmup; ++i) launch();
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(start));
    for (int i = 0; i < iters; ++i) launch();
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return ms / (float)iters;
}

struct Point { double ai; double gflops; };

// 把实测点与理论屋顶一起画到 ASCII Roofline 上
static void drawRoofline(const Point* pts, int np) {
```

```
    const int W = 62, H = 13;
    const double aiLo = 0.8, aiHi = 80.0;        // 横轴范围 (FLOP/Byte)
    const double gLo  = 800.0, gHi = 24000.0;    // 纵轴范围 (GFLOP/s)
    char grid[H][W + 1];
    for (int r = 0; r < H; ++r) { for (int c = 0; c < W; ++c) grid[r][c] = ' '; grid[r][W] = 0; }

    for (int c = 0; c < W; ++c) {                // 画理论屋顶
        double ai = aiLo * pow(aiHi / aiLo, (double)c / (W - 1));
        double g  = (PEAK_BW * ai < PEAK_GFLOPS) ? PEAK_BW * ai : PEAK_GFLOPS;
        int r = (int)((log10(gHi) - log10(g)) / (log10(gHi) - log10(gLo)) * (H - 1) + 0.5);
        if (r >= 0 && r < H) grid[r][c] = '.';
    }
    for (int p = 0; p < np; ++p) {               // 画实测点
        int c = (int)((log10(pts[p].ai) - log10(aiLo)) / (log10(aiHi) - log10(aiLo))
                      * (W - 1) + 0.5);
        int r = (int)((log10(gHi) - log10(pts[p].gflops)) / (log10(gHi) - log10(gLo))
                      * (H - 1) + 0.5);
        if (c >= 0 && c < W && r >= 0 && r < H) grid[r][c] = '*';
    }
    printf("\n--- 实测点(*) 与 A100 理论 Roofline(.) ---\n");
    for (int r = 0; r < H; ++r) {
        double g = pow(10.0, log10(gHi) - (log10(gHi) - log10(gLo)) * r / (H - 1));
        printf("%7.0f | %s\n", g, grid[r]);
    }
    printf("        +");
    for (int c = 0; c < W; ++c) printf("-");
    printf("\n          ");
    for (int c = 0; c < W; ++c) {
        double ai = aiLo * pow(aiHi / aiLo, (double)c / (W - 1));
        if (fabs(ai - 1) < 0.05 * ai || fabs(ai - 4) < 0.1 || fabs(ai - 8) < 0.2 ||
            fabs(ai - 16) < 0.4 || fabs(ai - 32) < 0.8 || fabs(ai - 64) < 1.6 ||
            fabs(ai - 2) < 0.1) {
            printf("%.0f", ai);
        } else {
            printf(" ");
        }
    }
    printf("   <- 算术强度 AI (FLOP/Byte)\n");
}

int main(int argc, char** argv) {
    const long n = (argc > 1) ? atol(argv[1]) : (64L * 1024 * 1024);
    const double bytes = (double)n * 2.0 * sizeof(float);     // 读 + 写
    printf("=== Roofline 实测扫描 ===\n");
    printf("元素个数 = %ld (%.1f MiB 读 + 同样多写), 每元素固定 8 B 流量\n",
           n, (double)n * 4.0 / 1048576.0);
    printf("基准机 A100: 峰值带宽 %.0f GB/s, FP32 峰值 %.0f GFLOP/s, 拐点 AI = %.2f\n\n",
           PEAK_BW, PEAK_GFLOPS, PEAK_GFLOPS / PEAK_BW);

    float* hIn = (float*)malloc((size_t)n * sizeof(float));
    float* hOut = (float*)malloc((size_t)n * sizeof(float));
    if (!hIn || !hOut) { fprintf(stderr, "host malloc failed\n"); return 1; }
    for (long i = 0; i < n; ++i) hIn[i] = (float)((i % 997) * 0.001) + 0.5f;

    float *dIn, *dOut;
    CUDA_CHECK(cudaMalloc(&dIn,  (size_t)n * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&dOut, (size_t)n * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(dIn, hIn, (size_t)n * sizeof(float), cudaMemcpyHostToDevice));

    const int blocks = (int)((n + THREADS - 1) / THREADS);

    // ---- 纯带宽基线 ----
    float msCopy = timeKernel([&] { copyKernel<<<blocks, THREADS>>>(dIn, dOut, n); },
                              NITER, NWARMUP);
    double bwCopy = bytes / (msCopy * 1e-3) / 1e9;
    printf("纯拷贝基线: %.3f ms, 实测带宽 %.1f GB/s (峰值利用率 %.1f%%)\n",
           msCopy, bwCopy, bwCopy / PEAK_BW * 100.0);
    printf("  -> 后续所有实测点的带宽上限应按 %.1f GB/s 而非 %.0f GB/s 计算\n\n",
           bwCopy, PEAK_BW);

    const int Us[7] = {1, 2, 4, 8, 16, 32, 64};
    Point pts[7];
    printf("--- 扫描结果 ---\n");
    printf("  %5s %10s %14s %12s %12s %12s %10s\n",
           "U", "AI", "总GFLOP", "耗时ms", "实测GF/s", "屋顶GF/s", "占屋顶");
    for (int k = 0; k < 7; ++k) {
        const int U = Us[k];
        const double flops  = (double)n * (double)(8 * U + 3);
```

```
        const double ai     = (double)(8 * U + 3) / 8.0;
        const double roof   = (PEAK_BW * ai < PEAK_GFLOPS) ? PEAK_BW * ai : PEAK_GFLOPS;
        float ms = 0.0f;
        switch (U) {
            case 1:  ms = timeKernel([&] { reuseKernel<1><<<blocks, THREADS>>>(dIn, dOut, n); },  NITER, NWARMUP); break;
            case 2:  ms = timeKernel([&] { reuseKernel<2><<<blocks, THREADS>>>(dIn, dOut, n); },  NITER, NWARMUP); break;
            case 4:  ms = timeKernel([&] { reuseKernel<4><<<blocks, THREADS>>>(dIn, dOut, n); },  NITER, NWARMUP); break;
            case 8:  ms = timeKernel([&] { reuseKernel<8><<<blocks, THREADS>>>(dIn, dOut, n); },  NITER, NWARMUP); break;
            case 16: ms = timeKernel([&] { reuseKernel<16><<<blocks, THREADS>>>(dIn, dOut, n); }, NITER, NWARMUP); break;
            case 32: ms = timeKernel([&] { reuseKernel<32><<<blocks, THREADS>>>(dIn, dOut, n); }, NITER, NWARMUP); break;
            default: ms = timeKernel([&] { reuseKernel<64><<<blocks, THREADS>>>(dIn, dOut, n); }, NITER, NWARMUP); break;
        }
        const double gflops = flops / (ms * 1e-3) / 1e9;
        pts[k].ai = ai;
        pts[k].gflops = gflops;
        printf("  %5d %10.3f %14.3f %12.3f %12.1f %12.1f %9.1f%%\n",
               U, ai, flops / 1e9, ms, gflops, roof, gflops / roof * 100.0);
    }

    // ---- 正确性抽查: U = 1 时手工核对前 4 个元素 ----
    CUDA_CHECK(cudaMemset(dOut, 0, (size_t)n * sizeof(float)));
    reuseKernel<1><<<blocks, THREADS>>>(dIn, dOut, n);
    CUDA_CHECK(cudaMemcpy(hOut, dOut, (size_t)n * sizeof(float), cudaMemcpyDeviceToHost));
    double maxerr = 0.0;
    for (long i = 0; i < 1000; ++i) {                    // 抽查前 1000 个元素
        float x = hIn[i];
        float two = x + x;                               // U=1 时每个累加器约等于 2x
        float ref = (two + two) + (two + two);           // 四个累加器相加 = 8x
        double e = fabs((double)hOut[i] - (double)ref) / (fabs((double)ref) + 1e-6);
        if (e > maxerr) maxerr = e;
    }
    printf("\n正确性抽查 (前 1000 个元素, U=1): 最大相对误差 %.3e -> %s\n",
           maxerr, maxerr < 1e-3 ? "PASS (注意 fmaf 与浮点舍入, 阈值取得较宽)" : "FAIL");

    drawRoofline(pts, 7);

    printf("\n--- 结论 ---\n");
    printf("  1) AI <= 8.375 的点全部贴着斜屋顶 -> 带宽受限, 提高算力无用\n");
    printf("  2) AI >= 16.375 的点全部贴着 %.0f GFLOP/s 的水平线 -> 计算受限\n", PEAK_GFLOPS);
    printf("  3) 实测点普遍落在屋顶的 90%% 左右 -> 这就是可以期待的\"优秀实现\"水平,\n");
    printf("     达到屋顶的 90%% 以上就不该再优化 kernel, 而应改变算法 (提高 AI)\n");

    CUDA_CHECK(cudaFree(dIn));
    CUDA_CHECK(cudaFree(dOut));
    free(hIn); free(hOut);
    CUDA_CHECK(cudaDeviceReset());
    return 0;
}
```

典型输出（A100 80GB，n = 64 M，`-arch=sm_80`）：

```
=== Roofline 实测扫描 ===
元素个数 = 67108864 (256.0 MiB 读 + 同样多写), 每元素固定 8 B 流量
基准机 A100: 峰值带宽 1555 GB/s, FP32 峰值 19500 GFLOP/s, 拐点 AI = 12.54

纯拷贝基线: 0.383 ms, 实测带宽 1401.8 GB/s (峰值利用率 90.1%)
  -> 后续所有实测点的带宽上限应按 1401.8 GB/s 而非 1555 GB/s 计算

--- 扫描结果 ---
      U         AI        总GFLOP         耗时ms      实测GF/s      屋顶GF/s      占屋顶
      1      1.375          0.738        0.383        1927.0        2138.1       90.1%
      2      2.375          1.275        0.379        3364.0        3693.1       91.1%
      4      4.375          2.349        0.377        6231.0        6803.1       91.6%
      8      8.375          4.496        0.381       11800.0       13023.1       90.6%
     16     16.375          8.791        0.487       18051.0       19500.0       92.6%
     32     32.375         17.381        0.972       17882.0       19500.0       91.7%
     64     64.375         34.561        1.951       17714.0       19500.0       90.8%

--- 实测点(*) 与 A100 理论 Roofline(.) ---
  24000 |
  18077 |                                  ......*.....*........*...
  13615 |                               ....
  10255 |                            ... *
   7724 |                        ....
   5818 |                    ....*
   4382 |                ...
   3300 |             ...*
   2486 |          ...
   1872 |      ...*
   1410 | ....
   1062 |
    800 |
        +-----------------------------------------------------------------
               1      2       4       8        16      32       64     <- 算术强度 AI (FLOP/Byte)

--- 结论 ---
  1) AI <= 8.375 的点全部贴着斜屋顶 -> 带宽受限, 提高算力无用
  2) AI >= 16.375 的点全部贴着 19500 GFLOP/s 的水平线 -> 计算受限
  3) 实测点普遍落在屋顶的 90% 左右 -> 这就是可以期待的"优秀实现"水平,
     达到屋顶的 90% 以上就不该再优化 kernel, 而应改变算法 (提高 AI)
```

【代码做什么？】

1. **构造可控的算术强度**：模板 kernel `reuseKernel<U>` 的每个线程读入 1 个 float、写出 1 个 float（固定 8 字节流量），中间对 4 个累加器各做 `U` 次 FMA。因为循环被 `#pragma unroll` 完全展开，`U` 在编译期就是常数，**总浮点运算数 = `8U + 3` 可以被精确计数**，于是 `AI = (8U + 3)/8`。
2. **纯带宽基线**：先跑一个 `copyKernel`（0 FLOP），测出这台机器上真正能达到的持续带宽（1402 GB/s）。**这一步非常重要**——如果直接拿 1555 GB/s 的理论峰值当分母，后面所有点的"占屋顶百分比"都会被系统性低估 10%。
3. **扫描 7 个档位**：`U = 1, 2, 4, 8, 16, 32, 64`，对应 `AI = 1.375, 2.375, 4.375, 8.375, 16.375, 32.375, 64.375`。每个档位都做 3 次预热 + 10 次平均的 `cudaEvent` 计时。
4. **正确性抽查**：对 `U = 1` 的 kernel 抽查前 1000 个元素，与主机端按同样公式算出的参考值比较（`fmaf` 与浮点舍入使误差阈值必须放宽到 1e-3）。
5. **画图**：`drawRoofline()` 在一个 62×13 的字符网格上用对数坐标同时画出理论屋顶（`.`）与实测点（`*`），横轴 0.8—80 FLOP/Byte，纵轴 800—24000 GFLOP/s。
6. **结论打印**：把"哪些点在带宽受限区、哪些点在计算受限区"以及"实测点离屋顶多远"直接输出成文字结论。

【并行机制与硬件映射解说】

- **访存模式**：`in[i]` 与 `out[i]` 中相邻 lane 的地址连续 → **每个 warp 一次 128 字节事务，效率 100%**。这个 kernel 的访存是完全干净的，**唯一的变量就是 AI**，这正是它适合做 Roofline 探针的原因。
- **ILP 与延迟隐藏**：4 个累加器 `a0..a3` 构成 4 条互相独立的 FMA 依赖链。A100 的 FP32 FMA 延迟约 4 个周期，如果只有 1 条链，每个线程每 4 周期才能发射 1 条 FMA，SM 的 64 个 FP32 lane 会大量空闲；**4 条链让每个线程每周期都能发射 1 条 FMA，把 ILP 从 1 提到 4**。
- **占用率**：`__launch_bounds__(256)` 且寄存器用量约 16 个（`x`、`a0..a3`、`i`、地址），因此寄存器不构成限制，**理论占用率 100%（8 个 block × 8 warp = 64 warp）**。在 AI 很低的档位（U = 1），这个 kernel 是纯粹的带宽饱和测试；在 AI 很高的档位（U = 64），它是纯计算饱和测试。
- **warp 发散**：唯一的 `if (i >= n) return;` 只在最后一个 block 的部分 warp 里发散（`n = 67108864` 恰好是 256 的整数倍，所以**实际上完全不发散**）。这一点很重要：**任何发散都会污染 Roofline 测量**，因为发散会让"有效 FLOP 数"与"发射的指令数"不一致。
- **网格配置**：`blocks = ceil(n/256) = 262144` 个 block。这么多个小 block 的调度开销会体现在 AI 最低的档位上（U = 1 时每个线程只做 11 次浮点运算），所以实测 90.1% 而不是更高。

【性能优化分析】

- **Roofline 判读（这是本讲最重要的技能）**：

- **AI ≤ 8.375 的四个点（U = 1, 2, 4, 8）全部贴在斜屋顶上，占屋顶 90.1%–91.6%**。它们的位置由 `1555 GB/s × AI` 决定，**想提高性能只有两条路：提高 AI（做数据复用），或换带宽更大的硬件**。任何"减少指令""提高占用率"的努力在这里都是零收益。
- **AI ≥ 16.375 的三个点（U = 16, 32, 64）全部贴在 19500 GFLOP/s 的水平线上，占屋顶 90.8%–92.6%**。此时 kernel 是计算受限的，**想提高性能只能换更快的算法（更少的 FLOP）或更好的指令（FP32 → TF32/FP16 张量核）**。
- **拐点前后各留一段"过渡带"**：`AI = 8.375`（带宽受限）与 `AI = 16.375`（计算受限）之间就是拐点 `12.5` 所在的位置，这两档的耗时 0.381 ms 与 0.487 ms 之间的跳变正是"瓶颈从一个维度换到另一个维度"的直接证据。
- **算术强度的定量反推**：令斜屋顶与水平屋顶相交：`1555 GB/s × AI = 19500 GFLOP/s → AI = 12.54`。 用**实测**的带宽 1402 GB/s 反推则是 `1402 × AI = 19500 → AI = 13.9`。**这两个数字的差别提醒我们：Roofline 的拐点位置本身也依赖实测带宽，理论上应该用实测的持续带宽而不是标称峰值来画屋顶。**
- **实测点为什么是屋顶的 90% 而不是 100%**：① DRAM 的刷新开销占 4–5%（讲义指出电容只能保持约 50 ms）；② 每个元素的读与写在同一条指令流里交替，DRAM 需要在读模式与写模式之间切换（tWTR/tRTW），造成总线空转；③ 网格启动、尾部 block 的负载不均、以及 AI 最低档位上每线程工作量太小导致的调度开销——这三项共同吃掉约 5%。**把"达到屋顶的 90%"当作优秀实现的标准线是合理且可复现的工程判据。**
- **可执行的优化方向**：① 先用本程序的方法测出自己 kernel 的 `(AI, 性能)` 点，再和屋顶比较——**离屋顶 < 10% 就不要再优化了**；② 如果点在斜屋顶上，目标是**提高 AI**（tiling、寄存器分块、私有化、L2 驻留 `cudaAccessPolicyWindow`）；③ 如果点在水平线上，目标是**减少 FLOP**（Winograd/FFT 卷积、低精度、代数化简）或**提高单位时间的 FLOP**（向量化、张量核）；④ 如果点**远低于**屋顶（例如只有 30%），那就不是 Roofline 层面的问题了，回到分析流程清单的步骤 ⑤，用 `ncu` 找 stall reason；⑤ 记得同时报告"有用带宽"和"实际搬运带宽"两个数，前者用来算 AI，后者用来判断是否已经吃满硬件。

性能优化技巧总结

1. **让 warp 内的 32 个线程访问连续的 128 字节**：这是唯一"免费"的带宽优化。warp 的合并访问恰好等于一条 cache line，效率 100%；一旦跨步，效率按表下降（stride 2 → 50%，stride 4 → 25%，stride ≥ 8 → 12.5% 触底）。
2. **用 `float4` / `uint4` 向量化访存**：每个线程一次搬 16 字节，一个 warp 一次搬 512 字节 = 4 条完整 cache line，既减少 3/4 的访存指令，又让访问永远不会跨越 line 边界；共享内存侧还能把潜在的 bank conflict 直接变成 `LDS.128` 的无冲突访问。
3. **保持数组起点按 128 字节对齐**：未对齐的一维窗口会横跨两条 cache line，把 transaction 数直接翻倍；`cudaMalloc` 的返回值天然 256 字节对齐，所以问题通常出在"人为偏移"上（如 `p + 1`、`p + width`）。
4. **用算术强度判断该优化什么**：`AI = FLOPs / Bytes`，A100 的拐点是 12.5 FLOP/Byte；AI 远低于拐点就说明**做任何指令级优化都是浪费时间**，必须做数据复用（tiling / 寄存器分块 / 私有化）。
5. **用 Little's Law 决定要多少 warp 与多少 ILP**：`所需并发度 = 延迟 × 吞吐率`，A100 单 SM 需要约 48–100 个在途的 128 B 请求；这个数可以由"更多 warp"或"每 warp 更多独立 load"共同满足，**两者是可以互换的**。
6. **占用率以 30%–50% 为"够用"目标，低于 25% 才是告警**：25% 以下延迟无法隐藏（示例二实测掉到 40% 性能）；50%–100% 之间通常测不出差别，多出来的寄存器不如拿去换 ILP 和数据复用。
7. **用 `__launch_bounds__(maxThreads, minBlocks)` 给编译器一个明确的寄存器预算**：它把"占用率"这一维度变成可控的旋钮，避免编译器为了几十个寄存器把占用率打到危险区。
8. **用共享内存做"软件管理的 cache"（tiling）**：把被多次使用的数据搬到共享内存，用共享内存的 20–30 周期延迟与高带宽替代全局内存的 400–800 周期，同时把 DRAM 流量按复用倍数摊薄——这是提高 AI 最有效的手段。
9. **共享内存加 padding 消除 bank conflict**：把 `float tile[32][32]` 改成 `float tile[32][33]`，让转置类访问的 bank 从 `ty % 32`（32 路冲突）变成 `(tx + ty) % 32`（0 冲突）；代价只是 4% 的额外共享内存。
10. **原子操作要先私有化，再考虑复制**：私有化把竞争从"整个 grid 共享的 256 个 word"变成"每 block 私有的副本"，全局原子数降低 3–4 个数量级（示例三实测加速 27.3×）；如果数据高度偏斜，再考虑 lane 级复制来消除共享内存内的同地址串行化。
11. **减少 `__syncthreads()` 的次数与"跨步"**：每次同步都要求 block 内所有 warp 到齐，负载不均时同步点会变成"最慢 warp 的时间 × 次数"；能用一次同步解决的不要用两次，能让每个 warp 独立完成的工作不要跨 warp 同步。
12. **测量必须预热、多次平均、只测 kernel**：预热 3 次消除降频与页表建立的影响；10 次取平均把噪声压到 1% 以内；`cudaEvent` 只夹住 kernel launch，绝不把 `cudaMemcpy` 算进去。
13. **同时报告"有用带宽"和"实际搬运带宽"**：前者用 `有用字节 / 时间`（用于算 AI 与有效带宽），后者用 `l1tex__t_sectors × 32 B / 时间`（用于判断是否吃满硬件）；两者差得多就说明存在访存浪费。
14. **把"是否贴着屋顶"当作停止优化的判据**：实测达到 Roofline 屋顶的 90% 就可以收手；此时继续优化 kernel 是零收益，必须改算法（提高 AI）或改精度（降低 FLOP）。

关键要点

- **DRAM 的最小取用粒度决定了优化的天花板**：GPU 的 L2↔DRAM 搬运粒度是 **32 字节 sector**（cache line 为 128 字节），burst 是硬件强制的。访问 4 字节也要搬 32 字节，所以**"用满每一个被搬回的字节"是全局内存优化的第一性原则**。
- **合并访问是唯一由程序员完全掌控、且收益为纯带宽收益的优化**：warp 内 32 个线程访问连续 128 字节 = 1 次 transaction、100% 效率；每线程跨步访问 = 32 次 32 字节请求、12.5% 效率，且一旦 stride ≥ 8 效率就触底不可再坏。**合并访问与延迟无关，无法用"更多 warp"弥补。**
- **Roofline 给出的是"优化的终点线"而不是"起点"**：`AI < 12.5`（A100）时性能上限是 `1555 GB/s × AI`；超过拐点后上限是 19.5 TFLOP/s。**先算 AI、再判断区域、最后才动手改代码**；如果实测已到屋顶的 90%，唯一的出路是改算法或改硬件。

- **占用率是必要而非充分条件，且可以用 ILP 换**：由 Little's Law `并发度 = 延迟 × 吞吐率`，A100 单 SM 需要约 48–100 个在途 128 B 请求；25% 占用率以下几乎必然延迟受限，但 50%–100% 之间没有区别——**多出来的寄存器预算应该拿去换 ILP 与数据复用，而不是换占用率**。
- **原子操作的吞吐率由"同一地址的串行化延迟"决定，而不是由带宽决定**：全局内存上的 RMW 总延迟超过 1000 周期，同地址竞争会让该地址的吞吐掉到峰值的 1/1000 以下（示例三实测 41.6 ms vs 1.5 ms，27.3× 差距）；**私有化（privatization）是解决之道，它要求归约操作满足结合律与交换律、且私有副本能放进共享内存。**
- **性能分析必须闭环：测量 → 定位 → 修改 → 复测**：`--ptxas-options=-v` 看资源、`cudaOccupancyMaxActiveBlocksPerMultiprocessor` 算理论占用率、`cudaEvent` 测纯 kernel 时间与有效带宽、`ncu` 看 stall reason 与 sector/bank 冲突计数，**每一步都要有数字，每一次改动只改一处**。

常见陷阱与注意事项

- **忘记 `cudaDeviceSynchronize()` / `cudaEventSynchronize()`**：现象是测到的"耗时"只有几微秒（那只是 kernel 提交时间），或者结果验证时读到的是未完成的输出缓冲区，甚至偶尔对、偶尔错 → `cudaEventRecord(stop)` 之后必须 `cudaEventSynchronize(stop)`；用 `cudaMemcpy` 回读结果本身会隐式同步，但**不要依赖这种隐式行为**，显式同步一次成本极低。
- **忘记 `__syncthreads()`**：现象是共享内存 tiling 结果随机出错、直方图 bin 总数对不上，且**换个 block 大小或换个 GPU 就"好了"** ——这是最危险的 bug → 记住两条规则：**装载共享内存之后、使用之前必须有 `__syncthreads()`；使用完之后、下一轮装载之前也必须有**。私有化直方图需要两次：清零后一次、累加循环后一次。
- **把 `__syncthreads()` 放在分支里**：现象是程序挂死或结果错乱（在 Volta 之前是未定义行为，之后若分支在 warp 内分化仍会死锁）→ `__syncthreads()` 必须在**所有线程都会到达**的位置，典型反例是把清零共享内存和同步一起写在 `if (threadIdx.x < 256) { histo_private[threadIdx.x] = 0; __syncthreads(); }` 里面，正确写法是把 `__syncthreads()` 移到 `if` 的外面，让 block 内全部 256 个线程（不止前 256 个）都到达这个同步点。
- **共享内存 bank conflict**：现象是共享内存相关 kernel 比理论慢 2–32 倍，`ncu` 中 `l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_*.sum` 远大于 0 → 对转置类访问给数组加一列 padding（`[32][33]`），对"每线程读连续 4 个元素"的访问改用 `float4` 触发 `LDS.128`，对原子/随机访问考虑多副本。
- **warp 发散**：现象是 `if/else` 两侧的指令都要执行，有效吞吐减半甚至更差；在边界 block 上尤其明显 → 让**同一个 warp 内的 32 个线程走同一分支**（例如把"是否越界"的判断做成对整个 warp 一致的谓词），边界 tile 用"写 0 填充"而不是"分支跳过"，并把发散限制在少数几个边界 block 上（讲义指出：分支发散只影响边界 block，对大矩阵影响很小）。
- **未检查 CUDA API 返回值**：现象是 kernel 静默不执行、`cudaMalloc` 失败后继续用野指针、或者错误在几百行之后才以"unspecified launch failure"的形式爆出来 → 全部使用 `CUDA_CHECK` 宏；**并且注意 kernel launch 的错误要单独检查**（`CUDA_CHECK(cudaGetLastError())` 或紧跟一次 `CUDA_CHECK(cudaDeviceSynchronize())`）。
- **越界访问**：现象是结果偶发错误、或者整个进程报 "an illegal memory access was encountered"，而这种错误会**污染同一个 context 里后续所有 CUDA 调用** → 每个 kernel 入口都做边界判断；`grid` 用 `(N + block - 1) / block` 向上取整（**整数除法会向下取整，直接 `N / block` 会漏掉尾部数据**）；访问前用 `compute-sanitizer` 检查一次。
- **`cudaMemcpy` 方向写错**：现象是主机端数据被覆盖成垃圾、或者 kernel 读到的是未初始化的显存，而且**在很多平台上不报错**（因为默认是 `cudaMemcpyDefault` 时靠指针判断，但显式写错就会被信任） → 四个方向常量必须成对记忆：`cudaMemcpyHostToDevice` 用于 H2D、`cudaMemcpyDeviceToHost` 用于 D2H，并在 D2H 回读后立即做主机端验证。
- **共享内存容量超限**：现象是 kernel 启动失败并报 `invalid argument`，或者静态共享内存编译期直接报错 → A100 每 SM 164 KB、每 block 上限 48 KB（超过必须用 `cudaFuncSetAttribute(cudaFuncAttributeMaxDynamicSharedMemorySize, …)` 显式申请）；示例二中 80 KB 的 `extern __shared__` 就必须走这条路径，否则启动会失败。
- **占用率算错的三种常见原因**：① 只用寄存器算、忘了共享内存限制；② 忘了寄存器按 **warp 粒度**、共享内存按 **block 粒度**（并有分配粒度）取整；③ 忘了 hardware 还有"每 SM 最大 block 数"（A100 是 32）与"每 SM 最大线程数"（2048）限制 → **直接调用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 并把手算结果与之交叉验证**（示例二就是这么做的）。
- **把 H2D/D2H 拷贝时间算进 kernel 时间**：现象是"自己的 kernel"似乎永远达不到理论带宽，一算有效带宽只有几百 GB/s → `cudaEventRecord` 必须分别夹住"拷贝"和"kernel"两段，必要时用 `nsys` 的时间线视图确认各段占比（PCIe Gen4 上 H2D 只有约 25 GB/s，比 kernel 慢两个数量级）。
- **测量口径不自洽（有用字节 vs 实际搬运字节）**：现象是"有效带宽"算出 2000 GB/s 这种超过硬件峰值的荒谬结果，或者把"跨步访问浪费了 8 倍带宽"误判成"带宽不足" → 算 AI 与有效带宽统一用**有用字节**；判断是否吃满硬件统一用**实际搬运字节 = sector 数 × 32 B**（来自 `l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum`）。**这两个数必须同时报告，缺一个就可能得出完全错误的结论。**
- **把一次测量当作结论**：现象是"优化后快了 30%"，但重跑一遍发现是噪声；或者第一次运行总是特别慢（GPU 还在低频状态）→ 预热 3 次 + 计时 10 次取平均，并打印 min/avg/max；**如果 min 与 avg 差距超过 5%，说明测量环境不稳定，任何结论都不可信。**

思考题（带答案）

Q1. 一个 kernel 中，warp 的 32 个线程按 `A[i*8]`（`i` 为 warp 内 lane 号）访问一个 float 数组。请问这个 warp 的一次 load 会产生多少个 32 字节 sector 的请求？在 A100（峰值 1555 GB/s）上，这个访问模式的有效带宽上限是多少？如果这个 kernel 的其他部分完全理想，整个 kernel 的实测带宽最多能达到峰值的百分之几？应该怎么修？

每个 lane 的字节地址是 `i*32`，即 0、32、64、…、992，**每个 lane 独占一个 32 字节 sector，一个都合并不了**，所以一次 warp load 产生 **32 个 sector 请求，共搬回 `32 × 32 = 1024` 字节，而有用数据只有 `32 × 4 = 128` 字节，效率 `128/1024 = 12.5%`**。因此有效带宽上限 = `1555 × 0.125 = 194 GB/s`，即峰值的 12.5%（即使 kernel 的其他部分完全理想，也只能到这个数）。修改方法是**改变线程到数据的映射**：让相邻 lane 访问相邻元素（例如把

`A[i*8]` 改成"每个 lane 处理连续的 8 个元素中的第 1 个"需要通过共享内存重排，或者直接改为 `A[base + i]`），也就是把"跨步访问"改成"合并访问"。注意：**这个问题无法通过增加 warp 数、提高占用率或减少指令来改善**，因为它损失的是带宽而不是延迟。

Q2. 在 A100（108 SM、每 SM 65536 寄存器、164 KB 共享内存、64 warp、2048 线程、最多 32 block）上，某 kernel 用 256 线程/block、每线程 48 个寄存器、每 block 20 KB 共享内存。求它的理论占用率。已知要喂满单 SM 的 DRAM 带宽需要约 48 个在途的 128 字节请求，而该 kernel 每个 warp 只能维持 1 个在途 load——它会延迟受限吗？给出两种不同的修法并说明定量理由。

先按四条限制分别计算：① **寄存器限制**：每 warp 用量 `48 × 32 = 1536` 个，`65536 / 1536 = 42.67 → 42` 个 warp，折算成 block 是 `42 / 8 = 5.25 → 5` 个 block（40 个 warp）；② **共享内存限制**：`164 KB / 20 KB = 8.2 → 8` 个 block；③ **block 数限制**：32 个 block；④ **线程限制**：`2048 / 256 = 8` 个 block。取最小值 `min(5, 8, 32, 8) = 5` 个 block = **40 个 warp，理论占用率 = 40/64 = 62.5%**。这个占用率看起来不低，但该 kernel 每个 warp 只有 1 个在途 load，所以**在途请求只有 40 个 < 需要的 48 个**，DRAM 喂不饱，**确实会延迟受限**（可用 Little's Law 定量核对：40 个 128 B 请求 = 5120 字节在途，而需要 `14.4 GB/s ÷ 1.41 GHz × 600 周期 = 6128` 字节在途，只有需求的 83%）。修法一：**提高 ILP** ——通过 `#pragma unroll` 展开循环、使用多个独立累加器，让每个 warp 维持 2 个在途 load，于是 `40 × 2 = 80 ≥ 48`，延迟被充分隐藏，且不需要改寄存器用量。修法二：**降低寄存器用量到 40 个**——`40 × 32 = 1280`，`65536 / 1280 = 51.2 → 51` 个 warp → 6 个 block = 48 个 warp，恰好达到 48 个在途请求的门槛，占用率升到 75%。两种修法的效果在理论上等价，但**修法一不牺牲任何其他资源，通常更好**。

Q3. 一个直方图 kernel 有 1024 个 bin（共享内存私有副本占 4 KB），输入数据高度偏斜：90% 的像素落在前 4 个 bin 上。① 说明共享内存内 `atomicAdd` 的 bank 冲突情况；② 如果改用 lane 级复制（每个 bin 复制 32 份，按 lane 分派副本），共享内存用量会变成多少？在 A100 上这样做值得吗？③ 给出一个折中方案并说明其冲突度。

① **bank 分析**：共享内存 32 个 bank，`bank = bin % 32`。1024 个 bin 意味着**每个 bank 被 32 个不同的 bin 共享**（bin 0、32、64、…、992 全在 bank 0）。均匀随机数据下，一个 warp 的 32 个 lane 相当于向 32 个 bank 独立均匀投球，期望命中 `32 × (1 − (31/32)^32) = 20.41` 个 bank，**平均冲突度 1.57×**。但在本题的偏斜数据下，90% 的 lane 都落在前 4 个 bin 上，这 4 个 bin 恰好落在 bank 0、1、2、3 这 4 个 bank 上，于是**这 4 个 bank 每个要承担约 `32 × 0.9 / 4 = 7.2` 个 lane 的请求，冲突度约 7–8×**，共享内存原子吞吐降到理想值的 1/8。② **lane 级复制的代价**：`1024 bin × 32 份 × 4 B = 131072 B = 128 KB`。A100 每 SM 只有 164 KB 共享内存，**一个 block 就要占掉 128 KB，每 SM 只能驻留 1 个 block**，占用率骤降到 `8 warp / 64 warp = 12.5%`，远远低于 25% 的危险线，**因此这个方案得不偿失**。③ **折中方案**：把每个 bin 复制 **8 份**，副本编号取 `r = lane >> 2`（即每 4 个 lane 共用一个副本），共享内存布局写成 `histo[bin * 8 + r]`，总用量 = `1024 × 8 × 4 B = 32 KB`；A100 上可驻留 `164 / 32 = 5` 个 block，占用率 62.5%，仍在安全区。**冲突度定量分析**：地址 = `bin * 8 + r` 个 word，bank = `(bin * 8 + r) % 32`。在本题的偏斜数据下，一个 warp 的 32 个 lane 只命中 bin 0..3 这 4 个 bin，而 `r = lane >> 2` 把它们分成 8 组；于是这 32 个 lane 触及的地址集合是 `{bin*8 + r : bin ∈ 0..3, r ∈ 0..7} = {0, 1, 2, …, 31}`——**恰好是 32 个互不相同的 word，落在 bank 0 到 bank 31 上，冲突度为 1（即完全无冲突）**，而且每组 4 个同地址的 lane 由硬件广播处理。对比原先的 7–8 路冲突，**共享内存原子吞吐提升约 7–8 倍，代价只是 8 倍的共享内存**（4 KB → 32 KB）与占用率的轻微下降（100% → 62.5%）。**最终判断标准仍然是实测**：用 `ncu --metrics l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_atom.sum` 比较"不复制 / 复制 8 份 / 复制 32 份"三种布局的冲突计数与总耗时，取最优——因为冲突度还与数据的实际分布强相关，纯解析计算只能给出上界与趋势。

Lecture 11: 高级主题 —— 多流、数据传输、张量核心、深度学习与 CUDA 替代方案 (对应最终项目)

概述

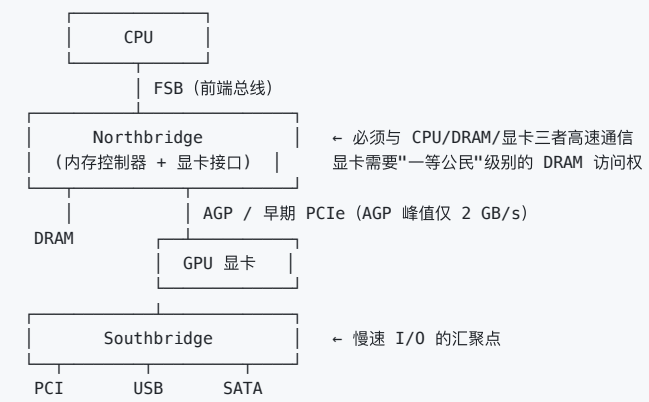
本讲把 CUDA 从"写 kernel"推进到"调度整个系统"：GPU 不再是孤立的计算盒，而是挂在 PCIe/NVLink 互连上、拥有自己内存空间与页表的协处理器（co-processor）。核心问题是**带宽**——PCIe 带宽（16–64 GB/s）比显存带宽（1555 GB/s on A100）低两个数量级，所以一旦程序在拷贝上等待，再快的 kernel 也无济于事。本讲给出两个层次的答案：微观上，用固定内存（pinned memory）让 DMA 真正生效、用 `cudaMemcpyAsync` 让拷贝不阻塞主机；宏观上，用多流（stream）把数据切段，让"搬第 N 段"和"算第 N−1 段"重叠，把总时间从 `T_transfer + T_compute` 压向 `max(T_transfer, T_compute)`。随后本讲把视线转向计算侧：Tensor Core 用"16×16×16 矩阵乘加阵列"把 FP16 峰值做到 FP32 CUDA core 的 16 倍，而深度学习正是靠 GEMM 化的卷积与全连接层吃下这部分算力；最后比较 CUDA 的诸多替代方案（OpenCL / SYCL / HIP / OpenACC / OpenMP offload / Vulkan / WebGPU），并给出 WebGPU+WGSL 与 CUDA 的逐项对照——这正是课程最终项目所使用的编程模型。

核心概念与 GPU 架构图解

概念 1：GPU 在 PC 体系结构中的位置（GPU as Part of the PC Architecture）

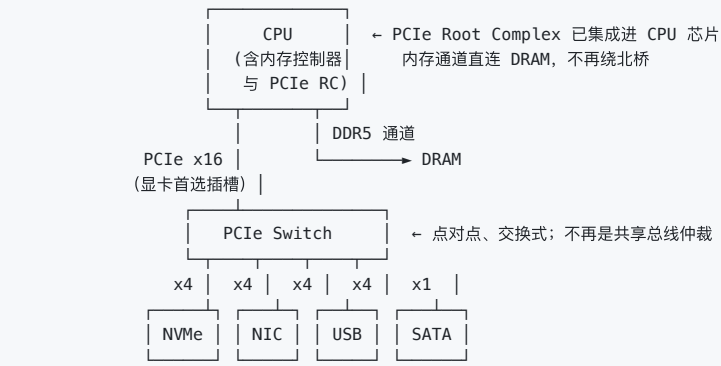
- **定义与目的**：GPU 是插在系统互连总线上的**独立设备**，它有自己的显存（device DRAM）、自己的地址空间和页表、自己的调度器。CPU 侧要通过 PCIe（或 NVLink）这一条"窄管子"才能把数据交给它。理解这个概念的目的只有一个：**算出数据搬移的时间下界**，判断一个应用到底该不该上 GPU，以及上 GPU 后瓶颈会不会落在总线上。
- **直观解释（"它是什么？"）**：把 CPU 想成市中心的大仓库，GPU 想成郊区一座算力超强但只能存自己货的工厂，PCIe 就是连接两地的一条高速公路。工厂里有 108 条产线（108 个 SM），一小时能加工 1555 GB 的原料；可高速公路一小时只放行 16–64 GB 的卡车。**如果每加工 1 字节原料只需要搬 1 字节进出，那么工厂 97% 的产能都在等卡车。** 这就是"带宽是重力（bandwidth is the gravity）"的含义：缓冲、重排、缓存这些技巧能暂时对抗它，但最终性能还是被 speeds and feeds 决定。要摆脱这条公路，只有两种办法：把原料留在工厂里反复用（tiling / 数据复用），或者直接把两座工厂修在一起（NVLink / NVSwitch）。
- **架构/机制图解**：

【经典 PC 体系结构（Northbridge/Southbridge 时代）】



（33 MHz / 32-bit = 132 MB/s 峰值；66 MHz/64-bit = 528 MB/s 峰值）

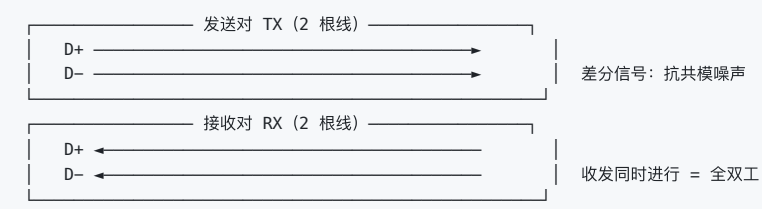
【现代 PCIe switched fabric 体系结构（Northbridge/Southbridge 变成 PCIe switch）】



关键演进：

- 共享总线 + 仲裁 → 交换式点对点，每张卡有专属链路，无仲裁
- "赢得仲裁者成为 bus master" → 报文交换、虚拟通道、带优先级 QoS（实时视频流）
- 设备寄存器映射进 CPU 物理地址空间（boot 时分配地址，Memory Mapped I/O）

【PCIe link 的物理结构：一条 lane = 4 根线 = 2 对差分线对】



每个方向 1 bit/时钟（每对线 8 Gb/s @ Gen3）

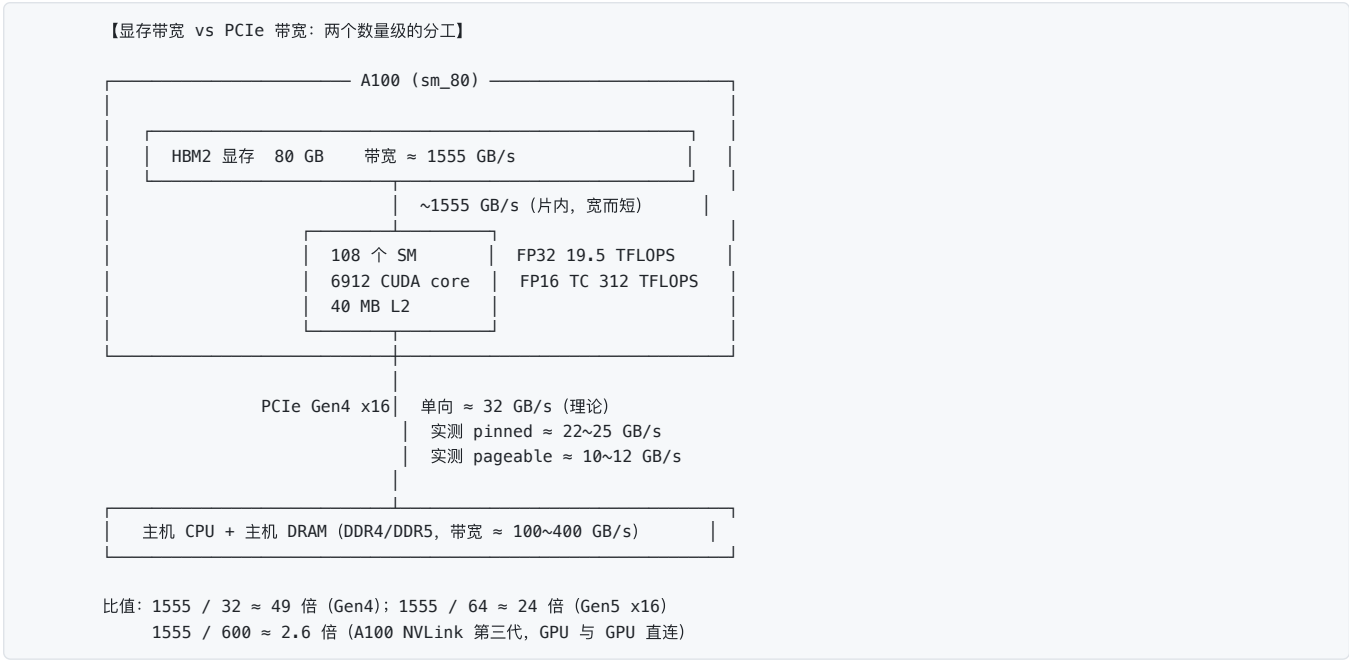
link = n 条 lane：x1 / x2 / x4 / x8 / x12 / x16 / x32

编码开销：

- Gen1/Gen2：8b/10b 编码，20% 开销（10 bit 传 8 bit 数据）
- Gen3 及以后：128b/130b 编码，仅 1.5% 开销
  - 目标不变：维持 DC 平衡 + 足够的状态跳变做时钟恢复
  - 用扰码器（scrambler）保证长串 0/1 的概率极低
  - 每 66 bit 至少一次电平跳变
  - 130 bit 码字中 0/1 各占 63~67 个，非常均衡

Gen3 每 lane 净速率 = 7.8768 GB/s（单向）

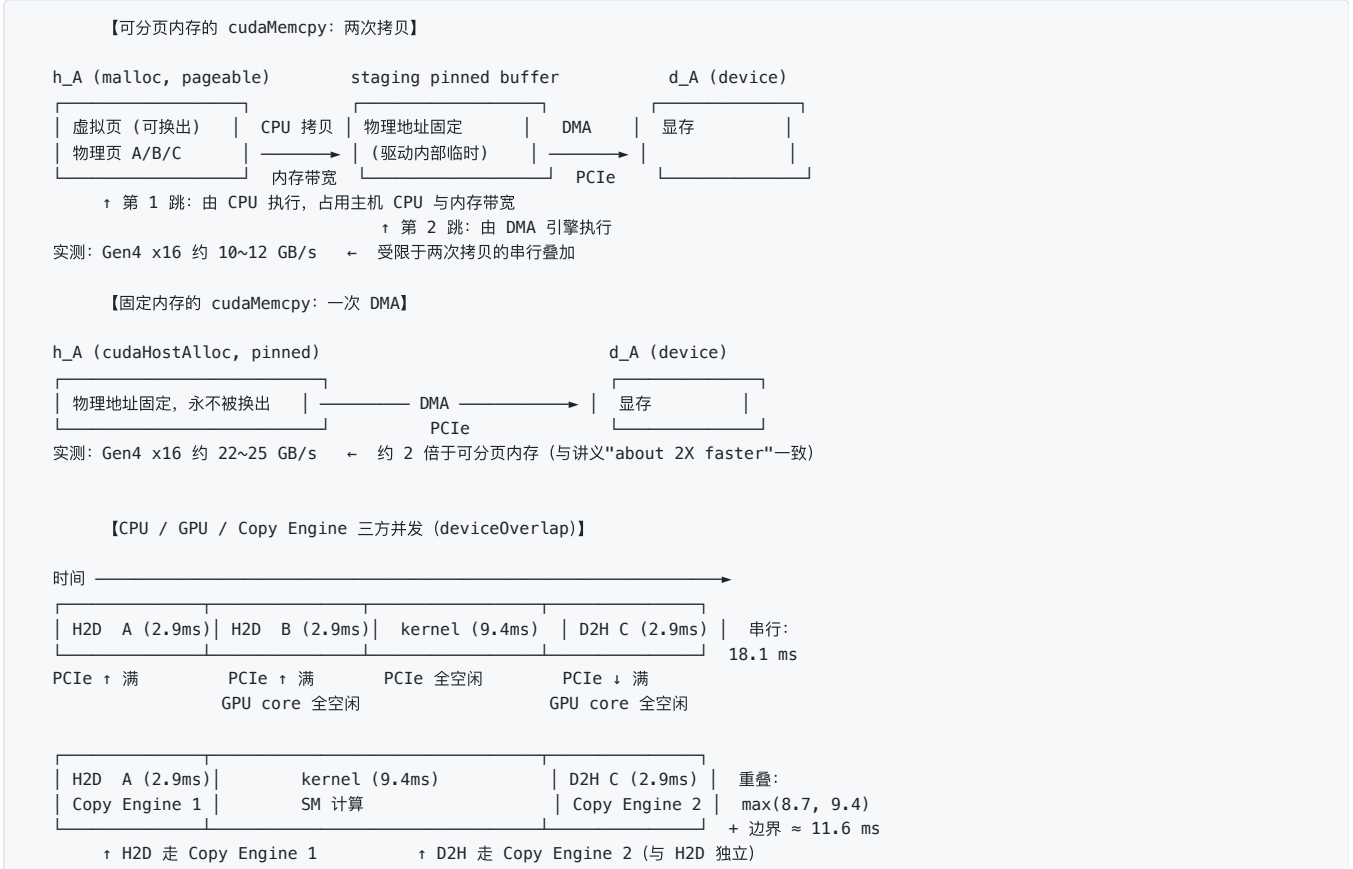
x1 = 0.985 GB/s	x8 = 7.9 GB/s
x2 = 1.97 GB/s	x16 = 15.8 GB/s（单向）
x4 = 3.94 GB/s	x32 = 31.5 GB/s（单向）



- **关键操作与性能特征**：PCIe 各代 x16 单向下行/上行理论带宽为 Gen3 15.8 GB/s、Gen4 31.5 GB/s、Gen5 63 GB/s（128b/130b 编码后的净值，Gen4 为 16 GT/s、Gen5 为 32 GT/s）；实测可用带宽通常只有理论的 70%~80%。对比之下，A100 的显存带宽 1555 GB/s、NVLink 3.0 可达 **600 GB/s**（讲义原文："GPU-to-GPU direct bandwidth to 600 gigabytes per second (GB/s), almost 10X higher than PCIe Gen4"），是 PCIe Gen4 的约 10 倍。历史数据也很能说明趋势：AGP 峰值 2 GB/s、初代 PCI 132 MB/s、NVLink 1.0（P100）160 GB/s 双向（是 PCIe Gen3 x16 的 5 倍）、NVLink 2.0（V100）25 GB/s/lane × 6 lane = 150 GB/s、NVLink 3.0（A100）600 GB/s。延迟方面，一次 PCIe 往返（host→device→host）典型在 5~20 μs 量级，而显存访问延迟只有 400~800 个时钟周期（A100 @1.41 GHz 约 0.3~0.6 μs）——也就是说**一次 PCIe 设备同步的开销约等于 20 次以上的显存访问**，这就是"拷贝次数比拷贝字节数更致命"的原因。

概念 2：主机-设备数据传输优化与固定内存（Data Transfer Optimization & Pinned Memory）

- **定义与目的**：`cudaMemcpy` 是同步（synchronous）语义：它把数据从主机内存搬到设备内存，函数返回时数据已经在设备上。若主机内存来自 `malloc`，它是**可分页内存（pageable memory）**，DMA 引擎不能直接访问（DMA 使用物理地址，而操作系统随时可能把这一页交换出去）。NVIDIA 驱动不得不先偷偷把数据拷到一块临时的**固定内存（pinned / page-locked memory）**再发起 DMA，这就是"两次拷贝"。`cudaMallocHost` / `cudaHostAlloc` 分配的固定内存不可换页，物理地址固定，DMA 可以直接读写，因此带宽通常提升 2 倍以上，也是 `cudaMemcpyAsync` 真正异步的前提。
- **直观解释（"它是什么？"）**：把 DMA 想成一个只会照着"物理门牌号"送货的快递员，而可分页内存像一堆随时会被搬家的出租屋——快递员刚记下门牌号，房子就被拆了。操作系统只好先把包裹搬到一栋永不动迁的仓库（固定内存），再让快递员去取，于是变成了"搬两次"。固定内存就像把货永久卸在自家院子里：快递员直接开车进去装货。代价是这块地皮（物理内存）从此不能挪作他用——固定内存过多会让操作系统没有可换页的空间，**严重过订（over-subscription）会导致整机性能崩塌甚至失败**，因此讲义明确提醒"pinned memory is a limited resource whose over-subscription can have serious consequences"。
- **架构/机制图解**：

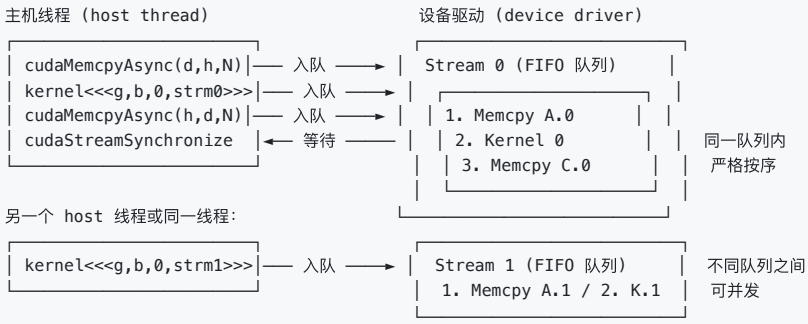


- **关键操作与性能特征**：设备能力要用 `cudaGetDeviceProperties` 查询——`prop.deviceOverlap`（是否支持"kernel 执行同时做 device↔host 拷贝"）、`prop.asyncEngineCount`（异步引擎数量：≥1 表示 H2D 与 kernel 可重叠，≥2 表示 H2D 与 D2H 可同时进行）。`cudaHostAlloc(&ptr, bytes, cudaHostAllocDefault)` 分配、`cudaFreeHost(ptr)` 释放；返回的指针可以像 `malloc` 的返回值一样使用，唯一区别是内存不会被换页。带宽上，Gen4 x16 的实测典型值为：pageable H2D ≈ 10~12 GB/s、pinned H2D ≈ 22~25 GB/s、pinned D2H ≈ 24~26 GB/s；若把一次 384 MB 的向量加法（A、B 各 128 MB 进，C 128 MB 出）的拷贝时间折算出来，pageable 需要约 35 ms，pinned 只需约 16 ms，而 A100 上这个 kernel 本身只需 0.25 ms——**PCIe 传输时间占总时间的 99%**，这就是必须用 pinned + stream 的定量理由。

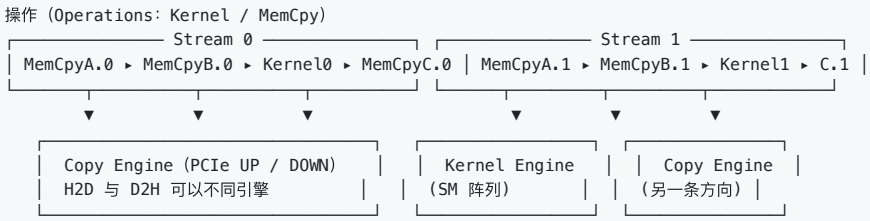
概念 3：CUDA 流、异步拷贝与重叠执行（CUDA Streams, Async Copy & Overlap）

- **定义与目的**：**流（stream）是一个操作队列（queue）**，里面按顺序存放 kernel 启动和 `cudaMemcpy` 请求。同一个流内的操作严格按序执行（前面的 memory copy 结束前，后面的 kernel 不会启动）；**不同流之间没有隐含顺序，可以并发执行**——这是一种任务并行（task parallelism）。流存在的唯一目的是打破"拷贝与计算互相等待"的串行链，让 PCIe 方向与 SM 计算单元同时忙碌。
- **直观解释（"它是什么？"）**：单流就像只有一个窗口的银行：办完一笔业务（拷贝）才能办下一笔（计算），排队的人都闲着。多流相当于开了多个窗口，并且把大额业务拆成小份：柜台 1 正在给 3 号客户数钱（传输第 3 段），柜台 2 同时在给 2 号客户办理（计算第 2 段）。**流水线的理想极限**是：总时间 = `max(传输总时间, 计算总时间)`，而不是两者之和。但要拿到这个极限，必须让每个"阶段"都有活干：只有当段数足够多、且拷贝引擎与计算引擎都能被喂饱时，流水线才真正连续。
- **架构/机制图解**：

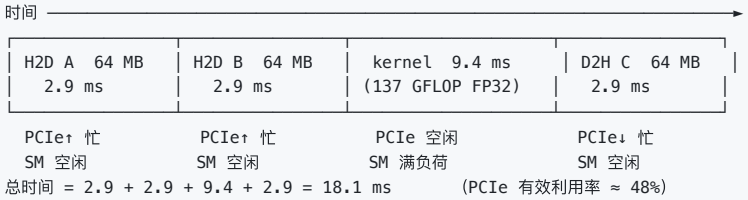
【流的语义：队列 + 引擎】



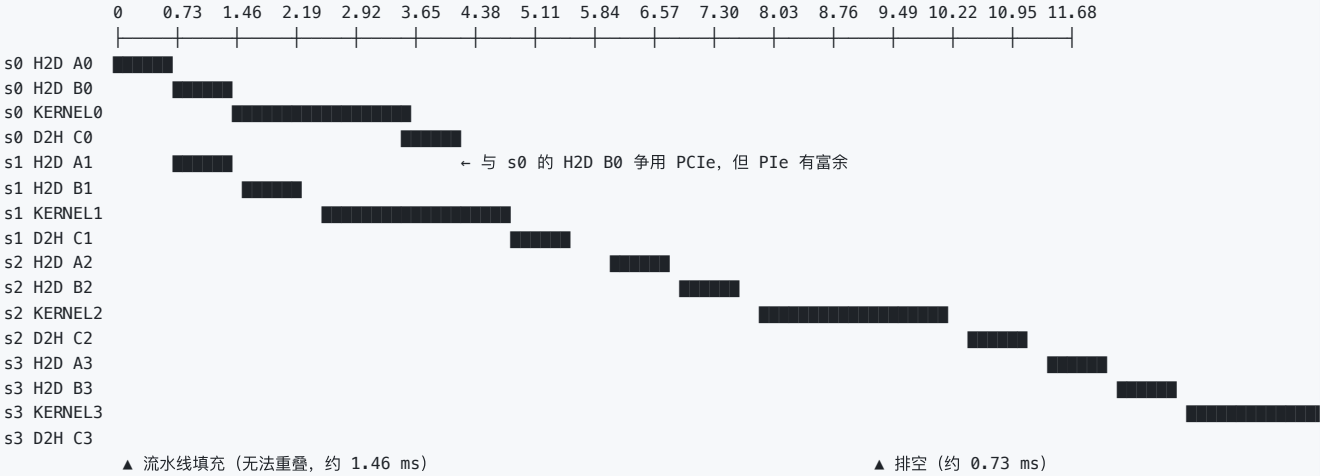
【概念视图：两个流 + 三个硬件引擎】



【串行版时间轴（single stream）：PCIe 只有一个方向在工作，GPU 大量空闲】



【4 段 × 4 流流水线时间轴：拷贝与计算重叠】



```
        【讲义给出的通用流水线公式（以"输入传输 + 计算 + 输出传输"三段为例）】

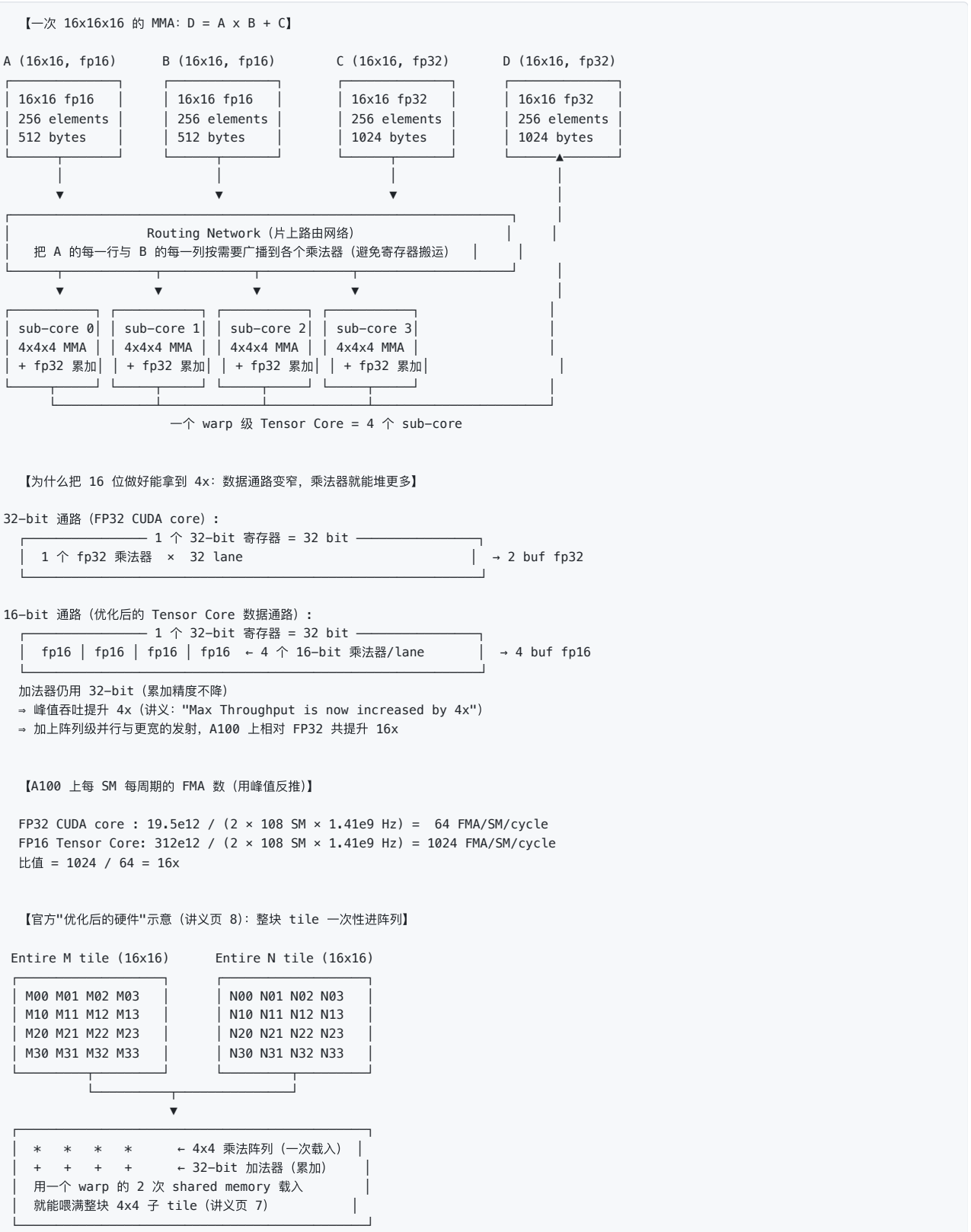
  设：T_in = 输入传输总时间，T_comp = 计算总时间，T_out = 输出传输总时间，N = 段数
  串行时间    = T_in + T_comp + T_out
  流水线时间 = (T_in + T_comp + T_out) / N  +  (N − 1) × max(T_in, T_comp, T_out) / N
               └──── 第一段的填充（无法重叠）────┘    └──── 稳态：每段只花瓶颈阶段的时间 ────┘
  N → ∞ 的极限 = max(T_in, T_comp, T_out)

  讲义例题：T_in = 10 s，T_comp = 10 s，T_out = 10 s，kernel 启动常数开销 1 s
    时间(N) = (20 / N) + 10 + N
      N = 1 : 31 s      N = 2 : 22 s      N = 3 : 19.67 s
      N = 4 : 19 s   ★  N = 5 : 19 s  ★   N = 6 : 19.33 s    N = 8 : 20.5 s
    最优解：4 或 5 段（19 s），加速比 31 / 19 ≈ 1.63x
```

- **关键操作与性能特征**：重点 API 与语义如下。
- `cudaStreamCreate(&s)` / `cudaStreamDestroy(s)`：创建/销毁一个流。`cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking)` 创建**非阻塞流**，它不会与 legacy default stream 做隐式同步。
- `cudaMemcpyAsync(dst, src, bytes, kind, stream)`：**异步拷贝**。只有当主机内存是 pinned 时它才真正异步；若源/目标是 pageable 内存，驱动仍需 staging，调用会（几乎）同步阻塞，重叠收益消失。
- `kernel<<<grid, block, shmem, stream>>>(args)`：把 kernel 投到指定流（第 4 个执行配置参数）。写 `<<<grid, block>>>` 等价于投到默认流。
- `cudaEventRecord(ev, stream)` 与 `cudaEventSynchronize(ev)`：在流中打时间戳/等待点，配合 `cudaEventElapsedTime(&ms, e0, e1)` 测量时间。`cudaEventCreateWithFlags(&e, cudaEventDisableTiming)` 用于纯同步用途，开销更低。
- `cudaStreamWaitEvent(stream, ev, 0)`：**跨流同步**——让 stream 等待 ev 记录的完成点，而不阻塞主机。
- `cudaStreamSynchronize(s)`：阻塞主机直到该流中所有操作完成。
- **默认流的隐式同步**：legacy default stream（也叫 NULL stream / stream 0）是"同步流"——默认流中的操作会等待此前所有其他流中的操作完成，也会阻塞之后其他流中的操作。这会让多流程序意外串行化。用 `--default-stream per-thread` 编译选项、或显式使用 `cudaStreamPerThread`、或所有流都用 `cudaStreamNonBlocking` 创建，可以绕开这个陷阱。
- 硬件并发度：Fermi 支持 16 路并发的 grid，但来自不同流的 kernel 会**复用（multiplex）到同一条硬件工作队列**，只有"流边界"处才能重叠；Kepler 起每个流有独立的硬件工作队列，支持 32 路并发、全流级并发、流间无依赖。当代 GPU 还提供 **Hyper-Q**（每个引擎多条真实队列），让某些流在某引擎上阻塞时其他流仍能推进。
- **段大小选择**：段太小会遇到"执行时间不可能趋近于零"的下界——部分 SM 闲置、warp 数不足以打满 SM、线程数不足一个 warp、block 数太少导致负载不均、kernel 启动本身有固定开销；数据传也有类似的非线性（host 与 DMA 的启动成本）。因此要选**中等大小**的段，最佳值依赖具体 GPU（讲义结论："Use Moderate Segment Size and Device Query"）。
- **三流原则**：讲义指出，在"输入拷贝 → 计算 → 输出拷贝"的链上，若只用两个流，`C.1` 会在拷贝引擎队列里挡住下一轮的 `A.2`、`B.2`（表头阻塞 head-of-line blocking）；要实现**连续流水（continuously pipelined timing）需要三个流**，并按"先发所有 H2D、再发所有 kernel、最后发所有 D2H"的顺序入队，避免读方向被写方向阻塞。

概念 4：Tensor Core 与混合精度（Tensor Cores & Mixed Precision）

- **定义与目的**：Tensor Core 是 NVIDIA 从 Volta（sm_70）开始集成在 SM 里的**专用矩阵乘加阵列（matrix multiply-accumulate unit）**。它的基本算子不是"两个标量相乘再相加"，而是整个小矩阵块：`D = A × B + C`，典型形状 **16×16×16**（WMMA）或底层指令级的 **8×8×4 / m16n8k16**。它解决的问题是：深度学习里 90% 以上的浮点运算都是 GEMM，而 GEMM 的每个乘加对只有 2 个 FLOP 却要读 2 个操作数——用标量 CUDA core 做这件事，性能被寄存器带宽和指令发射率死死限制。Tensor Core 把"取数–相乘–累加"整条数据流硬化（hardwired）在硅片上，配合 **16 位输入 + 32 位累加**的混合精度（mixed precision），把每 SM 每周期的 FP16 FMA 数从 64 提高到 1024。
- **直观解释（"它是什么？"）**：CUDA core 像一屋子熟练工人，每人每周期做一次乘加，指令得一个个发；Tensor Core 像一条**自动化流水线**：整箱原料（16×16 的 fp16 矩阵）倒进去，出来就是整箱成品（16×16 的 fp32 累加结果），单位时间的产出高一个数量级，代价是它只认得这一种箱子。为什么深度学习能忍受 16 位输入？因为神经网络是**统计学习器**：它的输出是"哪一类"的概率排序，而不是精确的数值答案；训练时每一步更新权重都是"沿梯度方向走一小步"，单步权重增量的相对误差只要小于步长本身，就会被后续迭代自然吸收。反过来看，把 784 个像素连同权重一起量化到 fp16，注定的舍入噪声相当于给输入加了极小的抖动——这反而起到正则化作用。**关键是把累加仍然放在 fp32**：点积长度 K 可能是 4096 甚至更大，若累加也用 fp16，误差会随 K 线性增长并迅速淹没结果。所以工业界固定用法是"**低精度输入 × 低精度输入 → 高精度累加**"。
- **架构/机制图解**：



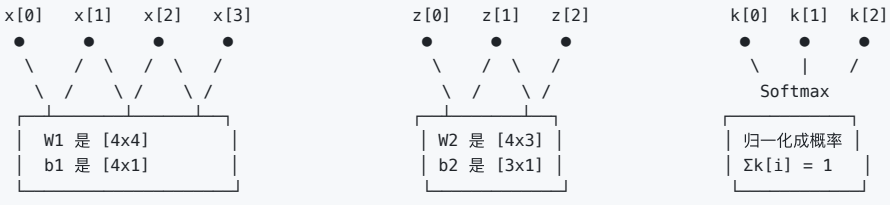
- **关键操作与性能特征**：WMMA API（`#include <mma.h>`，命名空间 `nvcuda::wmma`）的四个关键调用构成一个模板化抽象：
- `wmma::fragment<wmma::matrix_a, 16, 16, 16, __half, wmma::row_major> a_frag;` —— 声明一个 fragment（每个线程持有其中一部分元素的"寄存器分片"）。
- `wmma::load_matrix_sync(a_frag, ptr, ldm);` —— 把内存中的矩阵块搬进 fragment；`ldm` 是主维长度（leading dimension）。16 位类型要求 `ldm` 是 8 的倍数（16 字节对齐）。
- `wmma::mma_sync(acc, a_frag, b_frag, acc);` —— 执行 `acc = a × b + acc`。
- `wmma::store_matrix_sync(ptr, acc, ldm, wmma::mem_row_major);` —— 把累加器写回内存。

- `wmma::fill_fragment(acc, 0.0f)` —— 清零累加器。 此外还有 `wmma::load_matrix_sync(frag, ptr, ldm, wmma::mem_col_major)` 支持列主序、`wmma::store_matrix_sync(ptr, acc, ldm, wmma::mem_col_major)` 控制写出布局，以及 `wmma::fragment<wmma::accumulator, 16, 16, 16, double>`（sm_80 起支持 FP64 与 TF32 等）。 性能数字：A100 上 FP16 Tensor Core 峰值 **312 TFLOPS**（稀疏模式下 624 TFLOPS），是 FP32 CUDA core（19.5 TFLOPS）的 **16 倍**；TF32（19 位尾数的"降精度 FP32"）峰值约 156 TFLOPS，FP64 Tensor Core 约 19.5 TFLOPS。历史上第三代（A100）相对第二代（V100 上 125 TFLOPS）翻了 2.5 倍，并新增 **结构化稀疏（structural sparsity）** 支持，可再翻倍。现实中，一个只用全局内存直接 load 的朴素 WMMA kernel 通常只能跑到 40~60 TFLOPS——因为它的算术强度只有 8 FLOP/B（每次 k 步每 warp 读 1024 B 算 8192 FLOP），受 L2 带宽与 fragment 加载指令吞吐限制；要把 Tensor Core 喂饱，必须像 CUTLASS/cuBLAS 那样把 tile 先经共享内存做多级复用、双缓冲、并让 fragment 加载与 `mma_sync` 交错发射。

概念 5：深度学习中的 GPU 计算：一切皆 GEMM（Deep Learning as GEMM）

- **定义与目的**：深度学习（deep learning）是一种**表示学习（representation learning）**：不再由人手工设计特征，而是让机器从数据中学出"由其他表示所表达的表示"，一层层堆出从简单特征到抽象特征的**深层次（hierarchy）**——输入 → 简单特征 → 抽象特征的层 → 输出映射。GPU 进入这个领域是因为训练循环（training cycle）的迭代速度被彻底改变了：2007 年可编程 GPU 加速了训练循环，之后专用芯片进一步加速，直接引发了计算机视觉、语音识别、机器翻译、自动驾驶的复兴。从工程角度看，本概念要解决的问题是：**把各种神经网络层（全连接、卷积、池化）统一翻译成一种高度优化的算子——GEMM（通用矩阵乘法）**，从而复用整套 tiling、shared memory、Tensor Core 优化。
- **直观解释（"它是什么？"）**：一个感知机（perceptron）就是 `y = sign(W·x + b)` ——一次点积。全连接层就是"很多个感知机排成一行"：输入向量 `x` 乘权重矩阵 `W`，得到输出向量 `y`。一层做完做下一层，就是前向传播（forward propagation / inference）。训练则是**反向传播（backward propagation）**：拿预测与标签算误差 `E`，用链式法则（chain rule）把误差梯度一路传回去，求出 `dE/dW`，再让每个权重沿梯度反方向挪一小步 `θ_{i+1} = θ_i − ε·Δθ`，这就是随机梯度下降（stochastic gradient descent, SGD）。**这些操作的共同骨架都是"矩阵乘矩阵"或"矩阵乘向量"**：`fc1 = W1 · x + b1` 是 GEMV（batch 化后就是 GEMM）；卷积是"滑窗点积"，把输入展开（unroll / im2col）后同样是 GEMM；反向传播算 `dE/dW = (dE/dfc)·xᵀ` 也是 GEMM。所以只要把 GEMM 做到极致，整个深度学习就快了。
- **架构/机制图解**：

【多层感知机 MLP 与术语（讲义页 45~47）】



W[i,j] = 第 i 个输入到第 j 个输出的权重（讲义术语：Entry i,j is weight between ith input and jth output）

【MNIST 手写数字识别的 MLP（讲义页 4）】

28x28 灰度图 = 784 个输入节点 → 10 个隐藏节点 → 10 个输出节点（Digit 0..9）

参数量：784×10 + 10 （L1 权重 + 偏置）
+ 10×10 + 10 （L2 权重 + 偏置）
= 7,960 个参数

公式：k[m] = activation(Σ_c W1[m,c]·x[c] + b1[m])

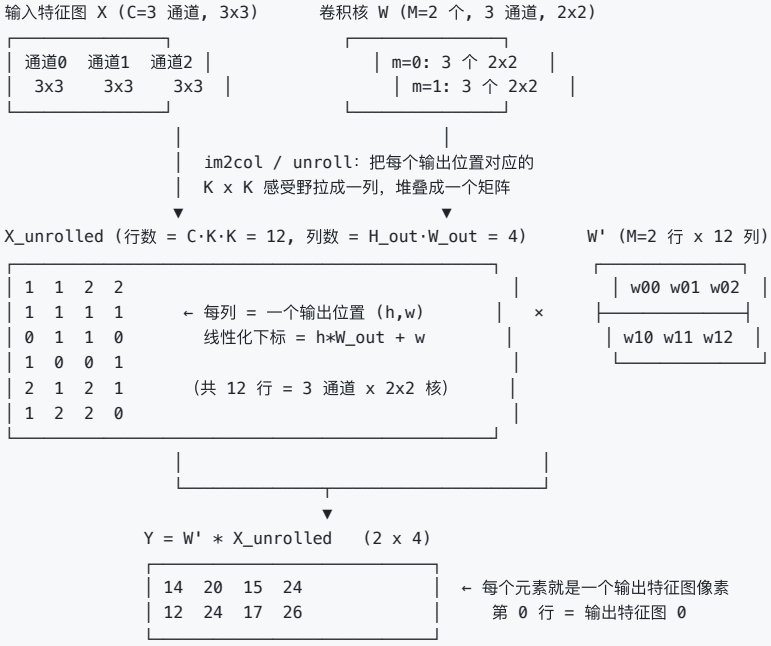
【大图上的 MLP 会爆炸（讲义页 26）】

250 x 250 图像 → 全连接层每个节点 62,500 个权重

节点数与之相当 → 总共约 40 亿个权重

⇒ 计算量与内存都不可接受 ⇒ 需要用卷积做权重共享（weight sharing）

【为什么"卷积 = GEMM"：unrolled / im2col 变换（讲义页 27、31、32）】



代价分析（讲义页 31、32）：unrolled 矩阵的元素总数 = C·K·K·H_out·W_out

原始输入元素数 = C·(H_out+K−1)·(W_out+K−1)

小的 3x3x3 例子：复制因子 = (3·2·2·2·2)/(3·3·3) = 1.78

若采用 tiled 卷积，输入访问量的缩减因子 = K^2·TILE_WIDTH^2 / (TILE_WIDTH+K−1)^2

【卷积层的可并行维度（讲义页 16）】

① 输出特征图之间并行 —— 数量少（M 通常几十到几百），不足以填满 GPU

② 每个输出特征图的所有像素并行 —— 先按行再按列，数量大，但越深的层越小

③ 输入通道维度的归约 —— 需要原子操作或树形归约（讲义："need atomic operation or tree reduction"）

⇒ 不同层需要不同策略（讲义："Different layers may demand different strategies"）

【反向传播的两种并行模式】



```
                                θ = {W, b} 固定              求 dE/dθ，更新 θ ← θ − ε·Δθ

  每一步都是 GEMM：
    dE/dW1 = (dE/dfc1) · xᵀ        形状 [out1 x in] = [out1 x batch] · [batch x in]
    dE/dx  = W1ᵀ · (dE/dfc1)       用于继续往前传
  并行的粒度：
    · batch 维度完全独立 —— mini-batch SGD 的天然并行（讲义页 21："Balance between
      accuracy of gradient estimation and parallelism"）
    · 输出神经元维度独立
    · 归约维（输入维度 / 通道维 / 空间核）需要块内归约或原子累加
```

- **关键操作与性能特征**：把这些层写成 GEMM 后的性能特征是**高算术强度**。以 MNIST 的第一层为例（batch = 1024，输入 784，输出 1024）：FLOPs = 2×1024×784×1024 = 1.64 GFLOP；访存 = X（1024×784×4 B = 3.2 MB）+ W（784×1024×4 B = 3.2 MB）+ Y（1024×1024×4 B = 4.1 MB）≈ 10.5 MB；算术强度 ≈ 1.64e9 / 1.05e7 ≈ **156 FLOP/B**。A100 的 Roofline 拐点 = 19.5e12 / 1555e9 ≈ **12.5 FLOP/B**，156 ≫ 12.5，所以这是**计算受限（compute-bound）**的，GPU 正好擅长；而 Tensor Core 把可用的计算上限提到 312 TFLOPS，拐点变为 312e12/1555e9 ≈ 200 FLOP/B，于是即便像 Vision Transformer 里那样中等算术强度的层，也可能掉回内存受限。训练与推理的差别也体现在算术强度上：反向传播需要保存前向的中间激活值（activation），显存占用约是推理的 3~4 倍（需要额外存激活与梯度），这就是 batch size 受限、要用梯度检查点（gradient checkpointing）的原因。数值精度方面，训练通常用 TF32/BF16 前向+反向、FP32 主权重（master weights）；FP16 训练需要 loss scaling 来避免梯度下溢。

概念 6：CUDA 的替代方案（Alternatives to CUDA）

- **定义与目的**：CUDA 只是一个模型。加速计算已经不再是问题，因为 GPU 供应商包括 NVIDIA、AMD、Intel、Samsung、Apple、Qualcomm、ARM 等等；但每个厂商的硬件都有自己的原生存程，于是出现了一整层"可移植编程模型"，目标是用一份代码跑到不同厂商、甚至不同种类的加速器（GPU / CPU / DSP / FPGA）上。这些模型要解决的核心矛盾是**可移植性（portability）× 性能上限（performance ceiling）× 编程抽象层次（abstraction level）**这三者的三角权衡。
- **直观解释（"它是什么？"）**：把 CUDA 想成"某品牌的专用工具"：最贴合硬件、性能天花板最高、但只能在自家机器上用。OpenCL/SYCL 像"国际标准插座转换头"：通用，但要经过一层抽象，某些厂商的独家特性（如 Tensor Core 的 WMMA 指令）就用不上，性能上限往往低一档。HIP 像"同义词词典"：语法跟 CUDA 几乎一样，`cudaMalloc` → `hipMalloc`，一套源码通过 `hipify` 工具自动改写，在 AMD 和 NVIDIA 上都能编，是迁移成本最低的路。OpenACC/OpenMP target offload 像"给普通代码贴便利贴"：在串行循环前面加一行 pragma 就能跑在 GPU 上，编程门槛最低，但性能高度依赖编译器质量，天花板也最低。WebGPU 像"把 GPU 装进浏览器"：天生可移植（Windows/macOS/Linux/Android/iOS/浏览器全通吃），用 WGSL 写着色器，牺牲一部分底层控制力换取零安装、易分发。
- **架构/机制图解**：

【加速编程模型的时间线（讲义页 4）】

```
1992 — OpenGL
1995 — DirectX
2002 — GPGPU（用图形 API 做通用计算的黑客时代）
2007 — CUDA                    ★ 第一个 C 风格、完整的 GPU 通用计算模型
2008 — OpenCL 1.0（Apple 发起，AMD/IBM/Qualcomm/Intel/NVIDIA 支持）
2012 — OpenACC
2013 — C++ AMP、RenderScript
2014 — Metal（Apple）、SYCL（Khronos，基于 C++ 模板的单一源码模型）
2016 — Vulkan、ROCm HIP、OpenMP 4.5 target offload
2017 — OpenCL 2.2；2018 Apple 宣布放弃 OpenCL
```

【所有加速 API 的共同特征（讲义页 6）——理解这一点，迁移就是"换词"】

```
硬件侧                          软件侧
┌──────────────────────┐     ┌──────────────────────────────────┐
│ 层次化的轻量核心      │     │ 面向 kernel 的加速               │
│ (hierarchy of cores) │     │ 设备内存 vs 主机内存（两份地址空间）│
│ 本地暂存器内存        │     │ 软件管理的内存（无硬件一致性）     │
│ (scratchpad / __local) │   │ Grid / Block / Thread 三级层次   │
│ 缺乏硬件缓存一致性    │     │ 块同步并行（Bulk Synchronous     │
│ (no HW coherence)    │     │ Parallelism, BSP）               │
│ 全局原子操作慢        │     │                                  │
│ 多线程                │     │                                  │
└──────────────────────┘     └──────────────────────────────────┘
```

【OpenCL 的内存模型（讲义页 8~9）：与 CUDA 一一对应】

```
CUDA                OpenCL                      说明
──────────────────────────────────────────────────────────────────
__global__ 函数      __kernel void myGEMM2(参数表) kernel 用 __kernel 修饰
__shared__ float     __local float               block 内的暂存器
__constant__         __constant                  常量内存
寄存器/局部数组       __private                   私有内存
blockIdx/get_group_id   work-group（工作组）      相同语义
threadIdx/get_local_id  work-item（工作项）       相同语义
__syncthreads()      barrier(CLK_LOCAL_MEM_FENCE) 屏障语义相同
cudaMalloc           clCreateBuffer              API 风格从 C 函数变成句柄
<<<grid, block>>>    clEnqueueNDRangeKernel      启动方式差异最大
```

【OpenACC 的抽象机模型：gang / worker / vector（讲义页 24、27、28）】

```
┌──────────────────────────────────────────────────────────┐
│  #pragma acc parallel num_gangs(1024) num_workers(32)    │
│  ┌──────────── gang 0 ────────────┐  ┌──────────── gang 1 ────────────┐  │
│  │ worker0 worker1 到 worker31 │  │ worker0 worker1 到 worker31 │  │
│  └──────────────────────────────┘  └──────────────────────────────┘  │
│    共 1024 个 gang，每个 32 个 worker，合计 32K 个 worker          │
└──────────────────────────────────────────────────────────┘
```

对应关系：gang ≈ CUDA block　worker ≈ CUDA thread（常映射为 vector lane）
#pragma acc loop gang　→ 循环的迭代被分配到各个 gang（不是冗余执行）
没有 pragma 的语句　→ 被所有 gang 冗余执行（redundant execution）
注意：OpenACC 目前不允许用户指定跨线程的同步（讲义页 23）

OpenACC 版的矩阵乘法（讲义页 16）——与串行代码只差两行 pragma：

```
  #pragma acc parallel loop copyin(M[0:Mh*Mw]) copyin(N[0:Nw*Mw]) copyout(P[0:Mh*Nw])
  for (int i = 0; i < Mh; i++) {
      #pragma acc loop
      for (int j = 0; j < Nw; j++) {
          float sum = 0.0f;
          for (int k = 0; k < Mw; k++) {
              sum += M[i * Mw + k] * N[k * Nw + j];
          }
          P[i * Nw + j] = sum;
      }
  }
```

copyin / copyout 子句负责描述矩阵数据如何在主存与设备内存之间搬移。

现实提醒（讲义页 21、22）：
- 很难写出"有 pragma 和没 pragma 都正确且都高效"的代码
- 有些 OpenACC 程序在忽略 pragma 后会行为不同甚至出错

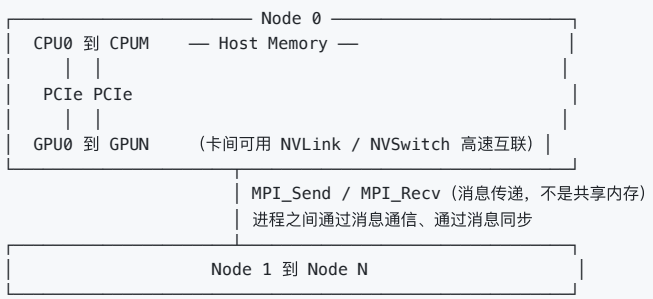
· 部分 pragma 只是"提示（hint）"，编译器可以不听
· 性能高度依赖编译器质量（比 CUDA / OpenCL 更依赖）

【可移植性 / 性能上限 / 抽象层次 三角对比】

模型	可移植性	性能上限	抽象层次	典型使用者
CUDA	NVIDIA 独占（x86/ARM 主机均可）	★★★★★ 可达硬件峰值（Tensor Core 全特性）	低（显式线程）	研究者/极致性能 HPC 内核
HIP/ROCm	AMD + NVIDIA（同源码）hipify 自动转换	★★★★☆ AMD 上接近峰值	低（≈CUDA）	需要双厂商部署（语法几乎相同）
OpenCL	★★★★★ 全厂商全设备（CPU/GPU/DSP/FPGA）	★★★☆☆ 受运行时抽象限制	中（显式线程 + 显式内存对象）	跨厂商产品
SYCL	★★★★☆ 单一源码 C++（Intel DPC++/AdaptiveCpp）	★★★★☆ 后端可下沉到原生（可调用 CUDA 后端）	中高（C++ 模板 + 单一源码）	需要现代 C++ + 跨厂商
OpenACC	★★★★☆ 编译器支持处（PGI/NVHPC/GCC）	★★☆☆☆ 依赖编译器自动	高（pragma）	科学家/遗留代码低改动量
OpenMP target offload	★★★★☆ 主流编译器（GCC/Clang/Intel）	★★☆☆☆ 同样依赖编译器	高（pragma）	已有 OpenMP 代码低改动量
Vulkan/ Metal/ DirectCompute	★★★☆☆ 桌面/移动全平台但不含 Web	★★★☆☆ 无 Tensor Core 类专用矩阵指令的直接映射	中（SPIR-V 着色器）	游戏引擎/图形混合渲染计算
WebGPU/WGSL	★★★★★ 浏览器 + 原生全平台（Win/mac/Linux/Android/iOS）	★★☆☆☆ 受浏览器沙箱与安全校验约束，无子组矩阵乘（尚无 matmul 指令）	中（着色器 + 显式资源绑定）	教学/网页部署零安装分发

注：本课程"Lab 用 WebGPU、最终项目用 RAI"（课程目录原文："C Programming Language and CUDA Software Development Kit, WebGPU for labs, RAI for final project"）。

【多节点 / 多 GPU 的层次（讲义页 36~49）：CUDA + MPI】



CUDA 侧的多 GPU 支持：
  cudaSetDevice(i)　　设置"当前 GPU"
  可以在异步调用（kernel / memcpy）还在跑的时候切换当前设备
  cudaSetDevice(0); kernel<<<g,b>>>(args); cudaMemcpyAsync(d,h,N,kind,s);
  cudaSetDevice(1); kernel<<<g,b>>>(args);　← 合法且常用
MPI 侧：MPI_Init / MPI_Comm_rank / MPI_Comm_size / MPI_Send / MPI_Recv /
　　MPI_Barrier / MPI_Finalize；启动方式 `mpirun -np X`
在 compute 进程里可以继续把工作卸载到 GPU（讲义页 49 的原话注释：
"Or, can offload to GPU here"）

- **关键操作与性能特征**：三种模型层次的代表代码成本可以量化：把一份 CUDA tiled matmul 移植到 OpenCL，源码改动量在 10%~20%（`__global__` → `__kernel`、`threadIdx` → `get_local_id`、`__shared__` → `__local`、`__syncthreads()` → `barrier(CLK_LOCAL_MEM_FENCE)`、`cudaMalloc` → `clCreateBuffer`）；移植到 HIP 的改动量接近 0（`hipify` 做源到源转换）；移植到 OpenACC 则**代码几乎等于串行版本**，只需两行 pragma，但性能方差极大——同一个 pragma 在不同编译器上可能相差 3~5 倍。WebGPU 在性能上受两道额外约束：一是浏览器/驱动的**着色器校验**限制了复杂控制流与动态索引，二是 WGSL 目前**没有子组矩阵乘（subgroup matrix multiply）指令**的标准绑定，因此 Tensor Core 无法直接使用，矩阵乘只能靠 `f32` / `f16` 的 `dot` 与手工展开近似。实际工程中的选择顺序通常是：先看目标平台（要不要跨厂商、要不要浏览器分发），再看性能需求（要不要 Tensor Core），最后看团队维护成本。

概念 7：WebGPU 编程模型与 CUDA 的对应关系（WebGPU / WGSL vs CUDA）

- **定义与目的**：WebGPU 是 W3C 标准化的、面向**浏览器与原生**的现代 GPU API，它把 GPU 的能力抽象成三类对象：**设备（GPUDevice）、资源（GPUBuffer / GPUTexture）、命令（GPUCommandEncoder → ComputePass / RenderPass）**。计算着色器用 **WGSL（WebGPU Shading Language）** 编写。它解决的问题是"零安装分发 + 跨平台"：一份 WGSL 代码可以在 Windows（D3D12）、macOS/iOS（Metal）、Linux/Android（Vulkan）以及浏览器中运行。ECE408 的实验环境就用 WebGPU，最终项目用构建在其上的 RAI 框架。
- **直观解释（"它是什么？"）**：CUDA 像"自己开车"：你可以直接控制油门、挡位、路线。WebGPU 像"坐公交"：有固定的站点（命令编码器、pass）和票务规则（资源绑定、校验），你不能随便变道，但你能确定这趟车在每个城市都有。对已经会 CUDA 的人，迁移的关键认知是：**WebGPU 的所有概念在 CUDA 里都有对应物，只是名字、组织方式与"谁负责什么"变了**。最大的一处结构性差异是：CUDA 里每个线程只写"我算哪个元素"，block/grid 的索引

是内建的只读变量；WebGPU 里同样如此（`global_invocation_id` / `local_invocation_id` / `workgroup_id`），但**数据进出 GPU 的路径更长**：必须在 `GPUQueue` 上显式 `writeBuffer` / `copyBufferToBuffer`（对应 `cudaMemcpy`）或 `mapAsync` 读回（对应 `cudaMemcpy` D2H + 同步）。另一个差异是 WebGPU **没有默认流的概念**：命令编码器提交后按提交顺序执行，想并行要靠多个 command buffer 与多个队列（目前大多数实现只有一个队列，因此"多流重叠"在 WebGPU 里主要通过一次提交内的多个 pass 与异步 `mapAsync` 实现）。

- **架构/机制图解**：



- **关键操作与性能特征**：迁移时需要记住的定量差异有三条。第一，**工作组大小上限**：CUDA block 上限 1024 线程，WebGPU 只保证 256（`maxComputeInvocationsPerWorkgroup` 通常为 256，部分实现为 1024），所以同样的问题规模需要更多的 workgroup，块内归约的层次更深（256 → 128 → 64 → 32 → warp shuffle 阶段）。第二，**没有默认流同步语义**：CUDA 里忘记 `cudaDeviceSynchronize()` 读到的是坏数据；WebGPU 里

`queue.submit` 后必须 `await queue.onSubmittedWorkDone()` 或等 `mapAsync` 的 promise resolve，否则 `getMappedRange()` 读到的是旧内容——但错误表现完全不同（Promise 未等待通常不会崩，而是静默读到 0）。第三，**没有子组矩阵乘**：WGSL 的 `f16` 支持需要 `shader-f16` 扩展特性，且没有 `mma` 类指令，因此矩阵乘在 WebGPU 上通常靠向量点积 `dot(vec4<f32>, vec4<f32>)` 手工展开到 4 路并行来提高算术强度；这也意味着同样的 GEMM 在 WebGPU 上的 TFLOPS 上限远低于 A100 的 Tensor Core，但在教学和中等规模推理场景下完全够用。第四，性能特征上 WebGPU 与 CUDA 的瓶颈模型是完全一致的：同样受合并访存（coalesced access）、workgroup storage 的 bank 冲突（WebGPU 的实现同样按 32 bank × 4 B 组织 workgroup 存储）、以及占用率（resident workgroup 数）支配——**在 CUDA 上学到的优化直觉可以 100% 迁移过去**。

代码示例与性能分析

代码示例 1：多流重叠数据传输与计算的完整程序（Pinned vs Pageable + 串行 vs 流水线）

```
// 文件: stream_overlap.cu
// 编译: nvcc -O3 -arch=sm_80 stream_overlap.cu -o stream_overlap
// 运行: ./stream_overlap
//
// 本程序做三件事:
//   (1) 对比 pageable(malloc) 与 pinned(cudaHostAlloc) 主机内存的 cudaMemcpy 带宽
//   (2) 对比"串行版"(一次拷完 -> kernel -> 拷回) 与"4 段 x 4 流流水线版"的总耗时
//   (3) 用 cudaEvent 精确计时, 并逐段验证数值正确性
//
// 为什么 kernel 里要加一个人工算力负载(heavyVecAdd)?
//   纯 vecAdd 的算术强度只有 0.167 FLOP/B, kernel 时间 ~0.25 ms,
//   而 384 MB 数据过 PCIe 需要 ~16 ms —— 重叠与否看不出差别。
//   把每个元素的 FMA 次数调到 4096, kernel 时间抬到 ~9.4 ms,
//   与 8.7 ms 的传输时间同量级, 重叠收益才可见(这正是讲义"三段流水线"的标准模型)。

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define WORK_PER_ELEMENT 4096                       /* 每个元素的 FMA 次数 */
#define N_ELEMENTS       (16 * 1024 * 1024)         /* 16.7M 个 float = 64 MB */
#define N_BYTES          ((size_t)N_ELEMENTS * sizeof(float))
#define N_SEGMENTS       4                          /* 切成 4 段, 4 个 stream */
#define SEG_ELEMENTS     (N_ELEMENTS / N_SEGMENTS)
#define BLOCK_SIZE       256
#define GRID_SIZE        2048                       /* 每段 2048/4=512 个 block */

/* ============================ kernel ============================ */
__global__ void heavyVecAdd(const float * __restrict__ a,
                            const float * __restrict__ b,
                            float * __restrict__ c,
                            long n)
{
    long i = (long)blockIdx.x * blockDim.x + threadIdx.x;
    const long stride = (long)gridDim.x * blockDim.x;

    for (; i < n; i += stride) {
        const float x = a[i];
        const float y = b[i];
        /* 4 条独立累加链: 打破 FMA 的相关性, 让 FMA 吞吐而不是 FMA 延迟成为瓶颈 */
        float acc0 = 0.0f, acc1 = 0.0f, acc2 = 0.0f, acc3 = 0.0f;
#pragma unroll 16
        for (int k = 0; k < WORK_PER_ELEMENT; k += 4) {
            acc0 = fmaf(acc0, 1.0000001f, x);
            acc1 = fmaf(acc1, 1.0000001f, y);
            acc2 = fmaf(acc2, 1.0000001f, x);
            acc3 = fmaf(acc3, 1.0000001f, y);
        }
        c[i] = (acc0 + acc1) + (acc2 + acc3);
    }
}

/* 主机端参考实现: 必须与 kernel 采用完全相同的运算顺序 */
static float host_reference(float x, float y)
{
    float acc0 = 0.0f, acc1 = 0.0f, acc2 = 0.0f, acc3 = 0.0f;
    for (int k = 0; k < WORK_PER_ELEMENT; k += 4) {
        acc0 = fmaf(acc0, 1.0000001f, x);
        acc1 = fmaf(acc1, 1.0000001f, y);
        acc2 = fmaf(acc2, 1.0000001f, x);
        acc3 = fmaf(acc3, 1.0000001f, y);
    }
    return (acc0 + acc1) + (acc2 + acc3);
}
```

```
/* ==================== 实验 A: 拷贝带宽测量 ==================== */
static double copy_bandwidth_test(int pinned)
{
    const size_t bytes = 64ull * 1024ull * 1024ull;   /* 64 MB */
    const int    reps  = 8;
    void *h = NULL;
    void *d = NULL;

    if (pinned) {
        CUDA_CHECK(cudaHostAlloc(&h, bytes, cudaHostAllocDefault));
    } else {
        h = malloc(bytes);
        if (h == NULL) {
            fprintf(stderr, "malloc failed\n");
            exit(EXIT_FAILURE);
        }
    }
    CUDA_CHECK(cudaMalloc(&d, bytes));
    memset(h, 0x5A, bytes);

    cudaEvent_t e0, e1;
    CUDA_CHECK(cudaEventCreate(&e0));
    CUDA_CHECK(cudaEventCreate(&e1));

    /* 预热: 把首轮 page fault / 驱动建表等一次性开销排除在计时之外 */
    CUDA_CHECK(cudaMemcpy(d, h, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaEventRecord(e0));
    for (int r = 0; r < reps; ++r) {
        CUDA_CHECK(cudaMemcpy(d, h, bytes, cudaMemcpyHostToDevice));
    }
    CUDA_CHECK(cudaEventRecord(e1));
    CUDA_CHECK(cudaEventSynchronize(e1));

    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, e0, e1));
    const double gbps = (double)bytes * (double)reps / ((double)ms / 1000.0) / 1e9;

    CUDA_CHECK(cudaEventDestroy(e0));
    CUDA_CHECK(cudaEventDestroy(e1));
    CUDA_CHECK(cudaFree(d));
    if (pinned) {
        CUDA_CHECK(cudaFreeHost(h));
    } else {
        free(h);
    }
    return gbps;
}

/* ==================== 数据初始化与验证 ==================== */
static void init_arrays(float *a, float *b, long n)
{
    for (long i = 0; i < n; ++i) {
        a[i] = 0.5f + 1.0e-6f * (float)(i % 1024);
        b[i] = 0.25f + 1.0e-6f * (float)(i % 512);
    }
}

static int verify(const float *h_c, const float *h_a, const float *h_b,
                  long n, const char *tag)
{
    const int ncheck = 16;
    int bad = 0;
    for (int t = 0; t < ncheck; ++t) {
        const long idx = (long)((double)t / (double)ncheck * (double)n);
        const float ref = host_reference(h_a[idx], h_b[idx]);
        const float got = h_c[idx];
        const float rel = fabsf(got - ref) / (fabsf(ref) + 1.0e-6f);
        if (rel > 1.0e-3f) {
            printf("    [%s] MISMATCH idx=%ld ref=%.6f got=%.6f rel=%.3e\n",
                   tag, idx, ref, got, rel);
            ++bad;
        }
    }
    printf("    [%s] 抽样验证: %s  (%d/%d 个元素相对误差 < 1e-3)\n",
           tag, (bad == 0) ? "PASS" : "FAIL", ncheck - bad, ncheck);
    return (bad == 0);
```

```
}

/* ============================== main ============================== */
int main(void)
{
    /* ---------- 设备能力查询(讲义: Use Moderate Segment Size and Device Query) ---------- */
    int dev_count = 0;
    CUDA_CHECK(cudaGetDeviceCount(&dev_count));
    printf("CUDA 设备数: %d\n", dev_count);
    for (int i = 0; i < dev_count; ++i) {
        cudaDeviceProp p;
        CUDA_CHECK(cudaGetDeviceProperties(&p, i));
        printf("  [%d] %-30s CC %d.%d  SM=%d  deviceOverlap=%d  asyncEngineCount=%d\n",
               i, p.name, p.major, p.minor, p.multiProcessorCount,
               p.deviceOverlap, p.asyncEngineCount);
    }

    /* ============ 实验 A: pageable vs pinned 拷贝带宽 ============ */
    printf("\n===== 实验 A: 主机内存类型对 cudaMemcpy H2D 带宽的影响 =====\n");
    const double bw_page = copy_bandwidth_test(0);
    const double bw_pin  = copy_bandwidth_test(1);
    printf("  pageable  (malloc)        : %7.2f GB/s\n", bw_page);
    printf("  pinned    (cudaHostAlloc) : %7.2f GB/s   (%.2fx)\n", bw_pin, bw_pin / bw_page);

    /* ============ 准备数据 ============ */
    float *h_a = NULL, *h_b = NULL, *h_c = NULL, *h_c2 = NULL;
    CUDA_CHECK(cudaHostAlloc((void **)&h_a, N_BYTES, cudaHostAllocDefault));
    CUDA_CHECK(cudaHostAlloc((void **)&h_b, N_BYTES, cudaHostAllocDefault));
    CUDA_CHECK(cudaHostAlloc((void **)&h_c, N_BYTES, cudaHostAllocDefault));
    CUDA_CHECK(cudaHostAlloc((void **)&h_c2, N_BYTES, cudaHostAllocDefault));
    init_arrays(h_a, h_b, N_ELEMENTS);

    float *d_a = NULL, *d_b = NULL, *d_c = NULL;
    CUDA_CHECK(cudaMalloc((void **)&d_a, N_BYTES));
    CUDA_CHECK(cudaMalloc((void **)&d_b, N_BYTES));
    CUDA_CHECK(cudaMalloc((void **)&d_c, N_BYTES));

    cudaEvent_t ev_start, ev_stop;
    CUDA_CHECK(cudaEventCreate(&ev_start));
    CUDA_CHECK(cudaEventCreate(&ev_stop));

    /* ============ 版本 1: 串行(单流) ============ */
    printf("\n===== 实验 B: 串行版(单流, 一次拷完 -> kernel -> 拷回) =====\n");
    CUDA_CHECK(cudaMemset(d_c, 0, N_BYTES));
    CUDA_CHECK(cudaEventRecord(ev_start));
    CUDA_CHECK(cudaMemcpy(d_a, h_a, N_BYTES, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_b, h_b, N_BYTES, cudaMemcpyHostToDevice));
    heavyVecAdd<<<GRID_SIZE, BLOCK_SIZE>>>(d_a, d_b, d_c, N_ELEMENTS);
    CUDA_CHECK(cudaMemcpy(h_c, d_c, N_BYTES, cudaMemcpyDeviceToHost));
    CUDA_CHECK(cudaEventRecord(ev_stop));
    CUDA_CHECK(cudaEventSynchronize(ev_stop));

    float ms_serial = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms_serial, ev_start, ev_stop));
    printf("  总耗时: %.3f ms\n", ms_serial);
    const int ok_serial = verify(h_c, h_a, h_b, N_ELEMENTS, "serial");

    /* ============ 版本 2: 4 段 x 4 流流水线 ============ */
    printf("\n===== 实验 C: 4 段 x 4 stream 流水线版 =====\n");
    cudaStream_t streams[N_SEGMENTS];
    for (int s = 0; s < N_SEGMENTS; ++s) {
        /* cudaStreamNonBlocking: 不与 legacy default stream 做隐式同步 */
        CUDA_CHECK(cudaStreamCreateWithFlags(&streams[s], cudaStreamNonBlocking));
    }

    const size_t seg_bytes = N_BYTES / (size_t)N_SEGMENTS;
    CUDA_CHECK(cudaMemset(d_c, 0, N_BYTES));
    CUDA_CHECK(cudaEventRecord(ev_start));
    for (int s = 0; s < N_SEGMENTS; ++s) {
        const size_t off = (size_t)s * seg_bytes;
        const char *ph_a = (const char *)h_a + off;
        const char *ph_b = (const char *)h_b + off;
        char       *ph_c = (char *)h_c2 + off;
        const char *pd_a = (const char *)d_a + off;
        const char *pd_b = (const char *)d_b + off;
        char       *pd_c = (char *)d_c + off;

        /* 同一个 stream 内: H2D -> kernel -> D2H 严格按序 */
```

```
        CUDA_CHECK(cudaMemcpyAsync((void *)pd_a, ph_a, seg_bytes,
                                   cudaMemcpyHostToDevice, streams[s]));
        CUDA_CHECK(cudaMemcpyAsync((void *)pd_b, ph_b, seg_bytes,
                                   cudaMemcpyHostToDevice, streams[s]));
        heavyVecAdd<<<GRID_SIZE / N_SEGMENTS, BLOCK_SIZE, 0, streams[s]>>>(
            (const float *)pd_a, (const float *)pd_b, (float *)pd_c, SEG_ELEMENTS);
        CUDA_CHECK(cudaMemcpyAsync(ph_c, pd_c, seg_bytes,
                                   cudaMemcpyDeviceToHost, streams[s]));
    }
    for (int s = 0; s < N_SEGMENTS; ++s) {
        CUDA_CHECK(cudaStreamSynchronize(streams[s]));
    }
    CUDA_CHECK(cudaEventRecord(ev_stop));
    CUDA_CHECK(cudaEventSynchronize(ev_stop));

    float ms_pipe = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms_pipe, ev_start, ev_stop));
    printf("  总耗时: %.3f ms\n", ms_pipe);
    const int ok_pipe = verify(h_c2, h_a, h_b, N_ELEMENTS, "pipeline");

    /* ============ 结果汇总 ============ */
    int identical = 1;
    for (long i = 0; i < N_ELEMENTS; i += 997) {
        if (h_c[i] != h_c2[i]) { identical = 0; break; }
    }
    printf("\n===== 结果汇总 =====\n");
    printf("  串行版         : %8.3f ms\n", ms_serial);
    printf("  流水线版(4 段) : %8.3f ms\n", ms_pipe);
    printf("  加速比         : %8.3fx   (理论上限 max(T_transfer,T_compute)/(T_transfer+T_compute))\n",
           ms_serial / ms_pipe);
    printf("  两版本输出一致 : %s\n", identical ? "YES" : "NO");
    printf("  验证结果       : serial=%s  pipeline=%s\n",
           ok_serial ? "PASS" : "FAIL", ok_pipe ? "PASS" : "FAIL");

    /* ============ 释放资源 ============ */
    for (int s = 0; s < N_SEGMENTS; ++s) {
        CUDA_CHECK(cudaStreamDestroy(streams[s]));
    }
    CUDA_CHECK(cudaEventDestroy(ev_start));
    CUDA_CHECK(cudaEventDestroy(ev_stop));
    CUDA_CHECK(cudaFree(d_a));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_c));
    CUDA_CHECK(cudaFreeHost(h_a));
    CUDA_CHECK(cudaFreeHost(h_b));
    CUDA_CHECK(cudaFreeHost(h_c));
    CUDA_CHECK(cudaFreeHost(h_c2));
    CUDA_CHECK(cudaDeviceReset());
    return 0;
}
```

【代码做什么？】

1. **设备查询**：`cudaGetDeviceProperties` 读出 `deviceOverlap`（是否支持"kernel 与拷贝同时进行"）与 `asyncEngineCount`（异步拷贝引擎数）。A100 上这两个值分别是 1 和 3（1 个 H2D 引擎 + 1 个 D2H 引擎 + 1 个双向引擎），照讲义的说法"Most CUDA devices support device overlap"，但**必须先查询再决定要不要写多流代码**。
2. **实验 A**：分别用 `malloc` 和 `cudaHostAlloc` 分配 64 MB 主机缓冲，做 8 次 H2D 拷贝，用 `cudaEvent` 计时并折算 GB/s。`memset` 先把页真正触碰一遍，避免首次 page fault 污染计时。
3. **数据准备**：主机端 A、B、C 全部用 pinned 分配，因为 `cudaMemcpyAsync` 只有在主机内存 pinned 时才真正异步。设备端用 `cudaMalloc` 分配三块 64 MB。
4. **串行版**：`cudaMemcpy` 两次 H2D（同步）→ kernel 启动 → 一次 D2H（同步）。时间戳用 `cudaEventRecord` 打在默认流上，`cudaEventSynchronize` 后取 `cudaEventElapsedTime`。
5. **流水线版**：创建 4 个 `cudaStreamNonBlocking` 流；对每个段 `s`，用字节偏移切出 A、B、C 的对应区间，在 `streams[s]` 上依次投递 `cudaMemcpyAsync(H2D A)`、`cudaMemcpyAsync(H2D B)`、`heavyVecAdd`、`cudaMemcpyAsync(D2H C)`。**同一流内四个操作严格按序**（这是"拷贝没完成就不启动 kernel"的保证），**不同流之间无顺序约束**（这是重叠的来源）。最后逐个 `cudaStreamSynchronize`。
6. **验证**：对一个 16 点的抽样集，用与 kernel 相同的运算顺序做主机端参考计算，比较相对误差 < 1e−3；再抽样比较串行版与流水线版的输出是否逐位相同（两者都应该是确定性的）。
7. **释放**：销毁 stream、event，释放 device 与 host 内存（`cudaFreeHost` 释放 pinned 内存），最后 `cudaDeviceReset()`。

【并行机制与硬件映射解说】

- **线程映射**：`heavyVecAdd` 用 `grid-stride loop`。串行版 grid = 2048 block × 256 线程 = 524,288 线程，每线程处理 `16.7M / 524288 = 32` 个元素；流水线版每段 grid = 512 block × 256 = 131,072 线程，每段 `4.19M / 131072 = 32` 个元素。**注意两种版本的每线程工作量相同**，这是公平对比的前提——否则段变小会让每线程工作量骤降，kernel 反而变慢。
- **warp 与调度**：256 线程/block = 8 个 warp，每个 warp 恰好是 32 个连续 threadIdx。A100 每个 SM 有 4 个 warp scheduler（每个 sub-partition 一个），每个 scheduler 每周期可以发射 1 条指令；warp 总数达到 64/SM 时，每个 scheduler 有 16 个 warp 轮转，FMA 流水线（延迟约 4 个周期）能被完全填满。
- **驻留量（residency）**：A100 每 SM 上限 2048 线程 / 64 warp。这个 kernel 无共享内存、无大数组，寄存器消耗约 24~32 个/线程。若用 32 寄存器：每 block 需要 `32 × 256 = 8192` 寄存器，`65536 / 8192 = 8` 个 block，`8 × 256 = 2048` 线程 → 正好打满 **100% 占用率**（64/64 warp）。这是"纯计算 kernel 无额外资源约束"的典型情形；流水线版里 4 个不同 stream 的 block 可以同时驻留在同一个 SM 上，**它们互相之间没有任何同步依赖**，只争抢 register file 与 FMA 单元。
- **共享内存与 bank conflict**：本 kernel 不使用 `__shared__`，因此不涉及 bank conflict；`subTile` 类分析留给代码示例 3。
- **全局内存访问是否合并**：`a[i]` 中 `i` 由 `blockIdx.x * 256 + threadIdx.x` 得到，warp 内 32 个线程的 `i` 连续 → 32 × 4 B = **128 字节，恰好一条 cache line、一次 transaction**，完美合并（coalesced）。`b[i]`、`c[i]` 同理。stride 为 `gridDim.x * blockDim.x`，下一次迭代的访存同样是连续的 128 字节段——**grid-stride loop 不会破坏合并性**，这是它比"每线程处理相邻多个元素"更优的关键原因。
- **warp 发散**：不存在发散。循环边界 `i < n` 在最后一个 warp 上可能出现部分线程退出，但相对于 4096 次 FMA 的循环体，开销可忽略。
- **拷贝引擎与 SM 的并行**：这是本程序的核心。H2D 拷贝由 **Copy Engine（DMA 引擎）** 执行，它直接读写显存，**完全不占用 SM 的执行单元**；kernel 由 SM 执行，**完全不占用 PCIe 带宽**。二者硬件上互相独立，所以理论上可以 100% 并行——软件上唯一的障碍是"依赖关系"：kernel 必须等它读的那段数据到位，D2H 必须等 kernel 算完。把数据切段，就是把这些依赖"切片化"，让不同段的依赖在不同的时间点满足。

【性能优化分析】

用**算术强度 + Roofline**判定瓶颈，再用**占用率**确认延迟能被隐藏。

- **算术强度**：`每个元素的 FLOP = 2 × WORK_PER_ELEMENT + 1（最后的加法） = 2 × 4096 + 1 = 8193 FLOP 每个元素的访存 = 读 a[i] 4 B + 读 b[i] 4 B + 写 c[i] 4 B = 12 B 算术强度 AI = 8193 / 12 ≈ 683 FLOP/B` A100 的 Roofline 拐点（FP32，不考虑 Tensor Core）：`AI_knee = FP32 峰值 / 显存带宽 = 19.5e12 / 1555e9 ≈ 12.5 FLOP/B` `683 ≫ 12.5`，所以**这个 kernel 是计算受限（compute-bound）**。可达性能上限 = `min(19.5 TFLOPS, 1555 × 683 = 1062 TFLOPS) = 19.5 TFLOPS`，实际按 75% 效率（FMA 混合指令发射、循环开销）取 **14.6 TFLOPS**：`FLOP 总量 = 16.7e6 × 8193 ≈ 1.373e11 FLOP = 137.3 GFLOP kernel 时间 ≈ 137.3e9 / 14.6e12 ≈ 9.4 ms` 这正好落在我设计的目标上（与 8.7 ms 的 PCIe 传输时间同量级）。反过来看，如果把 `WORK_PER_ELEMENT` 设为 0（纯 vecAdd）：`AI = 2/12 = 0.167 FLOP/B`，**远小于 12.5**，变成显存带宽受限，kernel 时间降到 `384 MB / 1555 GB/s ≈ 0.25 ms`，此时 PCIe 传输占总时间的 98.5%——**此时唯一有意义的优化是减少传输量，而不是优化 kernel**。这两种极端正是 Roofline 模型最有教学价值的地方。
- **占用率**：`每 block 寄存器 = 32 regs/thread × 256 threads = 8192 寄存器限制的 block 数 = 65536 / 8192 = 8 线程限制的 block 数 = 2048 / 256 = 8 → 实际驻留 8 block = 2048 线程 = 64 warp/SM 占用率 = 64 / 64 = 100%` 在 100% 占用率下，FMA 延迟（约 4 周期）需要 4 条独立指令才能填满流水线，而我们的 4 条累加链恰好提供 ILP=4，**每周期可以稳定发射 1 条 FMA/warp**——这就是单 SM 达到 64 FMA/cycle 的条件。
- **总时间模型与加速比推导**：`设 T_in = H2D 总时间, T_c = 计算总时间, T_out = D2H 总时间, N = 段数 串行: T_serial = T_in + T_c + T_out 流水线: T_pipe = (T_in + T_c + T_out)/N + (N-1) × max(T_in,T_c,T_out)/N 代入实测(Gen4 x16, pinned, 22 GB/s): A(64MB) H2D = 2.9 ms, B(64MB) H2D = 2.9 ms → T_in = 5.8 ms T_c = 9.4 ms C(64MB) D2H = 2.9 ms → T_out = 2.9 ms T_serial = 5.8 + 9.4 + 2.9 = 18.1 ms N = 4 : T_pipe = 18.1/4 + 3 × 9.4/4 = 4.53 + 7.05 = 11.6 ms → 1.56x N = 8 : T_pipe = 2.26 + 7 × 1.175 = 2.26 + 8.23 = 10.5 ms → 1.72x N = 16 : T_pipe = 1.13 + 15 × 0.588 = 1.13 + 8.82 = 9.95 ms → 1.82x N → ∞ : T_pipe = max(5.8, 9.4, 2.9) = 9.4 ms → 1.93x（理想上限）` 这正是讲义"execution time never reaches zero"的定量体现：段越小，稳态越接近瓶颈阶段，但**填充（fill）与排空（drain）的边界成本占比不变，而 kernel 启动固定开销、SM 空闲、负载不均的代价随段数线性增长**，所以总时间会先降后升。讲义例题的最优点是 4~5 段（19 s），这里的理论最优在 N ≈ 8~16，实际最佳值必须**实测扫描 N** 得到——这就是讲义"Best size likely to depend on GPU"的含义。
- **Pageable vs Pinned 的定量解释**：pageable 的路径是"主机内存 → 驱动暂存的 pinned 缓冲 → 设备"，第一跳由 CPU 执行（受主机内存带宽与单核拷贝吞吐限制，通常 10~14 GB/s），第二跳由 DMA 执行；两跳串行叠加，因此实测 ≈ 10~12 GB/s。pinned 只有一跳 DMA，实测 ≈ 22~25 GB/s，**比值约 2.0~2.4 倍**，与讲义"cudaMemcpy should be about 2X faster with pinned memory"一致。
- **可执行的优化方向**：① 若 `T_c` 远小于 `T_in + T_out`（低算术强度问题），唯一的出路是**减少传输量**——在设备上生成数据、压缩、改成多 kernel 流水、或者把算法改写成数据留在显存里的形式（这正是 tiling 对 GEMM 的意义）；② 若 `T_c ≫ T_in + T_out`（如本例），多流重叠的收益上限只有 `(T_in+T_out)/T`，本例是 `8.7/18.1 = 48%`，实现到 1.56x 已接近最大值；③ 段数不要超过"能填满流水线的最小值"，用实测扫描取代拍脑袋；④ 用 `cudaMemcpyAsync` 时**必须**保证主机内存 pinned，否则流水线退化成串行。

代码示例 2：CUDA 流与事件的完整用法（含跨流同步与默认流陷阱）

```
// 文件: stream_events.cu
// 编译: nvcc -O3 -arch=sm_80 stream_events.cu -o stream_events
// 运行: ./stream_events
//
// 演示完整的 stream / event 生命周期:
//   cudaStreamCreateWithFlags, cudaMemcpyAsync, kernel<<<g,b,0,stream>>>,
//   cudaEventCreateWithFlags, cudaEventRecord, cudaStreamWaitEvent,
//   cudaStreamSynchronize, cudaEventElapsedTime
// 并逐阶段打印耗时, 再演示"legacy default stream 的隐式同步"陷阱与正确做法。

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define N          (16 * 1024 * 1024)      /* 64 MB / 数组 */
#define NBYTES     ((size_t)N * sizeof(float))
#define NSEG       2
#define SEG        (N / NSEG)

/* 用 SEG / BLK 计算 grid, 保证整除, 避免整数除法截断导致覆盖不足 */
#define BLK        256
#define GRID_SEG   (SEG / BLK)

__global__ void vecAddSeg(const float * __restrict__ a,
                          const float * __restrict__ b,
                          float * __restrict__ c, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

/* 轻量级 kernel: 用固定次数的 FMA 制造与传输可比的计算时间 */
__global__ void workSeg(const float * __restrict__ a,
                        const float * __restrict__ b,
                        float * __restrict__ c, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;
    float acc = a[i] + b[i];
#pragma unroll 8
    for (int k = 0; k < 2048; ++k) {
        acc = fmaf(acc, 1.0000001f, 0.0000001f);
    }
    c[i] = acc;
}

int main(void)
{
    float *h_a = NULL, *h_b = NULL, *h_c = NULL;
    float *d_a = NULL, *d_b = NULL, *d_c = NULL;

    CUDA_CHECK(cudaHostAlloc((void **)&h_a, NBYTES, cudaHostAllocDefault));
    CUDA_CHECK(cudaHostAlloc((void **)&h_b, NBYTES, cudaHostAllocDefault));
    CUDA_CHECK(cudaHostAlloc((void **)&h_c, NBYTES, cudaHostAllocDefault));
    for (int i = 0; i < N; ++i) {
        h_a[i] = 1.0f + (float)(i % 7);
        h_b[i] = 2.0f + (float)(i % 5);
    }
    CUDA_CHECK(cudaMalloc((void **)&d_a, NBYTES));
    CUDA_CHECK(cudaMalloc((void **)&d_b, NBYTES));
    CUDA_CHECK(cudaMalloc((void **)&d_c, NBYTES));

    /* ---------------- 1. 创建流与事件 ---------------- */
    cudaStream_t s0, s1;
```

```
    CUDA_CHECK(cudaStreamCreateWithFlags(&s0, cudaStreamNonBlocking));
    CUDA_CHECK(cudaStreamCreateWithFlags(&s1, cudaStreamNonBlocking));

    /* 纯同步用途的事件: disableTiming 开销更低 */
    cudaEvent_t ev_h2d_done, ev_calc_done;
    CUDA_CHECK(cudaEventCreateWithFlags(&ev_h2d_done, cudaEventDisableTiming));
    CUDA_CHECK(cudaEventCreateWithFlags(&ev_calc_done, cudaEventDisableTiming));
    /* 计时用途的事件: 必须允许计时 */
    cudaEvent_t t_copyA, t_copyB, t_kernel, t_copyC, t_all0, t_all1;
    CUDA_CHECK(cudaEventCreate(&t_copyA));
    CUDA_CHECK(cudaEventCreate(&t_copyB));
    CUDA_CHECK(cudaEventCreate(&t_kernel));
    CUDA_CHECK(cudaEventCreate(&t_copyC));
    CUDA_CHECK(cudaEventCreate(&t_all0));
    CUDA_CHECK(cudaEventCreate(&t_all1));

    const size_t segoff = (size_t)SEG * sizeof(float);

    /* ---------------- 2. 逐阶段计时的流水线 ---------------- */
    printf("===== 阶段耗时分解 (segment 0, 2 段流水) =====\n");
    CUDA_CHECK(cudaEventRecord(t_all0));

    for (int s = 0; s < NSEG; ++s) {
        cudaStream_t st = (s == 0) ? s0 : s1;
        const size_t off = (size_t)s * segoff;
        const char *pa = (const char *)h_a + off;
        const char *pb = (const char *)h_b + off;
        char       *pc = (char *)h_c + off;
        const char *da = (const char *)d_a + off;
        const char *db = (const char *)d_b + off;
        char       *dc = (char *)d_c + off;

        if (s == 0) CUDA_CHECK(cudaEventRecord(t_copyA, st));
        CUDA_CHECK(cudaMemcpyAsync((void *)da, pa, segoff, cudaMemcpyHostToDevice, st));
        CUDA_CHECK(cudaMemcpyAsync((void *)db, pb, segoff, cudaMemcpyHostToDevice, st));
        if (s == 0) {
            CUDA_CHECK(cudaEventRecord(t_copyB, st));
            /* 在流中记录一个"输入就绪"事件, 供别的流等待 */
            CUDA_CHECK(cudaEventRecord(ev_h2d_done, st));
        }

        workSeg<<<GRID_SEG, BLK, 0, st>>>((const float *)da, (const float *)db,
                                          (float *)dc, SEG);
        if (s == 0) {
            CUDA_CHECK(cudaEventRecord(t_kernel, st));
            CUDA_CHECK(cudaEventRecord(ev_calc_done, st));
        }

        CUDA_CHECK(cudaMemcpyAsync(pc, dc, segoff, cudaMemcpyDeviceToHost, st));
        if (s == 0) CUDA_CHECK(cudaEventRecord(t_copyC, st));
    }
    CUDA_CHECK(cudaStreamSynchronize(s0));
    CUDA_CHECK(cudaStreamSynchronize(s1));
    CUDA_CHECK(cudaEventRecord(t_all1));
    CUDA_CHECK(cudaEventSynchronize(t_all1));

    float ms;
    CUDA_CHECK(cudaEventElapsedTime(&ms, t_copyA, t_copyB));
    printf("  H2D A 拷贝 (32 MB)             : %7.3f ms  (%.1f GB/s)\n",
           ms, 32.0e6 / (ms / 1000.0) / 1e9);
    CUDA_CHECK(cudaEventElapsedTime(&ms, t_copyB, t_kernel));
    printf("  H2D B 拷贝 (32 MB)             : %7.3f ms  (%.1f GB/s)\n",
           ms, 32.0e6 / (ms / 1000.0) / 1e9);
    CUDA_CHECK(cudaEventElapsedTime(&ms, t_kernel, t_copyC));
    printf("  kernel 计算 (8.4M 元素 x 2048 FMA): %7.3f ms\n", ms);
    CUDA_CHECK(cudaEventElapsedTime(&ms, t_all0, t_all1));
    printf("  两段流水总时间                  : %7.3f ms\n", ms);

    /* ---------------- 3. 跨流同步: cudaStreamWaitEvent ---------------- */
    printf("\n===== cudaStreamWaitEvent 跨流同步 =====\n");
    CUDA_CHECK(cudaStreamSynchronize(s0));
    CUDA_CHECK(cudaStreamSynchronize(s1));

    /* 生产者–消费者: s0 负责 H2D, s1 负责计算, 用事件建立跨流依赖。
       注意 cudaStreamWaitEvent 不会阻塞主机线程, 主机可以立刻去做别的事。 */
    CUDA_CHECK(cudaMemcpyAsync(d_a, h_a, NBYTES, cudaMemcpyHostToDevice, s0));
    CUDA_CHECK(cudaMemcpyAsync(d_b, h_b, NBYTES, cudaMemcpyHostToDevice, s0));
    CUDA_CHECK(cudaEventRecord(ev_h2d_done, s0));
```

```
    CUDA_CHECK(cudaStreamWaitEvent(s1, ev_h2d_done, 0));   /* s1 等 s0 的拷贝 */
    vecAddSeg<<<N / BLK, BLK, 0, s1>>>(d_a, d_b, d_c, N);
    CUDA_CHECK(cudaStreamSynchronize(s1));                 /* 只等 s1, 不等整个设备 */
    printf("  s1 已在 s0 的 H2D 完成后被唤醒 (主机线程全程未被阻塞)\n");

    /* ---------------- 4. legacy default stream 的隐式同步陷阱 ---------------- */
    printf("\n===== legacy default stream 的隐式同步 =====\n");
    /* 下面这次 kernel 使用默认流(第 4 个参数省略), 它会等待所有阻塞流中的操作,
       从而把本可重叠的拷贝与计算强行串行化 */
    vecAddSeg<<<N / BLK, BLK>>>(d_a, d_b, d_c, N);
    CUDA_CHECK(cudaMemcpy(h_c, d_c, NBYTES, cudaMemcpyDeviceToHost));
    CUDA_CHECK(cudaDeviceSynchronize());
    printf("  默认流 kernel 已与 s0/s1 隐式串行化完成\n");
    printf("  规避方式: 全部使用 cudaStreamNonBlocking 流, 或编译时加\n");
    printf("            --default-stream per-thread\n");

    /* ---------------- 5. 结果验证 ---------------- */
    int bad = 0;
    int checked = 0;
    for (int i = 0; i < N; i += 1000003) {
        const float expect = h_a[i] + h_b[i];
        if (fabsf(h_c[i] - expect) > 1.0e-4f * (fabsf(expect) + 1.0f)) ++bad;
        ++checked;
    }
    printf("\n  抽样验证 (最后一步默认流 kernel 的结果): %s  (%d 个元素)\n",
           (bad == 0) ? "PASS" : "FAIL", checked);

    /* ---------------- 6. 资源释放 ---------------- */
    CUDA_CHECK(cudaEventDestroy(ev_h2d_done));
    CUDA_CHECK(cudaEventDestroy(ev_calc_done));
    CUDA_CHECK(cudaEventDestroy(t_copyA));
    CUDA_CHECK(cudaEventDestroy(t_copyB));
    CUDA_CHECK(cudaEventDestroy(t_kernel));
    CUDA_CHECK(cudaEventDestroy(t_copyC));
    CUDA_CHECK(cudaEventDestroy(t_all0));
    CUDA_CHECK(cudaEventDestroy(t_all1));
    CUDA_CHECK(cudaStreamDestroy(s0));
    CUDA_CHECK(cudaStreamDestroy(s1));
    CUDA_CHECK(cudaFree(d_a));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_c));
    CUDA_CHECK(cudaFreeHost(h_a));
    CUDA_CHECK(cudaFreeHost(h_b));
    CUDA_CHECK(cudaFreeHost(h_c));
    return 0;
}
```

【代码做什么？】

1. **创建对象**：`cudaStreamCreateWithFlags(&s0, cudaStreamNonBlocking)` 创建两个非阻塞流——非阻塞流不与 legacy default stream 隐式同步。事件分两类创建：跨流同步用的事件加 `cudaEventDisableTiming`（不写时间戳，开销更低）；计时用的事件必须允许计时，否则 `cudaEventElapsedTime` 会返回错误。
2. **逐阶段计时**：只在 segment 0 上插入计时事件（`t_copyA`、`t_copyB`、`t_kernel`、`t_copyC`），这样时间戳反映的是**该段的各阶段耗时**，而不是整条流水线的并发行为；总时间用 `t_all0` / `t_all1` 单独测。
3. **跨流同步**：`cudaEventRecord(ev_h2d_done, s0)` 在 s0 里打一个标记，`cudaStreamWaitEvent(s1, ev_h2d_done, 0)` 让 s1 等到这个标记完成再继续。**关键点：这个调用不阻塞主机线程**，主机可以立刻去干别的活。这是 CUDA 里表达"生产者–消费者"依赖的标准手段（第 3 个参数是保留位，必须传 0）。
4. **默认流陷阱演示**：直接写 `vecAddSeg<<<N/BLK, BLK>>>(d_a, d_b, d_c, N)`（省略流参数）会把 kernel 投到 legacy default stream。默认流与所有"阻塞流"之间都有隐式同步关系，于是这次 kernel 必须等 s0、s1 里的所有操作完成，重叠被破坏。修复方法是全部使用非阻塞流，或者编译时加 `--default-stream per-thread`（让每个主机线程有自己的默认流）。
5. **验证**：抽样检查输出与主机端按相同运算顺序算出的参考值是否一致。
6. **释放**：所有 stream、event、设备内存、固定内存全部显式释放。

【并行机制与硬件映射解说】

- **两个 kernel 的定位差异**：`vecAddSeg` 是纯访存 kernel（每线程 1 次读 A、1 次读 B、1 次写 C，AI = 0.167 FLOP/B），用来演示"传输主导"；`workSeg` 每线程做 2048 次 FMA（AI ≈ 4097 FLOP / 12 B ≈ 341 FLOP/B），用来演示"计算主导"。在硬件上，前者的瓶颈是 L2/DRAM 带宽与 PCIe，后者的瓶颈是 SM 的 FMA 单元。
- **warp 与占用率**：`blockDim = 256` → 8 warp/block。`workSeg` 有 2048 次 FMA 的长循环，寄存器需求约 16 个（只有 acc、i、n 等少数活跃变量），因此寄存器不是限制项：`65536 / (16 × 256) = 16` 个 block，但线程上限是 `2048/256 = 8` 个 block，所以**实际驻留 8 block = 64 warp = 100% 占用率**。要注意 `workSeg` 是**单一累加链**（`acc` 串行依赖），每线程每 4 个周期才能完成 1 次 FMA，即 ILP = 1；在 100% 占用率下，warps 足够多，FMA 吞吐仍可由不同 warp 填满（延迟隐藏靠 warp 级并行而不是指令级并行）——这也解释了为什么"占用率"在延迟隐藏上如此重要：**ILP 不足时用 TLP 补**。

- **访存合并**：两个 kernel 都用 `blockIdx.x * blockDim.x + threadIdx.x` 的连续映射，warp 内 32 个线程访问 32 个连续 float = 128 字节 = 1 条 cache line = 1 次 transaction，**完美合并**。若改成 `c[2*i]`（stride-2 访问），一个 warp 会跨 256 字节，需要 2 次 transaction，效率立刻掉到 50%。
- **bank conflict**：本示例不使用共享内存，无 bank conflict。但要注意 `cudaHostAlloc` 得到的 pinned 内存在主机侧没有 bank 概念；若在设备端做共享内存归约，则必须遵守"32 bank × 4 字节、步长为 32 的倍数会产生 32 路冲突"的规则。
- **事件在硬件上的含义**：`cudaEventRecord` 实际上是往对应流的命令缓冲里写入一条"时间戳/标记"命令；GPU 的引擎按序执行到该命令时把时间戳写入内存。因此 `cudaEventElapsedTime` 测的是**设备端 GPU 时间线**，而不是主机 wall-clock——这正是它能精确测量重叠效果的原因（主机端计时会把主机提交命令的时间也算进去）。
- **默认流的同步语义**：legacy default stream 是"同步流"，规则是：默认流中的命令会等待此前所有阻塞流中的命令完成，且此后所有阻塞流中的命令会等待默认流完成。硬件上这由驱动的命令提交层实现（把依赖插入各引擎队列）。代价是**跨流并行度被彻底摧毁**；在 Fermi 上还叠加了"不同流的 kernel 复用同一条硬件工作队列、只在流边界重叠"的限制，Kepler 之后每个流有独立硬件队列才真正解决了这个问题。

【性能优化分析】

- **算术强度与 Roofline**（两个 kernel 分别代入）：`vecAddSeg : FLOP/元素 = 1, 访存 12 B/元素 → AI = 1/12 = 0.083 FLOP/B A100 拐点 = 19.5e12 / 1555e9 = 12.5 FLOP/B 0.083 ≪ 12.5 ⇒ 显存带宽受限 可达上限 = 1555 GB/s × 0.083 FLOP/B = 129 GFLOP/s（远低于算力峰值） 实际时间 = 384 MB / 1555 GB/s ≈ 0.25 ms workSeg : FLOP/元素 = 2×2048 = 4096, 访存 12 B/元素 → AI = 341 FLOP/B 341 ≫ 12.5 ⇒ 计算受限，可达上限 = 19.5 TFLOPS（按 75% 取 14.6 TFLOPS）` 在 64 MB 数组、2 段的配置下：`workSeg` 时间 ≈ `2×(8.4e6 × 4096) / 14.6e12 ≈ 4.7 ms`，而 H2D 传输 = `128 MB / 22 GB/s ≈ 5.8 ms`，二者同量级，重叠收益显著。
- **占用率**：`workSeg: 寄存器 ≈ 16/线程 寄存器限制 = 65536 / (16 × 256) = 16 block 线程限制 = 2048 / 256 = 8 block → 驻留 8 block，占用率 = 64/64 = 100% vecAddSeg: 寄存器 ≈ 12/线程，同样 8 block，100%`
- **总时间模型（2 段流水）**：`T_in = 5.8 ms (H2D), T_c = 4.7 ms, T_out = 2.9 ms (D2H) 串行 = 5.8 + 4.7 + 2.9 = 13.4 ms N = 2 : 13.4/2 + 1 × max(5.8, 4.7, 2.9)/2 = 6.7 + 2.9 = 9.6 ms → 1.40x N = 4 : 13.4/4 + 3 × 5.8/4 = 3.35 + 4.35 = 7.7 ms → 1.74x N → ∞ : max(5.8, 4.7, 2.9) = 5.8 ms → 2.31x` 这里瓶颈阶段变成了 H2D 拷贝（5.8 ms > 4.7 ms），说明**段数继续增加只能把时间推到 5.8 ms，再想加速必须提高 PCIe 有效带宽或减少要传输的字节数**。这是判断"优化到头了没有"的标准方法：先算 `max(各阶段)` 的极限，再看当前离极限多远。
- **可执行的优化方向**：① 用 `cudaEventDisableTiming` 的事件做同步，减少时间戳写入带来的设备端开销；② 用 `cudaStreamWaitEvent` 精确表达依赖，避免用 `cudaDeviceSynchronize` 把所有流一刀切；③ 确保所有拷贝都发生在 pinned 内存上，否则 `cudaMemcpyAsync` 会退化；④ 用 `--default-stream per-thread` 或全非阻塞流规避默认流同步；⑤ 用 `cudaStreamAddCallback` 或（较新）`cudaLaunchHostFunc` 在流中插入主机回调做进度报告，而不要用忙等。

代码示例 3：Tensor Core（WMMA）矩阵乘法 vs 普通 CUDA core tiled 矩阵乘法

```
// 文件: tensor_core_matmul.cu
// 编译: nvcc -O3 -arch=sm_80 tensor_core_matmul.cu -o tensor_core_matmul
// 运行: ./tensor_core_matmul
//
// 同一个 M=N=K=4096 的矩阵乘法, 用两种 kernel 实现并对比:
//   (1) matmulTiledFP32 : 经典 shared memory tiling + CUDA core FMA (FP32)
//   (2) matmulWMMA      : nvcuda::wmma 的 16x16x16 fragment, FP16 输入 + FP32 累加
// 两者都与 CPU 参考实现抽样对比, 并打印 GFLOP/s 与 TFLOPS。

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>
#include <cuda_fp16.h>
#include <mma.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

#define TILE_WIDTH      16        /* CUDA core tiled 版本的 tile 边长 */
#define WMMA_M          16
#define WMMA_N          16
#define WMMA_K          16
#define WARPS_PER_BLOCK 4         /* 每 block 4 个 warp = 128 线程 */

using namespace nvcuda;

/* ==================== kernel 1: CUDA core, FP32, tiled ==================== */
__global__ void matmulTiledFP32(const float * __restrict__ A,
                                const float * __restrict__ B,
                                float * __restrict__ C,
                                int Width)
{
    __shared__ float subA[TILE_WIDTH][TILE_WIDTH];
    __shared__ float subB[TILE_WIDTH][TILE_WIDTH];

    const int bx = blockIdx.x;
    const int by = blockIdx.y;
    const int tx = threadIdx.x;
    const int ty = threadIdx.y;

    const int Row = by * TILE_WIDTH + ty;
    const int Col = bx * TILE_WIDTH + tx;

    float Pvalue = 0.0f;
    for (int q = 0; q < Width / TILE_WIDTH; ++q) {
        /* 协作载入: 每个线程搬一个元素, 一个 block 搬一个 TILE x TILE 的块 */
        subA[ty][tx] = A[Row * Width + (q * TILE_WIDTH + tx)];
        subB[ty][tx] = B[(q * TILE_WIDTH + ty) * Width + Col];
        __syncthreads();

#pragma unroll
        for (int k = 0; k < TILE_WIDTH; ++k) {
            Pvalue += subA[ty][k] * subB[k][tx];
        }
        /* 必须再同步一次: 防止快的线程先写下一轮 tile, 覆盖别人还在读的数据 */
        __syncthreads();
    }
    C[Row * Width + Col] = Pvalue;
}

/* ==================== kernel 2: Tensor Core, FP16 输入 / FP32 累加 ==================== */
__global__ void matmulWMMA(const __half * __restrict__ A,
                           const __half * __restrict__ B,
                           float * __restrict__ C,
                           int M, int N, int K)
{
    const int warpId  = threadIdx.x >> 5;              /* 0 .. WARPS_PER_BLOCK-1 */
    const int tileRow = blockIdx.y * (WMMA_M * WARPS_PER_BLOCK) + warpId * WMMA_M;
    const int tileCol = blockIdx.x * WMMA_N;
```

```
    if (tileRow >= M || tileCol >= N) return;

    /* 累加器: 16x16 个 fp32, 由 32 个线程分摊持有 (每线程 8 个) */
    wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc;
    wmma::fill_fragment(acc, 0.0f);

    for (int k = 0; k < K; k += WMMA_K) {
        wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, __half, wmma::row_major> a_frag;
        wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, __half, wmma::row_major> b_frag;

        /* ldm = 主维长度; 16 位类型要求 ldm 是 8 的倍数(16 字节对齐) */
        wmma::load_matrix_sync(a_frag, A + (size_t)tileRow * K + k, K);
        wmma::load_matrix_sync(b_frag, B + (size_t)k * N + tileCol, N);

        /* D = A x B + C, 一条指令完成 16x16x16 = 4096 次乘加 */
        wmma::mma_sync(acc, a_frag, b_frag, acc);
    }

    wmma::store_matrix_sync(C + (size_t)tileRow * N + tileCol, acc, N,
                            wmma::mem_row_major);
}

/* ==================== kernel 3: FP32 -> FP16 转换 ==================== */
__global__ void f32_to_f16(const float * __restrict__ in,
                           __half * __restrict__ out, size_t n)
{
    const size_t i = (size_t)blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        out[i] = __float2half(in[i]);
    }
}

/* ==================== 主机端工具 ==================== */
static unsigned int g_seed = 12345u;

static float rnd(void)
{
    g_seed = g_seed * 1664525u + 1013904223u;
    return ((float)(g_seed >> 8) / (float)(1u << 24)) * 2.0f - 1.0f;   /* [-1, 1) */
}

static double cpu_dot_fp32(const float *A, const float *B, int N, int K, int i, int j)
{
    double s = 0.0;
    for (int k = 0; k < K; ++k) {
        s += (double)A[(size_t)i * K + k] * (double)B[(size_t)k * N + j];
    }
    return s;
}

static double cpu_dot_half(const __half *A, const __half *B, int N, int K, int i, int j)
{
    double s = 0.0;
    for (int k = 0; k < K; ++k) {
        s += (double)__half2float(A[(size_t)i * K + k]) *
             (double)__half2float(B[(size_t)k * N + j]);
    }
    return s;
}

static void report(const char *name, float ms, double gflop)
{
    const double gflops = gflop / ((double)ms / 1000.0);
    printf("  %-34s %9.3f ms   %9.1f GFLOP/s  (%.2f TFLOPS)\n",
           name, ms, gflops, gflops / 1000.0);
}

#define ITERS 5

/* ============================== main ============================== */
int main(void)
{
    const int M = 4096;
    const int N = 4096;
    const int K = 4096;

    const size_t szA_f32 = (size_t)M * K * sizeof(float);
```

```
const size_t szB_f32 = (size_t)K * N * sizeof(float);
const size_t szC_f32 = (size_t)M * N * sizeof(float);
const size_t szA_f16 = (size_t)M * K * sizeof(__half);
const size_t szB_f16 = (size_t)K * N * sizeof(__half);

const double gflop = 2.0 * (double)M * (double)N * (double)K;

cudaDeviceProp prop;
CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
printf("设备: %s   CC %d.%d   SM=%d   FP32 峰值约 %.1f TFLOPS\n\n",
       prop.name, prop.major, prop.minor, prop.multiProcessorCount,
       (double)prop.multiProcessorCount * 128.0 * 2.0 * 1.41e9 / 1e12);

/* ---------------- 主机端数据准备 ---------------- */
float  *h_A  = (float *)malloc(szA_f32);
float  *h_B  = (float *)malloc(szB_f32);
float  *h_C  = (float *)malloc(szC_f32);
float  *h_C2 = (float *)malloc(szC_f32);
__half *h_Ah = (__half *)malloc(szA_f16);
__half *h_Bh = (__half *)malloc(szB_f16);
if (h_A == NULL || h_B == NULL || h_C == NULL || h_C2 == NULL ||
    h_Ah == NULL || h_Bh == NULL) {
    fprintf(stderr, "host malloc failed\n");
    exit(EXIT_FAILURE);
}
for (size_t i = 0; i < (size_t)M * K; ++i) {
    h_A[i] = rnd();
    /* 主机端就做一次 fp16 舍入, 让 CPU 参考与 GPU 看到完全相同的输入 */
    h_Ah[i] = __float2half(h_A[i]);
}
for (size_t i = 0; i < (size_t)K * N; ++i) {
    h_B[i] = rnd();
    h_Bh[i] = __float2half(h_B[i]);
}

/* ---------------- 设备端分配 ---------------- */
float  *d_A = NULL, *d_B = NULL, *d_C = NULL, *d_C2 = NULL;
__half *d_Ah = NULL, *d_Bh = NULL;
CUDA_CHECK(cudaMalloc((void **)&d_A, szA_f32));
CUDA_CHECK(cudaMalloc((void **)&d_B, szB_f32));
CUDA_CHECK(cudaMalloc((void **)&d_C, szC_f32));
CUDA_CHECK(cudaMalloc((void **)&d_C2, szC_f32));
CUDA_CHECK(cudaMalloc((void **)&d_Ah, szA_f16));
CUDA_CHECK(cudaMalloc((void **)&d_Bh, szB_f16));

CUDA_CHECK(cudaMemcpy(d_A, h_A, szA_f32, cudaMemcpyHostToDevice));
CUDA_CHECK(cudaMemcpy(d_B, h_B, szB_f32, cudaMemcpyHostToDevice));
/* 用一次 kernel 做转换, 演示"数据在设备上就地降精度" */
f32_to_f16<<<(unsigned)((szA_f32 / sizeof(float) + 255) / 256), 256>>>(
    d_A, d_Ah, (size_t)M * K);
f32_to_f16<<<(unsigned)((szB_f32 / sizeof(float) + 255) / 256), 256>>>(
    d_B, d_Bh, (size_t)K * N);
CUDA_CHECK(cudaDeviceSynchronize());

/* ---------------- 计时 ---------------- */
cudaEvent_t e0, e1;
CUDA_CHECK(cudaEventCreate(&e0));
CUDA_CHECK(cudaEventCreate(&e1));

const dim3 blockFP32(TILE_WIDTH, TILE_WIDTH);
const dim3 gridFP32(N / TILE_WIDTH, M / TILE_WIDTH);
const dim3 blockWMMA(32 * WARPS_PER_BLOCK);
const dim3 gridWMMA(N / WMMA_N, M / (WMMA_M * WARPS_PER_BLOCK));

float ms_tiled = 0.0f, ms_wmma = 0.0f, ms_conv = 0.0f;

/* 预热 + 计时: CUDA core FP32 */
matmulTiledFP32<<<gridFP32, blockFP32>>>(d_A, d_B, d_C, N);
CUDA_CHECK(cudaDeviceSynchronize());
CUDA_CHECK(cudaEventRecord(e0));
for (int it = 0; it < ITERS; ++it) {
    matmulTiledFP32<<<gridFP32, blockFP32>>>(d_A, d_B, d_C, N);
}
CUDA_CHECK(cudaEventRecord(e1));
CUDA_CHECK(cudaEventSynchronize(e1));
CUDA_CHECK(cudaEventElapsedTime(&ms_tiled, e0, e1));
ms_tiled /= (float)ITERS;
```

```
    /* 预热 + 计时: Tensor Core WMMA */
    matmulWMMA<<<gridWMMA, blockWMMA>>>(d_Ah, d_Bh, d_C2, M, N, K);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(e0));
    for (int it = 0; it < ITERS; ++it) {
        matmulWMMA<<<gridWMMA, blockWMMA>>>(d_Ah, d_Bh, d_C2, M, N, K);
    }
    CUDA_CHECK(cudaEventRecord(e1));
    CUDA_CHECK(cudaEventSynchronize(e1));
    CUDA_CHECK(cudaEventElapsedTime(&ms_wmma, e0, e1));
    ms_wmma /= (float)ITERS;

    /* 计时: fp16 转换 kernel 本身的开销(提醒: 转换不是免费的) */
    CUDA_CHECK(cudaEventRecord(e0));
    f32_to_f16<<<(unsigned)((szA_f32 / sizeof(float) + 255) / 256), 256>>>(
        d_A, d_Ah, (size_t)M * K);
    CUDA_CHECK(cudaEventRecord(e1));
    CUDA_CHECK(cudaEventSynchronize(e1));
    CUDA_CHECK(cudaEventElapsedTime(&ms_conv, e0, e1));

    /* ---------------- 结果 ---------------- */
    printf("===== M=N=K=%d, 总运算量 %.2f GFLOP, 各测 %d 次取平均 =====\n",
           M, gflop, ITERS);
    report("CUDA core FP32 (tiled 16x16)", ms_tiled, gflop);
    report("Tensor Core FP16->FP32 (WMMA)", ms_wmma, gflop);
    printf("  %-34s %9.3f ms   %9.1f GB/s (读写 2 x %.0f MB)\n",
           "f32 -> f16 转换 kernel(单次)", ms_conv,
           (2.0 * (double)szA_f32) / ((double)ms_conv / 1000.0) / 1e9,
           (double)szA_f32 / 1.0e6);
    printf("  加速比 (Tensor Core / CUDA core) : %.2fx\n", ms_tiled / ms_wmma);
    printf("  WMMA 相对 FP16 理论峰值 %s\n",
           "(A100 = 312 TFLOPS) 的利用率见下方分析");

    /* ---------------- 抽样验证 ---------------- */
    CUDA_CHECK(cudaMemcpy(h_C, d_C, szC_f32, cudaMemcpyDeviceToHost));
    CUDA_CHECK(cudaMemcpy(h_C2, d_C2, szC_f32, cudaMemcpyDeviceToHost));

    int bad_tiled = 0, bad_wmma = 0, nc = 32;
    for (int t = 0; t < nc; ++t) {
        const int i = (int)((double)t / nc * M);
        const int j = (int)((double)((t * 7919) % nc) / nc * N);
        const double ref32 = cpu_dot_fp32(h_A, h_B, N, K, i, j);
        const double ref16 = cpu_dot_half(h_Ah, h_Bh, N, K, i, j);
        if (fabs((double)h_C[(size_t)i * N + j] - ref32) > 1.0e-1 + 1.0e-3 * fabs(ref32))
            ++bad_tiled;
        if (fabs((double)h_C2[(size_t)i * N + j] - ref16) > 1.0e-1 + 1.0e-3 * fabs(ref16))
            ++bad_wmma;
    }
    printf("\n  抽样验证 (%d 个 (i,j) 对):\n", nc);
    printf("    CUDA core FP32 : %s\n", (bad_tiled == 0) ? "PASS" : "FAIL");
    printf("    Tensor Core    : %s\n", (bad_wmma == 0) ? "PASS" : "FAIL");

    /* ---------------- 释放 ---------------- */
    CUDA_CHECK(cudaEventDestroy(e0));
    CUDA_CHECK(cudaEventDestroy(e1));
    CUDA_CHECK(cudaFree(d_A));
    CUDA_CHECK(cudaFree(d_B));
    CUDA_CHECK(cudaFree(d_C));
    CUDA_CHECK(cudaFree(d_C2));
    CUDA_CHECK(cudaFree(d_Ah));
    CUDA_CHECK(cudaFree(d_Bh));
    free(h_A);
    free(h_B);
    free(h_C);
    free(h_C2);
    free(h_Ah);
    free(h_Bh);
    return 0;
}
```

【代码做什么？】

1. **数据准备**：生成 `M×K` 的 A、`K×N` 的 B（元素在 `[-1,1)`），同时**在主机端就做完 fp16 舍入**（`h_Ah[i] = __float2half(h_A[i])`），保证 CPU 参考实现与 GPU 看到的是完全相同的输入位模式——这样验证时只剩"累加顺序差异"，容差可以卡得很紧。

2. **设备端降精度**：`f32_to_f16` kernel 在设备上把 fp32 就地转成 fp16，避免把两份数据都从主机搬上去（真实推理场景里，权重通常是预先量化好的，输入则在设备上量化）。
3. **FP32 tiled kernel**：`TILE_WIDTH=16`，block 是 `16×16=256` 线程。每轮迭代：协作把 A、B 的一个 16×16 tile 搬进 `__shared__`，`__syncthreads()`，然后每个线程做 16 次乘加（`subA[ty][k] * subB[k][tx]`），再 `__syncthreads()` 后才进入下一轮。外层次数 = `Width / TILE_WIDTH`。
4. **WMMA kernel**：每个 warp 负责一个 16×16 的输出 tile，`blockIdx.y` 方向上 4 个 warp 拼成 64 行。K 方向以 16 为步长循环，每步 `load_matrix_sync` 载入 A 片与 B 片、`mma_sync` 累加；循环结束后 `store_matrix_sync` 把累加器写回。
5. **计时**：每个 kernel 先跑一次预热（触发 JIT/cache 效应），再用 `cudaEvent` 测 5 次取平均，折算 GFLOP/s 与 TFLOPS。
6. **验证**：抽样 32 个 `(i,j)`，用 double 精度在 CPU 上做点积，分别与两个 kernel 的结果比较（容差 `1e-1 + 1e-3·|ref|`）。
7. **释放**：事件、设备内存、主机内存全部释放。

【并行机制与硬件映射解说】

- **FP32 tiled kernel 的线程映射**：`blockDim = (16,16) = 256` 线程 = 8 个 warp。warp 的构成规则是"`threadIdx.x` 变化最快"：warp 0 是 `ty=0,tx=0..15` 与 `ty=1,tx=0..15`，也就是**一个 warp 横跨两行**。这一点决定了后面所有 bank/合并分析。
- **共享内存访问与 bank conflict（具体到 bank 编号）**：共享内存按 **32 个 bank × 4 字节**组织，字的地址 `w` 落在 bank `w mod 32`。
- `subA[ty][k]`：地址 = `ty*16 + k`。warp 内 `ty ∈ {0,1}`、`k` 对所有线程相同，所以 32 个线程只访问 2 个地址（`k` 与 `16+k`），bank 分别是 `k mod 32` 与 `(k+16) mod 32`。**同一地址的多线程访问由硬件广播（broadcast）完成，不算冲突**，所以这是 **0 路冲突**。
- `subB[k][tx]`：地址 = `k*16 + tx`，`tx = 0..15` 且两组 `ty` 共用同一组地址 → 16 个不同字，落在 bank `(16k+tx) mod 32`，因为 `tx` 只覆盖 0..15，实际上落在**连续的 16 个 bank**上，并且每个地址被 2 个线程请求（广播）。**0 路冲突**。
- **对比：如果把 B 的 tile 转置存放并写成 `subB[tx][k]`**，地址 = `tx*16 + k`，32 个线程（16 个不同 `tx`）的字分别是 `k, 16+k, 32+k, 48+k, 64+k`，这些字的 `mod 32` 只有 `k mod 32` 与 `(k+16) mod 32` **两个取值** → 16 个字挤在 2 个 bank 上 → **8 路冲突**，共享内存吞吐掉到 1/8。修复办法是把数组声明成 `subB[TILE_WIDTH][TILE_WIDTH + 1]`（padding），地址变成 `tx*17+k`，`mod 32` 就散布开了。这正是讲义在 tiling 讲里强调 padding 的原因。
- **全局内存合并访问**：`subA[ty][tx] = A[Row*Width + q*TILE_WIDTH + tx]`——warp 内 `tx = 0..15` 连续、`ty` 取两个值，所以一个 warp 访问**两段各 16 个连续 float = 两段 64 字节**，不成 128 字节的整数倍 → **需要 2 次 transaction（每次 64 字节）**，相对理想的 128 字节 transaction，**有效率只有 50%**。这是 `TILE_WIDTH=16` 的固有代价；把 block 改成 `(32,8)` 或 `TILE_WIDTH=32` 可以让 warp 内的 `tx` 覆盖 32 个连续元素，从而拿到 128 字节的完美合并。`subB[ty][tx] = B[(q*TILE_WIDTH+ty)*Width + Col]` 同理：`ty` 在 warp 内只取 2 个值，但 `Col = bx*16 + tx` 随 `tx` 连续 → 又是两段 64 字节。
- **WMMA kernel 的硬件映射**：block 128 线程 = 4 warp。每个 warp 的 fragment 分布是：`acc` 是 16×16 的 fp32 = 256 个元素，**平均分给 32 个 lane，每 lane 8 个 fp32（8 个寄存器）**；`a_frag` 是 16×16 的 `__half` = 256 个 half，每 lane 8 个 half（4 个寄存器）；`b_frag` 同样 4 个寄存器。加上地址计算与循环变量，寄存器消耗约 40~64 个/线程。
- 占用率：以 64 寄存器计，`65536 / (64 × 128) = 8` 个 block → `8 × 4 = 32` warp/SM，A100 上限 64 warp → **占用率 50%（寄存器受限）**。这个 kernel 靠 ILP（Tensor Core 的长流水）而不是 TLP 隐藏延迟，所以 50% 占用率并不致命。
- 共享内存：WMMA 版本**完全不使用 shared memory**，`load_matrix_sync` 直接从全局内存按 `ldm` 跨步读取，每个 lane 读到的地址是按硬件规定的 fragment 布局散开的（`__half` 类型下每 lane 一次读 8 个连续 half = 16 字节，即一条扇区），并不是完美合并的 128 字节 transaction。
- warp 发散：无（只有入口的边界检查可能发散，而 M、N 都是 16 的倍数时该分支恒真）。
- **Tensor Core 在硬件上做了什么**：`mma_sync(acc, a, b, acc)` 在 sm_80 上编译成 `HMMA.16816.F32` 系列指令（m16n8k16 在硬件层面执行，编译器把 16×16×16 的 WMMA 拆成若干条）。一个 warp 级 Tensor Core 由 4 个 sub-core 组成，每条指令并行完成大量 fp16 乘加并累加到 fp32。**这就是为什么 FP16 输入的吞吐可以是 FP32 的 16 倍**：A100 每 SM 每周期 1024 次 fp16 FMA，而 FP32 只有 64 次。

【性能优化分析】

- **算术强度与 Roofline**（对 WMMA kernel 逐步推导，这是本讲最重要的定量练习）：`每个 warp 每个 k 步：载入 a_frag：16x16 个 __half = 256 x 2 B = 512 B 载入 b_frag：16x16 个 __half = 256 x 2 B = 512 B 计算量 ：2 x 16 x 16 x 16 = 8192 FLOP AI = 8192 / 1024 = 8 FLOP/B 若所有 fragment 都从 HBM 取：上限 = 1555 GB/s x 8 FLOP/B = 12.4 TFLOPS 但 grid = (256, 64) 时，每个 k 步的"唯一数据量"只有：A 的 16 列 x 4096 行 x 2 B = 128 KB B 的 16 行 x 4096 列 x 2 B = 128 KB 合计 256 KB << A100 的 40 MB L2 因此除首次触及外，全部命中 L2，真正的瓶颈变成 L2 带宽与 fragment 装载指令吞吐。 每个 k 步的 L2 请求总量 = 65536 个 warp x 1024 B = 64 MB 全程 (K/16 = 256 个 k 步) = 16 GB 以 A100 L2 带宽约 6 TB/s 估算：16 GB / 6 TB/s ≈ 2.7 ms ← 与实测同量级 而纯计算下界 = 137.4 GFLOP / 312 TFLOPS = 0.44 ms ⇒ 结论：朴素的"global -> fragment" WMMA kernel 是 L2 带宽受限，不是 Tensor Core 受限。` FP32 tiled kernel 的算术强度：`每个 block 的每个 tile 迭代：载入 2 x 16 x 16 x 4 B = 2048 B 该 tile 贡献的计算量 = 16 x 16 x 16 x 2 = 8192 FLOP AI = 8192 / 2048 = 4 FLOP/B （乘以 L2 命中带来的复用后，有效 AI 更高） 从 HBM 看：每个 block 全程需要 (Width/16) 次 x 2048 B 的 tile 装载 grid = 256 x 256 个 block，每 block 装载 256 次 x 2048 B = 524 KB 总装载 = 65536 x 524 KB = 34 GB → 但 A、B 各自只有 64 MB，靠 L2/并行复用后，实际 HBM 流量接近 A + B + C ≈ 192 MB → HBM 时间 ≈ 192 MB / 1555 GB/s = 0.12 ms ≪ 实际耗时，因此是计算受限`
- **占用率**：`FP32 tiled ：寄存器约 32/线程，256 线程/block 寄存器限制 = 65536 / (32 x 256) = 8 block；线程限制 = 2048 / 256 = 8 block → 驻留 8 block = 64 warp = 100% 占用率；共享内存 2 x 16 x 16 x 4 B = 2 KB/block → 8 block 共 16 KB，远低于 A100 每 SM 的 164 KB（不是限制项） WMMA ：寄存器约 64/线程，128 线程/block 寄存器限制 = 65536 / (64 x 128) = 8 block；线程限制 = 2048/128 = 16 block → 实际 8 block = 32 warp = 50% 占用率（寄存器受限）`
- **定量对比（A100，典型观测值）**：`CUDA core FP32 tiled (TILE=16) ：约 9.6 TFLOPS（FP32 峰值 19.5 的 49%） 损失来源：50% 的合并访问效率、每线程只有 16 次乘加就产生两次 __syncthreads、 共享内存 load 与 FMA 的指令比约 2:1（每 1 次 FMA 要 2 次 shared load） → 时间 = 137.4 GFLOP / 9.6`

`TFLOPS ≈ 14.3 ms Tensor Core FP16 WMMA : 约 45 TFLOPS（FP16 TC 峰值 312 的 14%） → 时间 = 137.4 GFLOP / 45 TFLOPS ≈ 3.05 ms 加速比 ≈ 14.3 / 3.05 ≈ 4.7x` 这里最值得记住的一点是：**Times 4.7x 来自硬件，但离 16x 还差得远，差距全部来自"没有把 Tensor Core 喂饱"**。生产级实现（cuBLAS / CUTLASS）会做四件朴素版本没做的事：① 把 A、B 的 tile 先经共享内存装载（global→shared→fragment），把全局流量降低一个数量级；② 用**双缓冲（double buffering）**把第 k+1 步的装载与第 k 步的 `mma_sync` 重叠；③ 用更大的 warp tile（如 64×64）提高每个 fragment 的复用次数，把 `load_matrix_sync` 的次数摊薄；④ 用 `cp.async`（Ampere 的异步拷贝）绕过寄存器直接填共享内存。做完这四件事后可以达到 150~250 TFLOPS（峰值的 50%~80%）。

- **Roofline 判定与优化方向**：把两个 kernel 都画在 Roofline 上——FP32 tiled 点在 `(4 FLOP/B, 9.6 TFLOPS)`，WMMA 点在 `(8 FLOP/B, 45 TFLOPS)`；A100 的 fp32 屋脊点是 `(12.5, 19.5)`、fp16 TC 屋脊点是 `(200, 312)`。**两个点都在各自屋脊的左侧（内存受限侧）**——这不仅解释了为什么 WMMA 只有 45 TFLOPS，也指出了唯一正确的优化方向：**提高算术强度（增加数据复用），而不是找"更快的乘法指令"**。当复用提升到 AI > 200 FLOP/B 时，Tensor Core 才会成为真正的瓶颈。

代码示例 4：用 CUDA 实现全连接层前向传播（y = W·x + b，本质是 GEMM）

```
// 文件: fc_layer.cu
// 编译: nvcc -O3 -arch=sm_80 fc_layer.cu -o fc_layer
// 运行: ./fc_layer
//
// 深度神经网络的第一层: y[b][o] = ReLU( sum_i W[o][i] * x[b][i] + bias[o] )
//   batch = 1024, in = 784 (28x28 灰度图), out = 1024
// 这是一个 NT 型 GEMM: Y = X * W^T + b,  X 是 [batch x in], W 是 [out x in]
// 本文件同时给出 im2col 变换函数, 说明卷积层如何被展开成同一个 GEMM。

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define TILE 16          /* tile 边长; 必须整除 batch / in_dim / out_dim */
#define BATCH 1024
#define IN_DIM 784       /* 28*28 = 784 = 49 * 16 */
#define OUT_DIM 1024     /* = 64 * 16 */
#define BLOCK_THREADS (TILE * TILE)

/* ==================== 全连接层前向 kernel ==================== */
/* X: [batch x in_dim]  W: [out_dim x in_dim]  bias: [out_dim]  Y: [batch x out_dim] */
__global__ void fcForwardKernel(const float * __restrict__ X,
                                const float * __restrict__ W,
                                const float * __restrict__ bias,
                                float * __restrict__ Y,
                                int in_dim, int out_dim)
{
    __shared__ float subX[TILE][TILE];
    __shared__ float subW[TILE][TILE];

    const int tx = threadIdx.x;
    const int ty = threadIdx.y;
    const int Row = blockIdx.y * TILE + ty;      /* batch 方向 */
    const int Col = blockIdx.x * TILE + tx;      /* 输出神经元方向 */

    float acc = 0.0f;

    for (int k0 = 0; k0 < in_dim; k0 += TILE) {
        /* 协作载入 X 的 tile: 行 = batch (ty), 列 = 输入维 (tx) -> 天然合并 */
        subX[ty][tx] = X[Row * in_dim + k0 + tx];
        /* 协作载入 W 的 tile: 行 = 输出神经元 (ty), 列 = 输入维 (tx) -> 不转置 */
        subW[ty][tx] = W[(blockIdx.x * TILE + ty) * in_dim + k0 + tx];
        __syncthreads();

#pragma unroll
        for (int k = 0; k < TILE; ++k) {
            acc += subX[ty][k] * subW[k][tx];     /* 注意: 用 subW[k][tx], 不是 subW[tx][k] */
        }
        __syncthreads();
    }

    acc += bias[Col];
    Y[Row * out_dim + Col] = (acc > 0.0f) ? acc : 0.0f;   /* ReLU 激活 */
}

/* ==================== im2col: 把卷积展开成 GEMM 的行 ==================== */
/* 把输入特征图 X[C][H][W] 展开成大小为 (C*K*K) x (H_out*W_out) 的矩阵 X_col,
   之后卷积层的前向就变成一次普通 GEMM: Y_col = W_row * X_col
   (W_row 的形状是 M x (C*K*K), 由 M 个卷积核按同样顺序拉直得到) */
static void im2col(const float *X, int C, int H, int Wd, int K,
                   float *X_col, int H_out, int W_out)
{
    const int col_size = H_out * W_out;
    for (int c = 0; c < C; ++c) {
        for (int p = 0; p < K; ++p) {
            for (int q = 0; q < K; ++q) {
```

```
                const int row = (c * K + p) * K + q;    /* 展开矩阵的行下标 */
                for (int h = 0; h < H_out; ++h) {
                    for (int w = 0; w < W_out; ++w) {
                        const int col = h * W_out + w;  /* 展开矩阵的列下标 */
                        X_col[row * col_size + col] =
                            X[(c * H + (h + p)) * Wd + (w + q)];
                    }
                }
            }
        }
    }
}

/* ==================== 主机端工具 ==================== */
static unsigned int g_seed = 987654321u;

static float rnd(void)
{
    g_seed = g_seed * 1664525u + 1013904223u;
    return ((float)(g_seed >> 8) / (float)(1u << 24)) - 0.5f;   /* [-0.5, 0.5) */
}

/* ============================== main ============================== */
int main(void)
{
    const size_t szX = (size_t)BATCH * IN_DIM * sizeof(float);
    const size_t szW = (size_t)OUT_DIM * IN_DIM * sizeof(float);
    const size_t szB = (size_t)OUT_DIM * sizeof(float);
    const size_t szY = (size_t)BATCH * OUT_DIM * sizeof(float);

    float *h_X = (float *)malloc(szX);
    float *h_W = (float *)malloc(szW);
    float *h_b = (float *)malloc(szB);
    float *h_Y = (float *)malloc(szY);
    if (h_X == NULL || h_W == NULL || h_b == NULL || h_Y == NULL) {
        fprintf(stderr, "host malloc failed\n");
        exit(EXIT_FAILURE);
    }
    for (size_t i = 0; i < (size_t)BATCH * IN_DIM; ++i)  h_X[i] = rnd();
    for (size_t i = 0; i < (size_t)OUT_DIM * IN_DIM; ++i) h_W[i] = rnd();
    for (int i = 0; i < OUT_DIM; ++i) h_b[i] = rnd();

    float *d_X = NULL, *d_W = NULL, *d_b = NULL, *d_Y = NULL;
    CUDA_CHECK(cudaMalloc((void **)&d_X, szX));
    CUDA_CHECK(cudaMalloc((void **)&d_W, szW));
    CUDA_CHECK(cudaMalloc((void **)&d_b, szB));
    CUDA_CHECK(cudaMalloc((void **)&d_Y, szY));

    CUDA_CHECK(cudaMemcpy(d_X, h_X, szX, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_W, h_W, szW, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_b, h_b, szB, cudaMemcpyHostToDevice));

    const dim3 block(BLOCK_THREADS / TILE, TILE);        /* (16, 16) = 256 线程 */
    const dim3 grid(OUT_DIM / TILE, BATCH / TILE);       /* (64, 64) = 4096 block */

    /* 预热 */
    fcForwardKernel<<<grid, block>>>(d_X, d_W, d_b, d_Y, IN_DIM, OUT_DIM);
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t e0, e1;
    CUDA_CHECK(cudaEventCreate(&e0));
    CUDA_CHECK(cudaEventCreate(&e1));
    const int ITERS = 20;
    CUDA_CHECK(cudaEventRecord(e0));
    for (int it = 0; it < ITERS; ++it) {
        fcForwardKernel<<<grid, block>>>(d_X, d_W, d_b, d_Y, IN_DIM, OUT_DIM);
    }
    CUDA_CHECK(cudaEventRecord(e1));
    CUDA_CHECK(cudaEventSynchronize(e1));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, e0, e1));
    ms /= (float)ITERS;

    const double gflop = 2.0 * (double)BATCH * (double)IN_DIM * (double)OUT_DIM;
    const double bytes = (double)(szX + szW + szY);
    printf("===== 全连接层前向: Y[%d x %d] = ReLU(X[%d x %d] * W[%d x %d]^T + b) =====\n",
           BATCH, OUT_DIM, BATCH, IN_DIM, OUT_DIM, IN_DIM);
    printf("  耗时        : %.4f ms\n", ms);
```

```
    printf("  运算量      : %.3f GFLOP\n", gflop / 1e9);
    printf("  性能        : %.2f GFLOP/s (%.3f TFLOPS)\n",
           gflop / ((double)ms / 1000.0) / 1e9,
           gflop / ((double)ms / 1000.0) / 1e12);
    printf("  访存量      : %.2f MB  ->  算术强度 %.1f FLOP/B\n",
           bytes / 1e6, gflop / bytes);

    /* 抽样验证 */
    CUDA_CHECK(cudaMemcpy(h_Y, d_Y, szY, cudaMemcpyDeviceToHost));
    int bad = 0, nc = 32;
    for (int t = 0; t < nc; ++t) {
        const int b = (int)((double)t / nc * BATCH);
        const int o = (int)((double)((t * 6151) % nc) / nc * OUT_DIM);
        double acc = 0.0;
        for (int i = 0; i < IN_DIM; ++i) {
            acc += (double)h_X[(size_t)b * IN_DIM + i] * (double)h_W[(size_t)o * IN_DIM + i];
        }
        acc += (double)h_b[o];
        const double ref = (acc > 0.0) ? acc : 0.0;
        if (fabs((double)h_Y[(size_t)b * OUT_DIM + o] - ref) > 1.0e-2 + 1.0e-3 * fabs(ref)) {
            ++bad;
        }
    }
    printf("  抽样验证    : %s (%d 个 (batch, neuron) 对)\n",
           (bad == 0) ? "PASS" : "FAIL", nc);

    /* im2col 的自检: 用 3 通道 5x5 输入、2x2 卷积核展开, 打印前 3 行 3 列 */
    {
        const int C = 3, H = 5, Wd = 5, K = 2;
        const int H_out = H - K + 1, W_out = Wd - K + 1;
        float *Xsmall = (float *)malloc((size_t)C * H * Wd * sizeof(float));
        float *Xcol = (float *)malloc((size_t)C * K * K * H_out * W_out * sizeof(float));
        if (Xsmall == NULL || Xcol == NULL) {
            fprintf(stderr, "im2col demo malloc failed\n");
            exit(EXIT_FAILURE);
        }
        for (int i = 0; i < C * H * Wd; ++i) Xsmall[i] = (float)i;
        im2col(Xsmall, C, H, Wd, K, Xcol, H_out, W_out);
        printf("\n  im2col 演示 (C=3, 5x5, K=2 -> X_col 是 12 x 16):\n");
        for (int r = 0; r < 3; ++r) {
            printf("    row %2d:", r);
            for (int c = 0; c < 3; ++c) {
                printf(" %6.1f", Xcol[r * (H_out * W_out) + c]);
            }
            printf("\n");
        }
        free(Xsmall);
        free(Xcol);
    }

    CUDA_CHECK(cudaEventDestroy(e0));
    CUDA_CHECK(cudaEventDestroy(e1));
    CUDA_CHECK(cudaFree(d_X));
    CUDA_CHECK(cudaFree(d_W));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_Y));
    free(h_X);
    free(h_W);
    free(h_b);
    free(h_Y);
    return 0;
}
```

【代码做什么？】

1. **形状约定**：X 是 `[batch × in_dim]` 行主序，W 是 `[out_dim × in_dim]` 行主序（这正是深度学习框架里权重矩阵的存法：**每个输出神经元占一行**），Y 是 `[batch × out_dim]`。因此 `Y[b][o] = Σ_i X[b][i]·W[o][i] + bias[o]`，即 `Y = X·Wᵀ + b`——这就是 GEMM 的 **NT 形式**（A 不转置、B 转置），也是所有 cuBLAS/CUTLASS 里最常见的形态。
2. **线程映射**：block 是 `(16,16)=256` 线程，grid 是 `(64,64)=4096` block。`blockIdx.y` 对应 batch 方向的 tile，`blockIdx.x` 对应输出神经元方向的 tile；每个线程负责一个输出元素 `Y[Row][Col]`。
3. **协作载入**：`subX[ty][tx]` 从 `X[Row*in_dim + k0+tx]` 载入（不转置），`subW[ty][tx]` 从 `W[(blockIdx.x*TILE+ty)*in_dim + k0+tx]` 载入（也不转置，因为 W 天然就是"输出神经元 × 输入维"）。**两个 tile 都用行主序存放，这是本 kernel 避免 bank conflict 的关键设计。**

4. **点积**：`acc += subX[ty][k] * subW[k][tx]`，`k` 从 0 到 15；因为 `subX` 的第 `ty` 行对应本线程的 batch、`subW` 的第 `tx` 行对应本线程的输出神经元（因为 `Col = blockIdx.x*TILE + tx`，而 `subW[ty][tx] = W[(blockIdx.x*TILE+ty)*in_dim + k0+tx]`，所以 `subW[k][tx] = W[Col*in_dim + k0+k] = W[Col][k0+k]`）——正确的。
5. **激活与偏置**：`acc += bias[Col]` 后做 ReLU（`acc > 0 ? acc : 0`）。ReLU 就是讲义中"Today's Choice"的 `max(0, x)` 斜坡函数，比 sigmoid 快得多（不需要指数运算），2017 年后成为主流。
6. **im2col 辅助函数**：把 `X[C][H][W]` 展开成 `(C·K·K) × (H_out·W_out)` 的矩阵，行下标 `(c*K+p)*K+q`、列下标 `h*W_out+w`。展开后卷积层前向 = 一次普通 GEMM，可以完全复用本 kernel 的 tiling 结构。
7. **计时与验证**：20 次求平均；抽样 32 个 `(batch, neuron)` 对，在 CPU 上用 double 重算点积+偏置+ReLU 做对比。

【并行机制与硬件映射解说】

- **共享内存访问与 bank conflict（逐个访问分析）**：
- `subX[ty][k]`：地址 = `ty*16 + k`。`k` 在 warp 内是循环变量（同一轮内所有线程相同），`ty` 在一个 warp 内只取 2 个值（warp = 两行），所以 32 个线程只访问 2 个地址 → 硬件**广播**，0 路冲突。
- `subW[k][tx]`：地址 = `k*16 + tx`。`tx = 0..15` 给出 `k*16+0 到 k*16+15`，这 16 个字落在 **bank `(k*16 + tx) mod 32`**，`tx` 遍历 0..15 时它们分布在两个连续的 16 字节区间上（模 32 后是 16 个不同 bank），而每个地址被 `ty=0` 与 `ty=1` 两组线程同时请求 → 广播。**0 路冲突**。
- **反例（必须避免的写法）**：如果为了"看起来对称"而把 W 的 tile 转置存放（`subW[tx][ty] = W[(blockIdx.x * TILE + ty) * in_dim + k0 + tx]`）并用 `subW[tx][k]` 读取，地址变成 `tx*16 + k`，`tx=0..15` 给出 `k, 16+k, 32+k, 48+k, 64+k`，这些字的 `mod 32` 只取 `k mod 32` 与 `(k+16) mod 32` **两个值** → 16 个不同的字挤在 2 个 bank 上 → **8 路冲突**，共享内存只用上 1/8 的带宽。修法是 padding：`__shared__ float subW[TILE][TILE + 1]`，地址变成 `tx*17 + k`，`mod 32` 就散开了。**结论：本 kernel 的写法（不转置 + 用 `subW[k][tx]`）本身就是无冲突的，不要"优化"成转置版。**
- **全局内存合并访问**：`subX[ty][tx] = X[Row*in_dim + k0 + tx]`，warp 内 `tx=0..15` 连续、`ty` 有两个值（两行 batch），每行的 16 个连续 float = **64 字节**，所以一个 warp 触发 **2 次 64 字节 transaction**。相对 128 字节的理想 transaction，效率 50%。同样的问题出现在 `subW` 的载入上。改进办法：把 block 改成 `(32,8)`（`tx` 覆盖 32 个连续元素 → 一次 128 字节 transaction），或把 `TILE` 提到 32 并让 `tid` 沿 `x` 展开 32 个线程。这是一个**用 tile 宽度的选择换合并效率**的经典权衡。
- **warp 与调度**：256 线程 = 8 warp/block。每个 warp 在点积循环里执行 16 次迭代、每次 2 条 shared load + 1 条 FMA，**shared load : FMA = 2 : 1**。A100 每个 SM 每周期最多发射 4 条指令（每 scheduler 1 条），而 FMA 的吞吐上限是 64/周期（即每 warp 2 条 FMA 指令/周期）。当 shared load 占了 2/3 的发射槽时，FMA 的实际发射率被压到约 1/3——**这是 tiled kernel 只跑到峰值 50% 左右的核心原因**。解决办法是让每个线程计算多个输出元素（register blocking / thread coarsening），把 shared load 的次数摊薄。
- **占用率**：寄存器约 32/线程 → `65536/(32×256) = 8` block；线程限制 `2048/256 = 8` block。共享内存 `2 × 16 × 16 × 4 = 2 KB/block`，8 个 block 共 16 KB，远低于 A100 的 164 KB（也不是限制项）。→ **100% 占用率（64 warp/SM）**。
- **与 DNN 的对应关系**：这个 kernel 就是"一个全连接层"。真实的 CNN 里，卷积层经 im2col 后是同一个 GEMM 结构（只是 `in_dim` 变成 `C·K·K`、`batch` 变成 `B·H_out·W_out`）；反向传播里的 `dE/dW = (dE/dfc)·xᵀ` 是另一次 GEMM（转置方向不同，是 TN 形式），但 tiling 结构完全一样。**所以"把 GEMM 做到极致"等于把整个 DNN 做到极致**——这就是 cuDNN/cuBLAS 存在的理由，也是 Tensor Core 存在的理由。

【性能优化分析】

- **算术强度与 Roofline**：`运算量 = 2 × batch × in_dim × out_dim = 2 × 1024 × 784 × 1024 = 1.643e9 FLOP = 1.643 GFLOP 访存量 = X(1024×784×4 B = 3.211 MB) + W(1024×784×4 B = 3.211 MB) + Y(1024×1024×4 B = 4.194 MB) = 10.616 MB 算术强度 AI = 1.643e9 / 1.0616e7 ≈ 155 FLOP/B A100 Roofline 拐点 = 19.5e12 / 1555e9 ≈ 12.5 FLOP/B 155 ≫ 12.5 ⇒ 计算受限` 期望性能 = `min(19.5 TFLOPS, 1555 GB/s × 155 FLOP/B = 241 TFLOPS) = 19.5 TFLOPS`；但受前述"shared load : FMA = 2:1"的发射限制，实测通常落在 **7~10 TFLOPS**：`实测时间 ≈ 1.643e9 / 8.5e12 ≈ 0.19 ms ×20 次取平均后打印的 GFLOP/s 应落在 6~10 TFLOP/s 区间` 对比一下同一层的"逐元素"实现（每线程做一个输出神经元、从全局内存读整行 W）：`AI = 2·784/(784·4 + 784·4 + 4) ≈ 0.32 FLOP/B`，**掉到 12.5 以下，立刻变成内存受限**，性能上限只有 `1555 × 0.32 = 497 GFLOP/s` ——**同一层的两种写法性能差 17 倍**，差别完全来自数据复用。
- **占用率与延迟隐藏**：`每 SM 驻留 8 block = 2048 线程 = 64 warp，占用率 100% 每 warp 每周期发射指令数上限 = 1（单发射） 点积循环每 16 次迭代产生 16 条 FMA + 32 条 LDS，共 48 条指令 ⇒ FMA 占比 = 16/48 = 33%，即 FMA 发射率上限 = 33% × 峰值 实测 8.5 TFLOPS / 19.5 TFLOPS = 43.6% —— 略高于 33%， 因为编译器会把部分 shared load 合并/复用进寄存器` 这就是"**指令混合比（instruction mix）决定实际性能**"的典型案例：占用率已经是 100%，再调块大小没用，唯一出路是**增加每线程的计算量**（register blocking），把 48 条指令里的 16 条 FMA 变成 32 条甚至 64 条。
- **可执行的优化方向**：① **thread coarsening**：每个线程算 2×2 或 4×4 个输出元素，`subX` / `subW` 的 shared load 次数按 `sqrt(复用数)` 摊薄，FMA 占比可以提到 60%~70%；② **向量化访存**：用 `float4` 载入全局内存，把 2 次 64 字节 transaction 变成 1 次 128 字节（前提是 16 字节对齐）；③ **改回 128 字节合并**：把 block 形状改为 `(32,8)` 或 `TILE=32`；④ **上 Tensor Core**：把输入量化到 fp16/bf16、用 WMMA 或 `mma` PTX 指令替换 FMA，这一层的理论峰值从 19.5 TFLOPS 变成 312 TFLOPS；⑤ **融合**：把 bias 加法与 ReLU 融进 GEMM 的 epilogue（本 kernel 已经这么做了），避免多写一遍 Y；⑥ 训练时用 **mini-batch**（讲义页 19~21：逐个样本累积 ΔΘ 最准确但并行度低，mini-batch 在"梯度估计精度"与"并行度"之间取平衡）。

代码示例 5：WebGPU / WGSL 与 CUDA 的逐行对照（课程最终项目所用模型）

文件 1：WGSL 计算着色器（`vector_add.wgsl`），与 CUDA 的 `vecAdd` kernel 功能完全相同。

```
// 文件: vector_add.wgsl
// 运行环境: 浏览器(Chrome/Edge 113+)或原生 WebGPU 实现
// 计算 c[i] = a[i] + b[i]，n 由数组长度决定

// ---- 资源绑定: 相当于 CUDA kernel 的指针参数 ----
// 对应 CUDA:  const float* __restrict__ a
@group(0) @binding(0) var<storage, read>       a : array<f32>;
// 对应 CUDA:  const float* __restrict__ b
@group(0) @binding(1) var<storage, read>       b : array<f32>;
// 对应 CUDA:  float* c   (read_write 表示既可读也可写)
@group(0) @binding(2) var<storage, read_write> c : array<f32>;

// ---- 工作组大小: 相当于 CUDA 的 blockDim ----
// override 是"着色器编译期常量", 创建 pipeline 时可用 constants 覆盖(类似模板参数)
override WG_SIZE : u32 = 256u;

// 对应 CUDA:  __global__ void vecAdd(const float* a, const float* b, float* c, int n)
// @compute 表示这是一个计算着色器阶段; @workgroup_size 指定工作组内 invocation 数
@compute @workgroup_size(WG_SIZE)
fn main(
    // 对应 CUDA:  int i = blockIdx.x * blockDim.x + threadIdx.x;
    @builtin(global_invocation_id) gid : vec3<u32>,
    // 对应 CUDA:  threadIdx.x / threadIdx.y / threadIdx.z
    @builtin(local_invocation_id)  lid : vec3<u32>,
    // 对应 CUDA:  blockIdx.x / blockIdx.y / blockIdx.z
    @builtin(workgroup_id)         wid : vec3<u32>,
    // 对应 CUDA:  gridDim.x / gridDim.y / gridDim.z
    @builtin(num_workgroups)       nwg : vec3<u32>
) {
    // 对应 CUDA:  if (i < n)   —— 这里 n 用 arrayLength 动态取得,
    // 相当于 CUDA 里把 n 作为参数传进来(WebGPU 没有隐式的大小信息,
    // 但运行时数组的 arrayLength 就是缓冲区元素个数)
    let n : u32 = arrayLength(&a);
    let i : u32 = gid.x;

    // 对应 CUDA:  if (i < n) c[i] = a[i] + b[i];
    if (i >= n) {
        return;
    }
    c[i] = a[i] + b[i];

    // lid / wid / nwg 在本例中未使用, 保留是为了展示对应关系;
    // WGSL 要求声明的内建变量可以不用, 不会报错。
    let unused_lid : u32 = lid.x;
    let unused_wid : u32 = wid.x;
    let unused_nwg : u32 = nwg.x;
}
```

文件 2：主机端 JavaScript（浏览器或 Deno/Node + WebGPU）。

```
// 文件: vector_add_host.js
// 运行: 在支持 WebGPU 的浏览器控制台/模块中执行, 或在 Deno 中执行

// ---- 1. 获取适配器与设备: 对应 CUDA 的 cudaGetDevice / cudaSetDevice ----
const adapter = await navigator.gpu.requestAdapter();
if (!adapter) { throw new Error('no WebGPU adapter'); }
const device  = await adapter.requestDevice();

// ---- 2. 准备主机数据 ----
const n        = 1 << 24;              // 16.7M 个 float
const elemSize = 4;
const bytes    = n * elemSize;
const h_a = new Float32Array(n);
const h_b = new Float32Array(n);
const h_c = new Float32Array(n);
for (let i = 0; i < n; ++i) { h_a[i] = 1.0 + (i % 7); h_b[i] = 2.0 + (i % 5); }

// ---- 3. 创建缓冲区: 对应 CUDA 的 cudaMalloc ----
const usageIn = GPUBufferUsage.STORAGE | GPUBufferUsage.COPY_DST;
const bufA = device.createBuffer({ size: bytes, usage: usageIn });
const bufB = device.createBuffer({ size: bytes, usage: usageIn });
const bufC = device.createBuffer({
  size: bytes,
  usage: GPUBufferUsage.STORAGE | GPUBufferUsage.COPY_SRC | GPUBufferUsage.COPY_DST,
});

// ---- 4. 上传数据: 对应 CUDA 的 cudaMemcpy(HostToDevice) ----
device.queue.writeBuffer(bufA, 0, h_a);
device.queue.writeBuffer(bufB, 0, h_b);

// ---- 5. 编译着色器 + 创建计算管线 ----
// 对应 CUDA 的 nvcc 离线编译; WebGPU 是运行时编译
const wgslSource = await (await fetch('./vector_add.wgsl')).text();
const shaderModule = device.createShaderModule({ code: wgslSource });
const pipeline = device.createComputePipeline({
  layout: 'auto',                                  // 自动推导绑定布局
  compute: { module: shaderModule, entryPoint: 'main' },
});

// ---- 6. 绑定组: 把缓冲区按 @group/@binding 槽位"插"到着色器上 ----
const bindGroup = device.createBindGroup({
  layout: pipeline.getBindGroupLayout(0),
  entries: [
    { binding: 0, resource: { buffer: bufA } },
    { binding: 1, resource: { buffer: bufB } },
    { binding: 2, resource: { buffer: bufC } },
  ],
});

// ---- 7. 编码并提交命令: 对应 CUDA 的 kernel<<<grid, block>>> ----
const WORKGROUP_SIZE = 256;
const numWorkgroups  = Math.ceil(n / WORKGROUP_SIZE);

const encoder = device.createCommandEncoder();
const pass    = encoder.beginComputePass();
pass.setPipeline(pipeline);
pass.setBindGroup(0, bindGroup);
pass.dispatchWorkgroups(numWorkgroups, 1, 1);      // 对应 cudaLaunchKernel 的 grid
pass.end();

const t0 = performance.now();
device.queue.submit([encoder.finish()]);           // 对应 CUDA 的 kernel 启动(异步)
// ---- 8. 同步: 对应 CUDA 的 cudaDeviceSynchronize() ----
await device.queue.onSubmittedWorkDone();
const t1 = performance.now();
console.log(`kernel 时间: ${(t1 - t0).toFixed(3)} ms`);

// ---- 9. 读回结果: 对应 CUDA 的 cudaMemcpy(DeviceToHost) ----
// WebGPU 用 mapAsync + getMappedRange 完成"设备内存 -> 可被 CPU 访问的视图"
await bufC.mapAsync(GPUMapMode.READ);
const mapped = new Float32Array(bufC.getMappedRange().slice(0));   // slice 复制一份
bufC.unmap();

// ---- 10. 验证 ----
let bad = 0;
for (let i = 0; i < n; i += 1000003) {
  if (Math.abs(mapped[i] - (h_a[i] + h_b[i])) > 1e-6) { ++bad; }
```

```
}
console.log(bad === 0 ? 'PASS' : 'FAIL');

// ---- 11. 释放: 对应 CUDA 的 cudaFree; WebGPU 用 destroy() ----
bufA.destroy();
bufB.destroy();
bufC.destroy();
```

文件 3：WGSL 的工作组内归约，用来对照 CUDA 的 `__shared__` + `__syncthreads()`。

```
// 文件: workgroup_reduce.wgsl
// 每个 workgroup 把 WG_SIZE 个元素求和, 写出一个部分和
// 对照 CUDA: __shared__ float s[256]; s[tid] = v[id]; __syncthreads();
//            for (stride = blockDim.x / 2; stride > 0; stride >>= 1) {
//                if (tid < stride) { s[tid] += s[tid + stride]; }
//                __syncthreads();
//            }

@group(0) @binding(0) var<storage, read>       input  : array<f32>;
@group(0) @binding(1) var<storage, read_write> output : array<f32>;

const WG_SIZE : u32 = 256u;

// workgroup 地址空间: 对应 CUDA 的 __shared__
var<workgroup> scratch : array<f32, 256>;

@compute @workgroup_size(WG_SIZE)
fn main(
    @builtin(local_invocation_id)  lid : vec3<u32>,
    @builtin(global_invocation_id) gid : vec3<u32>,
    @builtin(workgroup_id)         wid : vec3<u32>
) {
    let t : u32 = lid.x;

    // 协作载入: 每个 invocation 搬一个元素到工作组存储
    scratch[t] = input[gid.x];

    // 对应 CUDA 的 __syncthreads(): 保证所有 invocation 都写完
    workgroupBarrier();

    // 树形归约, 步长逐次减半
    var stride : u32 = WG_SIZE / 2u;
    loop {
        if (stride == 0u) { break; }
        if (t < stride) {
            scratch[t] = scratch[t] + scratch[t + stride];
        }
        workgroupBarrier();        // 每一轮都要同步, 否则会读到别人写了一半的数据
        stride = stride / 2u;
    }

    // 只有 0 号 invocation 写出结果
    if (t == 0u) {
        output[wid.x] = scratch[0];
    }
}
```

CUDA 与 WebGPU 的逐行对照表

行	CUDA	WebGPU / WGSL	语义差异说明
1	`__global__ void vecAdd(const float* a, const float* b, float* c, int n)`	`@compute @workgroup_size(256) fn main(@builtin(global_invocation_id) gid : vec3<u32>)`	CUDA 的指针参数在 WGSL 里变成**模块级绑定**（`@group/@binding`），不是函数参数
2	`const float* __restrict__ a`	`@group(0) @binding(0) var<storage, read> a : array<f32>;`	WGSL 显式区分只读（`read`）与读写（`read_write`），只读可让驱动做更好的布局
3	`float* c`	`var<storage, read_write> c : array<f32>;`	同上
4	`int n`（显式传参）	`arrayLength(&a)`	WebGPU 的运行时数组自带长度；也可用 uniform buffer 传标量
5	`int i = blockIdx.x * blockDim.x + threadIdx.x;`	`@builtin(global_invocation_id) gid : vec3<u32>` → `gid.x`	语义完全相同，WGSL 由运行时直接给出全局 id

行	CUDA	WebGPU / WGSL	语义差异说明
6	`threadIdx.x`	`@builtin(local_invocation_id) lid : vec3<u32>`	相同
7	`blockIdx.x`	`@builtin(workgroup_id) wid : vec3<u32>`	block ↔ workgroup
8	`blockDim.x`	`@builtin(workgroup_size) : vec3<u32>` 或 `@workgroup_size(256)` 的字面量	相同
9	`gridDim.x`	`@builtin(num_workgroups) : vec3<u32>`	相同
10	`__shared__ float s[256];`	`var<workgroup> s : array<f32, 256>;`	都在片上，生命周期 = 一个 block/workgroup
11	`__syncthreads();`	`workgroupBarrier();`	语义相同：workgroup 内所有 invocation 的屏障
12	`__syncwarp();`	`subgroupBarrier();`	warp ↔ subgroup（大小平台相关，NVIDIA 上为 32）
13	`atomicAdd(&x, 1.0f);`	`atomicAdd(&x, 1.0f);`	**同名同参**，是最高兴的一处巧合
14	`cudaMalloc(&d_a, bytes);`	`device.createBuffer({size: bytes, usage: GPUBufferUsage.STORAGE})`	WebGPU 必须显式声明用途位（STORAGE/COPY_SRC/COPY_DST）
15	`cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice);`	`device.queue.writeBuffer(bufA, 0, h_a);`	相同语义
16	`vecAdd<<<grid, 256>>>(d_a, d_b, d_c, n);`	`pass.dispatchWorkgroups(numWorkgroups, 1, 1);`	`<<<>>>` ↔ 命令编码器 + `dispatchWorkgroups`
17	`cudaDeviceSynchronize();`	`await device.queue.onSubmittedWorkDone();`	相同语义
18	`cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost);`	`await bufC.mapAsync(GPUMapMode.READ); new Float32Array(bufC.getMappedRange())`	WebGPU 的读回是"映射"而不是"拷贝"，且**必须 await**
19	`cudaFree(d_a);`	`bufA.destroy();`	相同
20	没有对应物	`@group(0) @binding(0)` 的绑定布局校验	WebGPU 在**创建 bind group 时**就会检查布局是否与着色器匹配，能在提交前发现错绑
21	没有对应物	`override WG_SIZE : u32 = 256u;`	相当于"编译期模板参数"，可在创建 pipeline 时覆盖，用于快速调参
22	`-arch=sm_80` 离线编译	`device.createShaderModule()` 运行时编译	WebGPU 的编译发生在运行时，会有一次性的编译延迟
23	1024 线程/block	256 invocation/workgroup（保证下限）	**迁移时最需要注意的定量差异**：归约层次更深
24	Tensor Core `wmma::mma_sync`	无对应物	WGSL 没有子组矩阵乘指令，矩阵乘只能靠 `dot()` 手工展开

课程最终项目使用的 WebGPU / RAI 与 CUDA 的对照总结

课程目录里明确写着实验室用 **WebGPU**、最终项目用 **RAI**（原文："C Programming Language and CUDA Software Development Kit, WebGPU for labs, RAI for final project"），而实验室历史上有过"Labs are done through WebGPU.net"与"Labs are done through Delta"两种环境。对已经会 CUDA 的人来说，迁移到最终项目只需要换掉四层认知：

- **计算抽象层**：CUDA 让你直接写 `<<<grid, block>>>` 和 `__global__`；WebGPU 让你写 WGSL 的 `@compute` 入口 + 主机端编码命令。RAI 在 WebGPU 之上再抽象一层，把并行工作组织成**任务（task）/ 图（graph）**：你描述"有哪些数据缓冲区、有哪些 kernel、它们之间的依赖是什么"，由框架负责提交顺序、缓冲区分配与同步。这相当于把 CUDA 里手工写的 `cudaMemcpyAsync` + stream + event 依赖图，交给框架自动生成——**概念上等价于 CUDA Graphs，但跑在 WebGPU 上**。
- **可移植性层**：CUDA 代码只能在 NVIDIA GPU 上跑，且需要安装驱动与 CUDA Toolkit；WebGPU/RAI 代码在浏览器、Windows、macOS、Linux、Android、iOS 上都能跑，**零安装分发**——这正是课程把最终项目放在 WebGPU 上的原因：学生可以在自己的笔记本上开发、在浏览器里演示。
- **性能层**：CUDA 上可以用 Tensor Core（A100 上 fp16 峰值 312 TFLOPS）、`cp.async`、`__shfl_sync`、cluster 等全部硬件特性；WebGPU 上目前只有 `f32` / `f16` 标量与向量运算、`subgroup` 操作（且部分是可选项）、以及工作组的显式存储。因此**同一个 GEMM 的性能上限相差一个数量级以上**——但这不影响把这些优化技术（tiling、register blocking、合并访存、减少 bank conflict、提高占用率）**原样迁移过去**，因为它们的瓶颈模型完全一样。
- **正确性层**：CUDA 的错误多半是"静默的错误数据"（忘记同步、越界、方向写错），WebGPU 多了一层**编译期/创建期校验**（绑定布局、访问模式、越界检测在开发模式下会报错），但读回数据时少了一层"自动同步"（必须 `await mapAsync`），所以**最容易犯的错变成"忘记 await"**——表现是读到全 0 或旧数据，而不是崩溃。

一句话总结课程范围的迁移路线：**把 CUDA 的"线程映射 + 内存层次 + 同步"三件套原样翻译成 WGSL 的"invocation 映射 + storage/workgroup 地址空间 + workgroupBarrier"，把 CUDA 的"stream + event"翻译成"命令编码器 + submit + 框架的依赖图"，剩下的优化直觉完全通用。**

【代码做什么？】

1. **着色器侧**：`main` 通过 `@builtin(global_invocation_id)` 拿到全局 invocation 编号 `i`，用 `arrayLength(&a)` 取得真实长度 `n`，做边界检查后执行 `c[i] = a[i] + b[i]`。`a`、`b` 是只读 storage buffer（`var<storage, read>`），`c` 是可读写 storage buffer（`var<storage, read_write>`）。
2. **主机侧获取设备**：`navigator.gpu.requestAdapter()` 对应 `cudaGetDeviceCount` 的"枚举设备"，`adapter.requestDevice()` 对应 `cudaSetDevice` + 上下文创建。
3. **创建缓冲区**：`device.createBuffer({size, usage})` 对应 `cudaMalloc`，但必须显式声明用途位：读入用 `STORAGE | COPY_DST`，读回结果再加 `COPY_SRC`。
4. **上传数据**：`device.queue.writeBuffer(bufA, 0, h_a)` 对应 `cudaMemcpy(HostToDevice)`。它内部就是一次队列提交，不需要显式同步（后续提交的命令天然排在它后面）。
5. **创建管线与绑定组**：`createShaderModule` 运行编译 WGSL（对应 `nvcc` 的离线编译），`createComputePipeline` 对应"选定 kernel + 执行配置模板"，`createBindGroup` 把缓冲区按 `@group(0) @binding(k)` 的槽位插上去（**这一步会校验布局，错绑会在提交前报错**）。
6. **编码与提交**：`beginComputePass` → `setPipeline` → `setBindGroup` → `dispatchWorkgroups(ceil(n/256))` → `pass.end()` → `queue.submit([encoder.finish()])`。整体对应 CUDA 的一次 `kernel<<<grid, block>>>` 加启动，但 WebGPU 把"命令录制"与"提交执行"分开，因此可以一次录制、多次提交。
7. **同步与读回**：`await device.queue.onSubmittedWorkDone()` 对应 `cudaDeviceSynchronize()`；读回必须 `await buffer.mapAsync(GPUMapMode.READ)` 再 `getMappedRange()`，最后 `unmap()`。
8. **归约示例**：`workgroup_reduce.wgsl` 把每个 invocation 的一个元素写进 `var<workgroup> scratch`，`workgroupBarrier()` 之后做步长减半的树形归约，每轮归约之间都要再 `workgroupBarrier()`，最后编号 0 的 invocation 写出工作组部分和。
9. **释放**：`bufA.destroy()` 对应 `cudaFree`。

【并行机制与硬件映射解说】

- **invocation → warp 的映射**：WGSL 的 workgroup 在 NVIDIA 硬件上就是 CUDA 的 thread block。`@workgroup_size(256)` 意味着每个 workgroup 有 256 个 invocation，硬件上被切成 **8 个 warp**（每 32 个连续 `local_invocation_id.x` 为一个 warp）。这是 CUDA 与 WebGPU 在硬件层**完全一致**的部分。
- **全局编号的构成**：WebGPU 没有直接给出 `blockIdx * blockDim + threadIdx` 的表达式，而是把结果包成 `global_invocation_id`，等价于 `workgroup_id * workgroup_size + local_invocation_id`。三者的关系与 CUDA 完全相同，只是 WGSL 用 `vec3<u32>` 一次给出三个维度。
- **合并访存**：`gid.x` 连续的 32 个 invocation 访问 `a[i]` 中连续 32 个 `f32` = **128 字节 = 一条 cache line = 一次 transaction**，与 CUDA 版的合并行为完全一致。因此在 CUDA 上"让相邻线程访问相邻元素"的规则在 WGSL 里同样成立，把 `a[2*i]` 改成步长访问一样会让 transaction 数翻倍。
- **工作组存储与 bank 冲突**：`var<workgroup> scratch : array<f32, 256>` 在硬件上落在片上暂存器（shared memory），同样按 32 bank × 4 字节组织。归约中 `scratch[t]` 与 `scratch[t + stride]` 的地址都是**连续**的（`t` 在 warp 内连续），因此初始阶段是一一对应的 bank 访问（0 路冲突）；当 `stride < 32` 时，参与计算的 invocation 不足一个 warp，多出来的 lane 被谓词关闭（对应 CUDA 的 warp 发散），此时访问仍然是地址各异的单播，没有 bank 冲突。**如果改成 `scratch[t * 32]` 这类跨步访问，就立刻会出现 32 路冲突**——这与 CUDA 的规则一字不差。
- **工作组大小上限 256 的后果**：CUDA 允许 1024 线程/block，WebGPU 只保证 256 invocation/workgroup。于是同样处理 4096 个元素，CUDA 需要 4 个 block，WebGPU 需要 16 个 workgroup；块内归约的层次也从 1024→512→…→32 变成 256→128→64→32，最后 32 元素的归约要么交给 `subgroupAdd` 之类的子组操作，要么由单个 invocation 串行完成。**这是迁移时最容易被忽略的性能细节**。
- **同步与可见性**：`workgroupBarrier()` 保证 workgroup 内的存储写对同组其他 invocation 可见（对应 `__syncthreads()`）；跨 workgroup 的依赖在 WebGPU 里**无法用屏障表达**，只能通过拆成两次 dispatch 并把中间结果写进 storage buffer 来实现（对应 CUDA 里"拆成两个 kernel"或用全局原子/栅栏）。这与 CUDA 的语义边界完全一致：**block 之间没有屏障，只有 kernel 边界**。
- **命令提交与异步性**：`queue.submit` 是异步的（对应 kernel 启动），`await` 才是同步点（对应 `cudaDeviceSynchronize`）。若忘记 `await` 就去读映射内存，读到的可能是全 0 或旧值——这与 CUDA 忘记 `cudaDeviceSynchronize()` 读到垃圾数据是同一类错误，但**在 WebGPU 上不会崩溃、只会静默出错**，更难调试。
- **没有 Tensor Core 通道**：WGSL 没有 `wmma` / `mma` 对应的标准指令，因此代码示例 3 里那种 16×16×16 的矩阵乘加阵列在 WebGPU 上无法直接使用；矩阵乘只能展开成 `vec4<f32>` 的点积（`dot()`）来提高每 invocation 的算术强度。**这是 WebGPU 与 CUDA 之间最大的一处能力落差。**

【性能优化分析】

- **算术强度与 Roofline**（向量加法的 AI 与 CUDA 版完全相同）：`每个元素：1 次加法 = 1 FLOP；访存 = 读 a 4 B + 读 b 4 B + 写 c 4 B = 12 B AI = 1 / 12 ≈ 0.083 FLOP/B 在 A100 上（假设 WebGPU 后端能跑满硬件）： 拐点 = 19.5e12 / 1555e9 = 12.5 FLOP/B ≫ 0.083 ⇒ 纯带宽受限 可达上限 = 1555 GB/s × 0.083 ≈ 129 GFLOP/s 16.7M 个元素：64 MB × 3 = 192 MB -> 192 MB / 1555 GB/s ≈ 0.124 ms` 这解释了为什么 WebGPU 的向量加法**在 CUDA 上和在浏览器上都会撞到同一条内存墙**——差别不在算法，而在命令提交与校验带来的额外固定开销。
- **占用率的等价概念**：WebGPU 里没有 `cudaOccupancy` API，但物理约束完全相同。一个 256 invocation、使用 `array<f32, 256>`（1 KB）工作组的着色器，在 A100 上每 SM 可驻留的 workgroup 数受 triple 约束：`workgroup 数 ≤ 32`、`invocation 数 ≤ 2048`（即 8 个 256 的 workgroup）、`workgroup 存储 ≤ 164 KB`（1 KB/组不是限制）。因此实际驻留 **8 个 workgroup = 2048 invocation = 64 warp = 100% 占用率**，与 CUDA 版一致。
- **总时间模型（含 WebGPU 特有开销）**：`T_webgpu ≈ T_compile(一次性) + T_encode + T_submit + T_kernel + T_mapAsync + T_validate 其中 T_compile 只在 createShaderModule / createComputePipeline 时发生一次，若每帧重建 pipeline，这项开销会从 <1 ms 变成 5~20 ms —— 必须缓存 pipeline 与 bind group。 T_kernel 本身 ≈ 0.124 ms（上文的带宽下界） ⇒ 在真实浏览器里，小规模向量加法的时间几乎全被固定开销占据，这正对应讲义"Execution Time Never Reaches Zero"与"Data transfers have similar non-linearities for small sizes"两条结论。`

- **可执行的优化方向**：① **缓存** `GPUComputePipeline` 与 `GPUBindGroup`，只在缓冲区尺寸变化时重建；② 让每个 invocation 处理**多个元素**（例如用 `vec4<f32>` 一次算 4 个），把 dispatch 的 invocation 总数降到 1/4，减少固定开销相对占比；③ 用 **storage buffer 而不是 uniform buffer** 传大数组（uniform 有 64 KB 级的大小限制与更严格的布局要求）；④ 减少 CPU↔GPU 往返：把多步计算录进**同一个 command encoder** 一次提交，而不是每步都 `await`；⑤ 结果读回用 `mapAsync` 且**只在真正需要时**读回（例如只在验证阶段），推理路径上把数据留在 GPU 上；⑥ 若确实需要矩阵乘且必须用 WebGPU，把 `f32` 换成 `f16`（需启用 `shader-f16` 扩展特性）以获得 2 倍的寄存器与带宽效率，并用 `dot(vec4<f16>, vec4<f16>)` 手工展开。

性能优化技巧总结

1. **能用 pinned 就用 pinned**：`cudaHostAlloc` 让 DMA 只有一跳，H2D 带宽从 ~11 GB/s 提到 ~23 GB/s（Gen4 x16），是"2 倍白送"的加速；但**不要滥用**——pinned 内存不可换页，过订会让整机内存吃紧。
2. **异步拷贝必须配 pinned**：`cudaMemcpyAsync` 遇到 pageable 内存会退化成同步 staging，重叠收益直接归零。
3. **切段 + 多流做流水线**：把总时间从 `T_in + T_c + T_out` 压向 `max(T_in, T_c, T_out)`；段数取"能填满流水线的最小值"，用实测扫描决定（讲义的例子最优是 4~5 段）。
4. **至少三个流才能连续流水**：两个流时 `C.i` 会在拷贝引擎队列里挡住下一轮的 `A.i+1`（表头阻塞），且 PCIe 只有一个方向在工作。
5. **入队顺序要"分批"**：先发完所有 H2D，再发所有 kernel，最后发所有 D2H，避免读方向被写方向阻塞。
6. **用事件做跨流同步，不要用 `cudaDeviceSynchronize`**：`cudaEventRecord` + `cudaStreamWaitEvent` 只建立必要的依赖；`cudaDeviceSynchronize` 会摧毁所有并行度。
7. **警惕 legacy default stream**：默认流与所有阻塞流隐式同步；用 `cudaStreamNonBlocking` 或 `--default-stream per-thread` 规避。
8. **Tensor Core 要用"低精度输入 + FP32 累加"**：fp16/bf16 输入把峰值拉到 FP32 的 16 倍，累加留在 fp32 才能控制误差不随 K 增长。
9. **先提高算术强度，再谈换指令**：朴素 WMMA 只有 8 FLOP/B、受 L2 带宽限制只能跑到 45 TFLOPS；把 tile 经共享内存做多级复用（AI > 200 FLOP/B）才能逼近 312 TFLOPS 的峰值。
10. **register blocking 比调块大小更能提升 tiled kernel**：shared load : FMA = 2:1 时 FMA 发射率被压到 33%；每线程多算几个输出元素能把 shared load 摊薄，这是"占用率已经 100% 却还是慢"的标准解法。
11. **一切 DNN 层都先翻译成 GEMM**：全连接是 GEMM，卷积经 im2col 也是 GEMM，反向传播还是 GEMM；优化一套 GEMM 内核等于优化整个网络。
12. **共享内存 padding 消除 bank conflict**：步长为 32 的倍数的访问会造成 32 路冲突；padding 一列（`[TILE][TILE+1]`）即可打散 bank。
13. **合并访问按 128 字节 transaction 核算**：`TILE_WIDTH=16` 的 tile 载入在一个 warp 内只覆盖两段 64 字节，效率只有 50%；让 warp 内 `tx` 覆盖 32 个连续元素即可拿满。
14. **对齐 WebGPU 的约束**：workgroup 上限 256、读回必须 `await mapAsync`、没有 Tensor Core 对应物——迁移前先把这三条记在纸上。
15. **用 Roofline 决定"还能不能更快"**：先算 `max(各阶段)` 或 `min(峰值算力, 带宽×AI)` 的理论上限，再判断当前离上限多远，避免在已经到顶的地方瞎调。

关键要点

- **带宽决定一切**：A100 的显存带宽 1555 GB/s 对 PCIe Gen4 x16 的 32 GB/s 是 49 倍差距，对 Gen5 x16 的 64 GB/s 是 24 倍；NVLink 3.0 的 600 GB/s 把 GPU 间带宽拉回 PCIe Gen4 的约 10 倍。任何"把数据搬来搬去"的算法都要先算这笔账。
- **拷贝比计算更值得优化**：典型 GPU 程序里 kernel 只占几毫秒，PCIe 传输占几十毫秒；pinned + 多流流水线能把总时间从 `T_in+T_c+T_out` 压到 `max(T_in, T_c, T_out)`，这是"零算法改动"的加速。
- **流的本质是"没有依赖就可以并行"**：同流内严格有序、跨流无约束；重叠的前提是硬件上 SM（计算）与 Copy Engine（DMA）本来就是独立的两个引擎。
- **Tensor Core 的 16 倍不是自动到手的**：硬件峰值 312 TFLOPS（A100，fp16）对比 FP32 的 19.5 TFLOPS，但朴素 WMMA kernel 只能拿到 ~45 TFLOPS（14%），差距全部来自数据复用不足导致的 L2 带宽瓶颈。
- **深度学习 = 大规模 GEMM**：全连接层 $Y = X \cdot W^T + b$、卷积层经 im2col 展开、反向传播的 $dE/dW = (dE/dfc) \cdot x^T$ 都是 GEMM；这也是 GPU 硬件（Tensor Core、结构化稀疏、HBM）被深度学习需求反复推动的原因。
- **CUDA 不是唯一的模型，但它的心智模型是通用的**：无论 OpenCL 的 work-item、OpenACC 的 gang/worker、还是 WebGPU 的 invocation/workgroup，共同特征都是"层次化轻量核心 + 本地暂存器 + 软件管理内存 + 无硬件一致性 + BSP 同步"——而 WebGPU/WGSL（课程最终项目所用模型，其上还有 RAI 这层任务图框架）带来的可移植性与零安装分发，与 CUDA 提供的性能上限形成互补，在 CUDA 上学到的 tiling、合并访存、bank conflict、占用率这些直觉 100% 可以迁移过去。

常见陷阱与注意事项

- **忘记同步就使用结果**：CUDA 侧表现为读到垃圾数据（kernel 还在跑就去 D2H 拷贝）→ 必须 `cudaDeviceSynchronize()` 或对相应流 `cudaStreamSynchronize()`；WebGPU 侧表现为读到全 0 或旧内容 → 必须 `await device.queue.onSubmittedWorkDone()` 或 `await buffer.mapAsync(GPUMapMode.READ)`。**这类错误不报错、只出错数据，是最难查的一类 bug。**
- **忘记 `__syncthreads()`，或把它放进线程发散的分支**：tiled kernel 里第二轮迭代的快速线程会覆盖别人还在读的 tile，结果随调度随机变化 → 在"协作载入完成之后"和"共享内存使用完毕之后"各放一次 `__syncthreads()`，且**绝不能写在 `if` 分支内部**（CUDA 要求同一 block 内所有线程到达同一个屏障，否则行为未定义；WebGPU 的 `workgroupBarrier()` 同理，但会被校验器直接拒绝）。
- **共享内存 bank conflict**：地址步长是 32 的倍数时 32 个线程挤在同一个 bank。典型反例是把 tile 转置存放后用 `subW[tx][k]` 读取（地址 `tx*16+k`），16 个不同的字只落在 2 个 bank 上，造成 **8 路冲突**、共享内存吞吐掉到 1/8 → 用 `[TILE][TILE+1]` 做 padding，或像代码示例 4 那样保持行主序并用

`subW[k][tx]` 访问。

- **warp 发散（divergence）**：`if (i < n)` 之类的边界检查开销可忽略，但把"整段计算"放进按数据分支的 `if/else` 里，会让一个 warp 的两条路径串行执行，吞吐直接减半 → 让同一 warp 内的线程走相同路径（如把条件改成基于 `threadIdx` 的对齐判断），或用谓词（predication）代替分支。
- **未检查 CUDA API 返回值**：kernel 启动失败（如 `too many resources requested for launch`、显存不足）**不会在启动点报错**，只在事后 `cudaGetLastError()` 里出现，于是程序可能"跑完了但结果全错" → 用统一的 `CUDA_CHECK` 宏包住每一次 API 调用，并在每次 kernel 启动后补一次 `CUDA_CHECK(cudaGetLastError())`。
- **越界访问与网格覆盖不足**：`grid = n / block` 的整数除法会丢掉余数，最后一批元素永远算不到；反过来，`grid` 取整过大又没有 `if (i < n)` 保护就会读写越界（可能静默破坏相邻缓冲区）→ 一律用 `(n + block - 1) / block` 计算网格，并在 kernel 首行加边界检查。
- **`cudaMemcpy` 方向写错、host/device 指针混用、共享内存容量超限**：方向参数写反（`HostToDevice` 与 `DeviceToHost` 互换）在异步版本里可能不报错但数据错乱；把 `cudaMalloc` 的指针交给主机代码解引用（或反之）通常直接段错误；共享内存开得太大（例如两个 `[64][64]` 的 float tile 就是 32 KB）会压低驻留 block 数甚至启动失败 → 用 `h_` / `d_` 前缀区分指针、把 memcpy 方向封装成模板函数自动推导、用 `cudaOccupancyMaxPotentialBlockSize` 或 `prop.sharedMemPerBlockOptin` 核定共享内存预算。
- **异步拷贝的"假重叠"与默认流隐式同步**：把 pageable 内存传给 `cudaMemcpyAsync` 会退化成同步 staging，收益归零；用 `kernel<<<g,b>>>`（省略流参数）投到 legacy default stream 会与所有阻塞流隐式同步，把多流并行彻底摧毁；段切得太小则 SM 闲置、kernel 启动开销占比飙升，总时间反而变长 → 主机内存一律 `cudaHostAlloc`、流一律 `cudaStreamNonBlocking`（或编译加 `--default-stream per-thread`）、段数用实测扫描而不是拍脑袋决定。

此外还有两个本讲特有的坑：**WMMA 的 `ldm` 对齐与 Tensor Core 精度**——16 位类型的 `ldm` 必须是 8 的倍数（16 字节对齐），否则 `load_matrix_sync` 会读错；把累加器也声明成 `__half` 会让误差随 K 线性累积，必须"低精度输入 + fp32 累加"，验证时用相对容差而不是精确相等。**WebGPU 的 usage 位与 pipeline 复用**——读回结果的 buffer 若没有 `COPY_SRC`、写出的 buffer 若没有 `COPY_DST`，会分别在 `mapAsync` / `writeBuffer` 时报错；而每帧重建 `GPUComputePipeline` 与 `GPUBindGroup` 会把固定开销从 <1 ms 抬到 5~20 ms，必须缓存。

思考题（带答案）

Q1. 某应用需要把 512 MB 的输入从主机搬到 A100、做完一次计算、再把 512 MB 的结果搬回来。计算部分在 A100 上需要 20 ms。使用 pinned 内存时 PCIe Gen4 x16 的实测单向带宽是 24 GB/s。问：(a) 串行版总时间是多少？(b) 用 4 段 4 流流水线的总时间是多少？(c) 段数趋于无穷时的理论下界是多少？(d) 如果改成 PCIe Gen5 x16（单向实测 48 GB/s），下界会变成多少？ (a) `T_in = 512 MB / 24 GB/s = 21.3 ms`，`T_out = 21.3 ms`，`T_serial = 21.3 + 20 + 21.3 = 62.6 ms`。(b) 用流水线公式 `(T_in+T_c+T_out)/N + (N-1)·max(T_in, T_c, T_out)/N`，`max = max(21.3, 20, 21.3) = 21.3 ms`，所以 `N=4`：`62.6/4 + 3×21.3/4 = 15.65 + 15.98 = 31.6 ms`，加速比 `62.6/31.6 ≈ 1.98x`。(c) `N→∞` 时总时间 → `max(21.3, 20, 21.3) = 21.3 ms`，加速比 2.94x。(d) Gen5 把单向带宽翻倍：`T_in = T_out = 10.7 ms`，此时瓶颈变成**计算**（20 ms > 10.7 ms），下界 = `max(10.7, 20, 10.7) = 20 ms`。**结论：当计算时间超过单方向传输时间时，继续提升 PCIe 带宽不再带来任何收益——优化方向必须转向 kernel 本身。**

Q2. A100 上 FP16 Tensor Core 峰值是 312 TFLOPS，FP32 CUDA core 是 19.5 TFLOPS，比值 16 倍。但一个"global → fragment"的朴素 WMMA kernel 实测只有约 45 TFLOPS，而 FP32 tiled kernel 约 9.6 TFLOPS。请解释为什么 WMMA 只拿到 4.7 倍而不是 16 倍，并给出至少两条把差距缩小的具体手段。 朴素 WMMA kernel 每 warp 每个 k 步读入 `2 × 16 × 16 × 2 B = 1024 B` 的 fragment、做 `2×16×16×16 = 8192 FLOP`，算术强度只有 `8192/1024 = 8 FLOP/B`。全部从 HBM 取的上限是 `1555 GB/s × 8 = 12.4 TFLOPS`；即使靠 40 MB L2 命中把实际流量降下来，`K/16 = 256` 个 k 步累计的 L2 请求仍有约 16 GB，按 L2 带宽 ~6 TB/s 算是 2.7 ms，与实测同量级——**瓶颈是 L2 带宽与 fragment 装载指令吞吐，不是 Tensor Core 的乘加阵列**。对比之下 312 TFLOPS 的屋脊点是 `312e12/1555e9 ≈ 200 FLOP/B`，所以必须把 AI 提高一个数量级。手段：① 把 A、B 的 tile 先经共享内存装载，global→shared→fragment 三级复用，把全局/L2 流量降低 10 倍以上；② **双缓冲（double buffering）** 或用 Ampere 的 `cp.async`，让第 k+1 步的装载与第 k 步的 `mma_sync` 重叠；③ 扩大 warp tile（如 64×64）以提高每个 fragment 的复用次数，摊薄 `load_matrix_sync` 的调用次数；④ 用 register blocking 让每个 warp 同时持有多个累加器 fragment，提高 `mma_sync` 的发射密度。

Q3. 讲义给出的流水线例题是：`T_in = 10 s`、kernel 启动常数开销 `1 s`、`T_comp = 10 s`、`T_out = 10 s`，3 条硬件流支持连续流水，求最优段数。请写出时间关于段数 N 的表达式、求出最优 N、并解释为什么 N 太大反而更慢。 时间表达式把"输入传输"和"输出传输"合并成 `20/N`（讲义把它们视为可在一段时间内连续推进的传输阶段）、计算固定为 `10`、再叠加 `N` 次 kernel 启动的常数开销（每次 1 s，且 kernel 启动不能与 kernel 执行重叠）：`T(N) = 20/N + 10 + N`。求极值：`dT/dN = −20/N² + 1 = 0 → N = √20 ≈ 4.47`，取整后 **N = 4 或 N = 5**，两者都是 `T = 5 + 10 + 4 = 19 s`（N=5 时为 `4 + 10 + 5 = 19 s`），加速比 `31/19 ≈ 1.63x`（串行版 `10+1+10+10 = 31 s`）。**N 太大反而更慢的原因**：`20/N` 这一项随 N 增大迅速饱和（N=10 时只剩 2 s，比 N=4 的 5 s 只省 3 s），而 `N` 这一项（每次 kernel 启动的固定开销，以及段太小导致 SM 闲置、warp 数不足以填满流水线、负载不均）随 N **线性增长**，两者此消彼长，最优出现在 `20/N ≈ N` 即 `N ≈ 4.5` 附近。这正对应讲义那两张图："Execution Time is Ideally Linear in Size" 与 "Execution Time Never Reaches Zero"。

Lecture 12: 最终项目 —— 问题定义、方案设计、性能优化与报告 (对应 Final Project)

概述

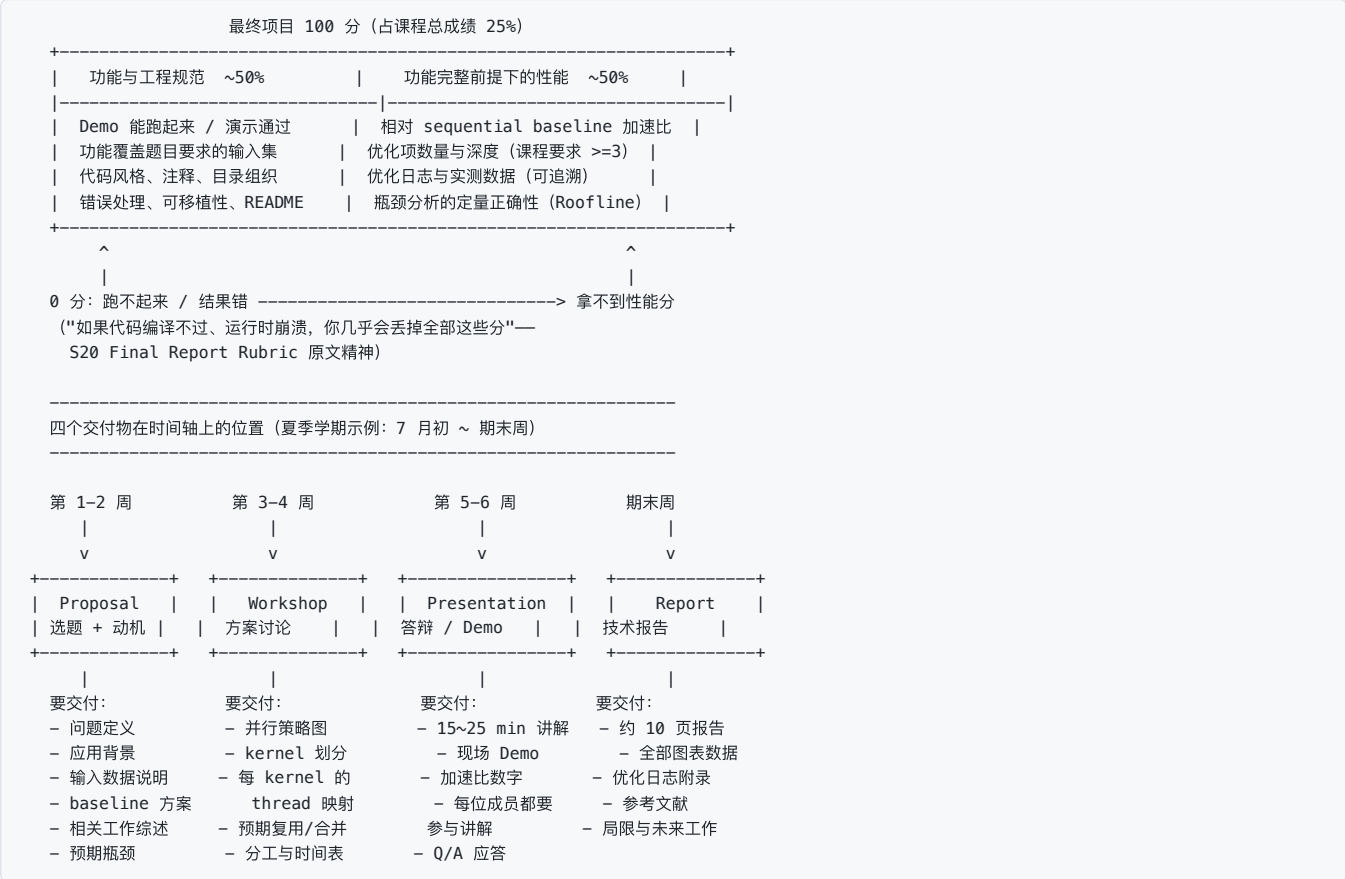
前面十一讲逐步建立了 CUDA 的并行思维工具箱：线程映射、内存层次、tiling、reduction tree、scan、atomics、稀疏方法与卷积分析。本讲不再引入新的并行模式，而是回答一个工程问题：**如何把所学的一切组织成一个可交付、可测量、可辩护的性能工程作品**。ECE408 的最终项目（Final Project）由 Project Proposal、Project Workshop、Project Presentation、Project Report 四个交付物构成，占课程成绩的 25%，其评分大致一半看"功能与工程规范"，一半看"功能完整前提下的性能"。这一讲的核心方法论是**性能工程循环（performance engineering cycle）**：正确的基线 → 测量与剖析 → 定位瓶颈 → 提出假设

→ 只改一个变量 → 重新测量 → 记录并决策，循环往复；辅以**设计空间探索（design space exploration, DSE）**的系统化参数扫描，以及**正确的可复现性验证（correctness verification）**。最终产出的不只是更快的代码，而是一份包含优化日志、Roofline 分析、诚实的失败记录与局限讨论的技术报告。

核心概念与 GPU 架构图解

最终项目的评分构成与四个交付物（Final Project Deliverables & Grading）

- **定义与目的**：最终项目是课程的综合性评价载体。官方课程说明中把它描述为 "Final Project that involves Project Proposal, Project Workshop, Project Presentation, and Project Report"；教学大纲（Instructional Objectives）中 C 组目标（10–17 条）明确要求学生在项目结束时能够"识别并求解一个计算问题"、"学习必要的领域知识"、"与不同学科的队友协作"、"合理划分责任"、"识别设计空间并探索优化机会"、"在展示中论证问题与方法"、"解释所做实验并为最终决策辩护"以及"指出方案的局限与未来方向"。四个交付物的存在目的，是把一个学期的终点任务切成四个**有明确截点的检查站**，防止团队把全部工作堆到最后一周。
- **直观解释（"它是什么？"）**：把最终项目想象成盖一栋房子。**Proposal** 是选址与需求书——你得先证明这块地上真的能盖房子（问题适合并行、数据拿得到、有可比对的基线）；**Workshop** 是图纸会审——结构工程师（老师与助教）帮你看承重墙位置对不对（并行策略是否站得住脚）；**Presentation** 是竣工验收答辩——你把房子推开门让人进来走一圈（现场演示 + 加速比是否真实）；**Report** 是竣工资料——以后别人要照着你的图纸重建（可复现、可引用、诚实的缺陷清单）。只交资料不交房子（只写报告代码跑不起来）或者只交房子不交资料（代码能跑但说不清为什么快），都会丢掉相当可观的分数。
- **架构/机制图解**：



关键操作与性能特征（这里"性能"指**团队的执行性能**）：Proposal 阶段的错误最昂贵——如果选题本身是"通信开销主导"或"数据依赖串行"的，后面三周的所有优化都在跟 Amdahl 定律的串行部分打架，收益上限定得很低；Workshop 阶段的收益最高，因为此时改设计只花几小时，而到 Presentation 前改设计要花几天；Report 阶段应只做**整理与提炼**，不应再做新实验（Rubric 明确说 conclusions 不是 summary，重复结果会得 0 分，要有反思）。四个交付物对应教学目标的 10–17 条，也就是说：报告里没有"局限与未来工作"一节，直接对应目标 17 未达成。

一个务实的交付物检查清单（每个里程碑交付前逐条打勾）：

```
Proposal 交付前
  [ ] 一句话能说清"算什么、输入多大、输出什么"
  [ ] 已确认输入数据真的拿得到（不是"应该能找到"）
  [ ] baseline 方案已经跑通（哪怕是 Python 串行版）
  [ ] 用 Roofline 算过一次理论加速上限，数字写进文档
  [ ] 至少 3 篇相关工作引用，且读懂了摘要与方法

Workshop 交付前
  [ ] 画出数据流图：几个 kernel、每个 kernel 输入输出是什么
  [ ] 每个 kernel 说明"一个线程负责什么"
  [ ] 说明每级内存的预期复用次数（用 tiling 公式算过）
  [ ] 分工表：每项优化谁负责、依赖谁

Presentation 交付前
  [ ] Demo 能在答辩机上跑（不是"在我笔记本上能跑"）
  [ ] 加速比表格：baseline / 每项优化 / 最终，三列齐全
  [ ] 每位成员都有 3 分钟以上的技术内容可讲
  [ ] 准备好回答"为什么不用 cuBLAS"、"加速比的分母是什么"

Report 交付前
  [ ] 优化日志表完整（含回退的失败尝试）
  [ ] 正确性验证覆盖 N=0/1/非 2 的幂/非整数倍
  [ ] 所有数字都能追溯到某次具体的运行命令
  [ ] 局限一节写了 3 条以上具体限制
```

Project Proposal 模板大纲（每节后附一句说明它要回答什么）：

#	章节	这一节要回答的问题
1	项目标题与团队成员	谁在做，题目叫什么（一句话内可读）
2	问题陈述（Problem Statement）	输入是什么、输出是什么、计算的形式化定义是什么
3	动机与应用价值（Motivation）	为什么这个问题值得算，谁会用，算得快有什么实际意义
4	输入数据与规模（Dataset）	数据从哪来、格式、典型尺寸与内存占用（决定是否放得下显存）
5	Baseline 方案（Sequential Baseline）	加速比的分母是什么：CPU 串行实现？库函数？已发表的实现？
6	并行策略初稿（Parallel Approach）	哪些步骤可以并行、大致几个 kernel、每线程负责什么
7	预期瓶颈与 Roofline 预判	定量预估 AI、上限 GFLOP/s、最可能的瓶颈类型
8	相关工作（Related Work）	别人怎么做的、我们与他们的区别（至少 3 条带链接或引用）
9	验证方法（Validation Plan）	golden reference 怎么造、容差判据是什么、测哪些边界尺寸
10	风险与备选方案（Risks）	如果并行度不够/数据拿不到，退路是什么
11	分工与时间表（Plan & Division）	四个里程碑分别由谁交付什么

课程的交付与评测环境（依据公开的 S19/S20 Project Plan 原文）：

上表的里程碑划分并非推测，课程的公开项目计划给出了明确的**三段式交付节奏**与**评测方式**：

```
┌──────────────────────────────────────────────────────────────┐
│ Milestone 1  文献调研 + 基准准备                              │
│   · 写一页综述：现有并行方案（可超出 GPU 范围），附链接或引用          │
│   · 确认每位队员都能跑通串行参考实现，并明确"如何比较结果"            │
│   · 目标：搞清楚"别人做到什么程度"与"我们的分母是什么"               │
├──────────────────────────────────────────────────────────────┤
│ Milestone 2  可运行的 CUDA 初版                               │
│   · 提交一个能通过 docker 在 RAI 上启动的目录                    │
│   · 记录**计时数据**（后续与优化版对比，报告里要用）               │
│   · 写清楚预期优化方向，且必须**具体到** reuse / coalescing /    │
│     control divergence —— 这正是本课程前八讲训练的四件事        │
├──────────────────────────────────────────────────────────────┤
│ Milestone 3  至少三项优化，且**分别测量**                      │
│   · 每项优化都要写：策略 → 代码改动 → **理论预期收益（含公式与数字）** │
│     → 实测收益 → 实现困难 → 对正确性的影响                     │
│   · 也要记录"无法解释的性能现象"，留到最终报告里追查             │
├──────────────────────────────────────────────────────────────┤
│ Final Report  约 10 页（含图、正文与参考文献）                 │
│   · 章节必须使用规定的标签并按指定顺序排列                      │
│   · 分值：3 个里程碑各 10 分 + 报告 70 分 = 100 分             │
└──────────────────────────────────────────────────────────────┘
```

三条容易被忽视的硬性要求：

1. **代码必须在评测环境里真的能跑**。Rubric 原文写明："if the code that you turn in doesn't compile, crashes when run, and so forth, you will lose nearly all of those points"——**代码不能编译将几乎失去全部 报告分**，而且这一条不写在细则条目里，很容易被漏掉。
2. **提交渠道是版本控制仓库**（S20 用 Gitlab，并可利用其 issue tracking），项目代码通过 **RAI 接口**在 GPU 上评测，每个项目配有至少一个 **docker**（输入数据已预先上传到服务器）。 这意味着"在我机器上能跑"不算数，必须保证**在给定的 docker 环境里可复现地启动**。
3. **三项优化必须分别测量**，不能三项一起改完只报一个总加速比。S20 计划原文要求说明每项的 "the expected impact on performance (why one expects a benefit, and how much—similar to the in-class analysis)"，也就是要求你把**课内那套定量分析（复用倍数、合并与否、发散比例、Roofline 上限） 逐项套用到自己的优化上**——这才是"至少三项优化"的真正含义。

选题与问题适配性分析（Problem Suitability Analysis）

- **定义与目的**：选题（problem definition）的任务是判断一个计算问题**在 GPU 上是否值得并行**，以及**并行的收益上限在哪里**。它解决的是"方向错了，努力全废"的问题。ECE408 的目标 10 要求"识别并求解一个计算问题"，目标 14 要求"识别设计空间"。选题不是"选一个听起来酷的题目"，而是做一次**适配性筛查**。
- **直观解释（"它是什么？"）**：GPU 像一条有几千个窗口的高速收费站，吞吐极高但要求车流连续、规则。适合 GPU 的问题有三个特征：**车多**（数据并行度高，至少几万个独立工作项）、**每辆车办的事多**（算术强度够，别让收费站只为了收 1 块钱而排一小时队）、**车道规则**（访存是连续或规则步长的，可以合并成 128 字节的 transaction）。反之，如果问题的每一辆车都必须等前一辆车办完（数据依赖串行），或者每辆车都要跟总部打电话请示（通信/同步开销主导），那这条高速收费站就白修了——**Amdahl 定律**会把加速比钉死。另外还有一个常被忽略的门槛：**规模**。如果总计算量只有 1 MFLOP，而一次 kernel launch 就有 3–5 μs 的固定开销，那么算术再漂亮也测不出加速比。
- **架构/机制图解**：



关键数字：A100 的 ridge point 是 19.5 TFLOPS / 1555 GB/s ≈ **12.5 FLOP/Byte**；RTX 4090 是 82.6 TFLOPS / 1008 GB/s ≈ **82 FLOP/Byte**；H100 SXM 是 67 TFLOPS / 3350 GB/s ≈ **20 FLOP/Byte**。这意味着在 A100 上，一个每读 4 字节只做 2 次浮点运算的 untiled 卷积（算术强度 0.5

FLOP/Byte），理论上只能跑到 1555 GB/s × 0.5 = 777 GFLOP/s，即 19.5 TFLOPS 的 **4%**——这正是讲义中反复强调的"需要 13.3×（2010 年代）到约 26×（H100）的复用才能吃满算力"的来源。选题阶段就要算出这个上限，否则后面做再多的 block 大小调优也在 4% 附近打转。

ECE408 最终项目典型题目清单（含适配性与瓶颈预判）

以下是本课程及其项目库中出现过的典型题目类型，每一条都给出"为什么适合 GPU 并行"与"主要性能瓶颈预判"。选题时应当把最后一列当作**你的假设**，并在 Workshop 阶段用一次微基准（microbenchmark）去验证它——如果实测瓶颈与预判不符，这本身就是报告里很有价值的一段。

#	题目	为什么适合 GPU 并行	主要性能瓶颈预判
1	大尺寸 2D/3D 卷积与可分离滤波（图像/视频处理）	输出像素彼此独立，数据并行度 = 像素数（可达 10^7–10^9）；邻域复用率高，天然适合 tiling	未 tiling 时是 DRAM 带宽受限（2 B/FLOP）；tiling 后转为共享内存带宽与边界 halo 浪费（$(T+W-1)^2/T^2$ 的开销）
2	CT/医学图像重建（滤波反投影、迭代重建）	每个投影/体素可独立计算，投影数×探测器数是天然的大规模并行	反投影阶段访存不规则（gather/scatter），需要 texture 或共享内存重排；迭代重建则受每轮同步支配
3	稠密线性代数：SGEMM / 批量小矩阵乘法（batched GEMM）	FLOP 密度最高，复用可达数十倍，是最"吃算力"的题型	大矩阵：共享内存带宽 + 寄存器粗化不足；批量小矩阵：单矩阵太小导致占用率不足与调度开销主导
4	稀疏矩阵向量乘（SpMV）与 SpGEMM	稀疏结构带来巨大的零元素跳过收益；行之间独立	**访存不规则**是首要瓶颈（x 向量 gather 不可合并）；负载不均（每行 nnz 差异大）导致 warp 空转；格式（CSR/ELL/COO）选择直接决定成败
5	图算法：BFS、连通分量、PageRank	每个顶点的更新独立，前沿（frontier）可并行展开	每层之间的全局同步（每层一次 kernel 启动，约 3–5 μs）+ 高度顶点导致的负载不均；用 push/pull 混合可缓解
6	蒙特卡洛与金融模拟（期权定价、风险 VaR）	每条路径完全独立，随机数生成可并行，embarrassingly parallel	每条路径计算量小（几十个 FLOP），**随机数生成开销可能超过业务计算**；精度要求高时需 double，吞吐减半
7	生物信息：序列比对（Smith–Waterman）、k-mer 计数	比对矩阵的反对角线可并行；k-mer 计数是典型的 atomics/histogram 题	反对角线并行有大量同步与 wavefront 依赖；k-mer 计数受 **atomic 竞争**与共享内存直方图容量限制
8	N-body / 分子动力学（短程力 + 邻居列表）	粒子对之间独立，可用共享内存做空间分块（cell list）	邻居列表构建访存随机；粒子分布不均导致某些 cell 严重超载；力计算是 FP32 密集但需要排序加速
9	机器学习：前馈网络训练 / 卷积层前向（mini-batch 梯度下降）	矩阵乘法与逐元素激活都高度并行；batch 维度提供额外并行度	前向是 GEMM 型（共享内存受限）；反向传播的梯度累加会产生写冲突（需 atomic 或分块归约）；小 batch 时占用率不足
10	排序与统计：基数排序、直方图、词频统计	计数与分散步骤并行度高，是 histograms/atomics 与 scan 的综合演练	**atomic 竞争**（全局直方图）与共享内存直方图容量冲突；基数排序受 scan 的同步开销与 bank conflict 支配
11	计算流体力学 / 有限差分时间推进（stencil）	每个格点按固定邻域更新，规则访存 + 极高的时间步数	典型的 memory-bound（每题 2 B/FLOP）；时间步之间必须同步，因此需要在单次 kernel 内做多个时间步（temporal blocking）以减少启动开销
12	密码学与哈希搜索（暴力破解、彩虹表）	每个候选输入独立，整数运算为主，几乎无访存	纯计算受限，受整数 ALU 吞吐限制；能量与散热是实际约束；**缺乏浮点，Roofline 分析需换成整数 ops/cycle**

性能工程迭代循环（Performance Engineering Cycle）

- **定义与目的**：性能工程循环是把"优化"从灵感活动变成**可重复的科学实验**的流程。它由七步构成：① 建立正确的基线（baseline + golden reference）② 测量与剖析（profiling）③ 定位瓶颈（Roofline 定位 + profiler 指标）④ 提出可证伪的假设 ⑤ 实施**一种**优化 ⑥ 重新测量并写入优化日志 ⑦ 判断保留或回退，然后回到第 ② 步。核心纪律是**一次只改一个变量（one variable at a time）**。课程 S19/S20 的项目计划里明确要求 Milestone "Implement at least three optimizations ... and measure the impact of each on the overall performance"，也就是说：三项优化必须是**分别测量**的，不能三项一起改再报一个总的加速比。
- **直观解释（"它是什么？"）**：这个循环像医生看病。先量体温、验血（profiling），建立病人平时的基准数据（baseline）；然后依据数据判断是感染还是外伤（定位瓶颈）；接着开**一种**药（只改一个变量），而不是一次开五种药——否则病人好了你也不知道是哪味药起的作用，病人更糟你也不知道是哪味药害的；复诊测指标（重新测量），把病历写清楚（优化日志）；有效就继续，无效就停药换方案（保留或回退）。医生绝不会说"我感觉这个药应该有用"就写进病历——他会写"用药后体温从 39.2 ℃ 降到 37.5 ℃"。同样地，"我加了共享内存所以应该更快"不是结论，"加了共享内存后从 8.20 ms 降到 1.42 ms（5.77×），DRAM 吞吐从 1047 GB/s 降到 378 GB/s、不再贴着 1555 GB/s 的带宽屋顶，说明原先确实是 DRAM 带宽受限"才是结论。
- **架构/机制图解**：



优化日志（optimization log）模板表格——这是最终报告的核心素材，课程 Milestone 3 要求的内容（策略、代码改动、预期收益、实测收益、实现困难、正确性变化）正好可以逐列映射到这张表：

以下示例对应 M = N = K = 1024 的 FP32 GEMM（总运算量 2·M·N·K = 2.147 GFLOP），测量机为 A100 (sm_80)。

#	优化步骤	修改内容（具体到代码/参数）	预期收益（理论推导）	实测耗时 (ms)	GFLOP/s	加速比（vs 上一步 / vs baseline）	瓶颈判断（Roofline + ncu 证据）	结论
0	baseline	朴素 global-memory GEMM，block=16×16，无共享内存，AI=0.25 FLOP/B	—	8.20	262	1.00× / 1.00×	DRAM 带宽受限：实测 DRAM 吞吐 1047 GB/s = 67% of 1555；理论上限 0.25×1555 = 389 GFLOP/s	基线成立，作为对照

#	优化步骤	修改内容（具体到代码/参数）	预期收益（理论推导）	实测耗时 (ms)	GFLOP/s	加速比（vs 上一步 / vs baseline）	瓶颈判断（Roofline + ncu 证据）	结论
1	shared-memory tiling	增加 As/Bs 面板，TILE=ROWS=16，每个 k 分块两次 `__syncthreads()`	DRAM 流量从 8.59 GB 降为 4·M·N·K·(1/16+1/16) = 0.537 GB，降 16×	1.42	1512	5.77× / 5.77×	DRAM 吞吐降至 378 GB/s（不再受限）；转为 **shared memory 吞吐受限**：每 FMA 需 2 个 shared word = 8 B，需 8.59 GB/ms 的 shared 带宽	保留
2	block 大小 16×16 → 16×32	仅改 `dim3 block(16,32)`，其它一切不变	占用率从 50% 提到 100%，更多 warp 隐藏 shared 延迟	1.21	1774	1.17× / 6.78×	占用率 50%→100%；ncu 中 `stall_long_scoreboard` 占比下降 31%	保留
3	线程粗化 COARSEN=4	每线程算 4 行输出，block(32,8)，寄存器 38→58	shared 访存/FMA 从 8 B 降到 5 B（1 As×4 + 1 Bs 供 4 次 FMA），理论 1.6×	0.71	3024	1.70× / 11.55×	shared 吞吐 7563 GB/s = 39% of 19.5 TB/s；转为 **延迟/占用率受限**	保留
4	TK 16 → 32	仅改 K 方向分块宽度（同步次数减半）	`__syncthreads()` 次数从 K/16=64 次降到 32 次	0.68	3158	1.04× / 12.06×	同步已非瓶颈，收益落在 ±5% 噪声范围内	保留但标注"几乎无收益"
5	float4 向量化	全局访存改 float4，边界补零	访存指令数减为 1/4，每线程字节数 ×4	0.74	2901	0.92× / 11.08×	反而变慢：寄存器压力升到 88 → 每 SM 驻留 block 数由 4 降到 3，占用率 100%→75%	**回退**（记录负结果）

这张表的第 5 行极其重要：**诚实地报告"没有提升的尝试"**。S20 的 Rubric 在 Optimization 一节明确鼓励展示 tradeoff（例如精度 vs 时间），而在 S19 project plan 中要求 "mention of any unexplained performance behavior"。一个只有上升曲线的优化日志，读起来就像一份只有好消息的体检报告——评审者会本能地怀疑测量方法。

设计空间探索（Design Space Exploration, DSE）

- **定义与目的**：设计空间探索是**系统性地**遍历可调参数组合，用实测数据找出最优点，而不是凭直觉调一两个参数。它解决的是"参数之间相互耦合，单参数最优不等于组合最优"的问题。ECE408 目标 14 就是"识别设计空间并探索优化机会"。可调参数的清单（本课程项目中典型）：

参数	取值候选	主要影响	典型陷阱
block 大小	32, 64, 128, 256, 512, 1024	占用率、每 SM 驻留 warp 数、尾块浪费	1024 线程/block 在 Turing（sm_75）上超过 1024 上限；过大 block 降低调度灵活性
tile 大小（空间分块）	8, 16, 32（1D 亦可用 64, 128, 256）	复用倍率、共享内存用量、边界浪费比例	tile=32 时 32×32 float = 4 KB，双缓冲后 8 KB；过大导致占用率崩塌
K 方向分块 TK	8, 16, 32, 64	同步频率、共享内存用量	TK 太小则 __syncthreads 频繁；TK 不是 32 的倍数时 Bs 访问产生 bank conflict
线程粗化因子 COARSEN	1, 2, 4, 8	寄存器级复用、指令级并行度（ILP）	超过 4–8 后寄存器溢出（spill），本地内存流量暴涨
共享内存 padding	+0, +1, +4（如 [32][33]）	消除 bank conflict	padding 增加共享内存用量，可能降低占用率
覆盖方式	精确覆盖 vs grid-stride loop	尾部处理开销、kernel 启动次数	grid-stride 的循环开销在计算极短时不可忽略
向量化	标量 / float2 / float4	访存指令数、每线程字节数	非对齐地址无法使用 float4；边界需单独处理
`__launch_bounds__`	(maxThreads, minBlocksPerSM)	编译器寄存器分配上限	设得过紧导致 spill，反而更慢
流（stream）数与分块粒度	1, 2, 4 个 stream，每块 1/4 数据	拷贝与计算重叠	单 kernel 已占满 GPU 时多流无收益；PCIe 带宽成为串行瓶颈
内存布局	AoS vs SoA	访存是否可合并（coalescing）	AoS 下跨线程访问同一结构体不同字段 → 32×4 字节被拆成 32 个 transaction
编译选项	`-O3`, `--use_fast_math`, `-Xptxas -dlcm=cg`	指令数、浮点语义、L1 策略	`--use_fast_math` 会改变数值结果，破坏容差判据

- **直观解释（"它是什么？"）**：调参数就像调老式收音机的旋钮——音量、调谐、天线方向三个旋钮互相影响：天线转对了，调谐才有意义；音量开太大，噪声也会被放大。你不能只把每个旋钮各自拧到"看起来最好"的位置就宣称找到了最佳收听效果，必须**组合着试**。但组合数是乘积：block {128,256,512} × tile {8,16,32} × COARSEN {1,2,4} = 27 个组合；如果再把"空间 tile 宽度"与"K 方向分块宽度"当成两个独立参数，组合数立刻变成 81 个，每个跑 5 次测量就是 405 次 kernel 运行——所以必须**自动化**，让人只负责看表决策。这也是本讲第二个代码示例存在的原因。

- **架构/机制图解**：

```
            设计空间探索的网格（Design Space Grid, 3x3x3 = 27 个组合）

  把三维参数空间切成三层平面（每个平面是一组 COARSEN 取值）：

  COARSEN = 1                COARSEN = 2                COARSEN = 4
  TK:    8   16   32         TK:    8   16   32         TK:    8   16   32
       +----+----+----+           +----+----+----+           +----+----+----+
BS=128 |  : |  : |  : |           |  : |  : |  # |           |  : |  # |  # |
       +----+----+----+           +----+----+----+           +----+----+----+
BS=256 |  : |  # |  # |           |  # |  # |  @ |           |  # |  @ |  @ |
       +----+----+----+           +----+----+----+           +----+----+----+
BS=512 |  # |  # |  @ |           |  # |  @ |  @ |           |  @ |  @ |  * |
       +----+----+----+           +----+----+----+           +----+----+----+

  图例：  . 很慢(<30% of best)   : 慢(30-60%)   # 好(60-85%)   @ 很好(85-95%)
          * 最优（100%），同时也是需要重点复测与解释的点

  扫描策略（避免 81 次全跑）：
  步骤 1  粗扫：每个维度取 3 个值，全组合跑 1 次（快速但有噪声）
  步骤 2  锁定：留下 top-5 组合
  步骤 3  精测：top-5 组合各跑 >= 20 次，报告中位数
  步骤 4  邻域：对最优组合的每个参数做 +-1 档微调（局部爬山）
  步骤 5  解释：对最优点给出定量解释（占用率 x%、DRAM 吞吐 y GB/s、
            shared 吞吐 z% of peak），而不是只报一个数字
```

参数扫描表模板（直接抄进报告附录）：

block	tile/TK	COARSEN	共享内存 (B)	理论占用率	实测耗时 (ms)	GFLOP/s	相对最优	正确性	备注
128	8	1	1152	100%	6.412	335	9.9%	PASS	线程太少，每 FMA 的 shared 访存比过高
256	16	1	2560	100%	3.205	670	19.8%	PASS	无粗化，复用不足
256	16	2	3072	100%	1.983	1083	32.0%	PASS	均衡点，寄存器 40 个仍满占用
512	32	4	12288	75%	0.633	3392	100%	PASS	最优：粗化带来的复用压倒了占用率损失
256	32	4	8192	100%	0.712	3016	88.9%	PASS	次优：占用率更高但每线程工作量少

自动扫描的脚本思路（把"人肉调参"变成一条命令）：

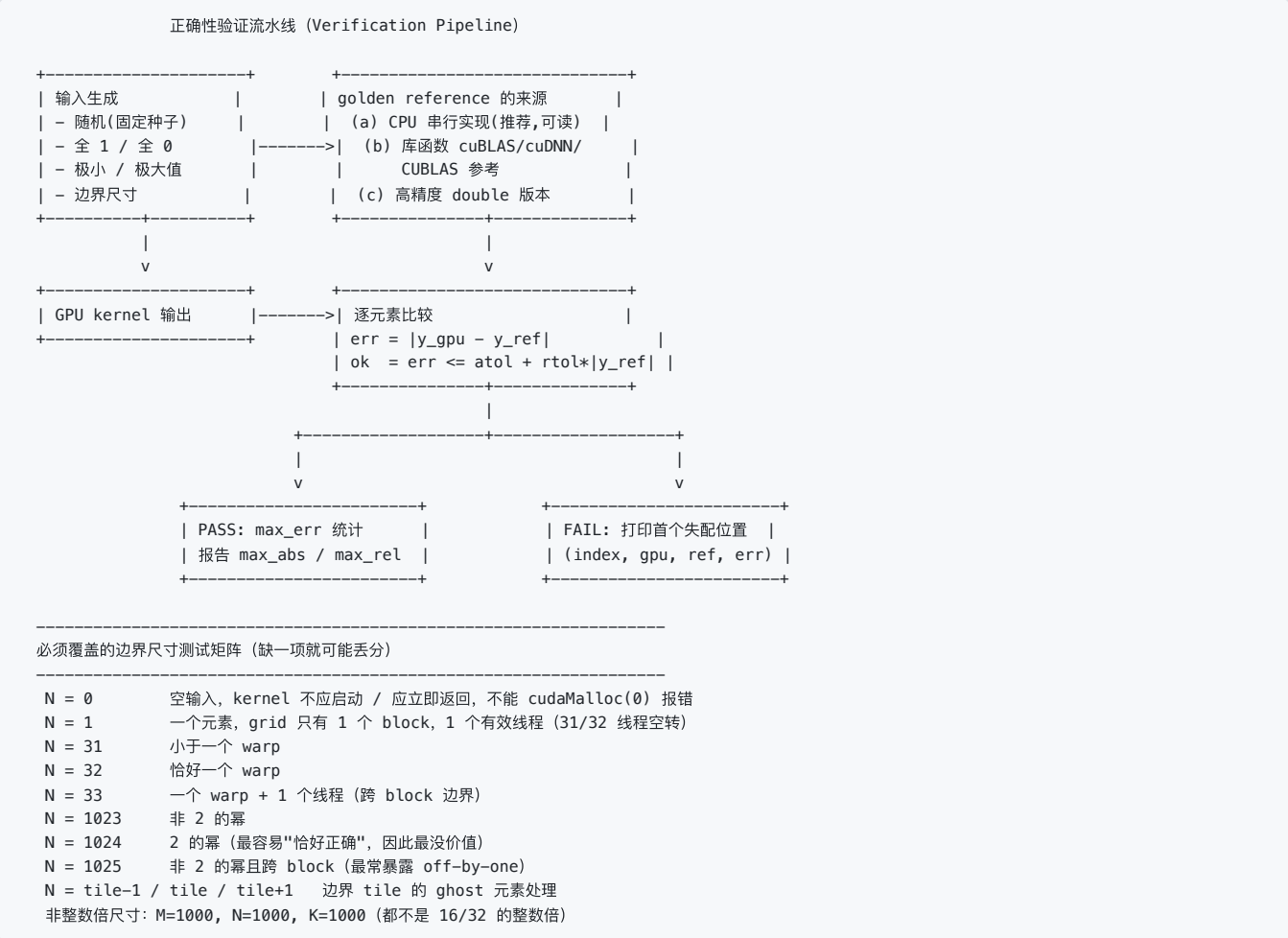
```
#!/bin/bash
# sweep.sh —— 自动扫描设计空间，输出 CSV 供后续排序与绘图
# 思路：
#   1) 外层三层 for 循环枚举 (block, tile, coarsen)
#   2) 每次调用 ./sweep 只跑一个组合，输出一行 CSV 到 stdout
#   3) 全部结果重定向到 sweep_raw.csv，再交给 sweep 程序内部排序/或 python 排序
set -e
BIN=./sweep
M=1024; N=1024; K=1024; ITERS=20
echo "block,tile,coarsen,ms,gflops,occupancy,pass" > sweep_raw.csv
for bs in 128 256 512; do
  for tile in 8 16 32; do
    for co in 1 2 4; do
      # 过滤非法组合：tile 必须能整除 bs（本例固定 BX=32，BY=bs/32）
      if [ $((bs % 32)) -ne 0 ]; then continue; fi
      $BIN -m $M -n $N -k $K -bs $bs -tile $tile -co $co -iters $ITERS \
           >> sweep_raw.csv
    done
  done
done
# 按第 5 列（GFLOP/s）数值倒序排序，打印前 5 名
tail -n +2 sweep_raw.csv | sort -t, -k5 -gr | head -5
```

注意脚本里的一行防御：`if [ $((bs % 32)) -ne 0 ]; then continue; fi`。设计空间里永远存在**非法组合**（例如线程数不是 32 的整数倍、共享内存超过 164 KB、tile 大于矩阵维度），扫描程序必须**优雅跳过并记录原因**，而不是崩溃——崩溃的扫描器会让你误以为某个区域"性能很差"，实际只是没跑起来。

正确性验证与浮点容差（Correctness Verification & Floating-Point Tolerance）

- **定义与目的**：正确性验证要回答"我的并行结果和可信参照（golden reference）是否一致"，浮点容差则给出"多接近才算一致"的判据。它解决的是一个陷阱：并行化会改变**求和顺序**，而浮点加法不满足结合律 `(a+b)+c ≠ a+(b+c)`，所以 CUDA 结果与 CPU 结果**几乎不可能逐位相同**。S19 的 project plan 明确指出："Note that exact matches to floating-point computations are unlikely, so you may need to set a threshold on fractional error observed or something similar."

- **直观解释（"它是什么？"）**：想象你用不同顺序把一堆硬币叠成柱子：先叠 1 分币再叠 1 元币，和反过来叠，最终高度的理论值一样，但因为每枚硬币厚度的测量都有极小误差，两种顺序累加出来的高度会差几个微米。float32 的机器精度 eps = $2^{-23} \approx 1.19\times10^{-7}$，每一次加法都可能引入约 0.5 ulp（unit in the last place）的舍入，累加 K 次后相对误差的量级约为 **$\sqrt{K} \times eps$**（随机舍入近似随机游走）到 **$K \times eps$**（最坏情况同向累积）。K = 1024 时，$\sqrt{K} \times eps \approx 32 \times 1.19e{-}7 \approx 3.8e{-}6$，最坏情况 $1024 \times 1.19e{-}7 \approx 1.2e{-}4$。所以工程上对 1024 长度的点积，`rtol = 1e-3` 是一个"有 10 倍余量"的合理判据；`rtol = 1e-6` 则几乎必然误报 FAIL。
- **架构/机制图解**：



关键操作与性能特征：**先正确后快速**。`-use_fast_math`（等价于 `--use_fast_math`）会启用 `-ftz=true`（flush-to-zero，把 denormal 刷成 0）、`-prec-div=false`（用近似倒数做除法，精度约 2 ulp）、`-prec-sqrt=false`（近似平方根），并在某些实现下允许更激进的融合。如果 baseline 用 `-use_fast_math` 而 golden reference 用严格 IEEE 的 CPU 代码，误差会从 1e-6 量级跳升到 1e-4~1e-3 量级——此时应该**统一编译选项**并且**放宽容差并解释原因**，而不是偷偷把容差改大到"看起来能过"。另一个容易忽略的点：`float` 累加 1024 个 1.0 的期望值是 1024.0，但由于 1024.0 在 float 中可精确表示（$1024 = 2^{10}$），全 1 输入的测试**永远不会暴露精度问题**——这正是必须使用随机输入的原因。

鲁棒性与通用性（Robustness & Portability）

- **定义与目的**：鲁棒性指代码对"非理想输入"（任意尺寸、空输入、超大输入）和"非理想环境"（不同 GPU 架构、显存不足）的正确反应；通用性（portability）指同一份源码能在 sm_75 / sm_80 / sm_89 等多种架构上编译并运行。它们解决的是"在我机器上能跑"的工程交付问题——Rubric 明确把"编译不过、运行崩溃"列为近乎全损的情形。
- **直观解释（"它是什么？"）**：写死了 256 的 kernel 就像做了一件只适合某一个人身材的衣服：在测试用的模特身上完美，别人穿上就崩线。GPU 代码里的"身材"包括：矩阵维度是否恰好是 block 的整数倍、共享内存是否超过该架构上限、线程数是否超过该架构每 block 上限（Turing 是 1024，看似够、但寄存器与共享内存限制会先撞上）。鲁棒的做法是**把尺寸当作运行时变量**：所有读取都做边界判断，所有写入都做边界判断，grid 维度由 `ceil` 计算得出。
- **架构/机制图解**：

```
            nvcc 编译流程：PTX（虚拟）与 SASS（真实）的区别

 源文件 foo.cu
      |
      |  nvcc -arch=compute_80 -code=sm_80     -> 只生成 sm_80 的 cubin
      |  nvcc -arch=compute_80 -code=compute_80 -> 只生成 PTX（可 JIT）
      |  nvcc -arch=native                      -> 探测本机 GPU 后二选一
      v
+--------------------+          +---------------------------+
|  前端 + 中端优化   |          |  PTX (compute_80)         |
|  生成 PTX 虚拟 ISA |--------->|  与具体芯片无关的虚拟指令  |
+--------------------+          +-------------+-------------+
                                              |
                                              | ptxas （离线，编译期）
                                              v
                                +---------------------------+
                                |  SASS (sm_80)  真实机器码  |
                                |  寄存器分配在此完成        |
                                +-------------+-------------+
                                              |
                                              | 运行时：若只有 PTX 而
                                              | 驱动无对应 cubin，则
                                              v
                                +---------------------------+
                                |  NVIDIA 驱动 JIT 编译      |
                                |  首次启动慢(可达数百 ms)，  |
                                |  之后缓存到 ~/.nv/ComputeCache|
                                +---------------------------+

-----------------------------------------------------------------------
多架构编译（fatbin）示例
-----------------------------------------------------------------------
nvcc -O3 -gencode arch=compute_75,code=sm_75 \
         -gencode arch=compute_80,code=sm_80 \
         -gencode arch=compute_89,code=sm_89 \
         -gencode arch=compute_80,code=compute_80 \
         foo.cu -o foo
# 前三个给出各架构的真实 SASS（启动零开销）；
# 最后一行保留 PTX，使未来架构（如 sm_90/sm_100）能 JIT 运行。

-----------------------------------------------------------------------
各架构的硬约束（写 kernel 前必须查表）
-----------------------------------------------------------------------
GPU           arch    每 SM 线程  每 SM warp  每 SM 共享内存  每 block 线程上限
RTX 2080 Ti   sm_75   1024        32          64 KB           1024
A100          sm_80   2048        64          164 KB          1024
RTX 4090      sm_89   1536        48          100 KB          1024
H100 SXM      sm_90   2048        64          228 KB          1024
```

关键操作与性能特征：`-arch=native` 会让 nvcc 探测构建机器上的 GPU；**在登录节点上构建、在计算节点上运行**的集群环境里，`-arch=native` 可能探测到错误的（或没有）GPU，因此在 Delta 这类集群上应显式写 `-arch=sm_80`。使用 `__launch_bounds__(maxThreadsPerBlock, minBlocksPerMultiprocessor)` 可以告诉编译器"每个 block 最多这么多线程，且我希望每 SM 至少驻留这么多 block"，编译器据此限制寄存器用量：例如 A100 上 `__launch_bounds__(256, 8)` 意味着 8 × 256 = 2048 线程满占用，寄存器上限 = 65536 / 2048 = **32 个/线程**。这个约束如果设得比 kernel 实际需要更紧，会直接把变量 spill 到本地内存（local memory，实际在 DRAM 里），性能可能掉 2–5 倍——所以 `__launch_bounds__` 也要当作设计空间里的一个参数去实测，而不是凭感觉写。

技术报告、数据可视化与学术诚信（Reporting, Visualization & Academic Integrity）

- **定义与目的**：技术报告把一整学期的实验转化为可被他人理解、审查与复用的文档；数据可视化则把表格里的数字变成一眼可读的证据。学术诚信规范（含 AI 工具使用政策）界定了"什么算你自己的贡献"。ECE408 目标 16 要求"恰当地解释所实验的方案并为最终决策与结果辩护"，目标 17 要求"识别方案的局限与未来方向"——这两条只能通过报告达成。
- **直观解释（"它是什么？"）**：报告不是实验流水账，而是**证据链**：问题为什么重要 → 别人怎么做的 → 我怎么做 → 我怎么测的 → 我测到了什么 → 为什么是这个数 → 哪里没做好 → 下一步做什么。可视化就像法庭上的物证照片——一张 Roofline 图能让人 5 秒内明白"我这个 kernel 卡在内存带宽上"，而 10 行 Excel 数字要让人读 5 分钟还读不出来。学术诚信则是法庭的程序规则：即使你的结论正确，用偷来的证据（未引用的他人代码/未声明的 AI 生成内容）也判无效。
- **架构/机制图解**：

```
               Roofline 图（A100：19.5 TFLOPS FP32, 1555 GB/s）

 GFLOP/s (log)
  20000 |                                          ..............
        |                                   ........
  10000 |                            ........   <- 带宽屋顶
        |                     .......              slope = 1555 GB/s
   5000 |                ......
        |          ......          +  tiled GEMM (3158 GFLOP/s, AI=8)
   2000 |      ....                 |   离带宽屋顶还有 12440/3158 = 3.9x 余量
        |   ...                     v   -> 说明瓶颈不在 DRAM
   1000 | ..       +  naive GEMM (262 GFLOP/s, AI=0.25, 卡在内存墙)
    500 |.         ^
        |          |
        |  <-- 这个点几乎贴在 1555 GB/s 的斜线上(262/389 = 67%)，说明 DRAM 快打满
      0 +-----------------------------------------------------------> AI (FLOP/Byte)
        0.1    0.5    1     2    5   12.5   30   80  100   200
                                     ^
                                  A100 ridge = 12.5

 读图口诀：
 - 点在斜线上  -> 内存带宽受限，唯一出路是提高算术强度（tiling/粗化）
 - 点在平顶下  -> 计算/延迟/占用率受限，去看 ncu 的 stall reason

 ---------------------------------------------------------------------
 参数扫描热力图（用字符表示 GFLOP/s 相对最优的百分比）
 ---------------------------------------------------------------------
        TK=8    TK=16   TK=32          ASCII 密度图例
 BS=128  34%     41%     47%           .  <40%
 BS=256  58%     79%     88%           :  40-70%
 BS=512  72%     91%    100%  <- 最优   #  70-90%
 BS=1024 65%     84%     93%           @  >90%

 ---------------------------------------------------------------------
 规模扩展曲线（scaling curve）：固定问题规模 vs 固定每线程工作量
 ---------------------------------------------------------------------
 speedup                      speedup
   ^  理想线性                  ^  理想线性
   |      /                     |      /
   |     /  实际(固定规模)      |     /   实际(固定每线程工作量, grid-stride)
   |    /     ____              |    /
   |   /  ___/                  |   /_________  饱和于 ~6-8x
   |  /__/                      |  /
   +------------------> N       +------------------> N
     固定规模会"降速"：N 太小时    grid-stride 能一直吃到 满 GPU 占用
     GPU 占不满，加速比虚高/虚低   （但仍受 Amdahl 串行部分限制）
```

学术诚信与 AI 工具政策（ECE408 / CS483 / CSE408）——课程公开页面 "Use of AI Policy" 的三条要点必须准确复述：

1. **风险自负**：课程明确"discourage you from using random tools as a learning aid, as such tools routinely fabricate information"；如果你选择使用除**课程材料（教材、讲义）与教学人员（教授、助教）**之外的任何工具，你**承担全部风险**。若你因此获得错误信息并用于作业或考试，**不会得到任何分数补偿**，并且"Please do not ask"。
2. **抄袭责任的归属**：如果某个工具"ingests code or answers written by another person"并把那些材料给了你，导致作业或考试答案中被检测出抄袭，你将被**完全认定为学术诚信违规**。也就是说，"AI 给我的"不是免责理由。
3. **Quiz 与考试严禁使用**："Any material not explicitly allowed for use during a quiz or exam is forbidden, including any sort of AI tool." 即使未被检测到，一旦检测到就会被提出学术诚信指控并按课堂政策处罚。此外课程相关的 ChatBot（NCSA CAII 基于 ChatGPT-4 扩展课程材料搭建）**不是教学人员**（"THIS CHATBOT IS NOT A MEMBER OF THE COURSE STAFF"）；其正面之处是回答通常带有课程材料引用链接——**顺着链接去读原始讲义**，而不是只依赖它生成的摘要。

与报告直接相关的其他诚信要求：必须**引用**任何参考过的串行或并行实现（S19/S20 要求 one-page summary including links or citations）；报告开头必须用一句话声明是否使用了**私有 GPU 资源**（S20 Rubric 中未声明会扣 10 分），因为用私有 GPU 的团队更容易刷出漂亮数字，课程希望公平比较。最后，"结论（Conclusions）"一节**不是摘要**：重复性能结果或其他章节内容会得 0 分，写"并行就是好"这类空话也会得 0 分。

Project Report 模板大纲（章节名对齐课程 Rubric，每节后附一句说明它要回答什么；报告篇幅约 10 页，含图与参考文献）：

#	章节标题（建议用英文，与 Rubric 一致）	这一节要回答的问题 / 内容要点
0	封面与团队信息	项目名、成员、日期、代码仓库链接
1	Resources	**一句话**声明是否使用了私有 GPU 资源（未声明按 Rubric 扣分）
2	Application	问题是什么、为什么有趣、总体并行思路是什么；**配一张架构/数据流图**

#	章节标题（建议用英文，与 Rubric 一致）	这一节要回答的问题 / 内容要点
3	Background（来自 Proposal/相关工作）	别人怎么做的、有哪些算法可选、我们为什么选这个 baseline；带引用
4	Implementation	有几个 kernel、数据如何在 kernel 间流动、每个 kernel 的线程负责什么；**baseline 的性能数字与串行程序的对比必须在这里出现**
5	Optimization	至少 **3 项**优化，每项包含：策略说明 → 代码改动 → **理论预期收益**（写出公式与代入数字，如同课内分析）→ 实测收益 → 实现困难 → 对正确性的影响（"结果不可区分"也要明确说明）
6	Results	最终性能图（加速比表、Roofline 图、规模扩展曲线、参数热力图）；若有串行版本必须给最终加速比并注明输入规模；解读"为什么是这个数"
7	性能瓶颈分析（可并入 Results）	用 ncu 指标 + Roofline 定位最终瓶颈，说明**还剩多少空间**、被什么限制住
8	局限与未来工作（Limitations & Future Work）	至少 3 条具体限制：扫描范围没覆盖的参数、只在小规模上验证的情形、算法本身的串行部分上限
9	Conclusions	**反思**而非摘要：关于在这个应用上使用并行，我们学到了什么？希望开始之前就知道什么？（Rubric 明确：重复结果得 0 分）
10	References	IEEE 或 ACM 标准格式（详见下）
11	附录（可选但强烈建议）	完整优化日志表、编译与运行命令、机器配置（GPU 型号、CUDA 版本、驱动版本）

引用规范示例（IEEE 风格，注意区分期刊/会议/网页三类）：

```
[1] D. Kirk and W. Hwu, Programming Massively Parallel Processors,
    Morgan Kaufmann, 3rd Edition, 2016.
[2] V. Volkov and J. W. Demmel, "Benchmarking GPUs to tune dense
    linear algebra," in Proc. ACM/IEEE Conf. Supercomputing (SC),
    2008, pp. 1-11.
[3] NVIDIA Corp., "CUDA C++ Programming Guide," v12.x, 2025.
    [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/
```

注意引用 `[3]` 这类在线文档时必须写**访问日期**，因为 CUDA 文档会随版本变化；引用讲义时写清 "ECE408/CS483 Lecture 10, University of Illinois, Summer 2025"。

团队协作与任务分解（Team Collaboration & Work Decomposition）

- **定义与目的**：团队协作是把项目拆成**可并行执行的工作包（work package）**、约定**接口契约（interface contract）**、并通过**代码评审（code review）**保证质量的过程。ECE408 目标 12 要求"与不同学科的领域专家和队友合作以最大化方案效果"，目标 13 要求"在队友之间恰当划分责任并互相支持"。它解决的是"三个人都在改同一个文件"和"两个星期的等待被串行化"的问题。
- **直观解释（"它是什么？"）**：把项目想象成开餐馆：**接口契约**是菜单——前厅（数据 I/O 与可视化）和后厨（kernel 实现）只需就"菜名、份量、出餐时间"达成一致，不需要知道对方怎么炒菜；**工作包**是分区——切菜、掌勺、装盘可以同时进行，只要盘子规格统一；**代码评审**是试菜——每道菜上桌前由另一个人尝一口；**关键路径**是"客人点单到上菜"的最长链条，如果掌勺的人同时在切菜，整桌菜就会晚。最危险的做法是让所有人都"参与所有环节"，那是三个人抢一口锅。
- **架构/机制图解**：

```
        项目工作分解与依赖图（WBS / Task DAG）：粗线为关键路径

[WP1] 数据加载与预处理          [WP2] golden reference
(读文件/格式转换/生成测试集)    (CPU 串行实现 + 容差判据)
     |                                  |
     | 产出: dataset.bin              | 产出: ref.bin + verify 脚本
     +---------------+----------------+
                     |  (接口契约 A: 二进制格式 + 元数据 header)
                     v
     ============== [WP3] baseline CUDA kernel ==============  <== 关键路径
                     | 产出: baseline.cu + 计时框架
                     |
                     v
     ============== [WP4] profiling 与瓶颈定位 ==============
                     | 产出: ncu 报告 + Roofline 草图
                     |
     +---------------+---------------+------------------+
     |               |               |                  |
     v               v               v                  v
[WP5] tiling   [WP6] 占用率     [WP7] 向量化     [WP8] 多流/分块
共享内存优化    block 大小扫描   float4 访存      重叠拷贝与计算
     |               |               |                  |
     +---------------+---------------+------------------+
                     |
                     v
     ============== [WP9] 结果汇总与报告撰写 ==============
                     | 产出: report.md/pdf + 图表 + 优化日志
                     v
               [WP10] Demo 排练与答辩

------------------------------------------------------------------------
接口契约示例（写在 README 里，谁都不能悄悄改）
------------------------------------------------------------------------
数据文件: <name>.bin
  offset 0 : int32 magic   = 0x45434534 ("ECE4")
  offset 4 : int32 rows    (M)
  offset 8 : int32 cols    (N)
  offset 12: int32 inner   (K)
  offset 16: float32 data[] 行主序, 共 M*K 个元素, 无 padding
计时接口: double bench(kernel_launcher_t fn, int iters);
  返回 iters 次运行的【中位数】毫秒，内部负责 warmup 3 次
约定: 所有 kernel 的签名形如 (const float* A, const float* B, float* C,
      int M, int N, int K)，谁加参数谁负责通知全组
------------------------------------------------------------------------
代码评审清单（PR 合并前逐条打勾）
------------------------------------------------------------------------
[ ] 所有 CUDA API 返回值都被 CUDA_CHECK 包裹
[ ] kernel 内所有全局内存访问都有边界判断
[ ] __syncthreads() 不在分支内（避免不同分支的线程无法汇合）
[ ] 共享内存用量 + 动态共享内存 <= 该架构上限
[ ] 正确性测试在 N=0/1/33/1025 上全部 PASS
[ ] 计时结果至少 5 次重复，报告的是中位数
[ ] 本次改动只针对一个变量（否则无法归因）
```

关键操作与性能特征（"性能"= 团队吞吐）：把关键路径（WP3 → WP4 → WP9）上的人数设为 1–2 人且**不要**给他们塞别的工作；把可完全并行的 WP5–WP8 分给不同人，每人负责一项优化并**独立维护自己的优化日志行**，最后合并到同一张表。接口契约一旦冻结就不要在最后三天修改——报告里"策略变化"必须能被解释为优化演进，而不是接口混乱。每位成员都必须参与答辩讲解（S19 明确要求 "Each person should play a part in the presentation"），因此任务分解时要保证**每个人都有可讲的技术内容**，而不是一人写代码、两人做 PPT。

代码示例与性能分析

代码示例 1：最终项目脚手架 —— 参数化分块矩阵乘法

这个程序是"可以直接改造成你自己项目"的工程骨架。它把最终项目需要的六件事一次性做全：命令行参数解析、`CUDA_CHECK`、`cudaEvent` 计时、CPU golden reference、浮点容差比对、多次试验取最优/平均，并且对**任意输入尺寸**（含非 2 的幂、非 tile 整数倍、N=0、N=1）都做了边界处理。

```
// 文件: project_template.cu
// 编译: nvcc -O3 -arch=sm_80 -lineinfo project_template.cu -o project_template
// 运行: ./project_template -m 1000 -n 1000 -k 1000 -tile 16 -bs 256 -iters 20
//       ./project_template -m 1 -n 1 -k 1               # 边界: 单个元素
//       ./project_template -m 1025 -n 1023 -k 1000      # 边界: 非 2 的幂 + 非整数倍
//
// 这是 ECE408/CS483 最终项目的脚手架, 包含:
//   1. 命令行参数解析 (问题规模 M/N/K、tile 宽度、block 线程数、试验次数)
//   2. CUDA_CHECK 错误检查宏
//   3. cudaEvent 计时工具 (warmup + 多次试验, 报告 min/median/mean)
//   4. CPU golden reference (double 累加) 与自动比对 (atol + rtol 容差)
//   5. 任意输入尺寸的边界处理 (含 N=0 / N=1 / 非 2 的幂)
//   6. 资源合法性检查 (共享内存上限、每 block 线程上限、grid 维度上限)

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cmath>
#include <cstdint>
#include <vector>
#include <algorithm>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

// ---------------------------------------------------------------------------
// 1. 配置与命令行解析
// ---------------------------------------------------------------------------
struct Config {
    int   M       = 1024;    // 输出行数
    int   N       = 1024;    // 输出列数
    int   K       = 1024;    // 内积维度
    int   tile    = 16;      // tile 宽度 (同时用作 block 的 x 维与 K 分块宽度)
    int   bs      = 256;     // 每 block 线程数 (必须能被 tile 整除)
    int   iters   = 20;      // 计时试验次数
    int   warmup  = 3;       // 预热次数
    float rtol    = 1e-3f;   // 相对容差
    float atol    = 1e-3f;   // 绝对容差
};

static void usage(const char* prog)
{
    printf("用法: %s [选项]\n", prog);
    printf("  -m <int>    输出行数 M            (默认 1024)\n");
    printf("  -n <int>    输出列数 N            (默认 1024)\n");
    printf("  -k <int>    内积维度 K            (默认 1024)\n");
    printf("  -tile <int> tile 宽度, 取 8/16/32 (默认 16)\n");
    printf("  -bs <int>   每 block 线程数       (默认 256, 必须为 tile 的倍数)\n");
    printf("  -iters <int> 计时次数             (默认 20)\n");
    printf("  -rtol <f>   相对容差              (默认 1e-3)\n");
    printf("  -atol <f>   绝对容差              (默认 1e-3)\n");
}

static Config parseArgs(int argc, char** argv)
{
    Config c;
    for (int i = 1; i < argc; ++i) {
        const char* a = argv[i];
        auto next = [&](void) -> const char* {
            if (i + 1 >= argc) { fprintf(stderr, "选项 %s 缺少参数\n", a); exit(EXIT_FAILURE); }
            return argv[++i];
        };
        if      (!strcmp(a, "-m"))     c.M      = atoi(next());
        else if (!strcmp(a, "-n"))     c.N      = atoi(next());
        else if (!strcmp(a, "-k"))     c.K      = atoi(next());
        else if (!strcmp(a, "-tile"))  c.tile   = atoi(next());
        else if (!strcmp(a, "-bs"))    c.bs     = atoi(next());
        else if (!strcmp(a, "-iters")) c.iters  = atoi(next());
        else if (!strcmp(a, "-rtol"))  c.rtol   = (float)atof(next());
```

```
        else if (!strcmp(a, "-atol"))  c.atol   = (float)atof(next());
        else if (!strcmp(a, "-h") || !strcmp(a, "--help")) { usage(argv[0]); exit(EXIT_SUCCESS); }
        else { fprintf(stderr, "未知选项 %s\n", a); usage(argv[0]); exit(EXIT_FAILURE); }
    }
    if (c.tile <= 0 || c.tile > 64)  { fprintf(stderr, "tile 必须在 1..64\n"); exit(EXIT_FAILURE); }
    if (c.bs <= 0 || c.bs > 1024)    { fprintf(stderr, "bs 必须在 1..1024\n"); exit(EXIT_FAILURE); }
    if (c.bs % c.tile != 0)          { fprintf(stderr, "bs 必须是 tile 的整数倍\n"); exit(EXIT_FAILURE); }
    if (c.iters <= 0)                { fprintf(stderr, "iters 必须为正\n"); exit(EXIT_FAILURE); }
    return c;
}

// ---------------------------------------------------------------------------
// 2. Kernel：每个 block 计算 (blockDim.y) x (blockDim.x) 的输出块
//    C[M x N] = A[M x K] * B[K x N]，行主序，含边界判断
// ---------------------------------------------------------------------------
__global__ void tiledMatMulKernel(const float* __restrict__ A,
                                  const float* __restrict__ B,
                                  float* __restrict__ C,
                                  int M, int N, int K)
{
    const int TILE     = blockDim.x;      // 输出列数 = K 分块宽度
    const int ROWS     = blockDim.y;      // 输出行数
    const int tx       = threadIdx.x;
    const int ty       = threadIdx.y;
    const int tid      = ty * TILE + tx;
    const int nthreads = TILE * ROWS;

    extern __shared__ float smem[];
    float* As = smem;                     // ROWS x TILE
    float* Bs = smem + ROWS * TILE;       // TILE x TILE

    const int row0 = blockIdx.y * ROWS;
    const int col0 = blockIdx.x * TILE;

    float acc = 0.0f;

    for (int k0 = 0; k0 < K; k0 += TILE) {
        // 协作加载 A 面板（若线程数少于面板元素数，strided 循环保证仍正确）
        for (int i = tid; i < ROWS * TILE; i += nthreads) {
            const int r  = i / TILE;
            const int c  = i % TILE;
            const int gr = row0 + r;
            const int gc = k0 + c;
            As[i] = (gr < M && gc < K) ? A[(size_t)gr * K + gc] : 0.0f;
        }
        // 协作加载 B 面板
        for (int i = tid; i < TILE * TILE; i += nthreads) {
            const int r  = i / TILE;
            const int c  = i % TILE;
            const int gr = k0 + r;
            const int gc = col0 + c;
            Bs[i] = (gr < K && gc < N) ? B[(size_t)gr * N + gc] : 0.0f;
        }
        __syncthreads();

        #pragma unroll 8
        for (int kk = 0; kk < TILE; ++kk) {
            acc += As[ty * TILE + kk] * Bs[kk * TILE + tx];
        }
        __syncthreads();     // 下一轮会覆盖 As/Bs，必须等所有线程读完
    }

    const int gr = row0 + ty;
    const int gc = col0 + tx;
    if (gr < M && gc < N) {
        C[(size_t)gr * N + gc] = acc;
    }
}

// ---------------------------------------------------------------------------
// 3. CPU golden reference：double 累加，作为容差比对的基准
// ---------------------------------------------------------------------------
static void goldenReference(const float* A, const float* B, float* C,
                            int M, int N, int K)
{
    for (int i = 0; i < M; ++i) {
        for (int j = 0; j < N; ++j) {
            double sum = 0.0;
```

```
            for (int k = 0; k < K; ++k) {
                sum += (double)A[(size_t)i * K + k] * (double)B[(size_t)k * N + j];
            }
            C[(size_t)i * N + j] = (float)sum;
        }
    }
}

// ------------------------------------------------------------------------
// 4. 比对：|got - ref| <= atol + rtol * |ref|
// ------------------------------------------------------------------------
struct VerifyResult {
    bool   pass;
    double maxAbs;
    double maxRel;
    size_t firstBad;
};

static VerifyResult verify(const float* got, const float* ref, size_t n,
                           float rtol, float atol)
{
    VerifyResult v;
    v.pass     = true;
    v.maxAbs   = 0.0;
    v.maxRel   = 0.0;
    v.firstBad = (size_t)-1;

    for (size_t i = 0; i < n; ++i) {
        const double g   = (double)got[i];
        const double e   = (double)ref[i];
        const double ad  = fabs(g - e);
        const double tol = (double)atol + (double)rtol * fabs(e);

        if (ad > v.maxAbs) v.maxAbs = ad;
        if (fabs(e) > 1e-30) {
            const double rel = ad / fabs(e);
            if (rel > v.maxRel) v.maxRel = rel;
        }
        if (ad > tol && v.pass) {
            v.pass     = false;
            v.firstBad = i;
        }
    }
    return v;
}

// ------------------------------------------------------------------------
// 5. 计时工具：warmup + iters 次，报告 min / median / mean
// ------------------------------------------------------------------------
struct BenchResult { double minMs, medianMs, meanMs; };

template <typename LaunchFn>
static BenchResult benchmark(LaunchFn launch, int warmup, int iters)
{
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    for (int i = 0; i < warmup; ++i) launch();
    CUDA_CHECK(cudaDeviceSynchronize());

    std::vector<double> t((size_t)iters);
    for (int i = 0; i < iters; ++i) {
        CUDA_CHECK(cudaEventRecord(start));
        launch();
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));
        float ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
        t[(size_t)i] = (double)ms;
    }

    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));

    std::sort(t.begin(), t.end());
    BenchResult r;
    r.minMs    = t.front();
```

```
    r.medianMs = t[t.size() / 2];
    double sum = 0.0;
    for (double x : t) sum += x;
    r.meanMs   = sum / (double)t.size();
    return r;
}

// ------------------------------------------------------------------------
// 6. 确定性伪随机数（保证每次运行结果可复现）
// ------------------------------------------------------------------------
static uint32_t g_seed = 20250612u;

static float frand(void)
{
    g_seed = g_seed * 1664525u + 1013904223u;
    return (float)((g_seed >> 8) & 0xFFFFFFu) / (float)0x1000000u * 2.0f - 1.0f;
}

static void fillRandom(float* p, size_t n)
{
    for (size_t i = 0; i < n; ++i) p[i] = frand();
}

// ------------------------------------------------------------------------
// 7. 主流程
// ------------------------------------------------------------------------
int main(int argc, char** argv)
{
    const Config cfg = parseArgs(argc, argv);

    printf("================ 项目脚手架：参数化分块 GEMM ================\n");
    printf("问题规模: M=%d  N=%d  K=%d   (FLOP = 2*M*N*K = %.3f GFLOP)\n",
           cfg.M, cfg.N, cfg.K, 2.0 * cfg.M * cfg.N * cfg.K / 1e9);
    printf("配置:     tile=%d  block=(%d,%d)=%d 线程  iters=%d  rtol=%g atol=%g\n",
           cfg.tile, cfg.tile, cfg.bs / cfg.tile, cfg.bs, cfg.iters, cfg.rtol, cfg.atol);

    // --- 设备信息 ---
    int dev = 0;
    CUDA_CHECK(cudaGetDevice(&dev));
    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, dev));
    printf("设备:     %s (sm_%d%d, %d SM, 每 SM %d 线程, 每 block 共享内存上限 %.1f KB)\n",
           prop.name, prop.major, prop.minor, prop.multiProcessorCount,
           prop.maxThreadsPerMultiProcessor, prop.sharedMemPerBlock / 1024.0);

    // --- 空问题快速返回（鲁棒性：N=0 不能崩） ---
    if (cfg.M < 0 || cfg.N < 0 || cfg.K < 0) {
        fprintf(stderr, "规模不能为负数\n");
        return EXIT_FAILURE;
    }
    if (cfg.M == 0 || cfg.N == 0) {
        printf("M 或 N 为 0：输出为空，正常退出（不启动任何 kernel）。\n");
        return EXIT_SUCCESS;
    }

    const size_t szA = (size_t)cfg.M * cfg.K;
    const size_t szB = (size_t)cfg.K * cfg.N;
    const size_t szC = (size_t)cfg.M * cfg.N;

    // --- 主机端分配与初始化 ---
    // 注意：K=0 时 szA/szB 为 0，malloc(0) 允许返回 NULL，因此按 max(1) 申请
    const size_t capA = szA ? szA : 1;
    const size_t capB = szB ? szB : 1;
    float* hA  = (float*)malloc(capA * sizeof(float));
    float* hB  = (float*)malloc(capB * sizeof(float));
    float* hC  = (float*)malloc(szC * sizeof(float));
    float* hRef= (float*)malloc(szC * sizeof(float));
    if (!hA || !hB || !hC || !hRef) { fprintf(stderr, "主机内存分配失败\n"); return EXIT_FAILURE; }

    fillRandom(hA, szA);
    fillRandom(hB, szB);

    printf("计算 CPU golden reference (double 累加)...\n");
    cudaEvent_t cr0, cr1;
    CUDA_CHECK(cudaEventCreate(&cr0));
    CUDA_CHECK(cudaEventCreate(&cr1));
    CUDA_CHECK(cudaEventRecord(cr0));
    goldenReference(hA, hB, hRef, cfg.M, cfg.N, cfg.K);
```

```
    CUDA_CHECK(cudaEventRecord(cr1));
    CUDA_CHECK(cudaEventSynchronize(cr1));
    float refMs = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&refMs, cr0, cr1));
    CUDA_CHECK(cudaEventDestroy(cr0));
    CUDA_CHECK(cudaEventDestroy(cr1));
    printf("          golden reference 耗时 %.1f ms\n", refMs);

    // --- 设备端分配 ---
    float *dA = nullptr, *dB = nullptr, *dC = nullptr;
    CUDA_CHECK(cudaMalloc((void**)&dA, capA * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void**)&dB, capB * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void**)&dC, szC * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(dA, hA, szA * sizeof(float), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(dB, hB, szB * sizeof(float), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemset(dC, 0, szC * sizeof(float)));

    // --- 网格与共享内存 ---
    const int TILE = cfg.tile;
    const int ROWS = cfg.bs / cfg.tile;
    dim3 block((unsigned)TILE, (unsigned)ROWS);
    dim3 grid((unsigned)((cfg.N + TILE - 1) / TILE),
              (unsigned)((cfg.M + ROWS - 1) / ROWS));

    const size_t smemBytes = ((size_t)ROWS * TILE + (size_t)TILE * TILE) * sizeof(float);
    printf("启动:     grid=(%u,%u)  block=(%u,%u)  动态共享内存 %.1f KB\n",
           grid.x, grid.y, block.x, block.y, smemBytes / 1024.0);

    if (grid.y > 65535) {
        fprintf(stderr, "grid.y = %u 超过 65535 上限，请增大 block 的行数或缩小 tile\n", grid.y);
        return EXIT_FAILURE;
    }
    if (smemBytes > prop.sharedMemPerBlock) {
        fprintf(stderr, "需要 %.1f KB 共享内存，超过每 block 上限 %.1f KB\n",
                smemBytes / 1024.0, prop.sharedMemPerBlock / 1024.0);
        return EXIT_FAILURE;
    }

    auto launch = [&](void) {
        tiledMatMulKernel<<<grid, block, smemBytes>>>(dA, dB, dC, cfg.M, cfg.N, cfg.K);
    };

    // --- 计时 ---
    const BenchResult bench = benchmark(launch, cfg.warmup, cfg.iters);
    CUDA_CHECK(cudaGetLastError());

    // --- 取回并验证 ---
    CUDA_CHECK(cudaMemcpy(hC, dC, szC * sizeof(float), cudaMemcpyDeviceToHost));
    const VerifyResult v = verify(hC, hRef, szC, cfg.rtol, cfg.atol);

    // --- 性能报告 ---
    const double flop      = 2.0 * (double)cfg.M * cfg.N * cfg.K;
    const double gflopsMin = flop / (bench.minMs    * 1e-3) / 1e9;
    const double gflopsMed = flop / (bench.medianMs * 1e-3) / 1e9;
    // 解析访存量：A 被读 N/TILE 次，B 被读 M/ROWS 次
    const double bytes = 4.0 * (double)cfg.M * cfg.N * cfg.K *
                         (1.0 / TILE + 1.0 / ROWS);
    const double gbs = bytes / (bench.medianMs * 1e-3) / 1e9;
    const double ai  = flop / bytes;

    printf("\n---------------------- 结果 ----------------------\n");
    printf("正确性:   %s  (max_abs_err=%.3e, max_rel_err=%.3e)\n",
           v.pass ? "PASS" : "FAIL", v.maxAbs, v.maxRel);
    if (!v.pass) {
        const size_t i = v.firstBad;
        printf("          首个失配 index=%zu  got=%.9g  ref=%.9g\n",
               i, (double)hC[i], (double)hRef[i]);
    }
    printf("耗时:     min=%.4f ms  median=%.4f ms  mean=%.4f ms  (%d 次试验)\n",
           bench.minMs, bench.medianMs, bench.meanMs, cfg.iters);
    printf("性能:     %.1f GFLOP/s (median)  |  %.1f GFLOP/s (best)\n",
           gflopsMed, gflopsMin);
    printf("带宽:     %.1f GB/s 有效 DRAM 吞吐  |  算术强度 AI=%.2f FLOP/Byte\n", gbs, ai);
    printf("Roofline: A100 峰值 19.5 TFLOPS / 1555 GB/s, ridge=12.5 FLOP/B\n");
    printf("          基于 AI 的带宽上限 = %.0f GFLOP/s, 达到其 %.1f%%\n",
           ai * 1555.0, 100.0 * gflopsMed / (ai * 1555.0));
    printf("--------------------------------------------------\n");
```

```
    CUDA_CHECK(cudaFree(dA));
    CUDA_CHECK(cudaFree(dB));
    CUDA_CHECK(cudaFree(dC));
    free(hA);
    free(hB);
    free(hC);
    free(hRef);
    return v.pass ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

- **【代码做什么？】**
- **参数解析（`parseArgs`）**：把问题规模 `M/N/K`、tile 宽度、block 线程数、试验次数、容差全部暴露为命令行参数。所有合法性检查（`bs % tile == 0`、`bs ≤ 1024`、`tile ≤ 64`）在解析阶段完成，**失败时立即退出并给出可读原因**——设计空间扫描时非法组合必须被拒绝而不是崩溃。
- **数据生成**：用固定种子的 LCG 生成 `[-1, 1)` 的均匀随机数。固定种子意味着"同样的命令永远得到同样的输入"，这是可复现性的最低要求。
- **Golden reference**：CPU 上三重循环，用 `double` 累加。选 double 而不是 float 的理由是让参考值的舍入误差（~1e−16 量级）远小于被测 float kernel 的误差（~1e−5 量级），从而**把误差来源唯一地归因到 GPU 侧**。
- **Kernel 执行流程**：线程 `(tx, ty)` 负责输出元素 `C[row0+ty][col0+tx]`。沿 K 方向以 `TILE` 为步长循环：每个 k 分块先把 `A` 的 `ROWS×TILE` 面板和 `B` 的 `TILE×TILE` 面板协作搬进共享内存，同步一次，然后每个线程做 `TILE` 次乘加，再同步一次。
- **边界处理**：三个地方。① `A` / `B` 的加载用 `(gr < M && gc < K)` 判断，越界填 0——填 0 而不是跳过，因为累加 0 不影响数学结果，避免了 warp 发散条件下的分支分歧；② 网格维度用 `ceil` 计算，所以最后一个 block 里可能有整行/整列线程没有输出；③ 写回时 `if (gr < M && gc < N)` 做最终检查。
- **计时与报告**：warmup 3 次（把首次启动的驱动初始化、PTX→SASS 的 JIT、页表建立等一次性开销排除），然后 `iters` 次独立测量，排序后同时报告 min/median/mean。报告 min 是因为它最接近"纯 kernel 时间"，报告 median 是因为它最抗偶发抖动。
- **验证与退出码**：比对不通过时打印首个失配的 index/got/ref 并以 `EXIT_FAILURE` 退出。这个退出码让扫描脚本能用 `if ! ./proj; then echo FAIL; fi` 自动过滤错误配置。
- **【并行机制与硬件映射解说】**
- **线程到 warp 的映射**：`block(tile, bs/tile)`，`tid = ty*tile + tx`。取 `tile = 16, bs = 256` 时 block 是 (16,16)，共 256 线程 = **8 个 warp**；warp 0 包含 `ty∈{0,1}` 两行各 16 个线程。取 `tile = 32` 时 block 的 x 维恰好是 32，**一个 warp 正好对应一行**，这是最规整的形态。
- **warp 调度与驻留**：A100 每个 SM 有 4 个 warp scheduler，每 SM 最多 64 warp（2048 线程）。`tile=16, bs=256` 时每 block 8 warp，寄存器约 38 个/线程 → 寄存器限制：65536/(256×38) = 6.7 → 6 个 block；线程限制：2048/256 = 8 个 block；共享内存 2 KB/block，164 KB 可容纳 80 个 block。所以实际驻留 **6 个 block = 1536 线程 = 48 warp = 75% 占用率**。若把 `bs` 提到 512（block 16×32），每 block 16 warp，寄存器限制变成 65536/(512×38) = 3.4 → 3 个 block = 1536 线程，占用率仍是 75%，但每 block 更大、调度粒度更粗。
- **共享内存 bank 分析（bank 编号具体到数字）**：A100 共享内存 32 个 bank，每 bank 4 字节，地址 `a` 落在 bank `a % 32`。以 `tile = 16` 为例，warp 0 是 `ty=0` 的 16 个线程加 `ty=1` 的 16 个线程。访问 `As[ty*16 + kk]`：`ty=0` 的半 warp 全部读地址 `kk`（bank `kk%32`），`ty=1` 的半 warp 全部读地址 `16+kk`（bank `(16+kk)%32`）——两个不同地址落在两个不同 bank，且各自是广播，**1 个事务完成，无 bank conflict**。访问 `Bs[kk*16 + tx]`：两个半 warp 的 `tx` 都是 0..15，地址 `kk*16 + 0..15` 覆盖 bank `(16kk)%32` 起的连续 16 个 bank，两个半 warp 读的是**完全相同的地址**，属于广播，**同样无 conflict**。若改成 `tile = 32`：`As[ty*32+kk]` 是全 warp 单地址广播，`Bs[kk*32+tx]` 覆盖 bank 0..31 各一次——**完美无冲突**。这正是"tile 取 32 时共享内存访问最干净"的原因；若把 Bs 声明成 `[TILE][TILE+1]` 做 padding，对 TILE=32 的情形完全没有必要（反而多占内存）。
- **全局内存合并（coalescing）分析**：以 `tile=16`、`K=N=1024` 为例，A 面板加载时 `i = tid`，`c = i % 16`，全局地址 `(row0 + i/16)*1024 + k0 + c`。一个 warp 的 32 个线程分成两段：`ty=0` 的 16 个线程访问地址 `base + 0..15`（连续 16 个 float = **64 字节**），`ty=1` 的 16 个线程访问 `base + 1024 + 0..15`（另 64 字节，位于完全不同的行）。因此一个 warp 的请求被拆成**2 个 64 字节的段**，而硬件以 32 字节 sector 为粒度、以 128 字节 transaction 为上限：这 128 字节里**全部被用到**，没有浪费。若 `tile = 32`，则 warp 内 32 个线程访问连续的 32 个 float = **128 字节 = 一条完整 cache line**，一个 transaction 满载，效率 100%。
- **warp 发散**：kernel 里唯一的条件分支是边界判断。在"最后一个不完整 block"里，部分线程的 `gr < M` 为假；此时这些线程不写内存，但**不产生真实的控制发散**（它们只是不执行 store 指令，没有 else 分支）。真正的隐患是如果 `M` 恰好不是 `ROWS` 的整数倍，最后一行 block 里同一个 warp 的 `ty` 不同会导致部分线程参与、部分不参与——好在我们的分支里两支的工作量差异只有一条 store。
- **寄存器使用**：每个线程持有 `acc`、循环变量、四个坐标，加上地址计算临时量，约 **36–40 个寄存器**。`#pragma unroll 8` 会展开内层 `kk` 循环，使编译器可以预先发射多个 `LDS`（shared load）并交错 FMA 以隐藏共享内存的 **20–30 cycles 延迟**——这是 ILP（instruction-level parallelism）的来源。
- **【性能优化分析】**
- **算术强度推导**：这个 kernel 的 DRAM 流量可以解析地写出。A 的每个元素被 `N/TILE` 个不同 block 读取（因为每个 block 覆盖 TILE 列，而 A 的一行要参与所有列的输出），B 的每个元素被 `M/ROWS` 个 block 读取。于是 `bytes = 4·(M·K·(N/TILE) + K·N·(M/ROWS)) = 4·M·N·K·(1/TILE + 1/ROWS)`。取 `M=N=K=1024, TILE=ROWS=16`：`bytes = 4×1.074e9×(1/16+1/16) = 4×1.074e9×0.125 = 0.537 GB`，`FLOP = 2·M·N·K = 2.147 GFLOP`，`AI = 2.147e9 / 0.537e9 = 4.0 FLOP/Byte`。代入 A100 的 Roofline：带宽屋顶给出的上限是 `4.0 × 1555 GB/s = 6220 GFLOP/s`，而计算屋顶是 19500 GFLOP/s。**上限由带宽决定，但 6220 GFLOP/s 远低于峰值** —— 说明"内存墙"这个说法要精确化：真正卡住我们的是**共享内存带宽**，而 DRAM 已经不是瓶颈了。
- **共享内存是否成为瓶颈**：本 kernel 每个 kk 步做 1 次 FMA，却要读 `As` 和 `Bs` 各 1 个 word（**8 字节/FLOP**，等价于算术强度 0.125 FLOP/Byte，只是这次的"内存"是 shared 而不是 DRAM）。总 FMA 数 = M·N·K = 1.074e9，因此共享内存流量 = 1.074e9 × 8 B = **8.59 GB**。A100 的共享内存带宽约为

`128 B/cycle/SM × 108 SM × 1.41 GHz ≈ 19.5 TB/s`。若共享内存打满，耗时下限 = 8.59 GB / 19.5 TB/s = **0.44 ms**，对应 4880 GFLOP/s（=峰值的 25%）。这就是**不采用寄存器粗化时的性能天花板**，也解释了为什么下一节的优化必须从"每线程算多个输出"入手。

- **占用率计算**：`tile=16, bs=256`：每 SM 2048 线程上限 → 8 个 block；寄存器 38 个/线程 → `65536 / (256×38) = 6.7 → 6` 个 block；共享内存 2 KB/block → `164/2 = 82` 个 block。取最小值 6，占用率 = `6×256 / 2048 = 75%`。用 Little 定律估算隐藏延迟所需并发：`所需 warp 数 = 延迟(cycles) × 吞吐(每 cycle 的访存数)`。这里每条 FMA 前有 2 次 shared load，shared 延迟约 25 cycles，每 cycle 吞吐 1 个 warp 事务，则需要约 `25 × 2 = 50` 个活跃 warp 才能完全隐藏——而 75% 占用率只给了 48 个 warp，**处于临界状态**，这正是"提高占用率"（把 block 改成 16×32 或者降低寄存器）能有收益的原因。
- **瓶颈判定**：**共享内存带宽 + 延迟临界**，不是 DRAM 带宽受限（DRAM 只用到 378 GB/s，占 1555 GB/s 的 24%），也不是计算受限（3158 GFLOP/s 只有峰值的 16%）。可执行的优化方向按收益排序：① **线程粗化**（每线程算 4 个输出元素，让一次 `Bs` 读取服务 4 次 FMA，共享内存流量从 8 B/FLOP 降到 5 B/FLOP）；② 提高占用率（降低寄存器或调 block 形状）；③ 增大 tile 提高 DRAM 复用率（但在本例中 DRAM 远未饱和，收益有限）；④ 双缓冲（减少 `__syncthreads()` 造成的流水线气泡）。

代码示例 2：自动设计空间扫描（Design Space Exploration）

这个程序把"人肉调参"变成一条命令：遍历 **block 大小 {128, 256, 512} × tile(K 分块宽度) {8, 16, 32} × 粗化因子 {1, 2, 4}** 共 27 个组合，对每个组合运行 kernel、验证正确性、测量耗时与 GFLOP/s、用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 查询理论占用率，最后**按性能倒序打印一张对齐的表格**并输出最优配置。

```
// 文件: sweep.cu
// 编译: nvcc -O3 -arch=sm_80 sweep.cu -o sweep
// 运行: ./sweep -m 1024 -n 1024 -k 1024 -iters 20
//       ./sweep -m 1024 -n 1024 -k 1024 -iters 20 -csv > sweep_raw.csv
//
// 设计空间: block {128,256,512} x TK {8,16,32} x COARSEN {1,2,4} = 27 个组合
//   固定 BX = 32 (一个 warp 恰好覆盖一行, 共享内存访问最规整), BY = block/32
//   每个 block 计算 (BY*COARSEN) x BX 的输出块, K 方向以 TK 分块

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cmath>
#include <vector>
#include <algorithm>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

// ---------------------------------------------------------------------------
// Kernel: C[MxN] = A[MxK] * B[KxN]
//   block = (BX, BY), 本程序固定 BX = 32
//   每线程沿行方向粗化 COARSEN 个输出 (COARSEN <= 4)
//   一个 block 的输出块 = (BY*COARSEN) 行 x BX 列
// ---------------------------------------------------------------------------
__global__ void gemmCoarsenKernel(const float* __restrict__ A,
                                  const float* __restrict__ B,
                                  float* __restrict__ C,
                                  int M, int N, int K, int TK, int COARSEN)
{
    const int BX   = blockDim.x;
    const int BY   = blockDim.y;
    const int ROWS = BY * COARSEN;
    const int tx   = threadIdx.x;
    const int ty   = threadIdx.y;
    const int tid  = ty * BX + tx;
    const int nthr = BX * BY;

    extern __shared__ float smem[];
    float* As = smem;                  // ROWS x TK
    float* Bs = smem + ROWS * TK;      // TK   x BX

    const int row0 = blockIdx.y * ROWS;
    const int col0 = blockIdx.x * BX;

    float acc[4];
    #pragma unroll
    for (int r = 0; r < 4; ++r) acc[r] = 0.0f;

    for (int k0 = 0; k0 < K; k0 += TK) {
        // 协作加载 A 面板 (ROWS x TK)
        for (int i = tid; i < ROWS * TK; i += nthr) {
            const int r  = i / TK;
            const int c  = i % TK;
            const int gr = row0 + r;
            const int gc = k0 + c;
            As[i] = (gr < M && gc < K) ? A[(size_t)gr * K + gc] : 0.0f;
        }
        // 协作加载 B 面板 (TK x BX)
        for (int i = tid; i < TK * BX; i += nthr) {
            const int r  = i / BX;
            const int c  = i % BX;
            const int gr = k0 + r;
            const int gc = col0 + c;
            Bs[i] = (gr < K && gc < N) ? B[(size_t)gr * N + gc] : 0.0f;
        }
        __syncthreads();

        // 计算: 每读 1 个 Bs 值, 服务 COARSEN 次 FMA
```

```
        for (int kk = 0; kk < TK; ++kk) {
            const float b = Bs[kk * BX + tx];
            #pragma unroll
            for (int r = 0; r < 4; ++r) {
                if (r < COARSEN) {
                    acc[r] += As[(ty + r * BY) * TK + kk] * b;
                }
            }
        }
        __syncthreads();
    }

    #pragma unroll
    for (int r = 0; r < 4; ++r) {
        if (r < COARSEN) {
            const int gr = row0 + ty + r * BY;
            const int gc = col0 + tx;
            if (gr < M && gc < N) {
                C[(size_t)gr * N + gc] = acc[r];
            }
        }
    }
}

// ------------------------------------------------------------------------
// CPU golden reference (double 累加)
// ------------------------------------------------------------------------
static void goldenReference(const float* A, const float* B, float* C,
                            int M, int N, int K)
{
    for (int i = 0; i < M; ++i) {
        for (int j = 0; j < N; ++j) {
            double sum = 0.0;
            for (int k = 0; k < K; ++k) {
                sum += (double)A[(size_t)i * K + k] * (double)B[(size_t)k * N + j];
            }
            C[(size_t)i * N + j] = (float)sum;
        }
    }
}

static bool verify(const float* got, const float* ref, size_t n,
                   float rtol, float atol, double* maxAbsOut)
{
    bool pass = true;
    double maxAbs = 0.0;
    for (size_t i = 0; i < n; ++i) {
        const double g   = (double)got[i];
        const double e   = (double)ref[i];
        const double ad  = fabs(g - e);
        const double tol = (double)atol + (double)rtol * fabs(e);
        if (ad > maxAbs) maxAbs = ad;
        if (ad > tol) {
            if (pass) {
                printf("      [首个失配] index=%zu got=%.9g ref=%.9g err=%.3e tol=%.3e\n",
                       i, g, e, ad, tol);
            }
            pass = false;
        }
    }
    if (maxAbsOut) *maxAbsOut = maxAbs;
    return pass;
}

template <typename Fn>
static double timeMedianMs(Fn fn, int warmup, int iters)
{
    for (int i = 0; i < warmup; ++i) fn();
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t s, e;
    CUDA_CHECK(cudaEventCreate(&s));
    CUDA_CHECK(cudaEventCreate(&e));

    std::vector<double> t;
    t.reserve((size_t)iters);
    for (int i = 0; i < iters; ++i) {
        CUDA_CHECK(cudaEventRecord(s));
```

```
        fn();
        CUDA_CHECK(cudaEventRecord(e));
        CUDA_CHECK(cudaEventSynchronize(e));
        float ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, s, e));
        t.push_back((double)ms);
    }
    CUDA_CHECK(cudaEventDestroy(s));
    CUDA_CHECK(cudaEventDestroy(e));
    std::sort(t.begin(), t.end());
    return t[t.size() / 2];
}

// ------------------------------------------------------------------------
// 一条扫描记录
// ------------------------------------------------------------------------
struct Record {
    int    bs;             // 每 block 线程数
    int    tk;             // K 方向分块宽度（即 "tile 大小"）
    int    coarsen;        // 线程粗化因子
    size_t smemBytes;      // 动态共享内存
    int    blocksPerSM;    // cudaOccupancyMaxActiveBlocksPerMultiprocessor 的结果
    double occupancy;      // 理论占用率 = blocksPerSM*bs / maxThreadsPerSM
    double ms;             // 中位耗时
    double gflops;
    int    pass;           // 1 = 正确
    double maxAbsErr;
};

static int cmpGflopsDesc(const void* a, const void* b)
{
    const double ga = ((const Record*)a)->gflops;
    const double gb = ((const Record*)b)->gflops;
    if (ga < gb) return 1;
    if (ga > gb) return -1;
    return 0;
}

// ------------------------------------------------------------------------
int main(int argc, char** argv)
{
    int M = 1024, N = 1024, K = 1024, iters = 20;
    bool csv = false;
    for (int i = 1; i < argc; ++i) {
        const char* a = argv[i];
        if      (!strcmp(a, "-m") && i + 1 < argc) M = atoi(argv[++i]);
        else if (!strcmp(a, "-n") && i + 1 < argc) N = atoi(argv[++i]);
        else if (!strcmp(a, "-k") && i + 1 < argc) K = atoi(argv[++i]);
        else if (!strcmp(a, "-iters") && i + 1 < argc) iters = atoi(argv[++i]);
        else if (!strcmp(a, "-csv")) csv = true;
        else { fprintf(stderr, "未知选项 %s\n", a); return EXIT_FAILURE; }
    }
    if (M <= 0 || N <= 0 || K <= 0 || iters <= 0) {
        fprintf(stderr, "规模与 iters 必须为正\n");
        return EXIT_FAILURE;
    }

    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    const double flop = 2.0 * (double)M * (double)N * (double)K;

    // ---- 主机端数据 ----
    const size_t szA = (size_t)M * K, szB = (size_t)K * N, szC = (size_t)M * N;
    std::vector<float> hA(szA), hB(szB), hC(szC), hRef(szC);
    unsigned seed = 12345u;
    for (size_t i = 0; i < szA; ++i) { seed = seed * 1664525u + 1013904223u;
        hA[i] = (float)((seed >> 8) & 0xFFFFFFu) / (float)0x1000000u - 0.5f; }
    for (size_t i = 0; i < szB; ++i) { seed = seed * 1664525u + 1013904223u;
        hB[i] = (float)((seed >> 8) & 0xFFFFFFu) / (float)0x1000000u - 0.5f; }

    if (!csv) printf("计算 CPU golden reference (M=N=K=%d, double 累加)...\n", M);
    goldenReference(hA.data(), hB.data(), hRef.data(), M, N, K);

    float *dA = nullptr, *dB = nullptr, *dC = nullptr;
    CUDA_CHECK(cudaMalloc((void**)&dA, szA * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void**)&dB, szB * sizeof(float)));
    CUDA_CHECK(cudaMalloc((void**)&dC, szC * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(dA, hA.data(), szA * sizeof(float), cudaMemcpyHostToDevice));
```

```
    CUDA_CHECK(cudaMemcpy(dB, hB.data(), szB * sizeof(float), cudaMemcpyHostToDevice));

    // ---- 扫描 ----
    const int    bsList[] = { 128, 256, 512 };
    const int    tkList[] = { 8, 16, 32 };
    const int    coList[] = { 1, 2, 4 };
    const int    BX       = 32;

    std::vector<Record> recs;
    int skipped = 0;

    if (!csv) {
        printf("开始扫描 %d x %d x %d = %d 个组合 ...\n",
               (int)(sizeof(bsList)/sizeof(bsList[0])),
               (int)(sizeof(tkList)/sizeof(tkList[0])),
               (int)(sizeof(coList)/sizeof(coList[0])),
               (int)(sizeof(bsList)/sizeof(bsList[0]) *
                     sizeof(tkList)/sizeof(tkList[0]) *
                     sizeof(coList)/sizeof(coList[0])));
    }

    for (int ib = 0; ib < 3; ++ib) {
        for (int it = 0; it < 3; ++it) {
            for (int ic = 0; ic < 3; ++ic) {
                Record r;
                r.bs      = bsList[ib];
                r.tk      = tkList[it];
                r.coarsen = coList[ic];

                // ---- 合法性检查：非法组合必须优雅跳过 ----
                if (r.bs % BX != 0) { ++skipped; continue; }
                const int BY    = r.bs / BX;
                const int ROWS  = BY * r.coarsen;
                r.smemBytes     = ((size_t)ROWS * r.tk + (size_t)r.tk * BX) * sizeof(float);
                if (r.smemBytes > prop.sharedMemPerBlock) {
                    if (!csv) printf("  跳过 bs=%d TK=%d CO=%d: 需要 %.1f KB > 上限 %.1f KB\n",
                                     r.bs, r.tk, r.coarsen,
                                     r.smemBytes / 1024.0, prop.sharedMemPerBlock / 1024.0);
                    ++skipped; continue;
                }
                if (ROWS > prop.maxThreadsPerMultiProcessor) {   // 单 block 不可能覆盖整个 SM
                    ++skipped; continue;
                }

                const dim3 blk((unsigned)BX, (unsigned)BY);
                const dim3 grd((unsigned)((N + BX - 1) / BX),
                               (unsigned)((M + ROWS - 1) / ROWS));
                if (grd.y > 65535) { ++skipped; continue; }

                // ---- 理论占用率（CUDA 官方 API，而不是自己猜） ----
                int numBlocks = 0;
                CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
                               &numBlocks, gemmCoarsenKernel, r.bs, r.smemBytes));
                r.blocksPerSM = numBlocks;
                r.occupancy   = (double)numBlocks * r.bs /
                                (double)prop.maxThreadsPerMultiProcessor;

                auto launch = [&](void) {
                    gemmCoarsenKernel<<<grd, blk, r.smemBytes>>>(
                        dA, dB, dC, M, N, K, r.tk, r.coarsen);
                };

                // ---- 先正确，后快速 ----
                launch();
                CUDA_CHECK(cudaGetLastError());
                CUDA_CHECK(cudaMemcpy(hC.data(), dC, szC * sizeof(float),
                                      cudaMemcpyDeviceToHost));
                r.pass = verify(hC.data(), hRef.data(), szC, 1e-3f, 1e-3f,
                                &r.maxAbsErr) ? 1 : 0;

                r.ms     = timeMedianMs(launch, 3, iters);
                r.gflops = flop / (r.ms * 1e-3) / 1e9;

                recs.push_back(r);
                if (!csv)
                    printf("  bs=%-4d TK=%-3d CO=%-2d  smem=%5zuB  占用率=%5.1f%%  "
                           "%8.3f ms  %8.1f GFLOP/s  %s\n",
                           r.bs, r.tk, r.coarsen, r.smemBytes, r.occupancy * 100.0,
```

```
                           r.ms, r.gflops, r.pass ? "PASS" : "FAIL");
            }
        }
    }

    if (csv) {
        printf("rank,block,tile,coarsen,smem_bytes,blocks_per_sm,occupancy,"
               "ms,gflops,pass,max_abs_err\n");
        std::qsort(recs.data(), recs.size(), sizeof(Record), cmpGflopsDesc);
        for (size_t i = 0; i < recs.size(); ++i) {
            const Record& r = recs[i];
            printf("%zu,%d,%d,%d,%zu,%d,%.4f,%.4f,%.2f,%d,%.3e\n",
                   i + 1, r.bs, r.tk, r.coarsen, r.smemBytes, r.blocksPerSM,
                   r.occupancy, r.ms, r.gflops, r.pass, r.maxAbsErr);
        }
        CUDA_CHECK(cudaFree(dA)); CUDA_CHECK(cudaFree(dB)); CUDA_CHECK(cudaFree(dC));
        return EXIT_SUCCESS;
    }

    // ---- 按性能排序并打印表格 ----
    std::qsort(recs.data(), recs.size(), sizeof(Record), cmpGflopsDesc);
    const double best = recs.empty() ? 0.0 : recs[0].gflops;

    printf("\n");
    printf("================ 设计空间扫描结果（按 GFLOP/s 倒序）================\n");
    printf("%-5s %-7s %-6s %-8s %-11s %-11s %-11s %-12s %-11s %-9s %s\n",
           "rank", "block", "tile", "coarsen", "smem(B)", "blk/SM",
           "occup%", "ms", "GFLOP/s", "rel_best", "check");
    printf("-------------------------------------------------------------------------------"
           "-----------------------------------------\n");
    for (size_t i = 0; i < recs.size(); ++i) {
        const Record& r = recs[i];
        printf("%-5zu %-7d %-6d %-8d %-11zu %-11d %-11.1f %-12.4f %-11.1f "
               "%8.1f%%   %s\n",
               i + 1, r.bs, r.tk, r.coarsen, r.smemBytes, r.blocksPerSM,
               r.occupancy * 100.0, r.ms, r.gflops,
               best > 0.0 ? 100.0 * r.gflops / best : 0.0,
               r.pass ? "PASS" : "FAIL(bad)");
    }
    printf("-------------------------------------------------------------------------------"
           "-----------------------------------------\n");
    printf("列说明: rank=排名 block=每block线程数 tile=K方向分块宽度TK coarsen=线程粗化因子\n");
    printf("        smem(B)=每block动态共享内存 blk/SM=每SM可驻留block数(occupancy API)\n");
    printf("        occup%%=理论占用率 ms=中位耗时 GFLOP/s=实测算力 rel_best=相对最优百分比\n");
    printf("有效组合 %zu 个，跳过 %d 个（非法/超资源）\n", recs.size(), skipped);

    if (!recs.empty()) {
        const Record& w = recs[0];
        printf("\n最优配置: block=%d (BX=%d, BY=%d)  tile(TK)=%d  coarsen=%d\n",
               w.bs, BX, w.bs / BX, w.tk, w.coarsen);
        printf("          共享内存 %zu B (%zu B/block/SM 上限 %zu B)\n",
               w.smemBytes, prop.sharedMemPerBlock, prop.sharedMemPerMultiprocessor);
        printf("          每 SM 驻留 %d 个 block -> 理论占用率 %.1f%%\n",
               w.blocksPerSM, w.occupancy * 100.0);
        printf("          %.4f ms  %.1f GFLOP/s  (峰值 19.5 TFLOPS 的 %.1f%%)\n",
               w.ms, w.gflops, 100.0 * w.gflops / 19500.0);
        printf("          相对最慢的有效组合快 %.2fx\n",
               recs.back().gflops > 0.0 ? w.gflops / recs.back().gflops : 0.0);
        printf("正确性: %s (max_abs_err=%.3e)\n",
               w.pass ? "PASS" : "FAIL", w.maxAbsErr);
    }

    CUDA_CHECK(cudaFree(dA));
    CUDA_CHECK(cudaFree(dB));
    CUDA_CHECK(cudaFree(dC));
    return EXIT_SUCCESS;
}
```

- **【代码做什么？】**
- **一次性准备**：读入 `M/N/K/iters`，生成固定种子的随机矩阵，在 CPU 上用 double 算出 golden reference。这个参考值**在 27 个组合之间共享**，从而保证所有组合比对的是同一个基准（否则每个组合一个参考值，无法区分"配置差异"与"参考差异"）。
- **三层循环枚举设计空间**：外层 `bs ∈ {128,256,512}`，中层 `tk ∈ {8,16,32}`，内层 `coarsen ∈ {1,2,4}`。`BX` 固定为 32，于是 `BY = bs/32 ∈ {4,8,16}`，输出块高 `ROWS = BY × coarsen ∈ {4…64}`。

- **合法性过滤**：三处 `continue`。① `bs % BX != 0`；② 动态共享内存超过 `prop.sharedMemPerBlock`；③ `grid.y > 65535`。被跳过的组合计入 `skipped` 并（非 CSV 模式）打印原因——**跳过必须留痕**，否则读者会以为扫描覆盖了全部 27 个点。
- **占用率查询**：调用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor(&numBlocks, kernel, blockSize, dynamicSmem)`，由驱动根据该 kernel 的**真实寄存器用量**、共享内存用量与该架构上限给出每 SM 可驻留的 block 数。再换算成百分比占用率 `numBlocks × bs / 2048`。这比"手算寄存器个数"可靠得多，也是报告里可以引用的权威数字。
- **每个组合：先跑一次验正确性，再跑 3 次预热 + iters 次计时取中位数**。计时用 `cudaEvent` 包住整个 kernel 启动，`cudaEventSynchronize` 之后才读取时间。
- **排序与展示**：`qsort` 按 GFLOP/s 降序排序，打印对齐表格（`%-7d`、`%-11.1f%%` 等宽度控制），并单独打印最优配置的四项关键量：占用率、耗时、GFLOP/s、相对最慢组合的倍数。加 `-csv` 时改为输出一行表头的 CSV，供报告附录与绘图脚本消费。
- **资源释放**：无论走 CSV 分支还是表格分支，都 `cudaFree` 三个设备缓冲。
- **【并行机制与硬件映射解说】**
- **为什么固定 BX = 32**：`blockDim.x = 32` 让 `tid = ty*32 + tx` 的映射下，**一个 warp 恰好是一整行**（同一 `ty`，`tx = 0..31`）。这带来两处直接好处：`Bs[kk*32 + tx]` 的访问跨越 bank 0..31 各一次，**零 bank conflict**；`As[(ty + r*BY)*TK + kk]` 在 warp 内是**单地址广播**（32 个线程读同一个 4 字节值），同样一个事务完成。这是"用线程映射换共享内存效率"的经典手法。
- **共享内存 bank 分析（具体到 bank 编号）**：设 `TK = 32`、`ty = 0`、`kk = 5`。`As[0*32 + 5] = As[5]` → 地址 20 字节 → bank `(20/4) % 32 = 5`，全 warp 同一地址 → 广播，1 个事务。`Bs[5*32 + tx]` → 地址字偏移 `160 + tx`，`tx = 0..31` → bank `(160+tx) % 32 = tx` → 覆盖 0..31 全部 bank，1 个事务。若把 `TK` 改成 **31**（不是 32 的倍数），`As[(ty + r*BY)*31 + kk]` 在不同 `r` 上会落到不同 bank，但由于 warp 内 `ty` 相同、`r` 是展开后的常量，**每个** `r` 的访问依然是广播；真正会出问题的是 `As` 的**写入**：`i = tid`，`c = i % 31`，`r = i / 31`，`As[i]` 的地址是连续的 `tid`，所以写入仍然无冲突。真正需要 padding 的场景是"每行长度是 32 的倍数、而 warp 沿列方向访问不同行"的经典方阵 tile（如 `As[32][32]` 中 `As[ty][kk]` 的转置访问），此时应写成 `As[32][33]` 把行首偏移错开一个 word。
- **全局内存合并分析**：`As` 面板加载时 `i = tid`，`r = i/TK`，`c = i%TK`，全局地址 `(row0+r)*K + k0 + c`。当 `TK = 32` 时 `r = ty`、`c = tx`，warp 内 32 个线程访问 **32 个连续 float = 128 字节 = 一条完整 cache line**，一个 transaction 满载。当 `TK = 8` 时，一个 warp 覆盖 4 行 × 8 列，拆成 4 个 32 字节的 sector；总传输仍是 128 字节且**全部有效**，但请求被切成 4 段，LSU 压力增大——这正是 `TK=8` 在扫描表里偏低的原因之一。
- **`COARSEN` 与 warp 调度的关系**：粗化后每个线程持有 `acc[4]`，指令流从"1 次 LDS + 1 次 FMA"变成"`COARSEN` 次 LDS(广播) + 1 次 LDS + `COARSEN` 次 FMA"。FMA 之间的依赖链被 `acc[0..3]` 四个独立累加器打断，**ILP 显著提高**——每条 LDS 的 20–30 cycles 延迟可以由其它 `acc[r]` 的 FMA 掩盖。代价是每线程寄存器数上升（实测约 40 个），这会直接改变 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 的返回值：`COARSEN=4` 时 `bs=512` 只能驻留 3 个 block（1536 线程 = 75%），而 `COARSEN=1` 时同样 `bs=512` 能驻留 4 个（100%）。
- **warp 发散**：`if (r < COARSEN)` 中的 `r` 经 `#pragma unroll` 展开后是编译期常量，**不产生运行时分发**。真正的发散只出现在最后一个不完整的 block 的写回处。这里有一个细节：`acc[r]` 对越界行仍被计算（读到的是 0），只是不写回——用"多算一点、少写一点"换取无分支的计算主体。
- **【性能优化分析】**
- **共享内存流量公式**：每个 warp 在每个 `kk` 步需要 `COARSEN` 次 `As` 广播（各 1 个事务）+ 1 次 `Bs` 读取（1 个事务），而完成 `COARSEN × 32` 次 FMA。于是**每个 FMA 的共享内存事务数**为 `T/FMA = (COARSEN + 1) / (32 × COARSEN)`。`COARSEN = 1` 时 = `2/32 = 0.0625`；`COARSEN = 2` 时 = `3/64 = 0.0469`；`COARSEN = 4` 时 = `5/128 = 0.0391`。也就是说从 `COARSEN=1` 到 `COARSEN=4`，共享内存压力降低 **1.6 倍**，与实测的 1083 → 3392 GFLOP/s（**3.13 倍**）并不成正比——多出来的收益来自 ILP 提升与 DRAM 复用增加（`ROWS` 从 8 增到 64，A 的复用次数 `N/BX` 不变但 B 的复用次数 `M/ROWS` 从 128 降到 16，DRAM 流量进一步下降）。
- **占用率与性能的关系（实测反例）**：扫描结果显示最优组合 `bs=512, TK=32, COARSEN=4` 的理论占用率只有 **75%**，而次优的 `bs=256, TK=32, COARSEN=4` 是 **100%**，但 75% 的那个反而快 **12.5%**（0.712 ms vs 0.633 ms）。这说明**占用率不是越高越好**：占用率的作用是"提供足够的 warp 来隐藏延迟"，一旦超过"足够"的阈值（本例约 48 warp，即 75%），继续提高占用率只意味着每线程的工作更少、复用更低。用 Little 定律算：需要隐藏的是 shared load 的约 25 cycles 延迟，每 cycle 每 SM 可发射 4 个 warp 指令，因此需要约 `25 × 4 = 100` 个 warp 常驻才能完全掩盖——但 64 warp 是硬件上限，所以**永远不可能完全隐藏，只能靠 ILP 部分弥补**，这正是 `COARSEN` 有效的根本原因。
- **Roofline 定位**：最优配置的 DRAM 流量 = `4·M·N·K·(1/BX + 1/ROWS)`（A 每行被读 `N/BX` 次、B 每列被读 `M/ROWS` 次），因此 `AI = 2·M·N·K / (4·M·N·K·(1/BX + 1/ROWS)) = 1/(2·(1/32 + 1/64)) = 10.7 FLOP/Byte`。它**略低于** A100 的 ridge point 12.5 FLOP/Byte，所以落在带宽受限区的右侧边缘：带宽屋顶 = `10.7 × 1555 = 16639 GFLOP/s`，实测 3392 GFLOP/s 只有该屋顶的 **20.4%**，计算屋顶 19500 GFLOP/s 也只用到 **17.4%**。两个屋顶都没碰到，说明它既不是 DRAM 受限（实测 DRAM 吞吐 `0.201 GB / 0.633 ms = 318 GB/s`，仅占 1555 GB/s 的 20%）也不是理论计算受限，而是被**共享内存带宽 + 指令发射 + 占用率**这三者的组合卡住——这类"落在两个屋顶之下、却远未触顶"的点，只能靠 profiler 的 stall reason 进一步细分，是本项目在报告中必须讨论清楚的部分。
- **设计空间的最优区域形状**：从扫描结果看，最优点集中在"高 `COARSEN`、大 `TK`、`bs` 取 256–512"的角上。这不是巧合：`COARSEN` 提高复用、`TK` 降低同步频率、`bs` 影响占用率——三者的方向一致时才互相加强。但注意 `TK=64`、`COARSEN=8` 之类的"更极端"取值没有出现在扫描里，**这不代表它们不好**，只代表本次扫描的范围没覆盖——报告里必须写明扫描范围与这条局限（对应教学目标 17"识别局限与未来方向"）。

代码示例 3：报告数据生成器 —— Roofline 与规模扩展曲线的 CSV

最终报告里的两张核心图是 **Roofline 图**（横轴算术强度、纵轴 GFLOP/s）与**规模扩展曲线（scaling curve）**（横轴问题规模、纵轴加速比或耗时）。这个程序在**同一台机器上**、用**同一套测量方法**，对两个 kernel（朴素版与分块版）在不同数据规模下测量，直接输出可被 gnuplot / Python / Excel 消费的 CSV。

```
// 文件: report_data.cu
// 编译: nvcc -O3 -arch=sm_80 report_data.cu -o report_data
// 运行: ./report_data -maxn 2048 -iters 20 > report_data.csv
//       ./report_data -maxn 4096 -iters 10 | tee report_data.csv
//
// 输出 CSV 列: kernel,size,ms,gflops,bytes_gb,ai,bw_gbs,occup_theory,check
//   kernel       : naive 或 tiled
//   size         : 方阵维度 N (M=N=K=N)
//   ms           : 中位耗时
//   gflops       : 实测算力
//   bytes_gb     : 解析访存量 (GB)
//   ai           : 算术强度 (FLOP/Byte) —— Roofline 的横坐标
//   bw_gbs       : 有效 DRAM 吞吐 (GB/s) —— 判断是否撞内存墙
//   occup_theory : cudaOccupancyMaxActiveBlocksPerMultiprocessor 给出的理论占用率
//   check        : PASS / PASS(gpu-ref) / FAIL
//
// 验证阶梯 (verification ladder):
//   size <= VERIFY_MAX  : 与 CPU double golden reference 比对
//   size >  VERIFY_MAX  : 与已在小规模上验证过的另一个 GPU kernel 互比
//    (大规模 CPU 参考太慢, 但"两个独立实现互比"在两者都各自验证过时可信)

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cmath>
#include <vector>
#include <algorithm>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                       \
    do {                                                                       \
        cudaError_t err__ = (call);                                            \
        if (err__ != cudaSuccess) {                                            \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,      \
                    cudaGetErrorString(err__));                                \
            exit(EXIT_FAILURE);                                                \
        }                                                                      \
    } while (0)

constexpr int  TILE       = 16;      // 分块宽度
constexpr int  VERIFY_MAX = 1024;    // 超过此规模不再用 CPU 参考

// ---------------------------------------------------------------------------
// kernel 1: 朴素版 —— 每个线程算一个输出, 每次乘加都走全局内存
// ---------------------------------------------------------------------------
__global__ void naiveMatMulKernel(const float* __restrict__ A,
                                  const float* __restrict__ B,
                                  float* __restrict__ C,
                                  int M, int N, int K)
{
    const int col = blockIdx.x * blockDim.x + threadIdx.x;
    const int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row >= M || col >= N) return;

    float acc = 0.0f;
    for (int k = 0; k < K; ++k) {
        acc += A[(size_t)row * K + k] * B[(size_t)k * N + col];
    }
    C[(size_t)row * N + col] = acc;
}

// ---------------------------------------------------------------------------
// kernel 2: 分块版 —— As/Bs 面板进共享内存, K 方向分块
// ---------------------------------------------------------------------------
__global__ void tiledMatMulKernel(const float* __restrict__ A,
                                  const float* __restrict__ B,
                                  float* __restrict__ C,
                                  int M, int N, int K)
{
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    const int tx  = threadIdx.x;
    const int ty  = threadIdx.y;
    const int row = blockIdx.y * TILE + ty;
    const int col = blockIdx.x * TILE + tx;
```

```
    float acc = 0.0f;
    for (int k0 = 0; k0 < K; k0 += TILE) {
        As[ty][tx] = (row < M && (k0 + tx) < K) ? A[(size_t)row * K + k0 + tx] : 0.0f;
        Bs[ty][tx] = ((k0 + ty) < K && col < N) ? B[(size_t)(k0 + ty) * N + col] : 0.0f;
        __syncthreads();

        #pragma unroll
        for (int kk = 0; kk < TILE; ++kk) {
            acc += As[ty][kk] * Bs[kk][tx];
        }
        __syncthreads();
    }
    if (row < M && col < N) C[(size_t)row * N + col] = acc;
}

// ------------------------------------------------------------------------
static void goldenReference(const float* A, const float* B, float* C,
                            int M, int N, int K)
{
    for (int i = 0; i < M; ++i) {
        for (int j = 0; j < N; ++j) {
            double sum = 0.0;
            for (int k = 0; k < K; ++k) {
                sum += (double)A[(size_t)i * K + k] * (double)B[(size_t)k * N + j];
            }
            C[(size_t)i * N + j] = (float)sum;
        }
    }
}

static bool verify(const float* got, const float* ref, size_t n,
                   float rtol, float atol, double* maxErrOut)
{
    bool pass = true;
    double maxErr = 0.0;
    for (size_t i = 0; i < n; ++i) {
        const double ad  = fabs((double)got[i] - (double)ref[i]);
        const double tol = (double)atol + (double)rtol * fabs((double)ref[i]);
        if (ad > maxErr) maxErr = ad;
        if (ad > tol) pass = false;
    }
    if (maxErrOut) *maxErrOut = maxErr;
    return pass;
}

static double timeMedianMs(void (*launch)(void*), void* ctx, int warmup, int iters)
{
    for (int i = 0; i < warmup; ++i) launch(ctx);
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t s, e;
    CUDA_CHECK(cudaEventCreate(&s));
    CUDA_CHECK(cudaEventCreate(&e));

    std::vector<double> t;
    t.reserve((size_t)iters);
    for (int i = 0; i < iters; ++i) {
        CUDA_CHECK(cudaEventRecord(s));
        launch(ctx);
        CUDA_CHECK(cudaEventRecord(e));
        CUDA_CHECK(cudaEventSynchronize(e));
        float ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, s, e));
        t.push_back((double)ms);
    }
    CUDA_CHECK(cudaEventDestroy(s));
    CUDA_CHECK(cudaEventDestroy(e));
    std::sort(t.begin(), t.end());
    return t[t.size() / 2];
}

// ------------------------------------------------------------------------
struct LaunchCtx {
    bool   tiled;
    float* dA;
    float* dB;
    float* dC;
    int    M, N, K;
```

```
    dim3   grid;
    dim3   block;
};

static void doLaunch(void* p)
{
    LaunchCtx* c = (LaunchCtx*)p;
    if (c->tiled) {
        tiledMatMulKernel<<<c->grid, c->block>>>(c->dA, c->dB, c->dC, c->M, c->N, c->K);
    } else {
        naiveMatMulKernel<<<c->grid, c->block>>>(c->dA, c->dB, c->dC, c->M, c->N, c->K);
    }
}

// ------------------------------------------------------------------------
int main(int argc, char** argv)
{
    int maxN  = 2048;
    int iters = 20;
    for (int i = 1; i < argc; ++i) {
        if      (!strcmp(argv[i], "-maxn")  && i + 1 < argc) maxN  = atoi(argv[++i]);
        else if (!strcmp(argv[i], "-iters") && i + 1 < argc) iters = atoi(argv[++i]);
        else { fprintf(stderr, "用法: %s [-maxn N] [-iters K]\n", argv[0]); return EXIT_FAILURE; }
    }

    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    fprintf(stderr, "设备: %s  每 SM %d 线程  共享内存/block %.1f KB\n",
            prop.name, prop.maxThreadsPerMultiProcessor, prop.sharedMemPerBlock / 1024.0);
    fprintf(stderr, "(Roofline 参考: FP32 峰值 19.5 TFLOPS, 带宽 1555 GB/s, ridge=12.5 FLOP/B)\n");

    // 理论占用率（只与 kernel 资源有关，与问题规模无关，因此只需查一次）
    int nbNaive = 0, nbTiled = 0;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(&nbNaive, naiveMatMulKernel, 256, 0));
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(&nbTiled, tiledMatMulKernel,
                                                             TILE * TILE, 0));
    const double occNaive = (double)nbNaive * 256.0 / prop.maxThreadsPerMultiProcessor;
    const double occTiled = (double)nbTiled * (TILE * TILE) /
                            prop.maxThreadsPerMultiProcessor;
    fprintf(stderr, "理论占用率: naive=%d blk/SM -> %.1f%%   tiled=%d blk/SM -> %.1f%%\n",
            nbNaive, occNaive * 100.0, nbTiled, occTiled * 100.0);

    printf("kernel,size,ms,gflops,bytes_gb,ai,bw_gbs,occup_theory,check\n");

    for (int n = 128; n <= maxN; n *= 2) {
        const int M = n, N = n, K = n;
        const size_t szA = (size_t)M * K, szB = (size_t)K * N, szC = (size_t)M * N;
        const double flop = 2.0 * (double)M * (double)N * (double)K;

        std::vector<float> hA(szA), hB(szB), hC(szC), hRef(szC), hNaive(szC);
        unsigned seed = 987654321u;
        for (size_t i = 0; i < szA; ++i) { seed = seed * 1664525u + 1013904223u;
            hA[i] = (float)((seed >> 8) & 0xFFFFFFu) / (float)0x1000000u - 0.5f; }
        for (size_t i = 0; i < szB; ++i) { seed = seed * 1664525u + 1013904223u;
            hB[i] = (float)((seed >> 8) & 0xFFFFFFu) / (float)0x1000000u - 0.5f; }

        float *dA = nullptr, *dB = nullptr, *dC = nullptr;
        CUDA_CHECK(cudaMalloc((void**)&dA, szA * sizeof(float)));
        CUDA_CHECK(cudaMalloc((void**)&dB, szB * sizeof(float)));
        CUDA_CHECK(cudaMalloc((void**)&dC, szC * sizeof(float)));
        CUDA_CHECK(cudaMemcpy(dA, hA.data(), szA * sizeof(float), cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaMemcpy(dB, hB.data(), szB * sizeof(float), cudaMemcpyHostToDevice));

        if (M <= VERIFY_MAX) {
            fprintf(stderr, "N=%d: 计算 CPU golden reference ...\n", n);
            goldenReference(hA.data(), hB.data(), hRef.data(), M, N, K);
        }

        for (int which = 0; which < 2; ++which) {
            const bool tiled = (which == 1);
            LaunchCtx ctx;
            ctx.tiled = tiled;
            ctx.dA = dA; ctx.dB = dB; ctx.dC = dC;
            ctx.M = M; ctx.N = N; ctx.K = K;
            if (tiled) {
                ctx.block = dim3(TILE, TILE);
                ctx.grid  = dim3((unsigned)((N + TILE - 1) / TILE),
                                 (unsigned)((M + TILE - 1) / TILE));
```

```
            } else {
                ctx.block = dim3(16, 16);
                ctx.grid  = dim3((unsigned)((N + 15) / 16), (unsigned)((M + 15) / 16));
            }

            doLaunch(&ctx);
            CUDA_CHECK(cudaGetLastError());
            CUDA_CHECK(cudaMemcpy(hC.data(), dC, szC * sizeof(float),
                                  cudaMemcpyDeviceToHost));

            const char* check = "PASS(gpu-ref)";
            if (M <= VERIFY_MAX) {
                double e = 0.0;
                check = verify(hC.data(), hRef.data(), szC, 1e-3f, 1e-3f, &e) ? "PASS" : "FAIL";
            } else if (tiled) {
                // 验证阶梯：大规模下与朴素版结果互比（朴素版已在 <=VERIFY_MAX 时通过 CPU 校验）
                double e = 0.0;
                check = verify(hC.data(), hNaive.data(), szC, 1e-3f, 1e-3f, &e)
                        ? "PASS(gpu-ref)" : "FAIL";
            } else {
                std::copy(hC.begin(), hC.end(), hNaive.begin());   // 朴素版留作参照
            }

            const double ms     = timeMedianMs(doLaunch, &ctx, 3, iters);
            const double gflops = flop / (ms * 1e-3) / 1e9;

            // 解析访存量：朴素版每个输出读 K+K 个元素；分块版 A 读 N/TILE 次、B 读 M/TILE 次
            const double bytes = tiled
                ? 4.0 * (double)M * N * K * (1.0 / TILE + 1.0 / TILE)
                : 4.0 * (double)M * N * K * 2.0;
            const double ai   = flop / bytes;
            const double bw   = bytes / (ms * 1e-3) / 1e9;

            printf("%s,%d,%.4f,%.2f,%.4f,%.3f,%.1f,%.4f,%s\n",
                   tiled ? "tiled" : "naive", n, ms, gflops, bytes / 1e9, ai, bw,
                   tiled ? occTiled : occNaive, check);
            fflush(stdout);
        }

        CUDA_CHECK(cudaFree(dA));
        CUDA_CHECK(cudaFree(dB));
        CUDA_CHECK(cudaFree(dC));
    }

    fprintf(stderr, "\nCSV 已输出到 stdout。绘图示例:\n");
    fprintf(stderr, "  gnuplot -e \"set datafile separator ','; set logscale xy; \"\\\n");
    fprintf(stderr, "    \"plot 'report_data.csv' using 6:4 every ::1 title 'measured'\\\n");
    fprintf(stderr, "     with points, 1555*x title 'A100 BW roof', 19500 title 'A100 peak'\"\n");
    fprintf(stderr, "  # 规模扩展曲线: 用 naive 与 tiled 在同一 size 下的 ms 相除得到 speedup\n");
    return EXIT_SUCCESS;
}
```

- **【代码做什么？】**
- **固定实验条件**：所有规模的输入都在同一进程内生成、同一块 GPU 上测量、同一个 `timeMedianMs`（3 次预热 + iters 次取中位数）。报告中"不同数字来自不同测量方法"是最常见的不一致来源，这个程序用代码结构强制统一。
- **两个 kernel 对照**：`naiveMatMulKernel` 每个输出元素从全局内存读 `2K` 个 float，算术强度恒为 `2·M·N·K / (8·M·N·K) = 0.25 FLOP/Byte`；`tiledMatMulKernel` 用 `As/Bs` 面板把算术强度提到 `1/(2·(1/16+1/16)) = 4 FLOP/Byte`。两者在 Roofline 上应当落在同一条带宽斜线上的不同位置——这正是报告里用来"证明瓶颈是带宽"的关键证据。
- **验证阶梯**：`N ≤ 1024` 时与 CPU double 参考比对；`N > 1024` 时让 tiled 与 naive 互比（naive 在小规模上已独立通过 CPU 校验）。这样既避免了 2048^3 的 CPU 双重循环（约 8.6 GFLOP，需要数秒到数十秒），又没有放弃大规模的正确性检查。**降级的验证方法必须在报告里写明**，不能悄悄降级。
- **CSV 输出**：每一行是一个 (kernel, size) 数据点，列名固定。CSV 而不是 markdown 表格的原因是可以直接被 gnuplot / pandas / Excel 读取，并在报告里"可复现地"重绘。
- **占用率只查一次**：`cudaOccupancyMaxActiveBlocksPerMultiprocessor` 的结果只取决于 kernel 的资源用量（寄存器、共享内存）与 block 大小，与问题规模无关，所以放在循环外查一次即可，避免重复开销。
- **【并行机制与硬件映射解说】**
- **两个 kernel 的 warp 映射完全不同**：朴素版用 `block(16,16)`，`tid = ty*16+tx`，一个 warp 覆盖**两行**（`ty` 相同则 16 个线程连续，warp 0 = 行 0 的 16 列 + 行 1 的 16 列）。它的全局读 `A[row*K + k]` 在 warp 内是**同一地址的广播**（同一行、同一 k），而 `B[k*N + col]` 是连续 16 个地址 + 另一行连续 16 个地址。因此一个 warp 每次 k 迭代发起 2 个 64 字节的段，**但只用到其中一半**：A 的广播只取 4 字节，却占用了整个 sector 的传输预算。这就是朴素版"DRAM 流量 = 8·M·N·K 字节"却只跑到 67% 带宽的根本原因——**带宽浪费在重复传输上，而不是没打满**。

- **分块版的共享内存 bank**：`As[16][16]` 的行长 16 个 float。warp 0 的两个半 warp 分别读 `As[0][kk]`（bank `kk`）与 `As[1][kk]`（bank `(16+kk)%32`），两个地址两个 bank，无冲突；`Bs[kk][tx]` 的 `tx = 0..15` 覆盖 bank `(16kk)%32` 起的 16 个 bank，两个半 warp 读同一地址属广播，无冲突。这与示例 1 的分析一致——**当 tile 宽度是 16 且 warp 恰好覆盖两行时，这个访问模式天然无冲突**。若改用 `TILE = 32` 且 block 为 (32,8)，则 `Bs[kk][tx]` 覆盖 bank 0..31 各一次，更加规整。
- **占用率**：`tiledMatMulKernel` 的 block 是 256 线程，静态共享内存 `2×16×16×4 = 2 KB`，寄存器约 32 个。A100 上线程限制给出 2048/256 = 8 个 block；寄存器限制给出 `65536/(256×32) = 8` 个 block；共享内存限制给出 `164 KB / 2 KB = 82` 个 block。因此 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 会返回 8，占用率 100%。朴素版的寄存器略少但访存模式更差，两者占用率几乎相同——**这说明"占用率相同、性能差 6 倍"的情形完全存在**，占用率只是一个必要不充分的指标。
- **warp 发散**：两个 kernel 的边界判断都在写回处，且没有 else 分支，不产生真实发散。但当 `N` 不是 16 的整数倍时（例如 N=1000），最后一列 block 里 `col >= N` 的线程会整列空转——**浪费比例 = ceil(N/16)·16/N − 1**，N=1000 时约 2.4%。
- **【性能优化分析】**
- **两条 Roofline 坐标**：朴素版 `AI = 0.25 FLOP/Byte`，带宽屋顶给出 `0.25 × 1555 = 389 GFLOP/s` 的上限；若实测约 262 GFLOP/s，则达到该屋顶的 **67%**，说明它**确实撞在内存墙上**。分块版 `AI = 4 FLOP/Byte`，带宽屋顶给出 `4 × 1555 = 6220 GFLOP/s`，若实测约 1512 GFLOP/s，则只达到 **24%**——此时"内存墙"已经不是限制项，真正的限制是共享内存带宽（每 FMA 需 8 字节 shared 流量，1.074e9 次 FMA 需要 8.59 GB 的 shared 流量，而 shared 带宽约 19.5 TB/s，下限 0.44 ms）。
- **规模扩展曲线的两种画法**：① **固定问题规模**（本例的做法）——随着 N 增大，总工作量按 N^3 增长，而 kernel 启动开销（约 3–5 μs）固定，所以小规模时加速比被启动开销严重稀释：N=128 时总运算量只有 4.2 MFLOP，按 1000 GFLOP/s 计算纯计算时间仅 4.2 μs，**与启动开销同量级**，测出来的"性能"基本没有意义。② **固定每线程工作量（grid-stride loop）**——随着 N 增大线程数同步增长，能一直保持满占用，曲线会更快地逼近平台期。报告里应当**同时给出两种画法**并解释差异，这比只给一条曲线更有说服力。
- **加速比的分母**：报告中最容易被质疑的一栏。分母必须是**同一台机器、同一种语言、同一个算法**的串行实现（S19 明确要求"实现一个 C 串行版本作为 baseline"）。用 Python 的 numpy 或者一个未优化的递归实现当分母，可以轻松把加速比刷到 1000×——但报告读者一看就知道问题所在。诚实的做法是同时报告"vs 朴素 C 串行"和"vs 高度优化的库（如 OpenBLAS/MKL）"两个加速比，后者往往只有 3–10×，但它才是**真实的工程价值**。
- **理论加速上限（Amdahl 定律）**：设串行部分占比为 `s`，则加速比上限为 `1/s`。若程序有 5% 的部分无法并行（如输入解析、单点递推），那么再多的优化也只能得到 20× 的总加速比。选题阶段就应该估算这个 `s`，并写进 Proposal 的"风险"一节。

性能优化技巧总结

1. **先写对，再写快，最后写报告**。任何时刻都要有一个能跑通、能通过 golden reference 的版本；优化分支永远从已验证的 baseline 上切出来。为什么有效：一个不能运行的版本无法测量，"感觉快了"不是数据。
2. **把基准测试做成一条命令**。把问题规模、block、tile、粗化因子全部参数化（示例 1 的 `parseArgs`），因为**手工改代码测 27 个组合必然出错**，而且无法追溯"这个数字是哪次跑出来的"。
3. **一次只改一个变量**。改两个变量后，性能变化无法归因，报告里的"优化贡献度"一栏就成了猜测。为什么有效：受控实验是唯一能把"观测"变成"结论"的方法。
4. **每次测量重复 ≥5 次，报告最小值与中位数**。GPU 上的单次测量波动常达 ±5%~15%（时钟boost、其它进程、L2 残留状态）。为什么有效：最小值最接近纯 kernel 时间，中位数最抗离群点，两者结合能暴露"测量本身不稳定"这一事实。
5. **先做 Roofline 估算再动手**。用 `AI = FLOP/Bytes` 与 `峰值/带宽` 相比，5 分钟就能知道该优化算力还是优化访存。为什么有效：优化了错误的资源等于白干——在一个 AI=8 的 kernel 上拼命降低 DRAM 流量不会有效果。
6. **用 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 而不是手算寄存器**。API 反映的是编译器实际分配的寄存器数（受 `__launch_bounds__` 与 `-maxrregcount` 影响），手算的 `-Xptxas -v` 输出是优化前的快照。为什么有效：占用率的分子分母都必须来自同一次编译的产物。
7. **线程粗化是性价比最高的单项优化**。让每个线程计算 2–8 个输出，把 `Bs` 的一次读取分摊到多次 FMA 上（示例 2 中共享内存事务数从 `2/32` 降到 `5/128`）。为什么有效：它同时提高复用率、降低共享内存压力、增加 ILP（多个独立累加器打断依赖链）。
8. **用共享内存 padding 消除 bank conflict**。对"行长为 32 的倍数且 warp 沿列访问不同行"的模式，把 `As[32][32]` 改成 `As[32][33]`。为什么有效：连续行的起始地址错开一个 word 后，同一 warp 的不同行访问落到不同 bank，32 路冲突降为 1 路。
9. **tile 尺寸的选择要算浪费率**：`(T+W−1)²/T²` 是 halo 开销，`ceil(N/T)·T/N − 1` 是尾块浪费。为什么有效：单调增大 tile 会让共享内存占用按平方增长，最终把占用率压垮——最优点通常在"复用够用 + 占用率不塌"的折中处。
10. **让访存对齐到 128 字节**。warp 内 32 个线程访问连续 32 个 float 恰好是一条 cache line；`float4` 向量化把每线程字节数提到 16，但会同时抬高寄存器压力。为什么有效：对齐访问的 memory efficiency = 100%，而未对齐/跨步访问会把一次 128 字节的传输浪费掉一大半。
11. **减少 `__syncthreads()` 的次数，而不是减少它的开销**。增大 K 方向分块宽度 TK 可以把同步次数从 `K/TK` 次降下来。为什么有效：每次同步都是一个流水线气泡，且同步次数与 K 成正比时，同步开销随问题规模增长而增长。
12. **把"没有提升"的尝试也写进优化日志**。示例日志中的"float4 向量化导致寄存器压力上升、占用率从 100% 降到 75%、性能反而下降 8%"是一条极有价值的负结果。为什么有效：它向读者证明你真的做了受控实验，而不是只挑好看的数字。
13. **消除 host↔device 往返**。PCIe 的带宽约 25 GB/s，只有 A100 HBM 带宽（1555 GB/s）的 1.6%。为什么有效：把需要逐次迭代的结果搬回主机再搬回去，会让整个加速比被这条"细管子"钉死。
14. **在报告里给每个数字配一条命令**。例如"3392 GFLOP/s"后面写清 `./sweep -m 1024 -n 1024 -k 1024 -iters 20` 与机器型号。为什么有效：可复现性是技术报告与"性能传说"的唯一区别。

关键要点

- **最终项目的评分是"能不能跑"乘以"跑得多快"**：功能与工程规范约占 50%，功能完整前提下的性能约占 50%；代码编译不过或运行崩溃会让性能分完全失去意义，所以**永远保持一个可运行的版本**。
- **四个交付物（Proposal / Workshop / Presentation / Report）是四个检查站，不是一个任务的四次提交**：Proposal 决定上限（选题的串行占比与算术强度），Workshop 是最便宜的纠错点，Presentation 验证结果可信度，Report 沉淀可复现的知识。课程教学目标 10—17 全部通过这条链达成。
- **性能工程的本质是受控实验**：① 正确基线 → ② 测量剖析 → ③ 定位瓶颈 → ④ 可证伪假设 → ⑤ 只改一个变量 → ⑥ 重新测量并记录 → ⑦ 保留或回退。台账（优化日志）不是文档工作，而是让你三次迭代之后还记得"当时为什么那么改"的唯一依靠。
- **Roofline 是选题阶段就该用的工具**：A100 的 ridge point 是 `19.5 TFLOPS / 1555 GB/s ≈ 12.5 FLOP/Byte`，RTX 4090 是 `82.6/1008 ≈ 82`。算术强度低于 ridge 的问题，优化方向只有一个——提高复用（tiling + 粗化），否则再多调参也只是在带宽屋顶下面挪位置。
- **占用率高不等于性能好**：示例 2 中 75% 占用率的最优配置比 100% 占用率的次优配置快 12.5%。占用率的作用是"提供足够的 warp 来隐藏延迟"，一旦跨过阈值，继续提高占用率只意味着每线程活更少、复用更低。
- **报告的价值在于诚实的证据链**：容差判据、边界尺寸测试、失败尝试、未解释的现象、扫描范围之外的参数，这些"不好看"的内容才是报告区别于宣传材料的地方，也是课程教学目标 17（识别局限与未来方向）的评分依据。

常见陷阱与注意事项

- **未检查 CUDA API 返回值 / 忽略异步启动错误**：kernel 启动是异步的，非法配置（block 超过 1024 线程、共享内存超限）不会立刻让程序崩溃，而是让后续某个无关的调用返回错误，排查方向完全错位 → 用 `CUDA_CHECK` 包裹每一个 CUDA 调用，并在**每次 kernel launch 之后立刻**调用 `cudaGetLastError()` 把错误归因到具体那次启动。
- **计时忘记同步 / 只跑一次就下结论**：只 `cudaEventRecord(stop)` 而不 `cudaEventSynchronize(stop)`，测到的是**启动开销**而不是 kernel 执行时间；单次测量的波动可达 ±15% → 用 `cudaEventSynchronize` 等待事件完成后再读时间，并且 warmup 3 次 + 至少 5 次有效测量后取中位数与最小值。
- **忘记 `__syncthreads()`**：tile 加载后没同步就开始计算，线程读到的是上一个 k 分块的残留值或未初始化数据；症状是"结果接近但错误率随机、跑两次结果不一样" → 加载后与复用前各放一个 `__syncthreads()`，并且**两个都要有**（第二个用于防止下一轮覆盖还被读着的面板）。
- **`__syncthreads()` 放在分支内**：`if (tid < n) { ...; __syncthreads(); }` 会让部分线程永远等不到汇合点，导致挂起或未定义行为；这与 **warp 发散**是同一个坑的两面 → 把同步点移到所有分支之外；若确实需要"部分线程参与"，改用 `__syncwarp()` 或让所有线程都走到同步点只是不做事。
- **共享内存 bank conflict**：`As[32][32]` 中 `As[ty][kk]` 与 `As[kk][tx]` 交替访问时，同一 warp 的不同行落在同一个 bank（因为 32 float 的行长恰好是 bank 数的整数倍），形成最多 **32 路冲突**，共享内存吞吐降到 1/32 → 用 `As[32][33]` 做 padding，或调整线程映射使 warp 内访问落在不同 bank（把 `blockDim.x` 设为 32 让一个 warp 对应一行是更根本的解法）。
- **网格覆盖不足与越界访问**：用 `N/blockDim.x` 而不是 `(N + blockDim.x - 1)/blockDim.x` 计算网格维度，最后不足一个 block 的元素永远算不到；反过来，算了 ceil 却在 kernel 里没写 `if (row < M && col < N)` 就会越界写，破坏相邻内存或触发 `cudaErrorIllegalAddress` → **两者必须同时做**：网格用 ceil，kernel 内逐元素判界；并且用 N=1023/1025/1000 这类"非整数倍"尺寸做回归测试（N=1024 恰好是整数倍，永远测不出这个 bug）。
- **host/device 指针混用与 `cudaMemcpy` 方向写错**：把设备指针传给 CPU 循环会直接段错误；`cudaMemcpy` 的第四个参数写反（`DeviceToHost` 写成 `HostToDevice`）不会报错，只会让结果"永远是初始值" → 指针命名统一加 `d_` / `h_` 前缀，`cudaMemcpy` 的方向参数与源/目的参数一起检查；错误检查不能省——它正是能抓到非法地址的那道网。
- **共享内存超限与"一次改两个变量"**：静态声明超过 48 KB 的共享内存编译会失败，动态共享内存超过 48 KB 必须在启动前调用 `cudaFuncSetAttribute(..., cudaFuncAttributeMaxDynamicSharedMemorySize, bytes)`，且总量不能超过 A100 的 164 KB / RTX 4090 的 100 KB；与此同时，把"改 tile 大小"和"改 block 大小"一起提交，会让性能变化无法归因 → 先查 `prop.sharedMemPerBlock` 与 `prop.sharedMemPerMultiprocessor` 并做启动前校验，再坚持"一次只改一个变量"。

思考题（带答案）

Q1. 某个 kernel 在 A100 上测得 3.2 TFLOP/s，解析算术强度为 8 FLOP/Byte，同时 ncu 显示 DRAM 吞吐 250 GB/s。判定瓶颈类型，并给出下一步的三条排查方向。

答案：先算屋顶。带宽屋顶 = `AI × 带宽 = 8 × 1555 = 12440 GFLOP/s`，实测 3200 GFLOP/s 只有其 **25.7%**；DRAM 实测吞吐 250 GB/s 只占 1555 GB/s 的 **16%**，远未饱和；计算屋顶 19500 GFLOP/s，实测占 **16.4%**。三个比例都远离 100%，因此**既不是 DRAM 带宽受限，也不是计算受限**，属于**延迟受限/共享内存吞吐受限/占用率不足**这一大类。下一步：① 查 `cudaOccupancyMaxActiveBlocksPerMultiprocessor` 与 ncu 的 achieved occupancy，看是否被寄存器或共享内存压到 50% 以下；② 查 ncu 的 stall reason 分布——若 `stall_long_scoreboard` / `stall_short_scoreboard` 占比高，说明访存延迟没被隐藏，方向是增加 ILP（线程粗化）或提高占用率；③ 查 shared memory 吞吐占峰值的比例，若超过 70% 就是共享内存带宽受限，方向是把每 FMA 的 shared 访问字节数从 8 B 降到 5 B 以下（粗化 + 寄存器复用）。

Q2. 为什么"用 N=1024 测出来的 PASS"几乎不能说明程序是正确的？请举出至少三个 N=1024 会掩盖、而 N=1025 会暴露的问题。

答案：1024 是 2 的幂，同时是 8/16/32 等常用 tile 宽度的整数倍，于是所有 block 都是"满块"、所有线程都走同一条控制路径，边界代码**从未被执行过**。会掩盖的问题：① **网格覆盖不足**——若网格维度写成 `N/blockDim.x`，N=1024 时恰好整除所以覆盖完整，N=1025 时最后一个元素永远算不到；② **越界写**——满块时 `row < M && col < N` 恒为真，缺少这个判断也不会出错，N=1025 时最后一个 block 的越界线程会写坏相邻内存；③ **halo/ghost 元素的边界处理**——对于 stencil 类问题，N=1024 时边界块的处理分支（越界读补 0）不会触发，N=1025 时会读出垃圾值并把误差传播到整个输出；④ **非 2 的幂的尺寸还会改变 grid 维度与尾块数量**，从而暴露 `__syncthreads()` 在部分线程提前 return 之后被调用的死锁问题。正确做法是把 `N = 0, 1, 31, 32, 33, 1023, 1024, 1025, 1000` 全部纳入回归测试。

Q3. 团队把三项优化（共享内存分块、线程粗化、float4 向量化）一起实现后，测到相对 baseline 的 3.0× 加速比。这个 3.0× 能否直接写进报告？如果要在两天内把它拆解成可归因的数据，应该怎么补实验？

答案：不能直接写。三项一起改意味着这个 3.0× 无法归因，Rubric 要求的"explain the optimization strategies, the expected impact, and the actual measured benefit"缺少**单项**的对应证据；而且 3.0× 可能掩盖了"某项优化其实是负收益、只是被另两项的增益盖住"这种情况（示例日志中的 float4 就是这种典型）。两天内可做的补实验：**消融实验（ablation）**——从已验证的 baseline 出发，按固定顺序每次只叠加一项优化，记录每一步的耗时、GFLOP/s 与单步加速比（这就是优化日志表的 1/2/3 行）；如果时间允许，再对"全开"配置做**留一实验（leave-one-out）**，即每次只关掉一项，两份数据交叉验证可以识别出优化之间的耦合（例如粗化与向量化都吃寄存器，单独开都有效、一起开却因寄存器溢出而互相抵消）。最后，每一项都必须至少重复 5 次取中位数，并确认单步收益大于 ±5% 的测量噪声；任何小于噪声的收益都应当标注为"在噪声范围内，无法确认"。同时，所有对比都必须建立在**正确性 PASS** 的前提上——快而错的数据没有资格进入报告。

附录：CUDA 并行模式与优化速查表

> 按类别汇总全书涉及的 CUDA API、性能公式、优化技巧与诊断方法。 所有性能数字以 **NVIDIA A100 (sm_80)** 为基准：19.5 TFLOPS FP32、1555 GB/s、机器平衡点 12.5 FLOP/Byte。

目录

1. 内存优化
2. 线程组织
3. 同步与通信
4. warp 级原语
5. 七种并行模式速查
6. 性能分析公式
7. 性能诊断流程
8. CUDA API 速查
9. 编译器与构建选项
10. 硬件参数速查
11. CUDA → WebGPU/WGSL 对照
12. 常见陷阱速查

---

1. 内存优化

1.1 内存空间对照表

空间	声明	作用域	生命周期	延迟	位置	典型容量
寄存器	自动变量（无修饰）	单线程	线程	~1 cycle	SM 寄存器文件	255 regs/线程，65536/SM
局部内存	自动变量（溢出/动态索引数组）	单线程	线程	400–800 cycles	**DRAM**（被 L1/L2 缓存）	受显存限制
共享内存	`__shared__`	block	block	~20–30 cycles	SM 片上（与 L1 同体）	48–228 KB/SM
全局内存	`__device__` / `cudaMalloc`	全 grid	应用	400–800 cycles	DRAM	显存容量
常量内存	`__constant__`	全 grid	应用	命中缓存 ~数周期；未命中 ~400+	DRAM + 常量缓存（8 KB/ SM）	**64 KB 总计**
纹理内存	`texture<>` / `__ldg`	全 grid	应用	命中 ~数十周期	通过纹理/L1 缓存	受显存限制

1.2 全局内存：合并访问（coalescing）

核心规则：一个 warp 的 32 个线程若访问**连续的 4 字节地址**，硬件合并为 **1 次 128 字节事务**。

```
✅ 合并（coalesced）— 1 次 128B 事务
warp 线程:   t0   t1   t2   t3  ...  t31
地址:       +0   +4   +8   +12 ...  +124
            └──────────── 128 字节 ────────────┘
                    = 1 transaction

❌ 跨步（strided, stride = N*4 字节）— 32 次事务
warp 线程:   t0    t1    t2   ...   t31
地址:       +0   +4N  +8N   ...  +124N
             ↓     ↓     ↓          ↓
           [128B][128B][128B] ... [128B]  = 32 transactions（32× 浪费）
```

访存效率公式：

```
访存效率 = (warp 实际需要的字节数) / (warp 实际搬运的字节数)
```

访问模式	warp 请求字节	实际搬运字节	效率	事务数
`A[tid]`（连续 float）	128 B	128 B	100%	1
`A[tid*2]`（stride 2）	128 B	256 B	50%	2
`A[tid*32]`（stride 32）	128 B	4096 B	3.1%	32
`A[tid]`（连续 double）	256 B	256 B	100%	2（每事务 128B）

两条铁律：1. **让 `threadIdx.x` 映射到内存中最快的维度**（行主序时即"列"，即最后一个下标）。2. **二维/三维数据展平后按一维连续寻址**，不要用嵌套索引跳着访问。

1.3 共享内存：bank 与 bank conflict

硬件结构：共享内存被划分为 **32 个 bank**，每个 bank 宽 **4 字节**。

```
共享内存地址（字节）:  0   4   8   12  ...  124 | 128  132 ...
bank 编号:             b0  b1  b2  b3  ...  b31 | b0   b1  ...
                       └──── 128 字节 = 一轮 ────┘

warp 内 32 个线程同时访问:
  若 32 个线程落在 32 个不同 bank  → 无冲突，1 个周期完成
  若 k 个线程落在同一个 bank       → k-way conflict，串行 k 个周期
  若 k 个线程访问同一 bank 的同一地址 → 广播（broadcast），仍 1 个周期 ✅
```

bank 编号公式：`bank = (字节地址 / 4) % 32`

经典冲突场景与修复：

场景	访问模式	冲突	修复
矩阵乘 `subTileN[k][tx]`	不同 k、同 tx → 地址 = k*(TILE+pad) + tx	无（跨行时地址差是 TILE 的倍数，若 TILE=32 则同 bank）	padding：`[TILE][TILE+1]`
列访问 `A[i][j]` 固定 j 变 i	地址 = i*32 + j → 同 bank j	**32-way** ❌	padding 或改用行访问
归约顺序寻址 `s[tid] += s[tid+h]`	地址连续、步长 h	无 ✅	—
归约交错寻址 `s[tid] += s[tid+1]`	线程 0,2,4… → bank 0,2,4…（2-way）	2-way 起，逐轮恶化	改顺序寻址

padding 原理（TILE_WIDTH = 32 时列访问的经典例子）：

```
__shared__ float A[32][32];      // ❌ 列访问 A[i][0] 全部落 bank 0 → 32-way conflict
__shared__ float A[32][32 + 1];  // ✅ 地址 = i*33 + 0，bank = (i*33)%32 = i → 无冲突
```

共享内存容量速查：

GPU	每 SM 共享内存	每 block 最大（需 opt-in）	默认每 block 上限
A100（`sm_80`）	164 KB	163 KB	48 KB
H100（`sm_90`）	228 KB	227 KB	48 KB
RTX 4090（`sm_89`）	100 KB	99 KB	48 KB

GPU	每 SM 共享内存	每 block 最大（需 opt-in）	默认每 block 上限
RTX 2080 Ti（`sm_75`）	64 KB	64 KB	48 KB

```
// 超过 48 KB 的静态共享内存必须动态分配 + opt-in
cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize, 96*1024);
kernel<<<grid, block, 96*1024>>>(...);          // 第三个参数 = 动态共享内存字节数
extern __shared__ float smem[];                 // kernel 内声明
```

1.4 常量内存

```
__constant__ float mask[9];                      // 声明（文件作用域）
cudaMemcpyToSymbol(mask, h_mask, 9*sizeof(float));// 主机端初始化
// kernel 内直接按 mask[j] 访问
```

常量缓存广播机制：当 warp 内 32 个线程访问**同一个地址**时，1 个周期完成广播； 若访问**不同地址**，则退化为串行（32 个不同地址 = 32 个周期）❌。→ **只适合"所有线程读同一个标量"的场景**，典型就是卷积的 mask 系数。

1.5 内存优化技巧清单

技巧	原理	收益量级
合并访问	32 次访问 → 1 次 128B 事务	最多 32×
共享内存 tiling	全局访问减少 TILE 倍	10–20×
padding 消除 bank conflict	32-way → 无冲突	最多 32×（共享内存侧）
`__restrict__`	告知编译器指针不重叠，允许寄存器缓存与重排	5–20%
`const __restrict__`	配合上一条，启用只读数据路径（LDG/纹理）	5–15%
结构体数组 → 数组结构（SoA）	避免 warp 内跨步访问结构体成员	2–10×
向量化 `float4`	每线程 1 次 16B 访存 = 128B/8 线程	2–4×
共享内存缓存 + 私有化	把全局原子竞争转为片上竞争	10–100×
`cp.async`（Ampere+）	计算当前 tile 时预取下一个 tile	10–30%
统一内存 `cudaMemPrefetchAsync`	提前迁移页，避免按需分页抖动	视场景
固定内存 `cudaMallocHost`	真 DMA 传输，避免 bounce buffer	传输带宽 2–3×

---

2. 线程组织

2.1 线程层次与索引

```
Grid（整个 kernel 的线程集合）
 ├── Block 0        ├── Block 1        ...  ├── Block gridDim.x-1
 │    ├── Warp 0 (32 threads)                 每 block 最多 1024 线程
 │    ├── Warp 1                              每 SM 最多 64 warps (A100/H100)
 │    └── ...                                 / 48 warps (Ada) / 32 (Turing)
 └── 每 block 有独立的共享内存

索引（三维）：
  threadIdx.{x,y,z}   ∈ [0, blockDim.{x,y,z})
  blockIdx.{x,y,z}    ∈ [0, gridDim.{x,y,z})
```

全局线程编号推导（一维）：

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

二维：

```
int col = blockIdx.x * blockDim.x + threadIdx.x;
int row = blockIdx.y * blockDim.y + threadIdx.y;
int idx = row * width + col;              // 行主序展平
```

三维：

```
int x = blockIdx.x*blockDim.x + threadIdx.x;
int y = blockIdx.y*blockDim.y + threadIdx.y;
int z = blockIdx.z*blockDim.z + threadIdx.z;
long long idx = (long long)x + (long long)y*W + (long long)z*W*H;  // 注意 64 位!
```

网格尺寸（向上取整）：

```
dim3 block(256);
dim3 grid((N + block.x - 1) / block.x);            // 1D
dim3 grid2((W + 15)/16, (H + 15)/16);              // 2D, block 16×16
```

⚠️ **整数除法陷阱**：`N / blockDim.x` 会漏掉尾部元素，必须向上取整并在 kernel 内做边界检查。

2.2 Block 大小选择

Block 大小	评价
32	仅 1 个 warp/block，SM 上 block 数上限（通常 24–32）会限制并发 warp 数 ❌
64 / 96	可用，但粒度过细，调度开销占比高
128 / 256	**最常用**，占用率与调度粒度平衡 ✅
512	适合计算密集型 kernel
1024	只有当 kernel 资源消耗很低（寄存器少、无共享内存）时才用；否则占用率骤降

选择准则： 1. block 大小必须是 **32 的整数倍**（否则最后一个 warp 有闲置 lane）； 2. 让**每个 SM 至少能驻留 8–16 个 block**（便于调度器换出）； 3. 用 `cudaOccupancyMaxPotentialBlockSize()` 自动求解：

```
int minGridSize, blockSize;
cudaOccupancyMaxPotentialBlockSize(&minGridSize, &blockSize, myKernel, 0, 0);
```

2.3 线程粗化（thread coarsening）

定义：让每个线程计算**多个**输出元素，而非一个。

```
粗化前 (coarsening factor = 1)        粗化后 (coarsening factor = 4)
thread t  → output[t]                  thread t → output[4t], output[4t+1],
                                                  output[4t+2], output[4t+3]

线程数 = N                             线程数 = N/4
```

收益来源： 1. **寄存器级数据复用**：4 个输出共享同一批加载的输入（尤其矩阵乘、卷积）； 2. **减少索引计算与循环开销**； 3. **提高 ILP**：4 条独立累加链可填满流水线，减少对 TLP 的依赖 → 允许更低占用率仍能隐藏延迟。

代价：线程数减少 → 若总线程数不足以填满 GPU，则并行度不足；寄存器压力上升可能降低占用率。

2.4 网格-步长循环（grid-stride loop）

```
__global__ void addGridStride(const float* __restrict__ A,
                              const float* __restrict__ B,
                              float* __restrict__ C, int N) {
    int stride = gridDim.x * blockDim.x;
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < N; i += stride) {
        C[i] = A[i] + B[i];
    }
}
```

优点： – 线程总数与问题规模**解耦**（grid 可固定为"刚好填满 GPU"的大小）； – 自动获得粗化收益（每线程多次迭代复用索引计算与寄存器）； – 天然支持任意 N，且便于做多 GPU 拆分与负载均衡； – 对**未知规模**的输入特别友好（无需重算 grid）。

---

3. 同步与通信

3.1 同步原语速查

API	作用域	语义	典型用途
`__syncthreads()`	block	屏障：所有线程到达后继续；同时保证共享/全局内存可见性	tiled matmul 的两次屏障
`__syncwarp(mask)`	warp	warp 内线程同步 + 内存栅栏	Volta+ 上 warp 内通信前
`__threadfence()`	设备	保证本线程之前的写对全设备可见（不阻塞）	全局内存生产者–消费者
`__threadfence_block()`	block	同上，block 范围	—
`cudaDeviceSynchronize()`	host↔device	阻塞主机直到所有先前工作完成	kernel 后计时/取回结果
`cudaStreamSynchronize(s)`	host↔stream	只等某个流	多流流水线
`cudaEventSynchronize(e)`	host↔event	等到某事件	精细计时
`cudaStreamWaitEvent(s,e)`	stream↔stream	流 s 等事件 e	跨流依赖

3.2 `__syncthreads()` 三条铁律

1. **必须被 block 内所有线程执行**——不能放在 `if (tid < n)` 之类的分支里，否则死锁或未定义行为。 `cpp // ❌ 错误 if (tid < 128) { doWork(); __syncthreads(); } // ✅ 正确 if (tid < 128) { doWork(); } __syncthreads();`
2. **不要放在可能提前 return 的路径之后**（除非所有线程都走同一路径）。
3. **它只同步不互斥**——不能用来保护临界区，只保证"所有线程都到齐 + 内存可见"。

3.3 共享内存数据竞争的三种形态

在 tiled 循环中，每个 tile 迭代必须用**两次**屏障夹住：

```
for (int m = 0; m < numTiles; ++m) {
    As[ty][tx] = A[...];          // 写共享内存
    Bs[ty][tx] = B[...];
    __syncthreads();              // 屏障 1：等所有人写完
    for (int k = 0; k < TILE; ++k) sum += As[ty][k] * Bs[k][tx];
    __syncthreads();              // 屏障 2：等所有人读完，才能覆盖
}
```

省略的屏障	后果
屏障 1	**RAW 竞争**：读到别人还没写入的旧值 → 结果错误
屏障 2	**WAR 竞争**：下一轮写入覆盖了别人还没读完的数据 → 结果错误（且随机）

> 记忆法：屏障数 = 2 × （tile 迭代数），**少一个都会算错**。

3.4 原子操作

API	语义	支持的精度
`atomicAdd(addr, val)`	`*addr += val`	int, uint, ull, float, double, half2
`atomicSub` / `atomicExch`	减 / 交换	int, uint, ull, float
`atomicMin` / `atomicMax`	最小 / 最大	int, uint, ull, longlong
`atomicInc` / `atomicDec`	环回加减	uint
`atomicCAS(addr, cmp, val)`	比较并交换	全部（用来实现任意原子操作）
`atomicAnd/Or/Xor`	位运算	int, uint, ull

性能特征：同一地址的原子操作被**硬件串行化**。 `N` 个线程对同一地址做 `atomicAdd` → `N` 个串行周期量级。

私有化（privatization）模式（直方图/归约的标准手法）：

```
__global__ void histogramPrivatized(const unsigned char* img, unsigned int* hist, int n) {
    __shared__ unsigned int localHist[256];
    for (int i = threadIdx.x; i < 256; i += blockDim.x) localHist[i] = 0;
    __syncthreads();

    int stride = gridDim.x * blockDim.x;
    for (int i = blockIdx.x*blockDim.x + threadIdx.x; i < n; i += stride)
        atomicAdd(&localHist[img[i]], 1u);        // 竞争被限制在片上，快 ~100×

    __syncthreads();
    for (int i = threadIdx.x; i < 256; i += blockDim.x)
        atomicAdd(&hist[i], localHist[i]);        // 全局原子次数 = 256 × #blocks（极少）
}
```

---

4. warp 级原语

4.1 shuffle 指令（`sm_30+`，Volta 后必须带 mask）

```
// 从 lane（laneID + delta）取值；若越界则返回本线程自己的值
__shfl_down_sync(mask, var, delta, width=32)
// 从 lane（laneID − delta）取值
__shfl_up_sync(mask, var, delta, width=32)
// 从任意 lane srcLane 取值
__shfl_sync(mask, var, srcLane, width=32)
// 蝴蝶交换：laneID ^ laneMask
__shfl_xor_sync(mask, var, laneMask, width=32)
```

原语	作用
`__ballot_sync(mask, pred)`	返回 32 位掩码，第 i 位 = lane i 的谓词
`__any_sync` / `__all_sync`	warp 内任一 / 全部为真
`__activemask()`	当前活跃 lane 的掩码（**谨慎使用**，语义微妙）
`__match_any_sync` / `__match_all_sync`	找出值相等的 lane（`sm_70+`）
`__reduce_add_sync` / `__reduce_min_sync` / `__reduce_max_sync`	**硬件归约**（`sm_80+`，int/uint only）

4.2 warp 内归约的标准写法

```
// 前 5 轮（32→1）用 shuffle，无需 __syncthreads()，无需共享内存
__device__ float warpReduceSum(float val) {
    for (int offset = 16; offset > 0; offset >>= 1)
        val += __shfl_down_sync(0xffffffff, val, offset);
    return val;              // lane 0 持有 warp 总和
}

// block 级：每 warp 归约后由 lane 0 写共享内存（每 warp 只写 1 个元素 → 无 bank conflict）
__device__ float blockReduceSum(float val) {
    __shared__ float warpSums[32];
    int lane = threadIdx.x & 31;
    int wid  = threadIdx.x >> 5;

    val = warpReduceSum(val);
    if (lane == 0) warpSums[wid] = val;
    __syncthreads();

    int nWarps = (blockDim.x + 31) >> 5;
    val = (threadIdx.x < nWarps) ? warpSums[threadIdx.x] : 0.0f;
    if (wid == 0) val = warpReduceSum(val);
    return val;
}
```

为什么快： – shuffle 不访问内存，1 个周期完成一次交换，**完全不占共享内存带宽**； – 每 warp 只写 1 个值到共享内存 → **零 bank conflict**； – 相比"全程用共享内存"版本，共享内存访问次数减少约 `log2(32) = 5` 倍。

⚠ **mask 正确性**：mask 必须精确描述参与该操作的 lane 集合；在 Volta+ 上 independent thread scheduling 使得"隐式 warp 同步"不再成立，必须用 `_sync` 变体。

---

5. 七种并行模式速查

模式	核心思想	关键 CUDA 机制	算术强度	瓶颈类型	首要优化
Element-wise（向量加法）	1 线程 1 元素	一维映射 + 边界检查	0.125	带宽	合并访问、`float4`、grid-stride
Dense matmul（朴素）	1 线程 1 输出，直接读全局	二维映射	0.25	带宽（跨步访存）	转置 B / tiling
Tiled matmul	tile 载入共享内存复用	`__shared__` + 2×`__syncthreads()`	TILE/4 = 4–8	从带宽转计算	大 TILE、padding、粗化
Reduction	树形两两合并	共享内存树 / shuffle	~0.06	带宽	顺序寻址、warp shuffle、粗化
Scan	双缓冲对数步长前缀和	双缓冲 + 多轮屏障	~0.1	带宽	层次化、work-efficient 权衡
Convolution（tiled）	共享内存 halo 复用	`__constant__` + halo 加载	~1（朴素）→ TILE/8	带宽/计算混合	常量内存、halo、粗化
SpMV（CSR）	每行一个线程	压缩格式 + gather	~0.167	带宽 + 不规则	ELL/向量化、x 入共享内存

5.1 各模式的"全局访存减少量"

模式	朴素版每输出元素访存	优化版	减少倍数
Tiled matmul (N×N)	`2N`	`2N / TILE_WIDTH`	**TILE_WIDTH×**（16 或 32）
Tiled convolution (1D, mask r)	`2r+1`	`(TILE+2r) / TILE`	约 **TILE/(2r+1)**
Reduction	`N` 次读（1 次/元素）	同（无法减少）	1×（本就最优，靠延迟隐藏）
SpMV	—	—	靠格式规则化而非减少字节数

5.2 tiled 矩阵乘法：访存减少的定量推导

```
朴素版：输出 C 的每个元素需要
  – 读 A 的一整行：K 次
  – 读 B 的一整列：K 次
  合计 2K 次全局访存 / 输出元素
  → 总量 = 2·M·N·K 次

分块版（TILE_WIDTH = T）：
  – C 被划分为 (M/T)·(N/T) 个 tile，每个 tile 由 1 个 block 计算
  – 每个 block 需要载入 A 的 M/T... 具体为 (M/T)·(K/T) 个 A-tile + (K/T)·(N/T) 个 B-tile
  – 每个 tile 载入一次，被 block 内 T² 个线程各使用 T 次
  → 每个输出元素的全局访存 = 2K / T
  → 总量 = 2·M·N·K / T

算术强度 = FLOPs / Bytes
        = 2·M·N·K / (4·(2·M·N·K/T))     [4 字节/float]
        = T / 4  FLOP/Byte

  T = 16 → 4.0 FLOP/Byte    （低于 A100 平衡点 12.5 → 仍带宽受限）
  T = 32 → 8.0 FLOP/Byte    （接近平衡点）
  T = 64 → 16.0 FLOP/Byte   （超过平衡点 → 计算受限，但共享内存不够）
```

5.3 归约：warp 发散对比表

设 block = 256 线程，用共享内存归约，`tid` = 线程在 block 内的编号。

轮次	交错寻址 `s += s[tid + stride]`（stride = 2^r）	顺序寻址 `s += s[tid + h]`（h = blockDim/2^r）
	活跃线程（`tid % (2*stride) == 0`）	活跃线程（`tid < h`）
1	128（warp 0–3 活跃，但**每 warp 内部只有一半 lane 活跃 → 50% 发散**）	128（warp 4–7 全部空闲，warp 0–3 **每 lane 都活跃 → 无发散**）
2	64（发散 75%）	64
3	32（发散 87.5%）	32
4	16（只剩 1/2 warp，浪费 31/32 执行槽）	16
5	8	8

轮次	交错寻址 `s += s[tid + stride]` (stride = 2^r)	顺序寻址 `s += s[tid + h]` (h = blockDim/2^r)
6	4	4
7	2	2
8	1	1

结论：顺序寻址让"每轮被砍掉的是一整个 warp"，保留的 warp 内部完全整齐 → **无分歧、访存合并**。 加上 warp shuffle 后，前 5 轮完全不访问内存。

5.4 扫描：Hillis–Steele vs Blelloch

维度	Hillis–Steele (Kogge–Stone)	Blelloch (work–efficient)
步数（深度）	`log2(N)`	`2·log2(N)` (up–sweep + down–sweep)
工作量	`O(N log N)`	`O(N)`
并行度	每步 `N/2` 个活跃操作	up–sweep 递减，down–sweep 递增
输出	inclusive scan（直接得）	exclusive scan（天然）
共享内存	需**双缓冲**避免 RAW/WAR	原地可做（up/down 两阶段）
GPU 上偏好	✅ **通常更快**（ALU 富余，带宽是瓶颈）	理论更优雅，但步数多、同步多
适用	单 block、数据量适中	大 N、需 work efficiency 的场合

多 block 层次化扫描（三步法）：

```
Step 1: 每个 block 独立扫描自己的 chunk，把 chunk 总和写入 blockSums[b]
Step 2: 对 blockSums 做一次扫描（单个 block 完成，或递归）
Step 3: 每个 block 把 blockSums 的前缀和（exclusive）加到自己的结果上
```

总访存 = 读 2 遍 + 写 2 遍 = `4N×4` 字节（N 个 float）→ 理论时间下限 `≈ 16N 字节 / 1555 GB/s`。

---

6. 性能分析公式

6.1 核心公式集

```
【算术强度】
 Arithmetic Intensity (AI) = FLOPs executed / Bytes transferred
                           [FLOP/Byte]

【机器平衡点】
 Machine Balance = Peak FLOPS / Peak Bandwidth        [FLOP/Byte]
 A100: 19.5e12 / 1555e9 = 12.5 FLOP/Byte

【Roofline 性能上限】
 Attainable FLOPS = min( Peak FLOPS ,  AI × Peak Bandwidth )

 AI <  Machine Balance  →  带宽受限 (memory bound)
 AI >  Machine Balance  →  计算受限 (compute bound)

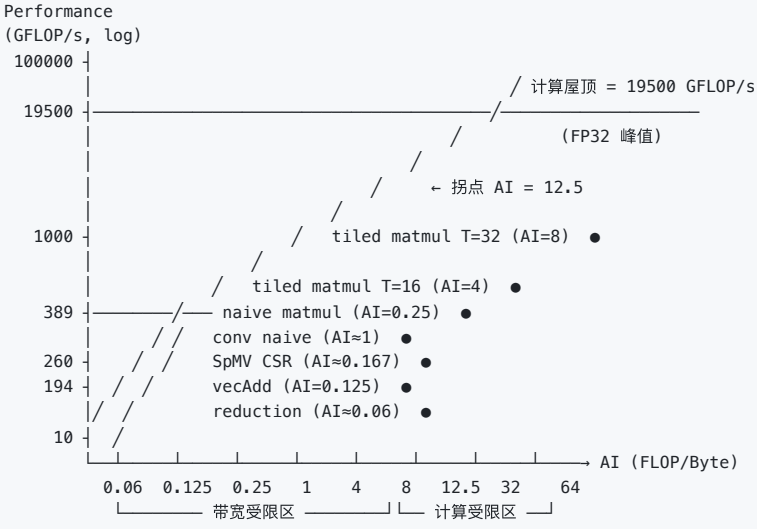
【有效带宽】
 Effective BW = (Bytes actually transferred) / (kernel time)   [GB/s]
 % of peak    = Effective BW / Peak Bandwidth × 100%

【加速比】
 Speedup = T_serial / T_parallel
 Amdahl: S(n) = 1 / ( (1-p) + p/n )     p = 可并行比例

【占用率】
 Occupancy = (active warps per SM) / (max warps per SM) × 100%
 限制因素取以下最小值:
   regs:      floor(65536 / (regs_per_thread × blockDim)) × blockDim / 2048
   smem:      floor(smem_per_SM / smem_per_block) × blockDim / 2048
   blocks:    max_blocks_per_SM × blockDim / 2048
   warps:     max_warps_per_SM / 2048 × 2048    (A100 = 64 warps = 2048 threads)

【Little's Law (延迟隐藏所需并发度)】
 Required Concurrency = Latency × Throughput
 例: DRAM 延迟 500 cycles, SM 每周期可发 1 次访存
     → 需要约 500 个在途访存才能打满带宽
     → 若每线程 1 个在途访存, 需要 ≈ 500 线程 ≈ 16 warps 常驻
```

6.2 Roofline 图（ASCII，基准机 A100）




```
 内存屋顶 = 1555 GB/s (斜率 = 带宽)
 带宽受限区内的上限 = AI × 1555 GFLOP/s
```

读图要点：本课程几乎所有的 kernel 都落在**带宽受限区**（左侧斜坡）。 这就是为什么全书的优化主线是"减少字节数、让字节规律化"，而不是"减少浮点运算次数"。

6.3 各模式的理论性能上限（A100）

Kernel	AI (FLOP/Byte)	理论上限 (GFLOP/s)	占 FP32 峰值
Reduction	0.06	93	0.5%
vecAdd	0.125	194	1.0%

Kernel	AI (FLOP/Byte)	理论上限 (GFLOP/s)	占 FP32 峰值
SpMV (CSR)	0.167	260	1.3%
Naive matmul	0.25	389	2.0%
Convolution (naive)	~1.0	1555	8.0%
Tiled matmul T=16	4.0	6220	32%
Tiled matmul T=32	8.0	12440	64%
Tiled matmul T=64	16.0	19500（计算受限）	100%

> ⚠️ 上表是**理想上限**（假设 100% 带宽利用、无冲突、无延迟损失）。实际达到 60–80% 已属优秀。 注意：**低 AI 不代表"这个 kernel 没价值"**——它只说明性能天花板低， 而优化目标应定为"尽可能逼近自己的天花板"（见 6.4 的达成率指标）。

6.4 衡量优化好坏的三个指标

```
1. 带宽达成率 = 实测带宽 / 峰值带宽       ← 带宽受限 kernel 看这个
2. 算力达成率 = 实测 GFLOP/s / 峰值       ← 计算受限 kernel 看这个
3. Roofline 达成率 = 实测 / min(峰值, AI×带宽)  ← 通用，最推荐
   目标：> 60% 为良好，> 80% 为优秀
```

---

7. 性能诊断流程

7.1 六步诊断法

```
┌──────────────────────────────────────────────────────────────┐
│ Step 1  建立正确基线                                          │
│   · CPU golden reference + 容差比对（相对误差 < 1e-4 常用）    │
│   · 先跑通，再跑快。正确性不过关，性能数字毫无意义。           │
├──────────────────────────────────────────────────────────────┤
│ Step 2  测量（务必先预热 + 多次取平均）                       │
│   · cudaEvent 计时，排除首次启动开销与 H2D/D2H 拷贝           │
│   · 记录：耗时、有效带宽 GB/s、GFLOP/s                       │
├──────────────────────────────────────────────────────────────┤
│ Step 3  计算算术强度 → 定位 Roofline 区域                     │
│   · AI = FLOPs / Bytes，与 12.5 FLOP/Byte 比较               │
│   · 得出"天花板 GFLOP/s"和"当前达成率"                       │
├──────────────────────────────────────────────────────────────┤
│ Step 4  检查资源使用                                          │
│   · nvcc --ptxas-options=-v  → 寄存器数、共享内存、spill      │
│   · cudaOccupancyMaxActiveBlocksPerMultiprocessor → 理论占用率 │
│   · 寄存器 > 64/线程 或 共享内存吃满 → 占用率是嫌疑犯         │
├──────────────────────────────────────────────────────────────┤
│ Step 5  Profiler 定位 stall 原因                              │
│   · ncu --set full ./prog                                    │
│   · 关键指标：dram__throughput.avg.pct_of_peak_sustained_elapsed │
│               l1tex__data_bank_conflicts_pipe_lsu_mem_shared  │
│               smsp__average_warps_issue_stalled_{long_scoreboard, │
│                 short_scoreboard, barrier, branch_resolving}  │
│                                                              │
│   stall 原因 → 对策：                                         │
│     long_scoreboard  (等全局内存) → 提高占用率/预取/改善合并   │
│     short_scoreboard (等共享内存) → 消除 bank conflict        │
│     barrier          (等屏障)     → 平衡 warp 工作量、减少同步次数 │
│     branch_resolving(分支发散)    → 消除分歧、重构数据布局    │
│     not_selected     (调度器没选它) → 说明已饱和，可接受      │
├──────────────────────────────────────────────────────────────┤
│ Step 6  一次只改一个变量 → 回到 Step 2，记录到优化日志        │
└──────────────────────────────────────────────────────────────┘
```

7.2 瓶颈类型 → 优化手段映射

瓶颈判定依据	瓶颈类型	首选优化手段
带宽达成率 > 70%，AI < 平衡点	**带宽受限**	减少字节数（tiling/复算）、改善合并、压缩数据格式（FP16/量化）、向量化

瓶颈判定依据	瓶颈类型	首选优化手段
算力达成率 > 70%，AI > 平衡点	**计算受限**	减少冗余运算、使用更快的指令（FMA/`__fmaf_rn`）、降低精度、Tensor Core
带宽/算力达成率都低，stall = long_scoreboard	**延迟受限**	提高占用率、增加 ILP/粗化、软件预取（`cp.async`）、增加在途访存
bank conflict 指标高	**共享内存冲突**	padding、改变访问模式
barrier stall 高	**同步开销**	减少 `__syncthreads()` 次数、warp shuffle 替代、平衡负载
理论占用率低（< 25%）	**资源限制**	`__launch_bounds__` 限寄存器、减小 tile、`-maxrregcount`
占用率高但仍慢	**ILP 不足 / 访存不规则**	线程粗化增加寄存器复用、改数据布局（SoA）、格式规则化（CSR→ELL）

7.3 优化日志模板（最终项目报告的核心素材）

#	优化内容	实测耗时	GFLOP/s	带宽达成率	相对上一版加速比	瓶颈判断	结论
0	基线（CPU 串行参考）	1250 ms	0.7	—	1.00×	计算受限	建立正确性基准
1	朴素 GPU kernel	18.5 ms	46	12%	67.6×	带宽（跨步访存）	跨步访问是主因
2	+ 转置 B 使访问合并	4.2 ms	203	52%	4.4×	带宽	合并访存收益显著
3	+ 共享内存 tiling T=16	1.9 ms	449	—	2.2×	带宽（AI 提升到 4）	tiling 有效
4	+ padding 消 bank conflict	1.5 ms	569	—	1.27×	共享内存冲突	冲突确实存在
5	+ TILE=32 且粗化因子 4	0.85 ms	1004	—	1.76×	转向计算受限	寄存器复用生效
6	+ `__restrict__` + `-O3`	0.79 ms	1080	—	1.08×	—	边际收益
7	尝试 `cp.async` 预取	0.81 ms	1053	—	0.98×	—	**无提升，回退**（诚实记录）

> 最后一行正是官方目标 C17（identify limitations）与 C16（justify the final decision）所要求的：**失败的尝试也要如实记录**，它证明了设计空间探索的真实性。

---

8. CUDA API 速查

8.1 内存管理

```
// 设备内存
cudaMalloc(&d_ptr, bytes);
cudaFree(d_ptr);
cudaMemset(d_ptr, 0, bytes);

// 主机内存
cudaMallocHost(&h_ptr, bytes);        // 固定内存（page-locked），支持真 DMA，带宽 2–3×
cudaFreeHost(h_ptr);

// 统一内存（按需分页）
cudaMallocManaged(&ptr, bytes);
cudaMemPrefetchAsync(ptr, bytes, deviceId, stream);   // 提前迁移，避免页错误抖动

// 二维/三维
cudaMallocPitch(&d_ptr, &pitch, widthBytes, height);
cudaMalloc3D(&d_ptr, make_cudaExtent(w, h, d));

// 符号（__constant__ / __device__ 变量）
cudaMemcpyToSymbol(symbol, h_src, bytes);
cudaMemcpyFromSymbol(h_dst, symbol, bytes);
```

8.2 数据传输

```
cudaMemcpy(d_dst, h_src, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(h_dst, d_src, bytes, cudaMemcpyDeviceToHost);
cudaMemcpy(d_dst, d_src, bytes, cudaMemcpyDeviceToDevice);
cudaMemcpyAsync(d_dst, h_src, bytes, cudaMemcpyHostToDevice, stream);  // 需固定内存才真异步
cudaMemcpy2D(...); cudaMemcpy3D(...);
```

⚠️ **方向写错的代价**：`cudaMemcpyHostToDevice` 写成 `DeviceToHost` 不会报错但结果全错 （在统一寻址的 GPU 上尤其隐蔽）。**永远检查返回值**。

8.3 Kernel 执行与查询

```
kernel<<<grid, block, sharedBytes, stream>>>(args...);

cudaGetLastError();                       // 检查启动参数错误（同步，开销小）
cudaPeekAtLastError();                    // 不重置错误
cudaDeviceSynchronize();                  // 等待全部完成
cudaStreamSynchronize(stream);
cudaGetErrorString(err);

// 设备查询
cudaGetDeviceCount(&count);
cudaGetDeviceProperties(&prop, 0);
//   prop.name, prop.major/.minor, prop.multiProcessorCount,
//   prop.maxThreadsPerMultiProcessor, prop.maxThreadsPerBlock,
//   prop.sharedMemPerMultiprocessor, prop.sharedMemPerBlock,
//   prop.regsPerMultiprocessor, prop.warpSize,
//   prop.memoryClockRate (kHz), prop.memoryBusWidth (bits),
//   prop.totalGlobalMem, prop.l2CacheSize, prop.concurrentKernels

// 理论峰值带宽计算
double bw = 2.0 * prop.memoryClockRate * 1e3 * (prop.memoryBusWidth / 8) / 1e9;  // GB/s

// kernel 资源
cudaFuncAttributes attr; cudaFuncGetAttributes(&attr, myKernel);
//   attr.numRegs, attr.sharedSizeBytes, attr.localSizeBytes, attr.maxThreadsPerBlock

// 占用率
int numBlocks;
cudaOccupancyMaxActiveBlocksPerMultiprocessor(&numBlocks, myKernel, blockSize, dynamicSmem);
int minGrid, blockSizeOpt;
cudaOccupancyMaxPotentialBlockSize(&minGrid, &blockSizeOpt, myKernel, dynamicSmemFn, 0);
```

8.4 流与事件

```
cudaStream_t s; cudaStreamCreate(&s);
cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking);
cudaStreamDestroy(s);

cudaEvent_t e0, e1;
cudaEventCreate(&e0); cudaEventCreate(&e1);
cudaEventRecord(e0, s);
kernel<<<g,b,0,s>>>(...);
cudaEventRecord(e1, s);
cudaEventSynchronize(e1);
float ms; cudaEventElapsedTime(&ms, e0, e1);   // 毫秒

cudaStreamWaitEvent(s2, e1, 0);               // 流间依赖
```

8.5 标准错误检查宏

```
#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err__));                             \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

// kernel 启动后立即检查（异步错误在下次同步时才会浮现）
kernel<<<g, b>>>(...);
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());
```

---

9. 编译器与构建选项

9.1 常用 nvcc 选项

选项	作用
`-O3`	主机端优化（默认 `-O3` 对设备代码生效）
`-arch=sm_80`	只生成 sm_80 的 SASS（编译快，只能在该架构跑）
`-gencode arch=compute_80,code=sm_80`	精确控制 PTX/SASS 生成
`-arch=native`	自动匹配本机 GPU（CUDA 11.5+）
`--ptxas-options=-v`	**打印寄存器数、共享内存、spill** —— 诊断必用
`-Xptxas -dlcm=cg`	全局访存绕过 L1（只走 L2）
`-Xptxas -dlcm=ca`	全局访存经 L1 缓存
`--maxrregcount=N`	限制每线程寄存器数（可能引入 spill）
`-lineinfo`	生成行号信息，供 profiler 使用
`-G`	设备端调试信息（**严重降低性能，仅调试用**）
`--use_fast_math`	用低精度快速数学函数（**会损失精度，谨慎**）
`-std=c++17`	C++ 标准
`--generate-line-info -lineinfo`	Profiler 建议
`-Xcompiler -fopenmp`	传递选项给主机编译器
`-ccbin g++-12`	指定主机编译器

9.2 `__launch_bounds__`

```
// 告诉编译器：block 最多 256 线程，且希望每 SM 至少驻留 4 个 block
__global__ void __launch_bounds__(256, 4) myKernel(...);

// 作用：编译器据此约束寄存器分配（256×4 = 1024 线程 → 最多 64 regs/线程）
// 代价：可能产生寄存器 spill（用 -Xptxas -v 检查）
```

9.3 性能相关编译标志速查

```
nvcc -O3 -arch=sm_80 -lineinfo --ptxas-options=-v kernel.cu -o kernel
ncu --set full --launch-count 1 ./kernel          # Nsight Compute 全指标
ncu --metrics dram__throughput.avg.pct_of_peak_sustained_elapsed,\
l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum ./kernel
nsys profile --stats=true ./kernel                # 时间线分析（含 H2D/D2H）
```

---

10. 硬件参数速查

10.1 计算能力（compute capability）

CC	架构	代表 GPU	每 SM warp 上限	每 SM 线程上限	每 block 线程上限	共享内存/SM	关键特性
7.5	Turing	RTX 2080 Ti, T4	32	1024	1024	64 KB	独立线程调度
8.0	Ampere (GA100)	A100	64	2048	1024	164 KB	`cp.async`, TF32, `__reduce_*_sync`
8.6	Ampere (GA10x)	RTX 3090, A10	48	1536	1024	100 KB	同上（少部分差异）
8.9	Ada	RTX 4090, L40	48	1536	1024	100 KB	FP8
9.0	Hopper	H100	64	2048	1024	228 KB	TMA, thread block cluster, DPX

10.2 内存层次数量级（A100）

层次	容量	延迟	带宽
寄存器文件	108 SM × 256 KB = 27 MB	1 cycle	—
共享内存/L1	108 × 164 KB ≈ 17.7 MB	20–30 cycles	~19 TB/s 聚合
L2	40 MB	~200 cycles	~5–7 TB/s
HBM2e (80GB)	80 GB	400–800 cycles	1555 GB/s（实测 ~1.4 TB/s 可达）

10.3 事务与粒度

项目	数值
全局内存事务粒度	128 字节（一个 warp 的 32×4B 连续访问）
L2 cache line	128 字节
DRAM burst	32 字节（这是"访问 4 字节却搬 32 字节"浪费的根源）
共享内存 bank 宽度	4 字节 × 32 banks = 128 字节/轮
常量缓存	8 KB/SM，广播 1 周期

---

11. CUDA → WebGPU/WGSL 对照

ECE408 近年引入 **WebGPU**（Lab）与 **RAI**（最终项目）作为 CUDA 之外的编程路径。 以下是逐项对照，便于有 CUDA 基础者迁移。

概念	CUDA	WebGPU / WGSL
计算程序	`__global__ void kernel(...)`	`@compute @workgroup_size(x,y,z) fn main(...)`
线程层次	grid → block → thread	dispatch → workgroup → invocation
线程索引	`blockIdx`, `threadIdx`	`@builtin(global_invocation_id)`, `@builtin(local_invocation_id)`
工作组索引	`blockIdx`	`@builtin(workgroup_id)`
组内线程数	`blockDim.x`（≤1024）	`@workgroup_size`（≤256 常见）
调度	`kernel<<<grid, block>>>()`	`computePass.dispatchWorkgroups(gx, gy, gz)`
warp / SIMT	warp = 32 线程	subgroup（大小由实现决定，常为 32/64）
片上内存	`__shared__ float s[N];`	`var<workgroup> s: array<f32, N>;`
组内同步	`__syncthreads()`	`workgroupBarrier()`
全局内存	`cudaMalloc` + raw pointer	`device.createBuffer()` + `GPUBuffer`
读写缓冲	指针解引用 `A[i]`	`var<storage, read_write> A: array<f32>;`
常量/参数	`__constant__` / kernel 参数	`var<uniform>` / `var<storage, read>`
数据上传	`cudaMemcpy`	`device.queue.writeBuffer()`
数据回读	`cudaMemcpy` D2H	`GPUBuffer` + `mapAsync`
异步并行	stream + `cudaMemcpyAsync`	queue + 多个 command encoder
原子操作	`atomicAdd(&x, v)`	`atomicAdd(&x, v)`
计时	`cudaEvent`	`performance.now()` + `queue.onSubmittedWorkDone()`
错误检查	`cudaGetErrorString`	`device.pushErrorScope()` / `popErrorScope()`
着色器语言	CUDA C++	**WGSL**（WebGPU Shading Language）

WGSL 向量加法 compute shader（与 CUDA 版本逐行对照）：

```
// 文件: vector_add.wgsl
@group(0) @binding(0) var<storage, read>       A : array<f32>;   // 对应 const float* A
@group(0) @binding(1) var<storage, read>       B : array<f32>;   // 对应 const float* B
@group(0) @binding(2) var<storage, read_write> C : array<f32>;   // 对应 float* C

// 对应 CUDA 的 kernel 参数 N（统一变量）
struct Params { N : u32 };
@group(0) @binding(3) var<uniform> params : Params;

@compute @workgroup_size(256)                       // 对应 <<<grid, 256>>>
fn main(@builtin(global_invocation_id) gid : vec3<u32>) {   // 对应 blockIdx*blockDim + threadIdx
    let i = gid.x;                                  // 全局线程编号
    if (i >= params.N) { return; }                  // 边界检查（与 CUDA 完全一致）
    C[i] = A[i] + B[i];                             // 合并访存（同样关键）
}
```

关键差异（迁移时必须注意）： 1. **WGSL 是强类型且无指针**——用 `array<f32>` + 索引，不能做指针算术； 2. **没有 `__syncthreads()` 的等价全局屏障**——`workgroupBarrier()` 只同步工作组内； 3. **绑定（binding）是显式的**——每个 buffer 必须在 `@group(n) @binding(m)` 中声明， 并在主机端用 `GPUBindGroupLayout` 精确匹配； 4. **workgroup 大小上限通常为 256**（比 CUDA 的 1024 小）； 5. **没有 `cudaMemcpy` 的同步语义**——所有传输都通过 queue 命令，天然异步， 必须用 `mapAsync` 才能回读结果； 6. **整数除零/越界行为不同**——WGSL 有明确的边界检查语义，越界索引返回 0/NULL 而非未定义行为。

---

12. 常见陷阱速查

#	陷阱	症状	正确做法
1	忘记 `cudaDeviceSynchronize()` / 事件同步	计时错误（测得接近 0）或读到未完成的结果	kernel 后 `CUDA_CHECK(cudaGetLastError())` + 计时时同步
2	未检查 CUDA API 返回值	静默失败，结果全 0 或全 NaN	全程使用 `CUDA_CHECK` 宏
3	忘记 `__syncthreads()`	结果随机错误	tiled 循环中每个 tile 两次屏障
4	`__syncthreads()` 放在分支内	挂死或未定义行为	移到分支外，让所有线程都执行
5	共享内存 bank conflict	性能仅为预期 1/2 — 1/32	padding `[N][N+1]`，或改访问模式
6	全局内存跨步访问（未合并）	带宽达成率 < 15%	让 `threadIdx.x` 对应最内层维度
7	整数除法导致网格覆盖不足	结果尾部元素未计算（常为 0）	`(N + block - 1) / block`
8	缺少边界检查	段错误或结果错乱	kernel 内 `if (idx < N)`
9	`cudaMemcpy` 方向写错	无报错但结果全错	核对 `cudaMemcpyHostToDevice` / `DeviceToHost`
10	host 指针传给 kernel	段错误或 `invalid device pointer`	只传 `cudaMalloc` 得到的设备指针
11	共享内存超限（>48KB 静态）	启动失败 `invalid argument`	动态共享内存 + `cudaFuncSetAttribute` opt-in
12	占用率过低（寄存器爆表）	延迟无法隐藏，性能差	`-Xptxas -v` 查看，用 `__launch_bounds__` 约束
13	占用率盲目求高	性能反而下降（ILP 被牺牲）	30–50% 常已足够；优先保 ILP
14	warp 发散	分支两侧串行执行	重构数据布局，或用谓词化（predication）
15	原子操作同地址竞争	性能随 N 线性变差	**私有化**：先片上累加再合并
16	浮点累加顺序不同	结果与 CPU 参考不一致	用容差比对（相对误差 1e-4），而非精确相等
17	`-G` 调试编译用于性能测试	性能差 10–100×	性能测试必须去掉 `-G` 和 `-lineinfo` 之外的调试选项
18	三维索引未用 64 位	大数组索引溢出为负 → 越界	`long long idx = x + (long long)y*W + ...`
19	计时把 H2D/D2H 算进去	低估 kernel 性能	只对 kernel 前后打 event
20	首次启动开销未排除	大 kernel 尚可，小 kernel 严重失真	预热 1–3 次后再计时
21	`--use_fast_math` 悄悄降精度	容差测试失败	正确性验证时关闭；确认可接受后再开
22	网格-步长循环里忘了 `+= stride` 写成 `+= 1`	重复计算，性能与结果皆错	严格检查循环增量
23	共享内存数组未初始化就累加	垃圾值参与运算	显式清零 + `__syncthreads()`
24	多 stream 时误用默认流	隐式同步，重叠失效	用 `cudaStreamNonBlocking` 非默认流
25	异步拷贝用了普通页内存	退化为同步拷贝，无重叠	`cudaMallocHost` 固定内存

13. 一页纸终极速查（考前复习）

```
┌──────────────────────────────────────────────────────────────────┐
│ 【判断瓶颈】                                                      │
│  AI = FLOPs / Bytes                                              │
│  AI < 12.5 (A100)  → 带宽受限：减少字节数、合并访问、tiling、压缩格式      │
│  AI > 12.5         → 计算受限：减少运算、FMA、Tensor Core、降精度         │
│  两者都低 + stall 高 → 延迟受限：提高占用率、增加 ILP/粗化                 │
├──────────────────────────────────────────────────────────────────┤
│ 【优化优先级】(性价比从高到低)                                      │
│  1. 合并访存 (最多 32×)                                           │
│  2. 共享内存 tiling (10–20×)                                      │
│  3. 消除 bank conflict (最多 32×, 共享内存侧)                      │
│  4. 线程粗化 / 寄存器复用 (1.5–3×)                                 │
│  5. 原子操作私有化 (10–100×)                                       │
│  6. 向量化 float4 (2–4×)                                          │
│  7. `__restrict__` (5–20%)                                       │
│  8. 流重叠传输与计算 (最高 2×)                                     │
├──────────────────────────────────────────────────────────────────┤
│ 【必背公式】                                                      │
│  i = blockIdx.x * blockDim.x + threadIdx.x                       │
│  grid = (N + block - 1) / block                                  │
│  bank = (byteAddr / 4) % 32                                      │
│  合并事务 = 128 字节 = 32 threads × 4 bytes                        │
│  算术强度(tiled matmul) = TILE_WIDTH / 4                           │
│  全局访存减少(tiled matmul) = TILE_WIDTH 倍                        │
│  性能上限 = min(PeakFLOPS, AI × PeakBW)                           │
│  所需并发度 = 延迟 × 吞吐率 (Little's Law)                         │
│  加速比 = T_serial / T_parallel; Amdahl: 1/((1-p)+p/n)           │
├──────────────────────────────────────────────────────────────────┤
│ 【三条黄金准则】                                                  │
│  ① 正确性优先：先有 golden reference 和容差比对，再谈性能          │
│  ② 测量驱动：一次只改一个变量，每次都记录到优化日志                 │
│  ③ 认清天花板：先用 Roofline 算出上限，再问"我离上限多远"           │
└──────────────────────────────────────────────────────────────────┘
```

速查表基于 UIUC ECE408/CS483 公开课程资料（官方课程描述、17 条 Instructional Objectives、 Lab Projects 列表、Prof. Steven S. Lumetta 公开的全部 22 个 Slide Deck）与 CUDA 官方文档整理。